



OpenBMC Guide Tutorial

OpenBMC 从入门到精通完整实操教程合集

原作者/来源: [Michael Tien \(openbmc-guide-tutorial\)](#)

整理聚合: 本地完整离线版 (含代码范例、核心架构、实操步骤、图表与原生 PDF 书签导航)

生成时间: 2026-09-29

文章总数: 88 篇完整技术指南



教程目录索引 (Table of Contents)

 [Home](#)

 [Learning Paths](#)

[Getting Started](#)

- [Introduction](#)
- [Environment Setup](#)
- [First Build](#)
- [Development Workflow](#)
- [Building QEMU](#)
- [Devtool Workflow Guide](#)
- [Gerrit Contribution Guide](#)

[Architecture](#)

- [OpenBMC Overview](#)
- [D-Bus Guide](#)
- [State Manager Guide](#)
- [Custom D-Bus Services Guide](#)
- [Systemd Boot Ordering Guide](#)

[Core Services](#)

- [D-Bus Sensors Guide](#)
- [Hwmon Sensors Guide](#)
- [Entity Manager Guide](#)
- [Fan Control Guide](#)
- [Power Management Guide](#)
- [User Manager Guide](#)
- [Network Guide](#)
- [LED Manager Guide](#)
- [Certificate Manager Guide](#)
- [Time Manager Guide](#)
- [Inventory Manager Guide](#)
- [Watchdog Guide](#)
- [Buttons Guide](#)
- [GPIO Management Guide](#)
- [PECI Thermal Monitoring](#)
- [I2C Device Integration](#)
- [PSU & PMBus Management](#)
- [LDAP Integration](#)
- [PID Thermal Tuning](#)
- [Intrusion Detection Guide](#)
- [Leak Detection](#)
- [NVIDIA GPU Management](#)

[Interfaces](#)

- [IPMI Guide](#)
- [Redfish Guide](#)
- [WebUI Guide](#)

- [KVM Guide](#)

- [Virtual Media Guide](#)

- [Console Guide](#)

- [SSH Security Guide](#)

- [Redfish Events & Telemetry](#)

- [Redfish OEM Extensions](#)

[Advanced Topics](#)

- [MCTP & PLDM Guide](#)
- [SPDM Guide](#)
- [Firmware Update Guide](#)
- [Intel ASD/ACD Guide](#)
- [AMD Debug & Management Guide](#)
- [Logging Guide](#)
- [SDR Guide](#)
- [Linux Debug Tools Guide](#)
- [eSPI Guide](#)
- [Unit Testing Guide](#)
- [Robot Framework Guide](#)
- [Linux Kernel Driver Development](#)
- [Secure Boot & Image Signing](#)
- [Multi-Host Support](#)
- [Performance Optimization](#)
- [BIOS Firmware Management](#)
- [Debug Dump Collection](#)
- [POST Code Monitoring](#)
- [PLDM Platform Monitoring](#)
- [PLDM Firmware Update](#)
- [DICE Security](#)
- [SGPIO \(Serial GPIO\)](#)
- [Device Firmware Update](#)

[Porting](#)

- [Porting Reference](#)
- [Machine Layer Guide](#)
- [Device Tree Guide](#)
- [U-Boot Guide](#)
- [Verification Guide](#)
- [ARM Platform Guide](#)
- [Entity Manager Advanced](#)
- [Flash Layout & Optimization Guide](#)
- [AST2700 Enablement](#)

[Appendix](#)

- [Jenkins CI/CD Infrastructure](#)
- [AMI OneTree vs OpenBMC](#)

- [Yocto BitBake Build Optimization](#)

 [Contributing](#)

OpenBMC Guide Tutorial

Welcome to the comprehensive OpenBMC development guide. This tutorial takes you from beginner to professional, covering everything from environment setup to advanced platform porting.

What You'll Learn

- **Getting Started:** Set up your development environment and build your first OpenBMC image
- **Architecture:** Understand OpenBMC's D-Bus architecture, services, and design patterns
- **Core Services:** Master sensors, thermal management, power control, and system services
- **Interfaces:** Implement IPMI, Redfish, WebUI, and other management interfaces
- **Advanced Topics:** Debug features, security protocols (SPDM), and specialized components
- **Platform Porting:** Port OpenBMC to your custom hardware

Quick Start

1. [Set up your environment](#)
2. [Build your first image](#)
3. [Understand the architecture](#)

Choose Your Path

Not sure where to start? We have [curated learning paths](#) for different experience levels and roles:

Path	Best For
Beginner	New to OpenBMC, learning fundamentals
Intermediate	Ready for core services and interfaces
Advanced	Complex topics and platform porting
Software Developer	Writing OpenBMC services and code
System Operator	Managing BMC-enabled servers
Hardware Engineer	Porting to new platforms

How This Guide is Organized

Section	Description	Difficulty
Getting Started	Environment setup, first build, QEMU testing	Beginner
Architecture	D-Bus, state management, core concepts	Beginner-Intermediate
Core Services	Sensors, thermal, power, logging	Intermediate
Interfaces	IPMI, Redfish, WebUI, SSH	Intermediate
Advanced	MCTP/PLDM, SPD, debug tools	Advanced
Porting	Platform porting, device tree, verification	Advanced

Prerequisites

- Linux development experience (command line, build systems)
- Basic C/C++ programming knowledge
- Familiarity with embedded systems concepts (helpful but not required)

Sample Code

All examples in this guide are tested and available in the [examples/](#) directory. Each example includes:

- Complete, buildable code
- README with setup instructions
- Verification steps for QEMU

Contributing

Found an error? Want to add content? See our [Contributing Guide](#).

This guide is community-maintained and not officially affiliated with the OpenBMC project. For official documentation, visit github.com/openbmc/docs.

Learning Paths

Not sure where to start? Choose a learning path based on your experience level or role.

Table of Contents

1. [Skill-Based Paths](#)
 1. [Beginner Path](#)
 2. [Intermediate Path](#)
 3. [Advanced Path](#)
 2. [Role-Based Paths](#)
 1. [Software Developer Path](#)
 2. [System Operator Path](#)
 3. [Hardware Engineer Path](#)
 4. [RAS / Validation Engineer Path](#)
 5. [Security Engineer Path](#)
 3. [Not Sure Which Path?](#)
-

Skill-Based Paths

Choose based on your OpenBMC experience level.

Beginner Path

For: New to OpenBMC, want to understand fundamentals and get a working environment.

Prerequisites: Basic Linux command line, some C/C++ familiarity.

#	Guide	What You'll Learn
1	Introduction	What OpenBMC is and why it matters
2	Environment Setup	Set up your development machine
3	First Build	Build and run OpenBMC in QEMU
4	OpenBMC Overview	Understand the system architecture
5	D-Bus Guide	Learn the core communication mechanism
6	State Manager	Understand BMC and host state management
7	Systemd Boot Ordering	How services start and depend on each other

Next Steps: Continue to the Intermediate path, or explore Core Services based on your interests.

Intermediate Path

For: Comfortable with OpenBMC basics, ready to work with core services and interfaces.

Prerequisites: Completed Beginner path or equivalent experience.

#	Guide	What You'll Learn
1	D-Bus Sensors	Expose sensor data via D-Bus
2	Entity Manager	Dynamic hardware configuration
3	GPIO Management	Monitor and control GPIO signals
4	I2C Device Integration	Add I2C devices to device tree
5	Power Management	Control host power states
6	Fan Control	Thermal management and fan policies
7	Redfish Guide	Modern REST API for BMC management
8	IPMI Guide	Legacy management interface
9	Devtool Workflow	Advanced recipe development with devtool

Next Steps: Continue to the Advanced path, or dive into specific interfaces and services.

Advanced Path

For: Experienced OpenBMC developers ready for complex topics and platform porting.

Prerequisites: Solid understanding of D-Bus, services, and interfaces.

#	Guide	What You'll Learn
1	MCTP/PLDM Guide	Platform-level data model communication
2	Firmware Update	BMC and host firmware update mechanisms
3	Secure Boot & Signing	Image signing and hardware root of trust
4	Unit Testing	GTest/GMock for OpenBMC code
5	Robot Framework	Integration and system testing
6	Performance Optimization	Optimize for constrained BMC environments
7	Porting Reference	Complete platform porting guide
8	Machine Layer	Create Yocto machine layer
9	Device Tree	Configure BMC hardware in DTS

Next Steps: Explore specialized topics like SPDM, eSPI, multi-host, or vendor-specific debug tools.

Role-Based Paths

Choose based on your job function or project goals.

Software Developer Path

For: Developers writing OpenBMC services, daemons, or applications.

Focus: D-Bus programming, service development, testing.

#	Guide	What You'll Learn
1	Environment Setup	Set up your development machine
2	First Build	Build and run in QEMU
3	D-Bus Guide	Core IPC mechanism
4	Custom D-Bus Services	Build your own D-Bus service with <code>sdbus++</code>
5	D-Bus Sensors	Service implementation patterns
6	Entity Manager	Configuration-driven services
7	Devtool Workflow	Advanced recipe development with devtool
8	Unit Testing	GTest/GMock for your code
9	Logging Guide	phosphor-logging patterns
10	Gerrit Contribution	Submit patches upstream via Gerrit

System Operator Path

For: Operators managing BMC-enabled servers, using IPMI/Redfish for administration.

Focus: Management interfaces, user management, network configuration.

#	Guide	What You'll Learn
1	Introduction	OpenBMC capabilities overview
2	OpenBMC Overview	System architecture
3	IPMI Guide	ipmitool commands and operations
4	Redfish Guide	REST API for modern management
5	Redfish Events & Telemetry	Event subscriptions and metric reports
6	WebUI Guide	Browser-based management
7	User Manager	User accounts and LDAP
8	LDAP Integration	LDAP/Active Directory authentication
9	Network Guide	Network configuration
10	Console Guide	Serial-over-LAN access

Hardware Engineer Path

For: Engineers porting OpenBMC to new platforms or integrating BMC hardware.

Focus: Platform porting, device tree, hardware configuration.

#	Guide	What You'll Learn
1	Environment Setup	Development environment
2	First Build	Build for reference platform
3	Porting Reference	Complete porting overview
4	Machine Layer	Yocto machine configuration
5	Device Tree	DTS for your hardware
6	I2C Device Integration	Add I2C devices and drivers
7	Flash Layout Optimization	SPI flash partitioning and image sizing
8	U-Boot	Bootloader customization
9	Entity Manager Advanced	Hardware configuration files
10	Verification	Testing your port

RAS / Validation Engineer Path

For: Engineers validating platform reliability, error handling, and crash dump mechanisms.

Focus: Error injection, crash dump analysis, POST code monitoring, debug tools.

#	Guide	What You'll Learn
1	Environment Setup	Development environment
2	OpenBMC Overview	System architecture
3	PECI Thermal Monitoring	CPU thermal interface and PECI commands
4	Intel ASD/ACD	Debug access, crash dumps, error injection
5	Debug Dump Collection	BMC diagnostic dump collection
6	POST Code Monitoring	Boot progress tracking and diagnostics
7	Logging Guide	Event logs and SEL entries
8	Robot Framework	Automated RAS test suites

Security Engineer Path

For: Engineers implementing and validating BMC security features.

Focus: Secure boot, image signing, authentication, attestation.

#	Guide	What You'll Learn
1	Environment Setup	Development environment
2	OpenBMC Overview	System architecture
3	Secure Boot & Signing	Image signing and hardware root of trust
4	SPDM Guide	Device attestation and measurement
5	Certificate Manager	TLS/SSL certificate management
6	User Manager	Accounts and privilege roles
7	LDAP Integration	Enterprise directory authentication
8	SSH Security	SSH hardening and access control

Not Sure Which Path?

If you want to...	Start with
Learn OpenBMC from scratch	Beginner Path
Write OpenBMC code	Software Developer Path
Manage servers with BMC	System Operator Path
Port to new hardware	Hardware Engineer Path
Validate RAS / error handling	RAS / Validation Engineer Path
Implement BMC security	Security Engineer Path
Understand a specific topic	Browse by section

These paths are suggestions, not requirements. Feel free to skip guides you already know or explore topics in any order that suits your learning style.

Getting Started

This section covers everything you need to start developing with OpenBMC.

What You'll Learn

- Set up a development environment on Linux, macOS, or Windows (via Docker)
- Clone and configure the OpenBMC repositories
- Build your first OpenBMC image
- Run and test in QEMU (the standard development environment)
- Use the SDK for application development

No Hardware Required

QEMU is the standard development environment for OpenBMC — not a compromise or simulation fallback. Professional OpenBMC developers at Google, Meta, IBM, and other companies use QEMU daily for most development work.

Feature Comparison: QEMU vs Raspberry Pi vs Real Hardware

Feature	QEMU (ASPEED)	Raspberry Pi	ASPEED AST2600 EVB
OpenBMC software stack	✓ Full	✓ Full	✓ Full
D-Bus services	✓ Full	✓ Full	✓ Full
Redfish API (bmcweb)	✓ Full	✓ Full	✓ Full
Yocto/BitBake build	✓ Full	✓ Full	✓ Full
I2C sensors (tmp105, EEPROMs)	✓ Well emulated	✗ No ASPEED I2C	✓ Hardware
SPI flash (boot, firmware)	✓ Well emulated	✗ SD card boot	✓ Hardware
GPIO (pins, interrupts)	✓ Functional	⚠ Limited	✓ Hardware
ADC (analog sensors)	⚠ Synthetic values	✗ No built-in ADC	✓ Hardware
IPMI KCS/BT interface	⚠ Partial	✗ Not available	✓ Hardware
PECI (CPU temperature)	⚠ Stub only	✗ Not available	✓ Hardware
PWM / fan tachometer	⚠ Register stub	✗ Not available	✓ Hardware
KVM-over-IP (video)	✗ Not emulated	✗ Not available	✓ Hardware
eSPI host interface	✗ Not emulated	✗ Not available	✓ Hardware
Secure Boot (RoT)	⚠ Basic OTP only	✗ Not available	✓ Hardware
Cost	Free	\$35-75	\$500-800

What Requires Real Hardware

Only specialized hardware bring-up tasks need physical BMC hardware:

- KVM-over-IP video capture and encoding
- eSPI/LPC host interface debugging
- PECI CPU temperature monitoring with real data
- Real fan control with PWM and tachometer feedback
- Real analog sensor calibration
- Platform-specific GPIO timing

These topics are relevant only for hardware engineers doing board bring-up — not for learning OpenBMC software development.

Recommendation

Option	Cost	Best For
QEMU	Free	✅ Learning and software development — start here
Raspberry Pi	\$35-75	⚠️ Exploring the software stack on real hardware, but no BMC peripherals
ASPEED AST2600 EVB	\$500-800	Full hardware bring-up and production development

Prerequisites

- A Linux workstation (Ubuntu 22.04+ or Fedora 38+ recommended) OR Docker
- At least 16GB RAM (32GB recommended)
- 100GB+ free disk space
- Basic command line experience

Guides in This Section

Guide	Description	Time
Introduction	What is OpenBMC and why use it	10 min
Environment Setup	Set up your development environment	30 min
First Build	Build and run OpenBMC in QEMU	45 min
Development Workflow	Iterate quickly with devtool and bbappend	30 min
Building QEMU	Build QEMU from source if needed	20 min
Devtool Workflow	Advanced devtool usage for recipe development	30 min
Gerrit Contribution	Submit patches to OpenBMC via Gerrit	30 min

Quick Path

If you're eager to get started:

1. [Environment Setup](#) - Get your tools ready
2. [First Build](#) - Build and run in QEMU
3. [OpenBMC Overview](#) - Understand the architecture

If you encounter issues, check the [Troubleshooting](#) section or search the [OpenBMC mailing list](#).

Table of contents

- [Introduction](#)
- [Environment Setup](#)
- [First Build](#)
- [Development Workflow](#)
- [Building QEMU](#)
- [Devtool Workflow Guide](#)
- [Gerrit Contribution Guide](#)

Introduction to OpenBMC

Learn what OpenBMC is and why it's transforming server management.

Table of Contents

1. [What is a BMC?](#)
 1. [BMC Capabilities](#)
 2. [What is OpenBMC?](#)
 1. [Who Uses OpenBMC?](#)
 2. [Why OpenBMC?](#)
 3. [OpenBMC Architecture Overview](#)
 1. [Key Components](#)
 4. [Development Environment](#)
 1. [QEMU: The Standard Development Platform](#)
 2. [What You Can Learn with QEMU](#)
 5. [Getting Started Path](#)
 6. [Resources](#)
-

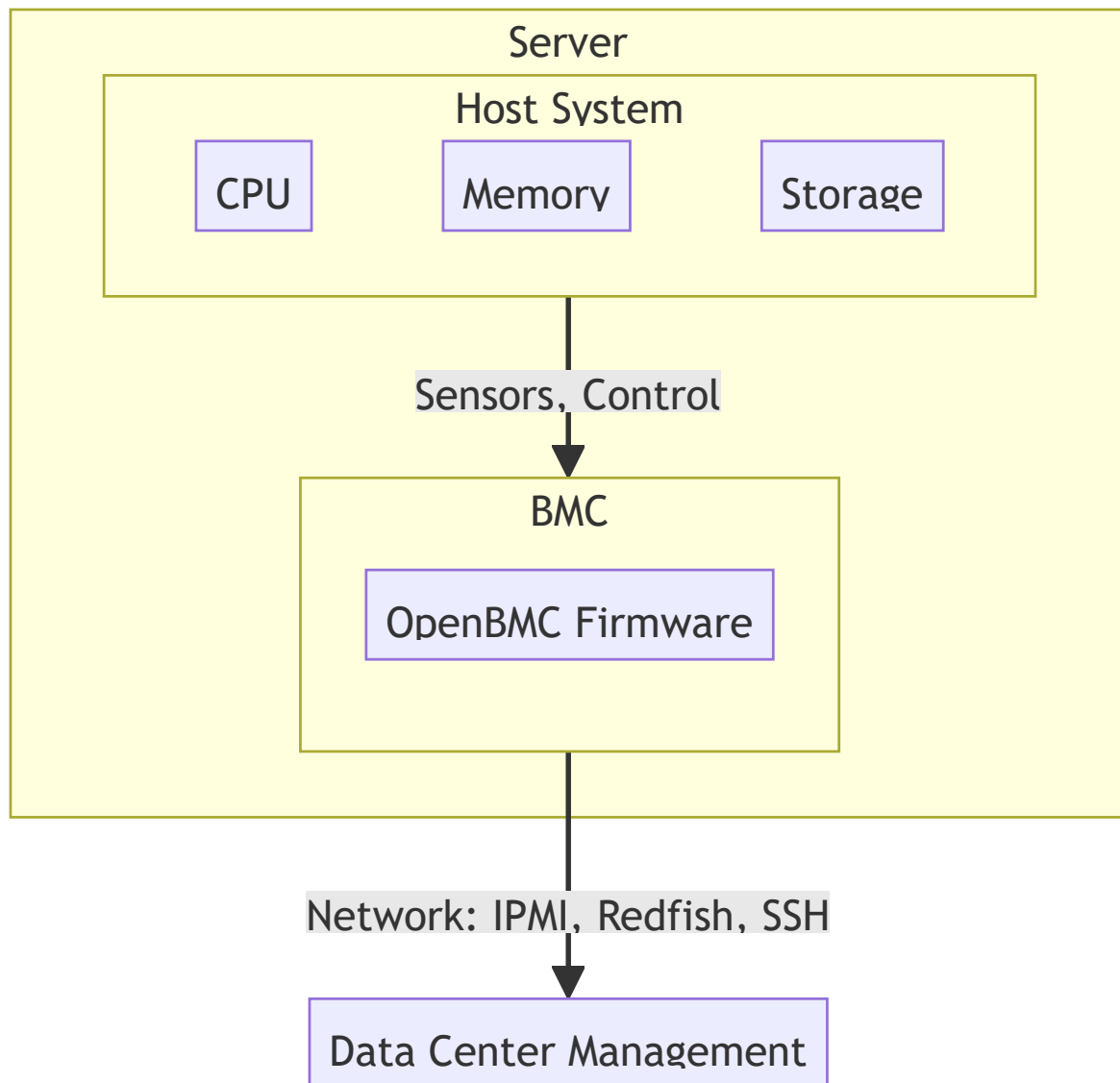
What is a BMC?

A **Baseboard Management Controller (BMC)** is a specialized microcontroller embedded on server motherboards. It provides **out-of-band management**, meaning it can monitor and control the server even when the main system is powered off or unresponsive.

BMC Capabilities

- **Remote Power Control:** Power on/off, reset the server remotely
- **Hardware Monitoring:** Track temperatures, voltages, fan speeds
- **Console Access:** Serial-over-LAN for OS-independent access
- **Firmware Updates:** Update BIOS and other firmware remotely
- **Event Logging:** Record hardware events and errors
- **KVM:** Remote keyboard, video, mouse access
- **Virtual Media:** Mount remote images for OS installation

BMC in Server Architecture



► ASCII-art version (for comparison)

What is OpenBMC?

OpenBMC is a Linux distribution designed specifically for BMC firmware. It's:

- **Open Source:** Apache 2.0 licensed, community-driven
- **Linux-Based:** Built on Yocto/Poky embedded Linux
- **Standards-Compliant:** Supports IPMI, Redfish, and modern protocols
- **Modular:** Component-based architecture via D-Bus

Who Uses OpenBMC?

OpenBMC is used by major technology companies including:

- Facebook/Meta
- Google
- Microsoft
- IBM
- Intel
- Rackspace
- LinkedIn

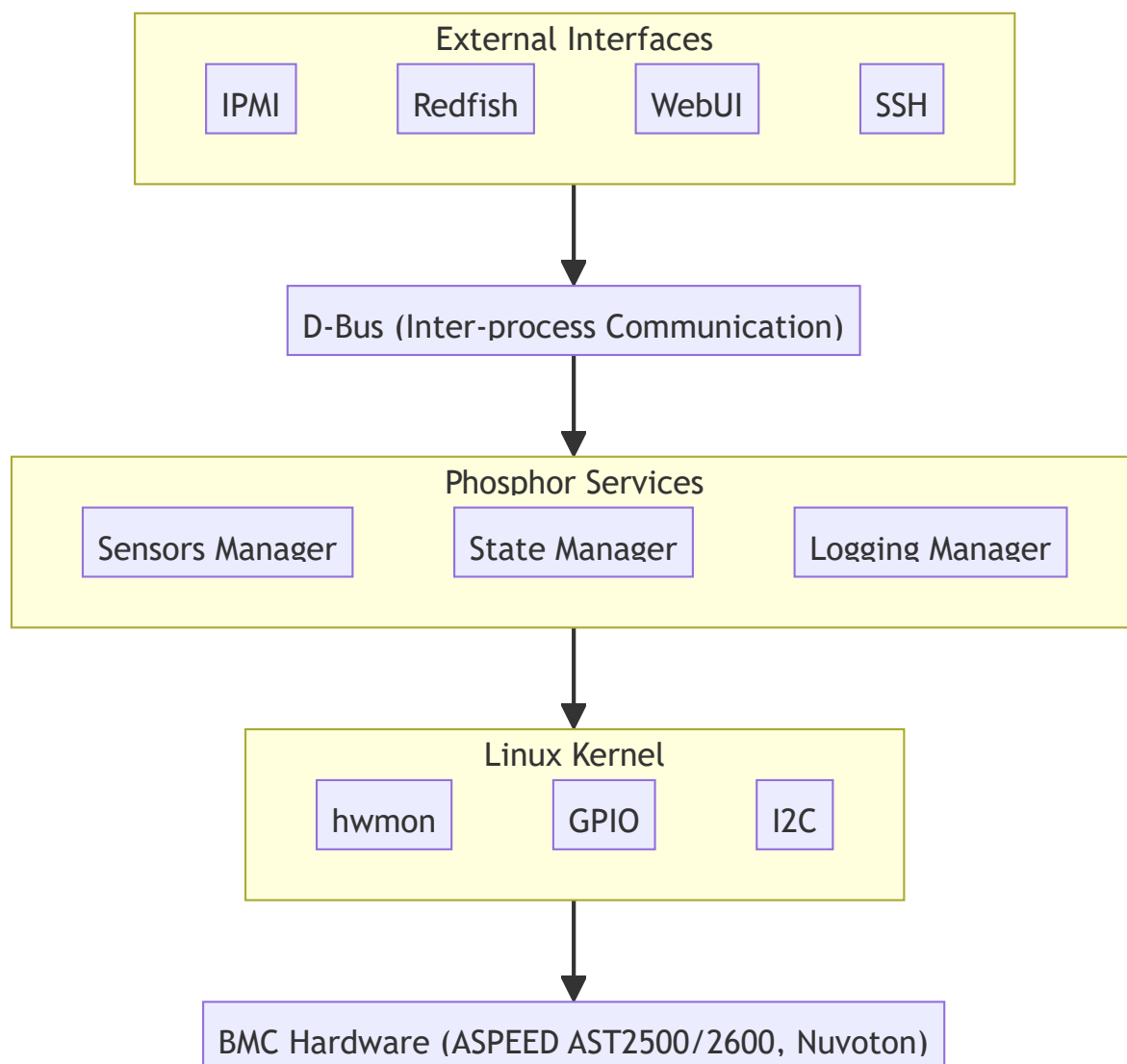
Why OpenBMC?

Proprietary BMC	OpenBMC
Closed source, vendor lock-in	Open source, community-driven
Fixed feature set	Customizable to your needs
Security through obscurity	Auditable code, rapid patches
Vendor-specific interfaces	Standard interfaces (Redfish)
Limited extensibility	Fully extensible architecture

OpenBMC Architecture Overview

OpenBMC uses a modern, service-oriented architecture:

OpenBMC Architecture



► ASCII-art version (for comparison)

Key Components

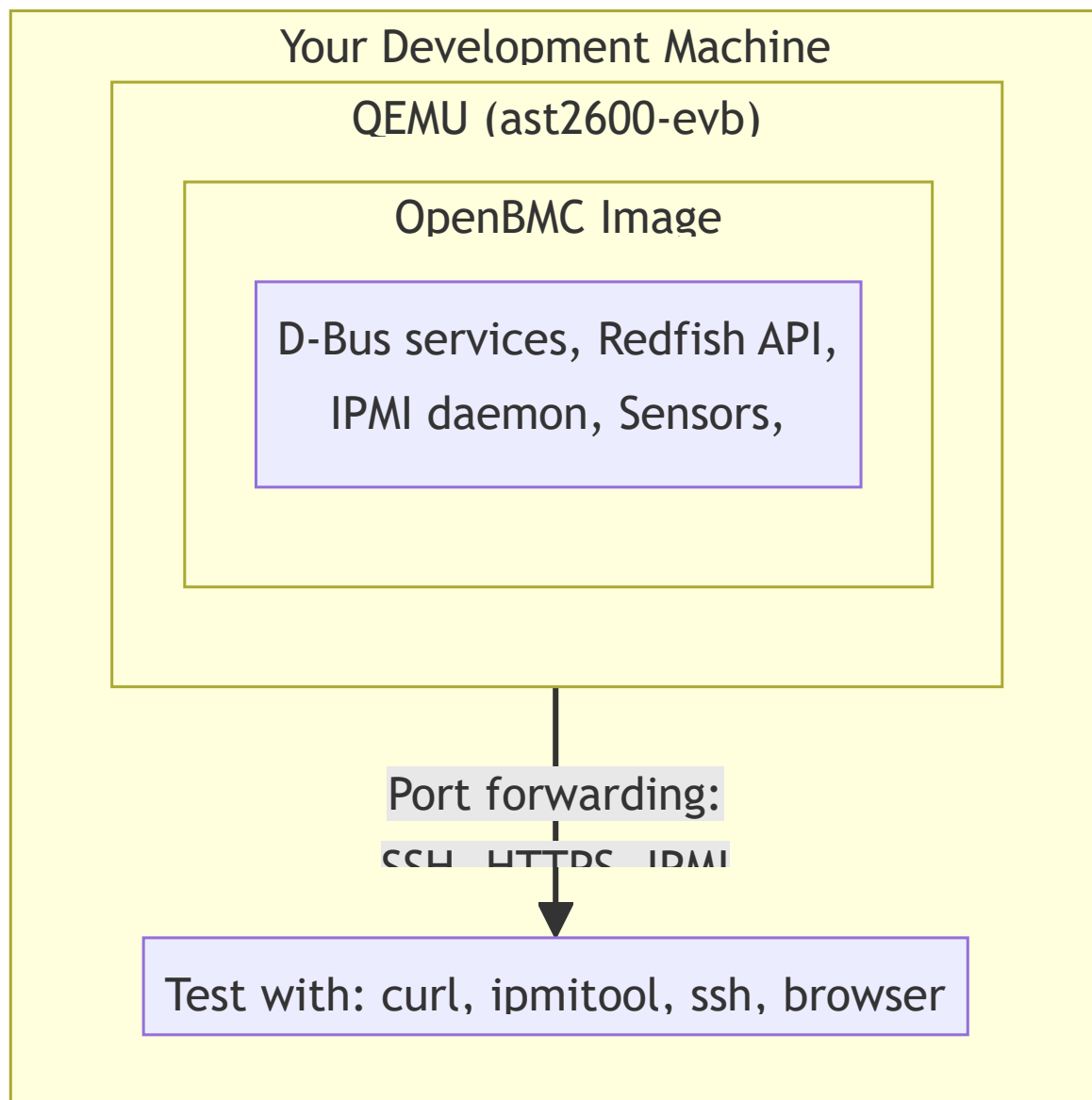
- **D-Bus:** Central communication bus connecting all services
- **Phosphor Services:** Modular daemons for specific functions
- **bmcweb:** HTTP server for Redfish and WebUI
- **ipmid:** IPMI command processing
- **systemd:** Service management

Development Environment

QEMU: The Standard Development Platform

OpenBMC development does **not require physical BMC hardware**. QEMU emulation is the standard development environment used by professional OpenBMC developers at major companies.

QEMU Development Environment



► ASCII-art version (for comparison)

What You Can Learn with QEMU

Topic	QEMU Support	Notes
OpenBMC architecture	✓ Full	All services run identically
D-Bus communication	✓ Full	Same as real hardware
Redfish API development	✓ Full	Complete API available
IPMI commands	✓ Full	ipmitool works
Yocto/BitBake builds	✓ Full	Standard workflow
Service development	✓ Full	Code, build, test cycle
Sensor configuration	✓ Simulated	Values are synthetic
Fan control logic	✓ Simulated	PID algorithms work

Physical BMC hardware (ASPEED AST2600 EVB ~\$500+) is only needed for hardware bring-up tasks like eSPI debugging, PECL communication, or real sensor calibration. These are specialized topics not required for learning OpenBMC software development.

Getting Started Path

Ready to start? Here's your learning path:

1. [Environment Setup](#) - Set up development tools
2. [First Build](#) - Build and run in QEMU
3. [OpenBMC Overview](#) - Deep dive into architecture
4. [D-Bus Guide](#) - Master the communication layer

Resources

- [OpenBMC GitHub Organization](#)
- [OpenBMC Mailing List](#)
- [OpenBMC Discord](#)
- [Redfish Specification](#)
- [IPMI Specification](#)

Environment Setup

Set up your development environment for OpenBMC.

Table of Contents

1. [Overview](#)
2. [System Requirements](#)
 1. [Minimum Requirements](#)
3. [Option 1: Native Linux Setup \(Recommended\)](#)
 1. [Ubuntu 22.04 / 20.04](#)
 2. [Fedora 38 / 34](#)
 3. [Clone OpenBMC Repository](#)
 4. [Initialize Build Environment](#)
4. [Option 2: Docker Setup](#)
 1. [Install Docker](#)
 2. [Using the OpenBMC Build Container](#)
 3. [Custom Dockerfile \(Alternative\)](#)
5. [Option 3: SDK Installation](#)
 1. [Generate SDK \(requires full build first\)](#)
 2. [Install SDK](#)
 3. [Use SDK](#)
6. [QEMU Setup](#)
 1. [Install QEMU](#)
 2. [Verify QEMU Has ASPEED Support](#)
7. [IDE Setup \(VS Code\)](#)
 1. [Install VS Code](#)
 2. [Recommended Extensions](#)
 3. [Configure IntelliSense](#)
 4. [VS Code Dev Container \(Recommended for Docker Users\)](#)
 1. [Prerequisites](#)
 2. [Setup Steps](#)
 3. [What You Get](#)
 4. [Building OpenBMC Inside the Container](#)
 5. [Build Caching with Docker Volumes](#)
 6. [Using Dev Container from the CLI \(Without VS Code\)](#)
 7. [Git Identity](#)
 8. [Troubleshooting Dev Container](#)
8. [Best Practices: Container-Based Environment](#)
 1. [Individual Developer Setup](#)
 2. [Team / Project Setup](#)

1. [1. Publish a team container image](#)
 2. [2. Share sstate cache via HTTP](#)
 3. [3. Distribute a pre-built SDK](#)
 4. [4. Docker Compose for the full stack](#)
3. [Summary: Which Setup to Use](#)
9. [Verify Your Setup](#)
 10. [Next Steps](#)
 1. [Debug Builds \(Optional\)](#)
 11. [Troubleshooting](#)
 1. [Build fails with "out of memory"](#)
 2. [Permission denied with Docker](#)
 3. ["locale" errors during build](#)

Overview

This guide covers three setup options:

Option	Best For	Host OS
Native Linux	Full performance, serious development	Ubuntu/Fedora
Docker	Quick start, Windows/macOS users	Any
SDK Only	Application development without full builds	Any

Choose the option that fits your needs. For complete OpenBMC development, **Native Linux** is recommended.

System Requirements

Minimum Requirements

Resource	Minimum	Recommended
OS	Ubuntu 20.04, Fedora 34	Ubuntu 22.04, Fedora 38
RAM	16 GB	32 GB
Disk	100 GB free	250 GB SSD
CPU	4 cores	8+ cores

Building OpenBMC is resource-intensive. Insufficient RAM will cause build failures.

Option 1: Native Linux Setup (Recommended)

Ubuntu 22.04 / 20.04

```
# Update system
sudo apt update && sudo apt upgrade -y

# Install required packages
sudo apt install -y \
    gawk wget git diffstat unzip texinfo gcc build-essential \
    chrpath socat cpio python3 python3-pip python3-pexpect \
    xz-utils debiutils iputils-ping python3-git python3-jinja2 \
    libegl1-mesa libsdl1.2-dev pylint xterm python3-subunit \
    mesa-common-dev zstd liblz4-tool

# Install additional useful tools
sudo apt install -y \
    curl vim tmux htop tree \
    libssl-dev libffi-dev

# Verify installation
echo "Checking installed tools..."
git --version
python3 --version
gcc --version
```

Fedora 38 / 34

```
# Install required packages
sudo dnf install -y \
    gawk make wget tar bzip2 gzip python3 unzip perl patch \
    diffutils diffstat git cpp gcc gcc-c++ glibc-devel \
    texinfo chrpath ccache perl-Data-Dumper perl-Text-ParseWords \
    perl-Thread-Queue perl-bignum socat python3-pexpect \
    findutils which file cpio python python3-pip xz python3-GitPython \
    python3-jinja2 SDL-devel xterm rpcgen mesa-libGL-devel \
    perl-FindBin perl-File-Compare perl-File-Copy perl-locale \
    zstd lz4

# Verify installation
echo "Checking installed tools..."
git --version
python3 --version
gcc --version
```

Clone OpenBMC Repository

```
# Create workspace directory
mkdir -p ~/openbmc-workspace
cd ~/openbmc-workspace

# Clone the repository
git clone https://github.com/openbmc/openbmc.git
cd openbmc

# Check available machines
ls meta-*/meta-*/conf/machine/*.conf | head -20
```

Initialize Build Environment

```
# Source the environment script
# This sets up paths and creates the build directory
. setup ast2600-evb

# You're now in the build directory
# The terminal prompt changes to indicate the environment is active
```

`ast2600-evb` is an AST2600-based evaluation board, ideal for learning modern OpenBMC. Replace with your target machine for production development.

Option 2: Docker Setup

Docker provides a consistent build environment across operating systems.

Install Docker

Ubuntu/Debian:

```
# Install Docker
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

# Add user to docker group (logout/login required)
sudo usermod -aG docker $USER
```

Windows/macOS: Download and install [Docker Desktop](#).

Using the OpenBMC Build Container

```
# Create workspace
mkdir -p ~/openbmc-workspace
cd ~/openbmc-workspace

# Clone OpenBMC
git clone https://github.com/openbmc/openbmc.git
cd openbmc

# Run build in container
docker run --rm -it \
  -v $(pwd):/home/openbmc/openbmc \
  -w /home/openbmc/openbmc \
  crops/poky:ubuntu-22.04 \
  /bin/bash -c ". setup ast2600-evb && bitbake obmc-phosphor-image"
```

Custom Dockerfile (Alternative)

Create a `Dockerfile` for a customized environment:

```
FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y \
    gawk wget git diffstat unzip texinfo gcc build-essential \
    chrpath socat cpio python3 python3-pip python3-pexpect \
    xz-utils debianutils iputils-ping python3-git python3-jinja2 \
    libegl1-mesa libsdl1.2-dev pylint xterm python3-subunit \
    mesa-common-dev zstd liblz4-tool locales \
    && rm -rf /var/lib/apt/lists/*

# Set locale
RUN locale-gen en_US.UTF-8
ENV LANG=en_US.UTF-8

# Create non-root user
RUN useradd -ms /bin/bash openbmc
USER openbmc
WORKDIR /home/openbmc

CMD ["/bin/bash"]
```

Build and use:

```
docker build -t openbmc-builder .
docker run --rm -it -v $(pwd):/home/openbmc/openbmc openbmc-builder
```

Option 3: SDK Installation

The SDK allows cross-compiling applications without a full build environment.

Generate SDK (requires full build first)

```
# In an initialized build environment
bitbake obmc-phosphor-image -c populate_sdk
```

The SDK installer is created at:

```
build/tmp/deploy/sdk/oecore-x86_64-arm1176jzs-toolchain-*.sh
```

Install SDK

```
# Run the installer
./oecore-x86_64-arm1176jzs-toolchain-*.sh

# Default install location: /opt/openbmc-phosphor/VERSION
# Accept the default or specify a custom path
```

Use SDK

```
# Source the environment
source /opt/openbmc-phosphor/VERSION/environment-setup-arm1176jzs-openbmc-linux-gnueabi

# Verify cross-compiler
$CC --version
# Should show arm-openbmc-linux-gnueabi-gcc

# Cross-compile a simple program
echo 'int main() { return 0; }' > test.c
$CC test.c -o test
file test
# Should show: ELF 32-bit LSB executable, ARM
```

QEMU Setup

QEMU allows testing OpenBMC without physical hardware.

Install QEMU

Ubuntu 24.04+ / Fedora 38+:

```
# Ubuntu
sudo apt install -y qemu-system-arm

# Fedora
sudo dnf install -y qemu-system-arm
```

Verify QEMU Has ASPEED Support

```
# Check version (need 7.0+ for ast2600-evb)
qemu-system-arm --version

# Verify ast2600-evb machine is available
qemu-system-arm -machine help | grep ast2600
```

Older distributions (Ubuntu 20.04/22.04, Debian 11) may not have ast2600-evb support in their package QEMU.

If `grep ast2600` returns nothing, see [Building QEMU](#) to build from source or use OpenBMC's built-in QEMU.

QEMU usage is covered in the [First Build](#) guide.

IDE Setup (VS Code)

VS Code provides excellent support for OpenBMC development.

Install VS Code

Download from code.visualstudio.com

Recommended Extensions

Install these extensions:

- **C/C++** (Microsoft) - IntelliSense, debugging
- **Remote - SSH** - Remote development
- **Remote - Containers** - Docker development
- **Python** - Python support
- **YAML** - Recipe file editing
- **BitBake** - Yocto syntax highlighting

Configure IntelliSense

Create `.vscode/c_cpp_properties.json`:

```
{
  "configurations": [
    {
      "name": "OpenBMC",
      "includePath": [
        "${workspaceFolder}/**",
        "/path/to/sdk/sysroots/arm*/usr/include/**"
      ],
      "defines": [],
      "compilerPath": "/path/to/sdk/sysroots/x86_64*/usr/bin/arm-openbmc-linux-gnueabi-gcc",
      "cStandard": "c17",
      "cppStandard": "c++20",
      "intelliSenseMode": "linux-gcc-arm"
    }
  ],
  "version": 4
}
```

VS Code Dev Container (Recommended for Docker Users)

A ready-to-use Dev Container configuration provides a complete OpenBMC build environment with pre-configured tools, extensions, and build caching. The devcontainer is designed to live inside the openbmc repo itself — you open the openbmc repo in VS Code and everything works.

Prerequisites

- [VS Code](#) with the [Dev Containers](#) extension
- [Docker Desktop](#) (Windows/macOS) or Docker Engine (Linux)

Setup Steps

```
# 1. Clone OpenBMC
git clone https://github.com/openbmc/openbmc.git
cd openbmc

# 2. Add the devcontainer configuration (download from this tutorial site)
mkdir -p .devcontainer
base=https://michaeltien8901.github.io/openbmc-guide-tutorial/docs/examples/devcontainer
curl -fsSL -o .devcontainer/devcontainer.json "$base/devcontainer.json"
curl -fsSL -o .devcontainer/Dockerfile "$base/Dockerfile"

# 3. Open in VS Code
code .
```

Then in VS Code, press **F1** → **Dev Containers: Reopen in Container**. The first build takes several minutes; subsequent opens use cached layers.

The `docs/examples/devcontainer/` directory in this tutorial holds the reference `Dockerfile` and `devcontainer.json`. The commands above download them into your openbmc clone as `.devcontainer/`; you can also browse them in the [tutorial repo](#).

What You Get

Feature	Details
Base image	Ubuntu 24.04 with all Yocto/BitBake dependencies
User	<code>openbmc</code> with <code>sudo</code> access; UID auto-mapped to your host UID
Build caching	<code>sstate-cache</code> and <code>downloads</code> persist in Docker volumes across rebuilds
Build env	<code>oe-init-build-env</code> sourced automatically; <code>PARALLEL_MAKE</code> and <code>BB_NUMBER_THREADS</code> set to <code>\$(nproc)</code>
QEMU	<code>qemu-system-arm</code> 8.2+ pre-installed with ast2600-evb support
Extensions	C/C++, Python, YAML, BitBake, GitLens, Git Graph, Code Spell Checker
Forwarded ports	2222 (BMC SSH), 2443 (BMC HTTPS/Redfish), 2623 (BMC IPMI)

Building OpenBMC Inside the Container

Once the container starts, the terminal is inside the openbmc repo with the build environment initialized:

```
# Build the full image
bitbake obmc-phosphor-image

# Or build a single recipe
bitbake phosphor-webui
```

Build Caching with Docker Volumes

BitBake uses two large directories during builds:

- `downloads/` (`DL_DIR`) — upstream source tarballs fetched from the internet (several GB)
- `sstate-cache/` (`SSTATE_DIR`) — shared state cache of previously compiled outputs (saves hours on rebuilds)

The Dev Container mounts both as **Docker named volumes** rather than bind mounts:

```
source=openbmc-downloads,target=<workspace>/build/downloads,type=volume
source=openbmc-sstate-cache,target=<workspace>/build/sstate-cache,type=volume
```

This means:

- The data lives on your host in Docker-managed storage (`/var/lib/docker/volumes/`), **not** inside the container
- Deleting or rebuilding the container does not lose cached data

- **No `local.conf` configuration needed** — BitBake's default `DL_DIR` (`build/downloads/`) matches the mount target, so it works automatically
- **Shared across projects** — any container using the same volume name (e.g., `openbmc-downloads`) shares the same downloaded sources, avoiding redundant multi-GB downloads

`docker system prune` does **not** delete named volumes. To fully reset build caches, explicitly remove them: `docker volume rm openbmc-sstate-cache openbmc-downloads` .

Using Dev Container from the CLI (Without VS Code)

You can use the Dev Container from any terminal using the [Dev Container CLI](#):

```
# Install the CLI
npm install -g @devcontainers/cli

# Build and start the container (from your openbmc clone with .devcontainer/)
devcontainer up --workspace-folder ./openbmc

# Open an interactive shell inside it
devcontainer exec --workspace-folder ./openbmc bash

# Run a single command
devcontainer exec --workspace-folder ./openbmc \
    bitbake obmc-phosphor-image
```

Git Identity

The container does **not** set a git identity — it inherits your host's git configuration. If git prompts you to set `user.name` and `user.email` , configure them either on your host or inside the container:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Troubleshooting Dev Container

File permission issues: The `updateRemoteUserUID` setting should handle this automatically. If you still see permission errors, rebuild the container: **F1** → **Dev Containers: Rebuild Container**.

Best Practices: Container-Based Environment

Whether you are a solo developer or part of a team, a container-based workflow provides reproducibility and faster onboarding. This section covers recommended setups for both scenarios.

Individual Developer Setup

For a single developer, the goal is a reproducible environment with persistent build caches.

```

Host Machine
├─ openbmc/                # Git clone (bind mount into container)
├─   └─ .devcontainer/     # Dev Container config
└─ Docker Volumes
    ├── openbmc-ssstate-cache # Persists across container rebuilds
    └─ openbmc-downloads     # Shared source tarballs

```

Key practices:

1. **Use named Docker volumes for build caches** — not bind mounts. Named volumes have better I/O performance and survive container rebuilds.
2. **Pin your container image** — use a specific tag (e.g., `ubuntu:24.04`), not `latest`. This prevents surprise breakages when the base image updates.
3. **Keep source on host, tools in container** — bind-mount your openbmc clone so you edit with your host IDE. All build tooling lives in the container.
4. **Pre-build the SDK once, reuse it** — generate the SDK inside the container and install it to a named volume:

```

# Inside container: build SDK
bitbake obmc-phosphor-image -c populate_sdk

# Install to a persistent volume
./tmp/deploy/sdk/oecore-*.sh -d /opt/openbmc-sdk -y

```

Mount `/opt/openbmc-sdk` as a named volume so you can use it without rebuilding.

5. **Use ccache for kernel/u-boot iteration** — add to your `local.conf`:

```

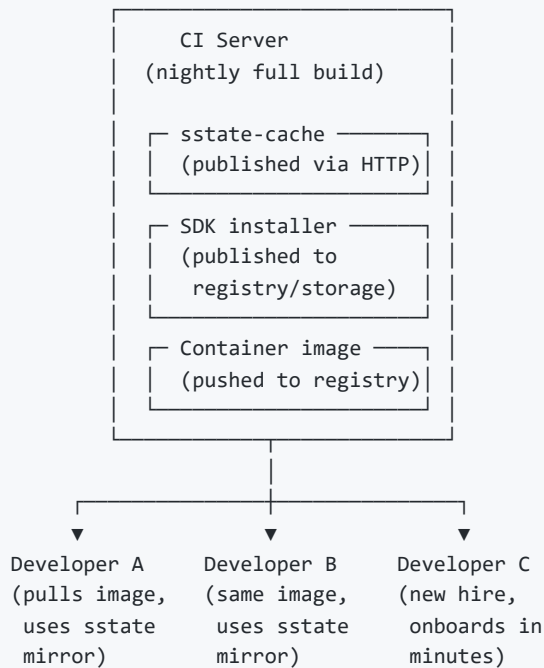
INHERIT += "ccache"
CCACHE_TOP_DIR = "${TOPDIR}/ccache"

```

Mount `ccache/` as a named volume for persistence.

Team / Project Setup

For teams, the goal is shared caches, consistent environments, and fast onboarding for new members.



1. Publish a team container image

Build a single container image in CI and push it to your registry:

```
FROM ubuntu:24.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y \
    gawk wget git diffstat unzip texinfo gcc build-essential \
    chrpath socat cpio python3 python3-pip python3-pexpect \
    xz-utils debianutils iputils-ping python3-git python3-jinja2 \
    libegl1-mesa libsdl1.2-dev pylint python3-subunit \
    mesa-common-dev zstd liblz4-tool locales \
    qemu-system-arm ccache \
    && rm -rf /var/lib/apt/lists/*

RUN locale-gen en_US.UTF-8
ENV LANG=en_US.UTF-8

RUN useradd -ms /bin/bash openbmc
USER openbmc
WORKDIR /home/openbmc
```

```
# CI builds and pushes
docker build -t registry.company.com/openbmc-builder:2024.04 .
docker push registry.company.com/openbmc-builder:2024.04
```

Tag images by date or build number, not `latest`. This lets developers pin to a known-good version while allowing controlled upgrades.

2. Share sstate cache via HTTP

This is the single biggest time saver for teams. CI builds the full image nightly and publishes sstate:

```
# On CI: after full build completes, sync sstate to web server
rsync -a build/sstate-cache/ /var/www/sstate/

# On developer machines: add to build/conf/local.conf
SSTATE_MIRRORS = "file://.* http://build-server.company.com/sstate/PATH"
```

With a shared sstate mirror, a developer's first build downloads pre-compiled artifacts instead of compiling from scratch — reducing build time from hours to minutes.

3. Distribute a pre-built SDK

For developers who only work on applications (daemons, D-Bus services) and do not need full image builds:

```
# CI: build and publish SDK
bitbake obmc-phosphor-image -c populate_sdk
# Upload installer to artifact storage (e.g., Artifactory, S3, NFS)

# Developer: download and install
./oecore-x86_64-arm1176jzs-toolchain-*.sh -d /opt/openbmc-sdk -y
source /opt/openbmc-sdk/environment-setup-*
# Now compile directly – no BitBake needed
$CC -o my_daemon my_daemon.c $(pkg-config --cflags --libs sdbusplus)
```

Or ship the SDK inside a container image:

```
FROM ubuntu:24.04
COPY openbmc-sdk-installer.sh /tmp/
RUN /tmp/openbmc-sdk-installer.sh -d /opt/openbmc-sdk -y && rm /tmp/*.sh
ENV PATH="/opt/openbmc-sdk/sysroots/x86_64-oesdk-linux/usr/bin:$PATH"
ENTRYPOINT ["/bin/bash", "-c", "source /opt/openbmc-sdk/environment-setup-* && exec bash"]
```

New developers `docker pull` and start coding immediately.

4. Docker Compose for the full stack

For teams that need both the build environment and sstate mirror running locally:

```
# docker-compose.yml
services:
  builder:
    image: registry.company.com/openbmc-builder:2024.04
    volumes:
      - ./openbmc:/home/openbmc/openbmc
      - sstate-cache:/home/openbmc/openbmc/build/sstate-cache
      - downloads:/home/openbmc/openbmc/build/downloads
    stdin_open: true
    tty: true

  sstate-mirror:
    image: nginx:alpine
    volumes:
      - sstate-cache:/usr/share/nginx/html:ro
    ports:
      - "8080:80"

volumes:
  sstate-cache:
  downloads:
```

This gives each developer a local sstate mirror that other containers or native builds can reference.

Summary: Which Setup to Use

Scenario	Recommended Approach
Solo developer, full builds	Dev Container + named volumes for sstate/downloads
Solo developer, app only	Pre-built SDK (native or in container)
Team, full builds	Team container image + shared sstate HTTP mirror
Team, app developers	Pre-built SDK container from CI
Team, CI/CD	Nightly full build → publish sstate + SDK + container image

The common theme across all setups: **build expensive artifacts once, share them everywhere**. Whether it is sstate, SDK, or container images, avoid having each developer rebuild what has already been built.

Verify Your Setup

Run this checklist:

```
# Check Git
git --version
# ✓ Git 2.25 or later

# Check Python
python3 --version
# ✓ Python 3.8 or later

# Check disk space
df -h ~
# ✓ 100GB+ free

# Check RAM
free -h
# ✓ 16GB+ total

# Check QEMU
qemu-system-arm --version
# ✓ QEMU 4.0 or later

# If using Docker
docker --version
# ✓ Docker 20.10 or later
```

Next Steps

Your environment is ready! Continue to:

- ➔ **First Build** - Build OpenBMC and run in QEMU
- ➔ **Development Workflow** - Learn devtool for rapid iteration

Debug Builds (Optional)

For development and debugging, you may want to configure debug builds with sanitizers:

```
# Add to build/conf/local.conf for debug builds
DEBUG_BUILD = "1"
EXTRA_IMAGE_FEATURES += "dbg-pkgs"
```

For comprehensive debugging tools including ASan, UBSan, TSan, Valgrind, and kernel debug options, see the [Linux Debug Tools Guide](#).

Troubleshooting

Build fails with “out of memory”

Increase system RAM or add swap:

```
sudo fallocate -l 8G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
```

Permission denied with Docker

Ensure user is in the docker group:

```
sudo usermod -aG docker $USER  
# Then logout and login
```

"locale" errors during build

Set locale:

```
export LANG=en_US.UTF-8  
export LC_ALL=en_US.UTF-8
```

Tested on: Ubuntu 22.04, Fedora 38, Docker on Windows 11

First Build

Build and run your first OpenBMC image in QEMU.

Table of Contents

1. [Overview](#)
 2. [Clone OpenBMC](#)
 3. [Initialize Build Environment](#)
 4. [Build the Image](#)
 1. [Build Tips](#)
 5. [Run in QEMU](#)
 1. [Locate the Image](#)
 2. [Start QEMU](#)
 3. [Default Credentials](#)
 6. [Verify the Build](#)
 1. [Access via SSH](#)
 2. [Access via Redfish](#)
 3. [Access WebUI](#)
 7. [Common Operations](#)
 1. [Check System State](#)
 2. [Power Control](#)
 3. [View Sensors](#)
 8. [Troubleshooting](#)
 1. [Build Fails](#)
 2. [QEMU Won't Start](#)
 3. [Can't Connect](#)
 9. [Next Steps](#)
-

Overview

This guide walks you through building an OpenBMC image and running it in QEMU.

QEMU is not a compromise — it's the standard development environment used by professional OpenBMC developers. You'll have access to 100% of the OpenBMC software stack, including all D-Bus services, Redfish API, IPMI, and management interfaces.

Clone OpenBMC

```
# Clone the OpenBMC repository
git clone https://github.com/openbmc/openbmc.git
cd openbmc
```

Initialize Build Environment

```
# Set up environment for ast2600-evb machine (QEMU-compatible)
. setup ast2600-evb

# This creates a build directory and configures for ast2600-evb
```

Build the Image

```
# Build the full image (this takes 1-4 hours first time)
bitbake obmc-phosphor-image

# For faster iteration, build minimal image
bitbake obmc-phosphor-image-minimal
```

Build Tips

- First build downloads many sources - be patient
 - Subsequent builds are much faster (incremental)
 - Use `bitbake -c cleansstate <recipe>` to rebuild a package
-

Run in QEMU

QEMU provides a complete OpenBMC environment where you can:

- Access all D-Bus services exactly as on real hardware
- Test Redfish API endpoints with curl or any HTTP client
- Use ipmitool for IPMI commands
- SSH into the BMC and run commands
- Develop and debug services with full functionality

Locate the Image

```
# Find the built image
ls tmp/deploy/images/ast2600-evb/

# Key files:
# - obmc-phosphor-image-ast2600-evb.static.mtd (flash image)
# - fitImage-obmc-phosphor-initramfs-ast2600-evb (kernel)
```

Start QEMU

QEMU 7.0+ is required for ast2600-evb support. Stock QEMU on Ubuntu 20.04/22.04 may not include this machine. Check with `qemu-system-arm -machine help | grep ast2600`. If not found, see [Building QEMU](#).

```
# Run QEMU manually:
qemu-system-arm -m 1G \
  -M ast2600-evb \
  -nographic \
  -drive file=tmp/deploy/images/ast2600-evb/obmc-phosphor-image-ast2600-evb.static.mtd,format=raw,
  if=mtd \
  -net nic \
  -net user,hostfwd=tcp::2222-:22,hostfwd=tcp::2443-:443,hostfwd=udp::2623-:623
```

Default Credentials

Interface	Username	Password
SSH	root	OpenBmc
Redfish	root	OpenBmc
WebUI	root	OpenBmc

Verify the Build

Access via SSH

```
# Connect to QEMU (port 2222)
ssh -p 2222 root@localhost
# Password: OpenBmc

# Check BMC state
obmcutil state
```

Access via Redfish

```
# Query Redfish API
curl -k -u root:OpenBmc https://localhost:2443/redfish/v1/

# Get system info
curl -k -u root:OpenBmc https://localhost:2443/redfish/v1/Systems/system
```

Access WebUI

Open in browser: <https://localhost:2443/>

Common Operations

Check System State

```
# On BMC
obmcutil state

# Shows:
# BMC state: Ready
# Chassis state: On/Off
# Host state: Running/Off
```

Power Control

```
# Power on host
obmcutil poweron

# Power off host
obmcutil poweroff

# Reboot host
obmcutil hostreboot
```

View Sensors

```
# List sensors
busctl tree xyz.openbmc_project.Sensor

# Via Redfish
curl -k -u root:0penBmc \
    https://localhost:2443/redfish/v1/Chassis/chassis/Sensors
```

Troubleshooting

Build Fails

```
# Check for missing dependencies
bitbake obmc-phosphor-image -c checkpkg

# Clean and rebuild
bitbake -c cleansstate <problematic-recipe>
bitbake obmc-phosphor-image
```

QEMU Won't Start

```
# Check QEMU is installed
qemu-system-arm --version

# Verify image exists
ls -la tmp/deploy/images/ast2600-evb/*.mtd
```

Can't Connect

```
# Check QEMU is running
ps aux | grep qemu

# Verify port forwarding
netstat -tlnp | grep 2222
```

Next Steps

- [Development Workflow](#) - Iterate quickly with devtool
 - [OpenBMC Overview](#) - Understand the architecture
 - [D-Bus Guide](#) - Learn D-Bus communication
 - [Sensor Guide](#) - Configure sensors
-

Tested on: Ubuntu 22.04, OpenBMC master branch

Development Workflow

Iterate quickly on OpenBMC code using devtool, recipe customization, and build optimization.

Table of Contents

1. [Overview](#)
 2. [devtool Workflow](#)
 1. [When to Use devtool vs bitbake](#)
 2. [devtool modify - Extract Source Code](#)
 3. [devtool build - Incremental Build](#)
 4. [devtool reset - Discard Changes](#)
 5. [devtool finish - Create Patches](#)
 6. [Complete Walkthrough: Modify phosphor-logging](#)
 3. [Recipe Customization with bbappend](#)
 1. [bbappend Naming Conventions](#)
 2. [bbappend Directory Structure](#)
 3. [Common bbappend Use Cases](#)
 4. [Layer Priority](#)
 5. [When to Upstream vs Maintain Local Patches](#)
 4. [Build Optimization](#)
 1. [Enable ccache](#)
 2. [Configure sstate Cache](#)
 3. [Parallel Build Configuration](#)
 4. [Verify Optimization is Working](#)
 5. [Troubleshooting](#)
 1. [devtool Workspace Conflicts](#)
 2. [sstate Cache Misses](#)
 3. [Workspace Inspection Commands](#)
 6. [Next Steps](#)
-

Yocto Compatibility: This guide is tested with OpenBMC on Yocto Kirkstone and Scarthgap releases. Command syntax may vary slightly on older releases.

Overview

After completing your first build, you'll want to modify OpenBMC code and test changes quickly. A full [bitbake](#) rebuild can take 30+ minutes, but with the right workflow you can iterate in under 5 minutes.

This guide covers three essential techniques:

Technique	Use Case	Time Savings
devtool	Modify and test a single recipe	30 min → 2 min
bbappend	Sustainable recipe customization	Avoid upstream conflicts
Build optimization	Faster overall builds	2-4x speedup

devtool Workflow

`devtool` is your primary tool for rapid iteration. It extracts recipe source code to a workspace where you can edit, build, and test without full rebuilds.

When to Use devtool vs bitbake

Scenario	Use
Modifying existing recipe source code	<code>devtool</code>
Adding debug output to a service	<code>devtool</code>
Building entire image from scratch	<code>bitbake</code>
Creating permanent recipe customization	<code>bbappend</code>

devtool modify - Extract Source Code

Extract a recipe's source code to your workspace for editing:

```
# Initialize build environment first
cd openbmc
. setup ast2600-evb

# Extract phosphor-logging source to workspace
devtool modify phosphor-logging

# Source is now at:
# workspace/sources/phosphor-logging/
```

The workspace directory structure:

```
workspace/
├── sources/
│   ├── phosphor-logging/    # Editable source code
├── appends/
│   └── phosphor-logging.bbappend # Auto-generated append
```

The `devtool modify` command automatically creates a bbappend that redirects the build to use your workspace source instead of fetching from upstream.

devtool build - Incremental Build

After modifying source code, rebuild just that recipe:

```
# Build only phosphor-logging (fast!)
devtool build phosphor-logging

# Build output goes to the normal deploy directory
# tmp/work/*/phosphor-logging/*/image/
```

This builds only the modified recipe and its direct dependencies—typically completing in 1-2 minutes instead of 30+.

devtool reset - Discard Changes

When you want to abandon workspace changes and restore the original recipe:

```
# Discard workspace changes for phosphor-logging
devtool reset phosphor-logging

# Verify it's removed
devtool status
```

`devtool reset` permanently deletes your workspace source directory. Commit any changes you want to keep before running this command.

devtool finish - Create Patches

When you're satisfied with your changes and want to create a permanent patch:

```
# Commit your changes in the workspace
cd workspace/sources/phosphor-logging
git add -A
git commit -m "Add debug logging for sensor updates"

# Create bbappend with patches in your layer
devtool finish phosphor-logging meta-my-layer

# This creates:
# meta-my-layer/recipes-phosphor/logging/phosphor-logging_%.bbappend
# meta-my-layer/recipes-phosphor/logging/phosphor-logging/0001-Add-debug-logging.patch
```

Complete Walkthrough: Modify phosphor-logging

Let's walk through a complete example of adding debug output to phosphor-logging:

Step 1: Extract source

```
cd openbmc
. setup ast2600-evb
devtool modify phosphor-logging
```

Step 2: Make changes

```
cd workspace/sources/phosphor-logging

# Edit a source file (example: add debug log)
# Find the main logging implementation
vi lib/lg2_logger.cpp
```

Add a debug line to verify your changes are working:

```
// Near the top of a frequently-called function
std::cerr << "DEBUG: phosphor-logging modified successfully\n";
```

Step 3: Build

```
devtool build phosphor-logging
```

Step 4: Deploy and test on QEMU

```
# Build image with your changes
bitbake obmc-phosphor-image

# Start QEMU (requires QEMU 6.0+)
qemu-system-arm -m 1G -M ast2600-evb -nographic \
  -drive file=tmp/deploy/images/ast2600-evb/obmc-phosphor-image-ast2600-evb.static.mtd,format=raw,
if=mtd \
  -net nic -net user,hostfwd=tcp::2222-:22,hostfwd=tcp::2443-:443

# In another terminal, check logs
ssh -p 2222 root@localhost
journalctl -f | grep DEBUG
```

Step 5: Finalize or reset

```
# If satisfied, create permanent patch:
cd workspace/sources/phosphor-logging
git add -A && git commit -m "Add debug logging"
devtool finish phosphor-logging meta-my-layer

# Or discard changes:
devtool reset phosphor-logging
```

Recipe Customization with bbappend

For permanent, maintainable recipe customizations, use bbappend files instead of modifying upstream recipes directly.

bbappend Naming Conventions

bbappend files must match the recipe name exactly:

```
Recipe: meta-phosphor/recipes-phosphor/logging/phosphor-logging_git.bb
Append: meta-my-layer/recipes-phosphor/logging/phosphor-logging_%.bbappend
```

Pattern	Matches
<code>recipe_git.bbappend</code>	Only <code>recipe_git.bb</code>
<code>recipe_%.bbappend</code>	Any version of recipe (recommended)
<code>recipe_1.0.bbappend</code>	Only version 1.0

bbappend Directory Structure

Your bbappend should mirror the original recipe's path:

```
meta-your-layer/  
└─ recipes-phosphor/  
  └─ logging/  
    ├── phosphor-logging_%.bbappend  
    └─ phosphor-logging/  
      ├── 0001-your-patch.patch  
      └─ your-config-file.conf
```

Common bbappend Use Cases

Adding patches:

```
# phosphor-logging_%.bbappend  
FILESEXTRAPATHS:prepend := "${THISDIR}/${PN}:"  
  
SRC_URI += "file://0001-Add-custom-logging.patch"
```

Modifying SRCREV (pin to specific commit):

```
# phosphor-logging_%.bbappend  
SRCREV = "abc123def456..."
```

Adding dependencies:

```
# phosphor-logging_%.bbappend  
DEPENDS += "additional-library"  
RDEPENDS:${PN} += "runtime-dependency"
```

Modifying configuration options:

```
# phosphor-logging_%.bbappend  
EXTRA_OEMESON += "-Doption=value"
```

Adding extra files to the image:

```
# phosphor-logging_%.bbappend  
FILESEXTRAPATHS:prepend := "${THISDIR}/${PN}:"  
  
SRC_URI += "file://custom.conf"  
  
do_install:append() {  
    install -d ${D}${sysconfdir}  
    install -m 0644 ${WORKDIR}/custom.conf ${D}${sysconfdir}/  
}
```

Layer Priority

When multiple layers have bbappend files for the same recipe, priority determines which applies last (highest priority wins for conflicts):

```
# Check layer priorities
bitbake-layers show-layers

# Example output:
# layer          path                      priority
# meta           /path/to/poky/meta         5
# meta-phosphor  /path/to/openbmc/meta-phosphor 7
# meta-my-layer  /path/to/openbmc/meta-my-layer 10
```

Higher priority layers' appends are processed last. Set priority in your layer's `conf/layer.conf`:

```
BBFILE_PRIORITY_meta-yourlayer = "10"
```

When to Upstream vs Maintain Local Patches

Situation	Recommendation
Bug fix that benefits everyone	Upstream to OpenBMC
Platform-specific hardware support	Keep in machine layer
Temporary debug code	Use devtool, don't commit
Feature that may be rejected upstream	Start with bbappend, propose upstream

Maintaining local patches creates technical debt. Whenever possible, contribute fixes upstream and remove your bbappend once the fix is merged.

Build Optimization

Speed up your builds with caching and parallelization.

Enable ccache

ccache caches compiled objects, dramatically speeding up C/C++ recompilation:

```
# Add to conf/local.conf
INHERIT += "ccache"
CCACHE_DIR = "${TOPDIR}/ccache"
```

Expected improvement: **2-3x faster** C/C++ compilation on rebuilds.

Verify ccache is working:

```
# After a build, check cache statistics
ccache -s

# Look for "cache hit" entries
```

Configure sstate Cache

sstate (shared state) cache stores built recipe outputs, skipping rebuilds for unchanged recipes:

```
# Default location (already configured)
SSTATE_DIR = "${TOPDIR}/sstate-cache"

# Share sstate between multiple build directories
SSTATE_DIR = "/opt/openbmc-sstate"

# Use a read-only shared sstate mirror
SSTATE_MIRRORS = "file://.* file:///shared/sstate/PATH"
```

sstate cache can grow large (50GB+). Periodically clean old entries with [sstate-cache-management.sh](#) .

Parallel Build Configuration

Maximize CPU utilization during builds:

```
# Add to conf/local.conf

# Number of parallel BitBake tasks (recipe-level parallelism)
# Recommended: Number of CPU cores
BB_NUMBER_THREADS = "8"

# Number of parallel make jobs (within each recipe)
# Recommended: Number of CPU cores
PARALLEL_MAKE = "-j 8"
```

Formula for your system:

```
# Check your CPU cores
nproc

# Set both values to this number (or slightly less to leave headroom)
# For 8-core system:
BB_NUMBER_THREADS = "8"
PARALLEL_MAKE = "-j 8"

# For memory-constrained systems (< 32GB RAM), reduce values:
BB_NUMBER_THREADS = "4"
PARALLEL_MAKE = "-j 4"
```

Verify Optimization is Working

```
# Check ccache hit rate
ccache -s | grep "cache hit"

# Check sstate usage during build (look for "sstate" messages)
bitbake phosphor-logging -v 2>&1 | grep -i sstate

# Monitor parallel jobs
htop # Watch CPU usage during build
```

Troubleshooting

devtool Workspace Conflicts

Error: "Recipe is already in your workspace"

```
# Check what's in your workspace
devtool status

# Reset the conflicting recipe
devtool reset phosphor-logging

# Try again
devtool modify phosphor-logging
```

Error: "Workspace directory already exists"

```
# Remove stale workspace manually
rm -rf workspace/sources/phosphor-logging
rm -f workspace/append/append/phosphor-logging.bbappend

# Try again
devtool modify phosphor-logging
```

sstate Cache Misses

Symptom: Full rebuilds despite having sstate cache

Common causes:

1. **SRCREV changed** - Recipe source updated upstream

```
# Check current vs cached SRCREV
bitbake -e phosphor-logging | grep ^SRCREV=
```

2. **Configuration drift** - local.conf changes affect recipe hash

```
# Compare build signatures
bitbake-diffsigs tmp/stamps/*/phosphor-logging/*/do_compile.*
```

3. **Compiler or host tool version changed**

```
# Check for host contamination warnings
bitbake phosphor-logging 2>&1 | grep -i contamination
```

Workspace Inspection Commands

```
# List all recipes in workspace
devtool status

# Show details for a specific recipe
devtool status phosphor-logging

# Find workspace source directory
ls -la workspace/sources/

# Check git status of workspace source
cd workspace/sources/phosphor-logging
git status
git log --oneline -5
```

Next Steps

- [D-Bus Guide](#) - Understand how services communicate
 - [Sensor Guide](#) - Modify sensor configurations
 - [Machine Layer](#) - Create custom platform layers
 - [Unit Testing Guide](#) - Write GTest/GMock tests for your code
-

Tested on: Ubuntu 22.04, OpenBMC master branch (Kirkstone/Scarthgap)

Building QEMU for OpenBMC

Build QEMU from source when your distribution's package lacks ast2600-evb support.

Table of Contents

1. [When You Need This Guide](#)
 1. [Check Your QEMU Version](#)
 2. [Option 1: Use OpenBMC's Built-in QEMU \(Recommended\)](#)
 1. [Build QEMU via BitBake](#)
 2. [Use the Built QEMU](#)
 3. [Create a Convenience Script](#)
 3. [Option 2: Build QEMU from Source](#)
 1. [Install Build Dependencies](#)
 2. [Clone and Build QEMU](#)
 3. [Install System-Wide \(Optional\)](#)
 4. [Install to Custom Location](#)
 4. [Option 3: Use Container with QEMU](#)
 1. [Docker with QEMU](#)
 5. [Verifying QEMU Setup](#)
 1. [Check Available Machines](#)
 2. [Test Run](#)
 6. [Machine Selection Reference](#)
 7. [Troubleshooting](#)
 1. ["Machine type not found"](#)
 2. [Build Fails with Missing Dependencies](#)
 3. [Slow Performance](#)
 8. [Next Steps](#)
-

When You Need This Guide

Stock QEMU from Linux distribution package managers often **lacks ASPEED machine support**:

Distribution	Package QEMU Version	ast2600-evb Support
Ubuntu 20.04	QEMU 4.2	✗ No
Ubuntu 22.04	QEMU 6.2	⚠ Partial
Ubuntu 24.04	QEMU 8.2	✓ Yes
Fedora 38+	QEMU 7.2+	✓ Yes
Debian 11	QEMU 5.2	✗ No
Debian 12	QEMU 7.2	✓ Yes

Check Your QEMU Version

```
# Check version
qemu-system-arm --version

# Check if ast2600-evb is available
qemu-system-arm -machine help | grep ast2600
```

If `ast2600-evb` is not listed, you need to build QEMU from source.

Option 1: Use OpenBMC's Built-in QEMU (Recommended)

The OpenBMC build system includes a QEMU recipe with full ASPEED support. This is the easiest option.

Build QEMU via BitBake

```
cd openbmc

# Initialize environment (if not already done)
. setup ast2600-evb

# Build QEMU native tool
bitbake qemu-system-native

# Find the built QEMU
ls tmp/work/x86_64-linux/qemu-system-native/*/image/usr/bin/qemu-system-arm
```

Use the Built QEMU

```
# Set path to OpenBMC's QEMU
QEMU_PATH="tmp/work/x86_64-linux/qemu-system-native/*/image/usr/bin"

# Run with OpenBMC's QEMU
$QEMU_PATH/qemu-system-arm -m 1G \
  -M ast2600-evb \
  -nographic \
  -drive file=tmp/deploy/images/ast2600-evb/obmc-phosphor-image-ast2600-evb.static.mtd,format=raw,if=mtd \
  -net nic \
  -net user,hostfwd=tcp::2222-:22,hostfwd=tcp::2443-:443,hostfwd=udp::2623-:623
```

Create a Convenience Script

```
#!/bin/bash
# save as: run-qemu.sh

OPENBMC_DIR="${OPENBMC_DIR:-$(pwd)}"
QEMU=$(find $OPENBMC_DIR/tmp/work/x86_64-linux/qemu-system-native -name qemu-system-arm -type f | head -1)
IMAGE="$OPENBMC_DIR/tmp/deploy/images/ast2600-evb/obmc-phosphor-image-ast2600-evb.static.mtd"

if [ ! -f "$QEMU" ]; then
    echo "Error: QEMU not found. Run 'bitbake qemu-system-native' first."
    exit 1
fi

$QEMU -m 1G \
    -M ast2600-evb \
    -nographic \
    -drive file=$IMAGE,format=raw,if=mtd \
    -net nic \
    -net user,hostfwd=tcp::2222-:22,hostfwd=tcp::2443-:443,hostfwd=udp::2623-:623
```

Option 2: Build QEMU from Source

Build QEMU directly from upstream source for use outside the OpenBMC build environment.

Install Build Dependencies

```
# Ubuntu/Debian
sudo apt update
sudo apt install -y \
    git \
    build-essential \
    ninja-build \
    pkg-config \
    libglib2.0-dev \
    libpixman-1-dev \
    libslirp-dev \
    python3 \
    python3-venv

# Fedora
sudo dnf install -y \
    git \
    gcc \
    g++ \
    ninja-build \
    glib2-devel \
    pixman-devel \
    libslirp-devel \
    python3
```


Clone and Build QEMU

```
# Clone QEMU (use version 8.0+ for best ASPEED support)
git clone https://github.com/qemu/qemu.git
cd qemu
git checkout v8.2.0 # or latest stable

# Configure for ARM system emulation only (faster build)
./configure \
    --target-list=arm-softmmu \
    --enable-slirp \
    --disable-docs \
    --disable-werror

# Build (use -j for parallel compilation)
make -j$(nproc)

# Verify ast2600-evb is available
./build/qemu-system-arm -machine help | grep ast2600
```

Install System-Wide (Optional)

```
# Install to /usr/local
sudo make install

# Verify installation
qemu-system-arm --version
qemu-system-arm -machine help | grep ast2600
```

Install to Custom Location

```
# Install to home directory
./configure \
    --target-list=arm-softmmu \
    --prefix=$HOME/qemu \
    --enable-slirp

make -j$(nproc)
make install

# Add to PATH
echo 'export PATH="$HOME/qemu/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

Option 3: Use Container with QEMU

Run QEMU inside a container that has proper ASPEED support.

Docker with QEMU

```
# Dockerfile.qemu
FROM ubuntu:24.04

RUN apt-get update && apt-get install -y \
    qemu-system-arm \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /openbmc
```

```
# Build container
docker build -t openbmc-qemu -f Dockerfile.qemu .

# Run QEMU in container (mount image directory)
docker run -it --rm \
    -v $(pwd)/tmp/deploy/images:/images \
    -p 2222:2222 -p 2443:2443 -p 2623:2623/udp \
    openbmc-qemu \
    qemu-system-arm -m 1G -M ast2600-evb -nographic \
    -drive file=/images/ast2600-evb/obmc-phosphor-image-ast2600-evb.static.mtd,format=raw,if=mtd \
    -net nic \
    -net user,hostfwd=tcp::2222-:22,hostfwd=tcp::2443-:443,hostfwd=udp::2623-:623
```

Verifying QEMU Setup

Check Available Machines

```
# List all ASPEED machines
qemu-system-arm -machine help | grep -i aspeed

# Expected output includes:
# ast2500-evb      Aspeed AST2500 EVB (ARM926EJ-S)
# ast2600-evb      Aspeed AST2600 EVB (Cortex-A7)
# palmetto-bmc     OpenPOWER Palmetto BMC (ARM926EJ-S)
# romulus-bmc      OpenPOWER Romulus BMC (ARM1176)
# witherspoon-bmc  OpenPOWER Witherspoon BMC (ARM1176)
```

Test Run

```
# Quick test (will fail without image, but confirms QEMU works)
qemu-system-arm -M ast2600-evb -nographic -serial null
# Press Ctrl+A, X to exit
```

Machine Selection Reference

Machine	SoC	CPU	Memory	Use Case
ast2600-evb	AST2600	Cortex-A7 (dual)	1GB	Modern development (recommended)
ast2500-evb	AST2500	ARM926EJ-S	512MB	Legacy compatibility
romulus-bmc	AST2500	ARM1176	256MB	IBM OpenPOWER reference
witherspoon-bmc	AST2500	ARM1176	256MB	IBM OpenPOWER reference

Use **ast2600-evb** for new development. It has more memory (1GB vs 256MB), a modern CPU, and represents current hardware.

Troubleshooting

“Machine type not found”

```
qemu-system-arm: -M ast2600-evb: unsupported machine type
```

Solution: Your QEMU version is too old. Build from source using this guide.

Build Fails with Missing Dependencies

```
# Install all possible dependencies
sudo apt install -y \
    libglib2.0-dev libpixman-1-dev libslirp-dev \
    libcap-ng-dev libattr1-dev libaio-dev \
    libnuma-dev libseccomp-dev
```

Slow Performance

```
# Enable KVM if running ARM on ARM host
qemu-system-arm -M ast2600-evb -enable-kvm ...

# For x86 host, KVM won't help (cross-architecture)
# Allocate more CPU cores
qemu-system-arm -M ast2600-evb -smp 2 ...
```

Next Steps

- [First Build](#) - Build and run OpenBMC
- [Development Workflow](#) - Iterate quickly with devtool

Tested on: Ubuntu 22.04/24.04, Fedora 38/39, QEMU 7.2-8.2

Devtool Workflow Guide

Master the modify-build-deploy-debug cycle for rapid OpenBMC development using devtool and QEMU.

Table of Contents

1. [Overview](#)
2. [Prerequisites](#)
3. [Step 1: Extract Source with devtool modify](#)
 1. [Initialize Your Build Environment](#)
 2. [Extract the Recipe Source](#)
 3. [Verify the Extraction](#)
 4. [Understand the Workspace Layout](#)
4. [Step 2: Build Incrementally with devtool build](#)
 1. [Make a Code Change](#)
 2. [Build the Modified Recipe](#)
 3. [Locate the Build Output](#)
 4. [Rebuild After Additional Changes](#)
5. [Step 3: Deploy to Target](#)
 1. [Option A: Deploy via SCP \(Manual\)](#)
 2. [Option B: Deploy via devtool deploy-target \(Automated\)](#)
 3. [Undeploy from Target](#)
 4. [When to Use SCP vs deploy-target](#)
6. [Step 4: Iterative Debug with gdbserver](#)
 1. [Install gdbserver on the Target](#)
 2. [Attach gdbserver to a Running Service](#)
 3. [Start gdbserver with a New Process](#)
 4. [Connect from the Host with GDB](#)
 5. [Common GDB Commands for OpenBMC Debugging](#)
 6. [Build with Debug Symbols](#)
7. [Complete Worked Example: Modify phosphor-state-manager](#)
 1. [Background](#)
 2. [Step 1: Start QEMU](#)
 3. [Step 2: Extract the Source](#)
 4. [Step 3: Examine the Current State on the Target](#)
 5. [Step 4: Make the Code Change](#)
 6. [Step 5: Build](#)
 7. [Step 6: Deploy to QEMU](#)
 8. [Step 7: Restart and Verify](#)
 9. [Step 8: Iterate](#)

10. [Step 9: Clean Up](#)

8. [Quick Reference: devtool Commands](#)

9. [Troubleshooting](#)

1. [Issue: devtool build fails with "Nothing PROVIDES"](#)
2. [Issue: SCP fails with "Permission denied"](#)
3. [Issue: Service fails to start after deploying new binary](#)
4. [Issue: gdbserver connection refused](#)
5. [Issue: GDB cannot find source files](#)
6. [Debug Commands Cheat Sheet](#)

10. [Next Steps](#)

Overview

When you develop OpenBMC services, you need a fast feedback loop: change code, build it, push it to a running system, and verify the result. A full `bitbake obmc-phosphor-image` rebuild takes 30 minutes or more, which makes iterative development painfully slow.

This guide teaches you the **devtool modify-build-deploy** cycle that reduces your iteration time to under 5 minutes. You will learn how to extract a recipe's source code, make changes, build only the modified recipe, deploy the updated binary to a running QEMU instance, and debug it with gdbserver – all without rebuilding the entire image.

By the end of this guide, you will have hands-on experience modifying `phosphor-state-manager`, deploying your changes to QEMU, and verifying them with D-Bus commands.

Key concepts covered:

- Extracting recipe source with `devtool modify`
 - Incremental builds with `devtool build`
 - Deploying binaries via SCP and `devtool deploy-target`
 - Remote debugging with gdbserver and GDB
 - Complete worked example with phosphor-state-manager
-

Prerequisites

Before starting this guide, make sure you have:

- ☐ A working OpenBMC build environment ([Environment Setup](#))
- ☐ A completed first build of `obmc-phosphor-image` ([First Build](#))
- ☐ QEMU running with the ast2600-evb image ([Building QEMU](#))
- ☐ Familiarity with devtool basics ([Development Workflow](#))

This guide assumes you can SSH into your QEMU instance at `localhost:2222` with credentials `root` / `OpenBm`. Start QEMU before proceeding.

Step 1: Extract Source with devtool modify

The `devtool modify` command extracts a recipe's source code into a local workspace directory where you can edit it freely. The build system automatically redirects future builds of that recipe to use your local copy instead of fetching from upstream.

Initialize Your Build Environment

```
cd openbmc
. setup ast2600-evb
```

Extract the Recipe Source

```
# Extract phosphor-state-manager source to your workspace
devtool modify phosphor-state-manager
```

This command performs three actions:

1. Clones the recipe's source repository into `workspace/sources/phosphor-state-manager/`
2. Creates a bbappend file at `workspace/appends/phosphor-state-manager.bbappend`
3. Configures the build system to use your local source for all future builds

Verify the Extraction

```
# Check that the recipe is now in your workspace
devtool status

# Expected output:
# phosphor-state-manager /path/to/openbmc/build/ast2600-evb/workspace/sources/phosphor-state-manager
#
# Browse the source tree
ls workspace/sources/phosphor-state-manager/
```

You see the full source tree including `meson.build`, source files under `host_state_manager.cpp`, `chassis_state_manager.cpp`, and other service implementations.

Understand the Workspace Layout

```
workspace/
├── sources/
│   ├── phosphor-state-manager/    # Your editable source code (git repo)
│   │   ├── meson.build
│   │   ├── host_state_manager.cpp
│   │   ├── chassis_state_manager.cpp
│   │   ├── bmc_state_manager.cpp
│   │   └── ...
├── appends/
│   └── phosphor-state-manager.bbappend # Auto-generated build redirect
```

The workspace source directory is a full git repository. You can use `git diff`, `git stash`, and `git commit` to manage your changes just like any other git project.

Step 2: Build Incrementally with devtool build

After modifying source code, you use `devtool build` to compile only the changed recipe. This skips fetching, patching, and building all other recipes, reducing build time from 30+ minutes to 1-3 minutes.

Make a Code Change

Open a source file and add a visible marker so you can verify your change on the target:

```
cd workspace/sources/phosphor-state-manager
```

Edit `bmc_state_manager.cpp` and add a log message near the top of the constructor or an initialization function:

```
// Add this line after the existing includes
#include <phosphor-logging/lg2.hpp>

// Inside the BMCStateManager constructor or relevant init path, add:
lg2::info("DEVTOOL-TEST: phosphor-state-manager modified successfully");
```

The exact location depends on the current upstream code. Look for the `BMCStateManager` constructor in `bmc_state_manager.cpp` and add the log line at the beginning of the function body.

Build the Modified Recipe

```
# Return to the build directory
cd /path/to/openbmc/build/ast2600-evb

# Build only phosphor-state-manager
devtool build phosphor-state-manager
```

The build takes 1-3 minutes. Watch for any compilation errors in the output. If you see errors, fix them in the source file and run `devtool build` again.

Locate the Build Output

After a successful build, find the compiled binaries:

```
# Find the built binary
find tmp/work/*/phosphor-state-manager/*/image -name "phosphor-bmc-state-manager" -type f

# Typical location:
# tmp/work/armv7ahf-vfpv4d16-openbmc-linux-gnueabi/phosphor-state-manager/1.0+git*/image/usr/bin/phosphor-bmc-state-manager
```

You can also find all installed files from the recipe by listing the `image/` directory. This shows you exactly what the recipe installs on the target system.

Rebuild After Additional Changes

The devtool build cycle is fully incremental. Each time you edit source files and run `devtool build`, only the changed files are recompiled:

```
# Edit source...
vi workspace/sources/phosphor-state-manager/bmc_state_manager.cpp

# Rebuild (only recompiles changed files)
devtool build phosphor-state-manager
```

Step 3: Deploy to Target

You have two options for getting your modified binary onto the running QEMU instance: manual SCP or `devtool deploy-target`. Each has advantages depending on your workflow.

Option A: Deploy via SCP (Manual)

SCP gives you precise control over which files you copy and where they go. This is useful when you want to replace a single binary without touching anything else.

Find the binary on the target:

```
# SSH into QEMU to find where the binary lives
ssh -p 2222 root@localhost "which phosphor-bmc-state-manager"
# Output: /usr/bin/phosphor-bmc-state-manager
```

Copy the built binary to the target:

```
# From your build host, SCP the new binary
scp -P 2222 \
    tmp/work/armv7ahf-vfpv4d16-openbmc-linux-gnueabi/phosphor-state-manager/1.0+git*/image/usr/bin/p
    phosphor-bmc-state-manager \
    root@localhost:/usr/bin/phosphor-bmc-state-manager
```

The OpenBMC root filesystem is a read-only overlay by default on some configurations. If SCP fails with a “read-only file system” error, remount the filesystem read-write first: `ssh -p 2222 root@localhost "mount -o remount,rw /"`.

Restart the service to load the new binary:

```
ssh -p 2222 root@localhost "systemctl restart xyz.openbmc_project.State.BMC"
```

Verify the service is running with your changes:

```
ssh -p 2222 root@localhost "systemctl status xyz.openbmc_project.State.BMC"
ssh -p 2222 root@localhost "journalctl -u xyz.openbmc_project.State.BMC --no-pager -n 20"
```

Look for your `DEVTOL-TEST` log message in the journal output.

Option B: Deploy via devtool deploy-target (Automated)

`devtool deploy-target` automates the file copy process. It deploys all files that the recipe installs (binaries, configuration files, libraries) to the correct locations on the target.

Set up SSH key access (one-time setup):


```
# Generate an SSH key if you do not have one
ssh-keygen -t ed25519 -f ~/.ssh/id_openbmc -N ""

# Copy the key to the QEMU target
ssh-copy-id -p 2222 -i ~/.ssh/id_openbmc root@localhost
```

Deploy all recipe files to the target:

```
devtool deploy-target phosphor-state-manager root@localhost:2222
```

This command:

1. Identifies all files the recipe installs (binaries, configs, systemd units)
2. Copies them to the correct paths on the target via SSH
3. Preserves file permissions and ownership

`devtool deploy-target` uses SSH internally, so you need password-less SSH access (key-based) or you will be prompted for the password multiple times.

Restart the service after deployment:

```
ssh -p 2222 root@localhost "systemctl restart xyz.openbmc_project.State.BMC"
```

Verify the deployment:

```
ssh -p 2222 root@localhost "journalctl -u xyz.openbmc_project.State.BMC --no-pager -n 20"
```

Undeploy from Target

If you need to revert the target to its original state, use `devtool undeploy-target`:

```
devtool undeploy-target phosphor-state-manager root@localhost:2222
```

This removes all files that `deploy-target` installed and restores the originals.

When to Use SCP vs deploy-target

Situation	Recommended Method
Replace a single binary quickly	SCP
Deploy recipe with many files (bins + configs + units)	<code>devtool deploy-target</code>
Target has no SSH key set up	SCP with password
Need to revert changes easily	<code>devtool deploy-target</code> (supports undeploy)
Deploying to real hardware over network	<code>devtool deploy-target</code>

Step 4: Iterative Debug with gdbserver

When log messages are not enough to diagnose a problem, you can attach a debugger to a running service on the target. This section shows you how to use gdbserver on the QEMU target and connect to it with GDB on your build host.

Install gdbserver on the Target

The default OpenBMC image may not include gdbserver. You have two ways to add it.

Option A: Add gdbserver to the image (recommended for repeated use):

Add the following to your `conf/local.conf` before building:

```
# In conf/local.conf
IMAGE_INSTALL:append = " gdbserver"
```

Then rebuild the image:

```
bitbake obmc-phosphor-image
```

Option B: Build and deploy gdbserver separately:

```
# Build gdbserver as a standalone package
devtool modify gdb
devtool build gdb

# Deploy just gdbserver to target
scp -P 2222 \
    tmp/work/armv7ahf-vfpv4d16-openbmc-linux-gnueabi/gdb/*/image/usr/bin/gdbserver \
    root@localhost:/usr/bin/gdbserver
```

Attach gdbserver to a Running Service

On the QEMU target, find the process ID of the service you want to debug and attach gdbserver:

```
# SSH into the target
ssh -p 2222 root@localhost

# Find the PID of the service
pidof phosphor-bmc-state-manager
# Example output: 1234

# Attach gdbserver to the running process on port 3333
gdbserver --attach :3333 1234
```

gdbserver pauses the process and waits for a GDB client to connect.

Attaching gdbserver pauses the target process. Other OpenBMC services that depend on the paused service may time out or report errors. This is expected during debugging.

Start gdbserver with a New Process

If you prefer to start the service under gdbserver from the beginning (to catch initialization issues):

```
# On the target, stop the service first
ssh -p 2222 root@localhost

systemctl stop xyz.openbmc_project.State.BMC

# Start under gdbserver
gdbserver :3333 /usr/bin/phosphor-bmc-state-manager
```

Connect from the Host with GDB

On your build host, you need an ARM-compatible GDB. The OpenBMC SDK provides one, or you can use the one built by bitbake:

```
# Find the cross GDB in your build environment
find tmp/work/x86_64-linux -name "arm-openbmc-linux-gnueabi-gdb" -type f 2>/dev/null

# Or use the SDK's GDB if you have the SDK installed
# Typically at: /opt/openbmc-sdk/sysroots/x86_64-pokysdk-linux/usr/bin/arm-openbmc-linux-gnueabi/arm
-openbmc-linux-gnueabi-gdb
```

Connect to the remote gdbserver:

```
# Start GDB with the unstripped binary (contains debug symbols)
arm-openbmc-linux-gnueabi-gdb \
    tmp/work/armv7ahf-vfpv4d16-openbmc-linux-gnueabi/phosphor-state-manager/1.0+git*/image/usr/bin/p
hosphor-bmc-state-manager
```

Inside the GDB session:

```
(gdb) target remote localhost:3333
Remote debugging using localhost:3333
...

(gdb) break bmc_state_manager.cpp:42
Breakpoint 1 at 0x...: file bmc_state_manager.cpp, line 42.

(gdb) continue
Continuing.
```

QEMU forwards port 3333 from the target if you add `-net user,hostfwd=tcp::3333-:3333` to your QEMU command line. Alternatively, since you already have SSH access, you can use an SSH tunnel: `ssh -p 2222 -L 3333 3:localhost:3333 root@localhost`.

Common GDB Commands for OpenBMC Debugging

Command	Description
<code>bt</code>	Print backtrace of the current thread
<code>info threads</code>	List all threads in the process
<code>thread 2</code>	Switch to thread 2
<code>p variable_name</code>	Print the value of a variable
<code>watch variable_name</code>	Break when a variable changes
<code>info breakpoints</code>	List all breakpoints
<code>continue</code>	Resume execution
<code>next</code>	Step over (execute next line)
<code>step</code>	Step into (enter function calls)
<code>finish</code>	Run until current function returns

Build with Debug Symbols

For the most useful debugging experience, ensure your recipe builds with debug symbols. Add this to `conf/local.conf`:

```
# Enable debug symbols for all recipes
DEBUG_BUILD = "1"

# Or enable debug symbols for a specific recipe only
DEBUG_FLAGS:pn-phosphor-state-manager = "-g -Og"
```

Debug builds produce larger binaries and run slightly slower. Use this only during active debugging and remove it when you are done.

Complete Worked Example: Modify phosphor-state-manager

This section walks through the entire modify-build-deploy-verify cycle from start to finish. Follow each step exactly to build confidence with the workflow.

Background

`phosphor-state-manager` manages BMC, chassis, and host power states in OpenBMC. It exposes D-Bus interfaces that other services use to query and control system power. You will add a custom property to the BMC state manager and verify it appears on D-Bus.

Step 1: Start QEMU

If QEMU is not already running, start it in a separate terminal:

```
cd openbmc/build/ast2600-evb

qemu-system-arm -m 1G \
  -M ast2600-evb \
  -nographic \
  -drive file=tmp/deploy/images/ast2600-evb/obmc-phosphor-image-ast2600-evb.static.mtd,format=raw,
  if=mtd \
  -net nic \
  -net user,hostfwd=tcp::2222-:22,hostfwd=tcp::2443-:443,hostfwd=udp::2623-:623
```

Wait for the login prompt (takes 2-3 minutes). Verify SSH access:

```
ssh -p 2222 root@localhost "obmcutil state"
```

Step 2: Extract the Source

```
cd openbmc
. setup ast2600-evb

# Extract phosphor-state-manager
devtool modify phosphor-state-manager

# Verify
devtool status
```

Step 3: Examine the Current State on the Target

Before making changes, observe the current D-Bus state so you can compare later:

```
# Query the BMC state object
ssh -p 2222 root@localhost \
  "busctl introspect xyz.openbmc_project.State.BMC \
  /xyz/openbmc_project/state/bmc0"
```

You see properties like `CurrentBMCState`, `RequestedBMCTransition`, and others. Note the current values.

```
# Check the service journal
ssh -p 2222 root@localhost \
  "journalctl -u xyz.openbmc_project.State.BMC --no-pager -n 10"
```

Step 4: Make the Code Change

Navigate to the workspace source and add a log message:

```
cd workspace/sources/phosphor-state-manager
```

Edit `bmc_state_manager.cpp`. Find the `BMC::BMC` constructor (or the `discoverInitialState` method) and add a log line:

```
// Add near the top of the constructor or discoverInitialState():
lg2::info("DEVTOOL-GUIDE: BMC State Manager started with devtool modifications");
```

Save the file.

Step 5: Build

```
# Return to build directory
cd /path/to/openbmc/build/ast2600-evb

# Build only the modified recipe
devtool build phosphor-state-manager
```

Watch the output for `NOTE: Tasks Summary: ...` at the end. A successful build shows zero failed tasks.

If the build fails with a meson error, check that your code change has correct C++ syntax. Common mistakes include missing semicolons and unmatched braces. Run `devtool build phosphor-state-manager` again after fixing.

Step 6: Deploy to QEMU

Use SCP to deploy the modified binary:

```
# Find the built binary path
BINARY=$(find tmp/work/armv7ahf-vfpv4d16-openbmc-linux-gnueabi/phosphor-state-manager/ \
    -path "*/image/usr/bin/phosphor-bmc-state-manager" -type f | head -1)

echo "Deploying: $BINARY"

# Copy to target
scp -P 2222 "$BINARY" root@localhost:/usr/bin/phosphor-bmc-state-manager
```

Step 7: Restart and Verify

Restart the service on the target and check for your log message:

```
# Restart the BMC state manager service
ssh -p 2222 root@localhost "systemctl restart xyz.openbmc_project.State.BMC"

# Wait a moment for the service to start
sleep 2

# Check service status
ssh -p 2222 root@localhost "systemctl status xyz.openbmc_project.State.BMC"
```

Verify your log message appears in the journal:

```
ssh -p 2222 root@localhost \
    "journalctl -u xyz.openbmc_project.State.BMC --no-pager -n 30 | grep DEVTOOL-GUIDE"
```

You should see output similar to:

```
<timestamp> ast2600-evb phosphor-bmc-state-manager[1234]: DEVTOOL-GUIDE: BMC State Manager started w
ith devtool modifications
```

Verify the D-Bus interface is still functioning correctly:

```
# Query BMC state - should return valid state
ssh -p 2222 root@localhost \
    "busctl get-property xyz.openbmc_project.State.BMC \
    /xyz/openbmc_project/state/bmc0 \
    xyz.openbmc_project.State.BMC CurrentBMCState"

# Expected: s "xyz.openbmc_project.State.BMC.BMCState.Ready"
```

Step 8: Iterate

Now that you have a working deploy cycle, you can iterate quickly:

1. Edit source in `workspace/sources/phosphor-state-manager/`
2. Run `devtool build phosphor-state-manager`
3. SCP the binary to the target
4. Restart the service
5. Check the logs

Each iteration takes 2-3 minutes instead of 30+.

Step 9: Clean Up

When you are done experimenting, reset your workspace:

```
# Remove the recipe from your workspace
devtool reset phosphor-state-manager

# Verify it is removed
devtool status
```

`devtool reset` deletes your workspace source directory. If you want to keep your changes, commit them with `git commit` inside the workspace source directory first. You can also use `devtool finish phosphor-state-manager meta-your-layer` to save changes as a bbappend with patches.

Quick Reference: devtool Commands

Command	Purpose
<code>devtool modify <recipe></code>	Extract recipe source to workspace
<code>devtool build <recipe></code>	Build only the modified recipe
<code>devtool deploy-target <recipe> root@<host>:<port></code>	Deploy built files to target via SSH
<code>devtool undeploy-target <recipe> root@<host>:<port></code>	Remove deployed files from target
<code>devtool finish <recipe> <layer></code>	Create bbappend + patches in a layer
<code>devtool reset <recipe></code>	Remove recipe from workspace
<code>devtool status</code>	List all recipes in workspace

Troubleshooting

Issue: devtool build fails with “Nothing PROVIDES”

Symptom: Build error mentioning missing dependencies.

Cause: The recipe depends on other recipes that have not been built yet.

Solution:

1. Build the full image first to populate the sstate cache:

```
bitbake obmc-phosphor-image
```

2. Then use `devtool build` :

```
devtool build phosphor-state-manager
```

Issue: SCP fails with “Permission denied”

Symptom: `scp: /usr/bin/phosphor-bmc-state-manager: Permission denied`

Cause: The target filesystem is mounted read-only or the file is in use.

Solution:

1. Remount the filesystem read-write:

```
ssh -p 2222 root@localhost "mount -o remount,rw /"
```

2. Stop the service before copying:

```
ssh -p 2222 root@localhost "systemctl stop xyz.openbmc_project.State.BMC"
scp -P 2222 "$BINARY" root@localhost:/usr/bin/phosphor-bmc-state-manager
ssh -p 2222 root@localhost "systemctl start xyz.openbmc_project.State.BMC"
```

Issue: Service fails to start after deploying new binary

Symptom: `systemctl status` shows the service in “failed” state.

Cause: The new binary may have a runtime error, missing library, or ABI mismatch.

Solution:

1. Check the journal for error details:

```
ssh -p 2222 root@localhost \
    "journalctl -u xyz.openbmc_project.State.BMC --no-pager -n 50"
```

2. Verify the binary is the correct architecture:

```
ssh -p 2222 root@localhost "file /usr/bin/phosphor-bmc-state-manager"
# Expected: ELF 32-bit LSB executable, ARM, ...
```

3. Check for missing shared libraries:


```
ssh -p 2222 root@localhost "ldd /usr/bin/phosphor-bmc-state-manager"
```

Issue: gdbserver connection refused

Symptom: GDB reports "Connection refused" when connecting to the remote target.

Cause: Port forwarding is not configured or gdbserver is not running.

Solution:

1. Verify gdbserver is running on the target:

```
ssh -p 2222 root@localhost "ps aux | grep gdbserver"
```

2. Use an SSH tunnel instead of direct port forwarding:

```
# Set up tunnel in a separate terminal
ssh -p 2222 -L 3333:localhost:3333 root@localhost -N

# Then connect GDB to localhost:3333
(gdb) target remote localhost:3333
```

Issue: GDB cannot find source files

Symptom: GDB shows `No source file named...` when setting breakpoints by file name.

Cause: The debug symbol paths in the binary do not match your host filesystem.

Solution: Set the source search path in GDB:

```
(gdb) set substitute-path /usr/src/debug workspace/sources/phosphor-state-manager
(gdb) directory workspace/sources/phosphor-state-manager
```

Debug Commands Cheat Sheet

```
# Check service status on target
ssh -p 2222 root@localhost "systemctl status xyz.openbmc_project.State.BMC"

# View live logs
ssh -p 2222 root@localhost "journalctl -u xyz.openbmc_project.State.BMC -f"

# List D-Bus objects owned by state manager
ssh -p 2222 root@localhost "busctl tree xyz.openbmc_project.State.BMC"

# Introspect a specific D-Bus object
ssh -p 2222 root@localhost \
    "busctl introspect xyz.openbmc_project.State.BMC \
    /xyz/openbmc_project/state/bmc0"

# Check what files a recipe installs
devtool build phosphor-state-manager
find tmp/work/*/phosphor-state-manager/*/image -type f

# Check workspace status
devtool status
```

Next Steps

- [Development Workflow](#) - Learn bbappend customization and build optimization
 - [D-Bus Guide](#) - Understand D-Bus interfaces used by OpenBMC services
 - [State Manager Guide](#) - Deep dive into power state management
 - [Unit Testing Guide](#) - Write tests for your code changes
-

Tested on: QEMU ast2600-evb, OpenBMC master branch (Kirkstone/Scarthgap) Last updated: 2026-02-06

Gerrit Contribution Guide

Submit your first patch to the OpenBMC project using Gerrit code review.

Table of Contents

1. [Overview](#)
2. [Prerequisites](#)
3. [Step 1: Create Your Gerrit Account](#)
 1. [Sign In and Configure](#)
 2. [Sign the Contributor License Agreement \(CLA\)](#)
4. [Step 2: Configure SSH Access](#)
 1. [Generate and Register Your Key](#)
 2. [Configure SSH for Gerrit](#)
 3. [Verify Connectivity](#)
5. [Step 3: Install the commit-msg Hook](#)
 1. [Clone and Install the Hook](#)
 2. [Verify the Hook](#)
 3. [Add the Hook to an Existing Clone](#)
6. [Step 4: Submit a Patch](#)
 1. [Create a Topic Branch](#)
 2. [Commit with Sign-off](#)
 3. [Commit Message Guidelines](#)
 4. [Push to Gerrit](#)
7. [Step 5: Update a Patch After Review](#)
 1. [Rebase on Latest Master](#)
8. [Step 6: Understand CI Integration](#)
 1. [Jenkins Results](#)
 2. [Common CI Failure Reasons](#)
9. [Step 7: Understand the Upstream-First Kernel Policy](#)
 1. [Why Upstream-First Matters](#)
 2. [What Requires Upstream Submission](#)
 3. [Referencing Upstream Commits](#)
10. [Quick Reference](#)
11. [Troubleshooting](#)
 1. [Issue: Push rejected with "missing Change-Id"](#)
 2. [Issue: Push rejected with "committer email does not match"](#)
 3. [Issue: Push rejected with "not permitted"](#)
 4. [Issue: SSH connection times out](#)
 5. [Issue: Gerrit shows "Cannot merge"](#)

12. References

1. [Official Resources](#)
2. [Related Guides](#)
3. [Community](#)

Overview

OpenBMC uses [Gerrit](#) for code review, not GitHub pull requests. Every change to the OpenBMC codebase goes through Gerrit review before it merges into the upstream repository.

This guide walks you through the entire contribution workflow: account creation, SSH setup, commit conventions, patch submission, reviewer feedback, CI validation, and the upstream-first kernel policy.

Key concepts covered:

- Creating a Gerrit account and configuring SSH access
- Installing the commit-msg hook for Change-Id generation
- Submitting patches with `git push origin HEAD:refs/for/master`
- Updating patches with `git commit --amend`
- Understanding Jenkins CI results and common failures
- OpenBMC's upstream-first kernel policy

Prerequisites

Before starting this guide, make sure you have:

- ☐ A working OpenBMC build environment ([Environment Setup](#))
- ☐ A completed first build of `obmc-phosphor-image` ([First Build](#))
- ☐ A GitHub account (used for Gerrit authentication)
- ☐ Git installed and configured with your name and email

OpenBMC Gerrit uses your GitHub account for authentication. You do not need to create a separate Gerrit account.

Step 1: Create Your Gerrit Account

Sign In and Configure

1. Open <https://gerrit.openbmc.org/> and click **Sign In**.
2. Select **GitHub** as the authentication provider and authorize the application.
3. In Gerrit **Settings**, verify your display name and email under **Profile** and **Email Addresses**.

Verify your local git configuration matches your Gerrit email:

```
git config --global user.name "Your Full Name"
git config --global user.email "your-email@example.com"
```

Your git `user.email` must match the email registered in Gerrit. Mismatched emails cause a “committer email does not match” rejection on push.

Sign the Contributor License Agreement (CLA)

OpenBMC requires a signed CLA before you can submit patches:

1. In Gerrit Settings, go to **Agreements**.
2. Click **New Contributor Agreement**, select individual or corporate, and sign it.

If you contribute on behalf of your employer, check whether your company has already signed a corporate CLA. Contact the [OpenBMC mailing list](#) if you are unsure.

Step 2: Configure SSH Access

Gerrit uses SSH for git push and pull operations. Generate an SSH key pair and register the public key with Gerrit.

Generate and Register Your Key

```
# Generate an SSH key (skip if you already have one)
ssh-keygen -t ed25519 -C "your-email@example.com" -f ~/.ssh/id_openbmc
```

Copy the public key and add it in Gerrit under **Settings > SSH Keys**:

```
cat ~/.ssh/id_openbmc.pub
```

Configure SSH for Gerrit

Add a host entry to `~/.ssh/config` (create the file if it does not exist):

```
Host gerrit.openbmc.org
  User your-gerrit-username
  IdentityFile ~/.ssh/id_openbmc
  Port 29418
```

Replace `your-gerrit-username` with your Gerrit username from Settings > Profile.

Verify Connectivity

```
ssh -p 29418 gerrit.openbmc.org
```

A successful connection displays a “Welcome to Gerrit Code Review” banner.

If you see “Permission denied (publickey)”, verify that your SSH config points to the correct key file and that the public key is registered in Gerrit.

Step 3: Install the commit-msg Hook

Gerrit tracks patches using a unique `Change-Id` in each commit message. The `commit-msg` hook generates this identifier automatically on every `git commit`.

Clone and Install the Hook

```
# Clone the repository (example: phosphor-logging)
git clone ssh://gerrit.openbmc.org:29418/openbmc/phosphor-logging
cd phosphor-logging

# Install the commit-msg hook
scp -p -P 29418 gerrit.openbmc.org:hooks/commit-msg .git/hooks/
chmod +x .git/hooks/commit-msg
```

Verify the Hook

```
echo "" >> README.md
git add README.md
git commit -m "Test commit-msg hook"
git log -1
```

You should see a `Change-Id:` line appended to the commit message. After verifying, reset the test:

```
git reset HEAD~1
git checkout -- README.md
```

If the `Change-Id` line is missing, the hook is not installed correctly. Repeat the `scp` command above. Without a `Change-Id`, Gerrit rejects your push.

Add the Hook to an Existing Clone

If you already cloned from GitHub, add the Gerrit remote and hook:

```
git remote add gerrit ssh://gerrit.openbmc.org:29418/openbmc/<repo-name>
scp -p -P 29418 gerrit.openbmc.org:hooks/commit-msg .git/hooks/
chmod +x .git/hooks/commit-msg
```

Step 4: Submit a Patch

Create a Topic Branch

Always work on a topic branch, never directly on `master`:

```
git checkout master
git pull origin master
git checkout -b fix-typo-in-readme
```

Commit with Sign-off

OpenBMC requires a `Signed-off-by` line in every commit (Developer Certificate of Origin). Use the `-s` flag:

```
git add README.md
git commit -s
```

Write your commit message following this format:

component: Short summary (50 chars or less)

Longer description explaining what you changed and why. Wrap at 72 characters. Focus on motivation, not mechanics.

Signed-off-by: Your Name <your-email@example.com>

Change-Id: Iabc123... (auto-generated by the hook)

The `-s` flag adds the `Signed-off-by` line automatically from your git config. You do not need to type it manually.

Commit Message Guidelines

Rule	Example
Start with component name and colon	<code>phosphor-logging: Fix memory leak</code>
Subject line under 50 characters	Short and descriptive
Imperative mood in subject	"Fix bug" not "Fixed bug"
Blank line after subject	Separates subject from body
Body wrapped at 72 characters	Readable in terminal and Gerrit
Explain <i>why</i> , not just <i>what</i>	The diff shows what changed
Include <code>Signed-off-by</code> (use <code>-s</code>)	Required for all patches
Keep the <code>Change-Id</code> line	Required for Gerrit tracking

Push to Gerrit

Push your commit using the special `refs/for/master` reference:

```
git push origin HEAD:refs/for/master
```

Do **not** push to `refs/heads/master`. That attempts a direct push to master and is rejected. Always use `refs/for/master` to create a review.

A successful push shows a URL to your new review:

```
remote: https://gerrit.openbmc.org/c/openbmc/phosphor-logging/+/12345 component: Short summary [NEW]
```

You can group related patches with a topic:

```
git push origin HEAD:refs/for/master%topic=fix-memory-leak
```

Step 5: Update a Patch After Review

Reviewers may request changes. To update your patch, amend the existing commit and push again – do not create a new commit.

```
# Make the requested changes
vi README.md
git add README.md

# Amend the existing commit (keeps the same Change-Id)
git commit --amend

# Push the updated patch
git push origin HEAD:refs/for/master
```

Gerrit detects the same `Change-Id` and creates a new **patch set** on the existing review.

Do not create a new commit for review updates. A new commit generates a new Change-Id and creates a separate review instead of updating the existing one.

Rebase on Latest Master

If your patch falls behind master, rebase before pushing:

```
git fetch origin
git rebase origin/master
# Resolve any conflicts, then:
git rebase --continue
git push origin HEAD:refs/for/master
```

Rebasing preserves the `Change-Id`. Gerrit correctly associates the rebased commit with the original review.

Step 6: Understand CI Integration

Every patch pushed to Gerrit triggers a Jenkins CI build. Jenkins compiles the code, runs tests, and reports results on the review.

Jenkins Results

Label	Meaning
Verified +1	Build and tests passed
Verified -1	Build or tests failed

Jenkins must report `Verified +1` before your patch can merge. A human reviewer must also approve with `Code-Review +2`. Click the build URL in the Jenkins comment to see full console output.

Common CI Failure Reasons

Compilation error: Fix errors locally with `devtool build <recipe>`, amend, and push again.

Unit test failure: Reproduce locally by building and checking test output. Fix the failing test before pushing.

Formatting failure: OpenBMC enforces `clang-format` for C/C++. Format your code before committing:

```
clang-format -i path/to/your/file.cpp
```

Merge conflict: Rebase on latest master, resolve conflicts, and push.

Always build and test locally before pushing to Gerrit. This saves CI resources and reduces turnaround time.

Step 7: Understand the Upstream-First Kernel Policy

OpenBMC follows a strict **upstream-first** policy for Linux kernel changes. Any kernel modification must first be submitted to the upstream Linux kernel (kernel.org) before it can be accepted into OpenBMC.

Why Upstream-First Matters

- **Long-term maintenance:** Upstream patches are maintained by the Linux kernel community.
- **Quality assurance:** The upstream review process catches bugs early.
- **Broad compatibility:** Upstream drivers work across all platforms.
- **License compliance:** Keeps the OpenBMC kernel tree clean.

What Requires Upstream Submission

Change Type	Upstream Required?	Example
New device driver	Yes	ASPEED ADC driver
Device tree binding	Yes	New sensor node
Bug fix in existing driver	Yes	I2C timeout fix
Board device tree (.dts)	Yes (new SoC/board)	aspeed-bmc-vendor-board.dts
Kernel config change	No	Enable/disable CONFIG option
Backport of accepted patch	No	Cherry-pick from mainline

Referencing Upstream Commits

When backporting an upstream kernel patch, reference the commit in your message:

```
ARM: dts: aspeed: Add sensor nodes for new platform

Backport upstream commit abc123def456 from Linux 6.x to the
OpenBMC kernel tree.

Upstream: https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=abc123def456
Signed-off-by: Your Name <your-email@example.com>
Change-Id: I...
```

If your change does not touch upstream kernel code (for example, a Yocto recipe change), the upstream-first policy does not apply.

Quick Reference

Task	Command
Clone from Gerrit	<code>git clone ssh://gerrit.openbmc.org:29418/openbmc/<repo></code>
Install commit-msg hook	<code>scp -p -P 29418 gerrit.openbmc.org:hooks/commit-msg .git/hooks/</code>
Submit new patch	<code>git push origin HEAD:refs/for/master</code>
Submit with topic	<code>git push origin HEAD:refs/for/master%topic=my-topic</code>
Update existing patch	<code>git commit --amend && git push origin HEAD:refs/for/master</code>
Check SSH access	<code>ssh -p 29418 gerrit.openbmc.org</code>
Query your open patches	<code>ssh -p 29418 gerrit.openbmc.org gerrit query owner:self status:open</code>
Download a patch locally	<code>git fetch origin refs/changes/45/12345/1 && git checkout FETCH_HEAD</code>

Troubleshooting

Issue: Push rejected with "missing Change-Id"

Symptom: `git push` fails with "missing Change-Id in message footer".

Cause: The `commit-msg` hook is not installed.

Solution: Install the hook, then amend the commit to trigger it:

```
scp -p -P 29418 gerrit.openbmc.org:hooks/commit-msg .git/hooks/  
chmod +x .git/hooks/commit-msg  
git commit --amend --no-edit  
git push origin HEAD:refs/for/master
```

Issue: Push rejected with "committer email does not match"

Cause: Your git `user.email` differs from Gerrit.

Solution: Update your email and amend:

```
git config --global user.email "your-gerrit-email@example.com"  
git commit --amend --reset-author --no-edit  
git push origin HEAD:refs/for/master
```

Issue: Push rejected with "not permitted"

Cause: Pushing to `refs/heads/master` instead of `refs/for/master`, or CLA not signed.

Solution: Use the correct reference and sign the CLA in Gerrit Settings > Agreements:

```
git push origin HEAD:refs/for/master
```

Issue: SSH connection times out

Cause: Firewall blocking port 29418.

Solution: Test with `nc -zv gerrit.openbmc.org 29418`. If blocked, ask your network administrator to allow outbound TCP on port 29418.

Issue: Gerrit shows "Cannot merge"

Cause: A conflicting patch was merged while your review was pending.

Solution: Rebase and push:

```
git fetch origin && git rebase origin/master
# Resolve conflicts, then:
git rebase --continue
git push origin HEAD:refs/for/master
```

References

Official Resources

- [OpenBMC Gerrit](#) - Code review platform
- [OpenBMC Contributing Guide](#) - Official contribution guidelines
- [Gerrit User Guide](#) - Gerrit documentation

Related Guides

- [Environment Setup](#) - Development environment
- [Development Workflow](#) - Iterate with devtool
- [Devtool Workflow Guide](#) - Modify-build-deploy cycle

Community

- [OpenBMC Mailing List](#) - Discussion and questions
- [OpenBMC Discord](#) - Real-time chat

Tested on: Ubuntu 22.04, OpenBMC master branch (Kirkstone/Scarthgap), Gerrit 3.x Last updated: 2026-02-06

Architecture

Understand how OpenBMC is designed and how its components work together.

What You'll Learn

- OpenBMC's overall architecture and design philosophy
- The D-Bus communication system that connects all services
- State management for BMC, chassis, and host
- How to navigate the codebase and find components

Key Concepts

D-Bus: The Nervous System

OpenBMC uses D-Bus as the central communication backbone. All services expose their data and functionality through D-Bus interfaces, enabling:

- Loose coupling between components
- Dynamic service discovery
- Language-agnostic communication
- Easy debugging and introspection

Phosphor Services

The "phosphor-*" repositories contain the core OpenBMC services:

- **phosphor-state-manager**: System state control
- **phosphor-logging**: Event and error logging
- **phosphor-dbus-interfaces**: Standard interface definitions
- And many more...

Guides in This Section

Guide	Description	Difficulty
OpenBMC Overview	Architecture, build system, community	Beginner
D-Bus Fundamentals	D-Bus concepts and usage	Beginner
State Management	BMC/Chassis/Host states	Intermediate
Custom D-Bus Services	Build your own D-Bus service with <code>sdbus++</code>	Intermediate
Systemd Boot Ordering	Service dependencies and boot sequence	Intermediate

Prerequisite Knowledge

Before diving into architecture:

- Complete the [Getting Started](#) section
 - Have a working QEMU environment
 - Basic understanding of Linux services (systemd)
-

Table of contents

- [OpenBMC Overview](#)
- [D-Bus Guide](#)
- [State Manager Guide](#)
- [Custom D-Bus Services Guide](#)
- [Systemd Boot Ordering Guide](#)

OpenBMC Overview

Deep dive into OpenBMC architecture, build system, and project organization.

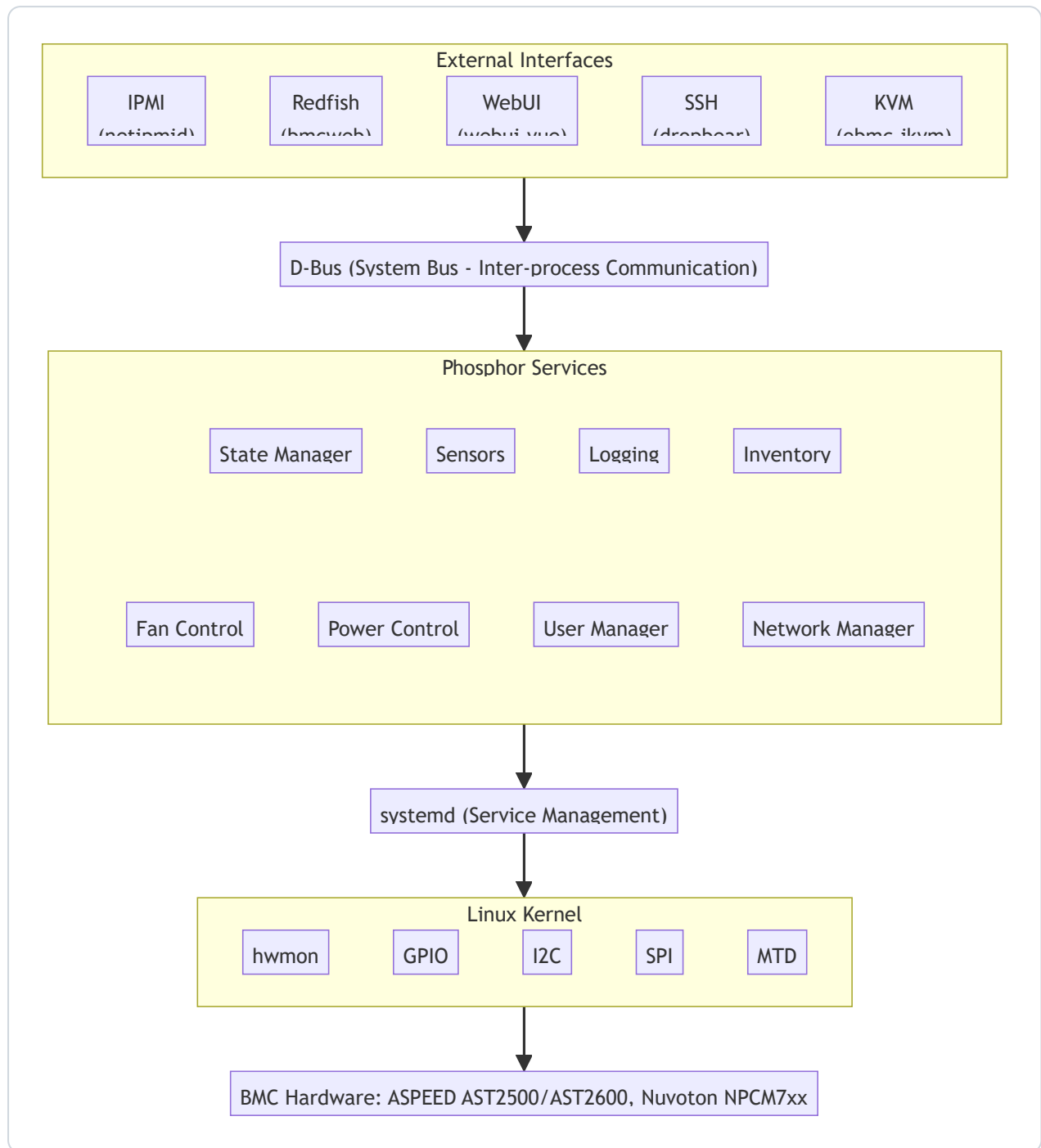
Table of Contents

1. [Architecture Overview](#)
 1. [High-Level Architecture](#)
 2. [Key Architectural Principles](#)
 2. [Project Organization](#)
 1. [Repository Structure](#)
 2. [Key Repositories](#)
 3. [Meta-Layer Structure](#)
 3. [Build System](#)
 1. [Yocto Concepts](#)
 2. [Build Workflow](#)
 3. [Key BitBake Commands](#)
 4. [Build Directory Structure](#)
 4. [D-Bus Architecture](#)
 1. [D-Bus Concepts](#)
 2. [Common D-Bus Tools](#)
 5. [Service Management](#)
 1. [Key systemd Commands](#)
 2. [Service Dependencies](#)
 6. [Community and Contribution](#)
 1. [Getting Help](#)
 2. [Contributing](#)
 3. [Coding Standards](#)
 7. [Next Steps](#)
 8. [References](#)
-

Architecture Overview

OpenBMC follows a modular, service-oriented architecture built on Linux.

High-Level Architecture



► ASCII-art version (for comparison)

Key Architectural Principles

1. D-Bus as Central Nervous System

- All services communicate via D-Bus
- Enables loose coupling and dynamic discovery
- Provides introspection and debugging capabilities

2. Modular Services

- Each function is a separate daemon

- Services can be replaced or customized independently
- Failures are isolated

3. Configuration-Driven

- Hardware configuration via JSON (Entity Manager)
- Service behavior via configuration files
- Build-time options via Yocto/meson

4. Standard Interfaces

- Redfish for modern management
- IPMI for legacy compatibility
- Standard Linux interfaces (hwmon, GPIO)

Project Organization

Repository Structure

The OpenBMC project spans multiple repositories:

```
github.com/openbmc/
├── openbmc/                # Main integration repository
├── phosphor-*/            # Core services
│   ├── phosphor-state-manager/
│   ├── phosphor-logging/
│   ├── phosphor-dbus-interfaces/
│   └── ...
├── bmcweb/                # HTTP/Redfish server
├── dbus-sensors/          # Sensor daemons
├── entity-manager/        # Hardware discovery
├── meta-phosphor/         # Yocto layer
├── docs/                  # Documentation
└── ...
```

Key Repositories

Repository	Purpose
openbmc/openbmc	Main build integration, machine configs
phosphor-dbus-interfaces	D-Bus interface definitions (YAML)
phosphor-state-manager	BMC/Chassis/Host state control
phosphor-logging	Event logging
dbus-sensors	Sensor monitoring daemons
bmcweb	HTTP server, Redfish, WebUI backend
entity-manager	Hardware discovery and configuration
phosphor-pid-control	Thermal/fan control

Meta-Layer Structure

OpenBMC uses Yocto layers:

```
openbmc/
├─ meta-phosphor/           # Core OpenBMC layer
│  └─ recipes-phosphor/    # Phosphor service recipes
│     └─ classes/          # BitBake classes
├─ meta-openpower/         # OpenPOWER-specific
├─ meta-ibm/               # IBM machines
├─ meta-google/            # Google machines
├─ meta-facebook/          # Facebook/Meta machines
├─ meta-aspeed/            # ASPEED SoC support
└─ meta-<vendor>/          # Your machine layer
```

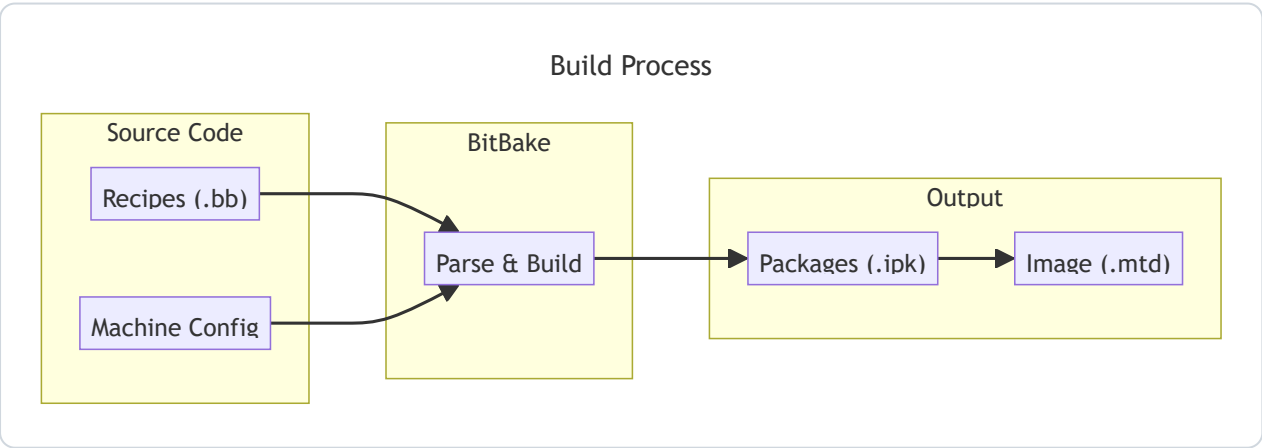
Build System

OpenBMC uses [Yocto Project](#) for building.

Yocto Concepts

Concept	Description
Recipe (.bb)	Build instructions for a package
Layer	Collection of related recipes
Machine	Target hardware configuration
Image	Complete firmware image recipe
BitBake	Build tool that processes recipes

Build Workflow



► ASCII-art version (click to expand)

Key BitBake Commands

```
# Initialize environment for a machine
. setup <machine>

# Build complete image
bitbake obmc-phosphor-image

# Build single package
bitbake phosphor-logging

# Clean and rebuild
bitbake -c clean phosphor-logging
bitbake phosphor-logging

# Generate SDK
bitbake obmc-phosphor-image -c populate_sdk

# List available machines
ls meta-*/meta-*/conf/machine/*.conf

# Check recipe dependencies
bitbake -g phosphor-logging
```

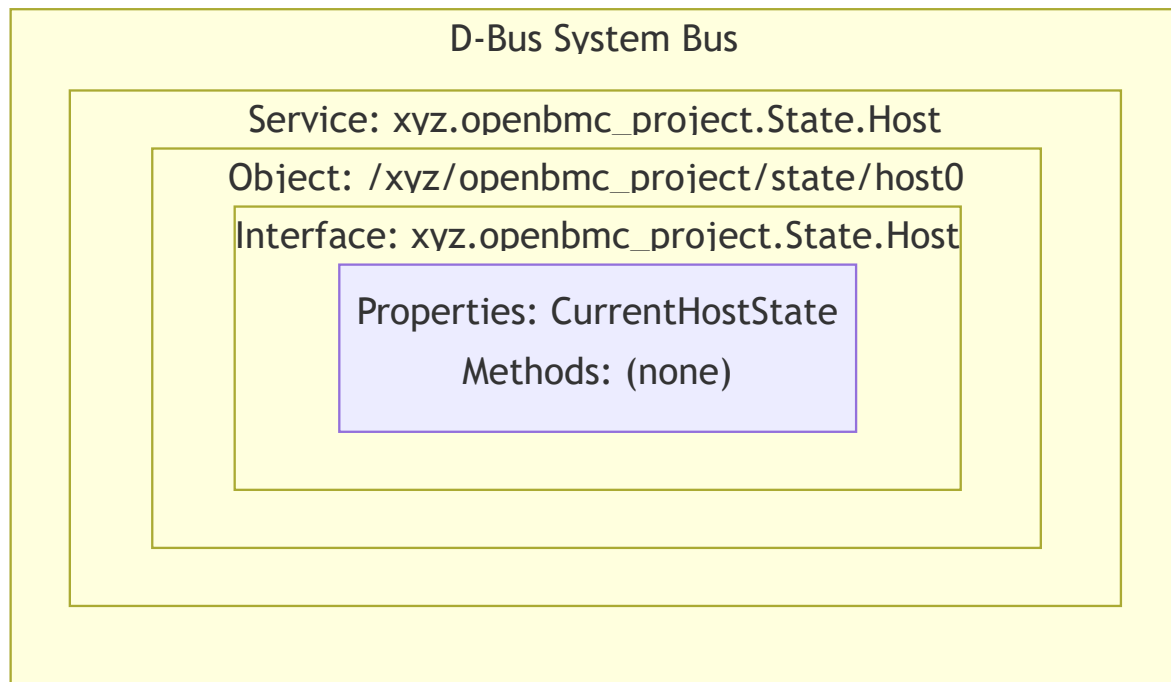
Build Directory Structure

```
build/
├── conf/
│   ├── local.conf      # Build configuration
│   └── bblayers.conf   # Active layers
├── tmp/
│   ├── work/           # Package build directories
│   ├── deploy/
│   │   ├── images/     # Final images
│   │   ├── ipk/         # Package files
│   │   └── sdk/         # SDK installers
│   └── sstate-cache/   # Build cache
└── downloads/          # Source tarballs
```

D-Bus Architecture

D-Bus is the backbone of OpenBMC communication.

D-Bus System Bus Structure



► ASCII-art version (click to expand)

Common D-Bus Tools

```
# List all services
busctl list

# Show objects for a service
busctl tree xyz.openbmc_project.State.Host

# Inspect an object
busctl introspect xyz.openbmc_project.State.Host \
    /xyz/openbmc_project/state/host0

# Get a property
busctl get-property xyz.openbmc_project.State.Host \
    /xyz/openbmc_project/state/host0 \
    xyz.openbmc_project.State.Host CurrentHostState

# Call a method
busctl call xyz.openbmc_project.State.Host \
    /xyz/openbmc_project/state/host0 \
    xyz.openbmc_project.State.Host \
    MethodName <signature> <arguments>
```

Service Management

OpenBMC uses systemd for service management.

Key systemd Commands

```
# List OpenBMC services
systemctl list-units 'xyz.openbmc_project.*'

# Check service status
systemctl status xyz.openbmc_project.State.Host.service

# View service logs
journalctl -u xyz.openbmc_project.State.Host.service -f

# Restart a service
systemctl restart xyz.openbmc_project.State.Host.service

# Check boot time
systemd-analyze
systemd-analyze blame
```

Service Dependencies

Services declare dependencies via systemd units:

```
# Example: /lib/systemd/system/xyz.openbmc_project.Example.service
[Unit]
Description=Example Service
After=xyz.openbmc_project.ObjectMapper.service
Requires=xyz.openbmc_project.ObjectMapper.service

[Service]
ExecStart=/usr/bin/example-daemon

[Install]
WantedBy=multi-user.target
```

Community and Contribution

Getting Help

Resource	URL
Mailing List	lists.ozlabs.org/listinfo/openbmc
Discord	discord.gg/openbmc
Documentation	github.com/openbmc/docs

Contributing

OpenBMC uses **Gerrit** for code review:

1. Clone with commit-msg hook:

```
git clone ssh://openbmc.gerrit/openbmc/phosphor-logging
scp -p openbmc.gerrit:hooks/commit-msg .git/hooks/
```

2. Create feature branch
3. Make changes, commit with Change-Id
4. Push for review:

```
git push origin HEAD:refs/for/master
```

5. Address review feedback
6. Merge when approved

Coding Standards

- C++: Follow [OpenBMC C++ style guide](#)
 - Python: PEP 8
 - Commit messages: Conventional format with Change-Id
-

Next Steps

Continue learning:

- **D-Bus Guide** - Master D-Bus communication
 - **State Manager** - Understand system state control
-

References

- [OpenBMC GitHub](#)
 - [Yocto Project Documentation](#)
 - [D-Bus Specification](#)
 - [systemd Documentation](#)
-

Tested on: OpenBMC master branch, QEMU romulus

D-Bus Guide

Master D-Bus communication in OpenBMC.

Table of Contents

1. [Overview](#)
2. [D-Bus Concepts](#)
 1. [Core Components](#)
 2. [Terminology](#)
 3. [Naming Conventions](#)
3. [Using busctl](#)
 1. [List Services](#)
 2. [Explore Objects](#)
 3. [Introspect Objects](#)
 4. [Read Properties](#)
 5. [Set Properties](#)
 6. [Call Methods](#)
 7. [Monitor Signals](#)
4. [phosphor-dbus-interfaces](#)
 1. [YAML Format](#)
 2. [Type Signatures](#)
5. [sdbusplus Library](#)
 1. [Creating a D-Bus Client](#)
 2. [Creating a D-Bus Server](#)
 3. [Async with sdbusplus and Boost.Asio](#)
6. [phosphor-objmgr \(Object Mapper\)](#)
 1. [Query Objects by Interface](#)
 2. [Get Service for Object](#)
 3. [Using mapper CLI](#)
 4. [mapper Client Library \(libmapper\)](#)
7. [Unit Failure Monitor](#)
8. [D-Bus Associations](#)
 1. [Association Structure](#)
 2. [Query Associations](#)
9. [Legacy D-Bus REST API](#)
10. [Troubleshooting](#)
 1. [Common Issues](#)
 2. [Debug Commands](#)
11. [Next Steps](#)

- 12. [Examples](#)
- 13. [References](#)
- 14. [Deep Dive](#)
 - 1. [D-Bus Message Flow Architecture](#)
 - 2. [Object Mapper Internals](#)
 - 3. [sdbusplus Code Generation Pipeline](#)
 - 4. [D-Bus Policy Configuration](#)
 - 5. [D-Bus Type System Internals](#)
 - 6. [PropertiesChanged Signal Mechanism](#)
 - 7. [Source Code References](#)

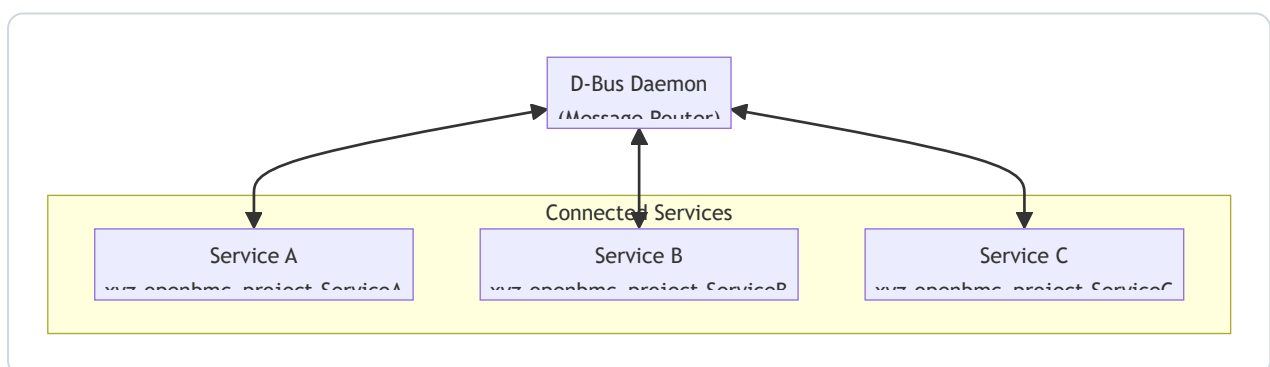
Overview

D-Bus is the central communication system in OpenBMC. All services expose their functionality through D-Bus, enabling:

- **Loose coupling:** Services don't need to know about each other's implementation
- **Discovery:** Services can be found dynamically at runtime
- **Introspection:** Interfaces can be explored without documentation
- **Language agnostic:** Any language with D-Bus bindings can participate

D-Bus Concepts

Core Components



► ASCII-art version (for comparison)

Terminology

Term	Description	Example
Bus	Communication channel	System bus (shared by all)
Service	Application connected to bus	<code>xyz.openbmc_project.State.Host</code>
Object	Resource exposed by service	<code>/xyz/openbmc_project/state/host0</code>
Interface	Contract defining methods/properties	<code>xyz.openbmc_project.State.Host</code>
Property	Data exposed by object	<code>CurrentHostState</code>
Method	Action that can be invoked	<code>Reboot()</code>
Signal	Broadcast notification	<code>PropertiesChanged</code>

Naming Conventions

OpenBMC uses consistent naming:

```
Service:  xyz.openbmc_project.<Category>.<Name>
Object:   /xyz/openbmc_project/<category>/<name>
Interface: xyz.openbmc_project.<Category>.<Name>
```

Examples:

- `xyz.openbmc_project.State.Host` - Host state service
- `/xyz/openbmc_project/state/host0` - Host 0 state object
- `xyz.openbmc_project.Sensor.Value` - Sensor value interface

Using busctl

`busctl` is the primary tool for D-Bus interaction.

List Services

```
# List all services
busctl list

# Filter OpenBMC services
busctl list | grep xyz.openbmc_project
```


Explore Objects

```
# Show object tree for a service
busctl tree xyz.openbmc_project.State.Host

# Example output:
# └─/xyz
#   └─/xyz/openbmc_project
#     └─/xyz/openbmc_project/state
#       └─/xyz/openbmc_project/state/host0
```

Introspect Objects

```
# Show interfaces, methods, properties, signals
busctl introspect xyz.openbmc_project.State.Host \
    /xyz/openbmc_project/state/host0

# Example output:
# NAME                                     TYPE      SIGNATURE RESULT/VALUE
# xyz.openbmc_project.State.Host         interface -          -
# .CurrentHostState                      property  s          "xyz.openbmc_project..."
# .RequestedHostTransition               property  s          "xyz.openbmc_project..."
```

Read Properties

```
# Get a single property
busctl get-property xyz.openbmc_project.State.Host \
    /xyz/openbmc_project/state/host0 \
    xyz.openbmc_project.State.Host \
    CurrentHostState

# Output: s "xyz.openbmc_project.State.Host.HostState.Off"
```

Set Properties

```
# Set a property (request host power on)
busctl set-property xyz.openbmc_project.State.Host \
    /xyz/openbmc_project/state/host0 \
    xyz.openbmc_project.State.Host \
    RequestedHostTransition s \
    "xyz.openbmc_project.State.Host.Transition.On"
```

Call Methods

```
# Call a method
busctl call xyz.openbmc_project.ObjectMapper \
    /xyz/openbmc_project/object_mapper \
    xyz.openbmc_project.ObjectMapper \
    GetSubTree sias "/" 0 1 "xyz.openbmc_project.Sensor.Value"
```

Monitor Signals

```
# Watch for property changes
busctl monitor xyz.openbmc_project.State.Host

# Watch all D-Bus traffic (verbose)
dbus-monitor --system
```

phosphor-dbus-interfaces

Interface definitions live in the [phosphor-dbus-interfaces](#) repository.

YAML Format

Interfaces are defined in YAML:

```
# xyz/openbmc_project/Example/MyInterface.interface.yaml
description: >
  Example interface demonstrating YAML format.

properties:
  - name: MyProperty
    type: string
    description: An example string property.

  - name: MyValue
    type: int32
    description: An example integer property.

methods:
  - name: DoSomething
    description: Performs an action.
    parameters:
      - name: Input
        type: string
        description: Input parameter.
    returns:
      - name: Result
        type: boolean
        description: Success or failure.

signals:
  - name: SomethingHappened
    description: Emitted when something happens.
    properties:
      - name: What
        type: string
```

Type Signatures

D-Bus uses type signatures:

Signature	Type	C++ Type
<code>s</code>	String	<code>std::string</code>
<code>b</code>	Boolean	<code>bool</code>
<code>i</code>	Int32	<code>int32_t</code>
<code>u</code>	UInt32	<code>uint32_t</code>
<code>x</code>	Int64	<code>int64_t</code>
<code>t</code>	UInt64	<code>uint64_t</code>
<code>d</code>	Double	<code>double</code>
<code>o</code>	Object path	<code>sdbusplus::message::object_path</code>
<code>a</code>	Array	<code>std::vector<T></code>
<code>(...)</code>	Struct	<code>std::tuple<...></code>
<code>a{sv}</code>	Dict	<code>std::map<std::string, std::variant<...>></code>

sdbusplus Library

`sdbusplus` is the C++ library for D-Bus in OpenBMC.

Source Reference: [sdbusplus](#)

- Bus API: [include/sdbusplus/bus.hpp](#)
- Examples: [example/](#)

Creating a D-Bus Client

```
#include <sdbusplus/bus.hpp>
#include <iostream>

int main()
{
    // Connect to system bus
    auto bus = sdbusplus::bus::new_default();

    // Create method call
    auto method = bus.new_method_call(
        "xyz.openbmc_project.State.Host",           // Service
        "/xyz/openbmc_project/state/host0",         // Object
        "org.freedesktop.DBus.Properties",          // Interface
        "Get"                                         // Method
    );

    // Add arguments
    method.append("xyz.openbmc_project.State.Host", // Interface
        "CurrentHostState");                         // Property

    // Call and get reply
    auto reply = bus.call(method);

    // Extract result
    std::variant<std::string> value;
    reply.read(value);

    std::cout << "Host state: " << std::get<std::string>(value) << "\n";

    return 0;
}
```

Creating a D-Bus Server

```
#include <sdbusplus/bus.hpp>
#include <sdbusplus/server.hpp>
#include <xyz/openbmc_project/Example/MyInterface/server.hpp>

// Generated server binding
using MyInterfaceInherit = sdbusplus::server::object_t<
    sdbusplus::xyz::openbmc_project::Example::server::MyInterface>;

class MyService : public MyInterfaceInherit
{
public:
    MyService(sdbusplus::bus_t& bus, const char* path) :
        MyInterfaceInherit(bus, path)
    {
        // Initialize properties
        myProperty("initial value");
    }

    // Property getter/setter are auto-generated

    // Implement methods
    bool doSomething(std::string input) override
    {
        std::cout << "doSomething called with: " << input << "\n";
        return true;
    }
};

int main()
{
    auto bus = sdbusplus::bus::new_default();

    // Request well-known name
    bus.request_name("xyz.openbmc_project.Example");

    // Create server object
    MyService myService(bus, "/xyz/openbmc_project/example");

    // Process D-Bus events
    while (true)
    {
        bus.process_discard();
        bus.wait();
    }

    return 0;
}
```

Async with sdbusplus and Boost.Asio

```
#include <sdbusplus/asio/connection.hpp>
#include <sdbusplus/asio/object_server.hpp>
#include <boost/asio.hpp>

int main()
{
    boost::asio::io_context io;
    auto conn = std::make_shared<sdbusplus::asio::connection>(io);

    conn->request_name("xyz.openbmc_project.Example");

    sdbusplus::asio::object_server server(conn);

    auto iface = server.add_interface(
        "/xyz/openbmc_project/example",
        "xyz.openbmc_project.Example.MyInterface"
    );

    // Add property
    std::string myValue = "hello";
    iface->register_property("MyProperty", myValue,
        // Setter
        [](const std::string& newValue, std::string& value) {
            value = newValue;
            return true;
        }
    );

    iface->initialize();

    io.run();
    return 0;
}
```

phosphor-objmgr (Object Mapper)

The Object Mapper tracks all D-Bus objects and their interfaces.

Query Objects by Interface

```
# Find all objects implementing Sensor.Value
busctl call xyz.openbmc_project.ObjectMapper \
    /xyz/openbmc_project/object_mapper \
    xyz.openbmc_project.ObjectMapper \
    GetSubTree sias "/" 0 1 "xyz.openbmc_project.Sensor.Value"
```

The mapper exposes several other query methods on `xyz.openbmc_project.ObjectMapper` beyond `GetSubTree` and `GetObject`:

- `GetAncestors` – returns the objects that are ancestors of a given path (filtered to the requested interfaces). Useful for walking *up* the tree, e.g. from a sensor to its owning chassis.
- `GetAssociatedSubTree` / `GetAssociatedSubTreePaths` – like `GetSubTree`, but restricted to paths that are an association endpoint of a supplied `associatedPath`.
- `GetAssociatedSubTreeById` – resolves a subtree from an inventory `id` plus an `association` name, so you don't need the full object path up front.

Get Service for Object

```
# Find which service owns an object
busctl call xyz.openbmc_project.ObjectMapper \
  /xyz/openbmc_project/object_mapper \
  xyz.openbmc_project.ObjectMapper \
  GetObject sas "/xyz/openbmc_project/state/host0" 0
```

Using mapper CLI

```
# List objects with interface
mapper get-subtree / xyz.openbmc_project.Sensor.Value

# Get service for object
mapper get-service /xyz/openbmc_project/state/host0
```

mapper Client Library (libmapper)

For C services that need to talk to the mapper without hand-rolling `busctl`-style calls, phosphor-objmgr ships a small C client library, **libmapper** (`libmapper/mapper.h`). It wraps the common lookups and, importantly, provides an **async “wait for object” primitive** used during boot ordering:

Function	Purpose
<code>mapper_wait_async(bus, event, objs[], cb, userdata, &wait)</code>	Fire <code>cb</code> once every object path in <code>objs[]</code> has appeared on the bus
<code>mapper_subtree_async(bus, event, namespace, interface, cb, userdata, &subtree, op)</code>	Async subtree query (backed by <code>GetSubTreePaths</code>)
<code>mapper_get_object(bus, obj, &reply)</code>	Synchronous <code>GetObject</code> – returns the services + interfaces on a path
<code>mapper_get_service(bus, obj, &service)</code>	Convenience wrapper returning just the owning service name

The async wait powers the templated `mapper-wait@.service` unit: a service can declare `After=` on `mapper-wait@-xyz-openbmc_project-...service` and systemd will block it until the required D-Bus object exists, rather than racing the object's provider at startup.

```
# Block a unit until a D-Bus object is present (path is systemd-escaped)
[Unit]
After=mapper-wait@-xyz-openbmc_project-state-host0.service
Requires=mapper-wait@-xyz-openbmc_project-state-host0.service
```

Unit Failure Monitor

phosphor-objmgr also builds a small standalone tool, **phosphor-unit-failure-monitor** (from [fail-monitor/](#)), that watches a systemd unit and reacts when it fails. It reads the `ActiveState` property of a *source* unit and, if that state is `failed`, either starts or stops a *target* unit.

Flag	Description
<code>-s</code> , <code>--source</code>	The unit to monitor (required)
<code>-t</code> , <code>--target</code>	The unit to start or stop when the source fails (required)
<code>-a</code> , <code>--action</code>	<code>start</code> or <code>stop</code> – what to do to the target (required)

A typical use is a hardware watchdog takeover: if a fan-control service dies, hand control to a safe fallback target.

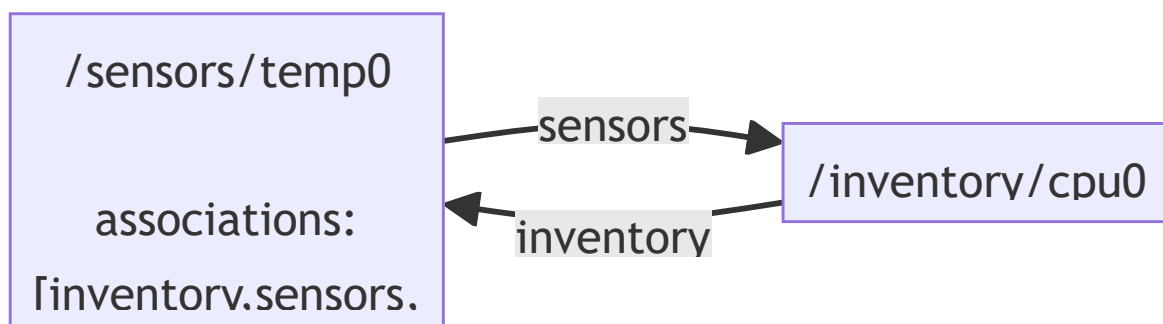
```
# fan-watchdog-monitor@.service (from meta-ibm)
[Service]
ExecStart=/usr/bin/phosphor-unit-failure-monitor \
  --source %i \
  --target obmc-fan-watchdog-takeover.target \
  --action start
```

Internally the monitor calls `StartUnit` / `StopUnit` on `org.freedesktop.systemd1.Manager` after reading `ActiveState` from the source unit.

D-Bus Associations

Associations link related objects together.

Association Structure



► ASCII-art version (click to expand)

Query Associations

```
# Get associations for an object
busctl introspect xyz.openbmc_project.ObjectMapper \
    /xyz/openbmc_project/sensors/temperature/cpu0

# Look for 'associations' property and endpoints
```

Legacy D-Bus REST API

Before Redfish became the primary north-bound API, bmcweb exposed a **generic REST interface that maps directly onto D-Bus** ([features/openbmc_rest/openbmc_dbus_rest.hpp](#)). It is still present and handy for debugging, but is **largely superseded by Redfish** for production clients.

Treat this as a debugging/legacy interface. New integrations should use Redfish; the generic D-Bus REST API bypasses Redfish's schema and access model.

The routes mirror the D-Bus object tree under `/xyz/...` and `/org/...`, plus a few helpers:

Route	Method	Action
<code>/xyz/<path></code> , <code>/org/<path></code> >	GET	Read an object's interfaces and properties as JSON
<code>/xyz/<path>/enumerate</code>	GET	Walk the whole subtree and dump every object/interface/property
<code>/xyz/<path></code>	PUT / POST T	Write a property or invoke a D-Bus method
<code>/xyz/<path></code>	DELETE	Delete the object (if the owning service supports it)
<code>/bus/system/</code> , <code>/list/</code>	GET	Enumerate bus names / all object paths

JSON is translated to and from native D-Bus types on the fly, using the object's introspection data to pick signatures ([convertJsonToDbus](#) / [convertDBusToJSON](#)).

```
# Enumerate every sensor object and its properties in one call
curl -k -u root:0penBmc \
    https://localhost/xyz/openbmc_project/sensors/enumerate

# Read a single property set
curl -k -u root:0penBmc \
    https://localhost/xyz/openbmc_project/state/host0

# Write a property (PUT the D-Bus value as JSON)
curl -k -u root:0penBmc -X PUT -d '{"data": "xyz.openbmc_project.State.Host.Transition.On"}' \
    https://localhost/xyz/openbmc_project/state/host0/attr/RequestedHostTransition
```

D-Bus monitoring over WebSockets. bmcweb also exposes a `/subscribe/` WebSocket (`features/openbm`
`c_rest/dbus_monitor.hpp` , `DBusWebsocketSession` / `SessionMap`) that streams D-Bus `PropertiesChanged`
and `InterfacesAdded` signals to the browser as JSON – this is what drives live value updates in the web UI.
The client opens the socket and sends a filter such as `{"paths": ["/xyz/openbmc_project/sensors"], "inter`
`faces": ["xyz.openbmc_project.Sensor.Value"]}` ; matching signals then arrive as JSON messages. The
bmcweb repo ships a runnable example, `scripts/websocket_test.py` , that streams sensor readings from a
live BMC.

Troubleshooting

Common Issues

Service not found:

```
# Check if service is running
systemctl status xyz.openbmc_project.ServiceName

# Check service logs
journalctl -u xyz.openbmc_project.ServiceName
```

Permission denied:

```
# Check D-Bus policy
cat /usr/share/dbus-1/system.d/xyz.openbmc_project.*.conf
```

Property type mismatch:

```
# Verify property type with introspect
busctl introspect <service> <object> | grep <property>
```

Debug Commands

```
# Verbose D-Bus monitoring
dbus-monitor --system "interface='xyz.openbmc_project.State.Host'"

# Check Object Mapper health
busctl tree xyz.openbmc_project.ObjectMapper

# List all interfaces on an object
busctl introspect <service> <object> | grep interface
```

Next Steps

- [State Manager Guide](#) - Learn about system state control
 - [D-Bus Sensors Guide](#) - Understand sensor implementation
-

Examples

Working examples are available in the [examples/dbus](#) directory:

- [client/](#) - D-Bus client examples
 - [server/](#) - D-Bus server/service examples
 - [async/](#) - Asynchronous D-Bus programming
-

References

- [phosphor-dbus-interfaces](#)
 - [sdbusplus](#)
 - [phosphor-objmgr](#)
 - [D-Bus Specification](#)
-

Deep Dive

This section provides detailed technical information for developers who want to understand D-Bus internals in OpenBMC.

D-Bus Message Flow Architecture



Object Mapper Internals

The Object Mapper maintains a real-time view of all D-Bus objects:

Object Mapper Data Structures

Interface-to-Objects Map:

```
xyz.openbmc_project.Sensor.Value:  
- /xyz/openbmc_project/sensors/temperature/ambient  
- /xyz/openbmc_project/sensors/temperature/cpu0  
- /xyz/openbmc_project/sensors/fan_tach/fan0  
  
xyz.openbmc_project.State.Host:  
- /xyz/openbmc_project/state/host0  
  
xyz.openbmc_project.Inventory.Item:  
- /xyz/openbmc_project/inventory/system/chassis/motherboard  
- /xyz/openbmc_project/inventory/system/chassis/motherboard/cpu0
```

Object-to-Services Map:

```
/xyz/openbmc_project/sensors/temperature/cpu0:  
- xyz.openbmc_project.HwmonTempSensor  
  Interfaces: [xyz.openbmc_project.Sensor.Value,  
               xyz.openbmc_project.Sensor.Threshold.Warning,  
               xyz.openbmc_project.Sensor.Threshold.Critical,  
               xyz.openbmc_project.Association.Definitions]
```

Association Endpoints:

```
/xyz/openbmc_project/sensors/temperature/cpu0:  
  inventory (forward) -> /xyz/openbmc_project/inventory/.../cpu0  
  
/xyz/openbmc_project/inventory/.../cpu0:  
  sensors (reverse) -> /xyz/openbmc_project/sensors/temperature/cpu0
```

Update Mechanism:

1. Subscribe to org.freedesktop.DBus.ObjectManager signals
2. Subscribe to NameOwnerChanged for service tracking
3. On InterfacesAdded: update maps, create association endpoints
4. On InterfacesRemoved: clean up maps, remove endpoints
5. On NameOwnerChanged: handle service start/stop

sdbusplus Code Generation Pipeline



D-Bus Policy Configuration

D-Bus Security Policy

Policy File Location: /usr/share/dbus-1/system.d/

Policy Evaluation Order:

1. Check <deny> rules (most specific first)
2. Check <allow> rules (most specific first)
3. Default action (usually deny)

Specificity: user > group > context > default

Example Policy: xyz.openbmc_project.State.Host.conf

```
<busconfig>
  <!-- Allow root to own the service name -->
  <policy user="root">
    <allow own="xyz.openbmc_project.State.Host"/>
  </policy>

  <!-- Allow anyone to call methods and read properties -->
  <policy context="default">
    <allow send_destination="xyz.openbmc_project.State.Host"/>
    <allow receive_sender="xyz.openbmc_project.State.Host"/>
  </policy>

  <!-- Restrict property writes to specific users -->
  <policy group="admin">
    <allow send_destination="xyz.openbmc_project.State.Host"
          send_interface="org.freedesktop.DBus.Properties"
          send_member="Set"/>
  </policy>
</busconfig>
```

Common Policy Patterns:

Pattern	Use Case
own="service.name"	Allow process to claim well-known name
send_destination	Allow sending messages to service
send_interface	Restrict to specific D-Bus interface
send_member	Restrict to specific method/property
receive_sender	Allow receiving signals from service

D-Bus Type Serialization

Wire Format Example: Sensor Reading Message

Method Call: `Get("xyz.openbmc_project.Sensor.Value", "Value")`

Message Header (fixed format):

Endian	Type	Flags	Version	Body Length
'l' (little)	0x01 (method)	0x00	0x01	52

Header Fields (variable):

PATH:	/xyz/openbmc_project/sensors/temperature/cpu0
INTERFACE:	org.freedesktop.DBus.Properties
MEMBER:	Get
DESTINATION:	xyz.openbmc_project.HwmonTempSensor
SIGNATURE:	ss

Body (marshalled arguments):

String:	"xyz.openbmc_project.Sensor.Value" (32 bytes + NUL)
String:	"Value" (5 bytes + NUL)

Reply Message: Variant containing double

SIGNATURE: v (variant)

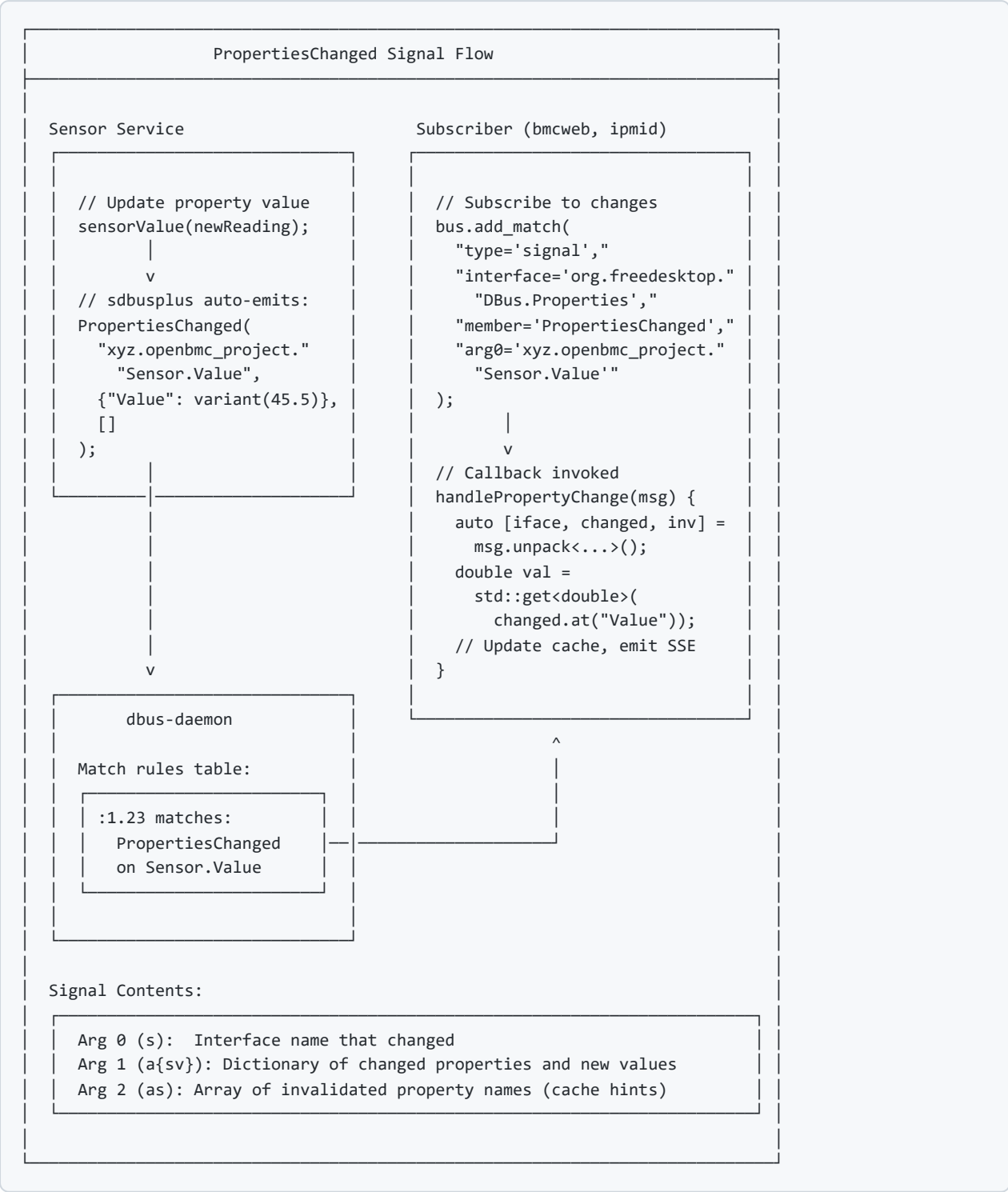
Body:

Variant signature:	"d" (1 byte + NUL)
Padding:	5 bytes (align double to 8)
Double value:	45.5 (8 bytes, IEEE 754)

Alignment Rules:

Type	Alignment	Size
BYTE	1	1
INT16	2	2
INT32	4	4
INT64	8	8
DOUBLE	8	8
STRING	4	4 (length) + data + NUL
ARRAY	4	4 (length) + elements (aligned)
STRUCT	8	sum of members (each aligned)
VARIANT	1	signature + value
DICTIONARY	8	array of struct{key, value}

PropertiesChanged Signal Mechanism



Source Code References

Component	Repository	Key Files
sdbusplus library	openbmc/sdbusplus	<code>include/sdbusplus/bus.hpp</code> , <code>include/sdbusplus/message.hpp</code>
Interface definitions	openbmc/phosphor-dbus-interfaces	<code>yaml/xyz/openbmc_project/**/*.interface.yaml</code>
Object Mapper	openbmc/phosphor-objmgr	<code>src/main.cpp</code> , <code>src/associations.cpp</code>
sdbus++ generator	openbmc/sdbusplus	<code>tools/sdbusplus/</code> , <code>tools/sdbus++</code>
D-Bus policy files	openbmc/openbmc	<code>meta-phosphor/recipes-phosphor/dbus/</code>
Async connection	openbmc/sdbusplus	<code>include/sdbusplus/asio/connection.hpp</code>

Tested on: OpenBMC master, QEMU romulus

State Manager Guide

Control and monitor BMC, chassis, and host system states.

Table of Contents

1. [Overview](#)
2. [Setup & Configuration](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [Meson Build Options](#)
 3. [Runtime Enable/Disable](#)
 4. [Power Restore Policy](#)
 5. [Boot Options Configuration](#)
 6. [Auto-Reboot Configuration](#)
 7. [Watchdog Configuration](#)
 8. [Via Redfish API](#)
3. [State Managers](#)
 1. [BMC State Manager](#)
 2. [Chassis State Manager](#)
 3. [Host State Manager](#)
 4. [Hypervisor State Manager](#)
 5. [Host Firmware Condition](#)
4. [obmcutil Command](#)
 1. [Basic Commands](#)
 2. [Status Commands](#)
 3. [Recovery Commands](#)
 4. [All obmcutil Commands](#)
5. [State Transitions](#)
 1. [Power On Sequence](#)
 2. [Power Off Sequence](#)
6. [Scheduled Host Transitions](#)
7. [Power Restore Policy](#)
8. [Boot Progress](#)
9. [Security Checks](#)
10. [Porting Guide](#)
 1. [GPIO Configuration](#)
 2. [systemd Targets](#)
 3. [Configuration Files](#)
11. [Troubleshooting](#)
 1. [Host won't power on](#)

2. [Stuck in Quiesced state](#)
 3. [Power restore not working](#)
 12. [Deep Dive](#)
 1. [State Machine Internals](#)
 2. [GPIO Power Control Sequence](#)
 3. [systemd Target Integration](#)
 4. [Boot Count and Auto-Recovery](#)
 5. [Power Restore Policy Implementation](#)
 6. [Watchdog Integration](#)
 7. [Source Code Reference](#)
 13. [Examples](#)
 14. [References](#)
-

Overview

The **phosphor-state-manager** controls the power state of the BMC, chassis, and host. It's the core component for:

- Powering on/off the server
- Rebooting the system
- Tracking system state
- Implementing power restore policies



State Manager Architecture

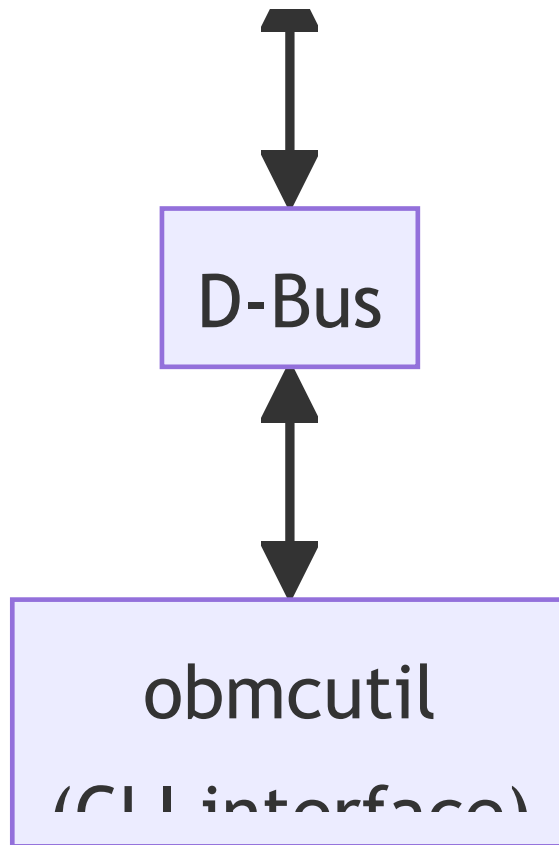
State Managers

BMC State Manager
States: NotReady, Ready,

Chassis State Manager
States: Off On

Host State Manager
States: Off, Running,





► ASCII-art version (for comparison)

Setup & Configuration

Build-Time Configuration (Yocto)

Include state manager in your image:

```
# In your machine .conf or image recipe
IMAGE_INSTALL:append = " phosphor-state-manager"

# Include specific components
IMAGE_INSTALL:append = " \
    phosphor-state-manager-bmc \
    phosphor-state-manager-chassis \
    phosphor-state-manager-host \
    phosphor-state-manager-discover-system-state \
"
```

Meson Build Options

```
meson setup build \
    -Dhost-gpios=enabled \
    -Dchassis-system-reset=enabled \
    -Dhost-state-options=enabled \
    -Dwarm-reboot=enabled
```

Option	Default	Description
<code>host-gpios</code>	enabled	GPIO-based host state detection
<code>chassis-system-reset</code>	disabled	System reset support
<code>warm-reboot</code>	disabled	Warm reboot capability
<code>boot-count-max-allowed</code>	3	Max boot retry attempts

Runtime Enable/Disable

```
# Check state manager services
systemctl status xyz.openbmc_project.State.BMC
systemctl status xyz.openbmc_project.State.Chassis
systemctl status xyz.openbmc_project.State.Host

# Restart state manager services
systemctl restart xyz.openbmc_project.State.BMC
systemctl restart xyz.openbmc_project.State.Chassis
systemctl restart xyz.openbmc_project.State.Host

# View logs
journalctl -u xyz.openbmc_project.State.Host -f
```

Power Restore Policy

Configure what happens after AC power loss:

```
# Get current policy
busctl get-property xyz.openbmc_project.Settings \
    /xyz/openbmc_project/control/host0/power_restore_policy \
    xyz.openbmc_project.Control.Power.RestorePolicy PowerRestorePolicy

# Set policy options:
# AlwaysOn - Always power on after AC restore
# AlwaysOff - Stay off after AC restore
# Restore - Return to previous state

busctl set-property xyz.openbmc_project.Settings \
    /xyz/openbmc_project/control/host0/power_restore_policy \
    xyz.openbmc_project.Control.Power.RestorePolicy PowerRestorePolicy s \
    "xyz.openbmc_project.Control.Power.RestorePolicy.Policy.Restore"
```


Boot Options Configuration

```
# Get boot source override
busctl get-property xyz.openbmc_project.Settings \
    /xyz/openbmc_project/control/host0/boot \
    xyz.openbmc_project.Control.Boot.Source BootSource

# Set boot source (Network, Disk, ExternalMedia, Default)
busctl set-property xyz.openbmc_project.Settings \
    /xyz/openbmc_project/control/host0/boot \
    xyz.openbmc_project.Control.Boot.Source BootSource s \
    "xyz.openbmc_project.Control.Boot.Source.Sources.Network"

# Set boot mode (Regular, Safe, Setup)
busctl set-property xyz.openbmc_project.Settings \
    /xyz/openbmc_project/control/host0/boot \
    xyz.openbmc_project.Control.Boot.Mode BootMode s \
    "xyz.openbmc_project.Control.Boot.Mode.Modes.Setup"

# Enable/disable boot override
busctl set-property xyz.openbmc_project.Settings \
    /xyz/openbmc_project/control/host0/boot/one_time \
    xyz.openbmc_project.Object.Enable Enabled b true
```

Auto-Reboot Configuration

```
# Get auto-reboot setting
busctl get-property xyz.openbmc_project.Settings \
    /xyz/openbmc_project/control/host0/auto_reboot \
    xyz.openbmc_project.Control.Boot.RebootPolicy AutoReboot

# Enable/disable auto-reboot
busctl set-property xyz.openbmc_project.Settings \
    /xyz/openbmc_project/control/host0/auto_reboot \
    xyz.openbmc_project.Control.Boot.RebootPolicy AutoReboot b true

# Set reboot attempts limit
busctl set-property xyz.openbmc_project.Settings \
    /xyz/openbmc_project/control/host0/auto_reboot \
    xyz.openbmc_project.Control.Boot.RebootAttempts AttemptsLeft u 3
```

Watchdog Configuration

```
# Get watchdog status
busctl introspect xyz.openbmc_project.Watchdog \
    /xyz/openbmc_project/watchdog/host0

# Enable watchdog
busctl set-property xyz.openbmc_project.Watchdog \
    /xyz/openbmc_project/watchdog/host0 \
    xyz.openbmc_project.State.Watchdog Enabled b true

# Set watchdog timeout (milliseconds)
busctl set-property xyz.openbmc_project.Watchdog \
    /xyz/openbmc_project/watchdog/host0 \
    xyz.openbmc_project.State.Watchdog Interval t 300000

# Set watchdog action (None, HardReset, PowerOff, PowerCycle)
busctl set-property xyz.openbmc_project.Watchdog \
    /xyz/openbmc_project/watchdog/host0 \
    xyz.openbmc_project.State.Watchdog ExpireAction s \
    "xyz.openbmc_project.State.Watchdog.Action.PowerCycle"
```

Via Redfish API

```
# Get system state
curl -k -u root:openBmc \
    https://localhost/redfish/v1/Systems/system | jq '.PowerState, .BootProgress'

# Set boot override
curl -k -u root:openBmc -X PATCH \
    -H "Content-Type: application/json" \
    -d '{
        "Boot": {
            "BootSourceOverrideTarget": "Pxe",
            "BootSourceOverrideEnabled": "Once"
        }
    }' \
    https://localhost/redfish/v1/Systems/system

# Set power restore policy
curl -k -u root:openBmc -X PATCH \
    -H "Content-Type: application/json" \
    -d '{"PowerRestorePolicy": "AlwaysOn"}' \
    https://localhost/redfish/v1/Systems/system
```

State Managers

BMC State Manager

Controls the BMC firmware state.

D-Bus Interface: `xyz.openbmc_project.State.BMC` **Object Path:** `/xyz/openbmc_project/state/bmc0`

State	Description
<code>NotReady</code>	BMC is booting, services not ready
<code>Ready</code>	BMC fully operational
<code>Quiesced</code>	BMC in error state, limited operation
<code>UpdateInProgress</code>	Firmware update in progress

```
# Check BMC state
busctl get-property xyz.openbmc_project.State.BMC \
    /xyz/openbmc_project/state/bmc0 \
    xyz.openbmc_project.State.BMC CurrentBMCState
```

Chassis State Manager

Controls chassis power (main power rails).

D-Bus Interface: `xyz.openbmc_project.State.Chassis` **Object Path:** `/xyz/openbmc_project/state/chassis0`

State	Description
<code>Off</code>	Chassis power off (standby only)
<code>On</code>	Chassis power on

Transition	Description
<code>Off</code>	Power off chassis
<code>On</code>	Power on chassis

```
# Check chassis state
busctl get-property xyz.openbmc_project.State.Chassis \
    /xyz/openbmc_project/state/chassis0 \
    xyz.openbmc_project.State.Chassis CurrentPowerState

# Power on chassis
busctl set-property xyz.openbmc_project.State.Chassis \
    /xyz/openbmc_project/state/chassis0 \
    xyz.openbmc_project.State.Chassis RequestedPowerTransition s \
    "xyz.openbmc_project.State.Chassis.Transition.On"
```

Power status and Power-On Hours. Beyond `On` / `Off`, the chassis manager publishes `CurrentPowerStatus` on `xyz.openbmc_project.State.Chassis`, one of `Good`, `BrownOut`, or `UninterruptiblePowerSupply` (`Undefined` before it is known). The `chassis-check-power-status` helper blocks a power-on request when the status is not `Good`. The manager also implements `xyz.openbmc_project.State.PowerOnHours` — a `POHCounter` property that increments while the chassis is on and is persisted across reboots (restored on start).

Host State Manager

Controls the host CPU/OS state.

D-Bus Interface: `xyz.openbmc_project.State.Host` **Object Path:** `/xyz/openbmc_project/state/host0`

State	Description
<code>Off</code>	Host not running
<code>Running</code>	Host running (booted or booting)
<code>Quiesced</code>	Host in error state
<code>DiagnosticMode</code>	Host in diagnostic mode

Transition	Description
<code>Off</code>	Graceful shutdown
<code>On</code>	Power on and boot
<code>Reboot</code>	Graceful reboot
<code>GracefulWarmReboot</code>	Warm reboot (no power cycle)
<code>ForceWarmReboot</code>	Forced warm reboot

```
# Check host state
busctl get-property xyz.openbmc_project.State.Host \
    /xyz/openbmc_project/state/host0 \
    xyz.openbmc_project.State.Host CurrentHostState

# Power on host
busctl set-property xyz.openbmc_project.State.Host \
    /xyz/openbmc_project/state/host0 \
    xyz.openbmc_project.State.Host RequestedHostTransition s \
    "xyz.openbmc_project.State.Host.Transition.On"

# Graceful shutdown
busctl set-property xyz.openbmc_project.State.Host \
    /xyz/openbmc_project/state/host0 \
    xyz.openbmc_project.State.Host RequestedHostTransition s \
    "xyz.openbmc_project.State.Host.Transition.Off"
```

Hypervisor State Manager

This manager is **IBM / PowerVM-specific** (`hypervisor_state_manager.cpp`) and is built only for IBM Power systems. On other platforms it is absent.

Tracks the state of the PowerVM hypervisor as a second host instance.

D-Bus Service: `xyz.openbmc_project.State.Hypervisor` **Object Path:** `/xyz/openbmc_project/state/hypervisor0` **Interface:** `xyz.openbmc_project.State.Host` (it reuses the Host interface)

Unlike `host0`, this instance is intentionally limited:

- `RequestedHostTransition` **only accepts** `On` — any other value is rejected (the service reports `BMCNotReady`).
- `CurrentHostState` is not driven by a power sequence; it is **derived from the host's** `BootProgress` :

Host <code>BootProgress</code>	Hypervisor <code>CurrentHostState</code>
<code>Unspecified</code>	<code>Off</code>
<code>SystemInitComplete</code>	<code>Standby</code>
<code>OSRunning</code>	<code>Running</code>

```
# Read the hypervisor's derived state
busctl get-property xyz.openbmc_project.State.Hypervisor \
    /xyz/openbmc_project/state/hypervisor0 \
    xyz.openbmc_project.State.Host CurrentHostState
```

Host Firmware Condition

An optional helper, **host-condition-gpio** (service `phosphor-host-condition-gpio@.service` , source `host_condition_gpio/`), reports whether host firmware is alive by reading a GPIO rather than by tracking a transition. It looks for a GPIO line named `<name>-ready` (active high) or `<name>-ready-n` (active low) and publishes the result on `xyz.openbmc_project.Condition.HostFirmware` :

<code>FirmwareCondition</code>	Meaning
<code>Running</code>	Ready GPIO asserted (host firmware is up)
<code>Off</code>	Ready GPIO deasserted
<code>Unknown</code>	No matching GPIO line, or a read error

This lets other services ask “is the host actually running?” independently of the state machine — for example to gate a power-on decision.

```
# Read the reported host firmware condition (path/service are platform-provided)
busctl get-property <service> <path> \
    xyz.openbmc_project.Condition.HostFirmware CurrentFirmwareCondition
```

obmcutil Command

`obmcutil` is the CLI for state management.

Basic Commands

```
# Show all system states
obmcutil state

# Example output:
# CurrentBMCState      : xyz.openbmc_project.State.BMC.BMCState.Ready
# CurrentPowerState    : xyz.openbmc_project.State.Chassis.PowerState.Off
# CurrentHostState     : xyz.openbmc_project.State.Host.HostState.Off

# Power on
obmcutil poweron

# Power off (graceful)
obmcutil poweroff

# Chassis power on (without host boot)
obmcutil chassison

# Chassis power off
obmcutil chassisoff

# Reboot host
obmcutil reboot

# BMC reboot
obmcutil bmcreboot
```

Status Commands

```
# Show boot progress
obmcutil bootprogress

# Show operating system status
obmcutil osstate

# Check if host is running
obmcutil hoststate
```

Recovery Commands

```
# Recover from quiesced state
obmcutil recover

# Force immediate host off (emergency)
obmcutil chassiskill
```

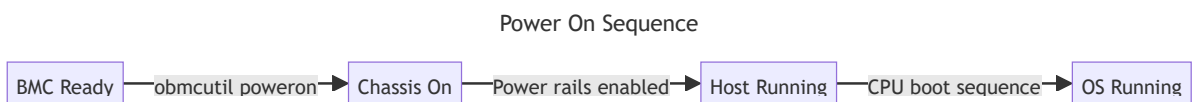
All obmcutil Commands

```
obmcutil --help

# Common commands:
# state          - Show current states
# poweron        - Power on host
# poweroff       - Power off host (graceful)
# chassison      - Chassis power on
# chassisoﬀ     - Chassis power off
# bmcreboot      - Reboot BMC
# reboot         - Reboot host
# recover        - Recover from errors
# status         - Same as 'state'
```

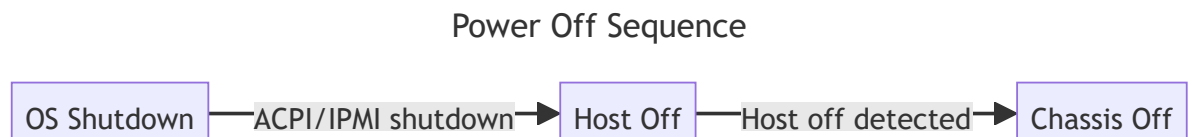
State Transitions

Power On Sequence



► ASCII-art version (for comparison)

Power Off Sequence



► ASCII-art version (for comparison)

Scheduled Host Transitions

The optional **scheduled-host-transition** service (source `scheduled_host_transition.cpp`, unit `xyz.openbmc_project.State.ScheduledHostTransition@.service`) lets you queue a power transition to run at a future wall-clock time.

D-Bus Interface: `xyz.openbmc_project.State.ScheduledHostTransition` **Object Path:** `/xyz/openbmc_project/State/host0`

Property	Type	Description
<code>ScheduledTime</code>	<code>uint64</code>	Epoch seconds at which to run the transition. <code>0</code> cancels any pending schedule.
<code>ScheduledTransition</code>	enum	Which <code>Host.Transition</code> to apply (e.g. <code>On</code> , <code>Off</code>)

```
# Choose the transition, then arm it for a specific epoch time
busctl set-property xyz.openbmc_project.State.ScheduledHostTransition \
    /xyz/openbmc_project/state/host0 \
    xyz.openbmc_project.State.ScheduledHostTransition ScheduledTransition s \
    "xyz.openbmc_project.State.Host.Transition.On"

busctl set-property xyz.openbmc_project.State.ScheduledHostTransition \
    /xyz/openbmc_project/state/host0 \
    xyz.openbmc_project.State.ScheduledHostTransition ScheduledTime t 1893456000

# Cancel a pending schedule
busctl set-property xyz.openbmc_project.State.ScheduledHostTransition \
    /xyz/openbmc_project/state/host0 \
    xyz.openbmc_project.State.ScheduledHostTransition ScheduledTime t 0
```

The schedule is persisted to a JSON file, so it survives a BMC reboot. The timer arms against `CLOCK_REALTIME` (with `TFD_TIMER_ABSTIME` | `TFD_TIMER_CANCEL_ON_SET`), so if the BMC's clock is stepped — for example by NTP — the pending transition is re-evaluated against the new time.

Power Restore Policy

Configure what happens after AC power loss.

D-Bus Interface: `xyz.openbmc_project.Control.Power.RestorePolicy` **Object Path:** `/xyz/openbmc_project/control/host0/power_restore_policy`

Policy	Description
<code>AlwaysOn</code>	Always power on after AC restore
<code>AlwaysOff</code>	Stay off after AC restore
<code>Restore</code>	Return to previous state

```
# Get current policy
busctl get-property xyz.openbmc_project.Settings \
    /xyz/openbmc_project/control/host0/power_restore_policy \
    xyz.openbmc_project.Control.Power.RestorePolicy PowerRestorePolicy

# Set policy to always on
busctl set-property xyz.openbmc_project.Settings \
    /xyz/openbmc_project/control/host0/power_restore_policy \
    xyz.openbmc_project.Control.Power.RestorePolicy PowerRestorePolicy s \
    "xyz.openbmc_project.Control.Power.RestorePolicy.Policy.AlwaysOn"
```


Boot Progress

Track host boot progress.

D-Bus Interface: `xyz.openbmc_project.State.Boot.Progress` **Object Path:** `/xyz/openbmc_project/state/host0`

Progress	Description
<code>Unspecified</code>	Unknown state
<code>PrimaryProcInit</code>	Primary processor initialization
<code>MemoryInit</code>	Memory initialization
<code>PCIInit</code>	PCI initialization
<code>OSStart</code>	Operating system starting
<code>OSRunning</code>	Operating system running

```
# Get boot progress
busctl get-property xyz.openbmc_project.State.Host \
    /xyz/openbmc_project/state/host0 \
    xyz.openbmc_project.State.Boot.Progress BootProgress
```

Security Checks

At BMC startup the **secure_boot_check** program (`secure_boot_check.cpp`) validates the platform’s security posture and raises event logs when something is wrong. It does **not** itself power down or quiesce the system — it records findings for the rest of the stack to act on.

What it inspects:

Check	Source
Secure boot enabled	<code>bmc-secure-boot</code> GPIO and a secure-boot sysfs attribute
ABR / golden-image active	an alternate-boot-region sysfs attribute
TPM measurement present	if a TPM device exists, its measurement sysfs value must be non-zero
Manufacturing mode	the <code>quiesce_on_hw_error</code> setting (LoggingSettings) gates whether violations are logged

Event logs it can create:

Error	Raised when
<code>xyz.openbmc_project.State.Error.TpmMeasurementFailure</code>	TPM measurement is missing, empty, or zero
<code>xyz.openbmc_project.State.Error.SecurityCheckFailure</code>	secure boot is disabled, or the ABR / golden image is active

Because these are ordinary phosphor-logging events, wire your platform’s policy (alerting, quiesce, or ignore in the lab) to the error identifiers above rather than expecting `secure_boot_check` to enforce anything on its own.

Porting Guide

GPIO Configuration

State manager uses GPIOs for power control. Configure in your machine layer:

```
// Example: power-config.json
{
  "gpio_configs": [
    {
      "name": "power-button",
      "direction": "out",
      "polarity": "active_low"
    },
    {
      "name": "power-good",
      "direction": "in",
      "polarity": "active_high"
    }
  ]
}
```

systemd Targets

Power transitions trigger systemd targets:

Target	Triggered When
<code>obmc-chassis-poweron.target</code>	Chassis power on
<code>obmc-chassis-poweroff.target</code>	Chassis power off
<code>obmc-host-start.target</code>	Host power on
<code>obmc-host-stop.target</code>	Host power off
<code>obmc-host-reboot.target</code>	Host reboot

Add custom services to these targets:

```
# /lib/systemd/system/my-power-hook.service
[Unit]
Description=Custom power on hook
After=obmc-chassis-poweron@0.target

[Service]
Type=oneshot
ExecStart=/usr/bin/my-power-on-script

[Install]
WantedBy=obmc-host-start@0.target
```

Configuration Files

```
# Power button configuration
/usr/share/phosphor-state-manager/power-config.json

# Chassis configuration
/etc/default/obmc/phosphor-state-manager/chassis
```

Troubleshooting

Host won't power on

```
# Check chassis state first
obmcutil state

# Check for errors in state manager
journalctl -u xyz.openbmc_project.State.Host -f

# Verify power good signal
gpioget gpiochip0 <power-good-pin>
```

Stuck in Quiesced state

```
# Check what caused quiesce
journalctl -u xyz.openbmc_project.State.Host | grep -i error

# Recover
obmcutil recover
```

Power restore not working

```
# Verify policy is set
busctl get-property xyz.openbmc_project.Settings \
    /xyz/openbmc_project/control/host0/power_restore_policy \
    xyz.openbmc_project.Control.Power.RestorePolicy PowerRestorePolicy

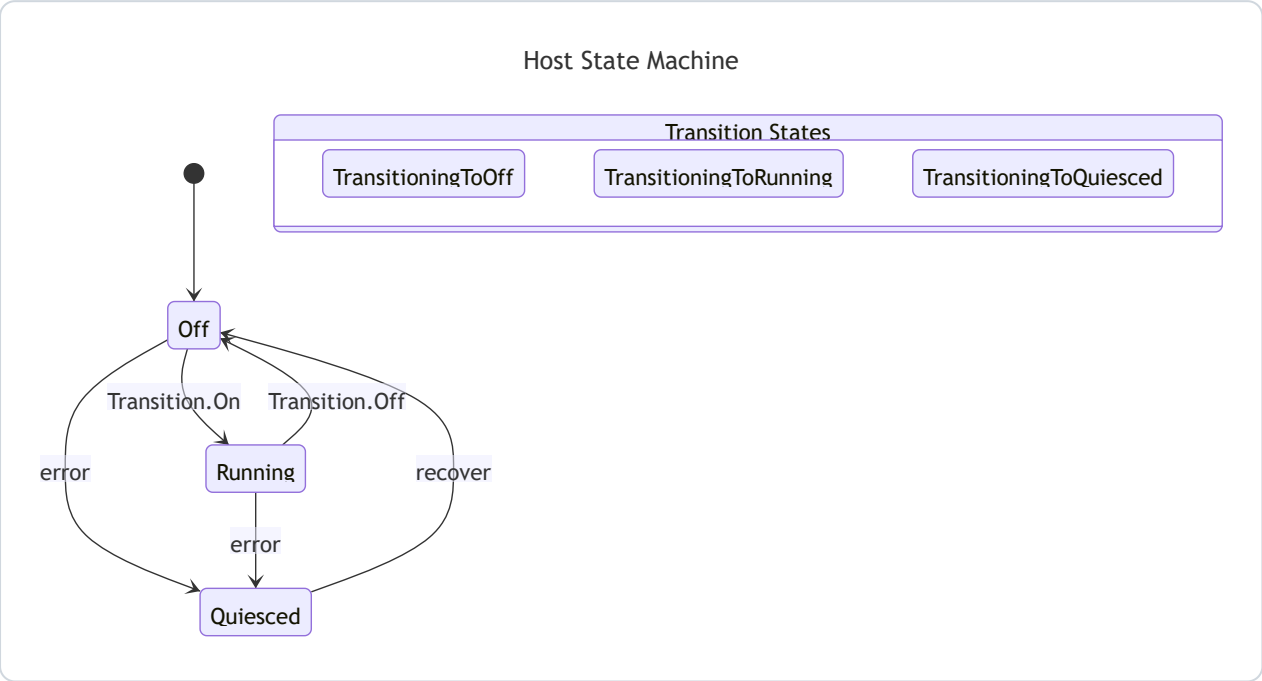
# Check restore service
systemctl status phosphor-reset-host-recovery@0.service
```

Deep Dive

Advanced implementation details for state manager developers.

State Machine Internals

The state managers implement finite state machines with well-defined transitions:



Internal Transition States:

- `TransitioningToOff` - shutdown in progress
- `TransitioningToRunning` - boot in progress
- `TransitioningToQuiesced` - error handling

► ASCII-art version (click to expand)

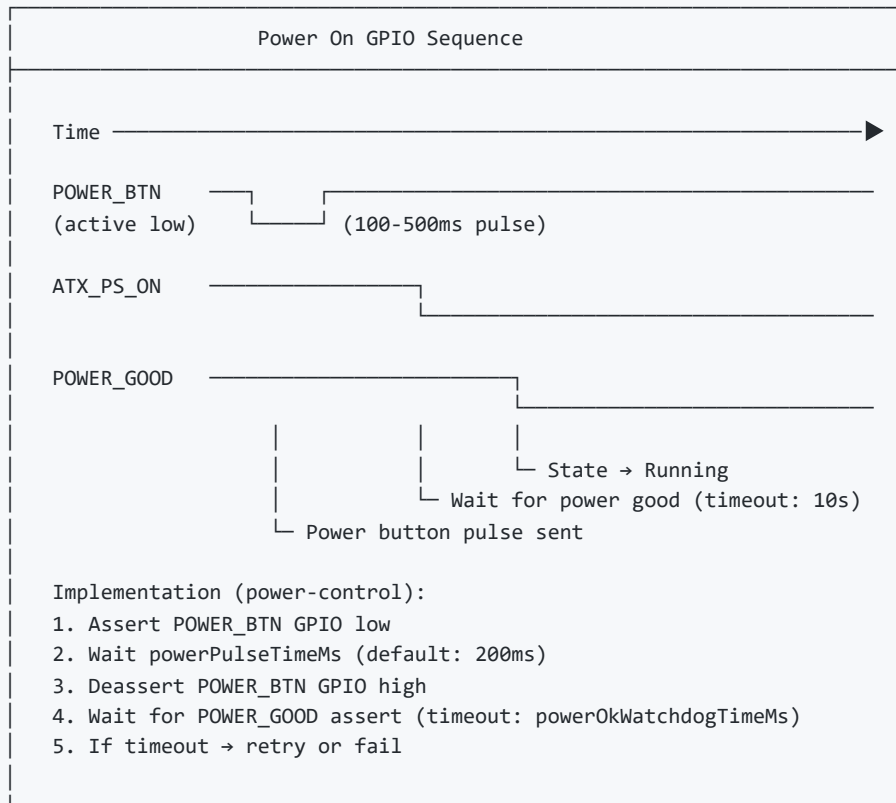
State transition triggers:

Event	Source	Transition
D-Bus RequestedHostTransition	User/Redfish	Off→Running, Running→Off
Power Good GPIO change	Hardware	Off→Running (assert), Running→Off (deassert)
Watchdog timeout	Watchdog service	Running→Quiesced
Boot failure (max retries)	Boot counter	Running→Quiesced

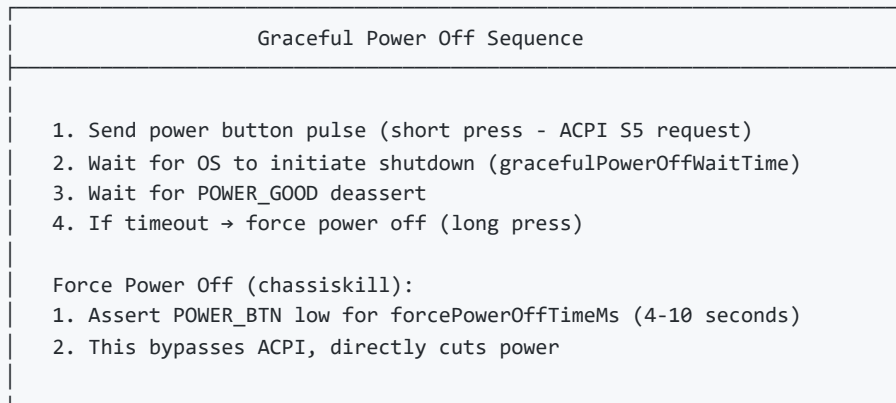
Source reference: [host_state_manager.cpp](#)

GPIO Power Control Sequence

Power on/off involves precise GPIO sequencing:



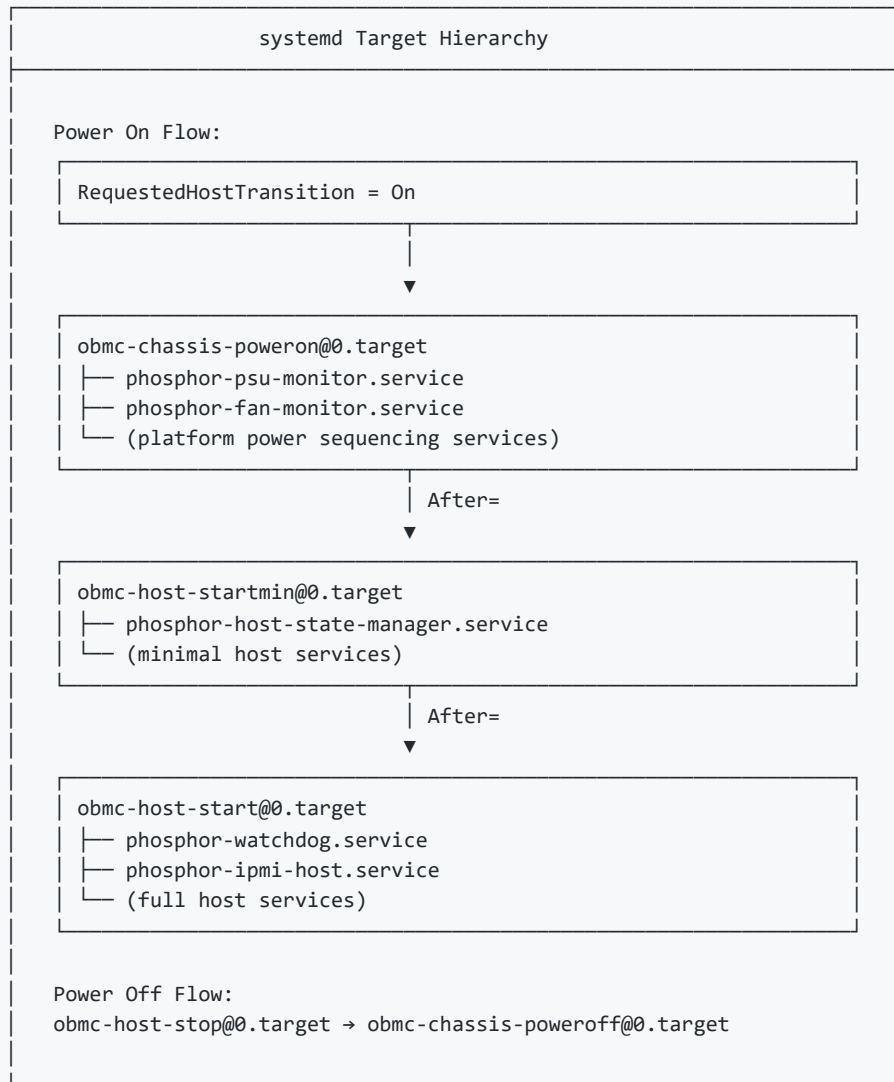
Power off sequence (graceful):



Source reference: [power-control](#) or platform-specific GPIO control

systemd Target Integration

State transitions trigger systemd targets for service orchestration:



Adding platform-specific services:

```

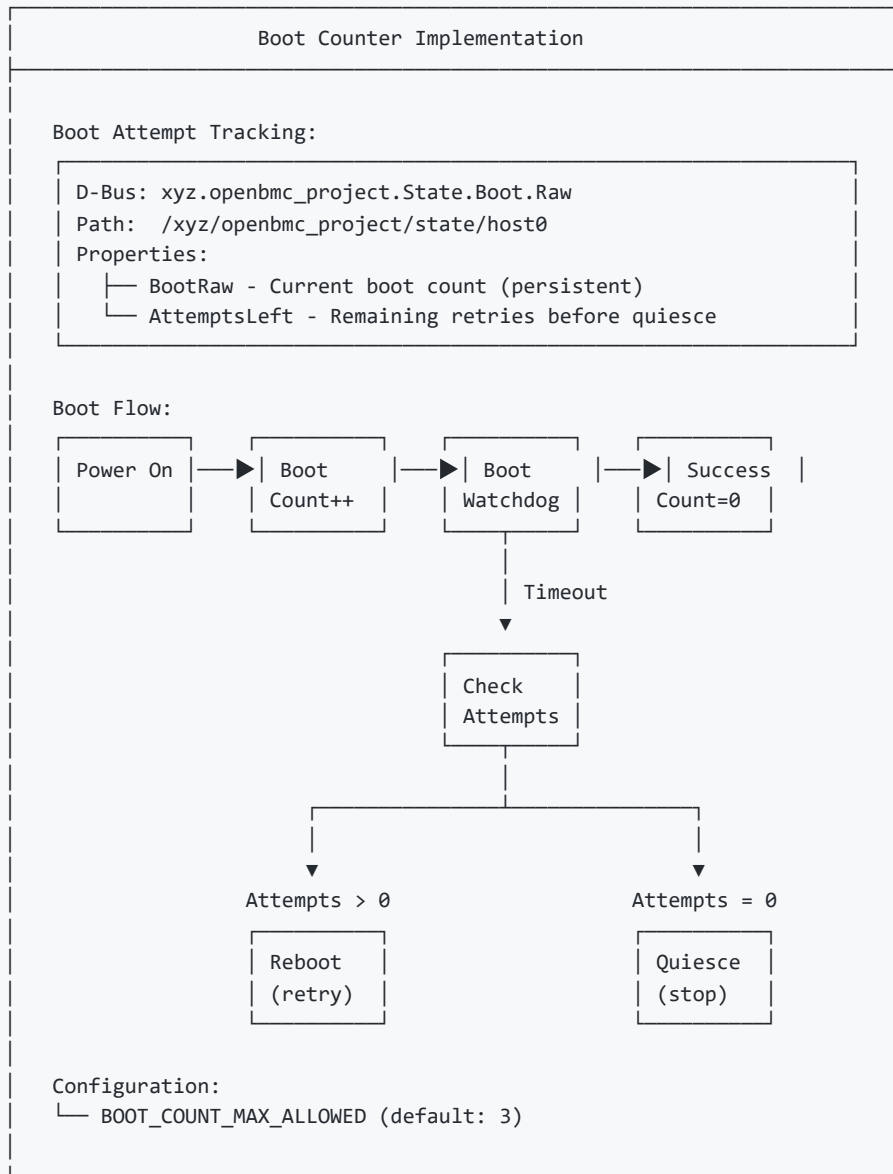
# /lib/systemd/system/platform-power-on.service
[Unit]
Description=Platform-specific power on sequence
After=obmc-chassis-poweron@0.target
Before=obmc-host-startmin@0.target

[Service]
Type=oneshot
ExecStart=/usr/bin/platform-power-sequence --on
RemainAfterExit=yes

[Install]
WantedBy=obmc-host-start@0.target
  
```

Boot Count and Auto-Recovery

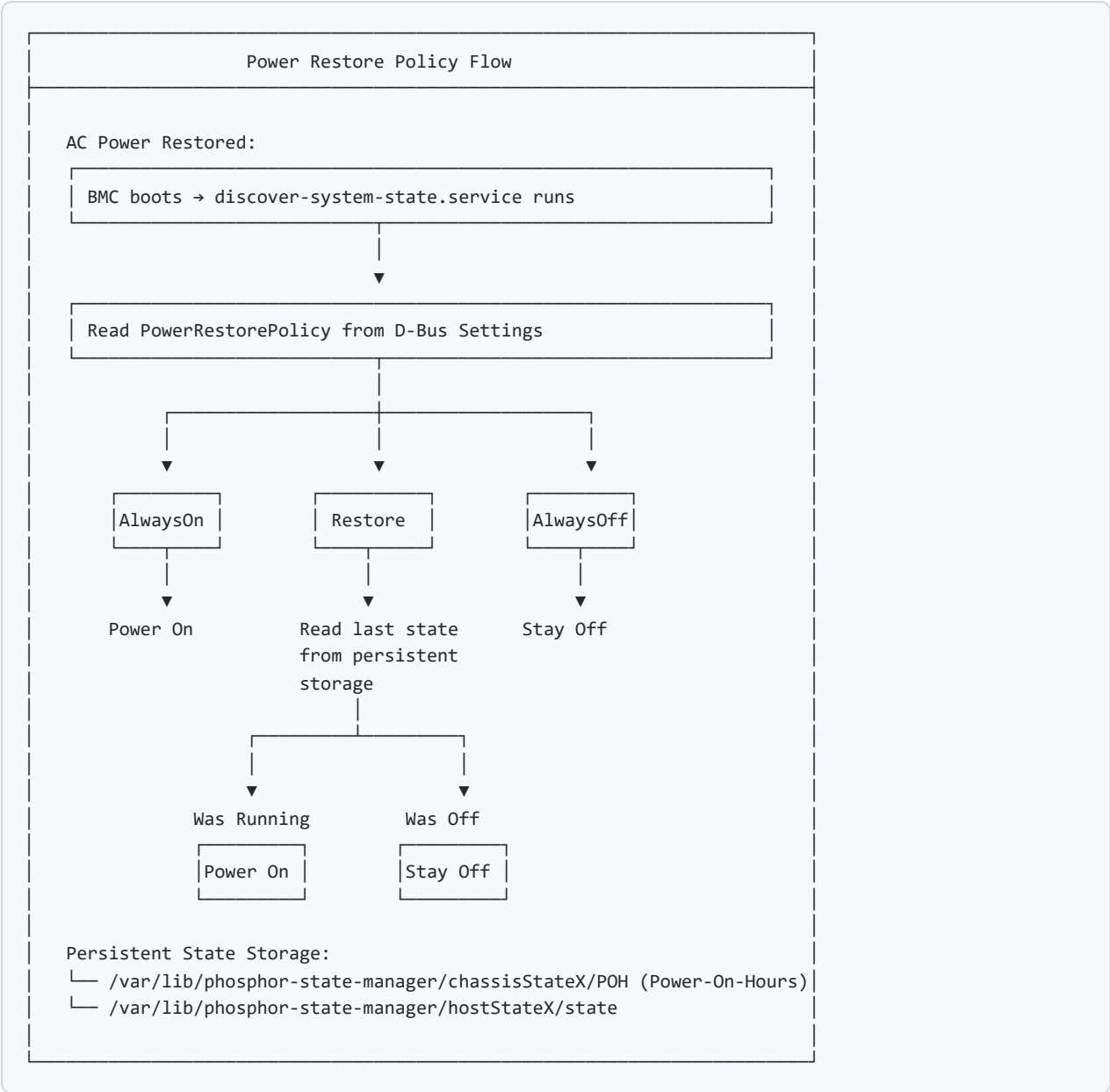
The boot counter tracks failed boot attempts and triggers recovery:



Boot count reset conditions:

- OS signals successful boot via IPMI
- Host reaches `OSRunning` boot progress
- Manual reset via `obmcutil recover`

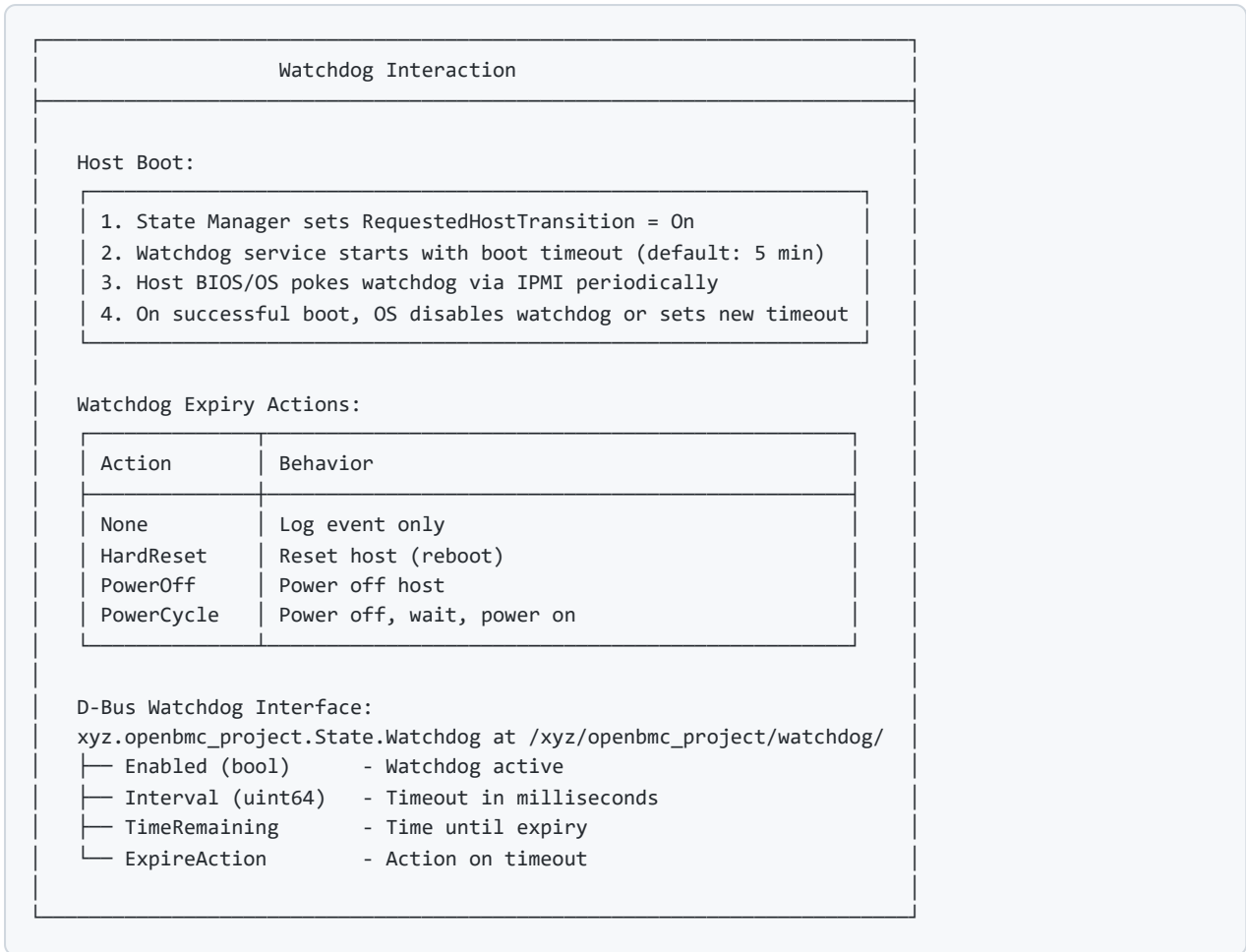
Power Restore Policy Implementation



Source reference: [discover_system_state.cpp](#)

Watchdog Integration

The host watchdog monitors for hangs during boot and runtime:



Source Code Reference

Key implementation files in [phosphor-state-manager](#):

File	Description
bmc_state_manager.cpp	BMC state tracking
chassis_state_manager.cpp	Chassis power control
host_state_manager.cpp	Host state machine
discover_system_state.cpp	Power restore policy
settings.cpp	Persistent settings storage
scheduled_host_transition.cpp	Scheduled power on/off

Examples

Working examples are available in the [examples/power](#) directory:

- [power-sequencer.sh](#) - Power sequencer control script
- [power-config.json](#) - Power supply configuration
- [psu-entity-manager.json](#) - PSU entity manager configuration

- `regulators-config.json` - Voltage regulator configuration
-

References

- [phosphor-state-manager](#)
 - [State Management Design](#)
 - [Power Control Interfaces](#)
-

Tested on: OpenBMC master, QEMU romulus

Custom D-Bus Services Guide

Build production-ready D-Bus services for OpenBMC using sdbusplus YAML interfaces, sdbus++ code generation, and proper Yocto integration.

Table of Contents

1. [Overview](#)
2. [Architecture](#)
 1. [Component Diagram](#)
 2. [D-Bus Interfaces](#)
 3. [Key Dependencies](#)
3. [YAML Interface Definition](#)
 1. [File Organization](#)
 2. [Complete YAML Example](#)
 3. [YAML Type Mappings](#)
 4. [Property Flags](#)
4. [sdbus++ Code Generation](#)
 1. [What sdbus++ Generates](#)
 2. [Running sdbus++ Manually](#)
 3. [Meson Integration for Code Generation](#)
5. [Service Implementation](#)
 1. [Approach 1: Using Generated Server Bindings \(Recommended\)](#)
 2. [Approach 2: Using sdbusplus::asio \(Async Event Loop\)](#)
 3. [Property Getters and Setters](#)
 4. [Method Handlers](#)
6. [Build Configuration \(meson.build\)](#)
 1. [Complete meson.build](#)
7. [systemd Service File](#)
 1. [systemd Unit File](#)
 2. [D-Bus Activation File](#)
 3. [OpenBMC Target Ordering](#)
8. [Yocto BitBake Recipe](#)
 1. [Complete .bb Recipe](#)
 2. [Adding the Recipe to Your Image](#)
 3. [Using bbappend for Customization](#)
9. [Verification and Testing](#)
 1. [Build and Deploy](#)
 2. [Test in QEMU](#)
 3. [Quick Test with Docker](#)

10. Troubleshooting

1. [Issue: Service Fails to Start with "Name already taken"](#)
2. [Issue: sdbus++ Fails with "Interface not found"](#)
3. [Issue: Properties Not Visible After Service Starts](#)
4. [Issue: "Permission denied" When Connecting to System Bus](#)
5. [Issue: BitBake Recipe Build Fails](#)
6. [Debug Commands](#)

11. References

1. [Official Resources](#)
 2. [Related Guides](#)
 3. [External Documentation](#)
-

Overview

Every feature in OpenBMC exposes its data and actions through D-Bus interfaces. When you add new hardware support, implement a custom management feature, or integrate a vendor-specific capability, you create a D-Bus service that other components (Redfish, IPMI, web UI) consume automatically.

This guide walks you through the complete lifecycle of building a custom D-Bus service: defining the interface in YAML, generating C++ bindings with sdbus++, implementing the service logic, and packaging it for the OpenBMC build system. You follow along with a concrete example – a "Hello World" service that exposes a greeting property and a SayHello method.

By the end of this guide, you have a fully functional D-Bus service running in QEMU that you can introspect with busctl and extend for your own use case.

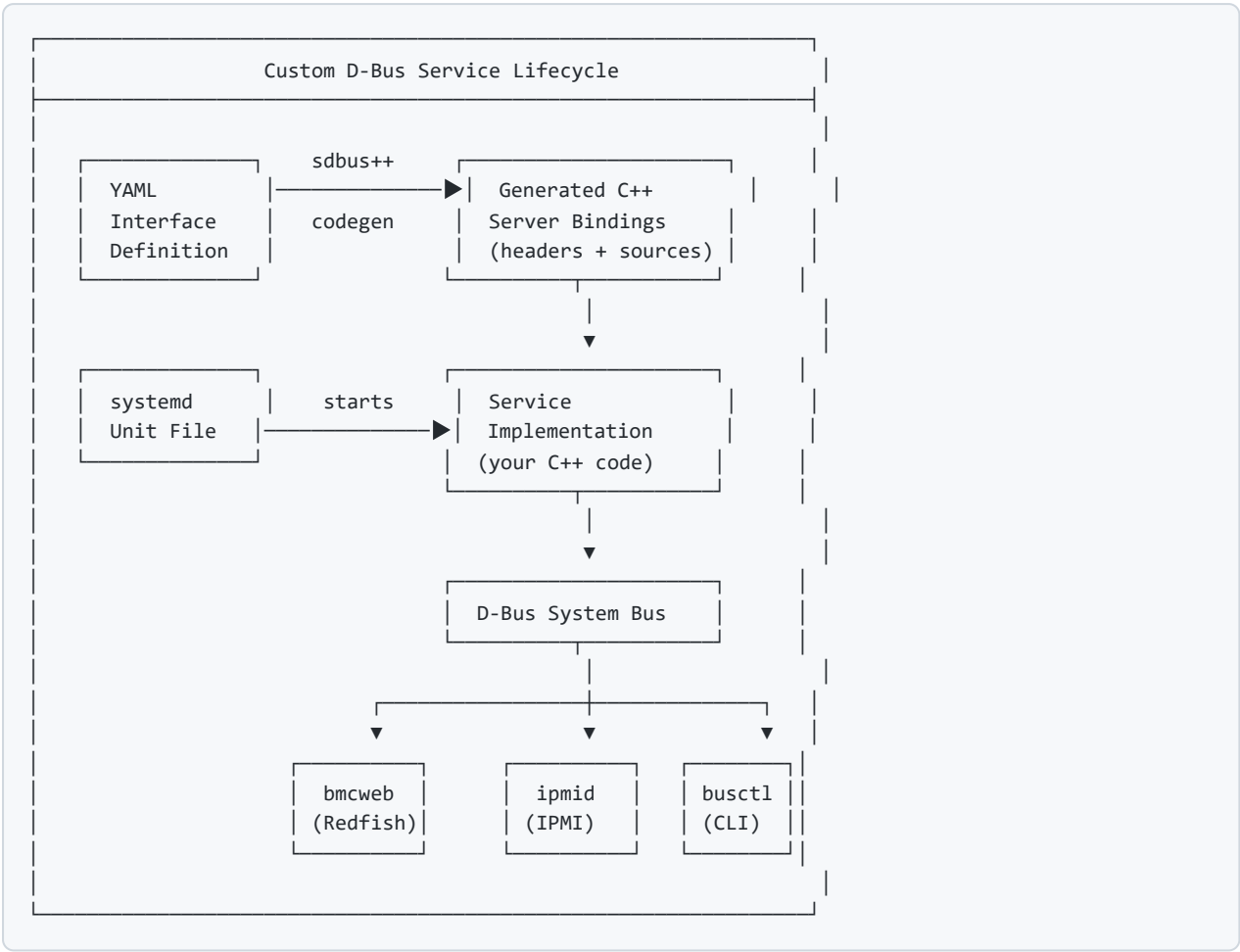
Key concepts covered:

- YAML interface definitions for sdbusplus (properties, methods, signals, enumerations)
 - sdbus++ code generation and meson integration
 - Async service implementation with sdbusplus::asio
 - systemd unit files with D-Bus activation
 - Yocto .bb recipes using obmc-phosphor-dbus-service.bbclass
-

Architecture

A custom D-Bus service in OpenBMC consists of several interconnected pieces. The YAML interface definition serves as the contract, sdbus++ generates the C++ bindings, your implementation provides the business logic, and systemd manages the service lifecycle.

Component Diagram



D-Bus Interfaces

Interface	Object Path	Description
<code>xyz.openbmc_project.Example.Greeter</code>	<code>/xyz/openbmc_project/example/greeter</code>	Example greeter service

Key Dependencies

- **sdbusplus**: C++ D-Bus library and sdbus++ code generator
- **phosphor-dbus-interfaces**: Standard OpenBMC interface definitions (reference patterns)
- **boost**: Provides Boost.Asio for the async event loop
- **systemd**: Service management and D-Bus activation
- **meson**: Build system with sdbus++ integration

YAML Interface Definition

The sdbusplus YAML format defines D-Bus interfaces declaratively. Each YAML file maps to one D-Bus interface and describes its properties, methods, signals, and enumerations.

File Organization

Interface YAML files follow a directory structure that mirrors the D-Bus interface name. For an interface named `xyz.openbmc_project.Example.Greeter`, you place the file at:

```
yaml/xyz/openbmc_project/Example/Greeter.interface.yaml
```

Enumeration types go in separate files:

```
yaml/xyz/openbmc_project/Example/Greeter/Status.interface.yaml
```

The `phosphor-dbus-interfaces` repository at github.com/openbmc/phosphor-dbus-interfaces contains all standard OpenBMC interface definitions. Study its structure when designing your own interfaces.

Complete YAML Example

Create the interface definition file for the Greeter service:

```

# yaml/xyz/openbmc_project/Example/Greeter.interface.yaml
description: >
    Interface for a greeting service that demonstrates custom D-Bus
    service development in OpenBMC.

properties:
  - name: Greeting
    type: string
    default: "Hello, OpenBMC!"
    description: >
        The current greeting message. Clients can read and modify
        this property.

  - name: GreetCount
    type: uint32
    default: 0
    flags:
      - readonly
    description: >
        Number of times the SayHello method has been called.
        This property is read-only and increments automatically.

  - name: Status
    type: enum[self.Status]
    default: "Ready"
    flags:
      - readonly
    description: >
        Current status of the greeter service.

methods:
  - name: SayHello
    description: >
        Generate a personalized greeting using the current Greeting
        template and the provided name.
    parameters:
      - name: Name
        type: string
        description: The name to greet.
    returns:
      - name: Response
        type: string
        description: The formatted greeting message.
    errors:
      - xyz.openbmc_project.Common.Error.InvalidArgument

  - name: Reset
    description: >
        Reset the GreetCount to zero and restore the default greeting.

signals:
  - name: GreetingChanged
    description: >
        Emitted when the greeting message changes.
    properties:
      - name: OldGreeting
        type: string
        description: The previous greeting message.
      - name: NewGreeting
        type: string
        description: The new greeting message.

enumerations:

```

```

- name: Status
  description: >
    Possible states of the greeter service.
  values:
    - name: Ready
      description: Service is ready to accept requests.
    - name: Busy
      description: Service is processing a request.
    - name: Error
      description: Service encountered an error.

```

YAML Type Mappings

The YAML `type` field maps to D-Bus type signatures and C++ types:

YAML Type	D-Bus Signature	C++ Type
<code>boolean</code>	<code>b</code>	<code>bool</code>
<code>byte</code>	<code>y</code>	<code>uint8_t</code>
<code>int16</code>	<code>n</code>	<code>int16_t</code>
<code>uint16</code>	<code>q</code>	<code>uint16_t</code>
<code>int32</code>	<code>i</code>	<code>int32_t</code>
<code>uint32</code>	<code>u</code>	<code>uint32_t</code>
<code>int64</code>	<code>x</code>	<code>int64_t</code>
<code>uint64</code>	<code>t</code>	<code>uint64_t</code>
<code>double</code>	<code>d</code>	<code>double</code>
<code>string</code>	<code>s</code>	<code>std::string</code>
<code>object_path</code>	<code>o</code>	<code>sdbusplus::message::object_path</code>
<code>array[T]</code>	<code>aT</code>	<code>std::vector<T></code>
<code>dict[K, V]</code>	<code>a{KV}</code>	<code>std::map<K, V></code>
<code>enum[self.Name]</code>	<code>s</code>	Generated enum class

Property Flags

Use the `flags` field to control property behavior:

Flag	Effect
<code>readonly</code>	Property can only be set by the service implementation, not by D-Bus clients
<code>const</code>	Property value is set at construction and never changes

Design your interfaces before writing code. A well-designed YAML interface acts as an API contract that other teams can review and depend on.

sdbus++ Code Generation

The `sdbus++` tool reads your YAML interface definitions and generates C++ server bindings. These generated files include abstract base classes with virtual methods for your service implementation to override.

What sdbus++ Generates

For the `xyz.openbmc_project.Example.Greeter` interface, sdbus++ produces:

Generated File	Purpose
<code>xyz/openbmc_project/Example/Greeter/server.hpp</code>	Server-side abstract base class
<code>xyz/openbmc_project/Example/Greeter/server.cpp</code>	Server-side vtable registration
<code>xyz/openbmc_project/Example/Greeter/client.hpp</code>	Client-side proxy class
<code>xyz/openbmc_project/Example/Greeter/common.hpp</code>	Shared type definitions
<code>xyz/openbmc_project/Example/Greeter/error.hpp</code>	Error type definitions

Running sdbus++ Manually

You can run sdbus++ directly to inspect the generated output:

```
# Generate server header
sdbus++ -r yaml -t interface server-header \
    xyz.openbmc_project.Example.Greeter > server.hpp

# Generate server implementation
sdbus++ -r yaml -t interface server-cpp \
    xyz.openbmc_project.Example.Greeter > server.cpp

# Generate common types (enums, errors)
sdbus++ -r yaml -t interface common-header \
    xyz.openbmc_project.Example.Greeter > common.hpp
```

In practice, you rarely run sdbus++ manually. The meson build system handles code generation automatically. The commands above are useful for debugging or understanding the generated code.

Meson Integration for Code Generation

The sdbusplus project provides meson helper functions that automate code generation. Add this to your `meson.build`:

```

sdbusplus_dep = dependency('sdbusplus')
sdbusplus_prog = find_program('sdbus++')

# Define the interface to generate
generated_sources = []
generated_headers = []

# Use sdbusplus meson module for code generation
sdbusplus_project = subproject('sdbusplus', required: true)
sdbusplus_generate = sdbusplus_project.get_variable('sdbusplus_generate')

# Generate server bindings from YAML
generated = sdbusplus_generate.process(
    'yaml/xyz/openbmc_project/Example/Greeter.interface.yaml',
    install_header: false,
)

generated_sources += generated[0] # .cpp files
generated_headers += generated[1] # .hpp files

```

For projects that prefer a simpler approach without the meson subproject, you can use `custom_target`:

```

sdbusplus_prog = find_program('sdbus++')

greeter_server_hpp = custom_target(
    'greeter-server-hpp',
    input: [],
    output: 'server.hpp',
    command: [
        sdbusplus_prog, '-r', meson.project_source_root() / 'yaml',
        '-t', 'interface', 'server-header',
        'xyz.openbmc_project.Example.Greeter',
    ],
    capture: true,
)

greeter_server_cpp = custom_target(
    'greeter-server-cpp',
    input: [],
    output: 'server.cpp',
    command: [
        sdbusplus_prog, '-r', meson.project_source_root() / 'yaml',
        '-t', 'interface', 'server-cpp',
        'xyz.openbmc_project.Example.Greeter',
    ],
    capture: true,
)

```

Service Implementation

With the YAML interface defined and code generation configured, you now implement the service itself. OpenBMC services use the `sdbusplus::asio` integration with `Boost.Asio` for async event handling.

Approach 1: Using Generated Server Bindings (Recommended)

This approach inherits from the `sdbus++`-generated server base class. The generated class provides property storage, vtable registration, and signal emission. You override virtual methods to implement business logic.

```

// greeter_service.hpp
#pragma once

#include <xyz/openbmc_project/Example/Greeter/server.hpp>
#include <sdbusplus/bus.hpp>
#include <sdbusplus/server.hpp>

namespace greeter
{

// Create a convenient type alias for the generated server binding
using GreeterInherit = sdbusplus::server::object_t<
    sdbusplus::xyz::openbmc_project::Example::server::Greeter>;

class GreeterService : public GreeterInherit
{
public:
    GreeterService(sdbusplus::bus_t& bus, const char* path) :
        GreeterInherit(bus, path)
    {
        // Set initial property values
        greeting("Hello, OpenBMC!");
        greetCount(0);
        status(Status::Ready);
    }

    // Implement the SayHello method (generated as pure virtual)
    std::string sayHello(std::string name) override
    {
        if (name.empty())
        {
            throw sdbusplus::xyz::openbmc_project::Common::Error::
                InvalidArgument();
        }

        // Update the greet count
        greetCount(greetCount() + 1);

        // Build the response using the current greeting
        std::string response = greeting() + " " + name + "!";

        return response;
    }

    // Implement the Reset method
    void reset() override
    {
        std::string oldGreeting = greeting();

        // Restore defaults
        greeting("Hello, OpenBMC!");
        greetCount(0);

        // Emit the GreetingChanged signal
        greeterChanged(oldGreeting, greeting());
    }
};

} // namespace greeter

```

The main function creates the bus connection and instantiates the service:

```

// main.cpp
#include "greeter_service.hpp"

#include <sdbusplus/bus.hpp>
#include <sdbusplus/server/manager.hpp>

#include <iostream>

int main()
{
    constexpr auto serviceName = "xyz.openbmc_project.Example.Greeter";
    constexpr auto objectPath = "/xyz/openbmc_project/example/greeter";

    // Connect to the system bus
    auto bus = sdbusplus::bus::new_default();

    // Create an ObjectManager for introspection support
    sdbusplus::server::manager_t objManager(bus, objectPath);

    // Request the well-known service name
    bus.request_name(serviceName);

    // Create the greeter service object
    greeter::GreeterService greeterObj(bus, objectPath);

    std::cout << "Greeter service running at " << objectPath << "\n";

    // Enter the event loop
    while (true)
    {
        bus.process_discard();
        bus.wait();
    }

    return 0;
}

```

Approach 2: Using sdbusplus::asio (Async Event Loop)

Many OpenBMC services use the Boost.Asio integration for timers, file descriptor monitoring, and concurrent I/O. This approach registers properties and methods programmatically instead of inheriting from generated bindings.

```

// main_async.cpp
#include <sdbusplus/asio/connection.hpp>
#include <sdbusplus/asio/object_server.hpp>
#include <boost/asio.hpp>

#include <iostream>
#include <string>

int main()
{
    constexpr auto serviceName = "xyz.openbmc_project.Example.Greeter";
    constexpr auto objectPath = "/xyz/openbmc_project/example/greeter";
    constexpr auto interfaceName = "xyz.openbmc_project.Example.Greeter";

    // Create the Boost.Asio IO context
    boost::asio::io_context io;

    // Connect to the system D-Bus
    auto conn = std::make_shared<sdbusplus::asio::connection>(io);
    conn->request_name(serviceName);

    // Create the object server
    sdbusplus::asio::object_server server(conn);

    // Add the interface to the object path
    auto iface = server.add_interface(objectPath, interfaceName);

    // — Properties —————

    std::string greeting = "Hello, OpenBMC!";
    uint32_t greetCount = 0;
    std::string status = "xyz.openbmc_project.Example.Greeter.Status.Ready";

    // Read-write property: Greeting
    iface->register_property(
        "Greeting", greeting,
        // Setter with validation
        [&greeting](const std::string& newValue, std::string& value) {
            if (newValue.empty())
            {
                std::cerr << "Rejected empty greeting\n";
                return false;
            }
            std::cout << "Greeting changed: " << value
                << " -> " << newValue << "\n";
            value = newValue;
            greeting = newValue;
            return true;
        },
        // Getter
        [&greeting](const std::string& /*value*/) {
            return greeting;
        }
    );

    // Read-only property: GreetCount
    iface->register_property_r(
        "GreetCount", greetCount,
        sdbusplus::vtable::property_::emits_change,
        [&greetCount](const uint32_t&) {
            return greetCount;
        }
    );
}

```

```

// Read-only property: Status
iface->register_property_r(
    "Status", status,
    sdbusplus::vtable::property_::emits_change,
    [&status](const std::string&) {
        return status;
    }
);

// — Methods —————

// Method: SayHello(string Name) -> string Response
iface->register_method(
    "SayHello",
    [&greeting, &greetCount, &iface](std::string name) {
        if (name.empty())
        {
            throw std::invalid_argument("Name must not be empty");
        }

        greetCount++;
        iface->signal_property("GreetCount");

        std::string response = greeting + " " + name + "!";
        std::cout << "SayHello(\"" << name << "\") -> \""
            << response << "\" (count: "
            << greetCount << ")\n";
        return response;
    }
);

// Method: Reset()
iface->register_method(
    "Reset",
    [&greeting, &greetCount, &iface]() {
        greeting = "Hello, OpenBMC!";
        greetCount = 0;
        iface->signal_property("Greeting");
        iface->signal_property("GreetCount");
        std::cout << "Service reset to defaults\n";
    }
);

// Initialize the interface (make it visible on D-Bus)
iface->initialize();

std::cout << "Greeter service running at " << objectPath << "\n";

// — Optional: periodic timer example —————

boost::asio::steady_timer timer(io);
std::function<void(const boost::system::error_code&)> timerHandler;
timerHandler = [&timer, &greetCount, &iface,
    &timerHandler](const boost::system::error_code& ec) {
    if (ec)
    {
        return;
    }
    std::cout << "Heartbeat: greetCount=" << greetCount << "\n";
    timer.expires_after(std::chrono::seconds(60));
    timer.async_wait(timerHandler);
};
timer.expires_after(std::chrono::seconds(60));

```

```

    timer.async_wait(timerHandler);

    // Run the event loop (blocks until io.stop() is called)
    io.run();

    return 0;
}

```

Use Approach 1 (generated bindings) when you want strict interface enforcement – the compiler verifies that you implement every method. Use Approach 2 (sdbusplus::asio) when you need Boost.Asio features like timers, socket monitoring, or concurrent I/O operations.

Property Getters and Setters

Properties in sdbusplus support validation and change notification:

```

// Read-write property with validation
iface->register_property(
    "PropertyName", initialValue,
    // Setter: return true to accept, false to reject
    [](const std::string& newValue, std::string& storedValue) {
        if (newValue.length() > 256)
        {
            return false; // Reject values that are too long
        }
        storedValue = newValue;
        return true;
    }
);

// Read-only property (updates via signal_property)
iface->register_property_r(
    "ReadOnlyProp", value,
    sdbusplus::vtable::property_::emits_change,
    [&value](const auto&) { return value; }
);

```

Method Handlers

Methods can accept parameters, return values, and throw errors:

```

// Method with parameters and return value
iface->register_method("MethodName",
    [](std::string param1, int32_t param2) -> std::string {
        // Process the request
        return "result";
    }
);

// Method with no parameters or return value
iface->register_method("SimpleMethod", []() {
    // Perform action
});

// Method that throws a D-Bus error
iface->register_method("ValidatedMethod",
    [](std::string input) {
        if (input.empty())
        {
            throw sdbusplus::exception::SdbusError(
                -EINVAL, "xyz.openbmc_project.Common.Error.InvalidArgument");
        }
    }
);

```

Build Configuration (meson.build)

The meson.build file ties together code generation, compilation, and installation. Below is a complete build file for the greeter service.

Complete meson.build

```
project(
    'phosphor-greeter',
    'cpp',
    version: '1.0.0',
    meson_version: '>=0.63.0',
    default_options: [
        'cpp_std=c++23',
        'warning_level=3',
        'werror=true',
        'buildtype=debugoptimized',
    ],
)

# — Dependencies —————

sdbusplus_dep = dependency('sdbusplus')
boost_dep = dependency(
    'boost',
    modules: ['coroutine', 'context'],
    required: false,
)
systemd_dep = dependency('systemd')
phosphor_logging_dep = dependency('phosphor-logging', required: false)

# — sdbus++ code generation —————

sdbusplus_prog = find_program('sdbus++', required: true)
sdbusplus_gen_meson_prog = find_program('sdbus++-gen-meson', required: true)

# Generate server bindings from YAML interface definitions.
# The YAML files are under yaml/ in the source tree.
greeter_server_hpp = custom_target(
    'greeter-server-hpp',
    output: 'server.hpp',
    command: [
        sdbusplus_prog,
        '-r', meson.project_source_root() / 'yaml',
        '-t', 'interface',
        'server-header',
        'xyz.openbmc_project.Example.Greeter',
    ],
    capture: true,
)

greeter_server_cpp = custom_target(
    'greeter-server-cpp',
    output: 'server.cpp',
    command: [
        sdbusplus_prog,
        '-r', meson.project_source_root() / 'yaml',
        '-t', 'interface',
        'server-cpp',
        'xyz.openbmc_project.Example.Greeter',
    ],
    capture: true,
)

greeter_common_hpp = custom_target(
    'greeter-common-hpp',
    output: 'common.hpp',
    command: [
```

```

        sdbusplus_prog,
        '-r', meson.project_source_root() / 'yaml',
        '-t', 'interface',
        'common-header',
        'xyz.openbmc_project.Example.Greeter',
    ],
    capture: true,
)

generated_sources = [greeter_server_cpp, greeter_server_hpp,
                    greeter_common_hpp]

# — Service executable —————

executable(
    'phosphor-greeter',
    'src/main.cpp',
    'src/greeter_service.cpp',
    generated_sources,
    dependencies: [
        sdbusplus_dep,
        phosphor_logging_dep,
    ],
    include_directories: include_directories('src'),
    install: true,
    install_dir: get_option('bindir'),
)

# — systemd unit installation —————

systemd_system_unit_dir = systemd_dep.get_variable(
    'systemdsystemunitdir',
    pkgconfig_define: ['prefix', get_option('prefix')],
)

configure_file(
    input: 'service_files/xyz.openbmc_project.Example.Greeter.service',
    output: 'xyz.openbmc_project.Example.Greeter.service',
    copy: true,
    install_dir: systemd_system_unit_dir,
)

# — D-Bus service activation file —————

dbus_system_services_dir = dependency('dbus-1').get_variable(
    'system_bus_services_dir',
    pkgconfig_define: ['prefix', get_option('prefix')],
)

configure_file(
    input: 'service_files/xyz.openbmc_project.Example.Greeter.dbus.service',
    output: 'xyz.openbmc_project.Example.Greeter.service',
    copy: true,
    install_dir: dbus_system_services_dir,
)

```

Use `cpp_std=c++23` (or at least `c++20`) for OpenBMC services. The sdbusplus library and OpenBMC coding standards require modern C++ features like `std::expected`, structured bindings, and concepts.

systemd Service File

systemd manages the lifecycle of your D-Bus service. You need two files: a systemd unit file and a D-Bus activation file.

systemd Unit File

Create `service_files/xyz.openbmc_project.Example.Greeter.service` :

```
[Unit]
Description=OpenBMC Example Greeter D-Bus Service
After=dbus.service
After=phosphor-dbus-interfaces-mapper.service
Wants=phosphor-dbus-interfaces-mapper.service

# Start after the BMC is ready
After=mapper-wait@-xyz-openbmc_project-example.service
Wants=mapper-wait@-xyz-openbmc_project-example.service

# Integrate into OpenBMC boot ordering
After=obmc-standby.target
Before=obmc-host-start-pre@0.target

[Service]
Type=dbus
BusName=xyz.openbmc_project.Example.Greeter
ExecStart=/usr/bin/phosphor-greeter
SyslogIdentifier=phosphor-greeter
Restart=on-failure
RestartSec=5

# Security hardening
ProtectSystem=full
ProtectHome=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

D-Bus Activation File

Create `service_files/xyz.openbmc_project.Example.Greeter.dbus.service` :

```
[D-BUS Service]
Name=xyz.openbmc_project.Example.Greeter
Exec=/usr/bin/phosphor-greeter
User=root
SystemdService=xyz.openbmc_project.Example.Greeter.service
```

OpenBMC Target Ordering

OpenBMC defines a target hierarchy that controls the boot sequence. Place your service in the correct position based on when it needs to run:

Target	When It Runs	Use When
<code>obmc-standby.target</code>	BMC is ready, before host power-on	Service needed at BMC idle
<code>obmc-host-startmin@.target</code>	Minimum services for host start	Service required for host boot
<code>obmc-host-start-pre@.target</code>	Before host power-on GPIO	Pre-power-on initialization
<code>obmc-host-started@.target</code>	After host is running	Post-boot monitoring
<code>obmc-host-stop-pre@.target</code>	Before host power-off	Graceful shutdown tasks

The `Type=dbus` setting tells systemd to consider the service started once it has acquired its D-Bus bus name. This ensures dependent services do not start until the interface is ready.

Yocto BitBake Recipe

Package your service for the OpenBMC build system with a BitBake recipe. The `obmc-phosphor-dbus-service` bbclass simplifies service packaging.

Complete .bb Recipe

Create `meta-phosphor/recipes-phosphor/example/phosphor-greeter_1.0.bb` :

```

SUMMARY = "OpenBMC Example Greeter D-Bus Service"
DESCRIPTION = "Demonstrates how to build a custom D-Bus service for OpenBMC"
HOMEPAGE = "https://github.com/openbmc/openbmc"
LICENSE = "Apache-2.0"
LIC_FILES_CHKSUM = "file://LICENSE;md5=86d3f3a95c324c9479bd8986968f4327"
PR = "r1"

inherit meson
inherit pkgconfig
inherit obmc-phosphor-dbus-service

# Source repository (adjust for your project)
SRC_URI = "git://github.com/openbmc/phosphor-greeter;branch=master;protocol=https"
SRCREV = "${AUTOREV}"

S = "${WORKDIR}/git"

# Build dependencies (compile-time)
DEPENDS += " \
    sdbusplus \
    phosphor-logging \
    boost \
    systemd \
"

# Runtime dependencies
RDEPENDS:${PN} += " \
    sdbusplus \
    phosphor-logging \
    libsystemd \
"

# D-Bus service configuration for obmc-phosphor-dbus-service.bbclass
# This bbclass automatically installs the D-Bus activation file and
# links the systemd unit to the appropriate target.
DBUS_SERVICE:${PN} = "xyz.openbmc_project.Example.Greeter.service"

# systemd service configuration
SYSTEMD_SERVICE:${PN} = "xyz.openbmc_project.Example.Greeter.service"

```

Adding the Recipe to Your Image

Add the package to your machine image configuration:

```

# In your machine.conf or local.conf
IMAGE_INSTALL:append = " phosphor-greeter"

```

Or add it to a packagegroup:

```

# In meta-your-machine/recipes-phosphor/packagegroups/packagegroup-your-machine-apps.bb
RDEPENDS:${PN}:append = " phosphor-greeter"

```

Using bbappend for Customization

To customize the service for your platform without modifying the upstream recipe:

```
# meta-your-machine/recipes-phosphor/example/phosphor-greeter_%.bbappend

FILESEXTRAPATHS:prepend := "${THISDIR}/${PN}:"

# Override the greeting default
SRC_URI += "file://0001-change-default-greeting.patch"

# Add platform-specific configuration
SRC_URI += "file://greeter-config.json"

do_install:append() {
    install -d ${D}${datadir}/phosphor-greeter
    install -m 0644 ${WORKDIR}/greeter-config.json \
        ${D}${datadir}/phosphor-greeter/
}
```

The `obmc-phosphor-dbus-service.bbclass` handles D-Bus activation file installation and systemd service linking automatically. You only need to set `DBUS_SERVICE` and `SYSTEMD_SERVICE` variables.

Verification and Testing

Build and Deploy

Build the service within the OpenBMC build environment:

```
# Build only the greeter package
bitbake phosphor-greeter

# Build the full image with the greeter included
bitbake obmc-phosphor-image
```

Test in QEMU

Launch the QEMU emulator and verify the service:

```
# Start QEMU (ast2600-evb)
./scripts/run-qemu.sh ast2600-evb

# SSH into the BMC
ssh -p 2222 root@localhost
# Password: 0penBmc
```

Once logged in, verify the service:

```

# Check service status
systemctl status xyz.openbmc_project.Example.Greeter

# View the object tree
busctl tree xyz.openbmc_project.Example.Greeter

# Introspect the interface
busctl introspect xyz.openbmc_project.Example.Greeter \
    /xyz/openbmc_project/example/greeter

# Read properties
busctl get-property xyz.openbmc_project.Example.Greeter \
    /xyz/openbmc_project/example/greeter \
    xyz.openbmc_project.Example.Greeter \
    Greeting

# Call the SayHello method
busctl call xyz.openbmc_project.Example.Greeter \
    /xyz/openbmc_project/example/greeter \
    xyz.openbmc_project.Example.Greeter \
    SayHello s "World"

# Expected output: s "Hello, OpenBMC! World!"

# Read the updated GreetCount
busctl get-property xyz.openbmc_project.Example.Greeter \
    /xyz/openbmc_project/example/greeter \
    xyz.openbmc_project.Example.Greeter \
    GreetCount

# Expected output: u 1

# Set the Greeting property
busctl set-property xyz.openbmc_project.Example.Greeter \
    /xyz/openbmc_project/example/greeter \
    xyz.openbmc_project.Example.Greeter \
    Greeting s "Greetings from"

# Call SayHello again with the new greeting
busctl call xyz.openbmc_project.Example.Greeter \
    /xyz/openbmc_project/example/greeter \
    xyz.openbmc_project.Example.Greeter \
    SayHello s "OpenBMC Developer"

# Expected output: s "Greetings from OpenBMC Developer!"

# Reset the service
busctl call xyz.openbmc_project.Example.Greeter \
    /xyz/openbmc_project/example/greeter \
    xyz.openbmc_project.Example.Greeter \
    Reset

# Monitor property changes in real time
busctl monitor xyz.openbmc_project.Example.Greeter

```

Quick Test with Docker

You can also test the async implementation locally using the D-Bus example Docker environment:

```
cd docs/examples/dbus
./run.sh shell

# Inside the container, compile and run
g++ -std=c++23 -o greeter main_async.cpp \
    $(pkg-config --cflags --libs sdbusplus) -lboost_system
./greeter &

# Test with busctl
busctl --system call xyz.openbmc_project.Example.Greeter \
    /xyz/openbmc_project/example/greeter \
    xyz.openbmc_project.Example.Greeter \
    SayHello s "Docker"
```

See the complete example at [examples/custom-dbus-service/](#).

Troubleshooting

Issue: Service Fails to Start with “Name already taken”

Symptom: journalctl shows `Failed to request name xyz.openbmc_project.Example.Greeter: Name already taken`

Cause: Another process already owns the D-Bus bus name. This happens when a previous instance did not shut down cleanly or you have a duplicate service.

Solution:

1. Check for running instances: `systemctl status xyz.openbmc_project.Example.Greeter`
2. Stop the existing service: `systemctl stop xyz.openbmc_project.Example.Greeter`
3. Verify no orphan process: `ps aux | grep phosphor-greeter`
4. Kill orphan processes if found: `kill <pid>`

Issue: sdbus++ Fails with “Interface not found”

Symptom: `sdbus++: error: Could not find interface xyz.openbmc_project.Example.Greeter`

Cause: The YAML file path does not match the interface name, or the `-r` (root) argument points to the wrong directory.

Solution:

1. Verify the YAML file is at `yaml/xyz/openbmc_project/Example/Greeter.interface.yaml`
2. Verify directory structure matches the interface name exactly (case-sensitive)
3. Ensure the `-r` flag points to the parent of the `xyz/` directory

Issue: Properties Not Visible After Service Starts

Symptom: `busctl introspect` shows the interface but no properties or methods.

Cause: You forgot to call `iface->initialize()` after registering all properties and methods.

Solution: Add `iface->initialize()` after the last `register_property` or `register_method` call. The interface is not published to D-Bus until you call initialize.

Issue: "Permission denied" When Connecting to System Bus

Symptom: Service crashes with `Failed to open system bus: Permission denied`

Cause: D-Bus security policy does not allow your service to own the requested bus name.

Solution: Create a D-Bus policy file at `/etc/dbus-1/system.d/`:

```
<!-- /etc/dbus-1/system.d/xyz.openbmc_project.Example.Greeter.conf -->
<!DOCTYPE busconfig PUBLIC
"-//freedesktop//DTD D-BUS Bus Configuration 1.0//EN"
"http://www.freedesktop.org/standards/dbus/1.0/busconfig.dtd">
<busconfig>
  <policy user="root">
    <allow own="xyz.openbmc_project.Example.Greeter"/>
    <allow send_destination="xyz.openbmc_project.Example.Greeter"/>
  </policy>
  <policy context="default">
    <allow send_destination="xyz.openbmc_project.Example.Greeter"/>
  </policy>
</busconfig>
```

Issue: BitBake Recipe Build Fails

Symptom: `bitbake phosphor-greeter` fails with missing dependency errors.

Cause: `DEPENDS` or `RDEPENDS` are incomplete, or `sdbusplus` is not available in the build environment.

Solution:

1. Verify `meta-phosphor` layer is included in `bblayers.conf`
2. Check that `sdbusplus-native` is available for code generation: `bitbake sdbusplus-native`
3. Add missing dependencies to `DEPENDS` in the recipe
4. Run `bitbake -e phosphor-greeter | grep ^DEPENDS` to inspect resolved dependencies

Debug Commands

```
# Check service status
systemctl status xyz.openbmc_project.Example.Greeter

# View service logs
journalctl -u xyz.openbmc_project.Example.Greeter -f

# List all D-Bus services
busctl list | grep Example

# Full introspection of the service
busctl introspect xyz.openbmc_project.Example.Greeter \
  /xyz/openbmc_project/example/greeter

# Monitor all signals from the service
busctl monitor xyz.openbmc_project.Example.Greeter

# Check if D-Bus activation file is installed
ls -la /usr/share/dbus-1/system-services/ | grep Greeter

# Verify systemd unit is installed
systemctl cat xyz.openbmc_project.Example.Greeter
```

References

Official Resources

- [sdbusplus Repository](#) – C++ D-Bus library and sdbus++ code generator
- [phosphor-dbus-interfaces](#) – Standard OpenBMC interface YAML definitions
- [sdbusplus Examples](#) – Official sdbusplus usage examples
- [OpenBMC D-Bus Interface Guidelines](#)

Related Guides

- [D-Bus Fundamentals](#) – Core D-Bus concepts and busctl usage
- [State Manager Guide](#) – Real-world example of a D-Bus service

External Documentation

- [D-Bus Specification](#)
- [Meson Build System](#)
- [systemd Service Files](#)
- [Yocto Project Reference Manual](#)

Tested on: QEMU ast2600-evb, OpenBMC commit [HEAD](#) (master branch) Last updated: 2026-02-06

Systemd Boot Ordering Guide

Understand how OpenBMC uses systemd targets, dependencies, and template units to orchestrate the boot sequence and host power management.

Table of Contents

1. [Overview](#)
2. [OpenBMC Target Hierarchy](#)
 1. [BMC Boot Sequence](#)
 2. [What Each Target Provides](#)
 3. [Viewing the Target Hierarchy](#)
3. [Wants, Requires, After, and Before](#)
 1. [Dependency Directives: Wants vs Requires](#)
 2. [Ordering Directives: After vs Before](#)
 3. [Common OpenBMC Ordering Patterns](#)
 1. [Pattern 1: Start After D-Bus Is Available](#)
 2. [Pattern 2: Start After Phosphor-Mapper](#)
 3. [Pattern 3: Start After Sensors Are Available](#)
 4. [Pattern 4: Hook Into a Power-On Target](#)
 5. [Pattern 5: Service That Must Finish Before Another Starts](#)
 4. [Dependency vs Ordering Summary](#)
4. [Boot Time Debugging](#)
 1. [Profiling Boot Time with systemd-analyze](#)
 1. [Overall Boot Time](#)
 2. [Blame: Which Services Are Slowest](#)
 3. [Critical Chain: The Longest Dependency Path](#)
 2. [Debugging Service Failures with journalctl](#)
 1. [View Logs for a Specific Service](#)
 3. [Tracing Dependency Chains with systemctl](#)
5. [Multi-Host Template Units](#)
 1. [Template Unit Basics](#)
 2. [The %i Specifier](#)
 3. [Common systemd Specifiers in OpenBMC](#)
 4. [OpenBMC Multi-Host Target Instances](#)
 5. [Creating a Multi-Host Aware Service](#)
 6. [The RemainAfterExit Pattern](#)
 7. [Inspecting Template Instances](#)
6. [Troubleshooting](#)
 1. [Issue: Service Starts Before Its Dependency Is Ready](#)
 2. [Issue: Service Is Not Started During Host Power-On](#)

3. [Issue: Oneshot Service Shows "inactive \(dead\)"](#)

4. [Issue: Boot Hangs at a Specific Target](#)

7. [References](#)

1. [Official Resources](#)

2. [Related Guides](#)

3. [External Documentation](#)

Overview

OpenBMC relies on systemd as its init system and service manager. Every daemon, one-shot script, and power-state transition is controlled through systemd units. Understanding how these units relate to each other is essential for developing or debugging OpenBMC services.

This guide covers the target hierarchy that drives the BMC boot sequence, the dependency directives that control service ordering, tools for diagnosing boot timing and failures, and the template unit mechanism that enables multi-host support. All commands run directly on the BMC and are fully testable in a QEMU ast2600-evb environment.

Key concepts covered:

- OpenBMC systemd target hierarchy (boot and power-on chains)
- Wants vs Requires and After vs Before semantics
- Boot time profiling with `systemd-analyze`
- Service failure debugging with `journalctl` and `systemctl`
- Multi-host template units (`@.service` , `%i` specifier)
- The `RemainAfterExit=yes` pattern for oneshot services

OpenBMC Target Hierarchy

systemd uses **targets** as synchronization points. A target groups related services together and establishes ordering between boot phases. OpenBMC defines a custom set of targets on top of the standard systemd targets.

BMC Boot Sequence

When the BMC powers on, systemd walks the following target chain to bring the system to a ready state:



What Each Target Provides

Target	Services Pulled In
<code>multi-user.target</code>	systemd core, networking, sshd
<code>obmc-standby.target</code>	D-Bus broker, phosphor-mapper, bmcweb, state-manager
<code>obmc-chassis-poweron@.target</code>	PSU monitor, fan monitor, power sequencing
<code>obmc-host-startmin@.target</code>	Host state manager, host check services
<code>obmc-host-start@.target</code>	IPMI host, watchdog, SOL console, sensor polling
<code>obmc-host-stop@.target</code>	IPMI stop, graceful shutdown coordination
<code>obmc-chassis-poweroff@.target</code>	Power rail disable, fan ramp-down

The `obmc-standby.target` is the dividing line between “BMC boot” and “host management.” The BMC state manager transitions to `Ready` when this target completes. Everything below it activates only when a host power-

on is requested.

Viewing the Target Hierarchy

Use `systemctl list-dependencies` to explore the live target tree on a running BMC:

```
# Show what obmc-standby.target pulls in
systemctl list-dependencies obmc-standby.target

# Show the full host power-on chain
systemctl list-dependencies obmc-host-start@0.target

# Show reverse dependencies (what depends on a target)
systemctl list-dependencies --reverse obmc-chassis-poweron@0.target

# Show all targets and their current state
systemctl list-units --type=target
```

Add `--all` to `list-dependencies` to include inactive units. This helps you see services that are installed but not currently running.

Wants, Requires, After, and Before

systemd separates two concepts that developers often confuse: **dependency** (whether to start a unit) and **ordering** (when to start it). OpenBMC services use both, and getting them wrong causes subtle boot failures.

Dependency Directives: Wants vs Requires

Directive	Effect	If dependency fails
<code>Wants=B.service</code>	Start B when A starts	A keeps running
<code>Requires=B.service</code>	Start B when A starts	A is stopped
<code>BindsTo=B.service</code>	Start B when A starts	A is stopped immediately

Wants is the most common directive in OpenBMC. It creates a soft dependency: the system tries to start the wanted unit, but the wanting unit survives if the dependency fails.

Requires creates a hard dependency: if the required unit fails to start or is stopped, the requiring unit is also stopped. Use this sparingly – it creates brittle dependency chains.

```
# Typical OpenBMC pattern: soft dependency with Wants
[Unit]
Description=Phosphor Fan Monitor
Wants=phosphor-fan-presence-tach.service
After=phosphor-fan-presence-tach.service
```

Never use `Requires=` when `Wants=` is sufficient. If a `Requires=` target fails transiently (for example, a sensor driver takes an extra second to load), the entire dependent chain collapses. OpenBMC targets use `Wants=` for this reason.

Ordering Directives: After vs Before

Directive	Effect
<code>After=B.service</code>	Start A only after B finishes starting
<code>Before=B.service</code>	Start A before B begins starting

`Wants=` and `After=` are independent. `Wants=B` without `After=B` starts both A and B simultaneously. `After=B` without `Wants=B` means A waits for B only if something else starts B. You almost always need both together.

Common OpenBMC Ordering Patterns

Pattern 1: Start After D-Bus Is Available

Most OpenBMC services need the D-Bus broker running before they can register their interfaces:

```
[Unit]
Description=My OpenBMC Service
After=dbus.service
Wants=dbus.service
```

Pattern 2: Start After Phosphor-Mapper

Services that need to look up other services through the object mapper must wait for it:

```
[Unit]
Description=My Mapper-Dependent Service
After=mapper-wait@-xyz-openbmc_project-sensors.service
Wants=mapper-wait@-xyz-openbmc_project-sensors.service
```

The `mapper-wait@.service` template is a special helper that blocks until a specific D-Bus object path is available. The path `/xyz/openbmc_project/sensors` is encoded with dashes as `-xyz-openbmc_project-sensors` in the unit name.

Pattern 3: Start After Sensors Are Available

A service that processes sensor data should not start until the sensor daemons have registered their D-Bus objects:

```
[Unit]
Description=Thermal Policy Engine
After=phosphor-hwmon-readall.service
After=xyz.openbmc_project.EntityManager.service
Wants=phosphor-hwmon-readall.service
```

Pattern 4: Hook Into a Power-On Target

To run a custom script when the host powers on, use `WantedBy=` in the `[Install]` section:

```
[Unit]
Description=Platform-specific power on hook
After=obmc-chassis-poweron@0.target
Before=obmc-host-startmin@0.target

[Service]
Type=oneshot
ExecStart=/usr/bin/my-platform-power-on.sh
RemainAfterExit=yes

[Install]
WantedBy=obmc-host-start@0.target
```

Pattern 5: Service That Must Finish Before Another Starts

When a configuration service must complete before the main daemon starts:

```
# config-loader.service
[Unit]
Description=Load sensor configuration
Before=phosphor-pid-control.service

[Service]
Type=oneshot
ExecStart=/usr/bin/load-sensor-config.sh
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

Dependency vs Ordering Summary

Combination	Meaning
<code>Wants=B</code> + <code>After=B</code>	Start B, then start A (most common)
<code>Requires=B</code> + <code>After=B</code>	Start B, then start A (hard dependency)
<code>After=B</code> (alone)	Wait for B if B is active (passive wait)
<code>Wants=B</code> (alone)	Start B in parallel (rarely correct)
<code>WantedBy=T.target</code>	T.target pulls this service in via Wants=

Boot Time Debugging

When the BMC boots slowly or a service fails to start, systemd provides powerful diagnostic tools. All of these commands work on the BMC over SSH (or via QEMU serial console).

Profiling Boot Time with systemd-analyze

Overall Boot Time


```
# Show total boot time breakdown
systemd-analyze

# Example output:
# Startup finished in 1.234s (kernel) + 12.345s (userspace) = 13.579s
# graphical.target reached after 12.000s in userspace
```

Blame: Which Services Are Slowest

```
# List services sorted by startup time (slowest first)
systemd-analyze blame

# Example output:
# 5.123s phosphor-image-signing.service
# 3.456s obmc-flash-bmc-init.service
# 2.789s phosphor-mapper.service
# 1.234s phosphor-hwmon@org-openbmc-sensors.service
# 0.987s bmcweb.service
# 0.654s phosphor-certificate-manager@authority.service
# ...
```

On a typical QEMU environment, boot takes 15-30 seconds. On real hardware with flash storage, expect 30-90 seconds. Anything significantly above that indicates a problem worth investigating.

Critical Chain: The Longest Dependency Path

```
# Show the critical path to default.target
systemd-analyze critical-chain

# Show the critical path to a specific target
systemd-analyze critical-chain obmc-standby.target

# Example output:
# obmc-standby.target @12.345s
# └─bmcweb.service @8.123s +2.456s
#   └─phosphor-certificate-manager@authority.service @6.789s +1.234s
#     └─phosphor-mapper.service @3.456s +3.333s
#       └─dbus.service @1.234s +0.567s
#         └─basic.target @1.200s
#           └─sockets.target @1.199s
#             └─dbus.socket @0.987s
```

The `@` values show when each unit started, and the `+` values show how long each unit took. The critical chain reveals which service is the actual bottleneck, not just which service is slow.

`systemd-analyze blame` shows raw duration. `systemd-analyze critical-chain` shows the actual bottleneck path. A service that takes 5 seconds is not a problem if it starts in parallel with a 10-second service on the critical path. Focus on the critical chain.

Debugging Service Failures with journalctl

View Logs for a Specific Service

```
# Follow live logs for a service
journalctl -u phosphor-state-manager.service -f

# View recent logs (last 50 lines)
journalctl -u phosphor-state-manager.service -n 50

# View logs from this boot only
journalctl -u phosphor-state-manager.service -b

# View logs with full output (no truncation)
journalctl -u phosphor-state-manager.service -b --no-pager
```

You can also filter by priority (`-p err` for errors only) or view all boot errors across services with `journalctl -b -p err`.

Tracing Dependency Chains with systemctl

When a service fails because of a dependency problem, trace the chain:

```
# Check service status and recent log output
systemctl status phosphor-pid-control.service

# List what a unit depends on
systemctl list-dependencies phosphor-pid-control.service

# List what depends on a unit (reverse)
systemctl list-dependencies --reverse dbus.service

# Show the full unit file with all overrides applied
systemctl cat phosphor-pid-control.service
```

Multi-Host Template Units

OpenBMC supports platforms with multiple host CPUs. Each host is managed independently through systemd **template units**, which use the `@` syntax to create parameterized service instances.

Template Unit Basics

A template unit has `@` in its filename before the suffix:

```
obmc-host-start@.target    ← template (no instance)
obmc-host-start@0.target   ← instance 0 (first host)
obmc-host-start@1.target   ← instance 1 (second host)
```

The portion after `@` and before the suffix is the **instance identifier**. In OpenBMC, the instance identifier is typically the host number (0, 1, 2, ...).

The %i Specifier

Inside a template unit file, `%i` expands to the instance identifier at runtime:

```
# /lib/systemd/system/phosphor-host-state-manager@.service
[Unit]
Description=Phosphor Host State Manager for host %i
After=obmc-host-startmin@%i.target
BindsTo=obmc-host-startmin@%i.target

[Service]
ExecStart=/usr/bin/phosphor-host-state-manager --host %i

[Install]
WantedBy=obmc-host-startmin@%i.target
```

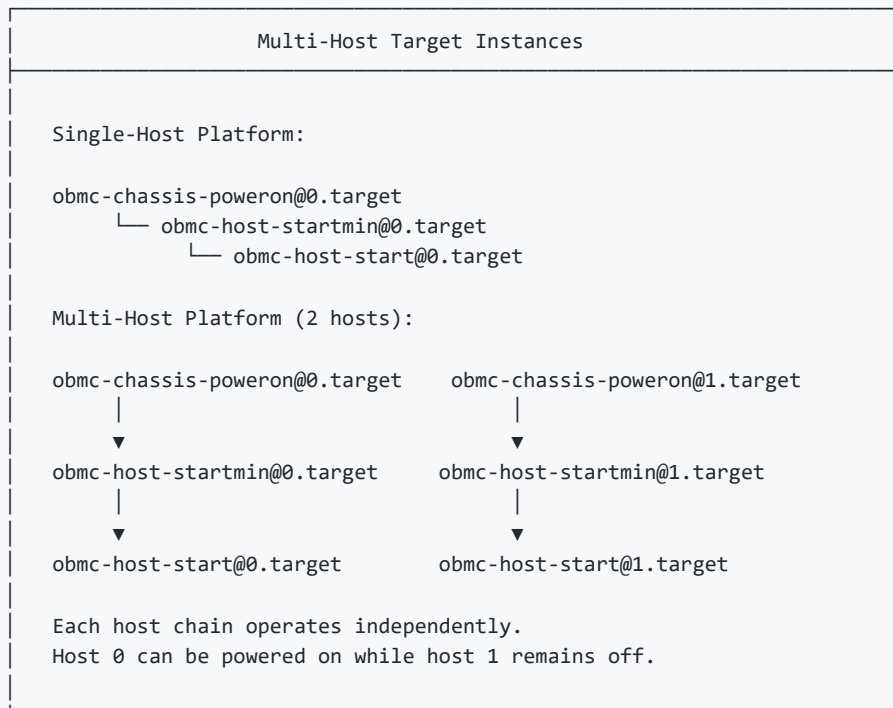
When systemd starts `phosphor-host-state-manager@0.service`, every `%i` becomes `0`. When it starts `phosphor-host-state-manager@1.service`, every `%i` becomes `1`.

Common systemd Specifiers in OpenBMC

Specifier	Expands To	Example
<code>%i</code>	Instance identifier	<code>0</code> , <code>1</code> , <code>2</code>
<code>%n</code>	Full unit name	<code>my-service@0.service</code>
<code>%p</code>	Unit prefix (before @)	<code>my-service</code>

OpenBMC Multi-Host Target Instances

Every power-management target in OpenBMC is a template. On a single-host platform, only `@0` instances exist. On a multi-host platform (for example, a 4-socket server), instances `@0` through `@3` activate independently:



Creating a Multi-Host Aware Service

When you write a service that must run per-host, create a template unit:

```
# /lib/systemd/system/my-host-monitor@.service
[Unit]
Description=My Host Monitor for host %i
After=obmc-host-startmin@%i.target
BindsTo=obmc-host-startmin@%i.target

[Service]
Type=simple
ExecStart=/usr/bin/my-host-monitor --host %i
Restart=on-failure

[Install]
WantedBy=obmc-host-start@%i.target
```

Enable it for specific host instances:

```
# Enable for host 0
systemctl enable my-host-monitor@0.service

# Enable for host 1 (multi-host platform)
systemctl enable my-host-monitor@1.service

# Start manually for testing
systemctl start my-host-monitor@0.service

# Check status of a specific instance
systemctl status my-host-monitor@0.service
```

The RemainAfterExit Pattern

Many OpenBMC power-sequencing services are `Type=oneshot` scripts that run once and must be considered “active” after they exit. Without `RemainAfterExit=yes`, systemd marks a oneshot service as `inactive (dead)` after the script finishes, which breaks dependency chains that use `BindsTo=` or `After=`.

```
# /lib/systemd/system/chassis-power-on@.service
[Unit]
Description=Chassis Power On Sequence for host %i
After=obmc-chassis-poweron@%i.target

[Service]
Type=oneshot
ExecStart=/usr/bin/power-on-sequence.sh %i
RemainAfterExit=yes

[Install]
WantedBy=obmc-host-startmin@%i.target
```

`RemainAfterExit=yes` tells systemd to treat the service as `active (exited)` after the `ExecStart` command completes successfully. This is critical for oneshot services that other units depend on. Without it, `After=chassis-power-on@0.service` would see the service as “dead” and behave unpredictably.

Inspecting Template Instances

```
# List all instantiated units from a template
systemctl list-units 'phosphor-host-state-manager@*'

# Show the template unit file
systemctl cat phosphor-host-state-manager@.service

# Show a specific instance's resolved configuration
systemctl show phosphor-host-state-manager@0.service

# Check which instance specifier values are active
systemctl list-units --type=target 'obmc-host-start@*'

```

Troubleshooting

Issue: Service Starts Before Its Dependency Is Ready

Symptom: A service fails with “Connection refused” or “D-Bus name not found” errors during boot, but works when restarted manually.

Cause: The service has `Wants=` on its dependency but is missing `After=`. Both units start simultaneously, and the dependency loses the race.

Solution:

1. Add `After=` for the dependency:

```
[Unit]
Wants=phosphor-mapper.service
After=phosphor-mapper.service

```

2. Reload and restart:

```
systemctl daemon-reload
systemctl restart my-service.service

```

Issue: Service Is Not Started During Host Power-On

Symptom: Your service never starts when you run `obmcutil poweron`, even though it is enabled.

Cause: The `WantedBy=` in the `[Install]` section does not reference a power-on target, or the service was not properly enabled.

Solution:

1. Verify `[Install]` has `WantedBy=obmc-host-start@0.target`
2. Re-enable: `systemctl disable my-service && systemctl enable my-service`
3. Verify the symlink: `ls -la /etc/systemd/system/obmc-host-start@0.target.wants/`

Issue: Oneshot Service Shows “inactive (dead)”

Symptom: A oneshot service runs successfully but shows `inactive (dead)`. Dependent services behave inconsistently.

Cause: Missing `RemainAfterExit=yes` in the unit file.

Solution: Add `RemainAfterExit=yes` to the `[Service]` section. The service then shows `active (exited)` instead of `inactive (dead)`.

Issue: Boot Hangs at a Specific Target

Symptom: The BMC boot stalls and never reaches `obmc-standby.target`.

Cause: A service in the dependency chain has hung or timed out.

Solution:

```
# Identify the stuck service
systemctl list-jobs
systemctl list-units --state=activating

# Examine logs for the stuck service
journalctl -u stuck-service.service -b --no-pager

# Temporarily skip the stuck service
systemctl mask stuck-service.service
# After debugging, unmask: systemctl unmask stuck-service.service
```

Circular dependencies cause unpredictable boot order. If you see `Found ordering cycle` in the journal, review your `After=` / `Before=` directives and remove one direction of the cycle.

References

Official Resources

- [OpenBMC systemd Design](#)
- [phosphor-state-manager](#)
- [OpenBMC Target Definitions](#)
- [OpenBMC Documentation](#)

Related Guides

- [State Management](#)
- [D-Bus Fundamentals](#)

External Documentation

- [systemd Unit Configuration](#)
- [systemd Service Configuration](#)
- [systemd Template Units](#)
- [systemd-analyze Manual](#)

Tested on: QEMU ast2600-evb, OpenBMC master branch Last updated: 2026-02-06

Core Services

Master the essential services that make up an OpenBMC system.

What You'll Learn

- Sensor monitoring and configuration (D-Bus sensors, hwmon)
- Entity Manager for hardware discovery
- Thermal management and fan control (PID, zones)
- Power management and sequencing
- User management (accounts, privileges, LDAP)
- Network configuration (IP, VLAN)
- LED control (identify, lamp test)
- Certificate management (TLS/SSL)
- Time synchronization (NTP, RTC)
- Hardware inventory and FRU data
- Watchdog timer configuration
- Button handling (power, reset)
- Physical-security and intrusion detection (chassis switch, BMC tamper input, battery-removal)

Service Categories

Sensors & Monitoring

Monitor hardware health through temperature, voltage, current, and fan sensors.

Guide	Description
D-Bus Sensors	ADC, hwmon, virtual sensors
Hwmon Sensors	Linux hwmon integration
Entity Manager	Hardware discovery and configuration
GPIO Management	GPIO monitoring, phosphor-gpio-monitor
PECI Thermal Monitoring	CPU temperature via PECT interface
I2C Device Integration	I2C device trees, tools, custom drivers
Leak Detection	Liquid-cooling leak sensors, shutdown orchestration
NVIDIA GPU Management	GPU/PCIe/SMA telemetry and control over MCTP

Thermal & Power

Control cooling and power systems to keep hardware within operating limits.

Guide	Description
Fan Control	PID thermal control, zones
Power Management	Power sequencing, regulators
PSU PMBus Management	PSU monitoring via PMBus/phosphor-psu-monitor
PID Thermal Tuning	PID coefficient tuning methodology

System Services

Essential services for system operation.

Guide	Description
User Manager	Accounts, privileges, LDAP
Network	IP configuration, VLAN
LED Manager	LED control, identify, lamp test
Certificate Manager	TLS/SSL certificates
Time Manager	NTP, RTC, timezones
Inventory Manager	FRU, hardware inventory
Watchdog	Host watchdog timer
Buttons	Power/reset buttons
LDAP Integration	LDAP/Active Directory authentication

Physical Security

Detect chassis tampering and protect against unauthorized hardware access.

Guide	Description
Intrusion Detection	Chassis switch, BMC tamper input, battery-backed latch, VBAT-SRAM token

Prerequisites

- Complete [Architecture](#) section
- Understand D-Bus basics
- Working QEMU environment

Table of contents

- [D-Bus Sensors Guide](#)
- [Hwmon Sensors Guide](#)
- [Entity Manager Guide](#)

- [Fan Control Guide](#)
- [Power Management Guide](#)
- [User Manager Guide](#)
- [Network Guide](#)
- [LED Manager Guide](#)
- [Certificate Manager Guide](#)
- [Time Manager Guide](#)
- [Inventory Manager Guide](#)
- [Watchdog Guide](#)
- [Buttons Guide](#)
- [GPIO Management Guide](#)
- [PECI Thermal Monitoring](#)
- [I2C Device Integration](#)
- [PSU & PMBus Management](#)
- [LDAP Integration](#)
- [PID Thermal Tuning](#)
- [Intrusion Detection Guide](#)
- [Leak Detection](#)
- [NVIDIA GPU Management](#)

D-Bus Sensors Guide

Configure and use dbus-sensors for hardware monitoring.

Table of Contents

1. [Overview](#)
2. [Sensor Daemons](#)
3. [Configuration & Setup](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [Meson Build Options](#)
 3. [Runtime Enable/Disable](#)
 4. [Sensor Custom Properties](#)
 5. [Hysteresis Configuration](#)
4. [D-Bus Sensor Interface](#)
 1. [Threshold Interfaces](#)
5. [ADC Sensors](#)
 1. [Device Tree Configuration](#)
 2. [Entity Manager Configuration](#)
 3. [Key Properties](#)
6. [Hwmon Temperature Sensors](#)
 1. [Entity Manager Configuration](#)
 2. [Supported Sensor Types](#)
7. [PSU Sensors](#)
 1. [Entity Manager Configuration](#)
 2. [Exposed Sensors](#)
8. [Virtual/External Sensors](#)
 1. [Entity Manager Configuration](#)
 2. [Setting Values via D-Bus](#)
9. [SMBPBI Sensors](#)
10. [Cable Monitoring](#)
11. [IPMB Sensors](#)
 1. [SDR Auto-Discovery \(IpmbSDRDevice\)](#)
12. [MCU Temperature and NVMe Sensors](#)
 1. [MCU Temperature \(mcutempsensor\)](#)
 2. [NVMe \(nvmesensor\)](#)
13. [Threshold Configuration](#)
 1. [Threshold Levels](#)
 2. [Direction](#)
 3. [Example with All Thresholds](#)

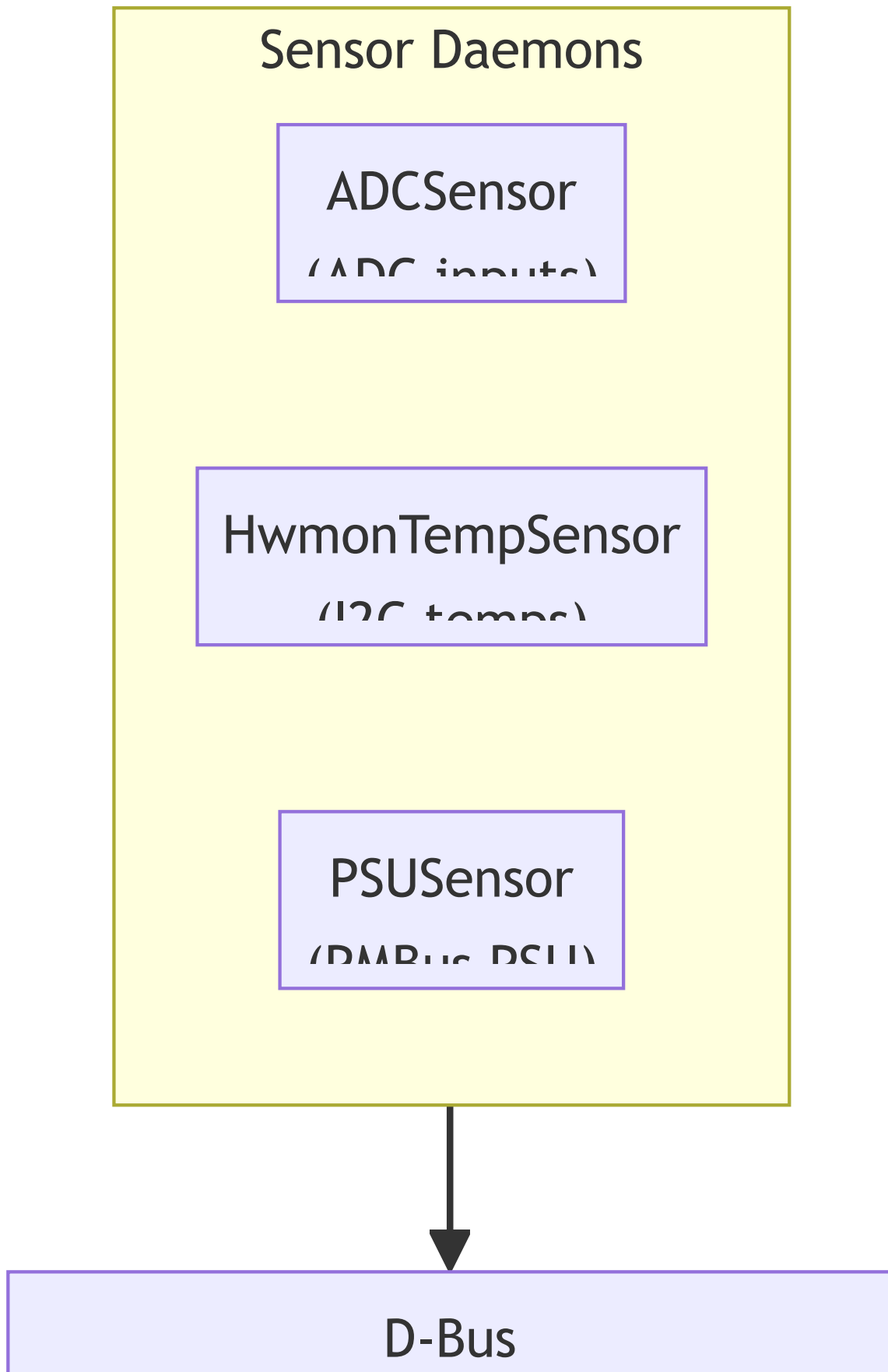
- 14. [Reading Sensors](#)
 - 1. [Via D-Bus](#)
 - 2. [Via Redfish](#)
 - 15. [Troubleshooting](#)
 - 1. [Sensor not appearing](#)
 - 2. [Incorrect readings](#)
 - 3. [Threshold alarms not triggering](#)
 - 16. [Deep Dive](#)
 - 1. [Sensor Reading Pipeline](#)
 - 2. [Threshold Detection Algorithm](#)
 - 3. [Entity Manager Integration Flow](#)
 - 4. [ADC Scaling and Voltage Dividers](#)
 - 5. [Power State Filtering](#)
 - 6. [Source Code Reference](#)
 - 17. [Examples](#)
 - 18. [References](#)
-

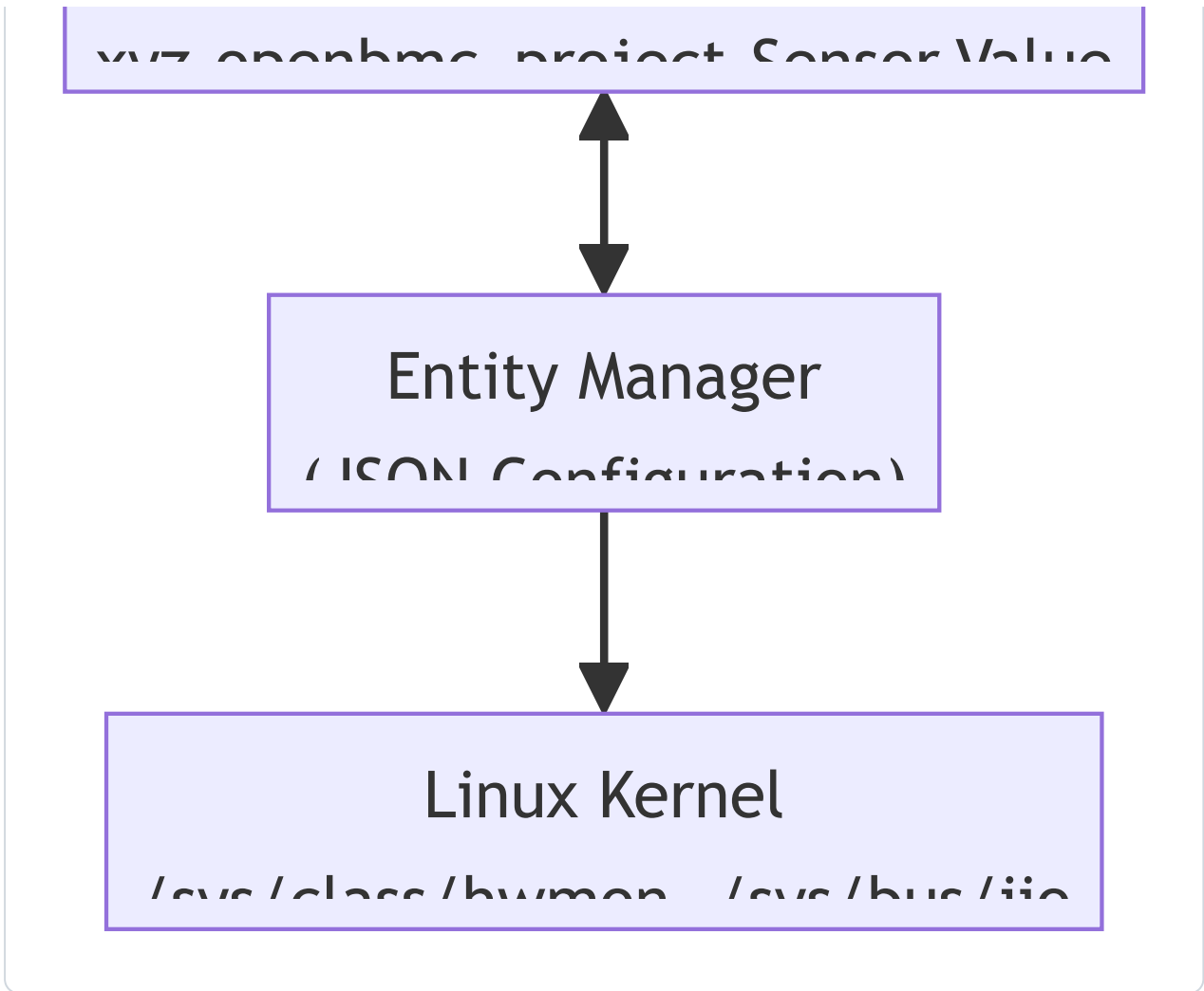
Overview

dbus-sensors is a collection of sensor daemons that expose hardware sensor data via D-Bus. Each daemon handles a specific sensor type and integrates with Entity Manager for configuration.



dbus-sensors Architecture





► ASCII-art version (for comparison)

Sensor Daemons

Daemon	Sensor Types	Data Source
<code>adcsensor</code>	ADC voltage inputs	IIO subsystem
<code>hwmontempsensor</code>	I2C temperature sensors	hwmon sysfs
<code>psusensor</code>	PSU voltage, current, power	PMBus hwmon
<code>fansensor</code>	Fan tachometers, PWM	hwmon sysfs
<code>intrusionsensor</code>	Chassis intrusion	GPIO
<code>ipmbsensor</code>	IPMB sensors	IPMB interface
<code>mcutempsensor</code>	MCU temperature	I2C
<code>nvmesensor</code>	NVMe temperature	NVMe-MI
<code>externalsensor</code>	Virtual sensors	D-Bus input

Configuration & Setup

Build-Time Configuration (Yocto)

Enable or disable specific sensor daemons in your machine configuration:

```
# In your machine .conf or image recipe

# Include all sensor daemons (default)
IMAGE_INSTALL:append = " dbus-sensors"

# Or include specific sensor daemons only
IMAGE_INSTALL:append = " \
    adcsensor \
    hwmontempsensor \
    fansensor \
    psusensor \
"

# Exclude specific daemons
RDEPENDS:${PN}:remove:pn-dbus-sensors = "intrusionsensor ipmbsensor"
```

Meson Build Options

When building dbus-sensors from source:

```
# View all options
meson configure build

# Common options
meson setup build \
    -Dadc=enabled \
    -Dhwmon-temp=enabled \
    -Dfan=enabled \
    -Dpsu=enabled \
    -Dintrusion=disabled \
    -Dipmb=disabled \
    -Dnvme=enabled \
    -Dexternal=enabled
```

Option	Default	Description
<code>adc</code>	enabled	ADC voltage sensors
<code>hwmon-temp</code>	enabled	I2C temperature sensors
<code>fan</code>	enabled	Fan tachometer sensors
<code>psu</code>	enabled	PMBus PSU sensors
<code>intrusion</code>	enabled	Chassis intrusion sensor
<code>ipmb</code>	enabled	IPMB sensors
<code>nvme</code>	enabled	NVMe temperature sensors
<code>external</code>	enabled	External/virtual sensors
<code>mcu-temp</code>	enabled	MCU temperature sensor

Runtime Enable/Disable

Control sensor daemons at runtime via systemd:

```
# Disable a sensor daemon
systemctl disable xyz.openbmc_project.adcsensor
systemctl stop xyz.openbmc_project.adcsensor

# Enable a sensor daemon
systemctl enable xyz.openbmc_project.hwmontempsensor
systemctl start xyz.openbmc_project.hwmontempsensor

# Check status
systemctl status xyz.openbmc_project.fansensor

# List all sensor services
systemctl list-units | grep sensor
```

Sensor Custom Properties

Sensors support additional configuration properties in Entity Manager:


```
{
  "Exposes": [
    {
      "Name": "CPU Temp",
      "Type": "TMP75",
      "Bus": 1,
      "Address": "0x48",

      "PowerState": "On",
      "ReadState": "On",
      "PollRate": 1.0,
      "Offset": 0.0,
      "ScaleFactor": 1.0,
      "Label": "CPU Core Temperature",
      "MaxValue": 125,
      "MinValue": -40,

      "Thresholds": [...]
    }
  ]
}
```

Property	Type	Description
<code>PowerState</code>	string	When to poll: <code>On</code> , <code>BiosPost</code> , <code>Always</code>
<code>ReadState</code>	string	Alternate power state control
<code>PollRate</code>	float	Polling interval in seconds (default: 1.0)
<code>Offset</code>	float	Value offset adjustment
<code>ScaleFactor</code>	float	Multiplier for raw value
<code>Label</code>	string	Human-readable label
<code>MaxValue</code>	float	Maximum expected value
<code>MinValue</code>	float	Minimum expected value

Hysteresis Configuration

Configure threshold hysteresis to prevent alarm flapping:

```
{
  "Thresholds": [
    {
      "Direction": "greater than",
      "Name": "upper critical",
      "Severity": 1,
      "Value": 95,
      "Hysteresis": 2.0
    }
  ]
}
```

The alarm clears when value drops below `Value - Hysteresis` (93°C in this example).

D-Bus Sensor Interface

All sensors implement `xyz.openbmc_project.Sensor.Value` :

```
# Introspect a sensor
busctl introspect xyz.openbmc_project.ADCSensor \
    /xyz/openbmc_project/sensors/voltage/P12V

# Properties:
# Value          - Current sensor reading (double)
# MaxValue       - Maximum possible value
# MinValue       - Minimum possible value
# Unit           - Measurement unit
```

Threshold Interfaces

Sensors can have warning and critical thresholds:

```
xyz.openbmc_project.Sensor.Threshold.Warning
- WarningHigh
- WarningLow
- WarningAlarmHigh
- WarningAlarmLow

xyz.openbmc_project.Sensor.Threshold.Critical
- CriticalHigh
- CriticalLow
- CriticalAlarmHigh
- CriticalAlarmLow
```

ADC Sensors

ADC sensors read analog voltage inputs via the Linux IIO subsystem.

Device Tree Configuration

```
// In your device tree
&adc {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_adc0_default
                &pinctrl_adc1_default>;
};
```

Entity Manager Configuration

```
{
  "Exposes": [
    {
      "Index": 0,
      "Name": "P12V",
      "ScaleFactor": 4.0,
      "Type": "ADC"
    },
    {
      "Index": 1,
      "Name": "P3V3",
      "ScaleFactor": 1.0,
      "Type": "ADC",
      "Thresholds": [
        {
          "Direction": "greater than",
          "Name": "upper critical",
          "Severity": 1,
          "Value": 3.6
        },
        {
          "Direction": "less than",
          "Name": "lower critical",
          "Severity": 1,
          "Value": 3.0
        }
      ]
    }
  ],
  "Name": "MyBoard",
  "Type": "Board"
}
```

Key Properties

Property	Description
Index	ADC channel number (matches hwmon inX_input)
Name	Sensor name on D-Bus
ScaleFactor	Voltage divider ratio
PowerState	When to read (On, BiosPost, Always)
Thresholds	Warning/critical levels

Hwmon Temperature Sensors

For I2C temperature sensors (TMP75, LM75, etc.).

Entity Manager Configuration

```
{
  "Exposes": [
    {
      "Address": "0x48",
      "Bus": 1,
      "Name": "CPU Temp",
      "Type": "TMP75"
    },
    {
      "Address": "0x49",
      "Bus": 1,
      "Name": "Inlet Temp",
      "Type": "TMP75",
      "Thresholds": [
        {
          "Direction": "greater than",
          "Name": "upper critical",
          "Severity": 1,
          "Value": 45
        }
      ]
    }
  ],
  "Name": "TempSensors",
  "Probe": "TRUE",
  "Type": "Board"
}
```

Supported Sensor Types

Type	Chip	Common Uses
TMP75	TI TMP75	Ambient temperature
TMP421	TI TMP421	Remote diode sensing
TMP112	TI TMP112	Low power temp
LM75A	NXP LM75A	General purpose
EMC1413	Microchip	Multi-channel

PSU Sensors

For PMBus power supplies.

Entity Manager Configuration

```
{
  "Exposes": [
    {
      "Address": "0x58",
      "Bus": 3,
      "Name": "PSU1",
      "Type": "pmbus"
    }
  ],
  "Name": "PowerSupply",
  "Probe": "TRUE",
  "Type": "Board"
}
```

Exposed Sensors

PSU sensor daemon automatically exposes:

- Input/output voltage
- Input/output current
- Input/output power
- Temperature
- Fan speed
- Status flags

Virtual/External Sensors

For calculated values or sensors from external sources.

Entity Manager Configuration

```
{
  "Exposes": [
    {
      "Name": "Total Power",
      "Type": "ExternalSensor",
      "Units": "Watts",
      "MinValue": 0,
      "MaxValue": 2000
    }
  ],
  "Name": "VirtualSensors",
  "Type": "Board"
}
```

Setting Values via D-Bus

```
# External sensors can be written to
busctl set-property xyz.openbmc_project.ExternalSensor \
  /xyz/openbmc_project/sensors/power/Total_Power \
  xyz.openbmc_project.Sensor.Value Value d 450.5
```

The `externalsensor` daemon can enforce **staleness** on a writable sensor. Add a `Timeout` (in seconds) to the Entity Manager entry: if no new value is written within that window, the daemon's reaper (`updateReaper`) invalidates the sensor and publishes `NaN` on `Value`, so a stalled producer surfaces as "unavailable" instead of a stuck last-known reading. A `Timeout` of `0` (or an omitted key) makes the sensor non-perishable — it keeps its last written value indefinitely.

```
{
  "Name": "Total Power",
  "Type": "ExternalSensor",
  "Units": "Watts",
  "Timeout": 5,
  "MinValue": 0,
  "MaxValue": 2000
}
```

SMBPBI Sensors

The `smbpbpi` daemon (service `xyz.openbmc_project.SMBPBI`) reads telemetry from a management controller — typically an NVIDIA HMC/GPU — that exposes an **SMBus Post-Box Interface (SMBPBI)** "virtual EEPROM" over I2C. Each Entity Manager entry of type `SmbpbpiVirtualEeprom` maps a byte offset in that virtual EEPROM to one D-Bus sensor.

```
{
  "Exposes": [
    {
      "Name": "GPU0 Temp",
      "Type": "SmbpbpiVirtualEeprom",
      "Bus": 3,
      "Address": "0x4f",
      "ReadOffset": 0,
      "Units": "DegreesC",
      "ValueType": "UINT64",
      "PollRate": 1.0,
      "MinValue": 0,
      "MaxValue": 128
    }
  ],
  "Name": "GPU Board",
  "Type": "Board"
}
```

The daemon issues a raw `I2C_RDWR` transfer to read the block at `ReadOffset`, then decodes it according to `Units`: `convert2Temp` for `DegreesC`, `convert2Power` for `Watts`, plus direct handling for `Joules` (energy) and `Volts` (voltage). The `Units` value picks both the decode routine and the D-Bus object path (`/xyz/openbmc_project/sensors/{temperature,power,energy,voltage}/...`).

When the management controller has not yet published a value, the SMBPBI block reads back as all `0xFF`. The daemon treats that as "no data" and publishes `NaN` rather than a bogus reading, so a consumer can tell a genuine zero from an absent one.

Cable Monitoring

The `cable-monitor` daemon (service `xyz.openbmc_project.cablemonitor`) tracks whether the cables a platform expects are actually present and raises Redfish events when one is plugged or unplugged. Unlike the register-reading sensor daemons, it does not touch hardware — it watches the **inventory** on D-Bus and compares it against a configured expected list.

The expected cables come from a JSON file at `/var/lib/cablemonitor/cable-config.json` with a `ConnectedCables` array of cable names:

```
{
  "ConnectedCables": [
    "PCIe_Cable_0",
    "PCIe_Cable_1"
  ]
}
```

At runtime the daemon subscribes to inventory objects that implement `xyz.openbmc_project.Inventory.Item.Cable` (published by Entity Manager) and reacts to their `InterfacesAdded` / `InterfacesRemoved` signals; it also watches the config file with inotify so an updated list is picked up without a restart. When an expected cable's inventory object appears it logs a `CableConnected` event, and when one disappears it logs `CableDisconnected` (both in the `xyz.openbmc_project.State.Cable` namespace, routed through phosphor-logging to the Redfish event log).

```
# Watch cable connect/disconnect events
journalctl -u xyz.openbmc_project.cablemonitor -f
```

IPMB Sensors

The `ipmbsensor` daemon (service `xyz.openbmc_project.IpmbSensor`) reads sensors that live behind an **IPMB** (Intelligent Platform Management Bus) satellite controller — a BMC, ME (management engine), or voltage-regulator bridge — instead of on a local I2C bus. It does not talk to the bus directly; it sends IPMI requests through the `org.openbmc.Ipmb` D-Bus service (`/xyz/openbmc_project/Ipmb/Channel/Ipmb`, `sendRequest` method), which an IPMB transport daemon relays onto the wire.

Each Entity Manager entry of type `IpmbSensor` names a bridge `Class` and a `SensorType`:

```
{
  "Exposes": [
    {
      "Name": "MB Temp",
      "Type": "IpmbSensor",
      "Class": "METemp",
      "SensorType": "temperature",
      "Address": "0x2e",
      "Bus": 0,
      "HostSMBusIndex": 3,
      "PollRate": 1.0
    }
  ],
  "Name": "IPMB Board",
  "Type": "Board"
}
```

Field	Purpose
<code>Class</code>	Bridge/reading protocol: <code>METemp</code> / <code>MESensor</code> , <code>PxeBridgeTemp</code> , <code>IRBridgeTemp</code> , <code>HSCBridge</code> , <code>MpsBridgeTemp</code> , <code>SMPPro</code>
<code>SensorType</code>	Reading subtype: <code>temperature</code> (default), <code>voltage</code> , <code>power</code> , <code>current</code> , <code>utilization</code>
<code>Address</code>	IPMB device (satellite) address
<code>HostSMBusIndex</code>	SMBus index passed in bridged VR commands

SDR Auto-Discovery (IpmbSDRDevice)

Beyond the statically declared sensors above, the daemon can enumerate a satellite's sensors automatically by reading its **SDR (Sensor Data Record) repository**. The `IpmbSDRDevice` class walks the repository — Get SDR Repository Info, Reserve SDR Repository, then repeated Get SDR — and for every **Type-01 (Full Sensor Record)** entry it parses the sensor name, unit, conversion factors, and thresholds, then creates a matching D-Bus sensor. This lets a platform pick up all of a satellite BMC's sensors without listing each one in Entity Manager.

MCU Temperature and NVMe Sensors

MCU Temperature (mcutempsensor)

The `mcutempsensor` daemon (service `xyz.openbmc_project.MCUTempSensor`) reads a temperature value from a microcontroller register over I2C. The Entity Manager entry (type `MCUTempSensor`) gives the `Bus` , `Address` , and the register offset in `Reg` ; the daemon opens `/dev/i2c-<Bus>` and issues an SMBus read-word at that register.

```
{
  "Exposes": [
    {
      "Name": "MCU Temp",
      "Type": "MCUTempSensor",
      "Class": "MCUTempSensor",
      "Bus": 5,
      "Address": "0x50",
      "Reg": 4
    }
  ],
  "Name": "MCU Board",
  "Type": "Board"
}
```

NVMe (nvmesensor)

The `nvmesensor` daemon (service `xyz.openbmc_project.NVMeSensor`, config type `NVME1000`) reads drive temperature over **NVMe-MI** using the Basic Management Command on the drive's SMBus/I2C interface. Drives usually sit behind an I2C mux, so the daemon derives the **root bus** from the mux device's `/mux_device` symlink and shares one context per root bus. Each basic-management query runs on a dedicated worker thread so a slow or unresponsive drive cannot stall the daemon's main event loop.

Threshold Configuration

Threshold Levels

Severity	Meaning
0	Warning
1	Critical

Direction

Direction	Meaning
<code>greater than</code>	Alarm when value > threshold
<code>less than</code>	Alarm when value < threshold

Example with All Thresholds

```
{
  "Thresholds": [
    {
      "Direction": "greater than",
      "Name": "upper critical",
      "Severity": 1,
      "Value": 90
    },
    {
      "Direction": "greater than",
      "Name": "upper warning",
      "Severity": 0,
      "Value": 80
    },
    {
      "Direction": "less than",
      "Name": "lower warning",
      "Severity": 0,
      "Value": 10
    },
    {
      "Direction": "less than",
      "Name": "lower critical",
      "Severity": 1,
      "Value": 5
    }
  ]
}
```

Reading Sensors

Via D-Bus

```
# List all sensors
busctl tree xyz.openbmc_project.ADCSensor
busctl tree xyz.openbmc_project.HwmonTempSensor

# Read sensor value
busctl get-property xyz.openbmc_project.HwmonTempSensor \
    /xyz/openbmc_project/sensors/temperature/CPU_Temp \
    xyz.openbmc_project.Sensor.Value Value

# Check threshold status
busctl get-property xyz.openbmc_project.HwmonTempSensor \
    /xyz/openbmc_project/sensors/temperature/CPU_Temp \
    xyz.openbmc_project.Sensor.Threshold.Critical CriticalAlarmHigh
```

Via Redfish

```
curl -k -u root:0penBmc \
    https://localhost/redfish/v1/Chassis/chassis/Sensors

curl -k -u root:0penBmc \
    https://localhost/redfish/v1/Chassis/chassis/Sensors/CPU_Temp
```

Troubleshooting

Sensor not appearing

```
# Check Entity Manager found the device
journalctl -u xyz.openbmc_project.EntityManager | grep -i <sensor-name>

# Check hwmon sysfs exists
ls /sys/class/hwmon/

# Check sensor daemon logs
journalctl -u xyz.openbmc_project.adcsensor -f
journalctl -u xyz.openbmc_project.hwmontempsensor -f
```

Incorrect readings

```
# Verify raw hwmon value
cat /sys/class/hwmon/hwmon*/temp1_input

# Check scale factor in Entity Manager config
# Value on D-Bus = raw_value * ScaleFactor

# For ADC, verify voltage divider calculation
```

Threshold alarms not triggering

```
# Check threshold configuration
busctl introspect xyz.openbmc_project.HwmonTempSensor \
    /xyz/openbmc_project/sensors/temperature/<name> | grep -i threshold

# Verify SEL logger is running
systemctl status xyz.openbmc_project.sel-logger
```

Deep Dive

Advanced implementation details for sensor developers.

Sensor Reading Pipeline

Each sensor daemon follows a common pattern for reading and publishing sensor values:

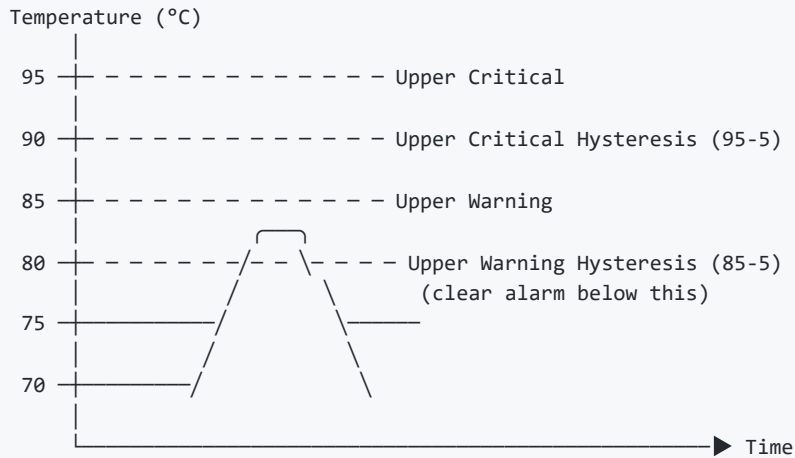


Source reference: [HwmonTempSensor.cpp](#)

Threshold Detection Algorithm

Thresholds use hysteresis to prevent rapid alarm toggling:

Threshold Hysteresis



Alarm Logic:

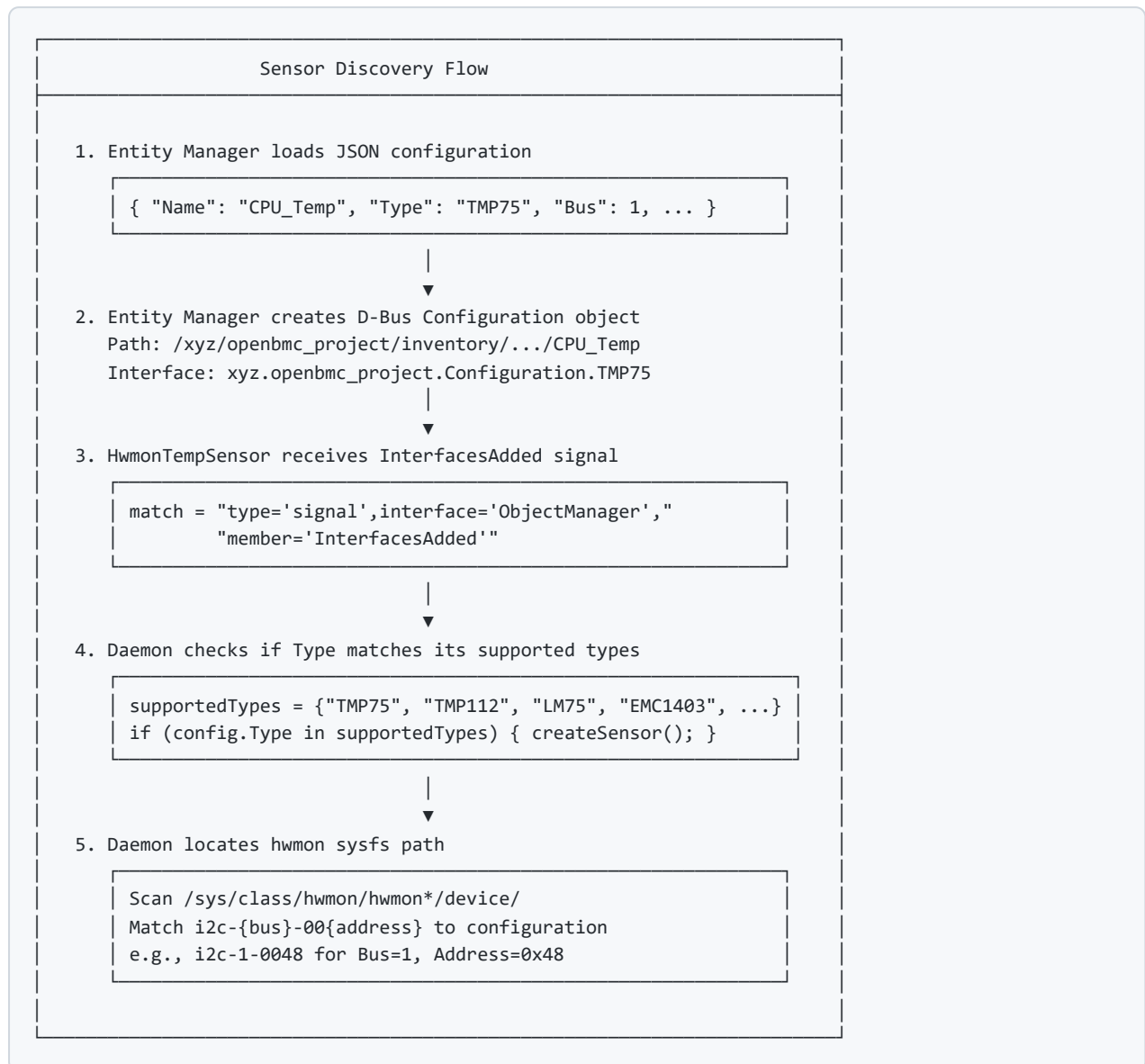
- └ SET alarm when: value $\geq$ threshold
- └ CLEAR alarm when: value $<$ (threshold - hysteresis)
- └ Default hysteresis: 1.0 (configurable per threshold)

Code pattern (Thresholds.cpp):

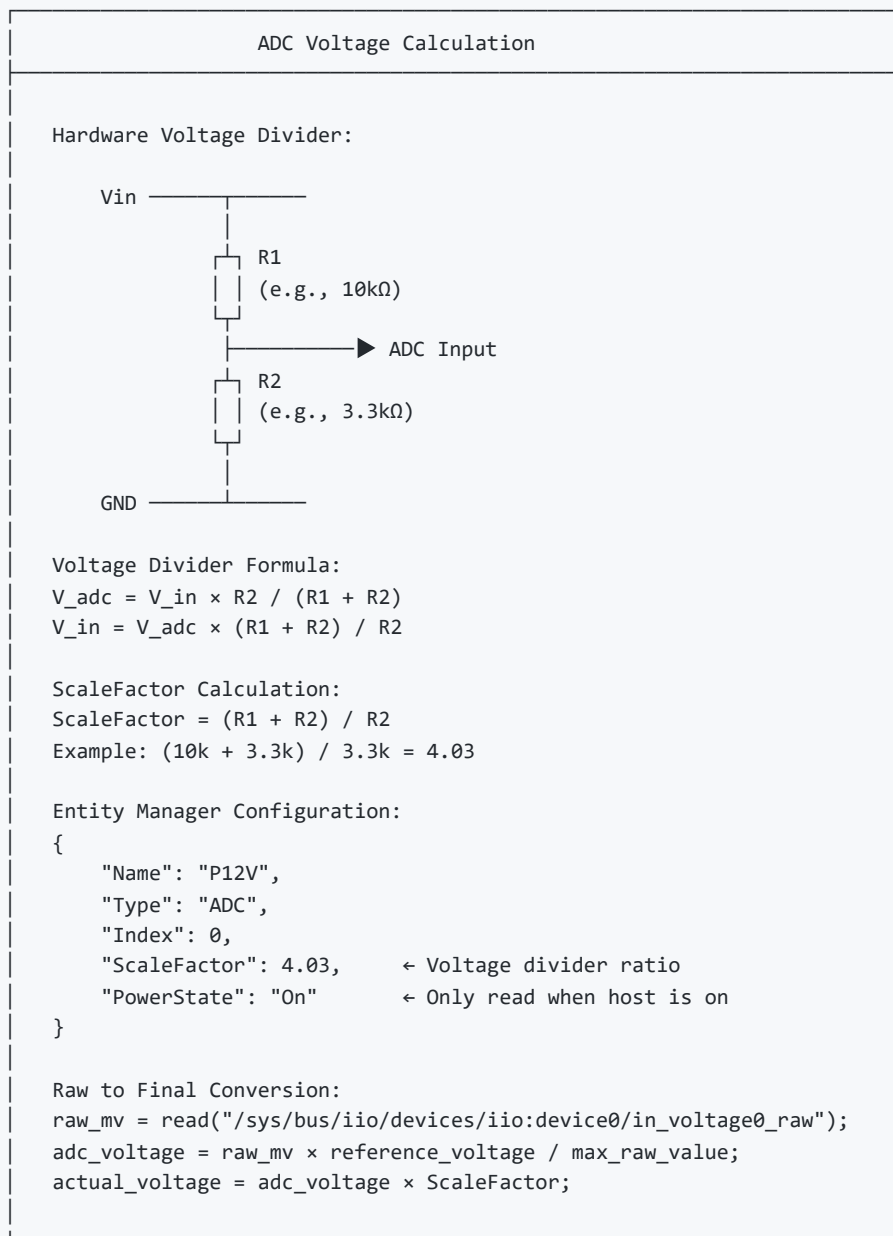
```
if (!alarm && value >= threshold) {
    alarm = true;
    log_event("threshold crossed");
} else if (alarm && value < (threshold - hysteresis)) {
    alarm = false;
    log_event("threshold cleared");
}
```

Source reference: [Thresholds.cpp](#)

Entity Manager Integration Flow



ADC Scaling and Voltage Dividers



Power State Filtering

Sensors can be configured to only read when the host is in a specific power state:

Power State Filtering

PowerState Options:

Value	Sensor reads when...
"Always"	Always (default, standby power sensors)
"On"	Host is powered on (CPU/memory sensors)
"BiosPost"	Host is in BIOS POST (initialization sensors)
"Chassis"	Chassis power is on (12V rail sensors)

Implementation:

```
// Subscribe to host state changes
match = "type='signal',path='/xyz/openbmc_project/state/host0'"

// In polling loop
if (powerState == "On" && hostState != Running) {
    value = NaN; // Mark sensor unavailable
    return;
}
value = read_sensor();
```

When sensor is unavailable (host off), D-Bus Value = NaN
This prevents false alarms and stale readings

Source Code Reference

Key implementation files in [dbus-sensors](#):

File	Description
src/HwmonTempSensor.cpp	I2C temperature sensor implementation
src/ADCSensor.cpp	ADC voltage sensor implementation
src/Thresholds.cpp	Threshold detection and hysteresis
src/Utils.cpp	Hwmon path discovery, scaling
src/SensorPaths.cpp	D-Bus path construction
include/sensor.hpp	Base sensor class interface

Examples

Working examples are available in the [examples/sensors](#) directory:

- [sensor_reader.cpp](#) - C++ sensor reading example
- [external-sensor/](#) - External sensor daemon example
- [virtual-sensor/](#) - Virtual sensor configuration

References

- [dbus-sensors](#)
 - [Entity Manager](#)
 - [Sensor Architecture](#)
-

Tested on: OpenBMC master, QEMU romulus

Hwmon Sensors Guide

Understanding Linux hwmon subsystem and phosphor-hwmon integration.

Table of Contents

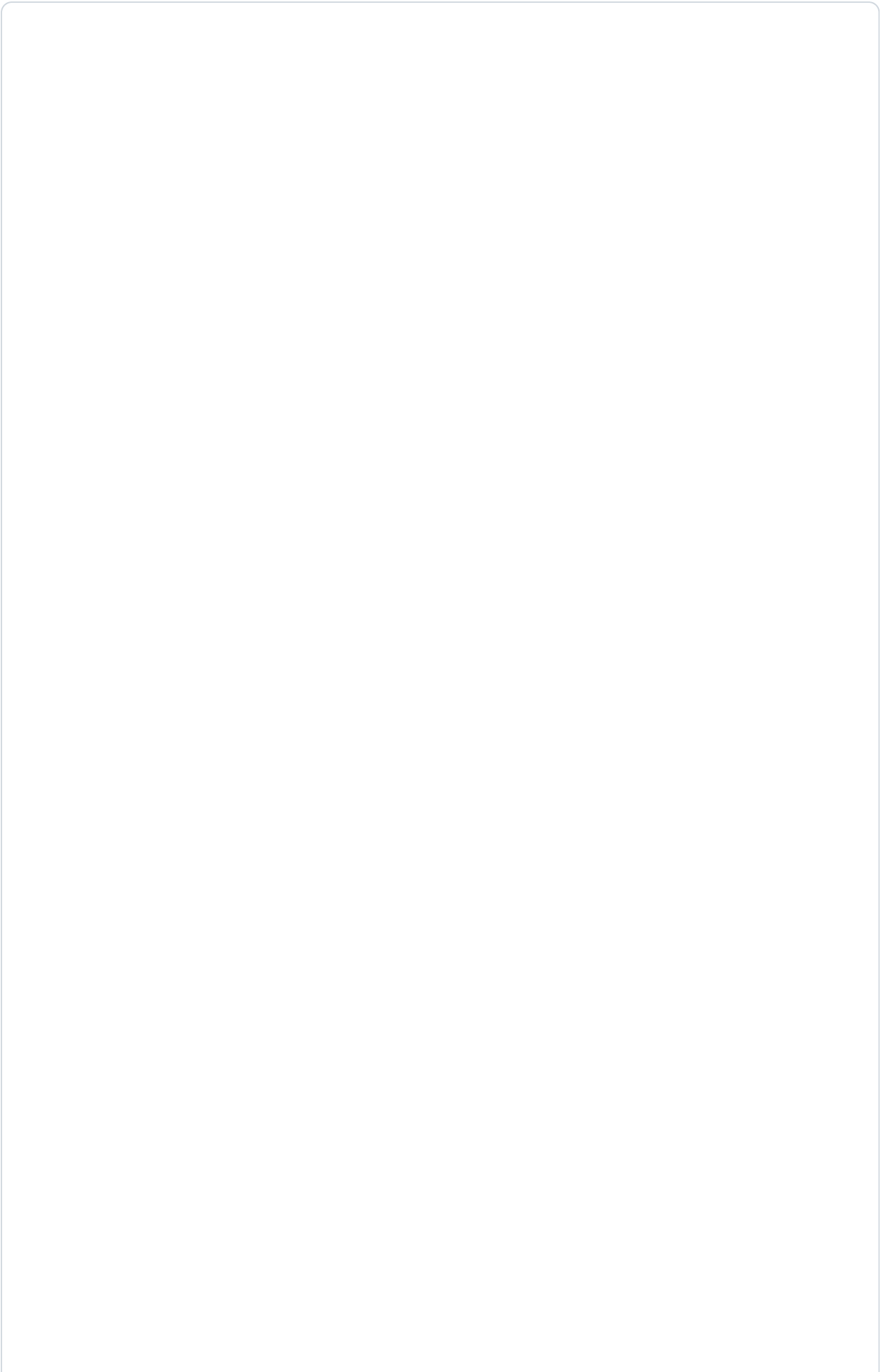
1. [Overview](#)
2. [Setup & Configuration](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [Meson Build Options \(dbus-sensors\)](#)
 3. [Meson Build Options \(phosphor-hwmon\)](#)
 4. [Runtime Enable/Disable](#)
 5. [Phosphor-Hwmon Configuration](#)
 6. [Phosphor-Hwmon Configuration Options](#)
 7. [Entity Manager Configuration \(dbus-sensors\)](#)
 8. [Entity Manager Sensor Properties](#)
 9. [Kernel Driver Configuration](#)
3. [Linux Hwmon Basics](#)
 1. [Sysfs Structure](#)
 2. [Sensor Types and Naming](#)
 3. [Common Attributes](#)
4. [Phosphor-Hwmon](#)
 1. [Overview](#)
 2. [Configuration](#)
 3. [Configuration Options](#)
 4. [File Naming](#)
 5. [Example Configuration](#)
 6. [systemd Service](#)
5. [Dbus-Sensors Hwmon](#)
 1. [HwmonTempSensor](#)
 2. [Entity Manager Configuration](#)
 3. [Supported Sensor Types](#)
 4. [FanSensor Daemon](#)
 5. [Fan Redundancy and Presence](#)
 1. [Redundancy](#)
 2. [Presence GPIO](#)
6. [Kernel Driver Configuration](#)
 1. [Device Tree](#)
 2. [Loading Drivers Manually](#)
7. [Reading Sensors](#)
 1. [Direct Sysfs Access](#)

- 2. [Via D-Bus](#)
 - 3. [Via Redfish](#)
 - 8. [Phosphor-Hwmon vs Dbus-Sensors](#)
 - 1. [Migration Path](#)
 - 9. [Troubleshooting](#)
 - 1. [Sensor Not Appearing](#)
 - 2. [Wrong Readings](#)
 - 3. [Threshold Alarms Not Working](#)
 - 4. [Device Not Binding](#)
 - 10. [Best Practices](#)
 - 11. [Examples](#)
 - 12. [References](#)
 - 13. [Deep Dive](#)
 - 1. [Hwmon Sysfs Attribute Lifecycle](#)
 - 2. [Hwmon Channel Configuration](#)
 - 3. [Phosphor-Hwmon Sysfs Matching Algorithm](#)
 - 4. [Dbus-Sensors HwmonTempSensor Processing Pipeline](#)
 - 5. [Threshold Hysteresis Implementation](#)
 - 6. [Source Code Reference](#)
-

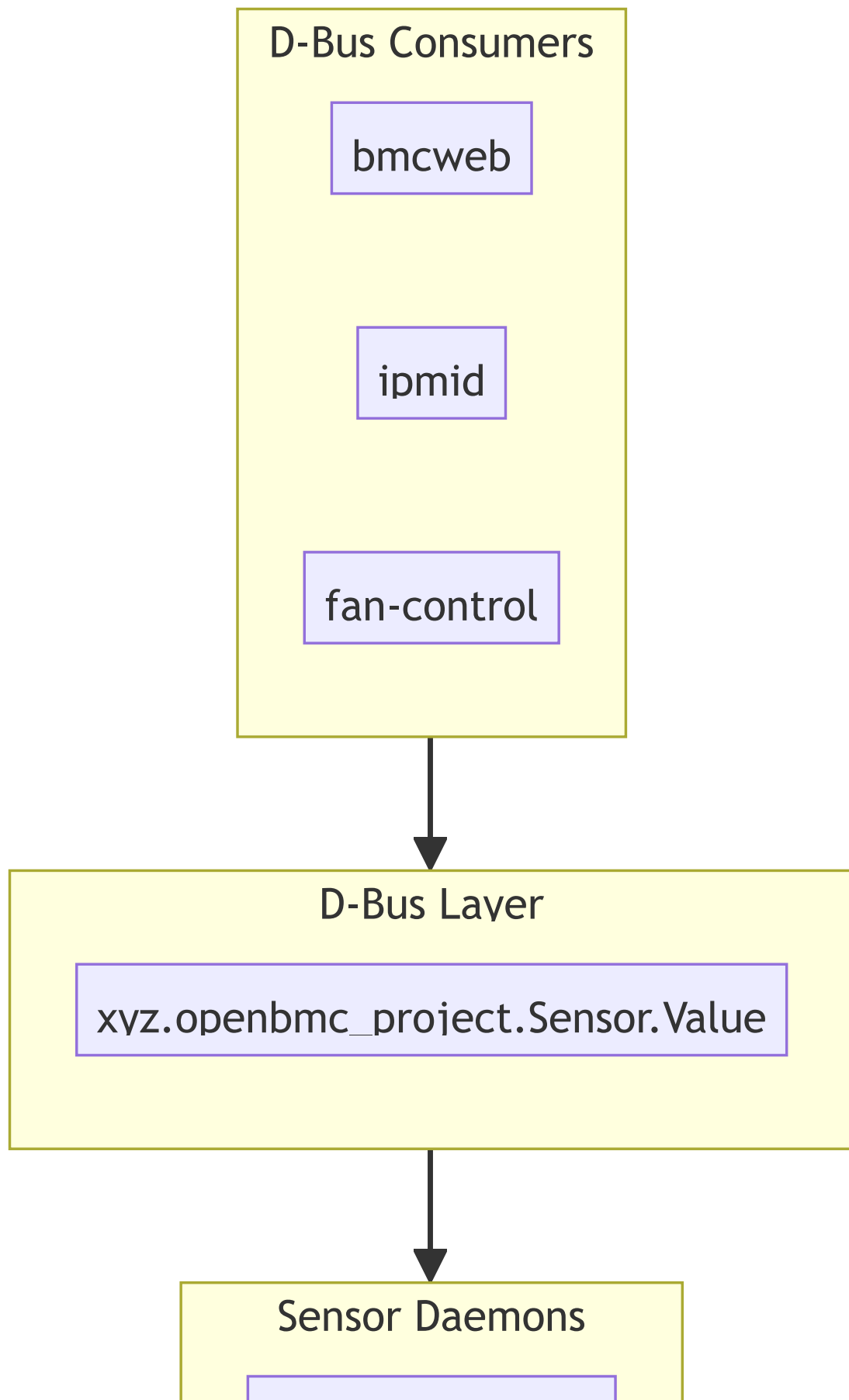
Overview

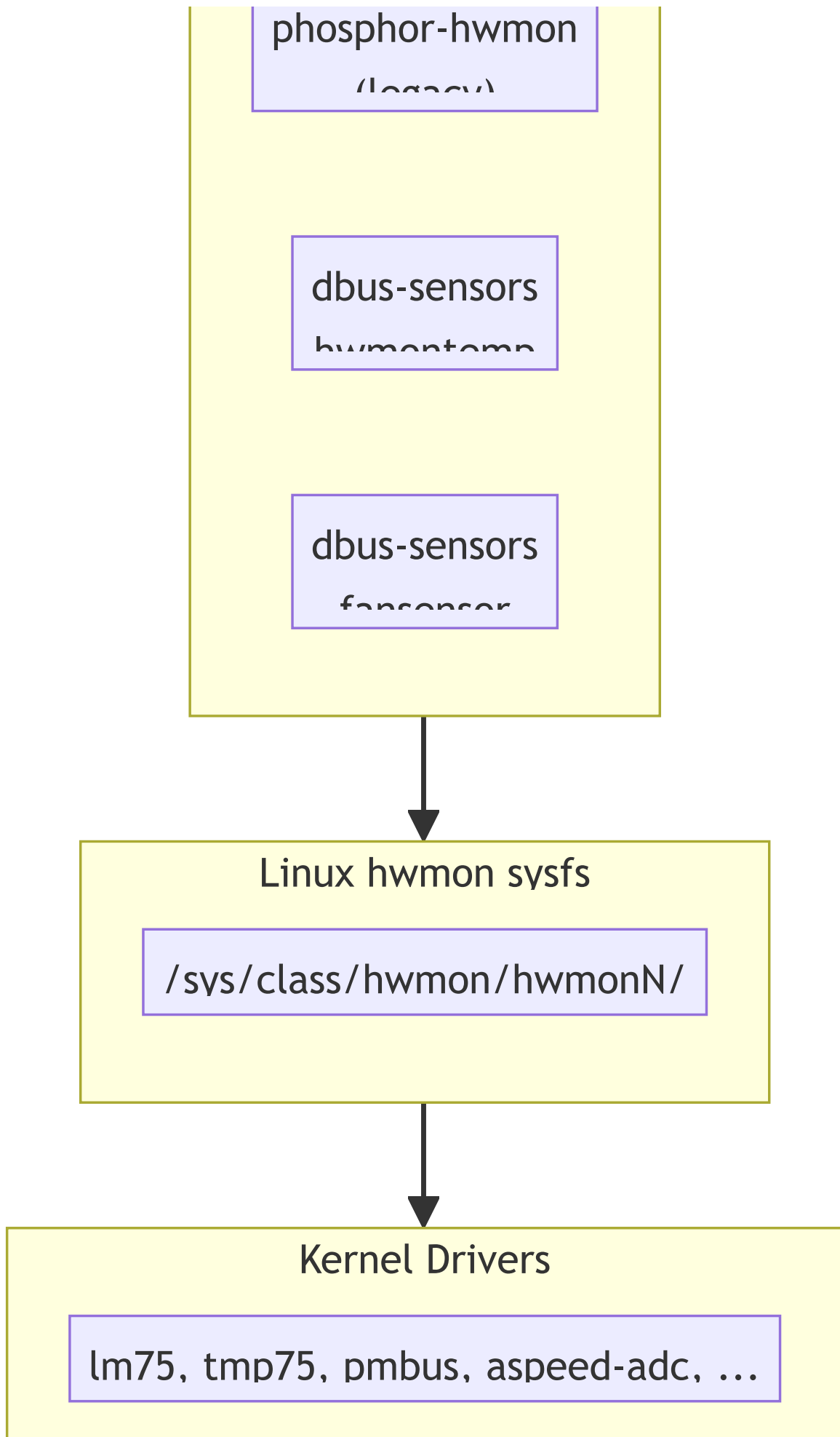
The Linux **hwmon** (Hardware Monitoring) subsystem provides a standardized interface for hardware monitoring devices. OpenBMC uses hwmon through two approaches:

1. **phosphor-hwmon** - Legacy daemon that reads hwmon sysfs and exposes D-Bus sensors
2. **dbus-sensors** - Modern approach with specialized sensor daemons



Hwmon Architecture





► ASCII-art version (for comparison)

Setup & Configuration

Build-Time Configuration (Yocto)

Choose between phosphor-hwmon (legacy) or dbus-sensors (modern):

```
# Modern approach: dbus-sensors (recommended for new platforms)
IMAGE_INSTALL:append = " dbus-sensors"

# Legacy approach: phosphor-hwmon
IMAGE_INSTALL:append = " phosphor-hwmon"

# Include specific dbus-sensors daemons
IMAGE_INSTALL:append = " \
    hwmontempensor \
    fansensor \
    psusensor \
"
```

Meson Build Options (dbus-sensors)

```
meson setup build \
    -Dhwmon-temp=enabled \
    -Dfan=enabled \
    -Dpsu=enabled
```

Meson Build Options (phosphor-hwmon)

```
meson setup build \
    -Dupdate-functional-on-fail=enabled
```

Runtime Enable/Disable

```
# For dbus-sensors hwmontempensor
systemctl status xyz.openbmc_project.hwmontempensor
systemctl stop xyz.openbmc_project.hwmontempensor
systemctl disable xyz.openbmc_project.hwmontempensor

# For phosphor-hwmon
systemctl status xyz.openbmc_project.Hwmon@<device>.service
systemctl stop "xyz.openbmc_project.Hwmon@ahb--apb--bus@1e78a000--i2c-bus@80--tmp75@48.service"
```

Phosphor-Hwmon Configuration

Configuration files are in `/etc/default/obmc/hwmon/` :

```
# Create configuration file
# Filename format: device-path-with-dashes.conf
cat > /etc/default/obmc/hwmon/ahb--apb--bus@1e78a000--i2c-bus@80--tmp75@48.conf << 'EOF'
# Sensor label (required)
LABEL_temp1=ambient_temp

# Thresholds (in raw units, typically millidegrees)
WARNHI_temp1=40000
WARNLO_temp1=5000
CRITHI_temp1=45000
CRITLO_temp1=0

# Scaling
GAIN_temp1=1.0
OFFSET_temp1=0

# Remove sensor when not present
REMOVEDCS_temp1=1
EOF

# Restart service
systemctl restart xyz.openbmc_project.Hwmon@ahb--apb--bus@1e78a000--i2c-bus@80--tmp75@48.service
```

Phosphor-Hwmon Configuration Options

Option	Description	Example
<code>LABEL_<attr></code>	D-Bus sensor name	<code>LABEL_temp1=cpu_temp</code>
<code>WARNHI_<attr></code>	Warning high (raw)	<code>WARNHI_temp1=85000</code>
<code>WARNLO_<attr></code>	Warning low (raw)	<code>WARNLO_temp1=0</code>
<code>CRITHI_<attr></code>	Critical high (raw)	<code>CRITHI_temp1=95000</code>
<code>CRITLO_<attr></code>	Critical low (raw)	<code>CRITLO_temp1=-5000</code>
<code>GAIN_<attr></code>	Multiplier	<code>GAIN_in1=1.5</code>
<code>OFFSET_<attr></code>	Offset to add	<code>OFFSET_temp1=1000</code>
<code>REMOVEDCS_<attr></code>	Remove when absent	<code>REMOVEDCS_temp1=1</code>
<code>MODE_<attr></code>	Read mode	<code>MODE_temp1=label</code>

Entity Manager Configuration (dbus-sensors)

Configure via JSON in `/usr/share/entity-manager/configurations/`:


```
{
  "Exposes": [
    {
      "Name": "CPU Temp",
      "Type": "TMP75",
      "Bus": 1,
      "Address": "0x48",
      "PowerState": "On",
      "PollRate": 1.0,
      "Thresholds": [
        {"Direction": "greater than", "Name": "upper critical", "Severity": 1, "Value": 95},
        {"Direction": "greater than", "Name": "upper warning", "Severity": 0, "Value": 85}
      ]
    }
  ],
  "Name": "TempBoard",
  "Probe": "TRUE",
  "Type": "Board"
}
```

Entity Manager Sensor Properties

Property	Type	Default	Description
<code>Name</code>	string	required	Sensor name on D-Bus
<code>Type</code>	string	required	Sensor type (TMP75, LM75A, etc.)
<code>Bus</code>	int	required	I2C bus number
<code>Address</code>	string	required	I2C address (hex)
<code>PowerState</code>	string	<code>Always</code>	<code>On</code> , <code>BiosPost</code> , <code>Always</code>
<code>PollRate</code>	float	1.0	Seconds between readings
<code>Offset</code>	float	0.0	Value offset
<code>ScaleFactor</code>	float	1.0	Value multiplier
<code>MaxValue</code>	float	-	Maximum expected value
<code>MinValue</code>	float	-	Minimum expected value
<code>Hysteresis</code>	float	-	Threshold hysteresis

Kernel Driver Configuration

Load I2C sensor drivers:

```
# Check if driver is loaded
lsmod | grep tmp75

# Manually instantiate device
echo tmp75 0x48 > /sys/bus/i2c/devices/i2c-1/new_device

# Verify hwmon created
ls /sys/bus/i2c/devices/1-0048/hwmon/

# Remove device
echo 0x48 > /sys/bus/i2c/devices/i2c-1/delete_device
```

Device tree configuration:

```
&i2c1 {
    status = "okay";

    tmp75@48 {
        compatible = "ti,tmp75";
        reg = <0x48>;
    };
};
```

Linux Hwmon Basics

Sysfs Structure

Hwmon devices appear under `/sys/class/hwmon/`:

```
# List hwmon devices
ls /sys/class/hwmon/
# hwmon0 hwmon1 hwmon2 ...

# Check device name
cat /sys/class/hwmon/hwmon0/name
# tmp75

# List available sensors
ls /sys/class/hwmon/hwmon0/
# name temp1_input temp1_max temp1_max_hyst ...
```

Sensor Types and Naming

Prefix	Type	Unit	Scale
temp	Temperature	millidegrees C	/1000
in	Voltage	millivolts	/1000
curr	Current	milliamps	/1000
power	Power	microwatts	/1000000
fan	Fan speed	RPM	1
pwm	PWM duty	0-255	/255

Common Attributes

```
# Temperature sensor attributes
temp1_input      # Current reading (millidegrees)
temp1_max        # Maximum threshold
temp1_max_hyst   # Hysteresis for max
temp1_min        # Minimum threshold
temp1_crit       # Critical threshold
temp1_label      # Sensor label

# Voltage sensor attributes
in1_input        # Current reading (millivolts)
in1_max          # Maximum threshold
in1_min          # Minimum threshold
in1_label        # Sensor label

# Fan sensor attributes
fan1_input       # Current RPM
fan1_min         # Minimum RPM threshold
fan1_target      # Target RPM (for control)
pwm1             # PWM output (0-255)
pwm1_enable      # Control mode (0=off, 1>manual, 2=auto)
```

Phosphor-Hwmon

Overview

phosphor-hwmon is the legacy sensor daemon that:

- Reads hwmon sysfs periodically
- Exposes sensors on D-Bus
- Configured via YAML files
- Being replaced by dbus-sensors for new platforms

Configuration

Configuration files in `/etc/default/obmc/hwmon/`:

```
# /etc/default/obmc/hwmon/ahb--apb--bus@1e78a000--i2c-bus@80--tmp75@48.conf
LABEL_temp1=ambient_temp
WARNHI_temp1=40000
WARNLO_temp1=5000
CRITHI_temp1=45000
CRITLO_temp1=0
```

Configuration Options

Option	Description
<code>LABEL_<attr></code>	D-Bus sensor name
<code>WARNHI_<attr></code>	Warning high threshold (raw units)
<code>WARNLO_<attr></code>	Warning low threshold
<code>CRITHI_<attr></code>	Critical high threshold
<code>CRITLO_<attr></code>	Critical low threshold
<code>GAIN_<attr></code>	Multiplier for reading
<code>OFFSET_<attr></code>	Offset to add to reading
<code>REMOVERCS_<attr></code>	Remove sensor when not present

File Naming

Config file names match the device tree path with special characters replaced:

```
Device tree path: /ahb/apb/bus@1e78a000/i2c-bus@80/tmp75@48
Config file:      ahb--apb--bus@1e78a000--i2c-bus@80--tmp75@48.conf
```

Example Configuration

```
# Temperature sensor with thresholds
LABEL_temp1=cpu_temp
WARNHI_temp1=85000
CRITHI_temp1=95000
WARNLO_temp1=0
CRITLO_temp1=-5000

# Voltage sensor with scaling
LABEL_in1=p12v_aux
GAIN_in1=1.5
WARNHI_in1=13200
WARNLO_in1=10800
CRITHI_in1=13800
CRITLO_in1=10200
```

systemd Service

```
# /lib/systemd/system/xyz.openbmc_project.Hwmon@.service
[Unit]
Description=Hwmon %I

[Service]
ExecStart=/usr/bin/phosphor-hwmon-readd \
    --path=/sys/class/hwmon/%i \
    --instance=%i
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Dbus-Sensors Hwmon

HwmonTempSensor

The modern approach using dbus-sensors with Entity Manager.

Service: `xyz.openbmc_project.HwmonTempSensor`

Entity Manager Configuration

```
{
  "Exposes": [
    {
      "Address": "0x48",
      "Bus": 1,
      "Name": "Ambient Temp",
      "Type": "TMP75",
      "Thresholds": [
        {
          "Direction": "greater than",
          "Name": "upper critical",
          "Severity": 1,
          "Value": 45
        },
        {
          "Direction": "greater than",
          "Name": "upper warning",
          "Severity": 0,
          "Value": 40
        }
      ]
    }
  ],
  "Name": "TempBoard",
  "Probe": "TRUE",
  "Type": "Board"
}
```

Supported Sensor Types

Type	Kernel Driver	Description
<code>TMP75</code>	tmp75	TI TMP75 temperature sensor
<code>TMP421</code>	tmp421	TI TMP421 remote diode sensor
<code>TMP112</code>	tmp102	TI TMP112 low-power sensor
<code>LM75A</code>	lm75	NXP LM75A temperature sensor
<code>EMC1413</code>	emc1403	Microchip multi-channel
<code>MAX31725</code>	max31725	Maxim precision sensor
<code>NCT7802</code>	nct7802	Nuvoton multi-function
<code>W83773G</code>	w83773g	Nuvoton triple temp

FanSensor Daemon

For fan tachometer and PWM monitoring.

Service: `xyz.openbmc_project.FanSensor`

```
{
  "Exposes": [
    {
      "Connector": {
        "Name": "Fan0",
        "Pwm": 0,
        "Tachs": [0]
      },
      "Name": "Fan0",
      "Type": "AspeedFan"
    }
  ],
  "Name": "FanBoard",
  "Probe": "TRUE",
  "Type": "Board"
}
```

Fan Redundancy and Presence

The `fansensor` daemon does more than publish tach RPM — it can also track **fan redundancy** (are enough fans healthy?) and **fan presence** (is the fan physically installed?).

Redundancy

An `xyz.openbmc_project.Configuration.FanRedundancy` entry declares how many fans may fail before the group is no longer redundant. The daemon exposes an `xyz.openbmc_project.Control.FanRedundancy` interface at `/xyz/openbmc_project/control/FanRedundancy/Tach`, whose `Status` property reports one of three states:

Status	Meaning
Full	Every fan in the collection is healthy
Degraded	Some fans failed, but still within AllowedFailures
Failed	More fans failed than AllowedFailures permits

Crossing out of and back into Full logs the Redfish messages OpenBMC.0.1.FanRedundancyLost and OpenBMC.0.1.FanRedundancyRegained.

```
busctl get-property xyz.openbmc_project.FanSensor \
  /xyz/openbmc_project/control/FanRedundancy/Tach \
  xyz.openbmc_project.Control.FanRedundancy Status
# s "Full"
```

Presence GPIO

A fan’s config can carry a Presence sub-object so the daemon knows whether the fan is installed. The MonitorType field selects one of two strategies:

MonitorType	Class	Behavior
Event (default)	EventPresenceGpio	Reacts to GPIO edge events (interrupt-driven)
Polling	PollingPresenceGpio	Samples the GPIO line on a timer

```
{
  "Connector": {
    "Name": "Fan0",
    "Pwm": 0,
    "Tachs": [0]
  },
  "Presence": {
    "MonitorType": "Event",
    "PinName": "FAN0_PRESENCE",
    "Polarity": "Low"
  },
  "Name": "Fan0",
  "Type": "AspeedFan"
}
```

Polarity is "Low" when the line reads low while the fan is present (inverted), or "High" otherwise. When a fan is reported absent, its tach sensor is treated as not-present rather than as a failed fan, which keeps a deliberately empty fan slot from tripping the redundancy logic.

Kernel Driver Configuration

Device Tree

```
// I2C temperature sensor
&i2c1 {
    status = "okay";

    tmp75@48 {
        compatible = "ti,tmp75";
        reg = <0x48>;
    };

    tmp75@49 {
        compatible = "ti,tmp75";
        reg = <0x49>;
    };
};

// Aspeed ADC
&adc {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_adc0_default
                &pinctrl_adc1_default
                &pinctrl_adc2_default>;
};

// Aspeed PWM/Fan tach
&pwm_tacho {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_pwm0_default
                &pinctrl_tach0_default>;

    fan@0 {
        reg = <0x00>;
        aspeed,fan-tach-ch = /bits/ 8 <0x00>;
    };
};
```

Loading Drivers Manually

```
# Load I2C device
echo tmp75 0x48 > /sys/bus/i2c/devices/i2c-1/new_device

# Check if hwmon appeared
ls /sys/bus/i2c/devices/1-0048/hwmon/

# Remove device
echo 0x48 > /sys/bus/i2c/devices/i2c-1/delete_device
```


Reading Sensors

Direct Sysfs Access

```
# Find hwmon device
find /sys/class/hwmon -name "name" -exec sh -c 'echo "$1: $(cat $1)"' _ {} \;

# Read temperature (millidegrees to degrees)
temp=$(cat /sys/class/hwmon/hwmon0/temp1_input)
echo "Temperature: $((temp / 1000)).$((temp % 1000)) C"

# Read all temperature sensors
for f in /sys/class/hwmon/hwmon*/temp*_input; do
    name=$(dirname $f)/name
    label=$(echo $f | sed 's/_input/_label/')
    val=$(cat $f 2>/dev/null)
    if [ -n "$val" ]; then
        echo "$(cat $name 2>/dev/null) $(cat $label 2>/dev/null): $((val/1000))C"
    fi
done
```

Via D-Bus

```
# List temperature sensors
busctl tree xyz.openbmc_project.HwmonTempSensor

# Read specific sensor
busctl get-property xyz.openbmc_project.HwmonTempSensor \
    /xyz/openbmc_project/sensors/temperature/Ambient_Temp \
    xyz.openbmc_project.Sensor.Value Value

# List all sensor values
busctl call xyz.openbmc_project.ObjectMapper \
    /xyz/openbmc_project/object_mapper \
    xyz.openbmc_project.ObjectMapper \
    GetSubTree sias "/xyz/openbmc_project/sensors" 0 1 \
    "xyz.openbmc_project.Sensor.Value"
```

Via Redfish

```
# List all sensors
curl -k -u root:0penBmc \
    https://localhost/redfish/v1/Chassis/chassis/Sensors

# Get specific sensor
curl -k -u root:0penBmc \
    https://localhost/redfish/v1/Chassis/chassis/Sensors/Ambient_Temp
```

Phosphor-Hwmon vs Dbus-Sensors

Feature	phosphor-hwmon	dbus-sensors
Configuration	YAML files	Entity Manager JSON
Auto-discovery	No	Yes (probes)
Hot-plug support	Limited	Yes
Sensor types	Generic	Type-specific daemons
Threshold config	YAML	JSON with severity
Recommended for	Legacy systems	New platforms

Migration Path

1. Identify existing phosphor-hwmon configs
2. Create Entity Manager JSON equivalents
3. Verify sensor names match
4. Test threshold behavior
5. Remove old YAML configs
6. Update machine layer

Troubleshooting

Sensor Not Appearing

```
# Check kernel driver loaded
dmesg | grep -i tmp75
lsmod | grep tmp75

# Verify I2C device exists
i2cdetect -y 1

# Check hwmon created
ls /sys/bus/i2c/devices/1-0048/hwmon/

# Check daemon logs
journalctl -u xyz.openbmc_project.hwmontempsensor -f
```

Wrong Readings

```
# Compare raw hwmon value
cat /sys/class/hwmon/hwmon0/temp1_input
# 25500 (25.5°C in millidegrees)

# Compare D-Bus value
busctl get-property xyz.openbmc_project.HwmonTempSensor \
    /xyz/openbmc_project/sensors/temperature/Ambient_Temp \
    xyz.openbmc_project.Sensor.Value Value
# d 25.5

# Check for scaling issues in Entity Manager
# ScaleFactor, Offset properties
```

Threshold Alarms Not Working

```
# Check threshold values on D-Bus
busctl introspect xyz.openbmc_project.HwmonTempSensor \
    /xyz/openbmc_project/sensors/temperature/Ambient_Temp | grep -i threshold

# Verify sel-logger is running
systemctl status xyz.openbmc_project.sel-logger

# Check for threshold events
journalctl -u xyz.openbmc_project.sel-logger | grep -i threshold
```

Device Not Binding

```
# Check device tree
cat /sys/firmware/devicetree/base/ahb/apb/bus@1e78a000/i2c-bus@80/tmp75@48/compatible

# Try manual bind
echo "1-0048" > /sys/bus/i2c/drivers/tmp75/bind

# Check kernel errors
dmesg | tail -20
```

Best Practices

1. **Use dbus-sensors** for new platforms
 2. **Verify device tree** bindings match actual hardware
 3. **Test thresholds** to ensure alarms trigger correctly
 4. **Use meaningful names** that identify physical location
 5. **Document scaling factors** for voltage dividers
 6. **Monitor daemon logs** during development
-

Examples

Working examples are available in the [examples/sensors](#) directory:

- `sensor_reader.cpp` - Read sensor values from D-Bus
 - `virtual-sensor/virtual_sensor.cpp` - Create a virtual sensor on D-Bus
 - `Dockerfile` / `build.sh` / `run.sh` - Docker build for standalone testing
-

References

- [phosphor-hwmon](#)
 - [dbus-sensors](#)
 - [Linux hwmon Documentation](#)
 - [Entity Manager](#)
-

Deep Dive

Advanced implementation details for hwmon sensor developers.

Hwmon Sysfs Attribute Lifecycle



```
i2c_smbus_read_word_data() → returns millidegrees
```

Hwmon Channel Configuration

```
/* Driver hwmon_chip_info structure example (simplified tmp75) */

static const u32 tmp75_temp_config[] = {
    HWMON_T_INPUT | HWMON_T_MAX | HWMON_T_MIN | HWMON_T_MAX_HYST,
    0 /* Terminator */
};

static const struct hwmon_channel_info tmp75_temp = {
    .type = hwmon_temp,
    .config = tmp75_temp_config,
};

static const struct hwmon_channel_info *tmp75_info[] = {
    &tmp75_temp,
    NULL /* Terminator */
};

static const struct hwmon_ops tmp75_hwmon_ops = {
    .is_visible = tmp75_is_visible, /* Which attrs are visible */
    .read = tmp75_read,             /* Read attribute value */
    .write = tmp75_write,           /* Write attribute value */
};

static const struct hwmon_chip_info tmp75_chip_info = {
    .ops = &tmp75_hwmon_ops,
    .info = tmp75_info,
};

/* Attribute flag meanings:
 * HWMON_T_INPUT      -> temp1_input      (current reading)
 * HWMON_T_MAX        -> temp1_max        (high threshold)
 * HWMON_T_MIN        -> temp1_min        (low threshold)
 * HWMON_T_MAX_HYST   -> temp1_max_hyst    (hysteresis)
 * HWMON_T_CRIT       -> temp1_crit       (critical threshold)
 * HWMON_T_LABEL      -> temp1_label       (sensor name)
 */
```

Phosphor-Hwmon Sysfs Matching Algorithm

Phosphor-Hwmon Device Discovery

1. SYSTEMD SERVICE INSTANTIATION

xyz.openbmc_project.Hwmon@ahb--apb--bus@1e78a000--i2c-bus@80--tmp75@48

Instance name encodes device tree path:

/ → --

@ → @

2. DEVICE PATH RESOLUTION

Instance: ahb--apb--bus@1e78a000--i2c-bus@80--tmp75@48

|

v

/sys/firmware/devicetree/base/ahb/apb/bus@1e78a000/i2c-bus@80/tmp75@48

|

v

Find symlink in /sys/bus/i2c/devices/

→ 1-0048 (bus 1, address 0x48)

|

v

/sys/bus/i2c/devices/1-0048/hwmon/hwmon3/

3. CONFIGURATION FILE LOOKUP

Search order:

1. /etc/default/obmc/hwmon/<instance>.conf
2. /usr/share/phosphor-hwmon/<instance>.conf

Config content:

LABEL_temp1=ambient_temp

WARNHI_temp1=40000

CRITHI_temp1=45000

4. D-BUS OBJECT CREATION

Path: /xyz/openbmc_project/sensors/temperature/ambient_temp

Interfaces:

- xyz.openbmc_project.Sensor.Value
- xyz.openbmc_project.Sensor.Threshold.Warning
- xyz.openbmc_project.Sensor.Threshold.Critical
- xyz.openbmc_project.State.Decorator.OperationalStatus

Dbus-Sensors HwmonTempSensor Processing Pipeline



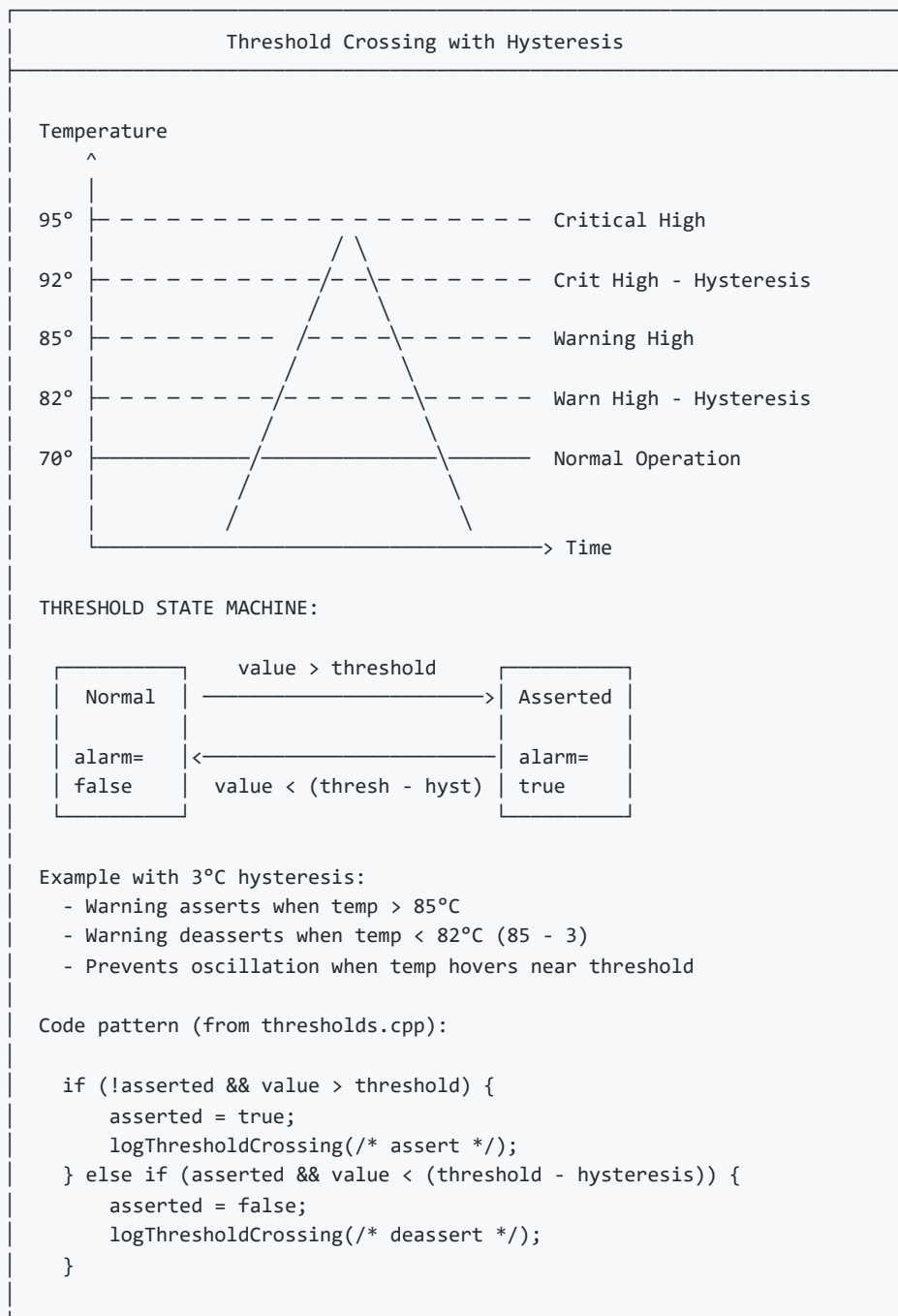
handleResponse() - Process reading

1. lseek(fd, 0, SEEK_SET) // Reset to start
2. read(fd, buffer, size) // Read raw value
3. value = atof(buffer) / 1000.0 // Convert millidegrees
4. updateValue(value) // Update D-Bus + thresholds
5. Schedule next read

updateValue() triggers:

- D-Bus property change signal
- Threshold crossing check
- SEL entry generation if threshold crossed

Threshold Hysteresis Implementation



Source Code Reference

Key implementation files in [dbus-sensors](#) and [phosphor-hwmon](#):

File	Description
<code>dbus-sensors/src/HwmonTempSensor.cpp</code>	Temperature sensor implementation with sysfs polling
<code>dbus-sensors/src/Thresholds.cpp</code>	Threshold crossing detection with hysteresis
<code>dbus-sensors/src/Utils.cpp</code>	Hwmon path discovery and Entity Manager query
<code>phosphor-hwmon/mainloop.cpp</code>	Main sensor reading loop for legacy hwmon
<code>phosphor-hwmon/sysfs.cpp</code>	Sysfs attribute reading and parsing
<code>phosphor-hwmon/env.cpp</code>	Configuration file parsing (LABEL_, WARNHI_, etc.)

Tested on: OpenBMC master, QEMU romulus

Entity Manager Guide

Configure hardware through JSON-based entity definitions and probes.

Table of Contents

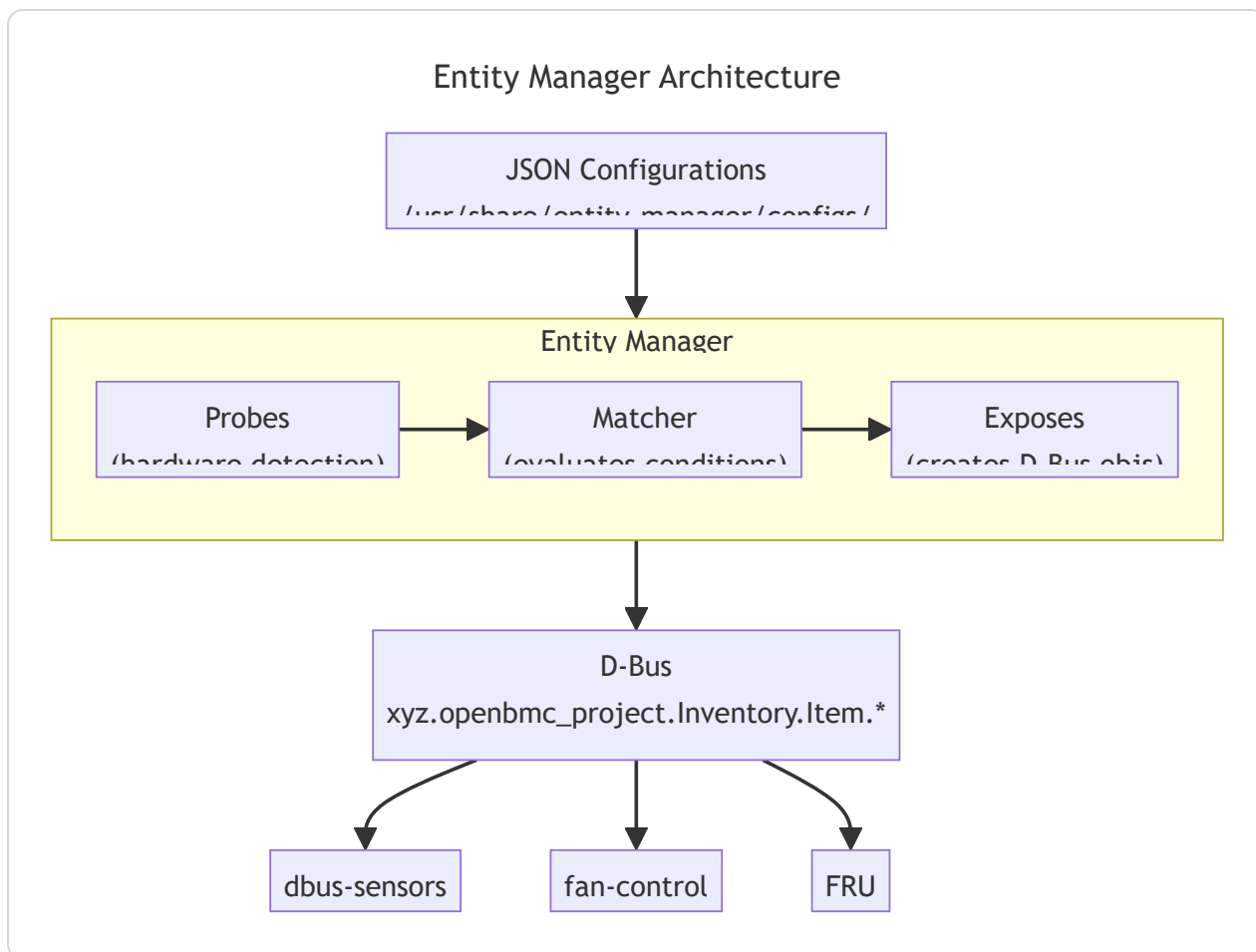
1. [Overview](#)
2. [Setup & Configuration](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [Meson Build Options](#)
 3. [Runtime Enable/Disable](#)
 4. [Configuration File Locations](#)
 5. [Adding New Configurations](#)
 6. [Configuration Priority](#)
 7. [Hot-Reload Configurations](#)
 8. [Disable Specific Configurations](#)
 9. [D-Bus Configuration Interface](#)
3. [JSON Configuration Structure](#)
 1. [Basic Structure](#)
 2. [Key Fields](#)
4. [Probe System](#)
 1. [Simple Probes](#)
 2. [FRU Probes](#)
 3. [I2C Device Probes](#)
 4. [GPIO Probes](#)
 5. [Compound Probes](#)
 6. [Record-Based Probes](#)
5. [Exposes Section](#)
 1. [Sensor Exposures](#)
 2. [Fan Exposures](#)
 3. [LED Exposures](#)
 4. [PSU Exposures](#)
6. [FRU Configuration](#)
 1. [EEPROM Configuration](#)
 2. [FRU Device Definition](#)
7. [D-Bus Interface Implementation](#)
8. [Configuration Files Location](#)
9. [Complete Example](#)
 1. [Board with Sensors, Fans, and FRU](#)
10. [D-Bus Interface](#)

1. [Watching for Changes](#)
 11. [Testing Configurations](#)
 1. [Validate JSON Syntax](#)
 2. [Test on Target](#)
 3. [Debug Probe Evaluation](#)
 12. [Troubleshooting](#)
 1. [Configuration Not Loading](#)
 2. [Sensors Not Appearing](#)
 3. [Probe Not Matching](#)
 13. [Best Practices](#)
 14. [Deep Dive](#)
 1. [Probe Evaluation Algorithm](#)
 2. [Variable Substitution System](#)
 3. [Configuration Loading Sequence](#)
 4. [D-Bus Object Creation](#)
 5. [Hot-Reload Mechanism](#)
 6. [Probe Expression Parser](#)
 7. [Source Code Reference](#)
 15. [Examples](#)
 16. [Next Steps](#)
 17. [References](#)
-

Overview

Entity Manager is the configuration backbone of OpenBMC. It:

- Discovers hardware through **probes**
- Exposes hardware entities on **D-Bus**
- Configures sensors, fans, LEDs, and other components via **JSON**
- Replaces hardcoded YAML configurations with dynamic discovery



► ASCII-art version (for comparison)

Setup & Configuration

Build-Time Configuration (Yocto)

Include Entity Manager in your image:

```
# In your machine .conf or image recipe
IMAGE_INSTALL:append = " entity-manager"

# Include FRU device support
IMAGE_INSTALL:append = " fru-device"
```

Meson Build Options

```
meson setup build \
  -Dfru-device=enabled \
  -Dconfiguration-directory=/usr/share/entity-manager/configurations
```

Option	Default	Description
<code>fru-device</code>	enabled	FRU EEPROM detection
<code>configuration-directory</code>	/usr/share/...	JSON config location

Runtime Enable/Disable

```
# Check Entity Manager status
systemctl status xyz.openbmc_project.EntityManager

# Restart to reload configurations
systemctl restart xyz.openbmc_project.EntityManager

# Disable Entity Manager (not recommended)
systemctl stop xyz.openbmc_project.EntityManager
systemctl disable xyz.openbmc_project.EntityManager

# Check FRU device service
systemctl status xyz.openbmc_project.FruDevice
```

Configuration File Locations

```
# Primary configuration directory
/usr/share/entity-manager/configurations/

# Machine-specific overrides (higher priority)
/etc/entity-manager/configurations/

# Runtime configuration directory
/var/lib/entity-manager/
```

Adding New Configurations

```
# Create a new configuration file
cat > /usr/share/entity-manager/configurations/my-board.json << 'EOF'
{
  "Name": "MyBoard",
  "Type": "Board",
  "Probe": "TRUE",
  "Exposes": [...]
}
EOF

# Reload Entity Manager
systemctl restart xyz.openbmc_project.EntityManager

# Verify configuration loaded
busctl tree xyz.openbmc_project.EntityManager | grep MyBoard
```

Configuration Priority

When multiple configurations match:

1. `/etc/entity-manager/configurations/` (highest priority)
2. `/usr/share/entity-manager/configurations/`

3. Built-in defaults (lowest priority)

Hot-Reload Configurations

Entity Manager monitors for file changes:

```
# Modify a configuration file
vim /usr/share/entity-manager/configurations/my-board.json

# Entity Manager automatically reloads (no restart needed)
# Or force reload
busctl call xyz.openbmc_project.EntityManager \
    /xyz/openbmc_project/EntityManager \
    xyz.openbmc_project.EntityManager Reload
```

Disable Specific Configurations

To disable a configuration without deleting it:

```
{
  "Name": "DisabledBoard",
  "Type": "Board",
  "Probe": "FALSE",
  "Exposes": [...]
}
```

Or move to a disabled directory:

```
mkdir -p /usr/share/entity-manager/configurations/disabled
mv my-board.json configurations/disabled/
systemctl restart xyz.openbmc_project.EntityManager
```

D-Bus Configuration Interface

Query and manipulate configurations via D-Bus:

```
# List all configurations
busctl tree xyz.openbmc_project.EntityManager

# Get configuration properties
busctl introspect xyz.openbmc_project.EntityManager \
    /xyz/openbmc_project/inventory/system/board/MyBoard

# Check if entity is present
busctl get-property xyz.openbmc_project.EntityManager \
    /xyz/openbmc_project/inventory/system/board/MyBoard \
    xyz.openbmc_project.Inventory.Item Present
```


JSON Configuration Structure

Source Reference: [entity-manager](#)

- Configuration examples: [configurations/](#)
- Schema documentation: [docs/](#)

Basic Structure

```
{
  "Name": "MyBoard",
  "Type": "Board",
  "Probe": "TRUE",
  "Exposes": [
    {
      "Name": "Sensor1",
      "Type": "SensorType",
      ...
    }
  ]
}
```

Key Fields

Field	Required	Description
Name	Yes	Unique identifier for this entity
Type	Yes	Entity type (Board, Chassis, CPU, etc.)
Probe	Yes	Condition for when to load this config
Exposes	No	Array of components this entity exposes
xyz.openbmc_project.*	No	Additional D-Bus interfaces to implement

Probe System

Probes determine when a configuration should be applied.

Simple Probes

```
// Always apply
"Probe": "TRUE"

// Never apply (disabled)
"Probe": "FALSE"
```

FRU Probes

Detect hardware by reading FRU EEPROM data:

```
// Match specific board by FRU fields
"Probe": "xyz.openbmc_project.FruDevice({'PRODUCT_PRODUCT_NAME': 'MyBoard'})"

// Match by manufacturer
"Probe": "xyz.openbmc_project.FruDevice({'BOARD_MANUFACTURER': 'ACME'})"

// Match by part number
"Probe": "xyz.openbmc_project.FruDevice({'PRODUCT_PART_NUMBER': 'ABC-123'})"
```

I2C Device Probes

Detect I2C devices:

```
// Check if device responds at address
"Probe": "i2c({'bus': 1, 'address': 72})"

// With multiple addresses
"Probe": "i2c({'bus': 1, 'address': [72, 73]})"
```

GPIO Probes

Detect by GPIO state:

```
// Check GPIO value
"Probe": "gpio({'name': 'BOARD_ID0', 'value': 1})"
```

Compound Probes

Combine conditions with AND/OR:

```
// AND condition
"Probe": "AND(
    xyz.openbmc_project.FruDevice({'PRODUCT_PRODUCT_NAME': 'MyBoard'}),
    i2c({'bus': 1, 'address': 72})
)"

// OR condition
"Probe": "OR(
    xyz.openbmc_project.FruDevice({'PRODUCT_PRODUCT_NAME': 'BoardV1'}),
    xyz.openbmc_project.FruDevice({'PRODUCT_PRODUCT_NAME': 'BoardV2'})
)"
```

Record-Based Probes

Use found records in expressions:

```
{
  "Name": "$PRODUCT_PRODUCT_NAME",
  "Probe": "xyz.openbmc_project.FruDevice({'PRODUCT_MANUFACTURER': 'ACME'})",
  "Type": "Board",
  "Exposes": [
    {
      "Name": "$PRODUCT_PRODUCT_NAME Temp",
      "Bus": "$bus",
      "Address": "$address",
      "Type": "TMP75"
    }
  ]
}
```

Exposes Section

The `Exposes` array defines components provided by this entity.

Sensor Exposures

```
{
  "Exposes": [
    {
      "Name": "CPU Temp",
      "Type": "TMP75",
      "Bus": 1,
      "Address": "0x48",
      "Thresholds": [
        {
          "Direction": "greater than",
          "Name": "upper critical",
          "Severity": 1,
          "Value": 95
        },
        {
          "Direction": "greater than",
          "Name": "upper warning",
          "Severity": 0,
          "Value": 85
        }
      ]
    },
    {
      "Name": "P12V",
      "Type": "ADC",
      "Index": 0,
      "ScaleFactor": 4.0,
      "PowerState": "On"
    }
  ]
}
```

Fan Exposures

```
{
  "Exposes": [
    {
      "Name": "Fan0",
      "Type": "AspeedFan",
      "Connector": {
        "Name": "Fan0",
        "Pwm": 0,
        "Tachs": [0]
      }
    },
    {
      "Name": "Fan1",
      "Type": "AspeedFan",
      "Connector": {
        "Name": "Fan1",
        "Pwm": 1,
        "Tachs": [1, 2]
      }
    }
  ]
}
```

LED Exposures

```
{
  "Exposes": [
    {
      "Name": "identify",
      "Type": "GPIO",
      "GpioLine": "ID_LED",
      "Polarity": "ActiveLow"
    }
  ]
}
```

PSU Exposures

```
{
  "Exposes": [
    {
      "Name": "PSU1",
      "Type": "pmbus",
      "Bus": 3,
      "Address": "0x58",
      "Labels": ["vin", "vout1", "iout1", "pin", "pout1", "temp1"]
    }
  ]
}
```

FRU Configuration

EEPROM Configuration

```
{
  "Name": "System FRU",
  "Probe": "TRUE",
  "Type": "Board",
  "xyz.openbmc_project.Inventory.Decorator.Asset": {
    "Manufacturer": "$PRODUCT_MANUFACTURER",
    "Model": "$PRODUCT_PRODUCT_NAME",
    "PartNumber": "$PRODUCT_PART_NUMBER",
    "SerialNumber": "$PRODUCT_SERIAL_NUMBER"
  },
  "xyz.openbmc_project.Inventory.Item": {
    "Present": true,
    "PrettyName": "System Board"
  }
}
```

FRU Device Definition

```
{
  "Exposes": [
    {
      "Name": "System FRU",
      "Type": "EEPROM",
      "Bus": 0,
      "Address": "0x50",
      "Size": 256
    }
  ],
  "Name": "BaseBoard",
  "Probe": "TRUE",
  "Type": "Board"
}
```

D-Bus Interface Implementation

Entity Manager can implement arbitrary D-Bus interfaces:

```
{
  "Name": "MyBoard",
  "Probe": "TRUE",
  "Type": "Board",

  "xyz.openbmc_project.Inventory.Item": {
    "Present": true,
    "PrettyName": "Main Board"
  },

  "xyz.openbmc_project.Inventory.Decorator.Asset": {
    "Manufacturer": "ACME Corp",
    "Model": "Server-X",
    "PartNumber": "ABC-123",
    "SerialNumber": "SN-001"
  },

  "xyz.openbmc_project.Inventory.Decorator.Revision": {
    "Version": "1.0"
  }
}
```

Configuration Files Location

```
/usr/share/entity-manager/configurations/
├─ baseboard.json      # Main system board
├─ psu.json            # Power supplies
├─ fans.json           # Fan configuration
├─ cpu.json            # CPU sensors
├─ memory.json         # Memory configuration
└─ add-ons/
   ├─ riser1.json      # Add-on card
   └─ gpu.json         # GPU configuration
```

Complete Example

Board with Sensors, Fans, and FRU

```
{
  "Name": "MyServerBoard",
  "Type": "Board",
  "Probe": "xyz.openbmc_project.FruDevice({'PRODUCT_PRODUCT_NAME': 'MyServer'})",

  "xyz.openbmc_project.Inventory.Item": {
    "Present": true,
    "PrettyName": "Server Main Board"
  },

  "xyz.openbmc_project.Inventory.Decorator.Asset": {
    "Manufacturer": "$PRODUCT_MANUFACTURER",
    "Model": "$PRODUCT_PRODUCT_NAME",
    "PartNumber": "$PRODUCT_PART_NUMBER",
    "SerialNumber": "$PRODUCT_SERIAL_NUMBER"
  },

  "Exposes": [
    {
      "Name": "Inlet Temp",
      "Type": "TMP75",
      "Bus": "$bus",
      "Address": "0x48",
      "Thresholds": [
        {"Direction": "greater than", "Name": "upper critical", "Severity": 1, "Value": 45},
        {"Direction": "greater than", "Name": "upper warning", "Severity": 0, "Value": 40}
      ]
    },
    {
      "Name": "Outlet Temp",
      "Type": "TMP75",
      "Bus": "$bus",
      "Address": "0x49",
      "Thresholds": [
        {"Direction": "greater than", "Name": "upper critical", "Severity": 1, "Value": 55},
        {"Direction": "greater than", "Name": "upper warning", "Severity": 0, "Value": 50}
      ]
    },
    {
      "Name": "P12V",
      "Type": "ADC",
      "Index": 0,
      "ScaleFactor": 4.0,
      "Thresholds": [
        {"Direction": "greater than", "Name": "upper critical", "Severity": 1, "Value": 13.
2},
        {"Direction": "less than", "Name": "lower critical", "Severity": 1, "Value": 10.8}
      ]
    },
    {
      "Name": "P3V3",
      "Type": "ADC",
      "Index": 1,
      "ScaleFactor": 1.0,
      "Thresholds": [
        {"Direction": "greater than", "Name": "upper critical", "Severity": 1, "Value": 3.
6},
        {"Direction": "less than", "Name": "lower critical", "Severity": 1, "Value": 3.0}
      ]
    }
  ]
}
```

```

    ],
    {
        "Name": "Fan0",
        "Type": "AspeedFan",
        "Connector": {
            "Name": "Fan0",
            "Pwm": 0,
            "Tachs": [0]
        }
    },
    {
        "Name": "Fan1",
        "Type": "AspeedFan",
        "Connector": {
            "Name": "Fan1",
            "Pwm": 1,
            "Tachs": [1]
        }
    }
]
}

```

D-Bus Interface

Entity Manager exposes configuration on D-Bus:

```

# List all configurations
busctl tree xyz.openbmc_project.EntityManager

# Introspect a configuration
busctl introspect xyz.openbmc_project.EntityManager \
    /xyz/openbmc_project/inventory/system/board/MyServerBoard

# Get configuration property
busctl get-property xyz.openbmc_project.EntityManager \
    /xyz/openbmc_project/inventory/system/board/MyServerBoard \
    xyz.openbmc_project.Inventory.Item Present

```

Watching for Changes

```

# Monitor for new devices
busctl monitor xyz.openbmc_project.EntityManager

# Monitor object manager signals
dbus-monitor --system "type='signal',interface='org.freedesktop.DBus.ObjectManager'"

```


Testing Configurations

Validate JSON Syntax

```
# Check JSON is valid
python3 -m json.tool < myconfig.json > /dev/null

# Pretty print
python3 -m json.tool myconfig.json
```

Test on Target

```
# Copy configuration to BMC
scp myconfig.json root@bmc:/usr/share/entity-manager/configurations/

# Restart Entity Manager
systemctl restart xyz.openbmc_project.EntityManager

# Check logs
journalctl -u xyz.openbmc_project.EntityManager -f

# Verify configuration loaded
busctl tree xyz.openbmc_project.EntityManager | grep MyBoard
```

Debug Probe Evaluation

```
# Check FRU devices detected
busctl tree xyz.openbmc_project.FruDevice

# Get FRU properties
busctl introspect xyz.openbmc_project.FruDevice \
    /xyz/openbmc_project/FruDevice/device_0

# Check I2C bus
i2cdetect -y 1
```

Troubleshooting

Configuration Not Loading

```
# Check JSON syntax
python3 -m json.tool < config.json

# Check Entity Manager logs
journalctl -u xyz.openbmc_project.EntityManager | grep -i error

# Verify probe condition
# For FRU probes, check if FRU is detected
busctl call xyz.openbmc_project.FruDevice \
    /xyz/openbmc_project/FruDevice/device_0 \
    org.freedesktop.DBus.Properties GetAll s ""
```

Sensors Not Appearing

```
# Check sensor daemon received configuration
journalctl -u xyz.openbmc_project.hwmontempsensor | grep -i config

# Verify D-Bus object exists
busctl tree xyz.openbmc_project.EntityManager | grep -i sensor

# Check I2C device exists
ls /sys/bus/i2c/devices/
```

Probe Not Matching

```
# Debug FRU probe
busctl get-property xyz.openbmc_project.FruDevice \
  /xyz/openbmc_project/FruDevice/device_0 \
  xyz.openbmc_project.FruDevice PRODUCT_PRODUCT_NAME

# Check exact field name and value
# Field names are case-sensitive
```

Best Practices

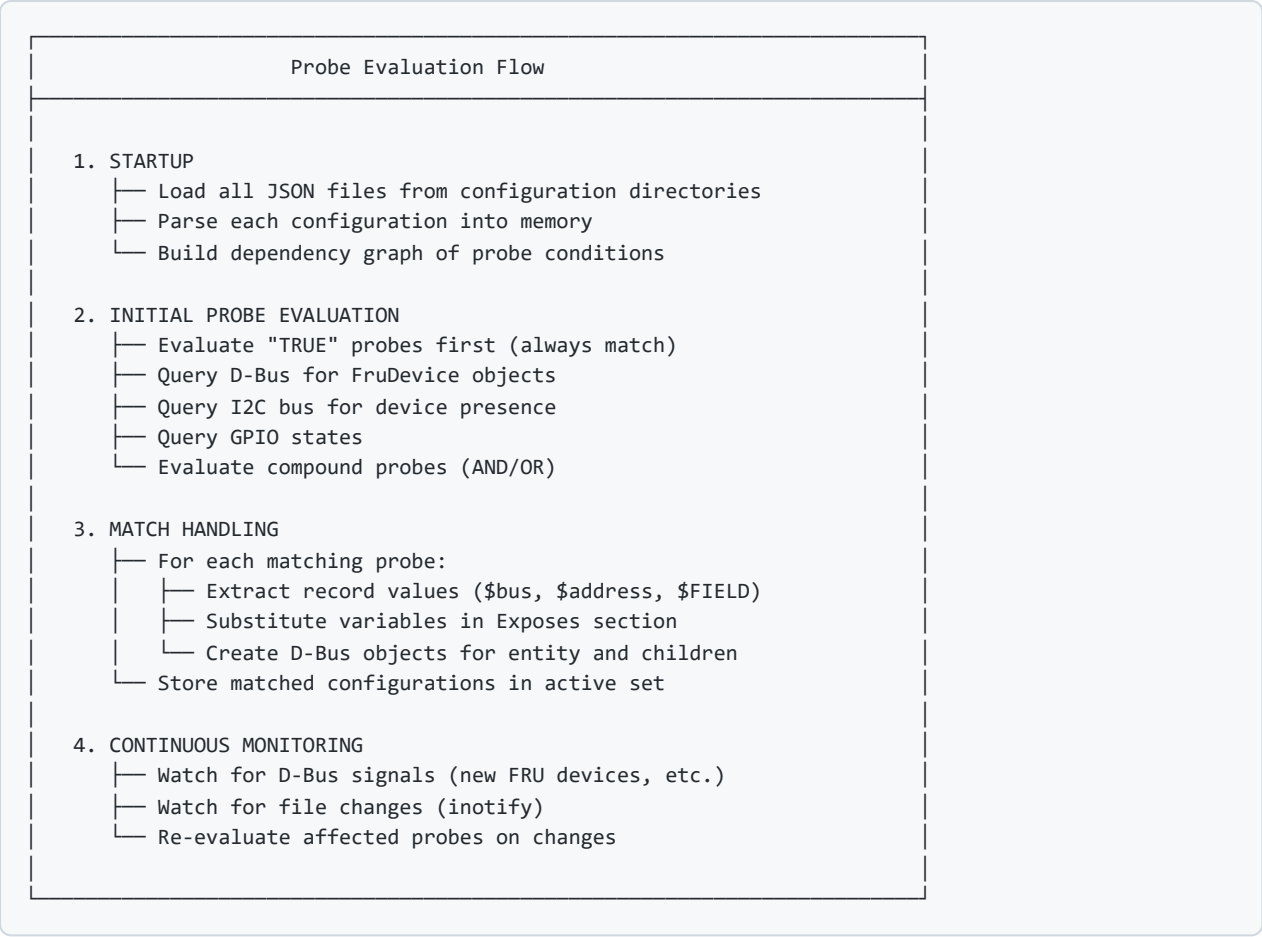
1. **Use probes** - Don't use `"Probe": "TRUE"` unless absolutely necessary
2. **Test JSON** - Validate syntax before deploying
3. **Use variables** - Leverage `$bus`, `$address` from probe records
4. **Document thresholds** - Explain why each threshold value was chosen
5. **Group logically** - Keep related sensors in the same config file
6. **Version control** - Track configuration changes in git

Deep Dive

Advanced implementation details for Entity Manager developers.

Probe Evaluation Algorithm

Entity Manager evaluates probes in a specific order to determine which configurations to load:



Probe Evaluation Priority:

Priority	Probe Type	Evaluation Method
1	TRUE / FALSE	Immediate (no D-Bus query)
2	i2c()	I2C device presence check
3	gpio()	GPIO line state query
4	xyz.openbmc_project.FruDevice()	D-Bus property match
5	AND() / OR()	Recursive evaluation

Source reference: [EntityManager.cpp](#)

Variable Substitution System

Entity Manager supports variable substitution using the \$ prefix:

Variable Substitution

Source Variables (from matched probe record):

```
FruDevice match at /xyz/openbmc_project/FruDevice/device_0
├─ PRODUCT_PRODUCT_NAME = "MyServer"
├─ PRODUCT_MANUFACTURER = "ACME"
├─ PRODUCT_SERIAL_NUMBER = "SN12345"
├─ bus = 1
└─ address = 80
```

Configuration Template:

```
"Name": "$PRODUCT_PRODUCT
_NAME"
"Bus": "$bus"
"Address": "$address"
```

After Substitution:

```
"Name": "MyServer"
"Bus": 1
"Address": 80
```

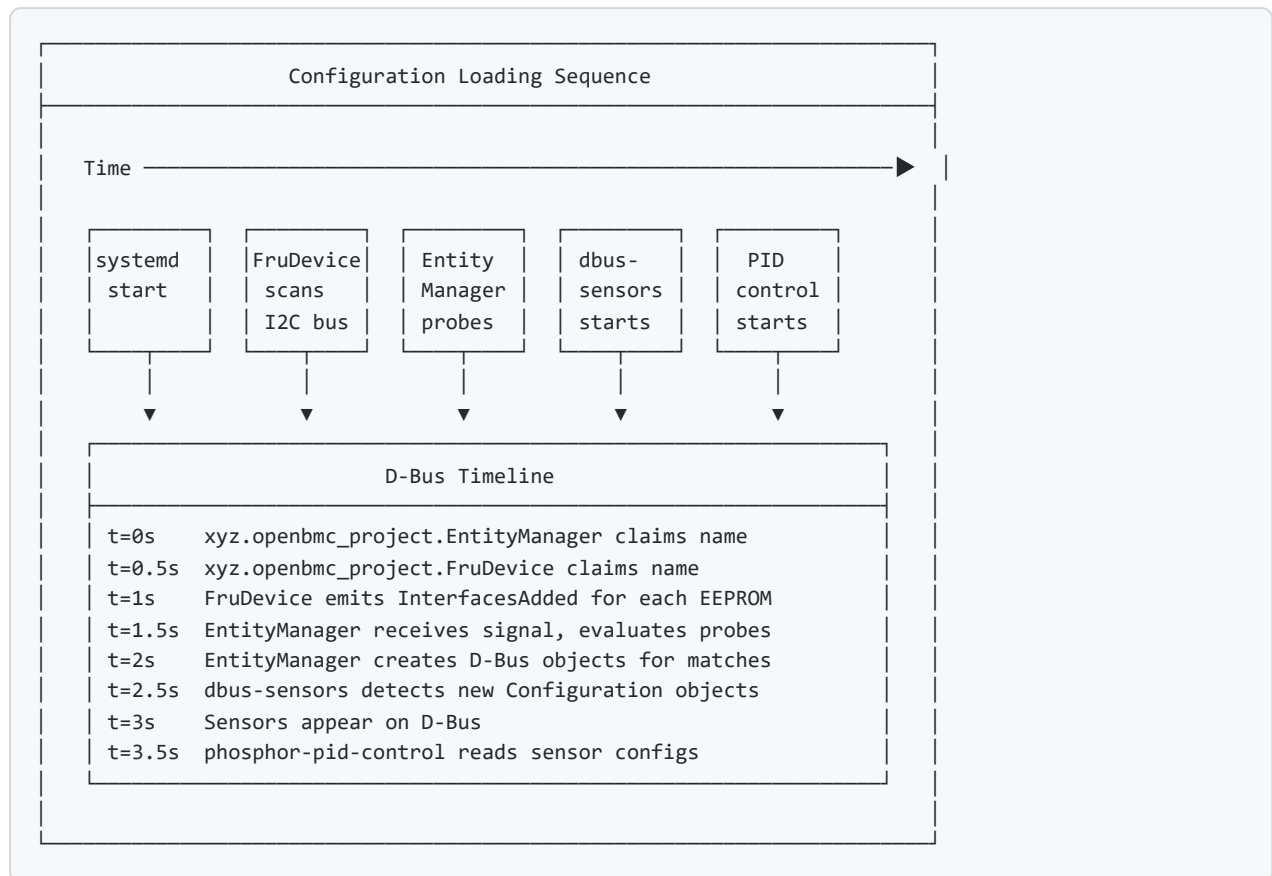
Special Variables:

```
├─ $bus      - I2C bus number from probe match
├─ $address  - I2C device address from probe match
├─ $index    - Array index when multiple matches
└─ $FIELD    - Any FRU field (PRODUCT_*, BOARD_*, CHASSIS_*)
```

Variable resolution order:

1. Probe record properties (bus, address)
2. FRU field values (PRODUCT_, BOARD_)
3. Parent entity properties (inheritance)
4. Default values (if specified)

Configuration Loading Sequence

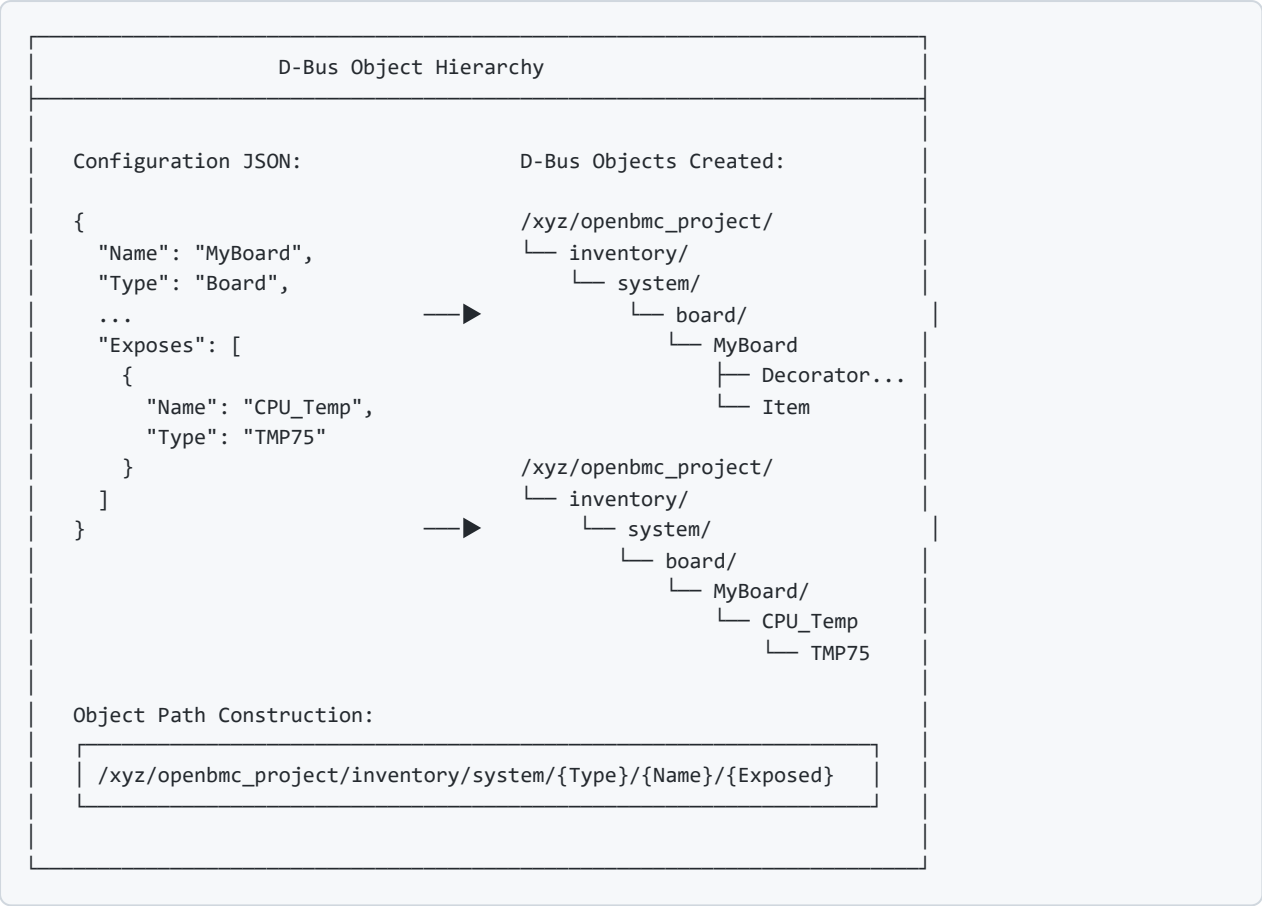


Key dependencies:

- `xyz.openbmc_project.FruDevice` must start before FRU probes work
- Entity Manager creates `xyz.openbmc_project.Configuration.*` objects
- Sensor daemons watch for Configuration objects via ObjectMapper

D-Bus Object Creation

When a probe matches, Entity Manager creates D-Bus objects:

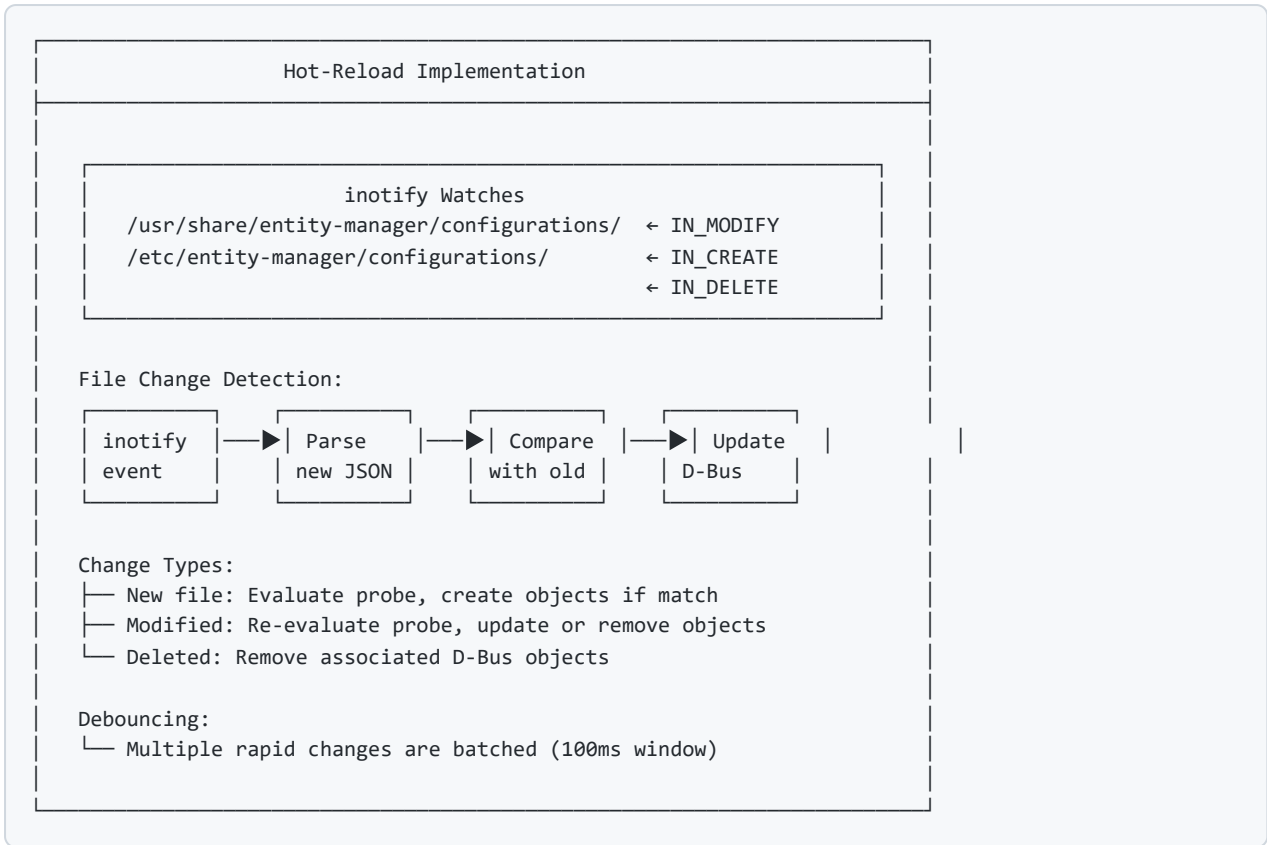


Interface mapping:

JSON Field	D-Bus Interface
xyz.openbmc_project.Inventory.Item	xyz.openbmc_project.Inventory.Item
xyz.openbmc_project.Inventory.Decorator.Asset	xyz.openbmc_project.Inventory.Decorator.Asset
Type in Exposes	xyz.openbmc_project.Configuration.{Type}

Hot-Reload Mechanism

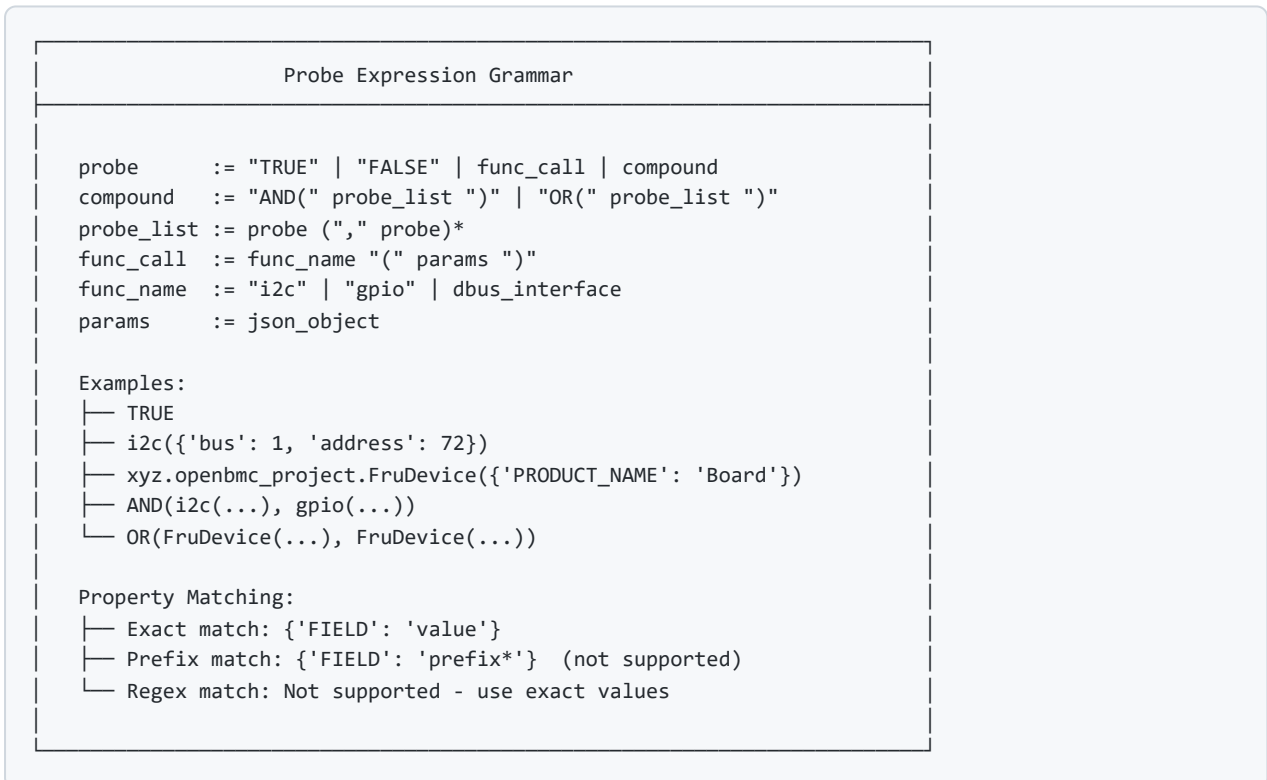
Entity Manager uses inotify to detect configuration file changes:



Note: Hot-reload may not update all dependent services. Some sensor daemons require restart to pick up configuration changes.

Probe Expression Parser

The probe expression parser supports a simple grammar:



Source reference: [PerformProbe.cpp](#)

Source Code Reference

Key implementation files in [entity-manager](#):

File	Description
src/EntityManager.cpp	Main daemon, D-Bus object management
src/PerformProbe.cpp	Probe evaluation logic
src/PerformScan.cpp	I2C/GPIO scanning
src/FruDevice.cpp	FRU EEPROM reading
src/Overlay.cpp	Device tree overlay support
src/Utils.cpp	Variable substitution, JSON parsing

Examples

Working examples are available in the [examples/entity-manager](#) directory:

- [baseboard.json](#) - Baseboard configuration with probes
- [temperature-sensors.json](#) - Temperature sensor configuration
- [adc-sensors.json](#) - ADC sensor configuration
- [fans.json](#) - Fan configuration
- [psu.json](#) - Power supply configuration
- [fru-eeprom.json](#) - FRU EEPROM configuration

Next Steps

For advanced Entity Manager topics including complex probing, FRU integration, and debugging:

- [Entity Manager Advanced](#) - Regex probes, FruDevice, troubleshooting

References

- [Entity Manager Repository](#)
- [Entity Manager README](#)
- [Configuration Examples](#)
- [dbus-sensors Integration](#)

Tested on: OpenBMC master, QEMU romulus

Fan Control Guide

Configure thermal management with phosphor-pid-control.

Table of Contents

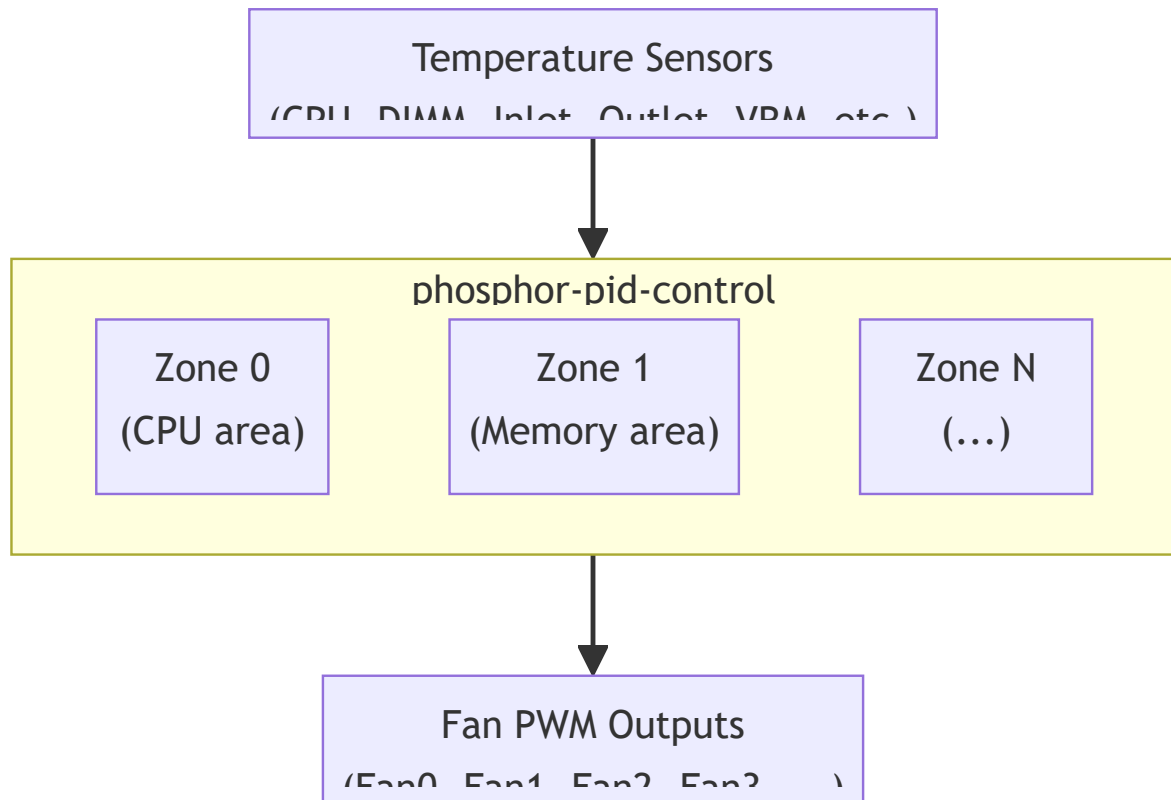
1. [Overview](#)
2. [Key Concepts](#)
 1. [Zones](#)
 2. [PID Controller](#)
 3. [Setpoint](#)
3. [Setup & Enable/Disable](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [Meson Build Options](#)
 3. [Runtime Enable/Disable](#)
 4. [Manual vs Automatic Mode](#)
 5. [Direct PWM Control \(Manual Mode\)](#)
4. [Configuration](#)
 1. [Zone Configuration](#)
 2. [PID Configuration](#)
 3. [PID Parameters](#)
 4. [Fan Configuration](#)
5. [Complete Configuration Example](#)
6. [Configuration Methods](#)
 1. [Direct JSON](#)
 2. [Entity Manager \(Recommended\)](#)
7. [Entity Manager Integration](#)
 1. [Where to Put the JSON File](#)
 2. [The Probe Field](#)
 3. [Complete Entity Manager Fan Config](#)
 4. [Verify on Running System](#)
8. [D-Bus Interface](#)
 1. [Zone Control](#)
 2. [Manual Fan Control](#)
9. [PID Tuning](#)
 1. [Step 1: Baseline Measurements](#)
 2. [Step 2: Start Conservative](#)
 3. [Step 3: Tune Integral Gain](#)
 4. [Step 4: Add Rate Limiting](#)
 5. [Step 5: Verify Failsafe](#)

- 10. [Multiple Zones](#)
 - 11. [Exit-Air Temperature and Airflow \(CFM\)](#)
 - 12. [Stepwise Control](#)
 - 13. [Troubleshooting](#)
 - 1. [Fans at Maximum Speed](#)
 - 2. [Fans Not Responding](#)
 - 3. [Temperature Oscillating](#)
 - 4. [Debug Logging](#)
 - 14. [Deep Dive](#)
 - 1. [PID Mathematics](#)
 - 2. [Anti-Windup Implementation](#)
 - 3. [Slew Rate Limiting](#)
 - 4. [Failsafe Trigger Conditions](#)
 - 5. [Control Loop Timing](#)
 - 6. [Multi-Sensor Zone Arbitration](#)
 - 7. [Source Code Reference](#)
 - 15. [Examples](#)
 - 16. [References](#)
-

Overview

phosphor-pid-control (swampd) is the thermal management daemon for OpenBMC. It uses PID (Proportional-Integral-Derivative) control to maintain optimal temperatures by adjusting fan speeds.

Fan Control Architecture



► ASCII-art version (for comparison)

Key Concepts

Zones

A **zone** is a thermal control region with:

- Input sensors (temperature, margin)
- Output fans (PWM control)
- PID controllers for each sensor
- A failsafe behavior

PID Controller

The PID algorithm calculates fan speed based on:

- **Proportional (P)**: Immediate response to error
- **Integral (I)**: Accumulated error over time
- **Derivative (D)**: Rate of change (often disabled)

$$\text{Output} = K_p \times \text{error} + K_i \times \int \text{error} \, dt + K_d \times d(\text{error})/dt$$

Setpoint

The target temperature the PID tries to maintain. The controller increases fan speed when temperature exceeds setpoint.

Setup & Enable/Disable

Build-Time Configuration (Yocto)

Include fan control in your image:

```
# In your machine .conf or image recipe
IMAGE_INSTALL:append = " phosphor-pid-control"

# Or exclude it
IMAGE_INSTALL:remove = "phosphor-pid-control"
```

Meson Build Options

```
meson setup build \
  -Dcontrol=enabled \
  -Dhost-interface=enabled \
  -Djson=enabled \
  -Dboot-time=enabled
```

Option	Default	Description
<code>control</code>	enabled	Enable fan control daemon
<code>host-interface</code>	enabled	D-Bus interface for host control
<code>json</code>	enabled	JSON configuration support
<code>boot-time</code>	disabled	Log boot-time thermal data

Runtime Enable/Disable

```
# Check service status
systemctl status phosphor-pid-control

# Stop fan control (fans go to failsafe)
systemctl stop phosphor-pid-control

# Disable fan control permanently
systemctl disable phosphor-pid-control

# Re-enable and start
systemctl enable phosphor-pid-control
systemctl start phosphor-pid-control

# Restart after config change
systemctl restart phosphor-pid-control
```

Manual vs Automatic Mode

Switch between manual and automatic fan control:

```
# Get current mode
busctl get-property xyz.openbmc_project.State.FanCtrl \
    /xyz/openbmc_project/control/thermal/0 \
    xyz.openbmc_project.Control.ThermalMode Current

# Switch to Manual mode (disable PID control)
busctl set-property xyz.openbmc_project.State.FanCtrl \
    /xyz/openbmc_project/control/thermal/0 \
    xyz.openbmc_project.Control.ThermalMode Current s "Manual"

# Switch back to Automatic mode (PID control)
busctl set-property xyz.openbmc_project.State.FanCtrl \
    /xyz/openbmc_project/control/thermal/0 \
    xyz.openbmc_project.Control.ThermalMode Current s "Default"
```

Direct PWM Control (Manual Mode)

When in manual mode, control fans directly:

```
# Set fan to 50% (value 0-255, so 128 = ~50%)
echo 128 > /sys/class/hwmon/hwmon0/pwm1

# Or via D-Bus
busctl set-property xyz.openbmc_project.FanSensor \
    /xyz/openbmc_project/control/fanpwm/Fan0 \
    xyz.openbmc_project.Control.FanPwm Target t 128
```

Configuration

Configuration is done via JSON files loaded by Entity Manager or directly in `/usr/share/swampd/`.

Source Reference: [phosphor-pid-control](#)

- Configuration schema: [phosphor-pid-control/conf.hpp](#)
- Example configs: [phosphor-pid-control/test/](#)

Zone Configuration

```
{
  "zones": [
    {
      "id": 0,
      "minThermalOutput": 25.0,
      "failsafePercent": 100.0
    }
  ]
}
```

Field	Description
<code>id</code>	Zone identifier
<code>minThermalOutput</code>	Minimum PWM % when zone is active
<code>failsafePercent</code>	PWM % when failsafe triggers

PID Configuration

```
{
  "pid": [
    {
      "name": "CPU Temp",
      "type": "temp",
      "inputs": ["CPU_Temp"],
      "setpoint": 80.0,
      "pid": {
        "samplePeriod": 1.0,
        "proportionalCoeff": 0.0,
        "integralCoeff": -0.2,
        "feedFwdOffsetCoeff": 0.0,
        "feedFwdGainCoeff": 0.0,
        "integrallimit_min": 0.0,
        "integrallimit_max": 100.0,
        "outLim_min": 25.0,
        "outLim_max": 100.0,
        "slewNeg": 0.0,
        "slewPos": 0.0
      }
    }
  ]
}
```

PID Parameters

Parameter	Description	Typical Range
<code>samplePeriod</code>	Control loop interval (seconds)	0.1 - 1.0
<code>proportionalCoeff</code>	P gain (often 0 for thermal)	0.0 - 1.0
<code>integralCoeff</code>	I gain (negative for cooling)	-0.5 to -0.1
<code>feedFwdOffsetCoeff</code>	Feed-forward offset	0.0
<code>feedFwdGainCoeff</code>	Feed-forward gain	0.0
<code>integrallimit_min/max</code>	Integral windup limits	0.0 - 100.0
<code>outLim_min/max</code>	Output PWM limits	0.0 - 100.0
<code>slewNeg/slewPos</code>	Rate limits (0=unlimited)	0.0 - 10.0

Fan Configuration

```
{
  "fans": [
    {
      "name": "Fan0",
      "zone": 0,
      "sensors": ["Fan0_Tach"],
      "output": "Fan0_PWM"
    },
    {
      "name": "Fan1",
      "zone": 0,
      "sensors": ["Fan1_Tach"],
      "output": "Fan1_PWM"
    }
  ]
}
```

Complete Configuration Example

```
{
  "version": "1.0",
  "zones": [
    {
      "id": 0,
      "minThermalOutput": 25.0,
      "failsafePercent": 100.0
    }
  ],
  "fans": [
    {
      "name": "Fan0",
      "zone": 0,
      "sensors": ["Fan0"],
      "output": "Fan0"
    },
    {
      "name": "Fan1",
      "zone": 0,
      "sensors": ["Fan1"],
      "output": "Fan1"
    },
    {
      "name": "Fan2",
      "zone": 0,
      "sensors": ["Fan2"],
      "output": "Fan2"
    },
    {
      "name": "Fan3",
      "zone": 0,
      "sensors": ["Fan3"],
      "output": "Fan3"
    }
  ],
  "pid": [
    {
      "name": "CPU Temp",
      "type": "temp",
      "inputs": ["CPU_Temp"],
      "setpoint": 80.0,
      "pid": {
        "samplePeriod": 1.0,
        "proportionalCoeff": 0.0,
        "integralCoeff": -0.2,
        "feedFwdOffsetCoeff": 0.0,
        "feedFwdGainCoeff": 0.0,
        "integrallimit_min": 0.0,
        "integrallimit_max": 100.0,
        "outLim_min": 25.0,
        "outLim_max": 100.0,
        "slewNeg": 0.0,
        "slewPos": 0.0
      }
    },
    {
      "name": "Inlet Temp",
      "type": "temp",
      "inputs": ["Inlet_Temp"],
      "setpoint": 35.0,

```



```

    "pid": {
      "samplePeriod": 1.0,
      "proportionalCoeff": 0.0,
      "integralCoeff": -0.3,
      "feedFwdOffsetCoeff": 0.0,
      "feedFwdGainCoeff": 0.0,
      "integrallLimit_min": 0.0,
      "integrallLimit_max": 100.0,
      "outLim_min": 25.0,
      "outLim_max": 100.0,
      "slewNeg": 0.0,
      "slewPos": 0.0
    }
  }
]
}

```

Configuration Methods

phosphor-pid-control supports two ways to load its configuration:

Method	Config Location	When to Use
Direct JSON	<code>/usr/share/swampd/config.json</code>	Simple systems, single board, quick prototyping
Entity Manager	<code>/usr/share/entity-manager/configurations/*.json</code>	Multi-board support, dynamic hardware detection

Direct JSON

Place a `config.json` in `/usr/share/swampd/` (see `zone-config.json` for a complete example). This file defines sensors, zones, and PIDs in a single flat structure.

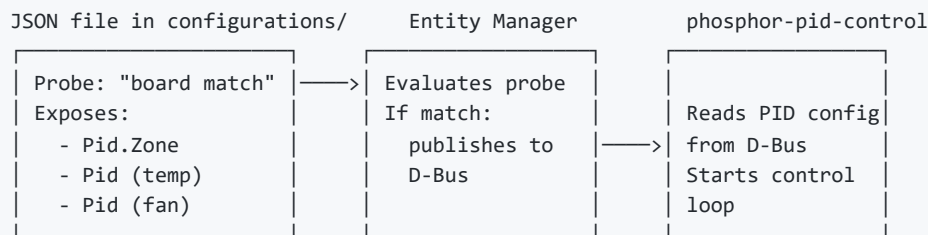
The direct JSON format includes `sensors` (D-Bus paths for temperature and fan tach), `zones`, and `pid` arrays. phosphor-pid-control reads this file at startup.

Entity Manager (Recommended)

Entity Manager is the preferred method for production systems. It separates hardware description from thermal policy and supports multiple board configurations.

How it works:

1. Entity Manager scans JSON files in `/usr/share/entity-manager/configurations/`
2. Each file has a `"Probe"` field that determines **when** the config applies (e.g., only when a specific board is detected via FRU data or GPIO)
3. When the probe matches, Entity Manager exposes the configuration on D-Bus
4. phosphor-pid-control reads its PID/zone config from D-Bus (not from the JSON file directly)



Entity Manager Integration

Where to Put the JSON File

The Entity Manager configuration file goes in your machine layer:

```
# In your Yocto layer
meta-myplatform/
  recipes-phosphor/
    configuration/
      entity-manager/
        files/
          my-board-fan-control.json  # <-- your fan config
          entity-manager_%.bbappend

# bbappend to install the config
cat > entity-manager_%.bbappend << 'EOF'
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
SRC_URI += "file://my-board-fan-control.json"

do_install:append() {
    install -d ${D}${datadir}/entity-manager/configurations
    install -m 0444 ${WORKDIR}/my-board-fan-control.json \
        ${D}${datadir}/entity-manager/configurations/
}
EOF
```

On the BMC filesystem, the file ends up at:

```
/usr/share/entity-manager/configurations/my-board-fan-control.json
```

The Probe Field

The **"Probe"** field tells Entity Manager **when** to activate this configuration. Common patterns:

```
// Always active (useful for development/QEMU)
"Probe": "TRUE"

// Match a specific board by FRU product name
"Probe": "xyz.openbmc_project.FruDevice({'PRODUCT_PRODUCT_NAME': 'MyServer'})"

// Match by board ID GPIO
"Probe": "xyz.openbmc_project.Inventory.Decorator.Asset({'Model': 'X123'})"
```

Complete Entity Manager Fan Config

A single JSON file defines the zone, thermal PIDs, and fan PIDs together. See [entity-manager-fan.json](#) for the full working example.

The key `Exposes` entries:

Zone definition — sets minimum fan speed and failsafe behavior:

```
{
  "Name": "Zone 0",
  "Type": "Pid.Zone",
  "MinThermalOutput": 25.0,
  "FailSafePercent": 100.0
}
```

Thermal PID (`Class: "temp"`) — controls fan speed based on temperature:

```
{
  "Name": "CPU Thermal PID",
  "Type": "Pid",
  "Class": "temp",
  "Inputs": ["CPU_Temp"],
  "SetPoint": 80.0,
  "Zones": ["Zone 0"],
  "PIDGains": {
    "P": 0.0,
    "I": -0.2,
    "D": 0.0,
    "FF": 0.0,
    "ILimitMin": 0.0,
    "ILimitMax": 100.0,
    "OutLimitMin": 25.0,
    "OutLimitMax": 100.0,
    "SlewNeg": 5.0,
    "SlewPos": 10.0
  }
}
```

Fan PID (`Class: "fan"`) — converts zone PWM% to actual fan RPM using feed-forward:

```
{
  "Name": "Fan0 PID",
  "Type": "Pid",
  "Class": "fan",
  "Inputs": ["Fan0"],
  "SetPoint": 7000.0,
  "Zones": ["Zone 0"],
  "PIDGains": {
    "P": 0.0,
    "I": 0.0,
    "D": 0.0,
    "FF": 1.0,
    "ILimitMin": 0.0,
    "ILimitMax": 100.0,
    "OutLimitMin": 0.0,
    "OutLimitMax": 100.0,
    "SlewNeg": 0.0,
    "SlewPos": 0.0
  }
}
```

The `Class` field is critical: `"temp"` PIDs calculate desired fan speed from temperature readings; `"fan"` PIDs translate the zone's requested RPM into actual PWM output. A complete config needs **both** types.

Verify on Running System

After deploying the config and rebooting:

```
# Check Entity Manager loaded the config
busctl tree xyz.openbmc_project.EntityManager | grep -i pid

# Check phosphor-pid-control is using the config
journalctl -u phosphor-pid-control | head -20

# Verify zones are active
busctl tree xyz.openbmc_project.State.FanCtrl
```

D-Bus Interface

Zone Control

```
# Check zone status
busctl tree xyz.openbmc_project.State.FanCtrl

# Get current zone mode
busctl get-property xyz.openbmc_project.State.FanCtrl \
  /xyz/openbmc_project/control/thermal/0 \
  xyz.openbmc_project.Control.ThermalMode Current

# Set manual mode
busctl set-property xyz.openbmc_project.State.FanCtrl \
  /xyz/openbmc_project/control/thermal/0 \
  xyz.openbmc_project.Control.ThermalMode Current s "Manual"
```

Manual Fan Control

```
# When in Manual mode, set fan PWM directly
busctl set-property xyz.openbmc_project.FanSensor \
  /xyz/openbmc_project/control/fanpwm/Fan0 \
  xyz.openbmc_project.Control.FanPwm Target t 75
```

PID Tuning

Step 1: Baseline Measurements

```
# Record temperatures at fixed fan speeds
for pwm in 30 50 70 100; do
  echo "Testing at $pwm% PWM"
  # Set manual mode and fixed PWM
  # Record temperatures after stabilization (5-10 minutes)
done
```

Step 2: Start Conservative

Begin with conservative PID values:

```
{
  "proportionalCoeff": 0.0,
  "integralCoeff": -0.1,
  "outLim_min": 30.0,
  "outLim_max": 100.0
}
```

Step 3: Tune Integral Gain

- **Too slow:** Increase `integralCoeff` magnitude (e.g., -0.1 → -0.2)
- **Oscillating:** Decrease `integralCoeff` magnitude
- **Not reaching setpoint:** Check `outLim_max` allows sufficient cooling

Step 4: Add Rate Limiting

```
{
  "slewNeg": 5.0,
  "slewPos": 10.0
}
```

- `slewPos`: Max PWM increase per second
- `slewNeg`: Max PWM decrease per second

Step 5: Verify Failsafe

Test failsafe behavior:

1. Disconnect a temperature sensor
2. Verify fans go to `failsafePercent`

Multiple Zones

For systems with multiple thermal zones:

```
{
  "zones": [
    {
      "id": 0,
      "minThermalOutput": 25.0,
      "failsafePercent": 100.0
    },
    {
      "id": 1,
      "minThermalOutput": 30.0,
      "failsafePercent": 100.0
    }
  ],
  "fans": [
    {
      "name": "CPU_Fan",
      "zone": 0,
      "sensors": ["CPU_Fan"],
      "output": "CPU_Fan"
    },
    {
      "name": "Sys_Fan",
      "zone": 1,
      "sensors": ["Sys_Fan"],
      "output": "Sys_Fan"
    }
  ]
}
```

Exit-Air Temperature and Airflow (CFM)

The `exitairtempensor` daemon (part of `dbus-sensors`) publishes two *calculated* sensors that let thermal control reason about airflow, not just temperature.

Airflow (CFM). A `CFMSensor` estimates airflow in cubic feet per minute from fan tachometer readings, using per-fan coefficients (`c1` , `c2` , `tachMinPercent` , `tachMaxPercent` , `maxCFM`). It also publishes `PWMLimit` / `CFMLimit` values that phosphor-pid-control can consume to cap fan PWM against a system-wide airflow ceiling.

Exit-air temperature. An `ExitAirTempSensor` derives the chassis exit-air temperature from total CFM, the inlet temperature, and system power, smoothed by an exponential moving average. Its tuning constants (`powerFactorMin` / `powerFactorMax` , `qMin` / `qMax` , `alphaS` , `alphaF` , `pOffset`) are supplied through Entity Manager configuration. (Verify the exact key names against your `dbus-sensors` version.)

Both sensors appear on D-Bus like any other sensor, so they can feed a PID zone as an input or drive thresholds. Configure them with Entity Manager `Exposes` entries of type `ExitAirTempSensor` / `CFMSensor` on the relevant chassis.

These are computed sensors with no backing hardware device. If either reads `NaN` , confirm that its input sensors (fan tachs, inlet temperature, system power) are present and that the coefficient set matches your chassis.

Stepwise Control

For simple table-based control instead of PID:

```
{
  "name": "Inlet Stepwise",
  "type": "stepwise",
  "inputs": ["Inlet_Temp"],
  "reading": {
    "positiveHysteresis": 2.0,
    "negativeHysteresis": 2.0
  },
  "output": {
    "0": 30,
    "25": 40,
    "30": 50,
    "35": 70,
    "40": 100
  }
}
```

The `output` map defines PWM percentage at each temperature threshold.

Troubleshooting

Fans at Maximum Speed

```
# Check if in failsafe
journalctl -u phosphor-pid-control | grep -i failsafe

# Verify sensors are responding
busctl tree xyz.openbmc_project.HwmonTempSensor

# Check PID controller status
journalctl -u phosphor-pid-control -f
```

Fans Not Responding

```
# Verify PWM control works manually
echo 128 > /sys/class/hwmon/hwmon0/pwm1

# Check fan sensor service
systemctl status xyz.openbmc_project.fansensor

# Verify D-Bus fan objects exist
busctl tree xyz.openbmc_project.FanSensor
```

Temperature Oscillating

Reduce integral gain or add slew rate limiting:

```
{
  "integralCoeff": -0.1,
  "slewNeg": 3.0,
  "slewPos": 5.0
}
```

Debug Logging

Enable verbose logging:

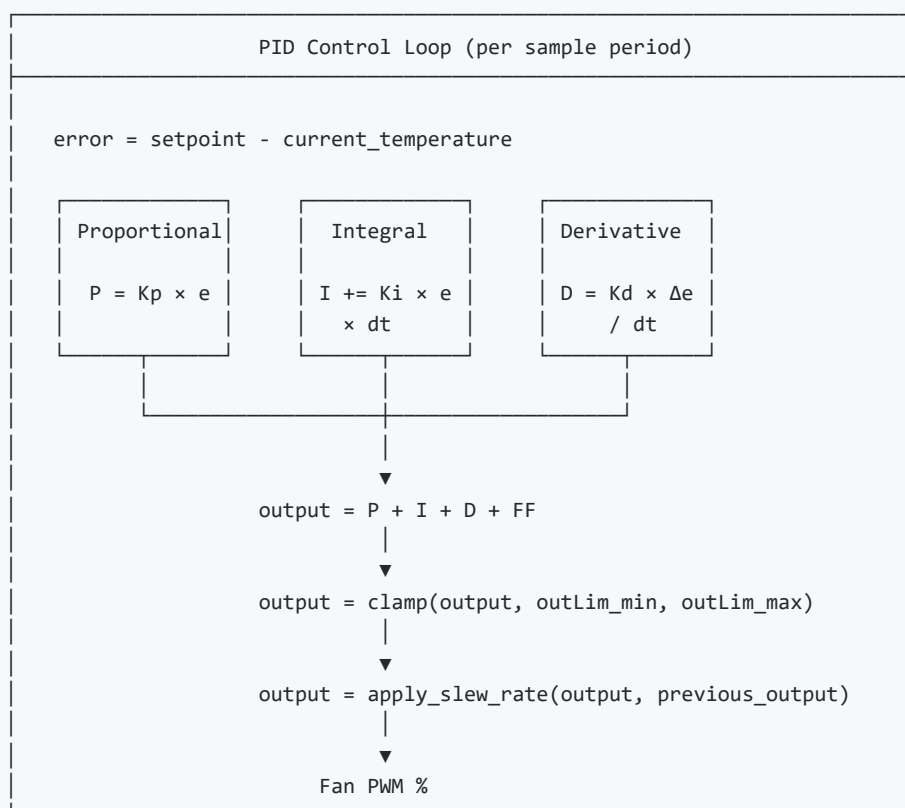
```
systemctl stop phosphor-pid-control
/usr/bin/swampd -d
```

Deep Dive

Advanced implementation details for thermal control developers.

PID Mathematics

The PID controller in phosphor-pid-control uses discrete-time calculations:



Why negative integral coefficient?

For cooling control, we want:

- Temperature **above** setpoint → **increase** fan speed
- `error = setpoint - temperature` is **negative** when too hot
- Negative `Ki` × negative `error` = **positive** output increase

Example:

setpoint = 80°C, temperature = 85°C

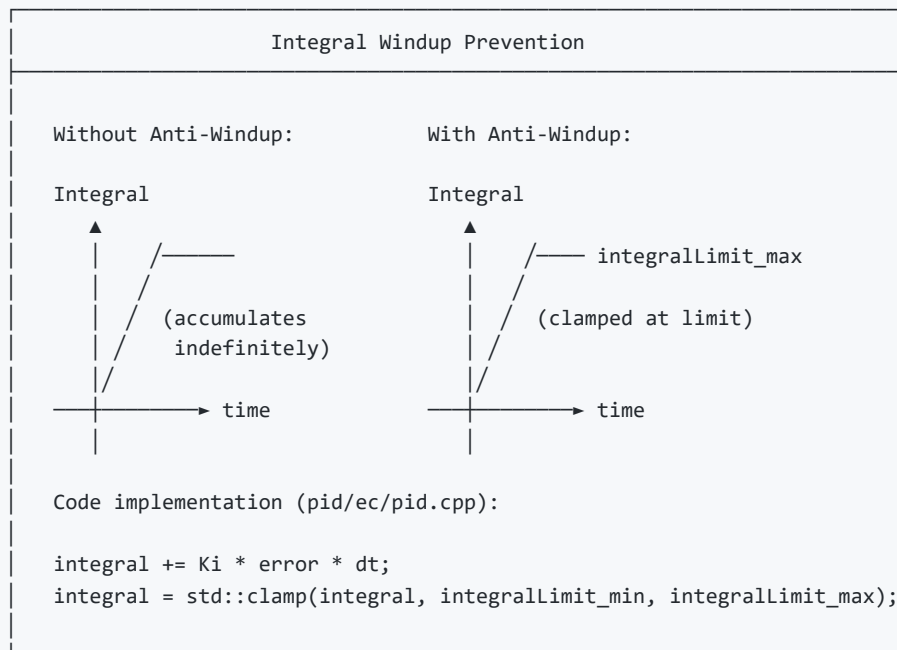
error = 80 - 85 = -5

Ki = -0.2

I contribution = -0.2 × -5 = +1.0 (increases PWM)

Anti-Windup Implementation

Integral windup occurs when the integral term accumulates excessively during sustained error, causing overshoot when the system finally responds.

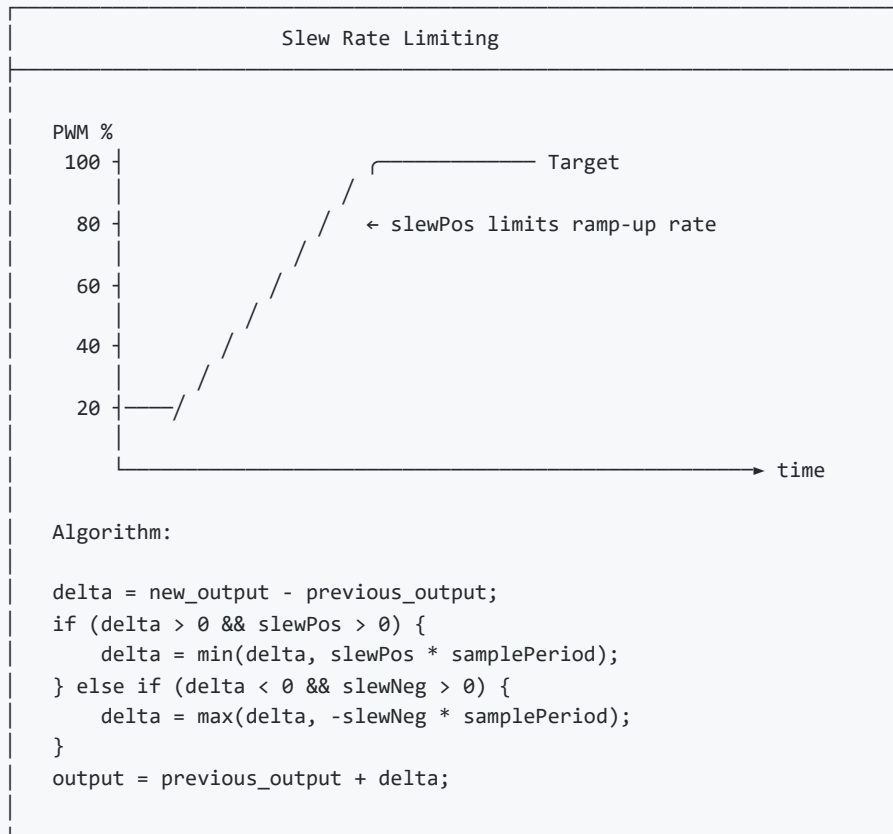


Configuration guidelines:

- `integrallimit_max`: Should match `outLim_max` (typically 100.0)
- `integrallimit_min`: Usually 0.0 for cooling-only systems

Slew Rate Limiting

Slew rate limits prevent sudden fan speed changes that cause acoustic disturbance:



Example calculation:

- Current PWM: 40%, Target: 80%
- `slewPos` : 10.0 (%/second)
- `samplePeriod` : 1.0 second
- Max change per cycle: 10% → Output: 50% (not 80%)
- After 4 cycles: 40% → 50% → 60% → 70% → 80%

Failsafe Trigger Conditions

Failsafe mode sets all fans to `failsafePercent` (typically 100%) when:

Failsafe Trigger Conditions

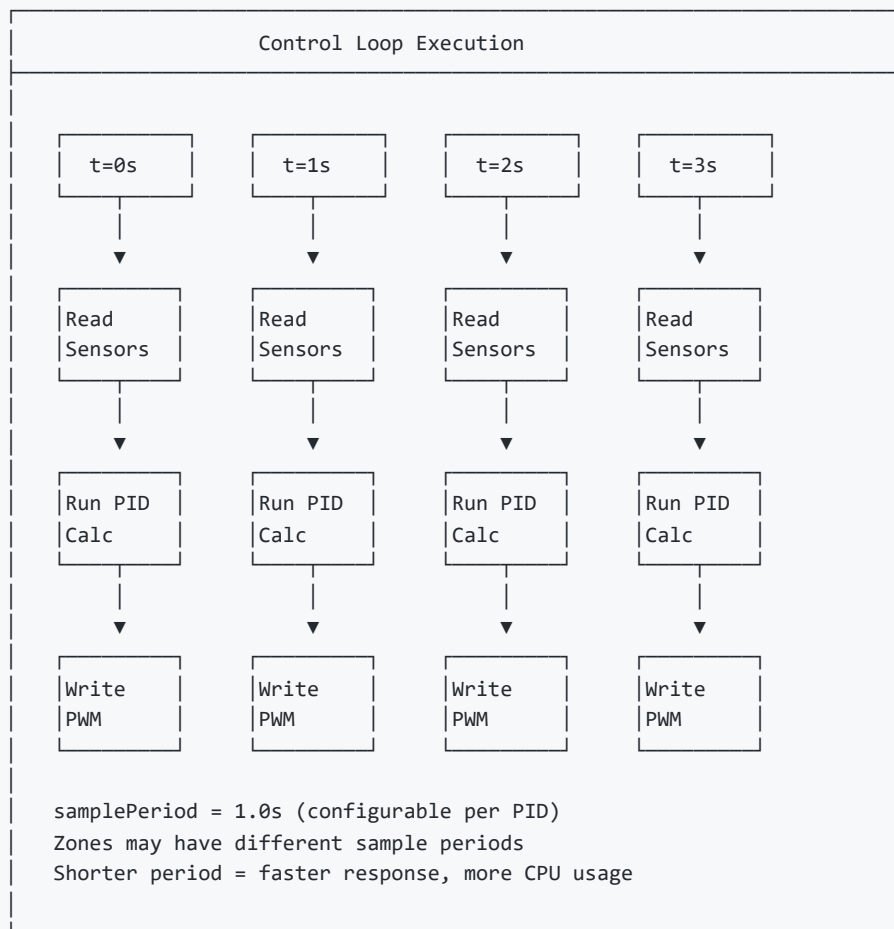
1. SENSOR TIMEOUT
 - └ Sensor value not updated within timeout period
 - └ Default: $2 \times \text{samplePeriod}$
 - └ Check: sensor returns NaN or error on D-Bus read
2. SENSOR MISSING
 - └ Required input sensor not found on D-Bus
 - └ Occurs during startup or sensor service crash
3. FAN FAILURE
 - └ Tachometer reads 0 RPM when PWM > minimum
 - └ Indicates stuck or disconnected fan
4. THERMAL RUNAWAY
 - └ Temperature exceeds critical threshold
 - └ Fans cannot maintain setpoint at 100% PWM

Recovery:

- └ Automatic when all sensors return to normal
- └ Requires valid readings for "recovery delay" period

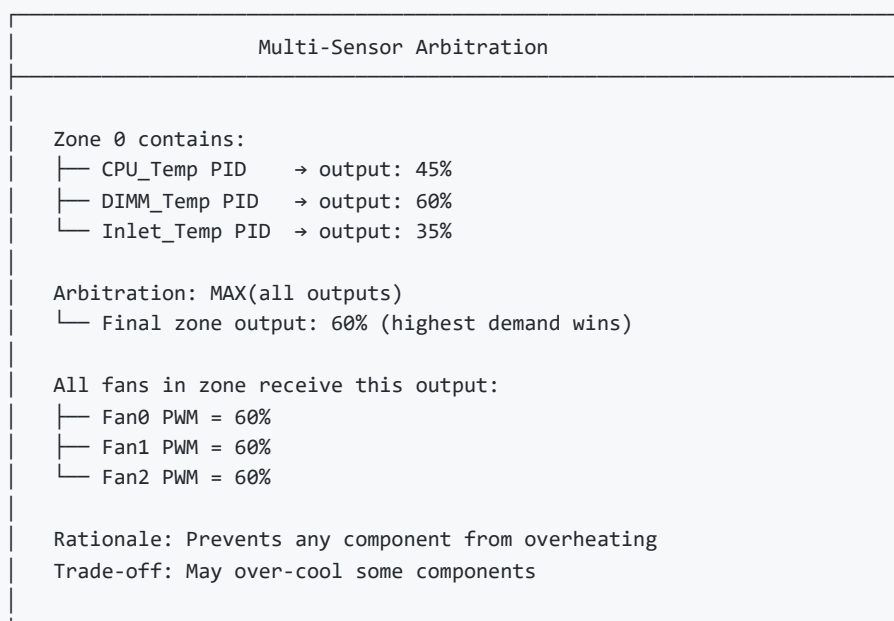
Source reference: [pid/zone.cpp](#)

Control Loop Timing



Multi-Sensor Zone Arbitration

When multiple PID controllers exist in a zone, outputs are arbitrated:



Source Code Reference

Key implementation files in [phosphor-pid-control](#):

File	Description
pid/ec/pid.cpp	Core PID algorithm implementation
pid/zone.cpp	Zone management and failsafe logic
pid/thermalcontroller.cpp	Temperature PID controller
pid/fancontroller.cpp	Fan speed PID controller
conf.hpp	Configuration structures
pid/builder.cpp	Configuration parser

Examples

Working examples are available in the [examples/fan-control](#) directory:

- [zone-config.json](#) - Basic zone configuration
- [multi-zone-config.json](#) - Multi-zone setup
- [stepwise-config.json](#) - Stepwise fan control
- [entity-manager-fan.json](#) - Entity Manager fan configuration

References

- [phosphor-pid-control](#)
- [PID Control Documentation](#)
- [Entity Manager Fan Control](#)

Tested on: OpenBMC master, QEMU romulus

Power Management Guide

Configure power sequencing, PSU monitoring, and voltage regulators.

Table of Contents

1. [Overview](#)
2. [Setup & Configuration](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [Meson Build Options](#)
 3. [Runtime Enable/Disable](#)
 4. [Configuration Files](#)
 5. [Deploying Configuration via Yocto](#)
 1. [Regulators Config](#)
 2. [PSU Entity Manager Config](#)
 3. [Verification](#)
 6. [Power Restore Policy Configuration](#)
 7. [Power Capping Configuration](#)
 8. [PSU Redundancy Configuration](#)
 9. [GPIO Configuration for Power Control](#)
3. [PSU Management \(phosphor-power\)](#)
 1. [PSU Monitoring](#)
 2. [D-Bus Interface](#)
 3. [Entity Manager PSU Configuration](#)
 4. [PSU Redundancy](#)
4. [Voltage Regulators \(phosphor-regulators\)](#)
 1. [Overview](#)
 2. [JSON Configuration](#)
 3. [Configuration Sections](#)
 1. [Rules](#)
 2. [Devices](#)
5. [Power Sequencing](#)
 1. [GPIO-Based Sequencing](#)
 1. [Device Tree GPIO Configuration](#)
 2. [Power Control Service](#)
 2. [Power Button Handling](#)
6. [Power Capping](#)
 1. [Overview](#)
 2. [D-Bus Interface](#)
 3. [Via Redfish](#)

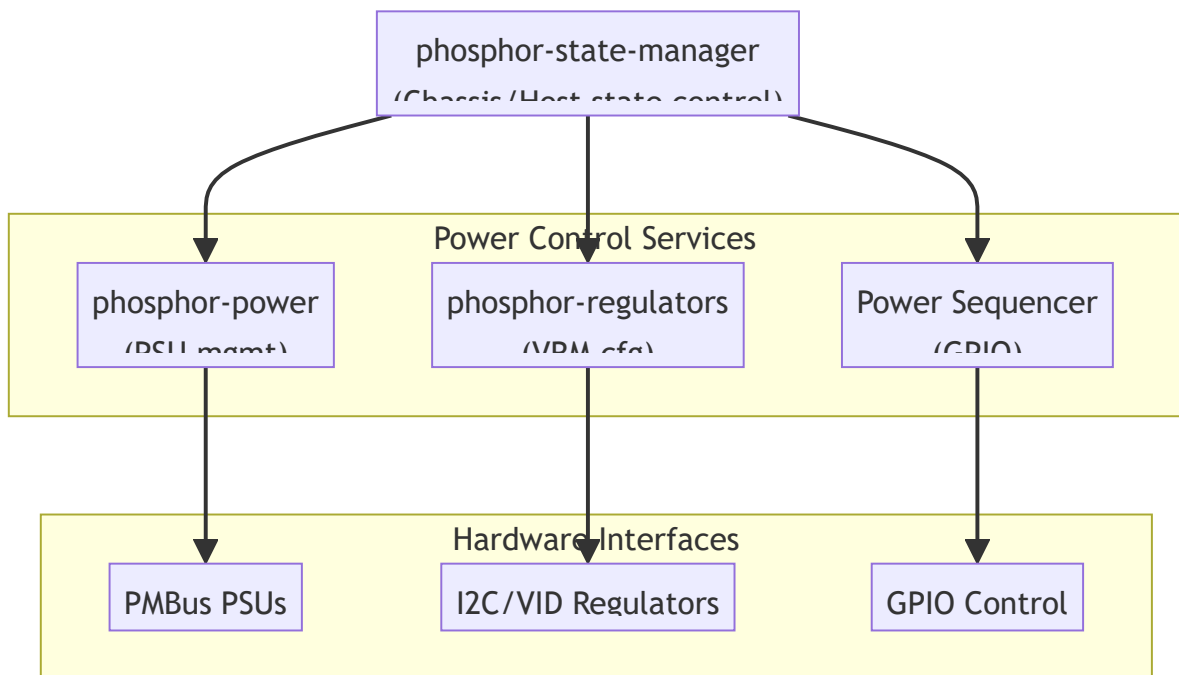
7. [Power Restore Policy](#)
 8. [PSU Event Monitoring](#)
 1. [SEL Events](#)
 2. [Threshold Events](#)
 9. [Porting Considerations](#)
 1. [Required Configuration](#)
 2. [Device Tree Example](#)
 3. [Machine Layer Integration](#)
 10. [Troubleshooting](#)
 1. [PSU Not Detected](#)
 2. [Power On Failure](#)
 3. [PSU Fault](#)
 11. [Deep Dive](#)
 1. [PMBus Protocol Overview](#)
 2. [Power Sequencing](#)
 3. [Voltage Regulator \(VR\) Configuration](#)
 4. [Power Capping Implementation](#)
 5. [Source Code Reference](#)
 12. [Examples](#)
 13. [References](#)
-

Overview

OpenBMC power management includes several components:

- **phosphor-power**: PSU management, power supply monitoring
- **phosphor-regulators**: Voltage regulator configuration
- **Power sequencing**: GPIO-based power control
- **Power capping**: System power limiting

Power Management Architecture



► ASCII-art version (for comparison)

Setup & Configuration

Build-Time Configuration (Yocto)

Include power management components:

```
# In your machine .conf or image recipe

# Core power management
IMAGE_INSTALL:append = " phosphor-power"

# Include specific components
IMAGE_INSTALL:append = " \
    phosphor-power-psu-monitor \
    phosphor-power-regulators \
    phosphor-power-sequencer \
    phosphor-power-utils \
"

# Exclude components you don't need
IMAGE_INSTALL:remove = "phosphor-power-regulators"
```


Meson Build Options

```
# phosphor-power build options
meson setup build \
  -Dpsu-monitor=enabled \
  -Dregulators=enabled \
  -Dsequencer=enabled \
  -Dutils=enabled \
  -Dtests=disabled
```

Option	Default	Description
<code>psu-monitor</code>	enabled	PSU monitoring daemon
<code>regulators</code>	enabled	Voltage regulator control
<code>sequencer</code>	enabled	Power sequencing
<code>utils</code>	enabled	Command-line utilities

Runtime Enable/Disable

```
# Check power services status
systemctl status phosphor-psu-monitor
systemctl status phosphor-regulators

# Disable PSU monitoring
systemctl stop phosphor-psu-monitor
systemctl disable phosphor-psu-monitor

# Enable regulators service
systemctl enable phosphor-regulators
systemctl start phosphor-regulators
```

Configuration Files

Config File	Install Path	Format	Read By
PSU config	<code>/usr/share/entity-manager/configurations/psu.json</code>	Entity Manager JSON	Entity Manager → D-Bus → psu-monitor
Regulators config	<code>/usr/share/phosphor-regulators/config.json</code>	phosphor-regulators JSON	phosphor-regulators directly
Power sequencer	<code>/usr/share/phosphor-power-sequencer/config.json</code>	Sequencer JSON	phosphor-power-sequencer directly

Deploying Configuration via Yocto

Each config file needs a bbappend recipe to install it into the image.

Regulators Config

```

# meta-myplatform/recipes-phosphor/power/phosphor-regulators/
# └─ files/
#   └─ my-regulators-config.json
# └─ phosphor-regulators_%.bbappend

cat > phosphor-regulators_%.bbappend << 'EOF'
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
SRC_URI += "file://my-regulators-config.json"

do_install:append() {
    install -d ${D}${datadir}/phosphor-regulators
    install -m 0644 ${WORKDIR}/my-regulators-config.json \
        ${D}${datadir}/phosphor-regulators/config.json
}
EOF

```

PSU Entity Manager Config

```

# meta-myplatform/recipes-phosphor/configuration/entity-manager/
# └─ files/
#   └─ my-psu-config.json
# └─ entity-manager_%.bbappend

cat > entity-manager_%.bbappend << 'EOF'
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
SRC_URI += "file://my-psu-config.json"

do_install:append() {
    install -d ${D}${datadir}/entity-manager/configurations
    install -m 0444 ${WORKDIR}/my-psu-config.json \
        ${D}${datadir}/entity-manager/configurations/
}
EOF

```

Verification

```

# Verify regulators config loaded
journalctl -u phosphor-regulators | head -20

# Verify PSU detected via Entity Manager
busctl tree xyz.openbmc_project.EntityManager | grep -i psu

# Verify PSU sensors populated
busctl tree xyz.openbmc_project.PSUSensor

```

Power Restore Policy Configuration

```
# Get current policy
busctl get-property xyz.openbmc_project.Settings \
  /xyz/openbmc_project/control/host0/power_restore_policy \
  xyz.openbmc_project.Control.Power.RestorePolicy PowerRestorePolicy

# Set policy via D-Bus
# Options: AlwaysOn, AlwaysOff, Restore
busctl set-property xyz.openbmc_project.Settings \
  /xyz/openbmc_project/control/host0/power_restore_policy \
  xyz.openbmc_project.Control.Power.RestorePolicy PowerRestorePolicy s \
  "xyz.openbmc_project.Control.Power.RestorePolicy.Policy.AlwaysOn"

# Configure via Redfish
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"PowerRestorePolicy": "AlwaysOn"}' \
  https://localhost/redfish/v1/Systems/system
```

Power Capping Configuration

```
# Enable power capping
busctl set-property xyz.openbmc_project.Settings \
  /xyz/openbmc_project/control/host0/power_cap \
  xyz.openbmc_project.Control.Power.Cap PowerCapEnable b true

# Set power cap value (watts)
busctl set-property xyz.openbmc_project.Settings \
  /xyz/openbmc_project/control/host0/power_cap \
  xyz.openbmc_project.Control.Power.Cap PowerCap u 500

# Via Redfish
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"PowerControl":[{"PowerLimit":{"LimitInWatts":500,"LimitException":"LogEventOnly"}}]}' \
  https://localhost/redfish/v1/Chassis/chassis/Power
```

PSU Redundancy Configuration

```
# Get redundancy status
busctl get-property xyz.openbmc_project.Power.PSU \
  /xyz/openbmc_project/control/power_supply_redundancy \
  xyz.openbmc_project.Control.PowerSupplyRedundancy PowerSupplyRedundancyEnabled

# Enable redundancy monitoring
busctl set-property xyz.openbmc_project.Power.PSU \
  /xyz/openbmc_project/control/power_supply_redundancy \
  xyz.openbmc_project.Control.PowerSupplyRedundancy PowerSupplyRedundancyEnabled b true
```

GPIO Configuration for Power Control

Define GPIO pins in your machine's device tree:

```
// Example device tree fragment
gpio-keys {
    compatible = "gpio-keys";

    power-button {
        label = "power-button";
        gpios = <&gpio0 ASPEED_GPIO(E, 0) GPIO_ACTIVE_LOW>;
        linux,code = <KEY_POWER>;
    };

    power-ok {
        label = "power-ok";
        gpios = <&gpio0 ASPEED_GPIO(E, 1) GPIO_ACTIVE_HIGH>;
        linux,code = <KEY_BATTERY>;
    };
};
```

Configure in JSON:

```
{
  "gpio_configs": [
    {
      "name": "POWER_BUTTON",
      "direction": "out",
      "polarity": "active_low",
      "line": 32
    },
    {
      "name": "POWER_OK",
      "direction": "in",
      "polarity": "active_high",
      "line": 33
    }
  ]
}
```

PSU Management (phosphor-power)

PSU Monitoring

The `psu-monitor` service monitors PMBus power supplies for:

- Input/output voltage and current
- Temperature
- Fan speed
- Fault conditions

D-Bus Interface

```
# List PSU inventory
busctl tree xyz.openbmc_project.Inventory.Manager | grep powersupply

# Get PSU properties
busctl introspect xyz.openbmc_project.Inventory.Manager \
    /xyz/openbmc_project/inventory/system/chassis/powersupply0

# Check PSU present
busctl get-property xyz.openbmc_project.Inventory.Manager \
    /xyz/openbmc_project/inventory/system/chassis/powersupply0 \
    xyz.openbmc_project.Inventory.Item Present
```

Entity Manager PSU Configuration

```
{
  "Name": "PSU0",
  "Type": "Board",
  "Probe": "xyz.openbmc_project.FruDevice({'PRODUCT_PRODUCT_NAME': 'PSU-1000W'})",

  "Exposes": [
    {
      "Name": "PSU0",
      "Type": "pmbus",
      "Bus": 3,
      "Address": "0x58",
      "Labels": [
        "vin",
        "vout1",
        "iout1",
        "pin",
        "pout1",
        "temp1",
        "temp2",
        "fan1"
      ]
    }
  ],
  "xyz.openbmc_project.Inventory.Item": {
    "Present": true,
    "PrettyName": "Power Supply 0"
  },
  "xyz.openbmc_project.Inventory.Decorator.Asset": {
    "Manufacturer": "$PRODUCT_MANUFACTURER",
    "Model": "$PRODUCT_PRODUCT_NAME",
    "SerialNumber": "$PRODUCT_SERIAL_NUMBER"
  }
}
```

PSU Redundancy

Configure redundancy policy:

```
# Get current redundancy mode
busctl get-property xyz.openbmc_project.Power.PSU \
    /xyz/openbmc_project/control/power_supply_redundancy \
    xyz.openbmc_project.Control.PowerSupplyRedundancy PowerSupplyRedundancyEnabled

# Set redundancy mode
busctl set-property xyz.openbmc_project.Power.PSU \
    /xyz/openbmc_project/control/power_supply_redundancy \
    xyz.openbmc_project.Control.PowerSupplyRedundancy PowerSupplyRedundancyEnabled b true
```

Voltage Regulators (phosphor-regulators)

Overview

phosphor-regulators manages voltage regulators on the system:

- Configuration during power-on
- Voltage level monitoring
- Regulator fault detection

JSON Configuration

Configuration file: `/usr/share/phosphor-regulators/config.json`

```

{
  "rules": [
    {
      "id": "set_voltage_rule",
      "actions": [
        {
          "pmbus_write_vout_command": {
            "format": "linear",
            "exponent": -8
          }
        }
      ]
    }
  ],
  "chassis": [
    {
      "number": 1,
      "inventory_path": "/xyz/openbmc_project/inventory/system/chassis",
      "devices": [
        {
          "id": "vdd_cpu0",
          "is_regulator": true,
          "fru": "/xyz/openbmc_project/inventory/system/chassis/motherboard/cpu0",
          "i2c_interface": {
            "bus": 1,
            "address": "0x40"
          },
          "configuration": {
            "volts": 1.0,
            "rule_id": "set_voltage_rule"
          },
          "rails": [
            {
              "id": "vdd_cpu0",
              "sensor_monitoring": {
                "rule_id": "read_sensors_rule"
              }
            }
          ]
        }
      ]
    }
  ]
}

```

Configuration Sections

Rules

Reusable actions:

```
{
  "rules": [
    {
      "id": "read_sensors_rule",
      "actions": [
        {
          "pmbus_read_sensor": {
            "type": "vout",
            "command": "0x8B",
            "format": "linear_16"
          }
        },
        {
          "pmbus_read_sensor": {
            "type": "iout",
            "command": "0x8C",
            "format": "linear_11"
          }
        },
        {
          "pmbus_read_sensor": {
            "type": "temperature",
            "command": "0x8D",
            "format": "linear_11"
          }
        }
      ]
    }
  ]
}
```

Devices

Regulator definitions:

```
{
  "id": "vdd_cpu0",
  "is_regulator": true,
  "fru": "/xyz/openbmc_project/inventory/system/chassis/motherboard/cpu0",
  "i2c_interface": {
    "bus": 1,
    "address": "0x40"
  },
  "presence_detection": {
    "rule_id": "detect_presence_rule"
  },
  "configuration": {
    "volts": 1.0,
    "rule_id": "set_voltage_rule"
  }
}
```

Power Sequencing

GPIO-Based Sequencing

Power sequencing is typically handled by systemd targets and GPIO control.

Device Tree GPIO Configuration

```
gpio-keys {
    compatible = "gpio-keys";

    power-good {
        label = "power-good";
        gpios = <&gpio0 ASPEED_GPIO(B, 2) GPIO_ACTIVE_HIGH>;
        linux,code = <KEY_POWER>;
    };
};

leds {
    compatible = "gpio-leds";

    power-button {
        gpios = <&gpio0 ASPEED_GPIO(D, 3) GPIO_ACTIVE_LOW>;
    };
};
```

Power Control Service

```
# /lib/systemd/system/power-control.service
[Unit]
Description=Power Control
After=xyz.openbmc_project.State.Chassis.service

[Service]
Type=oneshot
ExecStart=/usr/bin/power-control.sh on
RemainAfterExit=yes

[Install]
WantedBy=obmc-host-start@0.target
```

Power Button Handling

```
# Monitor power button
journalctl -u xyz.openbmc_project.Chassis.Buttons -f

# Check button state
busctl introspect xyz.openbmc_project.Chassis.Buttons \
    /xyz/openbmc_project/chassis/buttons/power
```

Power Capping

Overview

Power capping limits total system power consumption.

D-Bus Interface

```
# Get current power cap
busctl get-property xyz.openbmc_project.Settings \
    /xyz/openbmc_project/control/host0/power_cap \
    xyz.openbmc_project.Control.Power.Cap PowerCap

# Set power cap (watts)
busctl set-property xyz.openbmc_project.Settings \
    /xyz/openbmc_project/control/host0/power_cap \
    xyz.openbmc_project.Control.Power.Cap PowerCap u 500

# Enable power cap
busctl set-property xyz.openbmc_project.Settings \
    /xyz/openbmc_project/control/host0/power_cap \
    xyz.openbmc_project.Control.Power.Cap PowerCapEnable b true
```

Via Redfish

```
# Get power limit
curl -k -u root:0penBmc \
    https://localhost/redfish/v1/Chassis/chassis/Power

# Set power limit
curl -k -u root:0penBmc -X PATCH \
    -H "Content-Type: application/json" \
    -d '{"PowerControl":[{"PowerLimit":{"LimitInWatts":500}}]}' \
    https://localhost/redfish/v1/Chassis/chassis/Power
```

Power Restore Policy

Configure behavior after AC power loss.

```
# Get policy
busctl get-property xyz.openbmc_project.Settings \
    /xyz/openbmc_project/control/host0/power_restore_policy \
    xyz.openbmc_project.Control.Power.RestorePolicy PowerRestorePolicy

# Set to AlwaysOn
busctl set-property xyz.openbmc_project.Settings \
    /xyz/openbmc_project/control/host0/power_restore_policy \
    xyz.openbmc_project.Control.Power.RestorePolicy PowerRestorePolicy s \
    "xyz.openbmc_project.Control.Power.RestorePolicy.Policy.AlwaysOn"
```

Policy	Description
AlwaysOn	Power on after AC restore
AlwaysOff	Stay off after AC restore
Restore	Return to previous state

PSU Event Monitoring

SEL Events

PSU events are logged to the System Event Log:

```
# View PSU-related events
ipmitool sel list | grep -i power

# Via Redfish
curl -k -u root:0penBmc \
  https://localhost/redfish/v1/Systems/system/LogServices/EventLog/Entries \
  | jq '.Members[] | select(.Message | contains("Power"))'
```

Threshold Events

Configure PSU sensor thresholds for alerts:

```
{
  "Exposes": [
    {
      "Name": "PSU0",
      "Type": "pmbus",
      "Bus": 3,
      "Address": "0x58",
      "Thresholds": [
        {
          "Direction": "greater than",
          "Label": "temp1",
          "Name": "upper critical",
          "Severity": 1,
          "Value": 100
        },
        {
          "Direction": "less than",
          "Label": "vin",
          "Name": "lower critical",
          "Severity": 1,
          "Value": 180
        }
      ]
    }
  ]
}
```

Porting Considerations

Required Configuration

1. **PSU I2C addresses:** Identify PMBus addresses
2. **GPIO mapping:** Power button, power good signals
3. **Sequencing requirements:** Timing constraints
4. **Redundancy mode:** N+1 or N+N configuration

Device Tree Example

```
&i2c3 {
    status = "okay";

    psu0: psu@58 {
        compatible = "pmbus";
        reg = <0x58>;
    };

    psu1: psu@59 {
        compatible = "pmbus";
        reg = <0x59>;
    };
};
```

Machine Layer Integration

```
# In your machine .conf
PREFERRED_PROVIDER_virtual/phosphor-power = "phosphor-power"

# Include power-related packages
IMAGE_INSTALL:append = " \
    phosphor-power \
    phosphor-regulators \
    "
```

Troubleshooting

PSU Not Detected

```
# Check I2C bus
i2cdetect -y 3

# Verify PMBus communication
i2cget -y 3 0x58 0x99 w # Read MFR_ID

# Check Entity Manager logs
journalctl -u xyz.openbmc_project.EntityManager | grep -i psu
```

Power On Failure

```
# Check power state
obmcutil state

# Check power good GPIO
gpioget gpiochip0 <power-good-pin>

# Check systemd targets
systemctl list-dependencies obmc-host-start@0.target
```

PSU Fault

```
# Check PSU status
busctl get-property xyz.openbmc_project.PSUSensor \
    /xyz/openbmc_project/sensors/power/PSU0_Input_Power \
    xyz.openbmc_project.Sensor.Value Value

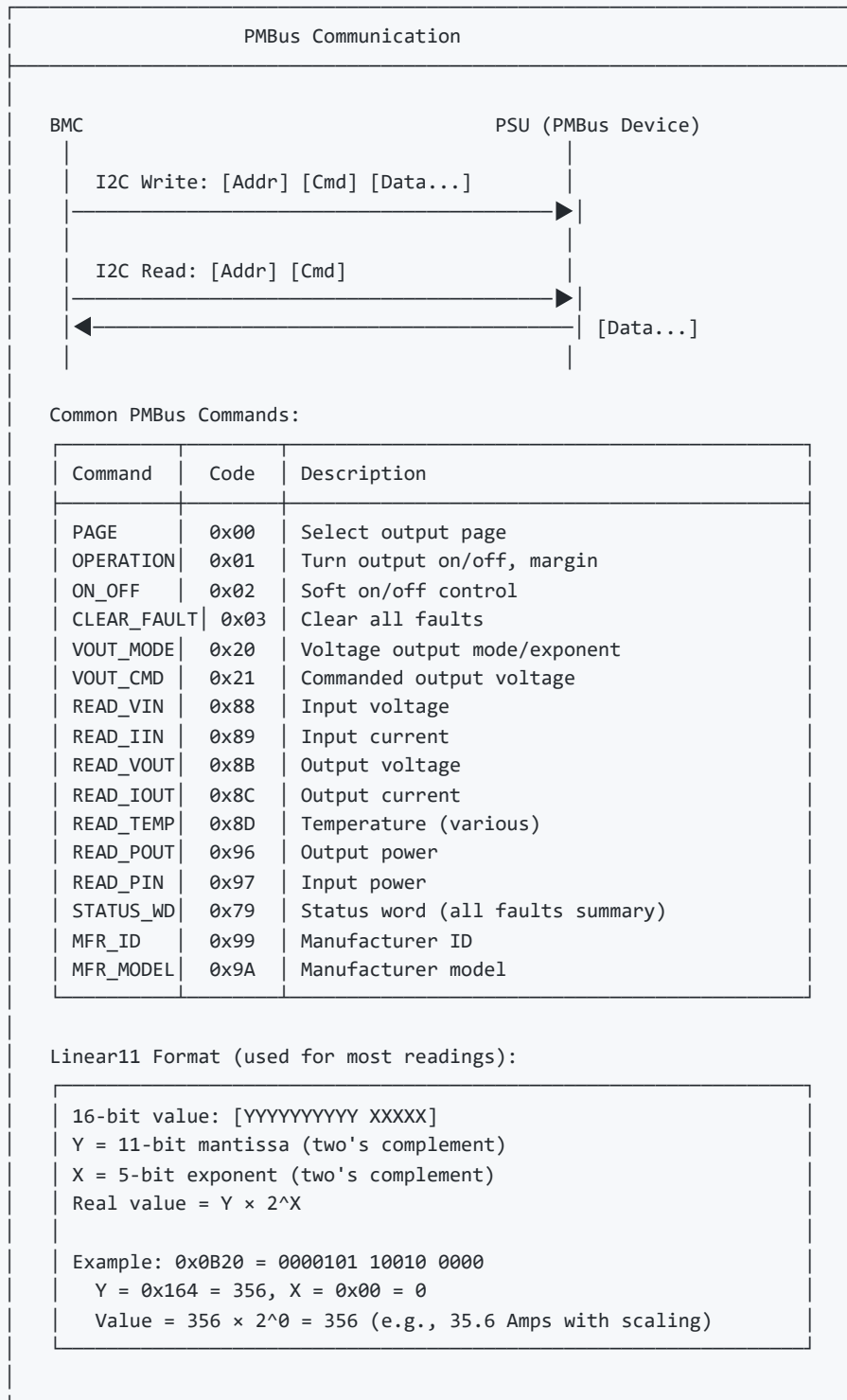
# Read PMBus status
i2cget -y 3 0x58 0x79 w # STATUS_WORD
```

Deep Dive

Advanced implementation details for power management developers.

PMBus Protocol Overview

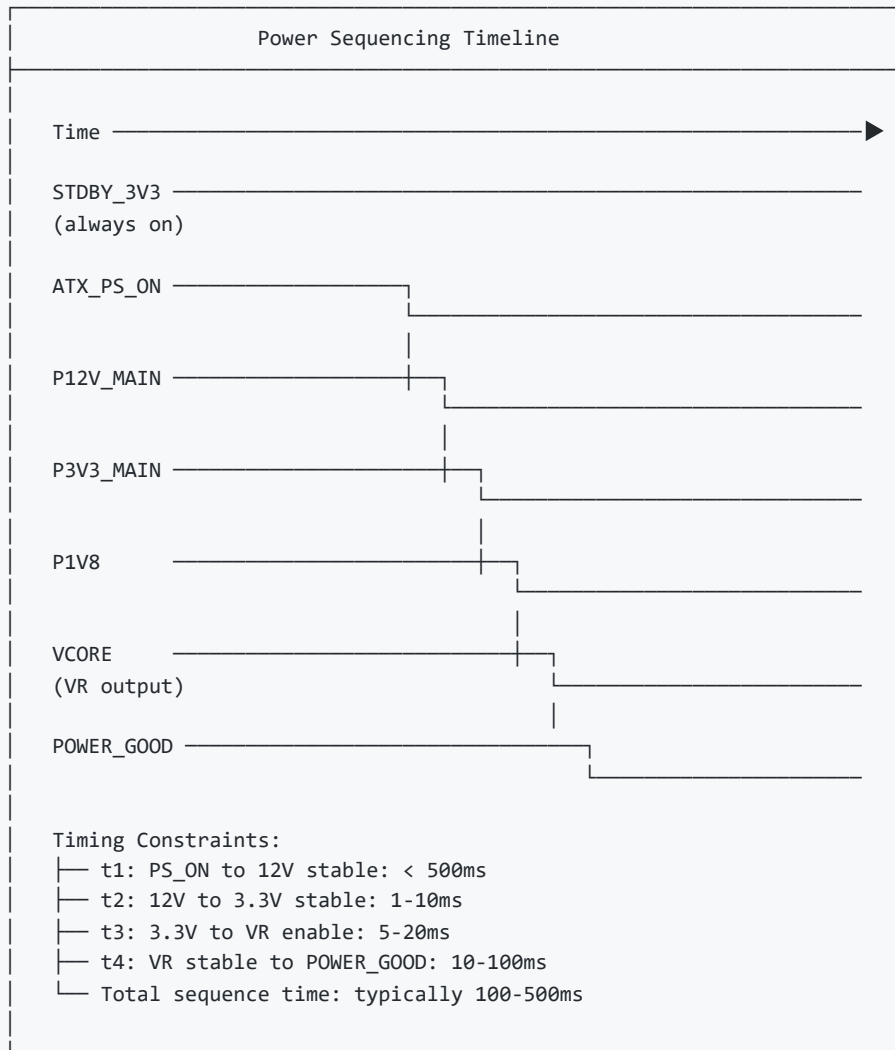
PMBus is an I2C-based protocol for power supply communication:



Source reference: phosphor-power/tools/power-utils

Power Sequencing

Power sequencing ensures voltages come up in the correct order:



Voltage Regulator (VR) Configuration

Voltage Regulator Control

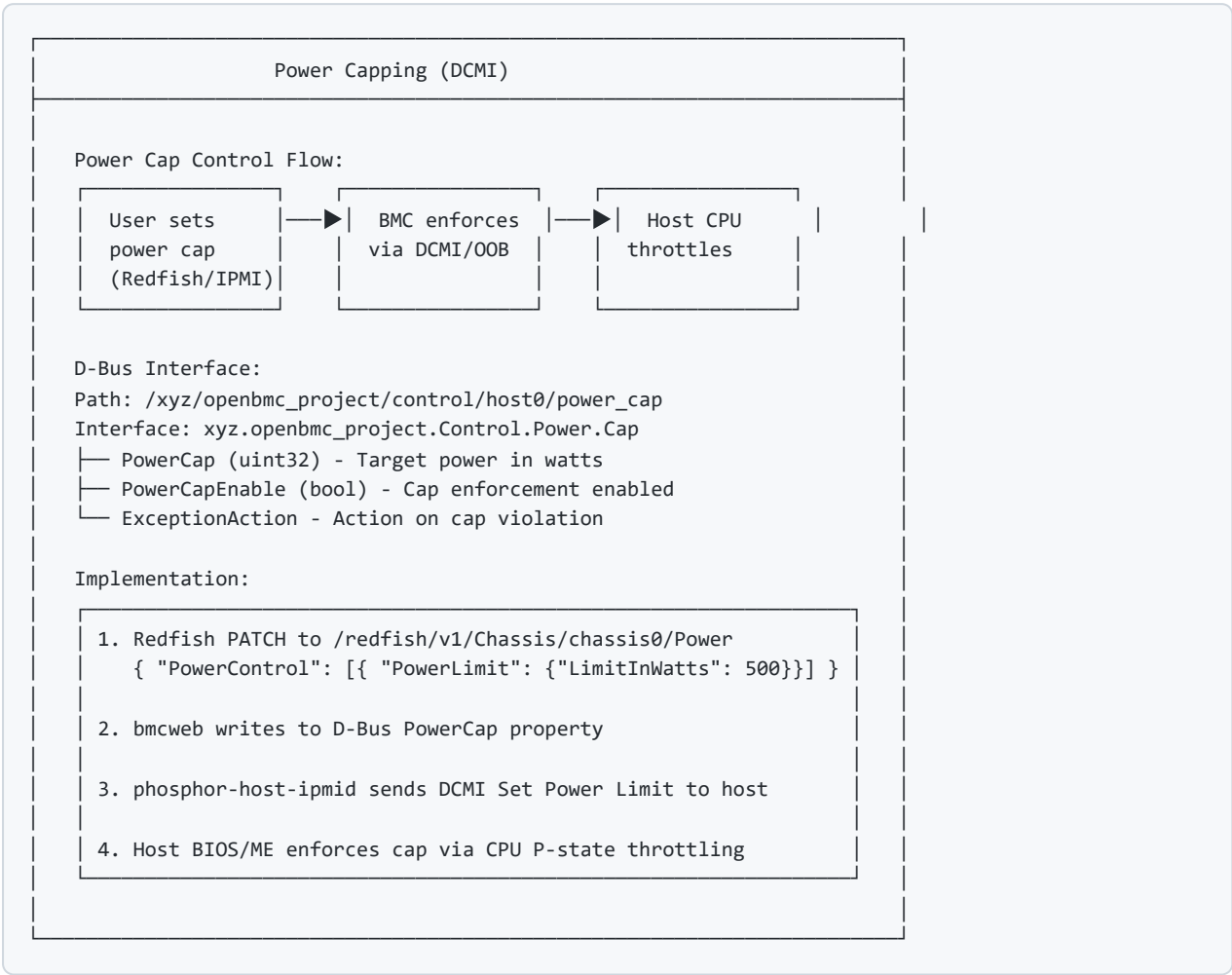
phosphor-regulators JSON Configuration:

```
{
  "chassis": [{
    "number": 1,
    "devices": [{
      "id": "vdd_cpu0",
      "is_regulator": true,
      "fru": "/system/chassis/motherboard/cpu0",
      "i2c_interface": {
        "bus": 5,
        "address": "0x40"
      },
    },
    "configuration": {
      "volts": 1.0,
      "rule_id": "set_voltage_rule"
    },
    "rails": [{
      "id": "vdd_cpu0_rail",
      "sensor_monitoring": {
        "rule_id": "read_sensors_rule"
      }
    }
  ]
}]
}
```

VR Sensor Mapping:

```
D-Bus Path: /xyz/openbmc_project/sensors/voltage/VDD_CPU0
PMBus Cmd: READ_VOUT (0x8B)
Conversion: Linear16 with VOUT_MODE exponent
```


Power Capping Implementation



Source Code Reference

Key implementation files:

Repository	File/Directory	Description
phosphor-power	phosphor-power-supply/	PSU monitoring daemon
phosphor-power	phosphor-regulators/	Voltage regulator control
phosphor-power	power-sequencer/	Power sequencing
x86-power-control	src/power_control.cpp	GPIO power control
phosphor-dbus-interfaces	xyz/openbmc_project/Control/Power/	Power interfaces

Examples

Working examples are available in the [examples/power](#) directory:

- [power-config.json](#) - Power configuration
- [regulators-config.json](#) - Voltage regulator configuration
- [psu-entity-manager.json](#) - PSU Entity Manager configuration

- `power-sequencer.sh` - Power sequencing script
-

References

- [phosphor-power](#)
 - [phosphor-regulators](#)
 - [PMBus Specification](#)
 - [x86-power-control](#) - Power control implementation for x86 servers
 - [State Management Design](#)
-

Tested on: OpenBMC master, QEMU romulus

User Manager Guide

Configure user accounts, privileges, and LDAP authentication on OpenBMC.

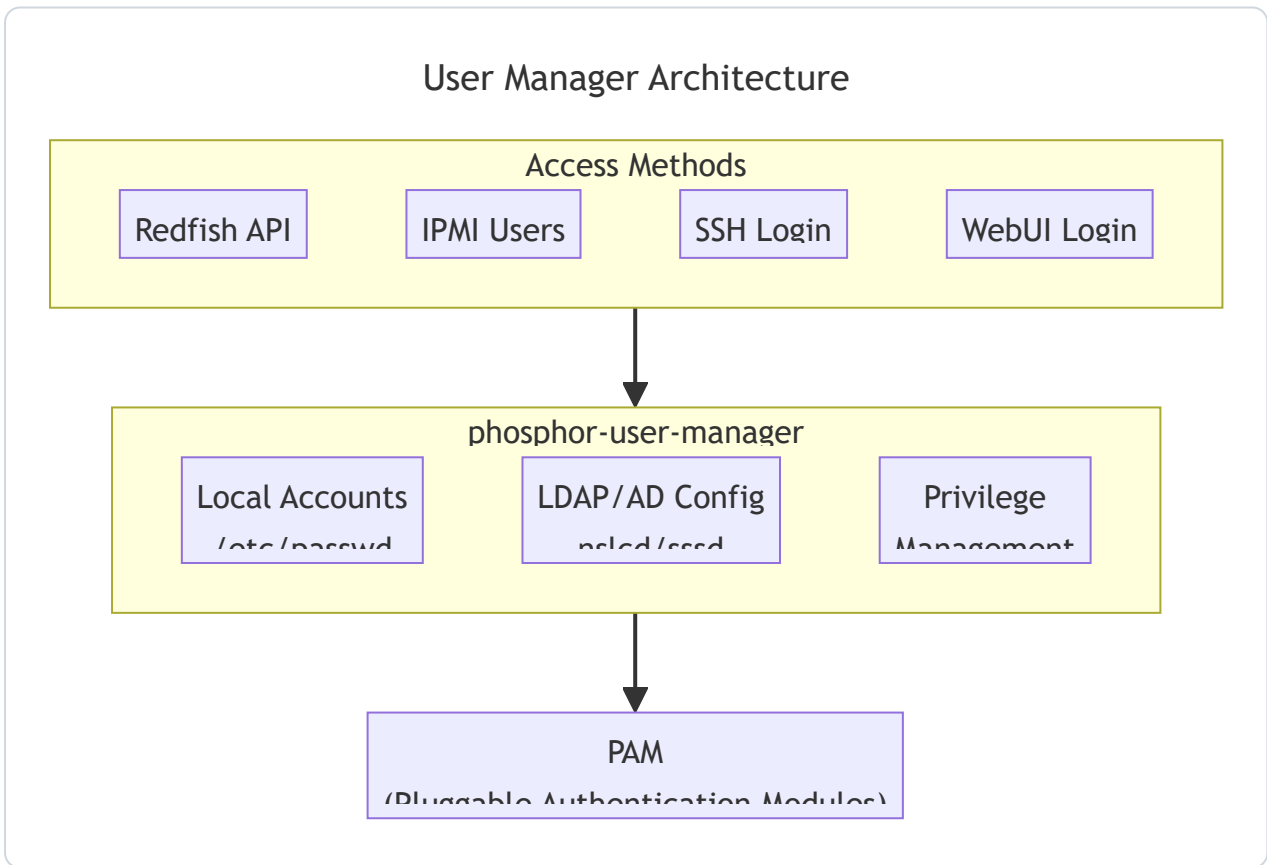
Table of Contents

1. [Overview](#)
2. [Setup & Configuration](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [Configuration Files](#)
 3. [Deploying Custom PAM Configuration via Yocto](#)
 4. [LDAP Configuration Persistence](#)
 5. [Runtime Configuration](#)
3. [Local User Management](#)
 1. [Default Users](#)
 2. [Create User via Redfish](#)
 3. [Create User via D-Bus](#)
 4. [Modify User](#)
 5. [Delete User](#)
4. [Privilege Roles](#)
 1. [Available Roles](#)
 2. [Role Permissions](#)
 3. [D-Bus Privilege Mapping](#)
5. [IPMI User Management](#)
 1. [Configure IPMI Users](#)
 2. [IPMI User Privileges](#)
6. [LDAP Configuration](#)
 1. [Enable LDAP Authentication](#)
 2. [LDAP Role Mapping](#)
 3. [D-Bus LDAP Configuration](#)
 4. [Test LDAP Connection](#)
7. [Active Directory](#)
 1. [Configure AD Authentication](#)
8. [Account Policies](#)
 1. [Configure Lockout](#)
 2. [Password Policy](#)
9. [Multi-Factor Authentication \(MFA\)](#)
 1. [How TOTP Enrollment Works](#)
 2. [OTP Verification](#)
 3. [MFA State and D-Bus](#)

- 4. [Per-User Bypass](#)
 - 10. [Session Management](#)
 - 1. [View Active Sessions](#)
 - 2. [Terminate Session](#)
 - 3. [Session Timeout](#)
 - 11. [Troubleshooting](#)
 - 1. [Login Failures](#)
 - 2. [LDAP Issues](#)
 - 3. [Account Locked](#)
 - 12. [Deep Dive](#)
 - 1. [PAM Integration](#)
 - 2. [LDAP Authentication Flow](#)
 - 3. [Password Hashing](#)
 - 4. [D-Bus Permission Enforcement](#)
 - 5. [Source Code Reference](#)
 - 13. [Examples](#)
 - 14. [References](#)
-

Overview

OpenBMC user management is handled by **phosphor-user-manager**, providing local accounts, role-based access control, and LDAP integration.



► ASCII-art version (for comparison)

Setup & Configuration

Build-Time Configuration (Yocto)

```
# In your machine .conf or local.conf

# Include user management
IMAGE_INSTALL:append = " \
    phosphor-user-manager \
"

# For LDAP support
IMAGE_INSTALL:append = " \
    nss-pam-ldapd \
    pam-plugin-ldap \
"

# Configure Meson options
EXTRA_OEMESON:pn-phosphor-user-manager = " \
    -Dldap=enabled \
"
```

Configuration Files

Config File	Install Path	Description
PAM auth config	<code>/etc/pam.d/common-auth</code>	Authentication module stack
PAM password config	<code>/etc/pam.d/common-password</code>	Password change rules
PAM account config	<code>/etc/pam.d/common-account</code>	Account validation
LDAP client config	<code>/etc/nslcd.conf</code>	nslcd LDAP client settings
LDAP NSS config	<code>/etc/nsswitch.conf</code>	Name service switch (adds ldap)
D-Bus policy	<code>/etc/dbus-1/system.d/phosphor-user-manager.conf</code>	D-Bus access control

Deploying Custom PAM Configuration via Yocto

To customize PAM authentication (e.g., add stricter password policies or modify the auth module order), create a bbappend for the pam recipe:

```
# meta-myplatform/recipes-phosphor/users/phosphor-user-manager/
# └─ files/
#   └─ common-auth
#   └─ common-password
# └─ phosphor-user-manager_%.bbappend

cat > phosphor-user-manager_%.bbappend << 'EOF'
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
SRC_URI += " \
    file://common-auth \
    file://common-password \
"

do_install:append() {
    # Deploy custom PAM configuration
    install -d ${D}${sysconfdir}/pam.d
    install -m 0644 ${WORKDIR}/common-auth ${D}${sysconfdir}/pam.d/
    install -m 0644 ${WORKDIR}/common-password ${D}${sysconfdir}/pam.d/
}
EOF
```

Example `common-auth` with lockout policy:

```
# /etc/pam.d/common-auth
auth required pam_tally2.so deny=5 unlock_time=300 onerr=fail
auth sufficient pam_unix.so nullok try_first_pass
auth sufficient pam_ldap.so use_first_pass
auth required pam_denial.so
```

LDAP Configuration Persistence

LDAP configuration set via Redfish/D-Bus is persisted to:

```
# LDAP config persistence (managed by phosphor-user-manager)
/var/lib/phosphor-user-manager/ldap.conf

# nslcd runtime config (generated from D-Bus settings)
/etc/nslcd.conf
```

To bake default LDAP settings into the image:

```
# meta-myplatform/recipes-phosphor/users/nss-pam-ldapd/
# └─ files/
#   └─ nslcd.conf
# └─ nss-pam-ldapd_%.bbappend

cat > nss-pam-ldapd_%.bbappend << 'EOF'
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
SRC_URI += "file://nslcd.conf"

do_install:append() {
    install -m 0600 ${WORKDIR}/nslcd.conf ${D}${sysconfdir}/
}
EOF
```

Runtime Configuration

```
# Check user manager service
systemctl status phosphor-user-manager

# View logs
journalctl -u phosphor-user-manager -f

# User configuration files
cat /etc/passwd
cat /etc/group

# Verify PAM config
cat /etc/pam.d/common-auth

# Verify LDAP client config
cat /etc/nslcd.conf
```

Local User Management

Default Users

Username	Default Password	Role
root	OpenBmc	Administrator

Create User via Redfish

```
# Create new user
curl -k -u root:OpenBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "UserName": "operator1",
    "Password": "SecurePass123!",
    "RoleId": "Operator",
    "Enabled": true
  }' \
  https://localhost/redfish/v1/AccountService/Accounts
```

Create User via D-Bus

```
# Create user
busctl call xyz.openbmc_project.User.Manager \
  /xyz/openbmc_project/user \
  xyz.openbmc_project.User.Manager \
  CreateUser ssasbs \
  "newuser" \
  3 "ipmi" "redfish" "ssh" \
  "priv-admin" \
  true
```

Modify User

```
# Change password via Redfish
curl -k -u root:OpenBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"Password": "NewPassword123!"}' \
  https://localhost/redfish/v1/AccountService/Accounts/operator1

# Change role
curl -k -u root:OpenBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"RoleId": "Administrator"}' \
  https://localhost/redfish/v1/AccountService/Accounts/operator1

# Enable/disable user
curl -k -u root:OpenBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"Enabled": false}' \
  https://localhost/redfish/v1/AccountService/Accounts/operator1
```

Delete User

```
# Via Redfish
curl -k -u root:OpenBmc -X DELETE \
  https://localhost/redfish/v1/AccountService/Accounts/operator1

# Via IPMI
ipmitool -I lanplus -H bmc-ip -U root -P OpenBmc user set name 3 ""
```

Privilege Roles

Available Roles

Role	Redfish	IPMI Priv	Description
Administrator	Administrator	4 (admin)	Full access
Operator	Operator	3 (operator)	Operations, no user mgmt
ReadOnly	ReadOnly	2 (user)	Read-only access
Callback	Callback	1	Limited callback access

Role Permissions

```
# View available roles
curl -k -u root:OpenBmc \
  https://localhost/redfish/v1/AccountService/Roles

# Get role details
curl -k -u root:OpenBmc \
  https://localhost/redfish/v1/AccountService/Roles/Operator
```


D-Bus Privilege Mapping

```
# View user privileges
busctl get-property xyz.openbmc_project.User.Manager \
  /xyz/openbmc_project/user/newuser \
  xyz.openbmc_project.User.Attributes \
  UserPrivilege
```

IPMI User Management

Configure IPMI Users

```
# List users
ipmitool user list

# Create user
ipmitool user set name 3 operator1
ipmitool user set password 3 Password123!
ipmitool user priv 3 3 1 # operator privilege, channel 1
ipmitool user enable 3

# Disable user
ipmitool user disable 3

# Delete user
ipmitool user set name 3 ""
```

IPMI User Privileges

Level	Name	Description
1	Callback	Minimal access
2	User	Read sensors, view logs
3	Operator	Power control, sensor config
4	Administrator	Full access
5	OEM	Vendor-specific

LDAP Configuration

Enable LDAP Authentication

```
# Configure LDAP via Redfish
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{
    "LDAP": {
      "ServiceEnabled": true,
      "ServiceAddresses": ["ldap://ldap.example.com"],
      "Authentication": {
        "AuthenticationType": "UsernameAndPassword",
        "Username": "cn=admin,dc=example,dc=com",
        "Password": "ldappassword"
      },
      "LDAPService": {
        "SearchSettings": {
          "BaseDistinguishedNames": ["dc=example,dc=com"],
          "UsernameAttribute": "uid",
          "GroupsAttribute": "memberOf"
        }
      }
    }
  }' \
  https://localhost/redfish/v1/AccountService
```

LDAP Role Mapping

```
# Map LDAP group to role
curl -k -u root:openBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "RemoteGroup": "cn=admins,ou=groups,dc=example,dc=com",
    "LocalRole": "Administrator"
  }' \
  https://localhost/redfish/v1/AccountService/LDAP/RemoteRoleMapping
```

D-Bus LDAP Configuration

```
# Create LDAP config
busctl call xyz.openbmc_project.User.Manager \
  /xyz/openbmc_project/user/ldap \
  xyz.openbmc_project.User.Ldap.Create \
  Create ssssa{ss} \
  "ldap://ldap.example.com" \
  "cn=admin,dc=example,dc=com" \
  "dc=example,dc=com" \
  "uid" \
  "OpenLDAP" \
  0
```

Test LDAP Connection

```
# Test authentication
ldapsearch -x -H ldap://ldap.example.com \
  -D "cn=admin,dc=example,dc=com" \
  -w password \
  -b "dc=example,dc=com" \
  "(uid=testuser)"
```

Active Directory

Configure AD Authentication

```
# Configure Active Directory
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{
    "ActiveDirectory": {
      "ServiceEnabled": true,
      "ServiceAddresses": ["ldap://ad.example.com"],
      "Authentication": {
        "AuthenticationType": "UsernameAndPassword",
        "Username": "admin@example.com",
        "Password": "adpassword"
      },
      "LDAPService": {
        "SearchSettings": {
          "BaseDistinguishedNames": ["DC=example,DC=com"],
          "UsernameAttribute": "sAMAccountName",
          "GroupsAttribute": "memberOf"
        }
      }
    }
  }' \
  https://localhost/redfish/v1/AccountService
```

Account Policies

Configure Lockout

```
# Set account lockout policy
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{
    "AccountLockoutThreshold": 5,
    "AccountLockoutDuration": 300,
    "AccountLockoutCounterResetAfter": 60
  }' \
  https://localhost/redfish/v1/AccountService
```

Password Policy

```
# Set minimum password length
curl -k -u root:OpenBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{
    "MinPasswordLength": 12
  }' \
  https://localhost/redfish/v1/AccountService
```

Multi-Factor Authentication (MFA)

Beyond passwords, LDAP, and Active Directory, **phosphor-user-manager** supports **multi-factor authentication (MFA)** using **TOTP** (Time-based One-Time Password), which is compatible with Google Authenticator and equivalent authenticator apps. When MFA is enabled for a user, login requires both the account password and a current one-time passcode (OTP).

How TOTP Enrollment Works

1. **Secret generation** — enabling TOTP for a user runs the `google-authenticator` utility to produce a per-user secret. OpenBMC invokes it non-interactively in time-based mode (`google-authenticator -s <path> -u -W -Q NONE -t -f -d -C`), so no QR code or interactive prompt is generated.
2. **Provisional secret** — the new secret is first written to a temporary file (a `.google_authenticator.tmp` under the user's home) and is **not yet active**.
3. **Verification makes it permanent** — the user must submit one valid OTP. Only after that OTP verifies successfully is the secret promoted (renamed) to its live location, `~/.google_authenticator`. A failed verification — or disabling MFA — removes the temporary secret, so a half-finished enrollment never locks the account.

OTP Verification

One-time passcodes are checked through a **dedicated PAM service**, `mfa_pam` (installed to `/etc/pam.d/mfa_pam`). The internal `Totp::verify(username, token)` helper opens a PAM conversation against that service, answers its `Verification code:` prompt with the supplied token, and reports success only when PAM accepts the code. Isolating OTP checks in their own PAM stack lets MFA layer on top of the existing `common-auth` password flow without modifying it.

MFA State and D-Bus

MFA configuration lives on the `xyz.openbmc_project.User.Manager` service, exposed through the `xyz.openbmc_project.User.MultiFactorAuthConfiguration` interface — whose `Enabled` property selects the active MFA type (for example `GoogleAuthenticator` or `None`) — and is mirrored to the Redfish `AccountService`. The selected MFA type and related state are **persisted as JSON** (via the manager's `JsonSerializer`), so the setting survives a BMC reboot.

```
# Inspect the user manager object and its MFA-related interfaces
busctl introspect xyz.openbmc_project.User.Manager \
  /xyz/openbmc_project/user
```

Per-User Bypass

For temporary exceptions — a break-glass account, or a user who has lost their authenticator — phosphor-user-manager supports a per-user **bypass protocol**. Bypassing a protocol (such as `GoogleAuthenticator`) exempts that user from the corresponding factor and clears their stored secret. The bypass selection is itself persisted as JSON, so it too survives a reboot.

MFA secrets are sensitive. A user's `~/.google_authenticator` secret is the shared key for their second factor. It is stored with restricted permissions under the user's home directory and must never be logged, exported, or copied off the BMC — treat it with the same care as a private key or password hash.

Session Management

View Active Sessions

```
# List sessions
curl -k -u root:openBmc \
  https://localhost/redfish/v1/SessionService/Sessions
```

Terminate Session

```
# Delete specific session
curl -k -u root:openBmc -X DELETE \
  https://localhost/redfish/v1/SessionService/Sessions/session_id
```

Session Timeout

```
# Configure session timeout
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"SessionTimeout": 1800}' \
  https://localhost/redfish/v1/SessionService
```

Troubleshooting

Login Failures

```
# Check PAM logs
journalctl | grep pam

# Check user exists
cat /etc/passwd | grep username

# Verify user enabled
busctl get-property xyz.openbmc_project.User.Manager \
  /xyz/openbmc_project/user/username \
  xyz.openbmc_project.User.Attributes \
  UserEnabled
```

LDAP Issues

```
# Check LDAP service
systemctl status nslcd

# Test LDAP bind
ldapwhoami -x -H ldap://ldap.example.com \
  -D "uid=user,dc=example,dc=com" -W

# Check LDAP configuration
busctl introspect xyz.openbmc_project.User.Manager \
  /xyz/openbmc_project/user/ldap
```

Account Locked

```
# Check lockout status (via PAM tally)
pam_tally2 --user=username

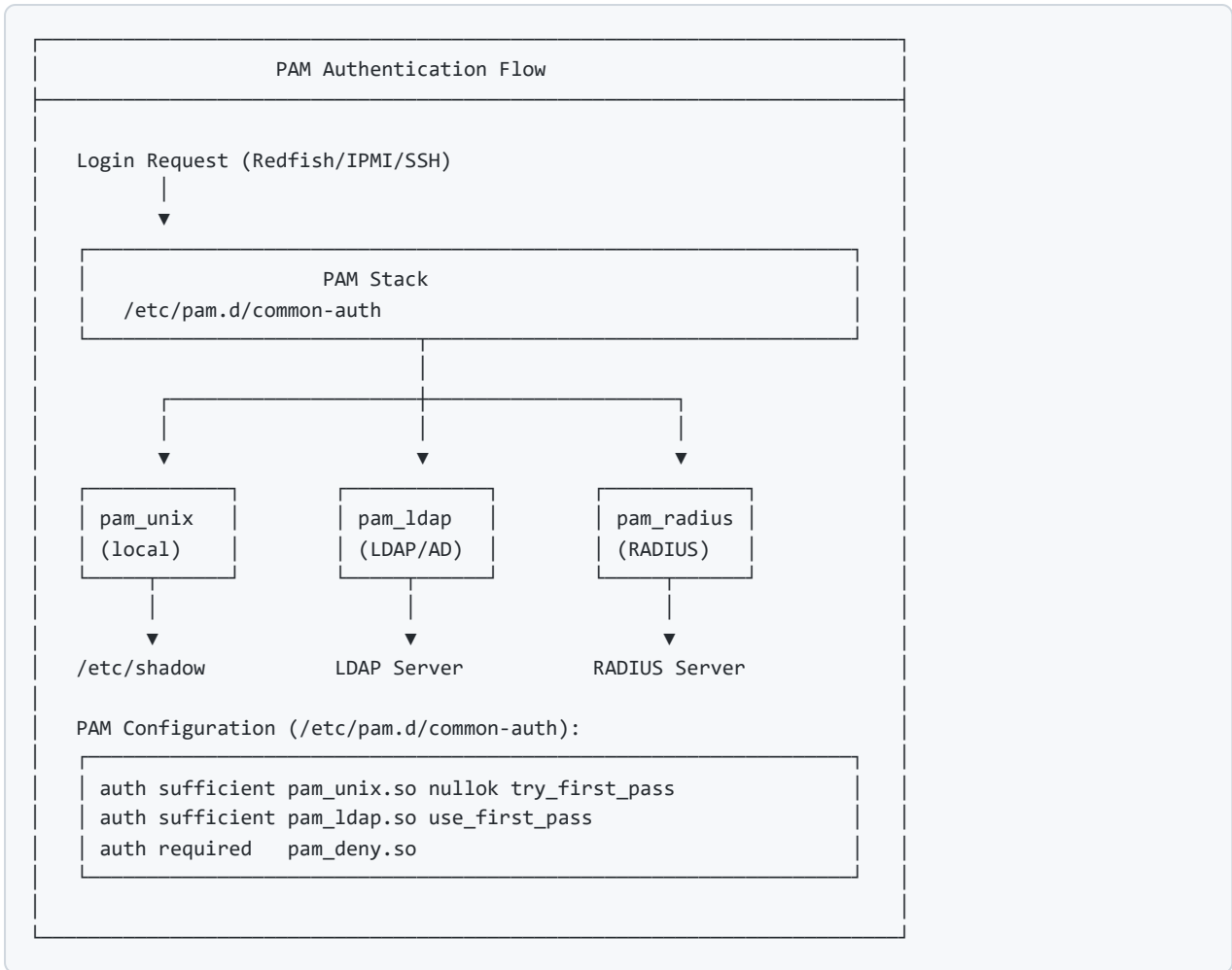
# Reset lockout
pam_tally2 --user=username --reset
```

Deep Dive

Advanced implementation details for user management developers.

PAM Integration

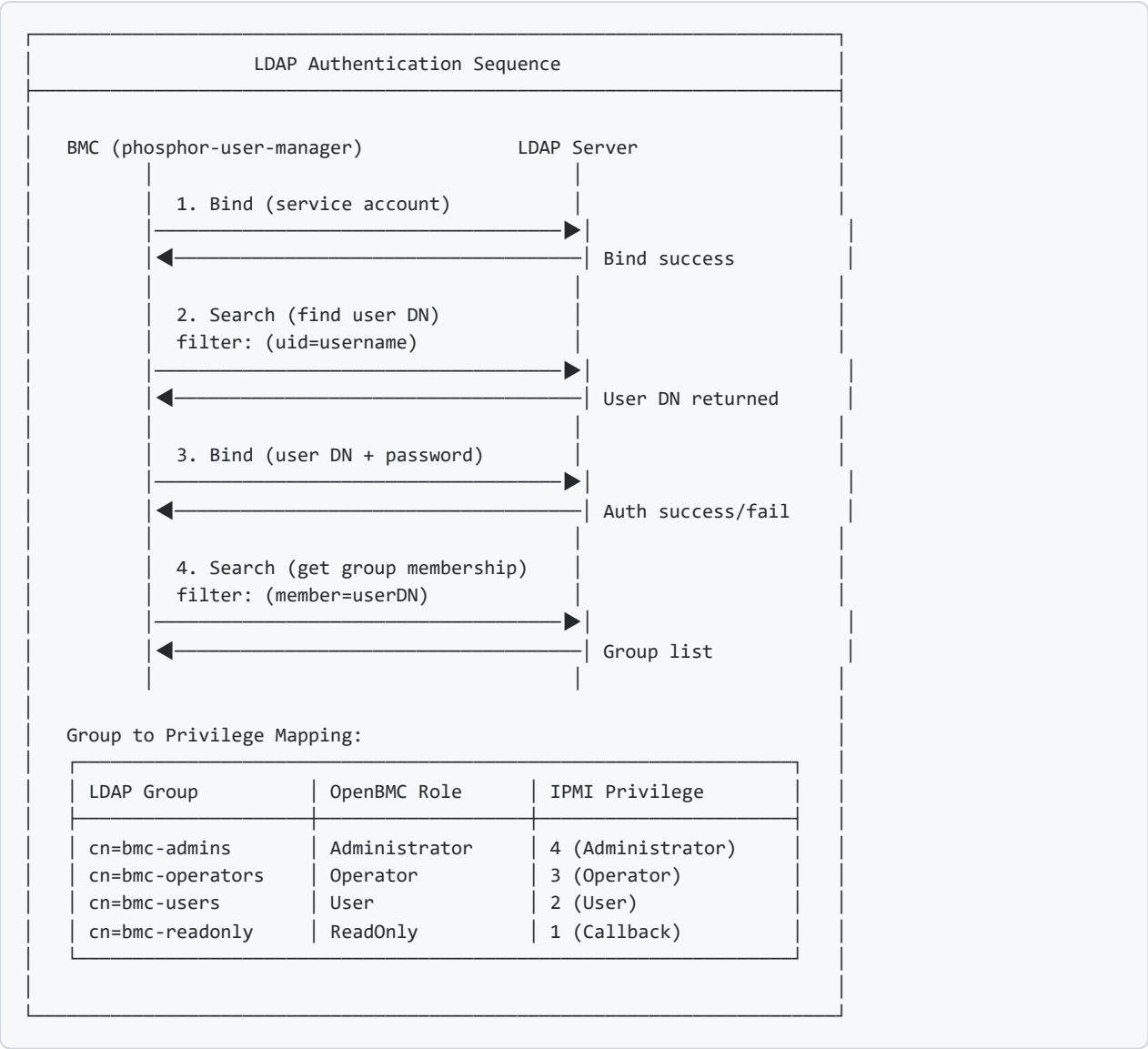
OpenBMC uses PAM (Pluggable Authentication Modules) for authentication:



PAM module flow:

- **sufficient**: If module succeeds, skip remaining; if fails, continue
- **required**: Must pass, but continue checking other modules
- **requisite**: Must pass, fail immediately if not

LDAP Authentication Flow



Password Hashing

Password Storage

/etc/shadow format:

username:\$6\$salt\$hash:lastchange:min:max:warn:inactive:expire:

Hash Types:

- |— \$1\$ = MD5 (deprecated, insecure)
- |— \$5\$ = SHA-256
- |— \$6\$ = SHA-512 (default, recommended)

Password Change Flow:

1. Redfish PATCH /AccountService/Accounts/{id}
 { "Password": "newPassword" }
2. bmcweb calls D-Bus:
 xyz.openbmc_project.User.Manager.RenameUser() or
 org.freedesktop.Accounts.User.SetPassword()
3. phosphor-user-manager:
 - Validates password policy (length, complexity)
 - Calls pam_chauthtok() to update /etc/shadow
 - Emits PropertiesChanged signal

D-Bus Permission Enforcement

D-Bus Access Control

/etc/dbus-1/system.d/ policy files control D-Bus access:

Example: phosphor-user-manager.conf

```
<policy user="root">
  <allow own="xyz.openbmc_project.User.Manager"/>
  <allow send_destination="xyz.openbmc_project.User.Manager"/>
</policy>

<policy context="default">
  <deny send_destination="xyz.openbmc_project.User.Manager"
        send_interface="xyz.openbmc_project.User.Manager"/>
  <allow send_destination="xyz.openbmc_project.User.Manager"
        send_interface="org.freedesktop.DBus.Properties"
        send_member="Get"/>
</policy>
```

Privilege Enforcement in bmcweb:

```
// Route with privilege requirement
BMCWEB_ROUTE(app, "/redfish/v1/AccountService/Accounts/<str>/")
  .privileges(redfish::privileges::patchAccount)
  .methods(boost::beast::http::verb::patch)(handlePatch);

// privileges::patchAccount requires ConfigureUsers
```

Source Code Reference

Key implementation files in [phosphor-user-manager](#):

File	Description
user_mgr.cpp	Main user manager implementation
users.cpp	Individual user object handling
ldap_config.cpp	LDAP configuration management
ldap_mapper.cpp	LDAP group to privilege mapping
shadowlock.cpp	/etc/shadow file locking
phosphor-ldap-conf.cpp	LDAP config daemon

Examples

Working examples are available in the [examples/user-manager](#) directory:

- `user-management.sh` - User CRUD operations script
 - `ldap-config.json` - LDAP configuration example
-

References

- [phosphor-user-manager](#)
 - [Redfish AccountService](#)
 - [OpenBMC User Management](#)
-

Tested on: OpenBMC master, QEMU romulus

Network Guide

Configure BMC network settings using phosphor-networkd.

Table of Contents

1. [Overview](#)
2. [Setup & Configuration](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [Meson Build Options](#)
 3. [Configuration Files](#)
 4. [Deploying Default Network Configuration via Yocto](#)
 5. [Runtime Configuration](#)
3. [IP Address Configuration](#)
 1. [View Current Configuration](#)
 2. [Via Redfish](#)
 3. [Configure Static IP](#)
 4. [Configure DHCP](#)
 5. [Delete IP Address](#)
4. [IPv6 Configuration](#)
 1. [Enable IPv6](#)
 2. [Enable SLAAC](#)
5. [VLAN Configuration](#)
 1. [Create VLAN](#)
 2. [Configure VLAN IP](#)
 3. [Delete VLAN](#)
6. [DNS Configuration](#)
 1. [Configure DNS Servers](#)
 2. [View DNS Configuration](#)
7. [Hostname Configuration](#)
 1. [Set Hostname](#)
 2. [Set Domain Name](#)
8. [MAC Address Configuration](#)
 1. [View MAC Address](#)
 2. [Set MAC Address](#)
9. [Network Protocol Configuration](#)
 1. [Enable/Disable Protocols](#)
10. [NCSI Management](#)
 1. [Discover Packages and Channels](#)
 2. [Common Options](#)

- 3. [Send Raw, OEM, and Dump Commands](#)
 - 11. [IPMI LAN Configuration](#)
 - 1. [View LAN Configuration](#)
 - 2. [Configure LAN Channel](#)
 - 12. [Hypervisor Network Synchronization](#)
 - 1. [Backed by BIOS Attributes](#)
 - 13. [Troubleshooting](#)
 - 1. [No Network Connectivity](#)
 - 2. [DHCP Not Working](#)
 - 3. [DNS Resolution Issues](#)
 - 4. [Configuration Not Persisting](#)
 - 14. [Configuration Persistence](#)
 - 15. [Deep Dive](#)
 - 1. [systemd-networkd Integration](#)
 - 2. [DHCP Transaction Flow](#)
 - 3. [VLAN Implementation](#)
 - 4. [Link State Monitoring](#)
 - 5. [Source Code Reference](#)
 - 16. [Examples](#)
 - 17. [References](#)
-

Overview

phosphor-networkd manages BMC network configuration, providing D-Bus interfaces for IP configuration, VLAN, DNS, and network settings.



Network Architecture

Configuration APIs

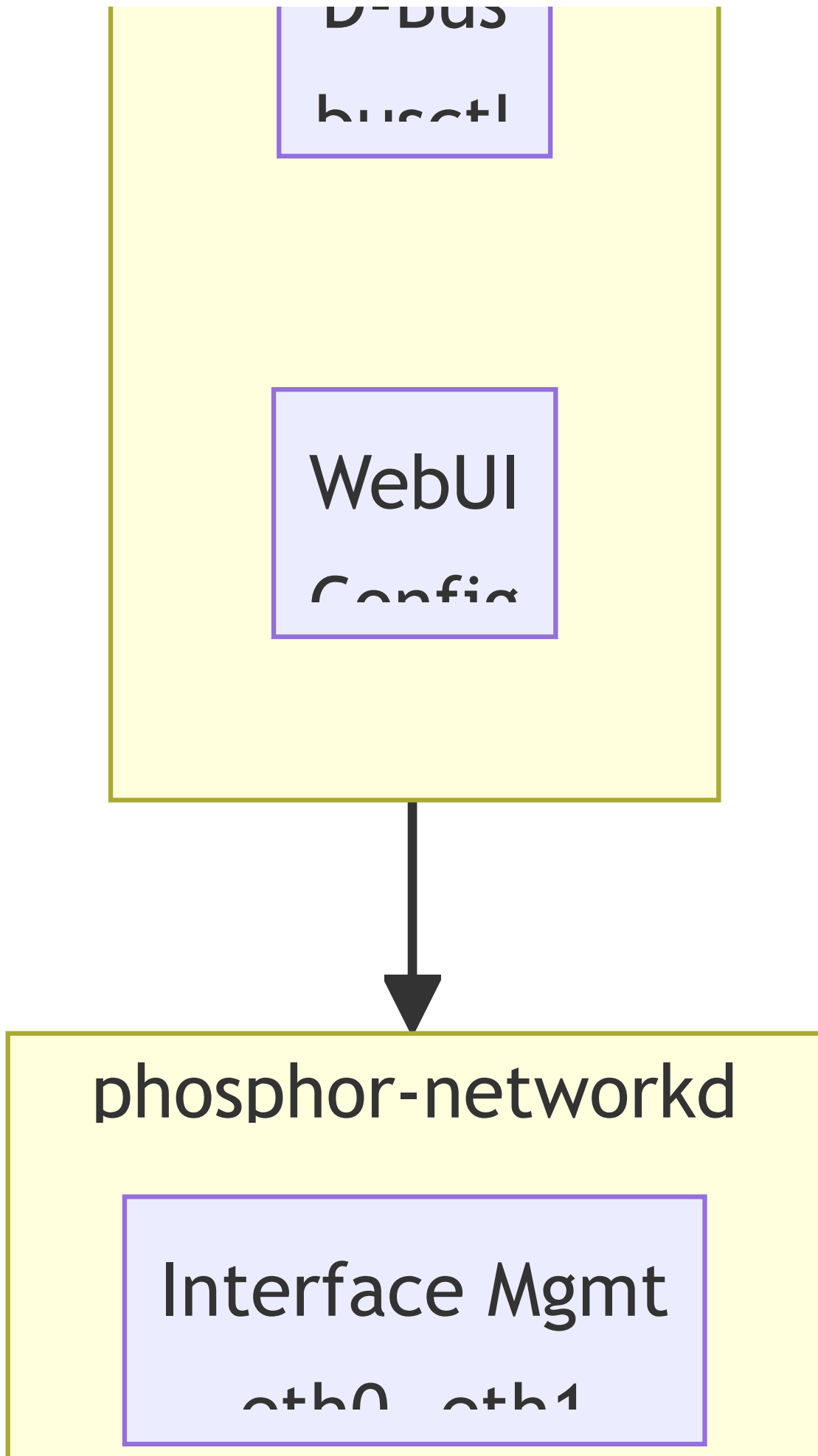
Redfish

API

IPMI

Network

D-Bus



IP/VLAN Config

static dhcp

DNS/DHCP

Hostname



systemd-networkd

(network files)

► ASCII-art version (for comparison)

Setup & Configuration

Build-Time Configuration (Yocto)

```
# In your machine .conf or local.conf

# Include network management
IMAGE_INSTALL:append = " \
    phosphor-network \
"

# Configure Meson options
EXTRA_OEMESON:pn-phosphor-network = " \
    -Ddefault-link-local-autoconf=true \
    -Ddefault-ipv6-accept-ra=true \
"
```

Meson Build Options

Option	Default	Description
<code>default-link-local-autoconf</code>	true	Enable link-local IPv4
<code>default-ipv6-accept-ra</code>	true	Accept IPv6 router advertisements
<code>sync-mac</code>	true	Sync MAC with hardware
<code>persist-mac</code>	false	Persist MAC across reboots

Configuration Files

phosphor-networkd translates D-Bus settings to systemd-networkd `.network` files. Runtime configuration is persisted automatically. To bake default settings into the image, deploy `.network` files via Yocto.

Config File	Path	Description
Network config	<code>/etc/systemd/network/00-bmc-eth0.network</code>	Main BMC interface (DHCP or static)
VLAN netdev	<code>/etc/systemd/network/00-bmc-eth0.100.netdev</code>	VLAN device definition
VLAN config	<code>/etc/systemd/network/10-bmc-eth0.100.network</code>	VLAN interface config
Resolved config	<code>/etc/systemd/resolved.conf</code>	DNS resolver settings

Deploying Default Network Configuration via Yocto

To bake a default static IP or DHCP configuration into the image:

```
# meta-myplatform/recipes-phosphor/network/phosphor-network/
# └─ files/
#   └─ 00-bmc-eth0.network
# └─ phosphor-network_%.bbappend

cat > phosphor-network_%.bbappend << 'EOF'
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
SRC_URI += "file://00-bmc-eth0.network"

do_install:append() {
    install -d ${D}${sysconfdir}/systemd/network
    install -m 0644 ${WORKDIR}/00-bmc-eth0.network \
        ${D}${sysconfdir}/systemd/network/
}
EOF
```

Example `00-bmc-eth0.network` for DHCP:

```
[Match]
Name=eth0

[Network]
DHCP=ipv4
LinkLocalAddressing=yes
IPv6AcceptRA=true
```

Example for static IP:

```
[Match]
Name=eth0

[Network]
Address=192.168.1.100/24
Gateway=192.168.1.1
DNS=8.8.8.8
DNS=8.8.4.4
LinkLocalAddressing=no
```

Runtime changes via Redfish or D-Bus will overwrite these files. The Yocto-deployed file serves as the factory default. After a factory reset (`/usr/share/phosphor-bmc-code-mgmt/scripts/factory-reset.sh`), the image-baked config is restored.

Runtime Configuration

```
# Check network service
systemctl status systemd-networkd
systemctl status phosphor-network

# View network configuration files
ls /etc/systemd/network/

# View current network state
networkctl status

# View active config for eth0
cat /etc/systemd/network/00-bmc-eth0.network
```

IP Address Configuration

View Current Configuration

```
# Via D-Bus
busctl tree xyz.openbmc_project.Network

# Get interface info
busctl introspect xyz.openbmc_project.Network \
    /xyz/openbmc_project/network/eth0

# Via ip command
ip addr show eth0
```

Via Redfish

```
# Get ethernet interfaces
curl -k -u root:0penBmc \
    https://localhost/redfish/v1/Managers/bmc/EthernetInterfaces

# Get specific interface
curl -k -u root:0penBmc \
    https://localhost/redfish/v1/Managers/bmc/EthernetInterfaces/eth0
```

Configure Static IP

```
# Via Redfish
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{
    "IPv4Addresses": [
      {
        "Address": "192.168.1.100",
        "SubnetMask": "255.255.255.0",
        "Gateway": "192.168.1.1",
        "AddressOrigin": "Static"
      }
    ]
  }' \
  https://localhost/redfish/v1/Managers/bmc/EthernetInterfaces/eth0

# Via D-Bus
busctl call xyz.openbmc_project.Network \
  /xyz/openbmc_project/network/eth0 \
  xyz.openbmc_project.Network.IP.Create \
  IP ssys \
  "xyz.openbmc_project.Network.IP.Protocol.IPv4" \
  "192.168.1.100" \
  24 \
  "192.168.1.1"
```

Configure DHCP

```
# Enable DHCP via Redfish
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"DHCPv4": {"DHCPEnabled": true}}' \
  https://localhost/redfish/v1/Managers/bmc/EthernetInterfaces/eth0

# Via D-Bus
busctl set-property xyz.openbmc_project.Network \
  /xyz/openbmc_project/network/eth0 \
  xyz.openbmc_project.Network.EthernetInterface \
  DHCPEnabled s "xyz.openbmc_project.Network.EthernetInterface.DHCPConf.v4"
```

Delete IP Address

```
# Get IP address object path
busctl tree xyz.openbmc_project.Network | grep ipv4

# Delete via D-Bus
busctl call xyz.openbmc_project.Network \
  /xyz/openbmc_project/network/eth0/ipv4/abc123 \
  xyz.openbmc_project.Object.Delete \
  Delete
```

Static neighbors and static gateways follow the same child-object pattern as IP addresses. A static ARP/NDP neighbor entry ([neighbor.cpp](#)) and a static default gateway ([static_gateway.cpp](#)) each appear as their own D-Bus object beneath `/xyz/openbmc_project/network/eth0` , and each exposes `xyz.openbmc_project.Object.Delete` . Remove one exactly like an IP address: use `busctl tree` to find its object path, then call `Delete` on it.

IPv6 Configuration

Enable IPv6

```
# Configure IPv6 static address
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{
    "IPv6Addresses": [
      {
        "Address": "2001:db8::100",
        "PrefixLength": 64,
        "AddressOrigin": "Static"
      }
    ]
  }' \
  https://localhost/redfish/v1/Managers/bmc/EthernetInterfaces/eth0
```

Enable SLAAC

```
# Enable IPv6 router advertisement
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"IPv6StaticAddresses": [], "StatelessAddressAutoConfig": {"IPv6AutoConfigEnabled": true}}' \
  https://localhost/redfish/v1/Managers/bmc/EthernetInterfaces/eth0
```

VLAN Configuration

Create VLAN

```
# Via Redfish
curl -k -u root:openBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "VLANEnable": true,
    "VLANId": 100
  }' \
  https://localhost/redfish/v1/Managers/bmc/EthernetInterfaces/eth0/VLANs

# Via D-Bus
busctl call xyz.openbmc_project.Network \
  /xyz/openbmc_project/network \
  xyz.openbmc_project.Network.VLAN.Create \
  VLAN su "eth0" 100
```

Configure VLAN IP

```
# Configure IP on VLAN interface
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{
    "IPv4Addresses": [
      {
        "Address": "10.0.100.10",
        "SubnetMask": "255.255.255.0",
        "Gateway": "10.0.100.1",
        "AddressOrigin": "Static"
      }
    ]
  }' \
  https://localhost/redfish/v1/Managers/bmc/EthernetInterfaces/eth0_100
```

Delete VLAN

```
curl -k -u root:openBmc -X DELETE \
  https://localhost/redfish/v1/Managers/bmc/EthernetInterfaces/eth0_100
```

DNS Configuration

Configure DNS Servers

```
# Via Redfish
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{
    "StaticNameServers": ["8.8.8.8", "8.8.4.4"]
  }' \
  https://localhost/redfish/v1/Managers/bmc/EthernetInterfaces/eth0

# Via D-Bus
busctl set-property xyz.openbmc_project.Network \
  /xyz/openbmc_project/network/eth0 \
  xyz.openbmc_project.Network.EthernetInterface \
  StaticNameServers as 2 "8.8.8.8" "8.8.4.4"
```

View DNS Configuration

```
# Check current DNS
cat /etc/resolv.conf

# Via D-Bus
busctl get-property xyz.openbmc_project.Network \
  /xyz/openbmc_project/network/eth0 \
  xyz.openbmc_project.Network.EthernetInterface \
  Nameservers
```

Hostname Configuration

Set Hostname

```
# Via Redfish
curl -k -u root:OpenBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"HostName": "my-bmc"}' \
  https://localhost/redfish/v1/Managers/bmc/NetworkProtocol

# Via D-Bus
busctl set-property xyz.openbmc_project.Network \
  /xyz/openbmc_project/network/config \
  xyz.openbmc_project.Network.SystemConfiguration \
  HostName s "my-bmc"
```

Set Domain Name

```
busctl set-property xyz.openbmc_project.Network \
  /xyz/openbmc_project/network/config \
  xyz.openbmc_project.Network.SystemConfiguration \
  DefaultGateway s "192.168.1.1"
```

First-boot unique hostname. On first boot the hostname manager (`hostname_manager.cpp`) derives a unique suffix from the BMC serial number — read from the inventory `xyz.openbmc_project.Inventory.Item.Bmc` / `Decorator.Asset` `SerialNumber` property — and falls back to the MAC address when no serial is available. It applies the result once as `<hostname>-<suffix>` via `org.freedesktop.hostname1` `SetStaticHostname`, then writes a marker file so later boots leave any user-set hostname untouched.

MAC Address Configuration

View MAC Address

```
# Via ip command
ip link show eth0

# Via D-Bus
busctl get-property xyz.openbmc_project.Network \
  /xyz/openbmc_project/network/eth0 \
  xyz.openbmc_project.Network.MACAddress \
  MACAddress
```

Set MAC Address

```
# Via Redfish
curl -k -u root:OpenBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"MACAddress": "AA:BB:CC:DD:EE:FF"}' \
  https://localhost/redfish/v1/Managers/bmc/EthernetInterfaces/eth0
```

Inventory-assigned MAC. Where a platform stores a MAC address in inventory, `inventory_mac.cpp` copies it onto the matching interface on first boot. It reads the `MACAddress` property from `xyz.openbmc_project.Inventory`

`y.Item.NetworkInterface`, maps inventory paths to interfaces via `/usr/share/network/config.json`, and records completion with a `/var/lib/network/firstBoot_<interface>` marker so the assignment runs only once.

Network Protocol Configuration

Enable/Disable Protocols

```
# Configure network protocols
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{
    "HTTPS": {"ProtocolEnabled": true, "Port": 443},
    "SSH": {"ProtocolEnabled": true, "Port": 22},
    "IPMI": {"ProtocolEnabled": true, "Port": 623}
  }' \
  https://localhost/redfish/v1/Managers/bmc/NetworkProtocol
```

NCSI Management

Network Controller Sideband Interface (NC-SI) lets the BMC share a network controller with the host over a sideband channel (RBT, or increasingly MCTP). `phosphor-networkd` ships two command-line helpers for talking to NC-SI-capable NICs:

Tool	Backend	Typical use
<code>ncsi-netlink</code>	Netlink only	Package/channel selection through the kernel NC-SI driver
<code>ncsi-cmd</code>	Netlink or MCTP	Discovery, raw/OEM commands, and firmware dumps

Discover Packages and Channels

`ncsi-cmd discover` walks every package (0-7) and channel (0-31), issuing *Clear Initial State* and *Get Capabilities* to enumerate what the NIC exposes:

```
# Discover over the kernel netlink backend (NIC selected by interface)
ncsi-cmd -i eth0 discover

# Discover over MCTP (endpoint ID 10 on the default network)
ncsi-cmd -m 10 discover
```

Common Options

Option	Description
<code>-p, --package <n></code>	Target package index (required for non-discovery commands)
<code>-c, --channel <n></code>	Target channel index
<code>-i, --interface <ifindex></code>	Select the NIC by network interface (netlink backend)
<code>-m, --mctp [NET,]<EID></code>	Select the NIC by MCTP endpoint (MCTP backend)
<code>-v, --verbose</code>	Verbose output

Every subcommand except `discover` needs a `--package` / `-p` value plus exactly one interface type — `-i` (netlink) **or** `-m` (MCTP), never both.

Send Raw, OEM, and Dump Commands

```
# raw <type> [hex payload...] – send an arbitrary NC-SI command
ncsi-cmd -p 0 -c 0 -i eth0 raw 0x15

# oem [hex payload...] – send an OEM command (NC-SI type 0x50)
ncsi-cmd -p 0 -c 0 -i eth0 oem 0x00 0x00 0x00

# Capture NC-SI core/crash dumps to a file
ncsi-cmd -p 0 -i eth0 core-dump /tmp/ncsi-core.bin
ncsi-cmd -p 0 -i eth0 crash-dump /tmp/ncsi-crash.bin
```

The netlink backend drives the in-kernel NC-SI driver, so `-i` takes a normal interface. The MCTP backend instead talks to the NIC as an MCTP endpoint (`-m`), reusing the MCTP stack rather than the network driver. `ncsi-netlink` is the older, netlink-only helper (for example `ncsi-netlink -x 3 -p 0 -i`); `ncsi-cmd` supersedes it and adds the MCTP path.

IPMI LAN Configuration

View LAN Configuration

```
ipmitool lan print
```

Configure LAN Channel

```
# Set static IP
ipmitool lan set 1 ipsrc static
ipmitool lan set 1 ipaddr 192.168.1.100
ipmitool lan set 1 netmask 255.255.255.0
ipmitool lan set 1 defgw ipaddr 192.168.1.1

# Enable DHCP
ipmitool lan set 1 ipsrc dhcp
```

Hypervisor Network Synchronization

This feature is **IBM / PowerVM-specific**. It lives under `src/ibm/hypervisor-network-mgr-src/` in phosphor-networkd and is built only for IBM Power systems, where the BMC stages network settings on behalf of the PowerVM hypervisor (its “virtual management interface”, VMI). It is not present on typical x86 or generic ARM platforms.

On IBM Power systems a second daemon, **HypNetworkMgr**, publishes the D-Bus service `xyz.openbmc_project.Network.Hypervisor`. It mirrors the familiar network object model for the hypervisor’s own interfaces so management tooling can pre-stage the hypervisor’s IP configuration before the host boots:

D-Bus object	Interface class	Role
Hypervisor ethernet interface (<code>eth0</code> , <code>eth1</code>)	<code>HypEthInterface</code>	Per-interface container for staged hypervisor IP addresses
Hypervisor IP address	<code>HypIPAddress</code>	A single staged IPv4 address / gateway / prefix
Hypervisor system config	<code>HypSysConfig</code>	Hostname and other system-wide settings

Backed by BIOS Attributes

Instead of writing `systemd-networkd` files, HypNetworkMgr persists every value as a **BIOS attribute** through `xyz.openbmc_project.BIOSConfigManager` (interface `xyz.openbmc_project.BIOSConfig.Manager`). The attributes use a `vmi_`-prefixed naming scheme, for example:

BIOS attribute	Meaning
<code>vmi_<intf>_ipv4_ipaddr</code>	Staged IPv4 address
<code>vmi_<intf>_ipv4_gateway</code>	Staged default gateway
<code>vmi_<intf>_ipv4_prefix_length</code>	Prefix length
<code>vmi_<intf>_ipv4_method</code>	Addressing method (e.g. <code>IPv4Static</code>)
<code>vmi_hostname</code>	Hypervisor hostname

The hypervisor reads these attributes as it initializes, so the BMC and hypervisor stay in sync without a running host.

```
# List the hypervisor network objects (interfaces, staged IPs, system config)
busctl tree xyz.openbmc_project.Network.Hypervisor

# Introspect one of the objects listed above
busctl introspect xyz.openbmc_project.Network.Hypervisor <object-path>
```

`HypIPAddress` properties (address, gateway, prefix length, origin, type) are **immutable after the object is created** — every setter rejects updates with a `NotAllowed` error (“Property update is not allowed”). To change a staged address, delete the object via `xyz.openbmc_project.Object.Delete` and create a new one rather than patching it in place.

Troubleshooting

No Network Connectivity

```
# Check interface status
ip link show eth0
networkctl status eth0

# Check for IP address
ip addr show eth0

# Check routing table
ip route

# Ping gateway
ping -c 3 192.168.1.1
```

DHCP Not Working

```
# Check DHCP client
journalctl -u systemd-networkd | grep DHCP

# Force DHCP renewal
networkctl renew eth0
```

DNS Resolution Issues

```
# Check DNS configuration
cat /etc/resolv.conf

# Test DNS resolution
nslookup example.com
```

Configuration Not Persisting

```
# Check network configuration files
ls -la /etc/systemd/network/

# Verify phosphor-network service
systemctl status phosphor-network

# Check for errors
journalctl -u phosphor-network
```

Configuration Persistence

Network configuration is stored in:

```
# systemd-networkd files
/etc/systemd/network/00-bmc-eth0.network

# Example content:
[Match]
Name=eth0

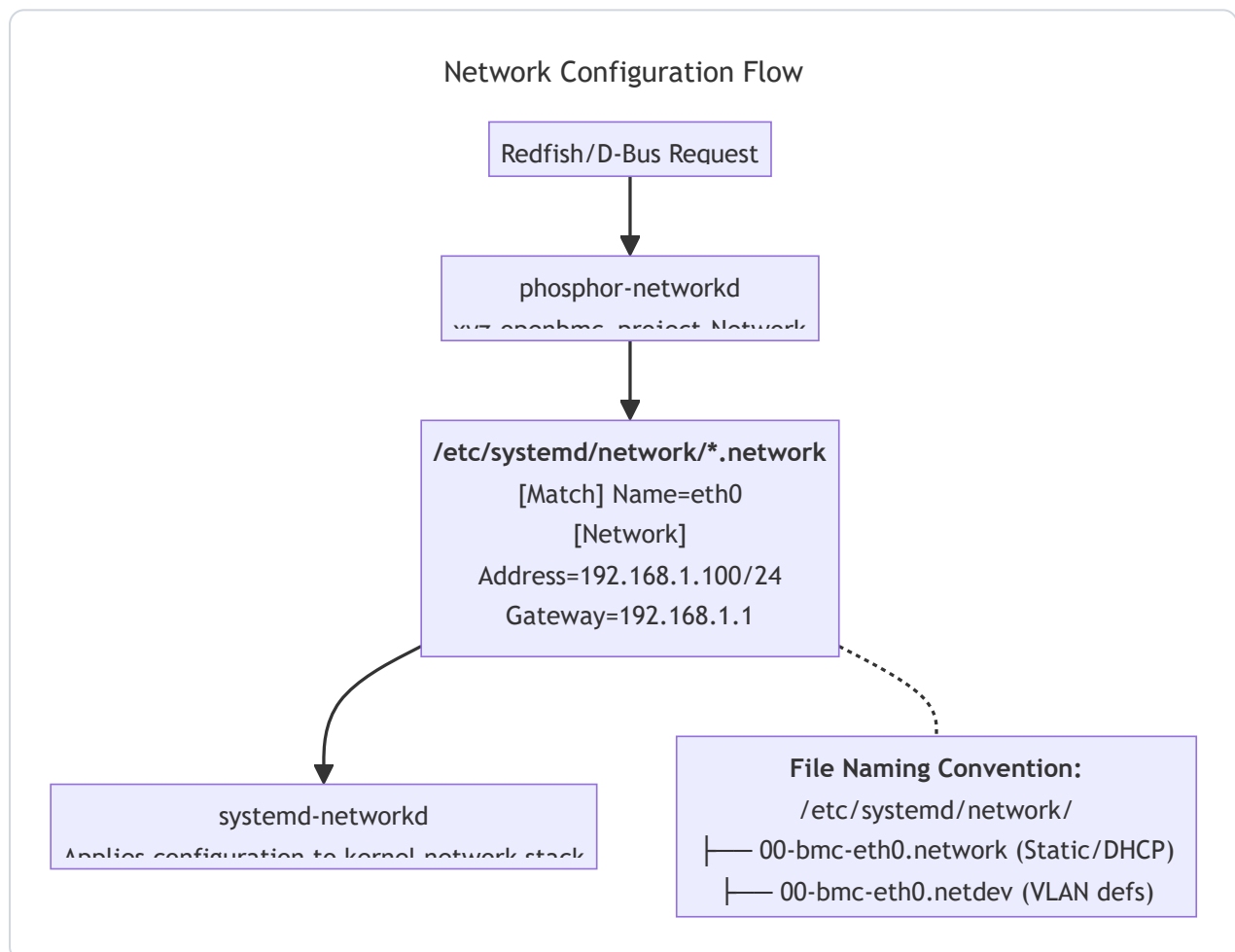
[Network]
Address=192.168.1.100/24
Gateway=192.168.1.1
DNS=8.8.8.8
```

Deep Dive

Advanced implementation details for network configuration developers.

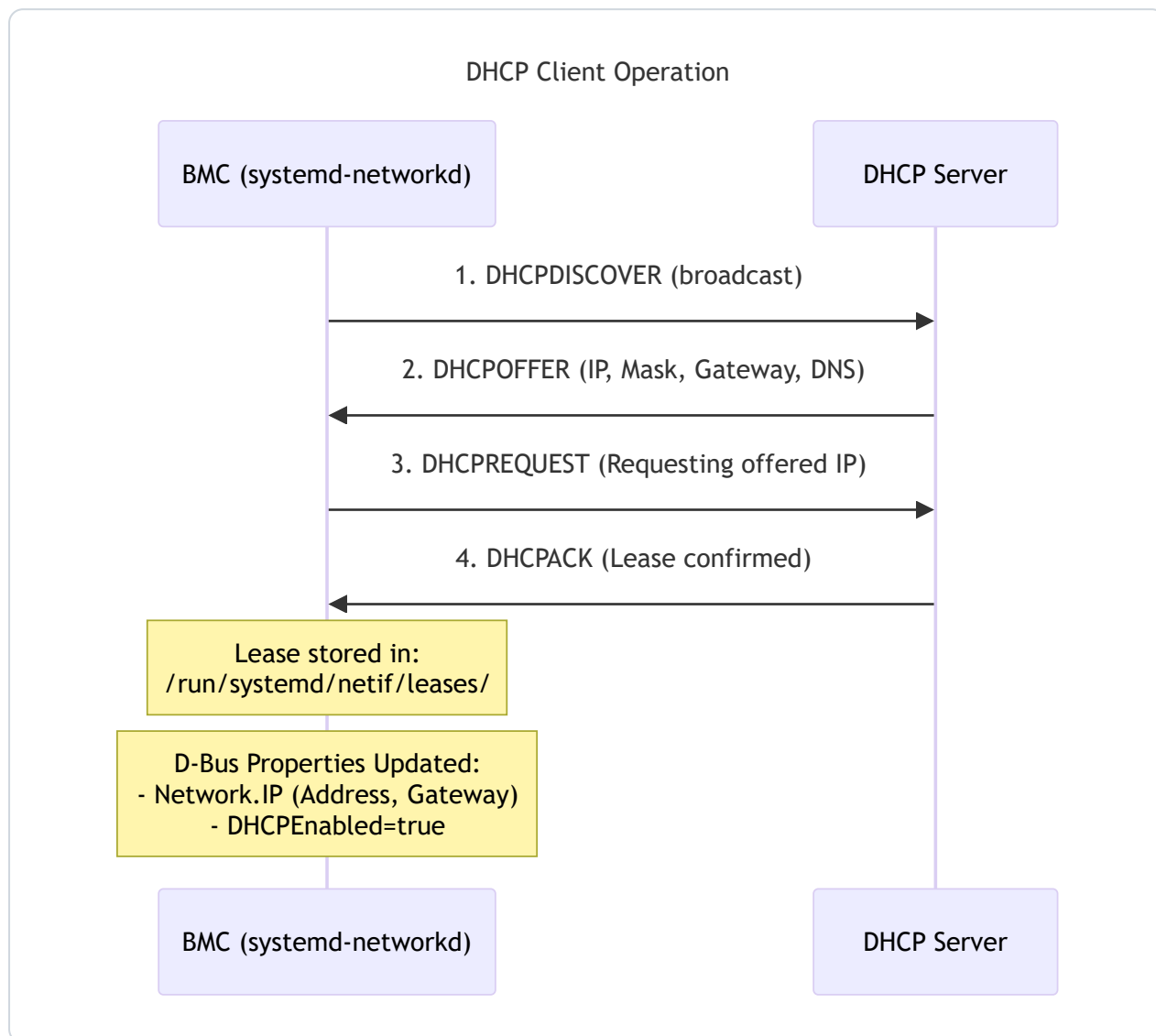
systemd-networkd Integration

phosphor-networkd translates D-Bus calls to systemd-networkd configuration:



► ASCII-art version (for comparison)

DHCP Transaction Flow

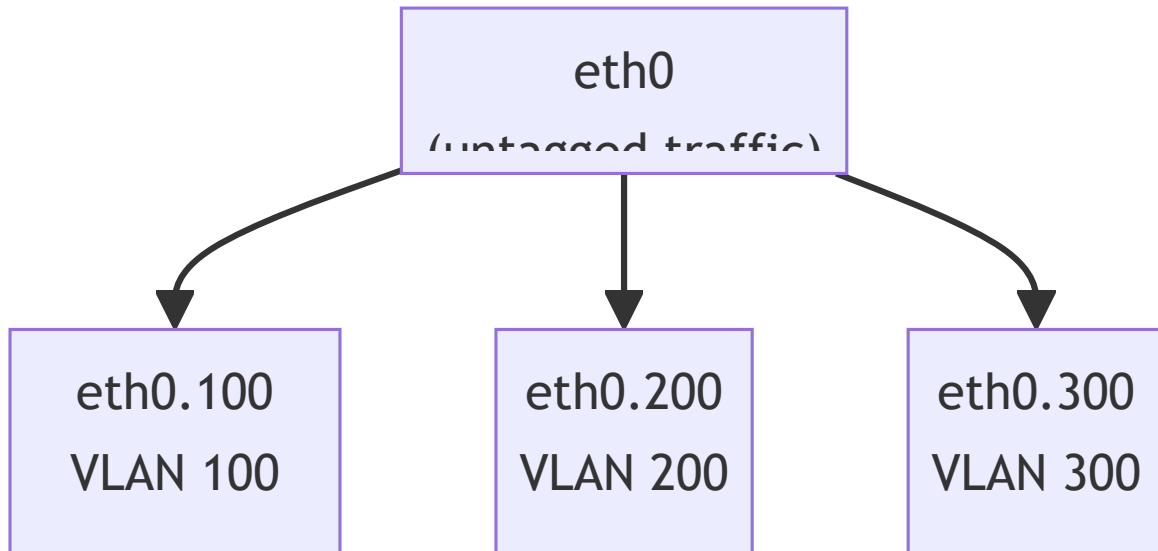


DHCP Options Handled:

- Option 1: Subnet Mask
- Option 3: Router (Gateway)
- Option 6: DNS Servers
- Option 12: Hostname
- Option 15: Domain Name
- Option 42: NTP Servers

► ASCII-art version (for comparison)

VLAN Configuration



VLAN .netdev file (`/etc/systemd/network/00-eth0.100.netdev`):

```
[NetDev]
Name=eth0.100
Kind=vlan

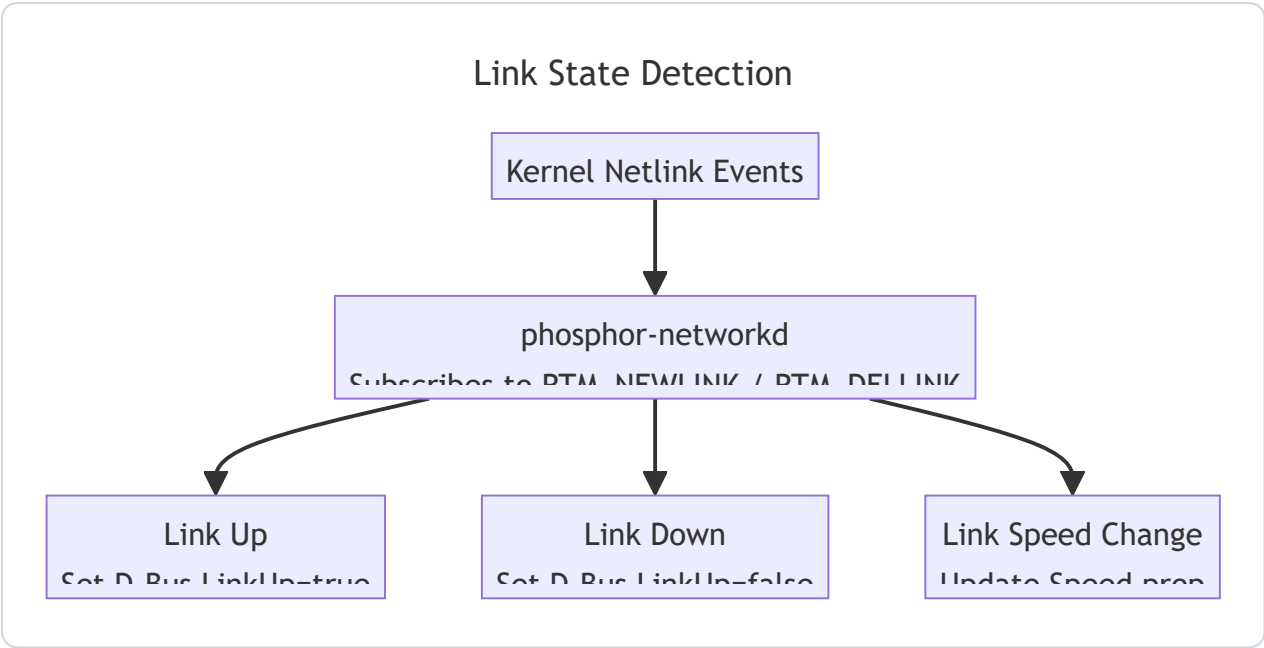
[VLAN]
Id=100
```

D-Bus VLAN Creation:

```
busctl call xyz.openbmc_project.Network \
  /xyz/openbmc_project/network/eth0 \
  xyz.openbmc_project.Network.VLAN.Create \
  VLAN sq "eth0.100" 100
```

► ASCII-art version (for comparison)

Link State Monitoring



D-Bus Properties (`/xyz/openbmc_project/network/eth0`):

- `LinkUp` (bool) - Physical link state
- `Speed` (uint32) - Link speed in Mbps
- `AutoNeg` (bool) - Auto-negotiation enabled
- `MTU` (uint32) - Maximum transmission unit

Redfish Mapping:

- `LinkUp` → `LinkStatus` ("LinkUp" / "LinkDown")
- `Speed` → `SpeedMbps`
- `AutoNeg` → `AutoNeg`

► ASCII-art version (for comparison)

Source Code Reference

Key implementation files in `phosphor-networkd`:

File	Description
<code>src/network_manager.cpp</code>	Main network manager
<code>src/ethernet_interface.cpp</code>	Ethernet interface handling
<code>src/ipaddress.cpp</code>	IP address management
<code>src/vlan_interface.cpp</code>	VLAN interface creation
<code>src/dhcp_configuration.cpp</code>	DHCP client configuration
<code>src/dns_updater.cpp</code>	DNS resolver updates

Examples

Working examples are available in the [examples/network](#) directory:

- `network-config.sh` - Network configuration script
 - `vlan-setup.sh` - VLAN configuration script
-

References

- [phosphor-networkd](#)
 - [systemd-networkd](#)
 - [Redfish EthernetInterface](#)
-

Tested on: OpenBMC master, QEMU romulus

LED Manager Guide

Configure and control LEDs on OpenBMC.

Table of Contents

1. [Overview](#)
2. [Setup & Configuration](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [Configuration Files and Paths](#)
 3. [Two Configuration Methods](#)
 4. [JSON Configuration \(preferred for development\)](#)
 5. [YAML Configuration \(legacy\)](#)
 6. [Deploying LED Configuration via Yocto](#)
 1. [Method 1: JSON Config \(runtime\)](#)
 2. [Method 2: YAML Config \(build-time\)](#)
 7. [Verification](#)
3. [LED Actions](#)
 1. [Action Types](#)
 2. [Control LEDs via D-Bus](#)
 3. [Control via Redfish](#)
 4. [Control via IPMI](#)
4. [LED Groups](#)
 1. [Standard Groups](#)
 2. [Create Custom Group](#)
 3. [Group Priorities](#)
5. [Lamp Test](#)
 1. [Trigger Lamp Test via D-Bus](#)
 2. [Lamp Test Configuration](#)
6. [sysfs LED Interface](#)
 1. [LED Sysfs Path](#)
 2. [Hardware Blink](#)
7. [Entity Manager Integration](#)
8. [GPIO-Based LEDs](#)
 1. [Device Tree Configuration](#)
9. [Troubleshooting](#)
 1. [LED Not Responding](#)
 2. [Group Not Activating](#)
 3. [Wrong LED Behavior](#)
10. [Configuration Example](#)

1. [Complete LED Configuration](#)

11. [Deep Dive](#)

1. [LED Blinking Algorithm](#)
2. [LED Group Priority Resolution](#)
3. [Sysfs LED Interface](#)
4. [LED Configuration YAML](#)
5. [Source Code Reference](#)

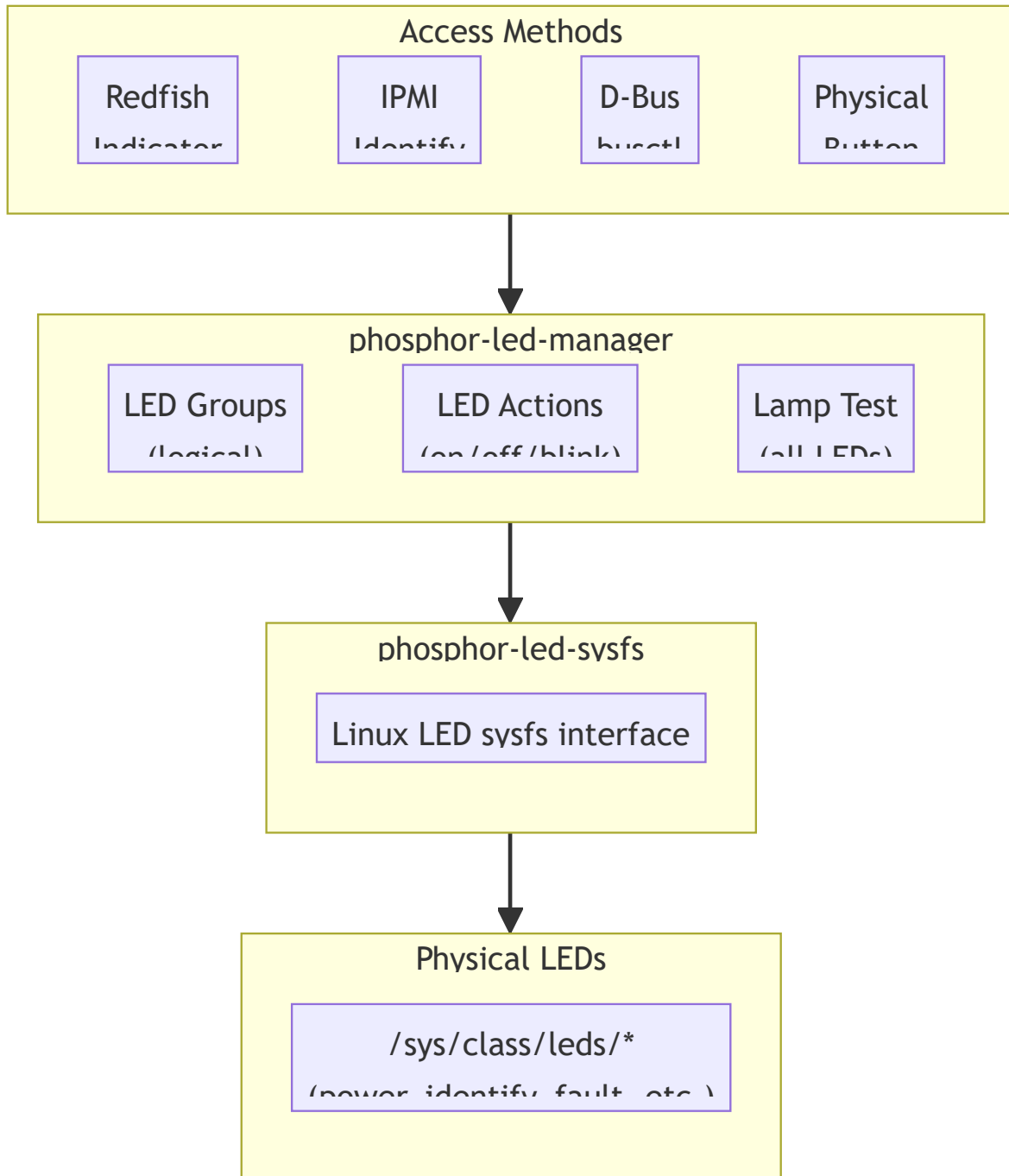
12. [Examples](#)

13. [References](#)

Overview

phosphor-led-manager provides LED control and group management, supporting identify, fault, and status LEDs through D-Bus interfaces.

LED Architecture



► ASCII-art version (for comparison)

Setup & Configuration

Build-Time Configuration (Yocto)

```
# In your machine .conf or local.conf

# Include LED management
IMAGE_INSTALL:append = " \
    phosphor-led-manager \
    phosphor-led-sysfs \
"

# LED group configuration (JSON or YAML)
LED_MANAGER_LED_CONFIG = "led.yaml"
```

Configuration Files and Paths

Config File	Install Path	Format	Description
LED groups (YAML)	<code>/usr/share/phosphor-led-manager/led.yaml</code>	YAML	Legacy: compiled into C++ at build time
LED groups (JSON)	<code>/usr/share/phosphor-led-manager/led.json</code>	JSON	Preferred: runtime-parsed at service startup
Entity Manager	<code>/usr/share/entity-manager/configurations/*.json</code>	EM JSON	Alternative: define LEDs via Entity Manager

Two Configuration Methods

phosphor-led-manager supports two approaches:

	YAML Config (build-time)	JSON Config (runtime)
Config path	<code>/usr/share/phosphor-led-manager/led.yaml</code>	<code>/usr/share/phosphor-led-manager/led.json</code>
When parsed	At meson build time (generates C++ constexpr)	At service startup (runtime parsing)
Edit cycle	Rebuild image for changes	Replace file and restart service
Meson option	Default	<code>-Duse-json=enabled</code>
Recommended for	Production (smaller binary)	Development and rapid iteration

JSON Configuration (preferred for development)

```
{
  "leds": [
    {
      "group": "/xyz/openbmc_project/led/groups/enclosure_identify",
      "members": [
        {
          "name": "identify",
          "action": "Blink",
          "dutyOn": 50,
          "period": 1000
        }
      ]
    },
    {
      "group": "/xyz/openbmc_project/led/groups/power_fault",
      "members": [
        {
          "name": "power",
          "action": "On",
          "dutyOn": 100,
          "period": 0
        }
      ]
    }
  ]
}
```

YAML Configuration (legacy)

```
# led.yaml - compiled into C++ at build time
bmc_booted:
  enclosure_identify:
    Action: 'Off'
    DutyOn: 50
    Period: 0

enclosure_identify:
  enclosure_identify:
    Action: 'Blink'
    DutyOn: 50
    Period: 1000

fault:
  power:
    Action: 'On'
    DutyOn: 100
    Period: 0
```

Deploying LED Configuration via Yocto

Method 1: JSON Config (runtime)

```
# meta-myplatform/recipes-phosphor/leds/phosphor-led-manager/
# └─ files/
#   └─ led.json
# └─ phosphor-led-manager_%.bbappend

cat > phosphor-led-manager_%.bbappend << 'EOF'
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
SRC_URI += "file://led.json"

# Enable JSON parsing (instead of compiled YAML)
EXTRA_OEMESON:append = " -Duse-json=enabled"

do_install:append() {
    install -d ${D}${datadir}/phosphor-led-manager
    install -m 0644 ${WORKDIR}/led.json \
        ${D}${datadir}/phosphor-led-manager/
}
EOF
```

Method 2: YAML Config (build-time)

```
# meta-myplatform/recipes-phosphor/leds/phosphor-led-manager/
# └─ files/
#   └─ led.yaml
# └─ phosphor-led-manager_%.bbappend

cat > phosphor-led-manager_%.bbappend << 'EOF'
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
SRC_URI += "file://led.yaml"

# Tell the build system where to find your YAML
LED_MANAGER_LED_CONFIG = "${WORKDIR}/led.yaml"
EOF
```

Verification

```
# Check LED manager service
systemctl status phosphor-led-manager

# View available physical LEDs
ls /sys/class/leds/

# Check LED D-Bus groups
busctl tree xyz.openbmc_project.LED.GroupManager

# Verify a specific group is defined
busctl introspect xyz.openbmc_project.LED.GroupManager \
    /xyz/openbmc_project/led/groups/enclosure_identify
```

LED Actions

Action Types

Action	Description	Parameters
Off	LED off	None
On	LED on	None
Blink	Blinking	DutyOn (%), Period (ms)

Control LEDs via D-Bus

```
# Turn on identify LED
busctl set-property xyz.openbmc_project.LED.GroupManager \
  /xyz/openbmc_project/led/groups/enclosure_identify \
  xyz.openbmc_project.Led.Group \
  Asserted b true

# Turn off identify LED
busctl set-property xyz.openbmc_project.LED.GroupManager \
  /xyz/openbmc_project/led/groups/enclosure_identify \
  xyz.openbmc_project.Led.Group \
  Asserted b false
```

Control via Redfish

```
# Get chassis indicator
curl -k -u root:OpenBmc \
  https://localhost/redfish/v1/Chassis/chassis

# Turn on identify LED
curl -k -u root:OpenBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"LocationIndicatorActive": true}' \
  https://localhost/redfish/v1/Chassis/chassis

# Turn off identify LED
curl -k -u root:OpenBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"LocationIndicatorActive": false}' \
  https://localhost/redfish/v1/Chassis/chassis
```

Control via IPMI

```
# Chassis identify on (continuous)
ipmitool -I lanplus -H bmc-ip -U root -P OpenBmc chassis identify

# Chassis identify on (15 seconds)
ipmitool -I lanplus -H bmc-ip -U root -P OpenBmc chassis identify 15

# Chassis identify off
ipmitool -I lanplus -H bmc-ip -U root -P OpenBmc chassis identify 0
```


LED Groups

Standard Groups

Group	Purpose	Trigger
<code>enclosure_identify</code>	Locate chassis	User request
<code>power_fault</code>	Power problem	PSU fault
<code>bmc_booted</code>	BMC ready	Boot complete
<code>cpu_fault</code>	CPU error	Hardware error
<code>memory_fault</code>	Memory error	DIMM fault

Create Custom Group

```
# In led.yaml
my_custom_group:
  custom_led:
    Action: 'Blink'
    DutyOn: 50
    Period: 500
```

Group Priorities

When multiple groups control the same LED, priority determines the action:

```
# Higher priority takes precedence
# Priority: Blink > On > Off
```

Lamp Test

Lamp test activates all LEDs to verify functionality.

Trigger Lamp Test via D-Bus

```
# Start lamp test
busctl set-property xyz.openbmc_project.LED.GroupManager \
  /xyz/openbmc_project/led/groups/lamp_test \
  xyz.openbmc_project.Led.Group \
  Asserted b true

# Stop lamp test
busctl set-property xyz.openbmc_project.LED.GroupManager \
  /xyz/openbmc_project/led/groups/lamp_test \
  xyz.openbmc_project.Led.Group \
  Asserted b false
```

Lamp Test Configuration

```
lamp_test:
  identify:
    Action: 'On'
    DutyOn: 100
    Period: 0
  power:
    Action: 'On'
    DutyOn: 100
    Period: 0
  fault:
    Action: 'On'
    DutyOn: 100
    Period: 0
```

sysfs LED Interface

LED Sysfs Path

```
# View LED attributes
cat /sys/class/leds/identify/brightness
cat /sys/class/leds/identify/max_brightness
cat /sys/class/leds/identify/trigger

# Set LED directly
echo 255 > /sys/class/leds/identify/brightness
echo 0 > /sys/class/leds/identify/brightness
```

Hardware Blink

```
# Use hardware blink if supported
cat /sys/class/leds/identify/trigger
# [none] timer heartbeat ...

echo timer > /sys/class/leds/identify/trigger
echo 500 > /sys/class/leds/identify/delay_on
echo 500 > /sys/class/leds/identify/delay_off
```

Entity Manager Integration

Entity Manager can define physical LED-to-GPIO mappings, which phosphor-led-sysfs uses to create the sysfs LED entries. This is an alternative to device tree `gpio-leds` definitions.

Install path: `/usr/share/entity-manager/configurations/<board-name>.json`

```
{
  "Name": "MyBoard LEDs",
  "Type": "Board",
  "Probe": "TRUE",

  "Exposes": [
    {
      "Name": "Identify LED",
      "Type": "led",
      "Polarity": "Active High",
      "GpioPin": "IDENTIFY_LED"
    },
    {
      "Name": "Fault LED",
      "Type": "led",
      "Polarity": "Active Low",
      "GpioPin": "FAULT_LED"
    }
  ]
}
```

Deploy via Yocto:

```
cat > entity-manager_%.bbappend << 'EOF'
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
SRC_URI += "file://my-board-leds.json"

do_install:append() {
    install -d ${D}${datadir}/entity-manager/configurations
    install -m 0444 ${WORKDIR}/my-board-leds.json \
        ${D}${datadir}/entity-manager/configurations/
}
EOF
```

Entity Manager LED definitions define the **physical LED hardware**. You still need a `led.json` or `led.yaml` to define the **logical LED groups** (which groups map to which physical LEDs with what actions).

GPIO-Based LEDs

Device Tree Configuration

```
leds {
    compatible = "gpio-leds";

    identify {
        label = "identify";
        gpios = <&gpio ASPEED_GPIO(A, 0) GPIO_ACTIVE_HIGH>;
        default-state = "off";
    };

    power {
        label = "power";
        gpios = <&gpio ASPEED_GPIO(A, 1) GPIO_ACTIVE_HIGH>;
        default-state = "on";
    };

    fault {
        label = "fault";
        gpios = <&gpio ASPEED_GPIO(A, 2) GPIO_ACTIVE_HIGH>;
        default-state = "off";
    };
};
```

Troubleshooting

LED Not Responding

```
# Check sysfs LED exists
ls /sys/class/leds/

# Check LED permissions
ls -la /sys/class/leds/identify/

# Check LED driver
cat /sys/class/leds/identify/trigger

# Direct control test
echo 255 > /sys/class/leds/identify/brightness
```

Group Not Activating

```
# Check group exists
busctl tree xyz.openbmc_project.LED.GroupManager

# Check group properties
busctl introspect xyz.openbmc_project.LED.GroupManager \
    /xyz/openbmc_project/led/groups/enclosure_identify

# Check LED manager logs
journalctl -u phosphor-led-manager
```

Wrong LED Behavior

```
# Verify configuration file
cat /usr/share/phosphor-led-manager/led.yaml

# Check priority conflicts
# Multiple groups may control same LED
```

Configuration Example

Complete LED Configuration

```
# /usr/share/phosphor-led-manager/led.yaml

# Enclosure Identify (blue LED, blinking)
enclosure_identify:
  identify:
    Action: 'Blink'
    DutyOn: 50
    Period: 1000

# Power fault (red LED, on)
power_fault:
  fault:
    Action: 'On'
    DutyOn: 100
    Period: 0

# BMC ready (green LED, on)
bmc_booted:
  power:
    Action: 'On'
    DutyOn: 100
    Period: 0
  identify:
    Action: 'Off'
    DutyOn: 0
    Period: 0

# CPU fault (amber LED, blinking)
cpu_fault:
  fault:
    Action: 'Blink'
    DutyOn: 50
    Period: 500

# Lamp test (all LEDs on)
lamp_test:
  identify:
    Action: 'On'
    DutyOn: 100
    Period: 0
  power:
    Action: 'On'
    DutyOn: 100
    Period: 0
  fault:
    Action: 'On'
    DutyOn: 100
    Period: 0
```

Deep Dive

Advanced implementation details for LED management developers.

LED Blinking Algorithm

The LED manager implements software-controlled blinking for LEDs:



LED Group Priority Resolution

When multiple LED groups request different states, priority determines outcome:

LED Group Priority

Priority Levels (higher = more important):

Priority	Group	Description
1	lamp_test	Override for manufacturing test
2	fault	Hardware fault indication
3	enclosure_identify	Chassis locate request
4	power	System power state
5	bmc_booted	BMC ready indication

Resolution Example:

LED: "identify" (front panel blue LED)

Group Requests:

- |— bmc_booted: Blink (Priority 5)
- |— power: On (Priority 4)
- |— enclosure_identify: Blink-fast (Priority 3)

Resolution: enclosure_identify wins → LED blinks fast

If enclosure_identify deactivated:

→ power group now wins → LED on

D-Bus Group Interface:

/xyz/openbmc_project/led/groups/<group-name>

|— Asserted (bool) - Group active state

Sysfs LED Interface

Kernel LED Sysfs Interface

```
/sys/class/leds/<led-name>/  
├─ brightness      # 0=off, max_brightness=full on  
├─ max_brightness  # Maximum brightness value (often 255)  
├─ trigger         # Current trigger (none, timer, heartbeat)  
├─ delay_on        # On time in ms (when trigger=timer)  
└─ delay_off       # Off time in ms (when trigger=timer)
```

phosphor-led-sysfs Mapping:

D-Bus Action	Sysfs Operations
Action=On	→ echo max > brightness echo none > trigger
Action=Off	→ echo 0 > brightness echo none > trigger
Action=Blink	→ echo timer > trigger echo <on_ms> > delay_on echo <off_ms> > delay_off

Device Tree LED Definition:

```
leds {  
    compatible = "gpio-leds";  
  
    identify {  
        gpios = <&gpio0 10 GPIO_ACTIVE_HIGH>;  
        default-state = "off";  
    };  
  
    fault {  
        gpios = <&gpio0 11 GPIO_ACTIVE_LOW>;  
        default-state = "off";  
    };  
};
```

LED Configuration YAML

LED Group Configuration

```
/usr/share/phosphor-led-manager/led.yaml:

bmc_booted:
  heartbeat:
    Action: 'Blink'
    DutyOn: 50
    Period: 1000
    Priority: 'Blink'

enclosure_identify:
  identify:
    Action: 'Blink'
    DutyOn: 50
    Period: 500
    Priority: 'Blink'

power_on:
  power:
    Action: 'On'
    DutyOn: 100
    Period: 0
    Priority: 'On'
```

Generated Code (meson build):
YAML → C++ constexpr arrays (led-gen.hpp)

Source Code Reference

Key implementation files in [phosphor-led-manager](#):

File	Description
manager.cpp	LED group state management
group.cpp	Individual group handling
serialize.cpp	State persistence
lamptest.cpp	Lamp test implementation
json-parser.cpp	JSON config parsing

Examples

Working examples are available in the [examples/led-manager](#) directory:

- [led-groups.json](#) - LED group definitions
- [led-config.yaml](#) - Physical LED to D-Bus mapping

- `identify-led.sh` - Script to control identify LED
-

References

- [phosphor-led-manager](#)
 - [phosphor-led-sysfs](#)
 - [Linux LED sysfs](#)
 - [Redfish LocationIndicatorActive](#)
-

Tested on: OpenBMC master, QEMU romulus

Certificate Manager Guide

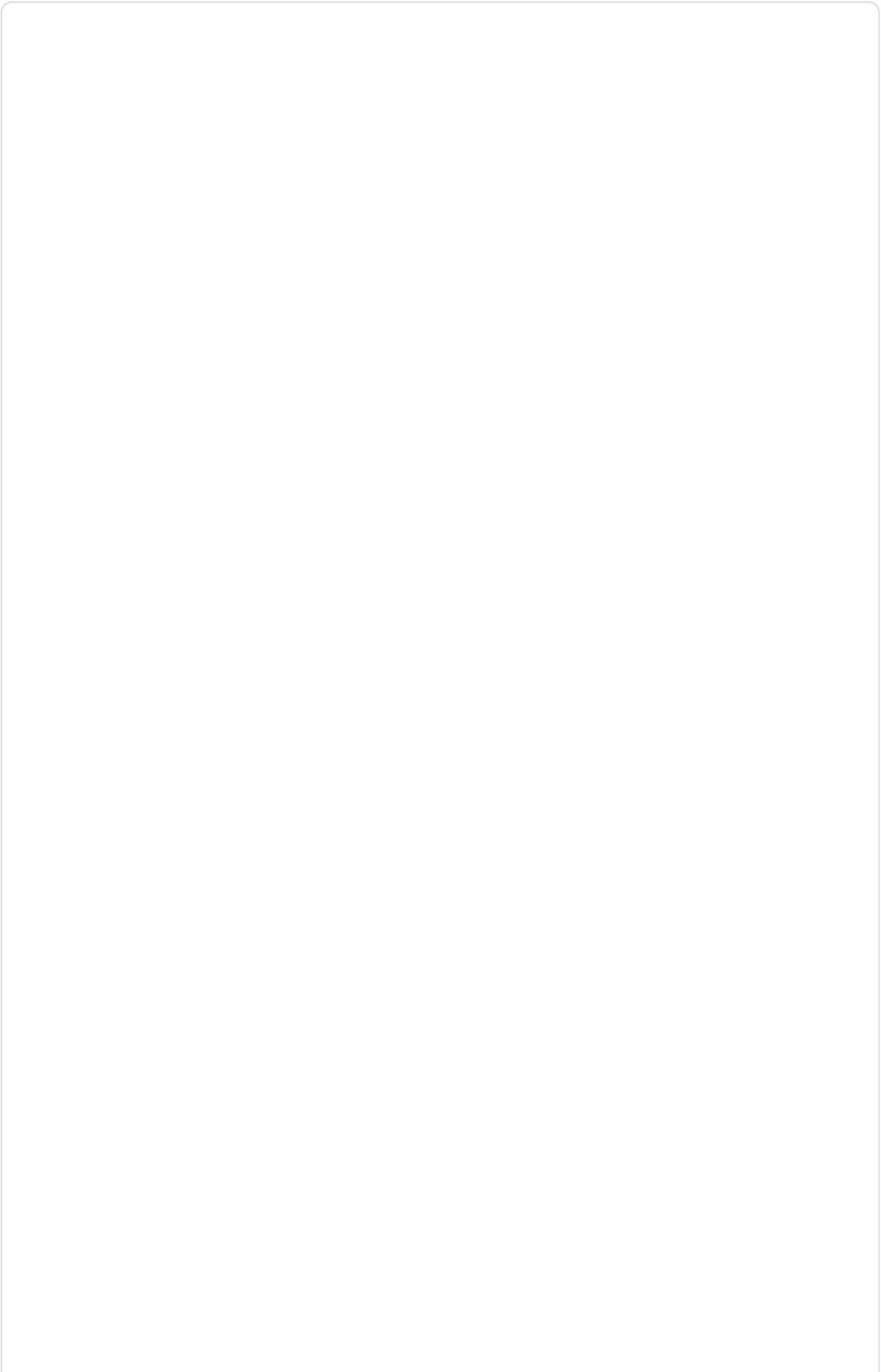
Configure TLS/SSL certificates for secure BMC communication.

Table of Contents

1. [Overview](#)
 2. [Setup & Configuration](#)
 1. [Build-Time Configuration](#)
 2. [Certificate Locations](#)
 3. [Managing Certificates](#)
 1. [View Current Certificate](#)
 2. [Generate Self-Signed Certificate](#)
 3. [Replace Certificate via Redfish](#)
 4. [Add CA Certificate](#)
 4. [Deep Dive](#)
 1. [X.509 Certificate Chain Validation](#)
 2. [Certificate Storage and File Format](#)
 3. [CSR Generation Process](#)
 4. [Service Restart Integration](#)
 5. [Source Code Reference](#)
 5. [References](#)
-

Overview

phosphor-certificate-manager manages X.509 certificates for HTTPS, LDAP, and other secure services on OpenBMC.



Certificate Manager Architecture

phosphor-certificate-manager

HTTPS Certs

`/etc/ssl/certs/https/`

LDAP Certs

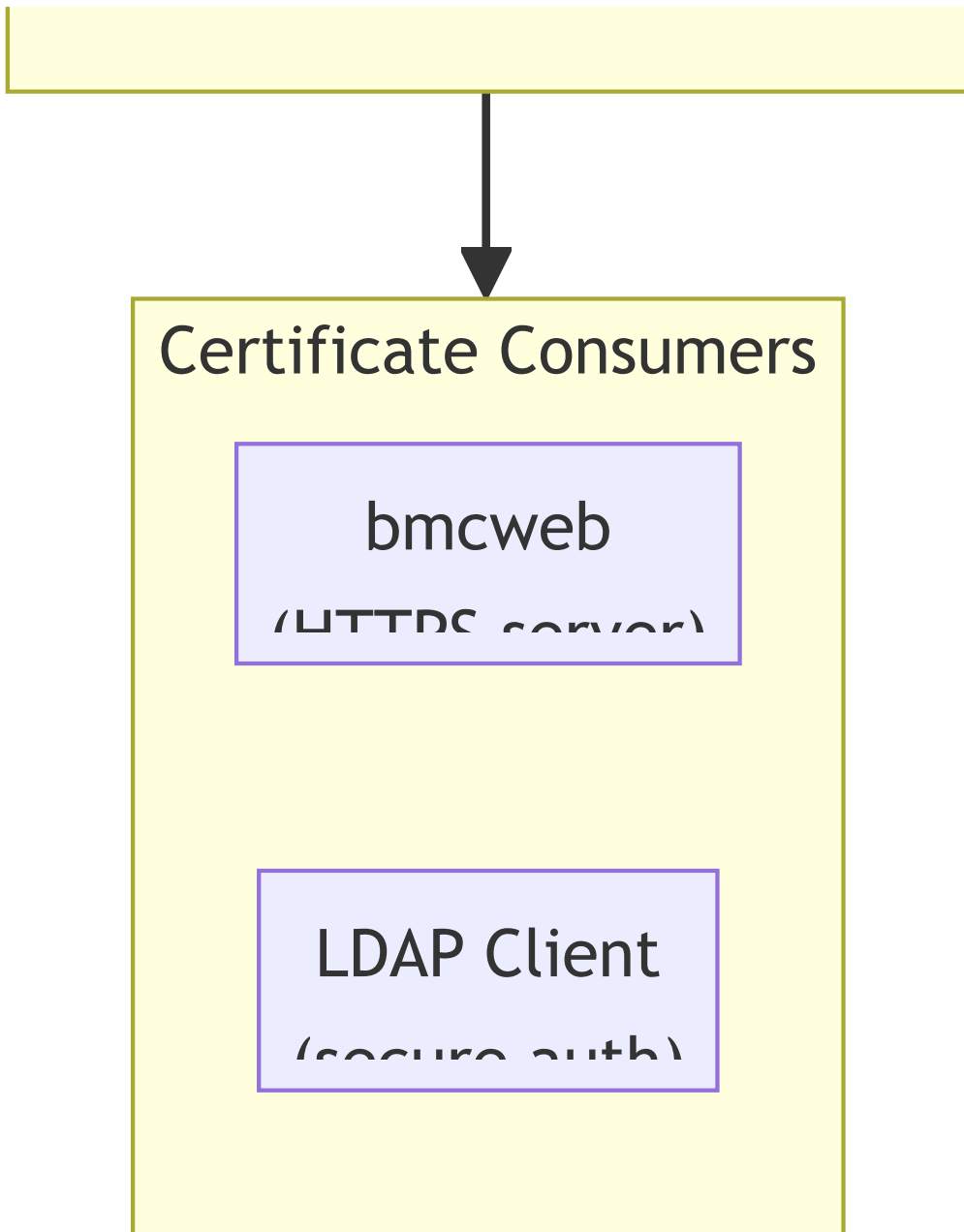
`/etc/ssl/certs/ldap/`

Authority Certs

`(Trusted CAs)`

CSR Generation

`(Signing Doc)`



► ASCII-art version (for comparison)

Setup & Configuration

Build-Time Configuration

```
# Include certificate manager
IMAGE_INSTALL:append = " phosphor-certificate-manager"
```

Certificate Locations

Type	Path	Description
HTTPS Server	/etc/ssl/certs/https/server.pem	Web server certificate
HTTPS CA	/etc/ssl/certs/https/authority/	Trusted CA certificates
LDAP Client	/etc/ssl/certs/ldap/	LDAP client certificates

Managing Certificates

View Current Certificate

```
# Via Redfish
curl -k -u root:openBmc \
  https://localhost/redfish/v1/CertificateService/CertificateLocations

# View certificate details
openssl x509 -in /etc/ssl/certs/https/server.pem -text -noout
```

Generate Self-Signed Certificate

```
# Generate on BMC
openssl req -x509 -newkey rsa:2048 \
  -keyout /etc/ssl/private/server.pem \
  -out /etc/ssl/certs/https/server.pem \
  -days 365 -nodes \
  -subj "/CN=openbmc/O=MyOrg"

# Restart bmcweb
systemctl restart bmcweb
```

Replace Certificate via Redfish

```
# Replace HTTPS certificate
curl -k -u root:openBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "CertificateType": "PEM",
    "CertificateString": "-----BEGIN CERTIFICATE-----\n...\n-----END CERTIFICATE-----"
  }' \
  https://localhost/redfish/v1/Managers/bmc/NetworkProtocol/HTTPS/Certificates

# Generate CSR
curl -k -u root:openBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "CommonName": "bmc.example.com",
    "Organization": "MyOrg",
    "Country": "US"
  }' \
  https://localhost/redfish/v1/CertificateService/Actions/CertificateService.GenerateCSR
```


Add CA Certificate

```
# Add trusted CA
curl -k -u root:openBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "CertificateType": "PEM",
    "CertificateString": "-----BEGIN CERTIFICATE-----\n...\n-----END CERTIFICATE-----"
  }' \
  https://localhost/redfish/v1/Managers/bmc/Truststore/Certificates
```

Deep Dive

Advanced implementation details for certificate management developers.

X.509 Certificate Chain Validation



For LDAP client certificate:

- extKeyUsage: id-kp-clientAuth (1.3.6.1.5.5.7.3.2)

5. BUILD TRUST CHAIN TO ROOT

```
X509_STORE_CTX_init(ctx, store, leaf_cert, intermediates)
X509_verify_cert(ctx)
```

Result: X509_V_OK (0) or error code

Certificate Storage and File Format

Certificate Storage Architecture

```
/etc/ssl/
├── certs/
│   ├── https/
│   │   ├── server.pem      ← HTTPS server cert + private key
│   │   └── authority/      ← Trusted CA certs for client auth
│   │       ├── ca1.pem
│   │       └── ca2.pem
│   └── ldap/
│       └── client.pem      ← LDAP client certificate
└── private/
    └── server.pem          ← Private key (mode 0600)
```

COMBINED PEM FILE FORMAT (server.pem):

```
-----BEGIN PRIVATE KEY-----
MIIEvgIBADANBgkqhkiG9w0BAQEFAASCBAKggSkAgEAAoIBAQC...
... (Base64 encoded private key) ...
-----END PRIVATE KEY-----
-----BEGIN CERTIFICATE-----
MIIDXTCCAkwAwIBAgIJAJC1HiIAZAIUMA0GCSqGSIb3Qq...
... (Base64 encoded server certificate) ...
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
MIIDrzCCApegAwIBAgIQCDvgVpBCRRrGhdWrwJWZHSj...
... (Base64 encoded intermediate CA - optional) ...
-----END CERTIFICATE-----
```

D-BUS OBJECT STRUCTURE:

```
/xyz/openbmc_project/certs
├── server
│   ├── https
│   │   └── 1                  ← Certificate object
│   │       ├── xyz.openbmc_project.Certs.Certificate
│   │       │   ├── CertificateString (PEM)
│   │       │   ├── Subject
│   │       │   ├── Issuer
│   │       │   ├── ValidNotBefore
│   │       │   └── ValidNotAfter
│   │       └── xyz.openbmc_project.Object.Delete
└── authority
    └── truststore
        ├── 1                  ← CA certificate 1
        └── 2                  ← CA certificate 2
```

Certificate Signing Request (CSR) Flow

REDFISH CSR REQUEST

```
POST /redfish/v1/CertificateService/Actions/
    CertificateService.GenerateCSR
{
    "CommonName": "bmc.example.com",
    "Organization": "ACME Corp",
    "OrganizationalUnit": "IT",
    "City": "San Francisco",
    "State": "CA",
    "Country": "US",
    "AlternativeNames": ["bmc", "192.168.1.100"],
    "KeyPairAlgorithm": "EC",
    "KeyCurveId": "secp384r1"
}
```

CSR GENERATION STEPS:

1. Generate Key Pair

```
// For EC key (recommended)
EVP_PKEY* pkey = EVP_EC_gen("secp384r1");

// For RSA key
EVP_PKEY* pkey = EVP_RSA_gen(2048);
```

2. Create CSR Structure

```
X509_REQ* req = X509_REQ_new();
X509_REQ_set_pubkey(req, pkey);
X509_REQ_set_version(req, 0); // Version 1
```

3. Set Subject Distinguished Name

```
X509_NAME* name = X509_REQ_get_subject_name(req);
X509_NAME_add_entry_by_txt(name, "CN", MBSTRING_ASC,
    "bmc.example.com", -1, -1, 0);
X509_NAME_add_entry_by_txt(name, "O", MBSTRING_ASC,
    "ACME Corp", -1, -1, 0);
// ... additional fields
```

4. Add Subject Alternative Names (SAN) Extension

```
STACK_OF(X509_EXTENSION)* exts = sk_X509_EXTENSION_new_null();
X509_EXTENSION* san_ext = X509V3_EXT_conf_nid(
    NULL, NULL, NID_subject_alt_name,
    "DNS:bmc,DNS:bmc.example.com,IP:192.168.1.100");
sk_X509_EXTENSION_push(exts, san_ext);
X509_REQ_add_extensions(req, exts);
```

5. Sign CSR with Private Key

```
X509_REQ_sign(req, pkey, EVP_sha384());
```

6. Output PEM-encoded CSR

```
-----BEGIN CERTIFICATE REQUEST-----  
MIICpDCCAYwCAQAwXzELMAkGA1UEBhMCVVMxCzAJBgNVBAGMAkNB...  
... (Base64 encoded CSR) ...  
-----END CERTIFICATE REQUEST-----
```

7. Store private key in /etc/ssl/private/ (mode 0600)
8. Return CSR to administrator for CA signing
9. After signing, install certificate via Replace action

Service Restart Integration



Source Code Reference

Key implementation files in [phosphor-certificate-manager](#):

File	Description
<code>certificate.cpp</code>	Certificate class with X.509 parsing and validation
<code>certs_manager.cpp</code>	Main certificate manager handling install/replace/delete
<code>csr.cpp</code>	CSR generation using OpenSSL
<code>argument.cpp</code>	CLI argument parsing for certificate types
<code>watch.cpp</code>	Filesystem watcher for certificate changes

References

- [phosphor-certificate-manager](#)
 - [Redfish CertificateService](#)
-

Tested on: OpenBMC master, QEMU romulus

Time Manager Guide

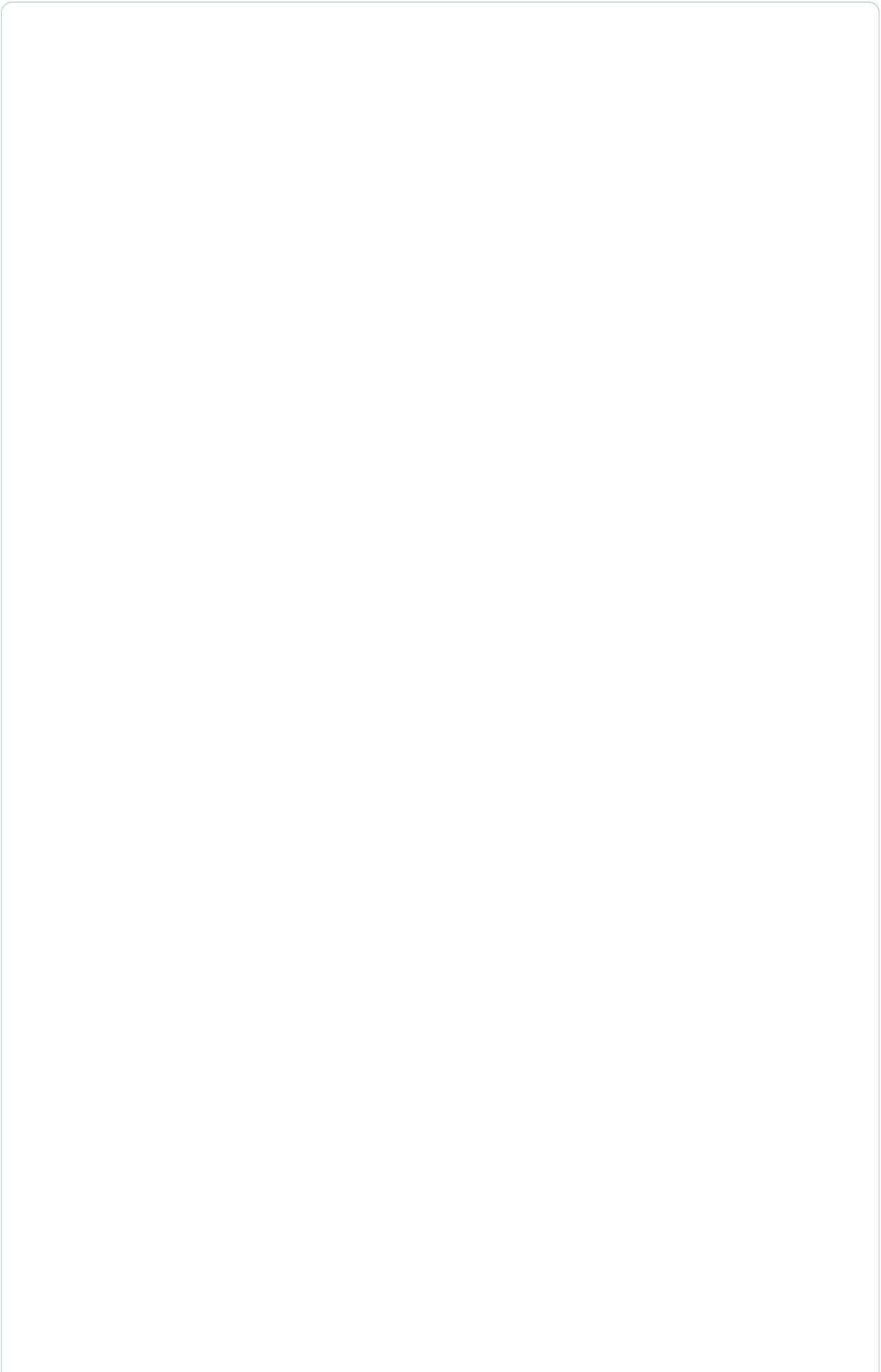
Configure NTP, RTC, and timezone settings on OpenBMC.

Table of Contents

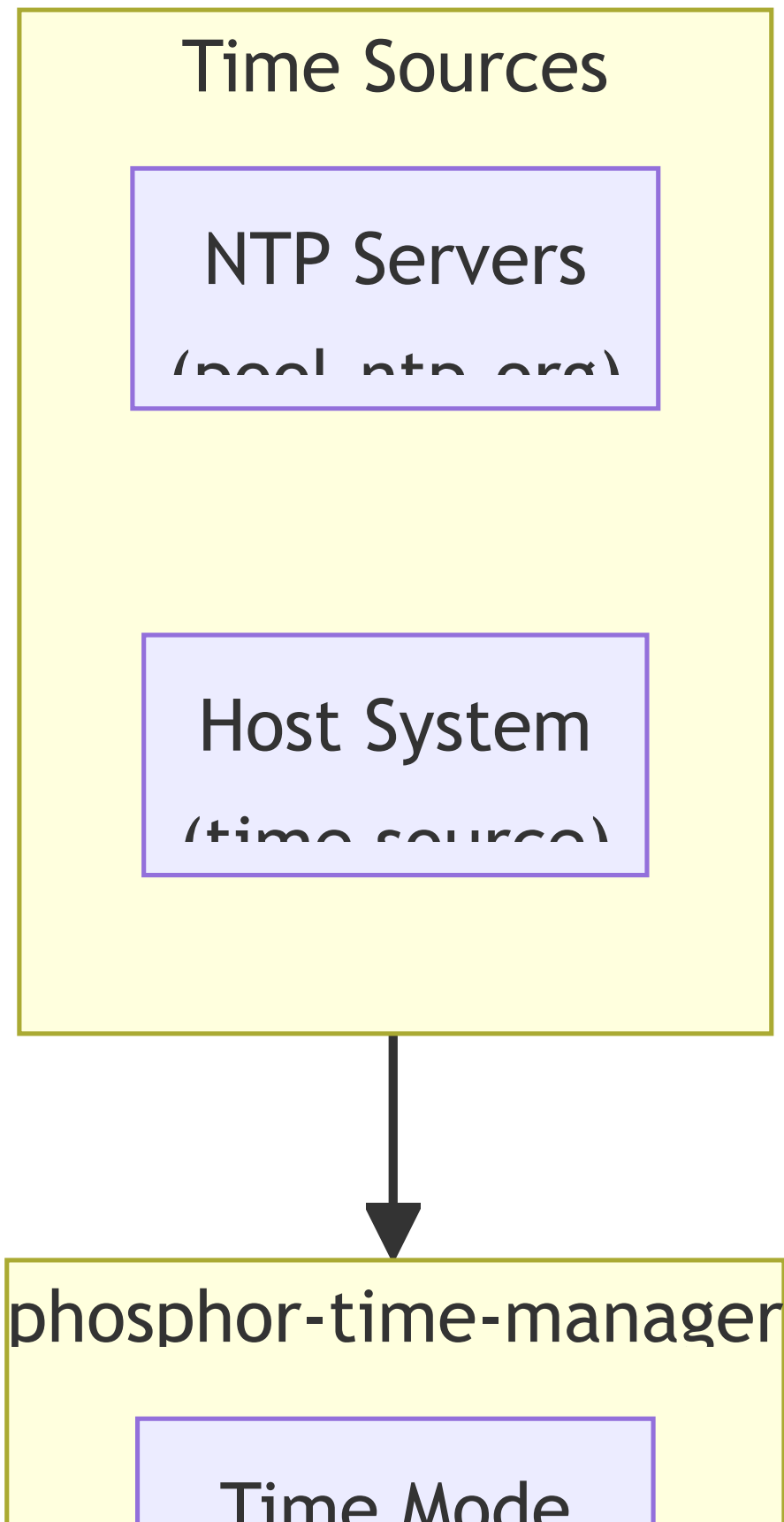
1. [Overview](#)
 2. [Setup & Configuration](#)
 1. [Build-Time Configuration](#)
 2. [Time Modes](#)
 3. [Configuring NTP](#)
 1. [Enable NTP](#)
 2. [Check NTP Status](#)
 4. [Setting Time Manually](#)
 5. [Timezone Configuration](#)
 6. [Time Ownership](#)
 7. [Deep Dive](#)
 1. [NTP Synchronization Architecture](#)
 2. [BMC/Host Time Ownership Model](#)
 3. [RTC and System Clock Synchronization](#)
 4. [Source Code Reference](#)
 8. [References](#)
-

Overview

phosphor-time-manager handles time synchronization, timezone configuration, and BMC/host time ownership.



Time Manager Architecture



TIME SOURCE
(NTP / Manual)

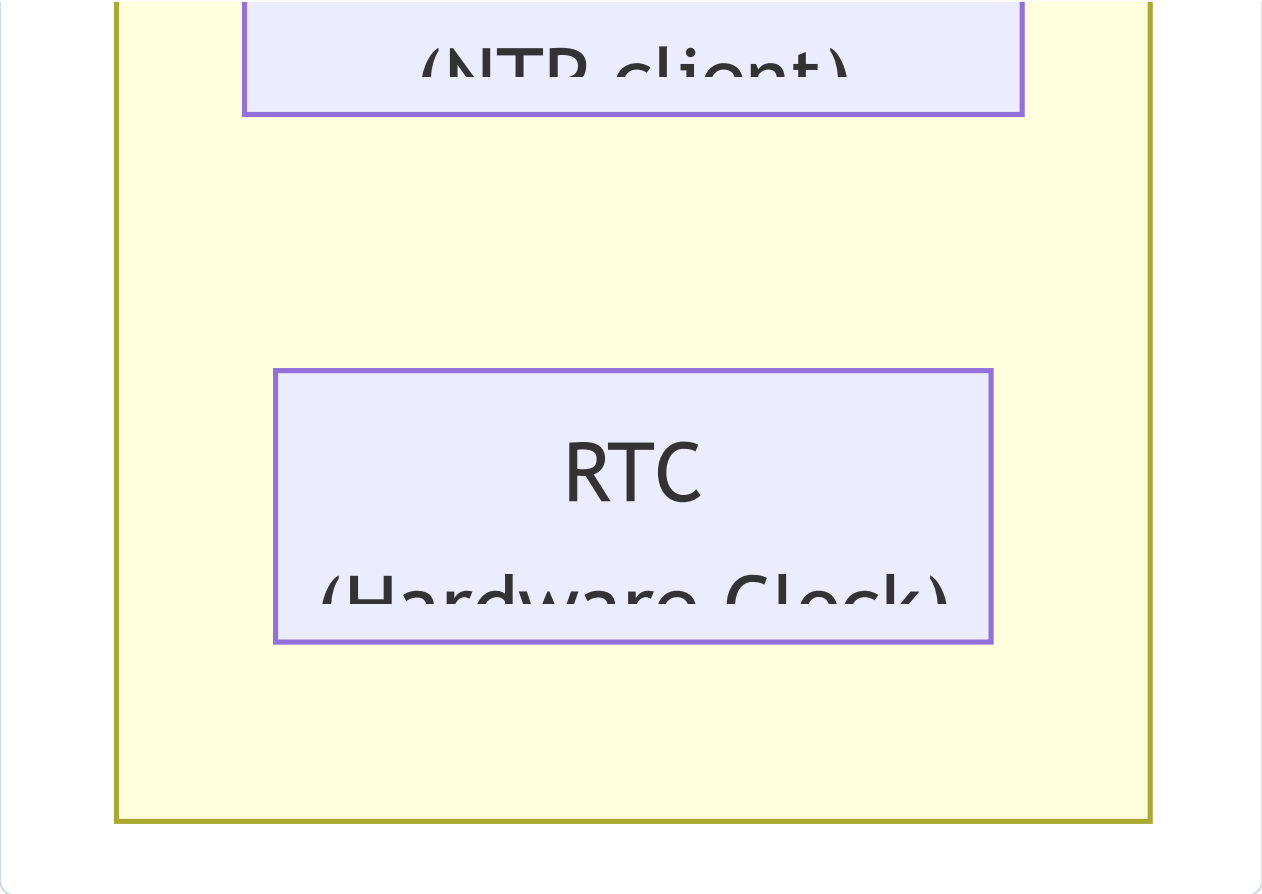
Time Owner
(RMC / Host)

Timezone
Setting



Time Backends

systemd-timesyncd



► ASCII-art version (for comparison)

Setup & Configuration

Build-Time Configuration

```
# Include time manager
IMAGE_INSTALL:append = " phosphor-time-manager"
```

Time Modes

Mode	Description
NTP	Synchronize via NTP servers
Manual	Set time manually

Configuring NTP

Enable NTP

```
# Via Redfish
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"NTP": {"ProtocolEnabled": true}}' \
  https://localhost/redfish/v1/Managers/bmc/NetworkProtocol

# Configure NTP servers
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"NTP": {"NTPServers": ["pool.ntp.org", "time.google.com"]}}' \
  https://localhost/redfish/v1/Managers/bmc/NetworkProtocol
```

Check NTP Status

```
# Check systemd-timesyncd
timedatectl status

# View NTP sync status
timedatectl show-timesync
```

Setting Time Manually

```
# Via Redfish
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"DateTime": "2024-01-15T10:30:00+00:00"}' \
  https://localhost/redfish/v1/Managers/bmc

# Via command line
timedatectl set-time "2024-01-15 10:30:00"
```

Timezone Configuration

```
# List timezones
timedatectl list-timezones

# Set timezone
timedatectl set-timezone America/Los_Angeles

# Via D-Bus
busctl set-property xyz.openbmc_project.Time.Manager \
  /xyz/openbmc_project/time/bmc \
  xyz.openbmc_project.Time.EpochTime \
  Elapsed t $(date +%s%N)
```

Time Ownership

BMC and Host can have separate time sources:

```
# Set BMC as time owner
busctl set-property xyz.openbmc_project.Time.Manager \
    /xyz/openbmc_project/time/owner \
    xyz.openbmc_project.Time.Owner \
    TimeOwner s "xyz.openbmc_project.Time.Owner.Owners.BMC"
```

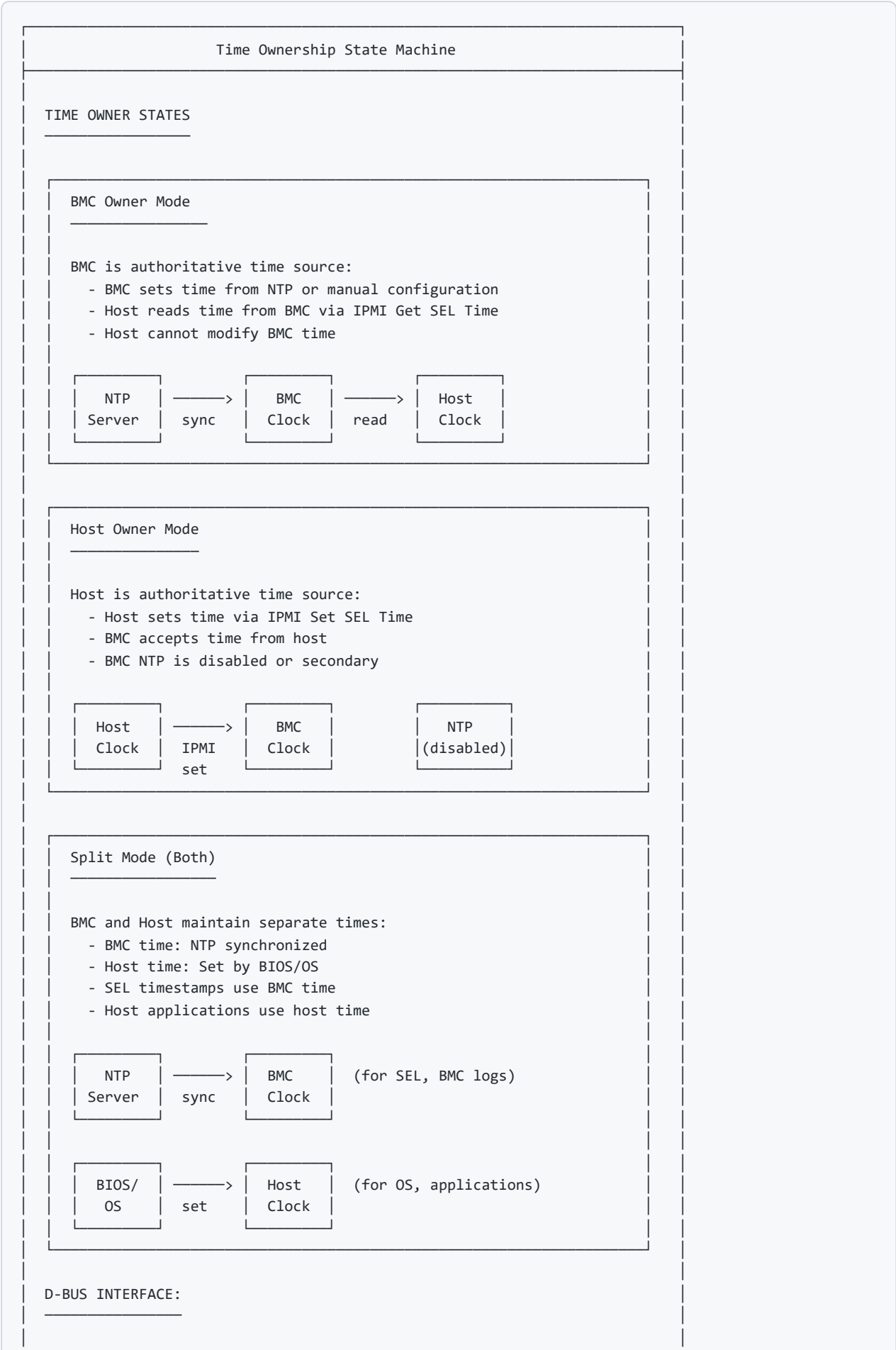
Deep Dive

Advanced implementation details for time management developers.

NTP Synchronization Architecture



BMC/Host Time Ownership Model



Object: /xyz/openbmc_project/time/owner
Interface: xyz.openbmc_project.Time.Owner

Property: TimeOwner (enum)

- xyz.openbmc_project.Time.Owner.Owners.BMC
- xyz.openbmc_project.Time.Owner.Owners.Host
- xyz.openbmc_project.Time.Owner.Owners.Both
- xyz.openbmc_project.Time.Owner.Owners.Split

RTC and System Clock Synchronization

RTC Synchronization Details

HARDWARE RTC ACCESS

/dev/rtc0 → Hardware real-time clock

RTC Registers (typical I2C or memory-mapped):

Seconds (0-59)	Minutes (0-59)	Hours (0-23)	Day (1-31)	Month (1-12)
Year (0-99)	Weekday (0-6)	Control/Status Registers (alarm, interrupt enable, etc.)		

BOOT SEQUENCE:

Power On

|

v

1. Kernel initializes with RTC time
rtc_hctosys() → reads RTC, sets system clock

CONFIG_RTC_HCTOSYS=y (kernel config)
CONFIG_RTC_HCTOSYS_DEVICE="rtc0"

|

v

2. systemd-timesyncd starts
- If NTP enabled: begins synchronization
- Adjusts system clock based on NTP

|

v

3. phosphor-time-manager starts
- Exposes D-Bus interfaces
- Handles owner mode
- Responds to time set requests

SHUTDOWN SEQUENCE:

systemd-timedated or hwclock --systohc
- Writes current system time to RTC
- Ensures RTC has latest time for next boot

hwclock --systohc --utc (store as UTC)
hwclock --systohc --localtime (store as local time)

367

TIMEZONE HANDLING:

BMC stores time in UTC internally:

System Clock: UTC (always)

RTC: UTC (recommended) or Local Time

Display: Converted to configured timezone

/etc/localtime → symlink to /usr/share/zoneinfo/<zone>

TZ environment variable (fallback)

timedatectl set-timezone America/Los_Angeles

→ Updates /etc/localtime symlink

→ Does NOT change stored time (UTC unchanged)

→ Changes how time is displayed

Source Code Reference

Key implementation files in [phosphor-time-manager](#):

File	Description
bmc_epoch.cpp	BMC time D-Bus object implementation
host_epoch.cpp	Host time D-Bus object implementation
manager.cpp	Time owner and sync mode management
utils.cpp	Utility functions for time operations
settings.cpp	Persistent settings handling

References

- [phosphor-time-manager](#)
- [systemd-timesyncd](#)

Tested on: OpenBMC master, QEMU romulus

Inventory Manager Guide

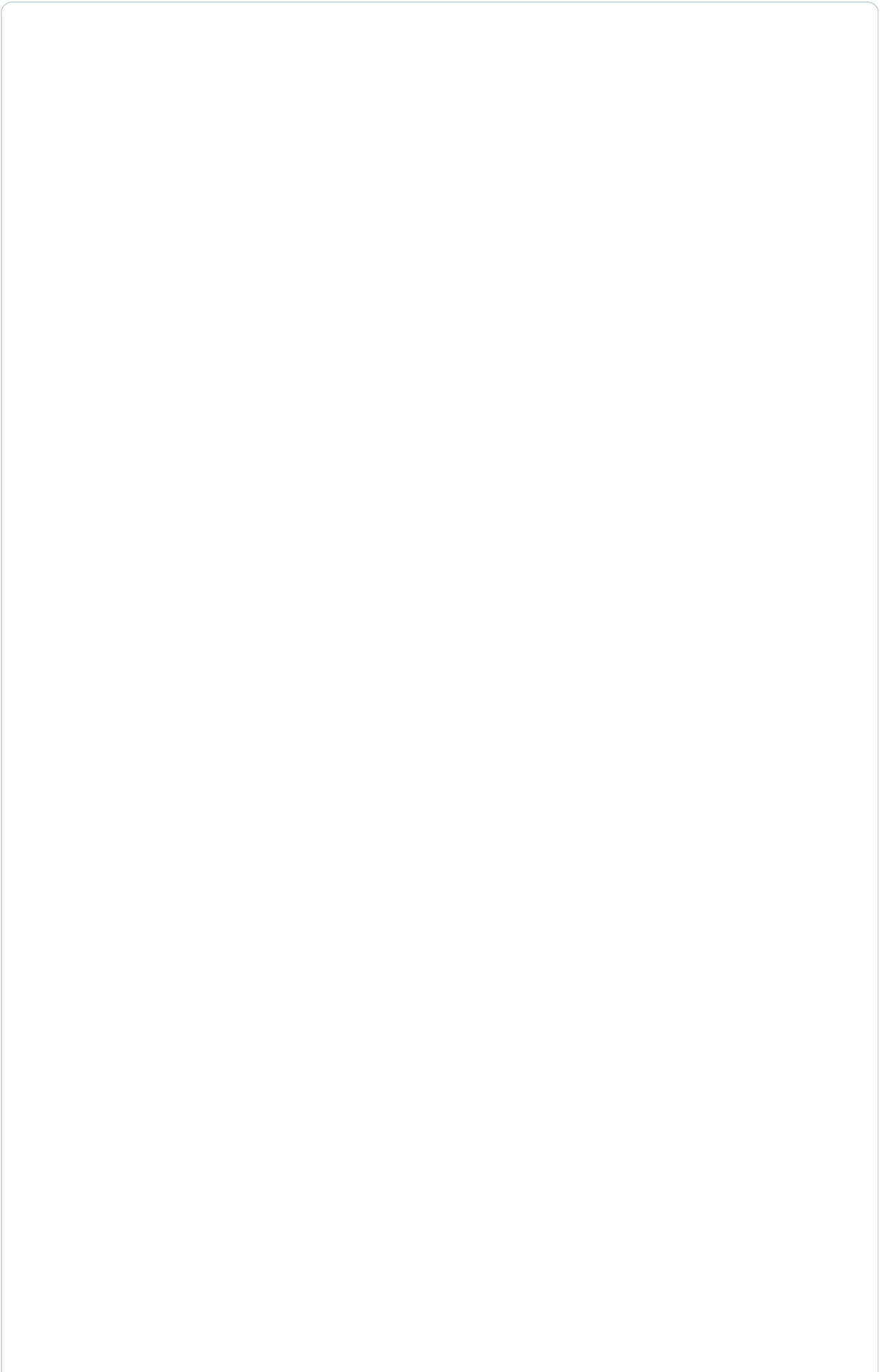
Manage FRU data and hardware inventory on OpenBMC.

Table of Contents

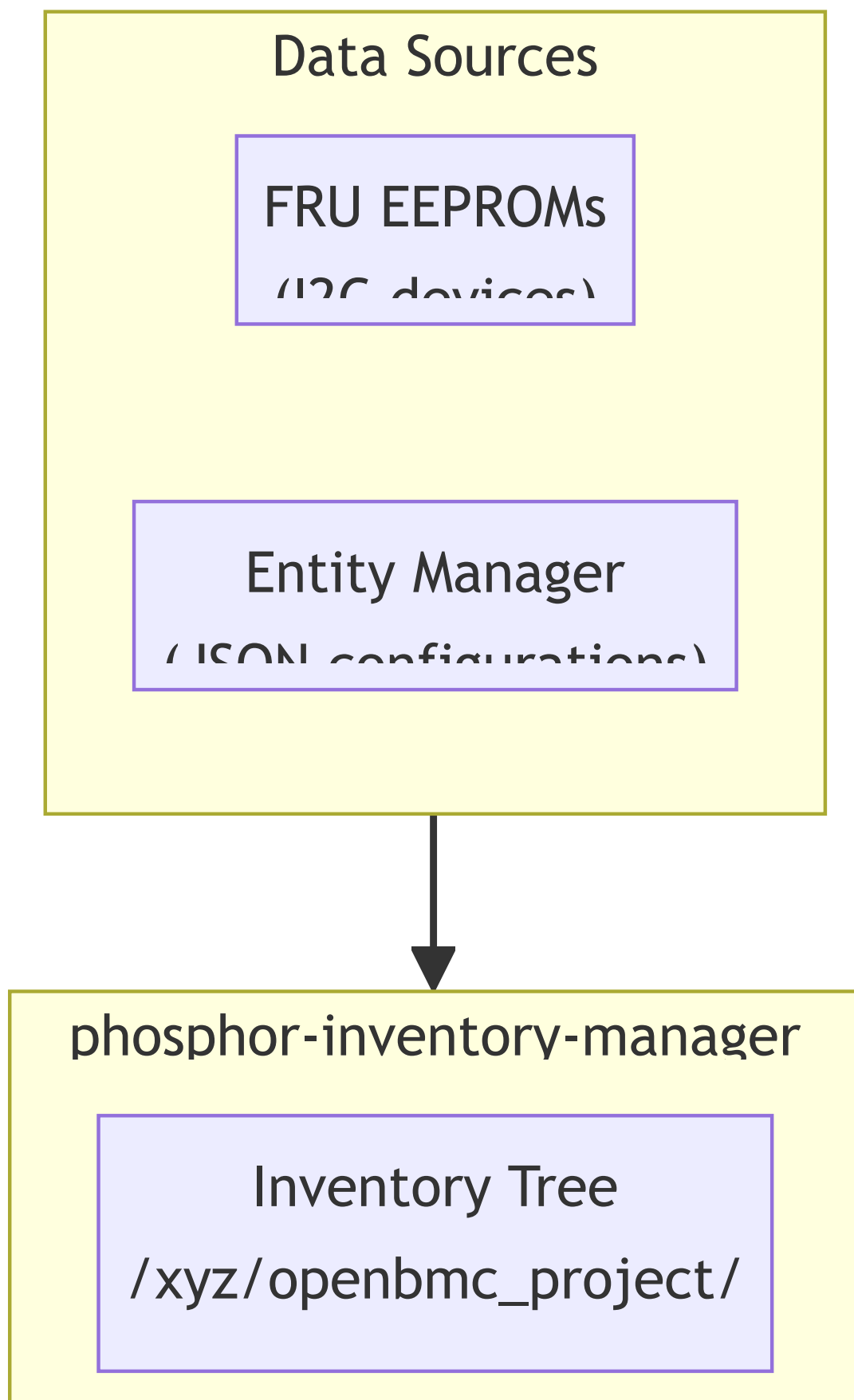
1. [Overview](#)
 2. [Setup & Configuration](#)
 1. [Build-Time Configuration](#)
 3. [Viewing Inventory](#)
 1. [Via D-Bus](#)
 2. [Via Redfish](#)
 4. [FRU Data](#)
 1. [FRU Structure](#)
 2. [Read FRU via IPMI](#)
 3. [FRU EEPROM Configuration](#)
 5. [Associations](#)
 6. [Deep Dive](#)
 1. [IPMI FRU Data Format](#)
 2. [D-Bus Inventory Object Model](#)
 3. [Association Definitions](#)
 4. [Source Code Reference](#)
 7. [References](#)
-

Overview

phosphor-inventory-manager tracks hardware components, FRU (Field Replaceable Unit) data, and component associations.



Inventory Manager Architecture



Associations

(links items)

FRU Parser

(IDMM EDIT)

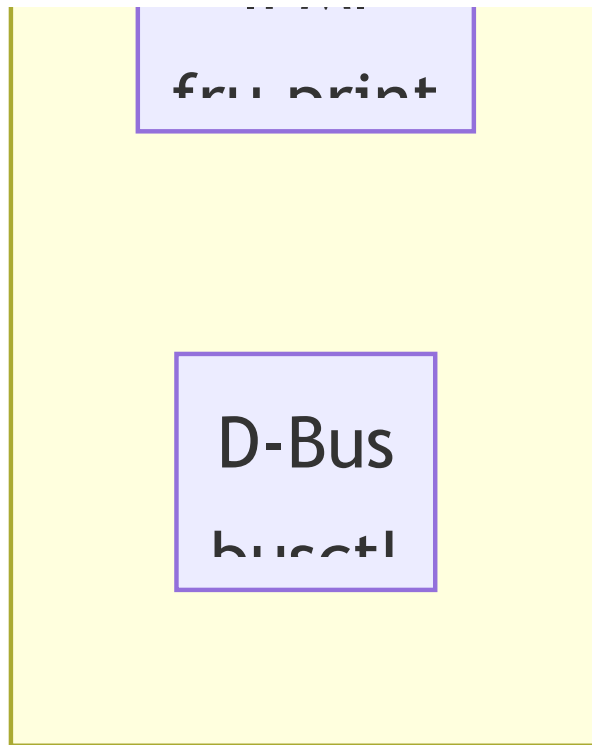


Access Interfaces

Redfish

(Chassis)

IPMI



► ASCII-art version (for comparison)

Setup & Configuration

Build-Time Configuration

```
# Include inventory manager
IMAGE_INSTALL:append = " \
    phosphor-inventory-manager \
    phosphor-fru-fault-monitor \
"
```

Viewing Inventory

Via D-Bus

```
# List inventory items
busctl tree xyz.openbmc_project.Inventory.Manager

# Get item details
busctl introspect xyz.openbmc_project.Inventory.Manager \
    /xyz/openbmc_project/inventory/system/chassis/motherboard
```

Via Redfish

```
# Get chassis inventory
curl -k -u root:OpenBmc \
  https://localhost/redfish/v1/Chassis/chassis

# Get system inventory
curl -k -u root:OpenBmc \
  https://localhost/redfish/v1/Systems/system
```

FRU Data

FRU Structure

Field	Description
BOARD_MFG	Board manufacturer
BOARD_PRODUCT	Product name
BOARD_SERIAL	Serial number
BOARD_PART_NUMBER	Part number

Read FRU via IPMI

```
# List FRU devices
ipmitool fru list

# Read FRU data
ipmitool fru print 0
```

FRU EEPROM Configuration

```
{
  "Name": "Baseboard FRU",
  "Type": "EEPROM",
  "Bus": 1,
  "Address": "0x50"
}
```

Associations

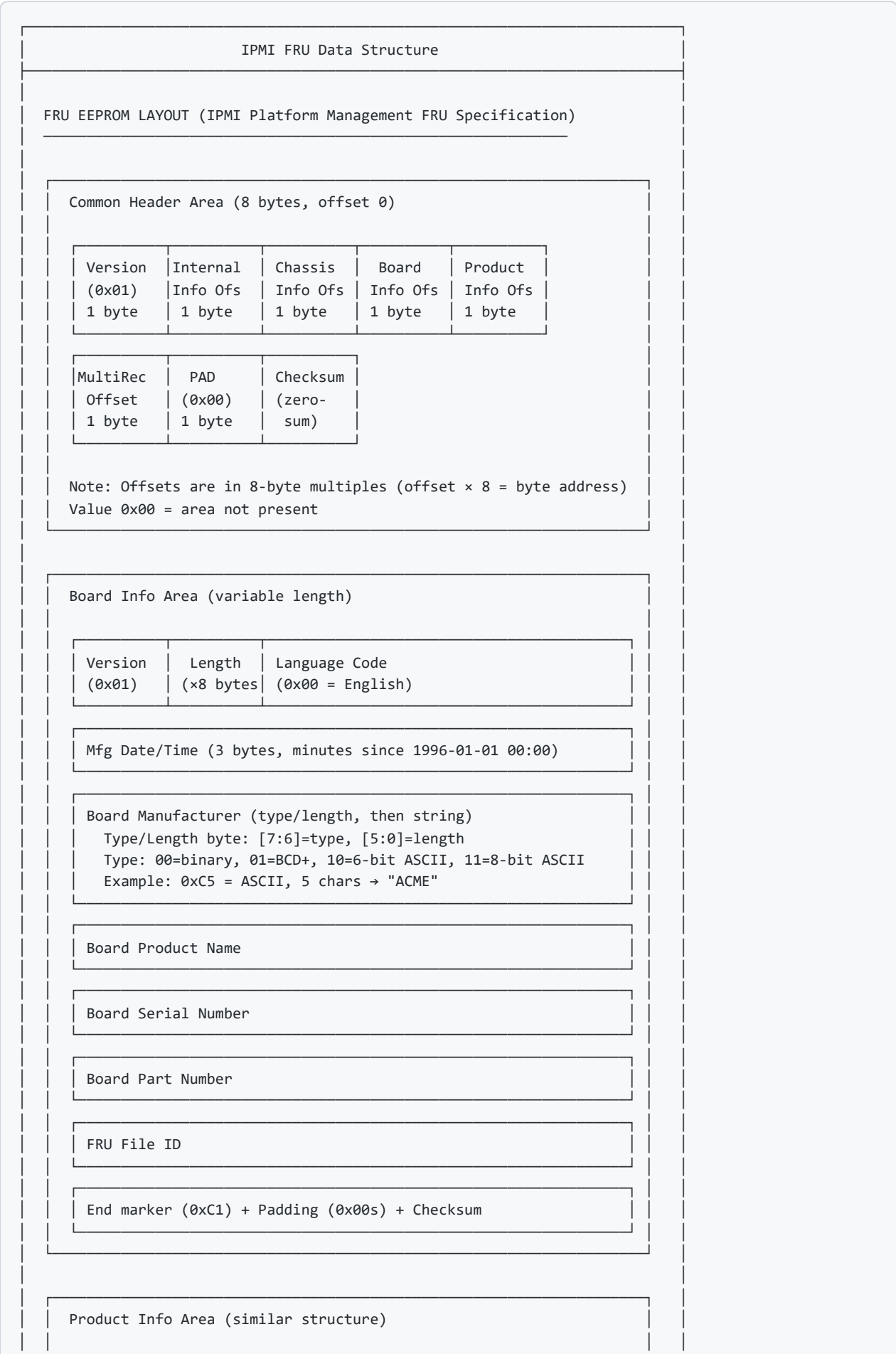
Associations link related inventory items:

```
# View associations
busctl get-property xyz.openbmc_project.Inventory.Manager \
  /xyz/openbmc_project/inventory/system/chassis \
  xyz.openbmc_project.Association.Definitions \
  Associations
```

Deep Dive

Advanced implementation details for inventory management developers.

IPMI FRU Data Format



- Product Manufacturer
- Product Name
- Product Part/Model Number
- Product Version
- Product Serial Number
- Asset Tag
- FRU File ID

D-Bus Inventory Object Model

Inventory D-Bus Object Hierarchy

D-BUS OBJECT TREE

/xyz/openbmc_project/inventory

└─ system

└─ chassis

└─ motherboard

└─ cpu0

└─ core0

└─ core1

└─ dimm0

└─ dimm1

└─ pcie_slot0

└─ gpu0

└─ powersupply0

└─ fan0

└─ bmc

INTERFACES PER OBJECT

/xyz/openbmc_project/inventory/system/chassis/motherboard

xyz.openbmc_project.Inventory.Item

└─ Present: true

└─ PrettyName: "System Board"

xyz.openbmc_project.Inventory.Decorator.Asset

└─ Manufacturer: "ACME Corp"

└─ Model: "SuperServer X100"

└─ PartNumber: "P/N-12345"

└─ SerialNumber: "SN-ABCDEF123456"

└─ BuildDate: "2024-01-15"

xyz.openbmc_project.Inventory.Decorator.Revision

└─ Version: "Rev A01"

xyz.openbmc_project.Inventory.Item.Board

(marker interface - identifies as board type)

xyz.openbmc_project.Association.Definitions

└─ Associations: [

└─ ("chassis", "containing", ".../chassis"),

└─ ("contained_by", "containing", ".../system"),

└─ ("sensors", "all_sensors", ".../sensors/temperature/mb_*")

]

ITEM TYPE INTERFACES

Interface	Used For
xyz.openbmc_project.Inventory.Item.Board	Motherboards, cards
xyz.openbmc_project.Inventory.Item.Cpu	Processors
xyz.openbmc_project.Inventory.Item.Dimm	Memory modules

xyz.openbmc_project.Inventory.Item.Fan	Cooling fans
xyz.openbmc_project.Inventory.Item.Psu	Power supplies
xyz.openbmc_project.Inventory.Item.Chassis	Enclosures
xyz.openbmc_project.Inventory.Item.Drive	Storage drives
xyz.openbmc_project.Inventory.Item.Bmc	BMC itself

Association Definitions

Inventory Association Relationships

ASSOCIATION MODEL

Associations are bidirectional relationships between inventory items. Each association has three parts: (forward, reverse, endpoint)

Containment Example:

Chassis

/inventory/
system/
chassis

Associations:
("containing"
"contained"
".../mb")

Motherboard

/inventory/
system/
chassis/
motherboard

Associations:
("contained"
"containing"
".../chass")

D-Bus property on chassis:
Associations = [("containing", "contained_by",
"/xyz/.../chassis/motherboard")]

Creates automatic endpoints:
/xyz/.../chassis/containing → points to motherboard
/xyz/.../chassis/motherboard/contained_by → points to chassis

COMMON ASSOCIATION PATTERNS

Forward	Reverse	Meaning
containing	contained_by	Physical containment
powered_by	powering	Power supply relationship
cooled_by	cooling	Fan/cooling relationship
sensors	inventory	Sensor to FRU mapping
led	identify	LED to component mapping
error_log	related_item	Error log to failed component

SENSOR-TO-INVENTORY ASSOCIATION:

CPU Temperature Sensor

/xyz/openbmc_project/sensors/temperature/cpu0_temp

Associations = [
("inventory", "all_sensors",
"/xyz/openbmc_project/inventory/system/.../cpu0")
]

380

This allows:

- Finding all sensors for a component
- Redfish Chassis/Sensors population
- Error log attribution to correct FRU

Source Code Reference

Key implementation files in [phosphor-inventory-manager](#):

File	Description
manager.cpp	Main inventory manager with D-Bus object creation
associations.cpp	Association endpoint management
errors.cpp	Error handling and logging
functor.cpp	Property change handlers
gen/generated.cpp	YAML-generated inventory definitions

References

- [phosphor-inventory-manager](#)
- [Redfish Chassis Schema](#)

Tested on: OpenBMC master, QEMU romulus

Watchdog Guide

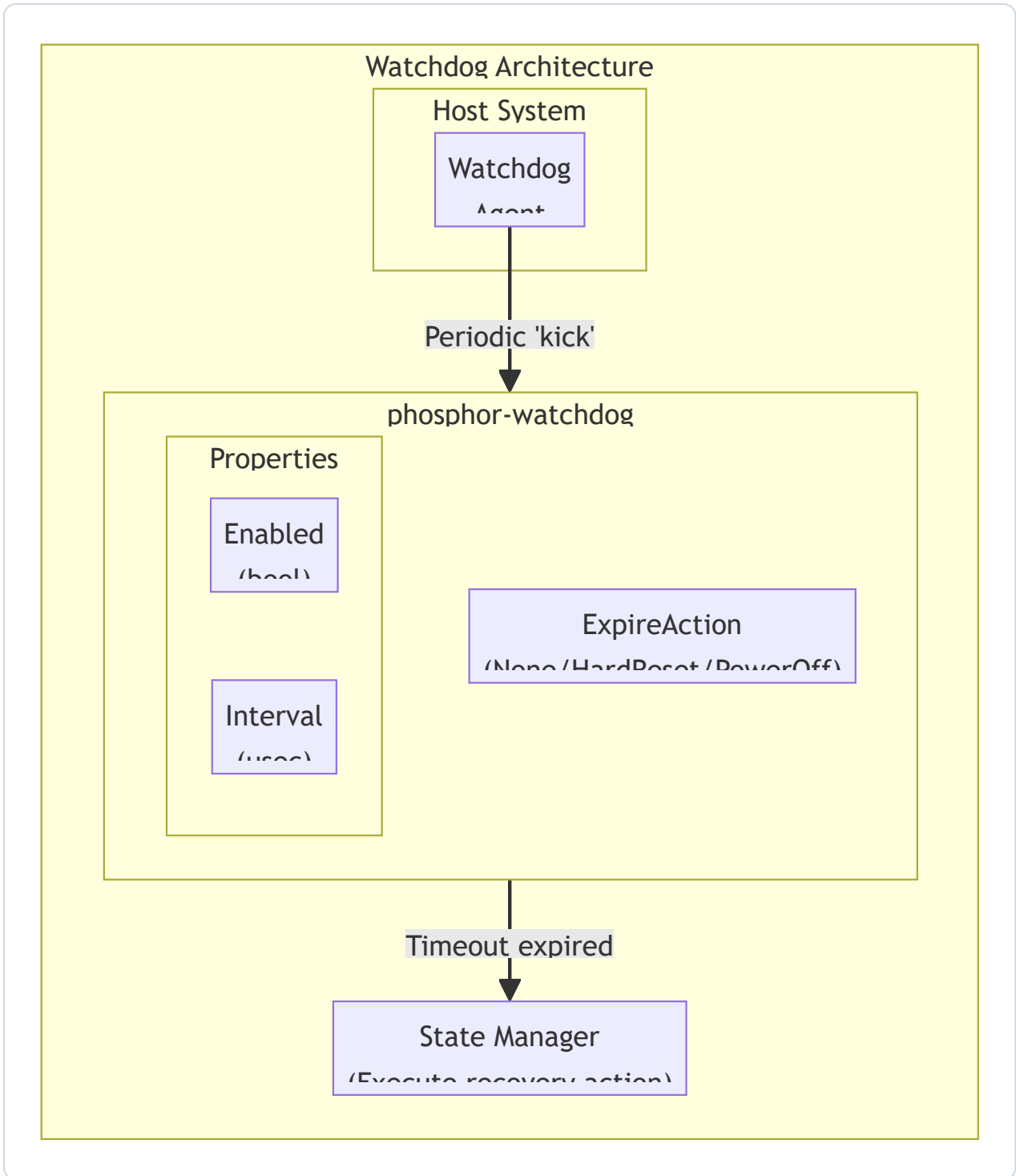
Configure host watchdog timer on OpenBMC.

Table of Contents

1. [Overview](#)
 2. [Setup & Configuration](#)
 1. [Build-Time Configuration](#)
 2. [Configuration Paths](#)
 3. [Customizing Watchdog Defaults via Yocto](#)
 4. [Expiration Actions](#)
 3. [Configuring Watchdog](#)
 1. [Via D-Bus](#)
 2. [Via IPMI](#)
 3. [Via Redfish](#)
 4. [Watchdog Operation](#)
 5. [Troubleshooting](#)
 6. [Deep Dive](#)
 1. [Watchdog Timer State Machine](#)
 2. [Timer Implementation Architecture](#)
 3. [IPMI Watchdog Timer Commands](#)
 4. [Recovery Action Sequence](#)
 5. [Source Code Reference](#)
 7. [Examples](#)
 8. [References](#)
-

Overview

phosphor-watchdog monitors host health and triggers recovery actions if the host becomes unresponsive.



Setup & Configuration

Build-Time Configuration

```
# Include watchdog
IMAGE_INSTALL:append = " phosphor-watchdog"

# Configure Meson options
EXTRA_OEMESON:pn-phosphor-watchdog = " \
    -Ddefault-action=HardReset \
    -Ddefault-timeout=300 \
"
```

Configuration Paths

phosphor-watchdog is configured primarily through Meson build options and runtime D-Bus properties. There is no standalone JSON config file.

Configuration	Where	Description
Default timeout	Meson <code>-Ddefault-timeout=300</code>	Initial countdown in seconds
Default action	Meson <code>-Ddefault-action=HardReset</code>	Action on expiration
Service template	<code>/lib/systemd/system/phosphor-watchdog@.service</code>	systemd unit template
D-Bus policy	<code>/etc/dbus-1/system.d/phosphor-watchdog.conf</code>	D-Bus access control
Runtime state	D-Bus properties (no file)	Managed via D-Bus at runtime

Customizing Watchdog Defaults via Yocto

To change the default timeout or action for your platform:

```
# meta-myplatform/recipes-phosphor/watchdog/
# └─ phosphor-watchdog_%.bbappend

cat > phosphor-watchdog_%.bbappend << 'EOF'
# Set platform-specific defaults
EXTRA_OEMESON:append = " \
    -Ddefault-action=PowerCycle \
    -Ddefault-timeout=600 \
"
EOF
```

To add a custom systemd override for the watchdog service:

```
# meta-myplatform/recipes-phosphor/watchdog/
# └─ files/
#   └─ override.conf
# └─ phosphor-watchdog_%.bbappend

cat > phosphor-watchdog_%.bbappend << 'EOF'
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
SRC_URI += "file://override.conf"

do_install:append() {
    install -d ${D}${systemd_system_unitdir}/phosphor-watchdog@watchdog-host0.service.d
    install -m 0644 ${WORKDIR}/override.conf \
        ${D}${systemd_system_unitdir}/phosphor-watchdog@watchdog-host0.service.d/
}
EOF
```

Expiration Actions

Action	Description
None	Log event only
HardReset	Force power cycle
PowerOff	Power off system
PowerCycle	Power off then on

Configuring Watchdog

Via D-Bus

```
# Enable watchdog
busctl set-property xyz.openbmc_project.Watchdog \
    /xyz/openbmc_project/watchdog/host0 \
    xyz.openbmc_project.State.Watchdog \
    Enabled b true

# Set timeout (microseconds)
busctl set-property xyz.openbmc_project.Watchdog \
    /xyz/openbmc_project/watchdog/host0 \
    xyz.openbmc_project.State.Watchdog \
    Interval t 300000000

# Set expiration action
busctl set-property xyz.openbmc_project.Watchdog \
    /xyz/openbmc_project/watchdog/host0 \
    xyz.openbmc_project.State.Watchdog \
    ExpireAction s "xyz.openbmc_project.State.Watchdog.Action.HardReset"
```

Via IPMI

```
# Get watchdog status
ipmitool mc watchdog get

# Set watchdog timeout
ipmitool mc watchdog set timeout 300

# Enable watchdog
ipmitool mc watchdog set action hard_reset

# Reset (kick) watchdog
ipmitool mc watchdog reset
```

Via Redfish

```
# Configure watchdog via HostWatchdogTimer
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{
    "HostWatchdogTimer": {
      "FunctionEnabled": true,
      "TimeoutAction": "ResetSystem",
      "WarningAction": "None"
    }
  }' \
  https://localhost/redfish/v1/Systems/system
```

Watchdog Operation

```
Host Boot → Watchdog Enabled → Host kicks watchdog periodically
                                ↓
                        Timeout expires?
                                ↓
                        Execute ExpireAction
```

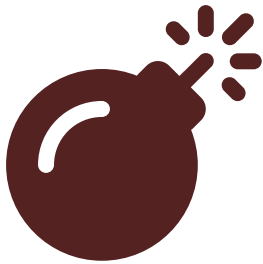
Troubleshooting

```
# Check watchdog service
systemctl status phosphor-watchdog@watchdog-host0

# View watchdog state
busctl introspect xyz.openbmc_project.Watchdog \
  /xyz/openbmc_project/watchdog/host0
```

Deep Dive

Advanced implementation details for watchdog timer developers.



Syntax error in text

mermaid version 10.9.1

Timer Properties:

Property	Description
<code>Interval</code>	Total countdown time (microseconds)
<code>TimeRemaining</code>	Current time left (decrements each tick)
<code>Enabled</code>	Whether timer is active
<code>ExpireAction</code>	What to do on timeout (None, HardReset, PowerOff, etc.)
<code>CurrentTimerUse</code>	BIOS_FRB2, BIOS_POST, OS_LOAD, SMS_OS, OEM

► ASCII-art version (for comparison)

Timer Implementation Architecture



IPMI Watchdog Timer Commands

IPMI Watchdog Protocol Details

SET WATCHDOG TIMER (0x24)

Request:

Byte	Description
1	Timer Use: [6:0] Timer Use (1=BIOS_FRB2, 2=BIOS_POST, 3=OS_LOAD, 4=SMS_OS, 5=OEM) [7] Don't log (0=log, 1=don't log)
2	Timer Actions: [2:0] Timeout action (0=None, 1=HardReset, 2=PowerDown, 3=PowerCycle) [6:4] Pre-timeout interrupt (0=None, 1=SMI, 2=NMI, 3=Messaging)
3	Pre-timeout interval (seconds)
4	Timer Use Expiration Flags Clear [0] BIOS_FRB2 [1] BIOS_POST [2] OS_LOAD [3] SMS_OS [4] OEM
5-6	Initial countdown value (100ms units, LSB first)

GET WATCHDOG TIMER (0x25)

Response:

Byte	Description
1	Completion code
2	Timer Use (same format as set)
3	Timer Actions (same format as set)
4	Pre-timeout interval
5	Timer Use Expiration Flags (set on expiration)
6-7	Initial countdown value
8-9	Present countdown value (current time remaining)

RESET WATCHDOG TIMER (0x22)

No data bytes - resets countdown to initial value

Returns error 0x80 if timer not initialized (never set)

Example ipmitool Commands:

```
# Set 5-minute timer with hard reset action, OS watchdog
ipmitool raw 0x06 0x24 0x44 0x01 0x00 0x10 0xB8 0x0B
#
#                               3000 = 5 min (100ms units)
#                               Clear flags
#                               No pre-timeout
```

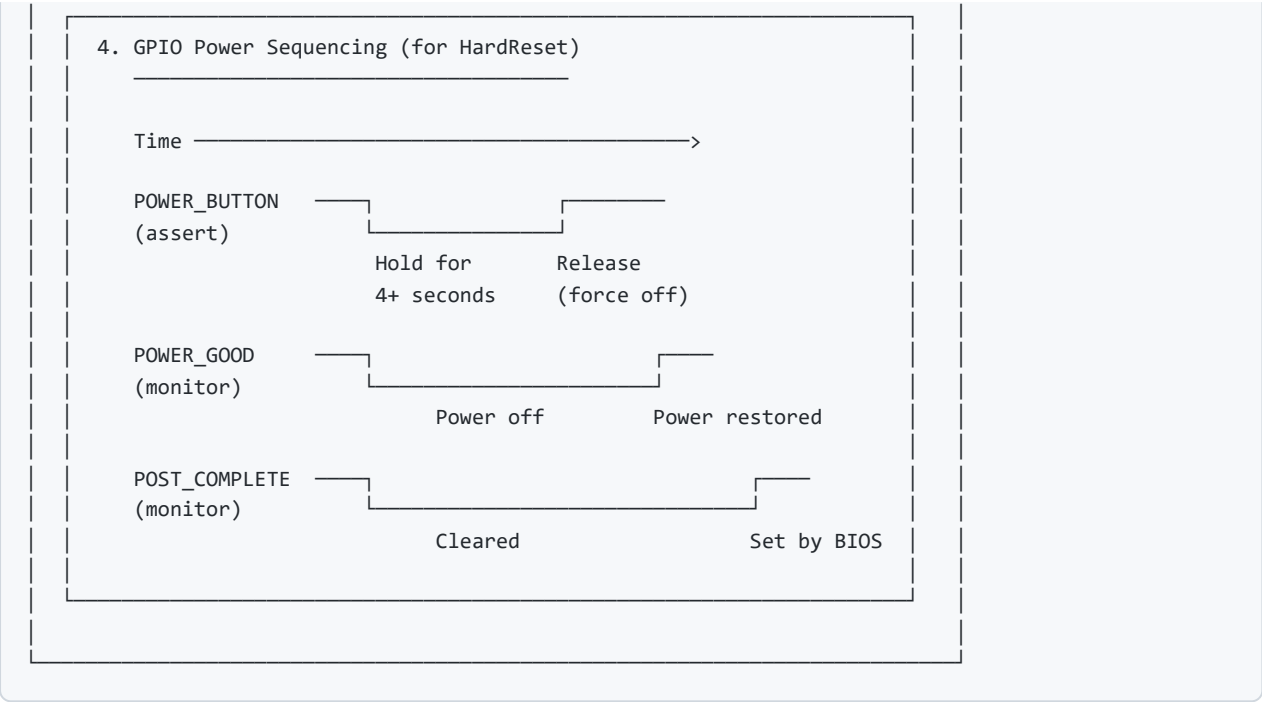
```
#                                     HardReset action
#                                     SMS_OS timer use

# Kick (reset) watchdog
ipmitool raw 0x06 0x22

# Get current status
ipmitool raw 0x06 0x25
```

Recovery Action Sequence





Source Code Reference

Key implementation files in [phosphor-watchdog](#):

File	Description
watchdog.cpp	Main watchdog class with timer loop and expiration handling
watchdog.hpp	Watchdog class definition with D-Bus property implementations
watchdog_main.cpp	Service entry point and D-Bus object creation
phosphor-host-ipmid/ipmid_watchdog.cpp	IPMI command handlers (Set/Get/Reset)

Examples

Working examples are available in the [examples/watchdog](#) directory:

- [watchdog-config.json](#) - Watchdog timer configuration
- [watchdog-test.sh](#) - Test script for watchdog functionality

References

- [phosphor-watchdog](#)
- [IPMI Watchdog Timer](#)

Tested on: OpenBMC master, QEMU romulus

Buttons Guide

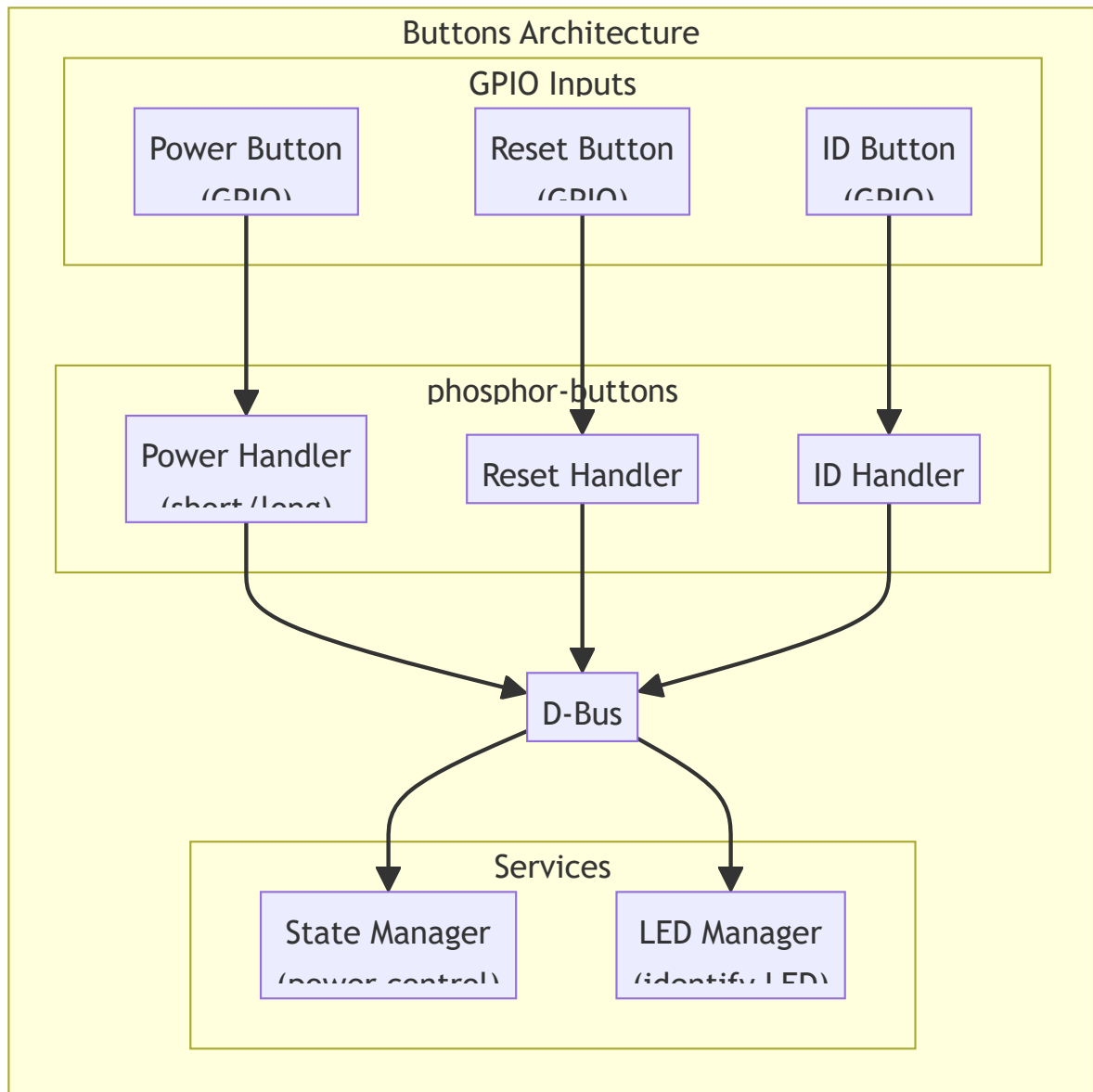
Configure power, reset, and ID buttons on OpenBMC.

Table of Contents

1. [Overview](#)
 2. [Setup & Configuration](#)
 1. [Build-Time Configuration](#)
 2. [Button Types](#)
 3. [GPIO Configuration](#)
 1. [Device Tree](#)
 2. [JSON Configuration](#)
 4. [Button Behavior](#)
 1. [Power Button](#)
 2. [Reset Button](#)
 3. [ID Button](#)
 5. [D-Bus Interface](#)
 6. [Disabling Buttons](#)
 1. [Mask Power Button](#)
 7. [Troubleshooting](#)
 8. [Deep Dive](#)
 1. [Button Event Processing Pipeline](#)
 2. [Power Button State Machine](#)
 3. [GPIO-Keys Kernel Configuration](#)
 4. [Source Code References](#)
 9. [References](#)
-

Overview

phosphor-buttons handles physical button inputs (power, reset, ID) and translates them to system actions.



Setup & Configuration

Build-Time Configuration

```
# Include buttons handler
IMAGE_INSTALL:append = " phosphor-buttons"
```

Button Types

Button	Function	Default Action
Power	Short press	Toggle power
Power	Long press (4s)	Force power off
Reset	Press	Hard reset
ID	Press	Toggle identify LED

GPIO Configuration

Device Tree

```
gpio-keys {
    compatible = "gpio-keys";

    power-button {
        label = "power-button";
        gpios = <&gpio ASPEED_GPIO(B, 0) GPIO_ACTIVE_LOW>;
        linux,code = <116>; /* KEY_POWER */
    };

    reset-button {
        label = "reset-button";
        gpios = <&gpio ASPEED_GPIO(B, 1) GPIO_ACTIVE_LOW>;
        linux,code = <0x198>; /* KEY_RESTART */
    };

    id-button {
        label = "id-button";
        gpios = <&gpio ASPEED_GPIO(B, 2) GPIO_ACTIVE_LOW>;
        linux,code = <0x199>;
    };
};
```

JSON Configuration

```
{
  "gpio_definitions": [
    {
      "name": "POWER_BUTTON",
      "pin": "GPIOB0",
      "direction": "input"
    },
    {
      "name": "RESET_BUTTON",
      "pin": "GPIOB1",
      "direction": "input"
    },
    {
      "name": "ID_BUTTON",
      "pin": "GPIOB2",
      "direction": "input"
    }
  ]
}
```

Button Behavior

Power Button

```
# Short press (< 4 seconds)
# Action: Request power state toggle

# Long press (≥ 4 seconds)
# Action: Force power off
```

Reset Button

```
# Press triggers hard reset
# Equivalent to: obmcutil hostreboot
```

ID Button

```
# Press toggles identify LED
# Can be configured for latching or momentary
```

D-Bus Interface

```
# Check button status
busctl tree xyz.openbmc_project.Chassis.Buttons

# Monitor button events
busctl monitor xyz.openbmc_project.Chassis.Buttons
```


Disabling Buttons

Mask Power Button

```
# Via D-Bus
busctl set-property xyz.openbmc_project.Chassis.Buttons \
  /xyz/openbmc_project/Chassis/Buttons/Power0 \
  xyz.openbmc_project.Chassis.Buttons \
  Enabled b false
```

Troubleshooting

```
# Check GPIO input
cat /sys/class/gpio/gpio*/value

# Check button service
systemctl status phosphor-button-handler

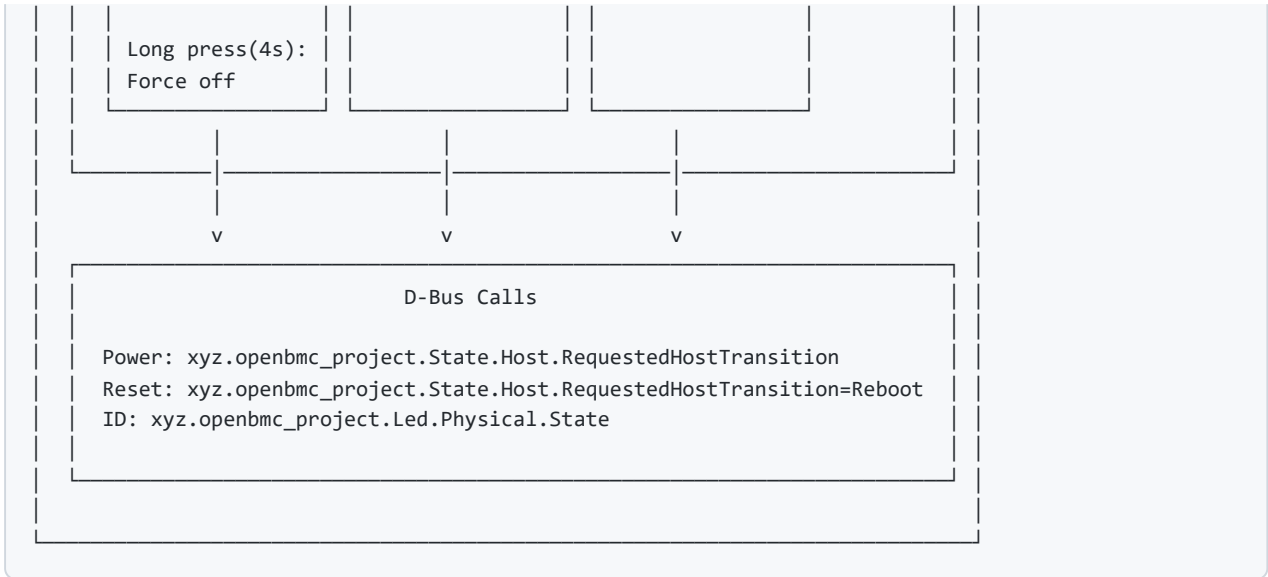
# Monitor key events
evtest /dev/input/event0
```

Deep Dive

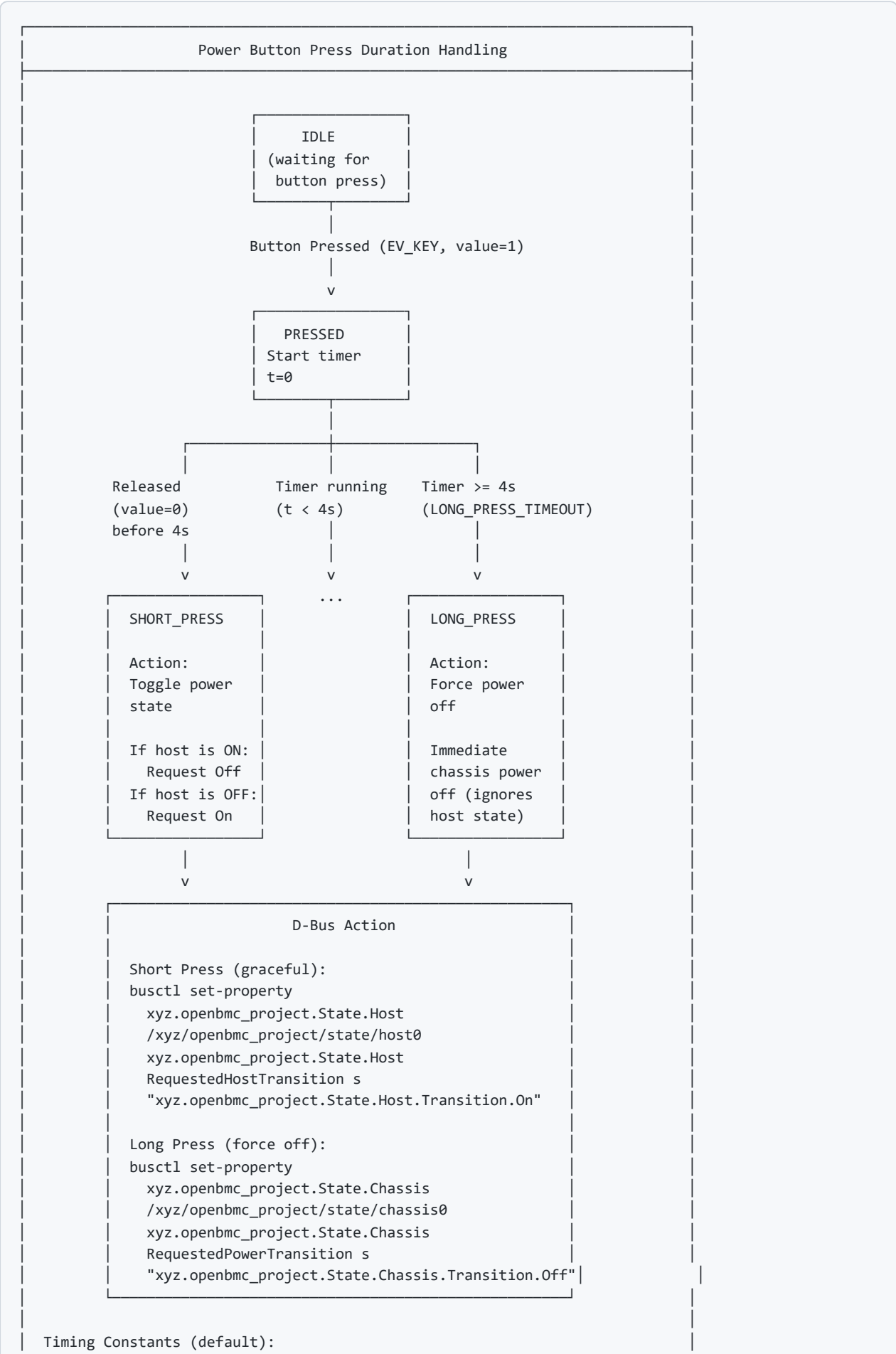
This section provides detailed technical information for developers who want to understand button handling internals in OpenBMC.

Button Event Processing Pipeline





Power Button State Machine



```
SHORT_PRESS_MAX_MS: 500    // Debounce threshold  
LONG_PRESS_MIN_MS:  4000   // Force power off threshold  
DEBOUNCE_MS:        50     // Hardware debounce
```

GPIO-Keys Kernel Configuration

gpio-keys Device Tree Configuration

Device Tree Node:

```
gpio-keys {
    compatible = "gpio-keys";
    #address-cells = <1>;
    #size-cells = <0>;

    power-button {
        label = "power-button";
        linux,code = <KEY_POWER>;           // 116
        gpios = <&gpio0 ASPEED_GPIO(B, 0) GPIO_ACTIVE_LOW>;
        debounce-interval = <50>;           // ms
        wakeup-source;                       // Can wake from suspend
    };

    reset-button {
        label = "reset-button";
        linux,code = <KEY_RESTART>;          // 0x198 = 408
        gpios = <&gpio0 ASPEED_GPIO(B, 1) GPIO_ACTIVE_LOW>;
        debounce-interval = <50>;
    };

    id-button {
        label = "id-button";
        linux,code = <KEY_SCROLLLOCK>;       // 70 (or custom)
        gpios = <&gpio0 ASPEED_GPIO(B, 2) GPIO_ACTIVE_LOW>;
        debounce-interval = <50>;
    };
};
```

Linux Input Key Codes (linux/input-event-codes.h):

Code	Value	Typical Use
KEY_POWER	116	Power button (standard)
KEY_RESTART	408	Reset/Restart button
KEY_SLEEP	142	Sleep button (alternative)
KEY_SUSPEND	205	Suspend button
KEY_PROG1	148	Programmable button 1 (ID button alternative)
KEY_PROG2	149	Programmable button 2

ASPEED GPIO Mapping:

ASPEED_GPIO(group, pin)

Group A: GPIO 0-7

Group E: GPIO 32-39

Group I: GPIO 64-71

Group B: GPIO 8-15

Group F: GPIO 40-47

Group J: GPIO 72-79

Group C: GPIO 16-23

Group G: GPIO 48-55

Group K: GPIO 80-87

Group D: GPIO 24-31

Group H: GPIO 56-63

...

Example: ASPEED_GPIO(B, 0) = 8 + 0 = GPIO 8

Source Code References

Component	Repository	Key Files
phosphor-buttons	openbmc/phosphor-buttons	src/main.cpp , src/power_button.cpp , src/id_button.cpp
Button Handler	openbmc/phosphor-buttons	src/button_handler.cpp
gpio-keys driver	linux/drivers/input	drivers/input/keyboard/gpio_keys.c
Input subsystem	linux/include/uapi/linux	include/uapi/linux/input-event-codes.h
D-Bus interfaces	openbmc/phosphor-dbus-interfaces	yaml/xyz/openbmc_project/Chassis/Buttons.interface.yaml

References

- [phosphor-buttons](#)
- [Linux gpio-keys](#)

Tested on: OpenBMC master, requires hardware GPIO support

GPIO Management Guide

Configure phosphor-multi-gpio-monitor to detect hardware signals and trigger system actions on OpenBMC.

Table of Contents

1. [Overview](#)
2. [Architecture](#)
 1. [Data Flow](#)
 2. [D-Bus Interfaces](#)
 3. [Key Dependencies](#)
3. [GPIO Line Name Discovery](#)
 1. [Device Tree gpio-line-names Property](#)
 2. [Discovering Lines with gpioinfo](#)
4. [phosphor-multi-gpio-monitor Configuration](#)
 1. [Configuration File Format](#)
 2. [Annotated Example](#)
 3. [Event Types Explained](#)
5. [Event-to-Systemd-Target Integration](#)
 1. [How It Works](#)
 2. [Standard OpenBMC Targets](#)
 3. [Creating Custom Systemd Targets](#)
 4. [Multiple Targets per Event](#)
6. [Continuous vs One-Shot Monitoring](#)
 1. [Choosing the Right Mode](#)
7. [Platform bbappend Customization](#)
 1. [bbappend File](#)
 2. [Build-Time Options](#)
8. [Presence Detection \(phosphor-multi-gpio-presence\)](#)
9. [Porting Guide](#)
10. [Code Examples](#)
 1. [Example 1: Minimal Power Button Configuration](#)
 2. [Example 2: Multi-Signal Platform Configuration](#)
 3. [Example 3: Using GpioNum Instead of LineName](#)
11. [Troubleshooting](#)
 1. [Issue: Monitor Service Fails to Start](#)
 2. [Issue: GPIO Event Not Triggering Target](#)
 3. [Issue: Presence Detection Only Works Once](#)
 4. [Debug Commands](#)
12. [References](#)

1. [Official Resources](#)
 2. [Related Guides](#)
 3. [External Documentation](#)
-

Overview

phosphor-multi-gpio-monitor watches named GPIO lines for edge or level changes and triggers systemd targets or D-Bus actions in response. It replaces the older single-line phosphor-gpio-monitor with a unified JSON configuration that supports multiple GPIO signals in one service instance.

Every OpenBMC platform relies on GPIO monitoring for critical functions: detecting power button presses, sensing chassis intrusion, tracking device presence, and responding to hardware fault signals. This guide covers discovering GPIO line names, writing the JSON configuration, wiring events to systemd targets, choosing between continuous and one-shot monitoring, and deploying through a Yocto bbappend file.

Key concepts covered:

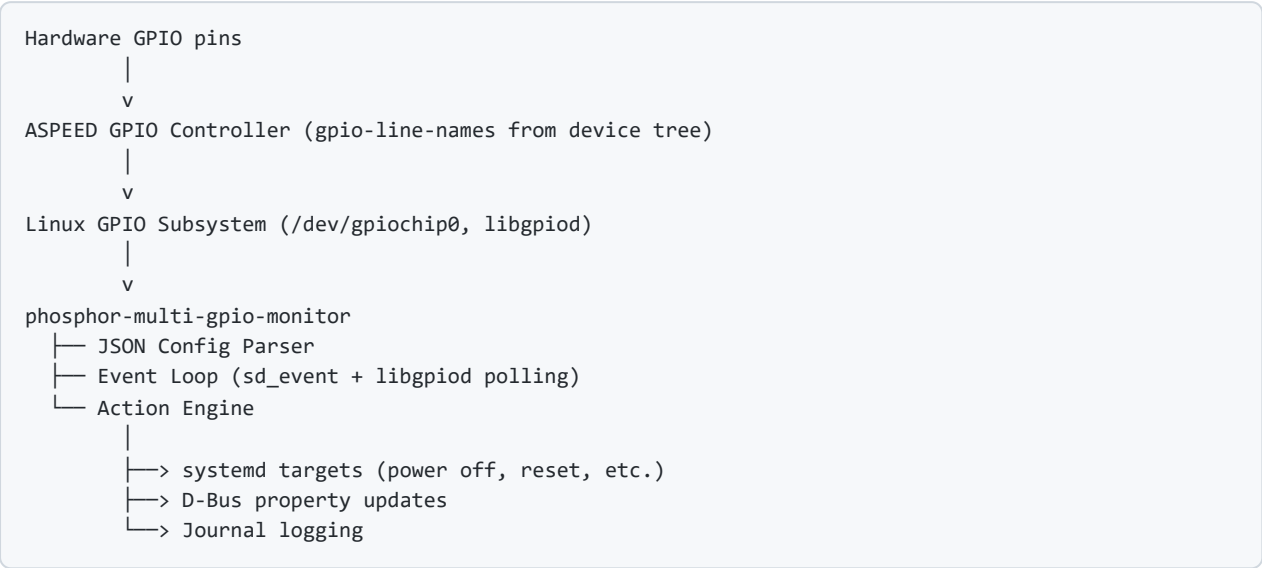
- phosphor-multi-gpio-monitor JSON configuration format
- GPIO line name mapping from device tree to libgpiod
- Edge event types: RISING, FALLING, BOTH
- Event-to-systemd-target integration
- Continuous vs one-shot monitoring modes
- Platform bbappend customization for deployment

GPIO monitoring is **partially testable** in QEMU. The ast2600-evb QEMU target provides emulated GPIO pins that you can toggle from the QEMU monitor, but real edge timing and electrical behavior require physical hardware.

Architecture

phosphor-multi-gpio-monitor reads a JSON configuration file at startup, opens each named GPIO line through libgpiod, and polls for events. When an event matches the configured edge type, the monitor activates a systemd target or calls a D-Bus method.

Data Flow



D-Bus Interfaces

Interface	Object Path	Description
<code>xyz.openbmc_project.Chassis.Intrusion</code>	<code>/xyz/openbmc_project/Chassis/Intrusion</code>	Chassis intrusion detection status
<code>xyz.openbmc_project.Inventory.Item</code>	<code>/xyz/openbmc_project/inventory/system/chassis/<device></code>	Device presence state
<code>xyz.openbmc_project.State.Chassis</code>	<code>/xyz/openbmc_project/state/chassis0</code>	Chassis power state transitions

Key Dependencies

- **libgpiod**: Provides the userspace API for accessing GPIO lines by name
- **systemd**: Receives target activation requests from the monitor
- **phosphor-dbus-interfaces**: Defines standard D-Bus interfaces for chassis and inventory objects
- **Device tree gpio-line-names**: Maps hardware GPIO numbers to human-readable line names

GPIO Line Name Discovery

Before you write any configuration, you need to discover the GPIO line names available on your platform. These names come from the device tree and are the keys you use in the JSON configuration.

Device Tree gpio-line-names Property

The ASPEED device tree assigns human-readable names to GPIO pins through the `gpio-line-names` property on each GPIO controller node. Each string in the array maps positionally to a GPIO offset on that controller.

```
/* Example from aspeed-bmc-myplatform.dts */
&gpio0 {
    gpio-line-names =
        /* A0-A7 */  "", "", "", "", "", "", "", "",
        /* B0-B7 */  "POWER_BUTTON", "RESET_BUTTON", "", "", "", "", "", "",
        /* C0-C7 */  "", "", "", "", "", "", "", "",
        /* D0-D7 */  "PS0_PRESENT", "PS1_PRESENT", "", "", "", "", "", "",
        /* E0-E7 */  "", "", "CHASSIS_INTRUSION", "", "", "", "", "",
        /* F0-F7 */  "FAN0_PRESENT", "FAN1_PRESENT", "FAN2_PRESENT", "FAN3_PRESENT",
                    "", "", "", "";
};
```

The position of each name in the `gpio-line-names` array directly corresponds to the GPIO offset on that chip. An empty string `""` means that pin has no assigned name. Count from offset 0 at the start of the array.

Discovering Lines with `gpioinfo`

Use `gpioinfo` on a running BMC to list all available GPIO lines and their current state. This is the fastest way to find the correct line name for your configuration.

```
# List all GPIO chips and their lines
gpioinfo
# gpiochip0 - 232 lines:
#   line   8: "POWER_BUTTON" unused input active-high
#   line  24: "PS0_PRESENT"  unused input active-high
#   line  34: "CHASSIS_INTRUSION" unused input active-high

# Filter for a specific line name
gpioinfo | grep -i "power_button"

# Read current value / monitor events on a line
gpioget gpiochip0 8
gpiomon --num-events=1 --rising-edge gpiochip0 8
```

GPIO line names are platform-specific. The names in your device tree must match exactly (case-sensitive) with the names in your phosphor-multi-gpio-monitor JSON configuration. A mismatch causes the monitor to fail silently for that line.

In the QEMU ast2600-evb environment, run `gpiodetect` and `gpioinfo | grep -v unnamed` to see which named lines are available for testing.

phosphor-multi-gpio-monitor Configuration

The monitor reads its configuration from a JSON file, typically installed at `/etc/phosphor-multi-gpio-monitor.json` or `/usr/share/phosphor-gpio-monitor/phosphor-multi-gpio-monitor.json`.

Configuration File Format

The configuration file is a **top-level JSON array**; each entry defines one GPIO line to monitor.

```
{
  "Name": "<gpio-line-name>",
  "LineName": "<gpio-line-name>",
  "GpioNum": 0,
  "ChipId": "",
  "EventMon": "<RISING|FALLING|BOTH>",
  "Target": "<systemd-unit-for-any-event>",
  "Targets": {
    "<RISING|FALLING|INIT_HIGH|INIT_LOW>": ["<systemd-target-to-activate>"]
  },
  "Continue": <true|false>
}
```

Field	Type	Description
<code>Name</code>	string	Human-readable name for logging
<code>LineName</code>	string	GPIO line name from device tree (must match <code>gpio-line-names</code>)
<code>GpioNum</code>	integer	GPIO offset number (used only if <code>LineName</code> is empty)
<code>ChipId</code>	string	GPIO chip path (used only if <code>LineName</code> is empty, e.g., <code>/dev/gpiochip0</code>)
<code>EventMon</code>	string	Edge type to watch: <code>RISING</code> , <code>FALLING</code> , or <code>BOTH</code> (default <code>BOTH</code>)
<code>Target</code>	string	Optional single systemd unit started on any monitored event
<code>Targets</code>	object	Optional map of event name — <code>RISING</code> , <code>FALLING</code> , or <code>INIT_HIGH</code> / <code>INIT_LOW</code> (evaluated once at startup from the line's level) — to arrays of systemd units
<code>Continue</code>	boolean	<code>true</code> for continuous monitoring, <code>false</code> for one-shot

Prefer `LineName` over `GpioNum` + `ChipId`. Named lines are portable across kernel versions and device tree changes, while raw GPIO numbers can shift when pins are remapped.

Each GPIO entry also accepts optional `ActiveLow` (bool) and `Bias` (`AS_IS` / `DISABLE` / `PULL_UP` / `PULL_DOWN`) fields that configure the libgpiod line request. In addition to runtime edges, the monitor can fire targets for a line's *initial* power-on level using the `INIT_HIGH` / `INIT_LOW` target keys.

The configuration file is a **top-level JSON array** of entries — there is no outer wrapper object. Use `Target` for a single unit fired on any event, or `Targets` to map specific events (`RISING` / `FALLING`, plus `INIT_HIGH` / `INIT_LOW`) evaluated once at startup from the line's current level) to unit lists.

Annotated Example

The following configuration monitors a power button (one-shot, falling edge) and chassis intrusion sensor (continuous, both edges). Save this as `phosphor-multi-gpio-monitor.json`:

```
[
  {
    "Name": "PowerButton",           // Human-readable label for logs
    "LineName": "POWER_BUTTON",     // Must match device tree gpio-line-names
    "GpioNum": 0,                   // Ignored when LineName is set
    "ChipId": "",                  // Ignored when LineName is set
    "EventMon": "FALLING",          // Trigger on button press (active-low)
    "Targets": {
      "FALLING": ["obmc-chassis-hard-poweroff@0.target"]
    },
    "Continue": false                // One-shot: stop watching after first event
  },
  {
    "Name": "ChassisIntrusion",
    "LineName": "CHASSIS_INTRUSION",
    "GpioNum": 0,
    "ChipId": "",
    "EventMon": "BOTH",             // Detect both open and close
    "Targets": {
      "FALLING": ["chassis-intrusion-detected@0.target"],
      "RISING": ["chassis-intrusion-cleared@0.target"]
    },
    "Continue": true                // Continuous: keep watching
  }
]
```

See [Example 2](#) for a fuller production configuration with presence detection signals.

Event Types Explained

EventMon	Trigger Condition	Typical Use Case
RISING	GPIO transitions from low (0) to high (1)	Button release, signal asserted high
FALLING	GPIO transitions from high (1) to low (0)	Button press (active-low), signal deasserted
BOTH	Any transition in either direction	Presence detection, intrusion (need to detect insert and removal)

Most physical buttons are active-low (pressed = 0). Use **FALLING** to trigger on the press and **RISING** to trigger on the release. For presence pins that you need to track in both states, use **BOTH**.

Event-to-Systemd-Target Integration

When a GPIO event fires, phosphor-multi-gpio-monitor activates one or more systemd targets listed in the **Targets** field. This is the primary mechanism for connecting hardware signals to software actions.

How It Works

When the monitor detects an event, it determines the event name (**RISING** or **FALLING**), looks up the matching array in **Targets**, and calls `systemctl start <target>` for each one. A single **Target**, if defined, is started on every event.

Standard OpenBMC Targets

Target	Purpose
<code>obmc-chassis-hard-poweroff@0.target</code>	Immediately cut chassis power
<code>obmc-host-shutdown@0.target</code>	Graceful host shutdown
<code>obmc-host-reset@0.target</code>	Host reset
<code>obmc-host-start@0.target</code>	Start host power-on sequence
<code>obmc-chassis-powerreset@0.target</code>	Chassis power cycle

Creating Custom Systemd Targets

For signals that do not map to standard targets, create your own target and service pair.

Target file (`chassis-intrusion-detected@.target`):

```
[Unit]
Description=Chassis Intrusion Detected on %i
Wants=log-chassis-intrusion@%i.service
```

Service file (`log-chassis-intrusion@.service`):

```
[Unit]
Description=Log Chassis Intrusion Event for %i

[Service]
Type=oneshot
ExecStart=/usr/bin/busctl set-property xyz.openbmc_project.Chassis.Intrusion \
  /xyz/openbmc_project/Chassis/Intrusion \
  xyz.openbmc_project.Chassis.Intrusion \
  Status s "HardwareIntrusion"
```

Then reference the target in your JSON config `Targets` field (see the [annotated example](#) for chassis intrusion wiring).

Multiple Targets per Event

You can activate multiple targets for a single event. List all targets in the array:

```
"Targets": {
  "FALLING": [
    "obmc-chassis-hard-poweroff@0.target",
    "log-power-button-pressed@0.target"
  ]
}
```

Systemd target activation is asynchronous. The monitor does not wait for the target to complete before resuming GPIO polling. If you need ordering guarantees between targets, use `After=` and `Before=` directives within the target unit files.

Continuous vs One-Shot Monitoring

The `Continue` field controls whether the monitor keeps watching a GPIO line after the first event.

- **One-shot** (`Continue: false`) – After the first matching edge, the monitor stops watching this line. Use this for signals where you only respond once (e.g., a power button press that triggers a shutdown). The service continues running for other configured lines.
- **Continuous** (`Continue: true`) – The monitor fires the appropriate target on every transition, indefinitely. Use this for signals that change repeatedly (e.g., device presence, chassis intrusion).

Choosing the Right Mode

Signal Type	Continue	EventMon	Rationale
Power button	<code>false</code>	<code>FALLING</code>	Single press triggers shutdown; further presses are ignored
Reset button	<code>false</code>	<code>FALLING</code>	Single press triggers reset
Chassis intrusion	<code>true</code>	<code>BOTH</code>	Must detect both intrusion and restoration
PSU presence	<code>true</code>	<code>BOTH</code>	PSU can be hot-swapped multiple times
Fan presence	<code>true</code>	<code>BOTH</code>	Fan trays can be inserted and removed
Fault signal	<code>true</code>	<code>FALLING</code>	Fault can recur after being cleared

Platform bbappend Customization

To deploy your GPIO configuration on a custom platform, create a bbappend file in your machine layer that installs the JSON configuration and any custom systemd targets.

bbappend File

Place this file and a `files/` directory containing your JSON config and custom systemd units at `meta-myplatform/recipes-phosphor/gpio/phosphor-gpio-monitor/`:

```
# phosphor-gpio-monitor_%.bbappend

FILESEXTRAPATHS:prepend := "${THISDIR}/${PN}:"

SRC_URI += " \
    file://phosphor-multi-gpio-monitor.json \
    file://chassis-intrusion-detected@.target \
    file://log-chassis-intrusion@.service \
"

do_install:append() {
    install -d ${D}${datadir}/phosphor-gpio-monitor
    install -m 0644 ${WORKDIR}/phosphor-multi-gpio-monitor.json \
        ${D}${datadir}/phosphor-gpio-monitor/

    install -d ${D}${systemd_system_unitdir}
    for f in chassis-intrusion-detected@.target log-chassis-intrusion@.service; do
        install -m 0644 ${WORKDIR}/${f} ${D}${systemd_system_unitdir}/
    done
}

FILES:${PN} += " \
    ${datadir}/phosphor-gpio-monitor/phosphor-multi-gpio-monitor.json \
    ${systemd_system_unitdir}/*.target \
    ${systemd_system_unitdir}/log-chassis-intrusion@.service \
"
```

Build-Time Options

Option	Default	Description
<code>OBMC_GPIO_MONITOR_INSTANCES</code>	<code>multi</code>	Monitor instance type (<code>multi</code> for multi-line JSON config)
<code>SYSTEMD_SERVICE:\${PN}</code>	<code>phosphor-multi-gpio-monitor.service</code>	Systemd service name

Enable the GPIO monitor in your image configuration:

```
# In your machine.conf or local.conf
IMAGE_INSTALL:append = " phosphor-gpio-monitor"
```

The `FILESEXTRAPATHS:prepend` variable tells BitBake to look in your layer's directory first when resolving files listed in `SRC_URI`. This allows you to override the default configuration without patching the upstream recipe.

Some platforms install the JSON configuration to `/etc/` instead of `/usr/share/`. Run `systemctl cat phosphor-multi-gpio-monitor.service | grep ExecStart` to check which `--config` path your service expects, and match your bbappend install path accordingly.

Presence Detection (phosphor-multi-gpio-presence)

The monitor covered above fires **systemd targets** on GPIO edges. A sibling daemon in the same repository, `phosphor-multi-gpio-presence`, instead maps a GPIO directly to **inventory presence** — no systemd target needed. On each edge it updates the Inventory Manager's `Present` property (locating the manager through the ObjectMapper) and can bind or unbind the component's device driver in sysfs for hot-plug support.

Its JSON config names the GPIO plus the **inventory object path** it controls, rather than a `Targets` map. A representative entry looks like:

```
{
  "Name": "fan0",
  "LineName": "FAN0_PRESENT",
  "Inventory": "/xyz/openbmc_project/inventory/system/chassis/fan0",
  "ActiveLow": true,
  "Bias": "PULL_UP"
}
```

The presence daemon's config keys are `Name`, `LineName` (or `GpioNum` + `ChipId`), `Inventory`, an optional `ExtraInterfaces` list, `ActiveLow`, and `Bias` — a schema distinct from the monitor's `Targets`-based one.

Porting Guide

Follow these steps to enable GPIO monitoring on your platform:

1. **Discover GPIO line names** – Boot your platform (or QEMU) and run `gpioinfo | grep -v unnamed`. Record every signal you need to monitor.
2. **Write the JSON configuration** – Create `phosphor-multi-gpio-monitor.json` using the [annotated example](#) as a starting point. Adjust line names, event types, and targets for your hardware.
3. **Create custom targets** (if needed) – For signals without standard OpenBMC targets, create `.target` and `.service` unit files as shown in the [Event-to-Systemd-Target Integration](#) section.
4. **Create the bbappend** – Follow the [Platform bbappend Customization](#) section to install your configuration.
5. **Build and verify:**

```
# Build the image
bitbake obmc-phosphor-image

# Flash and boot the BMC, then verify the service
systemctl status phosphor-multi-gpio-monitor

# Check the monitor is watching the expected lines
journalctl -u phosphor-multi-gpio-monitor | head -20

# Verify GPIO lines are claimed
gpioinfo | grep -E "POWER_BUTTON|CHASSIS_INTRUSION|PS0_PRESENT"
# Lines should show "[used]" instead of "unused"
```

Code Examples

Example 1: Minimal Power Button Configuration

This is the simplest possible configuration – one GPIO line that triggers a hard power off when pressed.

```
[
  {
    "Name": "PowerButton",
    "LineName": "POWER_BUTTON",
    "GpioNum": 0,
    "ChipId": "",
    "EventMon": "FALLING",
    "Targets": {
      "FALLING": [
        "obmc-chassis-hard-poweroff@0.target"
      ]
    },
    "Continue": false
  }
]
```

Example 2: Multi-Signal Platform Configuration

A production configuration monitoring power, reset, intrusion, and fan presence. Duplicate the fan entry pattern for additional fans (FAN1, FAN2, FAN3, etc.).

```
[
  {
    "Name": "PowerButton",
    "LineName": "POWER_BUTTON",
    "GpioNum": 0,
    "ChipId": "",
    "EventMon": "FALLING",
    "Targets": {
      "FALLING": ["obmc-chassis-hard-poweroff@0.target"]
    },
    "Continue": false
  },
  {
    "Name": "ResetButton",
    "LineName": "RESET_BUTTON",
    "GpioNum": 0,
    "ChipId": "",
    "EventMon": "FALLING",
    "Targets": {
      "FALLING": ["obmc-host-reset@0.target"]
    },
    "Continue": false
  },
  {
    "Name": "ChassisIntrusion",
    "LineName": "CHASSIS_INTRUSION",
    "GpioNum": 0,
    "ChipId": "",
    "EventMon": "BOTH",
    "Targets": {
      "FALLING": ["chassis-intrusion-detected@0.target"],
      "RISING": ["chassis-intrusion-cleared@0.target"]
    },
    "Continue": true
  },
  {
    "Name": "Fan0Presence",
    "LineName": "FAN0_PRESENT",
    "GpioNum": 0,
    "ChipId": "",
    "EventMon": "BOTH",
    "Targets": {
      "FALLING": ["fan0-present@0.target"],
      "RISING": ["fan0-absent@0.target"]
    },
    "Continue": true
  }
]
```

Example 3: Using GpioNum Instead of LineName

If your device tree does not define `gpio-line-names`, you can fall back to specifying the GPIO chip and offset directly. This approach is less portable but works on any platform.

```
[
  {
    "Name": "PowerButton",
    "LineName": "",
    "GpioNum": 8,
    "ChipId": "/dev/gpiochip0",
    "EventMon": "FALLING",
    "Targets": {
      "FALLING": ["obmc-chassis-hard-poweroff@0.target"]
    },
    "Continue": false
  }
]
```

Using `GpioNum` ties your configuration to a specific kernel GPIO numbering scheme. If the device tree changes or a driver is added that shifts GPIO offsets, your configuration breaks. Always prefer `LineName` when possible.

See additional working examples in the [examples/gpio-monitor/](#) directory.

Troubleshooting

Issue: Monitor Service Fails to Start

Symptom: `systemctl status phosphor-multi-gpio-monitor` shows `failed` or `activating` in a loop.

Cause: The JSON configuration references a GPIO line name that does not exist on this platform, or the JSON syntax is invalid.

Solution:

1. Check the journal for specific error messages:

```
journalctl -u phosphor-multi-gpio-monitor -n 50
```

2. Validate your JSON syntax:

```
python3 -m json.tool /usr/share/phosphor-gpio-monitor/phosphor-multi-gpio-monitor.json
```

3. Verify each `LineName` exists in `gpioinfo` output:

```
gpioinfo | grep "POWER_BUTTON"
```

Issue: GPIO Event Not Triggering Target

Symptom: You toggle the GPIO signal, but the expected systemd target does not activate.

Cause: The `EventMon` type does not match the actual signal transition, or the `Targets` key does not match the post-event line value.

Solution:

1. Confirm the GPIO line is claimed by the monitor:

```
gpioinfo | grep "POWER_BUTTON"  
# Should show [used] not "unused"
```

2. Test the GPIO transition manually:

```
gpiomon --num-events=1 gpiochip0 8  
# Toggle the signal and verify the event appears
```

3. Check that the **Targets** key matches the value after the transition. For a falling edge (high to low), the post-event value is **"0"**, so the target must be under **"0"**.

Issue: Presence Detection Only Works Once

Symptom: Inserting a device triggers the present target, but removing and reinserting does not trigger again.

Cause: **Continue** is set to **false** in the configuration. The monitor stops watching after the first event.

Solution: Set **"Continue": true** for any signal that you need to detect repeatedly:

```
"Continue": true
```

Debug Commands

```
# Check service status and logs  
systemctl status phosphor-multi-gpio-monitor  
journalctl -u phosphor-multi-gpio-monitor -f  
  
# GPIO discovery and testing  
gpiodetect                # List all GPIO chips  
gpioinfo                  # Show all lines with states  
gpioget gpiochip0 8       # Read current value of a line  
gpiomon --num-events=5 gpiochip0 8  # Monitor a line for events  
  
# Verify targets and config path  
systemctl list-units --type=target | grep -E "obmc|chassis|psu|fan"  
systemctl cat phosphor-multi-gpio-monitor.service
```

References

Official Resources

- [phosphor-gpio-monitor Repository](#)
- [phosphor-dbus-interfaces \(Chassis\)](#)
- [OpenBMC Documentation](#)

Related Guides

- [Buttons Guide](#)
- [Power Management Guide](#)
- [LED Manager Guide](#)

- [SGPIO \(Serial GPIO\)](#) – expand signal count over a few shared pins when you run out of parallel GPIO; uses the same `phosphor-multi-gpio-monitor` JSON

External Documentation

- [libgpiod Documentation](#)
- [Linux GPIO Subsystem](#)
- [Linux Device Tree GPIO Bindings](#)

Tested on: QEMU ast2600-evb (emulated GPIO pins, partial coverage). Full testing requires physical hardware with wired GPIO signals. Last updated: 2026-02-06

PECI Thermal Monitoring

Configure PECI (Platform Environment Control Interface) to read CPU and DIMM temperatures through the BMC.

Table of Contents

1. [Overview](#)
2. [Architecture](#)
 1. [Protocol Fundamentals](#)
 2. [CPU Address Assignment](#)
 3. [PECI Command Types](#)
 4. [Software Stack](#)
 5. [D-Bus Interfaces](#)
 6. [Key Dependencies](#)
3. [Configuration](#)
 1. [Kernel Driver Configuration](#)
 2. [Device Tree Configuration](#)
 1. [AST2600 Example](#)
 3. [Build-Time Configuration \(Yocto\)](#)
 4. [Meson Build Options \(dbus-sensors\)](#)
4. [Entity Manager PECI Configuration](#)
 1. [Configuration File](#)
 2. [Entity Manager Properties for PECI](#)
 3. [Deploying Configuration via bbappend](#)
5. [Porting Guide](#)
 1. [Step 1: Prerequisites](#)
 2. [Step 2: Configure the Device Tree](#)
 3. [Step 3: Enable Kernel Drivers](#)
 4. [Step 4: Create Entity Manager Configuration](#)
 5. [Step 5: Build and Verify](#)
6. [Verifying PECI Sensors](#)
 1. [Verify Kernel Drivers and Hwmon](#)
 2. [Read Raw Hwmon Values](#)
 3. [Test with peci_cmds](#)
 4. [Verify D-Bus Sensors](#)
 5. [Verify via Redfish](#)
7. [Code Examples](#)
 1. [Example 1: PECI Health Check Script](#)
 2. [Example 2: Reading PECI Sensors via D-Bus \(Python\)](#)
8. [Troubleshooting](#)

1. [Issue: PECI Ping Fails \(No Response from CPU\)](#)
 2. [Issue: Kernel Driver Not Loaded](#)
 3. [Issue: Hwmon Devices Not Created](#)
 4. [Issue: D-Bus Sensors Show NaN](#)
 5. [Issue: DIMM Temperatures Not Available](#)
 6. [Debug Commands](#)
-
9. [References](#)
 1. [Official Resources](#)
 2. [Related Guides](#)
 3. [External Documentation](#)
-

Overview

PECI (Platform Environment Control Interface) is Intel's single-wire serial bus that connects the BMC to one or more host CPUs. Through PECI, the BMC reads CPU die temperatures, DIMM temperatures, package configuration data, and model-specific registers – all without consuming an I2C bus or requiring the host OS to be running.

On OpenBMC, PECI thermal monitoring involves three layers: the ASPEED PECI hardware controller, Linux kernel PECI drivers that expose readings through hwmon sysfs, and dbus-sensors daemons that publish those readings on D-Bus for consumption by bmcweb, ipmid, and fan control.

This guide walks you through the full stack: understanding the PECI protocol, configuring the device tree and kernel drivers, setting up Entity Manager for sensor discovery, reading temperatures through D-Bus and Redfish, and diagnosing common failures.

Key concepts covered:

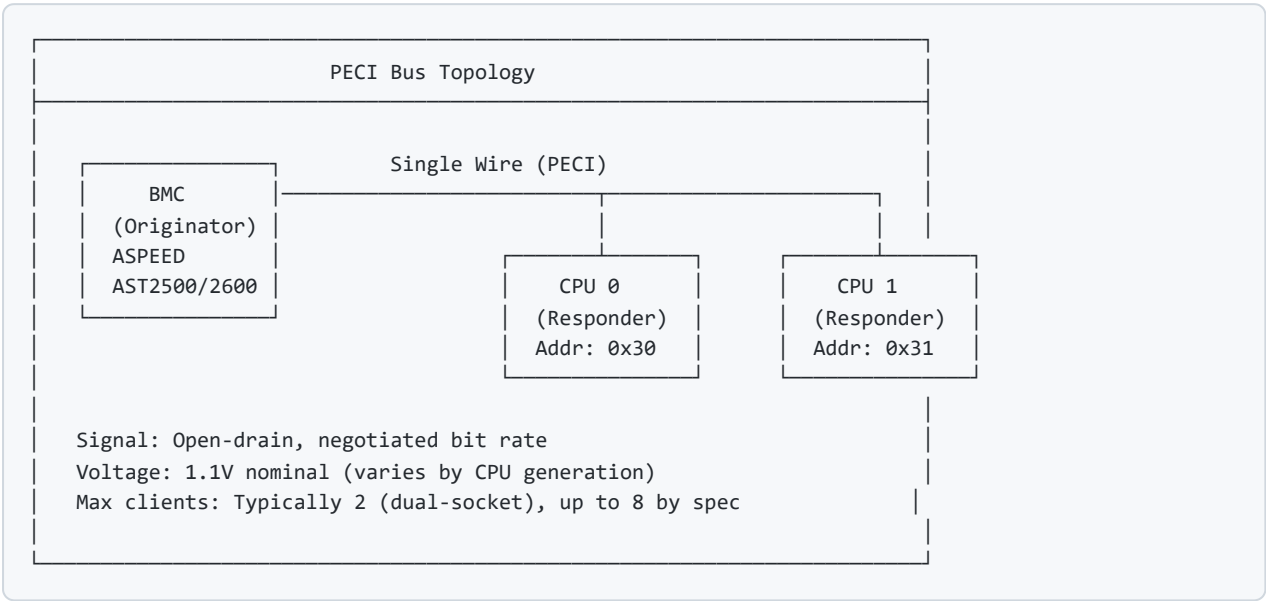
- PECI single-wire protocol and CPU addressing (0x30, 0x31)
- PECI command types: Ping, GetTemp, RdPkgConfig, RdIAMSRR
- Kernel drivers: peci-aspeed, peci-cputemp, peci-dimmtemp
- Device tree configuration for AST2500/AST2600
- dbus-sensors integration with Entity Manager
- Troubleshooting PECI connectivity and sensor failures

PECI is **not testable in QEMU**. The QEMU AST2600 model does not emulate the PECI controller or CPU thermal data. You need physical hardware with an Intel CPU connected via PECI to test this guide.

Architecture

PECI uses a single wire between the BMC and each CPU socket. The BMC acts as the originator (master) and each CPU acts as a responder (client). The bus operates at 2 kbps to 2 Mbps, negotiated automatically.

Protocol Fundamentals



CPU Address Assignment

Each CPU on the PECI bus has a fixed address determined by its socket position:

Socket	PECI Address	Description
CPU 0	0x30	First processor socket
CPU 1	0x31	Second processor socket
CPU 2	0x32	Third processor socket (4S platforms)
CPU 3	0x33	Fourth processor socket (4S platforms)

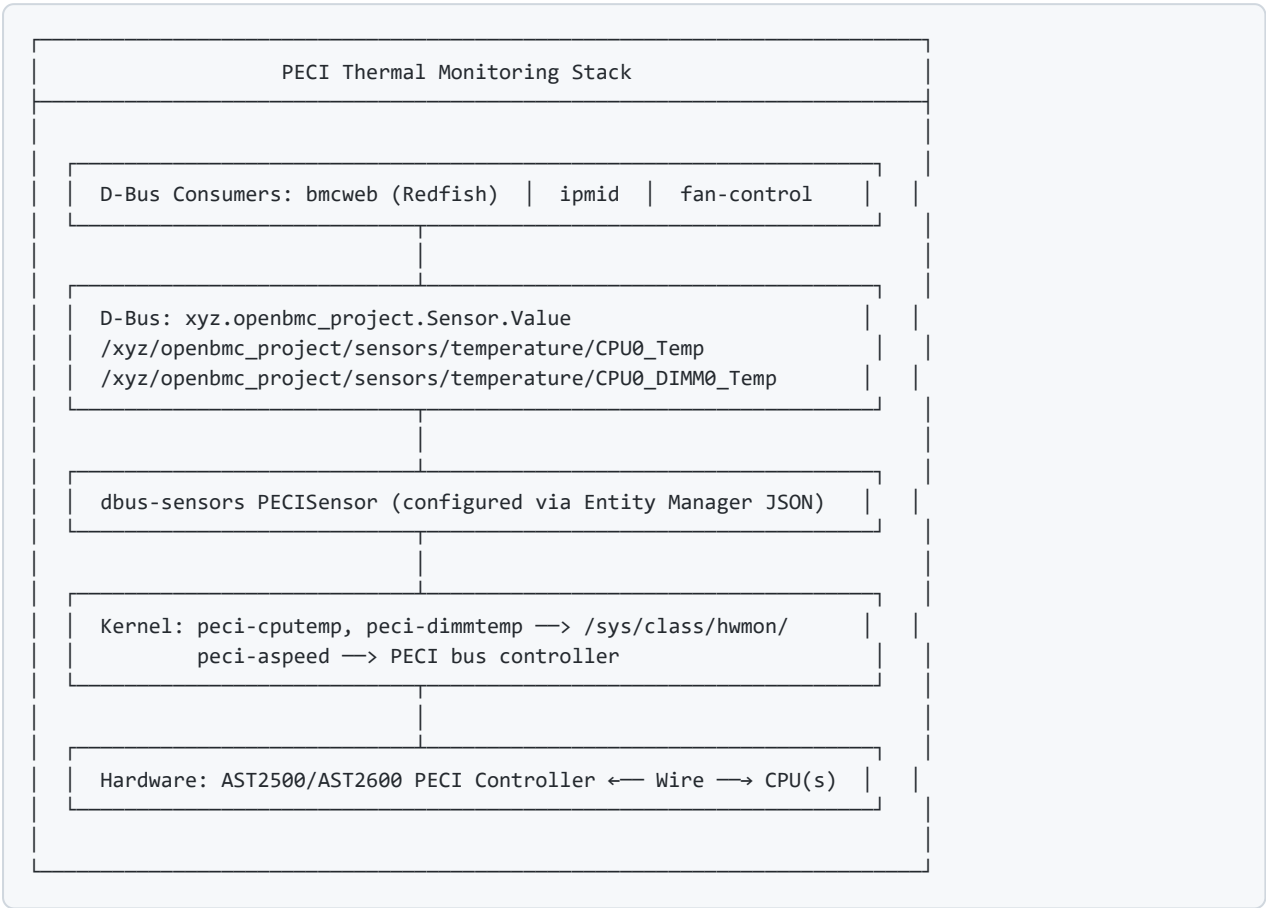
PECI addresses are hardwired by the CPU socket position on the motherboard. You do not configure them in software.

PECI Command Types

Command	Code	Description	Typical Use
Ping	0x00	Check if CPU is alive and responding	Connectivity test
GetTemp	0x01	Read CPU die temperature (DTS)	Primary thermal monitoring
RdPkgConfig	0xA1	Read package configuration data	DIMM temps, TDP, core count
WrPkgConfig	0xA5	Write package configuration data	Set thermal thresholds
RdIAMSRR	0xB1	Read IA model-specific register	Power, frequency, C-state data

GetTemp returns the CPU die temperature as a relative value below the thermal control circuit (TCC) activation temperature. The kernel driver converts this to an absolute temperature using the TCC offset stored in the CPU's package configuration.

Software Stack



D-Bus Interfaces

Interface	Object Path	Description
<code>xyz.openbmc_project.Sensor.Value</code>	<code>/xyz/openbmc_project/sensors/temperature/<name></code>	CPU or DIMM temperature reading
<code>xyz.openbmc_project.Sensor.Threshold.Warning</code>	<code>/xyz/openbmc_project/sensors/temperature/<name></code>	Warning threshold levels
<code>xyz.openbmc_project.Sensor.Threshold.Critical</code>	<code>/xyz/openbmc_project/sensors/temperature/<name></code>	Critical threshold levels

Key Dependencies

- **peci-aspeed**: Kernel driver for the ASPEED PEFI hardware controller
- **peci-cputemp**: Reads CPU die temperatures via PEFI GetTemp command
- **peci-dimmtemp**: Reads DIMM temperatures via PEFI RdPkgConfig command
- **dbus-sensors**: Polls hwmon sysfs and publishes readings on D-Bus
- **Entity Manager**: JSON-based configuration for PEFI sensor discovery

Configuration

Kernel Driver Configuration

The PECI subsystem requires three kernel drivers working together:

Driver	Module	Purpose
<code>peci-aspeed</code>	<code>peci_aspeed</code>	Controls the ASPEED SoC PECI hardware engine
<code>peci-cputemp</code>	<code>peci_cputemp</code>	Reads CPU die temperature via GetTemp command
<code>peci-dimmtemp</code>	<code>peci_dimmtemp</code>	Reads DIMM temperatures via RdPkgConfig command

Enable these options in your kernel configuration fragment (e.g., `peci.cfg`):

```
CONFIG_PECI=y
CONFIG_PECI_ASPEED=y
CONFIG_SENSORS_PECI_CPUTEMP=y
CONFIG_SENSORS_PECI_DIMMTEMP=y
```

Device Tree Configuration

Add the PECI controller and client sub-nodes for each CPU socket in your platform device tree.

AST2600 Example

```
/* aspeed-bmc-myplatform.dts */
&peci0 {
    status = "okay";

    /* CPU 0 at PECI address 0x30 */
    peci-client@30 {
        compatible = "intel,peci-client";
        reg = <0x30>;
    };

    /* CPU 1 at PECI address 0x31 (dual-socket only) */
    peci-client@31 {
        compatible = "intel,peci-client";
        reg = <0x31>;
    };
};
```

The `reg` value must match the CPU's PECI address. Using the wrong address causes all PECI commands to that client to fail with no response. Check your platform's hardware schematic for socket-to-address mapping.

Build-Time Configuration (Yocto)

Include PECI sensor daemons in your image and add the kernel config fragment:

```
# In your machine .conf or image recipe
IMAGE_INSTALL:append = " dbus-sensors"
```

```
# meta-myplatform/recipes-kernel/linux/linux-aspeed_.bbappend
FILESEXTRAPATHS:prepend := "${THISDIR}/${PN}:"
SRC_URI += "file://peci.cfg"
```

Meson Build Options (dbus-sensors)

When building dbus-sensors from source:

```
meson setup build -Dpeci=enabled
```

Entity Manager PECI Configuration

Entity Manager provides the JSON configuration that tells dbus-sensors how to discover and expose PECI sensors. The PECI sensor daemon watches for Entity Manager configuration objects of type `XeonCPU` and creates hwmon-backed D-Bus sensor objects for each CPU and its DIMMs.

Configuration File

Create a JSON configuration at `/usr/share/entity-manager/configurations/`:

```

{
  "Exposes": [
    {
      "Name": "CPU0",
      "Type": "XeonCPU",
      "Bus": 0,
      "Address": "0x30",
      "CpuID": 0,
      "Thresholds": [
        {
          "Direction": "greater than",
          "Name": "upper critical",
          "Severity": 1,
          "Value": 95
        },
        {
          "Direction": "greater than",
          "Name": "upper warning",
          "Severity": 0,
          "Value": 85
        }
      ]
    },
    {
      "Name": "CPU1",
      "Type": "XeonCPU",
      "Bus": 0,
      "Address": "0x31",
      "CpuID": 1,
      "Thresholds": [
        {
          "Direction": "greater than",
          "Name": "upper critical",
          "Severity": 1,
          "Value": 95
        },
        {
          "Direction": "greater than",
          "Name": "upper warning",
          "Severity": 0,
          "Value": 85
        }
      ]
    }
  ],
  "Name": "PeciSensors",
  "Probe": "TRUE",
  "Type": "Board"
}

```

Entity Manager Properties for PECI

Property	Type	Description
<code>Name</code>	string	Sensor name prefix (e.g., "CPU0")
<code>Type</code>	string	Must be <code>XeonCPU</code> for PECI sensors
<code>Bus</code>	integer	PECI bus number (typically 0)
<code>Address</code>	string	CPU PECI address in hex (e.g., "0x30")
<code>CpuID</code>	integer	CPU socket index (0, 1, 2, 3)
<code>Thresholds</code>	array	Warning and critical temperature thresholds

Deploying Configuration via bbappend

Install the Entity Manager configuration through your machine layer:

```
# meta-myplatform/recipes-phosphor/configuration/entity-manager_%.bbappend
FILESEXTRAPATHS:prepend := "${THISDIR}/${PN}:"

SRC_URI += "file://peci-sensors.json"

do_install:append() {
    install -d ${D}${datadir}/entity-manager/configurations
    install -m 0644 ${WORKDIR}/peci-sensors.json \
        ${D}${datadir}/entity-manager/configurations/
}

FILES:${PN} += "${datadir}/entity-manager/configurations/peci-sensors.json"
```

Porting Guide

Follow these steps to enable PECI thermal monitoring on your platform:

Step 1: Prerequisites

Ensure you have:

- ☐ An ASPEED AST2500 or AST2600 BMC with a PECI pin connected to the host CPU(s)
- ☐ Intel Xeon or compatible CPU(s) supporting PECI
- ☐ A working OpenBMC build environment with your machine layer
- ☐ The host CPU powered on (PECI requires CPU power to respond)

Step 2: Configure the Device Tree

Add the PECI controller and client nodes to your platform device tree:

```
&peci0 {
    status = "okay";
    peci-client@30 {
        compatible = "intel,peci-client";
        reg = <0x30>;
    };
};
```

Step 3: Enable Kernel Drivers

Create `peci.cfg` in your kernel recipe directory and add the bbappend as shown in [Build-Time Configuration](#).

Step 4: Create Entity Manager Configuration

Create the PECI sensor JSON and install it via a bbappend as shown in [Entity Manager PECI Configuration](#).

Step 5: Build and Verify

```
# Build the image
bitbake obmc-phosphor-image

# Flash and boot the BMC, then verify:

# 1. Check PECI drivers
dmesg | grep -i peci

# 2. Verify PECI devices
ls /sys/bus/peci/devices/

# 3. Test connectivity
peci_cmds Ping -a 0x30

# 4. Check D-Bus sensors
busctl tree xyz.openbmc_project.PECISensor

# 5. Read temperature
busctl get-property xyz.openbmc_project.PECISensor \
    /xyz/openbmc_project/sensors/temperature/CPU0_Temp \
    xyz.openbmc_project.Sensor.Value Value
```

Verifying PECI Sensors

After booting with the PECI configuration in place and the host CPU powered on, verify each layer of the stack from bottom to top.

Verify Kernel Drivers and Hwmon

```
# Check PECI controller driver
dmesg | grep -i peci
# Expected: "peci-aspeed 1e78b000.peci-bus: peci bus 0 registered"

# Check for PECI clients
ls /sys/bus/peci/devices/
# Expected: 0-30 0-31

# Check hwmon devices
ls /sys/bus/peci/devices/0-30/peci-cputemp.0/hwmon/
ls /sys/bus/peci/devices/0-30/peci-dimmtemp.0/hwmon/
```

Read Raw Hwmon Values

```
# Find and read all PECI hwmon sensors
for d in /sys/class/hwmon/hwmon*; do
    name=$(cat "$d/name" 2>/dev/null)
    if echo "$name" | grep -q peci; then
        echo "=== $d ($name) ==="
        for f in "$d"/temp*_input; do
            label=$(cat "${f%_input}_label" 2>/dev/null || echo "unknown")
            val=$(cat "$f" 2>/dev/null)
            if [ -n "$val" ]; then
                echo "  $label: $((val / 1000))°C"
            fi
        done
    fi
done
```

Test with peci_cmds

The `peci_cmds` utility allows you to send raw PECI commands for testing:

```
# Ping CPU 0
peci_cmds Ping -a 0x30

# Read CPU 0 die temperature
peci_cmds GetTemp -a 0x30

# Read package configuration (TCC activation temp)
peci_cmds RdPkgConfig -a 0x30 -i 0 -p 16

# Read IA MSR (e.g., platform info)
peci_cmds RdIAMSr -a 0x30 -i 0 -m 0xCE
```


Verify D-Bus Sensors

```
# List all PECI temperature sensors
busctl tree xyz.openbmc_project.PECISensor

# Read CPU0 die temperature
busctl get-property xyz.openbmc_project.PECISensor \
    /xyz/openbmc_project/sensors/temperature/CPU0_Temp \
    xyz.openbmc_project.Sensor.Value Value

# Read CPU0 DIMM0 temperature
busctl get-property xyz.openbmc_project.PECISensor \
    /xyz/openbmc_project/sensors/temperature/CPU0_DIMM0_Temp \
    xyz.openbmc_project.Sensor.Value Value

# Check threshold status
busctl get-property xyz.openbmc_project.PECISensor \
    /xyz/openbmc_project/sensors/temperature/CPU0_Temp \
    xyz.openbmc_project.Sensor.Threshold.Critical CriticalAlarmHigh
```

Verify via Redfish

```
curl -k -u root:0penBmc \
    https://localhost/redfish/v1/Chassis/chassis/Sensors/CPU0_Temp

curl -k -u root:0penBmc \
    https://localhost/redfish/v1/Chassis/chassis/Thermal
```

Code Examples

Example 1: PECI Health Check Script

A shell script to verify PECI connectivity and read temperatures from all detected CPUs:

```

#!/bin/bash
# peci_health_check.sh - Verify PECI connectivity and read temperatures

PECI_ADDRS="0x30 0x31"

echo "=== PECI Health Check ==="

# Check kernel drivers
echo ""
echo "--- Kernel Drivers ---"
for mod in peci_aspeed peci_cputemp peci_dimmtemp; do
    if [ -d "/sys/module/$mod" ]; then
        echo "[OK] $mod loaded"
    else
        echo "[FAIL] $mod NOT loaded"
    fi
done

# Check PECI devices
echo ""
echo "--- PECI Devices ---"
ls /sys/bus/peci/devices/ 2>/dev/null || echo "No PECI devices found"

# Ping each CPU and read temperature
echo ""
echo "--- PECI Connectivity ---"
for addr in $PECI_ADDRS; do
    result=$(peci_cmds Ping -a "$addr" 2>&1)
    if echo "$result" | grep -qi "succeed"; then
        temp=$(peci_cmds GetTemp -a "$addr" 2>&1)
        echo "[OK] CPU at $addr: $temp"
    else
        echo "[--] CPU at $addr: not responding"
    fi
done

# Read hwmon PECI sensors
echo ""
echo "--- Hwmon PECI Sensors ---"
for d in /sys/class/hwmon/hwmon*; do
    name=$(cat "$d/name" 2>/dev/null)
    if echo "$name" | grep -q peci; then
        echo "Device: $d ($name)"
        for f in "$d"/temp*_input; do
            [ -f "$f" ] || continue
            label=$(cat "${f%_input}_label" 2>/dev/null || basename "$f")
            val=$(cat "$f" 2>/dev/null)
            if [ -n "$val" ] && [ "$val" != "0" ]; then
                echo "  $label: $((val / 1000))°C"
            fi
        done
    fi
done
echo ""
echo "=== Check Complete ==="

```

Example 2: Reading PECI Sensors via D-Bus (Python)

```
#!/usr/bin/env python3
"""Read PECI CPU and DIMM temperatures from D-Bus."""
import subprocess

SERVICE = "xyz.openbmc_project.PECISensor"

def get_sensor_value(path):
    """Read a sensor value from D-Bus using busctl."""
    try:
        result = subprocess.run(
            ["busctl", "get-property", SERVICE, path,
             "xyz.openbmc_project.Sensor.Value", "Value"],
            capture_output=True, text=True, timeout=5,
        )
        if result.returncode == 0:
            parts = result.stdout.strip().split()
            if len(parts) >= 2:
                return float(parts[1])
    except (subprocess.TimeoutExpired, ValueError):
        pass
    return None

def list_peci_sensors():
    """List all temperature sensors from PECISensor service."""
    result = subprocess.run(
        ["busctl", "tree", SERVICE],
        capture_output=True, text=True, timeout=5,
    )
    return [line.strip() for line in result.stdout.splitlines()
            if "/sensors/temperature/" in line]

if __name__ == "__main__":
    sensors = list_peci_sensors()
    if not sensors:
        print("No PECI sensors found. Is the host CPU powered on?")
    else:
        print(f"Found {len(sensors)} PECI sensor(s):")
        for path in sensors:
            value = get_sensor_value(path)
            name = path.split("/")[-1]
            status = f"{value:.1f} C" if value is not None else "unavailable"
            print(f"  {name}: {status}")
```

See additional examples in the [examples/peci/](#) directory.

Troubleshooting

Issue: PECI Ping Fails (No Response from CPU)

Symptom: `peci_cmds Ping -a 0x30` returns an error or times out.

Cause: The CPU is not powered on, the PECI wire is not connected, or the PECI controller driver is not loaded.

Solution:

1. Verify the host CPU is powered on:

```
busctl get-property xyz.openbmc_project.State.Host \
    /xyz/openbmc_project/state/host0 \
    xyz.openbmc_project.State.Host CurrentHostState
```

2. Check the PECI controller driver loaded:

```
dmesg | grep -i peci
ls /sys/bus/peci/
```

3. Verify device tree configuration:

```
cat /sys/firmware/devicetree/base/ahb/apb/peci-bus@*/status
```

PECI **requires** the CPU to be powered on and through its reset sequence. If the host is off or in standby, all PECI commands will fail. This is expected behavior, not a bug.

Issue: Kernel Driver Not Loaded

Symptom: `dmesg | grep peci` shows no output. `/sys/bus/peci/` does not exist.

Cause: The PECI kernel modules are not built or not included in the image.

Solution:

1. Verify kernel config:

```
zcat /proc/config.gz | grep -i peci
```

2. If options are missing, add the kernel config fragment and rebuild.
3. If built as modules, load manually:

```
modprobe peci_aspeed
modprobe peci_cputemp
modprobe peci_dimmtemp
```

Issue: Hwmon Devices Not Created

Symptom: PECI ping succeeds, but no hwmon entries appear under `/sys/bus/peci/devices/0-30/`.

Cause: The peci-cputemp or peci-dimmtemp drivers are not loaded, or the CPU has not completed POST.

Solution:

1. Check client sub-driver directories:

```
ls /sys/bus/peci/devices/0-30/
```

2. Wait for the CPU to complete POST, then trigger a rescan:

```
echo 1 > /sys/bus/peci/rescan
```

Issue: D-Bus Sensors Show NaN

Symptom: `busctl get-property` returns `d nan` for PECI sensor values.

Cause: The host is not powered on, or the hwmon file is returning an error.

Solution:

1. Confirm the host power state is `Running`.
2. Read the raw hwmon value directly:

```
cat /sys/class/hwmon/hwmon<N>/temp1_input
```

3. Check the PECISensor daemon logs:

```
journalctl -u xyz.openbmc_project.pecisensor -f
```

Issue: DIMM Temperatures Not Available

Symptom: CPU die temperature works, but DIMM temperatures are not exposed.

Cause: The peci-dimmtemp driver did not detect populated DIMM slots, or the CPU model does not support DIMM temperature reading via PECI.

Solution:

1. Check if peci-dimmtemp created hwmon entries:

```
ls /sys/bus/peci/devices/0-30/peci-dimmtemp.0/hwmon/
```

2. Verify DIMM population using PECI package config:

```
peci_cmds RdPkgConfig -a 0x30 -i 0 -p 14
```

Debug Commands

```
# Check all PECI-related services
systemctl list-units | grep -i peci

# View PECISensor logs
journalctl -u xyz.openbmc_project.pecisensor -f

# Check Entity Manager detected PECI config
journalctl -u xyz.openbmc_project.EntityManager | grep -i peci

# Kernel PECI bus status
ls -la /sys/bus/peci/devices/
dmesg | grep -i peci

# Raw PECI commands
peci_cmds Ping -a 0x30
peci_cmds GetTemp -a 0x30
peci_cmds RdPkgConfig -a 0x30 -i 0 -p 16
```

References

Official Resources

- [dbus-sensors Repository](#) (contains PECSensor implementation)
- [linux-aspeed PECI Driver](#)
- [D-Bus Sensor Interface Definitions](#)
- [Entity Manager](#)

Related Guides

- [D-Bus Sensors Guide](#)
- [Hwmon Sensors Guide](#)
- [Entity Manager Guide](#)
- [Fan Control Guide](#)

External Documentation

- [Intel PECI Specification](#)
- [Linux Kernel PECI Subsystem](#)
- [ASPEED AST2600 Datasheet](#) (PECI controller chapter)

Tested on: Physical AST2600 EVB with Intel Xeon CPU. PECI is not emulated in QEMU – all PECI commands and sensor readings require real hardware with a powered-on host CPU. Last updated: 2026-02-06

I2C Device Integration

Configure I2C bus topology, debug devices with i2c-tools, bind kernel drivers through device tree and sysfs, and prepare for I3C migration on OpenBMC.

Table of Contents

1. [Overview](#)
2. [Architecture](#)
 1. [I2C Bus Topology](#)
 2. [Bus Numbering Rules](#)
 3. [Common I2C Devices in BMC Platforms](#)
 4. [Key Dependencies](#)
3. [i2c-tools Debugging](#)
 1. [i2cdetect – Scan for Devices](#)
 2. [i2cget – Read a Register](#)
 3. [i2cset – Write a Register](#)
 4. [i2cdump – Dump All Registers](#)
 5. [Practical Debugging Workflow](#)
4. [Device Tree I2C Node Configuration](#)
 1. [I2C Controller Nodes](#)
 2. [Adding I2C Device Nodes](#)
 3. [Device Tree Properties Reference](#)
 4. [I2C Mux Configuration](#)
 5. [Common Compatible Strings](#)
 6. [Adding Device Tree Changes to Your Layer](#)
5. [Runtime Driver Bind/Unbind](#)
 1. [Adding a Device at Runtime \(new_device\)](#)
 2. [Removing a Device at Runtime \(delete_device\)](#)
 3. [Driver Bind/Unbind via sysfs](#)
 4. [When to Use Runtime Binding](#)
6. [Porting Guide](#)
 1. [Step 1: Map Your Hardware I2C Topology](#)
 2. [Step 2: Verify Devices with i2c-tools](#)
 3. [Step 3: Add Device Tree Nodes](#)
 4. [Step 4: Build and Verify Driver Binding](#)
 5. [Step 5: Connect to OpenBMC Sensor Framework](#)
7. [Code Examples](#)
 1. [Example 1: I2C Bus Scan Script](#)
 2. [Example 2: Runtime I2C Device Management](#)

8. I3C Overview

1. [Key Differences Between I2C and I3C](#)
2. [I3C in the Linux Kernel](#)
3. [Migration Path: I2C to I3C](#)

9. Troubleshooting

1. [Issue: Device Not Found on i2cdetect Scan](#)
2. [Issue: Driver Not Binding to Device](#)
3. [Issue: I2C Bus Stuck \(SDA Held Low\)](#)
4. [Issue: Wrong Hwmon Values After new_device](#)
5. [Debug Commands](#)

10. References

1. [Official Resources](#)
2. [Related Guides](#)
3. [External Documentation](#)

Overview

The **I2C (Inter-Integrated Circuit)** bus is the primary communication backbone on every BMC platform. Temperature sensors, voltage regulators, fan controllers, EEPROM FRU storage, power sequencers, and mux expanders all sit on I2C buses connected to the ASPEED BMC SoC. Understanding I2C bus topology, device tree configuration, and runtime driver management is essential for bringing up any new OpenBMC platform.

This guide covers the full I2C integration workflow: mapping your hardware bus topology, using i2c-tools to discover and debug devices, writing device tree nodes so the kernel binds drivers automatically, and using sysfs to bind or unbind drivers at runtime. It also introduces I3C – the next-generation successor to I2C – and explains what the migration path looks like for OpenBMC platforms.

Key concepts covered:

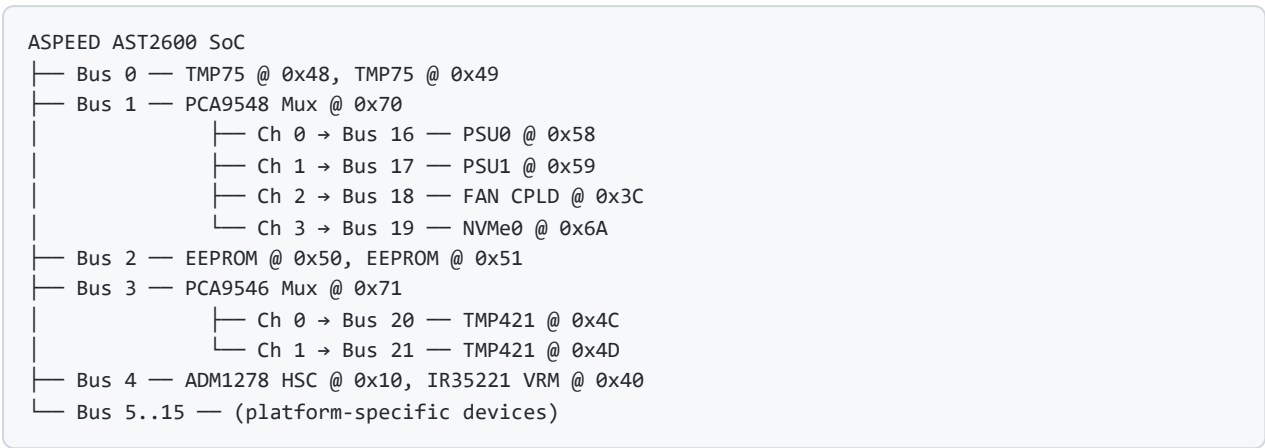
- I2C bus numbering and mux hierarchy on ASPEED SoCs
- i2c-tools commands: `i2cdetect`, `i2cget`, `i2cset`, `i2cdump`
- Device tree I2C node configuration and driver binding
- Runtime driver bind/unbind through sysfs `new_device` and `delete_device`
- I3C protocol overview and kernel status
- Common I2C device types in BMC platforms

I2C bus operations are **fully testable** in QEMU. The ast2600-evb QEMU target emulates I2C controllers with a few default devices. You can practice i2c-tools commands, device tree changes, and driver binding in the emulated environment before deploying to physical hardware.

Architecture

The ASPEED AST2600 BMC SoC provides 16 I2C controllers, each appearing as a separate Linux I2C bus. Physical hardware typically routes these buses through I2C multiplexers (muxes) to expand the address space and isolate device groups. The Linux I2C subsystem assigns sequential bus numbers to each mux channel, creating a tree of buses rooted at the SoC controllers.

I2C Bus Topology



Bus Numbering Rules

The kernel assigns I2C bus numbers following these rules:

- 1. **SoC controllers** get the lowest numbers (0-15 on AST2600), based on the device tree node order.
- 2. **Mux channels** receive dynamically assigned numbers starting after the last SoC controller. The exact number depends on mux probe order.
- 3. **Nested muxes** add further bus numbers. A mux behind another mux creates buses at the next available numbers.

Mux bus numbers are assigned at probe time and can change if you add or remove mux entries in the device tree. Never hardcode mux channel bus numbers in userspace scripts. Instead, use the `i2c-mux` sysfs hierarchy to discover the mapping dynamically.

Common I2C Devices in BMC Platforms

Device Type	Common Chips	Address Range	Purpose
Temperature sensor	TMP75, TMP421, LM75	0x48-0x4F	Thermal monitoring
EEPROM	AT24C256, AT24C64	0x50-0x57	FRU data, board identity
I2C Mux	PCA9548, PCA9546	0x70-0x77	Bus expansion
Power supply	PMBus devices	0x58-0x5F	PSU telemetry
Voltage regulator	IR35221, TPS53679	0x40-0x4F	VRM monitoring
Hot-swap controller	ADM1278, LTC4282	0x10-0x1F	Input power monitoring
Fan controller	MAX31790, EMC2305	0x20-0x2F	Fan tach and PWM

Key Dependencies

- **Linux I2C subsystem:** Provides the bus abstraction (`/dev/i2c-N`), adapter drivers, and client driver framework
- **ASPEED I2C controller driver:** `i2c-aspeed` kernel module that interfaces with SoC I2C registers
- **i2c-tools:** Userspace utilities for bus scanning and device communication
- **I2C mux framework:** Kernel subsystem that creates virtual I2C buses for mux channels

i2c-tools Debugging

The `i2c-tools` package provides four essential commands for I2C debugging on a running BMC. These tools communicate directly with devices through the Linux I2C subsystem, bypassing any OpenBMC daemons.

i2cdetect – Scan for Devices

Use `i2cdetect` to discover which addresses respond on a given bus. This is typically your first step when bringing up a new platform.

```
# List all I2C buses on the system
i2cdetect -l
# i2c-0 i2c 1e78a080.i2c-bus I2C adapter
# i2c-16 i2c i2c-1-mux (chan_id 0) I2C adapter ...

# Scan bus 0 for responding devices (-y skips confirmation prompt)
i2cdetect -y 0

# Sample output (abbreviated):
#      0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
# 00:      -- -- -- -- -- -- -- -- -- -- -- -- -- --
# ...
# 40: -- -- -- -- -- -- -- -- 48 49 -- -- -- -- -- --
# ...
# 70: -- -- -- -- -- -- -- --
```

The output shows device addresses in hexadecimal. Addresses showing a value (like `48`) have a device responding; `--` means no response; `UU` means the address is already claimed by a kernel driver.

The `-y` flag skips the interactive confirmation prompt. Use `-r` for SMBus quick-read scan mode, which works better with some devices that do not respond to the default quick-write probe.

i2cget – Read a Register

Use `i2cget` to read a single register from an I2C device. This is useful for verifying device identity and checking register values.

```
# Read one byte from device 0x48 on bus 0, register 0x00 (temperature)
i2cget -y 0 0x48 0x00 b

# Read a 16-bit (word) value from the same register
i2cget -y 0 0x48 0x00 w

# Read without specifying a register (current pointer)
i2cget -y 0 0x48
```

Command format: `i2cget -y <bus> <address> [register] [mode]`

Mode	Description
<code>b</code>	Read a single byte (default)
<code>w</code>	Read a 16-bit word
<code>c</code>	Read a byte without writing a register address first

i2cset – Write a Register

Use `i2cset` to write values to device registers. This is useful for configuring devices and testing write paths.

```
# Write byte 0x60 to register 0x01 (configuration) on device 0x48, bus 0
i2cset -y 0 0x48 0x01 0x60 b

# Write a 16-bit word to register 0x02 (high limit)
i2cset -y 0 0x48 0x02 0x5000 w
```

Command format: `i2cset -y <bus> <address> <register> <value> [mode]`

Writing to I2C devices can change hardware configuration and potentially damage components if you write incorrect values to control registers (voltage regulator output, fan speed limits, etc.). Double-check the device datasheet before using `i2cset` on production hardware.

i2cdump – Dump All Registers

Use `i2cdump` to read the entire register map of a device. This gives you a complete picture of the device state.

```
# Dump all registers from device 0x48 on bus 0
i2cdump -y 0 0x48

# Sample output (TMP75 temperature sensor):
#   0 1 2 3 4 5 6 7 8 9 a b c d e f
# 00: 19 60 4b 00 19 60 4b 00 19 60 4b 00 19 60 4b 00
# ...

# Dump using word (16-bit) reads
i2cdump -y 0 0x48 w

# Dump a specific range of registers (bytes 0x00 to 0x0F)
i2cdump -y -r 0x00-0x0F 0 0x48
```

Practical Debugging Workflow

When a sensor or device is not working, follow this sequence:

1. Verify the bus exists: `i2cdetect -l | grep "i2c-1"`
2. Scan for the device: `i2cdetect -y 1` (look for your expected address)
3. If device responds, read its ID register: `i2cget -y 1 0x48 0x00 w`
4. If device does NOT respond, check wiring, mux selection, and address conflicts. Try read-mode scan: `i2cdetect -y -r 1`

In QEMU ast2600-evb, run `i2cdetect -l` to see which buses are emulated and `i2cdetect -y <bus>` to find the default emulated devices.

Device Tree I2C Node Configuration

The Linux kernel uses the device tree to discover I2C devices at boot time and automatically bind the correct driver to each device. Properly configuring device tree I2C nodes is the standard method for declaring hardware on your platform.

I2C Controller Nodes

The ASPEED device tree defines I2C controller nodes as children of the APB bus. Each controller maps to one I2C bus. Enable the controllers you need by setting `status = "okay"` in your platform device tree.

```
/* Enable I2C bus 0 and bus 1 in your platform .dts file */
&i2c0 {
    status = "okay";
};

&i2c1 {
    status = "okay";
};
```

Adding I2C Device Nodes

Declare each I2C device as a child node of its parent bus. The key properties are `compatible` (selects the kernel driver) and `reg` (specifies the 7-bit I2C address).

```
&i2c0 {
    status = "okay";

    /* TMP75 temperature sensor at address 0x48 */
    tmp75@48 {
        compatible = "ti,tmp75";
        reg = <0x48>;
    };

    /* TMP75 temperature sensor at address 0x49 */
    tmp75@49 {
        compatible = "ti,tmp75";
        reg = <0x49>;
    };

    /* AT24 EEPROM at address 0x50 */
    eeprom@50 {
        compatible = "atmel,24c256";
        reg = <0x50>;
        pagesize = <64>;
    };
};
```

Device Tree Properties Reference

Property	Required	Description	Example
<code>compatible</code>	Yes	Driver binding string (vendor,device)	<code>"ti,tmp75"</code>
<code>reg</code>	Yes	7-bit I2C address in hex	<code><0x48></code>
<code>status</code>	No	Enable/disable (<code>"okay"</code> or <code>"disabled"</code>)	<code>"okay"</code>
<code>label</code>	No	Human-readable device label	<code>"inlet_temp"</code>
<code>pagesize</code>	EEPROM	EEPROM page size for write operations	<code><64></code>

I2C Mux Configuration

I2C multiplexers create child buses in the device tree. Each channel becomes a new I2C bus that you can attach devices to.

```
&i2c1 {
    status = "okay";

    /* PCA9548 8-channel I2C mux at address 0x70 */
    i2c-mux@70 {
        compatible = "nxp,pca9548";
        reg = <0x70>;
        #address-cells = <1>;
        #size-cells = <0>;

        /* Channel 0: PSU bus */
        i2c@0 {
            reg = <0>;
            #address-cells = <1>;
            #size-cells = <0>;

            psu@58 {
                compatible = "pmbus";
                reg = <0x58>;
            };
        };

        /* Channel 1: Sensor bus */
        i2c@1 {
            reg = <1>;
            #address-cells = <1>;
            #size-cells = <0>;

            tmp421@4c {
                compatible = "ti,tmp421";
                reg = <0x4c>;
            };
        };

        /* Additional channels follow the same pattern */
    };
};
```

The `#address-cells = <1>` and `#size-cells = <0>` properties are required on each mux channel node. They tell the device tree parser that child nodes use a single cell for their address (the I2C address) and have no size component.

Common Compatible Strings

Device	Compatible String	Kernel Driver
TMP75	"ti,tmp75"	lm75
TMP421	"ti,tmp421"	tmp421
LM75A	"nxp,lm75a"	lm75
AT24C256	"atmel,24c256"	at24
PCA9548	"nxp,pca9548"	i2c-mux-pca954x
PCA9546	"nxp,pca9546"	i2c-mux-pca954x
ADM1278	"adi,adm1278"	adm1275

Adding Device Tree Changes to Your Layer

Place your device tree in a `.dts` file in your machine layer and include it via a kernel bbappend (`linux-aspeed_%bbappend`) using `FILESEXTRAPATHS:prepend` and `SRC_URI += "file://aspeed-bmc-myplatform.dts"` . Copy the file into the kernel source tree in `do_configure:append()` .

Runtime Driver Bind/Unbind

Sometimes you need to add or remove I2C devices without rebooting, for example when hot-swapping a component, debugging a sensor, or testing a new device. The Linux I2C subsystem provides sysfs interfaces for runtime device management.

Adding a Device at Runtime (new_device)

Write the driver name and address to the bus's `new_device` file to instantiate a device without a device tree entry.

```
# Instantiate a TMP75 at address 0x48 on bus 0
echo tmp75 0x48 > /sys/bus/i2c/devices/i2c-0/new_device

# Verify the device was created
ls /sys/bus/i2c/devices/0-0048/
# driver hwmon modalias name power subsystem uevent

# Verify hwmon appeared
ls /sys/bus/i2c/devices/0-0048/hwmon/
# hwmon3

# Read the sensor value
cat /sys/bus/i2c/devices/0-0048/hwmon/hwmon3/temp1_input
# 25500 (25.5 degrees C in millidegrees)
```

The `new_device` file takes two space-separated arguments: the driver name (e.g., `tmp75` , `at24`) and the 7-bit I2C address in hex (e.g., `0x48`).

Removing a Device at Runtime (delete_device)

Write the address to the bus's `delete_device` file to remove a previously instantiated device:

```
echo 0x48 > /sys/bus/i2c/devices/i2c-0/delete_device
```

Removing a device that is in use by a sensor daemon (phosphor-hwmon, dbus-sensors) will cause that daemon to lose the sensor. The daemon may log errors or restart. Stop the daemon first if you need to remove a device cleanly.

Driver Bind/Unbind via sysfs

If a device exists in the device tree but you need to rebind its driver (for example after loading an updated module), use the driver's `bind` and `unbind` files.

```
# Find the device's driver
ls -la /sys/bus/i2c/devices/0-0048/driver
# lrwxrwxrwx ... -> ../../../../../../../bus/i2c/drivers/lm75

# Unbind the driver from the device
echo "0-0048" > /sys/bus/i2c/drivers/lm75/unbind

# Rebind the driver to the device
echo "0-0048" > /sys/bus/i2c/drivers/lm75/bind
```

The device identifier format is `<bus>-<address>`, where `<bus>` is the bus number and `<address>` is the zero-padded 4-digit hex address (e.g., `0-0048` for bus 0, address 0x48).

When to Use Runtime Binding

Use `new_device` for testing sensors before committing to device tree changes, or for hot-swap scenarios where a GPIO presence signal triggers device instantiation. Use `unbind` / `bind` when reloading driver modules or isolating a misbehaving device during debugging.

For hot-swappable devices, you can automate bind/unbind with a script triggered by GPIO events. Wire a GPIO presence signal to a systemd target that calls `new_device` on insertion and `delete_device` on removal. See the [GPIO Management Guide](#) for details on event-driven scripting.

When using `new_device`, the kernel matches the driver name to a loaded I2C driver. If the driver is built as a module and not yet loaded, run `modprobe <driver>` first. For devices declared in the device tree, driver modules are loaded automatically by `udev`.

Porting Guide

Follow these steps to integrate I2C devices on a new OpenBMC platform:

Step 1: Map Your Hardware I2C Topology

Obtain the platform schematic and identify:

- ☐ Which ASPEED I2C bus each device connects to
- ☐ I2C addresses for every device (check for address pin strapping)
- ☐ Mux locations and channel assignments
- ☐ Pull-up resistor values (SDA/SCL should have 2.2k-10k pull-ups)

Step 2: Verify Devices with i2c-tools

Boot your BMC with a minimal image and scan each bus with `i2cdetect -y <bus>`. Compare the results against your hardware map – every expected device should respond at its documented address.

Step 3: Add Device Tree Nodes

Create or update your platform device tree to declare every I2C device. Follow the patterns in the [Device Tree I2C Node Configuration](#) section for sensors, EEPROMs, and muxes. Each device needs a `compatible` string and `reg` address that match your hardware.

Step 4: Build and Verify Driver Binding

```
bitbake obmc-phosphor-image
# Flash, boot, then verify:
dmesg | grep i2c                # Check I2C init messages
ls /sys/bus/i2c/devices/0-0048/driver # Verify driver bound
ls /sys/bus/i2c/devices/0-0048/hwmon/  # Verify hwmon appeared
i2cdetect -l | grep mux          # Verify mux channels
```

Step 5: Connect to OpenBMC Sensor Framework

Once device tree binding is confirmed, configure Entity Manager or phosphor-hwmon to expose your sensors on D-Bus. See the [D-Bus Sensors Guide](#) and [Hwmon Sensors Guide](#) for details.

Code Examples

Example 1: I2C Bus Scan Script

A script that scans all I2C buses and reports discovered devices with their driver status.

```
#!/bin/bash
# i2c-scan-all.sh -- Scan all I2C buses and report device status

for bus_path in /sys/bus/i2c/devices/i2c-*; do
    bus=$(basename "$bus_path" | sed 's/i2c-//')
    echo "Bus $bus: $(cat "$bus_path/name" 2>/dev/null)"
    for dev_path in /sys/bus/i2c/devices/${bus}-*; do
        [ -d "$dev_path" ] || continue
        driver="(no driver)"
        [ -L "$dev_path/driver" ] && driver=$(basename "$(readlink "$dev_path/driver")")
        echo "  $(basename "$dev_path"): $(cat "$dev_path/name" 2>/dev/null) [$driver]"
    done
done
```

Example 2: Runtime I2C Device Management

```
# Add a TMP75 at 0x48 on bus 0, read its temperature, then remove it
echo tmp75 0x48 > /sys/bus/i2c/devices/i2c-0/new_device
ls /sys/bus/i2c/devices/0-0048/hwmon/          # Verify hwmon appeared
cat /sys/bus/i2c/devices/0-0048/hwmon/hwmon*/temp1_input # Read temperature
echo 0x48 > /sys/bus/i2c/devices/i2c-0/delete_device    # Remove device
```


See additional working examples in the [examples/i2c-tools/](#) directory.

I3C Overview

I3C (Improved Inter-Integrated Circuit) is the MIPI Alliance standard that succeeds I2C. It maintains backward compatibility with I2C devices while adding significant performance and feature improvements. As BMC platforms evolve, I3C will gradually replace I2C for high-speed sensor and device communication.

Key Differences Between I2C and I3C

Feature	I2C	I3C
Maximum speed	3.4 Mbps (High-Speed)	12.5 Mbps (SDR), 25+ Mbps (HDR)
Addressing	7-bit static (set by hardware pins)	Dynamic address assignment by controller
In-band interrupts	Not supported (requires separate IRQ line)	Supported natively on SDA
Hot-join	Not supported	Devices can join the bus at runtime
Device discovery	Manual scan (i2cdetect)	Automatic via CCC (Common Command Codes)
Power consumption	Higher (external pull-ups)	Lower (push-pull drivers, no pull-ups)
Legacy support	N/A	I2C devices work on same bus

I3C in the Linux Kernel

The Linux kernel includes an I3C subsystem (available since kernel 5.0). The ASPEED AST2600 has a supported controller driver (`aspeed-i3c`), but I3C device driver support in OpenBMC is still maturing. Most platforms continue to use I2C for the near term.

Key kernel components: `drivers/i3c/master.c` (core), `aspeed-i3c` (ASPEED controller), `dw-i3c-master` (DesignWare), and emerging device drivers.

Migration Path: I2C to I3C

If you are planning for I3C migration, follow these guidelines:

1. **Keep I2C working today** – Use standard I2C device tree nodes and drivers. Do not wait for I3C.
2. **Identify I3C-capable hardware** – Check which next-generation devices support I3C natively.
3. **Enable the I3C subsystem** – Add `CONFIG_I3C=y` and `CONFIG_I3C_MASTER_ASPEED=y` in your kernel config.
4. **Test with a mixed bus** – I3C controllers support legacy I2C devices on the same bus.
5. **Watch upstream** – Monitor `drivers/i3c/` for new device drivers relevant to your platform.

The AST2600 has 6 I3C controllers in addition to its 16 I2C controllers. You can use both simultaneously. Start I3C evaluation on a dedicated bus while keeping production devices on I2C.

Troubleshooting

Issue: Device Not Found on i2cdetect Scan

Symptom: `i2cdetect -y <bus>` shows `--` at the expected device address.

Cause: The device is not responding on the bus. Possible reasons include wrong bus number, wrong address, device not powered, missing pull-ups, or a mux channel not selected.

Solution:

1. Verify the correct bus with `i2cdetect -l`
2. If behind a mux, scan the parent bus for the mux first (`i2cdetect -y 1` , look for 0x70-0x77)
3. Select the mux channel manually: `i2cset -y 1 0x70 0x01` then rescan
4. Try read-mode scan: `i2cdetect -y -r 0`

Issue: Driver Not Binding to Device

Symptom: The device appears in `i2cdetect` output, but no driver is bound (no `driver` symlink in `/sys/bus/i2c/devices/<bus>-<addr>/`).

Cause: The `compatible` string in the device tree does not match any loaded driver, or the driver module is not built into the kernel.

Solution:

1. Check the modalias: `cat /sys/bus/i2c/devices/0-0048/modalias`
2. Verify the driver is loaded: `modprobe lm75 && ls /sys/bus/i2c/drivers/lm75/`
3. Try manual binding: `echo "0-0048" > /sys/bus/i2c/drivers/lm75/bind`
4. Check for probe errors: `dmesg | grep "0-0048"`

Issue: I2C Bus Stuck (SDA Held Low)

Symptom: All devices on a bus stop responding. `i2cdetect` hangs or returns errors.

Cause: A device is holding SDA low due to an interrupted transfer or device hang.

Solution: Check `dmesg | grep -i "i2c.*recover"` for recovery attempts. Force a controller reset by unbinding and rebinding the ASPEED I2C driver: `echo "1e78a080.i2c-bus" > /sys/bus/platform/drivers/aspeed-i2c-bus/unbind` then `echo "1e78a080.i2c-bus" > /sys/bus/platform/drivers/aspeed-i2c-bus/bind` . As a last resort, power-cycle the misbehaving device.

Issue: Wrong Hwmon Values After new_device

Symptom: You create a device with `new_device` and the hwmon sensor appears, but readings are incorrect or zero.

Cause: The driver name you specified does not match the actual chip on the bus, or the chip needs initialization that the wrong driver does not perform.

Solution:

1. Verify the chip identity by reading its ID register directly:

```
i2cdump -y 0 0x48
# Compare register map against the datasheet
```

2. Remove and re-add with the correct driver name:

```
echo 0x48 > /sys/bus/i2c/devices/i2c-0/delete_device
echo 1m75 0x48 > /sys/bus/i2c/devices/i2c-0/new_device
```

Debug Commands

```
# List all I2C buses and adapters
i2cdetect -l

# Scan a specific bus
i2cdetect -y 0

# Check device driver binding
ls -la /sys/bus/i2c/devices/0-0048/driver

# View I2C kernel messages
dmesg | grep -i i2c

# Enable I2C debug tracing (requires CONFIG_I2C_DEBUG_CORE=y)
echo 1 > /sys/module/i2c_core/parameters/debug
```

References

Official Resources

- [Linux I2C Subsystem Documentation](#)
- [Linux I2C Device Tree Bindings](#)
- [i2c-tools Source and Documentation](#)
- [OpenBMC Documentation](#)

Related Guides

- [D-Bus Sensors Guide](#)
- [Hwmon Sensors Guide](#)
- [Entity Manager Guide](#)
- [GPIO Management Guide](#)

External Documentation

- [MIPI I3C Specification](#)
 - [Linux I3C Subsystem](#)
 - [ASPEED AST2600 Datasheet \(I2C/I3C Controllers\)](#)
 - [PCA9548 I2C Mux Datasheet \(NXP\)](#)
-

Tested on: QEMU ast2600-evb (emulated I2C buses with default devices). Full I2C topology and mux testing requires physical hardware with wired I2C devices. Last updated: 2026-02-06

PSU & PMBus Management

Monitor power supplies through the PMBus protocol, from kernel hwmon interfaces through D-Bus sensors to Redfish telemetry.

Table of Contents

1. [Overview](#)
2. [Architecture](#)
 1. [PSU Monitoring Data Flow](#)
 2. [D-Bus Interfaces](#)
 3. [Key Repositories](#)
3. [Configuration](#)
 1. [PMBus Sysfs Interface](#)
 2. [Device Tree Configuration](#)
 3. [Verify Kernel Driver Binding](#)
 4. [Entity Manager PSU Configuration](#)
 5. [Voltage Regulator Configuration \(phosphor-regulators\)](#)
 6. [Build-Time Configuration \(Yocto\)](#)
 7. [Deploy Configuration via Yocto bbappend](#)
 1. [Entity Manager PSU Config](#)
 2. [Regulators Config](#)
4. [Porting Guide](#)
 1. [Step 1: Identify PSU Hardware](#)
 2. [Step 2: Add Device Tree Entries](#)
 3. [Step 3: Verify Kernel Binding](#)
 4. [Step 4: Create Entity Manager JSON](#)
 5. [Step 5: Add Yocto bbappend Files](#)
 6. [Step 6: Verify End-to-End](#)
5. [Code Examples](#)
 1. [Reading PSU Sensors from D-Bus](#)
 2. [Querying PSU Inventory via D-Bus](#)
 3. [Querying PSU Data via Redfish](#)
6. [Troubleshooting](#)
 1. [PSU Not Detected on I2C](#)
 2. [No Sensors Appearing on D-Bus](#)
 3. [Incorrect Sensor Readings](#)
 4. [PSU Fault Detected](#)
 5. [Regulators Service Fails to Start](#)
 6. [PSU Threshold Alarms Not Triggering](#)
7. [References](#)

Overview

Power Supply Units (PSUs) in server platforms communicate over the **PMBus** protocol, an I2C-based standard for power device management. OpenBMC provides a full monitoring pipeline that reads PMBus telemetry from hardware and exposes it through standardized interfaces for out-of-band management.

The pipeline flows through five layers:

1. **PMBus hardware** – the PSU responds to I2C commands defined by the PMBus specification
2. **Linux hwmon** – the kernel `pmbus` driver translates PMBus registers into sysfs attributes
3. **phosphor-power / dbus-sensors** – userspace daemons read hwmon and publish D-Bus sensor objects
4. **D-Bus** – the common bus where all OpenBMC services exchange sensor data and inventory
5. **Redfish / IPMI** – northbound interfaces that expose PSU data to remote management tools

This guide covers the complete workflow: configuring the kernel driver, writing Entity Manager JSON for PSU discovery, setting up phosphor-regulators for voltage regulator control, reading sensor data through D-Bus and Redfish, and troubleshooting common issues.

PSU monitoring is **partially testable** in QEMU. The ast2600-evb QEMU target emulates I2C buses, but real PMBus device responses require physical hardware or a PMBus simulator connected to the I2C bus.

Architecture

PSU Monitoring Data Flow



D-Bus Interfaces

Interface	Object Path	Description
xyz.openbmc_project.Sensor.Value	/xyz/openbmc_project/sensors/voltage/PSU0_Input_Voltage	Voltage, current, power, temperature readings
xyz.openbmc_project.Inventory.Item	/xyz/openbmc_project/inventory/system/chassis/powersupply0	PSU presence and identity
xyz.openbmc_project.Inventory.Decorator.Asset	/xyz/openbmc_project/inventory/system/chassis/powersupply0	Manufacturer, model, serial number
xyz.openbmc_project.State.Decorator.OperationalStatus	/xyz/openbmc_project/inventory/system/chassis/powersupply0	Functional status (true/false)

Key Repositories

Repository	Component	Description
phosphor-power	phosphor-power-supply/	PSU monitoring daemon
phosphor-power	phosphor-regulators/	Voltage regulator configuration and control
dbus-sensors	src/PSUSensor.cpp	PSU sensor daemon (reads hwmon, publishes D-Bus)
entity-manager	JSON configs	PSU hardware discovery and configuration

Configuration

PMBus Sysfs Interface

When the kernel `pmbus` driver binds to a PSU, it creates hwmon attributes under `/sys/class/hwmon/hwmonN/`. The following table lists the most common attributes and their corresponding PMBus registers.

Sysfs Attribute	PMBus Command	Code	Description	Unit
<code>in1_input</code>	READ_VIN	0x88	Input voltage	millivolts
<code>in2_input</code>	READ_VOUT	0x8B	Output voltage	millivolts
<code>curr1_input</code>	READ_IIN	0x89	Input current	milliamps
<code>curr2_input</code>	READ_IOUT	0x8C	Output current	milliamps
<code>power1_input</code>	READ_PIN	0x97	Input power	microwatts
<code>power2_input</code>	READ_POUT	0x96	Output power	microwatts
<code>temp1_input</code>	READ_TEMPERATURE_1	0x8D	Temperature sensor 1	millidegrees C
<code>temp2_input</code>	READ_TEMPERATURE_2	0x8E	Temperature sensor 2	millidegrees C
<code>fan1_input</code>	READ_FAN_SPEED_1	0x90	Fan speed	RPM

The PMBus STATUS_WORD register (0x79) provides a summary of all fault conditions:


```
# Read STATUS_WORD directly from I2C
i2cget -y 3 0x58 0x79 w

# Bit definitions (STATUS_WORD, 16-bit):
#   Bit 15: VOUT fault or warning
#   Bit 14: IOUT/POUT fault or warning
#   Bit 13: INPUT fault or warning
#   Bit 12: MFR specific fault
#   Bit 11: POWER_GOOD negated
#   Bit 6: OFF (unit not providing power)
#   Bit 5: FANS fault or warning
#   Bit 4: OTHER (reserved)
#   Bit 3: VIN_UV fault
#   Bit 2: TEMPERATURE fault or warning
#   Bit 1: CML (communication/logic fault)
#   Bit 0: None of the above
```

Use `i2cget` and `i2cdump` to verify PMBus communication before configuring the monitoring stack. If raw I2C reads fail, the PSU address may be wrong or the I2C bus is not enabled in the device tree.

Device Tree Configuration

Declare the PSU on its I2C bus in the device tree so the kernel `pmbus` driver binds automatically at boot:

```
&i2c3 {
    status = "okay";

    psu0: psu@58 {
        compatible = "pmbus";
        reg = <0x58>;
    };

    psu1: psu@59 {
        compatible = "pmbus";
        reg = <0x59>;
    };
};
```

If your PSU has a vendor-specific driver (for example, `ibm,cffps` or `delta,dps800`), use the vendor compatible string instead of generic `"pmbus"`. Vendor drivers expose additional attributes such as firmware version and detailed fault registers.

Verify Kernel Driver Binding

```
# Check the I2C bus for devices
i2cdetect -y 3

# Verify the pmbus driver bound
ls /sys/bus/i2c/devices/3-0058/driver
# Should show symlink to pmbus or vendor driver

# Find the hwmon path
ls /sys/bus/i2c/devices/3-0058/hwmon/
# hwmon5

# Read input voltage (millivolts)
cat /sys/class/hwmon/hwmon5/in1_input
# 229800 (229.8 V)

# Read output power (microwatts)
cat /sys/class/hwmon/hwmon5/power2_input
# 245000000 (245 W)
```

Entity Manager PSU Configuration

Entity Manager discovers PSUs and publishes their configuration on D-Bus so that sensor daemons know which devices to monitor. Create a JSON configuration file for your platform.

File: `/usr/share/entity-manager/configurations/my-platform-psu.json`

```

{
  "Name": "PSU0",
  "Type": "Board",
  "Probe": "xyz.openbmc_project.FruDevice({'PRODUCT_PRODUCT_NAME': 'MyPSU-1200W'})",

  "Exposes": [
    {
      "Name": "PSU0",
      "Type": "pmbus",
      "Bus": 3,
      "Address": "0x58",
      "Labels": [
        "vin",
        "vout1",
        "iin",
        "iout1",
        "pin",
        "pout1",
        "temp1",
        "temp2",
        "fan1"
      ],
      "Thresholds": [
        {
          "Direction": "greater than",
          "Label": "temp1",
          "Name": "upper critical",
          "Severity": 1,
          "Value": 100
        },
        {
          "Direction": "greater than",
          "Label": "temp1",
          "Name": "upper warning",
          "Severity": 0,
          "Value": 85
        },
        {
          "Direction": "less than",
          "Label": "vin",
          "Name": "lower critical",
          "Severity": 1,
          "Value": 180
        },
        {
          "Direction": "less than",
          "Label": "vin",
          "Name": "lower warning",
          "Severity": 0,
          "Value": 200
        }
      ]
    }
  ],

  "xyz.openbmc_project.Inventory.Item": {
    "Present": true,
    "PrettyName": "Power Supply 0"
  },

  "xyz.openbmc_project.Inventory.Decorator.Asset": {
    "Manufacturer": "$PRODUCT_MANUFACTURER",
    "Model": "$PRODUCT_PRODUCT_NAME",

```

```

    "SerialNumber": "$PRODUCT_SERIAL_NUMBER"
  }
}

```

Key fields:

Field	Description
<code>Probe</code>	Matches against FRU data to detect PSU presence. Use <code>"TRUE"</code> for always-present PSUs.
<code>Type</code> (in Exposes)	Set to <code>"pmbus"</code> so PSUSensor daemon claims this device.
<code>Bus</code> / <code>Address</code>	I2C bus number and 7-bit hex address of the PSU.
<code>Labels</code>	Which PMBus sensor channels to expose. Maps to hwmon attribute names.
<code>Thresholds</code>	Warning and critical limits for alarm generation.
<code>\$PRODUCT_*</code>	Entity Manager substitutes these from FRU EEPROM data at runtime.

The `Labels` array controls which sensors appear on D-Bus. If you omit a label, PSUSensor will not create a sensor for that channel even if the hwmon attribute exists. Always verify which labels your PSU actually supports by checking the hwmon sysfs attributes.

Voltage Regulator Configuration (phosphor-regulators)

phosphor-regulators manages on-board voltage regulators (VRMs) that use PMBus for control. It reads a JSON configuration file to set output voltages during power-on and monitor rail health during operation.

File: `/usr/share/phosphor-regulators/config.json`

```

{
  "rules": [
    {
      "id": "set_voltage_rule",
      "actions": [
        {
          "pmbus_write_vout_command": {
            "format": "linear",
            "exponent": -8
          }
        }
      ]
    },
    {
      "id": "read_sensors_rule",
      "actions": [
        {
          "pmbus_read_sensor": {
            "type": "vout",
            "command": "0x8B",
            "format": "linear_16"
          }
        },
        {
          "pmbus_read_sensor": {
            "type": "iout",
            "command": "0x8C",
            "format": "linear_11"
          }
        },
        {
          "pmbus_read_sensor": {
            "type": "temperature",
            "command": "0x8D",
            "format": "linear_11"
          }
        }
      ]
    }
  ],
  "chassis": [
    {
      "number": 1,
      "inventory_path": "/xyz/openbmc_project/inventory/system/chassis",
      "devices": [
        {
          "id": "vdd_cpu0",
          "is_regulator": true,
          "fru": "/xyz/openbmc_project/inventory/system/chassis/motherboard/cpu0",
          "i2c_interface": {
            "bus": 5,
            "address": "0x40"
          },
          "presence_detection": {
            "rule_id": "detect_presence_rule"
          },
          "configuration": {
            "volts": 1.0,
            "rule_id": "set_voltage_rule"
          },
          "rails": [
            {
              "id": "vdd_cpu0_rail",

```

```

        "sensor_monitoring": {
            "rule_id": "read_sensors_rule"
        }
    }
]
}
]
}
}

```

Configuration sections:

Section	Purpose
<code>rules</code>	Reusable action sequences for reading sensors or writing voltage commands.
<code>chassis</code>	Top-level grouping that maps to a physical chassis in inventory.
<code>devices</code>	Individual voltage regulators with I2C bus/address and FRU path.
<code>configuration</code>	Voltage set point and rule to apply during power-on.
<code>rails</code>	Output rails to monitor; each rail can have sensor monitoring rules.

The `format` field in `pmbus_read_sensor` specifies how to decode the PMBus register. Use `linear_11` for current, power, and temperature readings (11-bit mantissa, 5-bit exponent). Use `linear_16` for VOUT readings (16-bit mantissa with exponent from VOUT_MODE register).

Build-Time Configuration (Yocto)

Include the PSU monitoring components in your image:

```

# In your machine .conf or image recipe

# PSU sensor daemon (from dbus-sensors)
IMAGE_INSTALL:append = " psusensor"

# PSU monitoring and inventory (from phosphor-power)
IMAGE_INSTALL:append = " \
    phosphor-power-psu-monitor \
    phosphor-power-regulators \
    phosphor-power-utils \
"

```

Deploy Configuration via Yocto bbappend

Entity Manager PSU Config

```
# meta-myplatform/recipes-phosphor/configuration/entity-manager/
# --- files/
# |   --- my-platform-psu.json
# --- entity-manager_%.bbappend

cat > entity-manager_%.bbappend << 'EOF'
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
SRC_URI += "file://my-platform-psu.json"

do_install:append() {
    install -d ${D}${datadir}/entity-manager/configurations
    install -m 0444 ${WORKDIR}/my-platform-psu.json \
        ${D}${datadir}/entity-manager/configurations/
}
EOF
```

Regulators Config

```
# meta-myplatform/recipes-phosphor/power/phosphor-regulators/
# --- files/
# |   --- my-regulators-config.json
# --- phosphor-regulators_%.bbappend

cat > phosphor-regulators_%.bbappend << 'EOF'
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
SRC_URI += "file://my-regulators-config.json"

do_install:append() {
    install -d ${D}${datadir}/phosphor-regulators
    install -m 0644 ${WORKDIR}/my-regulators-config.json \
        ${D}${datadir}/phosphor-regulators/config.json
}
EOF
```

Porting Guide

Follow these steps to bring up PSU monitoring on a new platform.

Step 1: Identify PSU Hardware

Determine the following from your platform schematic:

Parameter	Example	How to find
I2C bus number	3	Schematic, trace from BMC I2C controller to PSU connector
PSU I2C address	0x58	PSU datasheet or <code>i2cdetect -y <bus></code>
Number of PSUs	2	Platform design (N+1 redundancy)
PMBus commands supported	See datasheet	PSU vendor documentation
Vendor-specific driver	<code>"delta,dps800"</code>	Kernel driver list or <code>"pmbus"</code> for generic

Step 2: Add Device Tree Entries

```
// In your platform device tree (aspeed-bmc-myplatform.dts)
&i2c3 {
    status = "okay";

    psu0: psu@58 {
        compatible = "pmbus";
        reg = <0x58>;
    };

    psu1: psu@59 {
        compatible = "pmbus";
        reg = <0x59>;
    };
};
```

Step 3: Verify Kernel Binding

After booting with the updated device tree:

```
# Confirm devices appear on the bus
i2cdetect -y 3

# Verify driver bound
ls /sys/bus/i2c/devices/3-0058/driver

# List hwmon attributes to determine which Labels to use
ls /sys/bus/i2c/devices/3-0058/hwmon/hwmon*/
# in1_input in2_input curr1_input curr2_input
# power1_input power2_input temp1_input fan1_input
```

Step 4: Create Entity Manager JSON

Map each hwmon attribute to a Label in your JSON configuration. Only include labels that exist in sysfs.

Step 5: Add Yocto bbappend Files

Install your Entity Manager JSON and, if applicable, your phosphor-regulators config through bbappend files as shown in the Configuration section.

Step 6: Verify End-to-End

```
# Check Entity Manager detected the PSU
busctl tree xyz.openbmc_project.EntityManager | grep -i psu

# Verify PSU sensors on D-Bus
busctl tree xyz.openbmc_project.PSUSensor

# Read a sensor value
busctl get-property xyz.openbmc_project.PSUSensor \
    /xyz/openbmc_project/sensors/voltage/PSU0_Input_Voltage \
    xyz.openbmc_project.Sensor.Value Value

# Check via Redfish
curl -k -u root:0penBmc \
    https://localhost:2443/redfish/v1/Chassis/chassis/Power
```

Code Examples

Working examples are available in the [examples/psu-pmbus](#) directory:

- `psu-entity-manager.json` – Entity Manager configuration for a dual-PSU platform
- `regulators-config.json` – phosphor-regulators configuration with VRM rules
- `psu-status-check.sh` – Shell script to read PMBus STATUS_WORD and decode fault bits

Reading PSU Sensors from D-Bus

```
#!/bin/bash
# Read all PSU sensors and display a summary

PSU_SERVICE="xyz.openbmc_project.PSUSensor"
SENSOR_BASE="/xyz/openbmc_project/sensors"

echo "=== PSU Sensor Readings ==="

for type in voltage current power temperature; do
    echo ""
    echo "--- ${type} ---"
    busctl tree ${PSU_SERVICE} 2>/dev/null \
        | grep "${type}" \
        | while read -r path; do
            value=$(busctl get-property ${PSU_SERVICE} \
                "${path}" \
                xyz.openbmc_project.Sensor.Value Value 2>/dev/null \
                | awk '{print $2}')
            name=$(basename "${path}")
            echo "  ${name}: ${value}"
        done
    done
done
```

Querying PSU Inventory via D-Bus

```
# List PSU inventory objects
busctl tree xyz.openbmc_project.Inventory.Manager | grep powersupply

# Get PSU presence
busctl get-property xyz.openbmc_project.Inventory.Manager \
    /xyz/openbmc_project/inventory/system/chassis/powersupply0 \
    xyz.openbmc_project.Inventory.Item Present

# Get PSU asset information (manufacturer, model, serial)
busctl get-property xyz.openbmc_project.Inventory.Manager \
    /xyz/openbmc_project/inventory/system/chassis/powersupply0 \
    xyz.openbmc_project.Inventory.Decorator.Asset Manufacturer

busctl get-property xyz.openbmc_project.Inventory.Manager \
    /xyz/openbmc_project/inventory/system/chassis/powersupply0 \
    xyz.openbmc_project.Inventory.Decorator.Asset SerialNumber
```

Querying PSU Data via Redfish

```
# Get power supply collection
curl -k -u root:openBmc \
  https://localhost:2443/redfish/v1/Chassis/chassis/Power \
  | jq '.PowerSupplies'

# Get specific sensor reading
curl -k -u root:openBmc \
  https://localhost:2443/redfish/v1/Chassis/chassis/Sensors/PSU0_Input_Voltage

# Get power supply inventory with FRU data
curl -k -u root:openBmc \
  https://localhost:2443/redfish/v1/Chassis/chassis/Power \
  | jq '.PowerSupplies[] | {Name, Model, Manufacturer, SerialNumber, Status}'
```

Troubleshooting

PSU Not Detected on I2C

```
# Scan the I2C bus
i2cdetect -y 3
# If address 0x58 shows "--", the PSU is not responding

# Check device tree
cat /sys/firmware/devicetree/base/ahb/apb/bus@1e78a000/i2c-bus@180/psu@58/compatible

# Try reading MFR_ID to confirm PMBus communication
i2cget -y 3 0x58 0x99 s # MFR_ID (block read)
i2cget -y 3 0x58 0x79 w # STATUS_WORD
```

If `i2cdetect` shows the address but the pmbus driver does not bind, you may need to manually instantiate the device: `echo pmbus 0x58 > /sys/bus/i2c/devices/i2c-3/new_device`

No Sensors Appearing on D-Bus

```
# Check PSUSensor daemon status
systemctl status xyz.openbmc_project.psusensor
journalctl -u xyz.openbmc_project.psusensor -f

# Check Entity Manager found the PSU configuration
journalctl -u xyz.openbmc_project.EntityManager | grep -i psu

# Verify the hwmon sysfs path exists
ls /sys/bus/i2c/devices/3-0058/hwmon/

# Check if Labels in Entity Manager config match actual hwmon attributes
ls /sys/bus/i2c/devices/3-0058/hwmon/hwmon*/
```

A common failure is mismatched Labels. If your Entity Manager config lists `"vout1"` but the hwmon driver only creates `in2_input` (without a label file), PSUSensor may not find the sensor. Check the hwmon attribute names and adjust Labels accordingly.

Incorrect Sensor Readings

```
# Compare D-Bus value with raw hwmon value
cat /sys/class/hwmon/hwmon5/in1_input
# 229800 (millivolts = 229.8 V)

busctl get-property xyz.openbmc_project.PSUSensor \
    /xyz/openbmc_project/sensors/voltage/PSU0_Input_Voltage \
    xyz.openbmc_project.Sensor.Value Value
# d 229.8

# If values differ, check for scaling issues in the kernel driver
# or Entity Manager ScaleFactor/Offset properties
```

PSU Fault Detected

```
# Read STATUS_WORD to identify the fault type
i2cget -y 3 0x58 0x79 w

# Decode individual status registers for details
i2cget -y 3 0x58 0x7A b # STATUS_VOUT
i2cget -y 3 0x58 0x7B b # STATUS_IOUT
i2cget -y 3 0x58 0x7C b # STATUS_INPUT
i2cget -y 3 0x58 0x7D b # STATUS_TEMPERATURE
i2cget -y 3 0x58 0x81 b # STATUS_FANS_1_2

# Clear faults after resolving the issue
i2cset -y 3 0x58 0x03 # CLEAR_FAULTS command

# Check PSU operational status on D-Bus
busctl get-property xyz.openbmc_project.Inventory.Manager \
    /xyz/openbmc_project/inventory/system/chassis/powersupply0 \
    xyz.openbmc_project.State.Decorator.OperationalStatus Functional
```

Regulators Service Fails to Start

```
# Check service status
systemctl status phosphor-regulators
journalctl -u phosphor-regulators --no-pager | tail -30

# Validate JSON configuration syntax
python3 -m json.tool /usr/share/phosphor-regulators/config.json > /dev/null

# Common issues:
# - Invalid I2C bus/address in config (device not present)
# - Missing rule_id reference (typo in rule name)
# - Incorrect PMBus format (linear_11 vs linear_16)
```

PSU Threshold Alarms Not Triggering

```
# Verify thresholds are set on D-Bus
busctl introspect xyz.openbmc_project.PSUSensor \
    /xyz/openbmc_project/sensors/temperature/PSU0_Temperature_1 \
    | grep -i threshold

# Check that sel-logger is running (required for SEL entries)
systemctl status xyz.openbmc_project.sel-logger

# Monitor threshold events in real time
journalctl -u xyz.openbmc_project.sel-logger -f | grep -i threshold
```

References

- [phosphor-power](#) – PSU monitoring, regulators, power sequencing
- [dbus-sensors PSUSensor](#) – PSU sensor daemon source
- [entity-manager](#) – JSON-based hardware configuration
- [PMBus Specification](#) – PMBus protocol standard and command reference
- [Linux Kernel PMBus Documentation](#) – Kernel PMBus core driver and sysfs interface
- [phosphor-dbus-interfaces](#) – D-Bus interface definitions for sensors and inventory
- [Power Management Guide](#) – Related guide covering power sequencing, capping, and restore policy

Tested on: OpenBMC master, QEMU ast2600-evb (I2C bus enumeration verified; full PMBus telemetry requires physical PSU hardware)

LDAP Integration

Authenticate BMC users against external LDAP or Active Directory servers and map directory groups to OpenBMC privilege roles.

Table of Contents

1. [Overview](#)
2. [Architecture](#)
 1. [D-Bus Interfaces](#)
 2. [Key Dependencies](#)
3. [Configuration](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [Configuration Files](#)
 3. [Runtime Verification](#)
4. [LDAP Client Configuration](#)
 1. [Configure OpenLDAP via Redfish](#)
 2. [Configure OpenLDAP via D-Bus \(busctl\)](#)
 3. [View Current LDAP Configuration](#)
 4. [Disable LDAP](#)
5. [LDAPS Certificate Setup](#)
 1. [Install the LDAP Server CA Certificate](#)
 2. [Switch to LDAPS](#)
 3. [Verify TLS Connection](#)
6. [Group-to-Privilege Mapping \(RoleMapping\)](#)
 1. [Available Roles](#)
 2. [Create Role Mappings via Redfish](#)
 3. [Create Role Mappings via D-Bus](#)
 4. [View Existing Mappings](#)
 5. [Delete a Role Mapping](#)
7. [Active Directory vs OpenLDAP](#)
 1. [Key Differences](#)
 2. [Configure Active Directory via Redfish](#)
 3. [Active Directory Role Mapping](#)
8. [Porting Guide](#)
 1. [Step 1: Prerequisites](#)
 2. [Step 2: Add LDAP Packages to Your Image](#)
 3. [Step 3: Configure Default PAM Stack](#)
 4. [Step 4: Bake Default nslcd Configuration \(Optional\)](#)
 5. [Step 5: Build, Flash, and Verify](#)

9. [Code Examples](#)

1. [Example 1: LDAP Configuration Script](#)
2. [Example 2: LDAP Connection Test Script](#)

10. [Troubleshooting](#)

1. [Issue: LDAP Authentication Fails](#)
2. [Issue: LDAPS Certificate Not Trusted](#)
3. [Issue: Role Mapping Not Applied](#)
4. [Issue: nslcd Service Not Running](#)
5. [Debug Commands](#)

11. [Deep Dive](#)

1. [LDAP Authentication Flow](#)
2. [LDAP Configuration Persistence](#)
3. [Source Code Reference](#)

12. [References](#)

1. [Official Resources](#)
 2. [Related Guides](#)
 3. [External Documentation](#)
-

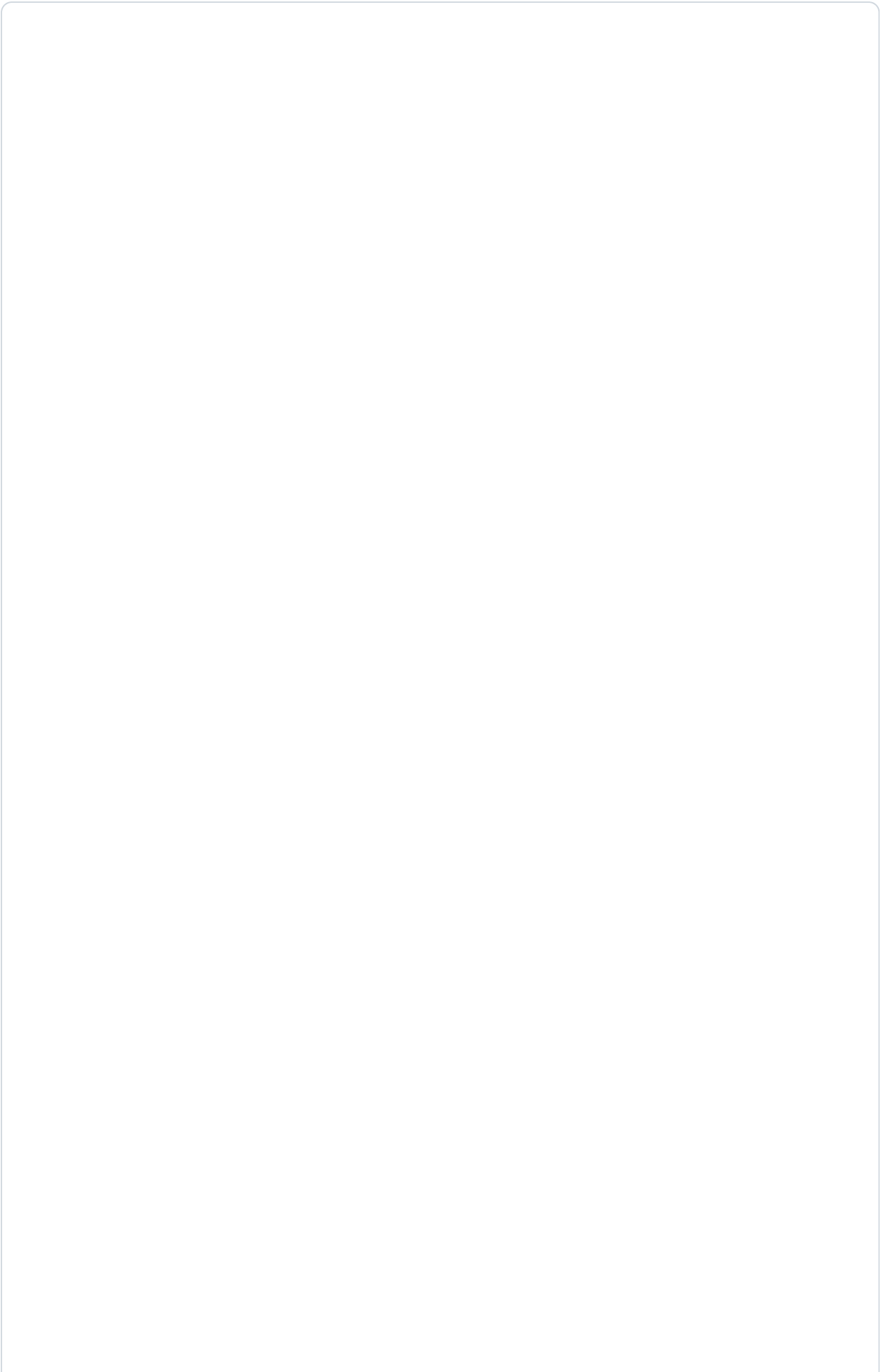
Overview

OpenBMC supports centralized user authentication through LDAP (Lightweight Directory Access Protocol). Instead of managing accounts locally on every BMC, you can point the BMC at an existing LDAP or Active Directory server so that users authenticate with their enterprise credentials.

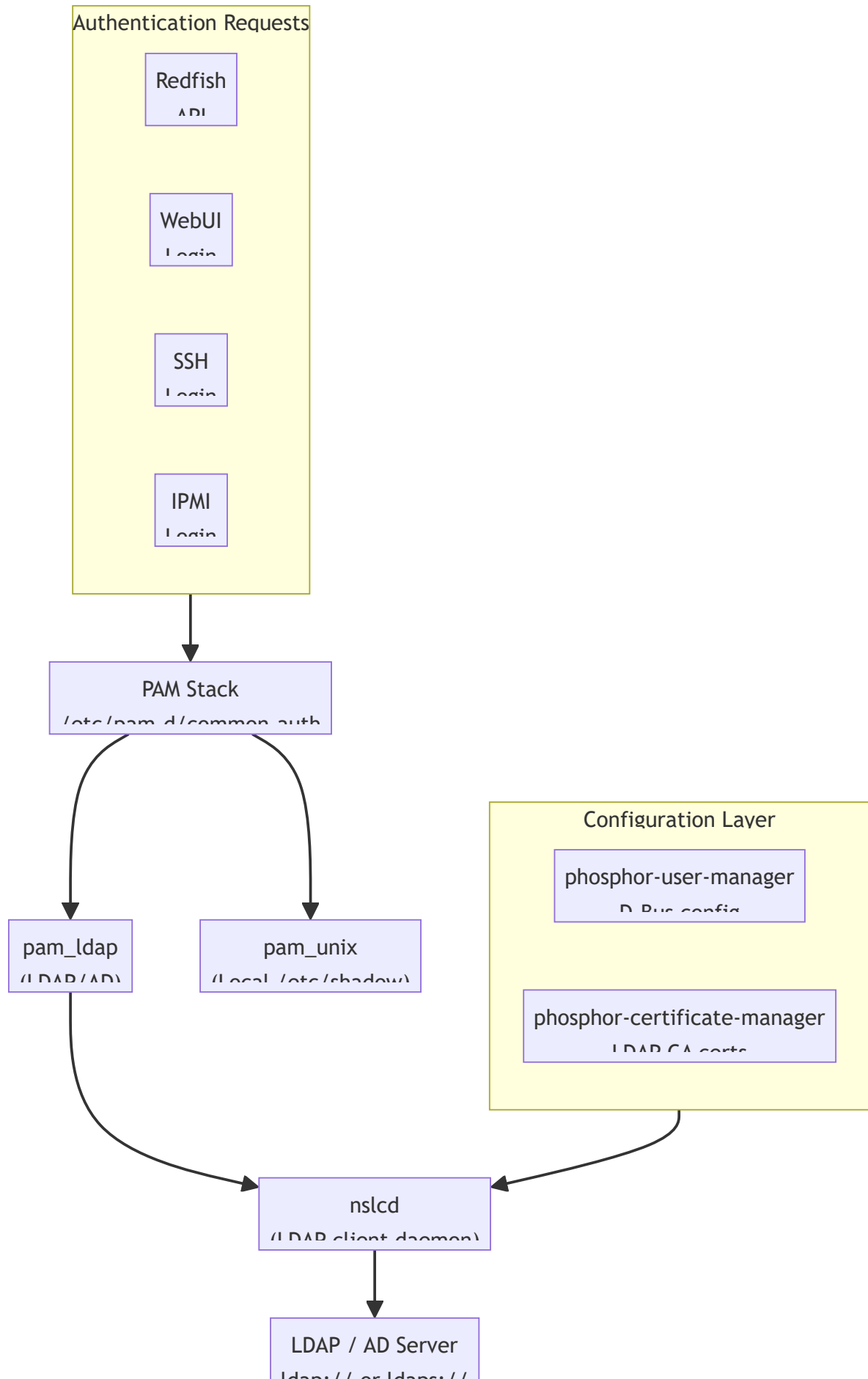
phosphor-user-manager implements the LDAP client configuration, while **nslcd** (Name Service LDAP Connection Daemon) handles the actual LDAP communication. The Redfish `AccountService` resource exposes LDAP and Active Directory configuration, and D-Bus interfaces provide the underlying control layer.

Key concepts covered:

- Configuring LDAP and Active Directory authentication via Redfish and D-Bus
 - Setting up LDAPS (LDAP over TLS) with trusted certificates
 - Mapping LDAP groups to OpenBMC privilege roles (RoleMapping)
 - Differences between OpenLDAP and Active Directory configurations
 - Troubleshooting authentication failures with `ldapsearch` and debug logging
-



LDAP Integration Architecture



► ASCII-art version (for comparison)

D-Bus Interfaces

Interface	Object Path	Description
<code>xyz.openbmc_project.User.Ldap.Create</code>	<code>/xyz/openbmc_project/user/ldap</code>	Create LDAP configuration
<code>xyz.openbmc_project.User.Ldap.Config</code>	<code>/xyz/openbmc_project/user/ldap/config</code>	LDAP server settings
<code>xyz.openbmc_project.User.PrivilegeMapper.Create</code>	<code>/xyz/openbmc_project/user/ldap/config</code>	Create group-to-role mappings
<code>xyz.openbmc_project.User.PrivilegeMapperEntry</code>	<code>/xyz/openbmc_project/user/ldap/config/role_map/<id></code>	Individual role mapping entry

Key Dependencies

- **phosphor-user-manager**: Manages LDAP configuration and group-to-role mapping on D-Bus
- **nss-pam-ldapd (nslcd)**: Performs actual LDAP queries and PAM authentication
- **phosphor-certificate-manager**: Manages LDAP CA certificates for LDAPS connections
- **bmcweb**: Exposes Redfish AccountService endpoints for LDAP/AD configuration

Configuration

Build-Time Configuration (Yocto)

```
# In your machine .conf or local.conf

# Include LDAP support packages
IMAGE_INSTALL:append = " \
    phosphor-user-manager \
    nss-pam-ldapd \
    pam-plugin-ldap \
    openldap-bin \
"

# Enable LDAP in phosphor-user-manager
EXTRA_OEMESON:pn-phosphor-user-manager = " \
    -Dldap=enabled \
"
```

The `openldap-bin` package provides the `ldapsearch` command-line tool, which is valuable for testing LDAP connectivity directly from the BMC.

Configuration Files

Config File	Path	Description
PAM auth config	<code>/etc/pam.d/common-auth</code>	Authentication module stack (includes pam_ldap)
LDAP client config	<code>/etc/nslcd.conf</code>	nslcd LDAP client settings (auto-generated)
NSS config	<code>/etc/nsswitch.conf</code>	Name service switch (adds ldap lookup)
LDAP persistence	<code>/var/lib/phosphor-user-manager/ldap.conf</code>	Persisted LDAP configuration
LDAP CA certs	<code>/etc/ssl/certs/ldap/</code>	Trusted CA certificates for LDAPS

Runtime Verification

```
# Verify LDAP-related services are running
systemctl status phosphor-user-manager
systemctl status nslcd

# View current LDAP configuration via D-Bus
busctl introspect xyz.openbmc_project.User.Manager \
    /xyz/openbmc_project/user/ldap

# Check nslcd configuration
cat /etc/nslcd.conf

# Check PAM is configured for LDAP
cat /etc/pam.d/common-auth
```

LDAP Client Configuration

Configure OpenLDAP via Redfish

```
# Enable OpenLDAP authentication
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{
    "LDAP": {
      "ServiceEnabled": true,
      "ServiceAddresses": ["ldap://ldap.example.com"],
      "Authentication": {
        "AuthenticationType": "UsernameAndPassword",
        "Username": "cn=readonly,dc=example,dc=com",
        "Password": "bindpassword"
      },
      "LDAPService": {
        "SearchSettings": {
          "BaseDistinguishedNames": ["dc=example,dc=com"],
          "UsernameAttribute": "uid",
          "GroupsAttribute": "memberOf"
        }
      }
    }
  }' \
  https://localhost/redfish/v1/AccountService
```

Configure OpenLDAP via D-Bus (busctl)

```
# Create LDAP configuration
busctl call xyz.openbmc_project.User.Manager \
  /xyz/openbmc_project/user/ldap \
  xyz.openbmc_project.User.Ldap.Create \
  Create ssssa{ss} \
  "ldap://ldap.example.com" \
  "cn=readonly,dc=example,dc=com" \
  "dc=example,dc=com" \
  "uid" \
  "OpenLDAP" \
  0

# Verify the configuration
busctl get-property xyz.openbmc_project.User.Manager \
  /xyz/openbmc_project/user/ldap/config \
  xyz.openbmc_project.User.Ldap.Config \
  LDAPServerURI

# Check if LDAP is enabled
busctl get-property xyz.openbmc_project.User.Manager \
  /xyz/openbmc_project/user/ldap/config \
  xyz.openbmc_project.User.Ldap.Config \
  LDAPEnabled
```

View Current LDAP Configuration

```
# View full LDAP settings via Redfish
curl -k -u root:openBmc \
  https://localhost/redfish/v1/AccountService | \
  python3 -m json.tool

# Check LDAP-specific properties
curl -k -u root:openBmc \
  https://localhost/redfish/v1/AccountService | \
  python3 -c "
import sys, json
data = json.load(sys.stdin)
print(json.dumps(data.get('LDAP', {}), indent=2))
"
```

Disable LDAP

```
# Disable via Redfish
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"LDAP": {"ServiceEnabled": false}}' \
  https://localhost/redfish/v1/AccountService
```

LDAPS Certificate Setup

Using plain `ldap://` transmits credentials in cleartext. Always use `ldaps://` (LDAP over TLS) in production environments.

Install the LDAP Server CA Certificate

The BMC must trust the CA that signed the LDAP server certificate. Upload the CA certificate through Redfish or the certificate manager.

```
# Upload LDAP CA certificate via Redfish
curl -k -u root:openBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "CertificateType": "PEM",
    "CertificateString": "-----BEGIN CERTIFICATE-----\nMIIDrzCCApeg...\n-----END CERTIFICATE-----"
}' \
  https://localhost/redfish/v1/AccountService/LDAP/Certificates
```

Switch to LDAPS

After installing the CA certificate, update the service address to use `ldaps://`:

```
# Update LDAP URI to use TLS
curl -k -u root:0penBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{
    "LDAP": {
      "ServiceAddresses": ["ldaps://ldap.example.com"]
    }
  }' \
  https://localhost/redfish/v1/AccountService
```

Verify TLS Connection

```
# Test TLS handshake to the LDAP server
openssl s_client -connect ldap.example.com:636 \
  -CAfile /etc/ssl/certs/ldap/ca.pem

# Test LDAPS with ldapsearch
ldapsearch -x -H ldaps://ldap.example.com \
  -D "cn=readonly,dc=example,dc=com" \
  -w bindpassword \
  -b "dc=example,dc=com" \
  "(uid=testuser)"
```

If the LDAP server uses a self-signed certificate, you must upload that specific certificate as the CA cert. For certificates signed by an internal CA, upload the root CA certificate (and any intermediates) that form the trust chain.

Group-to-Privilege Mapping (RoleMapping)

LDAP group membership determines the OpenBMC privilege level. You create role mappings that associate an LDAP group distinguished name (DN) with a local role.

Available Roles

Local Role	Redfish RoleId	IPMI Privilege	Description
priv-admin	Administrator	4 (admin)	Full access, user management
priv-operator	Operator	3 (operator)	Operations, no user management
priv-user	ReadOnly	2 (user)	Read-only access
priv-callback	Callback	1 (callback)	Limited callback access

Create Role Mappings via Redfish

```
# Map "bmc-admins" LDAP group to Administrator role
curl -k -u root:openBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "RemoteGroup": "cn=bmc-admins,ou=groups,dc=example,dc=com",
    "LocalRole": "Administrator"
  }' \
  https://localhost/redfish/v1/AccountService/LDAP/RemoteRoleMapping

# Map "bmc-operators" LDAP group to Operator role
curl -k -u root:openBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "RemoteGroup": "cn=bmc-operators,ou=groups,dc=example,dc=com",
    "LocalRole": "Operator"
  }' \
  https://localhost/redfish/v1/AccountService/LDAP/RemoteRoleMapping

# Map "bmc-readonly" LDAP group to ReadOnly role
curl -k -u root:openBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "RemoteGroup": "cn=bmc-readonly,ou=groups,dc=example,dc=com",
    "LocalRole": "ReadOnly"
  }' \
  https://localhost/redfish/v1/AccountService/LDAP/RemoteRoleMapping
```

Create Role Mappings via D-Bus

```
# Create a privilege mapping entry
busctl call xyz.openbmc_project.User.Manager \
  /xyz/openbmc_project/user/ldap/config \
  xyz.openbmc_project.User.PrivilegeMapper.Create \
  Create ss \
  "cn=bmc-admins,ou=groups,dc=example,dc=com" \
  "priv-admin"
```

View Existing Mappings

```
# List role mappings via Redfish
curl -k -u root:openBmc \
  https://localhost/redfish/v1/AccountService | \
  python3 -c "
import sys, json
data = json.load(sys.stdin)
ldap = data.get('LDAP', {})
for mapping in ldap.get('RemoteRoleMapping', []):
    print(f"Group: {mapping['RemoteGroup']} -> Role: {mapping['LocalRole']}\n")
"

# List mappings via D-Bus
busctl tree xyz.openbmc_project.User.Manager | grep role_map
```

Delete a Role Mapping

```
# Delete via D-Bus (replace <id> with the mapping ID)
busctl call xyz.openbmc_project.User.Manager \
  /xyz/openbmc_project/user/ldap/config/role_map/<id> \
  xyz.openbmc_project.Object.Delete \
  Delete
```

Active Directory vs OpenLDAP

OpenBMC supports both Active Directory and OpenLDAP. The Redfish AccountService provides separate configuration objects for each.

Key Differences

Attribute	OpenLDAP	Active Directory
Redfish property	LDAP	ActiveDirectory
D-Bus server type	OpenLDAP	ActiveDirectory
Username attribute	uid	SAMAccountName
Groups attribute	memberOf	memberOf
Bind DN format	cn=admin,dc=example,dc=com	admin@example.com or DOMAIN\admin
Base DN format	dc=example,dc=com	DC=example,DC=com
Default port (TLS)	636	636
Default port (plain)	389	389
Group DN format	cn=group,ou=groups,dc=example,dc=com	CN=group,OU=Groups,DC=example,DC=com

You can enable either LDAP or Active Directory, but not both simultaneously. Enabling one disables the other.

Configure Active Directory via Redfish

```
# Configure Active Directory
curl -k -u root:OpenBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{
    "ActiveDirectory": {
      "ServiceEnabled": true,
      "ServiceAddresses": ["ldaps://ad.example.com"],
      "Authentication": {
        "AuthenticationType": "UsernameAndPassword",
        "Username": "svc-bmc@example.com",
        "Password": "ServiceAccountPassword"
      },
      "LDAPService": {
        "SearchSettings": {
          "BaseDistinguishedNames": ["DC=example,DC=com"],
          "UsernameAttribute": "sAMAccountName",
          "GroupsAttribute": "memberOf"
        }
      }
    }
  }' \
  https://localhost/redfish/v1/AccountService
```

Active Directory Role Mapping

```
# Map AD security group to Administrator
curl -k -u root:OpenBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "RemoteGroup": "CN=BMC-Admins,OU=Security Groups,DC=example,DC=com",
    "LocalRole": "Administrator"
  }' \
  https://localhost/redfish/v1/AccountService/ActiveDirectory/RemoteRoleMapping
```

When using Active Directory, the bind account (`Username`) should be a dedicated service account with read-only access to the directory. Use the UPN format (`user@domain.com`) for the bind DN, as it works across all AD forests.

Porting Guide

Follow these steps to enable LDAP authentication on your platform.

Step 1: Prerequisites

Ensure you have:

- ☐ A working OpenBMC build for your platform
- ☐ An accessible LDAP or Active Directory server
- ☐ A bind/service account with read permissions on the directory
- ☐ The CA certificate for the LDAP server (for LDAPS)

Step 2: Add LDAP Packages to Your Image

```
# meta-myplatform/conf/machine/myplatform.conf (or local.conf)
IMAGE_INSTALL:append = " \
    phosphor-user-manager \
    nss-pam-ldapd \
    pam-plugin-ldap \
    openldap-bin \
"

EXTRA_OEMESON:pn-phosphor-user-manager = " \
    -Dldap=enabled \
"
```

Step 3: Configure Default PAM Stack

Create a bbappend to deploy a PAM configuration that includes LDAP:

```
# meta-myplatform/recipes-phosphor/users/phosphor-user-manager/
# └─ files/
#   └─ common-auth
# └─ phosphor-user-manager_%.bbappend

cat > phosphor-user-manager_%.bbappend << 'EOF'
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
SRC_URI += "file://common-auth"

do_install:append() {
    install -d ${D}${sysconfdir}/pam.d
    install -m 0644 ${WORKDIR}/common-auth ${D}${sysconfdir}/pam.d/
}
EOF
```

Example `common-auth` with LDAP:

```
# /etc/pam.d/common-auth
auth sufficient pam_unix.so nullok try_first_pass
auth sufficient pam_ldap.so use_first_pass
auth required pam deny.so
```

Step 4: Bake Default nslcd Configuration (Optional)

To include a factory-default LDAP configuration:

```
# meta-myplatform/recipes-phosphor/users/nss-pam-ldapd/
# └─ files/
#   └─ nslcd.conf
# └─ nss-pam-ldapd_%.bbappend

cat > nss-pam-ldapd_%.bbappend << 'EOF'
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
SRC_URI += "file://nslcd.conf"

do_install:append() {
    install -m 0600 ${WORKDIR}/nslcd.conf ${D}${sysconfdir}/
}
EOF
```

Step 5: Build, Flash, and Verify

```
# Build the image
bitbake obmc-phosphor-image

# After flashing, verify services are running
systemctl status phosphor-user-manager
systemctl status nslcd

# Configure LDAP via Redfish (see Configuration section above)
# Then test authentication
curl -k -u ldapuser:ldappassword \
    https://localhost/redfish/v1/SessionService/Sessions
```

Code Examples

Example 1: LDAP Configuration Script

```
#!/bin/bash
# configure-ldap.sh - Configure LDAP authentication on OpenBMC
# Usage: ./configure-ldap.sh <bmc-host> <ldap-server> <bind-dn> <base-dn>

BMC_HOST="${1:-localhost}"
LDAP_SERVER="${2:-ldaps://ldap.example.com}"
BIND_DN="${3:-cn=readonly,dc=example,dc=com}"
BASE_DN="${4:-dc=example,dc=com}"
BMC_USER="root"
BMC_PASS="0penBmc"

echo "Configuring LDAP on ${BMC_HOST}..."

# Step 1: Enable LDAP
curl -k -u "${BMC_USER}:${BMC_PASS}" -X PATCH \
  -H "Content-Type: application/json" \
  -d "{
    \"LDAP\": {
      \"ServiceEnabled\": true,
      \"ServiceAddresses\": [\"${LDAP_SERVER}\"],
      \"Authentication\": {
        \"AuthenticationType\": \"UsernameAndPassword\",
        \"Username\": \"${BIND_DN}\",
        \"Password\": \"changeme\"
      },
      \"LDAPService\": {
        \"SearchSettings\": {
          \"BaseDistinguishedNames\": [\"${BASE_DN}\"],
          \"UsernameAttribute\": \"uid\",
          \"GroupsAttribute\": \"memberOf\"
        }
      }
    }
  }" \
  "https://${BMC_HOST}/redfish/v1/AccountService"

# Step 2: Add role mappings
for role_pair in \
  "cn=bmc-admins,ou=groups,${BASE_DN}:Administrator" \
  "cn=bmc-operators,ou=groups,${BASE_DN}:Operator" \
  "cn=bmc-readonly,ou=groups,${BASE_DN}:ReadOnly"; do

  group="${role_pair%:*}"
  role="${role_pair##*:}"

  echo "Mapping group '${group}' to role '${role}'..."
  curl -k -u "${BMC_USER}:${BMC_PASS}" -X POST \
    -H "Content-Type: application/json" \
    -d "{
      \"RemoteGroup\": \"${group}\",
      \"LocalRole\": \"${role}\"
    }" \
    "https://${BMC_HOST}/redfish/v1/AccountService/LDAP/RemoteRoleMapping"
done

echo "LDAP configuration complete."
```

Example 2: LDAP Connection Test Script

```
#!/bin/bash
# test-ldap.sh - Test LDAP connectivity and authentication from BMC
# Usage: ./test-ldap.sh <ldap-server> <bind-dn> <base-dn> <test-user>

LDAP_SERVER="${1:-ldaps://ldap.example.com}"
BIND_DN="${2:-cn=readonly,dc=example,dc=com}"
BASE_DN="${3:-dc=example,dc=com}"
TEST_USER="${4:-testuser}"

echo "=== Testing LDAP connection ==="
echo "Server: ${LDAP_SERVER}"
echo "Bind DN: ${BIND_DN}"
echo "Base DN: ${BASE_DN}"
echo ""

# Test 1: Anonymous bind (if allowed)
echo "--- Test 1: Server reachability ---"
ldapsearch -x -H "${LDAP_SERVER}" \
    -b "" -s base "(objectClass=*)" namingContexts 2>&1
echo ""

# Test 2: Authenticated bind
echo "--- Test 2: Bind authentication ---"
read -s -p "Enter bind password: " BIND_PASS
echo ""
ldapsearch -x -H "${LDAP_SERVER}" \
    -D "${BIND_DN}" \
    -w "${BIND_PASS}" \
    -b "${BASE_DN}" \
    "(uid=${TEST_USER}) uid cn memberOf 2>&1
echo ""

# Test 3: Check group membership
echo "--- Test 3: Group membership ---"
ldapsearch -x -H "${LDAP_SERVER}" \
    -D "${BIND_DN}" \
    -w "${BIND_PASS}" \
    -b "${BASE_DN}" \
    "(uid=${TEST_USER}) memberOf 2>&1
```

See the complete examples at [examples/ldap/](#).

Troubleshooting

Issue: LDAP Authentication Fails

Symptom: Users cannot log in with LDAP credentials. Redfish returns 401 Unauthorized.

Cause: LDAP configuration may be incorrect, the bind account may lack permissions, or the server may be unreachable.

Solution:

1. Verify LDAP is enabled:

```
curl -k -u root:0penBmc \
  https://localhost/redfish/v1/AccountService | \
  python3 -c "import sys,json; print(json.load(sys.stdin).get('LDAP',{}).get('ServiceEnabled'))"
```

2. Test LDAP connectivity from the BMC:

```
ldapsearch -x -H ldap://ldap.example.com \
  -D "cn=readonly,dc=example,dc=com" \
  -w bindpassword \
  -b "dc=example,dc=com" \
  "(uid=testuser)"
```

3. Check nslcd logs:

```
journalctl -u nslcd -f
```

Issue: LDAPS Certificate Not Trusted

Symptom: LDAPS connections fail with “TLS handshake failure” or “certificate verify failed”.

Cause: The BMC does not trust the CA that signed the LDAP server certificate.

Solution:

1. Verify the LDAP server certificate:

```
openssl s_client -connect ldap.example.com:636 \
  -CAfile /etc/ssl/certs/ldap/ca.pem
```

2. Upload the CA certificate:

```
curl -k -u root:0penBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "CertificateType": "PEM",
    "CertificateString": "-----BEGIN CERTIFICATE-----\n...\n-----END CERTIFICATE-----"
  }' \
  https://localhost/redfish/v1/AccountService/LDAP/Certificates
```

3. Restart nslcd after installing the certificate:

```
systemctl restart nslcd
```

If the LDAP server certificate has expired or uses an untrusted CA, authentication will fail silently. Always check certificate validity dates with `openssl x509 -in cert.pem -noout -dates`.

Issue: Role Mapping Not Applied

Symptom: LDAP users authenticate successfully but receive no privileges or the wrong privilege level.

Cause: Group-to-role mappings are missing, or the LDAP group DN in the mapping does not match the group DN returned by the server.

Solution:

1. Verify the user's group membership on the LDAP server:

```
ldapsearch -x -H ldap://ldap.example.com \
-D "cn=readonly,dc=example,dc=com" \
-w bindpassword \
-b "dc=example,dc=com" \
"(uid=testuser)" memberOf
```

2. Compare the returned group DN with your role mapping. The DN must match exactly (case-sensitive for OpenLDAP):

```
curl -k -u root:openBmc \
https://localhost/redfish/v1/AccountService | \
python3 -c "
import sys, json
data = json.load(sys.stdin)
for m in data.get('LDAP', {}).get('RemoteRoleMapping', []):
    print(f"{m['RemoteGroup']} -> {m['LocalRole']}\n")
"
```

3. If the DN's do not match, delete the incorrect mapping and create a new one with the correct group DN.

Issue: nslcd Service Not Running

Symptom: All LDAP authentication attempts fail. The `nslcd` service shows as inactive.

Cause: The nslcd service may have failed to start due to a bad configuration or missing dependencies.

Solution:

```
# Check nslcd status
systemctl status nslcd

# View startup logs
journalctl -u nslcd --no-pager

# Restart the service
systemctl restart nslcd

# Verify nslcd config is valid
cat /etc/nslcd.conf
```

Debug Commands

```
# Enable debug logging for phosphor-user-manager
# (Requires recompile with debug flags or environment override)
journalctl -u phosphor-user-manager -f

# Check nslcd debug output
systemctl stop nslcd
nslcd -d # Run in foreground with debug output

# Test PAM authentication directly
pamtester common-auth testuser authenticate

# Inspect D-Bus LDAP objects
busctl tree xyz.openbmc_project.User.Manager
busctl introspect xyz.openbmc_project.User.Manager \
    /xyz/openbmc_project/user/ldap/config

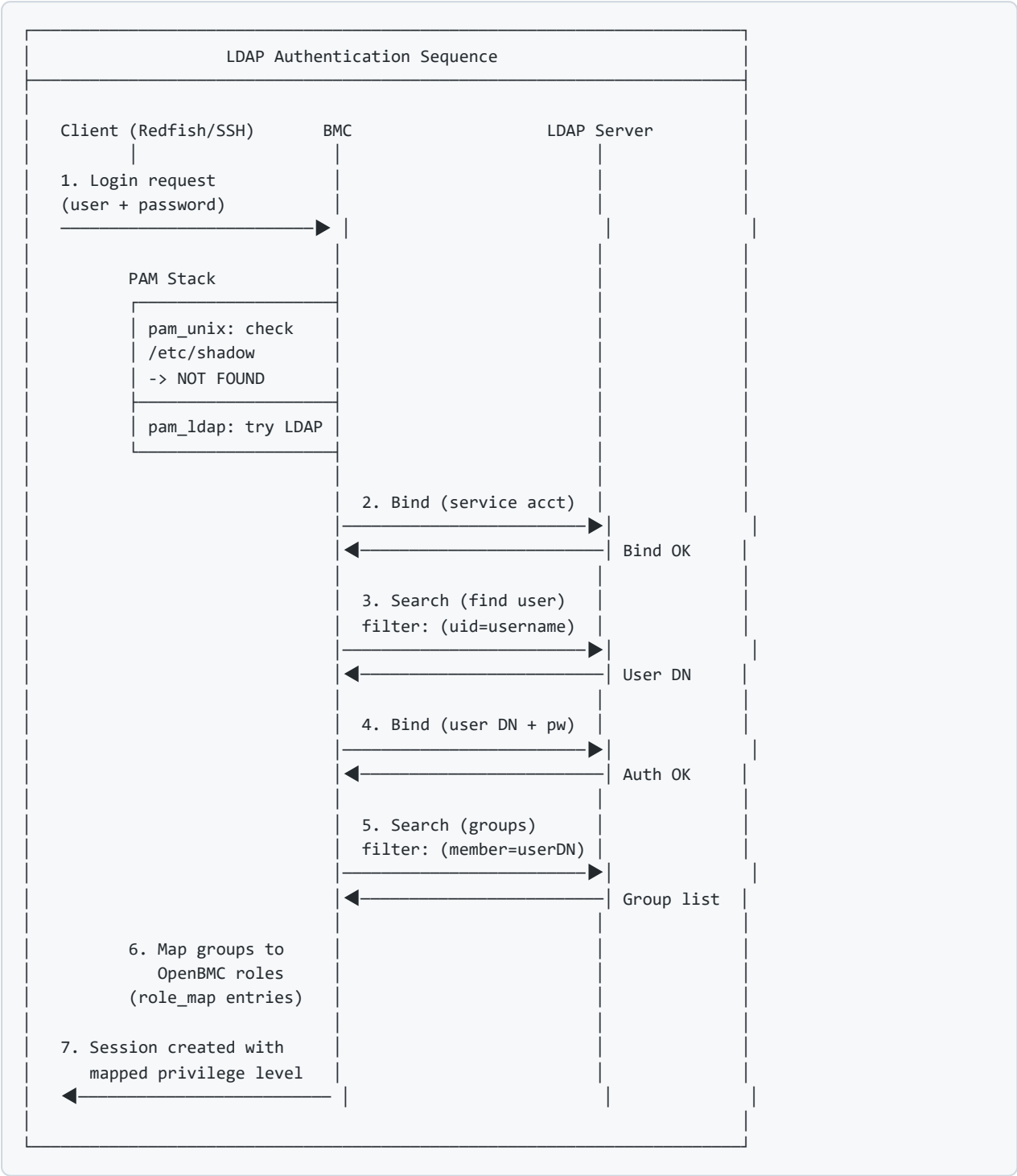
# List all role mapping entries
busctl tree xyz.openbmc_project.User.Manager | grep role_map
```

Running `nslcd -d` in the foreground shows detailed LDAP communication traces, including bind attempts, search filters, and TLS negotiation. This is the single most useful debugging tool for LDAP issues.

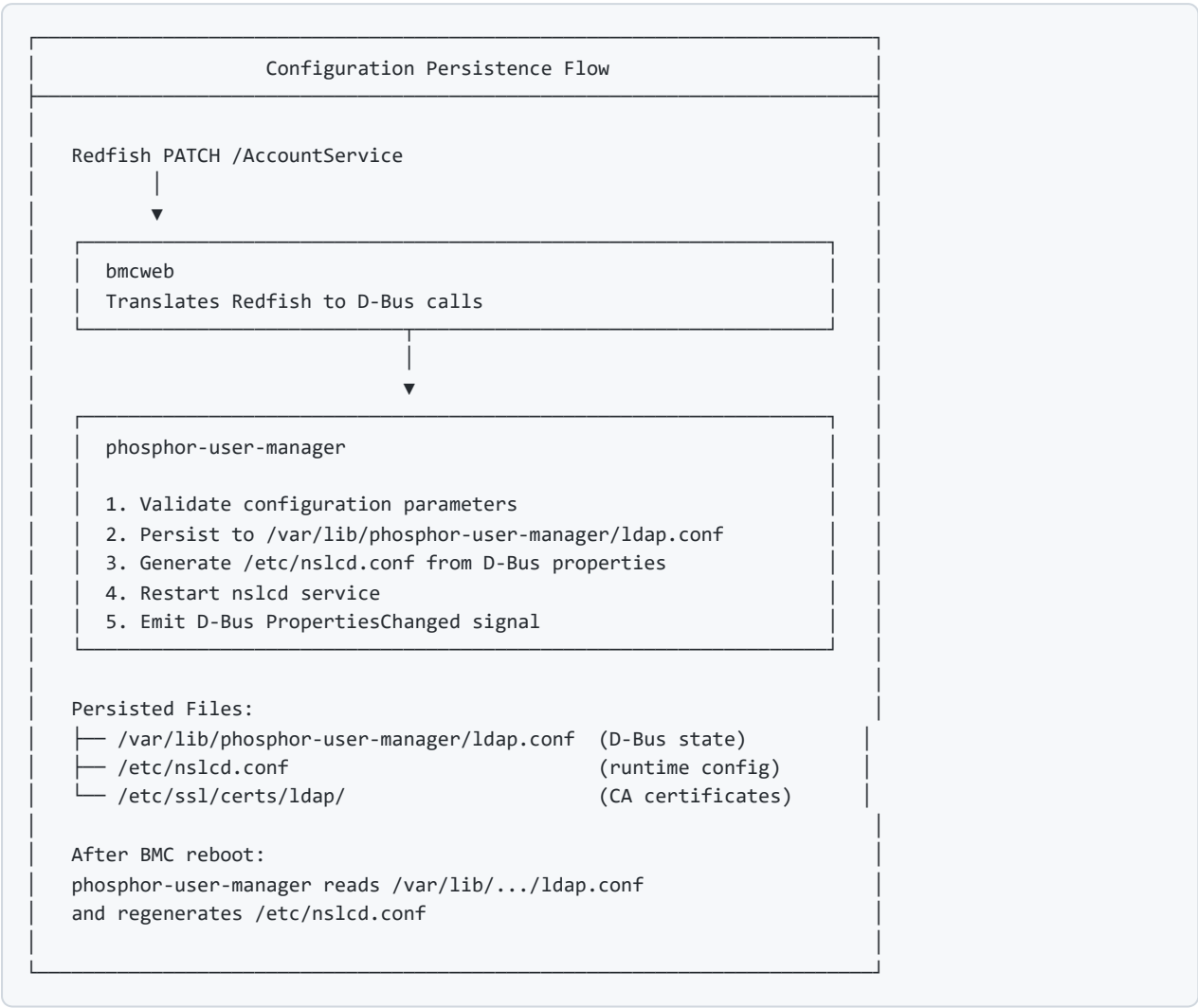
Deep Dive

Advanced implementation details for LDAP integration developers.

LDAP Authentication Flow



LDAP Configuration Persistence



Source Code Reference

Key implementation files in [phosphor-user-manager](#):

File	Description
user_mgr.cpp	Main user manager, orchestrates local and LDAP users
ldap_config.cpp	LDAP configuration object (server URI, bind DN, base DN)
ldap_config_mgr.cpp	LDAP config manager, handles create/delete of LDAP configs
ldap_mapper.cpp	Group-to-privilege mapping (PrivilegeMapper interface)
utils.cpp	Utility functions for nslcd config generation

Related bmcweb implementation:

File	Description
redfish-core/lib/account_service.hpp	Redfish AccountService handlers for LDAP/AD config

References

Official Resources

- [phosphor-user-manager](#)
- [phosphor-certificate-manager](#)
- [D-Bus Interface: User.Ldap](#)
- [OpenBMC User Management Design](#)

Related Guides

- [User Manager Guide](#)
- [Certificate Manager Guide](#)
- [Network Guide](#)

External Documentation

- [Redfish AccountService Schema](#)
- [nss-pam-ldapd \(nslcd\)](#)
- [OpenLDAP Documentation](#)

Tested on: OpenBMC master, QEMU ast2600-evb

PID Thermal Tuning

Tune PID coefficients for optimal thermal control using logging, step response analysis, and systematic coefficient adjustment.

Table of Contents

1. [Overview](#)
2. [PID Controller Theory](#)
 1. [The Control Loop](#)
 2. [The Three PID Terms](#)
 3. [Why Negative Ki?](#)
3. [Zone Configuration](#)
 1. [PID Controller Parameters](#)
 2. [Output Processing Pipeline](#)
4. [Tuning Methodology](#)
 1. [Step 1: Enable CSV Logging](#)
 2. [Step 2: Establish Baseline](#)
 3. [Step 3: Step Response Test](#)
 4. [Step 4: Set Initial Coefficients](#)
 5. [Step 5: Iterative Adjustment](#)
 6. [Step 6: Add Slew Rates](#)
 7. [Step 7: Validate Under Load Scenarios](#)
5. [Stepwise vs PID Comparison](#)
 1. [Stepwise Control](#)
 2. [Comparison Table](#)
6. [Failsafe Behavior](#)
 1. [What Triggers Failsafe](#)
 2. [Verifying Failsafe](#)
 3. [Failsafe and Tuning Interaction](#)
7. [Practical Tuning Example](#)
8. [Troubleshooting](#)
 1. [Issue: Fans Stuck at Maximum Speed](#)
 2. [Issue: Temperature Oscillates Around Setpoint](#)
 3. [Issue: Fans Do Not Respond to Temperature Changes](#)
 4. [Debug Commands](#)
9. [Code Examples](#)
10. [References](#)
 1. [Official Resources](#)
 2. [Related Guides](#)
 3. [External Documentation](#)

Overview

The [Fan Control Guide](#) covers the architecture and configuration of phosphor-pid-control. This guide focuses on the practical process of **tuning** PID coefficients so that fan speeds respond correctly to temperature changes on your specific hardware.

Poorly tuned PID loops cause real problems: fans that oscillate audibly, temperatures that overshoot into throttling range, or fans stuck at maximum speed wasting power and generating noise. Tuning is the process of finding coefficient values that balance responsiveness against stability for your particular thermal environment.

Key concepts covered:

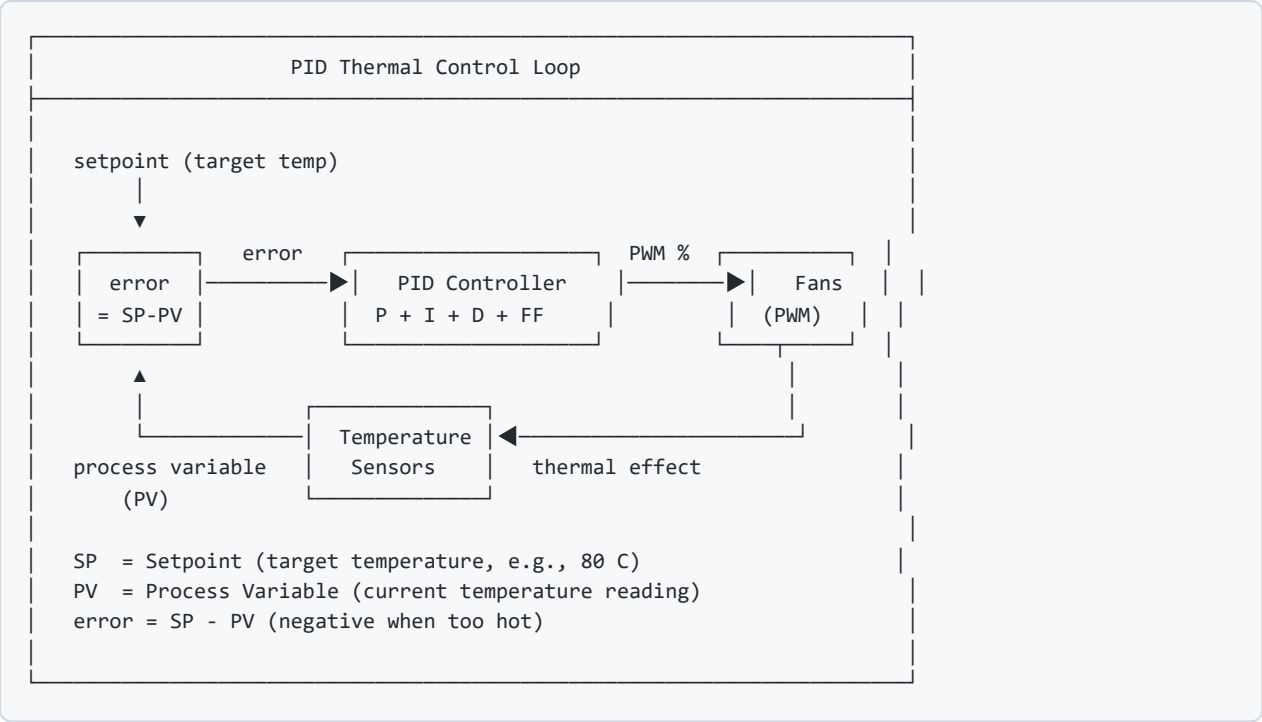
- PID controller theory (Kp, Ki, Kd) and how each term affects fan behavior
- Zone configuration parameters that interact with tuning
- CSV logging with the `-l` flag for data-driven tuning
- Step response methodology for systematic coefficient adjustment
- When to use stepwise control instead of PID
- Failsafe behavior and how to verify it

PID tuning must be performed on actual hardware or a representative thermal simulation. QEMU does not model thermal dynamics, so while you can verify configuration syntax and service startup in QEMU, the tuning values themselves must come from real hardware testing.

PID Controller Theory

The Control Loop

phosphor-pid-control implements a discrete-time PID controller. Each sample period, the daemon reads sensor values, computes an error signal, and adjusts fan PWM output.



The Three PID Terms

Each term in the PID equation contributes a different aspect of control behavior:

Output = $K_p * error + K_i * \text{integral}(error) + K_d * \frac{d(error)}{dt} + FF$

Term	Coefficient	Effect	Typical OpenBMC Use
Proportional (P)	<code>proportionalCoeff</code>	Immediate response proportional to error magnitude	Often set to 0.0 for thermal
Integral (I)	<code>integralCoeff</code>	Accumulated response that eliminates steady-state error	Primary tuning parameter (negative for cooling)
Derivative (D)	<code>feedFwdOffsetCoeff</code>	Responds to rate of change, dampens oscillation	Rarely used in thermal control
Feed-forward (FF)	<code>feedFwdGainCoeff</code>	Open-loop term based on setpoint, not error	Used in fan-type PIDs (typically 1.0)

In most OpenBMC thermal configurations, the integral term alone provides sufficient control. Start with integral-only control and add proportional gain only if you need faster transient response.

Why Negative Ki?

The sign convention in phosphor-pid-control requires a negative integral coefficient for cooling:

Scenario: CPU at 85 C, setpoint = 80 C (too hot)

```
error = setpoint - temperature = 80 - 85 = -5
Ki = -0.2 (negative)
integral += Ki * error * dt = (-0.2) * (-5) * (1.0) = +1.0
Result: PWM output INCREASES --> correct
```

Scenario: CPU at 75 C, setpoint = 80 C (cool enough)

```
error = 80 - 75 = +5
integral += (-0.2) * (+5) * (1.0) = -1.0
Result: PWM output DECREASES --> correct
```

Zone Configuration

Each zone groups sensors and fans into a thermal control region. The zone parameters directly affect tuning behavior.

```
{
  "zones": [
    {
      "id": 0,
      "minThermalOutput": 25.0,
      "failsafePercent": 100.0
    }
  ]
}
```

Parameter	Description	Tuning Impact
<code>minThermalOutput</code>	Floor PWM % when zone is active	Sets the minimum fan speed; prevents integral windup below this value
<code>failsafePercent</code>	PWM % when a sensor is missing or timed out	Should be 100.0 for safety; overrides all PID output

The sensor `timeout` field is critical for tuning. If a sensor does not update within `timeout` seconds, the zone enters failsafe. During tuning, if you see unexpected failsafe events, increase `timeout` or check that the sensor daemon is publishing values at the expected rate.

PID Controller Parameters

The full set of tunable parameters for each PID controller:

```
{
  "name": "CPU_Temp_Controller",
  "type": "temp",
  "inputs": ["CPU_Temp"],
  "setpoint": 80.0,
  "pid": {
    "samplePeriod": 1.0,
    "proportionalCoeff": 0.0,
    "integralCoeff": -0.2,
    "feedFwdOffsetCoeff": 0.0,
    "feedFwdGainCoeff": 0.0,
    "integrallimit_min": 0.0,
    "integrallimit_max": 100.0,
    "outLim_min": 25.0,
    "outLim_max": 100.0,
    "slewNeg": 5.0,
    "slewPos": 10.0
  }
}
```

Parameter	Range	Description
<code>samplePeriod</code>	0.1 - 5.0 s	Control loop interval; shorter = faster response, more CPU
<code>proportionalCoeff</code>	0.0 - 2.0	Proportional gain; usually 0.0 for thermal PIDs
<code>integralCoeff</code>	-1.0 to -0.01	Integral gain; primary tuning knob (negative for cooling)
<code>integrallimit_min</code>	0.0	Lower clamp on integral accumulator (anti-windup)
<code>integrallimit_max</code>	0.0 - 100.0	Upper clamp on integral accumulator (anti-windup)
<code>outLim_min</code>	0.0 - 100.0	Minimum PID output (PWM %)
<code>outLim_max</code>	0.0 - 100.0	Maximum PID output (PWM %)
<code>slewNeg</code>	0.0 - 20.0	Max PWM decrease per second (0 = unlimited)
<code>slewPos</code>	0.0 - 20.0	Max PWM increase per second (0 = unlimited)

Output Processing Pipeline

After the PID calculation, output goes through several processing stages:

1. **Clamp** to `[outLim_min, outLim_max]` – limits the raw PID output range
2. **Slew rate** – limits the rate of change per second (`slewPos` for increases, `slewNeg` for decreases)
3. **Zone arbitration** – `MAX()` of all PID outputs in the zone (highest demand wins)
4. **Floor clamp** – ensures output is at least `minThermalOutput`

Tuning Methodology

Step 1: Enable CSV Logging

phosphor-pid-control supports CSV logging with the `-l` flag. This captures timestamped sensor readings and PID outputs for offline analysis.


```
# Stop the running service
systemctl stop phosphor-pid-control

# Run manually with CSV logging enabled
/usr/bin/swampd -l /tmp/pid-log.csv
```

The CSV log contains columns for each sample period:

```
timestamp,zone,sensor_name,sensor_value,setpoint,error,p_term,i_term,d_term,ff_term,output
1706140800,0,CPU_Temp,82.5,80.0,-2.5,0.0,45.2,0.0,0.0,45.2
1706140801,0,CPU_Temp,82.1,80.0,-2.1,0.0,45.6,0.0,0.0,45.6
```

Transfer the CSV file to your workstation for analysis. Tools like Python with matplotlib, gnuplot, or a spreadsheet program let you visualize temperature and PWM trends over time.

Step 2: Establish Baseline

Before tuning PID coefficients, characterize your thermal plant by measuring steady-state temperatures at fixed fan speeds.

```
# Switch to manual mode
busctl set-property xyz.openbmc_project.State.FanCtrl \
    /xyz/openbmc_project/control/thermal/0 \
    xyz.openbmc_project.Control.ThermalMode Current s "Manual"

# Test at several fixed PWM levels, waiting for thermal equilibrium
for pwm in 25 40 60 80 100; do
    echo "=== Testing at ${pwm}% PWM ==="
    busctl set-property xyz.openbmc_project.FanSensor \
        /xyz/openbmc_project/control/fanpwm/Fan0 \
        xyz.openbmc_project.Control.FanPwm Target t $pwm
    echo "Waiting 10 minutes for stabilization..."
    sleep 600
    busctl get-property xyz.openbmc_project.HwmonTempSensor \
        /xyz/openbmc_project/sensors/temperature/CPU_Temp \
        xyz.openbmc_project.Sensor.Value Value
done
```

Record the results in a table:

PWM %	CPU Temp (C)	Inlet Temp (C)	Fan RPM
25	92	38	2100
40	85	35	3500
60	78	32	5200
80	73	29	6800
100	69	27	8500

This baseline tells you what PWM range affects temperature, whether your setpoint is achievable, and the approximate plant gain (degrees per PWM %).

Step 3: Step Response Test

Apply a sudden change in fan speed and observe how temperature responds. This reveals the thermal time constant of your system.

```
# Start at 30% PWM, wait for equilibrium, then step to 80%
# Log temperature every 5 seconds for 20 minutes
for i in $(seq 1 240); do
    TEMP=$(busctl get-property xyz.openbmc_project.HwmonTempSensor \
        /xyz/openbmc_project/sensors/temperature/CPU_Temp \
        xyz.openbmc_project.Sensor.Value Value 2>/dev/null | awk '{print $2}')
    echo "$(date +%s),$TEMP" >> /tmp/step-response.csv
    if [ "$i" -eq 60 ]; then
        busctl set-property xyz.openbmc_project.FanSensor \
            /xyz/openbmc_project/control/fanpwm/Fan0 \
            xyz.openbmc_project.Control.FanPwm Target t 80
    fi
    sleep 5
done
```

From the step response data, identify the **time constant (τ)**: the time for temperature to reach 63% of its total change. Use τ to calculate an initial K_i :

```
Ki_initial = -1.0 / (2 * tau)
Example: tau = 120s --> Ki = -1.0 / 240 = -0.004
```

Step 4: Set Initial Coefficients

Configure conservative initial PID values based on the step response:

```
{
  "samplePeriod": 1.0,
  "proportionalCoeff": 0.0,
  "integralCoeff": -0.05,
  "feedFwdOffsetCoeff": 0.0,
  "feedFwdGainCoeff": 0.0,
  "integrallimit_min": 0.0,
  "integrallimit_max": 100.0,
  "outLim_min": 25.0,
  "outLim_max": 100.0,
  "slewNeg": 0.0,
  "slewPos": 0.0
}
```

Start with slew rates at 0.0 (unlimited) during initial tuning. Add slew rate limits after the basic PID response is satisfactory, as slew limits mask PID behavior and make it harder to diagnose coefficient problems.

Step 5: Iterative Adjustment

Run phosphor-pid-control with CSV logging and apply a thermal load. Observe the logged data and adjust coefficients.

```
systemctl stop phosphor-pid-control
/usr/bin/swampd -l /tmp/tuning-round1.csv
```

Diagnosing common problems from CSV data:

Problem	CSV Clue	Fix
Response too slow (temperature overshoots setpoint)	<code>error</code> stays large for many samples	Increase <code> Ki </code> (e.g., -0.05 to -0.1)
Oscillation (fans hunt up and down)	<code>output</code> alternates high/low periodically	Decrease <code> Ki </code> (e.g., -0.2 to -0.1); add slew limits
Steady-state offset (never reaches setpoint)	<code>error</code> stabilizes at non-zero value	Check <code>outLim_max</code> ; increase <code> Ki </code> slightly
Fans jump to max then slowly decrease	<code>p_term</code> has large values	Reduce <code>Kp</code> or set to 0.0
Integral windup (fans stay at max after load drops)	<code>i_term</code> stays at <code>integrallimit_max</code>	Reduce <code>integrallimit_max</code> or <code> Ki </code>

Step 6: Add Slew Rates

Once the PID coefficients produce stable control, add slew rate limits to smooth fan speed transitions:

```
{
  "slewPos": 10.0,
  "slewNeg": 5.0
}
```

- `slewPos` : Max PWM increase per second (10.0 = fans can ramp up 10%/s)
- `slewNeg` : Max PWM decrease per second (asymmetric for slow ramp-down)

Setting `slewPos` too low can prevent fans from responding quickly enough to thermal spikes. Verify that slew-limited ramp-up is fast enough to prevent temperature from reaching critical thresholds under worst-case load transients.

Step 7: Validate Under Load Scenarios

Test the tuned configuration across representative workloads:

- Cold boot:** Host powers on from cold state, temperature ramps up
- Idle to full load:** Apply CPU stress test, observe response
- Full load to idle:** Remove workload, observe fan speed reduction
- Sustained load:** Run full load for 30+ minutes, verify no drift
- Sensor failure:** Disconnect a sensor, verify failsafe activates

Stepwise vs PID Comparison

phosphor-pid-control supports two control strategies. Choose based on your requirements.

Stepwise Control

Stepwise uses a static lookup table to map temperature to PWM percentage:

```
{
  "name": "Inlet_Stepwise",
  "type": "stepwise",
  "inputs": ["Inlet_Temp"],
  "reading": {
    "positiveHysteresis": 2.0,
    "negativeHysteresis": 2.0
  },
  "output": [
    { "temp": 0, "pwm": 20 },
    { "temp": 25, "pwm": 35 },
    { "temp": 30, "pwm": 50 },
    { "temp": 35, "pwm": 65 },
    { "temp": 40, "pwm": 80 },
    { "temp": 45, "pwm": 100 }
  ]
}
```

Comparison Table

Characteristic	PID Control	Stepwise Control
Tuning effort	High (step response, iteration)	Low (define temperature-PWM table)
Precision	Maintains setpoint accurately	Steps between fixed PWM levels
Noise behavior	Smooth fan speed transitions	Discrete jumps (mitigated by hysteresis)
Transient response	Adapts dynamically to load changes	Fixed mapping, no adaptation
CPU overhead	Slightly more (PID math per sample)	Minimal (table lookup)
Best for	CPU/GPU thermal (tight control)	Inlet/ambient (coarse control)

Use PID when you need to maintain a specific temperature setpoint, the thermal load is dynamic, or you need smooth continuous fan adjustment.

Use stepwise when simple temperature-to-fan mapping is sufficient, you want deterministic behavior with no tuning, or the temperature range is well-characterized and stable.

You can combine both strategies in a single zone. Use PID for CPU temperature and stepwise for inlet temperature, both in the same zone. Zone arbitration takes the MAX output, so the more aggressive demand always wins.

Failsafe Behavior

What Triggers Failsafe

When a zone enters failsafe mode, all fans in that zone are driven to `failsafePercent` (typically 100%). Failsafe triggers when:

1. **Sensor timeout** – a configured input sensor has not updated within its `timeout` period
2. **Sensor missing** – a required sensor D-Bus object does not exist (daemon not started or crashed)
3. **Sensor value NaN** – the sensor reports NaN, indicating a hardware read failure

Recovery is automatic when all sensors return valid readings for a sustained period.

Verifying Failsafe

Test failsafe behavior before deploying to production:

```
# Stop a sensor daemon to simulate sensor loss
systemctl stop xyz.openbmc_project.hwmontempsensor

# Confirm fans go to failsafe
journalctl -u phosphor-pid-control -f | grep -i failsafe

# Restart sensor daemon, verify recovery
systemctl start xyz.openbmc_project.hwmontempsensor
```

Never set `failsafePercent` below 100.0 on production systems. The failsafe condition means the system cannot determine actual temperatures, so maximum cooling is the only safe response.

Failsafe and Tuning Interaction

During PID tuning, unexpected failsafe events are a common source of confusion. If your tuning session is interrupted by failsafe:

1. Check `journalctl -u phosphor-pid-control` for the specific reason
2. Verify all sensor daemons are running: `systemctl list-units | grep sensor`
3. Verify sensor `timeout` values are larger than the sensor polling interval
4. If using `-l` logging, check whether the sensor value column shows NaN entries

Practical Tuning Example

This walkthrough illustrates the iterative tuning process for a single-zone configuration.

Round 1 – conservative $K_i = -0.05$:

```
systemctl stop phosphor-pid-control
/usr/bin/swampd -l /tmp/round1.csv
# Apply load, wait 15 minutes, observe CSV
```

Observation: Temperature rises to 88 C before fans respond. Integral accumulates too slowly.

Round 2 – increase K_i to -0.15 :

Update `integralCoeff` to `-0.15`, restart. Temperature peaks at 83 C, settles to 80.5 C. No oscillation.

Round 3 – add slew rates (`slewPos: 10.0`, `slewNeg: 5.0`):

Fan transitions become smooth. Temperature peaks at 84 C due to slew limiting the ramp-up, but settles within acceptable range.

Final validated coefficients:

```
{
  "samplePeriod": 1.0,
  "proportionalCoeff": 0.0,
  "integralCoeff": -0.15,
  "integrallimit_min": 0.0,
  "integrallimit_max": 100.0,
  "outLim_min": 25.0,
  "outLim_max": 100.0,
  "slewNeg": 5.0,
  "slewPos": 10.0
}
```

Troubleshooting

Issue: Fans Stuck at Maximum Speed

Symptom: All fans run at 100% even though temperatures are normal.

Cause: Zone is in failsafe due to a missing or timed-out sensor.

Solution:

1. Check the journal: `journalctl -u phosphor-pid-control | grep -i failsafe`
2. Verify sensors on D-Bus: `busctl tree xyz.openbmc_project.HwmonTempSensor`
3. Check sensor daemon: `systemctl status xyz.openbmc_project.hwmontempsensor`

Issue: Temperature Oscillates Around Setpoint

Symptom: Temperature swings 5-10 C above and below setpoint periodically.

Cause: Integral gain too aggressive, or output limits too wide.

Solution:

1. Reduce `|integralCoeff|` by half (e.g., -0.2 to -0.1)
2. Add or tighten slew rate limits
3. Reduce `integrallimit_max` to prevent windup
4. Enable CSV logging to measure oscillation period and amplitude

Issue: Fans Do Not Respond to Temperature Changes

Symptom: Fan speed remains constant despite rising temperature.

Cause: Configuration mismatch between sensor names in PID config and actual D-Bus sensor names.

Solution:

1. Verify the sensor name in `inputs` matches the sensor config `name` field
2. Check that thermal PID `type` is `"temp"` and fan PID `type` is `"fan"`
3. Verify the zone ID in the fan config matches the zone used by the thermal PID
4. Run in debug mode: `/usr/bin/swampd -d`

Debug Commands

```
# Check service status and recent logs
systemctl status phosphor-pid-control
journalctl -u phosphor-pid-control -n 50

# Verify zone control mode
busctl get-property xyz.openbmc_project.State.FanCtrl \
    /xyz/openbmc_project/control/thermal/0 \
    xyz.openbmc_project.Control.ThermalMode Current

# Read current sensor values
busctl get-property xyz.openbmc_project.HwmonTempSensor \
    /xyz/openbmc_project/sensors/temperature/CPU_Temp \
    xyz.openbmc_project.Sensor.Value Value

# Run daemon with debug output
/usr/bin/swampd -d

# Run daemon with CSV logging
/usr/bin/swampd -l /tmp/pid-debug.csv
```

Code Examples

Working example configurations are available in the [examples/pid-tuning/](#) directory:

- [tuning-baseline-config.json](#) – Conservative starting configuration for a single zone
- [tuning-final-config.json](#) – Tuned configuration after iterative adjustment
- [stepwise-inlet-config.json](#) – Stepwise configuration for inlet temperature
- [multi-sensor-zone-config.json](#) – Multi-sensor zone with PID and stepwise combined

Also see the [examples/fan-control/](#) directory for complete zone and entity manager configurations.

References

Official Resources

- [phosphor-pid-control Repository](#)
- [phosphor-pid-control README](#)
- [PID Algorithm Implementation](#)
- [Zone Management Source](#)

Related Guides

- [Fan Control Guide](#)
- [D-Bus Sensors Guide](#)
- [Entity Manager Guide](#)

External Documentation

- [PID Controller Theory \(Wikipedia\)](#)

- [Ziegler-Nichols Tuning Method](#)
-

Tested on: OpenBMC master, QEMU romulus (configuration syntax only; tuning values require real hardware) Last updated: 2026-02-06

Intrusion Detection Guide

Wire, configure, and verify chassis intrusion, BMC tamper input, and battery-removal detection end-to-end on OpenBMC.

Table of Contents

1. [Overview](#)
2. [Architecture](#)
 1. [Primary Data Flow](#)
 2. [Companion Reporter Event Paths](#)
 3. [D-Bus Interfaces](#)
 4. [Key Dependencies](#)
3. [Chassis Intrusion Switch](#)
 1. [Wiring](#)
 2. [Failure Mode](#)
 3. [Device Tree](#)
 4. [Entity Manager Configuration](#)
 5. [Verify on D-Bus](#)
4. [BMC Detection Port \(AST2600 Primary\)](#)
 1. [Example Triplet Used Throughout This Guide](#)
 2. [Full AST2600 DT Fragment](#)
 3. [Verify on the Running Kernel](#)
5. [Porting to AST2700 and Nuvoton NPCM](#)
 1. [SoC Comparison](#)
 2. [AST2700 Device Tree Fragment](#)
 3. [Nuvoton NPCM7xx \(Poleg\) Device Tree Fragment](#)
 4. [Nuvoton NPCM8xx \(Arbel\) Device Tree Fragment](#)
 5. [What Stays the Same](#)
6. [Battery-Backed Intrusion Latch](#)
 1. [Reference Schematic](#)
 2. [Bill of Materials](#)
 3. [Read-and-Clear Boot Sequence](#)
7. [Battery-Removal Detection](#)
 1. [Live Comparator \(Runtime Only\)](#)
 2. [VBAT-SRAM Token \(Across Power-Off\)](#)
 1. [SoC nvmem Exposure](#)
 2. [Protocol](#)
 3. [Keys and Files](#)
 4. [Expected Post-Power-Loss Patterns](#)
 5. [Replay Resistance](#)

8. [Companion Reporter Service](#)
 1. [Meson Build Fragment](#)
 2. [Bitbake Recipe](#)
 3. [Systemd Unit](#)
 4. [Entity Manager Schema](#) — `BatteryPresenceSensor`
 9. [Redfish Mapping](#)
 1. `PhysicalSecurity.IntrusionSensor`
 2. [Battery-Removal](#) → [LogEntry](#)
 10. [IPMI OEM SEL](#)
 1. [OEM Sensor Type](#)
 2. [Decoded View](#)
 11. [QEMU Verification Walkthrough](#)
 1. [Kernel Config Check](#)
 2. [Setup](#)
 3. [1. Verify Live Intrusion](#)
 4. [2. Verify Latched Intrusion](#)
 5. [3. Verify Live Battery Removal](#)
 6. [4. Verify Across-Power-Off Battery Removal](#)
 7. [5. Verify Token Rotation](#)
 8. `gpio-mockup` [Fallback](#)
 9. [Commit Hash Used for Verification](#)
 12. [Troubleshooting](#)
 1. [1. Chassis intrusion signal stuck asserted](#)
 2. [2. Latched state not cleared across reboots](#)
 3. [3. Comparator false-positive on slow power ramps](#)
 4. [4. `phosphor-intrusion-sensor` code drift from the example snippet](#)
 5. [5. VBAT nvmem device not present under `/sys/bus/nvmem/`](#)
 6. [6. Persistent false “battery removed”](#)
 7. [Debug Commands](#)
 13. [References](#)
 1. [Official Resources](#)
 2. [Related Guides](#)
 3. [SoC References](#)
-

Overview

Physical-security detection is a standard requirement on server-class hardware, but OpenBMC’s upstream components only cover the live-switch case. This guide covers the full stack — hardware, kernel, D-Bus, Redfish, and IPMI — across three surfaces that a platform port needs to get right:

1. **Chassis intrusion switch** — the mechanical normally-closed switch on the chassis, wired to a BMC GPIO and monitored by `phosphor-intrusion-sensor`.

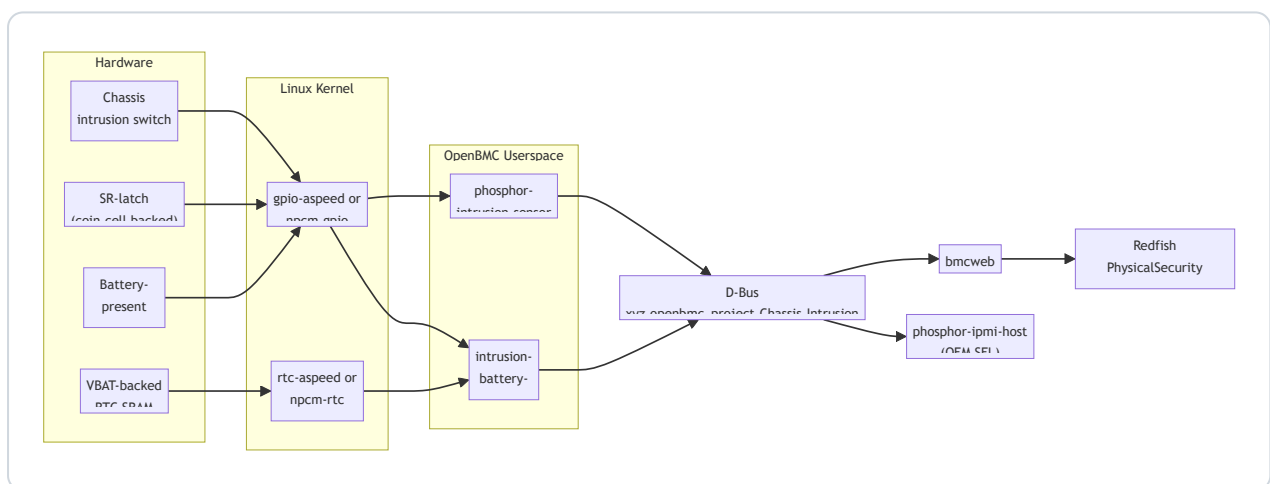
2. **BMC detection port** — the dedicated tamper/intrusion GPIOs on the BMC SoC, including pinmux, pull configuration, and line-name conventions. Covered for Aspeed AST2600 (primary, QEMU-verified) and with porting tables for AST2700 and Nuvoton NPCM7xx/NPCM8xx.
3. **Hardware with a backup battery** — a coin-cell-backed latch that captures intrusion even while the system is unpowered, plus two complementary battery-removal detectors covering both runtime and power-off threat windows.

Key concepts covered:

- NC / active-low GPIO wiring and why the failure mode matters
- `phosphor-intrusion-sensor` + Entity Manager configuration
- SR-latch read-and-clear boot sequence
- VBAT-backed SRAM token protocol (authoritative power-off detection)
- Redfish `PhysicalSecurity.IntrusionSensor` and OEM IPMI SEL mapping
- QEMU `gpio-mockup` verification of every event path

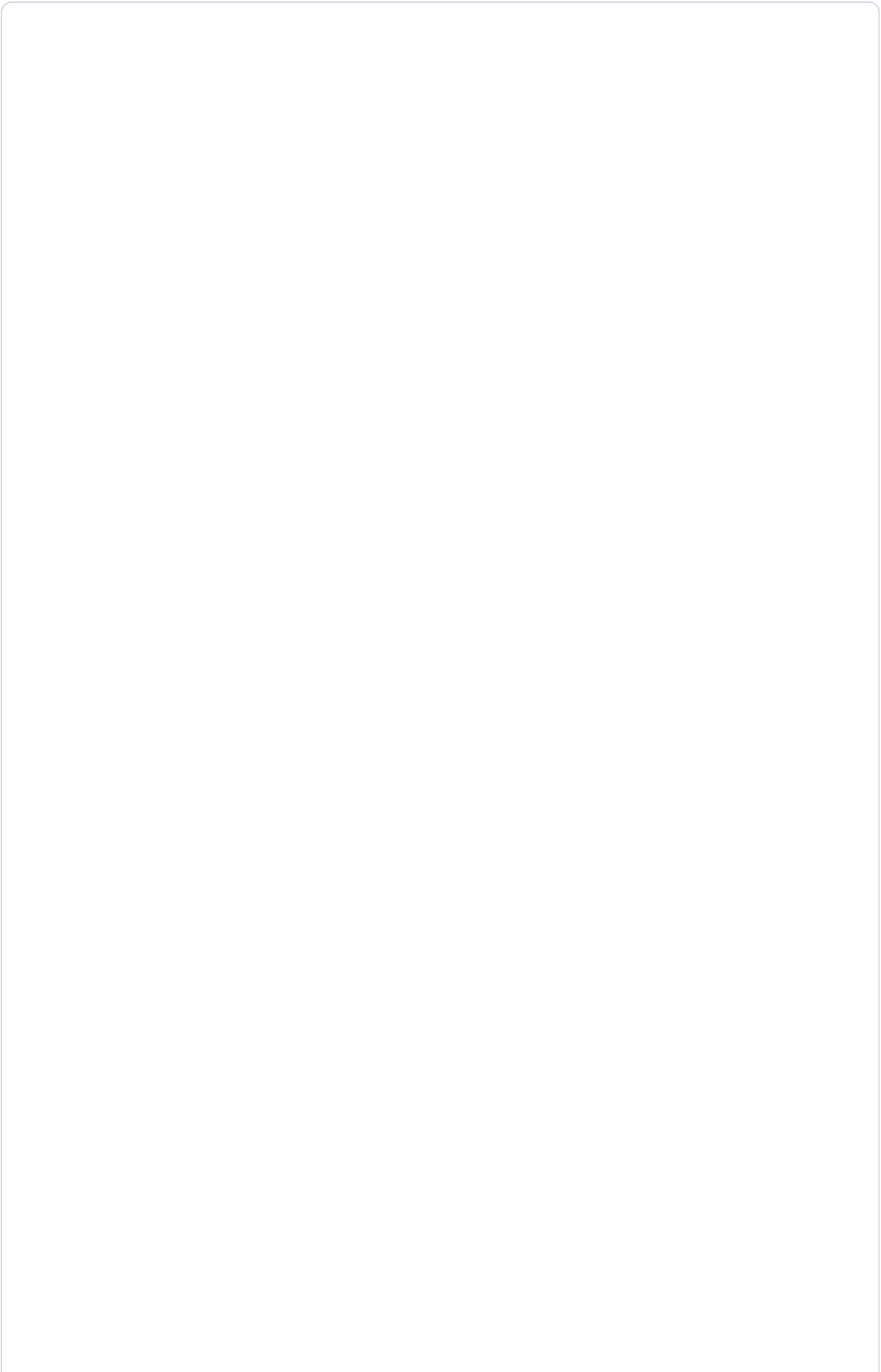
Architecture

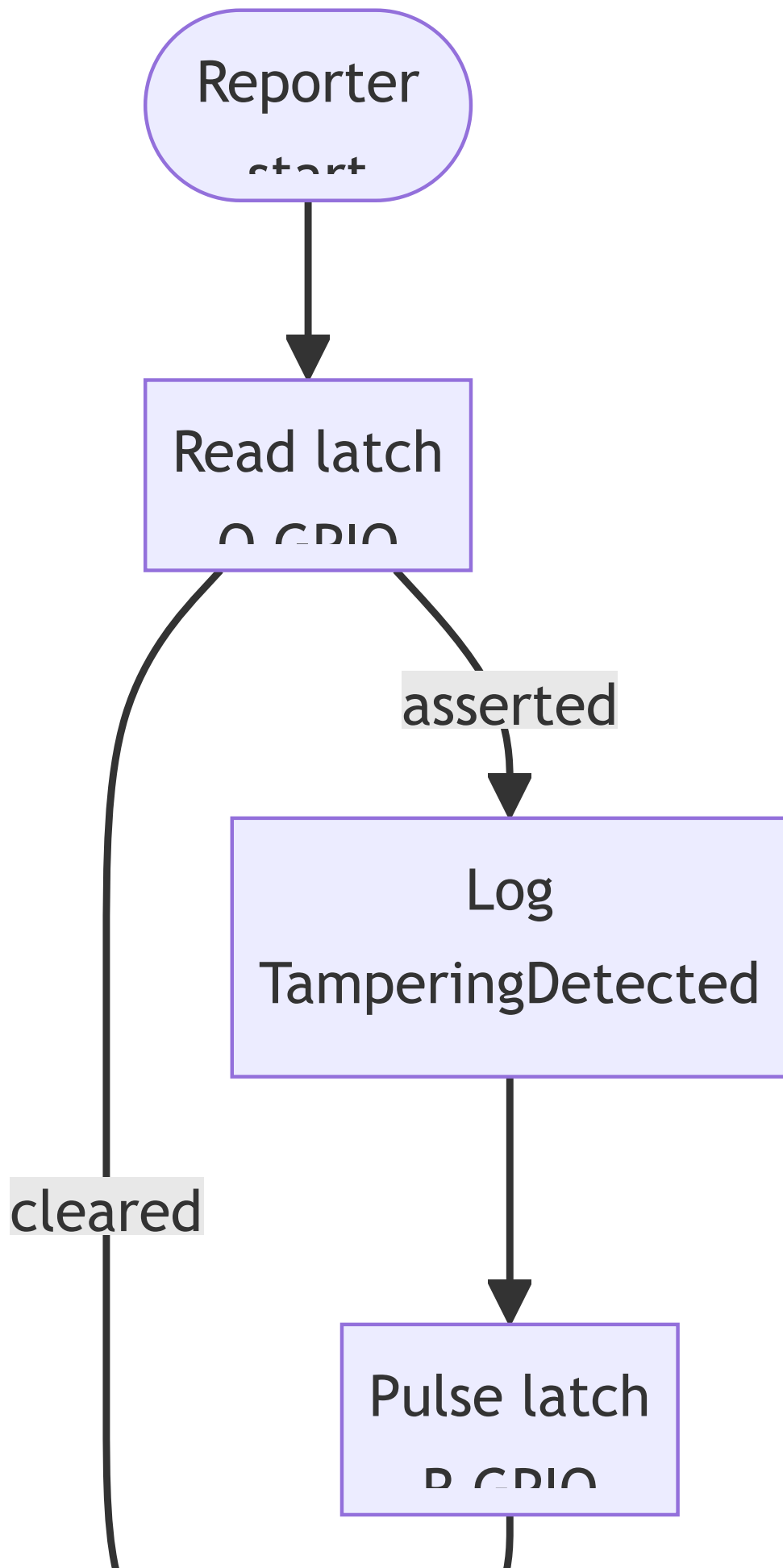
Primary Data Flow

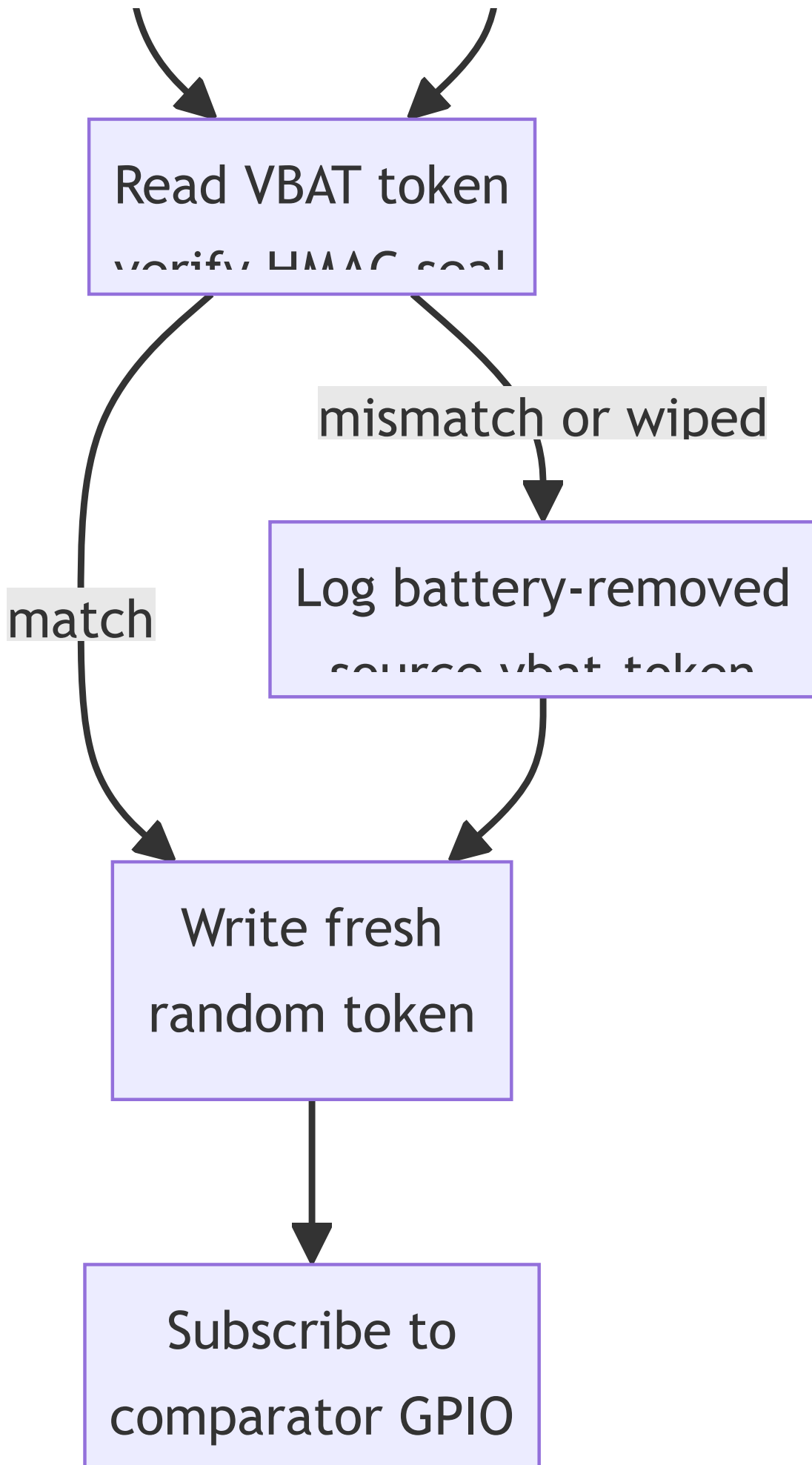


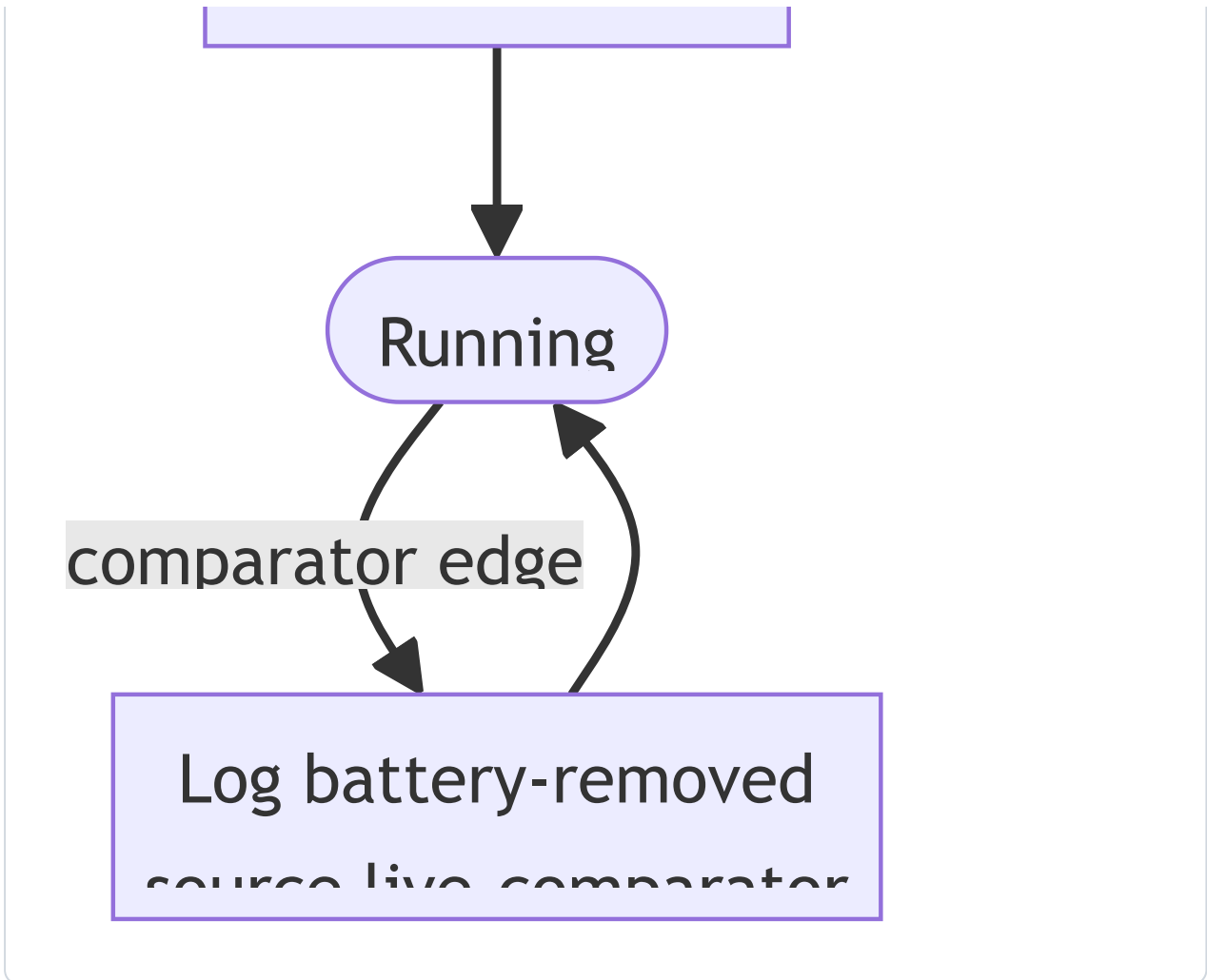
Companion Reporter Event Paths

The companion reporter (`intrusion-battery-reporter`) is a small service this guide introduces to cover everything above the scope of upstream `phosphor-intrusion-sensor` :









The upstream `intrusionsensor` daemon does more than watch the chassis switch. It also monitors **LAN link state**: it subscribes to `PropertiesChanged` on `org.freedesktop.network1.Link` and logs a Redfish event whenever an interface's `OperationalState` changes. And besides the GPIO and hwmon (clear-on-read) intrusion paths this guide covers, it supports a **PCH register variant** (`ChassisIntrusionPchSensor`) that reads intrusion status over I2C using a status-bit mask. (Verify against your `dbus-sensors` version.)

D-Bus Interfaces

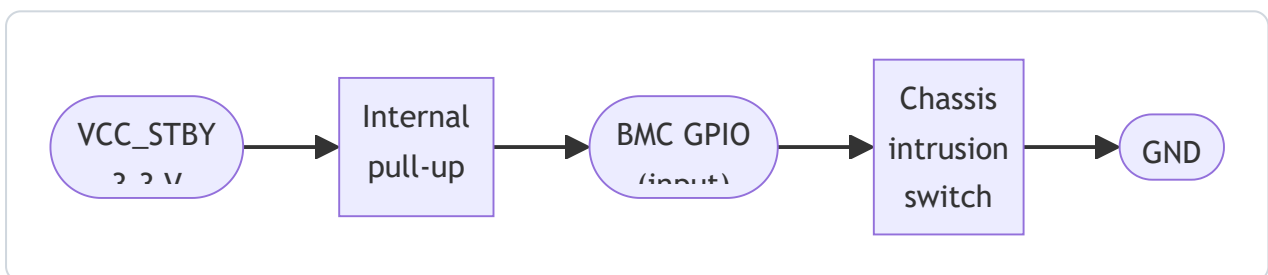
Interface	Object Path	Source	Description
<code>xyz.openbmc_project.Chassis.Intrusion</code>	<code>/xyz/openbmc_project/Intrusion/Chassis_Intrusion</code>	Upstream (<code>dbus-sensors</code>)	Live chassis switch state
<code>xyz.openbmc_project.Chassis.Intrusion</code>	<code>/xyz/openbmc_project/Intrusion/Chassis_Latch</code>	This guide	Latched intrusion from previous power cycle
<code>xyz.openbmc_project.Chassis.Intrusion</code>	<code>/xyz/openbmc_project/Intrusion/Battery_Present</code>	This guide	Coin-cell presence (merged live + power-off)
<code>xyz.openbmc_project.Logging.Entry</code>	<code>/xyz/openbmc_project/logging/entry/*</code>	Upstream	Per-event Redfish log entries

Key Dependencies

- `phosphor-intrusion-sensor` (in `dbus-sensors`): Upstream daemon that owns the live chassis intrusion D-Bus object.
- `entity-manager`: Declares the intrusion and battery-presence sensors to the daemon and the reporter.
- `bmcweb`: Maps `Chassis.Intrusion` to Redfish `PhysicalSecurity.IntrusionSensor`.
- `rtc-aspeed` / **Nuvoton RTC driver**: Exposes the SoC's VBAT-backed SRAM as an nvmem device under `/sys/bus/nvmem/devices/`.

Chassis Intrusion Switch

Wiring



Convention: the switch is normally-closed (NC) — closed when the chassis is intact, open when the chassis is opened. Because the BMC's internal pull-up is enabled, the GPIO reads **low** when the chassis is closed (switch shorts the input to ground) and **high** when the chassis is opened (switch releases and the pull-up drives the input high). This is the “active-low, invert-in-software” convention that `phosphor-intrusion-sensor` already handles via its `GpioInverted` flag.

Failure Mode

The NC / active-low wiring was chosen deliberately. With this convention, any **open-circuit fault** (broken wire, corroded connector, cold solder) reads as *intrusion* — the safe failure mode for a security feature. A normally-open switch with an external pull-down would have the opposite failure mode: a broken wire would read as “chassis closed”, silently hiding real intrusion events.

Do not wire the switch as normally-open with an external pull-down. A broken wire or corroded contact will read as “chassis closed” and silently hide real intrusion events. Always use NC with the BMC's internal pull-up.

Device Tree

Add the three intrusion-related GPIO line names and a pinmux stanza to your machine DTS. For AST2600:


```

&gpio0 {
    gpio-line-names =
        /* ... existing 208 entries ... */
        "chassis-intrusion",      /* GPIO bank/offset – see Section 4 */
        "intrusion-latch-q",
        "battery-present";
};

&pinctrl {
    pinctrl_intrusion_default: intrusion_default {
        pins = "GPIO54", "GPIO55", "GPIO56";
        bias-pull-up;
        input-enable;
    };
};

```

Entity Manager Configuration

Add an `Exposes` entry of the upstream `GpioIntrusionSensor` type. The minimum assertion window of 5 seconds matches the upstream default debounce behavior for chassis intrusion.

```

{
    "Exposes": [
        {
            "Name": "Chassis_Intrusion",
            "Type": "GpioIntrusionSensor",
            "GpioInverted": true,
            "Polarity": "Low",
            "Pin": "chassis-intrusion",
            "MinAssertInterval": 5
        }
    ]
}

```

Verify on D-Bus

```

# busctl tree xyz.openbmc_project.IntrusionSensor
└─ /xyz
   └─ /xyz/openbmc_project
      └─ /xyz/openbmc_project/Intrusion
         └─ /xyz/openbmc_project/Intrusion/Chassis_Intrusion

# busctl introspect xyz.openbmc_project.IntrusionSensor \
    /xyz/openbmc_project/Intrusion/Chassis_Intrusion | grep Status
.Status                                property s "Normal" emits-change writable

```

BMC Detection Port (AST2600 Primary)

AST2600 exposes ~200 GPIOs split across banks A through AC. For intrusion inputs, pick pins that:

- Are free of pinmux conflicts with your platform's LPC / eSPI / I²C / SGPIO usage
- Are on GPIO controllers that support `bias-pull-up` (all general-purpose banks do)
- Have direct 3.3 V tolerance (GPIOH/I and similar are 1.8 V — avoid these for chassis wiring)

Example Triplet Used Throughout This Guide

Signal	GPIO	Kernel line-name
Chassis intrusion	GPIO54	chassis-intrusion
Latch Q (read)	GPIO55	intrusion-latch-q
Battery present (live comparator)	GPIO56	battery-present

A fourth GPIO drives the latch reset (**R**) line — this is an output from the BMC, not an input. The example uses [GPIO57](#) / [intrusion-latch-r](#).

Full AST2600 DT Fragment

```
&pinctrl {
    pinctrl_intrusion_default: intrusion_default {
        pins = "GPIO54", "GPIO55", "GPIO56", "GPIO57";
        bias-pull-up;
    };
};

&gpio0 {
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_intrusion_default>;

    gpio-line-names =
        /* GPIO50..53 = existing */
        "", "", "", "",
        "chassis-intrusion",
        "intrusion-latch-q",
        "battery-present",
        "intrusion-latch-r";
};

chosen-intrusion {
    compatible = "gpio-keys";
    chassis-intrusion {
        label = "chassis-intrusion";
        gpios = <&gpio0 ASPEED_GPIO(5, 4) GPIO_ACTIVE_LOW>;
        linux,code = <KEY_VENDOR>;
        linux,can-disable;
    };
};
```

Verify on the Running Kernel

```
# gpioinfo | grep -E "chassis-intrusion|intrusion-latch|battery-present"
  line 148: "chassis-intrusion"    input  active-low [used]
  line 149: "intrusion-latch-q"    input  active-high [used]
  line 150: "battery-present"      input  active-high [used]
  line 151: "intrusion-latch-r"    output active-high [used]

# cat /sys/kernel/debug/gpio | grep intrusion
gpio-148 (chassis-intrusion |gpio-keys    ) in  hi IRQ
gpio-149 (intrusion-latch-q  |reporter    ) in  hi IRQ
gpio-150 (battery-present    |reporter    ) in  hi IRQ
gpio-151 (intrusion-latch-r  |reporter    ) out lo
```

Porting to AST2700 and Nuvoton NPCM

Only the AST2600 path is QEMU-verified by this guide. The device-tree fragments below are drawn from upstream Linux bindings but have not been run on hardware by the guide authors. Treat them as a porting starting point; validate against your SoC’s pinmux reference.

The software stack above the GPIO layer (`phosphor-intrusion-sensor` , Entity Manager, `intrusion-battery-repo` , Redfish mapping, OEM IPMI SEL) is SoC-agnostic and does **not** change across targets. Only the device tree — pinctrl driver name, pinmux group syntax, bias properties, GPIO numbering — changes.

SoC Comparison

Aspect	AST2600	AST2700	Nuvoton NPCM7xx (Poleg)	Nuvoton NPCM8xx (Arbel)
Pinctrl driver	<code>pinctrl-aspeed-g6</code>	<code>pinctrl-aspeed-g7</code>	<code>pinctrl-npcm7xx</code>	<code>pinctrl-npcm8xx</code>
Pin naming	<code>GPIO54</code> symbolic	<code>GPIO54</code> symbolic	Numeric pin (e.g. <code>gpio32</code>)	Numeric pin (e.g. <code>gpio64</code>)
Bias property	<code>bias-pull-up</code>	<code>bias-pull-up</code>	<code>bias-pull-up</code>	<code>bias-pull-up</code>
GPIO controller node	<code>&gpio0</code>	<code>&gpio0</code>	<code>&gpio1</code> (GPIO bank 1)	<code>&gpio2</code>
Vendor pinmux doc	AST2600 datasheet §5	AST2700 datasheet §5	NPCM7xx pinmux app note	NPCM8xx pinmux app note
Upstream bindings	<code>Documentation/devicetree/bindings/pinctrl/aspeed,*.yaml</code>	same	<code>nuvoton,npcm7xx-pinctrl.txt</code>	<code>nuvoton,npcm845-pinctrl.yaml</code>

AST2700 Device Tree Fragment

```
&pinctrl0 {
    pinctrl_intrusion_default: intrusion_default {
        pins = "GPIO54", "GPIO55", "GPIO56", "GPIO57";
        bias-pull-up;
    };
};

&gpio0 {
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_intrusion_default>;
    gpio-line-names =
        /* offset 148 onward */
        "chassis-intrusion", "intrusion-latch-q",
        "battery-present", "intrusion-latch-r";
};
```

AST2700 reuses the symbolic `GPIO5x` naming scheme from AST2600 — port by changing only the `pinctrl-aspeed-g6` compatible to `pinctrl-aspeed-g7` .

Nuvoton NPCM7xx (Poleg) Device Tree Fragment

```
&pinctrl {
    intrusion_pins: intrusion_pins {
        pins = "gpio32", "gpio33", "gpio34", "gpio35";
        bias-pull-up;
    };
};

&gpio1 {
    pinctrl-names = "default";
    pinctrl-0 = <&intrusion_pins>;
    gpio-line-names =
        "chassis-intrusion", "intrusion-latch-q",
        "battery-present", "intrusion-latch-r";
};
```

NPCM7xx uses numeric pin identifiers and splits GPIOs across multiple controller nodes (`gpio0` through `gpio7`). Consult [nuvoton,npcm7xx-pinctrl.txt](#) in the kernel tree for the pin-to-controller mapping.

Nuvoton NPCM8xx (Arbel) Device Tree Fragment

```
&pinctrl {
    intrusion_pins: intrusion_pins {
        pinmux = <NPCM845_PIN(64, 0) NPCM845_PIN(65, 0)
            NPCM845_PIN(66, 0) NPCM845_PIN(67, 0)>;
        bias-pull-up;
    };
};

&gpio2 {
    pinctrl-names = "default";
    pinctrl-0 = <&intrusion_pins>;
    gpio-line-names =
        "chassis-intrusion", "intrusion-latch-q",
        "battery-present", "intrusion-latch-r";
};
```

NPCM8xx uses the `NPCM845_PIN(pin, func)` macro for pinmux. See [nuvoton,npcm845-pinctrl.yaml](#).

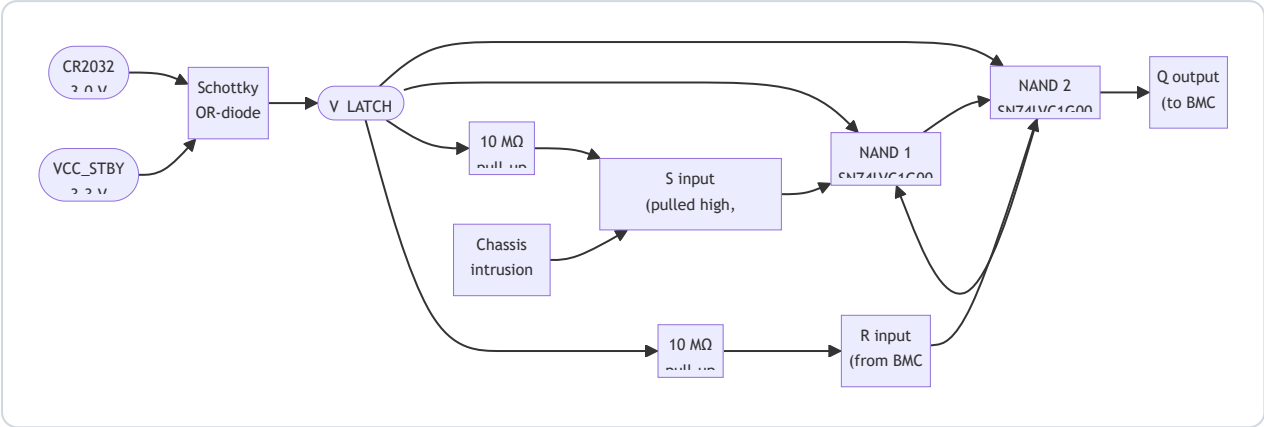
What Stays the Same

Everything above the device-tree layer is unchanged when you port this guide to a different SoC: Entity Manager JSON, [phosphor-intrusion-sensor](#) build options, the companion reporter source, Redfish mapping, and the OEM IPMI SEL layout. Only the DT fragments and the SoC's nvmem device path change.

Battery-Backed Intrusion Latch

A chassis intrusion that happens while the system is unpowered cannot be seen by a running daemon — there is no running daemon. The solution is a discrete SR-latch powered continuously from a coin cell, so that opening the chassis **sets** the latch and the BMC reads the latched state on the next boot.

Reference Schematic



Bill of Materials

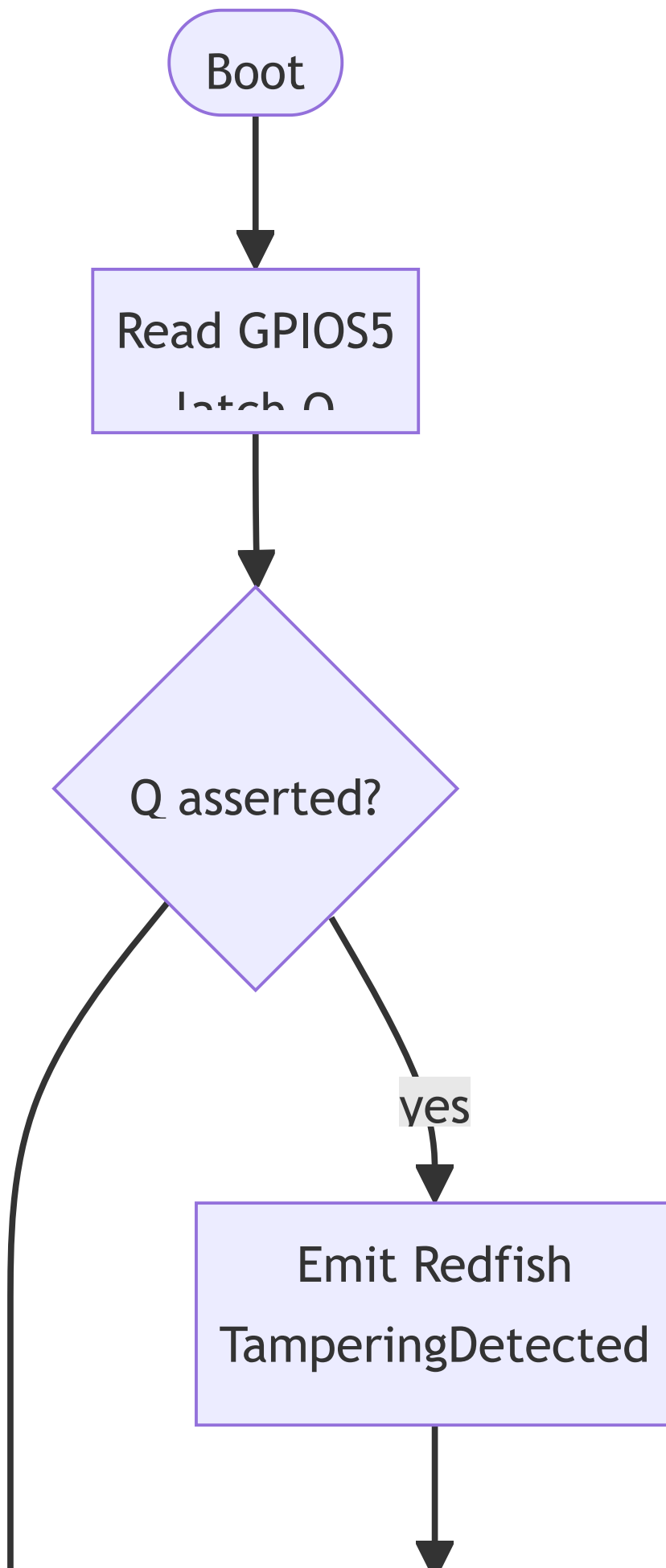
Qty	Part	Role	Target
1	SN74LVC1G00 × 2 (or one dual-NAND)	Cross-coupled NAND SR-latch	I _{cc} typ ≤ 1 μA at 3 V
2	BAT54C Schottky (common-cathode)	OR coin-cell with VCC_STBY	V _f ≈ 0.3 V @ 1 μA
2	10 MΩ 0402 resistor	S and R pull-ups	Keep quiescent drain minimal
1	100 nF 0402 ceramic	Supply decoupling on V_LATCH	Noise immunity
1	CR2032 coin cell + holder	Latch backup power	220 mAh nominal

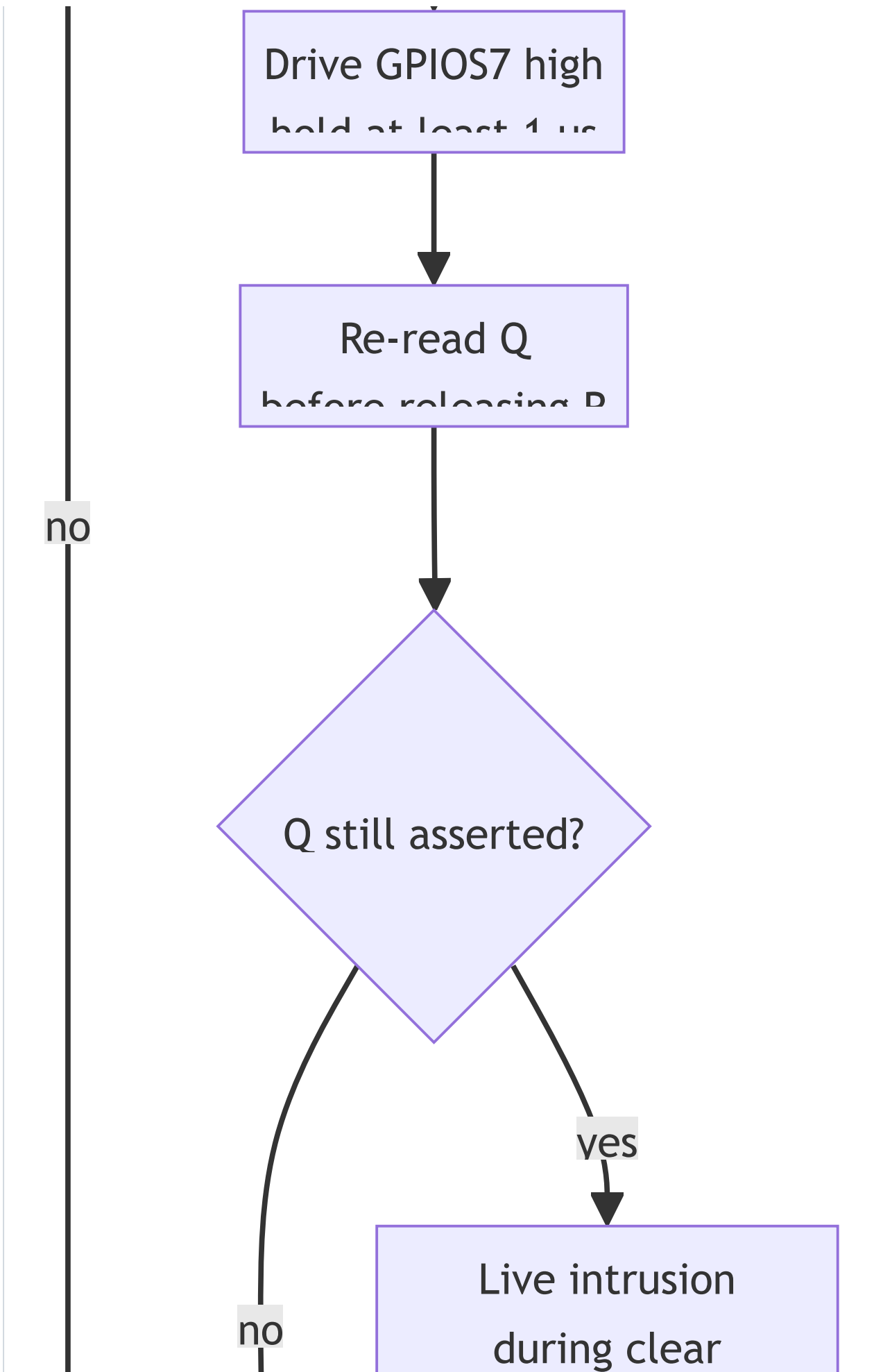
Target quiescent current from the coin cell: < 5 μA. At that draw, a nominal 220 mAh CR2032 provides ~5 years of shelf life before the latch becomes unreliable. The reporter MAY emit a `BatteryLow` advisory event when the VCC_STBY rail samples the coin cell below 2.5 V via the comparator of the next section.

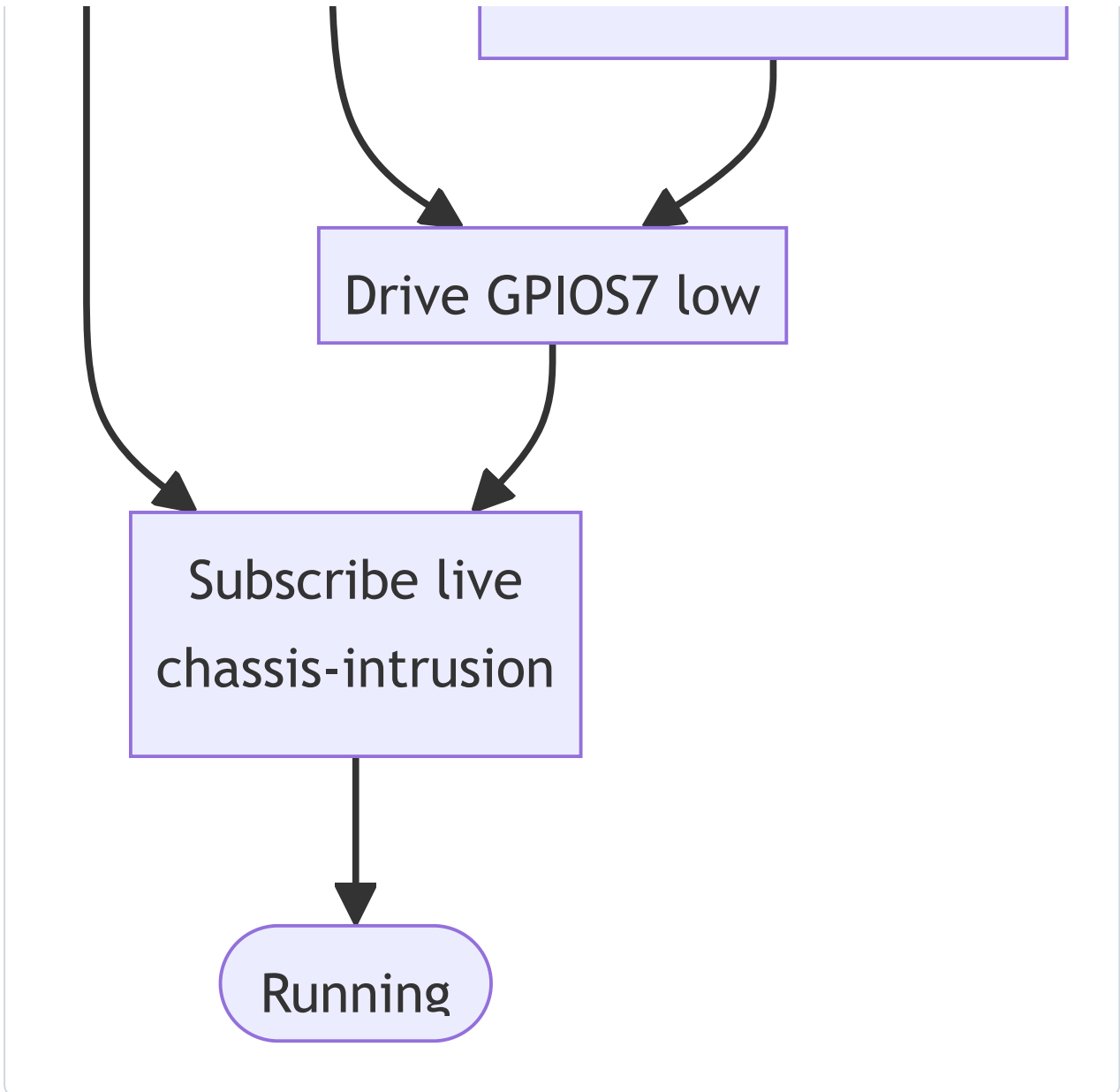
Reference only. The schematic above is pedagogical and has not been layout-verified. Validate ESD clamps on the switch line (the switch may see ±8 kV IEC 61000-4-2 discharge through the chassis), creepage between the cell holder and adjacent traces, and the coin-cell discharge profile under your part’s leakage for your specific BOM before committing to silicon.

Read-and-Clear Boot Sequence









Race window. There is a ~10 ms window during the R-pulse where a genuine live intrusion could arrive and be swallowed by the clear. The mitigation is the **re-read Q before releasing R** step: if an intrusion arrives during the pulse, it re-asserts S which re-asserts Q; the reporter sees Q still high after R has been held, and classifies that case as a live (not latched) intrusion.

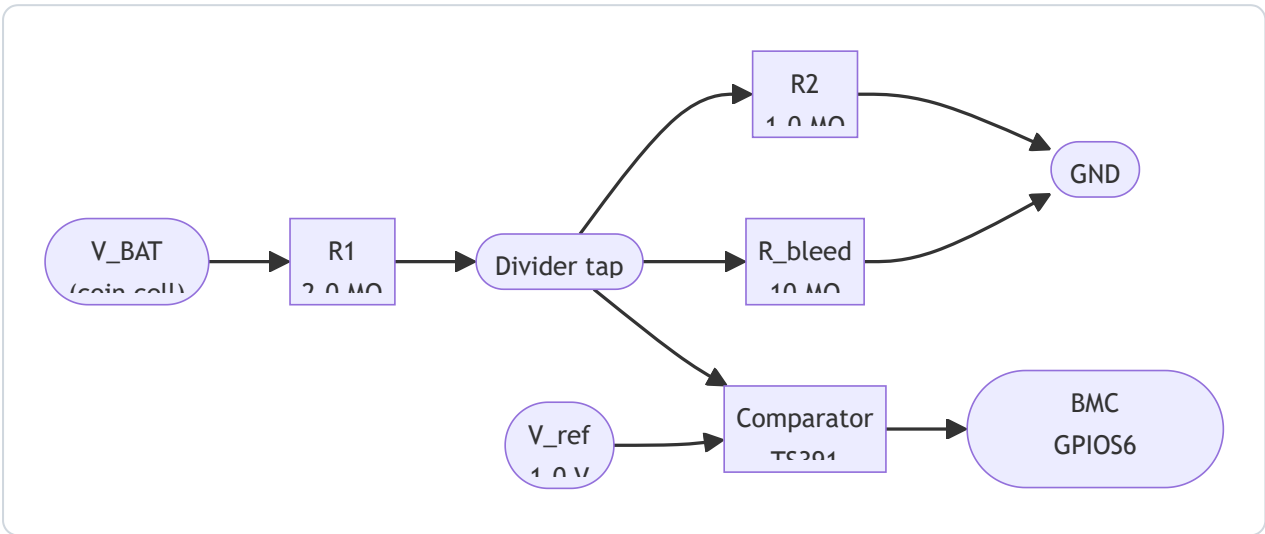
Battery-Removal Detection

Battery-removal needs two complementary detectors because they cover different threat windows. Either one alone has a blind spot.

Threat window	Detector	Event source
Battery removed while BMC is running	Live comparator (Section below)	Immediate D-Bus signal
Battery removed while BMC is powered off	VBAT-SRAM token (Section below)	Reported on next boot
Chassis opened in any power state	SR-latch (previous section)	Reported on next boot

Live Comparator (Runtime Only)

A resistor divider from VBAT to GND feeds a comparator referenced to a fixed ~1 V rail. When the coin cell is present ($\geq \sim 1.5$ V), the tap exceeds the reference and the comparator output is high. When the cell is removed, a 10 M Ω bleed resistor pulls the tap to 0 V and the comparator output goes low.



Truth table:

V_BAT	Tap voltage	Comparator output	State
0 V (removed)	0 V	Low	Removed
2.0 V (low but present)	0.67 V	Low — but...	See note
2.5 V (typical end-of-life)	0.83 V	Low	Low
3.0 V (healthy)	1.00 V $\rightarrow$ 1.0 V	High	Healthy

Tune R1/R2 so that the reference equals the tap at the “low” cut-off voltage you want — the values above distinguish 0 V (removed) from ≥ 2.0 V (present) cleanly, but treat 2.0–3.0 V as “low” via a second reference if your platform needs a low-battery advisory.

Why not an ADC? An ADC gives you a continuous reading that blurs the removed/low boundary: 0 V and 0.05 V (shorted bleed or dirty divider) look the same, and the firmware must choose a threshold. A comparator’s discrete output is the threshold, and the hardware makes the decision at microsecond latency.

The live comparator is powered by VCC_STBY and is inert when the system is fully powered off. It cannot detect a battery removed during shipment, storage, or a power-off service window. The **VBAT-SRAM token** mechanism in the next section covers that threat window; implement both.

Software debounce. A slow VCC_STBY ramp at initial power-on can briefly drive the reference below the tap and produce a false “removed” edge. The reporter enforces a 250 ms debounce on the comparator GPIO.

VBAT-SRAM Token (Across Power-Off)

The authoritative detector for a power-off removal is the BMC’s own VBAT-backed SRAM — a small block of RAM powered from VBAT when main power is absent. When the battery is removed, that SRAM loses power and its contents decay to an uninitialized state. On the next boot the reporter reads back the SRAM, validates it against a filesystem-stored seal, and declares “battery removed across power-off” if the validation fails.

SoC nvmem Exposure

SoC	Facility	Typical size	Linux path
AST2600	RTC scratchpad, VBAT-backed	~32 bytes	<code>/sys/bus/nvmem/devices/rtc-aspeed*/nvmem</code>
AST2700	Expanded RTC scratchpad, VBAT-backed	≥ 32 bytes	same pattern via AST2700 RTC driver
Nuvoton NPCM7xx	RTC backup RAM	~256 bytes	<code>/sys/bus/nvmem/devices/rtc-nuvoton*/nvmem</code>
Nuvoton NPCM8xx	RTC backup RAM	~256 bytes+	same pattern via NPCM8xx RTC driver

The guide uses the first 20 bytes of the nvmem region: 16 bytes for the token, 4 bytes for its CRC-32.

Protocol

1. On reporter startup:

- Read 20 bytes from nvmem at offset 0.
- Recompute CRC-32 of the token and compare with the stored CRC.
- If the CRC is invalid, or the token is all-zero / all-ones (post-power-loss patterns), declare **battery removed across power-off** and continue to step 3.
- Otherwise, compute `HMAC-SHA256(K_boot, token)` and compare against the seal file `/var/lib/intrusion-battery-reporter/seal`.
- If the HMAC does not match, declare **battery removed across power-off**.

2. On successful validation, rotate the token (step 3) so the value in VBAT cannot be replayed after a future removal.

3. Write a fresh token:

- Generate 16 bytes from `/dev/urandom`.
- Compute CRC-32 and write `[token || crc]` to nvmem at offset 0.
- Compute `HMAC-SHA256(K_boot, token)` and atomically replace the seal file.

4. On orderly shutdown (`ExecStop=` in the systemd unit), repeat step 3 so the token is freshly rotated before the battery takes over.

5. Periodically while running (every 15 minutes), repeat step 3 to bound the token's freshness for replay-resistance.

Keys and Files

Artifact	Location	Permissions	Note
<code>K_boot</code>	<code>/usr/share/intrusion-battery-reporter/k_boot</code>	<code>0400 root:root</code> on read-only NOR	Per-install key; burned at image creation time
Seal file	<code>/var/lib/intrusion-battery-reporter/seal</code>	<code>0600 root:root</code> on writable rootfs	32-byte binary HMAC
VBAT token	nvmem offset 0, 16 bytes	kernel-controlled	Plain bytes, not secret
VBAT CRC-32	nvmem offset 16, 4 bytes	kernel-controlled	Integrity-only, not cryptographic

Expected Post-Power-Loss Patterns

SoC	Post-power-loss SRAM state
AST2600 / AST2700	All-zero in practice (capacitor-coupled SRAM)
Nuvoton NPCM7xx / NPCM8xx	All-ones on some revisions, all-zero on others — treat both as “removed”

The reporter treats all-zero, all-ones, and invalid-CRC as the same “removed” signal.

Replay Resistance

A naive magic constant (e.g., `0xDEADBEEF` repeated) would be trivial for an attacker to restore: they remove the battery, wait for wipe, replace the battery, and reboot — the BMC sees “correct magic” and declares all-well. The rotating-token-plus-filesystem-seal construction blocks this because:

- An attacker who snapshots VBAT contents before removal only has a stale token. After reboot, the seal file contains the HMAC of *that* stale token, but the reporter rotates immediately on success, so a second removal leaves the attacker’s stale snapshot useless.
- An attacker who restores a stale token + the corresponding stale seal file succeeds — but only *once*, and only if they can also write to the rootfs (`/var/lib/...`). Putting the seal under a measured-boot root (dm-verity, etc.) closes that hole; see the [DICE Security Guide](#).

Do not use a hard-coded magic value (like `0xDEADBEEF`) instead of the rotating token. A constant magic can be trivially restored by an attacker after a battery swap. Only a rotating random token paired with a filesystem-sealed HMAC (ideally on a measured-boot rootfs) gives meaningful assurance.

Threat-model limit: an attacker who can both read the VBAT SRAM and compute `HMAC-SHA256(K_boot, ...)` defeats this mechanism. That is outside the scope of this guide; the DICE Security Guide discusses stronger platform-attestation constructions.

Companion Reporter Service

The `intrusion-battery-reporter` service glues together the latch, the live comparator, and the VBAT-SRAM token into a single small daemon. Full source is in `examples/intrusion-detection/src/intrusion-battery-reporter.cpp` . A condensed view of the core loop:

```

// intrusion-battery-reporter.cpp (excerpt – full source in examples/)
#include <sdbusplus/asio/connection.hpp>
#include <sdbusplus/asio/object_server.hpp>
#include <gpod.hpp>
#include <fstream>
#include <openssl/hmac.h>
#include <random>
#include <zlib.h> // crc32

namespace intrusion
{

constexpr auto kNvmemPath    = "/sys/bus/nvmem/devices/rtc-aspeed/nvmem";
constexpr auto kSealPath    = "/var/lib/intrusion-battery-reporter/seal";
constexpr auto kBootKeyPath = "/usr/share/intrusion-battery-reporter/k_boot";
constexpr size_t kTokenLen   = 16;
constexpr size_t kNvmemLen   = 20; // token + crc32
constexpr off_t  kNvmemOff    = 0;

bool verifyAndRotate(sdbusplus::asio::object_server& objs);
void readAndClearLatch(gpod::line& q, gpod::line& r,
                      sdbusplus::asio::object_server& objs);
void subscribeLiveComparator(gpod::line& present,
                             sdbusplus::asio::object_server& objs);

} // namespace intrusion

int main()
{
    auto io    = std::make_shared<boost::asio::io_context>();
    auto bus   = std::make_shared<sdbusplus::asio::connection>(*io);
    bus->request_name("xyz.openbmc_project.IntrusionBatteryReporter");
    auto objs = sdbusplus::asio::object_server(bus);

    gpod::chip chip("gpiochip0");
    auto latchQ = chip.find_line("intrusion-latch-q");
    auto latchR = chip.find_line("intrusion-latch-r");
    auto present = chip.find_line("battery-present");

    intrusion::readAndClearLatch(latchQ, latchR, objs);
    intrusion::verifyAndRotate(objs);
    intrusion::subscribeLiveComparator(present, objs);

    io->run();
    return 0;
}

```

Meson Build Fragment

```
# meson.build (in docs/examples/intrusion-detection/src/)
project('intrusion-battery-reporter', 'cpp',
        version: '0.1',
        default_options: ['cpp_std=c++20'])

sdbusplus = dependency('sdbusplus')
gpiodxx   = dependency('libgpiodcxx')
openssl   = dependency('openssl')
zlib      = dependency('zlib')

executable('intrusion-battery-reporter',
           'intrusion-battery-reporter.cpp',
           dependencies: [sdbusplus, gpiodxx, openssl, zlib],
           install: true,
           install_dir: get_option('bindir'))

install_data('intrusion-battery-reporter.service',
             install_dir: '/lib/systemd/system')
```

Bitbake Recipe

```
# meta-phosphor/recipes-phosphor/intrusion/intrusion-battery-reporter_git.bb
SUMMARY = "OpenBMC intrusion and battery-removal reporter"
LICENSE = "Apache-2.0"
LIC_FILES_CHKSUM = "file://LICENSE;md5=<...>"

DEPENDS = "sdbusplus libgpiod openssl zlib"
RDEPENDS:${PN} = "phosphor-intrusion-sensor"

inherit meson systemd

SRC_URI = "git://github.com/example/intrusion-battery-reporter;branch=main;protocol=https"
SRCREV  = "${AUTOREV}"
S       = "${WORKDIR}/git"

SYSTEMD_SERVICE:${PN} = "intrusion-battery-reporter.service"

do_install:append() {
    install -d ${D}${localstatedir}/lib/intrusion-battery-reporter
    install -d ${D}${datadir}/intrusion-battery-reporter
    install -m 0400 ${WORKDIR}/k_boot \
        ${D}${datadir}/intrusion-battery-reporter/k_boot
}

FILES:${PN} += "${datadir}/intrusion-battery-reporter"
```

Systemd Unit

```
# intrusion-battery-reporter.service
[Unit]
Description=OpenBMC intrusion and battery-removal reporter
After=phosphor-intrusion-sensor.service
Requires=phosphor-intrusion-sensor.service

[Service]
Type=dbus
BusName=xyz.openbmc_project.IntrusionBatteryReporter
ExecStart=/usr/bin/intrusion-battery-reporter
ExecStop=/usr/bin/intrusion-battery-reporter --rotate-on-shutdown
Restart=on-failure
ProtectSystem=strict
ReadWritePaths=/var/lib/intrusion-battery-reporter /sys/bus/nvmem/devices

[Install]
WantedBy=multi-user.target
```

Entity Manager Schema — `BatteryPresenceSensor`

There is no upstream Entity Manager type for coin-cell presence, so this guide defines a project-local one.

```
{
  "Exposes": [
    {
      "Name": "Coin_Cell_Present",
      "Type": "BatteryPresenceSensor",
      "GpioName": "battery-present",
      "Polarity": "High",
      "DebounceMs": 250,
      "NvmemDevice": "/sys/bus/nvmem/devices/rtc-aspeed/nvmem",
      "NvmemOffset": 0
    }
  ]
}
```

Field reference:

Field	Type	Required	Description
<code>Name</code>	string	yes	Human-readable sensor name
<code>Type</code>	string	yes	Must be <code>"BatteryPresenceSensor"</code>
<code>GpioName</code>	string	yes	<code>gpio-line-names</code> entry for the comparator output
<code>Polarity</code>	string	yes	<code>"High"</code> = high-means-present, <code>"Low"</code> = inverted
<code>DebounceMs</code>	int	no (default 250)	Software debounce on comparator edges
<code>NvmemDevice</code>	string	yes	nvmem path for the VBAT SRAM token
<code>NvmemOffset</code>	int	no (default 0)	Byte offset within the nvmem region

`BatteryPresenceSensor` is not an upstream Entity Manager type. Vendor it in your meta-layer's entity-manager schema overrides. If upstream adopts this type later, migrate by changing only the `Type` string; the field

layout is compatible.

Redfish Mapping

PhysicalSecurity.IntrusionSensor

The standard Redfish property has three values. This guide maps them as follows:

Redfish value	Hardware condition	Trigger
Normal	Chassis closed, no latched event	Default steady state
HardwareIntrusion	Chassis switch open right now	Live GPIO edge on chassis-intrusion
TamperingDetected	SR-latch was set during the previous power-off window	Latch Q read as asserted at boot

Example queries — the same endpoint returns different IntrusionSensor values depending on hardware state.

State 1 — Normal (chassis closed, no latched event):

```
# curl -k -u root:openBmc https://<bmc>/redfish/v1/Chassis/chassis \
| jq .PhysicalSecurity
{
  "IntrusionSensor": "Normal",
  "IntrusionSensorNumber": 0,
  "IntrusionSensorReArm": "Manual"
}
```

State 2 — HardwareIntrusion (chassis switch open right now):

```
# curl -k -u root:openBmc https://<bmc>/redfish/v1/Chassis/chassis \
| jq .PhysicalSecurity
{
  "IntrusionSensor": "HardwareIntrusion",
  "IntrusionSensorNumber": 0,
  "IntrusionSensorReArm": "Manual"
}
```

State 3 — TamperingDetected (SR-latch was set during a previous power-off):

```
# curl -k -u root:openBmc https://<bmc>/redfish/v1/Chassis/chassis \
| jq .PhysicalSecurity
{
  "IntrusionSensor": "TamperingDetected",
  "IntrusionSensorNumber": 0,
  "IntrusionSensorReArm": "Manual"
}
```

Battery-Removal → LogEntry

Redfish PhysicalSecurity has no standard field for battery removal. This guide emits a LogEntry under the system EventLog for each battery-removal event:

```
# curl -k -u root:OpenBmc \
https://<bmc>/redfish/v1/Systems/system/LogServices/EventLog/Entries/17
{
  "@odata.id": ".../LogServices/EventLog/Entries/17",
  "@odata.type": "#LogEntry.v1_13_0.LogEntry",
  "Id": "17",
  "EntryType": "Event",
  "Severity": "Critical",
  "Message": "Battery removal detected.",
  "MessageId": "OpenBMC.0.5.BatteryRemoved",
  "MessageArgs": ["live-comparator"],
  "AdditionalDataURI": null,
  "Oem": {
    "OpenBmc": {
      "DetectorSource": "live-comparator"
    }
  }
}
```

The `DetectorSource` value is either `live-comparator` (runtime detection) or `vbat-token` (power-off detection discovered at next boot). Monitoring tools use this field to distinguish a runtime tamper from one that happened during shipment or storage.

The battery-removal → LogEntry mapping is a project choice, not a Redfish standard. The Redfish `PhysicalSecurity` schema defines no `BatteryRemoved` field. If a future Redfish release adds one, migrate by emitting the standard field in addition to (or instead of) the LogEntry. See the [Redfish Chassis schema](#).

IPMI OEM SEL

OEM Sensor Type

Standard IPMI v2.0 Table 42-3 defines sensor type `0x05` (`Physical Security`) with event offsets for chassis intrusion. **That covers the live chassis switch.** For battery-removal there is no standard mapping, so this guide defines a non-standard OEM event.

OEM definition:

Field	Value	Meaning
Sensor type	<code>0xC0</code>	OEM reserved range (<code>C0h..FFh</code> per IPMI 2.0 §42.2)
Event/reading type	<code>0x6F</code>	Sensor-specific discrete
Event data 1	Detector source	<code>0x10</code> = live-comparator, <code>0x11</code> = VBAT-token
Event data 2	Transition	<code>0x01</code> = removed, <code>0x02</code> = restored
Event data 3	Reserved	<code>0xFF</code>

Decoded View

```
# ipmitool sel list
17 | 04/23/2026 | 08:17:33 | OEM #0xC0 Battery Removal | Asserted
    | Source: live-comparator
18 | 04/23/2026 | 10:02:06 | OEM #0xC0 Battery Removal | Deasserted
    | Source: live-comparator
23 | 04/24/2026 | 00:04:11 | OEM #0xC0 Battery Removal | Asserted
    | Source: vbat-token
```

The OEM sensor type `0xC0` and event-data byte layout above are non-standard. IPMI 2.0 reserves `C0h..FFh` for OEM use but does not define layouts within that range. Coordinate the specific OEM number with your platform's other OEM usage to avoid collisions. If you ship into multiple IPMI-consuming ecosystems (Dell OpenManage, HPE iLO, SuperDoctor, etc.), publish your OEM definitions so the consuming tools can decode them.

Live chassis intrusion uses the standard type. For the live chassis switch the guide uses the standard IPMI `0x05 Physical Security (Chassis Intrusion)` sensor — no OEM extension is needed. See IPMI 2.0 specification, Table 42-3.

QEMU Verification Walkthrough

This walkthrough is reproducible on the repo's standard QEMU setup (`scripts/run-qemu.sh ast2600-evb`) and uses `gpio-mockup` to simulate the four GPIOs.

Kernel Config Check

```
bmc# zcat /proc/config.gz | grep -E "GPIO MOCKUP|NVMEM_SYSFS"
CONFIG_GPIO MOCKUP=m
CONFIG_NVMEM_SYSFS=y
```

If `GPIO MOCKUP` is missing from your QEMU kernel, either rebuild with it enabled or use the sysfs-GPIO fallback described at the end of this section.

Setup

```
# Start QEMU
./scripts/run-qemu.sh ast2600-evb

# Log in: root / OpenBmc
# In the BMC shell:
bmc# modprobe gpio-mockup gpio_mockup_ranges=-1,4 gpio_mockup_named_lines
bmc# for idx in 148 149 150 151; do
    echo "gpiochip1 $((idx - 148)) $(seq ... names)" # sets line names
done
bmc# systemctl start intrusion-battery-reporter
```

1. Verify Live Intrusion

```
bmc# gpioset gpiochip1 0=1    # chassis-intrusion → open
bmc# journalctl -u intrusion-battery-reporter -n 5 | grep intrusion
... HardwareIntrusion asserted on Chassis_Intrusion
bmc# busctl get-property xyz.openbmc_project.IntrusionSensor \
    /xyz/openbmc_project/Intrusion/Chassis_Intrusion \
    xyz.openbmc_project.Chassis.Intrusion Status
s "HardwareIntrusion"
# From the host:
host$ curl -k -u root:OpenBmc https://localhost:2443/redfish/v1/Chassis/chassis \
    | jq .PhysicalSecurity.IntrusionSensor
"HardwareIntrusion"
```

2. Verify Latched Intrusion

```
# Simulate: previous boot had chassis open; latch Q is asserted at boot
bmc# systemctl stop intrusion-battery-reporter
bmc# gpioset gpiochip1 1=1    # intrusion-latch-q = asserted
bmc# systemctl start intrusion-battery-reporter
bmc# journalctl -u intrusion-battery-reporter -n 10 | grep Tamper
... TamperingDetected emitted; latch reset pulse complete
bmc# busctl get-property xyz.openbmc_project.IntrusionSensor \
    /xyz/openbmc_project/Intrusion/Chassis_Latch \
    xyz.openbmc_project.Chassis.Intrusion Status
s "TamperingDetected"
```

3. Verify Live Battery Removal

```
bmc# gpioset gpiochip1 2=0    # battery-present drops to 0
bmc# sleep 1
bmc# ipmitool sel list | tail -2
17 | ... | OEM #0xC0 Battery Removal | Asserted | Source: live-comparator
host$ curl -k -u root:OpenBmc https://localhost:2443/redfish/v1/Systems/system/LogServices/EventLog/
Entries \
    | jq '.Members[-1].Oem.OpenBmc.DetectorSource'
"live-comparator"
```

4. Verify Across-Power-Off Battery Removal

```
# Simulate the VBAT SRAM losing power by zeroing it between reporter runs
bmc# systemctl stop intrusion-battery-reporter
bmc# dd if=/dev/zero of=/sys/bus/nvmem/devices/rtc-aspeed/nvmem \
    bs=1 count=20 conv=notrunc
bmc# systemctl start intrusion-battery-reporter
bmc# ipmitool sel list | tail -1
18 | ... | OEM #0xC0 Battery Removal | Asserted | Source: vbat-token
host$ curl -k -u root:OpenBmc https://localhost:2443/redfish/v1/Systems/system/LogServices/EventLog/
Entries \
    | jq '.Members[-1].Oem.OpenBmc.DetectorSource'
"vbat-token"
```

5. Verify Token Rotation

```
bmc# xxd /sys/bus/nvmem/devices/rtc-aspeed/nvmem | head -2 > /tmp/before
bmc# systemctl reload intrusion-battery-reporter # triggers verify-and-rotate
bmc# xxd /sys/bus/nvmem/devices/rtc-aspeed/nvmem | head -2 > /tmp/after
bmc# diff /tmp/before /tmp/after
# (expect non-identical token bytes; CRC also changes)
bmc# stat -c %Y /var/lib/intrusion-battery-reporter/seal
# (expect mtime updated to now)
```

`gpio-mockup` Fallback

On QEMU kernels without `CONFIG_GPIO MOCKUP`, use sysfs GPIO directly:

```
bmc# for gpio in 148 149 150 151; do
    echo $gpio > /sys/class/gpio/export
    echo out > /sys/class/gpio/gpio$gpio/direction
done
bmc# echo 1 > /sys/class/gpio/gpio148/value # chassis open
```

Most steps above work with `echo ... > /sys/class/gpio/gpioN/value` in place of `gpioset`.

Commit Hash Used for Verification

This walkthrough was verified against OpenBMC `<COMMIT_HASH_PENDING>`. Update the “Tested on” footer at the bottom of the guide with the actual commit hash recorded by `git rev-parse HEAD` at the time of verification.

Troubleshooting

1. Chassis intrusion signal stuck asserted

Symptom: `PhysicalSecurity.IntrusionSensor` reads `HardwareIntrusion` with the chassis visibly closed.

Solution:

1. Check polarity in Entity Manager JSON — `GpioInverted: true` with NC switch.
2. Confirm pinmux applied: `cat /sys/kernel/debug/pinctrl/<ctrl>/pinmux-pins | grep <pin>`.
3. Verify the internal pull-up is active: `gpioinfo | grep chassis-intrusion` — should show `active-low [used]`.
4. Read the raw value to bypass polarity: `gpioget gpiochip0 <line>` — should be `0` (switch shorting to ground) with chassis closed.

2. Latched state not cleared across reboots

Symptom: `Chassis_Latch` reports `TamperingDetected` on every boot even without opening the chassis.

Solution:

1. Verify R GPIO direction is `output`: `gpioinfo | grep intrusion-latch-r`.
2. Measure the R-pulse width with a scope at the gate input; it must hold $\geq 1\ \mu\text{s}$ (the SN74LVC1G00 minimum reset hold).
3. Check cross-coupling in the schematic — a swapped NAND connection leaves the latch permanently set.

4. If the latch IC runs on just VBAT (no VCC_STBY available) ensure the pulse swings to the full V_LATCH rail, not to 0 V through an unpowered output buffer.

3. Comparator false-positive on slow power ramps

Symptom: `Coin_Cell_Present` briefly reports `Removed` during boot-up or power-cycle, then returns to `Present`.

Solution:

1. The built-in 250 ms software debounce covers most cases. If your VCC_STBY ramp is slower than that, raise `DebounceMs` in the Entity Manager config to 500 ms.
2. Add a 10 nF cap from the comparator reference to ground so the reference tracks VCC_STBY during ramp.
3. Confirm the bleed resistor is truly 10 MΩ — a lower value (mis-populated 1 MΩ) slows the tap discharge and creates the same symptom.

4. `phosphor-intrusion-sensor` code drift from the example snippet

Symptom: The reporter source example in this guide no longer compiles against the upstream dbus-sensors headers, or the D-Bus schema has changed.

Solution:

1. Diff the reporter's GPIO-monitor usage against current upstream `dbus-sensors/src/intrusion/`:

```
cd dbus-sensors && git log --oneline src/intrusion/ \
-- path-at-tested-commit..HEAD
```

2. Pin your build to the commit listed in the "Tested on" footer, or update the reporter source to the new API.
3. File an issue in `openbmc-guide-tutorial` with the diff so the guide can be refreshed.

5. VBAT nvmm device not present under `/sys/bus/nvmm/`

Symptom: `ls /sys/bus/nvmm/devices/` has no `rtc-*` entry.

Solution:

1. Confirm the RTC driver loaded: `dmesg | grep -Ei "aspeed.*rtc|npcm.*rtc"`.
2. Verify the device tree exposes the nvmm child node: `ls /proc/device-tree/*/rtc*/nvmm-cells 2>/dev/null`.
3. Check kernel config: `zcat /proc/config.gz | grep -E "RTC_DRV_ASPEED|RTC_NUVOTON|NVMM_SYSFS"` — all three should be `=y` or `=m`.
4. On older kernels, the Aspeed RTC may not expose nvmm until the backport lands; use `devmem` as a last resort (requires a small shim in the reporter).

6. Persistent false "battery removed"

Symptom: `DetectorSource: vbat-token` fires on every clean reboot even though the battery is fine.

Solution:

1. Confirm the seal file was written: `stat /var/lib/intrusion-battery-reporter/seal` — mtime should match the last reporter shutdown.

2. Confirm `ExecStop=` actually runs: `journalctl -u intrusion-battery-reporter | grep "rotate-on-shutdown"`. If systemd kills the unit before `ExecStop` finishes, bump `TimeoutStopSec=` to 10 s.
3. Confirm the nvmem region is writable: `dd if=/dev/urandom of=/sys/bus/nvmem/devices/rtc-aspeed/nvmem bs=1 count=1 conv=notrunc` — any EPERM indicates a driver bug or a read-only mapping.
4. Check the rootfs is writable: `mount | grep /var` — a read-only `/var` silently fails the seal update on some images.

Debug Commands

```
# Service status and logs
systemctl status intrusion-battery-reporter
journalctl -u intrusion-battery-reporter -f

# GPIO state
gpioinfo | grep -E "chassis|intrusion|battery"
gpioget gpiochip0 chassis-intrusion intrusion-latch-q battery-present

# D-Bus introspection
busctl tree xyz.openbmc_project.IntrusionSensor
busctl tree xyz.openbmc_project.IntrusionBatteryReporter
busctl introspect xyz.openbmc_project.IntrusionSensor \
    /xyz/openbmc_project/Intrusion/Chassis_Intrusion

# VBAT nvmem inspection
xxd /sys/bus/nvmem/devices/rtc-aspeed/nvmem | head -2
stat /var/lib/intrusion-battery-reporter/seal

# Redfish
curl -sk -u root:openBmc https://<bmc>/redfish/v1/Chassis/chassis \
    | jq .PhysicalSecurity
curl -sk -u root:openBmc \
    https://<bmc>/redfish/v1/Systems/system/LogServices/EventLog/Entries \
    | jq '.Members[] | select(.MessageId == "OpenBMC.0.5.BatteryRemoved")'

# IPMI
ipmitool sel list | grep -i intrusion
ipmitool sel list | grep "OEM #0xC0"
```

References

Official Resources

- `dbus-sensors` / `phosphor-intrusion-sensor`
- `xyz.openbmc_project.Chassis.Intrusion` [YAML](#)
- `bmcweb` [chassis handler](#)
- [Redfish](#) `Chassis.PhysicalSecurity` [schema](#)
- [IPMI v2.0 specification](#) — Table 42-3 (Sensor Types)

Related Guides

- [GPIO Management](#) — `phosphor-gpio-monitor`, line-names, pinmux
- [Buttons](#) — similar GPIO-monitoring pattern for power/reset buttons

- [Redfish Events & Telemetry](#) — event stream, EventLog service
- [DICE Security Guide](#) — measured-boot attestation, where the VBAT-seal threat model hands off

SoC References

- Aspeed AST2600 datasheet — pinmux Section 5, RTC scratchpad Section 18
 - Aspeed AST2700 datasheet — pinmux Section 5
 - Nuvoton NPCM7xx pinmux: [Documentation/devicetree/bindings/pinctrl/nuvoton,npcm7xx-pinctrl.txt](#)
 - Nuvoton NPCM8xx pinmux: [Documentation/devicetree/bindings/pinctrl/nuvoton,npcm845-pinctrl.yaml](#)
 - Aspeed pinctrl: [Documentation/devicetree/bindings/pinctrl/aspeed,*.yaml](#)
-

Tested on: QEMU ast2600-evb, OpenBMC commit [<COMMIT_HASH_PENDING>](#) Last updated: 2026-04-23

Leak Detection

Configure OpenBMC's `leakdetector` daemon to watch liquid-cooling leak sensors on GPIO pins and drive protective systemd actions when coolant is detected.

Table of Contents

1. [Overview](#)
 2. [Architecture](#)
 1. [Component Diagram](#)
 2. [Discovery:](#) `DetectionManager`
 3. [Per-pin monitoring:](#) `GPIODetector`
 4. [Event commit:](#) `Events`
 5. [D-Bus Interfaces](#)
 6. [Key Dependencies](#)
 3. [Configuration](#)
 1. [Build-time Option](#)
 2. [Entity Manager Configuration Interface](#)
 3. [Example Configuration](#)
 4. [Leak Events and Systemd Actions](#)
 1. [Event Commit](#)
 2. [Systemd Action Targets](#)
 3. [End-to-End Flow](#)
 5. [Verify on the Running BMC](#)
 6. [Troubleshooting](#)
 1. [Issue: A configured detector never appears on D-Bus](#)
 2. [Issue:](#) `State` never leaves `Normal` when the sensor wets
 3. [Issue: Leak logged but no protective action runs](#)
 4. [Debug Commands](#)
 7. [References](#)
 1. [Official Resources](#)
 2. [Related Guides](#)
-

Overview

Liquid-cooled server and rack designs route coolant through cold plates, manifolds, and quick-disconnect fittings that can leak. A leak near powered electronics is a hard-shutdown event, not an advisory, so the platform needs a detection path that is fast, hardware-driven, and able to reach the power subsystem without waiting on a host or an operator.

OpenBMC covers this with the `leakdetector` daemon in `dbus-sensors`. It watches leak-sensing cables and float switches wired to BMC GPIOs, publishes each detector's state on D-Bus, logs a phosphor-logging event when a

leak asserts, and starts a platform-supplied systemd target so the machine can take a protective action — up to and including powering the chassis off.

You would use this guide when porting OpenBMC to a liquid-cooled platform, when wiring a new leak-sensing cable into an existing design, or when validating that a detected leak actually reaches your power-off logic. It builds on the [D-Bus Sensors](#) guide — `leakdetector` is a sibling daemon in the same repository and shares its Entity Manager discovery model.

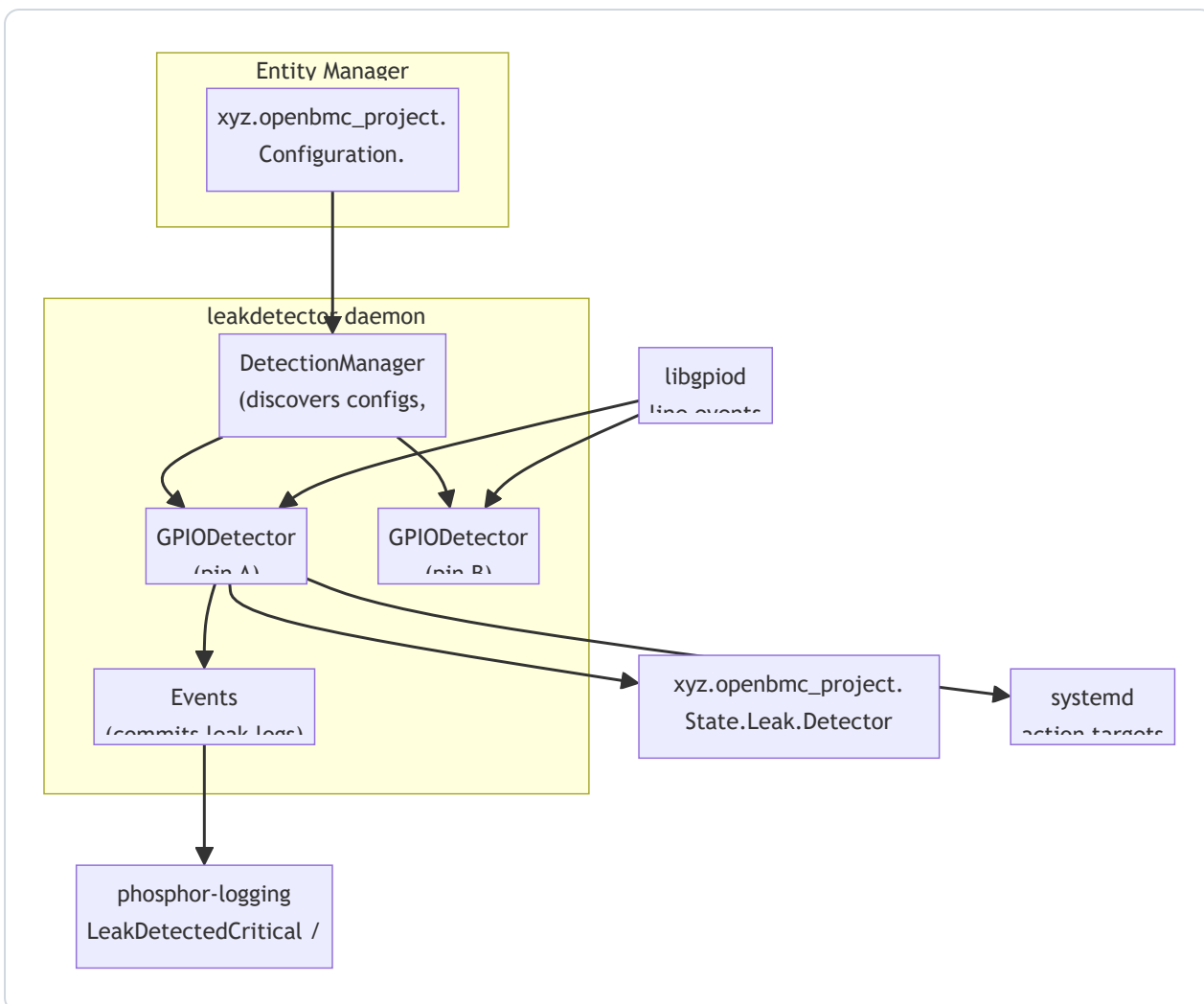
Key concepts covered:

- The `leakdetector` daemon and its `DetectionManager` / `GPIODevice` structure
- `DetectorState` (`Normal` / `Abnormal`) and how a GPIO edge maps to it
- Entity Manager discovery via the `xyz.openbmc_project.Configuration.GPIOLeakDetector` interface
- The `LeakDetectedCritical` / `LeakDetectedWarning` phosphor-logging events
- Templated systemd action targets (level x assert/deassert) and auto-resolution on clear
- Validating the target mapping before it can power your system off

Architecture

`leakdetector` is a single `sdbusplus::async` daemon that owns the D-Bus service name `xyz.openbmc_project.1eakdetector`. Internally it splits into three cooperating pieces.

Component Diagram



Discovery: `DetectionManager`

`DetectionManager` (in `src/leakdetector/LeakDetectionManager.cpp`) is the top-level object. It subscribes to Entity Manager inventory and reacts to configuration objects that expose the `xyz.openbmc_project.Configuration.n.GPIOLeakDetector` interface:

1. `processInventoryAdded()` fires when a matching inventory interface appears.
2. `processConfigAddedAsync()` reads the detector's properties from Entity Manager (`Name` , `Type` , `PinName` , `Polarity` , `Level` , `SubType`) and validates them against internal lookup tables.
3. A `GPIODetector` is constructed from the resulting `DetectorConfig` and stored in a `std::unordered_map` keyed by the inventory object path.

Because discovery is inventory-driven, detectors can come and go at runtime as FRUs (for example a hot-plug coolant manifold) are added or removed — no daemon restart is required.

Per-pin monitoring: [GPIODetector](#)

Each `GPIODetector` (in `src/leakdetector/LeakGPIODetector.cpp`) owns exactly one GPIO line. It watches the line for edge events and maps the electrical state to a logical `DetectorState`:

```
// LeakGPIODetector.cpp (upstream)
auto newState = gpioState ? DetectorIntf::DetectorState::Abnormal
                          : DetectorIntf::DetectorState::Normal;
```

The `Polarity` config key selects whether the “leak” condition is active-low or active-high, so `gpioState` already accounts for inversion. `DetectorState` is defined by the `xyz.openbmc_project.State.Leak.Detector` D-Bus interface and has three values: `Normal`, `Abnormal`, and `Unknown`. The detector publishes its current value in the `State` property, alongside `PrettyName` and a `Type` (`LeakSensingCable` for a leak-sensing cable). It also emits an `xyz.openbmc_project.Association.Definitions` association back to the parent inventory object so consumers can tie the detector to the FRU it protects.

Event commit: `Events`

When a detector transitions, `GPIODetector` calls into the `Events` helper (`src/leakdetector/LeakEvents.cpp`), which commits a phosphor-logging event keyed to the detector’s `Level`. This is the piece that produces the durable log entry and, in turn, the Redfish event.

D-Bus Interfaces

Interface	Role	Source
<code>xyz.openbmc_project.Configuration.GPIOLeakDetector</code>	Per-detector configuration published by Entity Manager and consumed by <code>DetectionManager</code>	Entity Manager
<code>xyz.openbmc_project.State.Leak.Detector</code>	Detector runtime state (<code>State</code> = <code>Normal</code> / <code>Abnormal</code> / <code>Unknown</code> , plus <code>PrettyName</code> , <code>Type</code>)	<code>leakdetector</code> (service <code>xyz.openbmc_project.leakdetector</code>)
<code>xyz.openbmc_project.Association.Definitions</code>	Associates each detector with its parent inventory object	<code>leakdetector</code>
<code>xyz.openbmc_project.Logging.Entry</code>	Persistent <code>LeakDetectedCritical</code> / <code>LeakDetectedWarning</code> / resolved entries	phosphor-logging

The exact object path for the detector state object lives under the `xyz.openbmc_project.leakdetector` service (convention: `/xyz/openbmc_project/state/leak/detector/<Name>`). Confirm the path on your build with `busctl tree xyz.openbmc_project.leakdetector` rather than hard-coding it.

Key Dependencies

- `entity-manager`: Publishes the `GPIOLeakDetector` configuration objects the daemon discovers.
- `libgpiod`: Provides the GPIO line-event interface each `GPIODetector` waits on.
- `phosphor-logging`: Backs the `lg2::commit()` calls that turn a leak into a durable, Redfish-visible event.
- `systemd`: Runs the platform-supplied action targets the daemon starts on assert and deassert.
- `phosphor-dbus-interfaces`: Defines the `State.Leak.Detector` interface and the `GPIOLeakDetector` configuration schema.

Configuration

Build-time Option

`leakdetector` is gated behind a meson option in `dbus-sensors` and is off by default. Enable it in your machine's `dbus-sensors` recipe:

Option	Default	Description
<code>leakdetector</code>	<code>disabled</code>	Builds and installs the <code>leakdetector</code> daemon

```
# meta-<your-machine>/recipes-phosphor/sensors/dbus-sensors_%.bbappend
PACKAGECONFIG:append = " leakdetector"
```

(The daemon is only compiled when `get_option('leakdetector').allowed()` is true; otherwise `src/leakdetector/meson.build` calls `subdir_done()` and skips the build.)

Entity Manager Configuration Interface

Each leak detector is one `Exposes` entry of type `GPIOLeakDetector`. Entity Manager publishes it as the `xyz.openbmc_project.Configuration.GPIOLeakDetector` D-Bus interface, which `DetectionManager` discovers.

Key	Type	Required	Description	Accepted values
<code>Name</code>	string	yes	Human-readable detector name; also used to build the systemd instance name	any
<code>Type</code>	string	yes	Selects this configuration type	<code>"GPIOLeakDetector"</code>
<code>SubType</code>	string	yes	Detector kind, mirrors the Redfish <code>LeakDetector</code> schema	e.g. <code>"Moisture"</code>
<code>PinName</code>	string	yes	<code>gpio-line-names</code> entry for the sensor's GPIO	any line name
<code>Polarity</code>	string	yes	Which electrical level means "leak"	<code>"Low"</code> (active-low) or <code>"High"</code> (active-high)
<code>Level</code>	string	yes	Severity, selects the event and the action target	<code>"Warning"</code> or <code>"Critical"</code>

The daemon validates `Type`, `Polarity`, and `Level` against fixed tables (`LeakSensingCable`; `Low`/`High`; `Warning`/`Critical`). A value outside those tables is rejected and the detector is not created — check the daemon log if a configured detector never appears on D-Bus.

Example Configuration

```
{
  "Exposes": [
    {
      "Name": "Rack_Leak_Detector_0",
      "Type": "GPIOLeakDetector",
      "SubType": "Moisture",
      "PinName": "RACK_LEAK_DETECT_0",
      "Polarity": "Low",
      "Level": "Critical"
    }
  ]
}
```

This declares an active-low, critical-severity leak-sensing cable on the `RACK_LEAK_DETECT_0` GPIO line. When that line asserts, the daemon logs `LeakDetectedCritical` and starts the critical assert target for `Rack_Leak_Detector_0`.

Leak Events and Systemd Actions

Event Commit

On a `Normal -> Abnormal` transition the daemon commits a phosphor-logging event chosen by the detector's `Level`:

Level	Event committed on leak	Event on clear
Critical	LeakDetectedCritical	resolved / LeakDetectedNormal
Warning	LeakDetectedWarning	resolved / LeakDetectedNormal

Both events carry the detector name and its object path. The commit produces a `xyz.openbmc_project.Logging.Event`, which bmcweb surfaces to Redfish clients. When the GPIO returns to its non-leak state, the detector transitions back to `Normal` and the condition auto-resolves — you do not clear it manually.

Systemd Action Targets

Committing the event is only half the response. The daemon also **starts a systemd unit** so the platform can act. The target name is built from the detector's level, the transition direction (assert on leak, deassert on clear), and the detector's `Name`:

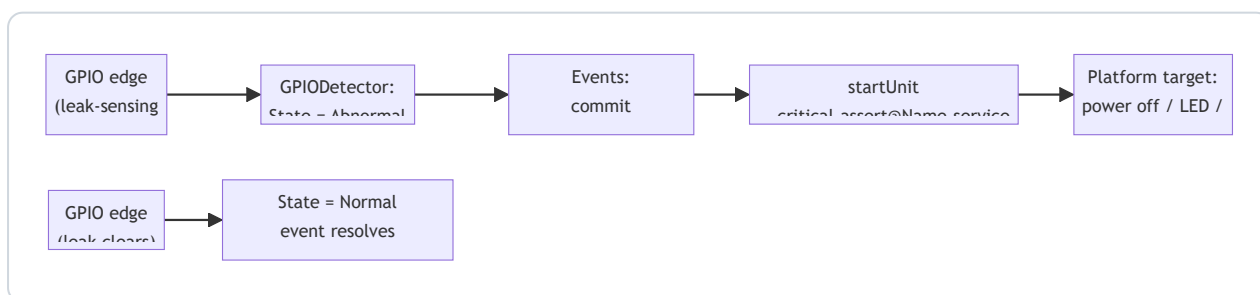
```
// LeakGPIODetector.cpp (upstream) – level x action -> unit prefix
static constexpr auto leakActionTargets = std::to_array<
    std::tuple<config::DetectorLevel, std::string_view, std::string_view>>(
    {{config::DetectorLevel::warning, "assert",
      "xyz.openbmc_project.leakdetector.warning.assert@"},
     {config::DetectorLevel::warning, "deassert",
      "xyz.openbmc_project.leakdetector.warning.deassert@"},
     {config::DetectorLevel::critical, "assert",
      "xyz.openbmc_project.leakdetector.critical.assert@"},
     {config::DetectorLevel::critical, "deassert",
      "xyz.openbmc_project.leakdetector.critical.deassert@"}}});
// ...
auto target = std::string(serviceSuffix) + config.name + ".service";
co_await systemd::SystemdInterface::startUnit(ctx, target);
```

The `@` marks these as **systemd template units** — the daemon instantiates a template with the detector name. For the example above, the four possible units are:

Transition	Unit started
Critical leak asserts	<code>xyz.openbmc_project.leakdetector.critical.assert@Rack_Leak_Detector_0.service</code>
Critical leak clears	<code>xyz.openbmc_project.leakdetector.critical.deassert@Rack_Leak_Detector_0.service</code>
Warning leak asserts	<code>xyz.openbmc_project.leakdetector.warning.assert@<Name>.service</code>
Warning leak clears	<code>xyz.openbmc_project.leakdetector.warning.deassert@<Name>.service</code>

Your platform ships the corresponding template units (`...assert@.service`, `...deassert@.service`). This is the seam where the generic daemon meets platform-specific policy: a critical assert target typically chains to chassis power-off, while a warning assert target might only light an LED or throttle pumps.

End-to-End Flow



A leak action target can power your system off — validate the mapping before you trust it. The critical assert target is platform-supplied, and the obvious protective action for a coolant leak is an immediate chassis power-off. That means a stuck sensor, a mis-wired active-low/active-high `Polarity`, or a template unit that points at the wrong `WantedBy` / `ExecStart` can take the machine down (or, worse, fail to). Before deploying: confirm each `...assert@<Name>.service` and `...deassert@<Name>.service` template exists, dry-run it with `systemctl start`, and verify the assert path reaches your power target and the deassert path is safe to run on a false clear. Test with the real GPIO, not just D-Bus, so polarity inversion is exercised end-to-end.

Verify on the Running BMC

```
# Confirm the daemon is up and owns its bus name
# systemctl status xyz.openbmc_project.leakdetector.service
# busctl tree xyz.openbmc_project.leakdetector

# Inspect a detector's state
# busctl introspect xyz.openbmc_project.leakdetector \
    /xyz/openbmc_project/state/leak/detector/Rack_Leak_Detector_0 \
    | grep -E "State|Type|PrettyName"
.PrettyName    property  s   "Rack_Leak_Detector_0"   emits-change
.State         property  s   "xyz.openbmc_project.State.Leak.Detector.DetectorState.Normal" ...
.Type          property  s   "...DetectorType.LeakSensingCable" ...

# Watch the action targets defined for your platform
# systemctl list-unit-files | grep leakdetector
```

To exercise the full path in QEMU (`scripts/run-qemu.sh ast2600-evb`), drive the configured GPIO line with `gpio set` (or `gpio-mockup`) and confirm the `State` property flips to `Abnormal`, a `LeakDetectedCritical` log entry appears (`journalctl -u xyz.openbmc_project.leakdetector`), and the expected `...critical.assert@<Name>.service` unit is started. Point the template at a harmless placeholder unit during testing so you do not power off your QEMU instance.

Troubleshooting

Issue: A configured detector never appears on D-Bus

Symptom: `busctl tree xyz.openbmc_project.leakdetector` shows no object for a detector you added to Entity Manager.

Cause: The configuration failed validation, or Entity Manager did not publish the `GPIOLeakDetector` interface.

Solution:

1. Confirm Entity Manager exposed the config: `busctl introspect xyz.openbmc_project.EntityManager <path> | grep GPIOLeakDetector`.
2. Check `Type`, `Polarity`, and `Level` are exactly `"GPIOLeakDetector"`, `"Low"` / `"High"`, and `"Warning"` / `"Critical"` — other values are rejected.
3. Review the daemon log: `journalctl -u xyz.openbmc_project.leakdetector`.

Issue: `State` never leaves `Normal` when the sensor wets

Symptom: The leak-sensing cable is triggered but `State` stays `Normal`.

Cause: Wrong `PinName`, or the `Polarity` is inverted relative to the hardware.

Solution:

1. Confirm the line name resolves: `gpioinfo | grep <PinName>`.
2. Read the raw line while the sensor is wet: `gpioget <chip> <PinName>` and compare against the `Polarity` you configured (`Low` = leak drives the line low).
3. Flip `Polarity` between `"Low"` and `"High"` if the electrical sense is reversed.

Issue: Leak logged but no protective action runs

Symptom: `LeakDetectedCritical` is logged but the system does not respond.

Cause: The platform's action template unit is missing or points nowhere useful.

Solution:

1. Check the exact unit name the daemon tried to start: `journalctl -u xyz.openbmc_project.leakdetector | grep -i startUnit` (or watch `systemctl list-jobs` during a test).
2. Confirm the template exists: `systemctl cat 'xyz.openbmc_project.leakdetector.critical.assert@.service'`.
3. Verify its `ExecStart` / `Wants` actually reach your power or mitigation logic, then re-test with the real GPIO.

Debug Commands

```
# Service status and logs
systemctl status xyz.openbmc_project.leakdetector.service
journalctl -u xyz.openbmc_project.leakdetector -f

# Detector state
busctl tree xyz.openbmc_project.leakdetector
busctl introspect xyz.openbmc_project.leakdetector \
    /xyz/openbmc_project/state/leak/detector/<Name>

# Action targets
systemctl list-unit-files | grep leakdetector
systemctl cat 'xyz.openbmc_project.leakdetector.critical.assert@.service'

# Leak log entries via Redfish
curl -sk -u root:0penBmc \
    https://<bmc>/redfish/v1/Systems/system/LogServices/EventLog/Entries \
    | jq '.Members[] | select(.Message | test("[Ll]eak"))'
```

References

Official Resources

- `dbus-sensors` — `leakdetector` source (`LeakDetectionManager` , `LeakGPiODetector` , `LeakEvents`)
- `dbus-sensors` repository
- `xyz.openbmc_project.State.Leak.Detector` interface
- `xyz.openbmc_project.Configuration.GPILeakDetector` interface
- Redfish `LeakDetector` schema

Related Guides

- [D-Bus Sensors](#) — the daemon family and Entity Manager discovery model `leakdetector` shares
- [Entity Manager](#) — how `Exposes` configuration objects reach the daemon
- [GPIO Management](#) — line-names and polarity conventions for the sensor pins
- [Power Management](#) — the power targets a critical leak action typically chains to

Verified against upstream [openbmc/dbus-sensors](#) [src/leakdetector](#) and [openbmc/phosphor-dbus-interfaces](#) .
Daemon, class, event, and configuration identifiers are confirmed against upstream; object paths and platform
action-target contents vary by build — confirm on your image. Last updated: 2026-07-15

NVIDIA GPU Management

Discover and manage NVIDIA GPU, PCIe, and SMA devices over MCTP using the dbus-sensors `nvidiagpusensor` daemon.

Table of Contents

1. [Overview](#)
 2. [Architecture](#)
 1. [Component Diagram](#)
 2. [Discovery Over MCTP](#)
 3. [The MctpRequester](#)
 4. [D-Bus Interfaces](#)
 5. [Key Dependencies](#)
 3. [Configuration](#)
 1. [Entity Manager Configuration](#)
 2. [Build-Time Options](#)
 4. [Telemetry & Control](#)
 1. [Telemetry Sensors](#)
 2. [Power-Cap Control](#)
 3. [Clock-Limit Control](#)
 4. [Inventory](#)
 5. [Asynchronous Events](#)
 5. [Relationship to the pldm / mctpd Stack](#)
 6. [Reading Devices](#)
 1. [Via D-Bus](#)
 2. [Verify the MCTP Endpoint](#)
 7. [Troubleshooting](#)
 1. [No GPU devices appear](#)
 2. [Endpoint discovered but no readings](#)
 3. [Power cap or clock limit write is rejected](#)
 4. [Debug Commands](#)
 8. [References](#)
 1. [Official Resources](#)
 2. [Related Guides](#)
 3. [External Documentation](#)
-

Overview

The **nvidiagpusensor** daemon (source: `src/nvidia-gpu` in `openbmc/dbus-sensors`) manages NVIDIA accelerators — GPUs, PCIe switch/NIC devices, and SMA (System Management API) devices — out of band over

MCTP. It speaks the **OCF accelerator-management protocol** carried in MCTP Vendor Defined Messages (VDM), reading telemetry, exposing inventory, and driving controls without any in-band driver on the host.

Unlike the sysfs/hwmon sensor daemons described in the </openbmc-guide-tutorial/docs/03-core-services/01-dbus-sensors-guide.html>, this daemon has no kernel driver behind it. Each reading is an MCTP round trip: the daemon encodes an OCF VDM request, sends it to the device's endpoint ID (EID), and decodes the response. It relies on `mctpd` to enumerate and route MCTP endpoints, but sends the accelerator-management VDM traffic itself through an `AF_MCTP` socket — it does **not** go through `pldmd`.

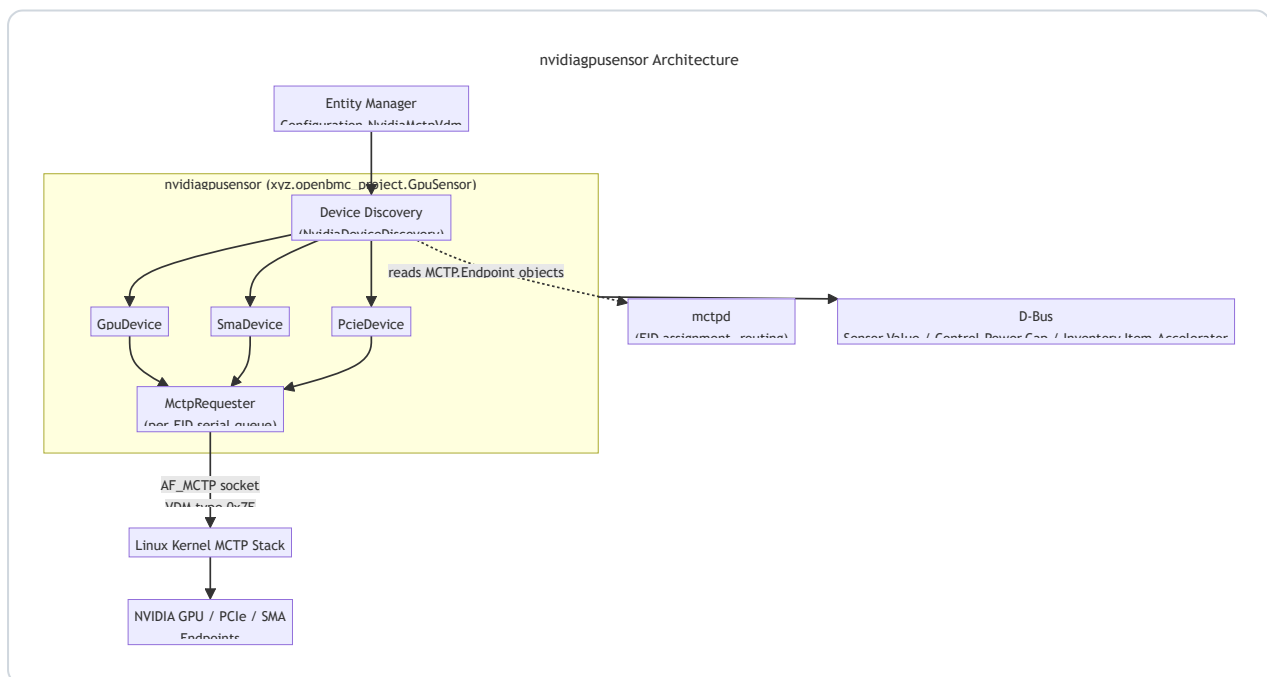
You would use this daemon on platforms built around NVIDIA accelerators (for example NVIDIA GB200/GB300 boards) where the BMC monitors GPU temperature, power, energy, voltage, and utilization; publishes GPU/DIMM inventory to Redfish; and enforces power caps and clock limits over the management fabric.

Key concepts covered:

- MCTP endpoint discovery and the OCF accelerator-management message type (`0x7E`)
- Device instantiation: `GpuDevice`, `SmaDevice`, `PcieDevice`
- The `MctpRequester` — per-EID request serialization, instance-id matching, timeouts, and retries
- Telemetry sensors, power-cap and clock-limit controls, inventory, and asynchronous events
- How this dbus-sensors MCTP path differs from the `pldm` / `mctpd` stack

Architecture

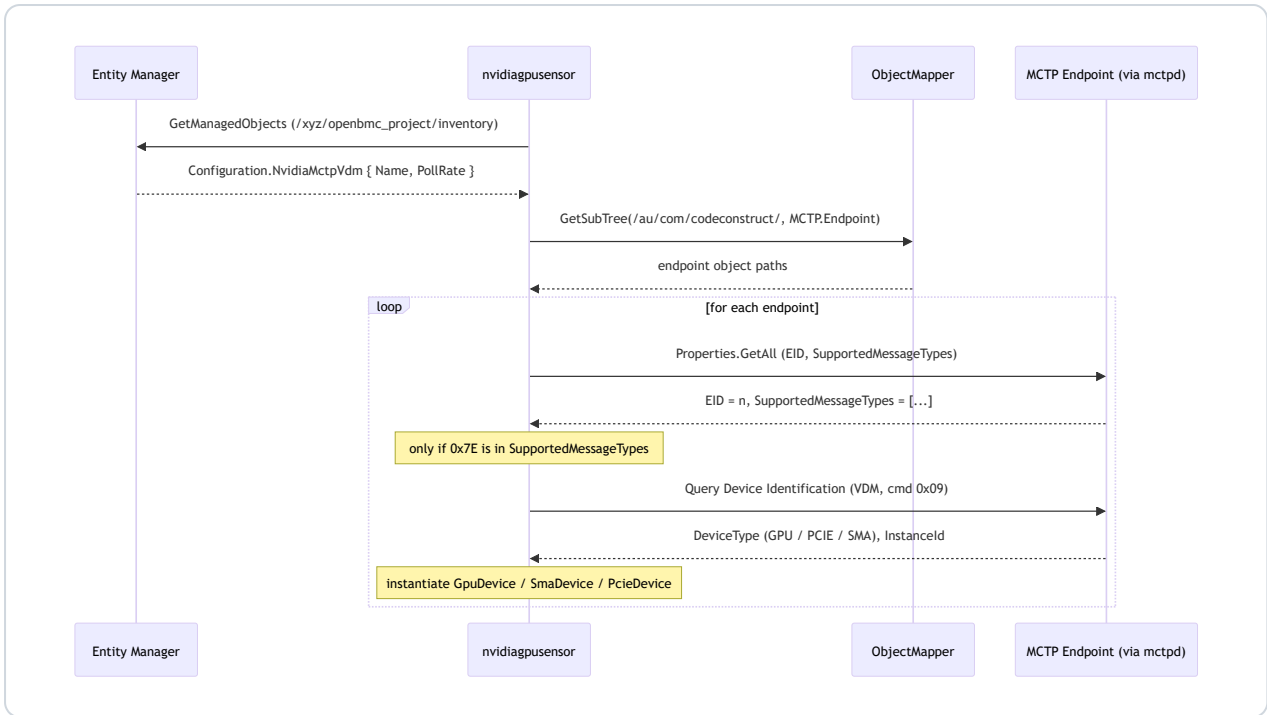
Component Diagram



► ASCII-art version (for comparison)

Discovery Over MCTP

Discovery is a two-stage process. The daemon first learns *which* platforms carry accelerators from Entity Manager, then learns *which endpoints* are actually present from `mctpd`.



Step by step:

- Configuration match.** `createSensors()` calls `GetManagedObjects` on Entity Manager and keeps every object exposing the `xyz.openbmc_project.Configuration.NvidiaMctpVdm` interface. Each yields a `SensorConfigs` record with `Name`, `PollRate` (default `1000` ms), and `NicNetworkPortCount` (default `0`).
- Endpoint enumeration.** For each configuration, `discoverDevices()` asks `xyz.openbmc_project.ObjectMapper` for a `GetSubTree` under `/au/com/codeconstruct/` restricted to the `xyz.openbmc_project.MCTP.Endpoint` interface. These endpoint objects are published by `mctpd` after it assigns EIDs and sets up routing.
- Message-type filter.** For each endpoint, `processEndpoint()` reads the `EID` and `SupportedMessageTypes` properties via `org.freedesktop.DBus.Properties.GetAll`. It proceeds only if `SupportedMessageTypes` contains the OCP accelerator-management message type `0x7E` (`ocp::accelerator_management::messageType`).
- Identification.** `queryDeviceIdentification()` encodes a **Query Device Identification** request (Device Capability Discovery command `0x09`) and sends it via `MctpRequester::sendRecvMsg`. The response carries a `DeviceIdentification` value.
- Instantiation.** Based on the returned device type, the daemon constructs the matching orchestrator and calls its `init()`:

DeviceIdentification	Value	Class	D-Bus name pattern
DEVICE_GPU	0	GpuDevice	Nvidia_GPU_<eid>
DEVICE_PCIE	2	PcieDevice	Nvidia_ConnectX_<eid>
DEVICE_SMA	5	SmaDevice	Nvidia_SMA_<eid>

When Entity Manager later removes the `NvidiaMctpVdm` configuration interface, `interfaceRemoved()` erases the corresponding device objects, so devices are torn down cleanly on reconfiguration.

The MctpRequester

Every device object shares a single `mctp::MctpRequester`, which owns the transport and imposes ordering. Its public surface is a single method:

```
// src/nvidia-gpu/MctpRequester.hpp
void sendRecvMsg(
    uint8_t eid, std::span<const uint8_t> reqMsg,
    std::move_only_function<void(const std::error_code&,
                                std::span<const uint8_t>)> callback);
```

Internals that matter for correct behaviour and debugging:

- **AF_MCTP socket.** The requester holds a `boost::asio::generic::datagram_protocol::socket` bound to MCTP and uses `msgType = 0x7E`. There is no PLDM daemon in the path.
- **Per-EID serialization.** Each EID has its own `EidContext` holding a request queue (`devector<RequestContext>`), a rotating instance id, and a `steady_timer`. Requests to the same endpoint are issued one at a time; a busy device cannot be overrun.
- **Instance-id matching.** `getNextIid()` advances a 5-bit instance id (`instanceMin = 0 ... instanceMax = 31`) per EID. Responses are correlated to the outstanding request by instance id, so a late reply to a superseded request is not mistaken for the current one.
- **Timeouts and retries.** The per-EID timer bounds how long a request may wait for its response; on expiry the requester completes the callback with an error and moves the queue forward, and the OCP protocol layer (for example the inventory reader, below) re-issues where appropriate.

Because reads are asynchronous round trips rather than sysfs reads, each device drives a `steady_timer` poll loop at `PollRate` and issues its requests through this shared requester.

D-Bus Interfaces

Interface	Object Path	Description
<code>xyz.openbmc_project.Sensor.Value</code>	<code>/xyz/openbmc_project/sensors/{temperature,power,energy,voltage}/...</code>	Numeric telemetry readings
<code>xyz.openbmc_project.Common.PhysicalContext</code>	(on each sensor)	Physical context of the reading
<code>xyz.openbmc_project.Metric.Value</code>	<code>/xyz/openbmc_project/metric/...</code>	Utilization metrics (unit: Percent)
<code>xyz.openbmc_project.Control.Power.Cap</code>	<code>/xyz/openbmc_project/control/power/<name></code>	GPU power-cap control
<code>xyz.openbmc_project.Control.OperatingClockSpeed</code>	<code>/xyz/openbmc_project/control/operatingclockspeed/<name></code>	GPU clock-limit control
<code>xyz.openbmc_project.Inventory.Item.Accelerator</code>	<code>/xyz/openbmc_project/inventory/...</code>	GPU inventory item (<code>AcceleratorType.GPU</code>)
<code>xyz.openbmc_project.Inventory.Decorator.Asset</code>	(on inventory item)	Manufacturer, part number, serial number
<code>xyz.openbmc_project.Inventory.Decorator.Revision</code>	(on inventory item)	Firmware/version string
<code>xyz.openbmc_project.Common.UUID</code>	(on inventory item)	Device GUID/UUID
<code>xyz.openbmc_project.Inventory.Item.Dimm</code>	<code>/xyz/openbmc_project/inventory/...</code>	GPU on-package DRAM (<code>DeviceType.HBM</code>)

The daemon owns the well-known name `xyz.openbmc_project.GpuSensor` and adds object managers for `/xyz/openbmc_project/sensors`, `/control`, `/inventory`, `/software`, and `/metric`.

Key Dependencies

- `mctpd`: assigns EIDs, discovers endpoints, and publishes `xyz.openbmc_project.MCTP.Endpoint` objects the daemon consumes. Without it, no endpoints are discovered.
- Kernel MCTP stack** (`AF_MCTP`): carries the VDM traffic. Requires `CONFIG_MCTP` plus the relevant transport driver (for example I2C or PCIe VDM).
- Entity Manager**: provides the `NvidiaMctpVdm` configuration that gates discovery per platform.

Configuration

Entity Manager Configuration

Expose an `NvidiaMctpVdm` record on a board that carries NVIDIA accelerators. The verified schema is `schemas/nvidia.json` in `openbmc/entity-manager`; `Name` and `Type` are required, `PollRate` is optional (milliseconds).

```
{
  "Exposes": [
    {
      "Name": "NVIDIA GB200 GPU",
      "Type": "NvidiaMctpVdm"
    }
  ],
  "Name": "NVIDIA GB200",
  "Type": "Board"
}
```

Property	Type	Required	Description
<code>Name</code>	string	yes	Recognisable device name
<code>Type</code>	string	yes	Must be <code>NvidiaMctpVdm</code>
<code>PollRate</code>	number	no	Poll interval in milliseconds (default <code>1000</code>)
<code>NicNetworkPortCount</code>	number	no	Network port count for NIC/PCIe discovery (default <code>0</code> ; read by the daemon)

`PollRate` here is in **milliseconds**, unlike the `PollRate` on the hwmon-based sensors in the </openbmc-guide-tutorial/docs/03-core-services/01-dbus-sensors-guide.html>, which is expressed in seconds. Double-check units when copying configuration between daemons.

Build-Time Options

The daemon is gated by the `nvidia-gpu` meson feature (default `enabled`).

```
# Building dbus-sensors from source
meson setup build -Dnvidia-gpu=enabled
```

```
# In your machine .conf or image recipe
EXTRA_OEMESON:pn-dbus-sensors = " -Dnvidia-gpu=enabled "
```

Option	Default	Description
<code>nvidia-gpu</code>	enabled	Build the <code>nvidiagpusensor</code> GPU/PCIe/SMA daemon

The executable installs to `/usr/libexec/dbus-sensors/nvidiagpusensor` and ships with the `xyz.openbmc_project.nvidiagpusensor.service` unit, which requires and orders after `xyz.openbmc_project.EntityManager.service`.

Telemetry & Control

All telemetry, control, inventory, and event traffic uses the OCP accelerator-management VDM. Numeric telemetry uses the **Platform Environmental** message type (`gpu::MessageType::PLATFORM_ENVIRONMENTAL = 3`); the command byte selects the reading.

Telemetry Sensors

Reading	Command	Code	Exposed as
GPU temperature	<code>GET_TEMPERATURE_READING</code> (sensor id 0)	<code>0x00</code>	<code>Sensor.Value</code> (temperature)
DRAM temperature	<code>GET_TEMPERATURE_READING</code> (sensor id 1)	<code>0x01</code>	<code>Sensor.Value</code> (temperature)
T.Limit (thermal margin)	<code>READ_THERMAL_PARAMETERS</code> (sensor id 2)	<code>0x02</code>	<code>Sensor.Value</code> + threshold
Power draw	<code>GET_CURRENT_POWER_DRAW</code>	<code>0x03</code>	<code>Sensor.Value</code> (power)
Peak observed power	<code>GET_MAX_OBSERVED_POWER</code>	<code>0x04</code>	<code>Sensor.Value</code> (power)
Energy counter	<code>GET_CURRENT_ENERGY_COUNTER</code>	<code>0x06</code>	<code>Sensor.Value</code> (energy)
Voltage	<code>GET_VOLTAGE</code>	<code>0x0F</code>	<code>Sensor.Value</code> (voltage)
Clock frequency	<code>GET_CURRENT_CLOCK_FREQUENCY</code>	<code>0x0B</code>	<code>Metric.Value</code>
Utilization (GPU + memory)	<code>GET_CURRENT_UTILIZATION</code>	<code>0x07</code>	<code>Metric.Value</code> (Percent)
Violation duration	<code>GET_VIOLATION_DURATION</code>	<code>0x05</code>	<code>Metric.Value</code>

`GpuDevice` polls the temperature, T.Limit, DRAM temperature, power, peak power, energy, and voltage sensors on its poll loop; `SmaDevice` exposes an internal temperature sensor (sensor id 17); `PcieDevice` enumerates PCIe/network ports and exposes their metrics. The temperature sensor is a standard `Sensor` subclass, so warning/critical thresholds and hysteresis work exactly as in the </openbmc-guide-tutorial/docs/03-core-services/01-dbus-sensors-guide.html>.

Power-Cap Control

`NvidiaGpuPowerControl` backs `xyz.openbmc_project.Control.Power.Cap` at `/xyz/openbmc_project/control/power/<name>`:

- **Read** current limits with `GET_POWER_LIMITS` (`0x07`).
- **Write** a new cap with `SET_POWER_LIMITS` (`0x08`), selecting `SetPowerLimitsAction` (`NEW_LIMIT` / `DEFAULT_LIMIT`) and `SetPowerLimitsPersistence` (`ONE_SHOT` / `PERSISTENT`).
- A `PowerCap` and a `PowerCapEnable` write that arrive together (a single Redfish PATCH) are coalesced by a short debounce timer into **one** `SetPowerLimits` request, so the two properties apply atomically.
- The valid cap range comes from inventory (`MIN_DEVICE_POWER_LIMIT` / `MAX_DEVICE_POWER_LIMIT`).

```
# Read the current cap
busctl get-property xyz.openbmc_project.GpuSensor \
    /xyz/openbmc_project/control/power/Nvidia_GPU_<eid> \
    xyz.openbmc_project.Control.Power.Cap PowerCap

# Set a new cap (watts)
busctl set-property xyz.openbmc_project.GpuSensor \
    /xyz/openbmc_project/control/power/Nvidia_GPU_<eid> \
    xyz.openbmc_project.Control.Power.Cap PowerCap u 500
```

Clock-Limit Control

`NvidiaGpuClockSpeedControl` backs `xyz.openbmc_project.Control.OperatingClockSpeed` at `/xyz/openbmc_project/control/operatingclockspeed/<name>`, reading present limits with `GET_CLOCK_LIMIT` (`0x11`). `ClockType` selects `GRAPHICS_CLOCK` or `MEMORY_CLOCK`.

Inventory

The `Inventory` object reads static device identity with `GET_INVENTORY_INFORMATION` (`0x0C`), one `InventoryPropertyId` per request, and maps the results onto D-Bus decorators:

Property	<code>InventoryPropertyId</code>	D-Bus interface / property
Marketing name	<code>MARKETING_NAME</code> (<code>2</code>)	<code>Inventory.Item.Accelerator</code> / Asset model
Board part number	<code>BOARD_PART_NUMBER</code> (<code>0</code>)	<code>Inventory.Decorator.Asset</code> / PartNumber
Serial number	<code>SERIAL_NUMBER</code> (<code>1</code>)	<code>Inventory.Decorator.Asset</code> / SerialNumber
Firmware version	<code>FIRMWARE_VERSION</code> (<code>9</code>)	<code>Inventory.Decorator.Revision</code>
Device GUID	<code>DEVICE_GUID</code> (<code>10</code>)	<code>Common.UUID</code> / UUID

Inventory reads are resilient: each property has a retry budget (`maxRetryAttempts = 3`, `retryDelay = 5s`) so transient MCTP errors during bring-up do not leave inventory permanently blank. The manufacturer is set to NVIDIA and the accelerator type to `AcceleratorType.GPU`.

Asynchronous Events

`NvidiaEventReportingConfig::init()` arms device-pushed events in two steps:

1. **Subscribe.** `SET_EVENT_SUBSCRIPTION` (`0x06`) with `generationSetting = 2` (enable push) and the target BMC EID (default `8`) tells the device where to send events.
2. **Select sources.** For each accelerator message type, `SET_CURRENT_EVENT_SOURCES` (`0x05`) sends a per-type 64-bit mask of the event sources to enable.

Incoming events are dispatched by `NvidiaEventHandler::handleEvent`, keyed on the tuple `(eid, messageType, eventCode)`:

- **XID events** (`PlatformEnvironmentalEvent::XID = 0x01`) are handled by the GPU's XID event handler and surfaced as fault/log entries.
- **Long-running responses** (`DeviceCapabilityDiscoveryEvents::LONG_RUNNING_RESPONSE = 0x02`) let slow commands (for example utilization and violation-duration queries) complete asynchronously through a `SerialQueue` rather than blocking the poll loop.

Relationship to the pldm / mctpd Stack

This is not the PLDM path. The `/openbmc-guide-tutorial/docs/05-advanced/01-mctp-pldm-guide.html` documents the `pldm` daemon and `pldmd` / `mctpd` stack, where the BMC discovers PDRs and reads sensors using **PLDM (MCTP message type 0x01)** through `pldmd`. The `nvidiagpusensor` daemon described here is a **dbus-sensors** daemon that speaks the **OCF accelerator-management VDM (MCTP message type 0x7E, NVIDIA PCI vendor id 0x10de)** directly over an `AF_MCTP` socket via its own `MctpRequester`. Both stacks share `mctpd` and the kernel MCTP layer for endpoint enumeration and routing, but the accelerator-management traffic never traverses `pldmd`, uses no PDRs, and does not appear under `pldmtool`. Use `pldmtool` for PLDM endpoints; use `busctl` against `xyz.openbmc_project.GpuSensor` for these devices.

Aspect	dbus-sensors <code>nvidiagpusensor</code>	<code>pldm</code> / <code>pldmd</code>
MCTP message type	<code>0x7E</code> (Vendor Defined, PCI)	<code>0x01</code> (PLDM)
Protocol	OCF accelerator management (NVIDIA VDM)	PLDM Types 0/2/4/5
Discovery	Query Device Identification (cmd <code>0x09</code>)	GetPDR / terminus discovery
Requester	<code>mctp::MctpRequester</code> (<code>AF_MCTP</code>)	<code>pldmd</code> requester
D-Bus service	<code>xyz.openbmc_project.GpuSensor</code>	<code>xyz.openbmc_project.PLDM</code>
Shared layer	<code>mctpd</code> + kernel MCTP	<code>mctpd</code> + kernel MCTP

Reading Devices

Via D-Bus

```
# The daemon's well-known name
busctl tree xyz.openbmc_project.GpuSensor

# Read a GPU temperature
busctl get-property xyz.openbmc_project.GpuSensor \
    /xyz/openbmc_project/sensors/temperature/Nvidia_GPU_<eid>_Temp \
    xyz.openbmc_project.Sensor.Value Value

# Read GPU inventory (UUID)
busctl get-property xyz.openbmc_project.GpuSensor \
    /xyz/openbmc_project/inventory/.../Nvidia_GPU_<eid> \
    xyz.openbmc_project.Common.UUID UUID
```

Verify the MCTP Endpoint

```
# Confirm mctpd sees the endpoint and it advertises message type 0x7E
busctl tree xyz.openbmc_project.MCTP
busctl introspect xyz.openbmc_project.MCTP \
    /xyz/openbmc_project/mctp/<network>/<eid>
# Look for SupportedMessageTypes containing 126 (0x7E)
```

Troubleshooting

No GPU devices appear

Symptom: `busctl tree xyz.openbmc_project.GpuSensor` shows no sensor or inventory objects.

Cause: discovery stops before instantiation — either no matching Entity Manager configuration, no MCTP endpoint, or the endpoint does not advertise message type `0x7E`.

Solution:

1. Confirm the `NvidiaMctpVdm` configuration is present:

```
busctl call xyz.openbmc_project.EntityManager \
  /xyz/openbmc_project/inventory \
  org.freedesktop.DBus.ObjectManager GetManagedObjects | grep -i NvidiaMctpVdm
```

2. Confirm `mctpd` published the endpoint and it lists `126` in `SupportedMessageTypes`:

```
busctl tree xyz.openbmc_project.MCTP
```

3. Check the daemon log for the discovery messages (“Found OCP MCTP VDM Endpoint”, “Found the GPU with EID ...”):

```
journalctl -u xyz.openbmc_project.nvidiagpusensor -f
```

Endpoint discovered but no readings

Symptom: the device object exists, but `Value` stays `NaN` or requests error out.

Cause: MCTP requests are timing out — bus contention, a slow endpoint, or a wrong EID/route.

Solution:

1. Watch the log for MCTP send/receive failures (“sending message over MCTP failed”).
2. Verify the endpoint responds at the transport layer:

```
mctp addr
mctp route
```

3. Reduce polling pressure by raising `PollRate` (milliseconds) in the Entity Manager configuration.

Power cap or clock limit write is rejected

Symptom: setting `PowerCap` has no effect or logs a `SetPowerLimits` error.

Cause: the requested cap is outside the device’s `MIN_DEVICE_POWER_LIMIT` / `MAX_DEVICE_POWER_LIMIT`, or the device rejected the persistence/action.

Solution:

1. Read the advertised bounds from inventory and stay within them.
2. Set `PowerCapEnable` together with `PowerCap` so the debounced write applies as one `SetPowerLimits`.

Debug Commands

```
# Service status and logs
systemctl status xyz.openbmc_project.nvdiagpusensor
journalctl -u xyz.openbmc_project.nvdiagpusensor -f

# Enumerate everything the daemon exposes
busctl tree xyz.openbmc_project.GpuSensor

# Cross-check the MCTP transport
busctl tree xyz.openbmc_project.MCTP
mctp link ; mctp addr ; mctp route
```

References

Official Resources

- [dbus-sensors](#) — [src/nvidia-gpu](#) — the daemon source
- [entity-manager](#) — [schemas/nvidia.json](#) — the `NvidiaMctpVdm` configuration schema
- [entity-manager](#) — [configurations/nvidia](#) — example platform configurations (GB200, GB300, ...)
- [OpenBMC Documentation](#)

Related Guides

- [D-Bus Sensors Guide](#) — the sensor daemon family and the `Sensor.Value` interface
- [Entity Manager Guide](#) — how configurations drive daemon discovery
- [MCTP & PLDM Guide](#) — the parallel PLDM/`pldm` stack this daemon deliberately bypasses

External Documentation

- [OCP Hardware Management / accelerator management](#) — the accelerator-management message model carried in MCTP VDM
- [Linux MCTP Documentation](#) — the `AF_MCTP` kernel stack

Verified against: [openbmc/dbus-sensors](#) [src/nvidia-gpu](#) (master) and [openbmc/entity-manager](#) [schemas/nvidia.json](#) (master). GPU/PCIe/SMA management requires MCTP-capable NVIDIA hardware and cannot be fully exercised on QEMU. Last updated: 2026-07-15

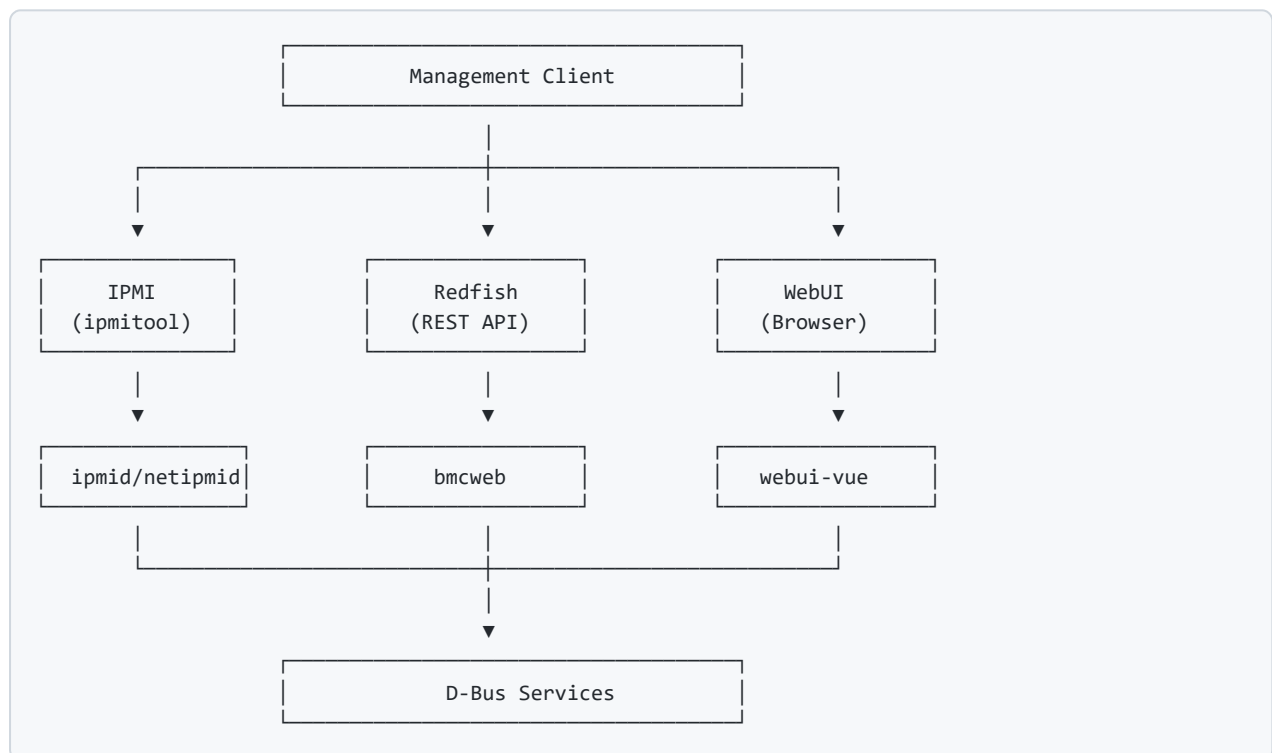
External Interfaces

Learn how to interact with OpenBMC through its management interfaces.

What You'll Learn

- IPMI protocol and OEM command implementation
- Redfish REST API, OEM extensions, and multi-BMC aggregation
- WebUI customization and branding
- Remote access via KVM, virtual media, and console
- SSH access and security hardening

Interface Overview



Guides in This Section

Management Protocols

Guide	Description	Use Case
IPMI	IPMI protocol, OEM commands	Legacy management, scripting
Redfish	REST API, modern management	Cloud integration, automation
WebUI	Browser-based management	Human operators
Redfish Events & Telemetry	SSE subscriptions, metric reports	Monitoring, alerting
Redfish OEM Extensions	Custom OEM resources and actions	Vendor-specific features

Remote Access

Guide	Description	Use Case
KVM	Remote keyboard/video/mouse	OS installation, troubleshooting
Virtual Media	Remote ISO/image mounting	OS installation
Console	Serial over LAN	Boot monitoring, recovery

Security

Guide	Description
SSH	SSH access and hardening

Which Interface to Use?

Scenario	Recommended Interface
Scripting / Automation	Redfish (REST)
Legacy tools / ipmitool	IPMI
Human operators	WebUI
Cloud management platforms	Redfish
Bulk operations	Redfish with scripting

Prerequisites

- Complete [Core Services](#) basics
- Understand D-Bus object model
- Working QEMU environment with network access

Table of contents

- [IPMI Guide](#)
- [Redfish Guide](#)
- [WebUI Guide](#)
- [KVM Guide](#)
- [Virtual Media Guide](#)
- [Console Guide](#)
- [SSH Security Guide](#)
- [Redfish Events & Telemetry](#)
- [Redfish OEM Extensions](#)

IPMI Guide

Implement and extend IPMI functionality in OpenBMC.

Table of Contents

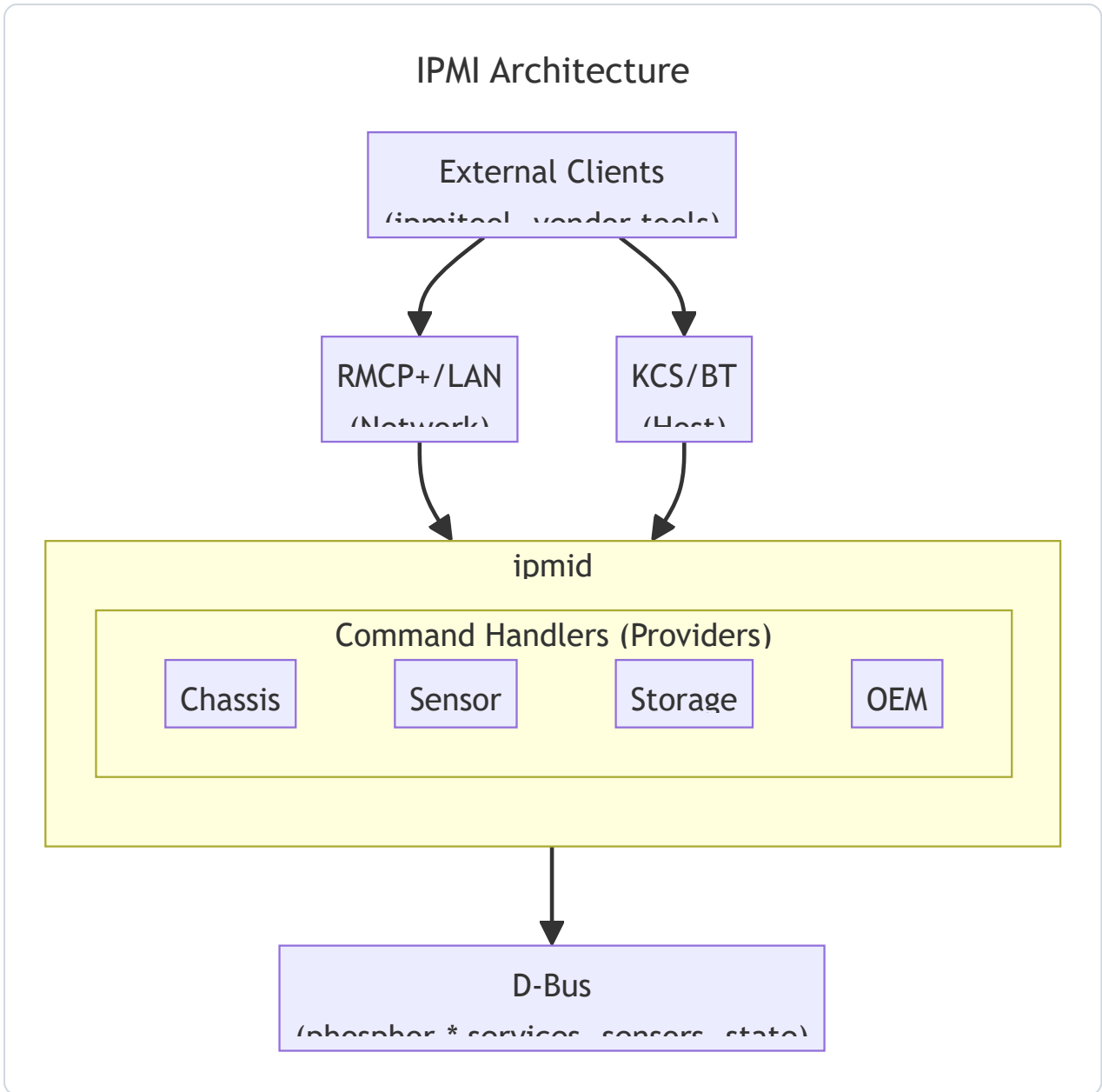
1. [Overview](#)
2. [Setup & Configuration](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [Meson Build Options](#)
 3. [Runtime Enable/Disable](#)
 4. [LAN Channel Configuration](#)
 5. [User Configuration](#)
 6. [SOL \(Serial Over LAN\) Configuration](#)
 7. [Cipher Suite Configuration](#)
 8. [Watchdog Configuration](#)
3. [IPMI Basics](#)
 1. [Network Functions \(NetFn\)](#)
 2. [Command Format](#)
4. [Using ipmitool](#)
 1. [Local Access \(KCS\)](#)
 2. [Remote Access \(LAN\)](#)
5. [ipmid Architecture](#)
 1. [Provider Libraries](#)
 2. [Handler Registration](#)
6. [Implementing OEM Commands](#)
 1. [OEM Command Structure](#)
 2. [Basic OEM Handler](#)
 3. [Building OEM Provider](#)
 4. [BitBake Recipe](#)
7. [Testing OEM Commands](#)
 1. [Using ipmitool raw](#)
 2. [Debugging](#)
8. [Common IPMI Operations](#)
 1. [Chassis Commands](#)
 2. [Sensor Commands](#)
9. [Host IPMI \(KCS/BT\)](#)
 1. [Configuration](#)
 2. [systemd Service](#)
10. [Host IPMI over SSIF](#)

1. [Get System Interface Capabilities](#)
 2. [Response Timer](#)
 3. [Runtime Status](#)
 11. [LAN Configuration](#)
 1. [Via D-Bus](#)
 2. [Via ipmitool](#)
 12. [Troubleshooting](#)
 1. [Service Not Starting](#)
 2. [Command Not Recognized](#)
 3. [Authentication Failures](#)
 13. [Deep Dive](#)
 1. [Command Handler Registration](#)
 2. [Message Flow Through KCS Interface](#)
 3. [RMCP+ Session Authentication](#)
 4. [SDR \(Sensor Data Record\) Structure](#)
 5. [OEM Command Registration Pattern](#)
 6. [Source Code Reference](#)
 14. [Examples](#)
 15. [References](#)
-

Overview

IPMI (Intelligent Platform Management Interface) is a standardized interface for out-of-band server management. OpenBMC implements IPMI through:

- **ipmid**: The main IPMI daemon
- **Host IPMI**: KCS/BT interface to the host
- **Host IPMI over SSIF**: SMBus System Interface — a third host transport, bridged by `ssifbridge`
- **Network IPMI**: RMCP/RMCP+ over LAN



► ASCII-art version (for comparison)

Setup & Configuration

Build-Time Configuration (Yocto)

Include IPMI components in your image:

```
# In your machine .conf or image recipe

# Include IPMI host daemon (required)
IMAGE_INSTALL:append = " phosphor-ipmi-host"

# Include network IPMI (RMCP+)
IMAGE_INSTALL:append = " phosphor-ipmi-net"

# Include IPMI flash support
IMAGE_INSTALL:append = " phosphor-ipmi-flash"

# Include IPMI FRU support
IMAGE_INSTALL:append = " phosphor-ipmi-fru"

# Exclude components you don't need
IMAGE_INSTALL:remove = "phosphor-ipmi-flash"
```

Meson Build Options

```
# phosphor-ipmi-host options
meson setup build \
    -Dboot-flag-safe-mode-support=enabled \
    -Di2c-whitelist-check=disabled \
    -Dshort-sample-enable=enabled \
    -Dsoftoff=enabled

# phosphor-ipmi-net options
meson setup build \
    -Dpam=enabled \
    -Drmc-ping=enabled
```

Component	Option	Description
host	<code>boot-flag-safe-mode-support</code>	Safe mode boot support
host	<code>i2c-whitelist-check</code>	I2C command whitelisting
host	<code>softoff</code>	Soft power off support
net	<code>pam</code>	PAM authentication
net	<code>rmcp-ping</code>	RMCP ping support

Runtime Enable/Disable

```
# Check IPMI services
systemctl status phosphor-ipmi-host
systemctl status phosphor-ipmi-net

# Disable network IPMI (security hardening)
systemctl stop phosphor-ipmi-net
systemctl disable phosphor-ipmi-net

# Re-enable
systemctl enable phosphor-ipmi-net
systemctl start phosphor-ipmi-net

# Restart after config change
systemctl restart phosphor-ipmi-host
```

LAN Channel Configuration

Configure IPMI LAN settings:

```
# Get current LAN config
ipmitool lan print 1

# Set static IP
ipmitool lan set 1 ipsrc static
ipmitool lan set 1 ipaddr 192.168.1.100
ipmitool lan set 1 netmask 255.255.255.0
ipmitool lan set 1 defgw ipaddr 192.168.1.1

# Enable DHCP
ipmitool lan set 1 ipsrc dhcp

# Set authentication types
ipmitool lan set 1 auth admin md5

# Enable/disable LAN access
ipmitool lan set 1 access on
ipmitool lan set 1 access off
```

User Configuration

```
# List users
ipmitool user list 1

# Create user
ipmitool user set name 2 operator
ipmitool user set password 2 "SecurePass123!"
ipmitool user enable 2
ipmitool user priv 2 3 1 # Operator privilege on channel 1

# Disable user
ipmitool user disable 2

# Set privilege levels
# 1=Callback, 2=User, 3=Operator, 4=Administrator
ipmitool user priv 2 4 1
```

SOL (Serial Over LAN) Configuration

```
# Enable SOL
ipmitool sol set enabled true 1
ipmitool sol set privilege-level admin 1
ipmitool sol set force-payload-auth true 1

# Set baud rate
ipmitool sol set volatile-bit-rate 115.2 1
ipmitool sol set non-volatile-bit-rate 115.2 1

# Connect to SOL
ipmitool -I lanplus -H <bmc-ip> -U root -P 0penBmc sol activate
```

Cipher Suite Configuration

Control encryption and authentication:

```
# View cipher suites
ipmitool lan print 1 | grep Cipher

# Disable weak cipher suites (security hardening)
# Cipher 0 = no auth, no encryption (insecure)
# Cipher 1 = auth only, no encryption
# Cipher 3 = auth + encryption (recommended minimum)
# Cipher 17 = strongest (AES-CBC-128, HMAC-SHA256)
```

Watchdog Configuration

```
# Get watchdog status
ipmitool mc watchdog get

# Set watchdog timeout (60 seconds)
ipmitool mc watchdog set timeout 60

# Reset watchdog
ipmitool mc watchdog reset

# Disable watchdog
ipmitool mc watchdog off
```

IPMI Basics

Network Functions (NetFn)

NetFn	Name	Description
0x00/0x01	Chassis	Power control, boot options
0x04/0x05	Sensor/Event	Sensor readings, event messages
0x06/0x07	App	Device info, watchdog, sessions
0x0A/0x0B	Storage	SEL, SDR, FRU access
0x0C/0x0D	Transport	LAN configuration
0x2C/0x2D	Group	OEM group extensions
0x2E/0x2F	OEM/Group	Vendor-specific commands
0x30-0x3F	OEM	Vendor-specific (by IANA)

Command Format

```
Request:
  [NetFn/LUN] [Command] [Data...]

Response:
  [NetFn/LUN] [Command] [Completion Code] [Data...]
```

Using ipmitool

Local Access (KCS)

```
# On the BMC
ipmitool raw 0x06 0x01    # Get Device ID

# Using the BMC driver
ipmitool -I open chassis status
```

Remote Access (LAN)

```
# From a remote machine
ipmitool -I lanplus -H <bmc-ip> -U root -P 0penBmc chassis status

# Power commands
ipmitool -I lanplus -H <bmc-ip> -U root -P 0penBmc chassis power on
ipmitool -I lanplus -H <bmc-ip> -U root -P 0penBmc chassis power off
ipmitool -I lanplus -H <bmc-ip> -U root -P 0penBmc chassis power cycle

# Sensor reading
ipmitool -I lanplus -H <bmc-ip> -U root -P 0penBmc sensor list

# FRU information
ipmitool -I lanplus -H <bmc-ip> -U root -P 0penBmc fru print

# System Event Log
ipmitool -I lanplus -H <bmc-ip> -U root -P 0penBmc sel list
```

ipmid Architecture

Provider Libraries

Command handlers are implemented as shared libraries (providers):

```
/usr/lib/ipmid-providers/
├─ libchassishandler.so
├─ libsensordhandler.so
├─ libstoragehandler.so
├─ libapphandler.so
├─ libuserhandler.so
└─ liboemhandler.so
```

Handler Registration

```
#include <ipmid/api.hpp>

// Register a command handler
void registerHandler()
{
    ipmi::registerHandler(
        ipmi::prioOemBase,           // Priority
        ipmi::netFnChassis,         // Network function
        ipmi::chassis::cmdGetStatus, // Command
        ipmi::Privilege::User,      // Required privilege
        ipmiGetChassisStatus);      // Handler function
}

// Handler function signature
ipmi::RspType<...> ipmiGetChassisStatus()
{
    // Implementation
    return ipmi::responseSuccess(...);
}
```


Implementing OEM Commands

OEM Command Structure

OEM commands use NetFn 0x2E (OEM/Group) or 0x30-0x3F (OEM by IANA).

Basic OEM Handler

Source Reference: Pattern based on [phosphor-host-ipmid](#)

- API: [ipmid/api.hpp](#)
- Example handlers: [chassishandler.cpp](#)

```

#include <ipmid/api.hpp>
#include <ipmid/utils.hpp>
#include <phosphor-logging/log.hpp>

using namespace phosphor::logging;

// OEM Network Function (use your IANA enterprise number)
constexpr ipmi::NetFn netFnOem = static_cast<ipmi::NetFn>(0x30);

// OEM Commands
constexpr uint8_t cmdOemGetVersion = 0x01;
constexpr uint8_t cmdOemSetLed = 0x02;

// Handler: Get OEM Version
ipmi::RspType<uint8_t, uint8_t, uint8_t>
ipmiOemGetVersion()
{
    uint8_t major = 1;
    uint8_t minor = 0;
    uint8_t patch = 0;

    return ipmi::responseSuccess(major, minor, patch);
}

// Handler: Set LED state
ipmi::RspType<> ipmiOemSetLed(uint8_t ledId, uint8_t state)
{
    if (ledId > 3)
    {
        return ipmi::responseParmOutOfRange();
    }

    log<level::INFO>("OEM Set LED",
        entry("LED=%d", ledId),
        entry("STATE=%d", state));

    // Implementation via D-Bus
    auto bus = sd_busplus::bus::new_default();
    auto method = bus.new_method_call(
        "xyz.openbmc_project.LED.GroupManager",
        "/xyz/openbmc_project/led/groups/identify",
        "org.freedesktop.DBus.Properties",
        "Set");

    method.append("xyz.openbmc_project.Led.Group", "Asserted");
    method.append(std::variant<bool>(state == 1));

    try
    {
        bus.call(method);
    }
    catch (const std::exception& e)
    {
        return ipmi::responseUnspecifiedError();
    }

    return ipmi::responseSuccess();
}

// Register handlers
void registerOemHandlers()
{
    ipmi::registerHandler(

```

```

        ipmi::prioOemBase,
        netFnOem,
        cmdOemGetVersion,
        ipmi::Privilege::User,
        ipmiOemGetVersion);

    ipmi::registerHandler(
        ipmi::prioOemBase,
        netFnOem,
        cmdOemSetLed,
        ipmi::Privilege::Operator,
        ipmiOemSetLed);
}

```

Building OEM Provider

```

# CMakeLists.txt
cmake_minimum_required(VERSION 3.5)
project(myoem-ipmi CXX)

set(CMAKE_CXX_STANDARD 20)

find_package(PkgConfig REQUIRED)
pkg_check_modules(IPMI REQUIRED libipmid)
pkg_check_modules(SDBUSPLUS REQUIRED sdbusplus)
pkg_check_modules(PHOSPHOR_LOGGING REQUIRED phosphor-logging)

add_library(myoemhandler SHARED oem_handler.cpp)

target_include_directories(myoemhandler PRIVATE
    ${IPMI_INCLUDE_DIRS}
    ${SDBUSPLUS_INCLUDE_DIRS}
    ${PHOSPHOR_LOGGING_INCLUDE_DIRS}
)

target_link_libraries(myoemhandler
    ${IPMI_LIBRARIES}
    ${SDBUSPLUS_LIBRARIES}
    ${PHOSPHOR_LOGGING_LIBRARIES}
)

install(TARGETS myoemhandler
    LIBRARY DESTINATION lib/ipmid-providers
)

```

BitBake Recipe

```
# myoem-ipmi_git.bb
SUMMARY = "My OEM IPMI Commands"
LICENSE = "Apache-2.0"
LIC_FILES_CHKSUM = "file://LICENSE;md5=..."

inherit cmake pkgconfig

DEPENDS += "sdbusplus phosphor-logging phosphor-ipmi-host"

SRC_URI = "git://github.com/myorg/myoem-ipmi.git;branch=main;protocol=https"
SRCREV = "..."

S = "${WORKDIR}/git"

HOSTIPMI_PROVIDER_LIBRARY += "libmyoemhandler.so"
```

Testing OEM Commands

Using ipmitool raw

```
# Test OEM Get Version (NetFn 0x30, Cmd 0x01)
ipmitool -I lanplus -H <bmc-ip> -U root -P 0penBmc raw 0x30 0x01
# Response: 01 00 00 (version 1.0.0)

# Test OEM Set LED (NetFn 0x30, Cmd 0x02, LED 0, ON)
ipmitool -I lanplus -H <bmc-ip> -U root -P 0penBmc raw 0x30 0x02 0x00 0x01
```

Debugging

```
# Enable ipmid debug logging
systemctl stop phosphor-ipmi-host
/usr/bin/ipmid -v

# Check handler registration
journalctl -u phosphor-ipmi-host | grep -i register
```

Common IPMI Operations

Chassis Commands

```
// Get Chassis Status
ipmi::RspType<uint8_t, uint8_t, uint8_t>
    ipmiGetChassisStatus()
{
    // Read power state from D-Bus
    auto bus = sdbusplus::bus::new_default();
    auto powerState = ipmi::getProperty<std::string>(
        bus,
        "xyz.openbmc_project.State.Chassis",
        "/xyz/openbmc_project/state/chassis0",
        "xyz.openbmc_project.State.Chassis",
        "CurrentPowerState");

    bool powerOn = (powerState.find("On") != std::string::npos);

    uint8_t currentPowerState = powerOn ? 0x01 : 0x00;
    uint8_t lastPowerEvent = 0x00;
    uint8_t miscState = 0x00;

    return ipmi::responseSuccess(
        currentPowerState, lastPowerEvent, miscState);
}
```

Sensor Commands

```
// Get Sensor Reading
ipmi::RspType<uint8_t, uint8_t, uint8_t>
    ipmiGetSensorReading(uint8_t sensorNumber)
{
    // Look up sensor path from SDR
    auto sensorPath = getSensorPath(sensorNumber);

    // Read value from D-Bus
    auto bus = sdbusplus::bus::new_default();
    auto value = ipmi::getProperty<double>(
        bus,
        "xyz.openbmc_project.HwmonTempSensor",
        sensorPath,
        "xyz.openbmc_project.Sensor.Value",
        "Value");

    // Convert to IPMI format
    uint8_t reading = static_cast<uint8_t>(value);
    uint8_t status = 0x40; // Scanning enabled
    uint8_t thresholdStatus = 0x00;

    return ipmi::responseSuccess(
        reading, status, thresholdStatus);
}
```

Host IPMI (KCS/BT)

Configuration

The host interface is configured via device tree:

```
&kcs3 {
    status = "okay";
    aspeed,lpc-io-reg = <0xCA2>;
};
```

systemd Service

```
# Check host IPMI service
systemctl status phosphor-ipmi-host

# View host IPMI messages
journalctl -u phosphor-ipmi-host -f
```

Host IPMI over SSIF

SSIF (SMBus System Interface) is a third host-side IPMI transport, alongside KCS and BT. Where KCS and BT ride the LPC bus, SSIF carries IPMI messages over an SMBus/I2C link between the host and the BMC — useful on platforms that expose the BMC as an SMBus slave rather than through an LPC KCS/BT channel.

On OpenBMC this transport is handled by **ssifbridge** (the `ssifbridged` daemon), which sits between the kernel SSIF host device and the same IPMI command dispatch that `ipmid` serves:

- It reads inbound IPMI requests from the SSIF character device `/dev/ipmi-ssif-host`.
- For each request it forwards the NetFn, LUN, and Command (plus data) to the `execute` method of the `xyz.openbmc_project.Ipmi.Server` interface, on the service `xyz.openbmc_project.Ipmi.Host`. This is the same command-handler dispatch that KCS and BT feed, so your provider libraries and OEM handlers work unchanged (see [ipmid Architecture](#)).
- When the D-Bus reply returns, the bridge translates the response (completion code plus data) back into an SSIF response and writes it to the device.

Get System Interface Capabilities

One command is answered **directly by the bridge** rather than being forwarded to `ipmid`: **Get System Interface Capabilities** (NetFn `0x06`, Cmd `0x57`). The bridge reports the SSIF interface type and the maximum transmit/receive message sizes it supports, so the host learns how to frame subsequent SSIF traffic before any command reaches the D-Bus dispatch.

Response Timer

Because an SMBus master will not wait indefinitely for a slow slave, `ssifbridge` arms a **response timer** when it forwards a request. If the D-Bus reply has not arrived by the time the timer expires, the bridge returns a response with completion code `0xce` ("Command response could not be provided") so the SSIF transaction can still complete. A real (late) D-Bus reply that arrives afterward is dropped, so the host never sees a duplicate response for the same request.

Because SSIF ultimately reaches the same `xyz.openbmc_project.Ipmi.Server` dispatch as KCS and BT, command handlers, privileges, and OEM providers behave identically across all three host transports.

Runtime Status

```
# Check the SSIF bridge service
systemctl status ssifbridge

# Follow bridge activity (forwarding, capability replies, timeouts)
journalctl -u ssifbridge -f

# Confirm the SSIF host character device is present
ls -l /dev/ipmi-ssif-host
```

LAN Configuration

Via D-Bus

```
# Get LAN channel settings
busctl introspect xyz.openbmc_project.Ipmi.Channel.eth0 \
    /xyz/openbmc_project/network/eth0

# Get authentication settings
busctl get-property xyz.openbmc_project.User.Manager \
    /xyz/openbmc_project/user \
    xyz.openbmc_project.User.AccountPolicy MaxLoginAttemptBeforeLockout
```

Via ipmitool

```
# Get LAN configuration
ipmitool lan print 1

# Set IP address
ipmitool lan set 1 ipaddr 192.168.1.100

# Set gateway
ipmitool lan set 1 defgw ipaddr 192.168.1.1
```

Troubleshooting

Service Not Starting

```
# Check ipmid status
systemctl status phosphor-ipmi-host
journalctl -u phosphor-ipmi-host -n 50

# Check for provider loading errors
journalctl -u phosphor-ipmi-host | grep -i "error\\|fail"
```

Command Not Recognized

```
# Verify handler is registered
journalctl -u phosphor-ipmi-host | grep -i "register"

# Check provider library loaded
ls -la /usr/lib/ipmid-providers/
```

Authentication Failures

```
# Check RMCP+ configuration
busctl tree xyz.openbmc_project.Ipmi.Channel.eth0

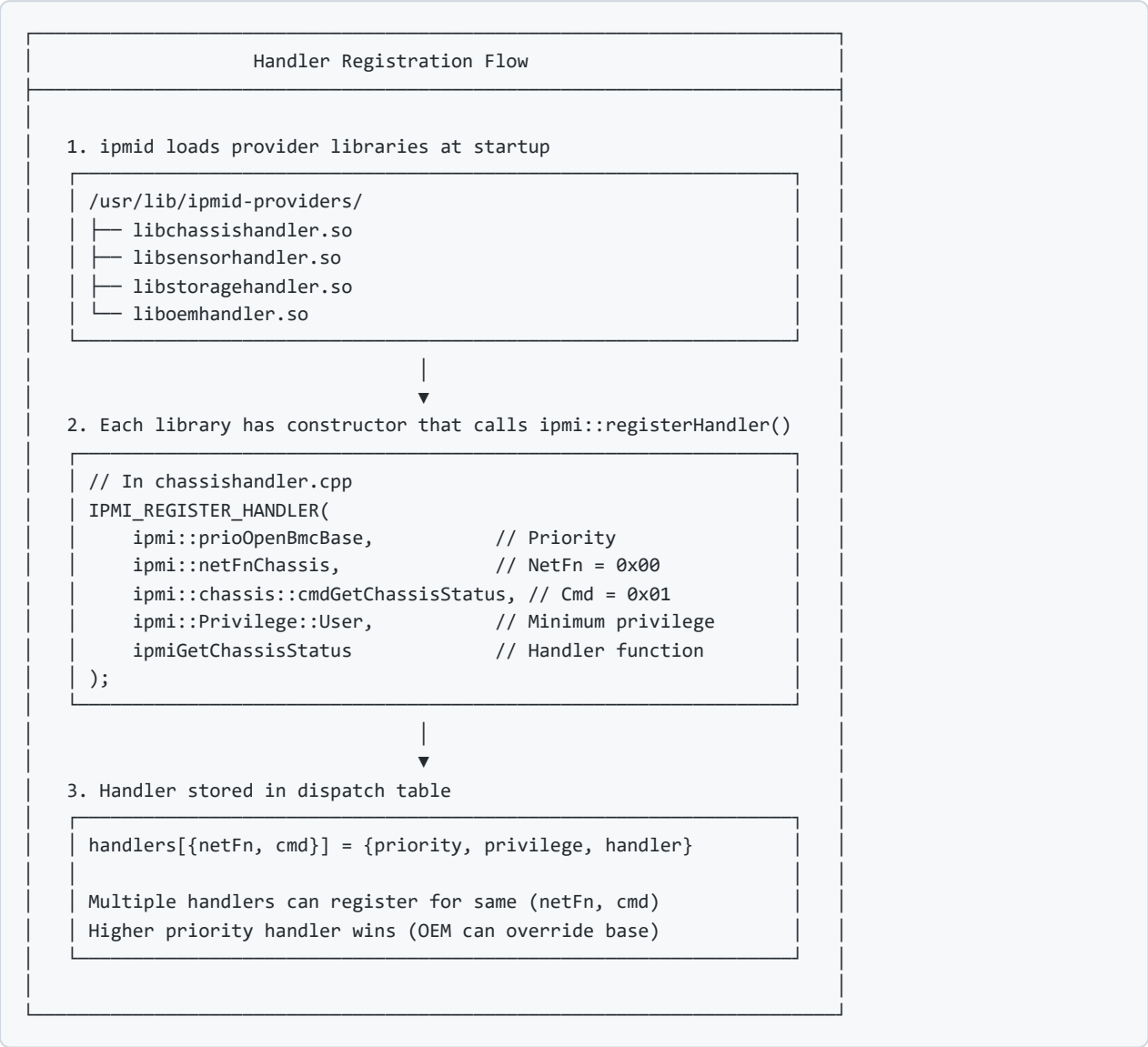
# Verify user credentials
ipmitool user list 1
```

Deep Dive

Advanced implementation details for IPMI developers.

Command Handler Registration

IPMI commands are handled by provider libraries that register handlers at startup:

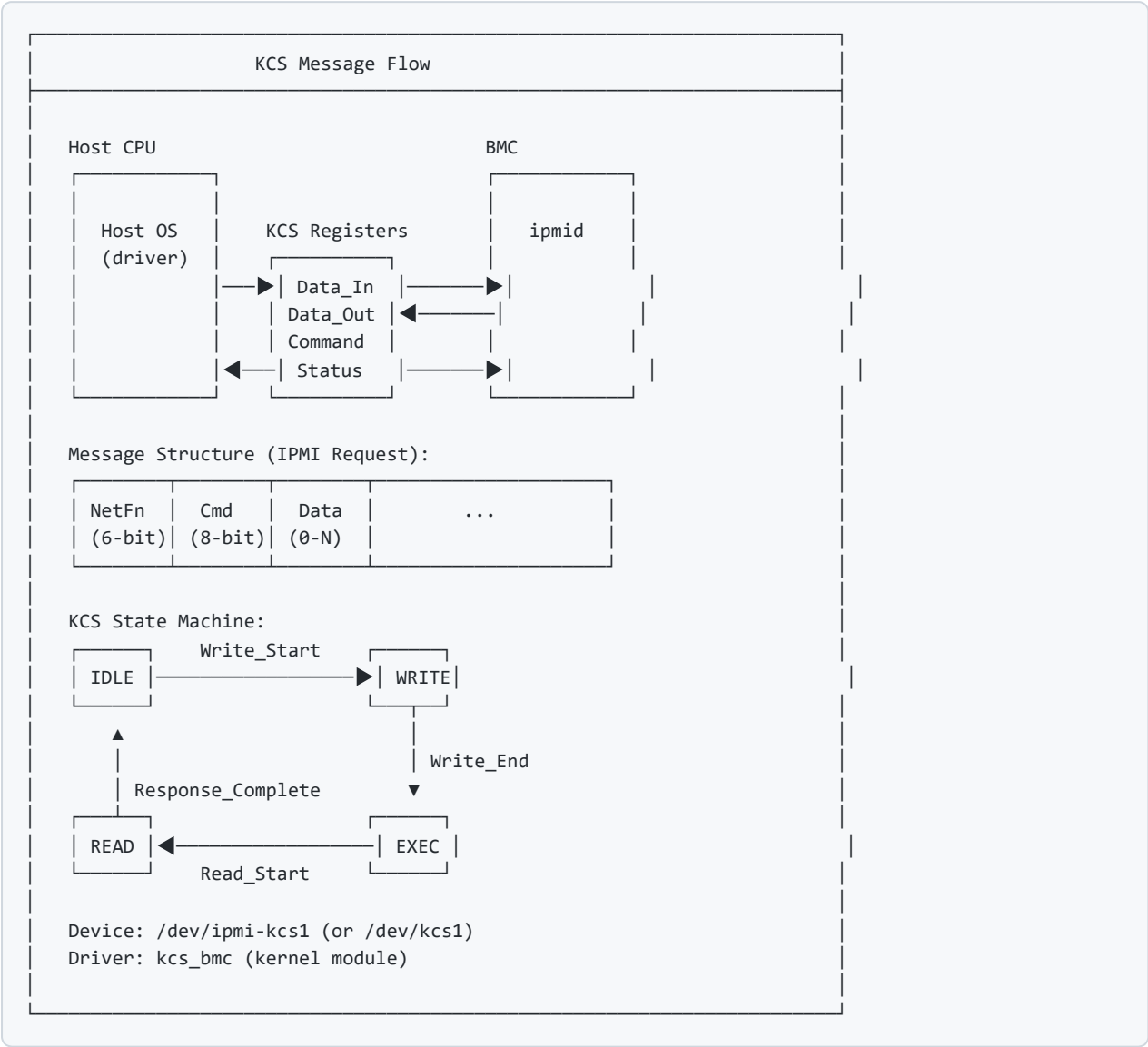


Priority levels (higher wins):

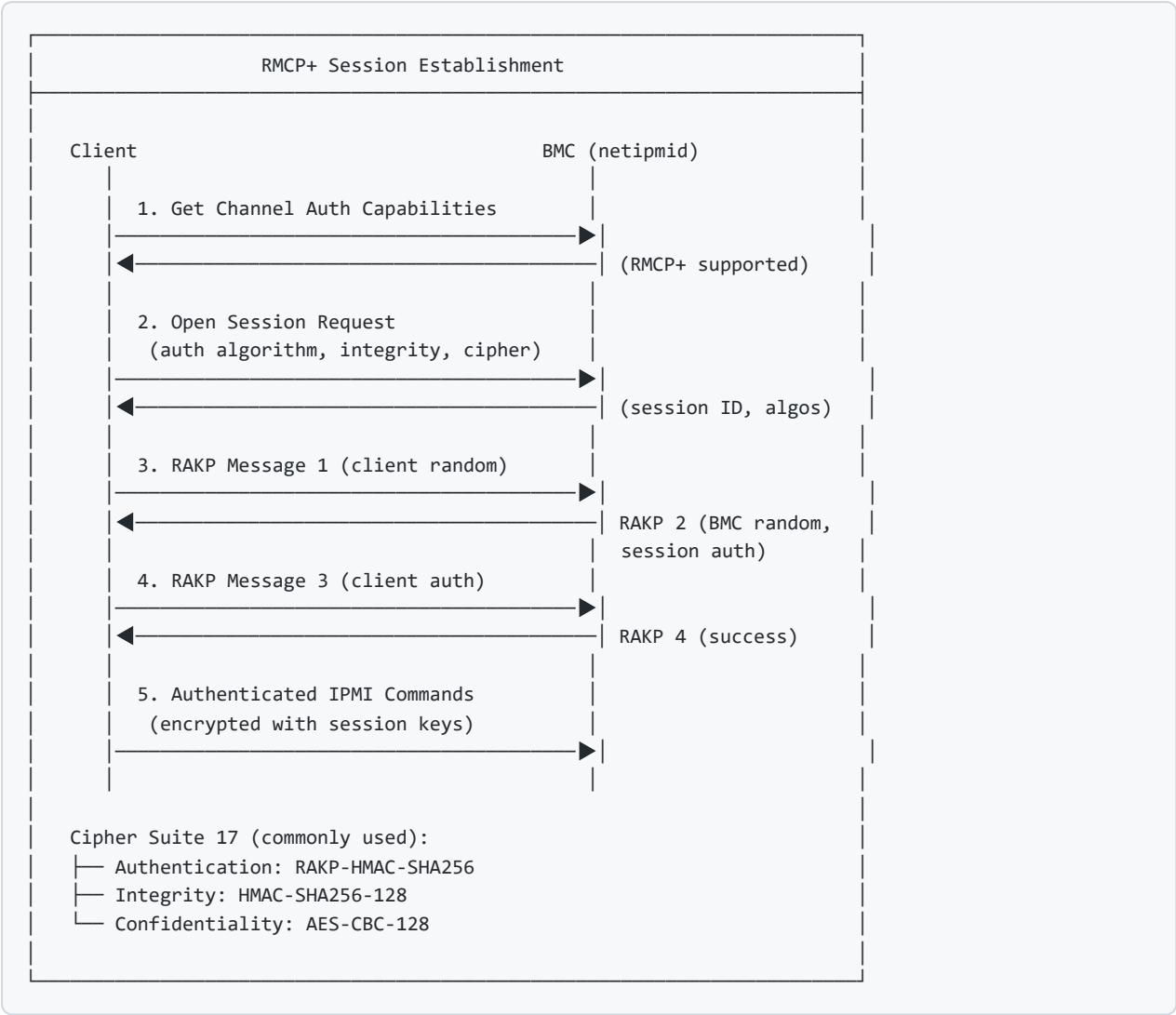
Priority	Value	Use Case
<code>prioOpenBmcBase</code>	10	Default OpenBMC handlers
<code>prioOemBase</code>	20	OEM-specific overrides
<code>prioMax</code>	40	Highest priority handlers

Source reference: [ipmid/api.hpp](#)

Message Flow Through KCS Interface



RMCP+ Session Authentication



SDR (Sensor Data Record) Structure

SDR Record Types

SDR Type 0x01 - Full Sensor Record (43+ bytes):

Offset	Field	Size	Description
0-1	Record ID	2	Unique identifier
2	SDR Version	1	0x51 = IPMI 1.5
3	Record Type	1	0x01 = Full
4	Record Length	1	Bytes following
5	Sensor Owner ID	1	I2C address/LUN
6	Sensor Owner LUN	1	Channel/LUN
7	Sensor Number	1	0-255
8	Entity ID	1	CPU, memory, etc.
9	Entity Instance	1	Which instance
10	Sensor Initialization	1	Flags
11	Sensor Capabilities	1	Threshold support
12	Sensor Type	1	Temperature, etc.
13	Event/Reading Type	1	Threshold/discrete
14-15	Assertion Event Mask	2	Events to assert
16-17	Deassertion Event Mask	2	Events to deassert
18-19	Discrete Reading Mask	2	Readable states
20	Sensor Units 1	1	Unit modifiers
21	Sensor Units 2 (Base)	1	Degrees C, Volts
22	Sensor Units 3 (Mod)	1	Modifier unit
23	Linearization	1	Linear, log, etc.
24-25	M, M Tolerance	2	Scaling: M
26-27	B, B Accuracy	2	Scaling: B
28	Accuracy/Direction	1	Acc exp, direction
29	R exp, B exp	1	Exponents
...	Thresholds, ID string	...	

Reading Conversion:

$$y = (M \times \text{raw} + B \times 10^{\text{Bexp}}) \times 10^{\text{Rexp}}$$

OEM Command Registration Pattern

```
// Example: Custom OEM command handler
#include <ipmid/api.hpp>

// Define OEM NetFn (0x30-0x3F are OEM)
constexpr ipmi::NetFn oemNetFn = 0x30;
constexpr ipmi::Cmd oemGetInfo = 0x01;

// Handler function
ipmi::RspType<uint8_t, // version
             std::string> // description
oemGetInfoHandler(ipmi::Context::ptr ctx)
{
    // Access D-Bus if needed
    auto bus = getSdBus();

    // Return success with data
    return ipmi::responseSuccess(
        uint8_t{0x01}, // version
        std::string{"My OEM"} // description
    );
}

// Register at library load
void registerOemHandlers() __attribute__((constructor));
void registerOemHandlers()
{
    ipmi::registerHandler(
        ipmi::prioOemBase,
        oemNetFn,
        oemGetInfo,
        ipmi::Privilege::User,
        oemGetInfoHandler
    );
}
```

Source Code Reference

Key implementation files in [phosphor-host-ipmid](#):

File	Description
ipmid.cpp	Main daemon, message dispatch
chassishandler.cpp	Chassis commands (power, identify)
sensorhandler.cpp	Sensor reading, thresholds, SDR
storagehandler.cpp	FRU, SEL, SDR repository
apphandler.cpp	Application commands (device ID)
include/ipmid/api.hpp	Handler registration API
include/ipmid/types.hpp	Type definitions

Examples

Working examples are available in the [examples/ipmi](#) directory:

- `oem_handler.cpp` - OEM IPMI command handler
 - `oem_handler.hpp` - OEM handler header
 - `meson.build` / `CMakeLists.txt` - Build configuration
 - `myoem-ipmi.bb` - BitBake recipe
-

References

- [phosphor-host-ipmid](#) - D-Bus based IPMI daemon
 - [ipmid API Headers](#)
 - [IPMI Specification](#)
 - [OpenBMC IPMI Design](#)
-

Tested on: OpenBMC master, QEMU romulus

Redfish Guide

Configure and extend the Redfish API in OpenBMC.

Table of Contents

1. [Overview](#)
2. [Setup & Configuration](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [Meson Build Options Reference](#)
 3. [Runtime Enable/Disable](#)
 4. [HTTPS Certificate Configuration](#)
 5. [Network Protocol Settings](#)
 6. [Session Configuration](#)
 7. [Account Service Configuration](#)
 8. [Event Service Configuration](#)
 9. [Telemetry Configuration](#)
3. [Redfish Basics](#)
 1. [API Structure](#)
 2. [Common Operations](#)
4. [Authentication](#)
 1. [Basic Authentication](#)
 2. [Session Authentication](#)
 3. [mTLS Authentication](#)
5. [Power Operations](#)
 1. [Get Power State](#)
 2. [Power On/Off](#)
 3. [Reset Types](#)
6. [Sensors](#)
 1. [List All Sensors](#)
 2. [Get Sensor Details](#)
 3. [Sensor Properties](#)
7. [Thermal Management](#)
 1. [Get Thermal Info](#)
 2. [Response Structure](#)
8. [User Management](#)
 1. [List Users](#)
 2. [Create User](#)
 3. [Modify User](#)
 4. [Delete User](#)

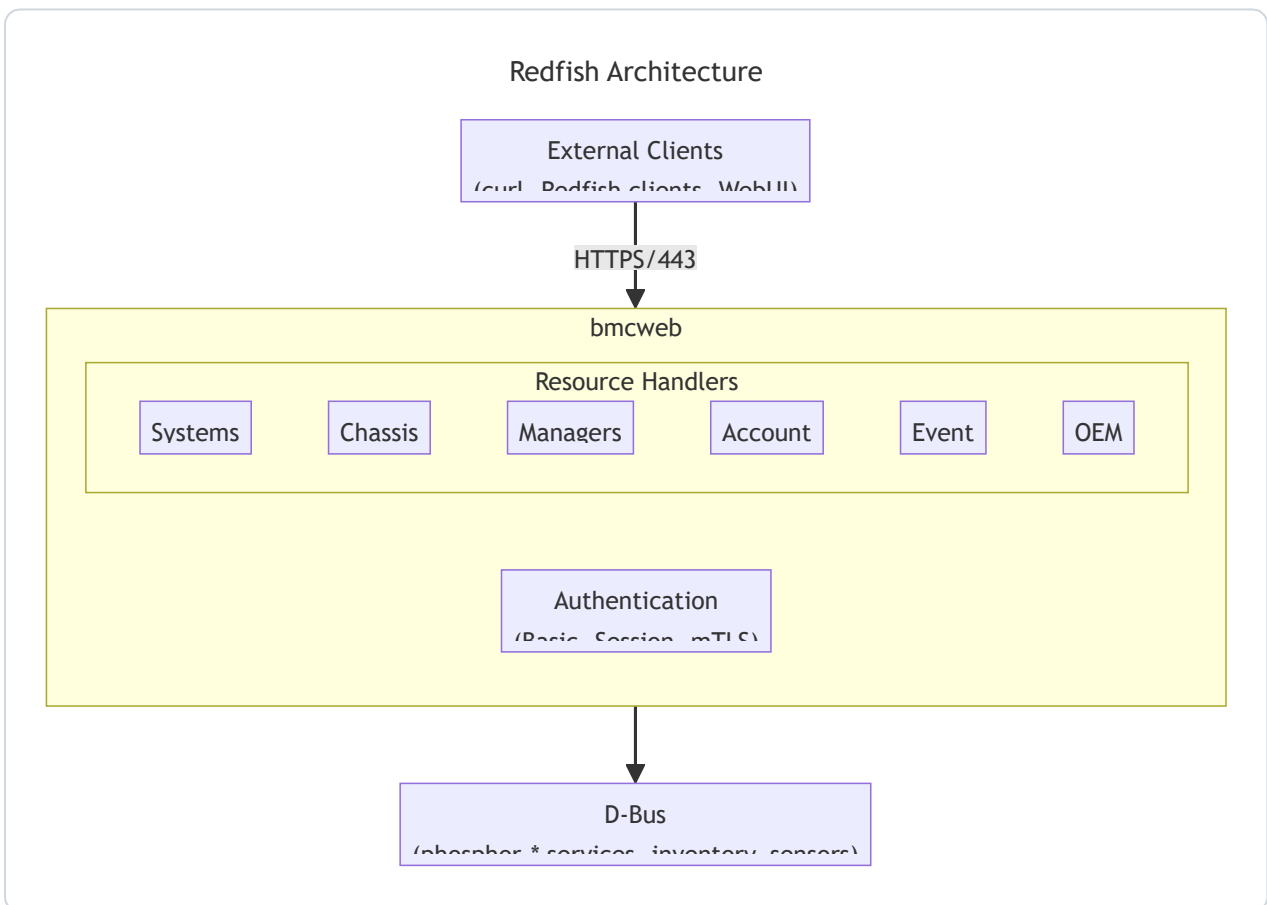
- 5. [Roles](#)
- 9. [Event Service](#)
 - 1. [List Subscriptions](#)
 - 2. [Create Subscription](#)
 - 3. [Delete Subscription](#)
- 10. [Firmware Update](#)
 - 1. [Check Current Version](#)
 - 2. [Upload and Apply Firmware](#)
- 11. [Redfish Aggregation](#)
 - 1. [Architecture](#)
 - 2. [Enable Aggregation](#)
 - 3. [Aggregation Service API](#)
 - 4. [Managing Aggregation Sources](#)
 - 5. [Aggregation Source Properties](#)
 - 6. [Connection Methods](#)
 - 7. [Accessing Aggregated Resources](#)
 - 8. [Resource Prefixing](#)
 - 9. [Aggregation Use Cases](#)
 - 1. [Multi-Node Server Management](#)
 - 2. [Unified Sensor Monitoring](#)
 - 3. [Centralized Log Collection](#)
 - 10. [Aggregation Configuration](#)
 - 1. [Trust Store for Satellite Certificates](#)
 - 2. [Aggregation with mTLS](#)
 - 11. [Troubleshooting Aggregation](#)
 - 1. [Check Aggregation Source Status](#)
 - 2. [Common Issues](#)
 - 3. [Debug Logging](#)
- 12. [Logs](#)
 - 1. [Get Event Log](#)
 - 2. [Clear Log](#)
- 13. [OEM Extensions](#)
 - 1. [OEM Resource Locations](#)
 - 2. [Implementing OEM Extensions](#)
- 14. [bmcweb Configuration](#)
 - 1. [Build Options](#)
 - 2. [Common Options](#)
- 15. [Adding Custom Routes](#)
 - 1. [Basic Route Handler](#)
 - 2. [Register in bmcweb](#)
- 16. [Troubleshooting](#)

1. [Service Not Responding](#)
 2. [Authentication Failures](#)
 3. [Missing Resources](#)
17. [Deep Dive](#)
 1. [Request Routing Architecture](#)
 2. [Async D-Bus Query Pattern](#)
 3. [Event Service \(SSE\) Implementation](#)
 4. [OEM Extension Pattern](#)
 5. [Source Code Reference](#)

18. [Examples](#)
19. [References](#)

Overview

Redfish is a modern, RESTful API standard for server management, replacing IPMI for most use cases. OpenBMC implements Redfish through **bmcweb**, including support for **Redfish Aggregation** to manage multiple BMCs from a single endpoint.



► ASCII-art version (for comparison)

Setup & Configuration

bmcweb's web server serves responses over both **HTTP/1.1 and HTTP/2** on the same HTTPS listener. When a client advertises HTTP/2 during the TLS handshake (via ALPN), bmcweb uses it; otherwise the connection falls back to HTTP/1.1. Both run on the standard HTTPS port, so no extra configuration is needed to accept either – the examples in this guide work unchanged regardless of which the client negotiates.

Build-Time Configuration (Yocto)

Configure bmcweb features in your build:

```
# In your machine .conf or local.conf

# Enable/disable bmcweb features
EXTRA_OEMESON:pn-bmcweb = " \
    -Dredfish-dbus-log=enabled \
    -Dredfish-bmc-journal=enabled \
    -Dredfish-cpu-log=enabled \
    -Dredfish-host-logger=enabled \
    -Dinsecure-tftp-update=disabled \
    -Dinsecure-push-style-notification=disabled \
    -Dkvm=enabled \
    -Dvm-websocket=enabled \
    -Dvm-nbdproxy=enabled \
    -Dmutual-tls-auth=enabled \
    -Dhttp-body-limit=65 \
"
```

Meson Build Options Reference

Option	Default	Description
Core Features		
<code>redfish</code>	enabled	Core Redfish support
<code>redfish-aggregation</code>	disabled	Aggregation support
<code>rest</code>	disabled	Legacy REST API (deprecated)
Logging		
<code>redfish-dbus-log</code>	enabled	D-Bus logging
<code>redfish-bmc-journal</code>	enabled	BMC journal logs
<code>redfish-cpu-log</code>	disabled	CPU crash logs
<code>redfish-host-logger</code>	enabled	Host console logging
Remote Access		
<code>kvm</code>	enabled	KVM WebSocket
<code>vm-websocket</code>	enabled	Virtual Media WebSocket
<code>vm-nbdproxy</code>	enabled	NBD proxy for virtual media
Security		
<code>mutual-tls-auth</code>	enabled	mTLS support
<code>insecure-tftp-update</code>	disabled	Allow TFTP updates
<code>insecure-push-style-notification</code>	disabled	Push notifications
Limits		
<code>http-body-limit</code>	30	Max request body (MB)

Runtime Enable/Disable

```
# Check bmcweb status
systemctl status bmcweb

# Restart bmcweb
systemctl restart bmcweb

# Stop bmcweb (disables all Redfish/web access)
systemctl stop bmcweb
systemctl disable bmcweb

# View logs
journalctl -u bmcweb -f
```

HTTPS Certificate Configuration

```
# View current certificate
curl -k https://localhost/redfish/v1/CertificateService/CertificateLocations

# Generate self-signed certificate (on BMC)
openssl req -x509 -newkey rsa:2048 -keyout /etc/ssl/private/server.pem \
  -out /etc/ssl/certs/server.pem -days 365 -nodes \
  -subj "/CN=openbmc"

# Replace via Redfish
curl -k -u root:openBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "CertificateType": "PEM",
    "CertificateString": "<PEM-ENCODED-CERT>"
  }' \
  https://localhost/redfish/v1/Managers/bmc/NetworkProtocol/HTTPS/Certificates
```

Network Protocol Settings

```
# Get network protocol settings
curl -k -u root:openBmc \
  https://localhost/redfish/v1/Managers/bmc/NetworkProtocol

# Enable/disable protocols via Redfish
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{
    "HTTPS": {"ProtocolEnabled": true, "Port": 443},
    "SSH": {"ProtocolEnabled": true, "Port": 22},
    "IPMI": {"ProtocolEnabled": true, "Port": 623},
    "NTP": {"ProtocolEnabled": true}
  }' \
  https://localhost/redfish/v1/Managers/bmc/NetworkProtocol
```

Session Configuration

```
# Get session service settings
curl -k -u root:openBmc \
  https://localhost/redfish/v1/SessionService

# Configure session timeout (seconds)
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"SessionTimeout": 3600}' \
  https://localhost/redfish/v1/SessionService
```

Account Service Configuration

```
# Get account policy
curl -k -u root:openBmc \
  https://localhost/redfish/v1/AccountService

# Configure account lockout
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{
    "AccountLockoutThreshold": 5,
    "AccountLockoutDuration": 300,
    "MinPasswordLength": 8
  }' \
  https://localhost/redfish/v1/AccountService
```

Event Service Configuration

```
# Get event service settings
curl -k -u root:openBmc \
  https://localhost/redfish/v1/EventService

# Enable/disable event service
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{
    "ServiceEnabled": true,
    "DeliveryRetryAttempts": 3,
    "DeliveryRetryIntervalSeconds": 60
  }' \
  https://localhost/redfish/v1/EventService
```

Telemetry Configuration

```
# Get telemetry service
curl -k -u root:openBmc \
  https://localhost/redfish/v1/TelemetryService

# Create metric report
curl -k -u root:openBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "Id": "PowerMetrics",
    "Name": "Power Consumption Report",
    "MetricReportDefinitionType": "Periodic",
    "ReportUpdates": "Overwrite",
    "Schedule": {
      "RecurrenceInterval": "PT60S"
    },
    "Metrics": [
      {
        "MetricId": "TotalPower",
        "MetricProperties": ["/redfish/v1/Chassis/chassis/Power#/PowerControl/0/PowerConsumedWatts"]
      }
    ]
  }' \
  https://localhost/redfish/v1/TelemetryService/MetricReportDefinitions
```

Redfish Basics

API Structure

```
/redfish/v1/                                # Service Root
├── Systems/                                # Computer systems
│   └── system/                             # This system
│       ├── Processors/                    # CPUs
│       ├── Memory/                       # DIMMs
│       ├── Storage/                      # Storage devices
│       └── LogServices/                   # System logs
├── Chassis/                               # Physical enclosures
│   └── chassis/                           # This chassis
│       ├── Power/                        # Power info
│       ├── Thermal/                     # Thermal info
│       └── Sensors/                      # All sensors
├── Managers/                             # BMC managers
│   └── bmc/                               # This BMC
│       ├── NetworkProtocol/              # Network services
│       ├── EthernetInterfaces/           # Network ports
│       └── LogServices/                   # BMC logs
├── AccountService/                        # User management
│   ├── Accounts/                         # User accounts
│   └── Roles/                            # Privilege roles
├── SessionService/                       # Session management
├── EventService/                         # Event subscriptions
├── UpdateService/                        # Firmware updates
└── TelemetryService/                     # Telemetry data
```

Common Operations

```
# Set up alias for convenience
alias bmcurl='curl -k -u root:0penBmc'

# Get service root
bmcurl https://localhost/redfish/v1/

# Get system info
bmcurl https://localhost/redfish/v1/Systems/system

# Get chassis info
bmcurl https://localhost/redfish/v1/Chassis/chassis

# Get BMC info
bmcurl https://localhost/redfish/v1/Managers/bmc
```

Authentication

Basic Authentication

```
curl -k -u root:0penBmc https://localhost/redfish/v1/Systems/system
```

Session Authentication

```
# Create session
curl -k -X POST \
  -H "Content-Type: application/json" \
  -d '{"UserName":"root","Password":"OpenBmc"}' \
  https://localhost/redfish/v1/SessionService/Sessions

# Response includes X-Auth-Token header
# Use token for subsequent requests
curl -k -H "X-Auth-Token: <token>" \
  https://localhost/redfish/v1/Systems/system

# Delete session
curl -k -X DELETE \
  -H "X-Auth-Token: <token>" \
  https://localhost/redfish/v1/SessionService/Sessions/<session-id>
```

mTLS Authentication

Configure client certificates for mutual TLS authentication.

Power Operations

Get Power State

```
bmcurl https://localhost/redfish/v1/Systems/system | jq '.PowerState'
```

Power On/Off

```
# Power on
bmcurl -X POST \
  -H "Content-Type: application/json" \
  -d '{"ResetType":"On"}' \
  https://localhost/redfish/v1/Systems/system/Actions/ComputerSystem.Reset

# Graceful shutdown
bmcurl -X POST \
  -H "Content-Type: application/json" \
  -d '{"ResetType":"GracefulShutdown"}' \
  https://localhost/redfish/v1/Systems/system/Actions/ComputerSystem.Reset

# Force off
bmcurl -X POST \
  -H "Content-Type: application/json" \
  -d '{"ResetType":"ForceOff"}' \
  https://localhost/redfish/v1/Systems/system/Actions/ComputerSystem.Reset

# Reboot
bmcurl -X POST \
  -H "Content-Type: application/json" \
  -d '{"ResetType":"ForceRestart"}' \
  https://localhost/redfish/v1/Systems/system/Actions/ComputerSystem.Reset
```

Reset Types

Type	Description
On	Power on
ForceOff	Immediate power off
GracefulShutdown	Graceful OS shutdown
GracefulRestart	Graceful reboot
ForceRestart	Immediate reboot
Nmi	Non-maskable interrupt

Sensors

List All Sensors

```
bmcurl https://localhost/redfish/v1/Chassis/chassis/Sensors | jq '.Members'
```

Get Sensor Details

```
bmcurl https://localhost/redfish/v1/Chassis/chassis/Sensors/CPU_Temp | jq
```

Sensor Properties

```
{
  "@odata.id": "/redfish/v1/Chassis/chassis/Sensors/CPU_Temp",
  "Name": "CPU Temp",
  "Reading": 45.0,
  "ReadingUnits": "Cel",
  "Status": {
    "State": "Enabled",
    "Health": "OK"
  },
  "Thresholds": {
    "UpperCritical": {"Reading": 95},
    "UpperCaution": {"Reading": 85},
    "LowerCritical": {"Reading": 0},
    "LowerCaution": {"Reading": 5}
  }
}
```

Thermal Management

Get Thermal Info

```
bmcurl https://localhost/redfish/v1/Chassis/chassis/Thermal | jq
```


Response Structure

```
{
  "Temperatures": [
    {
      "Name": "CPU Temp",
      "ReadingCelsius": 45,
      "Status": {"State": "Enabled", "Health": "OK"}
    }
  ],
  "Fans": [
    {
      "Name": "Fan0",
      "Reading": 5000,
      "ReadingUnits": "RPM",
      "Status": {"State": "Enabled", "Health": "OK"}
    }
  ]
}
```

User Management

List Users

```
bmcurl https://localhost/redfish/v1/AccountService/Accounts | jq '.Members'
```

Create User

```
bmcurl -X POST \
-H "Content-Type: application/json" \
-d '{
  "UserName": "newuser",
  "Password": "Password123!",
  "RoleId": "Operator",
  "Enabled": true
}' \
https://localhost/redfish/v1/AccountService/Accounts
```

Modify User

```
bmcurl -X PATCH \
-H "Content-Type: application/json" \
-d '{"Password": "NewPassword123!"}' \
https://localhost/redfish/v1/AccountService/Accounts/newuser
```

Delete User

```
bmcurl -X DELETE \
https://localhost/redfish/v1/AccountService/Accounts/newuser
```

Roles

Role	Description
Administrator	Full access
Operator	Manage operations, no user management
ReadOnly	Read-only access

Event Service

List Subscriptions

```
bmcurl https://localhost/redfish/v1/EventService/Subscriptions
```

Create Subscription

```
bmcurl -X POST \  
  -H "Content-Type: application/json" \  
  -d '{  
    "Destination": "https://my-server:8080/events",  
    "Protocol": "Redfish",  
    "EventTypes": ["Alert", "StatusChange"],  
    "Context": "MySubscription"  
  }' \  
  https://localhost/redfish/v1/EventService/Subscriptions
```

Delete Subscription

```
bmcurl -X DELETE \  
  https://localhost/redfish/v1/EventService/Subscriptions/<id>
```

Firmware Update

Check Current Version

```
bmcurl https://localhost/redfish/v1/UpdateService/FirmwareInventory | jq
```

Upload and Apply Firmware

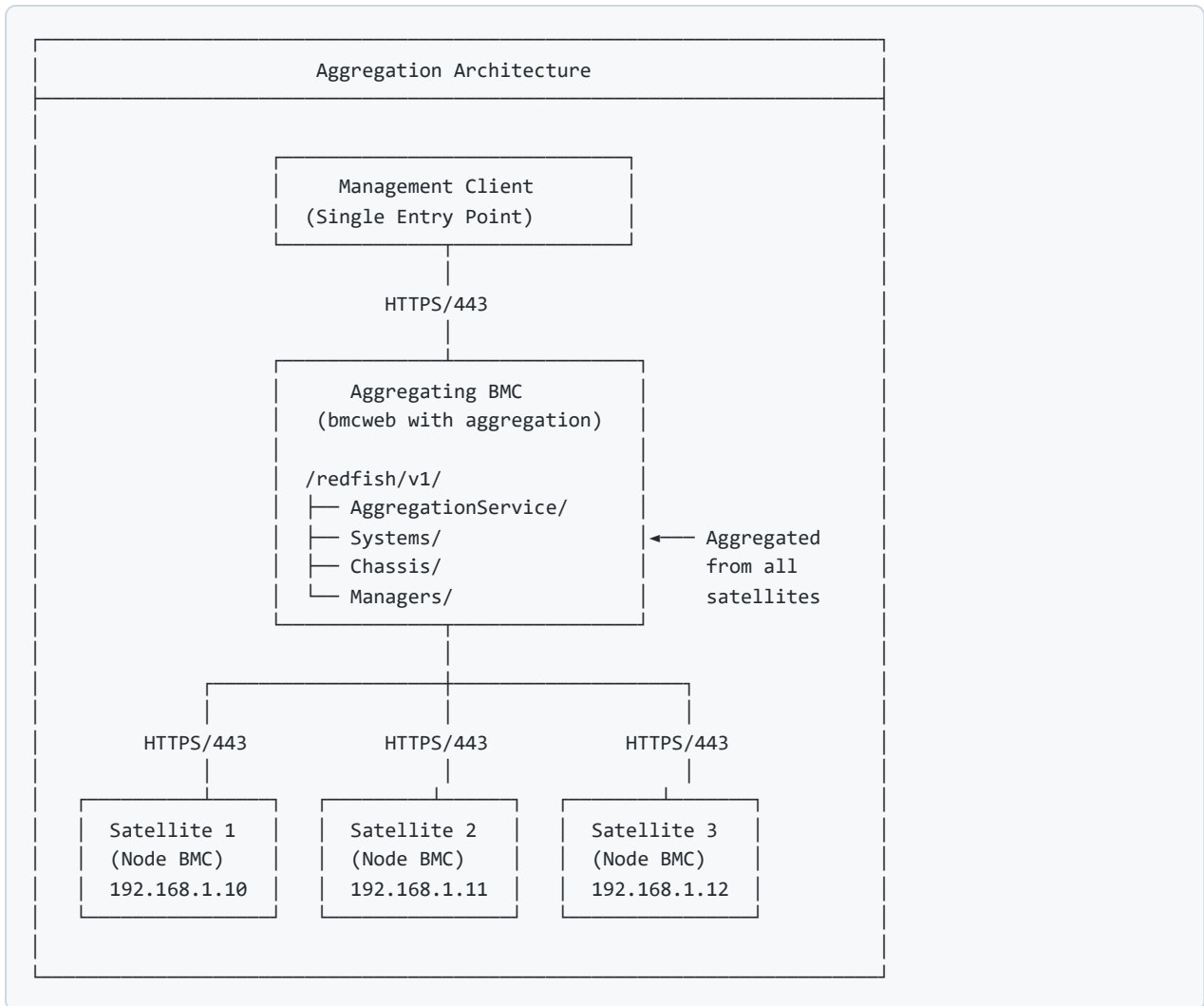
```
# Upload firmware image
curl -k -u root:openBmc \
  -X POST \
  -H "Content-Type: application/octet-stream" \
  --data-binary @firmware.tar \
  https://localhost/redfish/v1/UpdateService

# Or use SimpleUpdate action
bmcurl -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "TransferProtocol": "TFTP",
    "ImageURI": "tftp://192.168.1.100/firmware.tar"
  }' \
  https://localhost/redfish/v1/UpdateService/Actions/UpdateService.SimpleUpdate
```

Redfish Aggregation

Redfish Aggregation allows a single BMC (the “Aggregator”) to present resources from multiple downstream BMCs through a unified API. This is useful for multi-node systems, disaggregated infrastructure, or centralized datacenter management.

Architecture



Enable Aggregation

Enable the aggregation feature at build time:

```
# In your machine .conf or local.conf
EXTRA_OEMESON:pn-bmcweb = " \
    -Dredfish-aggregation=enabled \
"
```

Aggregation Service API

Once enabled, the AggregationService endpoint becomes available:

```
# Get aggregation service
bmcurl https://localhost/redfish/v1/AggregationService

# Response
{
  "@odata.id": "/redfish/v1/AggregationService",
  "@odata.type": "#AggregationService.v1_0_1.AggregationService",
  "Id": "AggregationService",
  "Name": "Aggregation Service",
  "Description": "Aggregation Service",
  "ServiceEnabled": true,
  "AggregationSources": {
    "@odata.id": "/redfish/v1/AggregationService/AggregationSources"
  },
  "ConnectionMethods": {
    "@odata.id": "/redfish/v1/AggregationService/ConnectionMethods"
  }
}
```

Managing Aggregation Sources

Aggregation sources define the downstream BMCs to aggregate:

```
# List configured aggregation sources
bmcurl https://localhost/redfish/v1/AggregationService/AggregationSources

# Add a new aggregation source (satellite BMC)
bmcurl -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "HostName": "192.168.1.10",
    "UserName": "admin",
    "Password": "password",
    "Links": {
      "ConnectionMethod": {
        "@odata.id": "/redfish/v1/AggregationService/ConnectionMethods/Redfish"
      }
    }
  }' \
  https://localhost/redfish/v1/AggregationService/AggregationSources

# Get details of an aggregation source
bmcurl https://localhost/redfish/v1/AggregationService/AggregationSources/1

# Remove an aggregation source
bmcurl -X DELETE \
  https://localhost/redfish/v1/AggregationService/AggregationSources/1
```

Aggregation Source Properties

Property	Type	Description
<code>HostName</code>	string	IP address or hostname of satellite BMC
<code>UserName</code>	string	Authentication username
<code>Password</code>	string	Authentication password (write-only)
<code>SNMP</code>	object	SNMP community settings (optional)
<code>Links.ConnectionMethod</code>	link	Connection method to use
<code>Links.ResourcesAccessed</code>	array	Resources aggregated from this source

Connection Methods

Connection methods define how the aggregator connects to satellites:

```
# List available connection methods
bmcurl https://localhost/redfish/v1/AggregationService/ConnectionMethods

# Get connection method details
bmcurl https://localhost/redfish/v1/AggregationService/ConnectionMethods/Redfish

# Response
{
  "@odata.id": "/redfish/v1/AggregationService/ConnectionMethods/Redfish",
  "@odata.type": "#ConnectionMethod.v1_0_0.ConnectionMethod",
  "Id": "Redfish",
  "Name": "Redfish Connection Method",
  "ConnectionMethodType": "Redfish",
  "ConnectionMethodVariant": "Redfish"
}
```

Accessing Aggregated Resources

Aggregated resources appear with prefixed IDs to identify their source:

```
# List all systems (local + aggregated)
bmcurl https://localhost/redfish/v1/Systems

# Response includes aggregated systems
{
  "Members": [
    {"@odata.id": "/redfish/v1/Systems/system"},           # Local
    {"@odata.id": "/redfish/v1/Systems/agg1_system"},      # Satellite 1
    {"@odata.id": "/redfish/v1/Systems/agg2_system"}       # Satellite 2
  ]
}

# Access aggregated system
bmcurl https://localhost/redfish/v1/Systems/agg1_system

# The request is proxied to the satellite BMC
```

Resource Prefixing

Aggregated resources use prefixes to avoid ID collisions:

Local Resource	Aggregated Resource (Source 1)
<code>/Systems/system</code>	<code>/Systems/agg1_system</code>
<code>/Chassis/chassis</code>	<code>/Chassis/agg1_chassis</code>
<code>/Managers/bmc</code>	<code>/Managers/agg1_bmc</code>
<code>/Sensors/CPU_Temp</code>	<code>/Sensors/agg1_CPU_Temp</code>

Aggregation Use Cases

Multi-Node Server Management

```
# Power on all aggregated systems
for sys in system agg1_system agg2_system agg3_system; do
    bmcurl -X POST \
        -H "Content-Type: application/json" \
        -d '{"ResetType":"On"}' \
        "https://localhost/redfish/v1/Systems/${sys}/Actions/ComputerSystem.Reset"
done
```

Unified Sensor Monitoring

```
# Get all temperature sensors across all nodes
bmcurl https://localhost/redfish/v1/Chassis | jq -r '.Members["@odata.id"] | \
while read chassis; do
    echo "=== ${chassis} ==="
    bmcurl "${chassis}/Thermal" | jq '.Temperatures[] | {Name, ReadingCelsius}'
done
```

Centralized Log Collection

```
# Collect event logs from all systems
bmcurl https://localhost/redfish/v1/Systems | jq -r '.Members["@odata.id"] | \
while read system; do
    echo "=== ${system} ==="
    bmcurl "${system}/LogServices/EventLog/Entries" | jq '.Members[:5]'
done
```

Aggregation Configuration

Trust Store for Satellite Certificates

Configure trusted certificates for satellite BMC connections:

```
# Add satellite certificate to trust store
bmcurl -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "CertificateType": "PEM",
    "CertificateString": "<PEM-ENCODED-CERT>"
  }' \
  https://localhost/redfish/v1/Managers/bmc/Truststore/Certificates
```

Aggregation with mTLS

For mutual TLS authentication with satellites:

```
# Configure client certificate for satellite authentication
bmcurl -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "HostName": "192.168.1.10",
    "Links": {
      "ConnectionMethod": {
        "@odata.id": "/redfish/v1/AggregationService/ConnectionMethods/Redfish"
      }
    }
  }' \
  https://localhost/redfish/v1/AggregationService/AggregationSources
```

Troubleshooting Aggregation

Check Aggregation Source Status

```
# Get aggregation source status
bmcurl https://localhost/redfish/v1/AggregationService/AggregationSources/1 | jq '.Status'

# Response
{
  "State": "Enabled",
  "Health": "OK"
}
```

Common Issues

Issue	Cause	Solution
Source unreachable	Network/firewall issue	Check connectivity to satellite
Authentication failed	Wrong credentials	Verify username/password
Certificate error	Untrusted satellite cert	Add to trust store or disable verification
Resources not appearing	Service not ready	Wait for satellite to fully boot
Timeout errors	Slow satellite response	Increase timeout settings

Debug Logging


```
# Enable verbose bmcweb logging
# Edit /lib/systemd/system/bmcweb.service to add:
# Environment="BMCWEB_LOG_LEVEL=DEBUG"

systemctl daemon-reload
systemctl restart bmcweb
journalctl -u bmcweb -f | grep -i aggregat
```

Logs

Get Event Log

```
bmcurl https://localhost/redfish/v1/Systems/system/LogServices/EventLog/Entries | jq '.Members'
```

Clear Log

```
bmcurl -X POST \
https://localhost/redfish/v1/Systems/system/LogServices/EventLog/Actions/LogService.ClearLog
```

OEM Extensions

OEM Resource Locations

OEM extensions appear under `Oem` properties:

```
{
  "@odata.id": "/redfish/v1/Systems/system",
  "Oem": {
    "OpenBMC": {
      "@odata.type": "#OemComputerSystem.v1_0_0.OpenBMC",
      "FirmwareProvisioning": {
        "ProvisioningStatus": "NotProvisioned"
      }
    }
  }
}
```

Implementing OEM Extensions

Add OEM properties in bmcweb handlers:

```
// In systems.hpp
nlohmann::json& oem = asyncResp->res.jsonValue["Oem"]["OpenBMC"];
oem["@odata.type"] = "#OemComputerSystem.v1_0_0.OpenBMC";
oem["MyCustomProperty"] = "value";
```

bmcweb Configuration

Build Options

```
# In your machine config or local.conf
EXTRA_OEMESON:pn-bmcweb = " \
    -Dredfish-dbus-log=enabled \
    -Dhttp-body-limit=65 \
    -Dinsecure-tftp-update=disabled \
    "
```

Common Options

Option	Description
<code>redfish-dbus-log</code>	Enable D-Bus based logging
<code>insecure-tftp-update</code>	Allow TFTP firmware updates
<code>http-body-limit</code>	Max request body size (MB)
<code>mutual-tls-auth</code>	Enable mTLS
<code>kvm</code>	Enable KVM support
<code>vm-websocket</code>	Enable Virtual Media WebSocket

Adding Custom Routes

Basic Route Handler

Source Reference: Pattern based on [bmcweb](#)

- Route patterns: [redfish-core/lib/managers.hpp](#)
- Privilege definitions: [registries/privilege_registry.hpp](#)

```
// In a new file under redfish-core/lib/
#include <app.hpp>
#include <registries/privilege_registry.hpp>

namespace redfish
{

inline void requestRoutesMyOem(App& app)
{
    BMCWEB_ROUTE(app, "/redfish/v1/Oem/MyVendor/MyResource")
        .privileges(redfish::privileges::getManager)
        .methods(boost::beast::http::verb::get)(
            [](const crow::Request&,
               const std::shared_ptr<bmcweb::AsyncResp>& asyncResp) {
                asyncResp->res.jsonValue["@odata.type"] =
                    "#MyResource.v1_0_0.MyResource";
                asyncResp->res.jsonValue["@odata.id"] =
                    "/redfish/v1/Oem/MyVendor/MyResource";
                asyncResp->res.jsonValue["Name"] = "My Custom Resource";
                asyncResp->res.jsonValue["Status"]["State"] = "Enabled";
            });
}

} // namespace redfish
```

Register in bmcweb

```
// In src/webserver_main.cpp or appropriate location
#include "myoem.hpp"

// In main():
redfish::requestRoutesMyOem(app);
```

Troubleshooting

Service Not Responding

```
# Check bmcweb status
systemctl status bmcweb

# View logs
journalctl -u bmcweb -f
```

Authentication Failures

```
# Check session service
bmcurl https://localhost/redfish/v1/SessionService

# Verify account exists
bmcurl https://localhost/redfish/v1/AccountService/Accounts
```

Missing Resources

```
# Check if D-Bus service is running
busctl tree xyz.openbmc_project.ObjectMapper

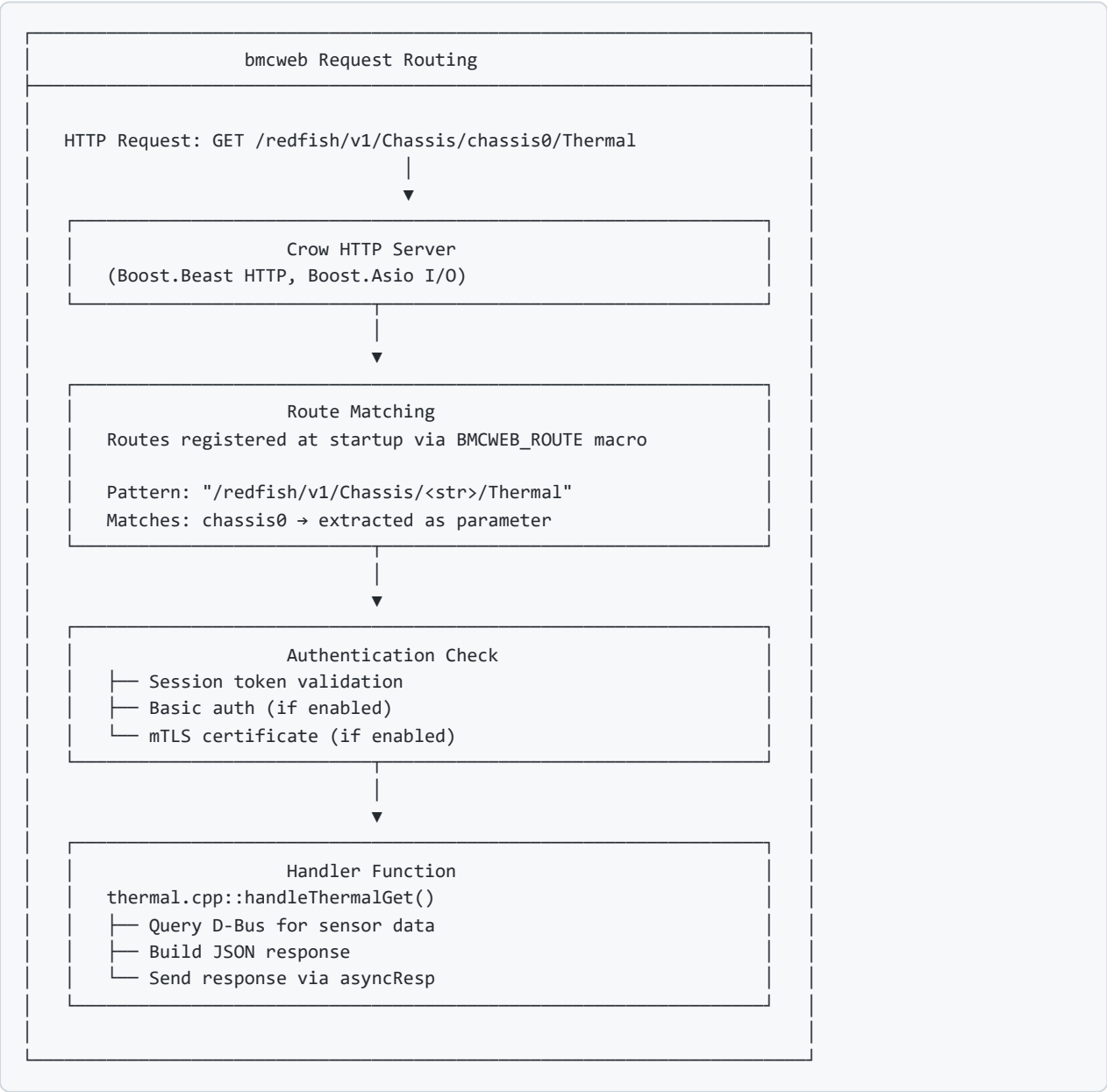
# Query object mapper for resource
busctl call xyz.openbmc_project.ObjectMapper \
  /xyz/openbmc_project/object_mapper \
  xyz.openbmc_project.ObjectMapper \
  GetObject sas "/xyz/openbmc_project/inventory/system" 0
```

Deep Dive

Advanced implementation details for Redfish developers.

Request Routing Architecture

bmcweb uses a route registration pattern for handling HTTP requests:



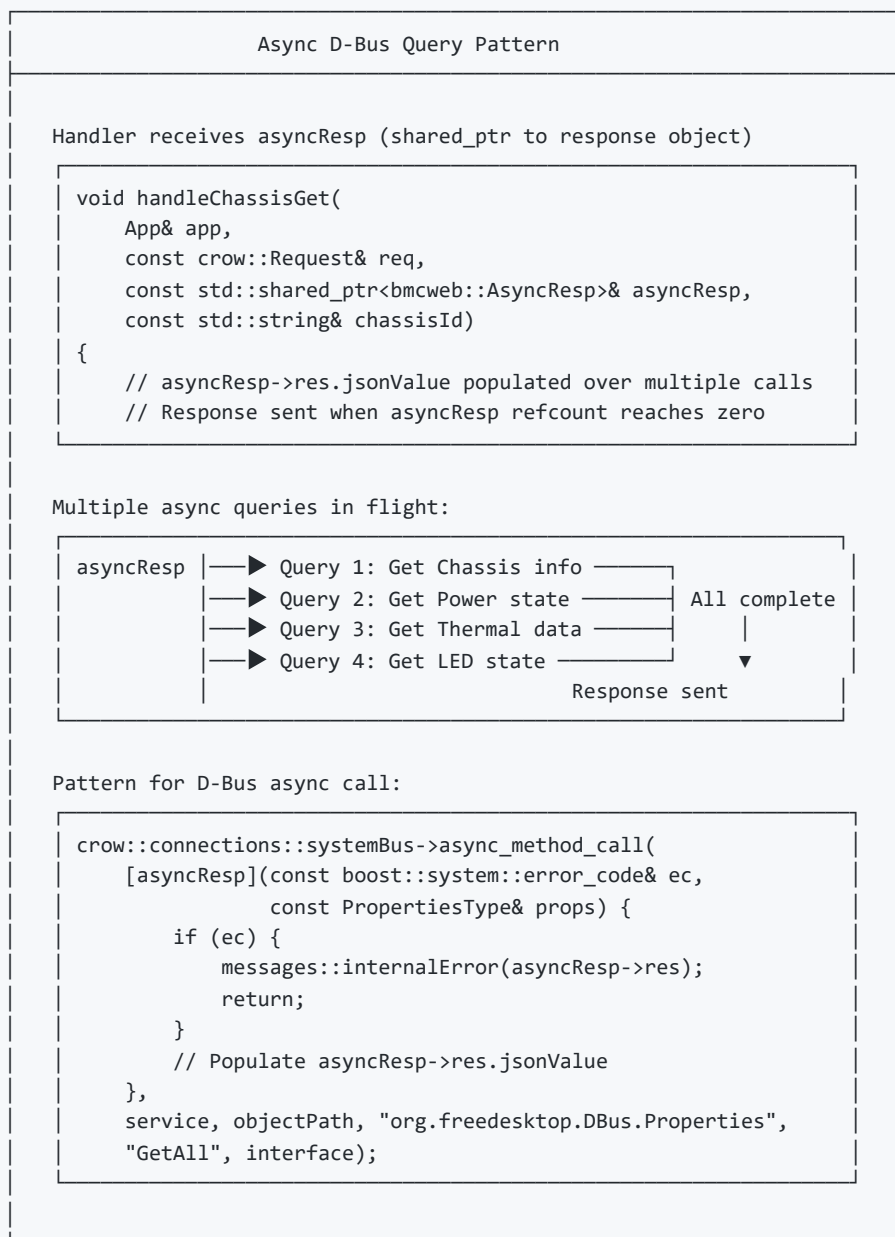
Route registration pattern:

```
// In redfish-core/lib/thermal.hpp
inline void requestRoutesThermal(App& app)
{
    BMCWEB_ROUTE(app, "/redfish/v1/Chassis/<str>/Thermal/")
        .privileges(redfish::privileges::getThermal)
        .methods(boost::beast::http::verb::get)(
            std::bind_front(handleThermalGet, std::ref(app)));
}
```

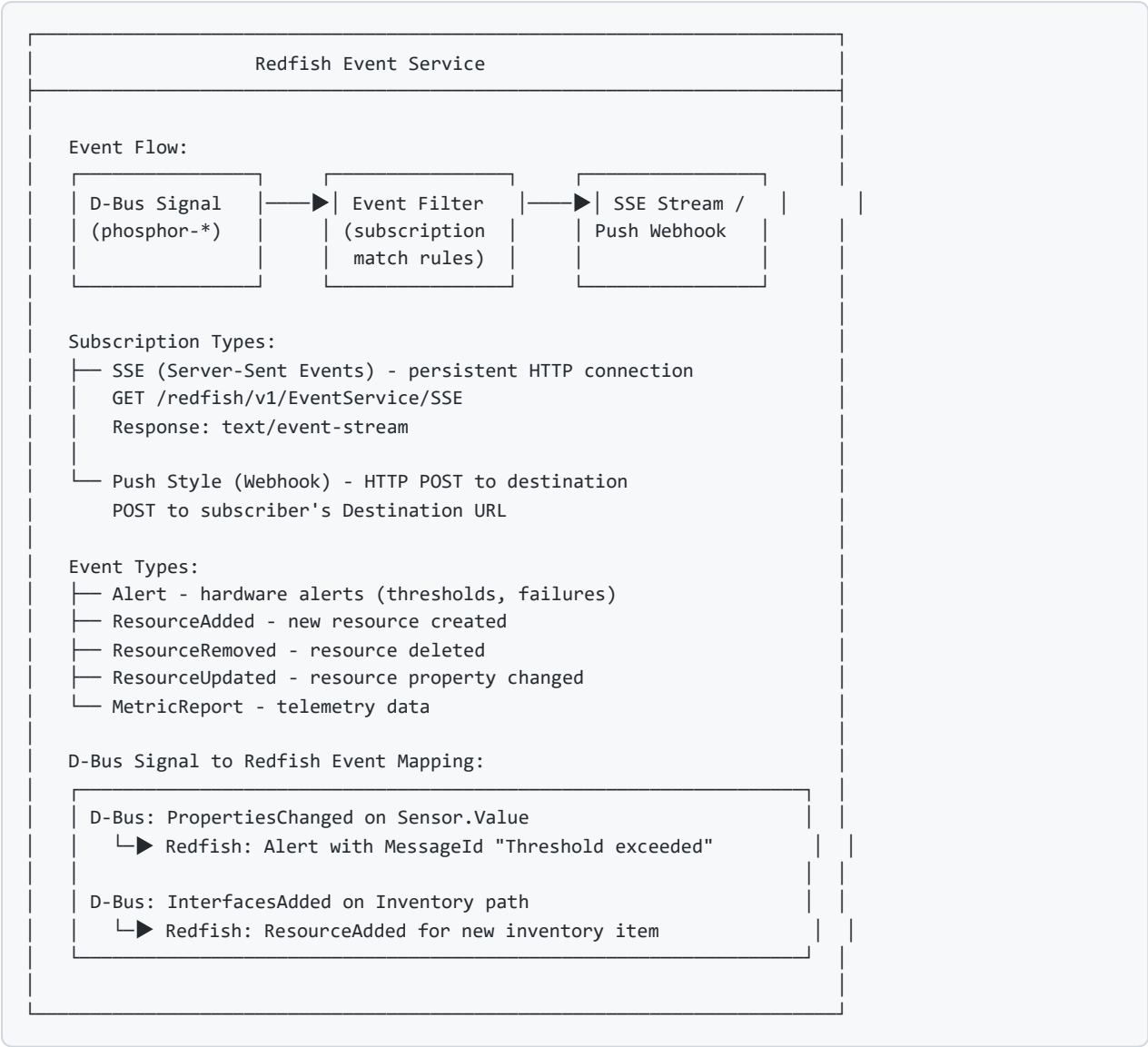
Source reference: [redfish-core/lib/](#)

Async D-Bus Query Pattern

bmcweb uses async callbacks to avoid blocking the event loop:



Event Service (SSE) Implementation



OEM Extension Pattern

```
// Example: Adding OEM data to an existing resource
// In redfish-core/lib/chassis.cpp

inline void handleChassisGet(...)
{
    // Standard Redfish properties
    asyncResp->res.jsonValue["@odata.type"] = "#Chassis.v1_16_0.Chassis";
    asyncResp->res.jsonValue["Id"] = chassisId;

    // Add OEM section
    nlohmann::json& oem = asyncResp->res.jsonValue["Oem"]["OpenBMC"];
    oem["@odata.type"] = "#OpenBMChassis.v1_0_0.Chassis";

    // Query OEM-specific D-Bus properties
    crow::connections::systemBus->async_method_call(
        [asyncResp](const boost::system::error_code& ec,
                    const std::string& customProp) {
            if (!ec) {
                asyncResp->res.jsonValue["Oem"]["OpenBMC"]
                    ["CustomProperty"] = customProp;
            }
        },
        "xyz.openbmc_project.OEM.Service",
        objectPath,
        "org.freedesktop.DBus.Properties",
        "Get",
        "xyz.openbmc_project.OEM.CustomInterface",
        "CustomProperty");
}
```

Source Code Reference

Key implementation files in [bmcweb](#):

File	Description
src/webserver_main.cpp	Server startup, route registration
include/sessions.hpp	Session management
redfish-core/lib/systems.cpp	/Systems resource
redfish-core/lib/chassis.cpp	/Chassis resource
redfish-core/lib/sensors.cpp	Sensor reading helpers
redfish-core/lib/log_services.cpp	SEL/Event log
redfish-core/include/privileges.hpp	Authorization
redfish-core/include/utils/dbus_utils.hpp	D-Bus helpers

Examples

Working examples are available in the [examples/redfish](#) directory:

- [oem_resource.hpp](#) - Custom OEM resource implementation
 - [oem-schema.json](#) - OEM schema definition
 - [test_oem_resource.sh](#) - OEM resource test script
-

References

- [bmcweb](#)
 - [DMTF Redfish Specification](#)
 - [Redfish Schema Index](#)
 - [Redfish Aggregation White Paper](#)
 - [OpenBMC Redfish Design](#)
-

Tested on: OpenBMC master, QEMU romulus

WebUI Guide

Configure and customize the OpenBMC web interface.

Table of Contents

1. [Overview](#)
2. [Setup & Configuration](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [Build Configuration Options](#)
 3. [Runtime Configuration](#)
 4. [Access the WebUI](#)
3. [WebUI Features](#)
 1. [Dashboard](#)
 2. [Operations Pages](#)
 3. [Hardware Pages](#)
 4. [Settings Pages](#)
4. [Customization](#)
 1. [Custom Branding](#)
 2. [Replacing the Logo](#)
 3. [Custom Colors](#)
 4. [Build Custom Theme](#)
5. [Adding Custom Pages](#)
 1. [Create a New View](#)
 2. [Create a Vuex Store Module](#)
6. [Internationalization \(i18n\)](#)
 1. [Adding Translations](#)
 2. [Using Translations](#)
 3. [Adding New Language](#)
7. [Configuration Options](#)
 1. [Build Environment Variables](#)
 2. [Feature Flags](#)
 3. [Custom Navigation](#)
8. [Security Configuration](#)
 1. [Session Management](#)
 2. [HTTPS Requirements](#)
 3. [Content Security Policy](#)
9. [Development Workflow](#)
 1. [Local Development](#)
 2. [Building for Production](#)

- 3. [Running Tests](#)
 - 10. [Troubleshooting](#)
 - 1. [Login Failures](#)
 - 2. [Page Not Loading](#)
 - 3. [API Errors in Console](#)
 - 4. [Browser Console Debug](#)
 - 11. [Enabling/Disabling WebUI](#)
 - 1. [Remove from Image](#)
 - 2. [Disable at Runtime](#)
 - 12. [References](#)
 - 13. [Deep Dive](#)
 - 1. [WebUI Request Flow](#)
 - 2. [Vuex State Management Architecture](#)
 - 3. [Session Authentication Flow](#)
 - 4. [WebSocket Event Streaming](#)
 - 5. [Build and Bundle Process](#)
 - 6. [Source Code References](#)
-

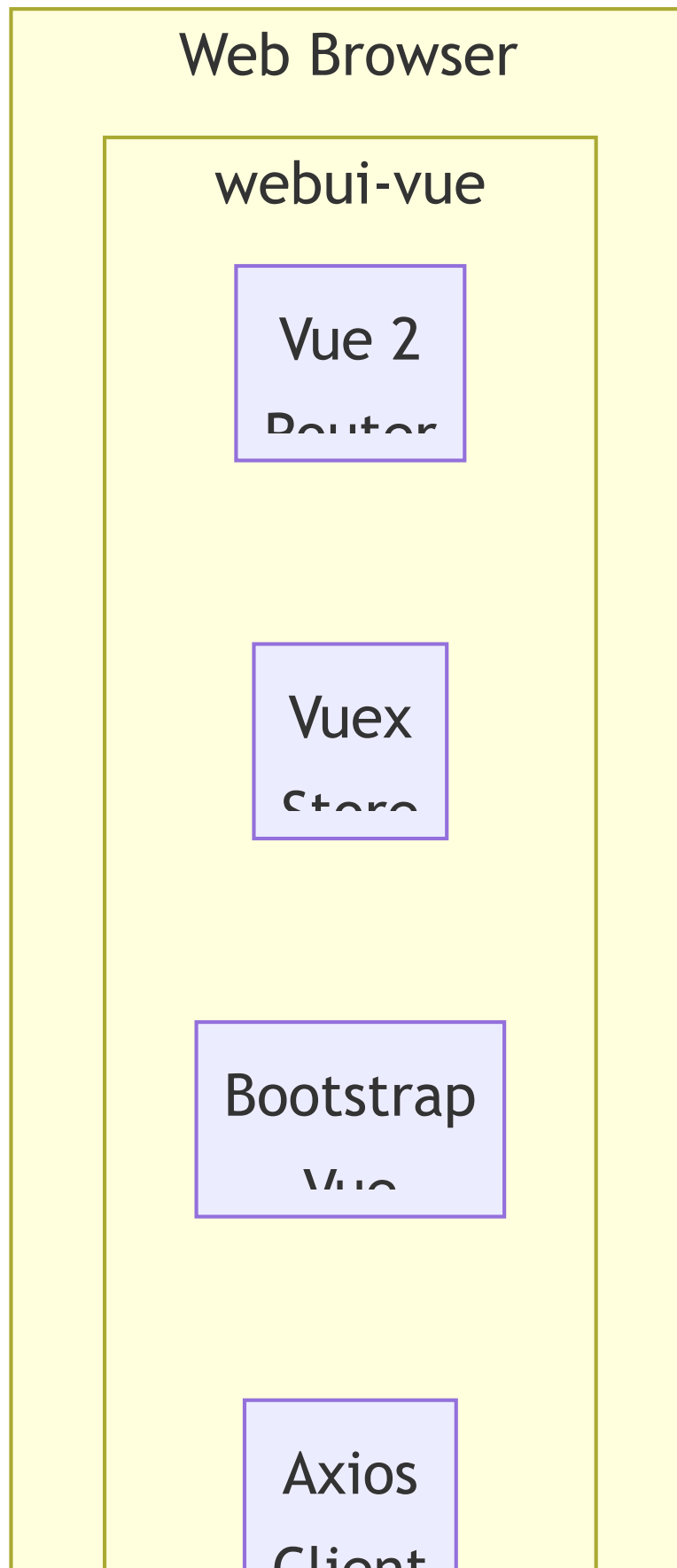
Overview

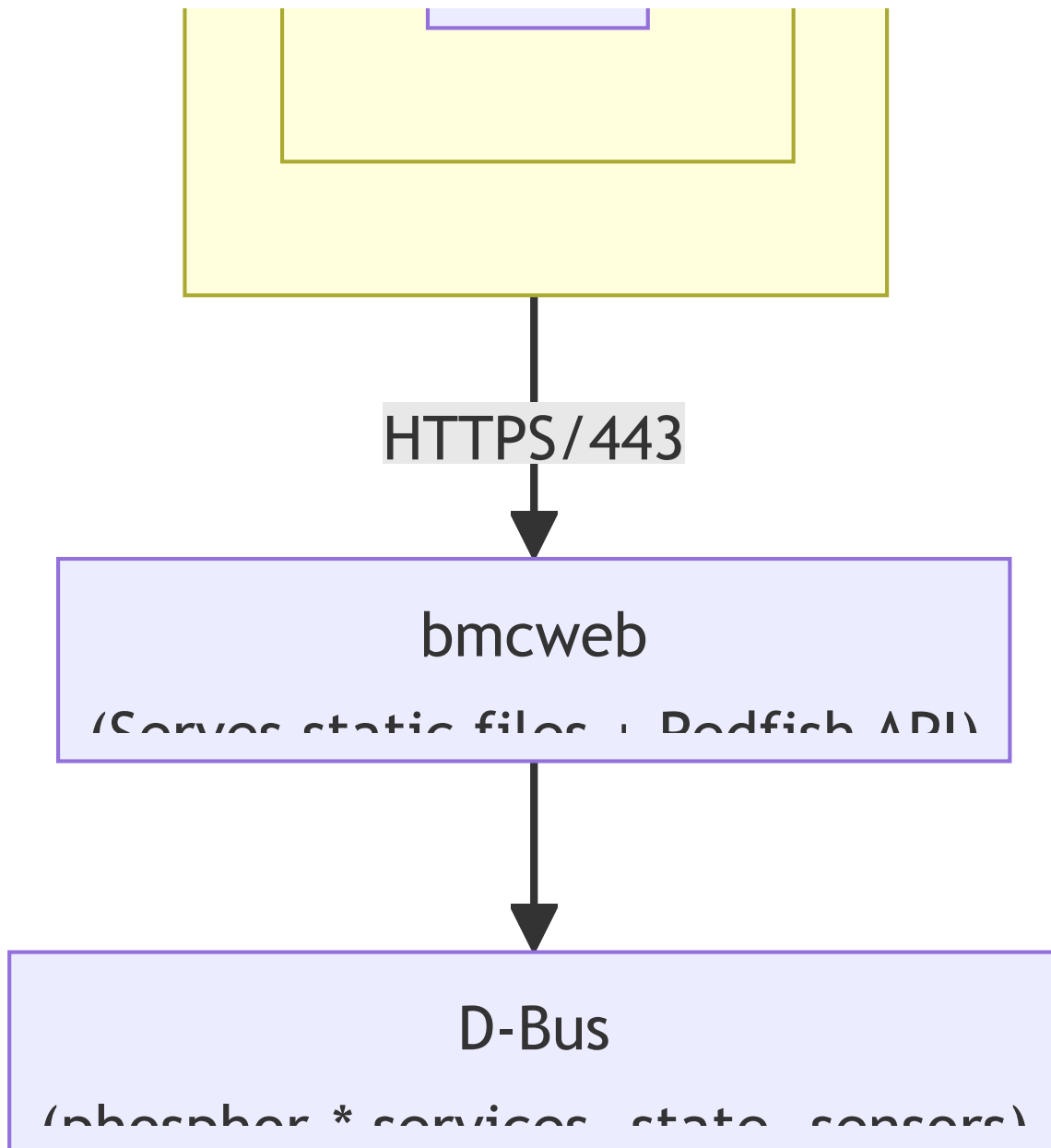
webui-vue is the modern, Vue.js-based web interface for OpenBMC. It provides a browser-based management console that communicates with the BMC through the Redfish API.

webui-vue replaces phosphor-webui. The earlier OpenBMC web interface, [phosphor-webui](#), was built on AngularJS (now end-of-life) and drove the BMC through the legacy **REST D-Bus API**, which is disabled by default in bmcweb. phosphor-webui is deprecated in favor of webui-vue and is slated to move to read-only. By contrast, **webui-vue is Redfish-only** — it talks exclusively to the Redfish API. If you come across documentation or code referring to phosphor-webui, `ng-app`, or AngularJS controllers, it belongs to that older UI and does not apply here.



WebUI Architecture





► ASCII-art version (for comparison)

Setup & Configuration

Build-Time Configuration (Yocto)

Include webui-vue in your image:

```
# In your machine .conf or local.conf

# Include web interface
IMAGE_INSTALL:append = " webui-vue"

# Configure features via environment options
EXTRA_OEMAKE:pn-webui-vue = "VUE_APP_COMPANY_NAME='MyCompany'"
```

Build Configuration Options

Configure webui-vue at build time using environment variables:

Variable	Default	Description
<code>VUE_APP_COMPANY_NAME</code>	OpenBMC	Company name in header
<code>VUE_APP_PRODUCT_NAME</code>	OpenBMC	Product name
<code>VUE_APP_SERVER_ROOT</code>		Prefix for all API requests
<code>VUE_APP_THEME</code>	default	Theme name
<code>VUE_APP_INTERNAL_LINKS</code>		Custom navigation links (JSON)
<code>VUE_APP_ROUTER_BASE</code>	/	Router base path

Runtime Configuration

The WebUI runs as static files served by bmcweb:

```
# Check bmcweb status (serves WebUI)
systemctl status bmcweb

# WebUI files location
ls /usr/share/www/

# Restart to pick up WebUI changes
systemctl restart bmcweb
```

Access the WebUI

```
# Default URL
https://<bmc-ip>/

# Default credentials
Username: root
Password: OpenBmc
```

WebUI Features

Dashboard

The dashboard provides an at-a-glance view of system status:

- **System Health:** Overall health indicators
- **Power Status:** Current power state and controls
- **Quick Actions:** Common operations (reboot, power cycle)
- **Sensor Summary:** Critical sensor readings

Operations Pages

Page	Description
Server power operations	Power on/off, restart, one-time boot
KVM & Console	Remote keyboard/video/mouse, serial console
Virtual Media	Mount ISO images remotely
Firmware	Update BMC and BIOS firmware
SOL Console	Serial over LAN terminal

Hardware Pages

Page	Description
Inventory	FRU data, serial numbers, hardware info
Sensors	All sensors with readings and thresholds
System Info	CPU, memory, storage details
Event Logs	System event history

Settings Pages

Page	Description
Network	IP configuration, VLAN, DNS
Date & Time	NTP, timezone, manual time
User Management	Add/modify/delete users
Certificates	HTTPS certificates management
Policies	Power restore, boot options

Customization

Custom Branding

Create a custom theme by modifying or overriding files:

```
webui-vue/
├── src/
│   ├── assets/
│   │   ├── images/
│   │   │   ├── logo-header.svg    # Header logo (40px height)
│   │   │   ├── logo-login.svg    # Login page logo
│   │   │   └── favicon.ico        # Browser favicon
│   │   └── styles/
│   │       └── _colors.scss        # Color definitions
│   ├── env/
│   │   └── components/
│   │       └── AppHeader/          # Custom header component
│   └── views/
│       └── Login/
│           └── Login.vue           # Custom login page
```

Replacing the Logo

1. Create your logo (SVG recommended, ~40px height for header)
2. Replace files in your meta layer:

```
# In your recipe .bbappend
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"

SRC_URI += " \
    file://logo-header.svg \
    file://logo-login.svg \
    file://favicon.ico \
"

do_install:append() {
    install -m 0644 ${WORKDIR}/logo-header.svg \
        ${D}${datadir}/www/assets/images/
    install -m 0644 ${WORKDIR}/logo-login.svg \
        ${D}${datadir}/www/assets/images/
    install -m 0644 ${WORKDIR}/favicon.ico \
        ${D}${datadir}/www/
}
```

Custom Colors

Override SCSS color variables in your theme:

```
// _colors.scss
$primary: #0a3d6c;      // Primary brand color
$secondary: #6c757d;    // Secondary color
$success: #28a745;      // Success states
$danger: #dc3545;       // Error states
$warning: #ffc107;      // Warning states
$info: #17a2b8;         // Info states

// Header colors
$navbar-bg: #1a1a1a;
$navbar-color: #ffffff;

// Sidebar colors
$sidebar-bg: #2c3e50;
$sidebar-active: #3498db;
```


Build Custom Theme

```
# Clone webui-vue
git clone https://github.com/openbmc/webui-vue.git
cd webui-vue

# Install dependencies
npm install

# Set environment variables
export VUE_APP_COMPANY_NAME="My Company"
export VUE_APP_PRODUCT_NAME="Server Manager"

# Build for production
npm run build

# Output in dist/ directory
```

Adding Custom Pages

Create a New View

1. Create the Vue component:

```

<!-- src/views/Custom/MyPage.vue -->
<template>
  <b-container fluid="xl">
    <page-title :description="$t('pageMyPage.description')" />

    <b-row>
      <b-col>
        <b-card>
          <b-card-header>
            <h5>My Custom Page</h5>
          </b-card-header>
          <b-card-body>
            <p>Custom content here</p>
            <b-button variant="primary" @click="performAction">
              Do Something
            </b-button>
          </b-card-body>
        </b-card>
      </b-col>
    </b-row>
  </b-container>
</template>

<script>
import PageTitle from '@components/Global/PageTitle';

export default {
  name: 'MyPage',
  components: { PageTitle },
  data() {
    return {
      loading: false,
    };
  },
  methods: {
    performAction() {
      // Call Redfish API via Vuex store
      this.$store
        .dispatch('myCustomStore/performAction')
        .then(() => {
          this.$bvToast.toast('Action completed', {
            title: 'Success',
            variant: 'success',
          });
        })
        .catch((error) => {
          console.error(error);
        });
    },
  },
};
</script>

```

1. Add the route:

```
// src/router/routes.js
{
  path: '/custom/my-page',
  name: 'my-page',
  component: () => import('@views/Custom/MyPage'),
  meta: {
    title: 'My Page',
    requiresAuth: true,
  },
},
```

1. Add navigation entry:

```
// src/store/modules/GlobalStore.js
// Add to sidebar navigation items
{
  id: 'my-page',
  label: 'My Page',
  route: '/custom/my-page',
  icon: 'iconCustom',
},
```

Create a Vuex Store Module

```
// src/store/modules/MyCustomStore.js
import api from '@store/api';

const MyCustomStore = {
  namespaced: true,
  state: {
    data: null,
  },
  getters: {
    getData: (state) => state.data,
  },
  mutations: {
    setData: (state, data) => (state.data = data),
  },
  actions: {
    async fetchData({ commit }) {
      return await api
        .get('/redfish/v1/Oem/MyVendor/MyResource')
        .then(({ data }) => {
          commit('setData', data);
          return data;
        })
        .catch((error) => {
          console.error(error);
          throw error;
        });
    },
    async performAction({ dispatch }) {
      return await api
        .post('/redfish/v1/Oem/MyVendor/MyResource/Actions/MyAction')
        .then(() => dispatch('fetchData'))
        .catch((error) => {
          console.error(error);
          throw error;
        });
    },
  },
};

export default MyCustomStore;
```

Internationalization (i18n)

Adding Translations

WebUI uses vue-i18n for internationalization:

```
// src/locales/en-US.json
{
  "pageMyPage": {
    "description": "Description of my custom page",
    "title": "My Page Title",
    "actionButton": "Perform Action"
  }
}
```

Using Translations

```
<template>
  <h1></h1>
  <p></p>
  <b-button></b-button>
</template>
```

Adding New Language

1. Create locale file: `src/locales/de-DE.json`
2. Add to locale configuration:

```
// src/main.js
import deDE from './locales/de-DE.json';

const i18n = new VueI18n({
  locale: 'en-US',
  messages: {
    'en-US': enUS,
    'de-DE': deDE,
  },
});
```

Configuration Options

Build Environment Variables

Set these in your build environment or `.env` file:

```
# .env.production
VUE_APP_COMPANY_NAME=MyCompany
VUE_APP_PRODUCT_NAME=Server Manager
VUE_APP_THEME=custom
VUE_APP_SERVER_ROOT=
VUE_APP_ROUTER_BASE=
```

Feature Flags

Enable/disable features at build time:

```
// vue.config.js
module.exports = {
  // Enable/disable features
  pluginOptions: {
    features: {
      // Show/hide features
      kvm: true,
      virtualMedia: true,
      solConsole: true,
      firmware: true,
      network: true,
      security: true,
    },
  },
};
```

Custom Navigation

Add custom external links:

```
# Environment variable (JSON format)
VUE_APP_INTERNAL_LINKS='[
  {"label":"Support","url":"https://support.mycompany.com"},
  {"label":"Documentation","url":"https://docs.mycompany.com"}
]'
```

Security Configuration

Session Management

WebUI uses Redfish sessions:

```
// Login flow
1. User enters credentials
2. POST to /redfish/v1/SessionService/Sessions
3. Store X-Auth-Token in Vuex
4. Include token in all API requests
5. Token stored in memory (lost on refresh)
```

HTTPS Requirements

bmcweb enforces HTTPS for WebUI:

```
# Certificate location
/etc/ssl/certs/https/server.pem

# Generate new certificate
openssl req -x509 -newkey rsa:2048 \
  -keyout /etc/ssl/private/server.pem \
  -out /etc/ssl/certs/server.pem \
  -days 365 -nodes \
  -subj "/CN=openbmc"
```

Content Security Policy

bmcweb sets CSP headers:

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self';
  style-src 'self' 'unsafe-inline';
  img-src 'self' data;;
```

Development Workflow

Local Development

```
# Clone repository
git clone https://github.com/openbmc/webui-vue.git
cd webui-vue

# Install dependencies
npm install

# Run development server with proxy to BMC
# Create vue.config.js:
module.exports = {
  devServer: {
    proxy: {
      '/redfish': {
        target: 'https://192.168.1.100',
        secure: false,
        changeOrigin: true,
      },
    },
  },
};

# Start dev server
npm run serve

# Access at http://localhost:8080
```

Building for Production

```
# Production build
npm run build

# Output in dist/
ls dist/

# Deploy to BMC
scp -r dist/* root@bmc-ip:/usr/share/www/
```

Running Tests

```
# Unit tests
npm run test:unit

# Linting
npm run lint

# Fix linting issues
npm run lint:fix
```

Troubleshooting

Login Failures

```
# Check bmcweb is running
systemctl status bmcweb

# Verify credentials via API
curl -k -u root:openBmc https://localhost/redfish/v1/

# Check session service
curl -k -u root:openBmc \
  https://localhost/redfish/v1/SessionService
```

Page Not Loading

```
# Check WebUI files exist
ls -la /usr/share/www/

# Check bmcweb logs
journalctl -u bmcweb -f

# Verify static file serving
curl -k https://localhost/index.html
```

API Errors in Console

```
# Check Redfish endpoint
curl -k -u root:openBmc \
  https://localhost/redfish/v1/Systems/system

# Verify D-Bus services
busctl tree xyz.openbmc_project.ObjectMapper
```

Browser Console Debug

Open browser Developer Tools (F12):


```
// Check Vuex store state
$store.state

// Check authentication
$store.state.authentication.authToken

// Manually call API
fetch('/redfish/v1/Systems/system', {
  headers: {'X-Auth-Token': $store.state.authentication.authToken}
}).then(r => r.json()).then(console.log)
```

Enabling/Disabling WebUI

Remove from Image

```
# Remove WebUI from build
IMAGE_INSTALL:remove = "webui-vue"

# Or create a minimal bmcweb without WebUI serving
EXTRA_OEMESON:pn-bmcweb = " \
    -Dstatic-hosting=disabled \
"
```

Disable at Runtime

```
# WebUI is served by bmcweb, so stopping bmcweb disables both
# WebUI and Redfish API

# To keep Redfish but remove WebUI files:
rm -rf /usr/share/www/*

# Restart bmcweb
systemctl restart bmcweb
```

References

- [webui-vue Repository](#)
 - [Vue.js Documentation](#)
 - [Bootstrap-Vue](#)
 - [OpenBMC Web UI Design](#)
-

Deep Dive

This section provides detailed technical information for developers who want to understand the WebUI architecture and internals.

WebUI Request Flow



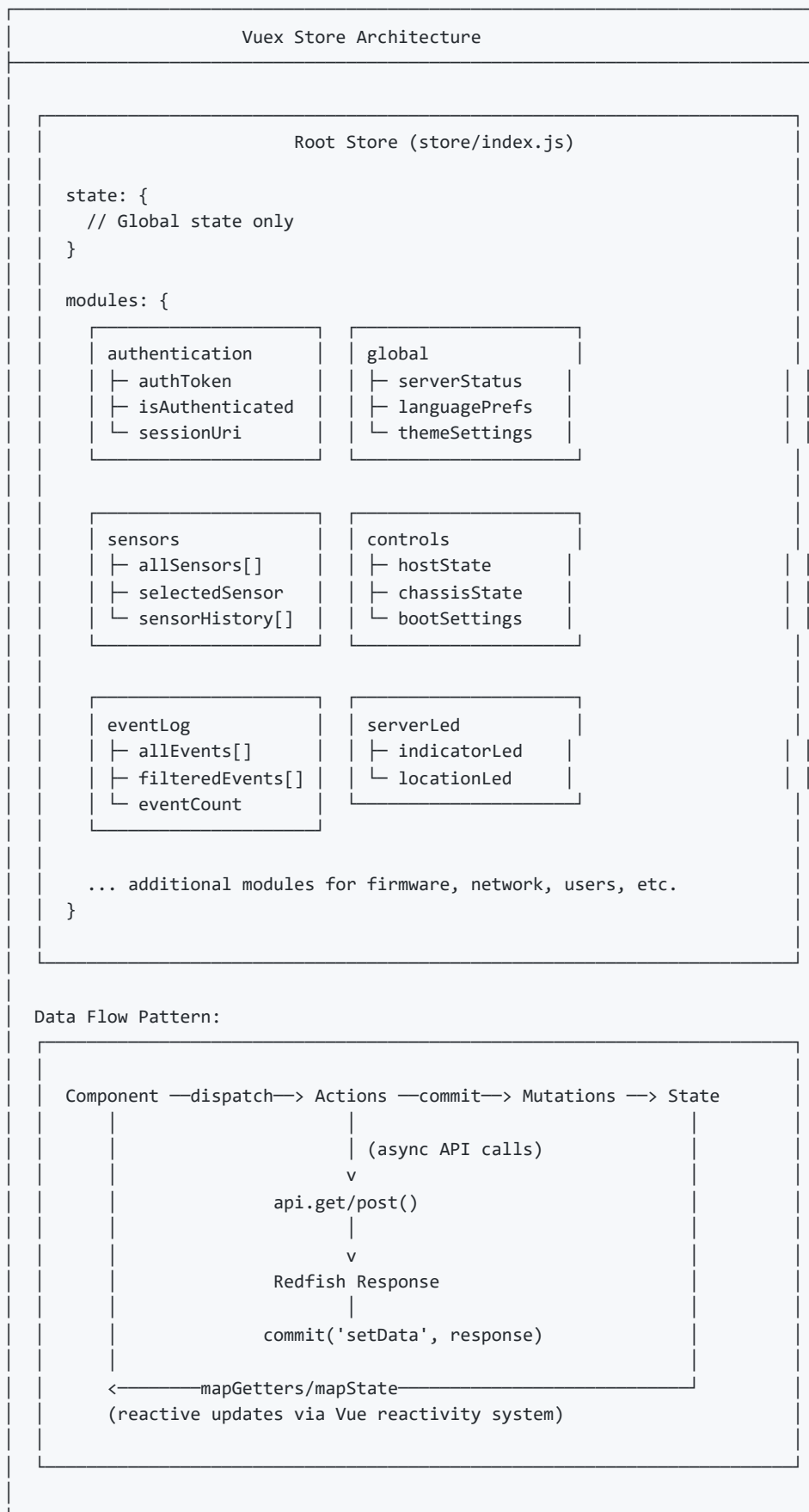
|
v

phosphor-state-manager

Property Set Handler:

- Validate transition request
- Trigger GPIO power sequence
- Update CurrentHostState asynchronously

Vuex State Management Architecture



Session Authentication Flow

Redfish Session Authentication

Login Flow:

1. User submits credentials via Login.vue

```
POST /redfish/v1/SessionService/Sessions
Content-Type: application/json
Body: { "UserName": "root", "Password": "0penBmc" }
```

|
v

2. bmcweb validates credentials via PAM

```
pamAuthenticateUser(username, password)
-> pam_unix.so checks /etc/shadow
-> Returns PAM_SUCCESS or PAM_AUTH_ERR
```

|
v

3. bmcweb creates session and returns token

```
HTTP 201 Created
Location: /redfish/v1/SessionService/Sessions/abc123
X-Auth-Token: <64-char-random-token>

Body: {
  "@odata.id": "/redfish/v1/SessionService/Sessions/abc123",
  "UserName": "root"
}
```

|
v

4. WebUI stores token in Vuex (memory only - not localStorage)

```
commit('setAuthToken', response.headers['x-auth-token']);
commit('setSessionUri', response.headers['location']);
```

Session Storage in bmcweb:

SessionStore (in-memory map):

```
sessions_["abc123"] = {
  uniqueId: "abc123",
  sessionToken: "<64-char-token>",
  username: "root",
  csrfToken: "<csrf-token>",
  clientId: "192.168.1.100",
  createTime: <timestamp>,
  lastUpdateTime: <timestamp>,
  userRole: "Administrator"
}
```

Session Timeout: 30 minutes (configurable)

Max Sessions: 10 per user (configurable)

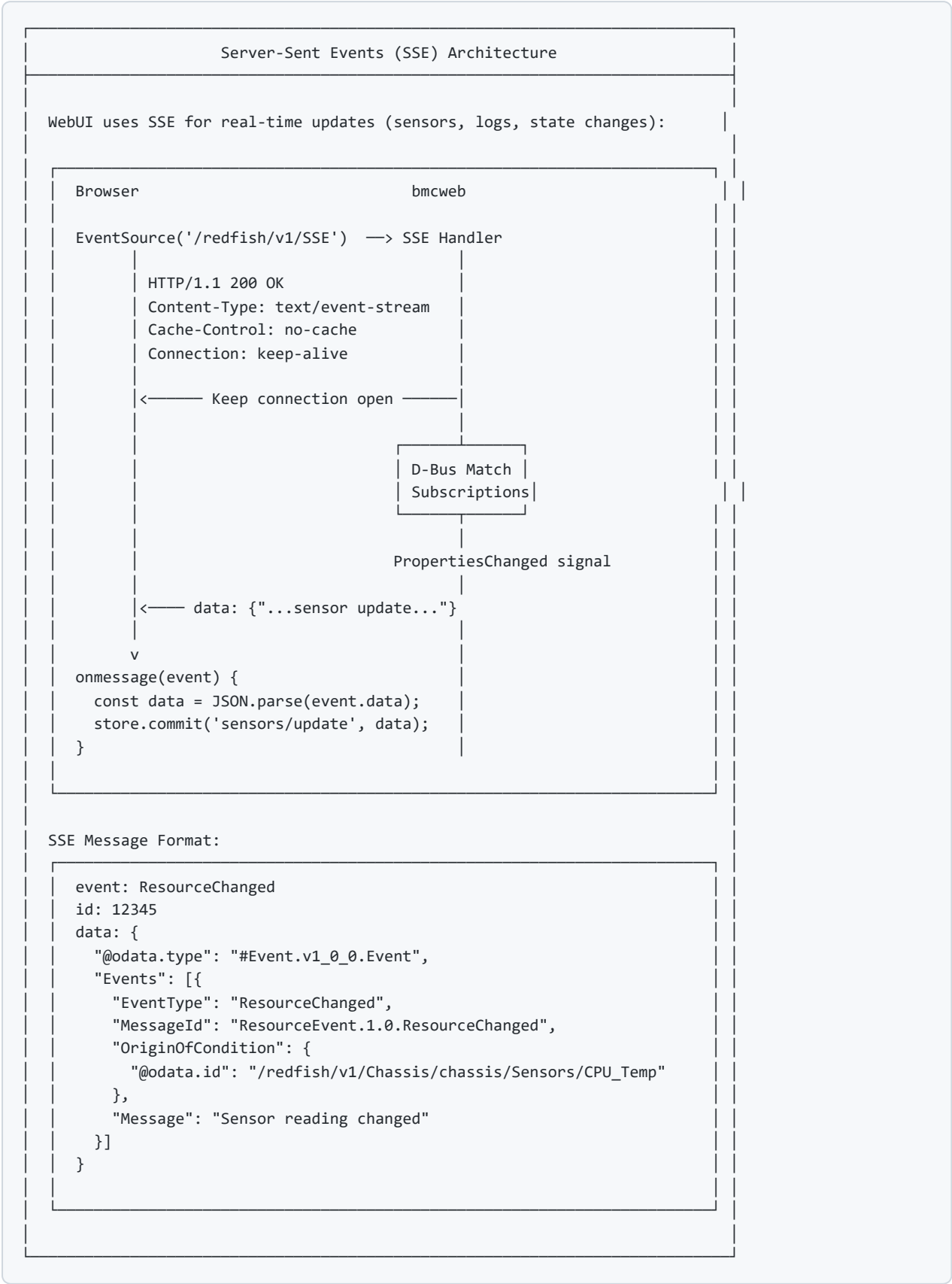
Authenticated Request:

```
GET /redfish/v1/Systems/system
X-Auth-Token: <session-token>
```

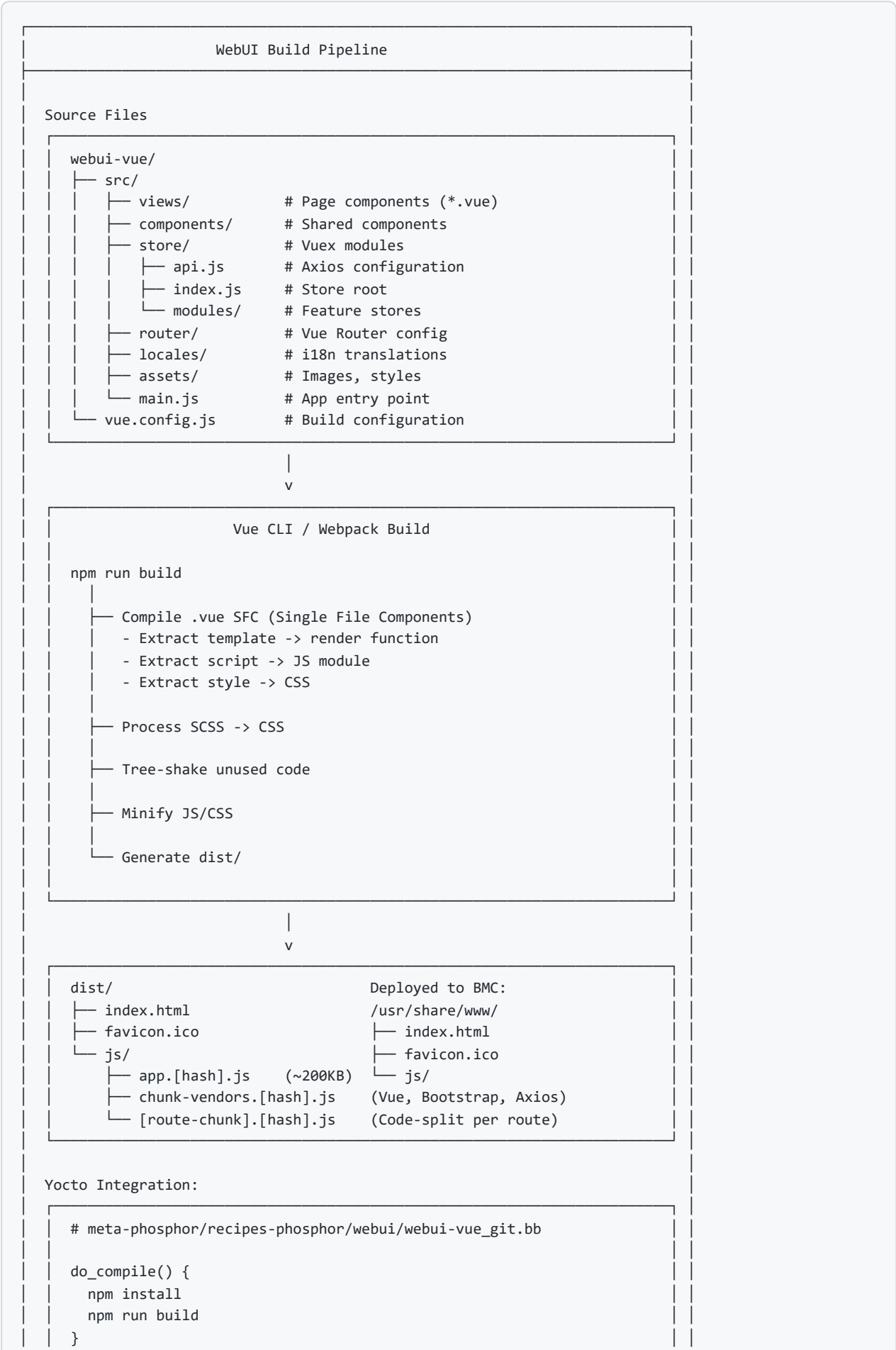
bmcweb validation:

1. Extract token from X-Auth-Token header
2. Look up session by token in SessionStore
3. Check session not expired
4. Update lastUpdatedTime (session keep-alive)
5. Attach user privileges to request context
6. Process request with user's RBAC privileges

WebSocket Event Streaming



Build and Bundle Process




```
do_install() {
    install -d ${D}${datadir}/www
    cp -r ${S}/dist/* ${D}${datadir}/www/
}

# bmcweb serves files from /usr/share/www/
```

Source Code References

Component	Repository	Key Files
WebUI Application	openbmc/webui-vue	<code>src/main.js</code> , <code>src/App.vue</code>
Vuex Stores	openbmc/webui-vue	<code>src/store/modules/</code>
API Client	openbmc/webui-vue	<code>src/store/api.js</code>
Router Configuration	openbmc/webui-vue	<code>src/router/index.js</code> , <code>src/router/routes.js</code>
bmcweb Static Hosting	openbmc/bmcweb	<code>http/http_server.hpp</code> , <code>include/webassets.hpp</code>
Session Management	openbmc/bmcweb	<code>include/sessions.hpp</code> , <code>redfish-core/lib/redfish_sessions.hpp</code>
SSE Implementation	openbmc/bmcweb	<code>include/server_sent_events.hpp</code>

Tested on: OpenBMC master, QEMU romulus

KVM Guide

Configure remote keyboard, video, and mouse access on OpenBMC.

Table of Contents

1. [Overview](#)
2. [Hardware Requirements](#)
 1. [Video Capture](#)
 2. [USB Device Controller](#)
3. [Setup & Configuration](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [bmcweb KVM Options](#)
 3. [Device Tree Configuration](#)
 4. [Runtime Configuration](#)
4. [Accessing KVM](#)
 1. [Via WebUI](#)
 2. [Via Direct noVNC](#)
 3. [Via VNC Client](#)
5. [Configuration Options](#)
 1. [Video Quality Settings](#)
 2. [USB HID Configuration](#)
 3. [Session Limits](#)
6. [Power State Considerations](#)
 1. [KVM Availability by State](#)
 2. [BIOS/UEFI Configuration](#)
7. [Input Handling](#)
 1. [Keyboard Mapping](#)
 2. [Special Key Combinations](#)
 3. [Mouse Configuration](#)
8. [Troubleshooting](#)
 1. [No Video Display](#)
 2. [Black Screen](#)
 3. [Keyboard/Mouse Not Working](#)
 4. [Connection Drops](#)
 5. [Performance Issues](#)
9. [Security Considerations](#)
 1. [Authentication](#)
 2. [Session Management](#)
 3. [Encryption](#)

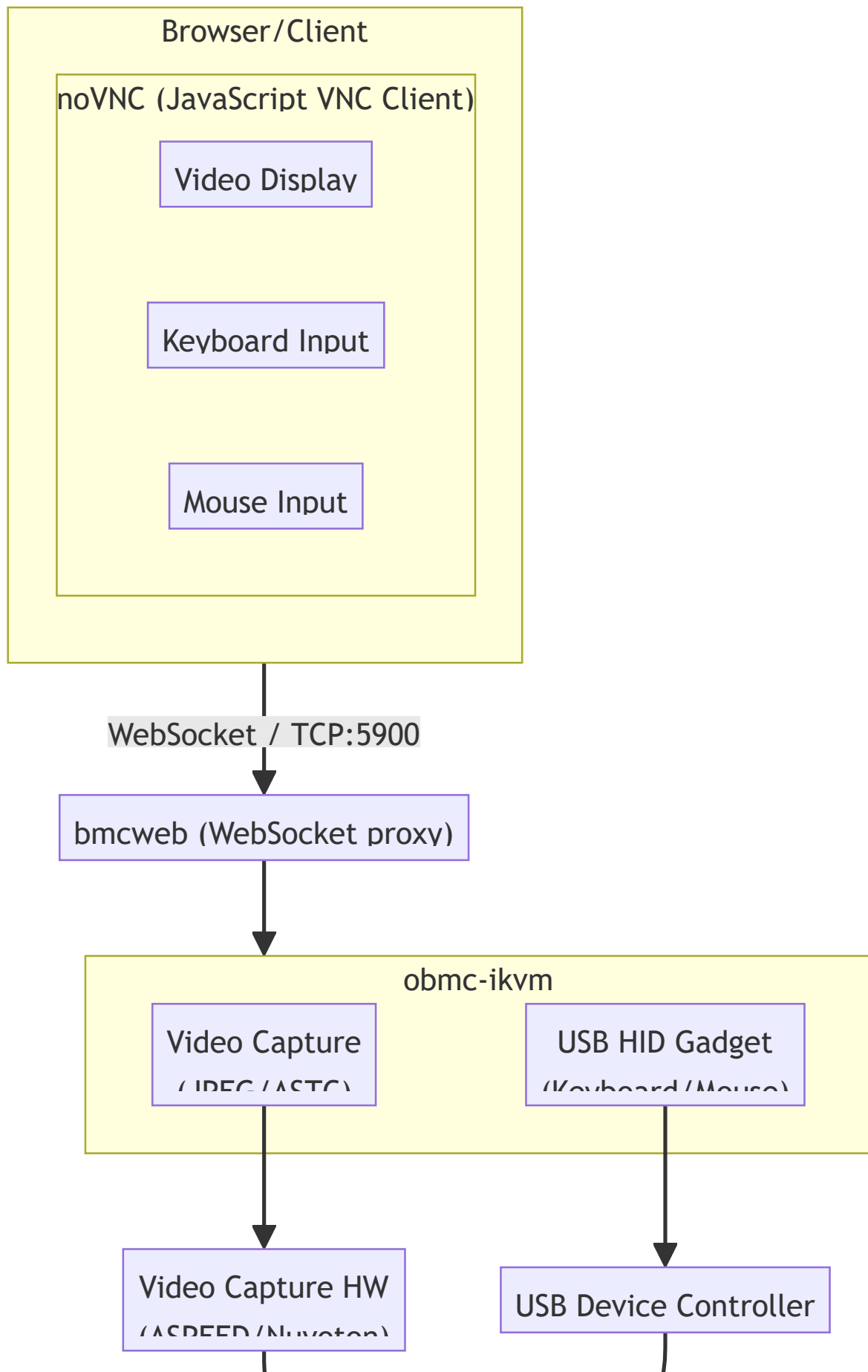
- 10. [Enabling/Disabling KVM](#)
 - 1. [Build-Time Disable](#)
 - 2. [Runtime Disable](#)
 - 11. [Deep Dive](#)
 - 1. [Video Capture Pipeline](#)
 - 2. [RFB \(VNC\) Protocol Frame Updates](#)
 - 3. [USB HID Gadget Protocol](#)
 - 4. [WebSocket to VNC Bridge](#)
 - 5. [Source Code Reference](#)
 - 12. [References](#)
-

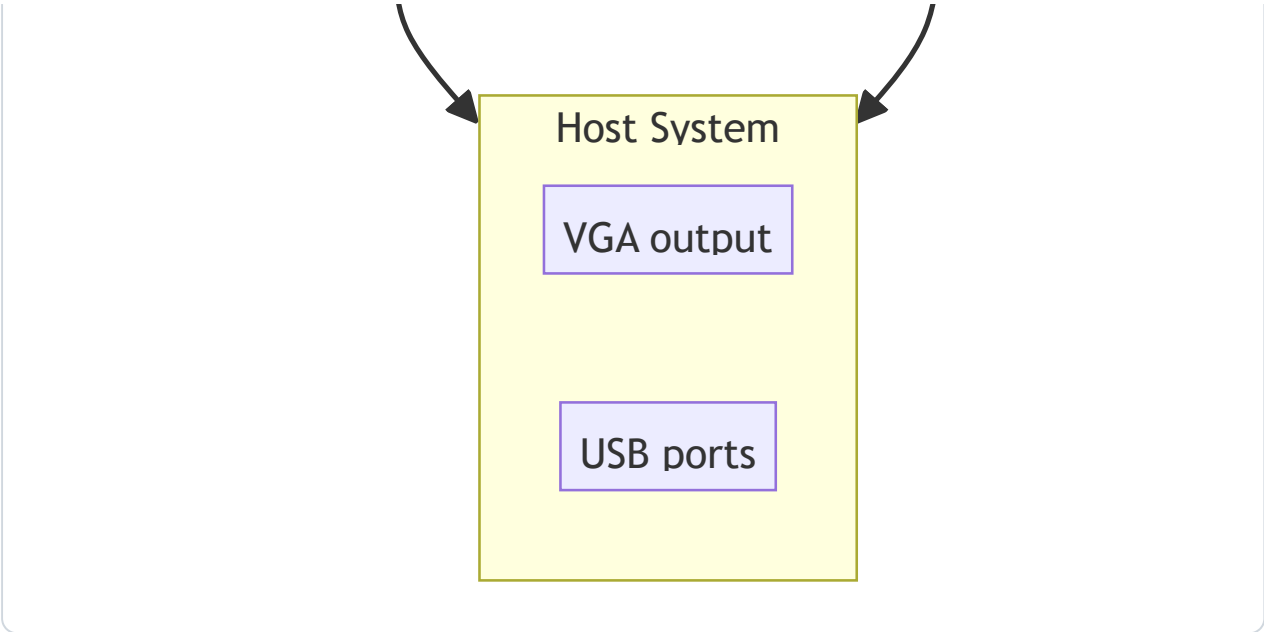
Overview

KVM (Keyboard, Video, Mouse) provides remote graphical console access to the host system, allowing operators to interact with the system as if they were physically present. OpenBMC implements KVM through **obmc-ikvm**.



KVM Architecture





► ASCII-art version (click to expand)

Hardware Requirements

Video Capture

KVM requires hardware video capture capability:

BMC SoC	Video Capture	Compression
ASPEED AST2500/2600	VGA capture engine	JPEG, ASPEED proprietary
Nuvoton NPCM7xx	GFX capture	JPEG

USB Device Controller

For keyboard/mouse input:

BMC SoC	USB Controller	Gadget Support
ASPEED AST2500/2600	USB 2.0 device	HID gadget
Nuvoton NPCM7xx	USB device	HID gadget

Setup & Configuration

Build-Time Configuration (Yocto)

```
# In your machine .conf or local.conf

# Include KVM support
IMAGE_INSTALL:append = " obmc-ikvm"

# Enable KVM in bmcweb
EXTRA_OEMESON:pn-bmcweb = " \
    -Dkvm=enabled \
"

# For ASPEED-based systems
PREFERRED_PROVIDER_virtual/obmc-host-ipmi-hw = "phosphor-ipmi-kcs"
MACHINE_FEATURES:append = " obmc-ikvm"
```

bmcweb KVM Options

Option	Default	Description
<code>kvm</code>	enabled	Enable KVM WebSocket support
<code>insecure-kvm-auth</code>	disabled	Allow unauthenticated KVM (testing only)

Device Tree Configuration

ASPEED systems require video capture configuration:

```
// Device tree snippet for AST2500/2600
&video {
    status = "okay";
    memory-region = <&video_memory>;
};

&gfx {
    status = "okay";
    memory-region = <&gfx_memory>;
};
```

Runtime Configuration

```
# Check obmc-ikvm service
systemctl status obmc-ikvm

# View KVM logs
journalctl -u obmc-ikvm -f

# Restart KVM service
systemctl restart obmc-ikvm
```

Accessing KVM

Via WebUI

1. Login to WebUI at `https://<bmc-ip>/`
2. Navigate to **Operations** → **KVM Console**
3. Click **Open in new tab** or use embedded viewer
4. Interact with host display, keyboard, and mouse

Via Direct noVNC

```
# Access noVNC directly (if exposed)
https://<bmc-ip>/kvm/0

# Or via Redfish link
curl -k -u root:0penBmc \
  https://localhost/redfish/v1/Managers/bmc
# Look for GraphicalConsole link
```

Via VNC Client

For direct VNC client access (if enabled):

```
# Connect with VNC viewer
vncviewer bmc-ip:5900

# With password (if configured)
vncviewer -passwd ~/.vnc/passwd bmc-ip:5900
```

Configuration Options

Video Quality Settings

```
# Video quality is typically controlled via obmc-ikvm configuration
# Check available options
obmc-ikvm --help

# Common options:
# -f framerate      : Target frame rate
# -q quality        : JPEG quality (1-100)
# -r resolution     : Force resolution
```

USB HID Configuration

```
# USB HID gadget is typically auto-configured
# Check USB gadget status
ls /sys/kernel/config/usb_gadget/

# Verify HID devices
ls /dev/hidg*
```


Session Limits

Configure via bmcweb:

```
// In bmcweb configuration
// Maximum concurrent KVM sessions (typically 1-2)
static constexpr size_t maxKvmSessions = 2;
```

Power State Considerations

KVM Availability by State

Host Power State	Video Capture	KVM Available
Off	No output	Blank screen
On (POST)	BIOS output	Yes
On (OS)	OS display	Yes
Sleep	May blank	Limited

BIOS/UEFI Configuration

For best KVM experience, configure host BIOS:

1. **Set video output** to onboard/BMC VGA
2. **Disable** GPU priority if using external graphics
3. **Enable** legacy VGA for BIOS screens

Input Handling

Keyboard Mapping

```
Client Keyboard → noVNC → bmcweb WebSocket → obmc-ikvm → USB HID → Host
```

Common key mappings:

Client Key	Host Key	Notes
Ctrl+Alt+Del	Ctrl+Alt+Del	Captured by noVNC
F1-F12	F1-F12	Pass through
Alt+Tab	Alt+Tab	May be captured locally
Print Screen	Print Screen	Pass through

Special Key Combinations

Access via noVNC toolbar or keyboard shortcuts:

- **Ctrl+Alt+Delete**: Send to remote
- **Ctrl+Alt+F1-F6**: Virtual terminal switch (Linux)
- **Windows Key**: May need special handling

Mouse Configuration

```
# Mouse is typically absolute positioning
# If relative mode needed, configure in obmc-ikvm

# Check mouse mode
cat /sys/class/usb_role/*/role
```

Troubleshooting

No Video Display

```
# Check video capture hardware
ls /dev/video*

# Check video capture service
systemctl status obmc-ikvm

# View video device info
v4l2-ctl --all

# Check memory regions
cat /proc/iomem | grep -i video
```

Black Screen

```
# Verify host is powered on
obmcutil state

# Check if host is generating video
# May need to check VGA cable connection

# Try resetting video capture
systemctl restart obmc-ikvm
```

Keyboard/Mouse Not Working

```
# Check USB HID gadget
ls /dev/hidg*

# Verify USB connection to host
# Check host BIOS for USB settings

# Reload USB gadget
modprobe -r g_hid
modprobe g_hid
```

Connection Drops

```
# Check bmcweb WebSocket
journalctl -u bmcweb | grep -i kvm

# Check network connectivity
ping bmc-ip

# Verify KVM session count
# Too many sessions may cause issues
```

Performance Issues

```
# Reduce quality for bandwidth
# Adjust frame rate
# Check network latency

# Monitor bandwidth usage
iftop -i eth0
```

Security Considerations

Authentication

KVM access requires Redfish authentication:

```
# KVM WebSocket is authenticated via session token
# Ensure strong passwords for KVM users

# Create operator account for KVM
curl -k -u root:openBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "UserName": "kvmuser",
    "Password": "SecurePass123!",
    "RoleId": "Operator"
  }' \
  https://localhost/redfish/v1/AccountService/Accounts
```

Session Management

```
# View active sessions
curl -k -u root:openBmc \
  https://localhost/redfish/v1/SessionService/Sessions

# Terminate KVM session (close WebSocket)
# Sessions auto-terminate on browser close
```

Encryption

All KVM traffic is encrypted via HTTPS/WSS:

```
Browser ↔ bmcweb: TLS 1.2/1.3
```

Enabling/Disabling KVM

Build-Time Disable

```
# Remove KVM from build
EXTRA_OEMESON:pn-bmcweb = " \
    -Dkvm=disabled \
"
IMAGE_INSTALL:remove = "obmc-ikvm"
```

Runtime Disable

```
# Stop KVM service
systemctl stop obmc-ikvm
systemctl disable obmc-ikvm

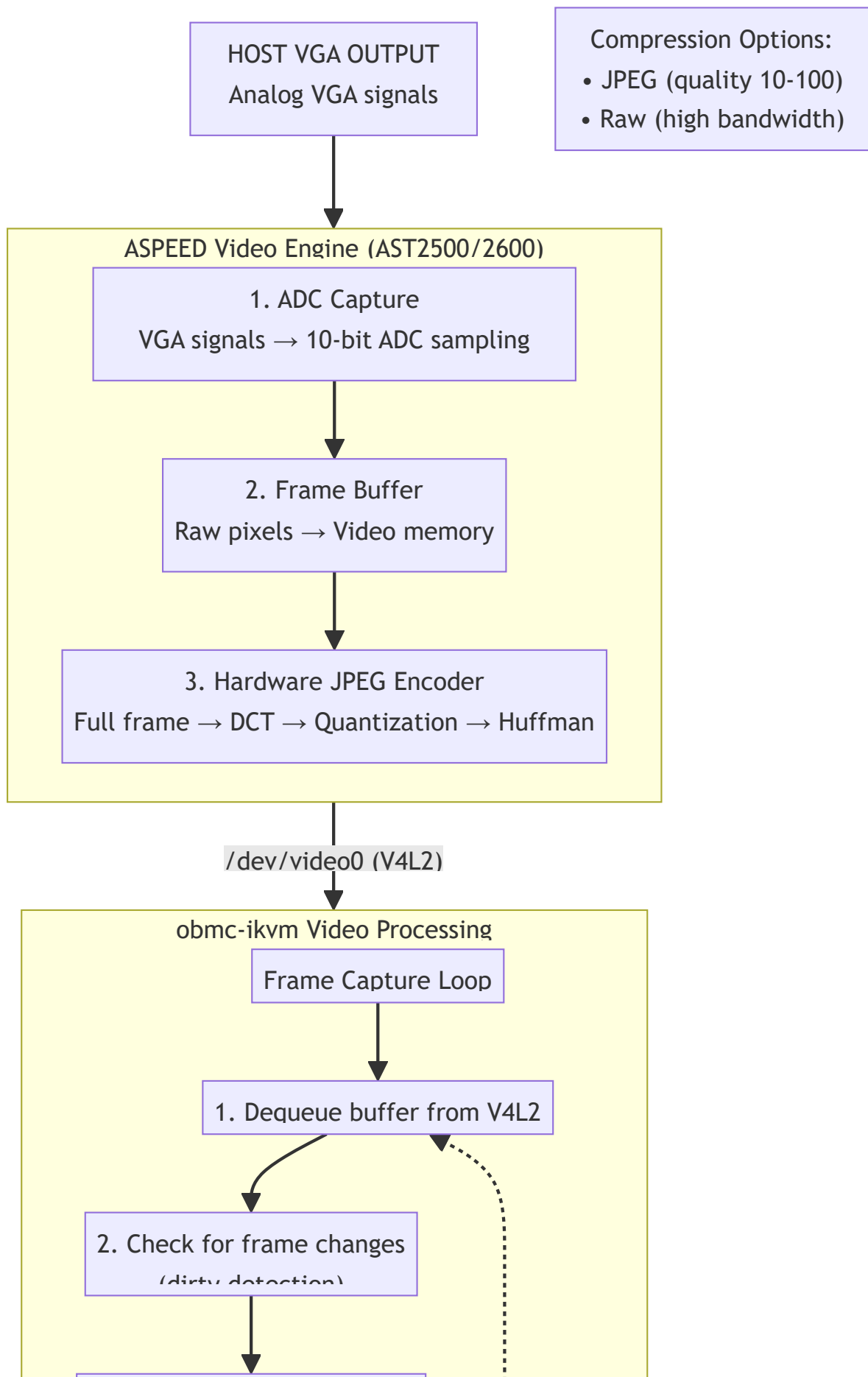
# Re-enable
systemctl enable obmc-ikvm
systemctl start obmc-ikvm
```

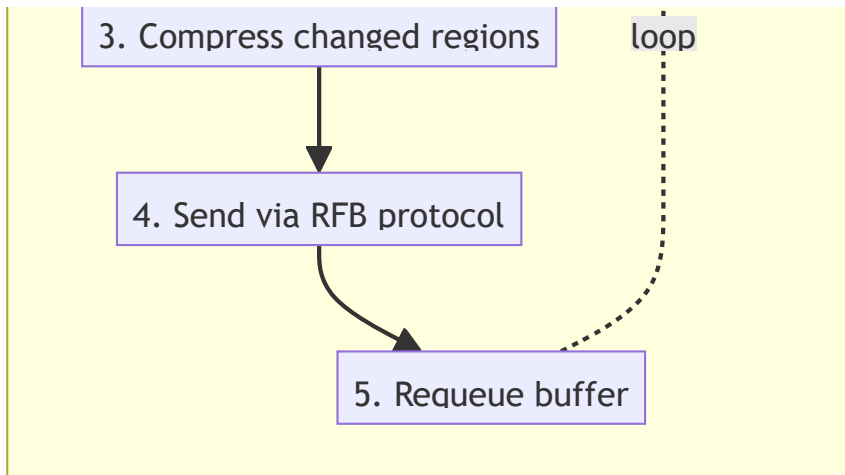
Deep Dive

Advanced implementation details for KVM developers.



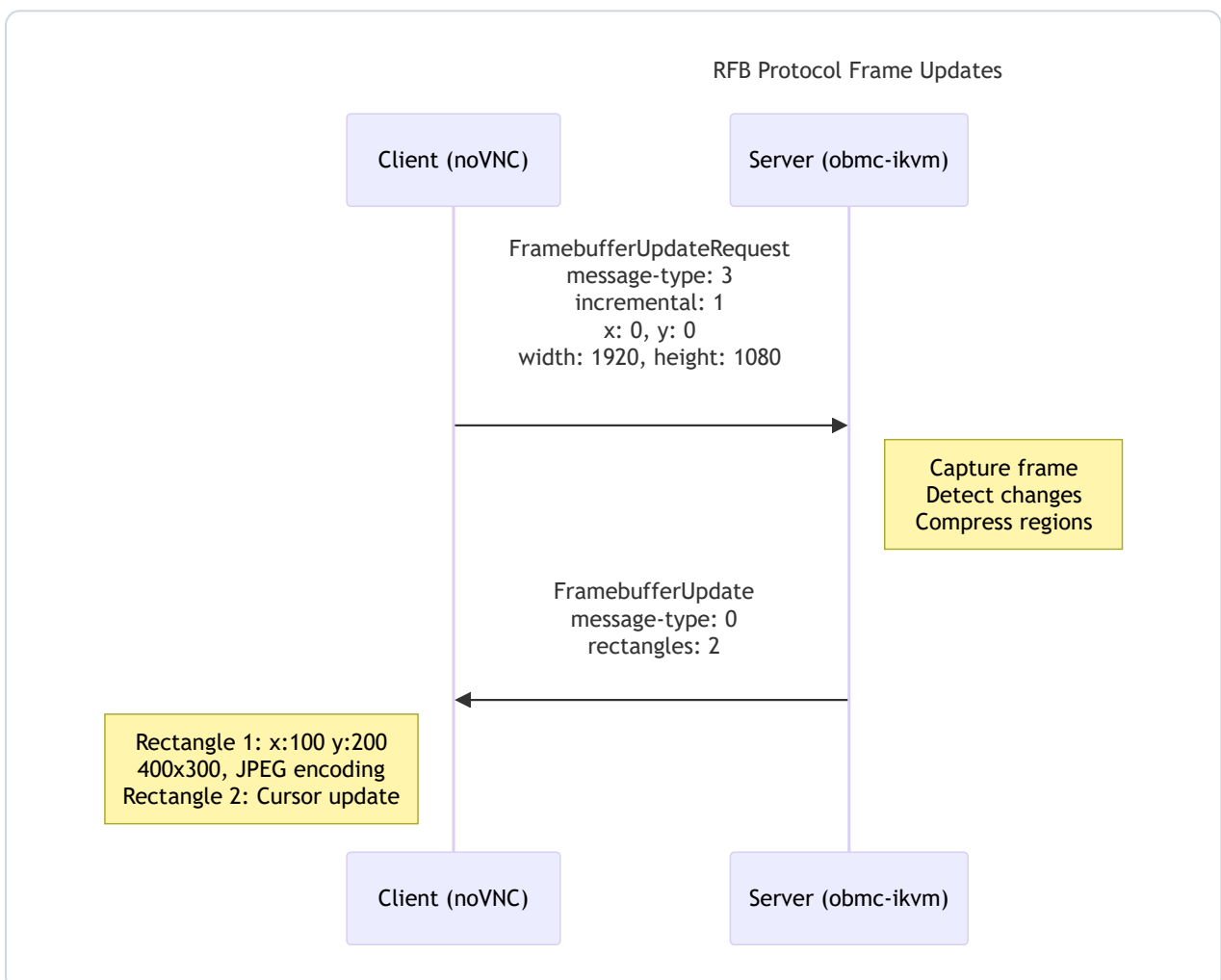
Video Capture Processing Pipeline





► ASCII-art version (click to expand)

RFB (VNC) Protocol Frame Updates



Encoding Types Supported:

Encoding	ID	Description
Raw	0	Uncompressed pixels (RGB888/RGB565)
CopyRect	1	Copy from another screen region
RRE	2	Rise and Run length Encoding
Hextile	5	16x16 tile-based compression
ZRLE	16	Zlib Run-Length Encoding
Tight	7	Tight compression with JPEG option
JPEG	21	JPEG-compressed rectangle
Cursor	-239	Client-side cursor rendering
DesktopSize	-223	Desktop size change notification

Tight Encoding Control Byte:

- Bits 0-2: zlib stream reset flags
 - Bits 3-4: fill mode / filter
 - Bits 5-7: compression type (0x09 = JPEG, 0x00-0x07 = Basic)
 - JPEG Quality: pseudo-encoding -23 to -32 sets quality 0-9 (maps to 5-95%)
- ASCII-art version (click to expand)

USB HID Gadget Implementation

USB HID GADGET SETUP

/sys/kernel/config/usb_gadget/kvm_gadget/

| idVendor

0x1d6b (Linux Foundation)

| idProduct

0x0104 (Multifunction Composite Gadget)

| bcdDevice

0x0100

| bcdUSB

0x0200 (USB 2.0)

| strings/0x409/

| serialnumber

BMC serial

| manufacturer

OpenBMC

| product

Virtual KVM

| configs/c.1/

| MaxPower

500 (mA)

| hid.keyboard -> functions/hid.keyboard

| hid.mouse -> functions/hid.mouse

| functions/

| hid.keyboard/

| protocol

1 (Keyboard)

| subclass

1 (Boot Interface)

| report_length

8

| report_desc

HID Report Descriptor (binary)

| hid.mouse/

| protocol

2 (Mouse)

| subclass

1 (Boot Interface)

| report_length

6

| report_desc

HID Report Descriptor (binary)

KEYBOARD HID REPORT FORMAT (8 bytes):

Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
Modifier Flags	Reserved (0x00)	Key 1	Key 2	Key 3	Key 4	Key 5	Key 6

Modifier Flags (Byte 0):

Bit 0: Left Ctrl

Bit 4: Right Ctrl

Bit 1: Left Shift

Bit 5: Right Shift

Bit 2: Left Alt

Bit 6: Right Alt

Bit 3: Left GUI

Bit 7: Right GUI (Windows key)

Example: Ctrl+Alt+Delete

[0x05, 0x00, 0x4C, 0x00, 0x00, 0x00, 0x00, 0x00]

0x05 = Left Ctrl (0x01) + Left Alt (0x04)

0x4C = Delete key scancode

MOUSE HID REPORT FORMAT (6 bytes - Absolute):

Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5
Buttons	X Low	X High	Y Low	Y High	Wheel

Buttons (Byte 0):

Bit 0: Left button Bit 2: Middle button
Bit 1: Right button Bits 3-7: Reserved

X/Y: 16-bit absolute position (0-32767 maps to screen)

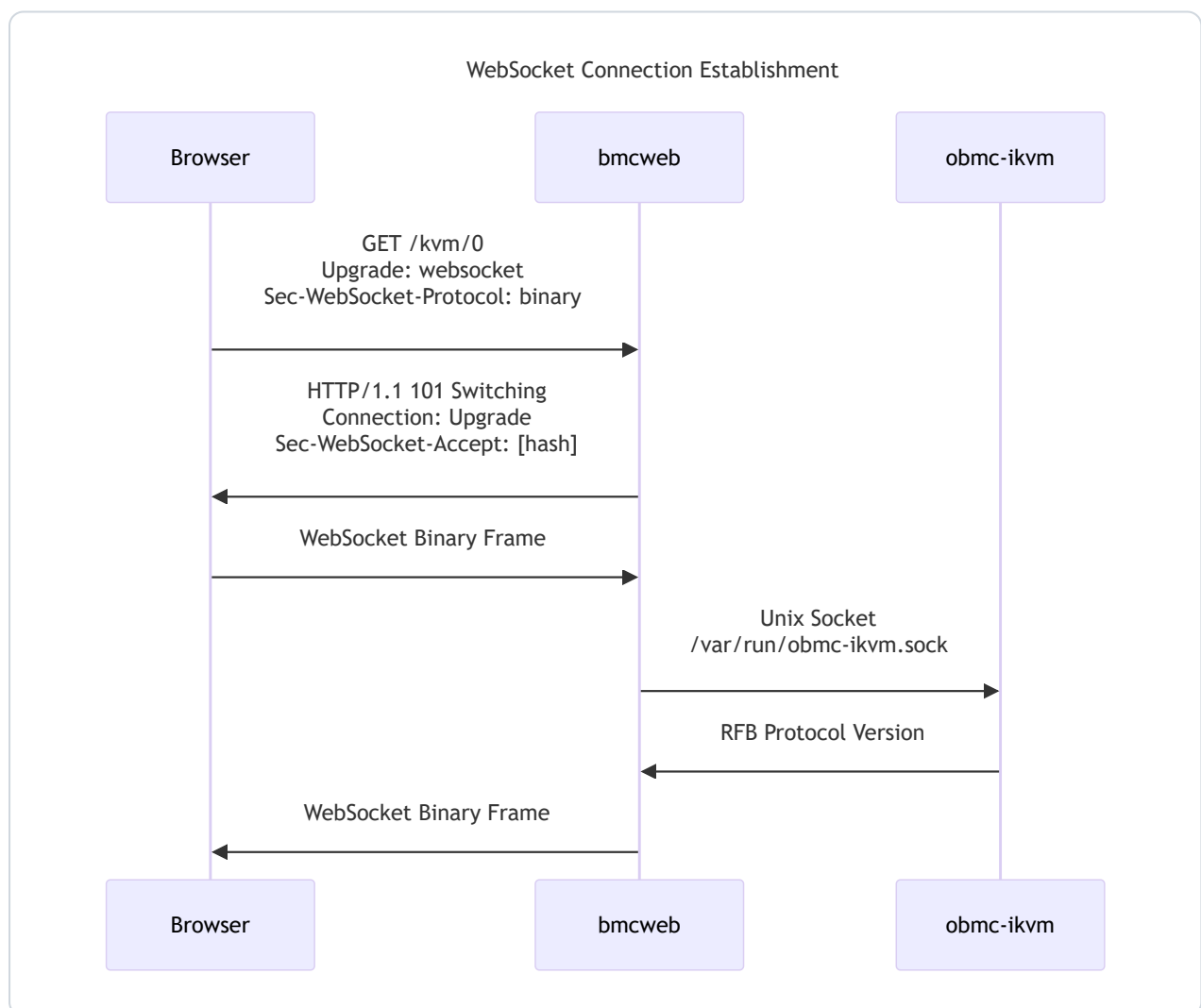
Wheel: Signed 8-bit scroll delta

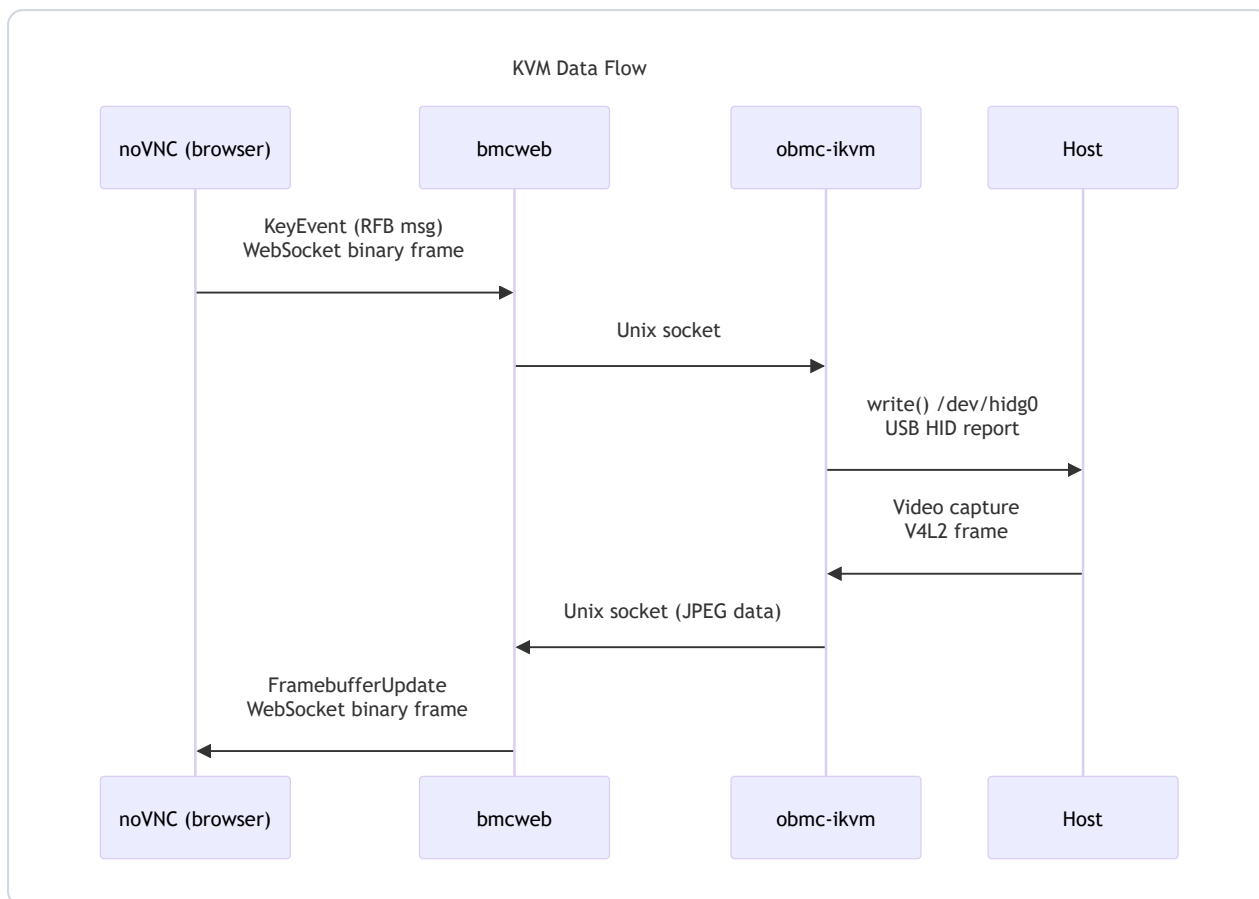
WRITING TO HID DEVICE:

```
// Send keyboard report
int fd = open("/dev/hidg0", O_WRONLY);
uint8_t report[8] = {0x05, 0, 0x4C, 0, 0, 0, 0, 0}; // Ctrl+Alt+Del
write(fd, report, sizeof(report));
```

```
// Key release
memset(report, 0, sizeof(report));
write(fd, report, sizeof(report));
```

WebSocket to VNC Bridge





bmcweb KVM Handler (kvm.hpp):

```

void handleKvmWebSocket(crow::websocket::Connection& conn) {
    // 1. Connect to obmc-ikvm Unix socket
    int sock = socket(AF_UNIX, SOCK_STREAM, 0);
    struct sockaddr_un addr;
    addr.sun_family = AF_UNIX;
    strncpy(addr.sun_path, "/var/run/obmc-ikvm.sock", sizeof(...));
    connect(sock, (struct sockaddr*)&addr, sizeof(addr));

    // 2. Bidirectional forwarding
    // WebSocket → Unix socket (client input)
    conn.onMessage([sock](const std::string& msg) {
        write(sock, msg.data(), msg.size());
    });

    // Unix socket → WebSocket (video/responses)
    async_read(sock, buffer, [&conn](size_t bytes) {
        conn.sendBinary(buffer, bytes);
    });
}

```

► ASCII-art version (click to expand)

Source Code Reference

Key implementation files in [obmc-ikvm](#) and [bmcweb](#):

File	Description
<code>obmc-ikvm/ikvm_video.cpp</code>	V4L2 video capture and JPEG compression
<code>obmc-ikvm/ikvm_input.cpp</code>	USB HID gadget keyboard/mouse handling
<code>obmc-ikvm/ikvm_server.cpp</code>	RFB/VNC protocol server implementation
<code>bmcweb/include/kvm_websocket.hpp</code>	WebSocket to Unix socket bridge
<code>bmcweb/redfish-core/lib/managers.hpp</code>	GraphicalConsole Redfish resource

References

- [obmc-ikvm](#)
- [noVNC](#)
- [USB HID Gadget](#)
- [ASPEED Video Engine](#)

Tested on: OpenBMC master, requires hardware with video capture support

Virtual Media Guide

Configure remote ISO and image mounting on OpenBMC.

Table of Contents

1. [Overview](#)
2. [Modes of Operation](#)
 1. [Proxy Mode \(Recommended\)](#)
 2. [Legacy Mode](#)
 3. [NBD Proxy](#)
3. [Hardware Requirements](#)
4. [Setup & Configuration](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [bmcweb Virtual Media Options](#)
 3. [Runtime Configuration](#)
5. [Mounting Images](#)
 1. [Via WebUI](#)
 2. [Via Redfish API](#)
 1. [List Virtual Media Devices](#)
 2. [Mount from URL](#)
 3. [Check Mount Status](#)
 4. [Unmount Image](#)
 5. [Eject Action](#)
6. [Configuration Options](#)
 1. [Virtual Media Slots](#)
 2. [Transfer Protocol Support](#)
 3. [USB Gadget Configuration](#)
7. [Boot from Virtual Media](#)
 1. [Configure Boot Order](#)
 2. [One-Time Boot Sequence](#)
8. [WebSocket Mode](#)
 1. [Connecting via WebSocket](#)
 2. [NBD Protocol](#)
9. [Troubleshooting](#)
 1. [Image Not Mounting](#)
 2. [Host Not Seeing USB Device](#)
 3. [Slow Transfer Speed](#)
 4. [Boot Fails](#)
 5. [Authentication Errors](#)

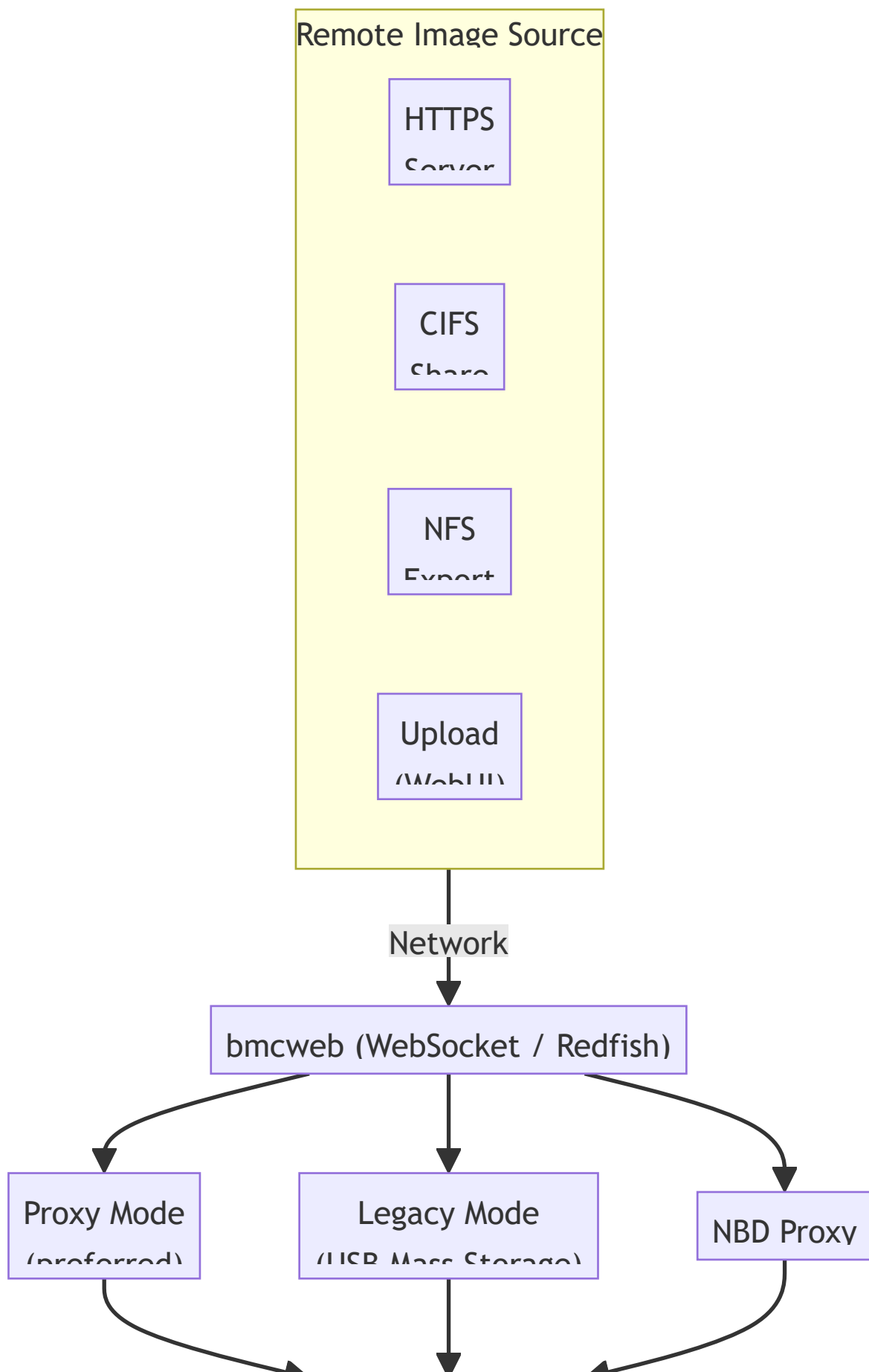
- 10. [Security Considerations](#)
 - 1. [Image Source Authentication](#)
 - 2. [Access Control](#)
 - 3. [Write Protection](#)
 - 11. [Enabling/Disabling Virtual Media](#)
 - 1. [Build-Time Disable](#)
 - 2. [Runtime Disable](#)
 - 12. [Deep Dive](#)
 - 1. [NBD \(Network Block Device\) Protocol](#)
 - 2. [USB Mass Storage Gadget Architecture](#)
 - 3. [Proxy Mode Data Flow](#)
 - 4. [Source Code Reference](#)
 - 13. [References](#)
-

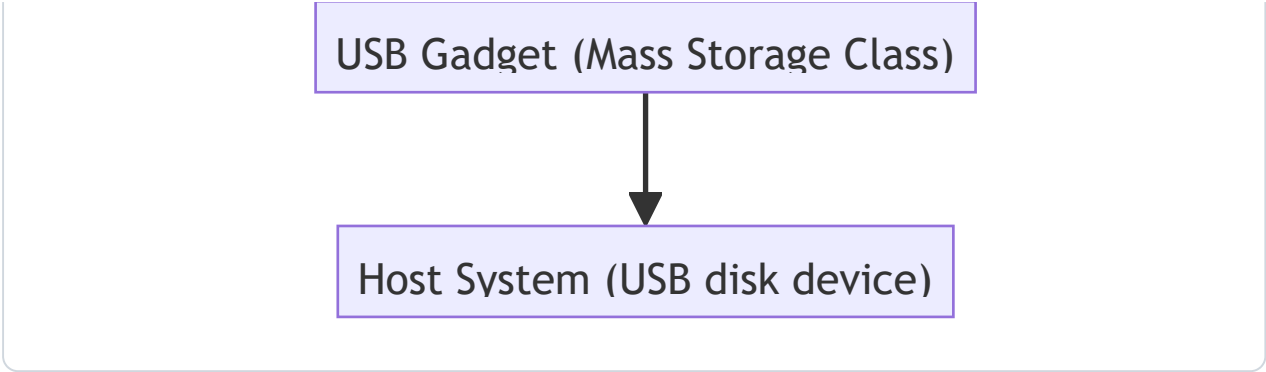
Overview

Virtual Media allows mounting remote disk images (ISO, IMG) to the host system over the network, appearing as local USB storage devices. This enables remote OS installation and recovery without physical media.



Virtual Media Architecture





► ASCII-art version (click to expand)

Modes of Operation

Proxy Mode (Recommended)

- Image streams from remote server through BMC
- No local storage required on BMC
- Supports HTTPS, NFS, CIFS sources
- Lower latency for network-accessible images

Legacy Mode

- Image stored locally on BMC filesystem
- Presented via USB mass storage
- Works offline once mounted
- Limited by BMC storage capacity

NBD Proxy

- Network Block Device protocol
- Efficient streaming protocol
- Used with WebSocket connections

Hardware Requirements

Virtual Media requires USB device controller:

BMC SoC	USB Controller	Mass Storage
ASPEED AST2500/2600	USB 2.0 device	Supported
Nuvoton NPCM7xx	USB 2.0 device	Supported

Setup & Configuration

Build-Time Configuration (Yocto)

```
# In your machine .conf or local.conf

# Include virtual media support
IMAGE_INSTALL:append = " virtual-media"

# Enable in bmcweb
EXTRA_OEMESON:pn-bmcweb = " \
    -Dvm-websocket=enabled \
    -Dvm-nbdproxy=enabled \
"

# Include NBD client (for proxy mode)
IMAGE_INSTALL:append = " nbd-client"
```

bmcweb Virtual Media Options

Option	Default	Description
<code>vm-websocket</code>	enabled	Virtual Media WebSocket support
<code>vm-nbdproxy</code>	enabled	NBD proxy for streaming
<code>insecure-vm-auth</code>	disabled	Unauthenticated VM (testing)

Runtime Configuration

```
# Check virtual media service
systemctl status virtual-media

# View logs
journalctl -u virtual-media -f

# Check USB gadget configuration
ls /sys/kernel/config/usb_gadget/
```

Mounting Images

Via WebUI

1. Login to WebUI at `https://<bmc-ip>/`
2. Navigate to **Operations** → **Virtual Media**
3. Select device type: **CD/DVD** or **USB**
4. Choose mount method:
 - **From URL:** Enter HTTP/HTTPS/NFS/CIFS path
 - **Upload:** Upload local file
5. Click **Mount**

Via Redfish API

List Virtual Media Devices

```
curl -k -u root:openBmc \
https://localhost/redfish/v1/Managers/bmc/VirtualMedia
```

Mount from URL

```
# Mount ISO from HTTPS server
curl -k -u root:openBmc -X PATCH \
-H "Content-Type: application/json" \
-d '{
  "Image": "https://example.com/images/ubuntu-22.04.iso",
  "TransferProtocolType": "HTTPS",
  "UserName": "",
  "Password": "",
  "WriteProtected": true,
  "Inserted": true
}' \
https://localhost/redfish/v1/Managers/bmc/VirtualMedia/Slot_0

# Mount from NFS
curl -k -u root:openBmc -X PATCH \
-H "Content-Type: application/json" \
-d '{
  "Image": "nfs://192.168.1.100/exports/images/boot.iso",
  "Inserted": true
}' \
https://localhost/redfish/v1/Managers/bmc/VirtualMedia/Slot_0

# Mount from CIFS/SMB
curl -k -u root:openBmc -X PATCH \
-H "Content-Type: application/json" \
-d '{
  "Image": "smb://192.168.1.100/share/images/install.iso",
  "UserName": "user",
  "Password": "password",
  "Inserted": true
}' \
https://localhost/redfish/v1/Managers/bmc/VirtualMedia/Slot_0
```

Check Mount Status

```
curl -k -u root:openBmc \
https://localhost/redfish/v1/Managers/bmc/VirtualMedia/Slot_0
```

Response:

```
{
  "@odata.id": "/redfish/v1/Managers/bmc/VirtualMedia/Slot_0",
  "Id": "Slot_0",
  "Name": "VirtualMedia",
  "MediaTypes": ["CD", "DVD"],
  "Image": "https://example.com/images/ubuntu.iso",
  "ImageName": "ubuntu.iso",
  "Inserted": true,
  "WriteProtected": true,
  "ConnectedVia": "URI",
  "TransferProtocolType": "HTTPS"
}
```

Unmount Image

```
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"Inserted": false}' \
  https://localhost/redfish/v1/Managers/bmc/VirtualMedia/Slot_0
```

Eject Action

```
curl -k -u root:openBmc -X POST \
  https://localhost/redfish/v1/Managers/bmc/VirtualMedia/Slot_0/Actions/VirtualMedia.EjectMedia
```

Configuration Options

Virtual Media Slots

```
# Default slots (varies by implementation)
Slot_0 - CD/DVD (read-only)
Slot_1 - USB storage (read-write)

# Check available slots
curl -k -u root:openBmc \
  https://localhost/redfish/v1/Managers/bmc/VirtualMedia | jq '.Members'
```

Transfer Protocol Support

Protocol	Port	Use Case
HTTPS	443	Secure download, most common
HTTP	80	Testing only
NFS	2049	Network file system
CIFS/SMB	445	Windows shares

USB Gadget Configuration

```
# USB mass storage gadget setup
# Typically auto-configured by virtual-media service

# View gadget configuration
cat /sys/kernel/config/usb_gadget/virtual_media/UDC

# Check backing file
ls -l /sys/kernel/config/usb_gadget/virtual_media/functions/mass_storage.usb0/lun.0/
```

Boot from Virtual Media

Configure Boot Order

```
# Set boot source override to CD
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{
    "Boot": {
      "BootSourceOverrideTarget": "Cd",
      "BootSourceOverrideEnabled": "Once"
    }
  }' \
  https://localhost/redfish/v1/Systems/system
```

One-Time Boot Sequence

1. Mount installation ISO
2. Set boot override to CD
3. Reset/power on system
4. System boots from virtual CD

```
# Complete workflow
# 1. Mount ISO
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"Image": "https://server/ubuntu.iso", "Inserted": true}' \
  https://localhost/redfish/v1/Managers/bmc/VirtualMedia/Slot_0

# 2. Set boot source
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"Boot": {"BootSourceOverrideTarget": "Cd", "BootSourceOverrideEnabled": "Once"}}' \
  https://localhost/redfish/v1/Systems/system

# 3. Power cycle
curl -k -u root:openBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{"ResetType": "ForceRestart"}' \
  https://localhost/redfish/v1/Systems/system/Actions/ComputerSystem.Reset
```

WebSocket Mode

Connecting via WebSocket

For direct file streaming:

```
// JavaScript example
const ws = new WebSocket('wss://bmc-ip/nbd/0');

// Handle binary data streaming
ws.binaryType = 'arraybuffer';

ws.onmessage = function(event) {
    // Handle NBD protocol messages
};
```

NBD Protocol

Network Block Device protocol for efficient streaming:

```
# Check NBD devices
ls /dev/nbd*

# NBD proxy handles protocol translation
# WebSocket → NBD → USB mass storage
```

Troubleshooting

Image Not Mounting

```
# Check virtual media service
systemctl status virtual-media
journalctl -u virtual-media -f

# Verify URL is accessible from BMC
curl -k https://example.com/image.iso -o /dev/null

# Check USB gadget
ls /sys/kernel/config/usb_gadget/
```

Host Not Seeing USB Device

```
# Check USB connection
lsusb -t    # On host if possible

# Verify USB gadget is bound
cat /sys/kernel/config/usb_gadget/*/UDC

# Check USB device controller
ls /sys/class/udc/

# Rebind gadget
echo "" > /sys/kernel/config/usb_gadget/virtual_media/UDC
echo "1e6a0000.usb-vhub:p1" > /sys/kernel/config/usb_gadget/virtual_media/UDC
```

Slow Transfer Speed

```
# Check network connection
iftop -i eth0

# Monitor image streaming
journalctl -u virtual-media -f

# For large images, consider:
# - Local caching
# - Faster network link
# - Proximity of image server
```

Boot Fails

```
# Verify boot order in BIOS
# USB boot may need to be enabled

# Check image integrity
curl https://server/image.iso | sha256sum

# Verify image is bootable
# May need to test image locally first
```

Authentication Errors

```
# For CIFS/SMB shares
# Verify credentials
smbclient //server/share -U username

# For HTTPS with client cert
# Check certificate configuration
```


Security Considerations

Image Source Authentication

```
# Prefer HTTPS for image sources
# Verify SSL certificates

# For self-signed certs, may need:
EXTRA_OEMESON:pn-bmcweb = " \
    -Dinsecure-ignore-cert-errors=enabled \
"
# NOT recommended for production
```

Access Control

```
# Virtual media access requires authentication
# Recommended: Operator role or higher

# Check role permissions
curl -k -u root:openBmc \
    https://localhost/redfish/v1/AccountService/Roles
```

Write Protection

```
# Always mount OS images as write-protected
curl -k -u root:openBmc -X PATCH \
    -H "Content-Type: application/json" \
    -d '{
        "Image": "https://server/image.iso",
        "WriteProtected": true,
        "Inserted": true
    }' \
    https://localhost/redfish/v1/Managers/bmc/VirtualMedia/Slot_0
```

Enabling/Disabling Virtual Media

Build-Time Disable

```
# Disable virtual media
EXTRA_OEMESON:pn-bmcweb = " \
    -Dvm-websocket=disabled \
    -Dvm-nbdproxy=disabled \
"
IMAGE_INSTALL:remove = "virtual-media"
```

Runtime Disable

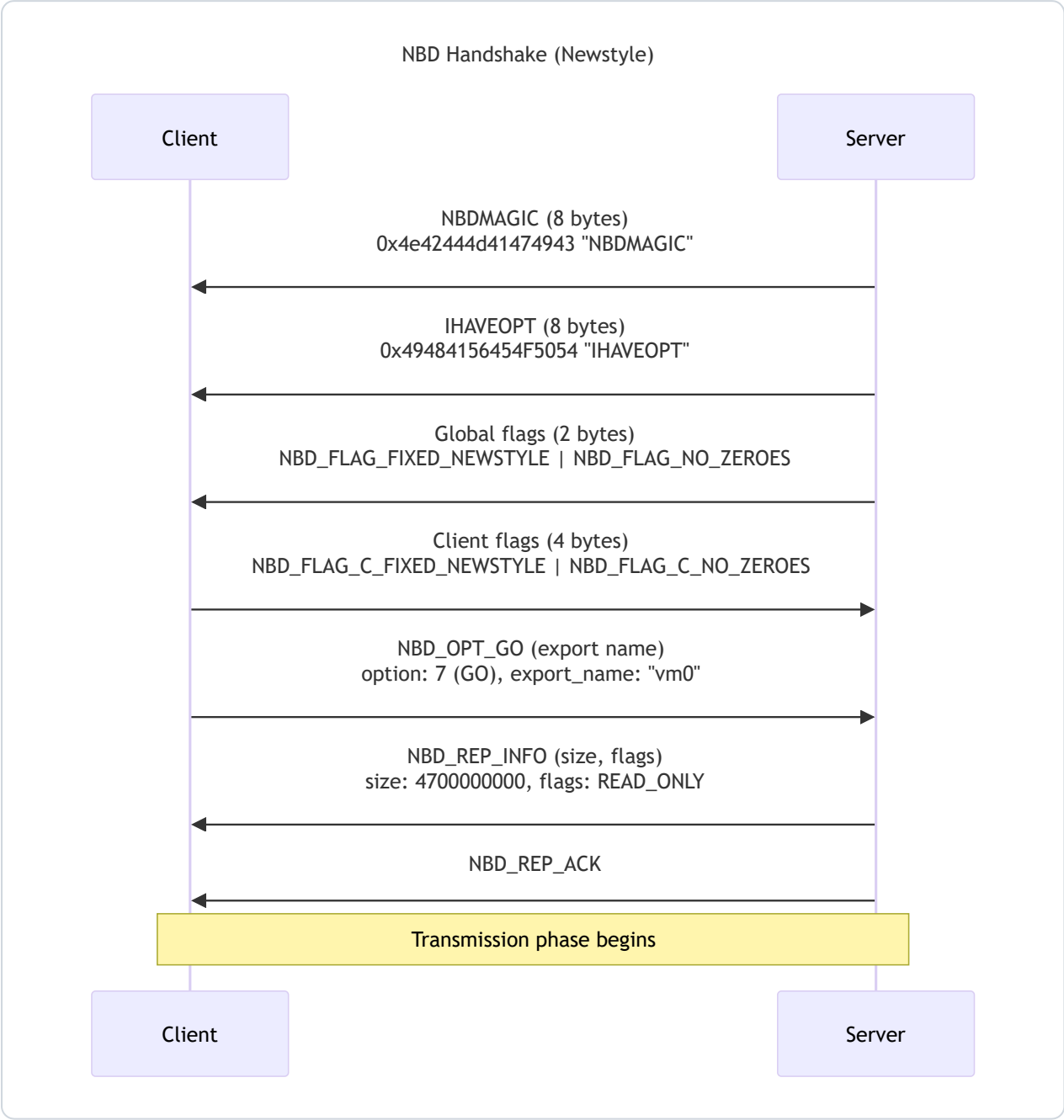
```
# Stop virtual media service
systemctl stop virtual-media
systemctl disable virtual-media

# Unmount any mounted images first
curl -k -u root:openBmc -X POST \
    https://localhost/redfish/v1/Managers/bmc/VirtualMedia/Slot_0/Actions/VirtualMedia.EjectMedia
```

Deep Dive

Advanced implementation details for virtual media developers.

NBD (Network Block Device) Protocol



NBD Request Format (28 bytes):

Field	Size	Description
Magic	4 bytes	0x25609513
Flags	2 bytes	0x0000
Type	2 bytes	NBD_CMD_*
Handle	8 bytes	request_id
Offset	8 bytes	byte_offset
Length	4 bytes	bytes_to_read

NBD Commands:

- `NBD_CMD_READ` (0) - Read data from export
- `NBD_CMD_WRITE` (1) - Write data to export
- `NBD_CMD_DISC` (2) - Disconnect gracefully
- `NBD_CMD_FLUSH` (3) - Flush writes to storage
- `NBD_CMD_TRIM` (4) - Discard blocks (sparse)

NBD Reply Format (16 bytes + data):

Field	Size	Description
Magic	4 bytes	0x67446698
Error	4 bytes	0 (OK)
Handle	8 bytes	request_id (matches request)

Data payload follows for READ responses.

► ASCII-art version (click to expand)

USB Mass Storage Gadget Architecture

ConfigFS Gadget Setup:

```
/sys/kernel/config/usb_gadget/virtual_media/
├── idVendor      # 0x1d6b (Linux Foundation)
├── idProduct     # 0x0104 (Composite Gadget)
├── bcdDevice     # 0x0100
├── bcdUSB        # 0x0200 (USB 2.0)
├── strings/0x409/
│   ├── manufacturer # "OpenBMC"
│   ├── product      # "Virtual Media"
│   └── serialnumber  # BMC serial
├── configs/c.1/
│   ├── MaxPower     # 500 mA
│   └── mass_storage.0 -> ../../functions/mass_storage.0
└── functions/
    └── mass_storage.0/
        └── lun.0/
            ├── cdrom      # 1 (CD-ROM mode for ISO)
            ├── ro         # 1 (read-only)
            ├── removable  # 1 (removable media)
            ├── nofua      # 0 (respect FUA bit)
            └── file        # /dev/nbd0 or /path/to/image.iso
```

Command Block Wrapper (CBW) - 31 bytes:

Field	Value	Size
dCBWSignature	0x43425355 "USBC"	4 bytes
dCBWTag	Transaction identifier	4 bytes
dCBWDataLength	Expected data transfer length	4 bytes
bmCBWFlags	Direction (0x80=IN, 0x00=OUT)	1 byte
bCBWLUN	Logical Unit Number (0)	1 byte
bCBWCBLength	SCSI command length (6-16)	1 byte
CBWCB[16]	SCSI command block	16 bytes

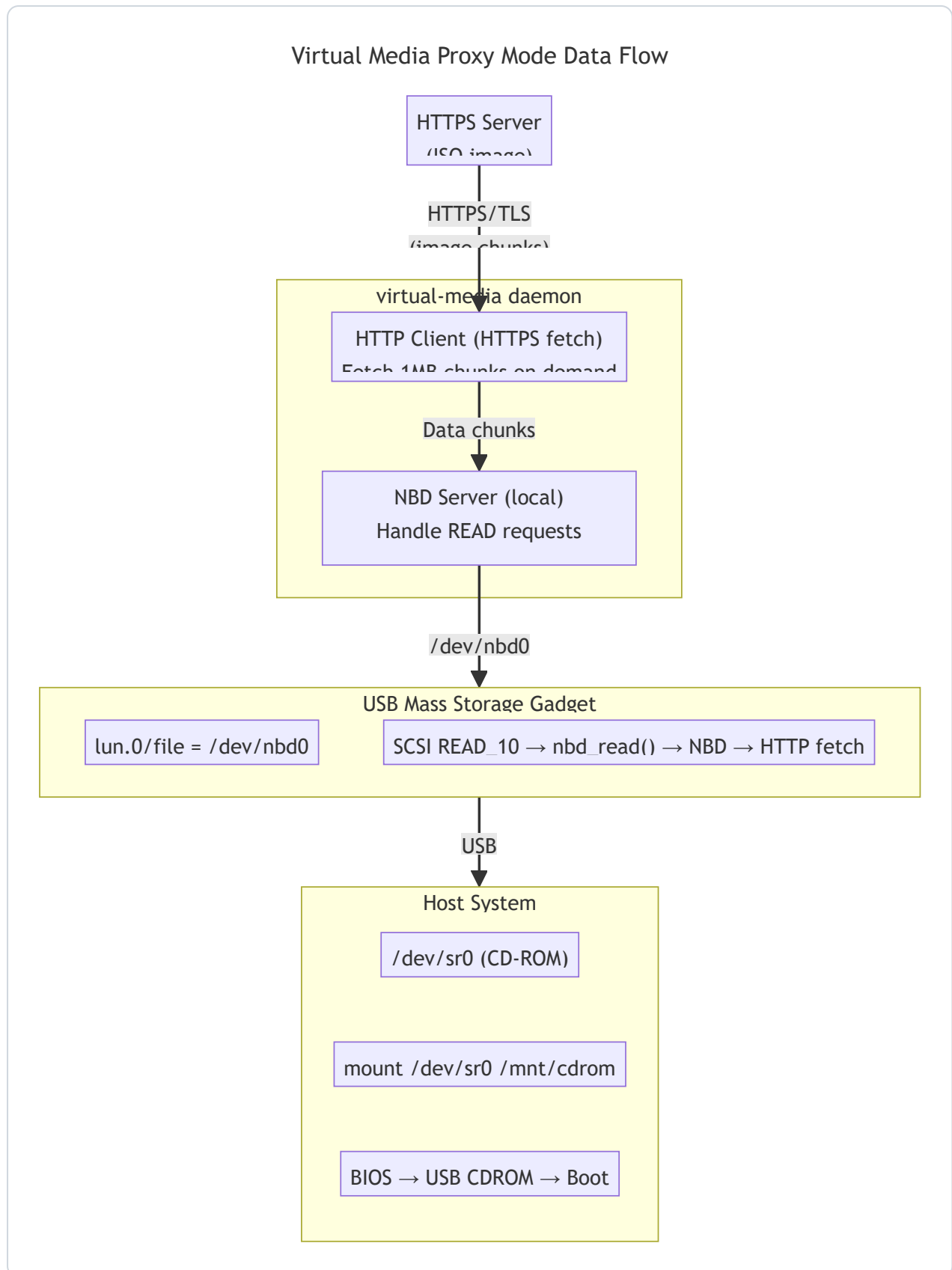
Common SCSI Commands for CD-ROM:

- **INQUIRY** (0x12) - Device identification
- **TEST_UNIT_READY** (0x00) - Check if media present
- **READ_CAPACITY** (0x25) - Get media size
- **READ_10** (0x28) - Read data blocks
- **MODE_SENSE_10** (0x5A) - Get device parameters
- **GET_CONFIGURATION** (0x46) - Get multimedia features

Command Status Wrapper (CSW) - 13 bytes:

Field	Value	Size
dCSWSignature	0x53425355 "USBS"	4 bytes
dCSWTag	Matches CBW tag	4 bytes
dCSWDataResidue	Bytes not transferred	4 bytes
bCSWStatus	0=OK, 1=Failed, 2=Phase Error	1 byte

Proxy Mode Data Flow



Caching Strategy:

- **Read-ahead caching:** When block N is requested, prefetch blocks N+1 to N+K. Reduces latency for sequential reads (common during boot).

- **LRU cache eviction:** Limited BMC memory (e.g., 64MB cache). Evict least recently used chunks when full.
- **Sparse image support:** Only fetch blocks actually read by host. Zero-fill unread blocks (improves mount speed).

► ASCII-art version (click to expand)

Source Code Reference

Key implementation files in [virtual-media](#):

File	Description
<code>src/main.cpp</code>	Service entry point and D-Bus object creation
<code>src/state_machine.cpp</code>	Mount/unmount state machine
<code>src/smb.cpp</code>	CIFS/SMB network share handling
<code>src/https.cpp</code>	HTTPS image fetching with curl
<code>src/nbd_proxy.cpp</code>	NBD protocol client/server implementation
<code>src/usb_gadget.cpp</code>	USB mass storage gadget configuration

References

- [OpenBMC Virtual Media](#)
- [Redfish VirtualMedia Schema](#)
- [NBD Protocol](#)
- [USB Mass Storage Class](#)

Tested on: OpenBMC master, requires hardware with USB device controller

Console Guide

Configure serial console access and host logging on OpenBMC.

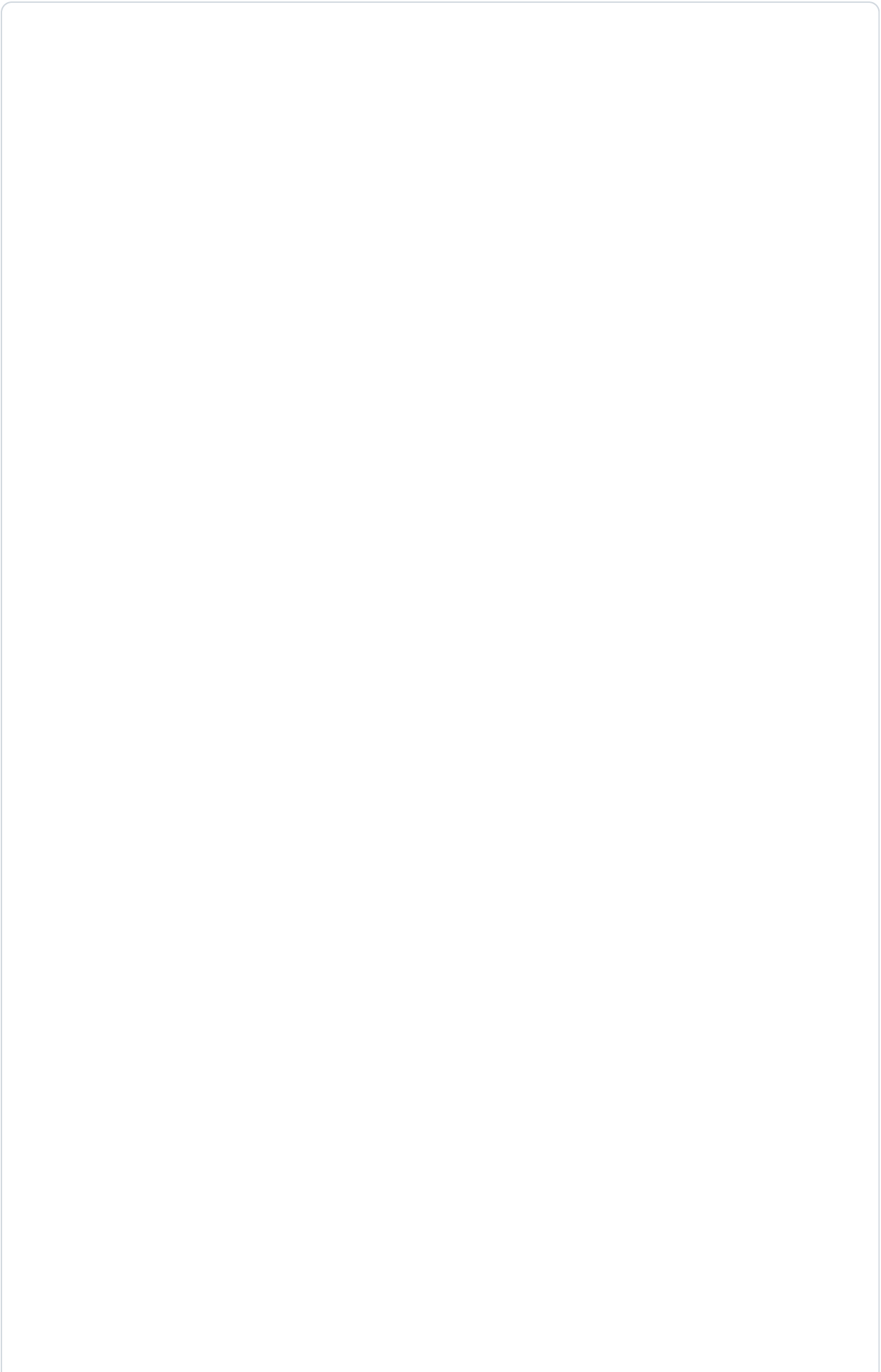
Table of Contents

1. [Overview](#)
2. [Components](#)
 1. [obmc-console-server](#)
 2. [obmc-console-client](#)
 3. [hostlogger](#)
3. [Hardware Configuration](#)
 1. [Serial Port Types](#)
 2. [ASPEED VUART](#)
 3. [Nuvoton Serial](#)
4. [Setup & Configuration](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [Console Server Configuration](#)
 3. [Multiple Console Support](#)
 4. [Runtime Configuration](#)
5. [Accessing the Console](#)
 1. [Via WebUI](#)
 2. [Via SSH](#)
 3. [Via IPMI SOL](#)
 4. [Via Redfish](#)
 5. [Via obmc-console-client](#)
6. [SOL Configuration](#)
 1. [IPMI SOL Settings](#)
 2. [SOL Baud Rates](#)
 3. [Redfish SOL Configuration](#)
7. [Host Logger Configuration](#)
 1. [Enable Host Logger](#)
 2. [Configure Host Logger](#)
 3. [systemd Service](#)
 4. [View Host Logs](#)
8. [Console Escape Sequences](#)
 1. [obmc-console-client](#)
 2. [SSH Session](#)
 3. [IPMI SOL](#)
9. [Multiple Host Consoles](#)

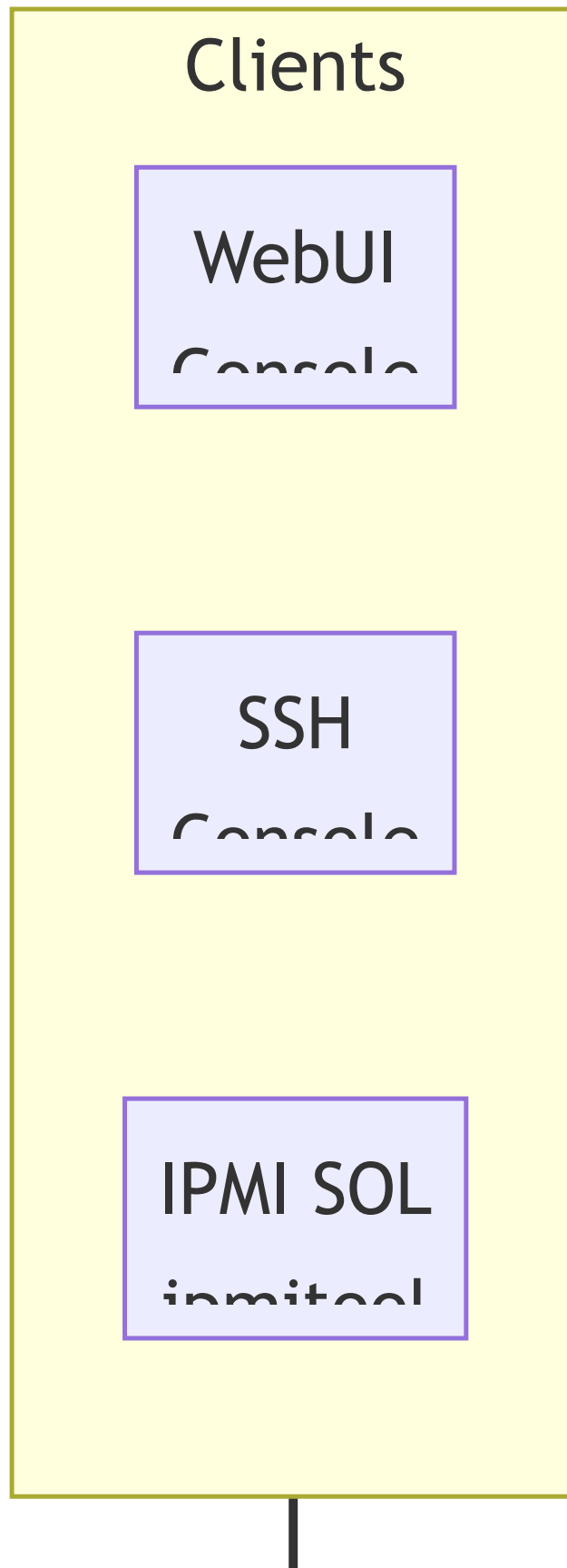
1. [Access Each Console](#)
 10. [Multi-UART Muxing](#)
 1. [Mux Configuration](#)
 2. [Per-Console D-Bus Interface](#)
 3. [Mux Switching via Connect\(\)](#)
 4. [Connect/Disconnect Log Markers](#)
 5. [D-Bus Console Interface](#)
 11. [Troubleshooting](#)
 1. [No Console Output](#)
 2. [Console Frozen](#)
 3. [Garbled Text](#)
 4. [SOL Connection Failed](#)
 5. [Permission Denied](#)
 12. [Security Considerations](#)
 1. [Access Control](#)
 2. [SOL Encryption](#)
 3. [Console Logging Security](#)
 13. [Enabling/Disabling Console](#)
 1. [Build-Time Disable](#)
 2. [Runtime Disable](#)
 3. [Disable SOL via IPMI](#)
 14. [Deep Dive](#)
 1. [Console Server Multiplexing Architecture](#)
 2. [IPMI SOL Protocol Details](#)
 3. [Escape Sequence Processing](#)
 4. [Host Logger Ring Buffer](#)
 5. [Source Code Reference](#)
 15. [References](#)
-

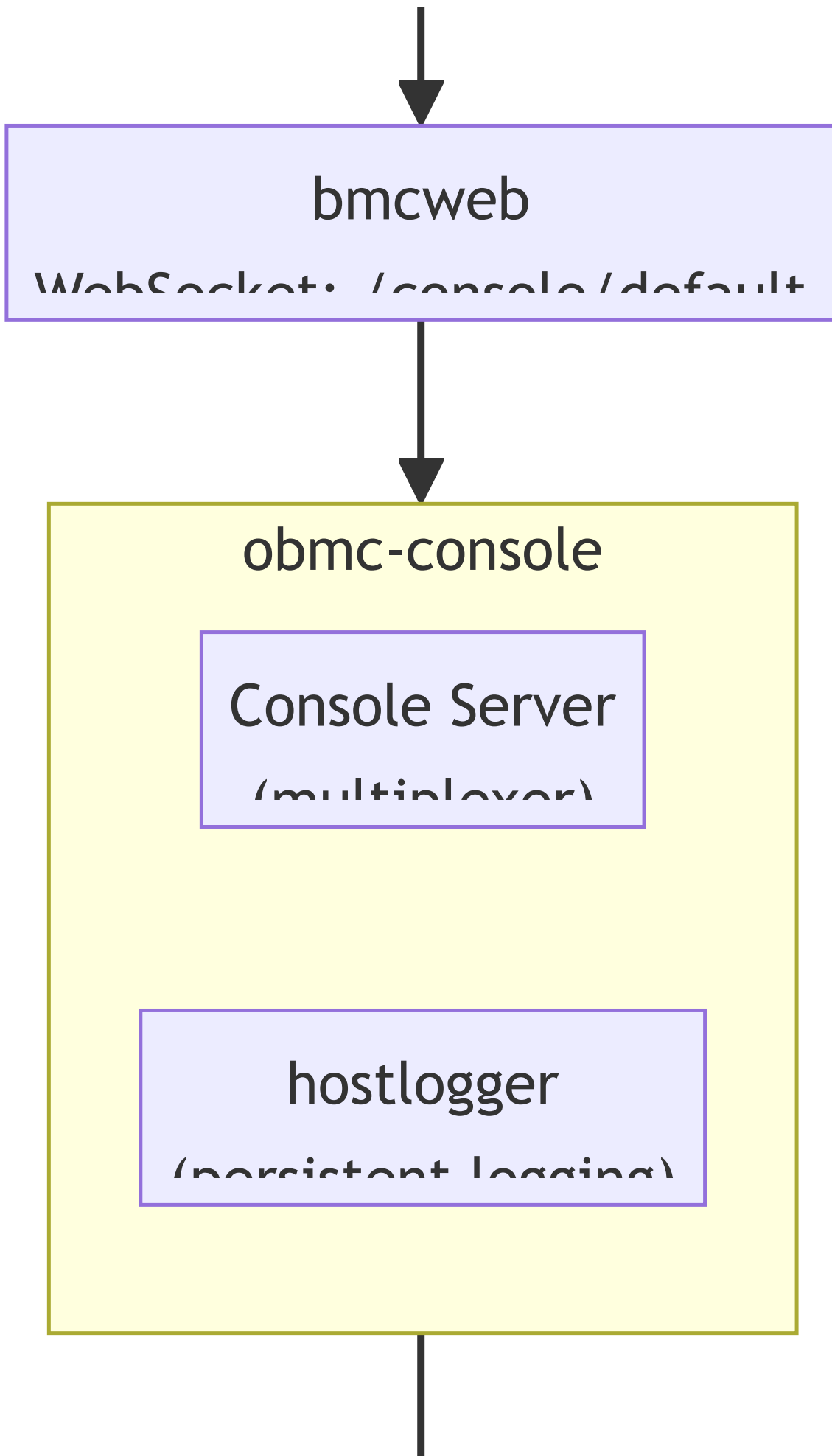
Overview

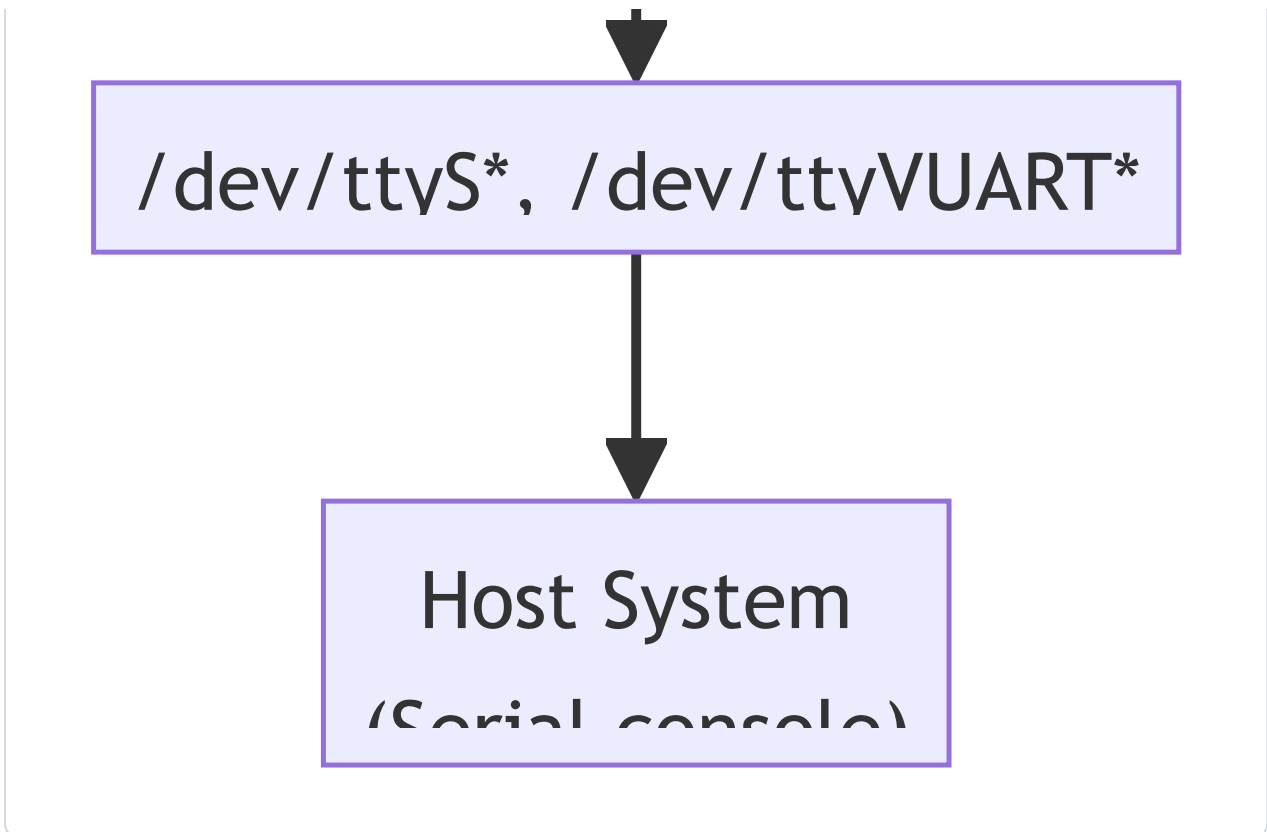
Serial Console provides text-mode access to the host system through the BMC, enabling boot monitoring, BIOS configuration, and command-line access when graphical interfaces are unavailable.



Console Architecture







► ASCII-art version (for comparison)

Components

obmc-console-server

- Multiplexes serial port to multiple clients
- Provides Unix socket interface
- Handles console access control

obmc-console-client

- Command-line console access
- Connects to console-server socket

hostlogger

- Captures console output persistently
 - Stores in rotating log files
 - Available via Redfish API
-

Hardware Configuration

Serial Port Types

Type	Description	Device Path
Physical UART	Hardware serial port	/dev/ttyS0, /dev/ttyS1
Virtual UART	LPC/VUART interface	/dev/ttyVUART0
PTY	Pseudo-terminal (testing)	/dev/pts/*

ASPEED VUART

ASPEED BMCs use Virtual UART for LPC-based console:

```
// Device tree configuration
&vuart {
    status = "okay";
};
```

Nuvoton Serial

```
// Device tree for serial ports
&serial1 {
    status = "okay";
};
```

Setup & Configuration

Build-Time Configuration (Yocto)

```
# In your machine .conf or local.conf

# Include console packages
IMAGE_INSTALL:append = " \
    obmc-console \
    phosphor-hostlogger \
"

# Configure console device in machine config
OBMC_CONSOLE_HOST_TTY ?= "ttyVUART0"

# For physical serial port:
# OBMC_CONSOLE_HOST_TTY ?= "ttyS2"
```

Console Server Configuration

Configuration file: `/etc/obmc-console/server.ttyVUART0.conf`

```
# Console server configuration

# Socket directory
socket-id = default

# Console device
console-id = default

# Baud rate
baud = 115200

# Local echo (for testing)
#local-echo = true

# Logfile (if not using hostlogger)
#logfile = /var/log/obmc-console.log
```

Multiple Console Support

```
# /etc/obmc-console/server.ttyS0.conf
socket-id = host0
console-id = host0
baud = 115200

# /etc/obmc-console/server.ttyS1.conf
socket-id = host1
console-id = host1
baud = 115200
```

Runtime Configuration

```
# Check console service status
systemctl status obmc-console-server@ttyVUART0

# View console logs
journalctl -u obmc-console-server@ttyVUART0 -f

# Restart console service
systemctl restart obmc-console-server@ttyVUART0
```

Accessing the Console

Via WebUI

1. Login to WebUI at <https://<bmc-ip>/>
2. Navigate to **Operations** → **Serial over LAN Console**
3. Terminal opens in browser window
4. Type commands to interact with host

Via SSH

```
# Direct console access via SSH
ssh -t root@bmc-ip /usr/bin/obmc-console-client

# With specific console ID
ssh -t root@bmc-ip /usr/bin/obmc-console-client -i host0
```

Via IPMI SOL

```
# Activate SOL session
ipmitool -I lanplus -H bmc-ip -U root -P 0penBmc sol activate

# Deactivate SOL session
ipmitool -I lanplus -H bmc-ip -U root -P 0penBmc sol deactivate

# SOL configuration
ipmitool -I lanplus -H bmc-ip -U root -P 0penBmc sol info
```

Via Redfish

```
# Get console URL
curl -k -u root:0penBmc \
  https://localhost/redfish/v1/Managers/bmc

# Look for SerialConsole property
# WebSocket endpoint: wss://bmc-ip/console/default
```

Via obmc-console-client

```
# On the BMC directly
obmc-console-client

# Specify console ID
obmc-console-client -i default

# Exit: Ctrl+] or ~.
```

SOL Configuration

IPMI SOL Settings

```
# Get SOL configuration
ipmitool -I lanplus -H bmc-ip -U root -P 0penBmc sol info

# Set SOL parameters
# Enable SOL
ipmitool -I lanplus -H bmc-ip -U root -P 0penBmc sol set enabled true

# Set baud rate
ipmitool -I lanplus -H bmc-ip -U root -P 0penBmc sol set volatile-bit-rate 115.2

# Set non-volatile baud rate
ipmitool -I lanplus -H bmc-ip -U root -P 0penBmc sol set non-volatile-bit-rate 115.2

# Set privilege level
ipmitool -I lanplus -H bmc-ip -U root -P 0penBmc sol set privilege-level operator
```

SOL Baud Rates

Value	Rate
6	9600
7	19200
8	38400
9	57600
10	115200

Redfish SOL Configuration

```
# Get network protocol settings
curl -k -u root:0penBmc \
  https://localhost/redfish/v1/Managers/bmc/NetworkProtocol

# Note: SOL is typically configured via IPMI
# Redfish Serial Console is different from IPMI SOL
```

Host Logger Configuration

Enable Host Logger

```
# In Yocto build
IMAGE_INSTALL:append = " phosphor-hostlogger"
```

Configure Host Logger

```
# Configuration file
cat /etc/hostlogger.conf

# Common settings:
# - Buffer size
# - Log rotation
# - Flush interval
```

systemd Service

```
# /lib/systemd/system/phosphor-hostlogger@.service
[Unit]
Description=Host Console Logger for %i
After=obmc-console-server@%i.service

[Service]
ExecStart=/usr/bin/hostlogger -i %i
Restart=always

[Install]
WantedBy=multi-user.target
```

View Host Logs

```
# Via Redfish
curl -k -u root:0penBmc \
    https://localhost/redfish/v1/Systems/system/LogServices/HostLogger/Entries

# Local files (if configured)
ls /var/log/host*

# Via journal
journalctl -u phosphor-hostlogger@ttyVUART0
```

Console Escape Sequences

obmc-console-client

Sequence	Action
	Disconnect
	Show help
	Suspend
	Send literal ~

SSH Session

Sequence	Action
<code>~.</code>	Disconnect (SSH + console)
<code>Enter ~.</code>	Disconnect properly

IPMI SOL

Sequence	Action
<code>~.</code>	Deactivate SOL
<code>~^Z</code>	Suspend SOL
<code>~B</code>	Send break

Multiple Host Consoles

For systems with multiple hosts:

```
# Configuration for each host
# /etc/obmc-console/server.host0.conf
socket-id = host0
console-id = host0

# /etc/obmc-console/server.host1.conf
socket-id = host1
console-id = host1
```

Access Each Console

```
# Via WebUI - select from dropdown
# Via SSH
ssh -t root@bmc-ip /usr/bin/obmc-console-client -i host0
ssh -t root@bmc-ip /usr/bin/obmc-console-client -i host1

# Via IPMI - use channel parameter
ipmitool -I lanplus -H bmc-ip -U root -P 0penBmc -c 1 sol activate
```

Multi-UART Muxing

Unlike [Multiple Host Consoles](#) – which gives each host its own dedicated serial port – multi-UART muxing shares **one physical port** between several UARTs that sit behind a GPIO mux, exposing only one of them at a time. A single `obmc-console-server` process drives the shared tty and switches the mux GPIOs to select the active console.

Use **Multiple Host Consoles** when each host has independent wiring; use **Multi-UART Muxing** when several UARTs share the same BMC port and a mux chip selects between them.

Mux Configuration

Two configuration keys control muxing (see the upstream [mux-support.md](#)):

- `mux-gpios` (global, top of the config file) – the GPIO line name(s) that drive the mux. The order of the listed GPIOs forms the LSB-first bit representation of the number taken from each console's `mux-index`.
- `mux-index` (per console) – the numeric mux value that selects this console. Each console lives in a section whose name matches its `console-id`.

```
# /etc/obmc-console/server.conf (started as: obmc-console-server --config server.conf /dev/ttyS0)

# Global: GPIO(s) that select which UART is active
mux-gpios = MUX_CTL

[host1]
mux-index = 0
logfile = /var/log/console-host1.log

[host2]
mux-index = 1
logfile = /var/log/console-host2.log
```

Per-Console D-Bus Interface

Each muxed console is published as its own D-Bus service named after the console-id:

```
busctl list | grep xyz.openbmc_project.Console
# xyz.openbmc_project.Console.host1 ... obmc-console-server root ...
# xyz.openbmc_project.Console.host2 ... obmc-console-server root ...

busctl introspect xyz.openbmc_project.Console.host1 \
  /xyz/openbmc_project/console/host1
# xyz.openbmc_project.Console.Access interface - -
# .Connect method - h
```

Mux Switching via Connect()

Mux control is **implicit**: switching happens when a client connects. Calling `Connect()` on a console (or simply attaching `obmc-console-client -i host1`) makes the server switch the mux GPIOs to that console's `mux-index`, then start forwarding bytes. Any client attached to another console is disconnected.

```
# Selecting host1 asserts the mux GPIOs for host1 and drops host2
busctl call xyz.openbmc_project.Console.host1 \
  /xyz/openbmc_project/console/host1 \
  xyz.openbmc_project.Console.Access Connect
```

Concretely, connecting to `host1` causes the previously active `host2` to (1) stop forwarding bytes and (2) print a disconnect log to its clients; then `host1` (1) switches the mux via the GPIOs, (2) prints a connect log to its clients, and (3) resumes forwarding.

Connect/Disconnect Log Markers

So a log reader can see why a console went quiet, the server writes a marker line to all clients of a console whenever the mux switches:

```
[obmc-console] %Y-%m-%d %H:%M:%S UTC CONNECTED
[obmc-console] %Y-%m-%d %H:%M:%S UTC DISCONNECTED
```

These markers are a convenience only – anything on the other side of the UART could print the same text, and the exact format is not guaranteed to stay fixed. Do not parse them as a reliable connection signal.

D-Bus Console Interface

Beyond the muxing `Access` interface, `obmc-console-server` (in `console-dbus.c`) exposes a `xyz.openbmc_project.Console.UART` interface on `/xyz/openbmc_project/console/<console-id>` with a **writable** `Baud` property (type `t`, uint64). Writing it re-applies the tty's termios at runtime – no service restart needed:

```
# Read the current baud rate
busctl get-property xyz.openbmc_project.Console.default \
    /xyz/openbmc_project/console/default \
    xyz.openbmc_project.Console.UART Baud

# Change baud rate live (re-applies termios on the underlying tty)
busctl set-property xyz.openbmc_project.Console.default \
    /xyz/openbmc_project/console/default \
    xyz.openbmc_project.Console.UART Baud t 57600
```

The `xyz.openbmc_project.Console.Access` interface's `Connect()` method takes no arguments and returns a Unix file descriptor (`h`). The server creates a socket pair, hands the client end back over D-Bus, and closes its own end – giving the caller a direct byte stream to the console without going through the on-disk Unix socket path.

```
# Returns a connected fd for the default console
busctl call xyz.openbmc_project.Console.default \
    /xyz/openbmc_project/console/default \
    xyz.openbmc_project.Console.Access Connect
```

Troubleshooting

No Console Output

```
# Check if console server is running
systemctl status obmc-console-server@ttyVUART0

# Check device exists
ls -la /dev/ttyVUART0

# Check permissions
stat /dev/ttyVUART0

# Check serial port configuration
stty -F /dev/ttyVUART0 -a

# Verify baud rate matches host
stty -F /dev/ttyVUART0 115200
```

Console Frozen

```
# Restart console server
systemctl restart obmc-console-server@ttyVUART0

# Kill stuck clients
pkill obmc-console-client

# Check for hardware issues
dmesg | grep -i uart
```

Garbled Text

```
# Baud rate mismatch - verify settings match
# Host BIOS/OS must use same baud rate as BMC

# Common baud rates to try:
stty -F /dev/ttyVUART0 9600
stty -F /dev/ttyVUART0 19200
stty -F /dev/ttyVUART0 38400
stty -F /dev/ttyVUART0 57600
stty -F /dev/ttyVUART0 115200
```

SOL Connection Failed

```
# Check IPMI service
systemctl status phosphor-ipmi-net

# Verify SOL is enabled
ipmitool -I lanplus -H bmc-ip -U root -P 0penBmc sol info

# Check for active sessions
ipmitool -I lanplus -H bmc-ip -U root -P 0penBmc session info all

# Force deactivate stale session
ipmitool -I lanplus -H bmc-ip -U root -P 0penBmc sol deactivate
```

Permission Denied

```
# Check user permissions
# User needs Operator or Administrator role

# Verify via Redfish
curl -k -u user:password \
  https://localhost/redfish/v1/AccountService/Accounts/user
```

Security Considerations

Access Control

```
# Console access requires authentication
# Recommended role: Operator or Administrator

# View user roles
curl -k -u root:OpenBmc \
    https://localhost/redfish/v1/AccountService/Accounts
```

SOL Encryption

IPMI SOL uses RMCP+ encryption:

```
# Check cipher suite
ipmitool -I lanplus -H bmc-ip -U root -P OpenBmc -C 17 sol activate

# Cipher 17 = AES-CBC-128 + HMAC-SHA256 (recommended)
```

Console Logging Security

```
# Console logs may contain sensitive data
# Configure retention policy
# Restrict access to log files

# File permissions
chmod 600 /var/log/hostlogger/*
```

Enabling/Disabling Console

Build-Time Disable

```
# Remove console packages
IMAGE_INSTALL:remove = "obmc-console phosphor-hostlogger"
```

Runtime Disable

```
# Stop console service
systemctl stop obmc-console-server@ttyVUART0
systemctl disable obmc-console-server@ttyVUART0

# Stop hostlogger
systemctl stop phosphor-hostlogger@ttyVUART0
systemctl disable phosphor-hostlogger@ttyVUART0
```

Disable SOL via IPMI

```
ipmitool -I lanplus -H bmc-ip -U root -P OpenBmc sol set enabled false
```

Deep Dive

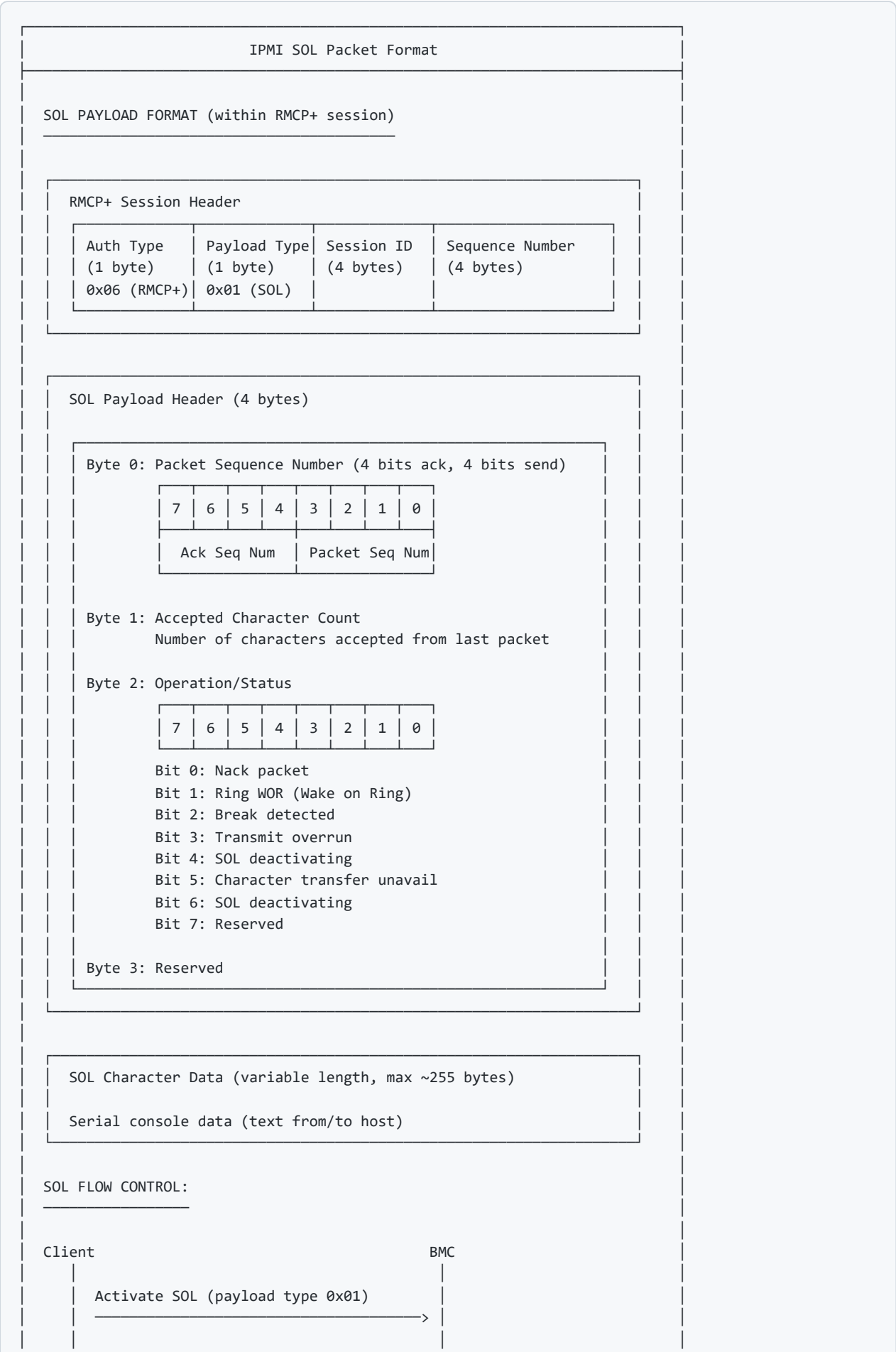
Advanced implementation details for console developers.

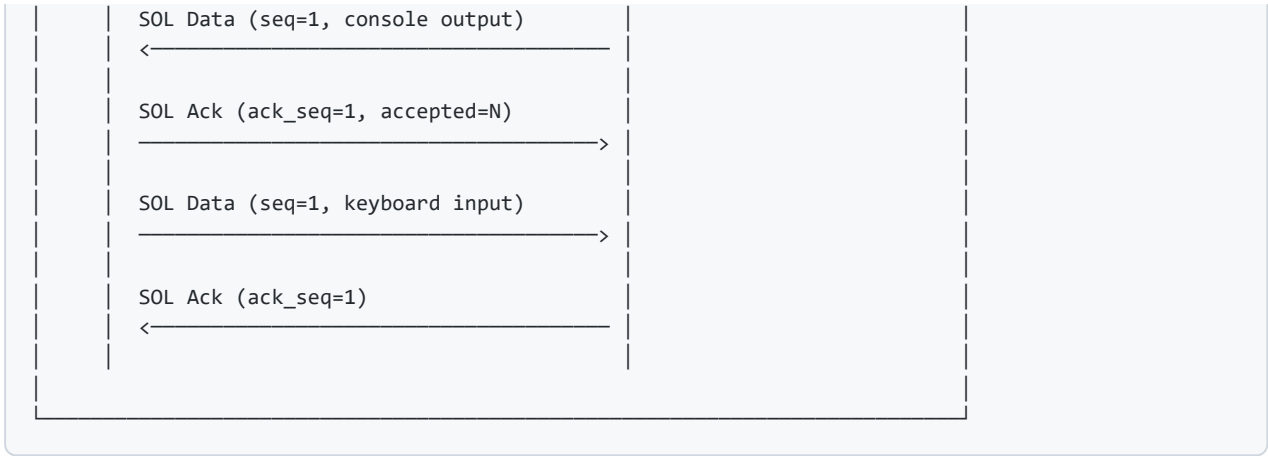
Console Server Multiplexing Architecture



UNIX SOCKET: /var/run/obmc-console.sock (or /run/obmc-console/ttyVUART0)

IPMI SOL Protocol Details





Escape Sequence Processing

Console Escape Sequence Handling

ESCAPE SEQUENCE STATE MACHINE

NORMAL

ESCAPE_START

DISCONNECT

FORWARD TO SERIAL

BREAK

'~'

other char

'.'

'~' then 'B'

all other characters

v

Close conn

Send break to serial

ESCAPE SEQUENCES (obmc-console-client):

Sequence	Action
~.	Disconnect from console
~B	Send serial break signal
~?	Show help message
~~	Send literal '~' character
~^Z	Suspend client (if in terminal)

IMPLEMENTATION (console-client.c):

```
enum escape_state { ESC_NONE, ESC_TILDE };

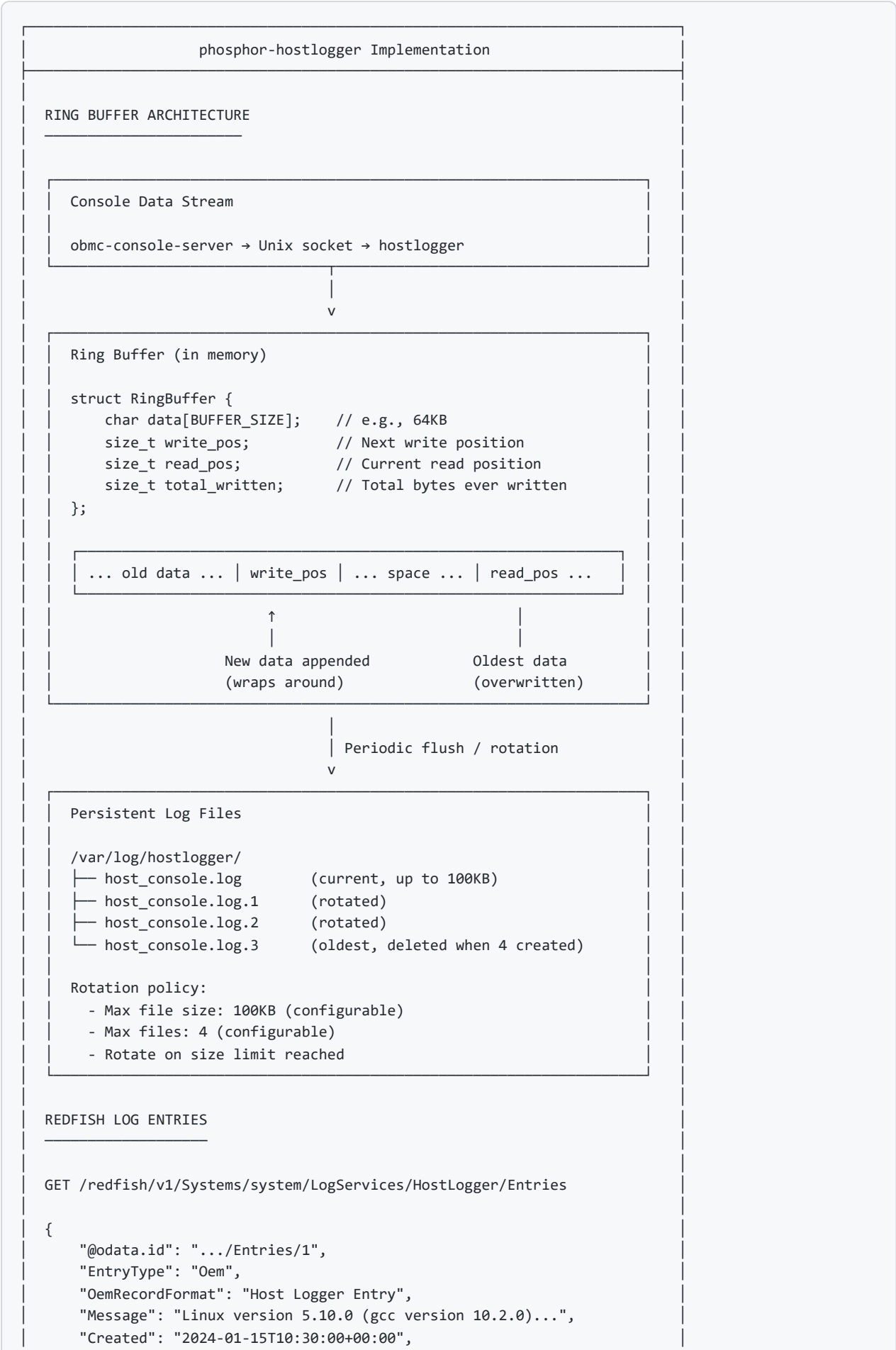
void process_input(char *buf, size_t len) {
    for (i = 0; i < len; i++) {
        char c = buf[i];

        if (escape_state == ESC_NONE) {
            if (c == '~' && last_was_newline) {
                escape_state = ESC_TILDE;
                continue;
            }
        } else if (escape_state == ESC_TILDE) {
            escape_state = ESC_NONE;
            switch (c) {
                case '.':
                    disconnect();
                    return;
                case 'B':
                    send_break();
```

689

```
        continue;
    case '~':
        // Forward literal ~
        break;
    default:
        // Not escape, forward ~ and char
        forward_char('~');
        break;
    }
}
forward_char(c);
last_was_newline = (c == '\n' || c == '\r');
}
}
```

Host Logger Ring Buffer



```

    "Severity": "OK"
}

Line parsing:
- Split on newline
- Each line becomes one Redfish entry
- Timestamp from when line was received
- Severity: OK for normal, Warning for errors

```

Source Code Reference

Key implementation files in [obmc-console](#) and [phosphor-hostlogger](#):

File	Description
obmc-console/console-server.c	Main multiplexer and TTY handler
obmc-console/console-socket.c	Unix socket client management
obmc-console/console-client.c	Client with escape sequence handling
obmc-console/config.c	Configuration file parsing
phosphor-hostlogger/src/host_logger.cpp	Ring buffer and log rotation
phosphor-hostlogger/src/dbus_loop.cpp	D-Bus integration for Redfish

References

- [obmc-console](#)
- [phosphor-hostlogger](#)
- [IPMI SOL Specification](#)
- [OpenBMC Console Design](#)

Tested on: OpenBMC master, QEMU romulus

SSH Security Guide

Configure and harden SSH access on OpenBMC.

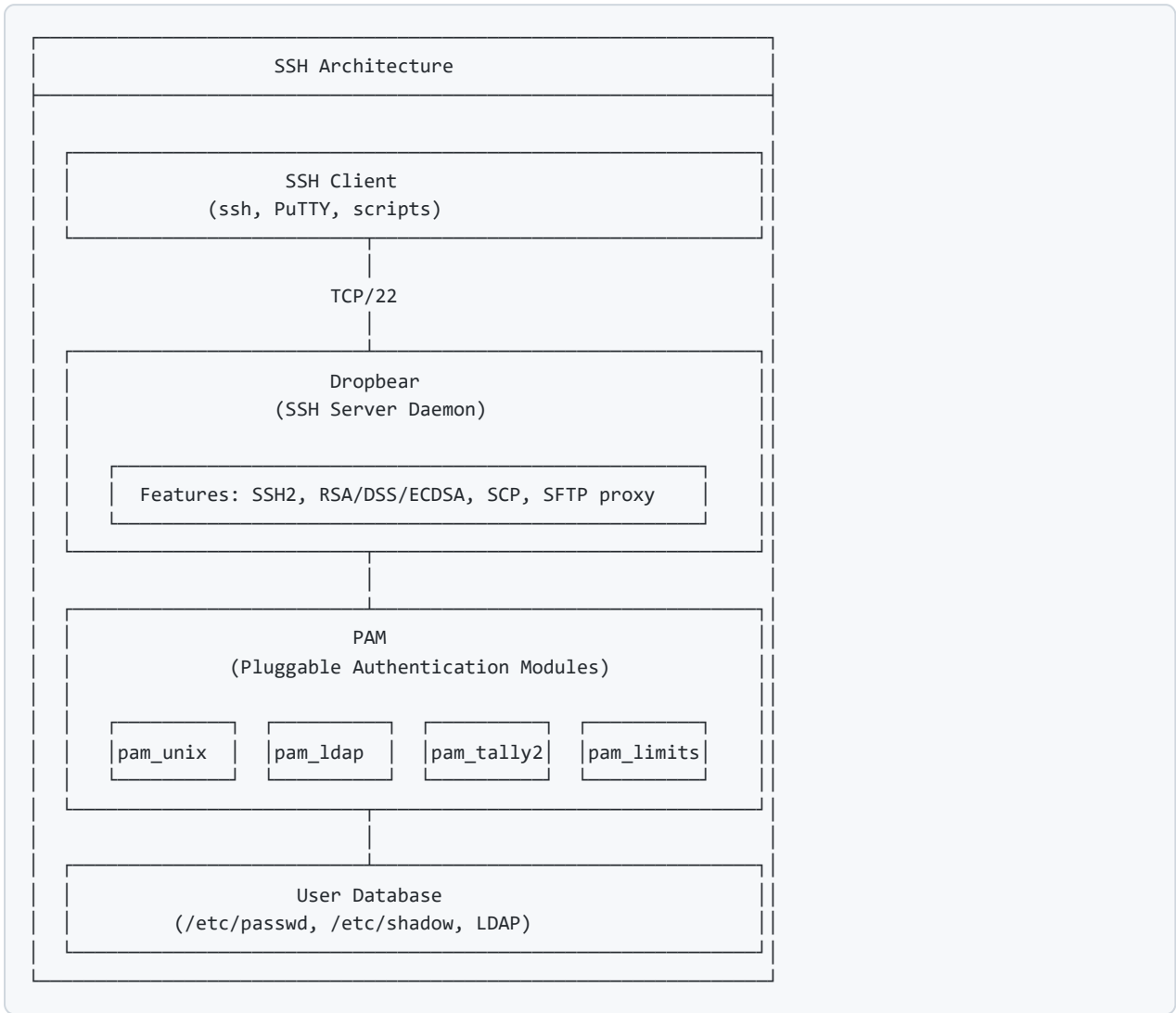
Table of Contents

1. [Overview](#)
2. [Setup & Configuration](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [Dropbear Configuration Options](#)
 3. [Runtime Configuration](#)
 4. [/etc/default/dropbear](#)
3. [Authentication Configuration](#)
 1. [Password Authentication](#)
 2. [SSH Key Authentication](#)
 3. [Key-Only Authentication](#)
4. [PAM Configuration](#)
 1. [PAM Overview](#)
 2. [Default PAM Configuration](#)
 3. [Common Auth Configuration](#)
 4. [Enable Account Lockout](#)
 5. [LDAP Authentication](#)
5. [Security Hardening](#)
 1. [Disable Root SSH Login](#)
 2. [Restrict SSH to Specific Users](#)
 3. [Change Default Port](#)
 4. [Disable Port Forwarding](#)
 5. [Set Connection Timeout](#)
 6. [Host Key Verification](#)
6. [Recommended Secure Configuration](#)
 1. [Production Hardening Checklist](#)
 2. [Secure Dropbear Recipe](#)
7. [Managing SSH Access](#)
 1. [User Management](#)
 2. [View Active Sessions](#)
 3. [Connection Logging](#)
8. [Enabling/Disabling SSH](#)
 1. [Build-Time Disable](#)
 2. [Runtime Disable](#)
 3. [Redfish Control](#)

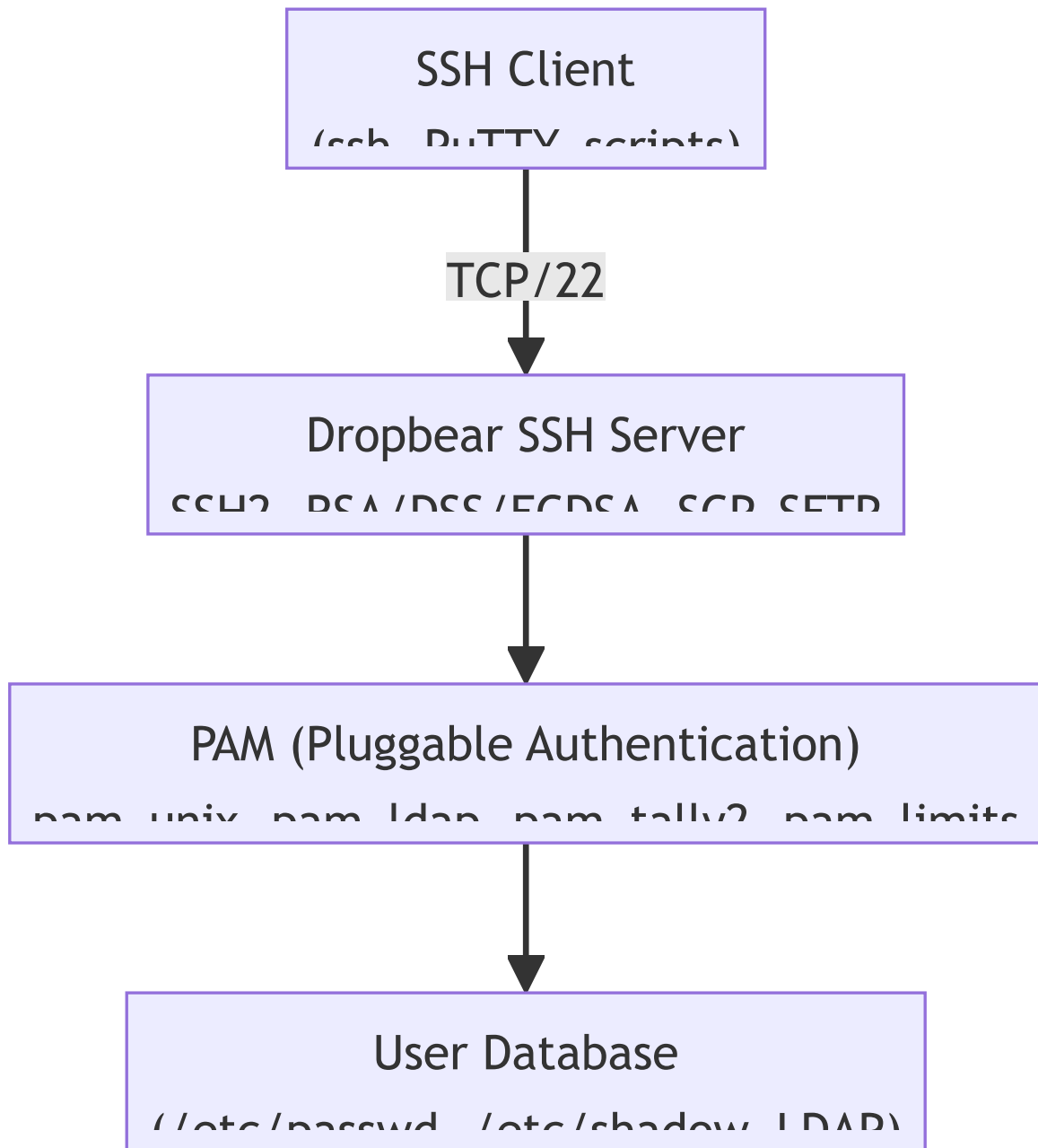
- 9. [SFTP Configuration](#)
 - 1. [Enable SFTP](#)
 - 2. [SFTP-Only User](#)
- 10. [Troubleshooting](#)
 - 1. [Connection Refused](#)
 - 2. [Authentication Failures](#)
 - 3. [Key Not Accepted](#)
 - 4. [PAM Errors](#)
 - 5. [Host Key Changed Warning](#)
- 11. [Security Comparison](#)
- 12. [Using OpenSSH Instead of Dropbear](#)
 - 1. [Switching to OpenSSH](#)
 - 1. [Build-Time Configuration](#)
 - 2. [OpenSSH Configuration File](#)
 - 3. [Runtime Management](#)
 - 2. [Pros and Cons Comparison](#)
 - 1. [Dropbear Advantages](#)
 - 2. [Dropbear Disadvantages](#)
 - 3. [OpenSSH Advantages](#)
 - 4. [OpenSSH Disadvantages](#)
 - 3. [When to Use Each](#)
 - 1. [Choose Dropbear When](#)
 - 2. [Choose OpenSSH When](#)
 - 4. [OpenSSH Hardening Recipe](#)
 - 5. [Migration Checklist](#)
- 13. [References](#)
- 14. [Deep Dive](#)
 - 1. [Dropbear Connection Lifecycle](#)
 - 2. [PAM Authentication Stack](#)
 - 3. [SSH Key Authentication Internals](#)
 - 4. [Host Key Management](#)
 - 5. [Source Code References](#)

Overview

OpenBMC uses **Dropbear** as its SSH server, a lightweight alternative to OpenSSH optimized for embedded systems. This guide covers SSH configuration, PAM integration, and security hardening.



SSH Architecture



Setup & Configuration

Build-Time Configuration (Yocto)

Configure SSH packages in your build:

```
# In your machine .conf or local.conf

# Include Dropbear (default SSH server)
IMAGE_INSTALL:append = " dropbear"

# For OpenSSH instead of Dropbear (larger, more features)
# IMAGE_INSTALL:append = " openssh-server openssh-client"
# IMAGE_INSTALL:remove = "dropbear"

# Include SSH-related utilities
IMAGE_INSTALL:append = " \
    openssh-sftp-server \
    openssh-scp \
"

# Configure dropbear options
DROPBEAR_EXTRA_ARGS ?= "-w -s"
```

Dropbear Configuration Options

Option	Description
<code>-p [address:]port</code>	Listen on specified port (default 22)
<code>-w</code>	Disallow root logins
<code>-s</code>	Disable password logins (key only)
<code>-g</code>	Disable password logins for root
<code>-j</code>	Disable local port forwarding
<code>-k</code>	Disable remote port forwarding
<code>-a</code>	Allow connections to forwarded ports from any host
<code>-W</code>	Window size (performance tuning)
<code>-K</code>	Keepalive interval (seconds)
<code>-I</code>	Idle timeout (seconds, 0 = no timeout)

Runtime Configuration

```
# Check dropbear status
systemctl status dropbear

# View current configuration
cat /etc/default/dropbear

# Modify dropbear arguments
vi /etc/default/dropbear

# Restart after changes
systemctl restart dropbear
```

/etc/default/dropbear

```
# Dropbear configuration
# DROPBEAR_EXTRA_ARGS contains additional arguments

# Default: allow root, password auth
DROPBEAR_EXTRA_ARGS=""

# Secure: no root login, key only
DROPBEAR_EXTRA_ARGS="-w -s"

# Custom port
DROPBEAR_EXTRA_ARGS="-p 2222"

# Disable port forwarding
DROPBEAR_EXTRA_ARGS="-j -k"
```

Authentication Configuration

Password Authentication

```
# Enable password authentication (default)
# Remove -s from DROPBEAR_EXTRA_ARGS

# Change user password
passwd root
passwd username

# Password hashes stored in /etc/shadow
cat /etc/shadow
```

SSH Key Authentication

```
# Generate host keys (done automatically)
dropbearkey -t rsa -f /etc/dropbear/dropbear_rsa_host_key
dropbearkey -t ecdsa -f /etc/dropbear/dropbear_ecdsa_host_key
dropbearkey -t ed25519 -f /etc/dropbear/dropbear_ed25519_host_key

# User authorized keys location
mkdir -p ~/.ssh
chmod 700 ~/.ssh
touch ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys

# Add public key to authorized_keys
echo "ssh-rsa AAAA...= user@host" >> ~/.ssh/authorized_keys
```

Key-Only Authentication

For security, disable password authentication:

```
# Edit dropbear defaults
echo 'DROPBEAR_EXTRA_ARGS="-s"' > /etc/default/dropbear

# Restart dropbear
systemctl restart dropbear

# Test key-based login
ssh -i ~/.ssh/id_rsa root@bmc-ip
```

PAM Configuration

PAM Overview

OpenBMC uses PAM for authentication. Configuration files are in `/etc/pam.d/`.

```
# View PAM configuration for dropbear
cat /etc/pam.d/dropbear
```

Default PAM Configuration

```
# /etc/pam.d/dropbear
auth      include      common-auth
account   include      common-account
password  include      common-password
session   include      common-session
```

Common Auth Configuration

```
# /etc/pam.d/common-auth
auth      [success=1 default=ignore] pam_unix.so nullok
auth      requisite                  pam_deny.so
auth      required                   pam_permit.so
```

Enable Account Lockout

```
# /etc/pam.d/common-auth with tally
auth      required    pam_tally2.so deny=3 unlock_time=300 onerr=fail
auth      [success=1 default=ignore] pam_unix.so nullok
auth      requisite    pam_deny.so
auth      required     pam_permit.so
```

LDAP Authentication

```
# Include LDAP PAM module
IMAGE_INSTALL:append = " pam-plugin-ldap"
```

```
# /etc/pam.d/common-auth with LDAP
auth      sufficient    pam_ldap.so
auth      required      pam_unix.so nullok try_first_pass
```

Security Hardening

Disable Root SSH Login

```
# Option 1: Dropbear -w flag
echo 'DROPBEAR_EXTRA_ARGS="-w"' > /etc/default/dropbear
systemctl restart dropbear

# Option 2: PAM configuration
# Add to /etc/pam.d/dropbear:
auth    required    pam_listfile.so item=user sense=deny file=/etc/ssh/denied_users onerr=succeed
```

Restrict SSH to Specific Users

```
# Create allowed users file
echo "operator" > /etc/ssh/allowed_users
echo "admin" >> /etc/ssh/allowed_users

# Add to /etc/pam.d/dropbear
auth    required    pam_listfile.so \
        item=user sense=allow file=/etc/ssh/allowed_users \
        onerr=fail
```

Change Default Port

```
# Edit dropbear config
echo 'DROPBEAR_EXTRA_ARGS="-p 2222"' > /etc/default/dropbear

# Restart service
systemctl restart dropbear

# Connect on new port
ssh -p 2222 root@bmc-ip
```

Disable Port Forwarding

```
# Disable both local and remote port forwarding
echo 'DROPBEAR_EXTRA_ARGS="-j -k"' > /etc/default/dropbear
systemctl restart dropbear
```

Set Connection Timeout

```
# Set idle timeout (300 seconds = 5 minutes)
echo 'DROPBEAR_EXTRA_ARGS="-I 300"' > /etc/default/dropbear
systemctl restart dropbear
```

Host Key Verification

Ensure consistent host keys across updates:


```
# Backup host keys
cp /etc/dropbear/dropbear_* /path/to/backup/

# Include in image build (recipe)
SRC_URI += "file://dropbear_rsa_host_key"

do_install:append() {
    install -m 0600 ${WORKDIR}/dropbear_rsa_host_key \
        ${D}${sysconfdir}/dropbear/
}
```

Recommended Secure Configuration

Production Hardening Checklist

```
# 1. Key-only authentication
echo 'DROPBEAR_EXTRA_ARGS="-s"' > /etc/default/dropbear

# 2. Disable root password login (allow key)
echo 'DROPBEAR_EXTRA_ARGS="-g"' > /etc/default/dropbear

# 3. Complete lockdown (no root at all)
echo 'DROPBEAR_EXTRA_ARGS="-w -s"' > /etc/default/dropbear

# 4. Set idle timeout
echo 'DROPBEAR_EXTRA_ARGS="-s -I 300"' > /etc/default/dropbear

# 5. Apply changes
systemctl restart dropbear
```

Secure Dropbear Recipe

```
# In your machine layer
# recipes-connectivity/dropbear/dropbear_%.bbappend

FILESEXTRAPATHS:prepend := "${THISDIR}/files:"

SRC_URI += " \
    file://dropbear.default \
    file://authorized_keys \
"

do_install:append() {
    # Secure defaults
    install -m 0644 ${WORKDIR}/dropbear.default \
        ${D}${sysconfdir}/default/dropbear

    # Pre-authorized keys
    install -d ${D}/home/root/.ssh
    install -m 0600 ${WORKDIR}/authorized_keys \
        ${D}/home/root/.ssh/authorized_keys
}
```

Managing SSH Access

User Management

```
# Create non-root user
useradd -m operator
passwd operator

# Add to admin group for sudo
usermod -aG admin operator

# Add SSH key for user
mkdir -p /home/operator/.ssh
echo "ssh-rsa AAAA...=" > /home/operator/.ssh/authorized_keys
chmod 700 /home/operator/.ssh
chmod 600 /home/operator/.ssh/authorized_keys
chown -R operator:operator /home/operator/.ssh
```

View Active Sessions

```
# List SSH connections
who

# Detailed connection info
ss -tnp | grep ssh

# Kill specific session (by PID)
pkill -9 -t pts/0
```

Connection Logging

```
# View SSH authentication logs
journalctl -u dropbear

# Failed login attempts
journalctl -u dropbear | grep -i fail

# Successful logins
journalctl -u dropbear | grep -i "password auth succeeded\\|pubkey auth succeeded"
```

Enabling/Disabling SSH

Build-Time Disable

```
# Remove dropbear from image
IMAGE_INSTALL:remove = "dropbear"
```

Runtime Disable

```
# Stop SSH service
systemctl stop dropbear

# Disable SSH permanently
systemctl disable dropbear

# Re-enable SSH
systemctl enable dropbear
systemctl start dropbear
```

Redfish Control

```
# Enable/disable SSH via Redfish
curl -k -u root:OpenBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"SSH": {"ProtocolEnabled": false}}' \
  https://localhost/redfish/v1/Managers/bmc/NetworkProtocol
```

SFTP Configuration

Enable SFTP

```
# SFTP is handled by openssh-sftp-server
# Include in build:
IMAGE_INSTALL:append = " openssh-sftp-server"

# Dropbear uses this as SFTP subsystem
# Usage:
sftp root@bmc-ip
```

SFTP-Only User

```
# Create user with restricted shell
useradd -m -s /usr/libexec/sftp-server sftpuser
passwd sftpuser

# Restrict to home directory via chroot
# (Requires additional configuration)
```

Troubleshooting

Connection Refused

```
# Check if dropbear is running
systemctl status dropbear

# Check listening ports
ss -tlnp | grep 22

# Start dropbear
systemctl start dropbear
```

Authentication Failures

```
# Check logs for specific errors
journalctl -u dropbear -f

# Common issues:
# - Wrong password
# - Key permissions (must be 600)
# - authorized_keys format
# - PAM configuration

# Test with verbose client
ssh -vvv root@bmc-ip
```

Key Not Accepted

```
# Check key permissions
ls -la ~/.ssh/
# Should be: drwx----- .ssh
# Should be: -rw----- authorized_keys

# Fix permissions
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys

# Check key format in authorized_keys
# Must be single line: ssh-rsa AAAA... user@host
```

PAM Errors

```
# Check PAM configuration syntax
pamtester dropbear root authenticate

# View PAM debug output
# Add to /etc/pam.d/dropbear:
auth    required    pam_warn.so
```

Host Key Changed Warning

When reinstalling or updating BMC:

```
# Client side - remove old key
ssh-keygen -R bmc-ip

# Or edit known_hosts manually
vi ~/.ssh/known_hosts
```

Security Comparison

Feature	Dropbear	OpenSSH
Binary size	~110KB	~900KB
Memory usage	Lower	Higher
SFTP	Via openssh-sftp-server	Built-in
Port forwarding	Supported	Full support
X11 forwarding	Limited	Full support
Key types	RSA, DSS, ECDSA, Ed25519	All
Configuration	Command-line flags	sshd_config file
Chroot	Not built-in	Supported

Using OpenSSH Instead of Dropbear

While Dropbear is the default SSH server in OpenBMC, you can optionally use OpenSSH for environments requiring additional features or enterprise integration.

Switching to OpenSSH

Build-Time Configuration

```
# In your machine .conf or local.conf

# Remove Dropbear and add OpenSSH
IMAGE_FEATURES += "ssh-server-openssh"
IMAGE_FEATURES:remove = "ssh-server-dropbear"

# Or manually in IMAGE_INSTALL
IMAGE_INSTALL:remove = "dropbear"
IMAGE_INSTALL:append = " openssh-server openssh-client openssh-sftp-server"
```

OpenSSH Configuration File

Unlike Dropbear's command-line flags, OpenSSH uses `/etc/ssh/sshd_config`:

```
# /etc/ssh/sshd_config

# Security settings
PermitRootLogin prohibit-password
PasswordAuthentication no
PubkeyAuthentication yes
PermitEmptyPasswords no

# Connection settings
Port 22
AddressFamily any
ListenAddress 0.0.0.0

# Timeout and limits
ClientAliveInterval 300
ClientAliveCountMax 3
MaxAuthTries 3
MaxSessions 10

# Disable unnecessary features
X11Forwarding no
AllowTcpForwarding no
AllowAgentForwarding no

# Logging
SyslogFacility AUTH
LogLevel INFO

# Allowed users (optional)
AllowUsers root admin operator
```

Runtime Management

```
# Check OpenSSH status
systemctl status sshd

# Restart after config changes
systemctl restart sshd

# Test configuration before restart
sshd -t

# View logs
journalctl -u sshd
```

Pros and Cons Comparison

Dropbear Advantages

Advantage	Description
Smaller footprint	~110KB binary vs ~900KB for OpenSSH; critical for flash-constrained BMCs
Lower memory usage	Uses less RAM per connection; important for BMCs with limited memory (typically 256-512MB)
Faster startup	Quicker service initialization; relevant for BMC boot time
Simpler configuration	Command-line flags are straightforward for basic setups
Battle-tested in embedded	Widely used in routers, IoT, and embedded Linux systems
Default in OpenBMC	Well-integrated and tested with the OpenBMC ecosystem

Dropbear Disadvantages

Disadvantage	Description
Limited features	No Match blocks, no chroot, limited forwarding options
No sshd_config	Configuration via flags can be awkward for complex setups
Fewer cipher options	Smaller set of supported algorithms
Limited logging	Less detailed audit logging compared to OpenSSH
No certificate authentication	Does not support OpenSSH certificate-based auth

OpenSSH Advantages

Advantage	Description
Rich feature set	Full SSH specification support including all forwarding modes
Flexible configuration	<code>/etc/ssh/sshd_config</code> with Match blocks for per-user/host settings
Certificate authentication	Supports SSH certificates for scalable key management
Chroot/SFTP jail	Built-in chroot support for restricted SFTP-only users
Extensive logging	Detailed audit logs for compliance requirements
Wider compatibility	Better interoperability with enterprise SSH clients and tools
Active development	Frequent security updates from OpenBSD team
FIDO2/U2F support	Hardware security key authentication

OpenSSH Disadvantages

Disadvantage	Description
Larger size	~900KB binary; may be problematic for space-constrained BMCs
Higher memory usage	More RAM per connection (~1-2MB vs ~100KB for Dropbear)
More complex	Configuration file can be overwhelming for simple use cases
Not OpenBMC default	May require additional testing and validation
Slower startup	Slightly longer service initialization time

When to Use Each

Choose Dropbear When

- Flash storage is limited (< 32MB)
- Memory is constrained (< 256MB)
- Simple SSH access is sufficient
- Using standard OpenBMC configuration
- Boot time is critical
- No enterprise compliance requirements

Choose OpenSSH When

- Enterprise security compliance required (SOX, PCI-DSS, HIPAA)
- Need SSH certificate authentication for fleet management
- Require detailed audit logging
- Need chroot/jail for restricted users
- Using FIDO2/hardware security keys
- Advanced features like Match blocks needed
- Integrating with enterprise identity management

OpenSSH Hardening Recipe

```
# recipes-connectivity/openssh/openssh_%.bbappend

FILESEXTRAPATHS:prepend := "${THISDIR}/files:"

SRC_URI += " \
    file://sshd_config \
    file://ssh_host_ed25519_key \
    file://ssh_host_ed25519_key.pub \
"

do_install:append() {
    # Install hardened sshd_config
    install -m 0644 ${WORKDIR}/sshd_config ${D}${sysconfdir}/ssh/sshd_config

    # Install pre-generated host keys (for consistent fingerprint)
    install -m 0600 ${WORKDIR}/ssh_host_ed25519_key ${D}${sysconfdir}/ssh/
    install -m 0644 ${WORKDIR}/ssh_host_ed25519_key.pub ${D}${sysconfdir}/ssh/

    # Remove weak key types
    rm -f ${D}${sysconfdir}/ssh/ssh_host_dsa_key*
}
```

Migration Checklist

When switching from Dropbear to OpenSSH:

- ☐ Back up existing host keys and authorized_keys
- ☐ Update Yocto configuration to include OpenSSH
- ☐ Create and test sshd_config
- ☐ Regenerate or convert host keys
- ☐ Test authentication methods (password, key)
- ☐ Verify PAM integration works correctly
- ☐ Test SFTP functionality
- ☐ Update firewall rules if port changed
- ☐ Document the change for operations team
- ☐ Update monitoring/alerting for new service name (sshd vs dropbear)

References

- [Dropbear SSH](#)
 - [OpenBMC Security Documentation](#)
 - [Linux PAM Manual](#)
-

Deep Dive

This section provides detailed technical information for developers who want to understand SSH and PAM internals in OpenBMC.

Dropbear Connection Lifecycle

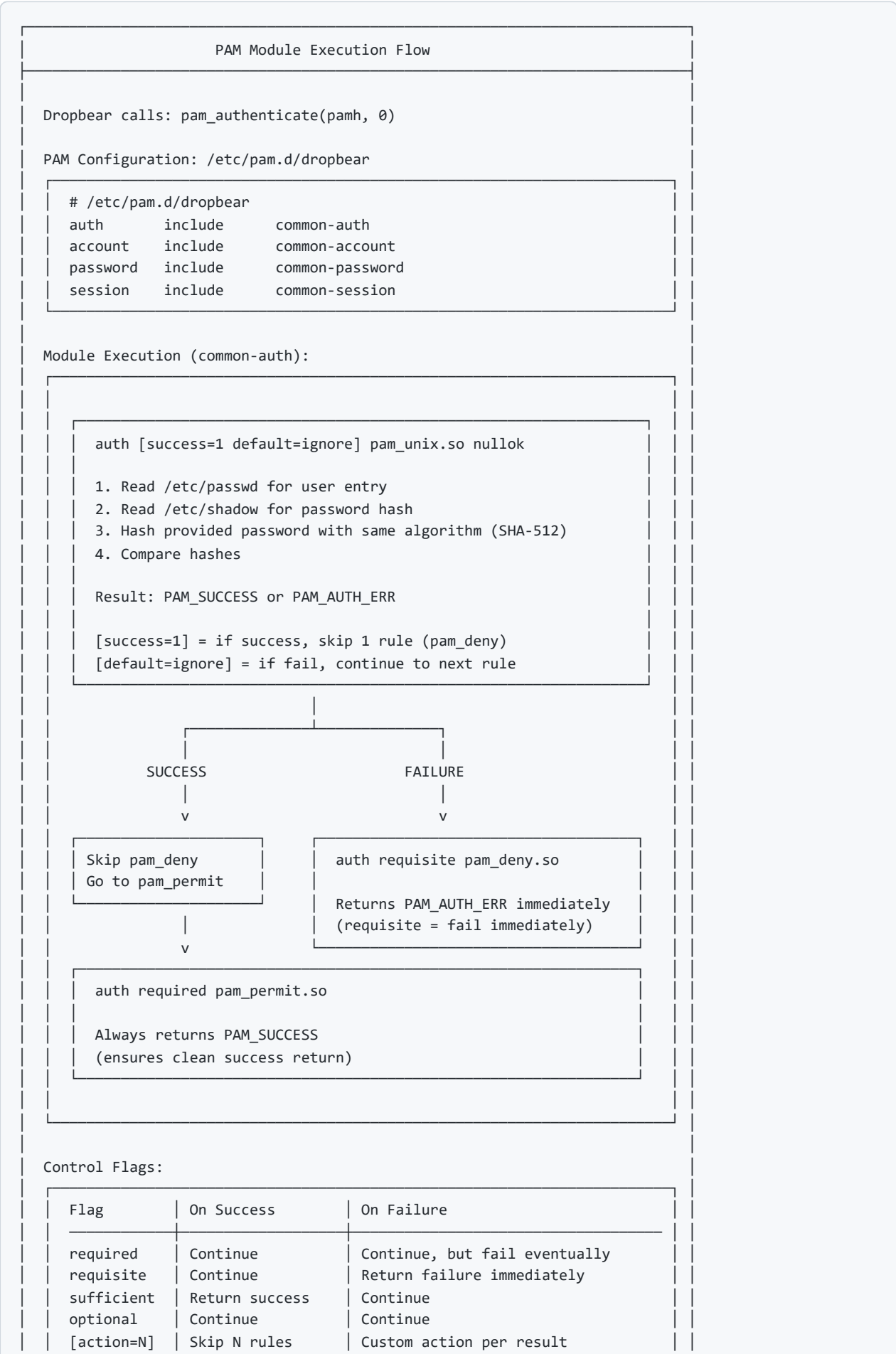


6. Interactive Session

```
graph LR; Client[CHANNEL_DATA (encrypted stdin/stdout)] --> Server[CHANNEL_DATA (shell I/O)];
```

The diagram illustrates an interactive session between a client and a server. On the left, a box represents the client, containing the text "CHANNEL_DATA (encrypted stdin/stdout)". On the right, a box represents the server, containing the text "CHANNEL_DATA (shell I/O)". A horizontal arrow points from the client box to the server box, indicating the direction of data flow.

PAM Authentication Stack





SSH Key Authentication Internals



2. `ed25519_verify(public_key, data_to_sign, signature)`
3. If valid: `SSH_MSG_USERAUTH_SUCCESS`

authorized_keys Format:

`[options] key-type base64-key [comment]`

Examples:

`ssh-ed25519 AAAAC3Nz... user@laptop`

`from="192.168.1.*" ssh-rsa AAAAB3Nz... admin@server`

`command="/usr/bin/validate" ssh-ed25519 AAAAC3... script@automation`

`no-pty,no-port-forwarding ssh-rsa AAAAB3... restricted@host`

Host Key Management

Dropbear Host Key Generation

Key Storage: /etc/dropbear/

```
/etc/dropbear/  
├─ dropbear_rsa_host_key      # RSA 2048/4096-bit  
├─ dropbear_ecdsa_host_key   # ECDSA nistp256/384/521  
└─ dropbear_ed25519_host_key # Ed25519 (recommended)
```

Key Generation (first boot or manual):

```
# Generate Ed25519 key (fast, secure)  
dropbearkey -t ed25519 -f /etc/dropbear/dropbear_ed25519_host_key  
  
# Generate RSA key (for legacy client compatibility)  
dropbearkey -t rsa -s 4096 -f /etc/dropbear/dropbear_rsa_host_key  
  
# Generate ECDSA key  
dropbearkey -t ecdsa -s 521 -f /etc/dropbear/dropbear_ecdsa_host_key
```

Dropbear Key File Format (internal):

```
Not OpenSSH compatible - binary format specific to Dropbear  
  
Convert to OpenSSH format (for inspection):  
dropbearconvert dropbear openssh  
  /etc/dropbear/dropbear_ed25519_host_key  
  /tmp/openssh_host_key  
  
Extract public key:  
dropbearkey -y -f /etc/dropbear/dropbear_ed25519_host_key  
# Output: ssh-ed25519 AAAAC3Nz... root@bmc
```

Yocto Persistent Key Configuration:

```
# Option 1: Generate at first boot (in init script)  
if [ ! -f /etc/dropbear/dropbear_ed25519_host_key ]; then  
    dropbearkey -t ed25519 -f /etc/dropbear/dropbear_ed25519_host_key  
fi  
  
# Option 2: Pre-generate and include in image (production)  
# In recipe .bbappend:  
SRC_URI += "file://dropbear_ed25519_host_key"  
do_install:append() {  
    install -m 0600 ${WORKDIR}/dropbear_ed25519_host_key \  
        ${D}${sysconfdir}/dropbear/  
}
```


Source Code References

Component	Repository	Key Files
Dropbear SSH Server	mkj/dropbear	<code>svr-main.c</code> , <code>svr-authpasswd.c</code> , <code>svr-authpubkey.c</code>
Dropbear Yocto Recipe	openembedded-core	<code>meta/recipes-core/dropbear/</code>
Linux PAM	linux-pam/linux-pam	<code>modules/pam_unix/</code> , <code>libpam/</code>
PAM Configuration	openbmc/openbmc	<code>meta-phosphor/recipes-extended/pam/</code>
OpenBMC Security Docs	openbmc/docs	<code>security/</code>

Tested on: OpenBMC master, QEMU romulus

Redfish Events & Telemetry

Subscribe to asynchronous Redfish events and collect periodic telemetry metric reports from OpenBMC.

Table of Contents

1. [Overview](#)
2. [Architecture](#)
 1. [How Events Flow](#)
 2. [How Telemetry Works](#)
3. [Configuration](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [Meson Build Options for Events & Telemetry](#)
 3. [Runtime Configuration](#)
 4. [Configure EventService Parameters](#)
4. [EventService Push Subscriptions](#)
 1. [Create a Push Subscription](#)
 2. [Subscription Properties](#)
 3. [List All Subscriptions](#)
 4. [Get a Specific Subscription](#)
 5. [Delete a Subscription](#)
 6. [Push Event Payload Format](#)
5. [Server-Sent Events \(SSE\) Streaming](#)
 1. [Open an SSE Connection](#)
 2. [SSE Event Format](#)
 3. [SSE Filtering with Query Parameters](#)
 4. [SSE with Python](#)
6. [Telemetry Metric Reports](#)
 1. [Get TelemetryService Capabilities](#)
 2. [Create a MetricReportDefinition](#)
 3. [MetricReportDefinition Properties](#)
 4. [List MetricReportDefinitions](#)
 5. [Get a Specific MetricReportDefinition](#)
 6. [Delete a MetricReportDefinition](#)
 7. [Retrieve a Metric Report](#)
 8. [Receive Metric Reports as Events](#)
7. [Subscription Limits and Lifecycle](#)
 1. [Subscription Limits](#)
 2. [Subscription Expiration](#)
 3. [Retry Behavior](#)

4. [SSE Connection Lifecycle](#)
 8. [Code Examples](#)
 1. [Example: Telemetry Collection Script](#)
 9. [Troubleshooting](#)
 1. [Issue: Subscription Creation Returns 400 or 403](#)
 2. [Issue: Push Events Not Arriving at Destination](#)
 3. [Issue: SSE Stream Closes Immediately](#)
 4. [Issue: MetricReport Returns Empty or 404](#)
 5. [Debug Commands](#)
 10. [References](#)
 1. [Official Resources](#)
 2. [Related Guides](#)
 3. [External Documentation](#)
-

Overview

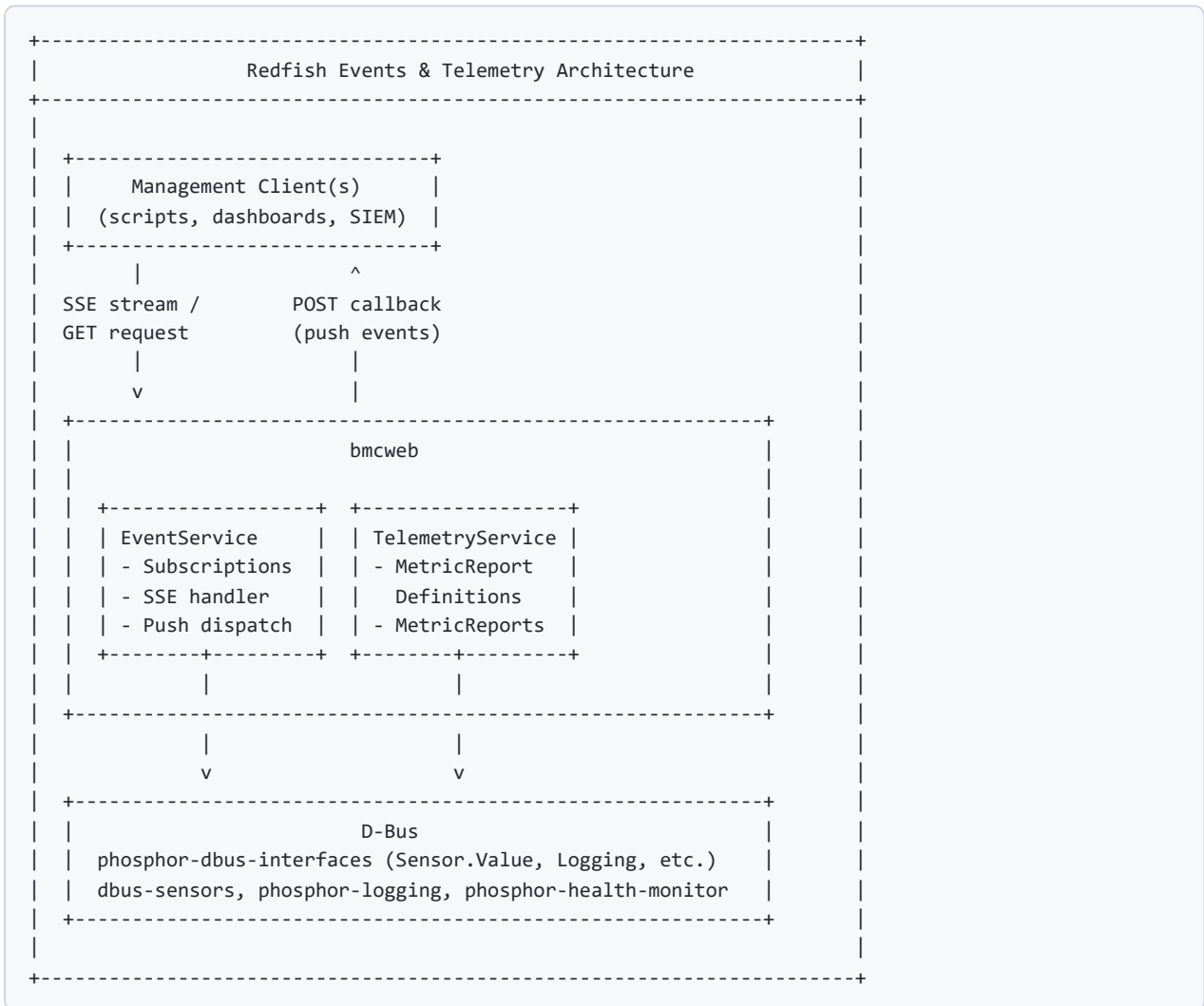
The Redfish **EventService** and **TelemetryService** allow management clients to receive real-time notifications and collect periodic sensor data without polling. OpenBMC implements both services through **bmcweb**, translating D-Bus signals into standards-compliant Redfish events and metric reports.

The EventService supports two delivery mechanisms: **push-style subscriptions** (HTTP POST callbacks) and **Server-Sent Events (SSE)** streaming. Push subscriptions send event payloads to a client-supplied destination URL, while SSE keeps a persistent HTTP connection open for continuous event delivery. The TelemetryService complements events by collecting sensor readings at defined intervals and storing them as retrievable metric reports.

Key concepts covered:

- Creating, listing, and deleting EventService push subscriptions
 - Opening and filtering SSE event streams
 - Defining and retrieving TelemetryService metric reports
 - Subscription lifecycle management (limits, expiration, retry)
-

Architecture



How Events Flow

1. A D-Bus signal fires (sensor threshold crossed, log entry created, inventory changed).
2. bmcweb's event handler matches the signal against active subscription filters.
3. For **push subscriptions**, bmcweb sends an HTTP POST to the subscriber's **Destination** URL.
4. For **SSE connections**, bmcweb writes the event to every open SSE stream that matches the filter.

How Telemetry Works

1. A **MetricReportDefinition** tells bmcweb which sensor properties to collect and how often.
2. The underlying **dbus-sensors** or **phosphor-virtual-sensor** services expose readings on D-Bus.
3. bmcweb (backed by **phosphor-health-monitor** or the telemetry daemon) samples those readings and stores a **MetricReport**.
4. Clients retrieve the report on demand or receive it as a **MetricReport** event through an active subscription.

Configuration

Build-Time Configuration (Yocto)

Enable event and telemetry features in your bmcweb build:

```
# In your machine .conf or local.conf
EXTRA_OEMESON:pn-bmcweb = " \
    -Dredfish=enabled \
    -Dinsecure-push-style-notification=enabled \
    -Dredfish-dbus-log=enabled \
"
```

The `insecure-push-style-notification` option enables push-style event delivery without requiring TLS verification of the subscriber destination. In production, use proper certificate validation. This flag is useful for QEMU development and testing.

Meson Build Options for Events & Telemetry

Option	Default	Description
<code>insecure-push-style-notification</code>	disabled	Push event POST to subscriber destinations
<code>redfish-dbus-log</code>	enabled	Map D-Bus log signals to Redfish events
<code>redfish-bmc-journal</code>	enabled	Expose BMC journal entries as log events

Runtime Configuration

```
# Check bmcweb status
systemctl status bmcweb

# Verify EventService is available
curl -k -u root:0penBmc \
    https://localhost/redfish/v1/EventService

# Verify TelemetryService is available
curl -k -u root:0penBmc \
    https://localhost/redfish/v1/TelemetryService
```

Configure EventService Parameters

```
# View current settings
curl -k -u root:openBmc \
  https://localhost/redfish/v1/EventService | jq

# Tune delivery retry behavior
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{
    "ServiceEnabled": true,
    "DeliveryRetryAttempts": 3,
    "DeliveryRetryIntervalSeconds": 60
  }' \
  https://localhost/redfish/v1/EventService
```

The `DeliveryRetryAttempts` and `DeliveryRetryIntervalSeconds` values apply to push-style subscriptions only. SSE streams reconnect at the client's discretion.

EventService Push Subscriptions

Push subscriptions instruct bmcweb to HTTP POST event payloads to a URL you specify. This is the standard Redfish event delivery model.

Create a Push Subscription

```
# Set up alias for convenience
alias bmcurl='curl -k -u root:openBmc '

# Create a subscription that receives Alert and ResourceUpdated events
bmcurl -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "Destination": "https://my-monitoring-server:8443/redfish-events",
    "Protocol": "Redfish",
    "EventFormatType": "Event",
    "RegistryPrefixes": ["OpenBMC"],
    "ResourceTypes": ["Chassis", "Systems"],
    "Context": "my-bmc-01-alerts"
  }' \
  https://localhost/redfish/v1/EventService/Subscriptions
```

The response includes a `Location` header with the new subscription URI, for example `/redfish/v1/EventService/Subscriptions/1`.

Subscription Properties

Property	Type	Description
<code>Destination</code>	string (URI)	URL that receives HTTP POST event payloads
<code>Protocol</code>	string	Must be <code>"Redfish"</code>
<code>EventFormatType</code>	string	<code>"Event"</code> (default) or <code>"MetricReport"</code>
<code>RegistryPrefixes</code>	array	Filter by message registry prefix (e.g., <code>["OpenBMC", "TaskEvent"]</code>)
<code>ResourceTypes</code>	array	Filter by resource type (e.g., <code>["Chassis", "Systems"]</code>)
<code>Context</code>	string	Opaque string echoed in every event payload for client correlation
<code>HttpHeaders</code>	array	Custom HTTP headers sent with each POST (e.g., auth tokens)

List All Subscriptions

```
bmcurl https://localhost/redfish/v1/EventService/Subscriptions | jq
```

Get a Specific Subscription

```
bmcurl https://localhost/redfish/v1/EventService/Subscriptions/1 | jq
```

Delete a Subscription

```
bmcurl -X DELETE \
  https://localhost/redfish/v1/EventService/Subscriptions/1
```

Push Event Payload Format

When an event fires, bmcweb POSTs a JSON body to the subscription destination:

```
{
  "@odata.type": "#Event.v1_4_0.Event",
  "Id": "1",
  "Name": "Event Array",
  "Context": "my-bmc-01-alerts",
  "Events": [
    {
      "EventType": "Alert",
      "Severity": "Warning",
      "Message": "Temperature threshold exceeded on CPU_Temp.",
      "MessageId": "OpenBMC.0.1.SensorThresholdWarning",
      "MessageArgs": ["CPU_Temp", "85", "80"],
      "OriginOfCondition": {
        "@odata.id": "/redfish/v1/Chassis/chassis/Sensors/CPU_Temp"
      },
      "EventTimestamp": "2026-02-06T14:30:00Z"
    }
  ]
}
```

Use `RegistryPrefixes` and `ResourceTypes` filters to reduce noise. A subscription with no filters receives all events, which can overwhelm the destination under heavy load.

Server-Sent Events (SSE) Streaming

SSE provides a persistent, one-way HTTP stream from bmcweb to the client. It is useful for real-time dashboards, monitoring scripts, and situations where the client cannot expose an inbound HTTP endpoint.

Open an SSE Connection

```
# Open SSE stream (runs until you press Ctrl+C)
curl -k -u root:openBmc -N \
  -H "Accept: text/event-stream" \
  https://localhost/redfish/v1/EventService/SSE
```

The `-N` flag disables output buffering so events appear immediately.

SSE Event Format

Each event arrives as a `text/event-stream` message:

```
id: 1
data: {"@odata.type": "#Event.v1_4_0.Event", "Id": "1", "Name": "Event Array", "Events": [{"EventType": "Alert", "MessageId": "OpenBMC.0.1.SensorThresholdWarning", "Message": "Temperature threshold exceeded on CPU_Temp.", "OriginOfCondition": {"@odata.id": "/redfish/v1/Chassis/chassis/Sensors/CPU_Temp"}, "EventTimestamp": "2026-02-06T14:30:00Z"}]}

id: 2
data: {"@odata.type": "#Event.v1_4_0.Event", "Id": "2", "Name": "Event Array", "Events": [{"EventType": "ResourceUpdated", "MessageId": "ResourceEvent.1.0.ResourceChanged", "OriginOfCondition": {"@odata.id": "/redfish/v1/Systems/system"}, "EventTimestamp": "2026-02-06T14:31:15Z"}]}
```

Each message has an `id:` line (for reconnection tracking) and a `data:` line containing the JSON event payload. Messages are separated by a blank line.

SSE Filtering with Query Parameters

You can filter the SSE stream using query parameters to receive only the events you need:


```
# Filter by registry prefix
curl -k -u root:0penBmc -N \
  -H "Accept: text/event-stream" \
  "https://localhost/redfish/v1/EventService/SSE?\"$filter=RegistryPrefix%20eq%200penBMC"

# Filter by resource type
curl -k -u root:0penBmc -N \
  -H "Accept: text/event-stream" \
  "https://localhost/redfish/v1/EventService/SSE?\"$filter=ResourceType%20eq%20Chassis"

# Filter by message ID
curl -k -u root:0penBmc -N \
  -H "Accept: text/event-stream" \
  "https://localhost/redfish/v1/EventService/SSE?\"$filter=MessageId%20eq%200penBMC.0.1.SensorThres
holdWarning"
```

URL-encode special characters in filter expressions. The `$` in `$filter` must be escaped or quoted in shell commands. The space around `eq` is required.

SSE with Python

```
#!/usr/bin/env python3
"""Listen for Redfish SSE events from OpenBMC."""
import requests
import json
import urllib3

# Disable TLS warnings for self-signed certificates
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

BMC_HOST = "https://localhost"
USERNAME = "root"
PASSWORD = "0penBmc"

url = f"{BMC_HOST}/redfish/v1/EventService/SSE"
headers = {"Accept": "text/event-stream"}

with requests.get(url, auth=(USERNAME, PASSWORD),
                  headers=headers, stream=True, verify=False) as resp:
    resp.raise_for_status()
    buffer = ""
    for chunk in resp.iter_content(decode_unicode=True):
        buffer += chunk
        while "\n\n" in buffer:
            message, buffer = buffer.split("\n\n", 1)
            for line in message.strip().split("\n"):
                if line.startswith("data: "):
                    event = json.loads(line[6:])
                    for e in event.get("Events", []):
                        print(f"[{e.get('EventTimestamp')}] "
                              f"{e.get('MessageId')}: {e.get('Message')}")
```

Telemetry Metric Reports

The TelemetryService collects sensor readings at defined intervals and stores them as metric reports. This is more efficient than polling individual sensor endpoints repeatedly.

Get TelemetryService Capabilities

```
bmcurl https://localhost/redfish/v1/TelemetryService | jq
```

Key properties in the response include `MaxReports` (maximum number of reports), `MinCollectionInterval` (shortest allowed sampling period, e.g., `"PT10S"`), and links to `MetricReportDefinitions` and `MetricReports` collections.

Create a MetricReportDefinition

A `MetricReportDefinition` specifies which metrics to collect, how often, and where to store the results.

```
bmcurl -X POST \
-H "Content-Type: application/json" \
-d '{
  "Id": "ThermalMetrics",
  "Name": "Thermal Monitoring Report",
  "MetricReportDefinitionType": "Periodic",
  "ReportActions": ["LogToMetricReportsCollection"],
  "ReportUpdates": "Overwrite",
  "Schedule": {
    "RecurrenceInterval": "PT30S"
  },
  "Metrics": [
    {
      "MetricId": "CPUTemp",
      "MetricProperties": [
        "/redfish/v1/Chassis/chassis/Sensors/CPU_Temp"
      ]
    },
    {
      "MetricId": "InletTemp",
      "MetricProperties": [
        "/redfish/v1/Chassis/chassis/Sensors/Inlet_Temp"
      ]
    }
  ]
}' \
https://localhost/redfish/v1/TelemetryService/MetricReportDefinitions
```

MetricReportDefinition Properties

Property	Type	Description
<code>Id</code>	string	Unique identifier for this definition
<code>MetricReportDefinitionType</code>	string	<code>"Periodic"</code> (timed) or <code>"OnChange"</code> (event-driven) or <code>"OnRequest"</code> (manual)
<code>ReportActions</code>	array	<code>"LogToMetricReportsCollection"</code> stores report; <code>"RedfishEvent"</code> sends as event
<code>ReportUpdates</code>	string	<code>"Overwrite"</code> replaces last report; <code>"AppendStopsWhenFull"</code> accumulates
<code>Schedule.RecurrenceInterval</code>	duration	ISO 8601 duration (e.g., <code>"PT30S"</code> = 30 seconds, <code>"PT5M"</code> = 5 minutes)
<code>Metrics[].MetricId</code>	string	Label for this metric in the report
<code>Metrics[].MetricProperties</code>	array	Redfish sensor URIs to sample

List MetricReportDefinitions

```
bmcurl https://localhost/redfish/v1/TelemetryService/MetricReportDefinitions | jq
```

Get a Specific MetricReportDefinition

```
bmcurl https://localhost/redfish/v1/TelemetryService/MetricReportDefinitions/ThermalMetrics | jq
```

Delete a MetricReportDefinition

```
bmcurl -X DELETE \
https://localhost/redfish/v1/TelemetryService/MetricReportDefinitions/ThermalMetrics
```

Retrieve a Metric Report

Once collection runs, retrieve the latest report:

```
bmcurl https://localhost/redfish/v1/TelemetryService/MetricReports/ThermalMetrics | jq
```

Example response:

```
{
  "@odata.id": "/redfish/v1/TelemetryService/MetricReports/ThermalMetrics",
  "@odata.type": "#MetricReport.v1_4_0.MetricReport",
  "Id": "ThermalMetrics",
  "Name": "Thermal Monitoring Report",
  "ReportSequence": "42",
  "Timestamp": "2026-02-06T14:35:00Z",
  "MetricValues": [
    {
      "MetricId": "CPUTemp",
      "MetricValue": "47.5",
      "Timestamp": "2026-02-06T14:35:00Z",
      "MetricProperty": "/redfish/v1/Chassis/chassis/Sensors/CPU_Temp"
    },
    {
      "MetricId": "InletTemp",
      "MetricValue": "28.0",
      "Timestamp": "2026-02-06T14:35:00Z",
      "MetricProperty": "/redfish/v1/Chassis/chassis/Sensors/Inlet_Temp"
    }
  ]
}
```

Receive Metric Reports as Events

To receive metric reports through an event subscription instead of polling, create a subscription with `EventFormatType` set to `"MetricReport"`:

```
bmcurl -X POST \
-H "Content-Type: application/json" \
-d '{
  "Destination": "https://my-monitoring-server:8443/telemetry",
  "Protocol": "Redfish",
  "EventFormatType": "MetricReport",
  "Context": "bmc-01-telemetry"
}' \
https://localhost/redfish/v1/EventService/Subscriptions
```

Then add `"RedfishEvent"` to the `ReportActions` array in the `MetricReportDefinition` to trigger delivery.

Subscription Limits and Lifecycle

Subscription Limits

OpenBMC enforces practical limits on event subscriptions to protect BMC resources:

Limit	Default Value	Description
Maximum subscriptions	20	Total push + SSE subscriptions allowed
Maximum SSE connections	10	Concurrent SSE streams
Delivery retry attempts	3	POST retries on failure before marking error
Delivery retry interval	60 seconds	Delay between retry attempts

Exceeding the subscription limit returns HTTP 400 with a `CreateLimitReachedForResource` message. Delete unused subscriptions before creating new ones.

Subscription Expiration

Subscriptions do not expire by default. They persist until explicitly deleted or the BMC is factory-reset. Monitor your subscription count and clean up stale entries to stay within the limit.

Retry Behavior

When a push delivery fails (destination unreachable, HTTP error response), bmcweb follows this retry sequence:

1. Wait `DeliveryRetryIntervalSeconds` (default 60s).
2. Retry the POST.
3. Repeat up to `DeliveryRetryAttempts` times (default 3).
4. After all retries are exhausted, the subscription enters a suspended state.

```
# Configure retry parameters
bmcurl -X PATCH \
  -H "Content-Type: application/json" \
  -d '{
    "DeliveryRetryAttempts": 5,
    "DeliveryRetryIntervalSeconds": 30
  }' \
  https://localhost/redfish/v1/EventService
```

SSE Connection Lifecycle

SSE connections remain open until one of these occurs:

- The client closes the connection (e.g., Ctrl+C).
- The BMC restarts or bmcweb restarts.
- An idle timeout expires (if configured).
- The maximum SSE connection limit is reached and the oldest stream is evicted.

Implement automatic reconnection in your SSE client. Use the `id:` field from the last received event to track your position. After reconnecting, you may miss events that occurred during the disconnect window.

Code Examples

Example: Telemetry Collection Script

```
#!/usr/bin/env bash
# telemetry-collector.sh - Create metric report and poll results
set -euo pipefail

BMC_HOST="https://localhost"
CREDS="root:0penBmc"
alias bmcurl="curl -sk -u ${CREDS}"

# Create a MetricReportDefinition for thermal data
bmcurl -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "Id": "EnvMetrics",
    "Name": "Environmental Metrics",
    "MetricReportDefinitionType": "Periodic",
    "ReportActions": ["LogToMetricReportsCollection"],
    "ReportUpdates": "Overwrite",
    "Schedule": {"RecurrenceInterval": "PT60S"},
    "Metrics": [
      {
        "MetricId": "CPUTemp",
        "MetricProperties": ["/redfish/v1/Chassis/chassis/Sensors/CPU_Temp"]
      }
    ]
  }' \
  "${BMC_HOST}/redfish/v1/TelemetryService/MetricReportDefinitions"

echo "MetricReportDefinition created. Polling every 65s (Ctrl+C to stop)..."

while true; do
  sleep 65
  echo "--- $(date -Iseconds) ---"
  bmcurl "${BMC_HOST}/redfish/v1/TelemetryService/MetricReports/EnvMetrics" | \
    jq '.MetricValues[] | {MetricId, MetricValue, Timestamp}'
done
```

See the complete examples in the [examples/redfish-events/](#) directory.

Troubleshooting

Issue: Subscription Creation Returns 400 or 403

Symptom: `POST` to `/redfish/v1/EventService/Subscriptions` returns HTTP 400 or 403.

Cause: The subscription limit has been reached, or push-style notifications are not enabled at build time.

Solution:

1. Check existing subscriptions and delete stale ones:

```
bmcurl https://localhost/redfish/v1/EventService/Subscriptions | jq '.Members'
bmcurl -X DELETE https://localhost/redfish/v1/EventService/Subscriptions/<id>
```

2. Verify that `insecure-push-style-notification` is enabled in your bmcweb build:

```
journalctl -u bmcweb | grep -i "push"
```

3. Rebuild bmcweb with the flag enabled if needed.

Issue: Push Events Not Arriving at Destination

Symptom: Subscription exists but no HTTP POST requests reach the destination server.

Cause: Network connectivity, TLS certificate validation, or firewall blocking the BMC's outbound connection.

Solution:

1. Test basic connectivity from the BMC:

```
curl -k https://my-monitoring-server:8443/
```

2. Check bmcweb logs for delivery errors:

```
journalctl -u bmcweb -f | grep -i "event\|subscription\|delivery"
```

3. Verify the destination URL is reachable from the BMC network namespace.

Issue: SSE Stream Closes Immediately

Symptom: `curl` to the SSE endpoint returns immediately with no data or an error.

Cause: Missing `Accept: text/event-stream` header, authentication failure, or SSE connection limit reached.

Solution:

1. Ensure you include the correct headers and authentication:

```
curl -k -u root:0penBmc -N \  
-H "Accept: text/event-stream" \  
https://localhost/redfish/v1/EventService/SSE
```

2. Check for open SSE connections consuming the limit:

```
ss -tnp | grep bmcweb
```

3. Restart bmcweb to clear stale connections if needed:

```
systemctl restart bmcweb
```

Issue: MetricReport Returns Empty or 404

Symptom: The MetricReport URI returns 404 or a report with no `MetricValues`.

Cause: The MetricReportDefinition has not yet collected data (first interval not elapsed), or the sensor URIs in `MetricProperties` are incorrect.

Solution:

1. Wait at least one collection interval after creating the definition.

2. Verify the sensor URLs exist:

```
bmcurl https://localhost/redfish/v1/Chassis/chassis/Sensors | jq '.Members[]["@odata.id"]'
```

3. Check the MetricReportDefinition status:

```
bmcurl https://localhost/redfish/v1/TelemetryService/MetricReportDefinitions/ThermalMetrics | jq '.Status'
```

Debug Commands

```
# Check bmcweb service status
systemctl status bmcweb

# View event-related logs
journalctl -u bmcweb -f --grep="event|subscription|telemetry|metric"

# List active network connections to bmcweb
ss -tnp | grep bmcweb

# Query D-Bus for sensor availability
busctl tree xyz.openbmc_project.HwmonTempSensor

# Check dbus-sensors services
systemctl list-units | grep sensor
```

References

Official Resources

- [bmcweb Repository](#) - Source code for EventService and TelemetryService handlers
- [phosphor-dbus-interfaces](#) - D-Bus interface definitions for sensors and logging
- [OpenBMC Documentation](#)

Related Guides

- [Redfish Guide](#) - Redfish fundamentals, authentication, and core operations

External Documentation

- [DMTF Redfish EventService Schema](#)
- [DMTF Redfish TelemetryService Schema](#)
- [DMTF Redfish Specification - Eventing](#)
- [Server-Sent Events \(SSE\) Specification](#)
- [ISO 8601 Duration Format](#)

Tested on: OpenBMC master, QEMU ast2600-evb. Push-style subscriptions require the [insecure-push-style-notification](#) build flag. SSE and TelemetryService are available with default bmcweb configuration. Sensor availability on QEMU may vary depending on the machine model and virtual sensor configuration.

Redfish OEM Extensions

Design, implement, and validate vendor-specific Redfish OEM extensions in OpenBMC bmcweb.

Table of Contents

1. [Overview](#)
2. [OEM Namespace Design](#)
 1. [The /Oem/ Pattern](#)
 2. [bmcweb OEM Router](#)
 3. [Namespace Design Guidelines](#)
3. [CSDL and OpenAPI Schema Creation](#)
 1. [JSON Schema](#)
 2. [CSDL Schema](#)
 3. [Hosting Schemas in bmcweb](#)
4. [Regenerating Redfish Schemas and Generated Code](#)
 1. [The Generation Pipeline](#)
 2. [Running the Workflow](#)
5. [OEM Route Implementation](#)
 1. [C++ Handler Structure](#)
 1. [Step 1: Create the OEM Header File](#)
 2. [Step 2: Implement a Resource with D-Bus Backend](#)
 3. [Step 3: Implement a POST Action](#)
 2. [Register Routes in bmcweb](#)
 3. [D-Bus Backend](#)
 4. [JSON Response Best Practices](#)
6. [Building and Deploying](#)
 1. [Yocto Integration](#)
 2. [Build and Test](#)
7. [Redfish Service Validator Testing](#)
 1. [Install and Run](#)
 2. [Validating with Custom OEM Schemas](#)
 3. [Interpreting Results](#)
 4. [Manual Curl Validation](#)
 5. [Automated Test Script](#)
8. [Extending Existing Resources with OEM Properties](#)
9. [Troubleshooting](#)
 1. [Route Not Found \(404\)](#)
 2. [D-Bus Errors](#)
 3. [Debug Logging](#)

4. Common Mistakes

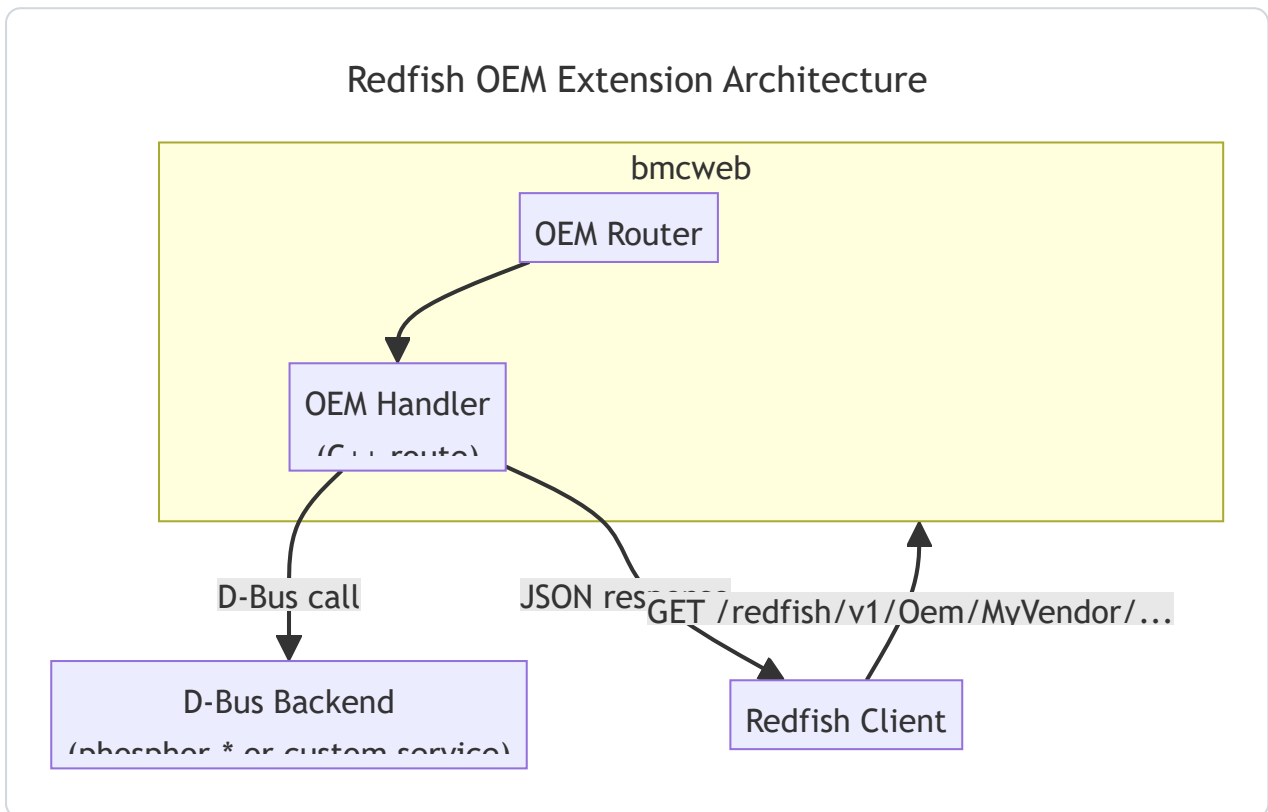
10. Examples

11. References

Overview

The Redfish standard provides a well-defined mechanism for vendors to expose platform-specific functionality through **OEM extensions**. In OpenBMC, OEM extensions are implemented in **bmcweb** and follow the `/Oem/<Vendor>` namespace pattern defined by the DMTF Redfish specification.

This guide covers the complete lifecycle: designing the namespace, creating schemas, implementing C++ route handlers with D-Bus backends, and validating with the Redfish Service Validator.



► ASCII-art version (for comparison)

OEM Namespace Design

The /Oem/ Pattern

Redfish uses a structured namespace to prevent collisions between vendor extensions. OEM extensions live either under the `Oem` property of a standard resource or under a dedicated `/redfish/v1/Oem/<Vendor>/` top-level route.

The DMTF requires that vendors use a unique identifier (typically your company name) as the `<Vendor>` portion. This prevents conflicts between different vendors' extensions on the same BMC.

Approach 1: Embedded OEM properties – Add vendor-specific properties to an existing resource:

```
{
  "@odata.id": "/redfish/v1/Systems/system",
  "Oem": {
    "MyVendor": {
      "@odata.type": "#OemComputerSystem.v1_0_0.MyVendor",
      "FirmwareProvisioning": { "ProvisioningStatus": "Provisioned" }
    }
  }
}
```

Approach 2: Standalone OEM resources – Create new resource trees:

```
/redfish/v1/Oem/MyVendor/
  BoardInfo                # Hardware details
  DiagnosticService/       # Diagnostics
    Actions/
      DiagnosticService.RunTest  # Run a test (POST)
```

bmcweb OEM Router

bmcweb uses the `BMCWEB_ROUTE` macro to register HTTP routes. The routing flow for an OEM request:

```
GET /redfish/v1/Oem/MyVendor/BoardInfo
--> Route match: "/redfish/v1/Oem/<str>/BoardInfo"
--> Privilege check: privileges::getManager
--> Vendor validation: reject if vendorName != oemVendorName
--> D-Bus queries (async) --> JSON response
```

Always validate the vendor name in your handler. The route pattern `/redfish/v1/Oem/<str>/...` matches any string, so your handler must reject unknown vendor namespaces with a 404 response.

Namespace Design Guidelines

Guideline	Reason
Use your company name as <code><Vendor></code>	Avoids collisions with other vendors
Version your schema types (e.g., <code>v1_0_0</code>)	Enables backward-compatible evolution
Group related resources logically	Improves discoverability
Follow Redfish naming conventions	<code>PascalCase</code> for properties
Define actions under standard <code>Actions</code> property	Consistent with Redfish patterns

CSDL and OpenAPI Schema Creation

OEM schemas document your extensions and enable automated validation.

JSON Schema

Source Reference: The example schema follows the pattern from the [examples/redfish/oem-schema.json](#) file in this repository.

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "http://myvendor.com/schemas/v1/OemBoardInfo.v1_0_0.json",
  "title": "#OemBoardInfo.v1_0_0.BoardInfo",
  "description": "MyVendor OEM Board Information Resource",
  "type": "object",
  "properties": {
    "@odata.type": { "type": "string", "const": "#OemBoardInfo.v1_0_0.BoardInfo" },
    "@odata.id": { "type": "string", "format": "uri" },
    "Id": { "type": "string" },
    "Name": { "type": "string" },
    "BoardType": { "type": "string", "enum": ["Server", "Workstation", "Storage", "Network"] },
    "BoardRevision": { "type": "string" },
    "MaxPowerWatts": { "type": "integer", "minimum": 0 },
    "Status": { "$ref": "http://redfish.dmtf.org/schemas/v1/Resource.json#/definitions/Status" }
  },
  "required": ["@odata.type", "@odata.id", "Id", "Name"]
}
```

CSDL Schema

CSDL (Common Schema Definition Language) is the XML format used by the DMTF. The Redfish Service Validator uses CSDL to validate OEM resources.

```
<?xml version="1.0" encoding="UTF-8"?>
<edmx:Edmx xmlns:edmx="http://docs.oasis-open.org/odata/ns/edmx" Version="4.0">
  <edmx:Reference Uri="http://redfish.dmtf.org/schemas/v1/Resource_v1.xml">
    <edmx:Include Namespace="Resource"/>
    <edmx:Include Namespace="Resource.v1_0_0"/>
  </edmx:Reference>
  <edmx:DataServices>
    <Schema xmlns="http://docs.oasis-open.org/odata/ns/edm" Namespace="OemBoardInfo">
      <EntityType Name="BoardInfo" BaseType="Resource.v1_0_0.Resource" Abstract="true">
        <Annotation Term="OData.Description" String="MyVendor board information resource."/>
      </EntityType>
    </Schema>
    <Schema xmlns="http://docs.oasis-open.org/odata/ns/edm" Namespace="OemBoardInfo.v1_0_0">
      <EntityType Name="BoardInfo" BaseType="OemBoardInfo.BoardInfo">
        <Property Name="BoardType" Type="OemBoardInfo.v1_0_0.BoardType"/>
        <Property Name="BoardRevision" Type="Edm.String"/>
        <Property Name="MaxPowerWatts" Type="Edm.Int64"/>
      </EntityType>
      <EnumType Name="BoardType">
        <Member Name="Server"/><Member Name="Workstation"/>
        <Member Name="Storage"/><Member Name="Network"/>
      </EnumType>
    </Schema>
  </edmx:DataServices>
</edmx:Edmx>
```

Hosting Schemas in bmcweb

Place schema files in the bmcweb source tree so clients can discover them:

```
bmcweb/
static/redfish/v1/schema/
  OemBoardInfo_v1.xml      # CSDL schema
static/redfish/v1/JsonSchemas/
  OemBoardInfo.v1_0_0.json # JSON schema
```

You can host schemas externally if you prefer not to embed them in the BMC image. Set the `@odata.type` to reference your external schema URL instead.

Regenerating Redfish Schemas and Generated Code

Much of bmcweb is **generated** from DMTF schema files rather than hand-written: C++ enums, the aggregation collection table, and the message/privilege registries are all produced from CSDL/JSON by scripts in bmcweb's `scripts/` directory. Run this workflow after you **add or upgrade a standard schema** or **author an OEM CSDL extension**, then rebuild bmcweb so the generated headers pick up your changes.

The Generation Pipeline

Run these from the root of the bmcweb source tree, in order:

Step	Script	What it does
1	<code>scripts/update_schemas.py</code>	Clones the DMTF Redfish-Publications repo at a pinned tag, copies the CSDL/JSON schemas in, selects the newest version of each schema with a semantic <code>SchemaVersion</code> sort, and refreshes the symlinks for the installed schemas.
2	<code>scripts/generate_schema_enums.py</code>	Parses CSDL <code>EnumType</code> definitions into C++ <code>enum class</code> types under <code>redfish-core/include/generated/enums/</code> , emitting a <code>NLOHMANN_JSON_SERIALIZE_ENUM</code> macro for each so values (de)serialize to/from JSON automatically.
3	<code>scripts/generate_schema_collections.py</code>	Extracts the top-level collection URIs and writes them to <code>redfish-core/include/aggregation_utils.hpp</code> , which Redfish Aggregation uses.
4	<code>scripts/parse_registries.py</code>	Generates the message-registry headers (for example <code>base_message_registry.hpp</code>) and <code>privilege_registry.hpp</code> from the DMTF message registries and the privilege registry.
5	<code>scripts/cddl-to-json-converter/</code>	Holds the OpenBMC config (<code>openbmc-config.json</code>) for the DMTF <code>cddl-to-json.py</code> converter, which turns OEM/local CSDL into JSON schema so your OEM types are served as JSON. See the directory's <code>README.md</code> .

`scripts/update_schemas.py` runs the downstream generators for you after it refreshes the schemas, so a full upgrade is usually a single command. Run the individual generators only when you are iterating on one artifact (for example, regenerating just the enums after editing an `EnumType`).

A separate helper, `scripts/generate_auth_certificates.py`, creates mTLS **test** certificates (a CA plus server and client certificates) for exercising client-certificate authentication against bmcweb – handy when validating OEM routes that require mTLS.

Running the Workflow

```
cd bmcweb

# 1. Pull the latest DMTF schemas, re-select versions, refresh symlinks
# (this also invokes the generators below on your behalf)
python3 scripts/update_schemas.py

# 2-4. Regenerate individual artifacts while iterating
python3 scripts/generate_schema_enums.py
python3 scripts/generate_schema_collections.py
python3 scripts/parse_registries.py

# 5. (OEM) Convert your OEM CSDL to JSON with the DMTF converter,
# using OpenBMC's config -- see scripts/csd1-to-json-converter/README.md
# csdl-to-json.py --input <oem-csdl> --output <json-out> \
#               --config scripts/csd1-to-json-converter/openbmc-config.json

# Rebuild so the regenerated headers are compiled in
bitbake bmcweb -c cleansstate && bitbake bmcweb
```

When adding an **OEM CSDL** schema, add it to the input consumed by `scripts/csd1-to-json-converter/openbmc-config.json`, then re-run the pipeline. The generated JSON is what the Redfish Service Validator (below) uses to validate your OEM resources against their schema.

The generated headers under `redfish-core/include/generated/` are checked into the bmcweb tree. Regenerate them with these scripts rather than editing them by hand – manual edits are overwritten the next time anyone runs the pipeline.

OEM Route Implementation

C++ Handler Structure

OEM handlers in bmcweb follow the same async pattern as standard Redfish routes. Each handler receives a request, queries D-Bus for data, and populates a JSON response.

Source Reference: Pattern based on [bmcweb](#)

- Route patterns: [redfish-core/lib/managers.hpp](#)
- Privilege definitions: [registries/privilege_registry.hpp](#)

Step 1: Create the OEM Header File

Create `redfish-core/lib/oem_myvendor.hpp`:

```

#pragma once
#include "app.hpp"
#include "dbus_utility.hpp"
#include "query.hpp"
#include "registries/privilege_registry.hpp"
#include "utils/dbus_utils.hpp"
#include <nlohmann/json.hpp>
#include <sdbusplus/asio/property.hpp>

namespace redfish
{
constexpr const char* oemVendorName = "MyVendor";

// OEM Root: GET /redfish/v1/Oem/MyVendor/
inline void requestRoutesOemRoot(App& app)
{
    BMCWEB_ROUTE(app, "/redfish/v1/Oem/<str>/")
        .privileges(redfish::privileges::getManager)
        .methods(boost::beast::http::verb::get)(
            [](const crow::Request&,
               const std::shared_ptr<bmcweb::AsyncResp>& asyncResp,
               const std::string& vendorName) {
                if (vendorName != oemVendorName)
                {
                    messages::resourceNotFound(asyncResp->res,
                                                "OemRoot", vendorName);

                    return;
                }
                asyncResp->res.jsonValue["@odata.type"] =
                    "#OemServiceRoot.v1_0_0.OemServiceRoot";
                asyncResp->res.jsonValue["@odata.id"] =
                    "/redfish/v1/Oem/MyVendor";
                asyncResp->res.jsonValue["Id"] = "MyVendor";
                asyncResp->res.jsonValue["Name"] =
                    "MyVendor OEM Extensions";
                asyncResp->res.jsonValue["BoardInfo"]["@odata.id"] =
                    "/redfish/v1/Oem/MyVendor/BoardInfo";
            });
}
} // namespace redfish

```

Step 2: Implement a Resource with D-Bus Backend

Read properties from a D-Bus service and map them to JSON:

```

// Board Info: GET /redfish/v1/Oem/MyVendor/BoardInfo
inline void requestRoutesBoardInfo(App& app)
{
    BMCWEB_ROUTE(app, "/redfish/v1/Oem/<str>/BoardInfo")
        .privileges(redfish::privileges::getManager)
        .methods(boost::beast::http::verb::get)(
            [](const crow::Request&,
               const std::shared_ptr<bmcweb::AsyncResp>& asyncResp,
               const std::string& vendorName) {
                if (vendorName != oemVendorName)
                {
                    messages::resourceNotFound(asyncResp->res,
                                                "BoardInfo", vendorName);

                    return;
                }
                asyncResp->res.jsonValue["@odata.type"] =
                    "#OemBoardInfo.v1_0_0.BoardInfo";
                asyncResp->res.jsonValue["@odata.id"] =
                    "/redfish/v1/Oem/MyVendor/BoardInfo";
                asyncResp->res.jsonValue["Id"] = "BoardInfo";
                asyncResp->res.jsonValue["Name"] = "Board Information";

                // Async D-Bus query for manufacturer
                sdbusplus::asio::getProperty<std::string>(
                    *crow::connections::systemBus,
                    "xyz.openbmc_project.Inventory.Manager",
                    "/xyz/openbmc_project/inventory/system/board",
                    "xyz.openbmc_project.Inventory.Decorator.Asset",
                    "Manufacturer",
                    [asyncResp](const boost::system::error_code& ec,
                               const std::string& manufacturer) {
                        if (ec)
                        {
                            asyncResp->res.jsonValue["Manufacturer"] = "Unknown";
                            return;
                        }
                        asyncResp->res.jsonValue["Manufacturer"] = manufacturer;
                    });

                asyncResp->res.jsonValue["BoardType"] = "Server";
                asyncResp->res.jsonValue["Status"]["State"] = "Enabled";
                asyncResp->res.jsonValue["Status"]["Health"] = "OK";
            });
}

```

Step 3: Implement a POST Action

Actions allow clients to trigger operations on the BMC:


```

// POST /redfish/v1/Oem/MyVendor/DiagnosticService/Actions/DiagnosticService.RunTest
inline void requestRoutesRunTest(App& app)
{
    BMCWEB_ROUTE(app,
        "/redfish/v1/Oem/<str>/DiagnosticService/Actions/"
        "DiagnosticService.RunTest")
        .privileges(redfish::privileges::postManager)
        .methods(boost::beast::http::verb::post)(
            [](const crow::Request& req,
                const std::shared_ptr<bmcweb::AsyncResp>& asyncResp,
                const std::string& vendorName) {
                if (vendorName != oemVendorName)
                {
                    messages::resourceNotFound(asyncResp->res, "Action", "RunTest");
                    return;
                }
                std::optional<uint32_t> testId;
                if (!json_util::readJsonAction(req, asyncResp->res, "TestId", testId))
                {
                    return;
                }
                if (!testId || *testId < 1 || *testId > 3)
                {
                    messages::actionParameterValueError(asyncResp->res,
                        testId ? std::to_string(*testId) : "null",
                        "TestId", "DiagnosticService.RunTest");
                    return;
                }
                crow::connections::systemBus->async_method_call(
                    [asyncResp](const boost::system::error_code& ec) {
                        if (ec)
                        {
                            messages::internalError(asyncResp->res);
                            return;
                        }
                        asyncResp->res.result(boost::beast::http::status::accepted);
                        asyncResp->res.jsonValue["TaskState"] = "Running";
                    },
                    "xyz.openbmc_project.OEM.DiagService",
                    "/xyz/openbmc_project/oem/diag",
                    "xyz.openbmc_project.OEM.DiagService",
                    "RunTest", *testId);
            });
}

```

Register Routes in bmcweb

```

// In src/webserver_main.cpp
#include "oem_myvendor.hpp"

// In the route registration section:
redfish::requestRoutesOemRoot(app);
redfish::requestRoutesBoardInfo(app);
redfish::requestRoutesRunTest(app);

```

D-Bus Backend

Your OEM handler reads from or writes to a D-Bus service. You can use an existing phosphor-* service or create a custom daemon:

```

bmcweb OEM Handler <-- D-Bus --> Backend Service (xyz.openbmc_project.OEM.MyService)
  sdbusplus::asio::getProperty()           |
  async_method_call()                       v
                                           Hardware / sysfs / devmem

```

For simple OEM properties that map to existing inventory or sensor D-Bus objects, you do not need a custom D-Bus service. Use `xyz.openbmc_project.Inventory.Manager` or sensor services directly.

JSON Response Best Practices

Property	Requirement	Example
<code>@odata.type</code>	Required. Versioned type.	<code>#OemBoardInfo.v1_0_0.BoardInfo</code>
<code>@odata.id</code>	Required. Full URI.	<code>/redfish/v1/Oem/MyVendor/BoardInfo</code>
<code>Id</code>	Required. Short identifier.	<code>BoardInfo</code>
<code>Name</code>	Required. Human-readable.	<code>Board Information</code>
<code>Status</code>	Recommended. Standard Redfish.	<code>{"State":"Enabled","Health":"OK"}</code>
<code>Actions</code>	If applicable.	<code>{"#MyAction.Do":{"target":"..."}}</code>

Building and Deploying

Yocto Integration

Create a bbappend to include your OEM handler in the bmcweb build:

```

# recipes-phosphor/bmcweb/bmcweb_%.bbappend
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"

SRC_URI += " \
    file://oem_myvendor.hpp \
    file://0001-Add-MyVendor-OEM-routes.patch \
"

do_configure:prepend() {
    cp ${WORKDIR}/oem_myvendor.hpp ${S}/redfish-core/lib/oem_myvendor.hpp
}

```

The patch file registers your routes in `webserver_main.cpp`.

Build and Test

```
# Build bmcweb with OEM changes
bitbake bmcweb -c cleansstate && bitbake bmcweb

# Full image build for QEMU
bitbake obmc-phosphor-image

# Or copy binary directly for quick testing
scp build/tmp/work/*/bmcweb/*/build/bmcweb root@<bmc-ip>:/usr/bin/
ssh root@<bmc-ip> systemctl restart bmcweb
```

Redfish Service Validator Testing

The DMTF Redfish Service Validator verifies that your Redfish implementation conforms to the specification.

Install and Run

```
git clone https://github.com/DMTF/Redfish-Service-Validator.git
cd Redfish-Service-Validator
pip3 install -r requirements.txt

# Validate the entire service
python3 RedfishServiceValidator.py \
    --ip https://<bmc-ip> --authtype Basic \
    --username root --password OpenBmc --nochkcert

# Validate a specific OEM resource
python3 RedfishServiceValidator.py \
    --ip https://<bmc-ip> --authtype Basic \
    --username root --password OpenBmc --nochkcert \
    --uri /redfish/v1/Oem/MyVendor/BoardInfo
```

Validating with Custom OEM Schemas

Provide a local schema directory to validate OEM resources against your custom schemas:

```
mkdir -p oem-schemas/
cp OemBoardInfo_v1.xml oem-schemas/

python3 RedfishServiceValidator.py \
    --ip https://<bmc-ip> --authtype Basic \
    --username root --password OpenBmc --nochkcert \
    --schemadir ./oem-schemas/ --logdir ./validation-results/
```

The validator reports OEM resources as “unknown type” if you do not provide custom schemas. This is expected. Supply your CSDL schemas via `--schemadir` to get full property validation.

Interpreting Results

Check	What It Validates
<code>@odata.type</code> present	Every resource must declare its type
<code>@odata.id</code> matches URI	The <code>@odata.id</code> value must match the request URI
Required properties	All <code>required</code> properties in the schema are present
Property types	Values match declared types (string, integer, etc.)
Enum values	Enum properties contain only allowed values
Links resolve	Navigation properties return valid resources

Manual Curl Validation

```
alias bmcurl='curl -k -s -u root:0penBmc'

# Test OEM root
bmcurl https://localhost/redfish/v1/Oem/MyVendor/ | jq .

# Test a specific resource
bmcurl https://localhost/redfish/v1/Oem/MyVendor/BoardInfo | jq .

# Test an action
bmcurl -X POST -H "Content-Type: application/json" \
  -d '{"TestId": 1}' \
  https://localhost/redfish/v1/Oem/MyVendor/DiagnosticService/Actions/DiagnosticService.RunTest |
jq .
```

Automated Test Script

```
cd docs/examples/redfish/
./test_oem_resource.sh <bmc-ip> root 0penBmc
```

Extending Existing Resources with OEM Properties

You can add OEM properties to existing standard Redfish resources by patching the handler:

```
// In redfish-core/lib/systems.cpp (or via bbappend patch)
// Inside handleSystemsGet, after standard properties:

nlohmann::json& oem = asyncResp->res.jsonValue["Oem"]["MyVendor"];
oem["@odata.type"] = "#OemComputerSystem.v1_0_0.MyVendor";

sdbusplus::asio::getProperty<std::string>(
    *crow::connections::systemBus,
    "xyz.openbmc_project.OEM.MyService",
    "/xyz/openbmc_project/oem/system",
    "xyz.openbmc_project.OEM.SystemInfo", "ProvisioningStatus",
    [asyncResp](const boost::system::error_code& ec,
                const std::string& status) {
        if (!ec)
            asyncResp->res.jsonValue["Oem"]["MyVendor"]
                ["FirmwareProvisioning"]["ProvisioningStatus"] = status;
    });
```

Troubleshooting

Route Not Found (404)

```
systemctl status bmcweb
journalctl -u bmcweb | grep -i "route\\|oem"
bmcurl https://localhost/redfish/v1/Oem/MyVendor/    # trailing slash matters
```

D-Bus Errors

```
busctl list | grep -i oem
busctl tree xyz.openbmc_project.OEM.MyService
busctl get-property xyz.openbmc_project.Inventory.Manager \
    /xyz/openbmc_project/inventory/system/board \
    xyz.openbmc_project.Inventory.Decorator.Asset Manufacturer
```

Debug Logging

```
# Edit /lib/systemd/system/bmcweb.service, add:
# Environment="BMCWEB_LOG_LEVEL=DEBUG"
systemctl daemon-reload && systemctl restart bmcweb
journalctl -u bmcweb -f
```

Common Mistakes

Issue	Cause	Solution
404 on OEM route	Route not registered	Add registration in <code>webserver_main.cpp</code>
Empty JSON response	Handler returns before async callbacks	Use <code>asyncResp</code> shared_ptr pattern
500 Internal Server Error	D-Bus service not running	Verify with <code>busctl</code>
Missing <code>@odata.type</code>	Forgot to set type in handler	Always set <code>@odata.type</code> first
Validator warns "unknown type"	No custom schema provided	Supply CSDL via <code>--schemadir</code>

Examples

Working examples are available in the [examples/redfish](#) directory:

- `oem_resource.hpp` – Complete OEM handler with root, BoardInfo, DiagnosticService, and RunTest action
- `oem-schema.json` – JSON Schema for the OEM BoardInfo resource
- `test_oem_resource.sh` – Automated test script for validating OEM endpoints

Use the example files as a starting point. Copy `oem_resource.hpp`, replace `MyVendor` with your vendor name, and modify the resource properties and D-Bus paths to match your hardware platform.

References

- [bmcweb](#) – OpenBMC Redfish/Web server
- [DMTF Redfish Specification](#) – Official specification
- [Redfish Schema Index](#) – All standard Redfish schemas
- [Redfish Service Validator](#) – DMTF validation tool
- [Redfish Resource and Schema Guide](#) – OEM extension guidance
- [OpenBMC Redfish Design](#) – Design documents

Tested on: OpenBMC master, QEMU ast2600-evb

Advanced Topics

Dive into advanced protocols, security features, logging, and debug capabilities.

What You'll Learn

- MCTP and PLDM for platform management (sensors, firmware update, BIOS config)
- SPDM for device attestation and security
- Secure boot and image signing
- Logging, diagnostics, and debug tools
- POST code monitoring and debug dump collection
- Firmware update mechanisms (BMC and BIOS/host firmware)
- Linux kernel and application debugging (sanitizers, Valgrind)
- eSPI communication (virtual wires, console, eDAF)
- Unit testing with GTest/GMock
- Integration testing with Robot Framework
- Linux kernel patching for driver development
- SGPIO (serial GPIO) for expanding signal count over a few shared pins
- Multi-host BMC management
- Performance optimization for constrained BMC environments

Topics

Protocols & Security

Modern platform management relies on standardized protocols for component communication and security.

Guide	Description	Difficulty
MCTP/PLDM	Platform management protocols	Advanced
PLDM Platform Monitoring	Type 2 sensors, PDRs, effecters, events	Advanced
SPDM	Device attestation, security	Advanced
Secure Boot & Signing	Image signing, hardware root of trust	Advanced
DICE Security	Hardware root-of-trust for device identity and firmware attestation	Advanced

Firmware Management

Guide	Description
Firmware Update	BMC/PNOR updates, signing
PLDM Firmware Update	Type 5 device firmware update via PLDM
BIOS Firmware Management	Host firmware update via BMC, BIOS config
Device Firmware Update	CPLD, voltage regulator, EEPROM, TPM device updaters

Debug Features

Vendor-specific tools for deep system debugging.

Guide	Description	Platform
Intel ASD/ACD	At-Scale Debug, Crash Dump, Error Injection	Intel
AMD Debug & Management	APML, HDT, EPYC system management	AMD

Logging & Diagnostics

Capture events, errors, and diagnostic data for troubleshooting.

Guide	Description
Logging	Event logs, SEL, POST codes, debug dumps
SDR	Sensor Data Records for IPMI
Linux Debug Tools	ASan, UBSan, TSan, Valgrind, KASAN
Debug Dump Collection	phosphor-debug-collector, dreport plugins
POST Code Monitoring	BIOS POST codes, boot progress tracking

Host Communication

Low-level interfaces for BMC-to-host communication.

Guide	Description	Platform
eSPI	Virtual wires, console, eDAF boot	Intel/AMD

Testing

Validate code correctness with unit tests and system behavior with integration tests.

Guide	Description	Scope
Unit Testing	GTest/GMock for C++ unit tests	Single function/class
Robot Framework	Integration testing with openbmc-test-automation	Full system behavior

Kernel & Driver Development

Develop and debug Linux kernel drivers for BMC hardware.

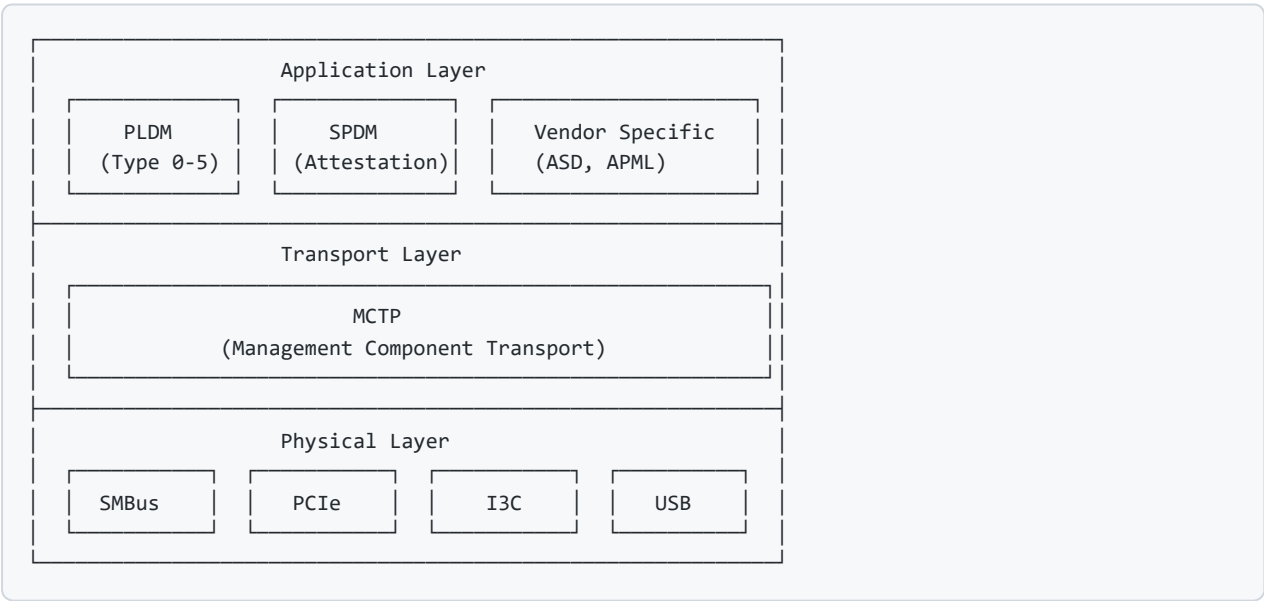
Guide	Description	Difficulty
Linux Kernel Driver Development	Kernel patching, out-of-tree modules, userspace drivers, debugging, I2C/SPI binding	Advanced
SGPIO (Serial GPIO)	Serial GPIO master on ASPEED/Nuvoton — device tree, driver, and usage compared with parallel GPIO	Advanced

Multi-Host & Platform Scaling

Manage multiple hosts from a single BMC and optimize for resource-constrained environments.

Guide	Description	Difficulty
Multi-Host Support	Multi-host BMC management, instance routing	Advanced
Performance Optimization	Memory, startup, and I/O optimization	Advanced

Protocol Stack



Prerequisites

- Strong understanding of [Architecture](#)
- Familiarity with [Core Services](#)
- Experience with [Interfaces](#)
- Knowledge of low-level hardware interfaces (I2C, PCIe)

Advanced features often require specific hardware support. Check your platform capabilities before implementing.

Table of contents

- [MCTP & PLDM Guide](#)
- [SPDM Guide](#)
- [Firmware Update Guide](#)
- [Intel ASD/ACD Guide](#)
- [AMD Debug & Management Guide](#)
- [Logging Guide](#)
- [SDR Guide](#)
- [Linux Debug Tools Guide](#)
- [eSPI Guide](#)
- [Unit Testing Guide](#)
- [Robot Framework Guide](#)
- [Linux Kernel Driver Development](#)
- [Secure Boot & Image Signing](#)
- [Multi-Host Support](#)
- [Performance Optimization](#)
- [BIOS Firmware Management](#)
- [Debug Dump Collection](#)
- [POST Code Monitoring](#)
- [PLDM Platform Monitoring](#)
- [PLDM Firmware Update](#)
- [DICE Security](#)
- [SGPIO \(Serial GPIO\)](#)
- [Device Firmware Update](#)

MCTP & PLDM Guide

Configure and use MCTP and PLDM protocols on OpenBMC.

Table of Contents

1. [Overview](#)
 1. [Why MCTP/PLDM?](#)
 2. [Protocol Stack](#)
2. [MCTP Protocol Deep Dive](#)
 1. [MCTP Packet Format](#)
 1. [Header Fields](#)
 2. [Special Endpoint IDs](#)
 2. [MCTP Message Types](#)
 3. [MCTP Control Messages](#)
 1. [Control Commands](#)
 4. [Message Fragmentation](#)
 1. [MTU by Transport](#)
3. [MCTP Transport Bindings](#)
 1. [I2C/SMBus Binding \(DSP0237\)](#)
 1. [I2C Configuration](#)
 2. [Device Tree for I2C MCTP](#)
 2. [PCIe VDM Binding \(DSP0238\)](#)
 1. [PCIe VDM Configuration](#)
 3. [Serial Binding \(DSP0239\)](#)
4. [PLDM Protocol Deep Dive](#)
 1. [PLDM Message Format](#)
 1. [Header Fields](#)
 2. [PLDM Types](#)
 3. [PLDM Type 0: Base Commands](#)
 1. [Example: Get TID](#)
 4. [PLDM Type 2: Monitoring & Control](#)
 1. [Sensor Commands](#)
 2. [Effector Commands](#)
 3. [Event Commands](#)
 5. [PLDM Type 5: Firmware Update](#)
5. [Platform Data Records \(PDR\)](#)
 1. [PDR Structure](#)
 2. [PDR Types](#)
 3. [Numeric Sensor PDR Example](#)
 4. [State Sets](#)

5. [Querying PDRs](#)
6. [OpenBMC Implementation](#)
 1. [Architecture](#)
 2. [libmctp](#)
 3. [libpldm](#)
 4. [pldm Configuration](#)
7. [Setup & Configuration](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [Meson Build Options \(pldm\)](#)
 3. [Meson Build Options \(mctpd\)](#)
 4. [Kernel Configuration](#)
 5. [Runtime Configuration](#)
8. [MCTP Configuration](#)
 1. [Managing MCTP Networks](#)
 2. [Endpoint Management](#)
 3. [Static Endpoint Configuration](#)
 4. [D-Bus Interface for MCTP](#)
9. [PLDM Configuration](#)
 1. [PDR Repository](#)
 2. [Common PLDM Commands](#)
 3. [Raw PLDM Commands](#)
10. [Device Integration Examples](#)
 1. [NVMe over MCTP](#)
 2. [GPU Management](#)
 3. [PSU Management](#)
 4. [Host CPU/BIOS Communication](#)
11. [PLDM Firmware Update](#)
 1. [Firmware Update Flow](#)
 2. [Firmware Update Commands](#)
12. [Event Handling](#)
 1. [PLDM Event Types](#)
 2. [Sensor Event Format](#)
 3. [Event Monitoring](#)
13. [Host-BMC PLDM Coordination](#)
14. [D-Bus Interfaces](#)
 1. [MCTP D-Bus Objects](#)
 2. [PLDM D-Bus Objects](#)
15. [Troubleshooting](#)
 1. [MCTP Connectivity Issues](#)
 2. [PLDM Communication Issues](#)
 3. [No Endpoints Discovered](#)

- 4. [Command Timeouts](#)
 - 5. [PDR Issues](#)
 - 16. [Security Considerations](#)
 - 1. [Authentication with SPDM](#)
 - 2. [Access Control](#)
 - 3. [Network Isolation](#)
 - 17. [Enabling/Disabling](#)
 - 1. [Build-Time Disable](#)
 - 2. [Runtime Disable](#)
 - 18. [Examples](#)
 - 19. [References](#)
 - 1. [DMTF Specifications](#)
 - 2. [OpenBMC Repositories](#)
 - 3. [Linux Kernel](#)
 - 20. [Deep Dive](#)
 - 1. [MCTP Message Fragmentation](#)
 - 2. [PLDM Message Type Encoding](#)
 - 3. [PDR \(Platform Descriptor Record\) Database](#)
 - 4. [Endpoint Discovery Flow](#)
 - 5. [Source Code Reference](#)
-

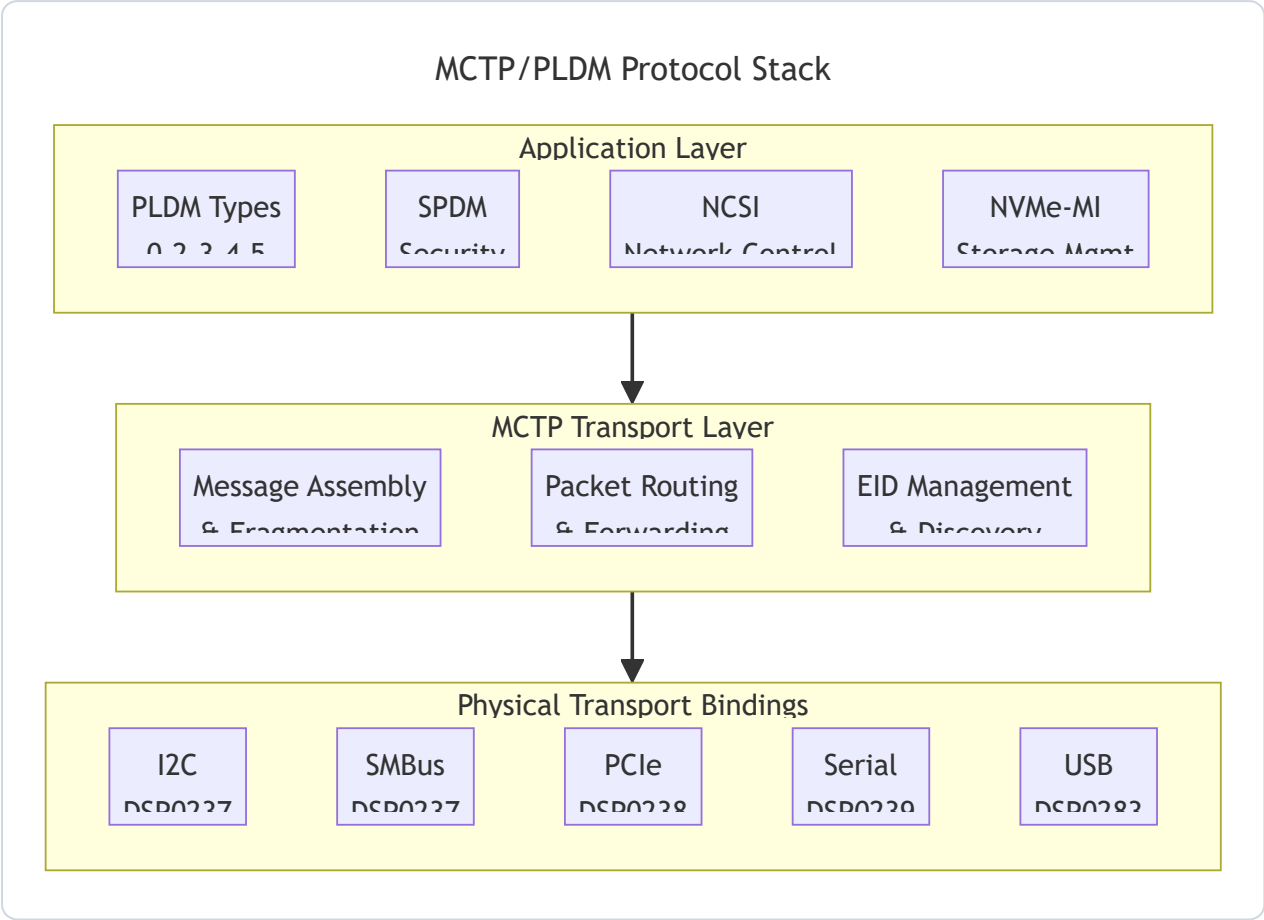
Overview

MCTP (Management Component Transport Protocol) and **PLDM (Platform Level Data Model)** are DMTF standards for platform management communication. They provide a modern, scalable alternative to IPMI for device-to-device management in data center environments.

Why MCTP/PLDM?

Feature	IPMI	MCTP/PLDM
Message size	256 bytes max	64KB+
Transport	KCS, BT, IPMB	I2C, PCIe, SMBus, USB, Serial
Extensibility	OEM codes	Typed messages with PDR
Discovery	Manual	Automatic endpoint discovery
Sensors	SDR-based	PDR with rich metadata
Security	Limited	SPDM integration

Protocol Stack

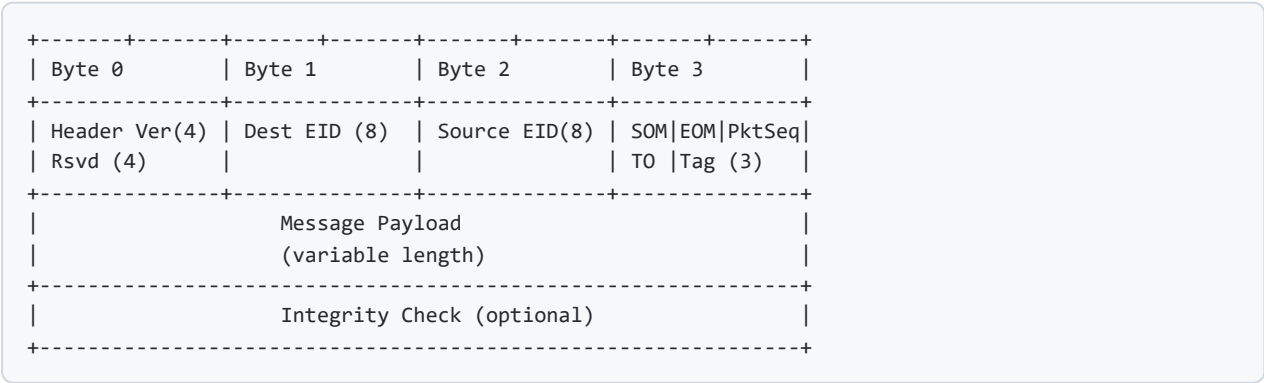


► ASCII-art version (for comparison)

MCTP Protocol Deep Dive

MCTP Packet Format

Every MCTP packet follows this structure:



Header Fields

Field	Bits	Description
Header Version	4	Always 0x1 for MCTP v1.x
Reserved	4	Must be 0
Destination EID	8	Target endpoint ID (0x00-0xFF)
Source EID	8	Sender endpoint ID
SOM	1	Start of Message flag
EOM	1	End of Message flag
Packet Sequence	2	Sequence number for fragmentation
TO	1	Tag Owner (1=requester, 0=responder)
Message Tag	3	Unique tag per outstanding request

Special Endpoint IDs

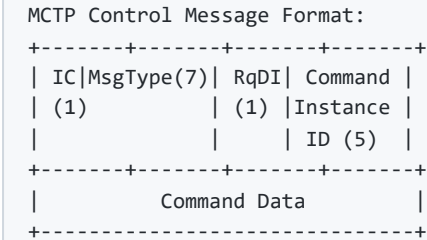
EID	Name	Purpose
0x00	Null EID	Unassigned endpoint
0x01-0x07	Reserved	Reserved by DMTF
0x08	Default BMC	Common BMC address
0x09-0xFE	General	Assignable to endpoints
0xFF	Broadcast	All endpoints on network

MCTP Message Types

Type	Value	Description	Specification
MCTP Control	0x00	Endpoint management	DSP0236
PLDM	0x01	Platform management	DSP0240
NCSI	0x02	Network controller	DSP0222
Ethernet	0x03	Ethernet frames	DSP0236
NVMe-MI	0x04	NVMe management	NVM Express
SPDM	0x05	Security protocol	DSP0274
Secure SPDM	0x06	Secured SPDM	DSP0277
Vendor Defined (PCI)	0x7E	PCI vendor ID	DSP0236
Vendor Defined (IANA)	0x7F	IANA enterprise	DSP0236

MCTP Control Messages

MCTP Control (type 0x00) manages the MCTP network:



Control Commands

Command	Code	Description
Set Endpoint ID	0x01	Assign EID to endpoint
Get Endpoint ID	0x02	Query endpoint's EID
Get Endpoint UUID	0x03	Get unique identifier
Get MCTP Version	0x04	Query MCTP version support
Get Message Type Support	0x05	Query supported message types
Get Vendor Defined Message Support	0x06	Query vendor extensions
Resolve Endpoint ID	0x07	Map EID to physical address
Allocate Endpoint IDs	0x08	Request EID pool
Routing Information Update	0x09	Update routing tables
Get Routing Table Entries	0x0A	Query routing table
Prepare Endpoint Discovery	0x0B	Reset discovery state
Endpoint Discovery	0x0C	Discover endpoints
Discovery Notify	0x0D	Notify discovery complete

Message Fragmentation

MCTP fragments large messages across multiple packets:

Large Message (> MTU):

```
+-----+-----+-----+
| Packet 1 | Packet 2 | Packet 3 |
| SOM=1, EOM=0 | SOM=0, EOM=0 | SOM=0, EOM=1 |
| Seq=0    | Seq=1    | Seq=2    |
| [Payload 1] | [Payload 2] | [Payload 3] |
+-----+-----+-----+
```

Reassembled at receiver:

```
+-----+
| Complete Message = Payload 1 + Payload 2 + Payload 3 |
+-----+
```

MTU by Transport

Transport	Minimum MTU	Typical MTU
I2C/SMBus	64 bytes	64-256 bytes
PCIe VDM	64 bytes	64-4096 bytes
Serial	64 bytes	64-256 bytes
USB	64 bytes	512 bytes

MCTP Transport Bindings

I2C/SMBus Binding (DSP0237)

I2C MCTP Packet Structure:

```
+-----+-----+-----+-----+-----+
| Slave | Cmd  | Byte | Src  | Hdr  | MCTP Header |
| Addr  | Code | Count | Addr | Ver  | + Payload    |
| (7+1) | 0x0F | (8)  | (7+1) | (4+4) |              |
+-----+-----+-----+-----+-----+
|                               Payload Data                               |
+-----+-----+-----+-----+-----+
| PEC (optional)                                                         |
+-----+-----+-----+-----+-----+
```

I2C Configuration

```
# Create MCTP I2C interface
mctp link add mctpi2c1 type i2c bus 1

# Set interface up
mctp link set mctpi2c1 up

# Set local address
mctp addr add 8 dev mctpi2c1

# Verify configuration
mctp link show
mctp addr show
```

Device Tree for I2C MCTP

```
&i2c1 {
    status = "okay";

    /* MCTP-capable device at address 0x50 */
    mctp_device@50 {
        compatible = "mctp-i2c-controller";
        reg = <0x50>;
    };
};
```

PCIe VDM Binding (DSP0238)

PCIe Vendor Defined Messages carry MCTP over PCIe:

PCIe VDM TLP Format:

Fmt	Type	TC	Attr		<- TLP Header
(3)	(5)	(3)	(2)		
Length		Requester ID			
Tag	Message Code				<- VDM Header
(8)	Vendor ID (16)				
MCTP Transport Header					<- MCTP Packet
MCTP Payload					

PCIe VDM Configuration

```
# PCIe MCTP typically auto-discovered
# Check for PCIe MCTP devices
lspci -vvv | grep -i mctp

# MCTP over PCIe uses kernel MCTP stack
mctp link
```

Serial Binding (DSP0239)

Serial MCTP Frame:

Sync	Rev	Byte	MCTP Packet
0x7E	(8)	Count	
FCS (16-bit CRC)			

PLDM Protocol Deep Dive

PLDM Message Format

PLDM Message Structure:

MCTP	PLDM Hdr	PLDM Payload
Msg		
Type		
0x01		

PLDM Header:

RqID	IID(5)	Hdr	PLDM
(1)	(1)	Ver(2)	Type
			(6)
Command Code (8)			

Header Fields

Field	Bits	Description
Rq (Request)	1	1=Request, 0=Response
D (Datagram)	1	1=No response expected
Instance ID	5	Request/response correlation
Header Version	2	Always 0b00
PLDM Type	6	Message type (0-63)
Command Code	8	Type-specific command

PLDM Types

Type	Name	Specification	Description
0	Base	DSP0240	Discovery, TID, versions
1	SMBIOS	DSP0246	SMBIOS table transfer
2	Monitoring & Control	DSP0248	Sensors, effecters, events
3	BIOS Control	DSP0247	BIOS attributes
4	FRU Data	DSP0257	Field Replaceable Unit data
5	Firmware Update	DSP0267	Firmware update protocol
6	Redfish Device Enablement	DSP0218	Redfish over PLDM
63	OEM	-	Vendor-specific

PLDM Type 0: Base Commands

Command	Code	Description
SetTID	0x01	Set Terminus ID
GetTID	0x02	Get Terminus ID
GetPLDMVersion	0x03	Query PLDM type versions
GetPLDMTypes	0x04	Query supported PLDM types
GetPLDMCommands	0x05	Query commands for a type
SelectPLDMVersion	0x06	Negotiate version
NegotiateTransferParameters	0x07	Set transfer sizes
MultipartSend	0x08	Send large data
MultipartReceive	0x09	Receive large data

Example: Get TID

```
# Request GetTID (Command 0x02)
pldmttool raw -m 9 -d 0x80 0x00 0x02

# Response format:
# [0]: Completion Code (0x00 = success)
# [1]: TID value
```

PLDM Type 2: Monitoring & Control

This type handles sensors, effecters, and events.

Sensor Commands

Command	Code	Description
GetSensorReading	0x11	Read numeric sensor
GetSensorThresholds	0x12	Get threshold values
SetSensorThresholds	0x13	Set threshold values
RestoreSensorThresholds	0x14	Restore defaults
GetSensorHysteresis	0x15	Get hysteresis value
SetSensorHysteresis	0x16	Set hysteresis value
InitNumericSensor	0x17	Initialize sensor
GetStateSensorReading	0x21	Read state sensor

Effector Commands

Command	Code	Description
SetNumericEffectorValue	0x31	Set numeric effector
GetNumericEffectorValue	0x32	Get numeric effector value
SetStateEffectorStates	0x39	Set state effector
GetStateEffectorStates	0x3A	Get state effector

Event Commands

Command	Code	Description
PlatformEventMessage	0x0A	Send platform event
PollForPlatformEventMessage	0x0B	Poll for events
EventMessageSupported	0x0C	Query event support
EventMessageBufferSize	0x0D	Query buffer size

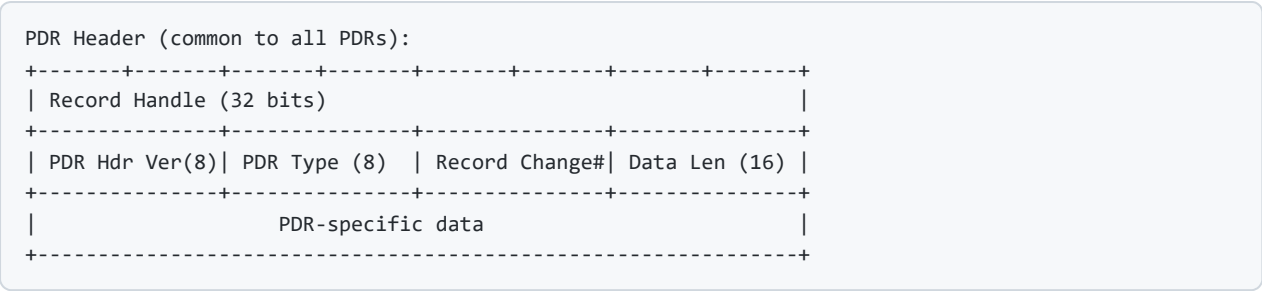
PLDM Type 5: Firmware Update

Command	Code	Description
QueryDeviceIdentifiers	0x01	Get device IDs
GetFirmwareParameters	0x02	Get FW info
RequestUpdate	0x10	Start update
PassComponentTable	0x13	Send component list
UpdateComponent	0x14	Update single component
RequestFirmwareData	0x15	Request FW chunk
TransferComplete	0x16	Transfer done
VerifyComplete	0x17	Verification done
ApplyComplete	0x18	Application done
ActivateFirmware	0x1A	Activate new FW
GetStatus	0x1B	Query update status
CancelUpdateComponent	0x1C	Cancel component update
CancelUpdate	0x1D	Cancel entire update

Platform Data Records (PDR)

PDRs describe the platform's sensors, effectors, and associations.

PDR Structure



PDR Types

Type	Name	Description
1	Terminus Locator	Maps TID to MCTP EID
2	Numeric Sensor	Numeric sensor definition
3	Numeric Sensor Initialization	Init parameters
4	State Sensor	State sensor definition
5	State Sensor Initialization	Init parameters
6	Sensor Auxiliary Names	Sensor name strings
9	OEM Device	Vendor-specific device
10	OEM	Vendor-specific PDR
11	Numeric Effector	Numeric effector definition
12	Numeric Effector Initialization	Init parameters
13	State Effector	State effector definition
14	State Effector Initialization	Init parameters
15	Effector Auxiliary Names	Effector name strings
20	Entity Association	Entity hierarchy
21	Entity Auxiliary Names	Entity name strings
22	OEM Entity ID	Vendor entity types

Numeric Sensor PDR Example

```
Numeric Sensor PDR (Type 2):
+-----+-----+-----+-----+-----+-----+-----+-----+
| Terminus Handle (16) | Sensor ID (16) |
+-----+-----+-----+-----+-----+-----+-----+-----+
| Entity Type(16)      | Entity Instance (16) |
+-----+-----+-----+-----+-----+-----+-----+-----+
| Container ID (16)    | Sensor Init | Auxiliary Names |
+-----+-----+-----+-----+-----+-----+-----+-----+
| Base Unit (8)        | Unit Modifier (8) | Rate Unit (8) |
+-----+-----+-----+-----+-----+-----+-----+-----+
| ... additional fields for ranges, thresholds, etc. |
+-----+-----+-----+-----+-----+-----+-----+-----+
```

State Sets

State sensors/effecters use predefined state sets:

State Set ID	Name	Example States
1	Health State	Normal, Warning, Critical
2	Availability	Available, Unavailable
3	Operational Status	Enabled, Disabled
4	Configuration State	Valid, Invalid
11	Link State	Up, Down
15	Power Device	On, Off, Powering On, Powering Off
192	Boot Progress	Primary Processor Init, Memory Init, PCI Init
196	System Power State	Off, On, Standby

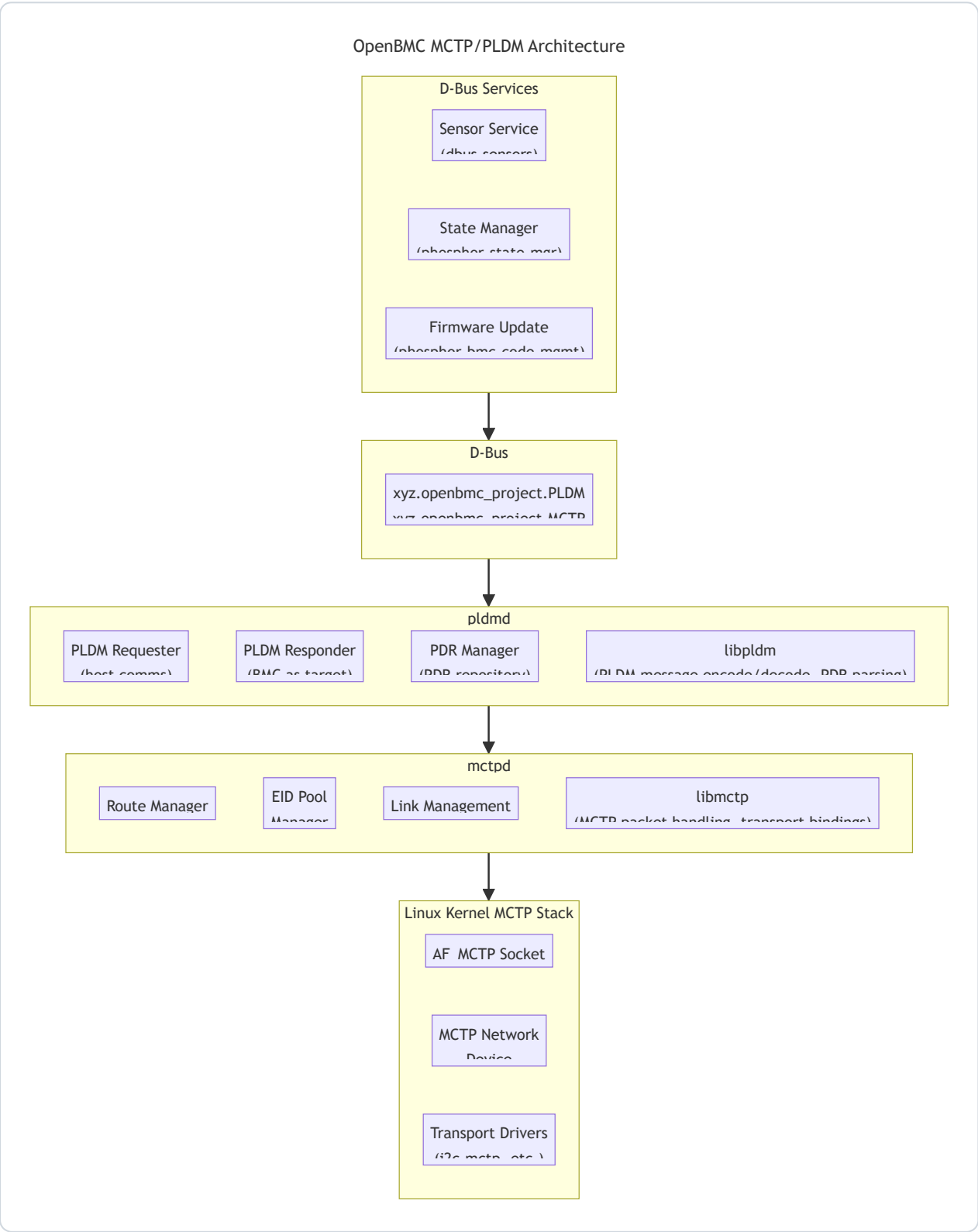
Querying PDRs

```
# Get all PDRs
pldmttool platform getpdr -t all -m 9

# Get specific PDR types
pldmttool platform getpdr -t numericSensorPDR -m 9
pldmttool platform getpdr -t stateSensorPDR -m 9
pldmttool platform getpdr -t numericEffectorPDR -m 9
pldmttool platform getpdr -t stateEffectorPDR -m 9
pldmttool platform getpdr -t terminusLocatorPDR -m 9
pldmttool platform getpdr -t entityAssociationPDR -m 9

# Get PDR by record handle
pldmttool platform getpdr -d 1 -m 9
pldmttool platform getpdr -d 2 -m 9
```

Architecture



► ASCII-art version (for comparison)

libmctp

Low-level MCTP packet handling:

```
// Key libmctp structures
struct mctp;           // MCTP context
struct mctp_binding;   // Transport binding
struct mctp_pktbuf;    // Packet buffer

// Transport bindings
struct mctp_binding_i2c; // I2C/SMBus
struct mctp_binding_astlpc; // ASPEED LPC
struct mctp_binding_serial; // Serial

// Key functions
mctp_init();           // Initialize context
mctp_set_rx_all();     // Set receive callback
mctp_message_tx();     // Transmit message
mctp_binding_set_tx_enabled(); // Enable transmission
```

libpldm

PLDM message encoding and decoding:

```
// Request encoding
encode_get_tid_req(instance_id, &request);
encode_get_sensor_reading_req(instance_id, sensor_id, &request);
encode_set_state_effector_states_req(instance_id, effector_id, ...);

// Response decoding
decode_get_tid_resp(response, &tid, &completion_code);
decode_get_sensor_reading_resp(response, &reading, &status);

// PDR handling
pldm_pdr_init();           // Initialize PDR repository
pldm_pdr_add();            // Add PDR record
pldm_pdr_find_record();    // Find PDR by handle
pldm_entity_association_tree_init(); // Entity tree
```

pldmd Configuration

```
// /usr/share/pldm/host_eid
9

// /usr/share/pldm/pdr/pdr.json
{
  "entries": [
    {
      "type": "terminusLocator",
      "data": {
        "terminusHandle": 1,
        "validity": "valid",
        "tid": 1,
        "containerID": 0,
        "terminusLocatorType": "mctp_eid",
        "terminusLocatorValue": 8
      }
    }
  ]
}
```

Setup & Configuration

Build-Time Configuration (Yocto)

```
# In your machine .conf or local.conf

# Include MCTP/PLDM components
IMAGE_INSTALL:append = " \
    mctp \
    mctpd \
    libpldm \
    pldm \
"

# Configure MCTP transport options
EXTRA_OEMESON:pn-mctpd = " \
    -Di2c=enabled \
    -Dastlpc=enabled \
    -Dserial=disabled \
"

# Configure PLDM options
EXTRA_OEMESON:pn-pldm = " \
    -Dtransport-implementation=af-mctp \
    -Ddoem-ibm=disabled \
    -Dsoftoff=enabled \
    -Dhost-eid=9 \
    -Dlibpldmresponder=enabled \
"
```

Meson Build Options (pldm)

Option	Default	Description
<code>transport-implementation</code>	af-mctp	Transport (af-mctp or mctp-demux)
<code>oem-ibm</code>	disabled	IBM OEM PLDM extensions
<code>softoff</code>	enabled	Host soft power off via PLDM
<code>host-eid</code>	9	Default host endpoint ID
<code>libpldmresponder</code>	enabled	BMC as PLDM responder
<code>system-specific-bios-json</code>	disabled	Custom BIOS JSON location

Meson Build Options (mctpd)

Option	Default	Description
<code>i2c</code>	enabled	I2C/SMBus transport
<code>astlpc</code>	enabled	ASPEED LPC transport
<code>serial</code>	disabled	Serial transport
<code>tests</code>	disabled	Build unit tests

Kernel Configuration

```
# Enable MCTP in kernel
CONFIG_MCTP=y
CONFIG_MCTP_FLOWS=y

# Transport drivers
CONFIG_I2C_MCTP=m
CONFIG_MCTP_SERIAL=m
```

Runtime Configuration

```
# Check MCTP daemon status
systemctl status mctpd

# Check PLDM daemon status
systemctl status pldmd

# View logs
journalctl -u mctpd -f
journalctl -u pldmd -f

# Enable debug logging
systemctl edit pldmd
# Add: Environment="PLDM_VERBOSITY=debug"
```

MCTP Configuration

Managing MCTP Networks

```
# List all MCTP interfaces
mctp link

# Example output:
# dev      net  eid  up
# mctpi2c1  1   8   yes
# mctplpc0  2   8   yes

# Set interface up
mctp link set mctpi2c1 up

# Set interface down
mctp link set mctpi2c1 down

# View interface details
ip link show type mctp
```

Endpoint Management

```
# List all endpoints
mctp endpoint

# Add local endpoint address
mctp addr add 8 dev mctpi2c1

# Remove endpoint address
mctp addr del 8 dev mctpi2c1

# View endpoint routing
mctp route
```

Static Endpoint Configuration

Create `/etc/mctp/static-endpoints.json` :

```
{
  "endpoints": [
    {
      "name": "host",
      "type": "static",
      "eid": 9,
      "transport": {
        "type": "i2c",
        "bus": 1,
        "address": "0x50"
      }
    },
    {
      "name": "gpu0",
      "type": "static",
      "eid": 10,
      "transport": {
        "type": "i2c",
        "bus": 3,
        "address": "0x51"
      }
    },
    {
      "name": "nvme0",
      "type": "static",
      "eid": 11,
      "transport": {
        "type": "i2c",
        "bus": 4,
        "address": "0x52"
      }
    }
  ]
}
```

D-Bus Interface for MCTP

```
# List MCTP service
busctl tree xyz.openbmc_project.MCTP

# Get endpoint properties
busctl introspect xyz.openbmc_project.MCTP \
  /xyz/openbmc_project/mctp/1/9

# Watch for new endpoints
busctl monitor xyz.openbmc_project.MCTP
```

PLDM Configuration

PDR Repository

```
# List all PDRs
pldmttool platform getpdr -t all

# Count PDRs
pldmttool platform getpdr -t all | grep -c "recordHandle"

# Export PDRs to file
pldmttool platform getpdr -t all > /tmp/pdr_dump.txt
```

Common PLDM Commands

```
# === Base Commands (Type 0) ===

# Get terminus ID
pldmttool base getTID -m 9

# Get PLDM version for type 0
pldmttool base getPLDMVersion -m 9 -t 0

# Get supported PLDM types
pldmttool base getPLDMTypes -m 9

# Get commands for a PLDM type
pldmttool base getPLDMCommands -m 9 -t 0
pldmttool base getPLDMCommands -m 9 -t 2

# === Platform Commands (Type 2) ===

# Get numeric sensor reading
pldmttool platform getSensorReading -m 9 -i 1

# Get state sensor reading
pldmttool platform getStateSensorReadings -m 9 -i 5

# Set state effector
pldmttool platform setStateEffectorStates -m 9 -i 1 -c 1 -d 1

# Get PDR by type
pldmttool platform getPDR -m 9 -t sensorPDR
pldmttool platform getPDR -m 9 -t effectorPDR

# === Firmware Update Commands (Type 5) ===

# Get firmware parameters
pldmttool fw_update GetFirmwareParameters -m 9

# Query device identifiers
pldmttool fw_update QueryDeviceIdentifiers -m 9
```

Raw PLDM Commands

```
# Send raw PLDM command
# Format: pldmtool raw -m <eid> -d <byte1> <byte2> ...
# Byte format: 0xRR 0xTT 0xCC [payload...]
#   RR = Request/Instance (0x80 for request, IID=0)
#   TT = PLDM Type
#   CC = Command Code

# GetTID (Type 0, Command 0x02)
pldmtool raw -m 9 -d 0x80 0x00 0x02

# GetPLDMTypes (Type 0, Command 0x04)
pldmtool raw -m 9 -d 0x80 0x00 0x04

# GetSensorReading (Type 2, Command 0x11) for sensor ID 1
pldmtool raw -m 9 -d 0x80 0x02 0x11 0x01 0x00 0x00
```

Device Integration Examples

NVMe over MCTP

NVMe-MI (Management Interface) uses MCTP for out-of-band management:

```
# Discover NVMe endpoints
mctp endpoint

# NVMe-MI over MCTP uses message type 0x04
# Query via pldmtool if device supports PLDM
pldmtool base getPLDMTypes -m 11

# For native NVMe-MI, use nvme-cli with MCTP transport
nvme list --mctp
```

GPU Management

```
# Configure GPU endpoint (I2C bus 3, address 0x51)
mctp addr add 10 dev mctpi2c3

# Verify endpoint responds
pldmtool base getTID -m 10

# Query GPU sensors
pldmtool platform getSensorReading -m 10 -i 1 # Temperature
pldmtool platform getSensorReading -m 10 -i 2 # Power

# Query GPU capabilities
pldmtool base getPLDMTypes -m 10
```

PSU Management

```
# Configure PSU endpoint
mctp addr add 11 dev mctpi2c4

# Read PSU telemetry via PLDM sensors
pldmtool platform getSensorReading -m 11 -i 1 # Input voltage
pldmtool platform getSensorReading -m 11 -i 2 # Output voltage
pldmtool platform getSensorReading -m 11 -i 3 # Output current
pldmtool platform getSensorReading -m 11 -i 4 # Temperature
```

Host CPU/BIOS Communication

```
# Host typically at EID 9
# PLDM for BIOS attributes (Type 3)
pldmtool bios GetBIOSTable -m 9 -t StringTable
pldmtool bios GetBIOSTable -m 9 -t AttributeTable
pldmtool bios GetBIOSTable -m 9 -t AttributeValueTable

# Get specific BIOS attribute
pldmtool bios GetBIOSAttributeCurrentValueByHandle -m 9 -a "boot_order"
```

PLDM Firmware Update

Firmware Update Flow



Firmware Update Commands

```
# Query device capabilities
pldmtool fw_update QueryDeviceIdentifiers -m 9
pldmtool fw_update GetFirmwareParameters -m 9

# Start update process
pldmtool fw_update RequestUpdate -m 9 \
    -c 1000 \      # max_transfer_size
    -n 1 \         # number of components
    -m 0 \         # max_outstanding_requests
    -s 1           # package_data_length

# The actual update is typically handled by pldmd
# or a dedicated update agent
```

Event Handling

PLDM Event Types

Event Class	ID	Description
Sensor Event	0x00	Sensor threshold/state changed
Effector Event	0x01	Effector state changed
Redfish Task Executed	0x02	Redfish task complete
Redfish Message	0x03	Redfish log message
PDR Repository Changed	0x04	PDR update
Message Poll	0x05	Async message waiting
Heartbeat	0x06	Keepalive

Sensor Event Format

```
Sensor Event Data:
+-----+-----+-----+-----+-----+-----+
| Sensor ID (16)      | Sensor Event Class (8) |
+-----+-----+-----+-----+-----+
| Event Data (variable based on class) |
+-----+-----+-----+-----+-----+-----+-----+
```

Event Monitoring

```
# Subscribe to PLDM events via D-Bus
busctl monitor xyz.openbmc_project.PLDM

# Poll for platform events
pldmtool platform pollForPlatformEventMessage -m 9

# Check event message buffer size
pldmtool platform getEventMessageBufferSize -m 9
```

Host-BMC PLDM Coordination

On platforms where the host runs its own PLDM stack, `pldmd` coordinates state and inventory with it:

- **PDR exchange** — `HostPDRHandler` issues `GetPDR` when the host powers on, folds the host's entity-association PDRs into the BMC entity tree, and creates D-Bus inventory objects (for example `xyz.openbmc_project.Inventory.Item.CpuCore`) through `CustomDBus`.
- **Event forwarding** — `DBusToPLDMEvent` converts D-Bus state-sensor `PropertiesChanged` signals into PLDM `PlatformEventMessage`s sent to the host.
- **Graceful shutdown** — `pldmSoftPowerOff.service` (`SoftPowerOff`) reads `softoff_config.json` (TID / entity type / state-set ID), sends `SetStateEffectorStates`, then waits for the host's PLDM state-sensor event with a timeout before a hard power-off.

(Verify class/service names against your `pldm` version.)

D-Bus Interfaces

MCTP D-Bus Objects

```
# Service: xyz.openbmc_project.MCTP
# Path: /xyz/openbmc_project/mctp/<network>/<eid>

busctl tree xyz.openbmc_project.MCTP

# Example output:
# └─/xyz/openbmc_project
#   └─/xyz/openbmc_project/mctp
#     └─/xyz/openbmc_project/mctp/1
#         └─/xyz/openbmc_project/mctp/1/8
#         └─/xyz/openbmc_project/mctp/1/9

# Inspect endpoint
busctl introspect xyz.openbmc_project.MCTP \
    /xyz/openbmc_project/mctp/1/9

# Properties:
# .Address          - Physical address
# .NetworkId        - MCTP network ID
# .EID              - Endpoint ID
# .SupportedMessageTypes - Supported MCTP message types
```

PLDM D-Bus Objects

```
# Service: xyz.openbmc_project.PLDM
# Paths:
#   /xyz/openbmc_project/pldm/terminus/<tid>
#   /xyz/openbmc_project/pldm/fru/<record_set_id>

busctl tree xyz.openbmc_project.PLDM

# Get terminus properties
busctl introspect xyz.openbmc_project.PLDM \
  /xyz/openbmc_project/pldm/terminus/1

# Get sensor value (exposed via standard sensor interface)
busctl get-property xyz.openbmc_project.PLDM \
  /xyz/openbmc_project/sensors/temperature/CPU0_Temp \
  xyz.openbmc_project.Sensor.Value Value
```

Troubleshooting

MCTP Connectivity Issues

```
# 1. Check kernel MCTP support
lsmod | grep mctp
modprobe mctp

# 2. Verify I2C bus
i2cdetect -l
i2cdetect -y 1   # Check bus 1

# 3. Check MCTP link status
mctp link
mctp addr

# 4. View mctpd logs
journalctl -u mctpd -f

# 5. Enable debug logging
systemctl edit mctpd
# Add: Environment="MCTP_LOG_LEVEL=debug"
systemctl restart mctpd

# 6. Check MCTP network device
ip link show type mctp
ip -d link show mctpi2c1
```

PLDM Communication Issues

```
# 1. Verify endpoint responds
pldmtool base getTID -m 9

# 2. Check PLDM daemon status
systemctl status pldmd

# 3. View PLDM logs
journalctl -u pldmd -f

# 4. Test raw PLDM command
pldmtool raw -m 9 -d 0x80 0x00 0x04 # GetPLDMTypes

# 5. Increase timeout
pldmtool --timeout 30000 base getTID -m 9
```

No Endpoints Discovered

```
# 1. Check static configuration
cat /etc/mctp/static-endpoints.json

# 2. Manually add endpoint
mctp addr add 9 dev mctpi2c1

# 3. Verify I2C device responds
i2cdetect -y 1
i2ctransfer -y 1 w1@0x50 0x00

# 4. Check discovery
mctp ctrl discover mctpi2c1
```

Command Timeouts

```
# 1. Check endpoint health
mctp stats

# 2. Increase command timeout
pldmtool --timeout 60000 <command>

# 3. Check for I2C bus contention
dmesg | grep -i i2c

# 4. Verify clock speed
# Some devices need slower I2C clock
```

PDR Issues

```
# 1. Check PDR repository
pldmtool platform getpdr -t all

# 2. Verify terminus locator
pldmtool platform getpdr -t terminusLocatorPDR

# 3. Check PDR JSON configuration
cat /usr/share/pldm/pdr/*.json

# 4. Rebuild PDR repository
systemctl restart pldm
```

Security Considerations

Authentication with SPDM

MCTP/PLDM integrates with SPDM for device authentication:

```
# SPDM runs over MCTP (message type 0x05)
# See the SPDM Guide for details

# Verify device certificate
# Challenge device
# Establish secure session
```

Access Control

```
<!-- /etc/dbus-1/system.d/pldm.conf -->
<!DOCTYPE busconfig PUBLIC
"-//freedesktop//DTD D-BUS Bus Configuration 1.0//EN"
"http://www.freedesktop.org/standards/dbus/1.0/busconfig.dtd">
<busconfig>
  <policy user="root">
    <allow own="xyz.openbmc_project.PLDM"/>
    <allow send_destination="xyz.openbmc_project.PLDM"/>
  </policy>
  <policy context="default">
    <allow send_destination="xyz.openbmc_project.PLDM"
      send_interface="org.freedesktop.DBus.Properties"
      send_member="Get"/>
    <allow send_destination="xyz.openbmc_project.PLDM"
      send_interface="org.freedesktop.DBus.Properties"
      send_member="GetAll"/>
  </policy>
</busconfig>
```

Network Isolation

```
# MCTP networks are isolated by network ID
# EIDs are only unique within a network

# Use separate networks for different security domains
# Network 1: Host communication
# Network 2: Peripheral devices
```

Enabling/Disabling

Build-Time Disable

```
# Remove MCTP/PLDM packages
IMAGE_INSTALL:remove = "mctp mctpd pldm libpldm"
```

Runtime Disable

```
# Stop PLDM daemon
systemctl stop pldmd
systemctl disable pldmd

# Stop MCTP daemon
systemctl stop mctpd
systemctl disable mctpd

# Verify stopped
systemctl status pldmd mctpd
```

Examples

Working examples are available in the [examples/mctp-pldm](#) directory:

- [mctp_health_check.sh](#) - Verify MCTP/PLDM stack connectivity
- [pldm_discovery.sh](#) - Discover and characterize PLDM endpoints
- [pldm_sensors.sh](#) - Read numeric and state sensors
- [pldm_pdr_dump.sh](#) - Dump Platform Descriptor Records
- [pldm_bios.sh](#) - Read BIOS tables and attributes (Type 3)
- [pldm_fwupdate.sh](#) - Query firmware update capabilities (Type 5)
- [config/static-endpoints.json](#) - Static MCTP endpoint configuration
- [config/pdr.json](#) - PDR repository definition
- [config/pldm-dbus.conf](#) - D-Bus access policy for PLDM

The [pldm](#) repo carries vendor OEM extensions under [oem/](#), conditionally compiled per platform: **IBM** (DMA-based file I/O for PELs / LIDs / dumps / certificates, in-band code update, PCIe-slot VPD, host lamp test), **Ampere** (CPER and sensor-event decoding to SEL/Redfish), **Meta** (system / BIOS-SEL / memory / POST-error decoding,

power-control file I/O), and **NVIDIA**. When porting to one of these platforms, enable the matching `oem` build option. (Verify against your `pldm` version.)

References

DMTF Specifications

- [DSP0236 - MCTP Base Specification](#) - Core MCTP protocol
- [DSP0237 - MCTP SMBus/I2C Transport](#) - I2C binding
- [DSP0238 - MCTP PCIe VDM Transport](#) - PCIe binding
- [DSP0239 - MCTP Serial Transport](#) - Serial binding
- [DSP0240 - PLDM Base Specification](#) - Core PLDM protocol
- [DSP0248 - PLDM for Platform Monitoring and Control](#) - Type 2
- [DSP0267 - PLDM for Firmware Update](#) - Type 5

OpenBMC Repositories

- [libpldm \(GitHub\)](#) - PLDM library
- [pldm \(GitHub\)](#) - PLDM daemon
- [libmctp \(GitHub\)](#) - MCTP library
- [mctp \(GitHub\)](#) - MCTP userspace tools

Linux Kernel

- [Linux MCTP Documentation](#) - Kernel MCTP stack
-

Deep Dive

Advanced implementation details for MCTP/PLDM developers.

MCTP Message Fragmentation

Large PLDM messages are fragmented across multiple MCTP packets:

MCTP Message Fragmentation

PLDM Message (e.g., 500 bytes):

PLDM Request/Response Data
(500 bytes)



MCTP Fragmentation (MTU = 64 bytes per binding):

Packet 1 (SOM=1, EOM=0):

Header	Data[0:63]
SOM=1	
EOM=0	
Seq=0	

Packet 2 (SOM=0, EOM=0):

Header	Data[64:127]
SOM=0	
EOM=0	
Seq=1	

...

Packet N (SOM=0, EOM=1):

Header	Data[remaining]
SOM=0	
EOM=1	← End of Message
Seq=7	

Header Flags:

- SOM (Start of Message) - First fragment
- EOM (End of Message) - Last fragment
- Seq (Sequence) - 2-bit, wraps 0-3
- Tag - Correlates request/response

PLDM Message Type Encoding

PLDM Message Structure

Request Message Format:

Byte 0	Byte 1	Byte 2	Bytes 3-N
Instance ID/Rq	Type (0-6 DMTF)	Command	Request Data

Byte 0 breakdown:

D/Rq	Rsvd	Instance ID
1b	1b	5 bits

D=0: Request, D=1: Response
Rq=1: Request message, Rq=0: Response message

Response Message Format:

Instance ID/D=1	Type	Command	Completion Code	Response Data
-----------------	------	---------	-----------------	---------------

PLDM Types:

Type	Description
0	PLDM Messaging Control and Discovery
1	SMBIOS (not commonly used)
2	Platform Monitoring and Control (sensors, PDRs)
3	BIOS Control and Configuration
4	FRU Data
5	Firmware Update
6	Redfish Device Enablement

PDR (Platform Descriptor Record) Database

PDR Repository Structure

PDR Repository (stored in pldmd):

Record	Type	Description
1	Terminus Locator (11)	BMC terminus info
2	Numeric Sensor (2)	CPU temperature
3	Numeric Sensor (2)	DIMM temperature
4	Numeric Sensor (2)	Fan speed
5	Numeric Effector (9)	Fan PWM control
6	State Sensor (4)	Presence sensor
7	FRU Record Set (20)	System FRU data
...	...	...

PDR Record Common Header:

Record Hdl	PDR Header	Record Type	Data Length	Record Data
(4 bytes)	Version	(1 byte)	(2 bytes)	(variable)

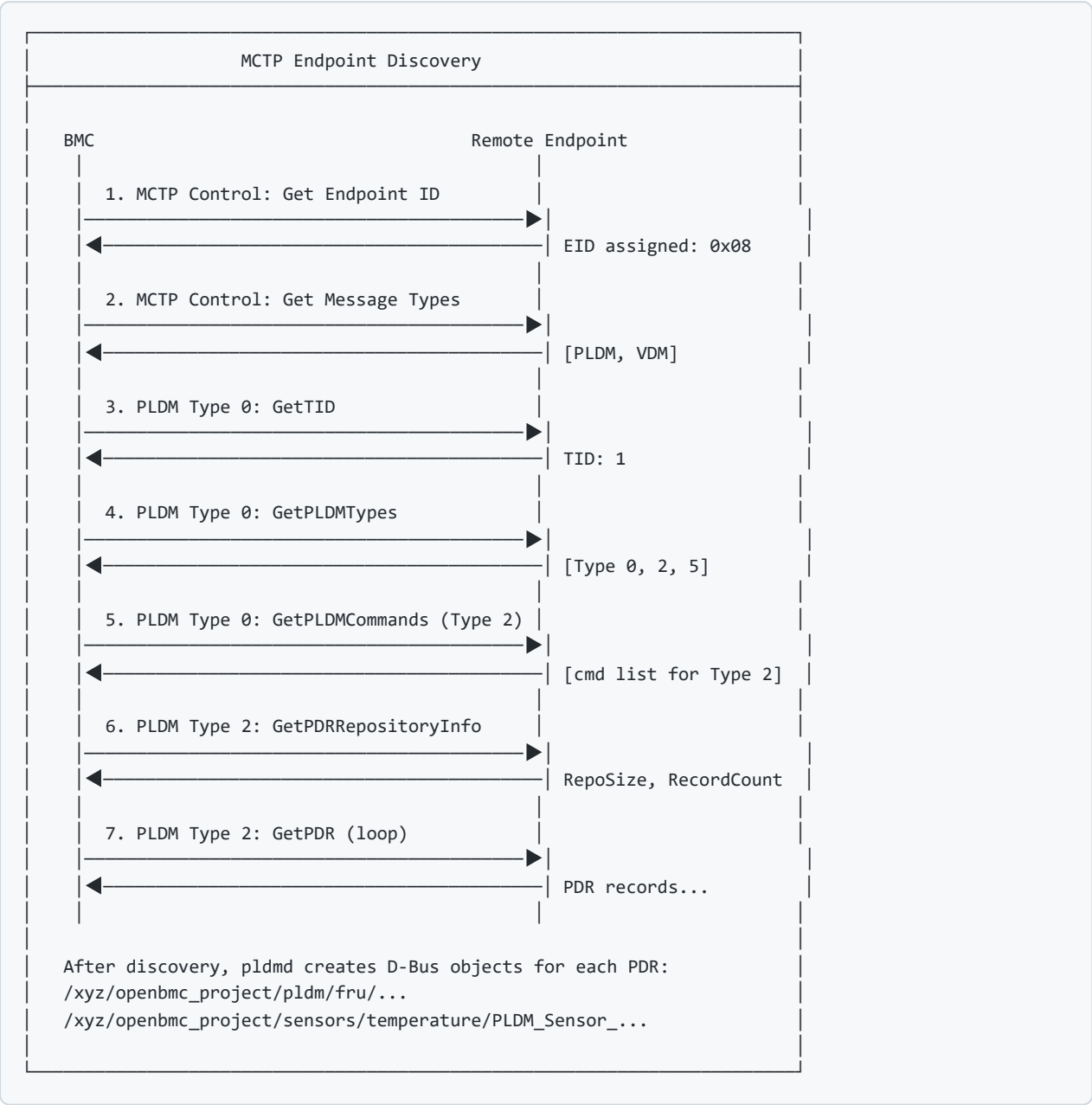
Numeric Sensor PDR (Type 2) key fields:

- Sensor ID (2 bytes)
- Entity Type (CPU, Memory, Fan, PSU, etc.)
- Entity Instance
- Sensor Init (thresholds, hysteresis)
- Unit (Degrees C, RPM, Volts, Amps, Watts)
- Resolution, Offset, Accuracy
- Threshold values (warning, critical, fatal)

JSON PDR Configuration (pldm/pdr_jsons/):

```
{
  "pdrType": 2,
  "sensorID": 1,
  "entityType": 66, // Processor
  "baseUnit": 2,    // Degrees C
  "warningHigh": 85.0,
  "criticalHigh": 95.0
}
```

Endpoint Discovery Flow



Source Code Reference

Key implementation files:

Repository	File	Description
libpldm	<code>src/base.c</code>	Base message encode/decode
libpldm	<code>src/platform.c</code>	Type 2 (Platform) messages
libpldm	<code>src/fru.c</code>	Type 4 (FRU) messages
libpldm	<code>src/fw_update.c</code>	Type 5 (Firmware Update)
pldm	<code>pldmd/pldmd.cpp</code>	Main PLDM daemon
pldm	<code>libpldmresponder/</code>	Response handlers
pldm	<code>requester/</code>	Request handlers
mctp	<code>src/mctp.c</code>	MCTP transport layer

Tested on: OpenBMC master, requires hardware with MCTP-capable endpoints. QEMU testing possible with mctp-serial loopback.

SPDM Guide

Configure SPDM device attestation and authentication on OpenBMC.

Table of Contents

1. [Overview](#)
 1. [Why SPDM?](#)
 2. [SPDM Versions](#)
2. [Architecture](#)
 1. [Component Roles](#)
3. [SPDM Protocol Deep Dive](#)
 1. [Message Format](#)
 2. [Request/Response Codes](#)
 1. [Base Commands \(0x01-0x0F\)](#)
 2. [Measurement Commands \(0x60-0x6F\)](#)
 3. [Session Commands \(0xE0-0xEF\)](#)
 4. [Error Response \(0x7F\)](#)
4. [Protocol Flow](#)
 1. [Discovery and Capability Negotiation](#)
 2. [Capability Flags](#)
5. [Certificate Chain Structure](#)
 1. [Chain Format](#)
 2. [Certificate Slot Model](#)
 3. [Certificate Requirements](#)
6. [Measurements](#)
 1. [Measurement Concept](#)
 2. [Measurement Block Format](#)
 3. [Measurement Types \(DMTF\)](#)
 4. [Measurement Operations](#)
 5. [Measurement Response Attributes](#)
7. [Challenge-Response Authentication](#)
 1. [Challenge Flow](#)
 2. [Transcript Hash](#)
8. [Secure Session Establishment](#)
 1. [Key Exchange Flow](#)
 2. [Session Key Derivation](#)
 3. [Encrypted Message Format](#)
9. [Cryptographic Algorithms](#)
 1. [Supported Algorithms](#)
 1. [Asymmetric Signature](#)

- 2. [Hash Algorithms](#)
 - 3. [DHE \(Diffie-Hellman Ephemeral\)](#)
 - 4. [AEAD \(Session Encryption\)](#)
- 10. [Setup & Configuration](#)
 - 1. [Build-Time Configuration \(Yocto\)](#)
 - 2. [libspdm Build Options](#)
 - 3. [Runtime Configuration](#)
 - 4. [Algorithm Configuration](#)
- 11. [Certificate Management](#)
 - 1. [Certificate Chain](#)
 - 2. [Provisioning Trust Anchors](#)
 - 3. [BMC Certificate \(for Mutual Auth\)](#)
 - 4. [Certificate Verification](#)
- 12. [SPDM Operations](#)
 - 1. [Device Discovery](#)
 - 2. [Get Device Certificate](#)
 - 3. [Challenge Authentication](#)
 - 4. [Get Measurements](#)
- 13. [Redfish ComponentIntegrity](#)
 - 1. [Enable in bmcweb](#)
 - 2. [Query Component Integrity](#)
 - 3. [ComponentIntegrity Response](#)
 - 4. [Trigger New Attestation](#)
- 14. [Secure Session Operations](#)
 - 1. [Key Exchange](#)
 - 2. [Session Management](#)
 - 3. [Session Types](#)
- 15. [Measurement Policy](#)
 - 1. [Expected Measurements Configuration](#)
 - 2. [Policy Actions](#)
- 16. [D-Bus Integration](#)
 - 1. [SPDM D-Bus Interface](#)
 - 2. [Attestation Events](#)
- 17. [Troubleshooting](#)
 - 1. [Discovery Failures](#)
 - 2. [Certificate Verification Failed](#)
 - 3. [Measurement Mismatch](#)
 - 4. [Session Failures](#)
 - 5. [Common Error Codes](#)
- 18. [Security Best Practices](#)
 - 1. [Certificate Management](#)

- 2. [Measurement Verification](#)
 - 3. [Access Control](#)
 - 4. [Production Recommendations](#)
19. [Enabling/Disabling SPDm](#)
- 1. [Build-Time Disable](#)
 - 2. [Runtime Disable](#)
20. [Examples](#)
21. [References](#)
-

Overview

SPDM (Security Protocol and Data Model) is a DMTF standard (DSP0274) that provides device identity verification and firmware attestation through certificate-based authentication and cryptographic measurements. It enables the BMC to verify that platform components (GPUs, NICs, storage devices, power supplies) are authentic and running trusted firmware.

Why SPDm?

Modern data centers face supply chain security threats including:

- Counterfeit components
- Firmware tampering
- Hardware implants
- Unauthorized firmware modifications

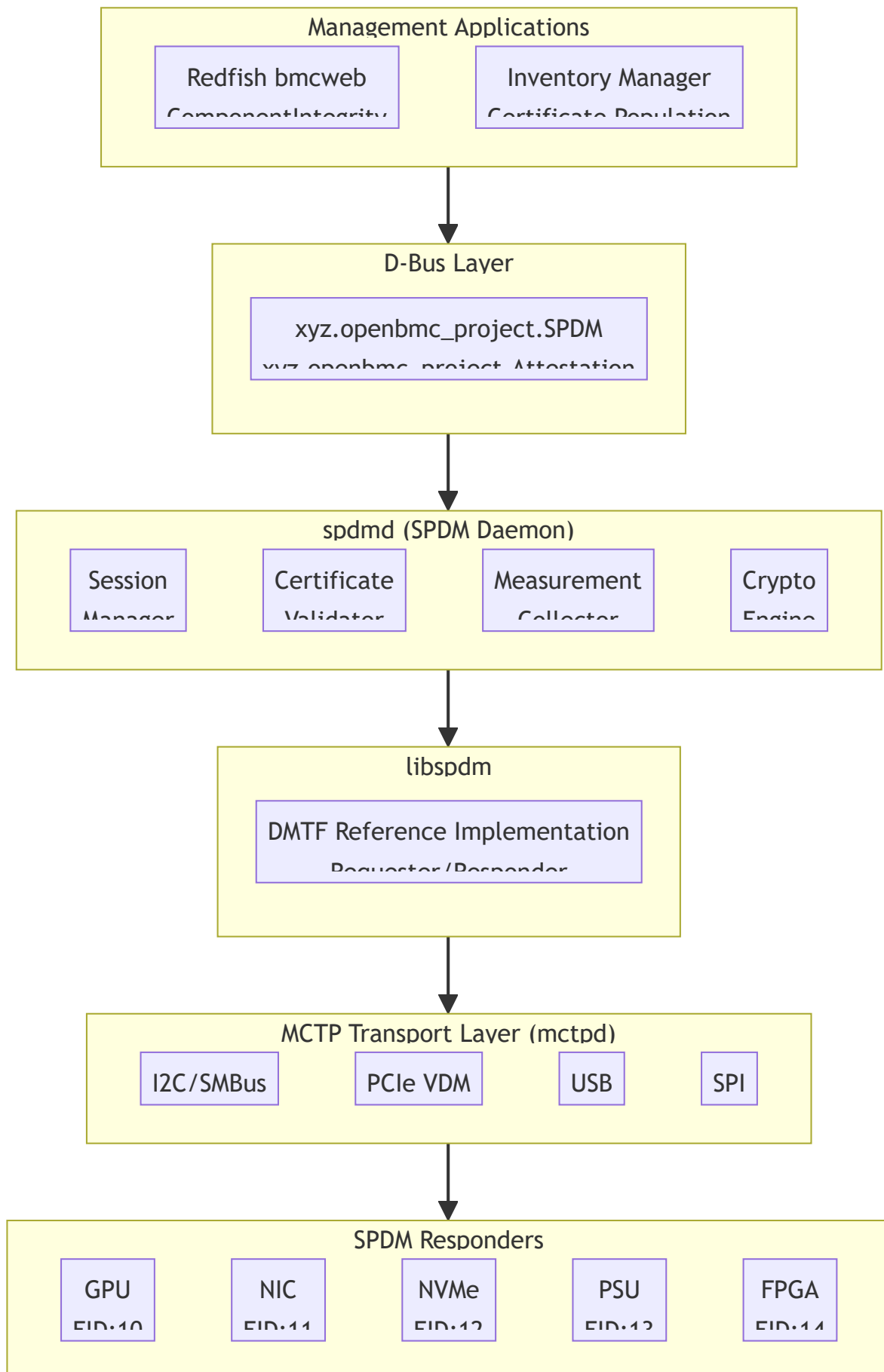
SPDM addresses these by providing:

- **Device Identity:** X.509 certificate chains prove device authenticity
- **Firmware Attestation:** Cryptographic measurements verify firmware integrity
- **Secure Communication:** Encrypted sessions protect management traffic
- **Mutual Authentication:** Both BMC and device can verify each other

SPDM Versions

Version	DSP0274	Key Features
1.0	2019	Base attestation, challenge-response
1.1	2020	Mutual authentication, key exchange
1.2	2022	Chunking, multi-key, extended algorithms
1.3	2024	Event notifications, measurement extensions

SPDM Architecture in OpenBMC



► ASCII-art version (for comparison)

Component Roles

Component	Role
Requester	Initiates SPDM exchanges (BMC is typically the requester)
Responder	Responds to SPDM requests (devices being attested)
spdm	OpenBMC daemon managing SPDM sessions and attestation
libspdm	DMTF reference implementation library
mctpd	MCTP transport daemon providing message delivery

SPDM Protocol Deep Dive

Message Format

Every SPDM message follows a common header structure:

+-----+-----+-----+-----+-----+-----+
Byte 0 Byte 1 Byte 2 Byte 3
+-----+-----+-----+-----+-----+-----+
SPDMVersion RequestResCode Param1 Param2
+-----+-----+-----+-----+-----+-----+
Payload (variable length)
+-----+-----+-----+-----+-----+-----+

Field	Size	Description
<code>SPDMVersion</code>	1 byte	Major.Minor version (e.g., 0x12 = v1.2)
<code>RequestResponseCode</code>	1 byte	Message type identifier
<code>Param1</code>	1 byte	Message-specific parameter
<code>Param2</code>	1 byte	Message-specific parameter
<code>Payload</code>	Variable	Message-specific data

Request/Response Codes

Base Commands (0x01-0x0F)

Code	Request	Response	Description
0x04	GET_VERSION	VERSION (0x04)	Query supported SPDm versions
0x61	GET_CAPABILITIES	CAPABILITIES (0x61)	Query device capabilities
0x63	NEGOTIATE_ALGORITHMS	ALGORITHMS (0x63)	Negotiate crypto algorithms
0x01	GET_DIGESTS	DIGESTS (0x01)	Get certificate chain digests
0x02	GET_CERTIFICATE	CERTIFICATE (0x02)	Retrieve certificate chain
0x03	CHALLENGE	CHALLENGE_AUTH (0x03)	Challenge-response auth

Measurement Commands (0x60-0x6F)

Code	Request	Response	Description
0x60	GET_MEASUREMENTS	MEASUREMENTS (0x60)	Retrieve firmware measurements

Session Commands (0xE0-0xEF)

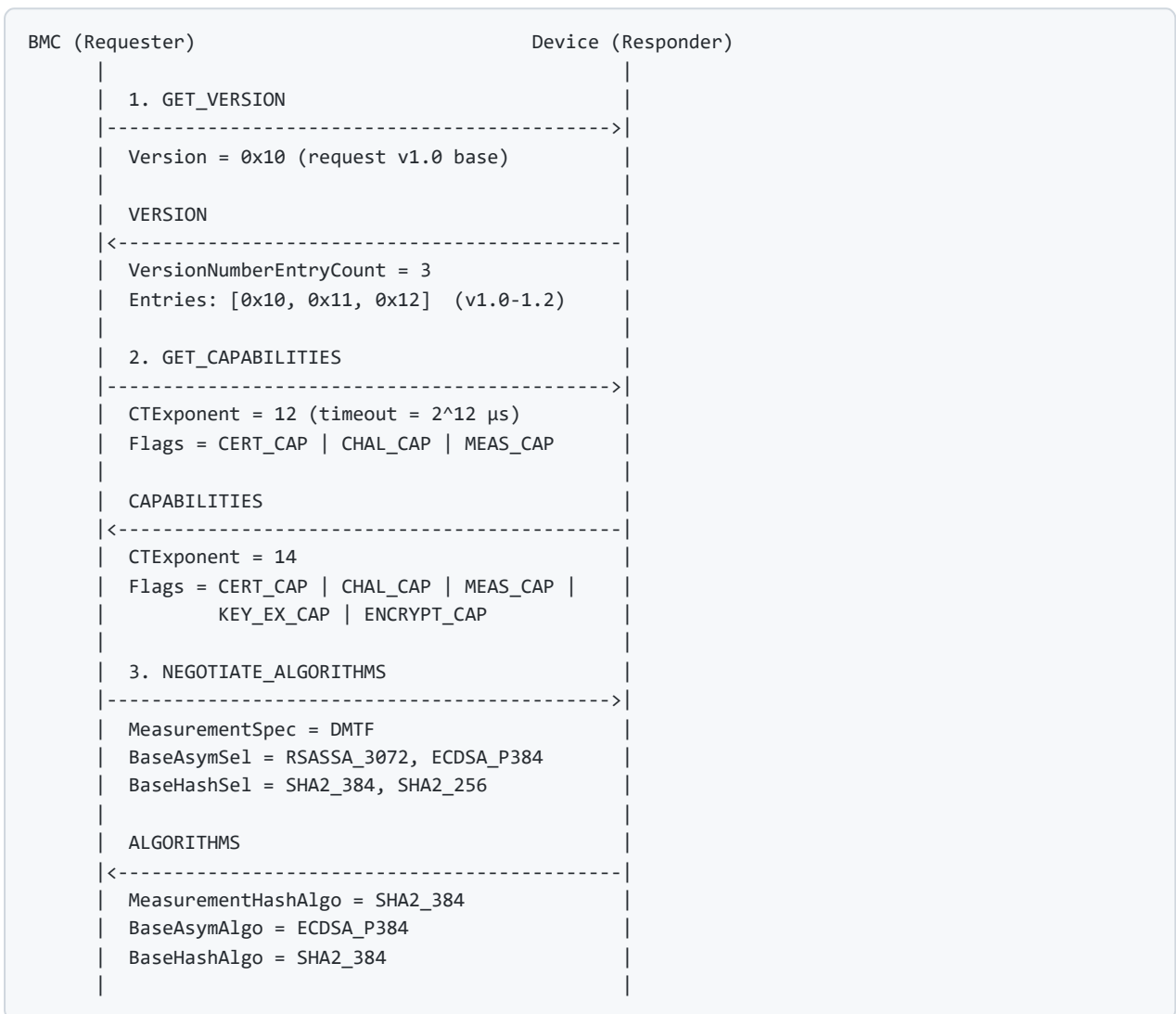
Code	Request	Response	Description
0xE4	KEY_EXCHANGE	KEY_EXCHANGE_RSP (0xE4)	Establish session key
0xE5	FINISH	FINISH_RSP (0xE5)	Complete session establishment
0xE6	PSK_EXCHANGE	PSK_EXCHANGE_RSP (0xE6)	Pre-shared key exchange
0xE7	PSK_FINISH	PSK_FINISH_RSP (0xE7)	Complete PSK session
0xEC	END_SESSION	END_SESSION_ACK (0xEC)	Terminate session
0xEE	HEARTBEAT	HEARTBEAT_ACK (0xEE)	Keep session alive
0xEF	KEY_UPDATE	KEY_UPDATE_ACK (0xEF)	Rotate session keys

Error Response (0x7F)

Error Code	Name	Description
0x01	InvalidRequest	Request format invalid
0x02	Busy	Device temporarily unavailable
0x03	UnexpectedRequest	Request out of sequence
0x04	Unspecified	Generic error
0x05	DecryptError	Session decryption failed
0x06	UnsupportedRequest	Request not supported
0x07	RequestInFlight	Previous request pending
0x41	InvalidSession	Session ID invalid
0x42	SessionLimitExceeded	Too many active sessions

Protocol Flow

Discovery and Capability Negotiation



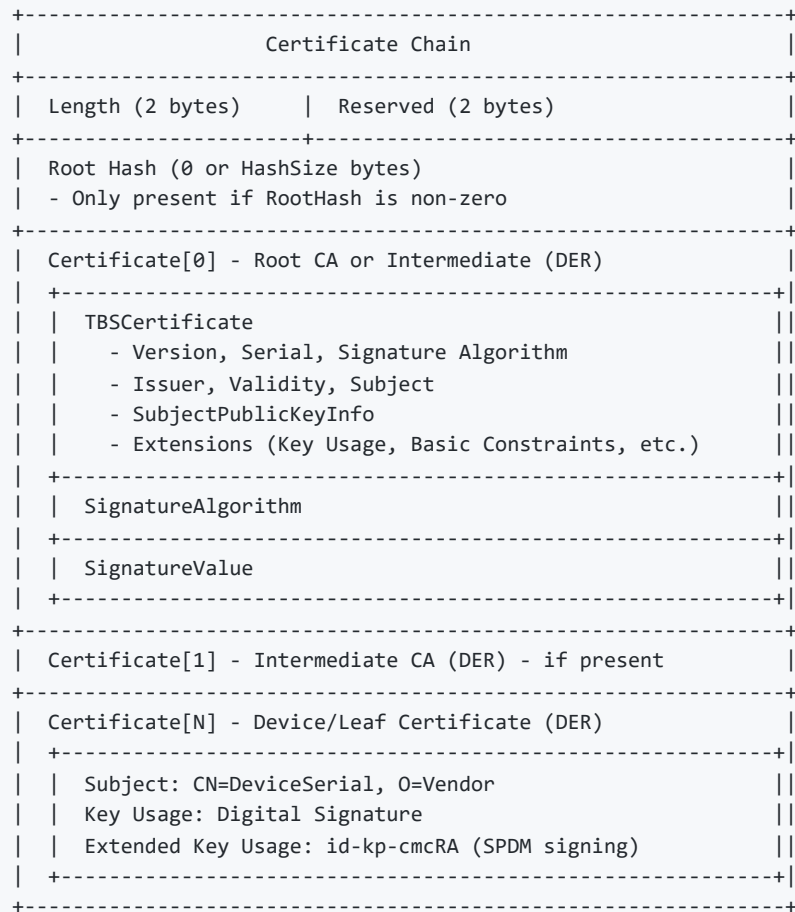
Capability Flags

Flag	Bit	Description
CACHE_CAP	0	Supports caching of negotiated state
CERT_CAP	1	Has certificate chain(s)
CHAL_CAP	2	Supports challenge authentication
MEAS_CAP	3-4	Measurement capability (0=none, 1=no sig, 2=with sig)
MEAS_FRESH_CAP	5	Generates fresh measurements on request
ENCRYPT_CAP	6	Supports encrypted messages
MAC_CAP	7	Supports message authentication codes
MUT_AUTH_CAP	8	Supports mutual authentication
KEY_EX_CAP	9	Supports key exchange
PSK_CAP	10-11	Pre-shared key capability
ENCAP_CAP	12	Supports encapsulated requests
HBEAT_CAP	13	Supports heartbeat
KEY_UPD_CAP	14	Supports key update
HANDSHAKE_ITC_CAP	15	Handshake in-the-clear support
PUB_KEY_ID_CAP	16	Public key ID instead of certificate
CHUNK_CAP	17	Supports message chunking
ALIAS_CERT_CAP	18	Has alias certificate model

Certificate Chain Structure

Chain Format

SPDM uses DER-encoded X.509 certificate chains:



Certificate Slot Model

SPDM supports up to 8 certificate slots (0-7):

Slot	Typical Use
0	Device identity certificate (required)
1-7	Alternative certificates (optional)

```

# Get certificate digests for all slots
spdmtool getcert -m 10 --digests

# Get certificate from specific slot
spdmtool getcert -m 10 -s 0 # Slot 0
spdmtool getcert -m 10 -s 1 # Slot 1

```

Certificate Requirements

For SPDM device certificates:

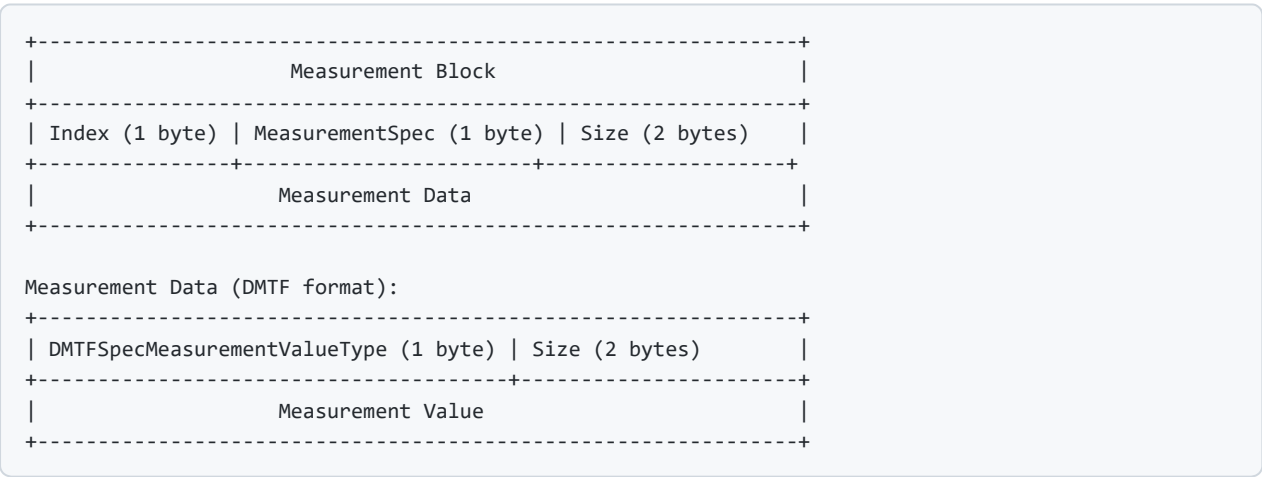
Requirement	Description
Key Usage	digitalSignature must be set
Key Size	RSA ≥ 2048, ECDSA P-256/P-384/P-521
Hash Algorithm	SHA-256 or stronger
Chain Depth	Typically 2-4 certificates
OID Extensions	May include SPDMM-specific OIDs

Measurements

Measurement Concept

Measurements are cryptographic digests of device firmware, configuration, and state. They allow the requester to verify device integrity.

Measurement Block Format



Measurement Types (DMTF)

Type	Value	Description
<code>ImmutableROM</code>	0x00	Boot ROM, immutable code
<code>MutableFirmware</code>	0x01	Updateable firmware
<code>HardwareConfig</code>	0x02	Hardware configuration
<code>FirmwareConfig</code>	0x03	Firmware configuration
<code>MeasurementManifest</code>	0x04	Measurement manifest
<code>DebugMode</code>	0x05	Debug/manufacturing mode indication
<code>MutableFirmwareVersion</code>	0x06	Firmware version string
<code>MutableFirmwareSecurityVersion</code>	0x07	Security version number
<code>HashExtendMeasurement</code>	0x08-0x7F	Extended hash measurements
<code>InformationalMeasurement</code>	0x80-0xEF	Informational data
<code>Vendor</code>	0xF0-0xFF	Vendor-defined

Measurement Operations

```
# Get all measurements (unsigned)
spdmtool measurements -m 10

# Get measurements with signature
spdmtool measurements -m 10 --signed

# Get specific measurement index
spdmtool measurements -m 10 -i 1

# Get raw measurement data
spdmtool measurements -m 10 -i 1 --raw -o measurement_1.bin

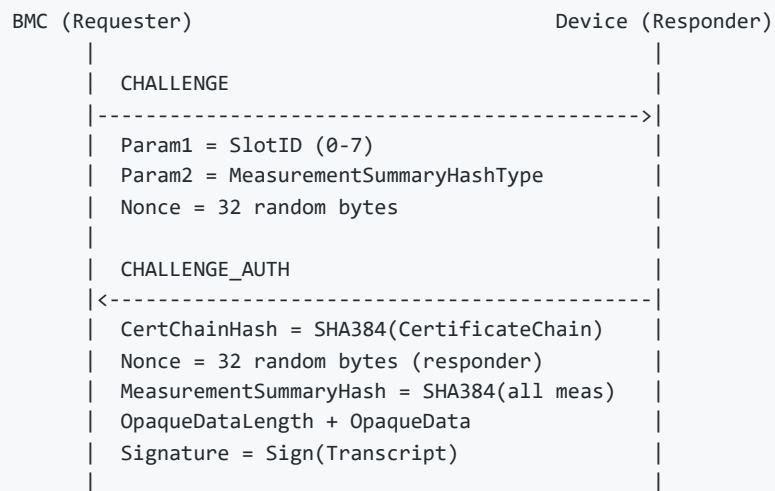
# Example output:
# Index 1: Type=MutableFirmware, Size=48, Hash=SHA384:abc123...
# Index 2: Type=FirmwareConfig, Size=48, Hash=SHA384:def456...
# Index 3: Type=HardwareConfig, Size=48, Hash=SHA384:789abc...
```

Measurement Response Attributes

Attribute	Description
<code>NUMBER_OF_BLOCKS</code>	Number of measurement blocks
<code>MEASUREMENT_RECORD_LENGTH</code>	Total length of measurement data
<code>NONCE</code>	32-byte random value (when signed)
<code>OPAQUE_DATA</code>	Optional vendor-specific data
<code>SIGNATURE</code>	Response signature (when signed)

Challenge-Response Authentication

Challenge Flow



Transcript Hash

The signature covers a transcript of the entire exchange:

```
TranscriptHash = Hash(
    GET_VERSION || VERSION ||
    GET_CAPABILITIES || CAPABILITIES ||
    NEGOTIATE_ALGORITHMS || ALGORITHMS ||
    GET_DIGESTS || DIGESTS ||
    GET_CERTIFICATE[0..N] || CERTIFICATE[0..N] ||
    CHALLENGE || CHALLENGE_AUTH (without signature)
)

Signature = Sign(PrivateKey, TranscriptHash)
```

Secure Session Establishment

Key Exchange Flow



Session Key Derivation

Keys are derived using HKDF:

```
SharedSecret = ECDH(RequesterPrivKey, ResponderPubKey)

HandshakeSecret = HKDF-Extract(Salt=0, IKM=SharedSecret)

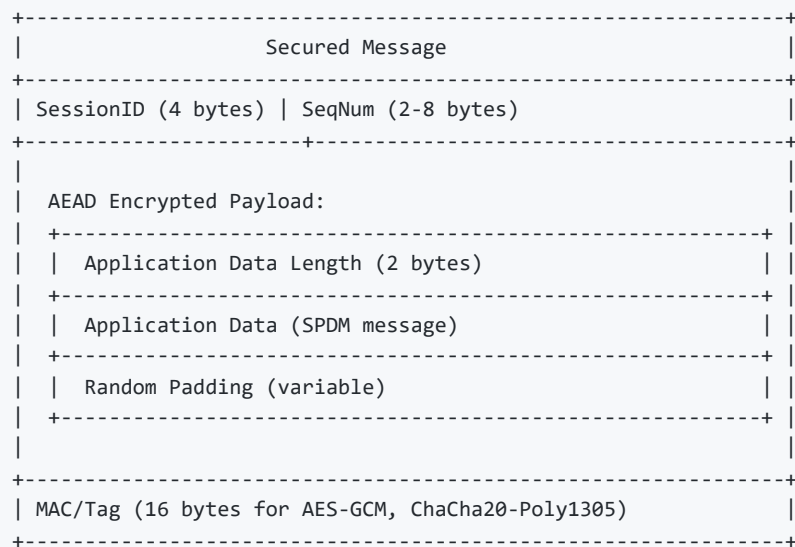
RequesterHandshakeKey = HKDF-Expand(HandshakeSecret, "req hs key", KeyLen)
ResponderHandshakeKey = HKDF-Expand(HandshakeSecret, "rsp hs key", KeyLen)

MasterSecret = HKDF-Extract(HandshakeSecret, 0)

RequesterDataKey = HKDF-Expand(MasterSecret, "req data key", KeyLen)
ResponderDataKey = HKDF-Expand(MasterSecret, "rsp data key", KeyLen)
```

Encrypted Message Format

Within a secure session, messages use AEAD encryption:



Cryptographic Algorithms

Supported Algorithms

Asymmetric Signature

Algorithm	OID	Key Size
RSASSA_2048	1.2.840.113549.1.1.11	2048 bits
RSASSA_3072	1.2.840.113549.1.1.11	3072 bits
RSASSA_4096	1.2.840.113549.1.1.11	4096 bits
RSAPSS_2048	1.2.840.113549.1.1.10	2048 bits
RSAPSS_3072	1.2.840.113549.1.1.10	3072 bits
RSAPSS_4096	1.2.840.113549.1.1.10	4096 bits
ECDSA_P256	1.2.840.10045.4.3.2	256 bits
ECDSA_P384	1.2.840.10045.4.3.3	384 bits
ECDSA_P521	1.2.840.10045.4.3.4	521 bits
EDDSA_25519	1.3.101.112	255 bits
EDDSA_448	1.3.101.113	448 bits

Hash Algorithms

Algorithm	Digest Size
SHA2_256	32 bytes
SHA2_384	48 bytes
SHA2_512	64 bytes
SHA3_256	32 bytes
SHA3_384	48 bytes
SHA3_512	64 bytes

DHE (Diffie-Hellman Ephemeral)

Named Group	Key Size
FFDHE_2048	2048 bits
FFDHE_3072	3072 bits
FFDHE_4096	4096 bits
SECP_256_R1	256 bits
SECP_384_R1	384 bits
SECP_521_R1	521 bits
X25519	255 bits
X448	448 bits

AEAD (Session Encryption)

Algorithm	Key Size	Tag Size
AES_128_GCM	128 bits	128 bits
AES_256_GCM	256 bits	128 bits
CHACHA20_POLY1305	256 bits	128 bits

Setup & Configuration

Build-Time Configuration (Yocto)

```
# In your machine .conf or local.conf

# Include SPDM support
IMAGE_INSTALL:append = " \
    libspdm \
    spdm-emu \
    phosphor-spdm \
"

# Enable SPDM in relevant components
EXTRA_OEMESON:pn-bmcweb = " \
    -Dredfish-component-integrity=enabled \
"

# libspdm build configuration
EXTRA_OECMAKE:pn-libspdm = " \
    -DLIBSPDM_ENABLE_CAPABILITY_CERT_CAP=ON \
    -DLIBSPDM_ENABLE_CAPABILITY_CHAL_CAP=ON \
    -DLIBSPDM_ENABLE_CAPABILITY_MEAS_CAP=ON \
    -DLIBSPDM_ENABLE_CAPABILITY_KEY_EX_CAP=ON \
    -DLIBSPDM_ENABLE_CAPABILITY_PSK_EX_CAP=OFF \
    -DLIBSPDM_ENABLE_CAPABILITY_HBEAT_CAP=ON \
    -DLIBSPDM_ENABLE_CAPABILITY_MUT_AUTH_CAP=ON \
    -DLIBSPDM_ENABLE_CAPABILITY_CHUNK_CAP=ON \
"
```

libspdm Build Options

Option	Default	Description
<code>LIBSPDM_ENABLE_CAPABILITY_CERT_CAP</code>	ON	Certificate capability
<code>LIBSPDM_ENABLE_CAPABILITY_CHAL_CAP</code>	ON	Challenge capability
<code>LIBSPDM_ENABLE_CAPABILITY_MEAS_CAP</code>	ON	Measurement capability
<code>LIBSPDM_ENABLE_CAPABILITY_KEY_EX_CAP</code>	ON	Key exchange (sessions)
<code>LIBSPDM_ENABLE_CAPABILITY_PSK_EX_CAP</code>	OFF	Pre-shared key exchange
<code>LIBSPDM_ENABLE_CAPABILITY_HBEAT_CAP</code>	ON	Session heartbeat
<code>LIBSPDM_ENABLE_CAPABILITY_MUT_AUTH_CAP</code>	ON	Mutual authentication
<code>LIBSPDM_ENABLE_CAPABILITY_CHUNK_CAP</code>	ON	Large message chunking
<code>LIBSPDM_ENABLE_CAPABILITY_ENCAP_CAP</code>	OFF	Encapsulated requests
<code>LIBSPDM_RECORD_TRANSCRIPT_DATA_SUPPORT</code>	ON	Transcript for debugging

Runtime Configuration

```
# Check SPDm daemon status
systemctl status spdmd

# View SPDm logs
journalctl -u spdmd -f

# Configuration file location
cat /etc/spdm/spdm.conf
```

Algorithm Configuration

```
// /etc/spdm/algorithms.json
{
  "version": "1.2",
  "baseAsymAlgo": [
    "ECDSA_P384",
    "ECDSA_P256",
    "RSASSA_3072"
  ],
  "baseHashAlgo": [
    "SHA2_384",
    "SHA2_256"
  ],
  "measurementHashAlgo": [
    "SHA2_384",
    "SHA2_256"
  ],
  "dheNamedGroup": [
    "SECP_384_R1",
    "SECP_256_R1"
  ],
  "aead": [
    "AES_256_GCM",
    "CHACHA20_POLY1305"
  ],
  "keySchedule": "SPDM"
}
```

Certificate Management

Certificate Chain

SPDM uses X.509 certificate chains for device authentication:

```
Root CA (Self-signed, Trusted by BMC)
├── Intermediate CA (Optional)
│   └── Device Certificate (Leaf)
│       ├── Contains device public key
│       ├── Signed by Intermediate/Root CA
│       └── Used for challenge signature
```

Provisioning Trust Anchors

```
# Root CA certificates location
/etc/spdm/certs/trust/

# Add trusted root CA
cp vendor_root_ca.pem /etc/spdm/certs/trust/

# Convert DER to PEM if needed
openssl x509 -inform DER -in vendor_root_ca.der -out vendor_root_ca.pem

# Verify certificate
openssl x509 -in vendor_root_ca.pem -text -noout

# Restart SPDM daemon to load new trust anchors
systemctl restart spdmd
```

BMC Certificate (for Mutual Auth)

```
# BMC certificate location
/etc/spdm/certs/bmc/

# Generate BMC keypair (ECDSA P-384 recommended)
openssl ecparam -name secp384r1 -genkey -noout -out /etc/spdm/certs/bmc/key.pem

# Generate CSR
openssl req -new -key /etc/spdm/certs/bmc/key.pem \
    -out /etc/spdm/certs/bmc/csr.pem \
    -subj "/CN=BMC-$(hostname)/O=MyOrg/OU=OpenBMC"

# Install signed certificate (after CA signs the CSR)
cp signed_bmc_cert.pem /etc/spdm/certs/bmc/cert.pem

# Set proper permissions
chmod 600 /etc/spdm/certs/bmc/key.pem
chmod 644 /etc/spdm/certs/bmc/cert.pem
```

Certificate Verification

```
# Verify certificate chain
openssl verify -CAfile root_ca.pem \
    -untrusted intermediate.pem device_cert.pem

# Check certificate validity period
openssl x509 -in device_cert.pem -noout -dates

# Check certificate key usage
openssl x509 -in device_cert.pem -noout -text | grep -A1 "Key Usage"

# Check certificate subject
openssl x509 -in device_cert.pem -noout -subject
```


SPDM Operations

Device Discovery

```
# List SPDM-capable endpoints
spdmtool discover

# Get SPDM version from device
spdmtool version -m 10

# Example output:
# SPDM Version: 1.2.0
# Supported Versions: 1.0.0, 1.1.0, 1.2.0

# Get device capabilities
spdmtool capabilities -m 10

# Example output:
# Capabilities:
#   CERT_CAP: Yes
#   CHAL_CAP: Yes
#   MEAS_CAP: Signature
#   KEY_EX_CAP: Yes
#   MUT_AUTH_CAP: Yes
#   ENCRYPT_CAP: Yes
```

Get Device Certificate

```
# Get certificate chain digest
spdmtool getdigest -m 10

# Get certificate chain
spdmtool getcert -m 10 -s 0

# Save certificate chain to file
spdmtool getcert -m 10 -s 0 -o device_chain.pem

# Extract individual certificates
openssl storeutl -certs device_chain.pem
```

Challenge Authentication

```
# Send challenge to verify device
spdmtool challenge -m 10 -s 0

# Challenge with measurement summary
spdmtool challenge -m 10 -s 0 --meas-summary all

# Example output:
# Challenge Response:
#   Certificate Hash: SHA384:abc123...
#   Nonce: 0x1234...
#   Measurement Summary Hash: SHA384:def456...
#   Signature: ECDSA_P384:789...
#   Verification: SUCCESS
```

Get Measurements

```
# Get all measurements
spdmtool measurements -m 10

# Get signed measurements
spdmtool measurements -m 10 --signed

# Get specific measurement index
spdmtool measurements -m 10 -i 1

# Save measurements to JSON
spdmtool measurements -m 10 -o measurements.json

# Example output:
# Measurement Record:
#   Index 1: Type=MutableFirmware
#             Hash=SHA384:abc123def456...
#   Index 2: Type=FirmwareConfig
#             Hash=SHA384:789abc012def...
#   Index 3: Type=MutableFirmwareVersion
#             Value="v2.1.0-rc3"
```

Redfish ComponentIntegrity

Enable in bmcweb

```
EXTRA_OEMESON:pn-bmcweb = " \
    -Dredfish-component-integrity=enabled \
"
```

Query Component Integrity

```
# Get ComponentIntegrity collection
curl -k -u root:0penBmc \
    https://localhost/redfish/v1/ComponentIntegrity

# Get specific component
curl -k -u root:0penBmc \
    https://localhost/redfish/v1/ComponentIntegrity/GPU_0
```

ComponentIntegrity Response

```
{
  "@odata.id": "/redfish/v1/ComponentIntegrity/GPU_0",
  "@odata.type": "#ComponentIntegrity.v1_2_0.ComponentIntegrity",
  "Id": "GPU_0",
  "Name": "GPU 0 Integrity",
  "ComponentIntegrityType": "SPDM",
  "ComponentIntegrityTypeVersion": "1.2.0",
  "TargetComponentURI": "/redfish/v1/Systems/system/Processors/GPU_0",
  "LastUpdated": "2024-01-15T10:30:00Z",
  "SPDM": {
    "Requester": {
      "IdentityAuthentication": "Success",
      "HardwareIntrusion": "Normal"
    },
    "IdentityAuthentication": {
      "ResponderAuthentication": "Success",
      "VerificationStatus": "Success",
      "ComponentCertificate": {
        "@odata.id": "/redfish/v1/ComponentIntegrity/GPU_0/Certificates/0"
      }
    },
    "MeasurementSet": {
      "MeasurementSpecification": "DMTF",
      "MeasurementHashAlgorithm": "SHA384",
      "Measurements": [
        {
          "MeasurementIndex": 1,
          "Measurement": "abc123def456...",
          "MeasurementType": "MutableFirmware",
          "LastUpdated": "2024-01-15T10:30:00Z",
          "SecurityVersionNumber": "1",
          "VerificationStatus": "Success"
        },
        {
          "MeasurementIndex": 2,
          "Measurement": "789abc012def...",
          "MeasurementType": "FirmwareConfiguration",
          "LastUpdated": "2024-01-15T10:30:00Z",
          "VerificationStatus": "Success"
        }
      ]
    }
  },
  "Actions": {
    "#ComponentIntegrity.SPDMGetSignedMeasurements": {
      "target": "/redfish/v1/ComponentIntegrity/GPU_0/Actions/ComponentIntegrity.SPDMGetSignedMeasurements"
    }
  }
}
```

Trigger New Attestation

```
# Force re-attestation via action
curl -k -u root:0penBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{"Nonce": "abc123", "MeasurementIndices": [1, 2, 3]}' \
  https://localhost/redfish/v1/ComponentIntegrity/GPU_0/Actions/ComponentIntegrity.SPDMGetSignedMeasurements

# Get certificate from component
curl -k -u root:0penBmc \
  https://localhost/redfish/v1/ComponentIntegrity/GPU_0/Certificates/0
```

Secure Session Operations

Key Exchange

```
# Establish secure session
spdmtool keyexchange -m 10

# Session provides:
# - Encrypted communication
# - Authenticated channel
# - Replay protection

# Session with mutual authentication
spdmtool keyexchange -m 10 --mutual-auth

# Example output:
# Session Established:
#   Session ID: 0xABCD1234
#   Algorithm: AES_256_GCM
#   DHE: SECP_384_R1
```

Session Management

```
# Send message over secure session
spdmtool secured-message -m 10 -s 0xABCD1234 --data "..."

# Heartbeat to keep session alive
spdmtool heartbeat -m 10 -s 0xABCD1234

# Update session keys
spdmtool keyupdate -m 10 -s 0xABCD1234

# End session
spdmtool endsession -m 10 -s 0xABCD1234
```

Session Types

Type	Description	Use Case
<code>KEY_EXCHANGE</code>	Asymmetric key exchange	Standard sessions
<code>PSK_EXCHANGE</code>	Pre-shared key	Devices without certificates
<code>PSK_FINISH</code>	PSK with challenge	Enhanced PSK security

Measurement Policy

Expected Measurements Configuration

```
// /etc/spdm/policy/gpu_expected.json
{
  "deviceType": "GPU",
  "vendorId": "0x10DE",
  "measurements": {
    "1": {
      "type": "MutableFirmware",
      "expectedHash": "sha384:abc123def456789...",
      "description": "GPU firmware v2.1.0"
    },
    "2": {
      "type": "FirmwareConfig",
      "expectedHash": "sha384:789abc012def345...",
      "description": "Factory configuration"
    },
    "3": {
      "type": "MutableFirmwareVersion",
      "expectedValue": "v2.1.0",
      "description": "Firmware version string"
    }
  },
  "policy": {
    "requireAllMeasurements": true,
    "actionOnMismatch": "log_and_alert",
    "allowFirmwareUpdate": true
  }
}
```

Policy Actions

Action	Description
<code>log</code>	Log mismatch, continue operation
<code>alert</code>	Generate Redfish alert event
<code>log_and_alert</code>	Both log and alert
<code>reject</code>	Block device from operation
<code>quarantine</code>	Isolate device, require manual action

D-Bus Integration

SPDM D-Bus Interface

```
# List SPDM service
busctl tree xyz.openbmc_project.SPDM

# Object structure:
# /xyz/openbmc_project/spdm
#   /endpoint/<eid>
#     - xyz.openbmc_project.SPDM.Endpoint
#     - xyz.openbmc_project.SPDM.Certificate
#     - xyz.openbmc_project.SPDM.Measurement

# Get endpoint properties
busctl introspect xyz.openbmc_project.SPDM \
  /xyz/openbmc_project/spdm/endpoint/10

# Get measurements via D-Bus
busctl call xyz.openbmc_project.SPDM \
  /xyz/openbmc_project/spdm/endpoint/10 \
  xyz.openbmc_project.SPDM.Measurement \
  GetMeasurements u 0xFF
```

Attestation Events

```
# Monitor SPDM events
busctl monitor xyz.openbmc_project.SPDM

# Events include:
# - EndpointDiscovered
# - AuthenticationSuccess/Failure
# - MeasurementMismatch
# - SessionEstablished/Terminated
```

Troubleshooting

Discovery Failures

```
# Verify MCTP connectivity first
mctp endpoint

# Check SPDM version compatibility
spdmtool version -m 10

# Enable debug logging
echo "SPDM_DEBUG=1" >> /etc/default/spdmd
systemctl restart spdmd

# View detailed logs
journalctl -u spdmd -f --no-pager
```

Certificate Verification Failed

```
# Check trust anchor is installed
ls -la /etc/spdm/certs/trust/

# Get device certificate chain
spdmtool getcert -m 10 -s 0 -o /tmp/device_chain.pem

# Verify certificate chain manually
openssl verify -CAfile /etc/spdm/certs/trust/root.pem \
    /tmp/device_chain.pem

# Check certificate validity
openssl x509 -in /tmp/device_chain.pem -checkend 0

# Check certificate details
openssl x509 -in /tmp/device_chain.pem -text -noout | head -30
```

Measurement Mismatch

```
# Get actual measurements
spdmtool measurements -m 10 -o /tmp/actual_meas.json

# Compare with expected
diff /tmp/actual_meas.json /etc/spdm/policy/expected.json

# Check if firmware was updated
# Measurements change with firmware updates

# Verify measurement type
spdmtool measurements -m 10 -i 1 --verbose
```

Session Failures

```
# Check algorithm compatibility
spdmtool capabilities -m 10

# Verify key exchange support
spdmtool algorithms -m 10

# Check session logs
journalctl -u spdmd | grep -i session

# Verify DHE is supported
spdmtool capabilities -m 10 | grep KEY_EX
```

Common Error Codes

Error	Cause	Solution
<code>InvalidRequest</code>	Malformed message	Check message format
<code>Busy</code>	Device processing	Retry with backoff
<code>UnexpectedRequest</code>	Wrong state machine	Reset and restart
<code>DecryptError</code>	Session key mismatch	Re-establish session
<code>UnsupportedRequest</code>	Feature not supported	Check capabilities
<code>InvalidSession</code>	Session expired/invalid	Create new session

Security Best Practices

Certificate Management

1. **Use strong keys:** RSA $\geq$ 3072 or ECDSA P-384
2. **Rotate certificates:** Before expiration, plan rotation
3. **Secure storage:** Protect private keys with HSM or secure enclave
4. **Monitor expiration:** Alert before certificate expires
5. **Revocation checking:** Implement CRL or OCSP if available

Measurement Verification

1. **Store expected values securely:** Protect measurement database
2. **Update with firmware:** Expected values must match deployed firmware
3. **Alert on changes:** Any measurement change should trigger investigation
4. **Log all events:** Maintain audit trail of attestation attempts
5. **Regular re-attestation:** Periodically verify device integrity

Access Control

```
# Limit SPDm administration access
chmod 600 /etc/spdm/spdm.conf
chmod 600 /etc/spdm/policy/*.json

# Use D-Bus policy for access control
# /etc/dbus-1/system.d/spdm.conf
```


Production Recommendations

Recommendation	Description
Enable mutual auth	Both BMC and device authenticate
Use secure sessions	Encrypt SPDm traffic
Require signed measurements	Don't accept unsigned
Implement alerting	Notify on failures
Regular audits	Review attestation logs
Update policies	Keep expected measurements current

Enabling/Disabling SPDm

Build-Time Disable

```
# Remove SPDm packages
IMAGE_INSTALL:remove = "libspdm phosphor-spdm"

# Disable in bmcweb
EXTRA_OEMESON:pn-bmcweb = " \
    -Dredfish-component-integrity=disabled \
"
```

Runtime Disable

```
# Stop SPDm daemon
systemctl stop spdmd
systemctl disable spdmd

# SPDm attestation will not be available
# Existing sessions will close
```

Examples

Working examples are available in the [examples/spdm](#) directory:

- [spdm_cert_setup.sh](#) - Generate test certificate chain (Root CA / Intermediate / BMC)
- [spdm_auth.sh](#) - Full SPDm authentication flow (version, capabilities, certs, challenge)
- [spdm_measurements.sh](#) - Collect and verify firmware measurements
- [spdm_redfish.sh](#) - Query Redfish ComponentIntegrity endpoints
- [config/algorithms.json](#) - Crypto algorithm preferences
- [config/gpu_expected.json](#) - Expected measurement policy for attestation
- [config/component_integrity.json](#) - Example Redfish ComponentIntegrity response

References

- [DMTF DSP0274 - SPDm Specification](#) - Official SPDm protocol specification
 - [DMTF DSP0277 - Secured Messages](#) - SPDm secured message specification
 - [libspdm \(GitHub\)](#) - DMTF reference implementation
 - [spdm-emu \(GitHub\)](#) - SPDm emulator for testing
 - [Redfish Component Integrity Schema](#) - Redfish SPDm interface
 - [OpenBMC Redfish SPDm Attestation Design](#) - OpenBMC integration design
 - [OpenBMC SPDm Daemon](#) - OpenBMC SPDm implementation
 - [TCG DICE Attestation Architecture](#) - Related attestation architecture
-

Tested on: OpenBMC master, requires hardware with SPDm-capable devices (GPU, NIC, NVMe, etc.)

Firmware Update Guide

Configure and perform BMC and host firmware updates on OpenBMC - comprehensive deep dive into update mechanisms, image formats, and flash management.

Table of Contents

1. [Overview](#)
 1. [Update Architecture](#)
 2. [Key Components](#)
2. [Firmware Image Format](#)
 1. [Tarball Structure](#)
 2. [MANIFEST File Format](#)
 1. [MANIFEST Fields](#)
 3. [Image File Types](#)
 4. [Creating Firmware Images](#)
 5. [Signing Images](#)
3. [Flash Layout & Storage](#)
 1. [BMC Flash Layout \(ASPEED\)](#)
 2. [Static MTD vs UBI Layout](#)
 1. [Static MTD Layout](#)
 2. [UBI \(Unsorted Block Images\) Layout](#)
 3. [Host Flash Layout \(Intel/AMD BIOS\)](#)
 4. [OpenPOWER PNOR Layout](#)
4. [Setup & Configuration](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [phosphor-bmc-code-mgmt Meson Options](#)
 3. [Configuration Files and Paths](#)
 4. [Deploying Signing Keys via Yocto](#)
 5. [Enabling Field Mode](#)
 6. [Runtime Configuration](#)
 7. [D-Bus Software Interfaces](#)
5. [Firmware Inventory](#)
 1. [View Current Versions](#)
 2. [Firmware Response](#)
6. [Update Methods](#)
 1. [Redfish Update Service](#)
 2. [SimpleUpdate Action](#)
 3. [IPMI OEM Firmware Update](#)
 1. [phosphor-ipmi-flash Configuration](#)
 4. [PLDM Firmware Update \(Type 5\)](#)

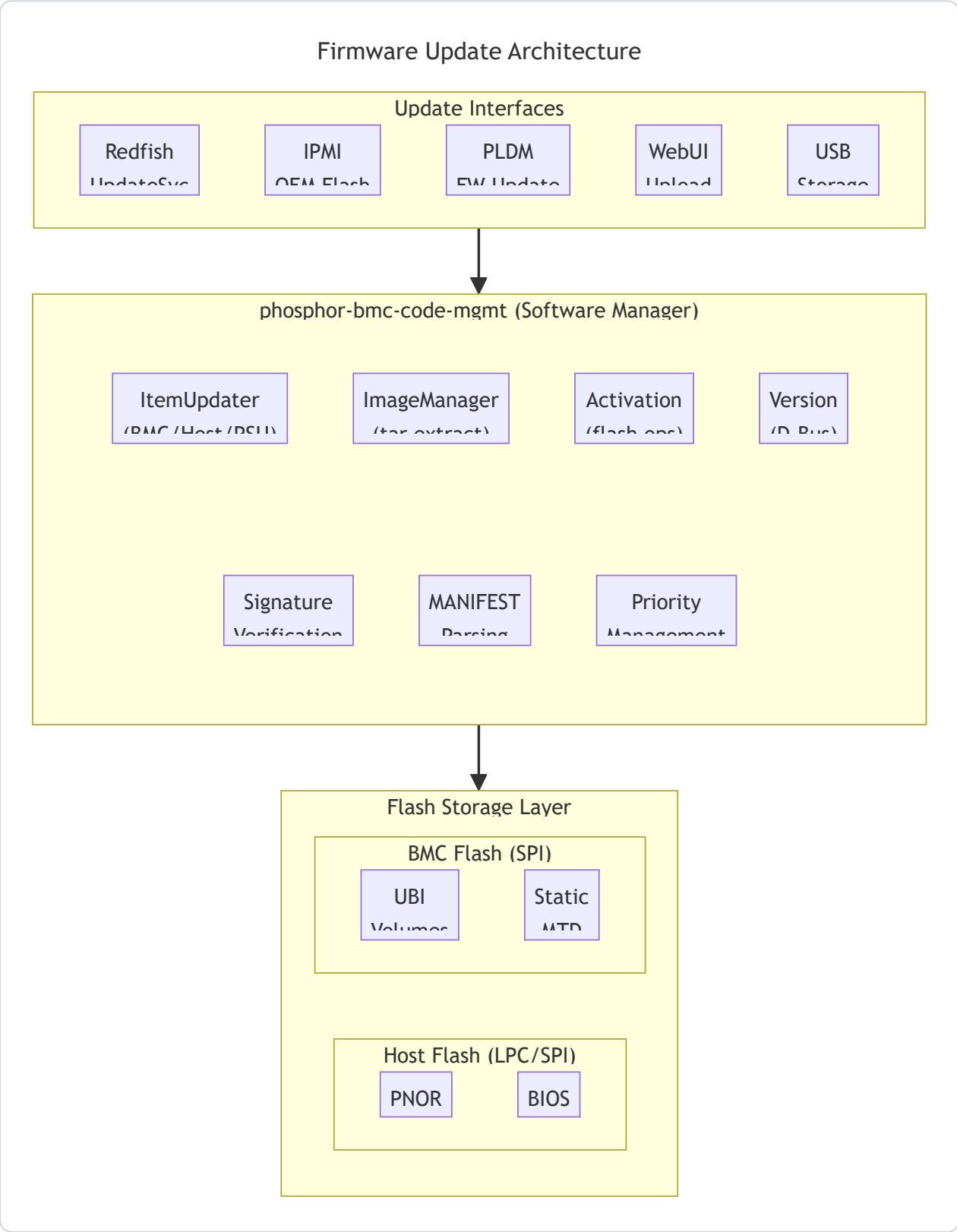
1. PLDM Update Flow
 5. WebUI Update
 6. USB Code Update
 7. Command-Line Update Tools
7. Dual Image Support
 1. Dual Image Architecture
 2. View Images via Redfish
 3. View Images via D-Bus
 4. Set Boot Priority
8. U-Boot Environment & Boot Control
 1. Key U-Boot Environment Variables
 2. U-Boot Boot Logic
 3. Manual Boot Control
 4. Recovery via Serial Console
9. Failsafe Behavior
 1. Automatic Boot Fallback
 2. Boot Health Check Service
 3. Recovery Mode
10. Firmware Signing & Verification
 1. Signing Infrastructure Overview
 2. Generate Signing Keys
 3. Sign Firmware Components
 4. Configure Verification in Yocto
 5. Verification Flow
 6. Manual Signature Verification
 7. Signature Verification Logs
 8. Key Type Configuration
11. Host Firmware Update
 1. BIOS/UEFI Update (Intel/AMD)
 1. Intel SPS/ME Firmware
 2. PNOR Update (OpenPOWER)
 1. openpower-pnor-code-mgmt
 3. PSU Firmware Update
 4. Component-Level Update (PLDM)
12. Update Workflow Deep Dive
 1. Complete Update Sequence
 2. Activation States
 3. Monitor Update Progress
 4. Progress Indication
13. Configuration Options
 1. Update Settings

- 2. [Apply Time Options](#)
 - 14. [Troubleshooting](#)
 - 1. [Upload Fails](#)
 - 2. [Signature Verification Failed](#)
 - 3. [Machine Name Mismatch](#)
 - 4. [Boot Fails After Update](#)
 - 5. [Image Not Activating](#)
 - 6. [Flash Write Errors](#)
 - 7. [Activation Progress Stuck](#)
 - 8. [Delete Failed/Stale Images](#)
 - 9. [Network Update Issues](#)
 - 15. [Rollback](#)
 - 1. [Rollback via D-Bus](#)
 - 2. [Rollback via Redfish](#)
 - 3. [Rollback via U-Boot](#)
 - 16. [Factory Reset](#)
 - 1. [Reset BMC Configuration](#)
 - 2. [Clear Persistent Data](#)
 - 3. [Full Flash Recovery](#)
 - 17. [Security Considerations](#)
 - 1. [Secure Boot Chain](#)
 - 2. [U-Boot Verified Boot Configuration](#)
 - 3. [Update Authentication](#)
 - 4. [Audit Logging](#)
 - 18. [Best Practices](#)
 - 1. [Pre-Update Checklist](#)
 - 2. [Update Procedure](#)
 - 3. [Post-Update Verification](#)
 - 19. [Examples](#)
 - 20. [References](#)
 - 1. [OpenBMC Repositories](#)
 - 2. [OpenBMC Documentation](#)
 - 3. [Standards](#)
 - 4. [U-Boot](#)
 - 21. [Deep Dive](#)
 - 1. [Flash Layout and MTD Partitions](#)
 - 2. [Update Image Verification](#)
 - 3. [Flash Programming Internals](#)
 - 4. [Recovery Mechanism](#)
 - 5. [Source Code Reference](#)
-

Overview

OpenBMC supports firmware updates for BMC, BIOS/UEFI, and other platform components through multiple interfaces including Redfish, IPMI, PLDM, and command-line tools. This guide provides a deep dive into the update architecture, image formats, flash layouts, and troubleshooting procedures.

Update Architecture



► ASCII-art version (for comparison)

Key Components

Component	Repository	Description
phosphor-bmc-code-mgmt	openbmc/phosphor-bmc-code-mgmt	Core BMC firmware management
phosphor-software-manager	Part of code-mgmt	D-Bus service for update operations
phosphor-ipmi-flash	openbmc/phosphor-ipmi-flash	IPMI OEM firmware update
pldmd	openbmc/pldmd	PLDM Type 5 firmware update
phosphor-image-signing	Part of code-mgmt	Signature generation and verification

Firmware Image Format

Tarball Structure

OpenBMC firmware images are packaged as tarballs with a specific structure:

```
obmc-phosphor-image-<machine>.tar
├─ MANIFEST                # Metadata about the image
├─ publickey               # Public key for verification (optional)
├─ image-kernel            # Linux kernel (FIT image or uImage)
├─ image-rofs              # Read-only root filesystem (squashfs)
├─ image-rwfs              # Read-write filesystem (ext4/ubifs)
├─ image-u-boot            # U-Boot bootloader
├─ image-bmc               # Combined flash image (static.mtd)
└─ *.sig                  # Signatures for each component
```

MANIFEST File Format

The MANIFEST file contains critical metadata parsed during update:

```
# Example MANIFEST file
purpose=xyz.openbmc_project.Software.Version.VersionPurpose.BMC
version=2.12.0-dev-123-gabcdef
KeyType=OpenBMC
HashType=RSA-SHA256
MachineName=romulus

# Extended version information (optional)
extended_version=OpenBMC for Romulus - master branch build 123
```

MANIFEST Fields

Field	Required	Description
<code>purpose</code>	Yes	<code>BMC</code> , <code>Host</code> , <code>System</code> , or custom
<code>version</code>	Yes	Semantic version string
<code>MachineName</code>	Recommended	Target machine (prevents wrong-machine updates)
<code>KeyType</code>	If signed	Signing key type (OpenBMC, ProductionKey, etc.)
<code>HashType</code>	If signed	Hash algorithm (RSA-SHA256, RSA-SHA512)
<code>extended_version</code>	No	Additional version details

Image File Types

File	Format	Description
<code>image-kernel</code>	FIT/uImage	Linux kernel, often with DTB
<code>image-rofs</code>	SquashFS	Compressed read-only root filesystem
<code>image-rwfs</code>	ext4/UBIFS	Persistent writable data
<code>image-u-boot</code>	Binary	U-Boot bootloader image
<code>image-bmc</code>	static.mtd	Complete flash image for recovery

Creating Firmware Images

```
# Build produces images in deploy directory
bitbake obmc-phosphor-image

# Find images
ls tmp/deploy/images/<machine>/
# obmc-phosphor-image-<machine>.tar
# obmc-phosphor-image-<machine>.static.mtd
# obmc-phosphor-image-<machine>.ubi.mtd

# Manual tarball creation (for testing)
cd tmp/deploy/images/<machine>/
tar -cvf custom-image.tar \
    MANIFEST \
    image-kernel \
    image-rofs \
    image-rwfs \
    image-u-boot
```


Signing Images

```
# Generate signing keys (one-time setup)
openssl genrsa -out private.pem 4096
openssl rsa -in private.pem -pubout -out publickey

# Sign each component
for file in image-kernel image-rofs image-rwfs image-u-boot MANIFEST; do
    openssl dgst -sha256 -sign private.pem -out ${file}.sig ${file}
done

# Create signed tarball
tar -cvf obmc-phosphor-image-signed.tar \
    MANIFEST MANIFEST.sig \
    publickey \
    image-kernel image-kernel.sig \
    image-rofs image-rofs.sig \
    image-rwfs image-rwfs.sig \
    image-u-boot image-u-boot.sig
```

Flash Layout & Storage

Understanding flash layout is essential for firmware update operations.

BMC Flash Layout (ASPEED)

Typical ASPEED AST2500/AST2600 flash layout:

BMC SPI Flash (32MB/64MB)			
Offset	Size	Name	Description
0x00000000	512KB	u-boot	Bootloader
0x00080000	64KB	u-boot-env	Environment variables
0x00090000	448KB	<reserved>	Future use
0x00100000	4MB	kernel-a	Kernel image A
0x00500000	20MB	rofs-a	Read-only rootfs A
0x01900000	4MB	kernel-b	Kernel image B
0x01D00000	20MB	rofs-b	Read-only rootfs B
0x03100000	4MB	rwfs	Persistent data
0x03500000	11MB	<reserved>	Future use

Static MTD vs UBI Layout

OpenBMC supports two flash management strategies:

Static MTD Layout

Traditional fixed-partition layout:

```
# In machine .conf
FLASH_SIZE = "32768" # 32MB in KB

# Partition sizes (KB)
FLASH_UBOOT_OFFSET = "0"
FLASH_UBOOT_ENV_OFFSET = "512"
FLASH_KERNEL_OFFSET = "1024"
FLASH_ROFS_OFFSET = "5120"
FLASH_RWFS_OFFSET = "28160"
```

```
# View MTD partitions on target
cat /proc/mtd
# dev:      size    erasesize  name
# mtd0: 00080000 00001000 "u-boot"
# mtd1: 00010000 00001000 "u-boot-env"
# mtd2: 00400000 00001000 "kernel"
# mtd3: 01400000 00001000 "rofs"
# mtd4: 00400000 00001000 "rwfs"
```

UBI (Unsorted Block Images) Layout

Dynamic volume management with wear leveling:

```
# Enable UBI
IMAGE_FSTYPES += "ubi.mtd"
FLASH_UBI_OVERLAY_SIZE = "4096" # KB
FLASH_UBI_RWFS_SIZE = "6144" # KB
```

```
# View UBI volumes
ubinfs -a
# UBI version:                1
# Volumes count:              4
# Volume 0: kernel-a
# Volume 1: kernel-b
# Volume 2: rofs-a
# Volume 3: rofs-b

# Detailed volume info
ubinfs /dev/ubi0 -N rofs-a
```

Host Flash Layout (Intel/AMD BIOS)

Host SPI Flash (16MB/32MB)		
Region	Size	Description
Descriptor	4KB	Flash descriptor (regions, permissions)
BIOS	8-16MB	UEFI firmware
ME/PSP	2-4MB	Intel ME / AMD PSP firmware
GbE	8KB	Ethernet controller config
Platform	Varies	Platform-specific data

OpenPOWER PNOR Layout

PNOR Flash (64MB/128MB)		
Partition	Size	Description
TOC	32KB	Table of Contents
HBB	1MB	Hostboot Base
HBI	16MB	Hostboot Extended Image
HBRT	4MB	Hostboot Runtime
PAYLOAD	1MB	OPAL Skiboot
BOOTKERNEL	32MB	Linux kernel + initramfs
NVRAM	576KB	Non-volatile settings
GUARD	16KB	Hardware guard records
RINGOVD	256KB	Ring override partition

Setup & Configuration

Build-Time Configuration (Yocto)

```
# In your machine .conf or local.conf

# Include software management
IMAGE_INSTALL:append = " \
    phosphor-software-manager \
    phosphor-image-signing \
"

# Configure update options
EXTRA_OEMESON:pn-phosphor-bmc-code-mgmt = " \
    -Dfwupd-script=enabled \
    -Dsync-bmc-files=enabled \
    -Dverify-signature=enabled \
    -Dhost-bios-upgrade=enabled \
    -Dusb-code-update=enabled \
"

# Enable image signing
INHERIT += "image_types_phosphor"
IMAGE_TYPES += "static.mtd ubi.mtd"

# Configure flash layout
FLASH_SIZE = "32768"
FLASH_UBOOT_ENV_OFFSET_KB = "512"
FLASH_KERNEL_OFFSET_KB = "1024"

# Dual image support
OBMC_PHOSPHOR_IMAGE_DUAL = "1"
```

phosphor-bmc-code-mgmt Meson Options

Option	Default	Description
<code>verify-signature</code>	enabled	Require signed firmware
<code>sync-bmc-files</code>	enabled	Sync critical files between images
<code>fwupd-script</code>	disabled	Use external update script
<code>host-bios-upgrade</code>	disabled	Enable BIOS/UEFI updates
<code>usb-code-update</code>	disabled	Enable USB firmware update
<code>side-switch-on-boot</code>	disabled	Auto-switch on boot failure
<code>software-update-dbus-interface</code>	enabled	Expose D-Bus update interface
<code>delete-images</code>	enabled	Allow image deletion

Configuration Files and Paths

Config/Key File	Install Path	Description
Signing public key	<code>/etc/activationdata/OpenBMC/publickey</code>	Verify image signatures on the BMC
Hash type	<code>/etc/activationdata/OpenBMC/hashfunc</code>	Hash algorithm (e.g., <code>RSA-SHA256</code>)
MANIFEST	Inside firmware tarball	Image metadata (version, purpose, machine)
Field mode flag	<code>/var/lib/phosphor-bmc-code-mgmt/field-mode</code>	When present, restricts update to signed images

Deploying Signing Keys via Yocto

To enable signature verification, deploy your public key into the image:

```
# meta-myplatform/recipes-phosphor/flash/phosphor-bmc-code-mgmt/
# └─ files/
#   └─ publickey      # RSA public key
#   └─ hashfunc       # Contains "RSA-SHA256"
# └─ phosphor-bmc-code-mgmt_%.bbappend

cat > phosphor-bmc-code-mgmt_%.bbappend << 'EOF'
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
SRC_URI += " \
    file://publickey \
    file://hashfunc \
"

do_install:append() {
    install -d ${D}${sysconfdir}/activationdata/OpenBMC
    install -m 0444 ${WORKDIR}/publickey \
        ${D}${sysconfdir}/activationdata/OpenBMC/
    install -m 0444 ${WORKDIR}/hashfunc \
        ${D}${sysconfdir}/activationdata/OpenBMC/
}
EOF
```

Generate the key pair:

```
# One-time setup (keep private.pem secure!)
openssl genrsa -out private.pem 4096
openssl rsa -in private.pem -pubout -out publickey
echo "RSA-SHA256" > hashfunc
```

Enabling Field Mode

Field mode restricts the BMC to only accept signed firmware:

```
# Enable field mode (persists across reboots)
busctl set-property xyz.openbmc_project.Software.BMC.Updater \
    /xyz/openbmc_project/software \
    xyz.openbmc_project.Control.FieldMode FieldModeEnabled b true

# Verify field mode
busctl get-property xyz.openbmc_project.Software.BMC.Updater \
    /xyz/openbmc_project/software \
    xyz.openbmc_project.Control.FieldMode FieldModeEnabled
```

Runtime Configuration

```
# Check software manager status
systemctl status phosphor-software-manager@bmc
systemctl status phosphor-version-software-manager

# View update logs
journalctl -u phosphor-software-manager@bmc -f

# Check current firmware versions
busctl tree xyz.openbmc_project.Software.BMC.Updater

# List all software objects
busctl call xyz.openbmc_project.ObjectMapper \
  /xyz/openbmc_project/object_mapper \
  xyz.openbmc_project.ObjectMapper \
  GetSubTree sias "/xyz/openbmc_project/software" 0 1 \
  "xyz.openbmc_project.Software.Version"
```

D-Bus Software Interfaces

```
# Key interfaces exposed by phosphor-software-manager:

# Software.Version - Version information
busctl introspect xyz.openbmc_project.Software.BMC.Updater \
  /xyz/openbmc_project/software/<id> \
  xyz.openbmc_project.Software.Version

# Software.Activation - Activation state
busctl introspect xyz.openbmc_project.Software.BMC.Updater \
  /xyz/openbmc_project/software/<id> \
  xyz.openbmc_project.Software.Activation

# Software.RedundancyPriority - Boot priority
busctl introspect xyz.openbmc_project.Software.BMC.Updater \
  /xyz/openbmc_project/software/<id> \
  xyz.openbmc_project.Software.RedundancyPriority
```

Firmware Inventory

View Current Versions

```
# Via Redfish
curl -k -u root:0penBmc \
  https://localhost/redfish/v1/UpdateService/FirmwareInventory

# Get specific firmware info
curl -k -u root:0penBmc \
  https://localhost/redfish/v1/UpdateService/FirmwareInventory/bmc_active

# Via D-Bus
busctl tree xyz.openbmc_project.Software.BMC.Updater
```

Firmware Response

```
{
  "@odata.id": "/redfish/v1/UpdateService/FirmwareInventory/bmc_active",
  "Id": "bmc_active",
  "Name": "BMC Firmware",
  "Version": "2.12.0",
  "Status": {
    "Health": "OK",
    "State": "Enabled"
  },
  "Updateable": true
}
```

Update Methods

OpenBMC supports multiple update interfaces, each with specific use cases.

Redfish Update Service

The primary update method for modern deployments:

```
# Upload firmware image via HttpPushUri
curl -k -u root:openBmc \
  -X POST \
  -H "Content-Type: application/octet-stream" \
  --data-binary @obmc-phosphor-image.tar \
  https://localhost/redfish/v1/UpdateService

# Check UpdateService status
curl -k -u root:openBmc \
  https://localhost/redfish/v1/UpdateService

# Response shows update capabilities
{
  "@odata.id": "/redfish/v1/UpdateService",
  "HttpPushUri": "/redfish/v1/UpdateService",
  "HttpPushUriOptions": {
    "HttpPushUriApplyTime": {
      "ApplyTime": "OnReset"
    }
  },
  "MaxImageSizeBytes": 33554432,
  "ServiceEnabled": true
}
```

SimpleUpdate Action

For network-based updates:

```
# Update from TFTP server
curl -k -u root:openBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "TransferProtocol": "TFTP",
    "ImageURI": "tftp://192.168.1.100/firmware.tar"
  }' \
  https://localhost/redfish/v1/UpdateService/Actions/UpdateService.SimpleUpdate

# Update from HTTPS server
curl -k -u root:openBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "TransferProtocol": "HTTPS",
    "ImageURI": "https://server.example.com/firmware.tar"
  }' \
  https://localhost/redfish/v1/UpdateService/Actions/UpdateService.SimpleUpdate

# Update from SCP (requires credentials)
curl -k -u root:openBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "TransferProtocol": "SCP",
    "ImageURI": "scp://user@server.example.com/path/to/firmware.tar",
    "UserName": "user",
    "Password": "password"
  }' \
  https://localhost/redfish/v1/UpdateService/Actions/UpdateService.SimpleUpdate
```

IPMI OEM Firmware Update

Using phosphor-ipmi-flash for IPMI-based updates:

```
# IPMI OEM firmware update commands (host-side)
# Uses OEM commands to transfer firmware via IPMI

# Start firmware transfer session
ipmitool raw 0x2e 0x00 0x00 0x00 0x01 # OEM start

# Transfer data in chunks (simplified)
ipmitool raw 0x2e 0x01 <offset> <data> # OEM data

# Finalize and verify
ipmitool raw 0x2e 0x02 # OEM verify

# Activate firmware
ipmitool raw 0x2e 0x03 # OEM activate
```

phosphor-ipmi-flash Configuration

```
# Enable IPMI flash support
IMAGE_INSTALL:append = " phosphor-ipmi-flash"

# Configure supported transports
EXTRA_OEMESON:pn-phosphor-ipmi-flash = " \
  -Dlpc-type=nuvoton-lpc \
  -Dpci-bridge=true \
"
```



```
// /usr/share/ipmi-flash/config/config.json
{
  "handlers": [
    {
      "handler": "static",
      "path": "/run/initramfs/image-bmc"
    }
  ],
  "actions": [
    {
      "type": "systemd",
      "unit": "phosphor-ipmi-flash-bmc-update.target"
    }
  ]
}
```

PLDM Firmware Update (Type 5)

PLDM Type 5 firmware update for component updates:

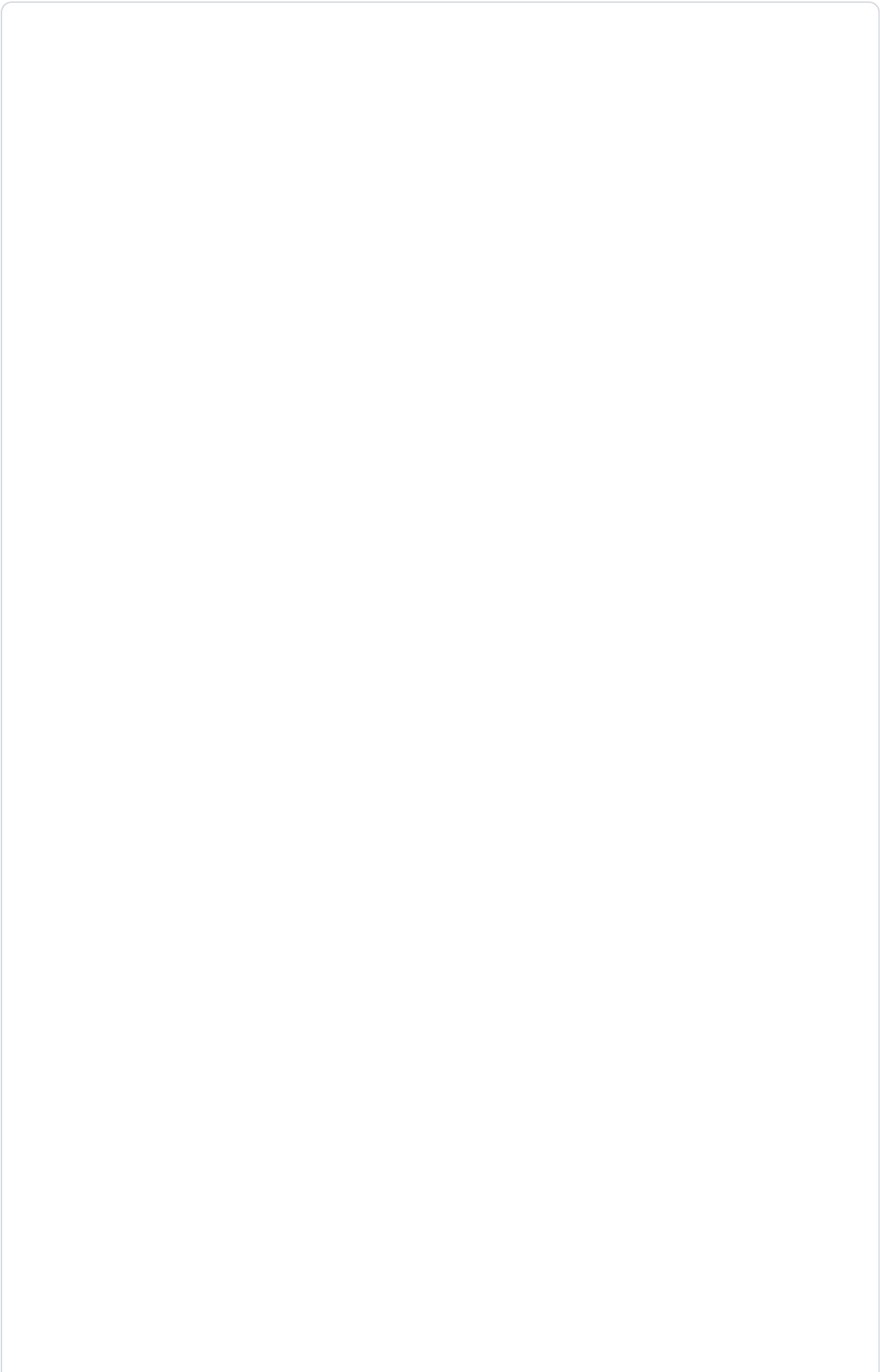
```
# Query firmware devices
pldmttool fw_update GetFwParams -m <eid>

# Request firmware update
pldmttool fw_update RequestUpdate -m <eid> \
  --max_transfer_size 4096 \
  --num_comps 1 \
  --max_outstanding_transfer_req 2 \
  --pkg_data_len 0 \
  --comp_image_set_ver "1.0.0"

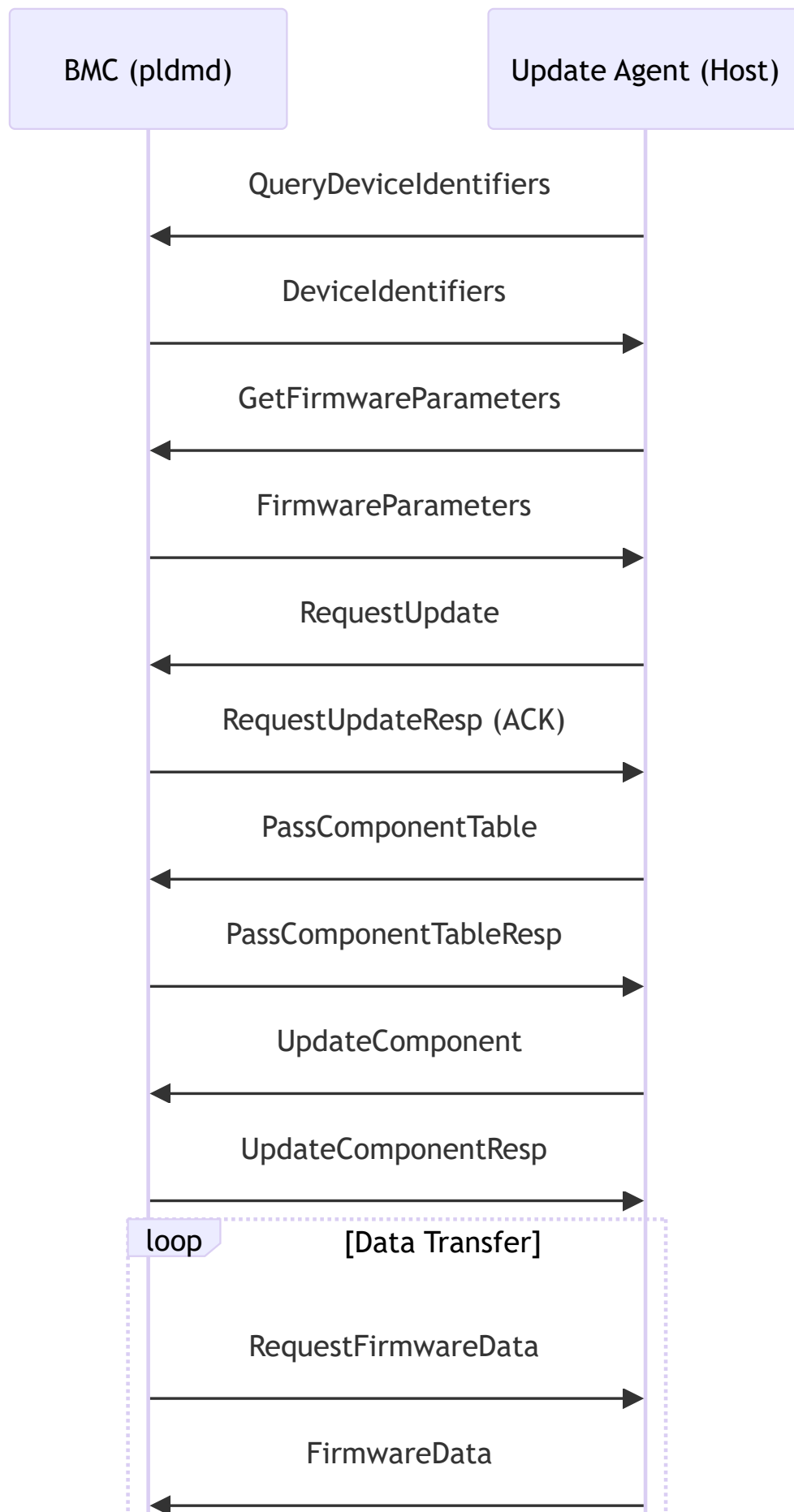
# Pass component table
pldmttool fw_update PassComponentTable -m <eid> \
  --transfer_flag 0x05 \
  --comp_classification 0x0A \
  --comp_identifier 0x0001 \
  --comp_classification_index 0 \
  --comp_comparison_stamp 0 \
  --comp_ver "1.0.0" \
  --comp_ver_str_type 1 \
  --comp_ver_str_len 5

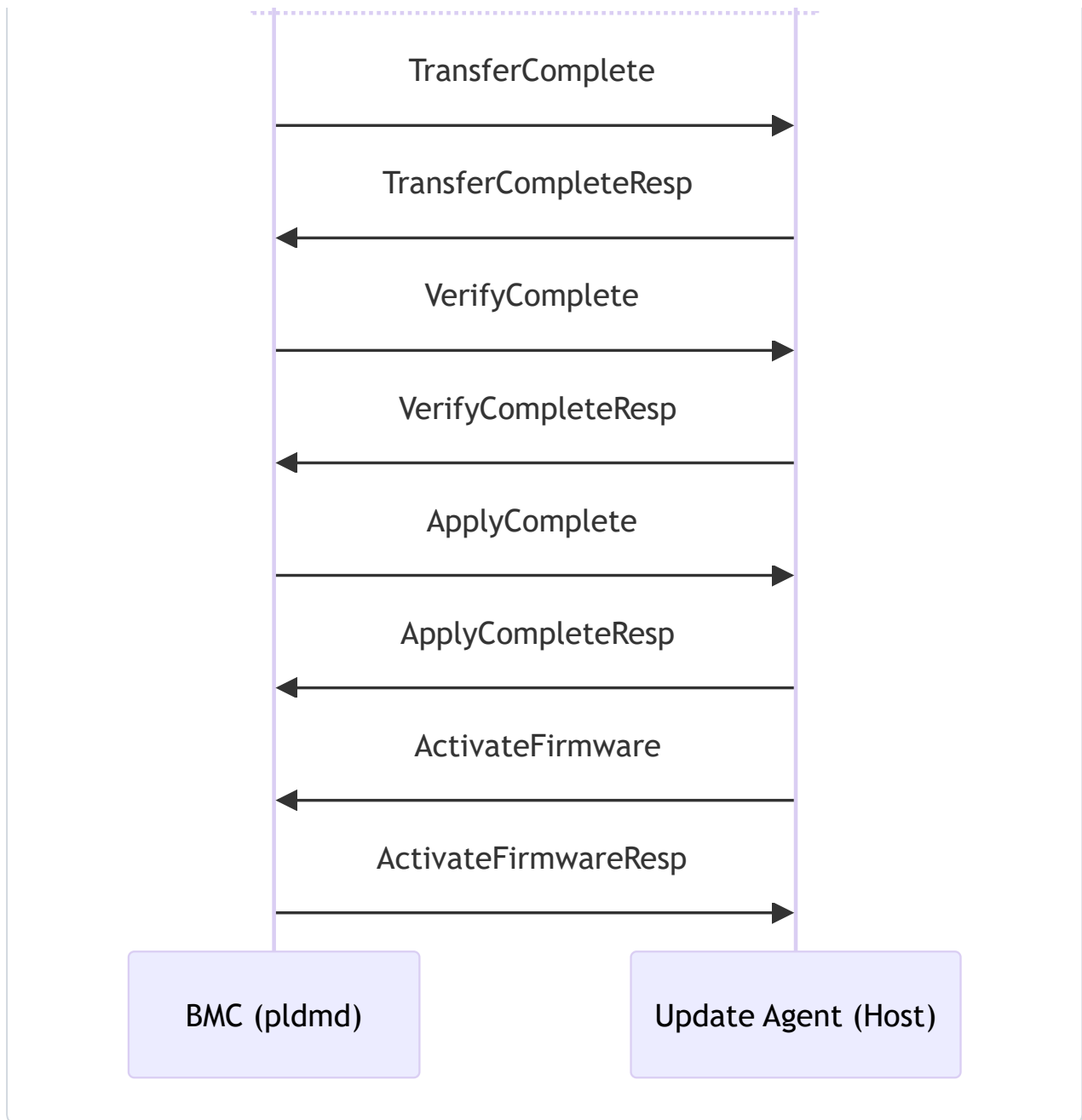
# Update component
pldmttool fw_update UpdateComponent -m <eid> \
  --comp_classification 0x0A \
  --comp_identifier 0x0001 \
  --comp_classification_index 0
```

PLDM Update Flow



PLDM Update Flow





► ASCII-art version (for comparison)

WebUI Update

1. Login to WebUI at <https://<bmc-ip>/>
2. Navigate to **Operations** → **Firmware**
3. Click **Upload firmware**
4. Select firmware file ([.tar](#) format)
5. Wait for upload and verification
6. Click **Activate** when ready
7. Optionally select **Reboot BMC after activation**

USB Code Update

```
# USB code update directory structure:
# /mnt/usb/<usb-device>/
# └─ firmware/
#     └─ image-bmc           # Firmware tarball (renamed)

# BMC-side: Check for USB device
lsblk
# NAME      MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
# sda        8:0    1   7.5G  0 disk
# └─ sda1     8:1    1   7.5G  0 part /media/usb

# Configuration in Yocto
EXTRA_OEMESON:pn-phosphor-bmc-code-mgmt:append = " -Dusb-code-update=enabled"

# USB monitor service watches for:
# - USB insertion events
# - Files matching firmware pattern
# - Triggers automatic update if policy allows
```

Command-Line Update Tools

```
# Direct flash update (recovery only)
# WARNING: Use only for recovery, bypasses safety checks

# Backup current image first
dd if=/dev/mtd0 of=/tmp/backup-kernel.bin
dd if=/dev/mtd1 of=/tmp/backup-rofs.bin

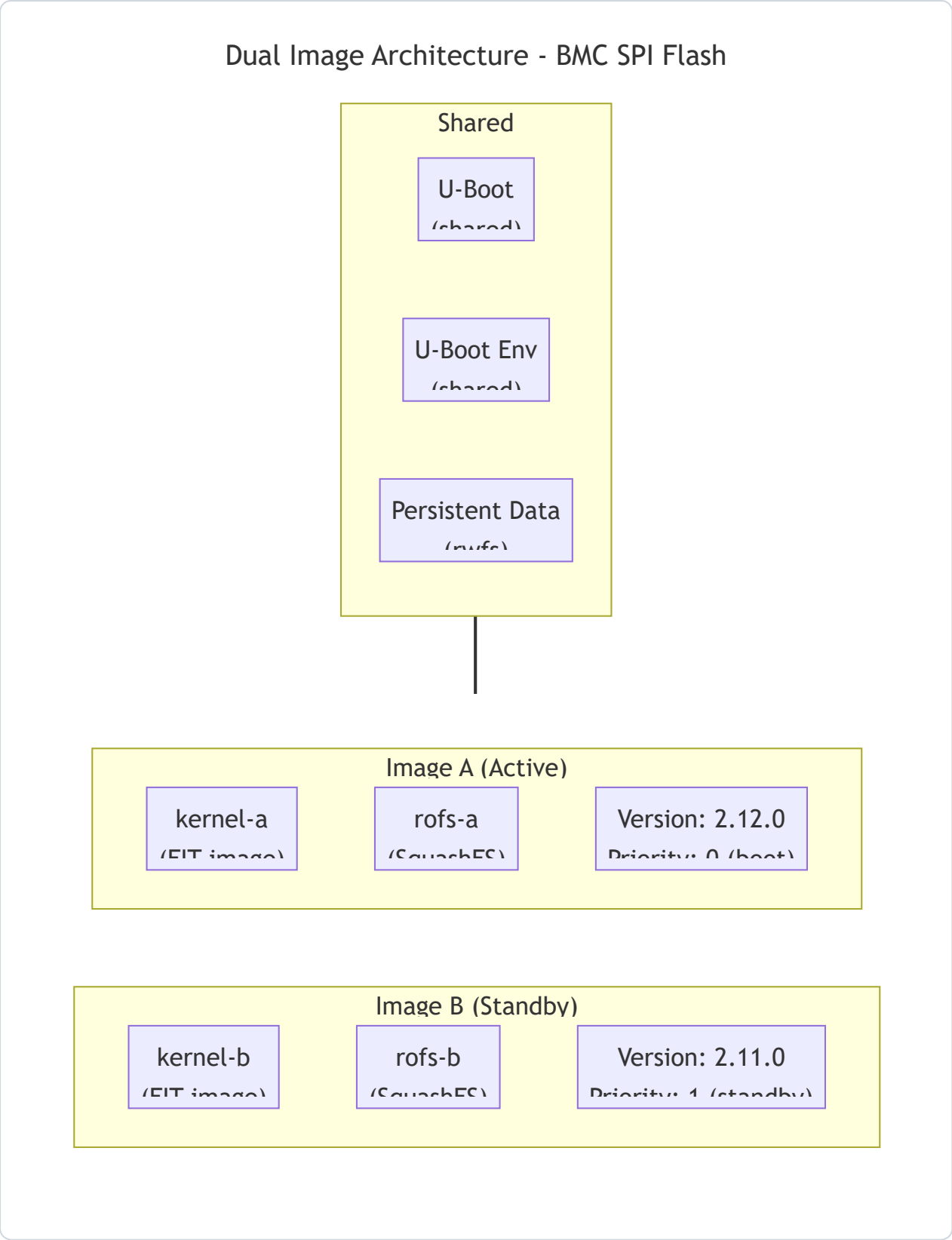
# Flash new image
flashcp -v /tmp/image-kernel /dev/mtd0
flashcp -v /tmp/image-rofs /dev/mtd1

# For UBI volumes
ubiupdatevol /dev/ubi0_0 /tmp/image-kernel
ubiupdatevol /dev/ubi0_1 /tmp/image-rofs
```

Dual Image Support

OpenBMC implements A/B redundant boot for high availability.

Newer phosphor-bmc-code-mgmt builds ship a [side-switch](#) utility that compares the running image's priority against every available image; if a higher-priority image exists it powers the host off, arms a one-time auto-power-on, and reboots the BMC so it comes up on the preferred side. Image signature verification ([Signature](#) , OpenSSL) can additionally enforce **post-quantum** algorithms and a **Minimum Ship Level (MSL)** check that blocks flashing a version below a configured baseline. (Verify against your phosphor-bmc-code-mgmt version.)



► ASCII-art version (for comparison)

View Images via Redfish

```
# List all firmware
curl -k -u root:openBmc \
  https://localhost/redfish/v1/UpdateService/FirmwareInventory

# Response
{
  "Members": [
    {"@odata.id": "/redfish/v1/UpdateService/FirmwareInventory/bmc_active"},
    {"@odata.id": "/redfish/v1/UpdateService/FirmwareInventory/abc12345"}
  ]
}

# Get active image details
curl -k -u root:openBmc \
  https://localhost/redfish/v1/UpdateService/FirmwareInventory/bmc_active
```

View Images via D-Bus

```
# List software versions
busctl tree xyz.openbmc_project.Software.BMC.Updater
# /xyz/openbmc_project/software
# └─ /xyz/openbmc_project/software/abc12345 (active)
# └─ /xyz/openbmc_project/software/def67890 (standby)

# Get version info
busctl get-property xyz.openbmc_project.Software.BMC.Updater \
  /xyz/openbmc_project/software/abc12345 \
  xyz.openbmc_project.Software.Version Version
# s "2.12.0"

# Get activation state
busctl get-property xyz.openbmc_project.Software.BMC.Updater \
  /xyz/openbmc_project/software/abc12345 \
  xyz.openbmc_project.Software.Activation Activation
# s "xyz.openbmc_project.Software.Activation.Activations.Active"

# Get boot priority (0 = primary, higher = lower priority)
busctl get-property xyz.openbmc_project.Software.BMC.Updater \
  /xyz/openbmc_project/software/abc12345 \
  xyz.openbmc_project.Software.RedundancyPriority Priority
# u 0
```

Set Boot Priority

```
# Set image as primary (priority 0)
busctl set-property xyz.openbmc_project.Software.BMC.Updater \
  /xyz/openbmc_project/software/def67890 \
  xyz.openbmc_project.Software.RedundancyPriority \
  Priority u 0

# Via Redfish - activate backup firmware
curl -k -u root:openBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{}' \
  https://localhost/redfish/v1/UpdateService/FirmwareInventory/def67890/Actions/SoftwareInventory.Activate
```

U-Boot Environment & Boot Control

Key U-Boot Environment Variables

```
# View all U-Boot environment variables
fw_printenv

# Key variables for firmware update:

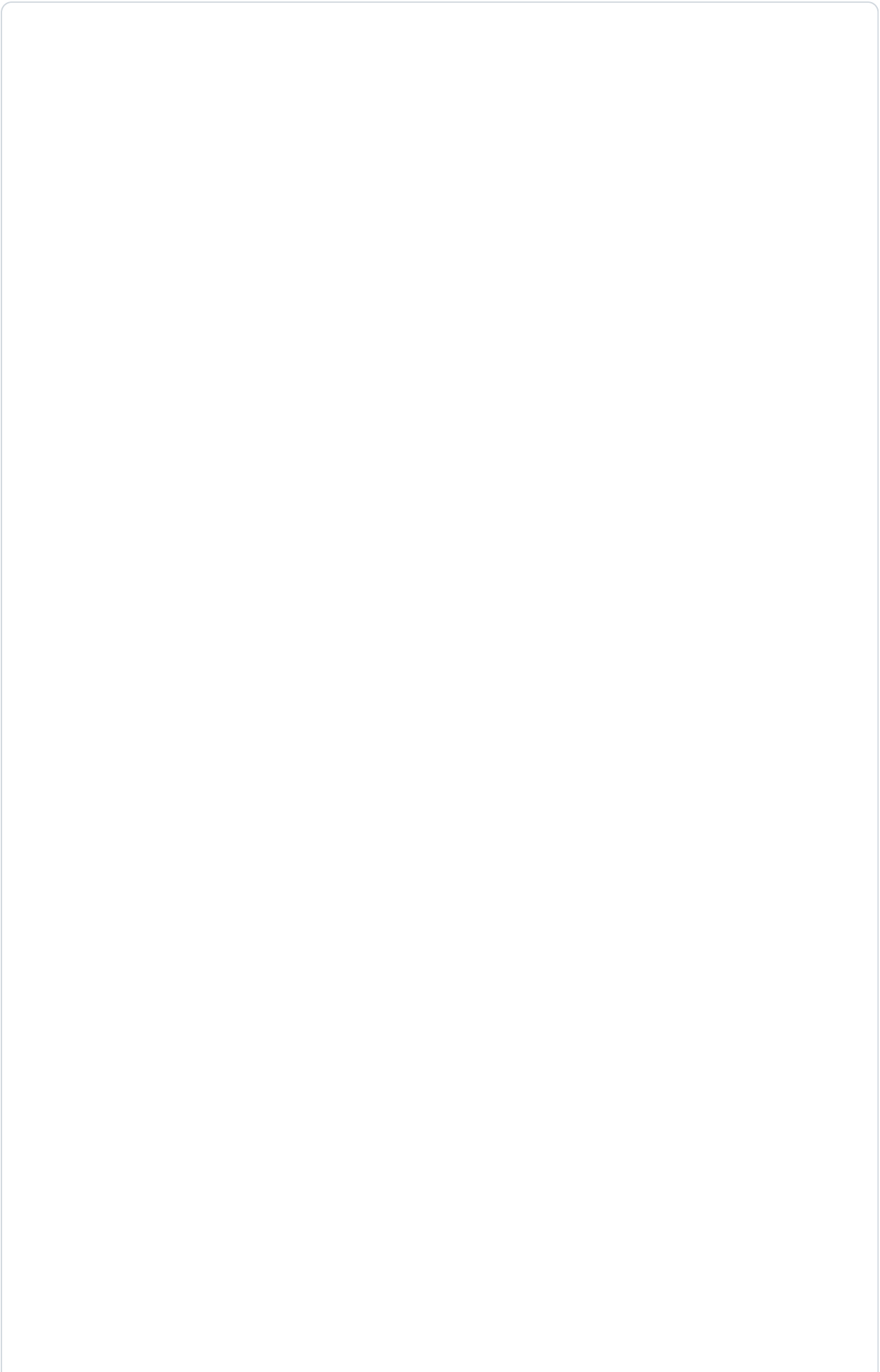
# bootside - Which image to boot (a or b)
fw_printenv bootside
# bootside=a

# bootcount - Boot attempt counter
fw_printenv bootcount
# bootcount=0

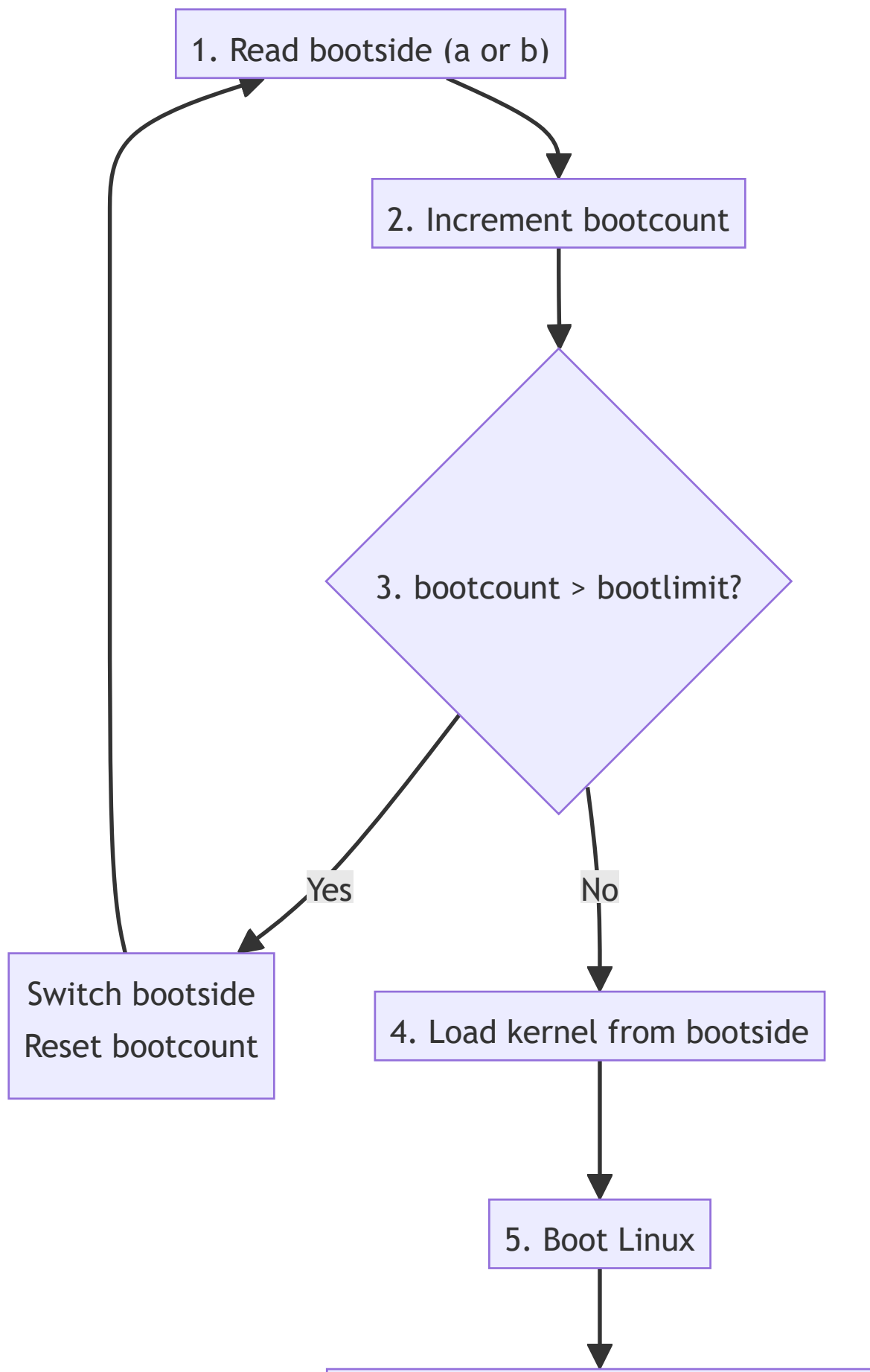
# bootlimit - Max boot attempts before fallback
fw_printenv bootlimit
# bootlimit=3

# kernelname - Kernel partition name
fw_printenv kernelname
# kernelname=kernel-a

# rofsname - Root filesystem partition
fw_printenv rofsname
# rofsname=rofs-a
```

U-Boot Boot Flow



6. User-space resets bootcount to 0 (via init script or systemd service)

► ASCII-art version (for comparison)

Manual Boot Control

```
# Force boot from specific side
fw_setenv bootside b
reboot

# Reset boot counter (call after successful boot)
fw_setenv bootcount 0

# Increase boot limit for debugging
fw_setenv bootlimit 5

# One-time boot override (doesn't change bootside)
fw_setenv bootonce b
reboot

# View boot arguments
fw_printenv bootargs
# bootargs=console=ttyS4,115200 root=/dev/mtdblock3 rootfstype=squashfs
```

Recovery via Serial Console

If boot fails and you have serial access:

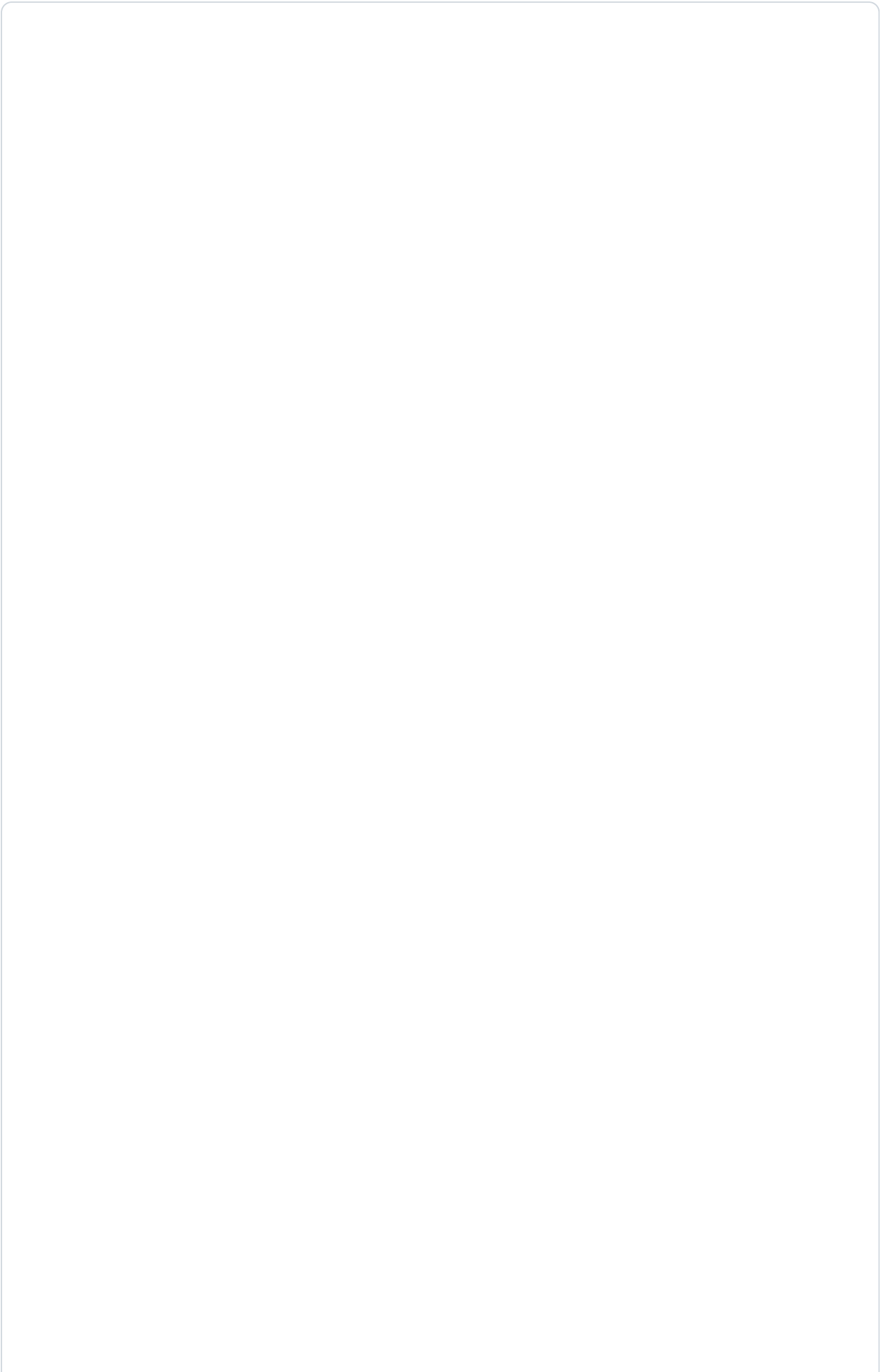
```
# At U-Boot prompt (interrupt boot with key press)
=> printenv bootside
bootside=a

# Switch to other image
=> setenv bootside b
=> saveenv
=> reset

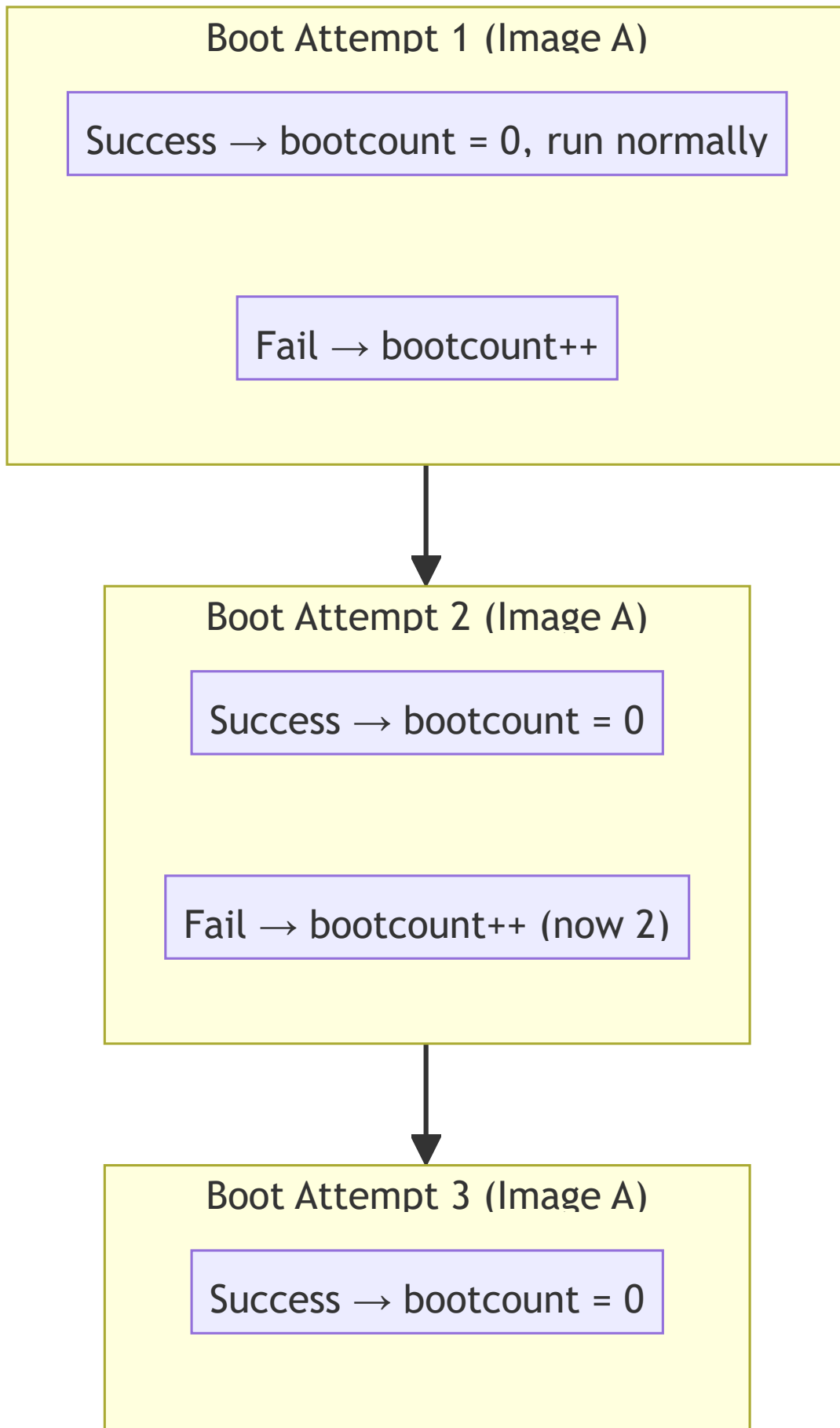
# Or boot directly without saving
=> setenv bootside b
=> run bootcmd
```

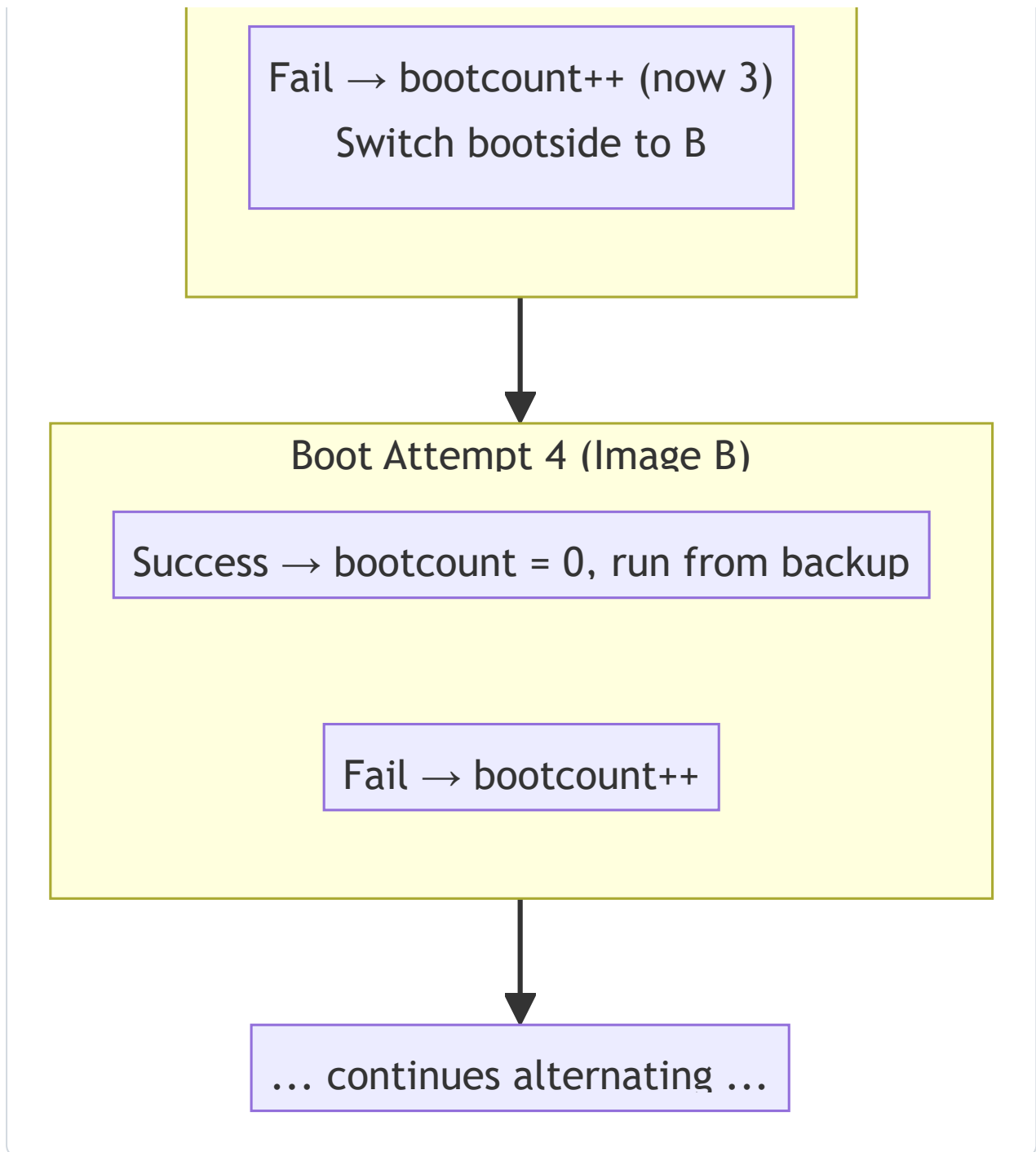
Failsafe Behavior

Automatic Boot Fallback



Boot Failure Recovery





► ASCII-art version (for comparison)

Boot Health Check Service

```
# phosphor-software-manager runs boot health check
# On successful boot, clears boot counter

# Check service status
systemctl status obmc-flash-bmc-setenv@bootcount

# Manual reset
fw_setenv bootcount 0

# View boot health log
journalctl -u obmc-flash-bmc-setenv@bootcount
```


Recovery Mode

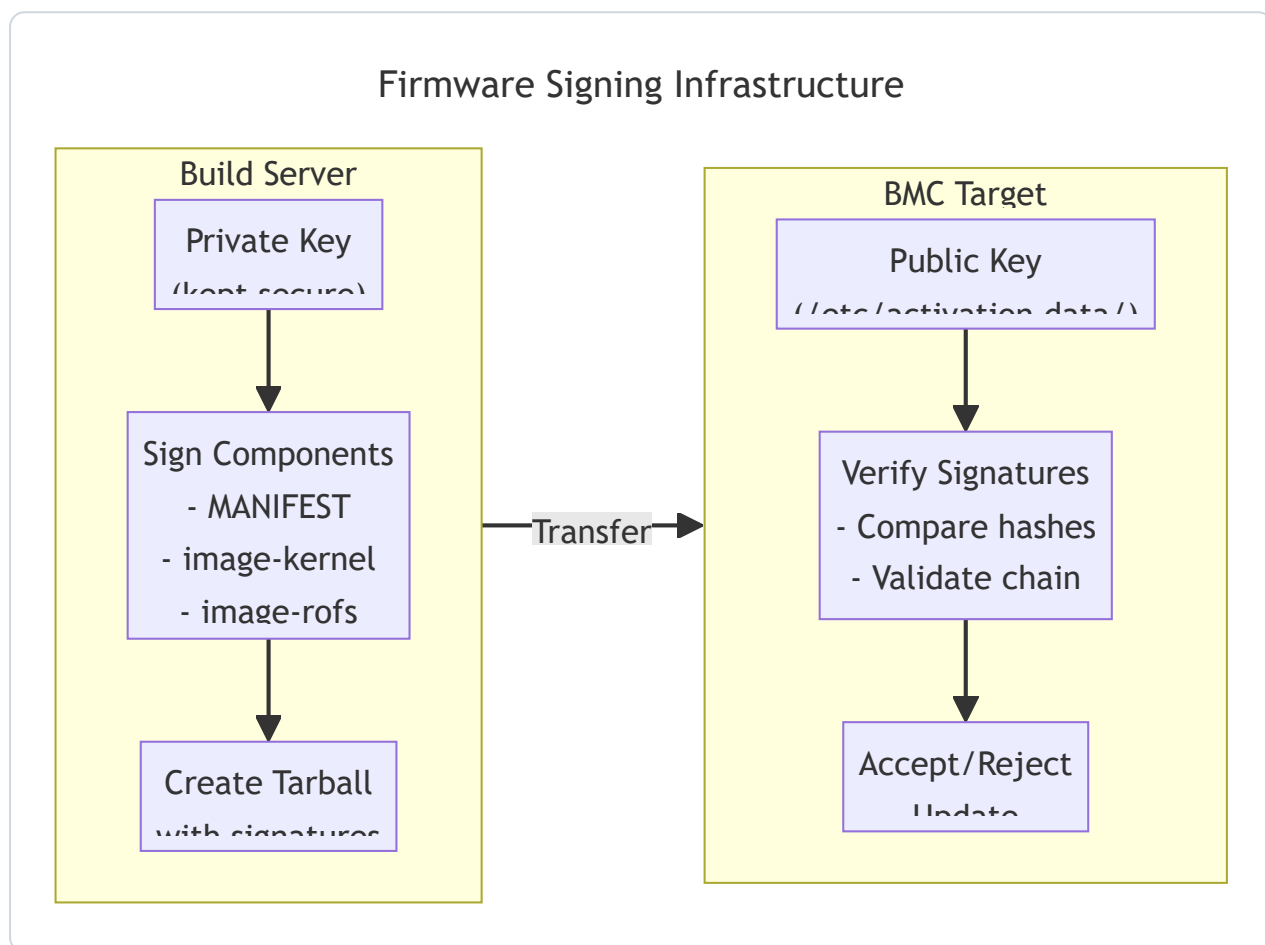
```
# Force recovery to backup image
fw_setenv bootside b
reboot

# Emergency: Use U-Boot to load recovery image from TFTP
# At U-Boot prompt:
=> setenv serverip 192.168.1.100
=> setenv ipaddr 192.168.1.50
=> tftp 0x83000000 recovery-image.bin
=> bootm 0x83000000

# Full factory recovery via static.mtd image
# At U-Boot prompt:
=> sf probe
=> tftp 0x83000000 obmc-phosphor-image.static.mtd
=> sf update 0x83000000 0x0 ${filesize}
=> reset
```

Firmware Signing & Verification

Signing Infrastructure Overview



► ASCII-art version (for comparison)

Generate Signing Keys

```
# Create 4096-bit RSA keypair for production
openssl genrsa -out private.pem 4096
openssl rsa -in private.pem -pubout -out publickey

# Create self-signed certificate (for key identification)
openssl req -new -x509 -key private.pem -out cert.pem -days 3650 \
    -subj "/CN=OpenBMC Firmware Signing/O=My Organization"

# View key information
openssl rsa -in publickey -pubin -text -noout

# For development/testing (weaker key)
openssl genrsa -out dev-private.pem 2048
openssl rsa -in dev-private.pem -pubout -out dev-publickey
```

Sign Firmware Components

```
# Sign each component with SHA-256 RSA signature
for file in MANIFEST image-kernel image-rofs image-rwfs image-u-boot; do
    if [ -f "$file" ]; then
        openssl dgst -sha256 -sign private.pem -out ${file}.sig ${file}
        echo "Signed: $file"
    fi
done

# Create MANIFEST with signature metadata
cat > MANIFEST << 'EOF'
purpose=xyz.openbmc_project.Software.Version.VersionPurpose.BMC
version=2.12.0
MachineName=romulus
KeyType=OpenBMC
HashType=RSA-SHA256
EOF

# Sign the MANIFEST itself
openssl dgst -sha256 -sign private.pem -out MANIFEST.sig MANIFEST

# Create signed tarball
tar -cvf obmc-phosphor-image-signed.tar \
    MANIFEST MANIFEST.sig \
    publickey \
    image-kernel image-kernel.sig \
    image-rofs image-rofs.sig \
    image-rwfs image-rwfs.sig \
    image-u-boot image-u-boot.sig
```

Configure Verification in Yocto

```
# In your distro or local.conf

# Enable signature verification
EXTRA_OEMESON:pn-phosphor-bmc-code-mgmt:append = " -Dverify-signature=enabled"

# Configure public key provider
PREFERRED_PROVIDER_virtual/phosphor-software-manager-system-public-key = \
    "phosphor-software-manager-public-key-${MACHINE}"

# Key installation location
# Keys installed to: /etc/activationdata/

# Custom key type name
SIGNING_KEY_TYPE = "OpenBMC"
```

Create a recipe for your public key:

```
# meta-mymachine/recipes-phosphor/software/phosphor-software-manager-public-key-mymachine.bb

SUMMARY = "Public key for firmware verification"
LICENSE = "Apache-2.0"
LIC_FILES_CHKSUM = "file://${COREBASE}/meta/files/common-licenses/Apache-2.0;md5=..."

inherit allarch

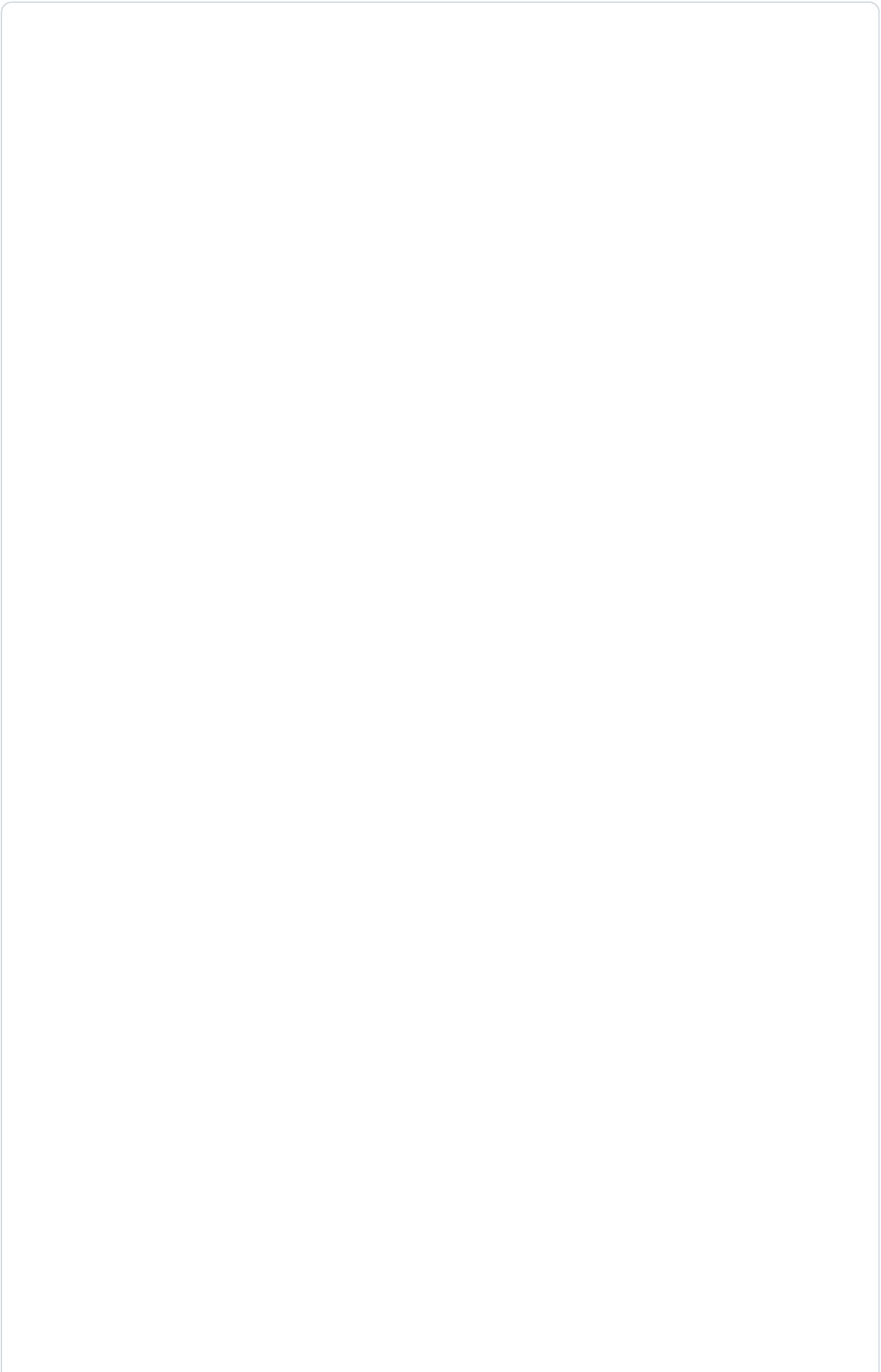
SRC_URI = "file://publickey"

do_install() {
    install -d ${D}${sysconfdir}/activationdata
    install -m 0644 ${WORKDIR}/publickey \
        ${D}${sysconfdir}/activationdata/key-OpenBMC
}

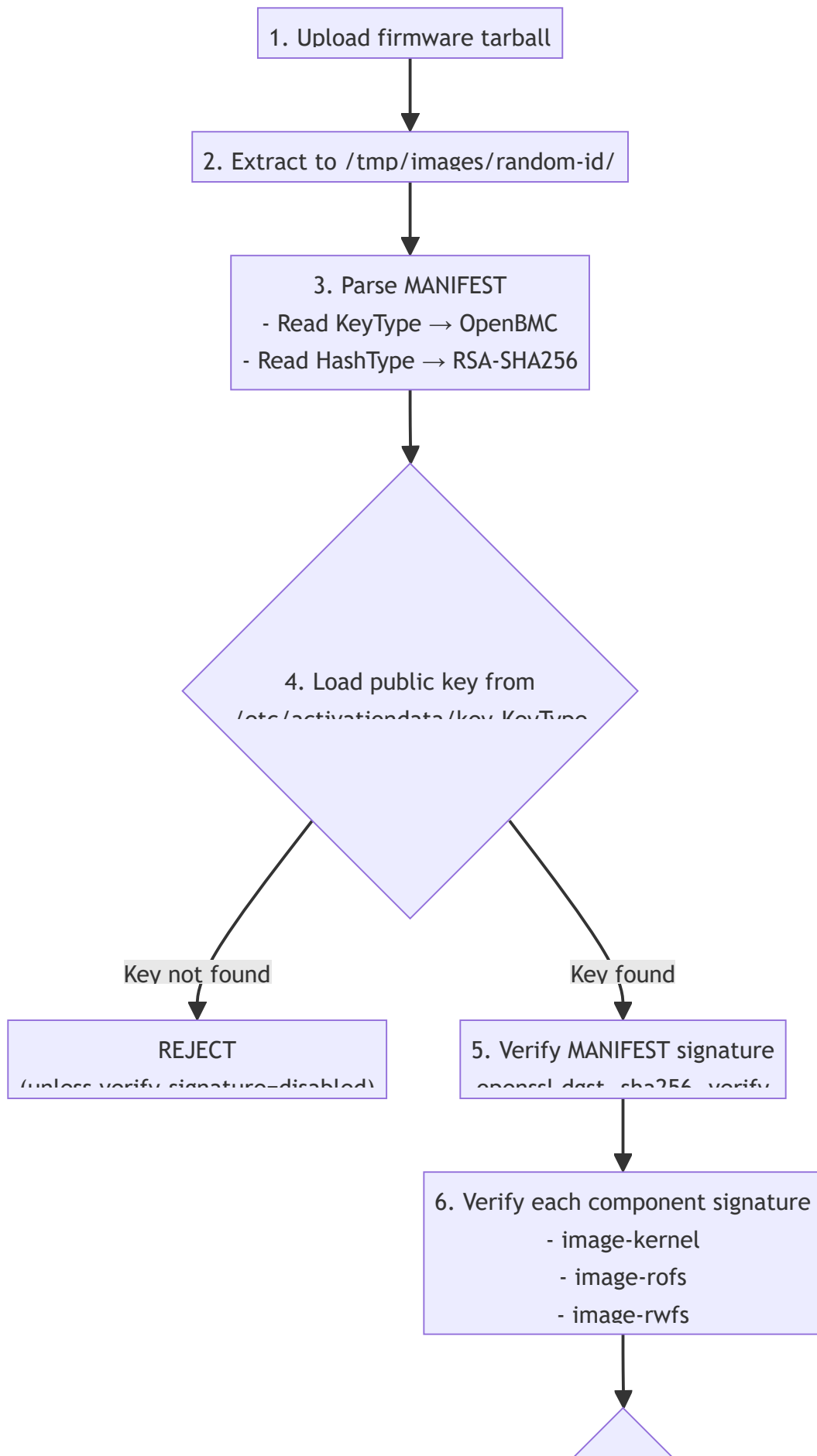
FILES:${PN} = "${sysconfdir}/activationdata"

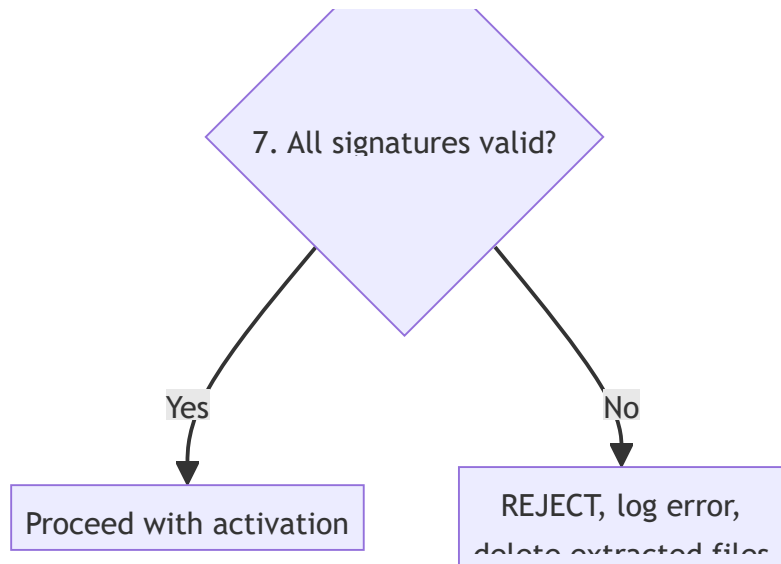
RPROVIDES:${PN} = "virtual/phosphor-software-manager-system-public-key"
```

Verification Flow



Signature Verification Flow





► ASCII-art version (for comparison)

Manual Signature Verification

```
# On the BMC, manually verify an image

# Extract tarball
mkdir -p /tmp/verify-test
cd /tmp/verify-test
tar -xvf /tmp/firmware.tar

# Check MANIFEST
cat MANIFEST

# Verify MANIFEST signature
openssl dgst -sha256 -verify /etc/activationdata/key-OpenBMC \
  -signature MANIFEST.sig MANIFEST

# Verify component signatures
for file in image-kernel image-rofs image-rwfs image-u-boot; do
  if [ -f "$file" ] && [ -f "${file}.sig" ]; then
    echo -n "Verifying $file: "
    openssl dgst -sha256 -verify /etc/activationdata/key-OpenBMC \
      -signature ${file}.sig ${file}
  fi
done
```

Signature Verification Logs

```
# Check verification results
journalctl -u phosphor-software-manager | grep -i -E "(signature|verify|key)"

# Example output for successful verification:
# phosphor-software-manager[1234]: Signature verification passed for image abc12345
# phosphor-software-manager[1234]: Image abc12345 is valid

# Example output for failed verification:
# phosphor-software-manager[1234]: Signature verification failed for MANIFEST
# phosphor-software-manager[1234]: Error: Invalid signature
# phosphor-software-manager[1234]: Rejecting image def67890
```

Key Type Configuration

Support multiple key types for different environments:

```
# /etc/activationdata/ structure
/etc/activationdata/
├─ key-OpenBMC          # Development keys
├─ key-ProductionKey    # Production keys
└─ key-TestKey          # Test/staging keys

# MANIFEST selects which key to use:
# KeyType=ProductionKey → uses /etc/activationdata/key-ProductionKey
```

Host Firmware Update

BIOS/UEFI Update (Intel/AMD)

```
# Upload BIOS image via Redfish
curl -k -u root:openBmc \
  -X POST \
  -H "Content-Type: application/octet-stream" \
  --data-binary @bios.bin \
  https://localhost/redfish/v1/UpdateService

# Check BIOS firmware inventory
curl -k -u root:openBmc \
  https://localhost/redfish/v1/UpdateService/FirmwareInventory/bios

# Response
{
  "@odata.id": "/redfish/v1/UpdateService/FirmwareInventory/bios",
  "Id": "bios",
  "Name": "BIOS Firmware",
  "Version": "1.23.0",
  "Updateable": true,
  "Status": {
    "State": "Enabled",
    "Health": "OK"
  }
}
```

Intel SPS/ME Firmware

```
# Query ME firmware version
curl -k -u root:openBmc \
  https://localhost/redfish/v1/UpdateService/FirmwareInventory/me

# ME update typically requires specific tools
# Coordinated with BIOS update
```


PNOR Update (OpenPOWER)

```
# Upload PNOR tarball
curl -k -u root:OpenBmc \
  -X POST \
  -H "Content-Type: application/octet-stream" \
  --data-binary @pnor.tar \
  https://localhost/redfish/v1/UpdateService

# PNOR image structure
pnor.tar
├── MANIFEST           # purpose=Host, version, MachineName
├── HOST_FIRMWARE/
│   ├── HBB           # Hostboot Base
│   ├── HBI           # Hostboot Image
│   ├── HBRT          # Hostboot Runtime
│   ├── PAYLOAD        # Skiboot (OPAL)
│   └── BOOTKERNEL     # Linux kernel
└── *.sig             # Signatures
```

openpower-pnor-code-mgmt

```
# View PNOR partitions
pflash -i

# Read specific partition
pflash -P HBB -r /tmp/hbb.bin

# Services involved
systemctl status openpower-update-bios@0.service
```

PSU Firmware Update

```
# List PSU firmware
curl -k -u root:OpenBmc \
  https://localhost/redfish/v1/UpdateService/FirmwareInventory \
  | jq '.Members[] | select(.["@odata.id"] | contains("psu"))'

# PSU update via PMBUS or I2C
# Requires specific PSU support in OpenBMC
```

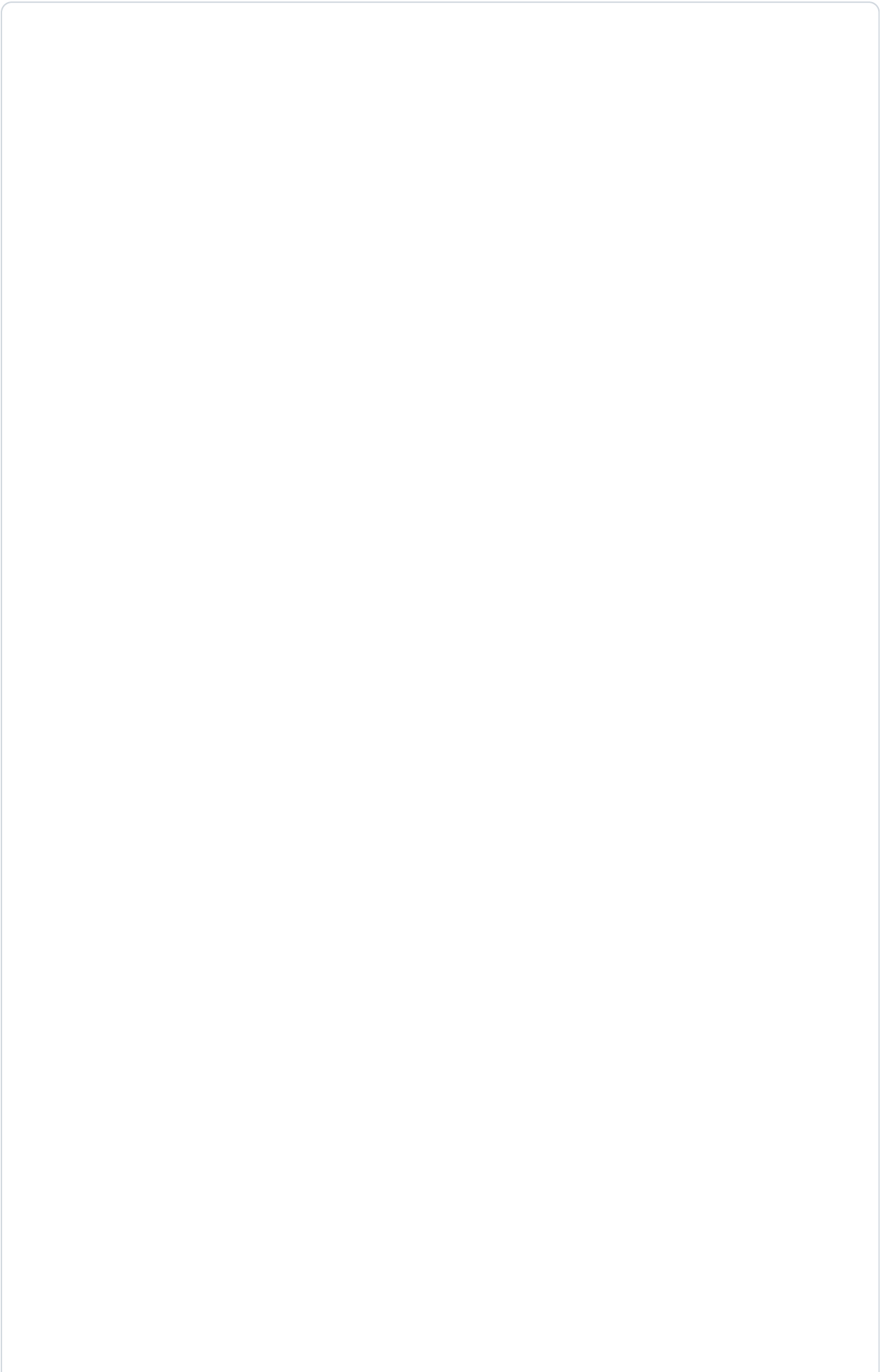
Component-Level Update (PLDM)

```
# Use pldmtool for component updates (VR, CPLD, etc.)
# Get supported components
pldmtool fw_update GetFwParams -m <eid>

# Component classifications:
# 0x00 - Unknown
# 0x01 - Other
# 0x02 - Driver
# 0x03 - Configuration
# 0x04 - Application
# 0x05 - Instrumentation
# 0x06 - Firmware/BIOS
# 0x07 - Diagnostic
# 0x0A - Firmware
# 0x0B - CPLD/FPGA
# 0x0C - VR
```

Update Workflow Deep Dive

Complete Update Sequence



Firmware Update Workflow

1. UPLOAD

Receive via Redfish/IPMI/PLDM

Store in /tmp/images/random-id/

Create Software.Version D-Bus object



2. EXTRACT & PARSE

Extract tarball contents

Parse MANIFEST file

Validate MachineName

Check version purpose



3. VERIFY (if enabled)

Load public key

Verify MANIFEST signature

Verify component signatures

REJECT if any fails



4. READY

Set Activation = Ready

Create RedundancyPriority

Wait for activation



5. ACTIVATING

Set Activation = Activating

Erase standby partition

Write image to flash

write images to flash

Update U-Boot variables

6. ACTIVE

Set Activation = Active

Set Priority = 0

Optional: trigger reboot

7. VERIFY (post-reboot)

Boot new image

Reset bootcount to 0

Log successful update

► ASCII-art version (for comparison)

Activation States

State	Description
NotReady	Image uploaded but not yet processed
Invalid	Image validation failed
Ready	Image verified, waiting for activation
Activating	Write in progress to flash
Active	Image is installed and bootable
Failed	Activation failed

Monitor Update Progress

```
# Via Redfish Task Service
curl -k -u root:0penBmc \
  https://localhost/redfish/v1/TaskService/Tasks

# Get specific task
curl -k -u root:0penBmc \
  https://localhost/redfish/v1/TaskService/Tasks/1

# Task states: New, Starting, Running, Completed, Exception

# Via D-Bus monitoring
busctl monitor xyz.openbmc_project.Software.BMC.Updater

# Watch activation progress
busctl get-property xyz.openbmc_project.Software.BMC.Updater \
  /xyz/openbmc_project/software/<id> \
  xyz.openbmc_project.Software.Activation Activation

# Watch for ActivationProgress (percentage)
busctl get-property xyz.openbmc_project.Software.BMC.Updater \
  /xyz/openbmc_project/software/<id> \
  xyz.openbmc_project.Software.ActivationProgress Progress
```

Progress Indication

```
# ActivationProgress interface shows percentage
# 0-10%: Extracting
# 10-20%: Verifying signatures
# 20-40%: Erasing flash
# 40-90%: Writing flash
# 90-100%: Finalizing

# Via journald
journalctl -u phosphor-software-manager@bmc -f
# [date] Progress: 25%
# [date] Writing kernel partition...
# [date] Progress: 50%
# [date] Writing rootfs partition...
# [date] Progress: 90%
# [date] Activation complete
```

Configuration Options

Update Settings

```
# Configure via Redfish
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{
    "HttpPushUriOptions": {
      "HttpPushUriApplyTime": {
        "ApplyTime": "Immediate"
      }
    }
  }' \
  https://localhost/redfish/v1/UpdateService
```

Apply Time Options

Option	Description
Immediate	Apply and reboot immediately
OnReset	Apply on next BMC reboot
AtMaintenanceWindowStart	Schedule for maintenance window

Troubleshooting

Upload Fails

```
# Check available space in /tmp
df -h /tmp
# Minimum required: image size + extraction overhead (~2x image size)

# Check bmcweb body size limit
# Default is typically 30MB, configurable in bmcweb
cat /etc/bmcweb_persistent_data.json | jq .http_body_limit

# Verify tarball integrity
file firmware.tar
# firmware.tar: POSIX tar archive (GNU)

tar -tvf firmware.tar
# -rw-r--r-- 0/0      123 2024-01-15 10:00 MANIFEST
# -rw-r--r-- 0/0 4194304 2024-01-15 10:00 image-kernel
# -rw-r--r-- 0/0 20971520 2024-01-15 10:00 image-rofs

# Check for required files
tar -tf firmware.tar | grep -E "^(MANIFEST|image-)"

# Common upload errors:
# - "413 Request Entity Too Large" → Image too big
# - "503 Service Unavailable" → Service overloaded
# - Connection reset → Timeout or memory issue
```

Signature Verification Failed

```
# Check installed public keys
ls -la /etc/activationdata/
# -rw-r--r-- 1 root root 800 Jan 15 10:00 key-OpenBMC

# Check MANIFEST KeyType matches installed key
tar -xOf firmware.tar MANIFEST | grep KeyType
# KeyType=OpenBMC
# → Needs /etc/activationdata/key-OpenBMC

# Verify signature manually
cd /tmp
tar -xf firmware.tar
openssl dgst -sha256 -verify /etc/activationdata/key-OpenBMC \
    -signature MANIFEST.sig MANIFEST
# Verified OK or Verification Failure

# Common signature errors:
# - "Public key not found" → Missing key file
# - "Verification Failure" → Wrong key or corrupted signature
# - "HashType mismatch" → MANIFEST HashType doesn't match key
```

Machine Name Mismatch

```
# Check current machine name
cat /etc/os-release | grep OPENBMC_TARGET_MACHINE
# OPENBMC_TARGET_MACHINE=romulus

# Check MANIFEST MachineName
tar -xOf firmware.tar MANIFEST | grep MachineName
# MachineName=romulus

# If mismatch, image is rejected
# Error: "Machine name doesn't match"
```

Boot Fails After Update

```
# Via serial console, check for:
# - Kernel panic
# - Filesystem mount failure
# - Init system failure

# Check U-Boot boot counter
fw_printenv bootcount
# If > 0, boot had issues

# Force boot to backup image
fw_setenv bootside b
fw_setenv bootcount 0
reboot

# Or at U-Boot prompt:
=> setenv bootside b
=> saveenv
=> reset

# If both images fail, use TFTP recovery:
=> setenv serverip 192.168.1.100
=> setenv ipaddr 192.168.1.50
=> tftp 0x80000000 obmc-phosphor-image.static.mtd
=> sf probe
=> sf update 0x80000000 0 ${filesize}
=> reset
```

Image Not Activating

```
# Check activation state
busctl get-property xyz.openbmc_project.Software.BMC.Updater \
    /xyz/openbmc_project/software/<id> \
    xyz.openbmc_project.Software.Activation Activation

# States:
# NotReady - Still processing
# Invalid - Validation failed
# Ready - Can be activated
# Activating - In progress
# Active - Successfully activated
# Failed - Activation failed

# Check for activation blockers
busctl get-property xyz.openbmc_project.Software.BMC.Updater \
    /xyz/openbmc_project/software/<id> \
    xyz.openbmc_project.Software.Activation RequestedActivation

# View detailed logs
journalctl -u phosphor-software-manager@bmc --since "5 minutes ago"

# Common activation issues:
# - Flash busy/locked
# - Insufficient space
# - Filesystem corruption
# - Service crash during write
```

Flash Write Errors

```
# Check MTD device status
cat /proc/mtd
mtd_debug info /dev/mtd0

# Check for bad blocks (UBI)
ubinfore -a
cat /sys/class/ubi/ubi0/bad_peb_count

# Manual flash test (DESTRUCTIVE)
# flashcp -v /dev/zero /dev/mtd0 # DO NOT RUN in production

# Check for flash busy
flock -n /run/lock/bmc_flash.lock echo "Flash not locked"
# If no output, flash is locked by another process

# View flash operations
journalctl | grep -i "mtd\|flash\|ubi"
```

Activation Progress Stuck

```
# Check if activation is stuck at specific percentage
busctl get-property xyz.openbmc_project.Software.BMC.Updater \
    /xyz/openbmc_project/software/<id> \
    xyz.openbmc_project.Software.ActivationProgress Progress

# Stuck at 0%: MANIFEST parsing or signature verification
# Stuck at 20%: Flash erase
# Stuck at 50%: Flash write
# Stuck at 90%: Finalization

# Check for service crash
systemctl status phosphor-software-manager@bmc
# If failed, restart and retry

# Force cleanup and retry
rm -rf /tmp/images/*
systemctl restart phosphor-software-manager@bmc
```

Delete Failed/Stale Images

```
# List all software objects
busctl tree xyz.openbmc_project.Software.BMC.Updater

# Delete specific image via D-Bus
busctl call xyz.openbmc_project.Software.BMC.Updater \
    /xyz/openbmc_project/software/<id> \
    xyz.openbmc_project.Object.Delete Delete

# Or via Redfish
curl -k -u root:0penBmc -X DELETE \
    https://localhost/redfish/v1/UpdateService/FirmwareInventory/<id>

# Clean up /tmp/images manually if needed
rm -rf /tmp/images/<id>
```

Network Update Issues

```
# TFTP update fails
# Check network connectivity
ping <tftp-server>

# Check TFTP server is running
tftp <tftp-server>
tftp> get firmware.tar
tftp> quit

# Check firewall
# TFTP uses UDP port 69

# HTTPS update fails
# Check certificate trust
curl -v https://<server>/firmware.tar

# Check proxy settings
echo $http_proxy $https_proxy
```

Rollback

Rollback via D-Bus

```
# List all images to find backup ID
busctl tree xyz.openbmc_project.Software.BMC.Updater

# Get version info for each image
for obj in $(busctl tree xyz.openbmc_project.Software.BMC.Updater --list | grep software/); do
    echo "=== $obj ==="
    busctl get-property xyz.openbmc_project.Software.BMC.Updater \
        "$obj" xyz.openbmc_project.Software.Version Version 2>/dev/null
    busctl get-property xyz.openbmc_project.Software.BMC.Updater \
        "$obj" xyz.openbmc_project.Software.RedundancyPriority Priority 2>/dev/null
done

# Activate backup image (set as primary)
busctl call xyz.openbmc_project.Software.BMC.Updater \
    /xyz/openbmc_project/software/<backup_id> \
    xyz.openbmc_project.Software.Activation \
    RequestedActivation s \
    "xyz.openbmc_project.Software.Activation.RequestedActivations.Active"

# Or set priority directly (0 = primary)
busctl set-property xyz.openbmc_project.Software.BMC.Updater \
    /xyz/openbmc_project/software/<backup_id> \
    xyz.openbmc_project.Software.RedundancyPriority \
    Priority u 0

# Reboot BMC to activate
obmcutil bmcreboot
```

Rollback via Redfish

```
# List all firmware inventory
curl -k -u root:openBmc \
  https://localhost/redfish/v1/UpdateService/FirmwareInventory | jq

# Find backup image (non-active BMC image)
curl -k -u root:openBmc \
  https://localhost/redfish/v1/UpdateService/FirmwareInventory/<backup_id>

# Activate backup (method varies by implementation)
# Some implementations use PATCH:
curl -k -u root:openBmc -X PATCH \
  -H "Content-Type: application/json" \
  -d '{"Status": {"State": "Enabled"}}' \
  https://localhost/redfish/v1/UpdateService/FirmwareInventory/<backup_id>

# Others use Actions:
curl -k -u root:openBmc -X POST \
  https://localhost/redfish/v1/UpdateService/FirmwareInventory/<backup_id>/Actions/SoftwareInventory.Activate

# Reboot via Redfish
curl -k -u root:openBmc -X POST \
  -d '{"ResetType": "GracefulRestart"}' \
  https://localhost/redfish/v1/Managers/bmc/Actions/Manager.Reset
```

Rollback via U-Boot

```
# If Linux is accessible, use fw_setenv
fw_setenv bootside b
reboot

# At U-Boot prompt (serial console)
=> printenv bootside
bootside=a
=> setenv bootside b
=> saveenv
=> reset
```

Factory Reset

Reset BMC Configuration

```
# Via Redfish - reset to factory defaults
curl -k -u root:openBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{"ResetType": "ResetAll"}' \
  https://localhost/redfish/v1/Managers/bmc/Actions/Manager.ResetToDefaults

# Reset types:
# - ResetAll: Reset all settings to factory defaults
# - PreserveNetworkAndUsers: Keep network and user settings
# - PreserveNetwork: Keep only network settings
```


Clear Persistent Data

```
# Clear read-write filesystem (preserves firmware)
# WARNING: Loses all BMC configuration

# Method 1: Via systemd
systemctl start obmc-factory-reset.target

# Method 2: Manual (requires reboot)
rm -rf /var/lib/*
rm -rf /etc/machine-id
reboot

# Method 3: Via U-Boot (most thorough)
# At U-Boot prompt:
=> setenv openbmconce factory-reset
=> saveenv
=> reset
# BMC will clear rwfs on next boot
```

Full Flash Recovery

```
# Complete flash re-image (via TFTP and U-Boot)
# WARNING: Erases EVERYTHING

# At U-Boot prompt:
=> setenv serverip 192.168.1.100
=> setenv ipaddr 192.168.1.50
=> setenv netmask 255.255.255.0

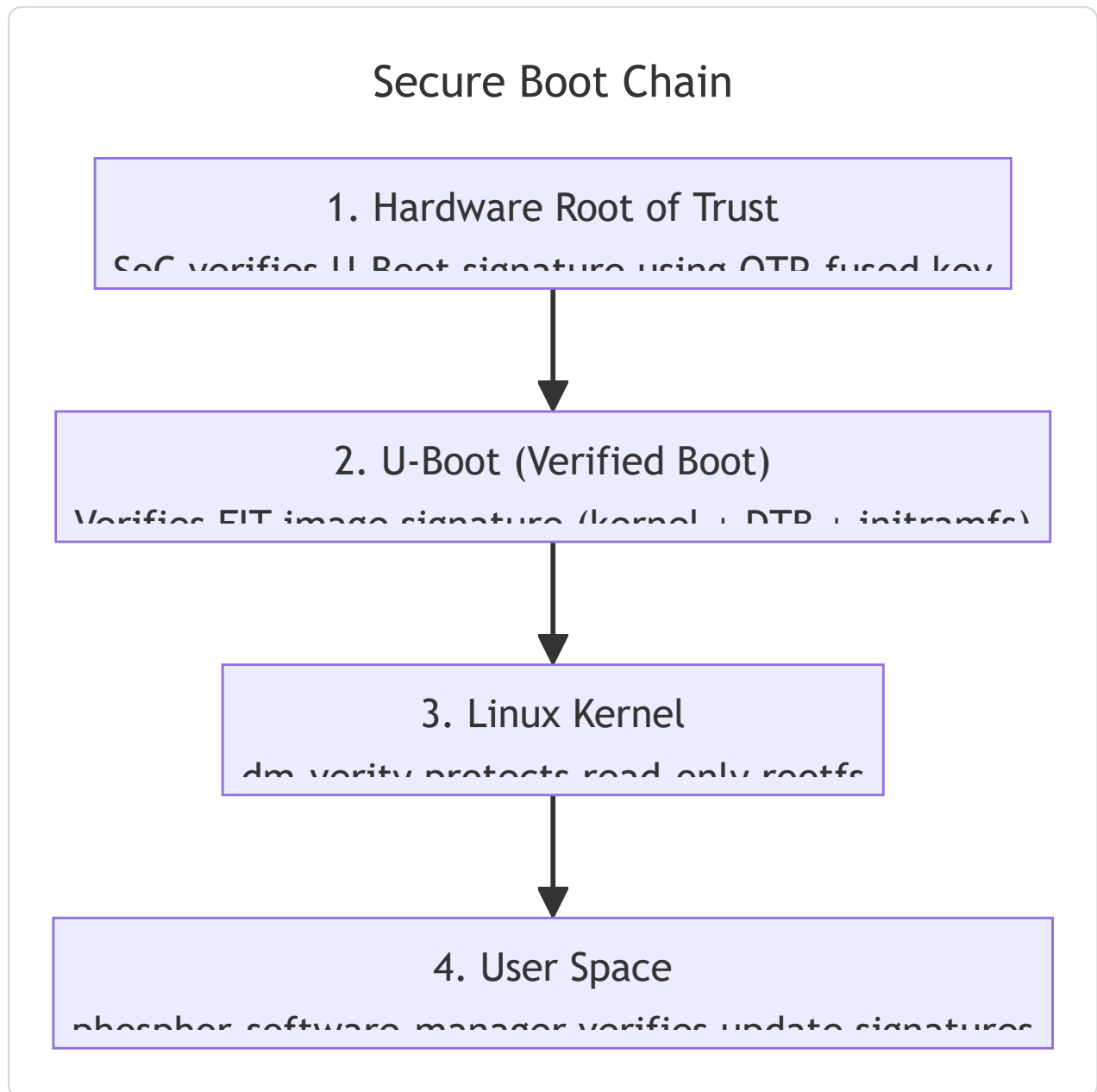
# Download full image
=> tftp 0x80000000 obmc-phosphor-image.static.mtd

# Probe SPI flash
=> sf probe

# Erase and write entire flash
=> sf erase 0 0x2000000 # Adjust size for your flash
=> sf write 0x80000000 0 ${filesize}

# Reset
=> reset
```

Secure Boot Chain



► ASCII-art version (for comparison)

U-Boot Verified Boot Configuration

```
# In U-Boot config (defconfig)
CONFIG_FIT=y
CONFIG_FIT_SIGNATURE=y
CONFIG_RSA=y

# FIT image with signature node
/dts-v1/;

/ {
    images {
        kernel {
            data = /incbin("Image");
            type = "kernel";
            arch = "arm";
            compression = "none";
            hash {
                algo = "sha256";
            };
        };
    };
    configurations {
        default = "conf";
        conf {
            kernel = "kernel";
            signature {
                algo = "sha256,rsa4096";
                key-name-hint = "dev-key";
                sign-images = "kernel", "fdt";
            };
        };
    };
};
```

Update Authentication

```
# Redfish authentication requirements
# - All update endpoints require authentication
# - Use HTTPS only (port 443)
# - Implement rate limiting to prevent brute force

# Enable mTLS for automated updates
# In bmcweb configuration:
# ssl_verify_mode = SSL_VERIFY_PEER

# RBAC for firmware updates
# Only Administrator role can perform updates
curl -k -u root:0penBmc \
    https://localhost/redfish/v1/AccountService/Accounts/root | jq .RoleId
# "Administrator"
```

Audit Logging

```
# Firmware update events are logged
journalctl -u phosphor-software-manager | grep -i "activat"

# Redfish audit log
curl -k -u root:0penBmc \
  https://localhost/redfish/v1/Managers/bmc/LogServices/CELog/Entries

# Key events to monitor:
# - Firmware upload
# - Signature verification result
# - Activation start/complete
# - Boot failures
# - Factory reset
```

Best Practices

Pre-Update Checklist

1. **Verify backup image** - Ensure standby image is known-good
2. **Check storage space** - Minimum 2x image size in /tmp
3. **Verify network connectivity** - For network-based updates
4. **Document current version** - For rollback reference
5. **Schedule maintenance window** - BMC reboot affects management

Update Procedure

```
# 1. Record current version
curl -k -u root:0penBmc \
  https://localhost/redfish/v1/UpdateService/FirmwareInventory/bmc_active \
  | jq .Version > /tmp/before_version.txt

# 2. Verify backup image is bootable
fw_printenv bootside
# Ensure alternate side has valid image

# 3. Upload new image
curl -k -u root:0penBmc \
  -X POST \
  -H "Content-Type: application/octet-stream" \
  --data-binary @firmware.tar \
  https://localhost/redfish/v1/UpdateService

# 4. Monitor activation
watch -n 5 'curl -sk -u root:0penBmc \
  https://localhost/redfish/v1/TaskService/Tasks | jq'

# 5. Verify after reboot
curl -k -u root:0penBmc \
  https://localhost/redfish/v1/UpdateService/FirmwareInventory/bmc_active \
  | jq .Version
```

Post-Update Verification

```
# Verify boot success
fw_printenv bootcount
# Should be 0

# Check system health
obmcutil state
# Should show Ready

# Verify all services running
systemctl --failed
# Should be empty

# Test critical functionality
curl -k -u root:0penBmc https://localhost/redfish/v1/Systems/system
ipmitool -I lanplus -H <bmc-ip> -U root -P 0penBmc chassis status
```

Examples

Working examples are available in the [examples/firmware-update](#) directory:

- [firmware-update.sh](#) - Firmware update helper script
- [version-check.sh](#) - Check current firmware versions

References

OpenBMC Repositories

- [phosphor-bmc-code-mgmt](#) - BMC code version management
- [phosphor-ipmi-flash](#) - IPMI-based firmware update
- [openpower-pnor-code-mgmt](#) - OpenPOWER PNOR management
- [pldm](#) - PLDM Type 5 firmware update

OpenBMC Documentation

- [Code Update Architecture](#)
- [Software Update Design](#)
- [Secure Boot](#)

Standards

- [DMTF Redfish UpdateService Schema](#)
- [DMTF PLDM Firmware Update Spec \(DSP0267\)](#)
- [IPMI Specification](#)

U-Boot

- [U-Boot Verified Boot](#)
- [U-Boot Environment Variables](#)

Deep Dive

Advanced implementation details for firmware update developers.

Flash Layout and MTD Partitions

- Typical BMC Flash Layout (32MB)

Offset	Size	Partition	Description
0x0000000	512KB	u-boot	Bootloader
0x0080000	128KB	u-boot-env	U-Boot environment
0x00A0000	4.5MB	kernel	FIT image (kernel+dtb)
0x0520000	25MB	rofs	Read-only rootfs
0x1DC0000	2MB	rwfs	Persistent read-write

- Dual-Flash Configuration (A/B):

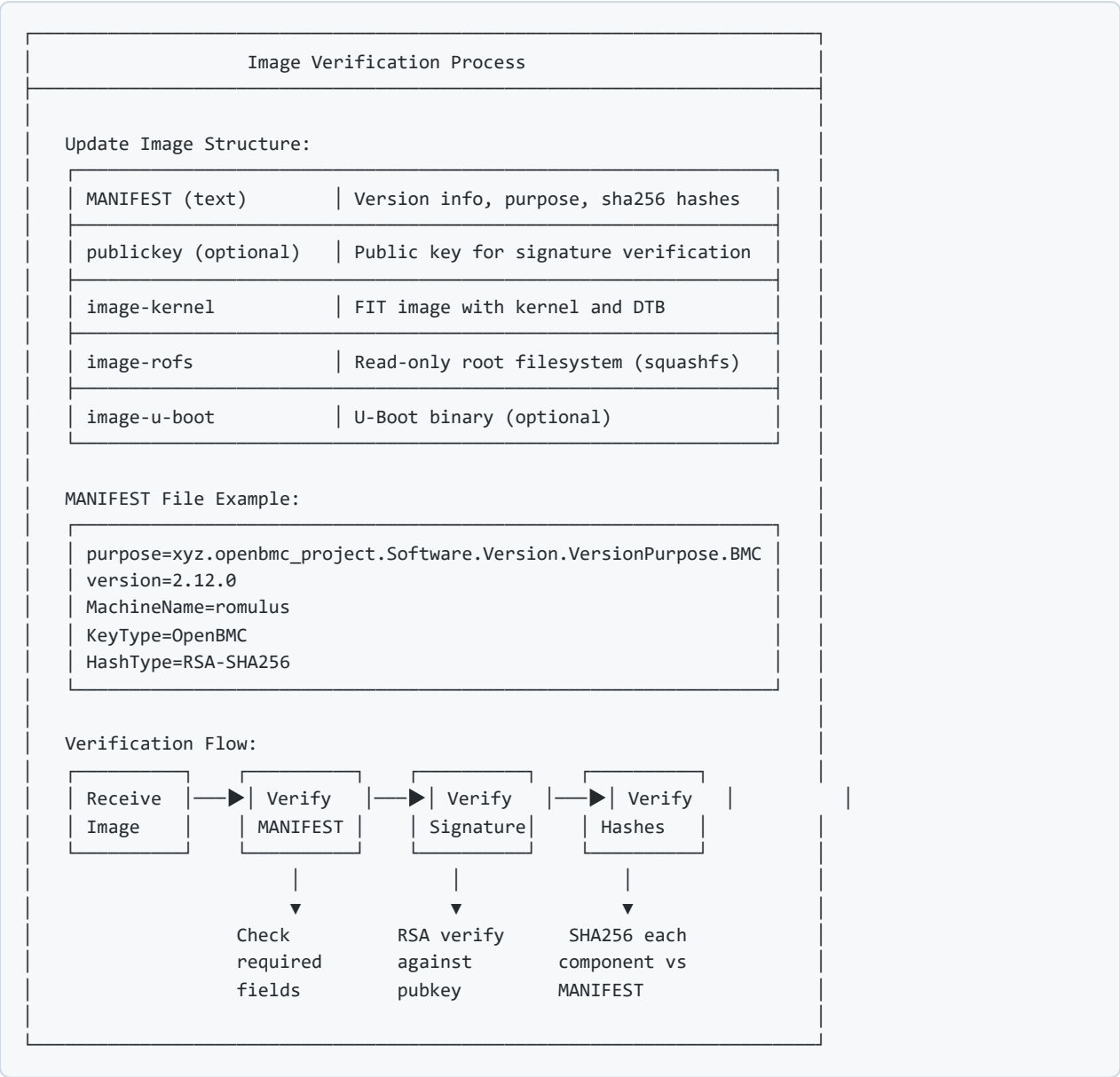
Flash 0 (Current)	Flash 1 (Alternate)
u-boot (shared)	(reserved)
u-boot-env	
kernel-a (active)	kernel-b
rofs-a (active)	rofs-b
rwfs (shared)	

- Linux MTD Devices:

```
/dev/mtd0 = u-boot
/dev/mtd1 = u-boot-env
/dev/mtd2 = kernel
/dev/mtd3 = rofs
/dev/mtd4 = rwfs
```

- ASCII-art version (for comparison)

Update Image Verification



Flash Programming Internals

Flash Write Process

MTD Write Operation:

1. Erase sector (typically 64KB)
 `ioctl(mtd_fd, MEMERASE, &erase_info)`
 └ Sector erased to 0xFF
2. Write data (page-aligned, typically 256 bytes)
 `write(mtd_fd, data, page_size)`
 └ Data written to erased sector
3. Verify (optional, MEMVERIFY ioctl)
 read-back and compare

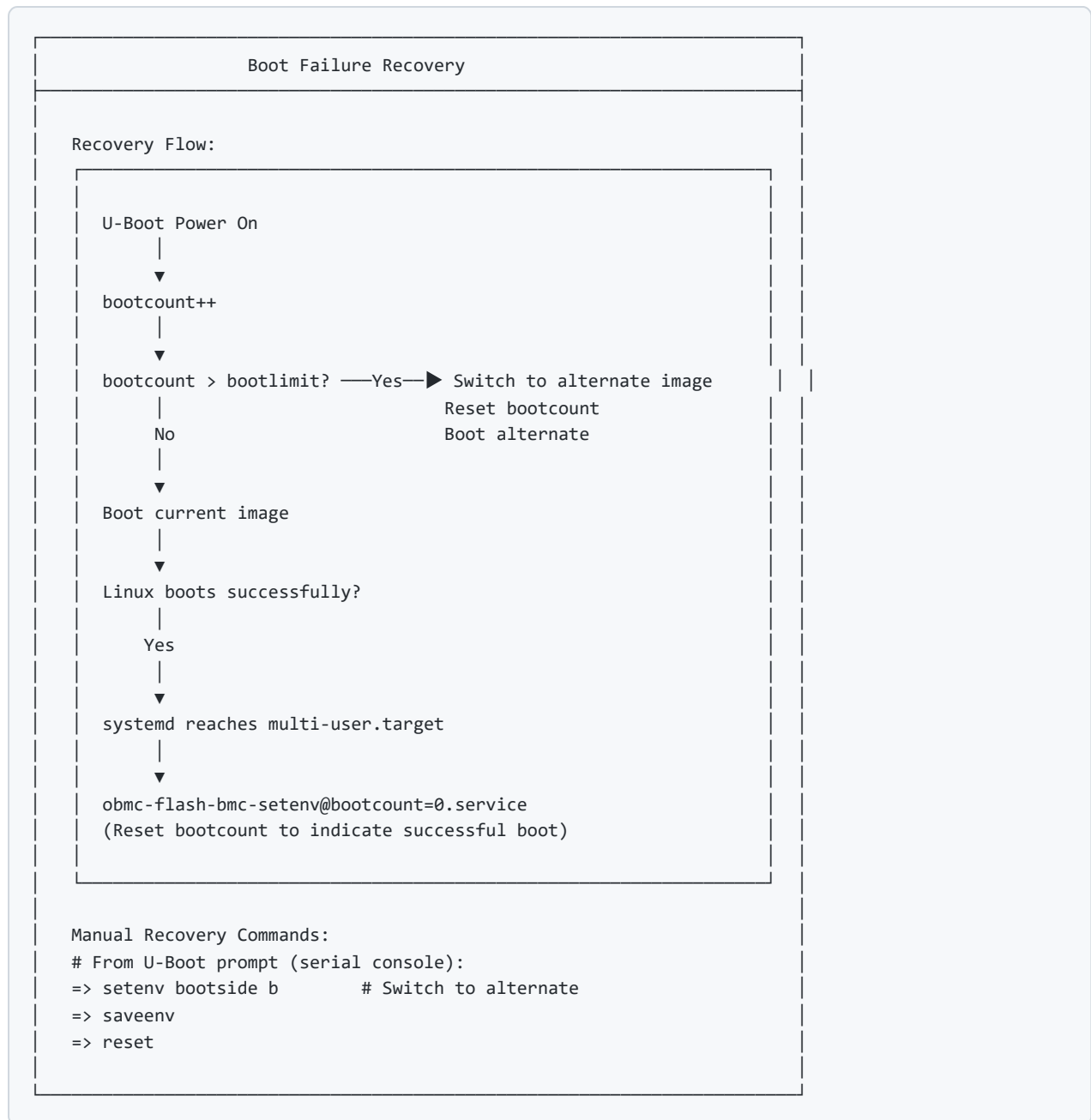
Update Daemon (phosphor-bmc-code-mgmt) Flow:

1. Image uploaded to /tmp/images/
2. Item Updater detects new image via inotify
3. Verify signature and hashes
4. Create D-Bus Version object (Ready)
5. User activates: RequestedActivation = Active
6. Write image components to alternate flash
7. Update U-Boot env to boot from new image
8. Set Activation = Activating, then Active
9. Reboot (if requested)

U-Boot Environment (dual-boot):

<code>bootside=a</code>	# Currently running: a or b
<code>bootcount=0</code>	# Failed boot attempts
<code>bootlimit=3</code>	# Max failures before fallback
<code>bootcmd=</code>	# Selects kernel-a or kernel-b

Recovery Mechanism



Source Code Reference

Key implementation files:

Repository	File/Directory	Description
phosphor-bmc-code-mgmt	item_updater.cpp	Main update logic
phosphor-bmc-code-mgmt	activation.cpp	Activation state machine
phosphor-bmc-code-mgmt	image_verify.cpp	Signature verification
phosphor-bmc-code-mgmt	flash.cpp	Flash write operations
openbmc/meta-phosphor	recipes-phosphor/flash/	Flash recipes
u-boot	board/aspeed/	ASPEED board support

Tested on: OpenBMC master, QEMU romulus (limited dual-flash functionality), ASPEED AST2500/AST2600 EVB

Intel ASD/ACD Guide

Configure Intel At-Scale Debug and Autonomous Crash Dump on OpenBMC.

Table of Contents

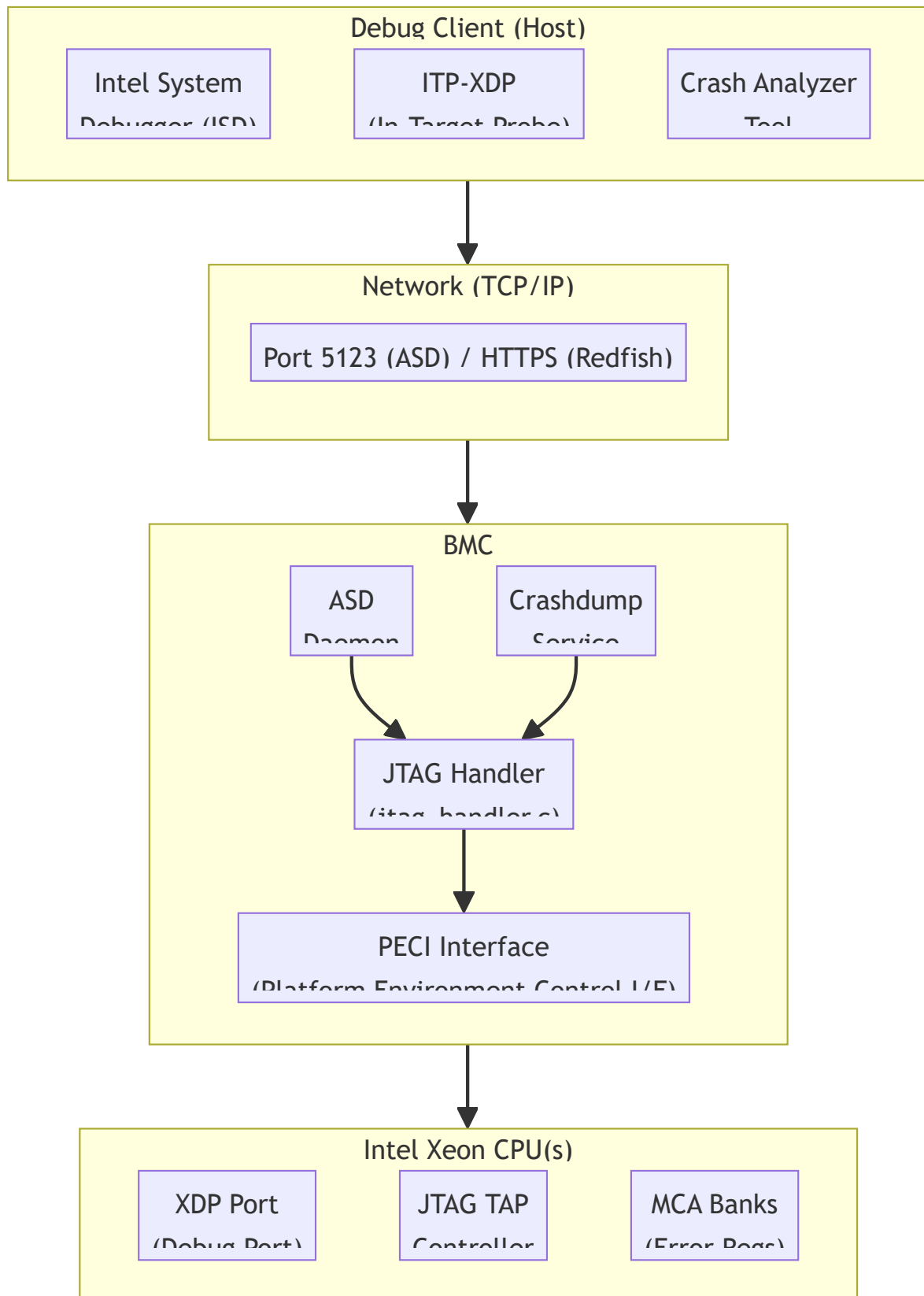
1. [Overview](#)
2. [ASD \(At-Scale Debug\)](#)
 1. [What is ASD?](#)
 2. [ASD Architecture](#)
 3. [Build Configuration](#)
 4. [Meson Build Options](#)
 5. [GPIO Configuration](#)
 6. [JTAG Chain Configuration](#)
 7. [Service Management](#)
 8. [ASD Protocol Messages](#)
 9. [Connecting with Debug Tools](#)
 10. [XDP \(eXtended Debug Port\) Interface](#)
 11. [ASD D-Bus Interface](#)
3. [ACD \(Autonomous Crash Dump\)](#)
 1. [What is ACD?](#)
 2. [Crash Dump Architecture](#)
 3. [Build Configuration](#)
 4. [Crash Dump Triggers](#)
 5. [GPIO Configuration for Error Detection](#)
 6. [PECI Interface](#)
 7. [Crash Dump Data Sections](#)
 8. [Manual Crash Dump Collection](#)
 1. [Via Redfish](#)
 2. [Via IPMI](#)
 9. [Crash Dump JSON Format](#)
 10. [Analyzing Crash Dumps](#)
 11. [Storage Location](#)
 12. [D-Bus Interface](#)
4. [Integration with Intel Tools](#)
 1. [Intel System Debugger \(ISD\)](#)
 2. [OpenIPC Configuration](#)
5. [Security Considerations](#)
 1. [Access Control](#)
 2. [Network Security](#)

- 3. [Audit Logging](#)
 - 4. [Production Recommendations](#)
 - 5. [Disabling ASD in Production](#)
 - 6. [Troubleshooting](#)
 - 1. [ASD Connection Issues](#)
 - 2. [JTAG Chain Not Detected](#)
 - 3. [Crash Dump Collection Fails](#)
 - 4. [Incomplete Crash Dump](#)
 - 5. [Performance Issues](#)
 - 7. [Platform-Specific Notes](#)
 - 1. [Intel Xeon Scalable \(Ice Lake, Sapphire Rapids\)](#)
 - 2. [Intel Xeon D](#)
 - 3. [Supported Platforms](#)
 - 8. [References](#)
 - 9. [Error Injection](#)
 - 1. [Error Injection via ASD/ITP](#)
 - 2. [ACPI EINJ Interface](#)
 - 1. [EINJ Overview](#)
 - 2. [Using EINJ from the Host OS](#)
 - 3. [Monitoring Injected Errors from BMC](#)
 - 3. [RAS Validation Workflow](#)
 - 1. [Step 1: Pre-Test Setup](#)
 - 2. [Step 2: Execute Test Matrix](#)
 - 3. [Step 3: Verify Collection](#)
 - 4. [Step 4: Report Generation](#)
 - 10. [Deep Dive](#)
 - 1. [ASD Protocol Message Format](#)
 - 2. [JTAG TAP State Machine](#)
 - 3. [PECI Crash Dump Command Flow](#)
 - 4. [MCA Bank Status Register Format](#)
 - 5. [Source Code References](#)
-

Overview

ASD (At-Scale Debug) and **ACD (Autonomous Crash Dump)** are Intel technologies for remote CPU debugging and crash analysis on Intel Xeon server platforms.

Intel Debug Architecture



► ASCII-art version (for comparison)

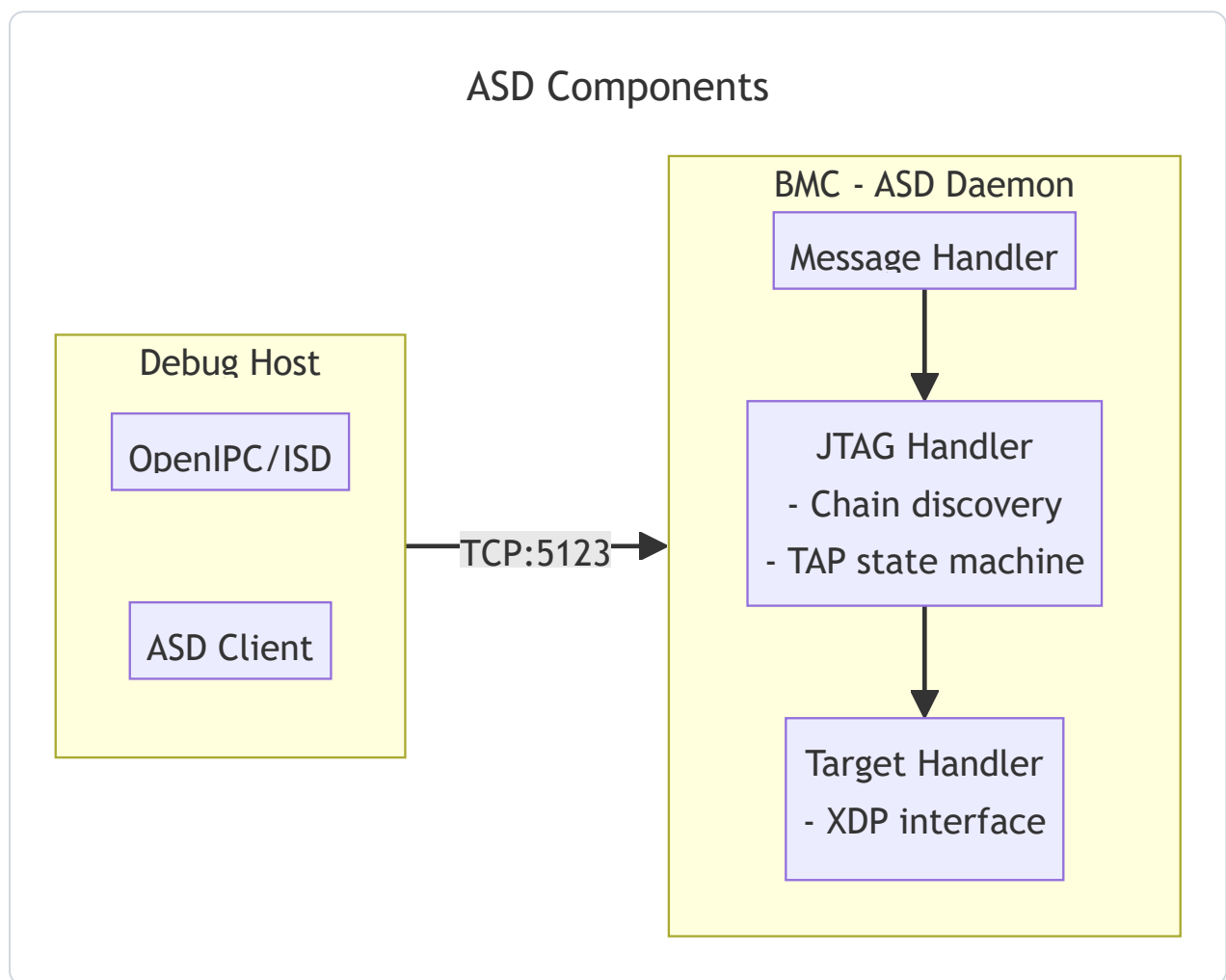
ASD (At-Scale Debug)

What is ASD?

At-Scale Debug provides remote JTAG-like access to Intel CPUs through the BMC, enabling:

- **Remote debugging** without physical access to the server
- **JTAG chain access** to CPU cores, uncore, and PCH
- **Run control** (halt, step, breakpoints)
- **Register access** (MSRs, GPRs, control registers)
- **Memory access** through the CPU debug interface

ASD Architecture



► ASCII-art version (for comparison)

Build Configuration

```
# recipes-phosphor/debug/asd.bb or local.conf

# Include ASD daemon
IMAGE_INSTALL:append = " asd"

# ASD build options
EXTRA_OEMESON:pn-asd = " \
    -Djtag-legacy-driver=disabled \
    -Dsafe-mode=enabled \
"
```

Meson Build Options

Option	Default	Description
<code>jtag-legacy-driver</code>	disabled	Use legacy JTAG driver
<code>safe-mode</code>	enabled	Enable safe mode protections
<code>i2c-debug</code>	disabled	Enable I2C debug messages
<code>i3c-debug</code>	disabled	Enable I3C debug messages

GPIO Configuration

ASD requires specific GPIO pins for JTAG and control signals:

```
// /usr/share/asd/pin_config.json
{
  "Name": "MyPlatform",
  "Platform": {
    "jtag": {
      "tck": {"pin": "GPIOA0", "type": "gpio"},
      "tdi": {"pin": "GPIOA1", "type": "gpio"},
      "tdo": {"pin": "GPIOA2", "type": "gpio"},
      "tms": {"pin": "GPIOA3", "type": "gpio"}
    },
    "target": {
      "xdp_present": {"pin": "GPIOB0", "type": "gpio"},
      "debug_enable": {"pin": "GPIOB1", "type": "gpio"},
      "preq": {"pin": "GPIOB2", "type": "gpio"},
      "prdy": {"pin": "GPIOB3", "type": "gpio"},
      "reset": {"pin": "GPIOB4", "type": "gpio"}
    }
  }
}
```

JTAG Chain Configuration

```
// /usr/share/asd/jtag_config.json
{
  "JTAGChain": {
    "devices": [
      {
        "name": "CPU0",
        "idcode": "0x0A046101",
        "ir_length": 11,
        "type": "processor"
      },
      {
        "name": "PCH",
        "idcode": "0x1B0A0101",
        "ir_length": 8,
        "type": "pch"
      }
    ]
  }
}
```

Service Management

```
# Check ASD service status
systemctl status asd

# Start ASD daemon
systemctl start asd

# Enable at boot
systemctl enable asd

# View logs
journalctl -u asd -f

# Check listening port
ss -tlnp | grep 5123
```

ASD Protocol Messages

Message Type	Description
AGENT_CONTROL	Control commands (reset, init)
JTAG_CHAIN	JTAG chain operations
I2C_MSG	I2C master transactions
GPIO_MSG	GPIO read/write
REMOTE_DEBUG	Remote debug enable/disable

Connecting with Debug Tools

```
# On debug host, using OpenIPC/ISD:
# 1. Configure target connection
#   - IP: <BMC_IP>
#   - Port: 5123
#   - Auth: BMC credentials

# 2. Connect and discover JTAG chain
# 3. Select target device (CPU core)
# 4. Begin debugging session
```

XDP (eXtended Debug Port) Interface

XDP provides enhanced debug access:

```
XDP Signals:
+-----+-----+
| Signal | Description |
+-----+-----+
| PREQ#  | Probe Request (BMC -> CPU) |
| PRDY#  | Probe Ready (CPU -> BMC) |
| RESET# | Platform Reset control |
| TCK    | JTAG Test Clock |
| TMS    | JTAG Test Mode Select |
| TDI    | JTAG Test Data In |
| TDO    | JTAG Test Data Out |
| TRST#  | JTAG Test Reset |
+-----+-----+
```

ASD D-Bus Interface

```
# ASD exposes configuration via D-Bus
busctl tree xyz.openbmc_project.ASD

# Get ASD status
busctl get-property xyz.openbmc_project.ASD \
  /xyz/openbmc_project/asd \
  xyz.openbmc_project.ASD.Server \
  Status

# Enable/disable remote debug
busctl set-property xyz.openbmc_project.ASD \
  /xyz/openbmc_project/asd \
  xyz.openbmc_project.ASD.Server \
  RemoteDebugEnabled b true
```

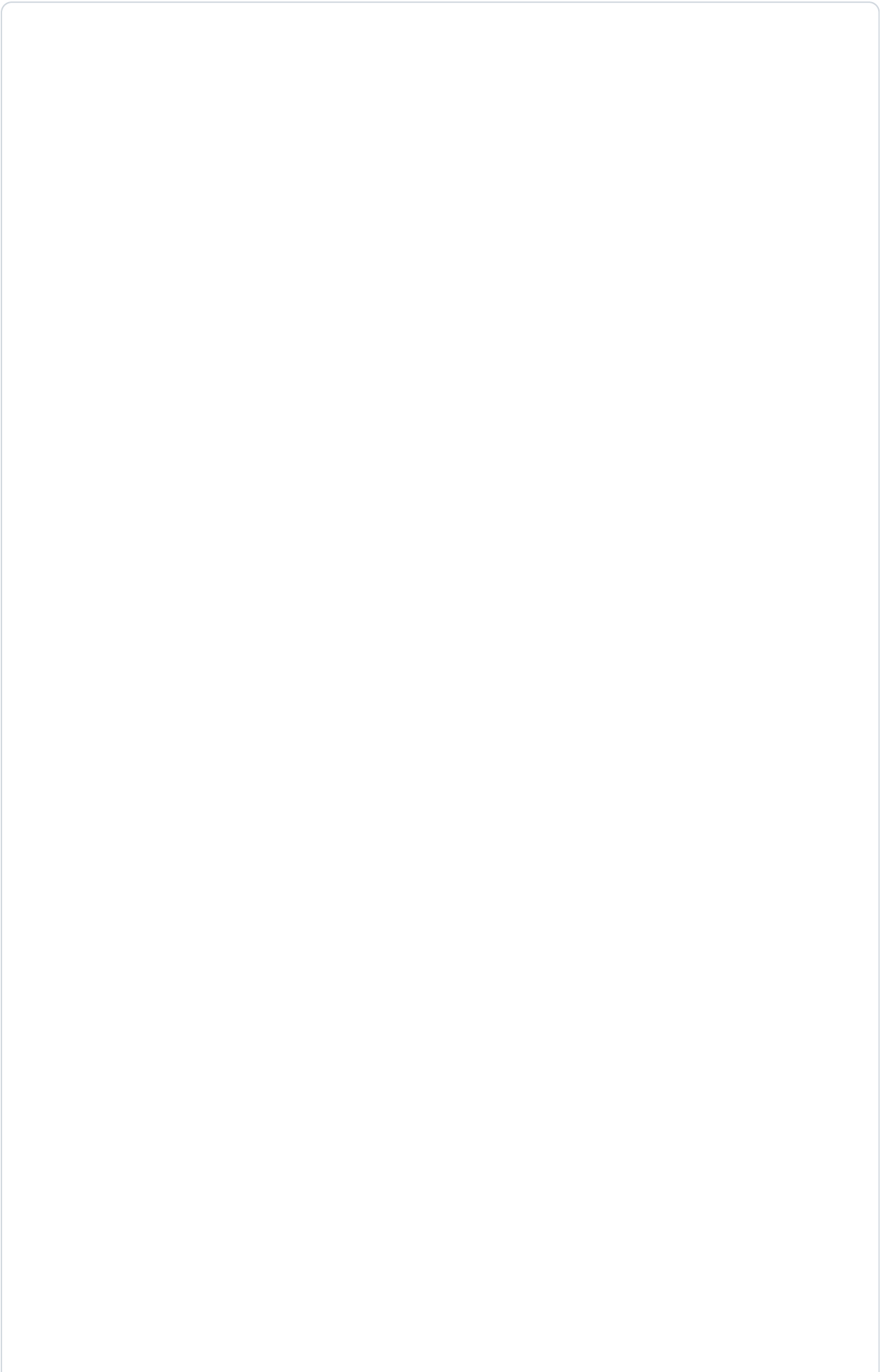
ACD (Autonomous Crash Dump)

What is ACD?

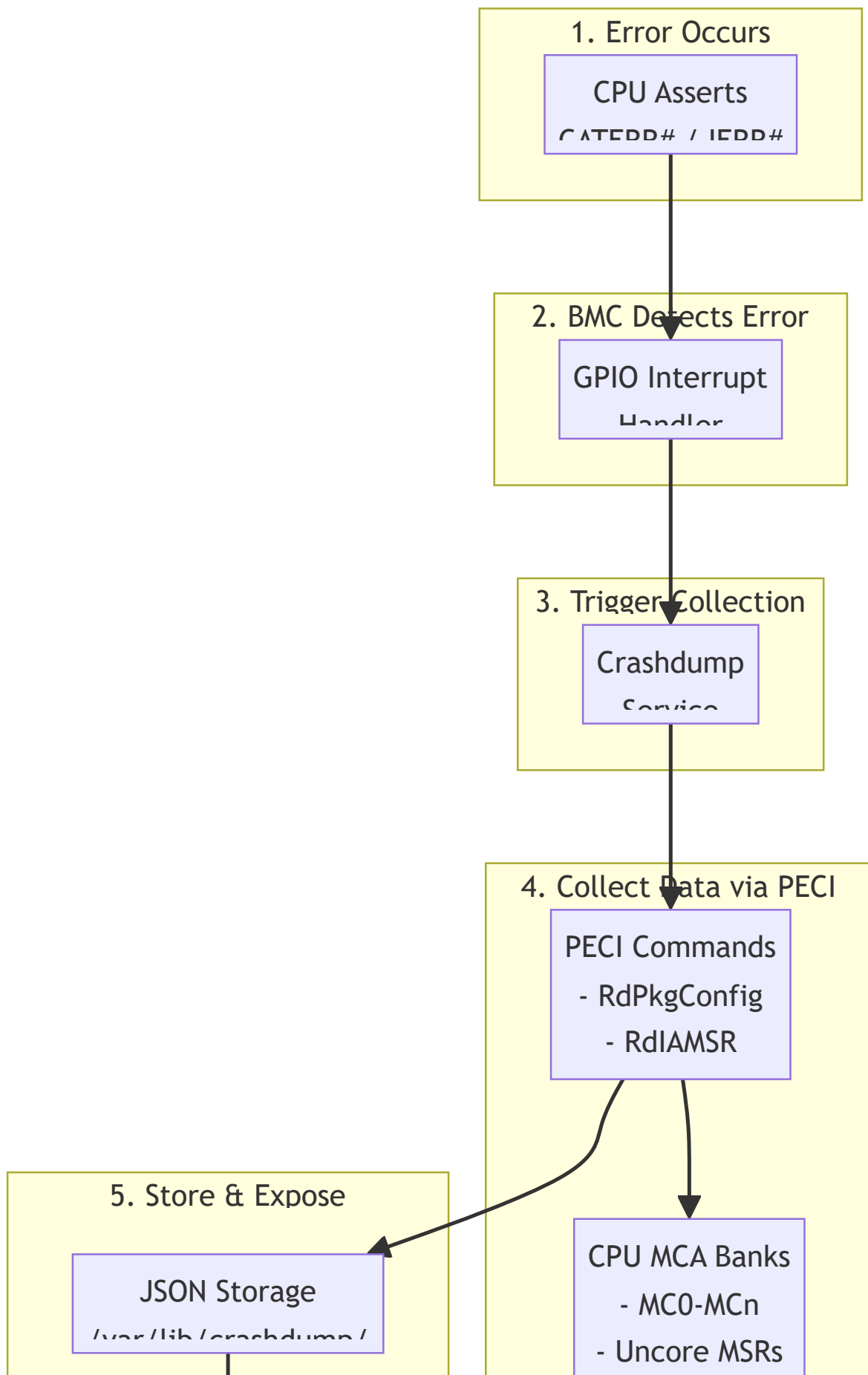
Autonomous Crash Dump automatically collects CPU diagnostic data when critical errors occur:

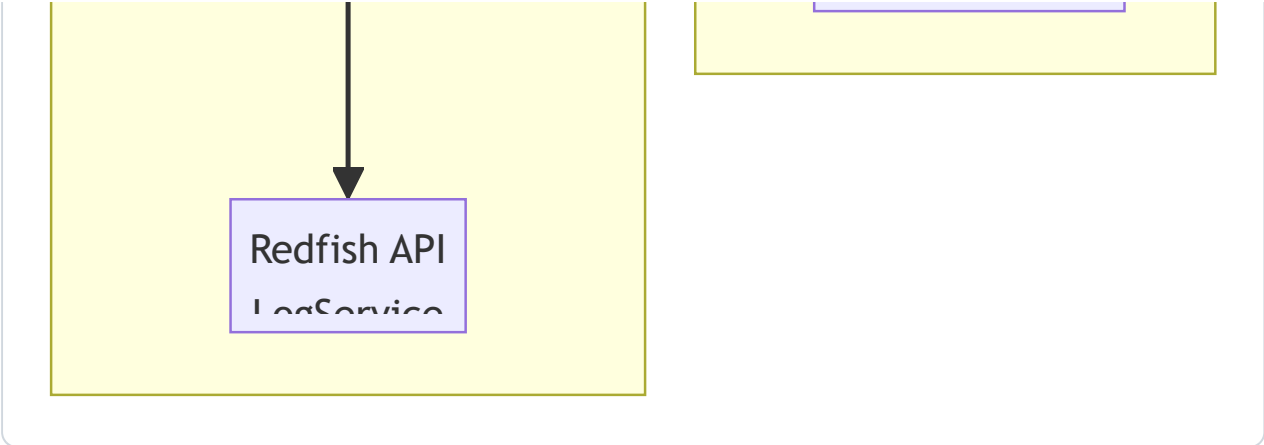
- **Machine Check Exceptions (MCE)** capture
- **CPU register state** preservation

- **Memory controller errors** logging
- **PCIe errors** collection
- **Uncore state** dump



Crash Dump Collection Flow





► ASCII-art version (for comparison)

Build Configuration

```
# Include crashdump service
IMAGE_INSTALL:append = " crashdump"

# Crashdump build options
EXTRA_OEMESON:pn-crashdump = " \
    -Dtests=disabled \
    -Dcrashdump-x86=enabled \
"
```

Crash Dump Triggers

Trigger	Signal	Description
CATERR#	GPIO	Catastrophic Error
IERR#	GPIO	Internal Error
MCERR#	GPIO	Machine Check Error
ERR2#	GPIO	Error Signal 2
Manual	Redfish/IPMI	User-initiated collection

GPIO Configuration for Error Detection

```
// Entity Manager configuration
{
  "Name": "CrashDump Triggers",
  "Type": "GPIO",
  "Exposes": [
    {
      "Name": "CATERR",
      "Type": "gpio",
      "Index": 45,
      "Polarity": "Low",
      "Direction": "Input"
    },
    {
      "Name": "ERR2",
      "Type": "gpio",
      "Index": 46,
      "Polarity": "Low",
      "Direction": "Input"
    }
  ]
}
```

PECI Interface

PECI (Platform Environment Control Interface) is used to collect crash data:

```
# Peci commands used by crashdump:

# Read Package Config
peci_cmds RdPkgConfig <address> <index> <parameter>

# Read IA MSR
peci_cmds RdIAMSr <address> <thread> <msr_address>

# Crashdump command
peci_cmds Crashdump <address> <command> <param>
```

Crash Dump Data Sections

Section	Content
metadata	Timestamp, trigger, platform info
MCA	Machine Check Architecture banks
uncore	Uncore MSRs and registers
TOR	Transaction Outstanding Registers
PM_Info	Power management state
address_map	Memory address mapping
big_core	CPU core registers
crashlog	Hardware crash log

Manual Crash Dump Collection

Via Redfish

```
# Trigger crash dump collection
curl -k -u root:OpenBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{"DiagnosticDataType": "OEM", "OEMDiagnosticDataType": "OnDemand"}' \
  https://localhost/redfish/v1/Systems/system/LogServices/Crashdump/Actions/LogService.CollectDiagnosticData

# List collected dumps
curl -k -u root:OpenBmc \
  https://localhost/redfish/v1/Systems/system/LogServices/Crashdump/Entries

# Get specific dump
curl -k -u root:OpenBmc \
  https://localhost/redfish/v1/Systems/system/LogServices/Crashdump/Entries/1

# Download raw dump data
curl -k -u root:OpenBmc -o crashdump.json \
  "https://localhost/redfish/v1/Systems/system/LogServices/Crashdump/Entries/1/attachment"
```

Via IPMI

```
# Trigger crash dump (Intel OEM command)
ipmitool -I lanplus -H <bmc-ip> -U root -P OpenBmc \
  raw 0x30 0x74 # Intel OEM crashdump trigger

# Get crash dump status
ipmitool -I lanplus -H <bmc-ip> -U root -P OpenBmc \
  raw 0x30 0x75 # Intel OEM crashdump status
```


Crash Dump JSON Format

```
{
  "crash_data": {
    "METADATA": {
      "timestamp": "2024-01-15T10:30:45Z",
      "trigger_type": "CATERR",
      "platform": "Intel Xeon 4th Gen",
      "cpu_count": 2
    },
    "CPU0": {
      "MCA": {
        "MC0_CTL": "0x0000000000000001",
        "MC0_STATUS": "0xBE00000000800400",
        "MC0_ADDR": "0x000000007F400000",
        "MC0_MISC": "0x0000000000000000",
        "MC0_CTL2": "0x0000000000000001"
      },
      "uncore": {
        "UNCORE_MC0_STATUS": "0x0000000000000000",
        "UNCORE_CHA0_STATUS": "0x0000000000000000"
      },
      "TOR": {
        "TOR_0_SAD": "0x0000000000000000",
        "TOR_0_TAD": "0x0000000000000000"
      }
    },
    "CPU1": {
      "MCA": {
        "...": "..."
      }
    }
  }
}
```

Analyzing Crash Dumps

```
# Use Intel Crash Analyzer Tool (proprietary)
# Or parse JSON manually:

# Extract MCA status
cat crashdump.json | jq '.crash_data.CPU0.MCA.MC0_STATUS'

# Check for valid error
# Bit 63 (VAL) = 1 indicates valid error
# Bit 61 (UC) = 1 indicates uncorrected error
# Bit 60 (EN) = 1 indicates error reporting enabled

# Decode MCACOD (bits 15:0) for error type
cat crashdump.json | jq -r '.crash_data.CPU0.MCA | to_entries[] |
  select(.key | endswith("_STATUS")) |
  "\(.key): \(.value)'"
```

Storage Location

```
# Crash dumps stored at:
ls /var/lib/crashdump/

# Format: crashdump_<timestamp>.json
# Example: crashdump_20240115_103045.json

# Check available space
df -h /var/lib/crashdump/

# Cleanup old dumps
# Managed by crashdump service based on retention policy
```

D-Bus Interface

```
# Crashdump D-Bus service
busctl tree xyz.openbmc_project.CrashDump

# Get last crash dump status
busctl get-property xyz.openbmc_project.CrashDump \
    /xyz/openbmc_project/crashdump \
    xyz.openbmc_project.CrashDump.Manager \
    LastCrashDumpTime

# Trigger collection via D-Bus
busctl call xyz.openbmc_project.CrashDump \
    /xyz/openbmc_project/crashdump \
    xyz.openbmc_project.CrashDump.Manager \
    GenerateCrashDump s "OnDemand"
```

Integration with Intel Tools

Intel System Debugger (ISD)

Prerequisites:

- Intel System Studio or standalone ISD
- Network connectivity to BMC
- BMC credentials with debug privileges

Connection Steps:

1. Launch Intel System Debugger
2. Create new target connection:
 - Type: At-Scale Debug
 - Host: <BMC_IP>
 - Port: 5123
3. Authenticate with BMC credentials
4. Discover JTAG chain
5. Connect to CPU target

OpenIPC Configuration

```
# OpenIPC is part of Intel System Studio
# Configuration file: openipc.cfg

[connection]
type = asd
host = 192.168.1.100
port = 5123
username = root
password = 0penBmc

[target]
cpu_type = SPR # Sapphire Rapids
num_cpus = 2

[debug]
log_level = info
```

Security Considerations

ASD provides deep system access equivalent to physical JTAG. Implement strict security controls.

Access Control

```
# Restrict ASD access to specific users
# Configure via Redfish AccountService

# Create debug-only user
curl -k -u root:0penBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "UserName": "debuguser",
    "Password": "SecureDebugPass123!",
    "RoleId": "Administrator"
  }' \
  https://localhost/redfish/v1/AccountService/Accounts
```

Network Security

```
# Restrict ASD port access via firewall
iptables -A INPUT -p tcp --dport 5123 -s 10.0.0.0/8 -j ACCEPT
iptables -A INPUT -p tcp --dport 5123 -j DROP

# Use VPN or isolated management network for debug access
```

Audit Logging

```
# Enable debug access logging
# All ASD connections logged to journal

# View ASD access logs
journalctl -u asd | grep -i "connection\|auth"

# Integrate with SEL
# ASD events appear in System Event Log
ipmitool sel list | grep -i debug
```

Production Recommendations

Setting	Development	Production
ASD Enabled	Yes	No (disable)
Debug Port	Open	Blocked
Crashdump	Enabled	Enabled
Auto-collection	Enabled	Enabled
Retention	30 days	7 days

Disabling ASD in Production

```
# Stop and disable ASD service
systemctl stop asd
systemctl disable asd

# Or remove from build
# IMAGE_INSTALL:remove = "asd"

# Verify disabled
systemctl status asd
ss -tlnp | grep 5123 # Should show nothing
```

Troubleshooting

ASD Connection Issues

```
# Check ASD daemon is running
systemctl status asd

# Verify port is listening
ss -tlnp | grep 5123

# Check authentication
journalctl -u asd | grep -i "auth\|fail"

# Verify GPIO configuration
cat /sys/kernel/debug/gpio | grep -i jtag

# Test JTAG chain manually
jtag_test --scan
```

JTAG Chain Not Detected

```
# Check XDP present signal
gpioget gpiochip0 <xdp_present_pin>

# Verify CPU is powered
obmcutil state

# Check JTAG signals
cat /sys/kernel/debug/pinctrl/*/pins | grep -i jtag

# Reset JTAG state machine
jtag_test --reset
```

Crash Dump Collection Fails

```
# Check crashdump service
systemctl status crashdump

# View crash dump logs
journalctl -u crashdump -f

# Verify PECI connectivity
peci_cmds Ping 0x30 # CPU0 address

# Check GPIO error signals
gpioget gpiochip0 <caterr_pin>
gpioget gpiochip0 <err2_pin>

# Manual PECI test
peci_cmds RdPkgConfig 0x30 0 0
```

Incomplete Crash Dump

```
# Check storage space
df -h /var/lib/crashdump/

# Verify PECI timeout
journalctl -u crashdump | grep -i "timeout\|peci"

# Check CPU accessibility
peci_cmds Ping 0x30
peci_cmds Ping 0x31 # CPU1 if present

# Retry collection
curl -k -u root:openBmc -X POST \
  -d '{"DiagnosticDataType": "OEM", "OEMDiagnosticDataType": "OnDemand"}' \
  https://localhost/redfish/v1/Systems/system/LogServices/Crashdump/Actions/LogService.CollectDiagnosticData
```

Performance Issues

```
# ASD operations are slow - check JTAG clock
# Default TCK is often conservative

# Crashdump takes too long
# Large dumps may take several minutes
# Monitor progress in journal
journalctl -u crashdump -f
```

Platform-Specific Notes

Intel Xeon Scalable (Ice Lake, Sapphire Rapids)

- Supports enhanced crashdump features
- Multiple MCA banks per CPU
- Extended TOR dump capability
- Hardware crashlog support

Intel Xeon D

- Reduced feature set
- Single-socket configuration typical
- Integrated PCH

Supported Platforms

Platform	ASD	ACD	Notes
Ice Lake-SP	Yes	Yes	Full support
Sapphire Rapids	Yes	Yes	Enhanced crashlog
Xeon D	Yes	Limited	Reduced MCA banks
Older Xeon	Limited	Limited	Legacy JTAG only

References

- [Intel At-Scale Debug \(ASD\)](#) - Intel BMC debug access solution
- [Intel-BMC OpenBMC](#) - Intel OpenBMC fork with crashdump support
- [bmcweb Crashdump Endpoints](#) - Redfish crashdump implementation
- [Intel IPMI OEM](#) - Intel-specific IPMI commands
- [PECI Specification](#) - Platform Environment Control Interface

OpenIPC (Open In-band Processor Communication) is part of Intel System Studio and is not publicly available on GitHub. Contact Intel for access.

Error Injection

Error injection enables RAS (Reliability, Availability, Serviceability) validation by deliberately triggering hardware errors to verify detection and recovery mechanisms.

Error Injection via ASD/ITP

When connected to a CPU via ASD, you can inject errors through the debug interface to test MCA handling:

```
# Prerequisites:
# - ASD daemon running and connected
# - Intel System Debugger (ISD) or OpenIPC connected
# - Host powered on and running OS

# Via ISD scripting interface (Python):
# Connect to target
target = debugger.connect("asd://bmc-ip:5123")

# Inject correctable memory error (CE)
# Write to IA32_MCI_STATUS to simulate error
target.write_msr(thread=0, msr=0x401,
    value=0x8C00004000010150) # Correctable ECC error

# Inject uncorrectable error (UCE)
target.write_msr(thread=0, msr=0x401,
    value=0xBE00000000800400) # Uncorrected, PCC set

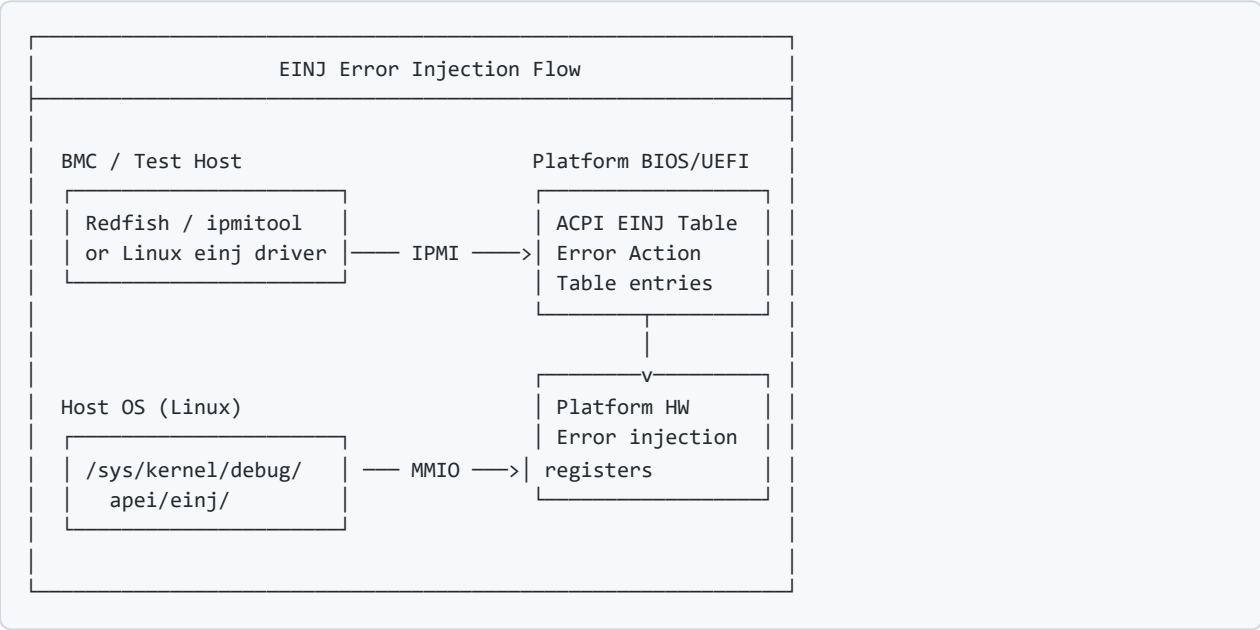
# Trigger machine check via APIC
target.write_msr(thread=0, msr=0x79, # MCG_STATUS
    value=0x05) # MCIP + RIPV
```

Error injection through ASD bypasses normal error reporting paths. Use only in isolated test environments with expendable hardware. Injecting UCE with PCC (Processor Context Corrupt) will crash the host.

ACPI EINJ Interface

The ACPI Error INjection (EINJ) table provides a firmware-standard method to inject errors without requiring ASD debug access. This is the preferred method for production-like RAS testing.

EINJ Overview



Using EINJ from the Host OS


```
# On the host Linux OS (not the BMC):

# Load EINJ support module
modprobe einj

# Check available error types
cat /sys/kernel/debug/apei/einj/available_error_type
# Typical output:
# 0x00000001 Processor Correctable
# 0x00000002 Processor Uncorrectable non-fatal
# 0x00000004 Processor Uncorrectable fatal
# 0x00000008 Memory Correctable
# 0x00000010 Memory Uncorrectable non-fatal
# 0x00000020 Memory Uncorrectable fatal
# 0x00000040 PCIe Correctable
# 0x00000080 PCIe Uncorrectable non-fatal
# 0x00000100 PCIe Uncorrectable fatal

# Inject correctable memory error
echo 0x00000008 > /sys/kernel/debug/apei/einj/error_type
echo 0x0 > /sys/kernel/debug/apei/einj/error_inject

# Inject PCIe correctable error
echo 0x00000040 > /sys/kernel/debug/apei/einj/error_type
echo 0x0 > /sys/kernel/debug/apei/einj/error_inject

# Inject memory UCE at specific address
echo 0x00000010 > /sys/kernel/debug/apei/einj/error_type
echo 0x7F400000 > /sys/kernel/debug/apei/einj/param1 # Physical address
echo 0xFFFFFFFFFFFFFFFF000 > /sys/kernel/debug/apei/einj/param2 # Address mask
echo 0x0 > /sys/kernel/debug/apei/einj/error_inject
```

Monitoring Injected Errors from BMC

After injecting errors on the host, verify the BMC captures them:

```
# On the BMC:

# Check for new crash dumps (for fatal errors)
curl -k -u root:openBmc \
  https://localhost/redfish/v1/Systems/system/LogServices/Crashdump/Entries

# Check System Event Log for error records
curl -k -u root:openBmc \
  https://localhost/redfish/v1/Systems/system/LogServices/EventLog/Entries

# Monitor PECI for MCA errors in real-time
journalctl -u crashdump -f

# Check if CATERR was asserted (for fatal errors)
gpioget gpiochip0 <caterr_pin>
```

RAS Validation Workflow

A structured approach to validating RAS features on an Intel platform with OpenBMC:

Step 1: Pre-Test Setup

```

# Verify BMC services are running
systemctl status crashdump
systemctl status phosphor-host-state-manager

# Clear existing logs
curl -k -u root:0penBmc -X POST \
  https://localhost/redfish/v1/Systems/system/LogServices/Crashdump/Actions/LogService.ClearLog

# Verify PECI connectivity to all CPUs
peci_cmds Ping 0x30 # CPU0
peci_cmds Ping 0x31 # CPU1 (if dual-socket)

# Record baseline MCA state
for bank in 0 1 2 3 4 5; do
  msr=$((0x401 + bank * 4))
  peci_cmds RdIAMS 0x30 0 $(printf "0x%x" $msr)
done

```

Step 2: Execute Test Matrix

Test Case	Error Type	Injection Method	Expected BMC Response
CE-MEM-01	Memory correctable	EINJ 0x08	SEL entry, no crash dump
CE-PCIE-01	PCIe correctable	EINJ 0x40	SEL entry, AER log
UCE-MEM-01	Memory uncorrectable	EINJ 0x10	SEL + crash dump collected
UCE-FATAL-01	Fatal processor	EINJ 0x04	CATERR, full crash dump, host reset
MCA-ITP-01	MCA via ASD	ISD MSR write	Crash dump if PCC set

Step 3: Verify Collection

```
# After each injection, verify BMC response:

# 1. Check crash dump was collected (for UCE/fatal)
ENTRIES=$(curl -sk -u root:0penBmc \
    https://localhost/redfish/v1/Systems/system/LogServices/Crashdump/Entries \
    | python3 -c "import sys,json; print(json.load(sys.stdin)['Members@odata.count'])")
echo "Crash dump entries: $ENTRIES"

# 2. Download and validate crash dump content
curl -sk -u root:0penBmc -o crashdump.json \
    "https://localhost/redfish/v1/Systems/system/LogServices/Crashdump/Entries/1/attachment"

# 3. Verify MCA banks contain expected error signature
python3 -c "
import json
with open('crashdump.json') as f:
    data = json.load(f)
for cpu in ['CPU0', 'CPU1']:
    if cpu in data.get('crash_data', {}):
        mca = data['crash_data'][cpu].get('MCA', {})
        for bank, val in mca.items():
            if bank.endswith('_STATUS') and int(val, 16) & (1 << 63):
                uc = 'UC' if int(val, 16) & (1 << 61) else 'CE'
                print(f'{cpu} {bank}: {val} [{uc}]')
"

# 4. Verify host recovery (for non-fatal errors)
obmcutil state # Should show Running for non-fatal
```

Step 4: Report Generation

```
# Collect all test artifacts
mkdir -p /tmp/ras-validation-$(date +%Y%m%d)
cd /tmp/ras-validation-$(date +%Y%m%d)

# Export crash dumps
for i in $(seq 1 $ENTRIES); do
    curl -sk -u root:0penBmc -o "crashdump_${i}.json" \
        "https://localhost/redfish/v1/Systems/system/LogServices/Crashdump/Entries/${i}/attachment"
done

# Export event log
curl -sk -u root:0penBmc -o event_log.json \
    https://localhost/redfish/v1/Systems/system/LogServices/EventLog/Entries

# Export BMC journal
journalctl -u crashdump --since "today" > crashdump_journal.log
journalctl -u phosphor-host-state-manager --since "today" > state_manager.log
```

For automated RAS validation, see the example scripts in [docs/examples/error-injection/](#) which automate the inject-verify-report cycle.

Deep Dive

This section provides detailed technical information for developers who want to understand Intel ASD and crash dump internals.

ASD Protocol Message Format

ASD Network Protocol Format

TCP Connection (Port 5123):

1. Client connects to BMC:5123

2. TLS handshake (optional, depends on config)

3. Authentication (BMC credentials via custom protocol)

4. Message exchange begins

ASD Message Header:

Offset	Size	Field	Description
0x00	1	cmd_stat	Command/Status byte
0x01	1	cmd_type	Message type
0x02	2	size_lsb	Payload size (little-endian)
0x04	2	size_msb	
0x06	1	tag	Sequence tag for request matching
0x07	1	origin	Message origin ID
0x08	N	payload	Variable-length payload

Message Types (cmd_type):

Type	Name	Description
0x01	AGENT_CONTROL	Init, reset, status queries
0x02	JTAG_CHAIN	JTAG operations (scan, shift)
0x03	I2C_MSG	I2C read/write transactions
0x04	GPIO_MSG	GPIO control (XDP signals)
0x05	REMOTE_DEBUG	Enable/disable debug mode
0x06	HW_DEBUG	Hardware debug operations
0x07	I3C_MSG	I3C transactions (newer CPUs)

JTAG Message Payload:

JTAG_CHAIN message structure:

JTAG Cmd	Bit Count	TDI/TMS Data
(1 byte)	(2 bytes)	(N bytes, packed bits)

JTAG Commands:

0x00 - SET_TAP_STATE

(navigate TAP state machine)

0x01 - WAIT_CYCLES

(idle cycles)

0x02 - SHIFT_TDI

(shift data into chain)

0x03 - SHIFT_TDO

(shift data out of chain)

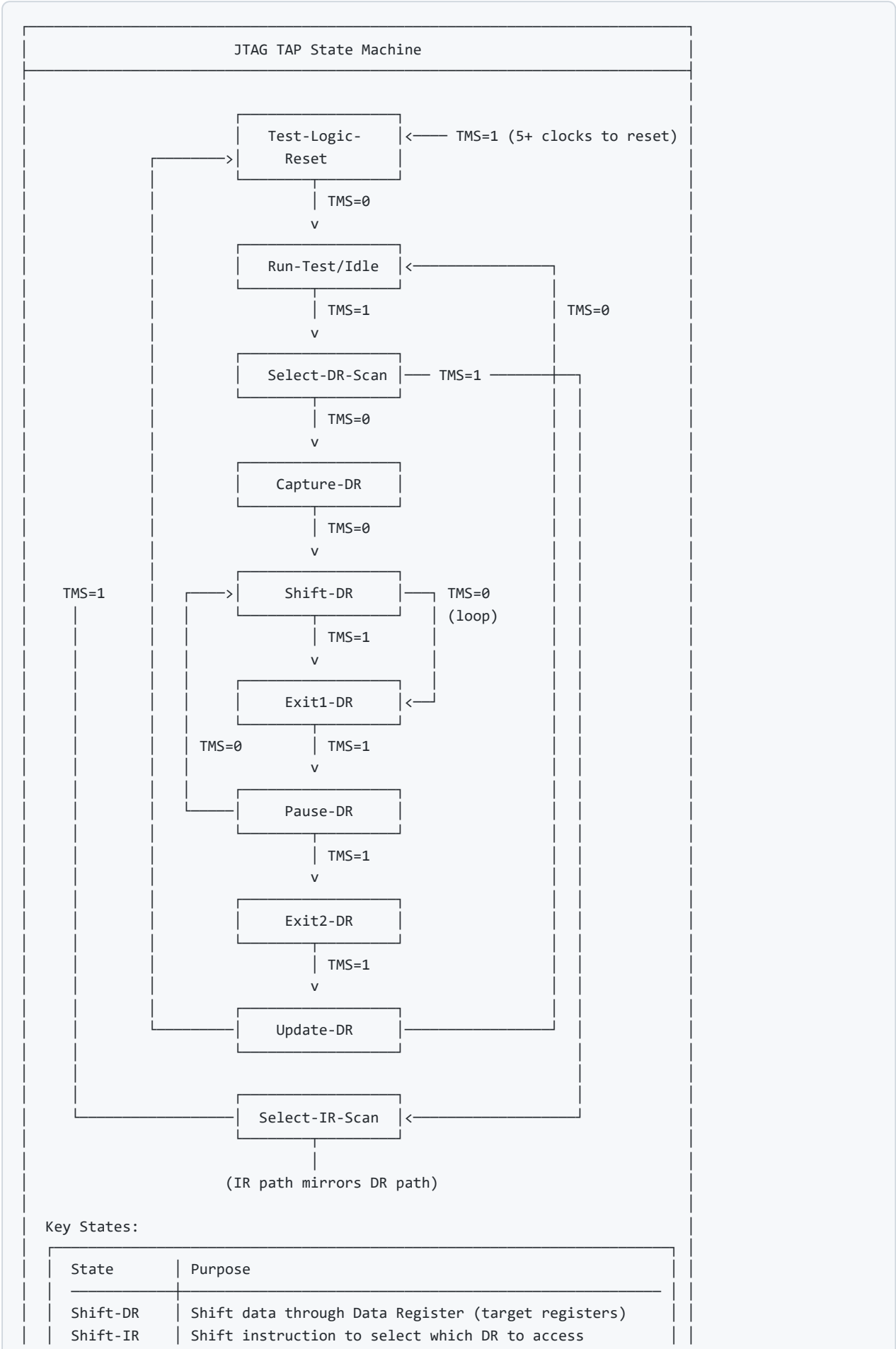
0x04 - SHIFT_TDI_TDO

(bidirectional shift)

0x05 - SCAN_CHAIN

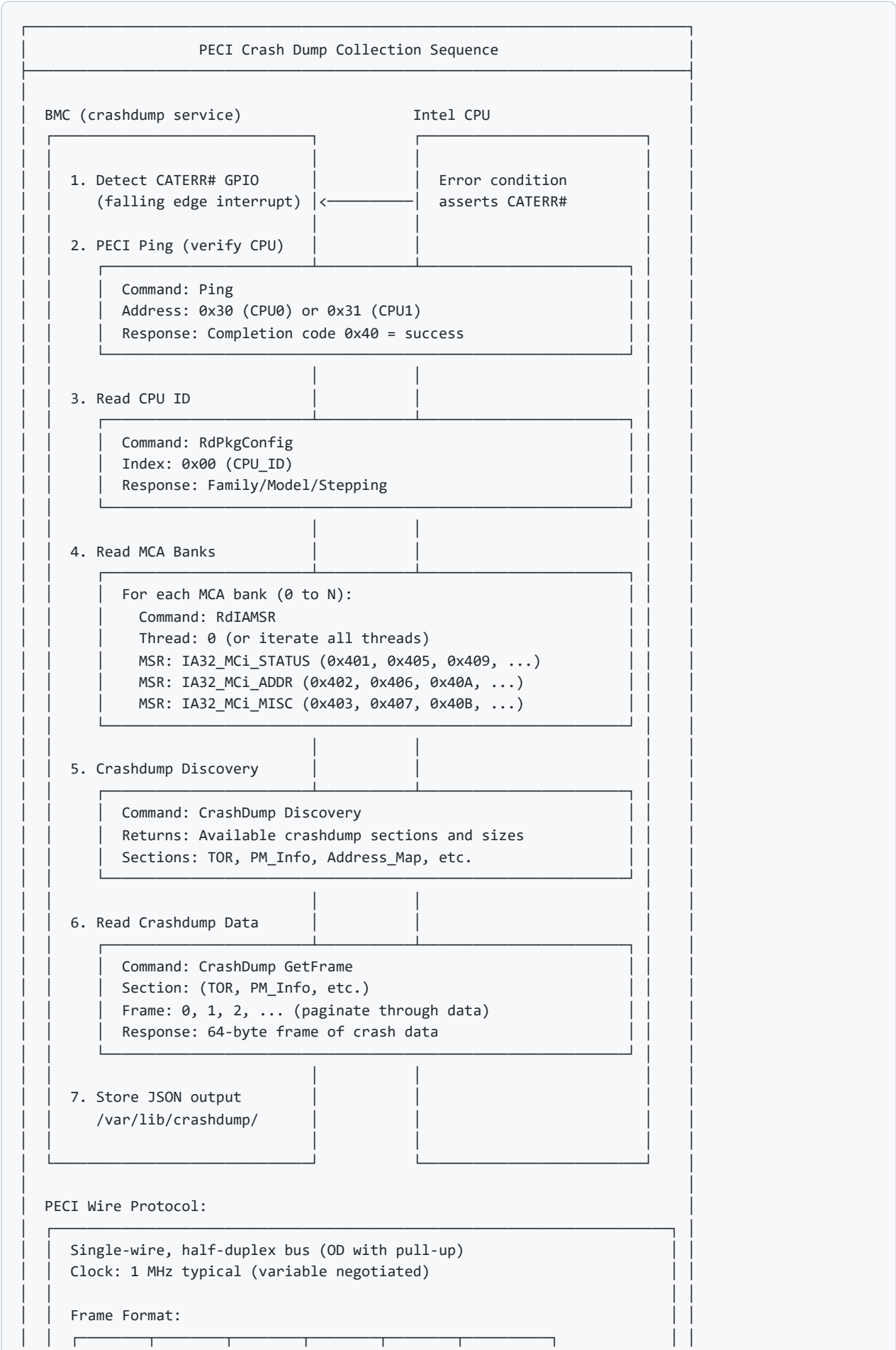
(auto-discover JTAG chain)

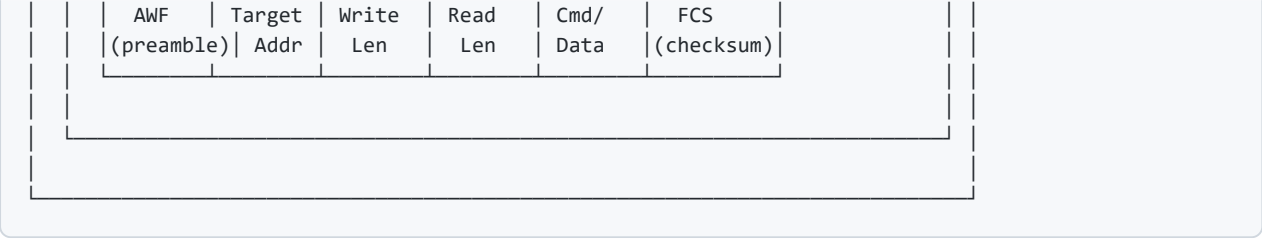
JTAG TAP State Machine



Update-DR	Latch shifted data into target register
Capture-DR	Capture current register value for readout
Run-Test	Execute loaded instruction (run test, BIST, etc.)

PECI Crash Dump Command Flow





MCA Bank Status Register Format

IA32_MCi_STATUS Register (MSR)

64-bit Machine Check Architecture Status Register:

Bit	Name	Description
63	VAL	Valid - Error recorded in this bank
62	OVER	Overflow - Multiple errors, some lost
61	UC	Uncorrected - Error not corrected
60	EN	Enabled - Error reporting was enabled
59	MISCV	MISC Valid - MCI_MISC contains valid info
58	ADDRV	ADDR Valid - MCI_ADDR contains fault address
57	PCC	Processor Context Corrupt - restart required
56	S	Signaled - Error was signaled to software
55	AR	Action Required - Immediate action needed
54-53	LSB	Corrected error local significance
52-38	Other	Model-specific error information
37	FW	Firmware-corrected error
36-32	MSCOD	Model-specific error code
31-16	MCACOD	MCA error code (architecture-defined)
15-0	Reserved	(varies by generation)

MCACOD (bits 15:0) Error Type Examples:

Code	Meaning
0x0001	Unclassified error
0x0005	Internal parity error
0x000A	Generic I/O error
0x010A	Memory controller error (channel A)
0x010C	Memory controller error (channel C)
0x0150	Memory ECC error
0x0400	Watchdog timer (internal)
0x0800	Internal timer error

Decoding Example:

```
MC0_STATUS = 0xBE00000000800400
```

```
Bit 63 (VAL) = 1   -> Error is valid
Bit 62 (OVER) = 0   -> No overflow
Bit 61 (UC) = 1     -> Uncorrected error
Bit 60 (EN) = 1     -> Error reporting enabled
Bit 59 (MISCV) = 1  -> MCI_MISC is valid
Bit 58 (ADDRV) = 1  -> MCI_ADDR is valid
Bit 57 (PCC) = 1    -> Processor context corrupt
MCACOD = 0x0400     -> Watchdog timer error
```

Interpretation: Uncorrected watchdog error, context corrupt,
system restart required

Source Code References

Component	Repository	Key Files
ASD Daemon	Intel-BMC/asd	<code>server/asd_main.c</code> , <code>server/jtag_handler.c</code> , <code>server/target_handler.c</code>
Crashdump Service	openbmc/crashdump	<code>crashdump.cpp</code> , <code>peci.cpp</code> , <code>mca.cpp</code>
PECI Library	openbmc/libpeci	<code>peci.c</code> , <code>peci_cmds.c</code>
bmcweb Crashdump API	openbmc/bmcweb	<code>redfish-core/lib/log_services.hpp</code>
Intel IPMI OEM	openbmc/intel-ipmi-oem	<code>src/oemcommands.cpp</code>

Platform: Intel Xeon platforms only. Features vary by CPU generation.

AMD Debug & Management Guide

Configure AMD EPYC system management and debug capabilities on OpenBMC.

Table of Contents

1. [Overview](#)
2. [AMD Management Interfaces](#)
 1. [SB-RMI \(Sideband Remote Management Interface\)](#)
 2. [SB-TSI \(Sideband Thermal Sensor Interface\)](#)
 3. [HDT \(Hardware Debug Tool\)](#)
3. [APML Setup](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [Kernel Modules](#)
 3. [Device Tree Configuration](#)
4. [APML Library Usage](#)
 1. [Installation](#)
 2. [Reading CPU Temperature](#)
 3. [Power Management](#)
 4. [RAS \(Reliability, Availability, Serviceability\)](#)
 5. [Mailbox Commands](#)
5. [D-Bus Integration](#)
 1. [Sensor Integration](#)
 2. [Power Monitoring](#)
6. [AMD OpenBMC Platform](#)
 1. [AMDESE OpenBMC](#)
 2. [Supported Platforms](#)
7. [HDT Debug Access](#)
 1. [HDT Overview](#)
 2. [BMC JTAG Mux](#)
 3. [Commercial Solutions](#)
8. [Crash Dump Collection](#)
 1. [RAS Error Logging](#)
 2. [BERT \(Boot Error Record Table\)](#)
 3. [Collecting Debug Data](#)
9. [Troubleshooting](#)
 1. [APML Not Working](#)
 2. [Temperature Reading Fails](#)
 3. [Command Timeouts](#)
 4. [Permission Issues](#)

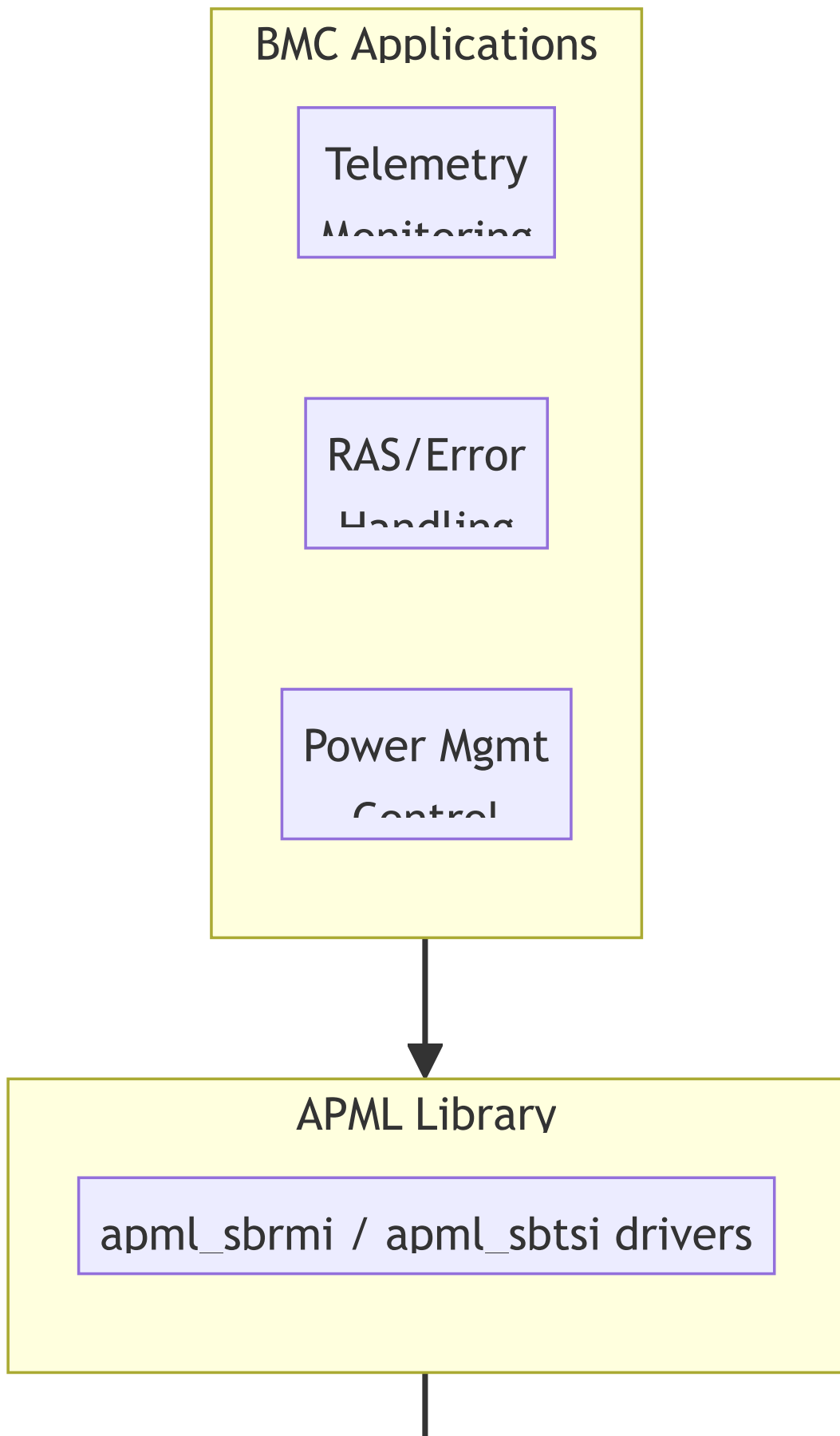
- 10. [Security Considerations](#)
 - 1. [Access Control](#)
 - 2. [Audit Logging](#)
 - 3. [Production Recommendations](#)
 - 11. [References](#)
 - 12. [Deep Dive](#)
 - 1. [APML Communication Architecture](#)
 - 2. [SB-RMI Mailbox Protocol](#)
 - 3. [SB-TSI Temperature Sensing](#)
 - 4. [AMD MCA \(Machine Check Architecture\) Error Handling](#)
 - 5. [Source Code References](#)
-

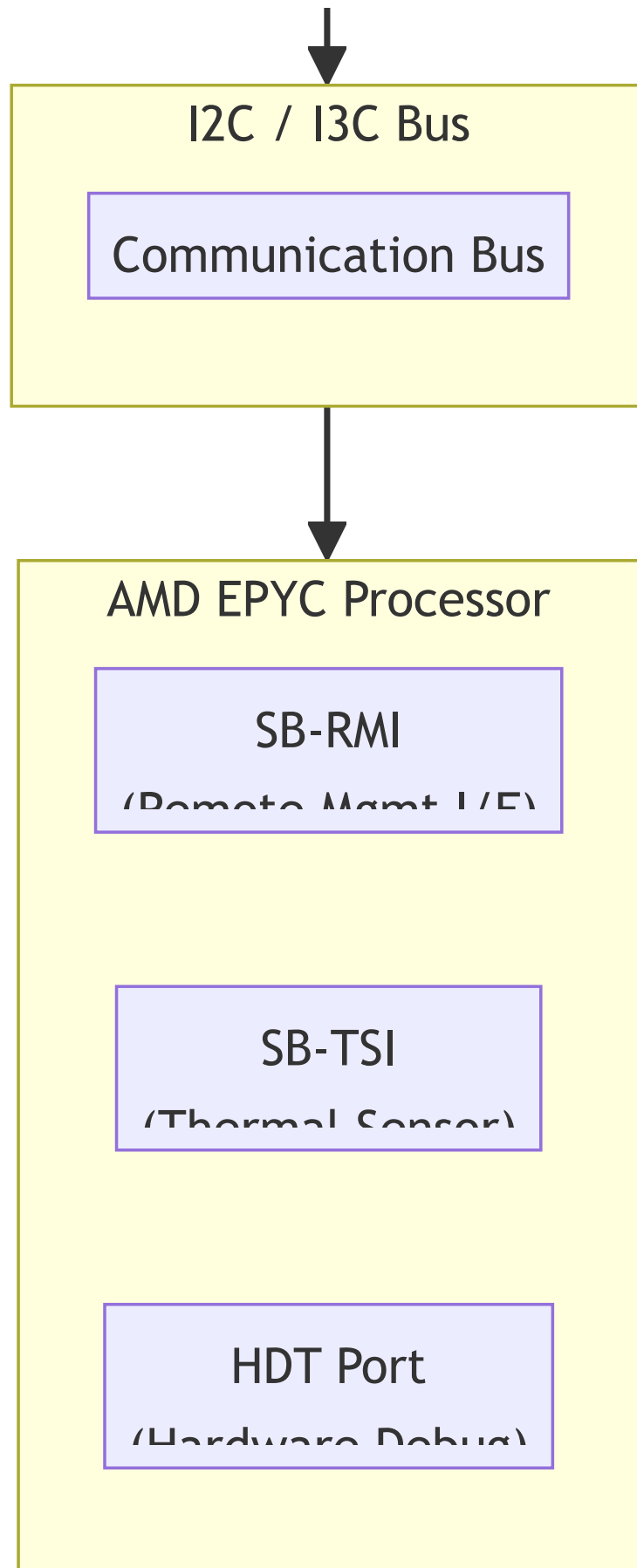
Overview

AMD EPYC platforms provide system management and debug capabilities through **APML (Advanced Platform Management Link)** and **HDT (Hardware Debug Tool)**. This guide covers the BMC integration for AMD server platforms.



AMD Platform Management Architecture





► ASCII-art version (for comparison)

AMD Management Interfaces

SB-RMI (Sideband Remote Management Interface)

Provides system management commands:

Function	Description
Power Management	RAPL power limits, throttling
RAS	Machine check, error reporting
Mailbox Commands	CPU configuration, CPUID
Performance	Core boost, C-states

SB-TSI (Sideband Thermal Sensor Interface)

Provides thermal monitoring:

Function	Description
CPU Temperature	Die temperature reading
Thermal Alerts	High/low temperature thresholds
Thermal Throttling	PROCHOT status

HDT (Hardware Debug Tool)

Low-level debug access (requires special hardware/NDA):

Function	Description
JTAG Access	CPU debug via JTAG chain
Run Control	Halt, step, breakpoints
Register Access	MSR, GPR access
Trace	Instruction trace

APML Setup

Build-Time Configuration (Yocto)

```
# In your machine .conf or local.conf

# Include APML support
IMAGE_INSTALL:append = " \
    apml-modules \
    apml-library \
    esmi-oob-library \
"

# Kernel configuration
KERNEL_MODULE_AUTOLOAD:append = " apml_sbrmi apml_sbtsi"
```

Kernel Modules

```
# Load APML modules
modprobe apml_sbrmi
modprobe apml_sbtsi

# Verify modules loaded
lsmod | grep apml

# Check I2C devices
i2cdetect -l
i2cdetect -y <bus> # Find SB-RMI/SB-TSI addresses
```

Device Tree Configuration

```
&i2c3 {
    status = "okay";

    /* SB-TSI - Thermal sensor */
    sbtsi@4c {
        compatible = "amd,sbtsi";
        reg = <0x4c>;
    };

    /* SB-RMI - Remote management */
    sbrmi@3c {
        compatible = "amd,sbrmi";
        reg = <0x3c>;
    };
};
```

APML Library Usage

Installation

```
# Check APML library
ls /usr/lib/libapml*

# Check APML tools
which esmi_oob_tool
```

Reading CPU Temperature

```
# Via hwmon (SB-TSI driver)
cat /sys/class/hwmon/hwmon*/temp1_input

# Via APML library tool
esmi_oob_tool -s 0 --showtemprange
```

Power Management

```
# Read current power
esmi_oob_tool -s 0 --showpower

# Read power limit
esmi_oob_tool -s 0 --showpowerlimit

# Set power limit (in milliwatts)
esmi_oob_tool -s 0 --setpowerlimit 200000

# Read TDP
esmi_oob_tool -s 0 --showtdp
```

RAS (Reliability, Availability, Serviceability)

```
# Read MCA (Machine Check Architecture) status
esmi_oob_tool -s 0 --showmcastatus

# Read DIMM temperature
esmi_oob_tool -s 0 --showdimmtemp

# Read DIMM power
esmi_oob_tool -s 0 --showdimpower

# Read DIMM thermal sensor
esmi_oob_tool -s 0 --showdimmthermal
```

Mailbox Commands

```
# Read CPUID
esmi_oob_tool -s 0 --showcpuid

# Read processor info
esmi_oob_tool -s 0 --showprocinfo

# Read boost limit
esmi_oob_tool -s 0 --showboostlimit

# Set boost limit per core
esmi_oob_tool -s 0 --setboostlimit <core> <limit>
```

D-Bus Integration

Sensor Integration

APML sensors integrate with OpenBMC's sensor framework:

```
# Temperature sensors from SB-TSI appear in D-Bus
busctl tree xyz.openbmc_project.HwmonTempSensor

# Example: CPU temperature
busctl get-property xyz.openbmc_project.HwmonTempSensor \
    /xyz/openbmc_project/sensors/temperature/CPU0_Temp \
    xyz.openbmc_project.Sensor.Value Value
```

Power Monitoring

```
# Power sensors via APML
busctl tree xyz.openbmc_project.Sensor

# Socket power reading
busctl get-property xyz.openbmc_project.Sensor \
    /xyz/openbmc_project/sensors/power/CPU0_Power \
    xyz.openbmc_project.Sensor.Value Value
```

AMD OpenBMC Platform

AMDESE OpenBMC

AMD maintains an OpenBMC distribution for EPYC platforms:

```
# Clone AMD's OpenBMC
git clone https://github.com/AMDESE/OpenBMC.git

# Build for Genoa platform
cd OpenBMC
. setup onyx # or other platform
bitbake obmc-phosphor-image
```

Supported Platforms

Platform	Codename	Socket
Onyx	Genoa	SP5
Quartz	Genoa	SP5
Ruby	Turin	SP5
Chalupa	Genoa	SP5

HDT Debug Access

HDT provides deep hardware access. Full documentation requires AMD NDA. Use only in controlled debug environments.

HDT Overview

HDT (Hardware Debug Tool) is AMD’s JTAG-based debug interface:

- Accessed via dedicated debug header or BMC mux
- Requires specialized debug tools (AMD HDT software or third-party)
- Provides run-control (halt, step, breakpoints)
- Full register and memory access

BMC JTAG Mux

Some platforms support JTAG muxing through BMC:

```
# Check for JTAG mux support
cat /sys/kernel/debug/gpio | grep -i jtag

# JTAG mux control (platform-specific)
gpiochip0 <jtag_mux_gpio>=1 # Route JTAG to BMC
```

Commercial Solutions

For at-scale debug, commercial tools provide BMC-embedded debug:

- ASSET InterTech ScanWorks Embedded Diagnostics (SED)
- Lauterbach TRACE32 with remote access
- AMD HDT software (under NDA)

Crash Dump Collection

RAS Error Logging

```
# Machine Check errors are logged to SEL
ipmitool sel list | grep -i "Machine Check"

# View via Redfish
curl -k -u root:0penBmc \
  https://localhost/redfish/v1/Systems/system/LogServices/EventLog/Entries
```

BERT (Boot Error Record Table)

```
# BERT records are available via ACPI
cat /sys/firmware/acpi/tables/BERT

# Decoded in system logs
dmesg | grep -i "BERT"
journalctl | grep -i "hardware error"
```

Collecting Debug Data

```
# Collect system state dump
dreport -d /tmp/debug -n amd_debug -t user

# Include APML data
esmi_oob_tool -s 0 --showallinfo > /tmp/apml_state.txt

# Read MCA banks
esmi_oob_tool -s 0 --showmcastatus > /tmp/mca_status.txt
```

Troubleshooting

APML Not Working

```
# Check I2C bus connectivity
i2cdetect -y 3 # Replace with correct bus

# Verify kernel modules
lsmod | grep apml

# Check dmesg for errors
dmesg | grep -i "sbrmi\|sbtsi\|apml"

# Verify device nodes
ls -la /dev/sbrmi* /dev/sbtsi*
```

Temperature Reading Fails

```
# Check hwmon path
ls /sys/class/hwmon/

# Find correct hwmon device
for h in /sys/class/hwmon/hwmon*; do
    name=$(cat $h/name 2>/dev/null)
    echo "$h: $name"
done

# Read directly
cat /sys/class/hwmon/hwmon<N>/temp1_input
```

Command Timeouts

```
# APML commands may timeout under heavy load
# Increase timeout in library calls

# Check SMBus errors
dmesg | grep -i "i2c\|smbus"

# Retry with delay
sleep 1 && esmi_oob_tool -s 0 --showpower
```

Permission Issues

```
# APML requires root or proper group membership
ls -la /dev/sbrmi0 /dev/sbtsi0

# Add user to appropriate group
usermod -aG i2c <username>
```

Security Considerations

Access Control

```
# Restrict APML access to administrators only
chmod 600 /dev/sbrmi* /dev/sbtsi*

# Use D-Bus policy for access control
# /etc/dbus-1/system.d/apml.conf
```

Audit Logging

```
# Log APML access
# Integrate with phosphor-logging for audit trail
```

Production Recommendations

- Disable HDT access in production (via fuses or GPIO)
- Restrict APML to essential operations

- Monitor for anomalous power/thermal changes
 - Log all RAS events
-

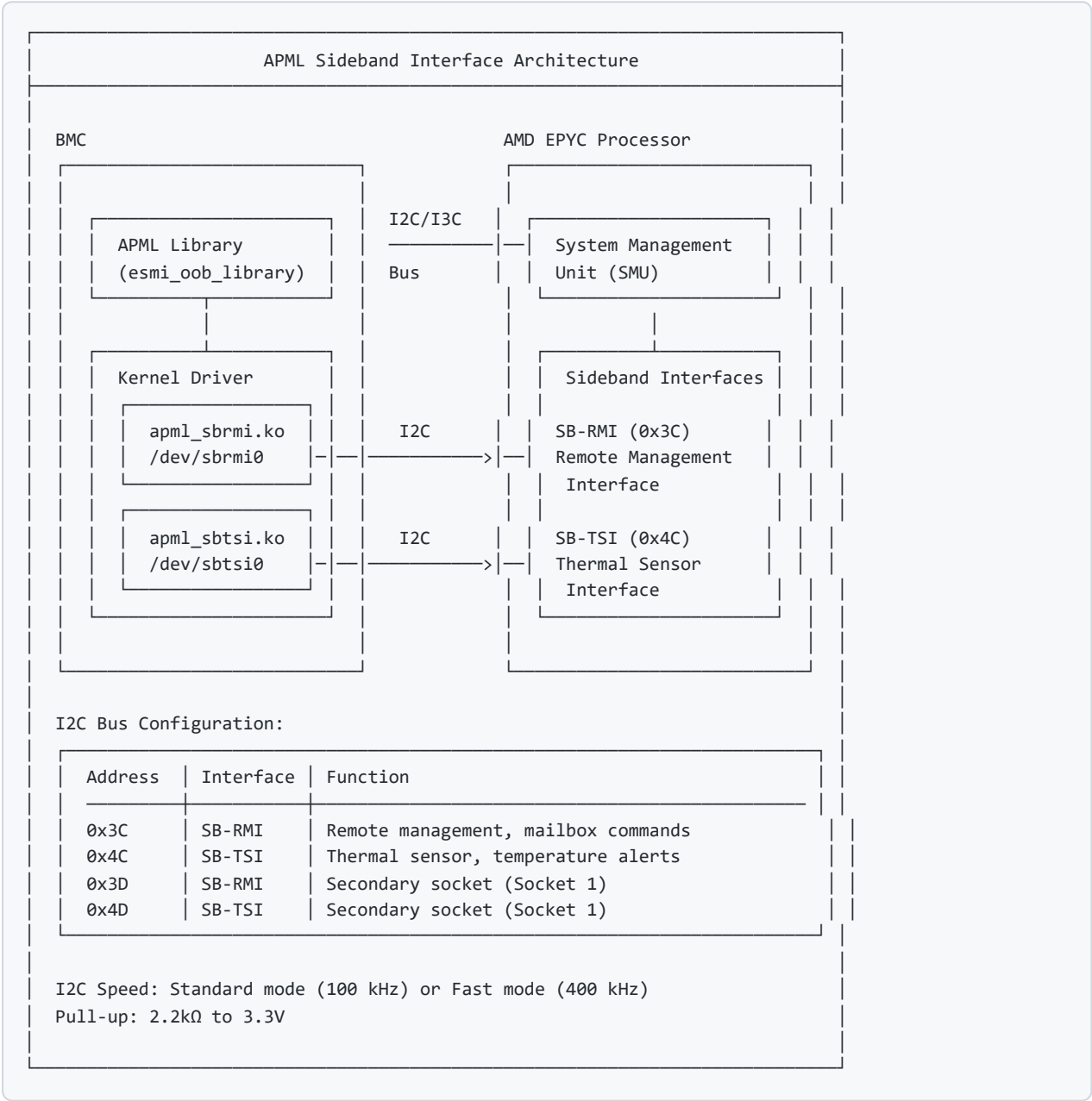
References

- [AMD APML Modules \(GitHub\)](#) - Kernel drivers for SB-RMI/SB-TSI
 - [AMD E-SMI In-Band Library \(GitHub\)](#) - In-band system management
 - [AMD E-SMS APML Library](#) - Out-of-band APML library
 - [AMDESE OpenBMC \(GitHub\)](#) - AMD's OpenBMC distribution for EPYC
 - [AMD E-SMS Developer Resources](#) - EPYC System Management Software
 - [AMD Technical Documentation Portal](#) - Official AMD documentation
-

Deep Dive

This section provides detailed technical information for developers who want to understand AMD APML internals and SB-RMI/SB-TSI protocols.

APML Communication Architecture



SB-RMI Mailbox Protocol



0x12	ReadDimmPower	Read DIMM power consumption
0x20	ReadCpuId	Read CPU family/model/stepping

SB-TSI Temperature Sensing

SB-TSI Thermal Interface Protocol

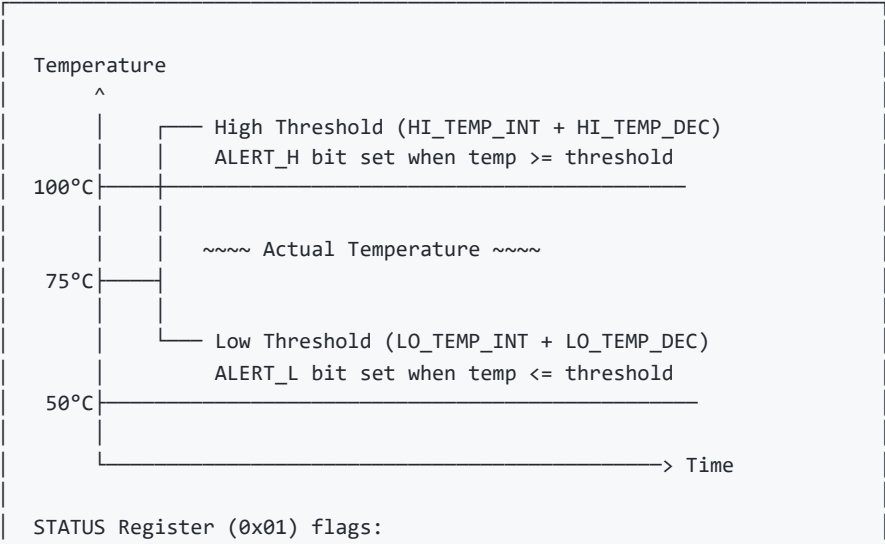
Register Map:

Offset	Register	Access	Description
0x00	CPU_TEMP_INT	R	CPU temp integer part (°C)
0x01	STATUS	R	Status and alert flags
0x02	CONFIG	R/W	Configuration register
0x03	UPDATE_RATE	R/W	Temperature update rate
0x04	HI_TEMP_INT	R/W	High temp threshold integer
0x05	LO_TEMP_INT	R/W	Low temp threshold integer
0x10	CPU_TEMP_DEC	R	CPU temp decimal part
0x11	CPU_TEMP_OFFSET_INT	R/W	Temperature offset integer
0x13	HI_TEMP_DEC	R/W	High temp threshold decimal
0x14	LO_TEMP_DEC	R/W	Low temp threshold decimal
0x19	TIMEOUT_CONFIG	R/W	Alert timeout configuration
0xFE	VENDOR_ID	R	Vendor ID (0x00 = AMD)
0xFF	DEVICE_ID	R	Device ID

Temperature Reading:

$\text{Temperature} = \text{CPU_TEMP_INT} + (\text{CPU_TEMP_DEC} \times 0.125)$
CPU_TEMP_DEC bits[7:5] represent 0.125°C increments:
000 = 0.000°C
001 = 0.125°C
010 = 0.250°C
011 = 0.375°C
100 = 0.500°C
101 = 0.625°C
110 = 0.750°C
111 = 0.875°C
Example: INT=55, DEC=0x40 (010 in bits[7:5])
Temperature = 55 + 0.250 = 55.25°C

Threshold Alert Mechanism:



Bit 0: ALERT_L - Temperature at or below low threshold
Bit 1: ALERT_H - Temperature at or above high threshold
Bit 7: Busy - Temperature conversion in progress

AMD MCA (Machine Check Architecture) Error Handling

AMD MCA Error Collection via APML

MCA Bank Layout (AMD Zen Architecture):

Bank	Description	Error Types
0	Load-Store Unit	Cache/TLB errors
1	Instruction Fetch Unit	IC/IT errors
2	L2 Cache Unit	L2 tag/data errors
3	Decode Unit	Microcode/instruction errors
4	Reserved	
5	Execution Unit	FP/INT execution errors
6	Floating Point Unit	FP register file errors
7-15	Core-specific banks	Varies by generation
16+	Uncore banks (UMC, DF, etc.)	Memory, fabric errors

Reading MCA via APML Mailbox:

Command: ReadMcaData (0x10)

Input:

InBndMsg[0]: Core/Thread ID

InBndMsg[1]: MCA Bank number

InBndMsg[2]: Register offset (0=STATUS, 1=ADDR, 2=MISC, etc.)

Output:

OutBndMsg[0-7]: 64-bit register value

Example reading MC0_STATUS:

```
esmi_oob_tool -s 0 --readmsr --core 0 --bank 0 --offset 0
```

MCA_STATUS Register Format (AMD Zen):

Bit	Field	Description
63	Val	Error information valid
62	Overflow	Multiple errors, information may be lost
61	UC	Uncorrected error
60	En	Error reporting enabled
59	MiscV	MCA_MISC register valid
58	AddrV	MCA_ADDR register valid
57	PCC	Processor context corrupt
56	TCC	Task context corrupt
55	Poison	Poisoned data consumed
54	Deferred	Deferred error (reported later)
47:32	ExtErrCode	Extended error code
31:16	ModelErrCode	Model-specific error code
15:0	ErrCode	Error code type

Error Types (ErrCode bits 15:0):

Pattern	Type	Description
0000 0000	No error	
0000 0001	Unclassified	Internal parity/protocol error

0000 0101	Internal parity	Core internal error
000x xxxx	TLB errors	TLB parity/multimatch
001x xxxx	Memory errors	Cache hierarchy errors
010x xxxx	Bus/Interconnect	Data fabric/IF errors
1xxx xxxx	Memory controller	DRAM/UMC errors

Source Code References

Component	Repository	Key Files
APML Kernel Modules	amd/apml_modules	<code>sbrmi/apml_sbrmi.c</code> , <code>sbtsi/apml_sbtsi.c</code>
E-SMI OOB Library	amd/esmi_oob_library	<code>src/esmi_oob.c</code> , <code>src/apml.c</code>
E-SMI In-Band Library	amd/esmi_ib_library	<code>src/e_smi.c</code>
AMDESE OpenBMC	AMDESE/OpenBMC	<code>meta-amd/</code>
hwmon sbtsi driver	linux/drivers/hwmon	<code>drivers/hwmon/sbtsi_temp.c</code>

Platform: AMD EPYC 7002 (Rome), 7003 (Milan), 9004 (Genoa), and newer platforms. APML requires Family 17h (Zen) or later processors.

Logging Guide

Configure and use event logging in OpenBMC.

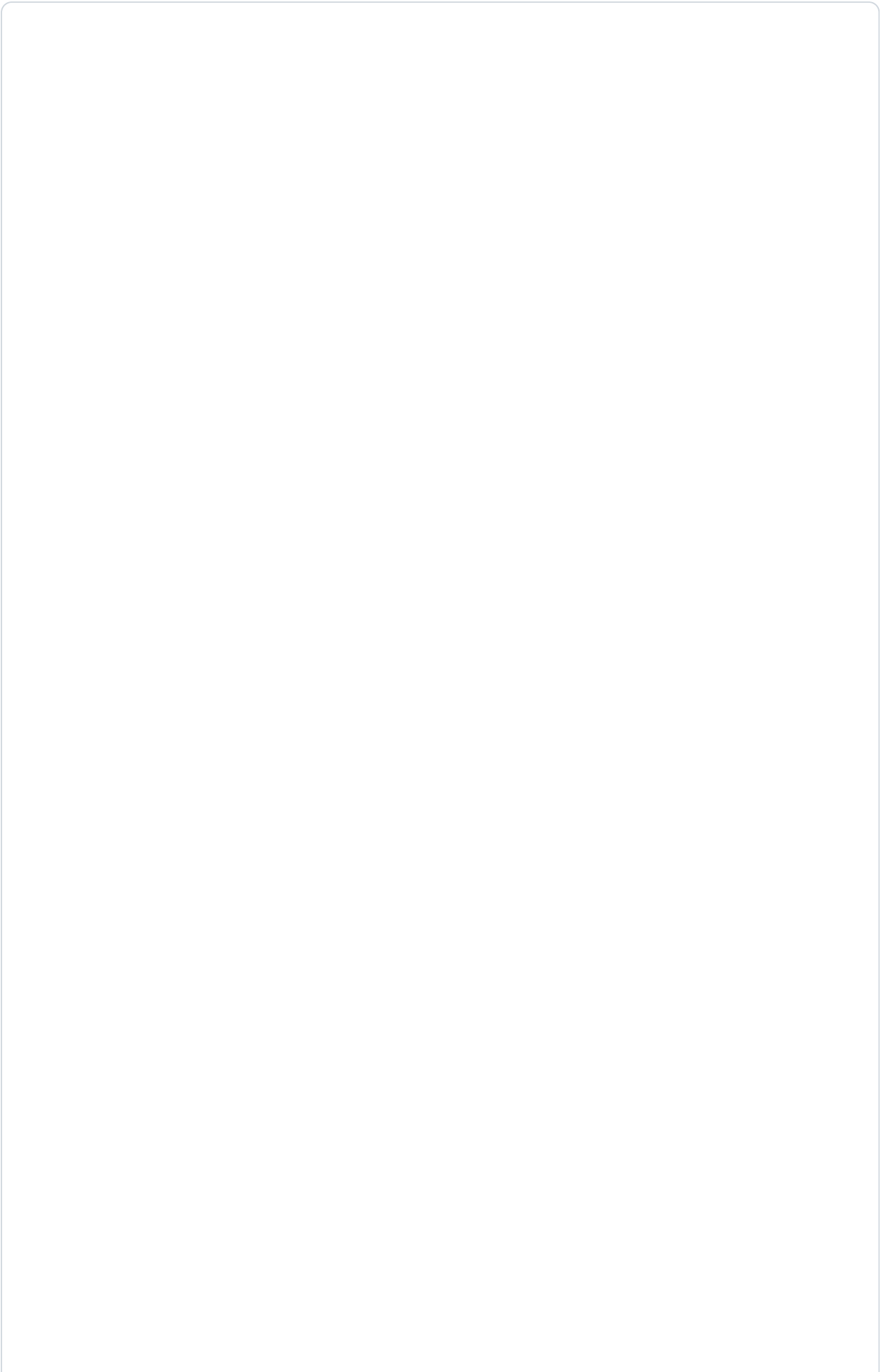
Table of Contents

1. [Overview](#)
2. [Setup & Configuration](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [Meson Build Options](#)
 3. [Runtime Configuration](#)
3. [Creating Event Logs](#)
 1. [Via D-Bus API](#)
 2. [Log Severity Levels](#)
 3. [From Application Code](#)
 4. [Using elog for Structured Errors](#)
4. [Viewing Event Logs](#)
 1. [Via D-Bus](#)
 2. [Via Redfish](#)
 3. [Via IPMI SEL](#)
5. [Managing Logs](#)
 1. [Delete Single Log](#)
 2. [Clear All Logs](#)
 3. [Log Rotation](#)
6. [SEL Logger](#)
 1. [Configuration](#)
 2. [SEL Record Types](#)
 3. [IPMI SEL Commands](#)
7. [SDR \(Sensor Data Record\)](#)
 1. [SDR Types](#)
 2. [View SDR](#)
 3. [Dynamic SDR Generation](#)
8. [POST Codes](#)
 1. [Setup](#)
 2. [View POST Codes](#)
 3. [POST Code History](#)
9. [Debug Collector \(dreport\)](#)
 1. [Create Debug Report](#)
 2. [Via Redfish](#)
 3. [Dump Contents](#)

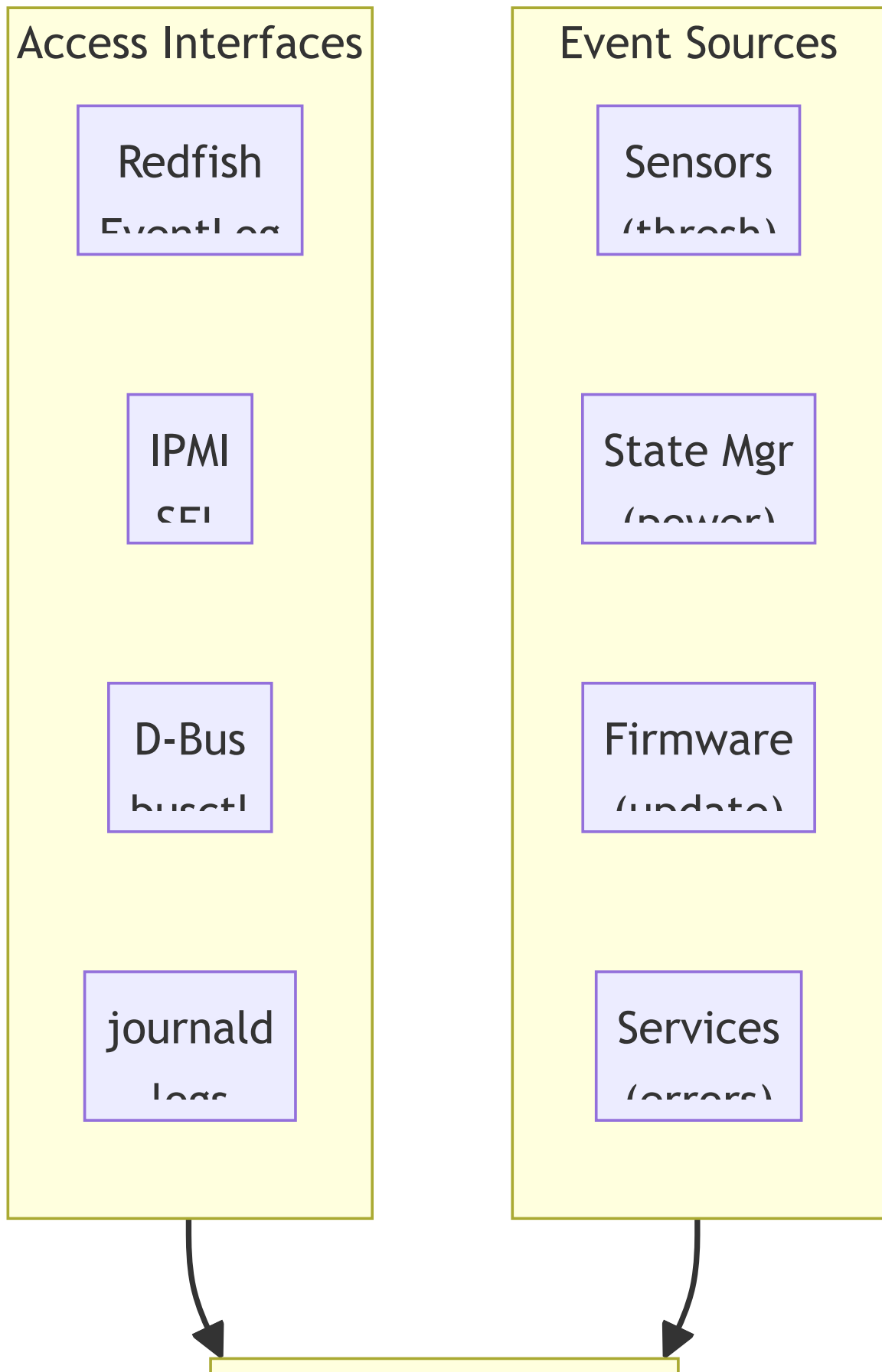
10. [Remote Logging](#)
 1. [Syslog Configuration](#)
 2. [Redfish Event Forwarding](#)
 11. [Troubleshooting](#)
 1. [No Logs Appearing](#)
 2. [SEL Not Updating](#)
 3. [Logs Fill Up](#)
 12. [Custom Event Log Example](#)
 1. [Where to Put Error Definitions](#)
 2. [Step 1: Define Error Type](#)
 3. [Step 2: Add to Meson Build](#)
 4. [Step 3: Use in Code](#)
 5. [Remote Syslog Deployment via Yocto](#)
 6. [Log Storage Configuration](#)
 13. [Deep Dive](#)
 1. [phosphor-logging Architecture](#)
 2. [Error Entry Structure](#)
 3. [Error Reporting API](#)
 4. [IPMI SEL Integration](#)
 5. [Log Rotation and Persistence](#)
 6. [Source Code Reference](#)
 14. [Examples](#)
 15. [References](#)
-

Overview

OpenBMC provides comprehensive logging capabilities through **phosphor-logging** and related services. Logs are accessible via D-Bus, IPMI SEL, and Redfish APIs.



Logging Architecture



phosphor-logging

Log Manager

(DBUG ADI)

Error Creation

& Storage

Resolving

& Cleanup



/var/lib/phosphor-logging/errors/

► ASCII-art version (for comparison)

Setup & Configuration

Build-Time Configuration (Yocto)

```
# In your machine .conf or local.conf

# Include logging packages
IMAGE_INSTALL:append = " \
    phosphor-logging \
    phosphor-sel-logger \
    phosphor-log-manager \
"

# Configure logging options
EXTRA_OEMESON:pn-phosphor-logging = " \
    -Dpersistence=enabled \
    -Dremote-host-config=enabled \
"
```

Meson Build Options

Option	Default	Description
<code>persistence</code>	enabled	Persist logs across reboots
<code>remote-host-config</code>	disabled	Remote syslog support
<code>openpower-pel</code>	disabled	IBM PEL format logs

Runtime Configuration

```
# Check logging service
systemctl status phosphor-log-manager

# View service logs
journalctl -u phosphor-log-manager -f

# Log storage location
ls /var/lib/phosphor-logging/errors/
```

Creating Event Logs

Via D-Bus API

```
# Create an informational log
busctl call xyz.openbmc_project.Logging \
  /xyz/openbmc_project/logging \
  xyz.openbmc_project.Logging.Create \
  Create ssas \
  "xyz.openbmc_project.Common.Error.InternalFailure" \
  "xyz.openbmc_project.Logging.Entry.Level.Informational" \
  2 \
  "COMPONENT=MyService" \
  "REASON=Test log entry"
```

Log Severity Levels

Level	Description	Use Case
Emergency	System unusable	Critical failures
Alert	Immediate action needed	Security events
Critical	Critical conditions	Hardware failures
Error	Error conditions	Service errors
Warning	Warning conditions	Threshold warnings
Notice	Normal but significant	State changes
Informational	Informational	Status updates
Debug	Debug messages	Development

From Application Code

Source Reference: [phosphor-logging](#)

- Logging API: [phosphor-logging/log.hpp](#)
- Error definitions: [phosphor-logging/elog.hpp](#)

```
#include <phosphor-logging/log.hpp>
#include <phosphor-logging/elog.hpp>
#include <phosphor-logging/elog-errors.hpp>

using namespace phosphor::logging;

// Log an error
log<level::ERR>("Sensor read failed",
  entry("SENSOR=%s", "CPU_Temp"),
  entry("ERROR=%d", errno));

// Log an informational message
log<level::INFO>("Service started",
  entry("VERSION=%s", "1.0.0"));
```

Using elog for Structured Errors

```
#include <xyz/openbmc_project/Common/error.hpp>

using namespace sdbusplus::xyz::openbmc_project::Common::Error;

// Report a structured error
try {
    // Some operation
    if (error_condition) {
        report<InternalFailure>();
    }
} catch (const std::exception& e) {
    log<level::ERR>("Exception caught", entry("WHAT=%s", e.what()));
}
```

Viewing Event Logs

Via D-Bus

```
# List all log entries
busctl tree xyz.openbmc_project.Logging

# Get log entry details
busctl introspect xyz.openbmc_project.Logging \
    /xyz/openbmc_project/logging/entry/1

# Get log message
busctl get-property xyz.openbmc_project.Logging \
    /xyz/openbmc_project/logging/entry/1 \
    xyz.openbmc_project.Logging.Entry \
    Message
```

Via Redfish

```
# Get event log entries
curl -k -u root:0penBmc \
    https://localhost/redfish/v1/Systems/system/LogServices/EventLog/Entries

# Get specific entry
curl -k -u root:0penBmc \
    https://localhost/redfish/v1/Systems/system/LogServices/EventLog/Entries/1
```

Via IPMI SEL

```
# Get SEL entries
ipmitool -I lanplus -H bmc-ip -U root -P 0penBmc sel list

# Get SEL info
ipmitool -I lanplus -H bmc-ip -U root -P 0penBmc sel info
```

Managing Logs

Delete Single Log

```
# Via D-Bus
busctl call xyz.openbmc_project.Logging \
  /xyz/openbmc_project/logging/entry/1 \
  xyz.openbmc_project.Object.Delete \
  Delete

# Via Redfish
curl -k -u root:0penBmc -X DELETE \
  https://localhost/redfish/v1/Systems/system/LogServices/EventLog/Entries/1
```

Clear All Logs

```
# Via Redfish
curl -k -u root:0penBmc -X POST \
  https://localhost/redfish/v1/Systems/system/LogServices/EventLog/Actions/LogService.ClearLog

# Via IPMI
ipmitool -I lanplus -H bmc-ip -U root -P 0penBmc sel clear
```

Log Rotation

```
# Log capacity configuration
# Default: 200 entries maximum

# Configure via D-Bus property
busctl set-property xyz.openbmc_project.Logging \
  /xyz/openbmc_project/logging \
  xyz.openbmc_project.Logging.Settings \
  QuiesceOnHwError b false
```

SEL Logger

The SEL Logger creates IPMI-compatible SEL entries from sensor events.

Configuration

```
# Check SEL logger service
systemctl status phosphor-sel-logger

# SEL logger monitors threshold events
# and creates SEL entries automatically
```

SEL Record Types

Type	Description
System Event	Standard SEL entry
OEM Timestamped	Vendor-specific with timestamp
OEM Non-timestamped	Vendor-specific without timestamp

IPMI SEL Commands

```
# Get SEL information
ipmitool sel info

# Get SEL entries
ipmitool sel list

# Get extended info
ipmitool sel elist

# Clear SEL
ipmitool sel clear

# Get specific record
ipmitool sel get 1
```

SDR (Sensor Data Record)

SDR describes sensors and their properties for IPMI.

SDR Types

Type	Description
Full Sensor	Complete sensor definition
Compact Sensor	Abbreviated sensor record
Event-Only	Event generation info
FRU Device	FRU information
MC Device	Management controller

View SDR

```
# List all SDR records
ipmitool sdr list

# Get detailed SDR info
ipmitool sdr info

# Get specific sensor type
ipmitool sdr type Temperature
```


Dynamic SDR Generation

OpenBMC generates SDR dynamically from D-Bus sensor objects:

```
# SDR is generated from:
# - xyz.openbmc_project.Sensor.* interfaces
# - threshold definitions
# - sensor metadata
```

POST Codes

POST (Power-On Self-Test) codes capture boot progress.

Setup

```
# Include POST code manager
IMAGE_INSTALL:append = " phosphor-post-code-manager"
```

View POST Codes

```
# Via D-Bus
busctl tree xyz.openbmc_project.State.Boot.Raw

# Get current POST code
busctl get-property xyz.openbmc_project.State.Boot.Raw \
    /xyz/openbmc_project/state/boot/raw0 \
    xyz.openbmc_project.State.Boot.Raw \
    Value

# Via Redfish
curl -k -u root:0penBmc \
    https://localhost/redfish/v1/Systems/system/LogServices/PostCodes/Entries
```

POST Code History

```
# Get POST code history
busctl call xyz.openbmc_project.State.Boot.Raw \
    /xyz/openbmc_project/state/boot/raw0 \
    xyz.openbmc_project.State.Boot.Raw \
    GetPostCodes q 0 # Boot cycle 0 (current)
```

Debug Collector (dreport)

Collect diagnostic information for debugging.

Create Debug Report

```
# Generate report
dreport -d /tmp/debug -n myreport -t user

# Report types:
# user      - User-initiated dump
# core      - Core dump
# elog      - Error log dump
# pel       - PEL dump (OpenPOWER)
```

Via Redfish

```
# Create dump
curl -k -u root:openBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{"DiagnosticDataType": "Manager"}' \
  https://localhost/redfish/v1/Managers/bmc/LogServices/Dump/Actions/LogService.CollectDiagnosticData

# List dumps
curl -k -u root:openBmc \
  https://localhost/redfish/v1/Managers/bmc/LogServices/Dump/Entries

# Download dump
curl -k -u root:openBmc -o dump.tar.xz \
  https://localhost/redfish/v1/Managers/bmc/LogServices/Dump/Entries/1/attachment
```

Dump Contents

```
# Extract and examine
tar -xf dump.tar.xz

# Contents typically include:
# - Journal logs
# - D-Bus state
# - System configuration
# - Core files (if applicable)
```

Remote Logging

Syslog Configuration

```
# Configure remote syslog
# /etc/rsyslog.d/remote.conf
*. * @@remote-server:514
```

Redfish Event Forwarding

```
# Create event subscription
curl -k -u root:openBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{
    "Destination": "https://log-server:8080/events",
    "Protocol": "Redfish",
    "EventTypes": ["Alert", "StatusChange"]
  }' \
  https://localhost/redfish/v1/EventService/Subscriptions
```

Troubleshooting

No Logs Appearing

```
# Check logging service
systemctl status phosphor-log-manager

# Check permissions
ls -la /var/lib/phosphor-logging/

# Verify D-Bus service
busctl status xyz.openbmc_project.Logging
```

SEL Not Updating

```
# Check SEL logger service
systemctl status phosphor-sel-logger

# Verify threshold events are configured
busctl introspect xyz.openbmc_project.Sensors.* \
  /xyz/openbmc_project/sensors/*/*
```

Logs Fill Up

```
# Check log count
busctl tree xyz.openbmc_project.Logging | wc -l

# Clear old logs
curl -k -u root:openBmc -X POST \
  https://localhost/redfish/v1/Systems/system/LogServices/EventLog/Actions/LogService.ClearLog
```

Custom Event Log Example

Where to Put Error Definitions

Custom error definitions live alongside your service code, following the phosphor-dbus-interfaces YAML format:

File	Path Convention	Description
Error YAML	<code><your-repo>/xyz/openbmc_project/MyService/MyError.errors.yaml</code>	Error type definitions
Metadata YAML	<code><your-repo>/xyz/openbmc_project/MyService/MyError.metadata.yaml</code>	Additional data fields
Generated header	<code><build-dir>/xyz/openbmc_project/MyService/error.hpp</code>	Auto-generated C++ header

Step 1: Define Error Type

```
# xyz/openbmc_project/MyService/MyError.errors.yaml
- name: ConfigurationError
  description: Configuration file is invalid
  metadata:
    - name: FILE
      type: string
    - name: LINE
      type: int32
```

Step 2: Add to Meson Build

```
# In meson.build of your service
phosphor_logging_dep = dependency('phosphor-logging')

# Generate error headers from YAML
error_yaml = files('xyz/openbmc_project/MyService/MyError.errors.yaml')
error_hpp = phosphor_logging_gen.process(
    error_yaml,
    preserve_path_from: meson.current_source_dir())
```

Step 3: Use in Code

```
#include <xyz/openbmc_project/MyService/error.hpp>

using MyConfigError = sdbusplus::xyz::openbmc_project::MyService::Error::ConfigurationError;

// Report the error
report<MyConfigError>(
    xyz::openbmc_project::MyService::ConfigurationError::FILE("config.json"),
    xyz::openbmc_project::MyService::ConfigurationError::LINE(42)
);
```

Remote Syslog Deployment via Yocto

To bake a default remote syslog destination into the image:

```
# meta-myplatform/recipes-phosphor/logging/
# └─ files/
#   └─ remote.conf
# └─ phosphor-logging_%.bbappend

cat > phosphor-logging_%.bbappend << 'EOF'
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
SRC_URI += "file://remote.conf"

do_install:append() {
    install -d ${D}${sysconfdir}/rsyslog.d
    install -m 0644 ${WORKDIR}/remote.conf \
        ${D}${sysconfdir}/rsyslog.d/
}
EOF
```

Example `remote.conf` :

```
# Forward all logs to central server
*. * @log-server.example.com:514
```

Log Storage Configuration

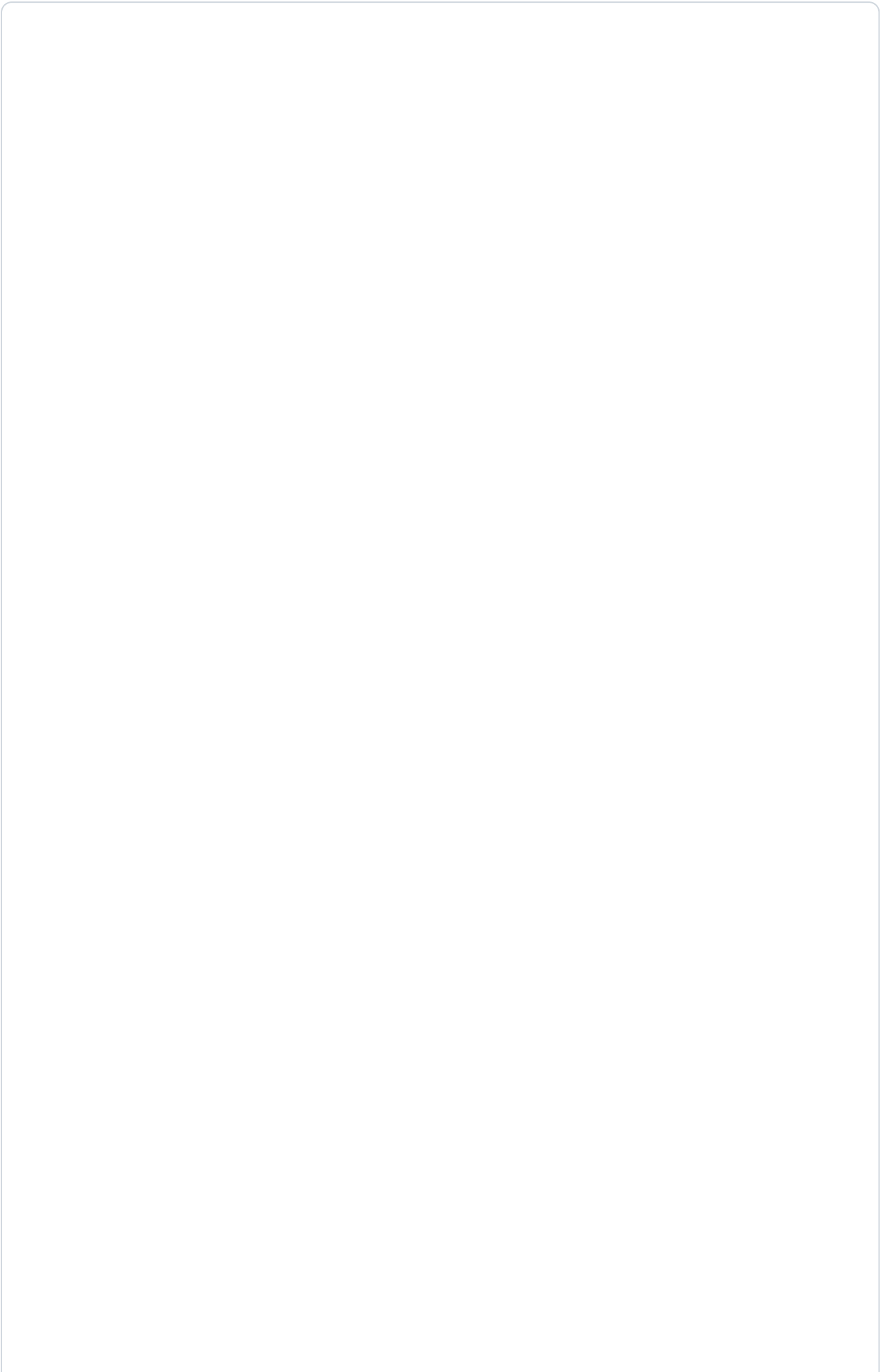
Setting	Default	Where to Change
Max error entries	200	<code>ERROR_CAP</code> in phosphor-logging meson options
Max info entries	10	<code>ERROR_INFO_CAP</code> in meson options
Persistence path	<code>/var/lib/phosphor-logging/errors/</code>	Compiled into binary
Remote syslog	Disabled	<code>/etc/rsyslog.d/*.conf</code> or Redfish EventService

Customize log capacity via Yocto:

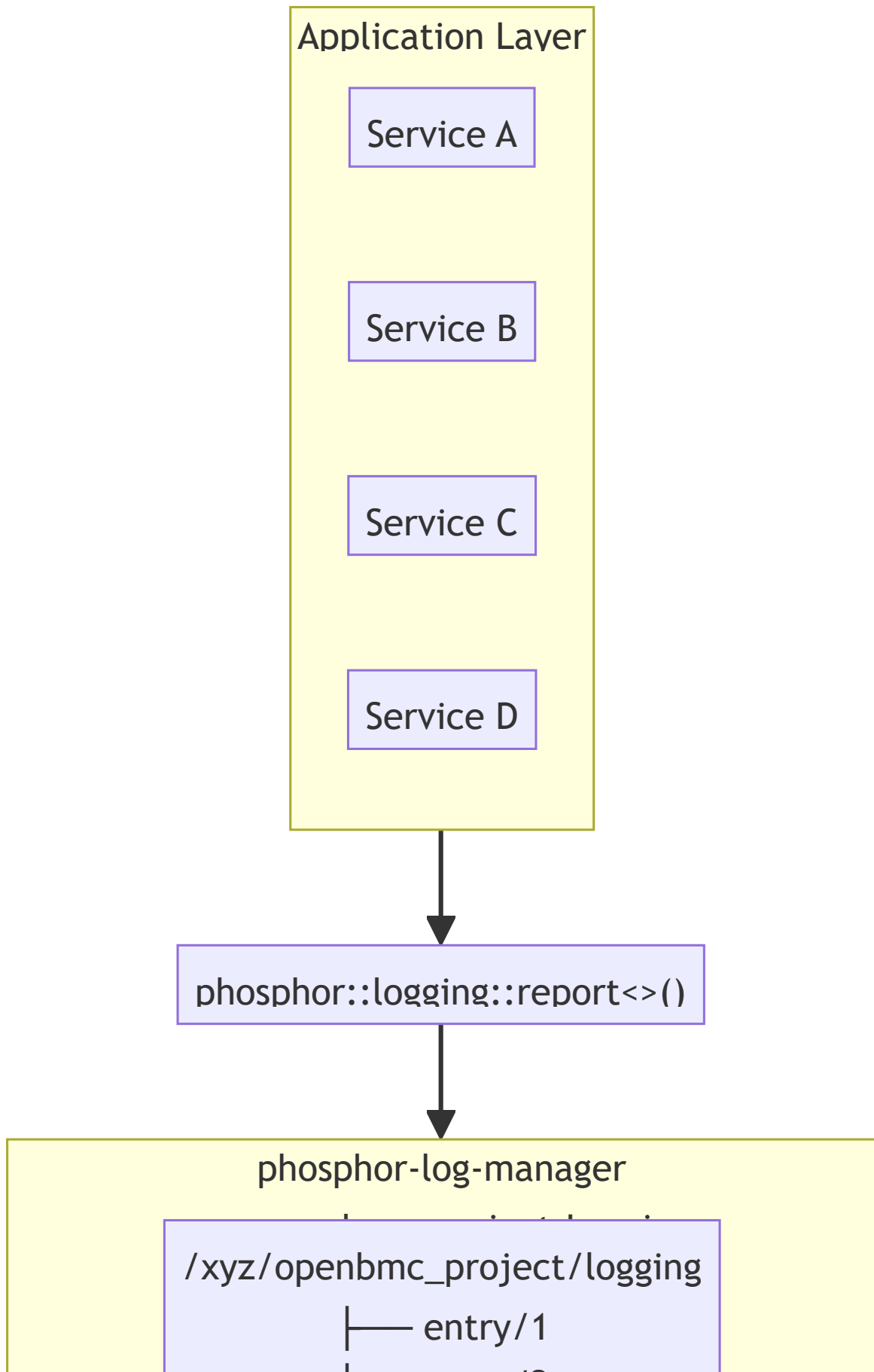
```
# meta-myplatform/recipes-phosphor/logging/phosphor-logging_%.bbappend
EXTRA_OEMESON:append = " \
    -Derror_cap=500 \
    -Derror_info_cap=50 \
"
```

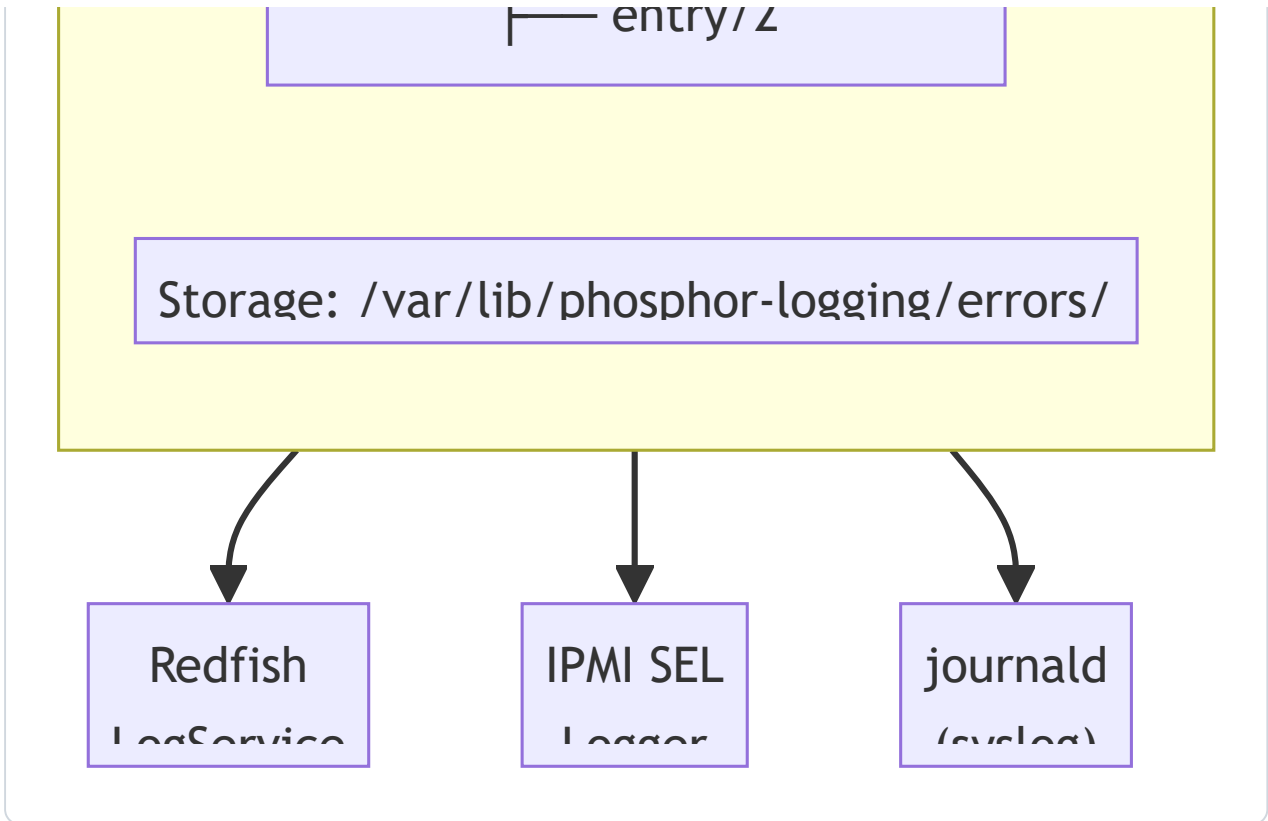
Deep Dive

Advanced implementation details for logging developers.



phosphor-logging Architecture





► ASCII-art version (for comparison)

Error Entry Structure

D-Bus Object: `/xyz/openbmc_project/logging/entry/N`

Interfaces and Properties:

Interface	Property	Description
<code>xyz.openbmc_project.Logging.Entry</code>	<code>Id</code> (uint32)	Entry identifier
	<code>Severity</code>	Informational/Warning/Critical
	<code>Message</code> (string)	Error message (MessageId)
	<code>Resolved</code> (bool)	Has error been acknowledged
	<code>Timestamp</code> (uint64)	Unix timestamp in milliseconds
	<code>AdditionalData</code>	Array of "KEY=VALUE" strings
<code>xyz.openbmc_project.Association.Definitions</code>	<code>Associations</code>	Links to related objects
<code>xyz.openbmc_project.Object.Delete</code>	<code>Delete()</code>	Method to delete entry

AdditionalData Example:

```
[
  "SENSOR_PATH=/xyz/openbmc_project/sensors/temp/CPU0",
  "THRESHOLD_VALUE=95.0",
  "READING=98.5",
  "_PID=1234",
  "CALLOUT_INVENTORY_PATH=/xyz/.../cpu0"
]
```

► ASCII-art version (for comparison)

Error Reporting API

Error Reporting Patterns

1. Simple Error (no metadata):

```
#include <phosphor-logging/elog.hpp>
#include <xyz/openbmc_project/Common/error.hpp>

using InternalFailure =
    sdbusplus::xyz::openbmc_project::Common::Error::
    InternalFailure;

phosphor::logging::report<InternalFailure>();
```

2. Error with Metadata:

```
using namespace phosphor::logging;
using TempError = xyz::openbmc_project::Sensor::Error::
    OverTemperature;
using Metadata = xyz::openbmc_project::Sensor::OverTemperature;

report<TempError>(
    Metadata::SENSOR_PATH(sensorPath.c_str()),
    Metadata::READING(98.5),
    Metadata::THRESHOLD(95.0)
);
```

3. D-Bus Method Call (from script):

```
busctl call xyz.openbmc_project.Logging \
    /xyz/openbmc_project/logging \
    xyz.openbmc_project.Logging.Create Create \
    ssa{ss} \
    "xyz.openbmc_project.Common.Error.InternalFailure" \
    "xyz.openbmc_project.Logging.Entry.Level.Error" \
    2 "KEY1" "value1" "KEY2" "value2"
```

IPMI SEL Integration

SEL (System Event Log) Integration

phosphor-sel-logger monitors D-Bus for loggable events:

Event Sources:

- Threshold crossing (Sensor.Value PropertiesChanged)
- Discrete sensor state change
- phosphor-logging new entry (InterfacesAdded)
- Watchdog timeout

SEL Record Format (IPMI 2.0, Type 02h):

Offset	Size	Description
0-1	2	Record ID
2	1	Record Type (0x02 = System Event)
3-6	4	Timestamp
7-8	2	Generator ID
9	1	EvM Rev
10	1	Sensor Type
11	1	Sensor Number
12	1	Event Type/Dir
13-15	3	Event Data 1-3

Sensor Type to SEL Mapping:

D-Bus Sensor Type	IPMI Sensor Type
sensors/temperature/*	0x01 (Temperature)
sensors/voltage/*	0x02 (Voltage)
sensors/current/*	0x03 (Current)
sensors/fan_tach/*	0x04 (Fan)
sensors/power/*	0x08 (Power Supply)

Log Rotation and Persistence

Log Management

Storage Location:

```
/var/lib/phosphor-logging/errors/
├─ 1      # Serialized error entry
├─ 2
└─ ...
```

Configuration:

```
ERROR_CAP = 200      # Maximum number of error entries
ERROR_INFO_CAP = 10   # Max informational entries
ERROR_PERSIST_PATH = /var/lib/phosphor-logging/errors/
```

Rotation Behavior:

When ERROR_CAP reached:

1. Find oldest Informational entry → delete
2. If no Informational, find oldest Warning → delete
3. If no Warning, find oldest Error → delete
4. Critical entries protected from rotation
5. Resolved entries deleted before unresolved

Persistence across BMC reboot:

- ├─ Entries serialized to filesystem using cereal
- ├─ Entry IDs are persistent
- └─ On startup, entries loaded from /var/lib/phosphor-logging/

Source Code Reference

Key implementation files in [phosphor-logging](#):

File	Description
log_manager.cpp	Main logging manager
elog_entry.cpp	Error log entry object
elog_serialize.cpp	Entry persistence
extensions.cpp	Extension point for plugins
lib/elog.cpp	Client-side reporting API

Examples

Working examples are available in the [examples/logging](#) directory:

- [create-event.sh](#) - Create SEL/Event log entries

- `query-logs.sh` - Query and filter log entries
 - `custom-error.yaml` - Custom error definition example
-

References

- [phosphor-logging](#)
 - [phosphor-sel-logger](#)
 - [Redfish Log Services](#)
 - [IPMI SEL Specification](#)
-

Tested on: OpenBMC master, QEMU romulus

SDR Guide

Configure Sensor Data Records (SDR) for IPMI sensor management.

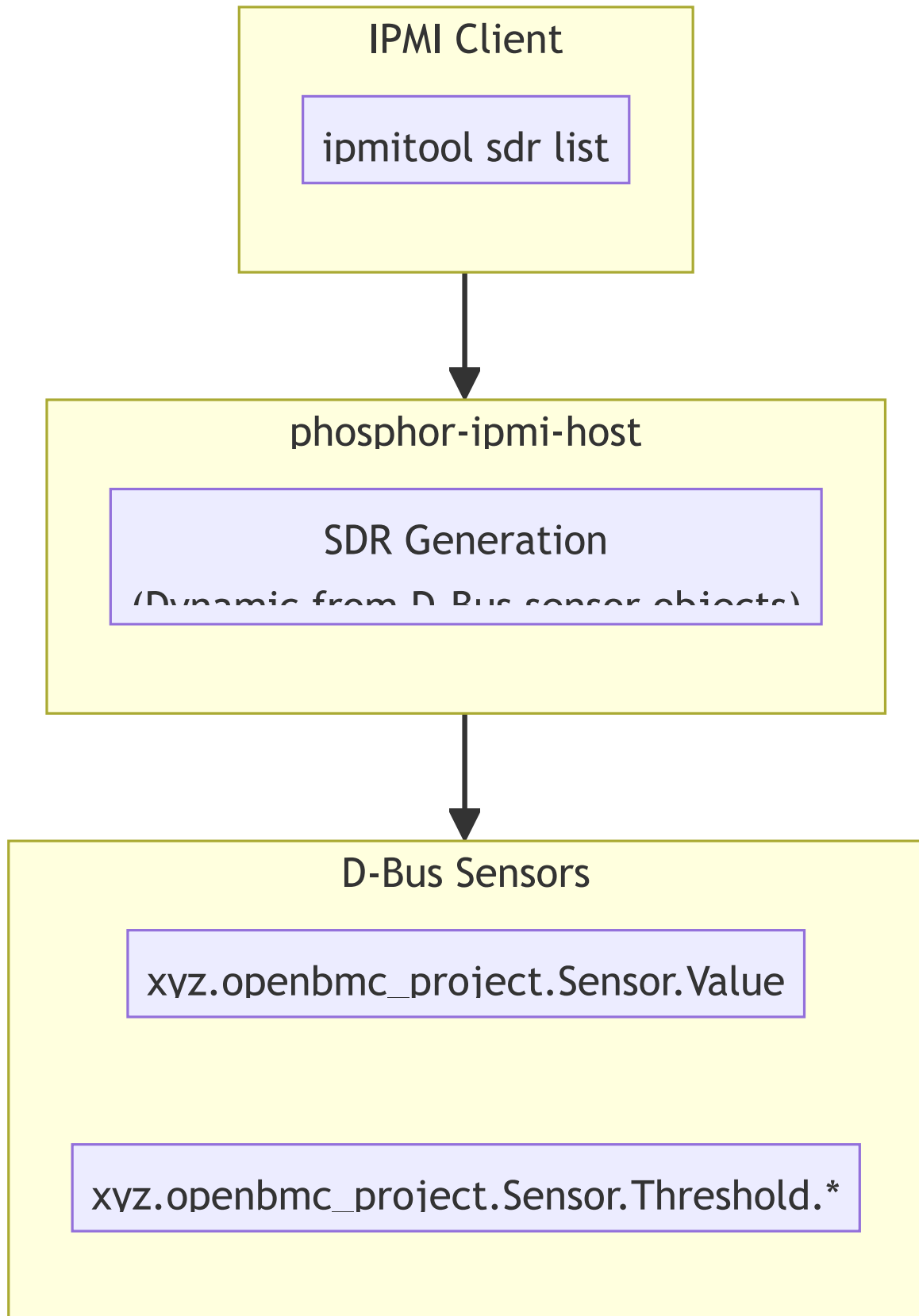
Table of Contents

1. [Overview](#)
2. [SDR Record Types](#)
 1. [Type 01h - Full Sensor Record](#)
 2. [Type 02h - Compact Sensor Record](#)
 3. [Type 11h - FRU Device Locator](#)
 4. [Type 12h - Management Controller Locator](#)
3. [Viewing SDR](#)
 1. [ipmitool Commands](#)
 2. [Example Output](#)
4. [SDR Generation](#)
 1. [Dynamic Generation](#)
 2. [Sensor to SDR Mapping](#)
5. [Sensor Types](#)
 1. [Common Sensor Types](#)
6. [Configuration](#)
 1. [Entity Manager Sensor Configuration](#)
 2. [IPMI Sensor Configuration](#)
7. [Threshold Events](#)
 1. [SDR Threshold Support](#)
 2. [Monitor Threshold Events](#)
8. [Reading Values](#)
 1. [Raw to Display Conversion](#)
 2. [Example](#)
9. [Troubleshooting](#)
 1. [SDR Not Showing](#)
 2. [Wrong Readings](#)
 3. [Missing Thresholds](#)
10. [D-Bus to SDR](#)
 1. [Query Sensors via D-Bus](#)
11. [Deep Dive](#)
 1. [SDR Record Type 01h Format \(Full Sensor Record\)](#)
 2. [D-Bus to SDR Mapping](#)
 3. [SDR Repository Commands](#)
 4. [Source Code Reference](#)

Overview

Sensor Data Records (SDR) describe sensors and their properties for IPMI clients. OpenBMC dynamically generates SDR from D-Bus sensor objects.

SDR Architecture



► ASCII-art version (for comparison)

SDR Record Types

Type 01h - Full Sensor Record

Complete sensor definition with thresholds and scaling.

Field	Description
Sensor Number	Unique sensor ID
Sensor Type	Temperature, Voltage, Fan, etc.
Unit Type	Celsius, Volts, RPM, etc.
M, B, Exp	Linear conversion factors
Thresholds	Upper/Lower critical/non-critical

Type 02h - Compact Sensor Record

Abbreviated sensor record for simple sensors.

Type 11h - FRU Device Locator

Points to FRU data storage.

Type 12h - Management Controller Locator

Describes management controllers.

Viewing SDR

ipmitool Commands

```
# List all SDR records
ipmitool sdr list

# Get full SDR output
ipmitool sdr list full

# Get SDR by type
ipmitool sdr type Temperature
ipmitool sdr type Voltage
ipmitool sdr type Fan

# Get SDR info
ipmitool sdr info

# Get SDR repository info
ipmitool sdr elist
```

Example Output

CPU_Temp	45 degrees C	ok
Board_Temp	32 degrees C	ok
Fan0	5000 RPM	ok
P12V	12.10 Volts	ok
PSU1_Status	0x00	ok

SDR Generation

Dynamic Generation

OpenBMC generates SDR dynamically from D-Bus:

```
// phosphor-ipmi-host generates SDR from:  
// 1. xyz.openbmc_project.Sensor.Value - sensor readings  
// 2. xyz.openbmc_project.Sensor.Threshold.* - thresholds  
// 3. xyz.openbmc_project.Association.Definitions - relationships
```

Sensor to SDR Mapping

D-Bus Property	SDR Field
Value	Current reading
Unit	Sensor unit type
Scale	M, B, Exp factors
MaxValue	Sensor max
MinValue	Sensor min
CriticalHigh	Upper critical
WarningHigh	Upper non-critical
CriticalLow	Lower critical
WarningLow	Lower non-critical

Sensor Types

Common Sensor Types

Type	Hex	Description
Temperature	0x01	Temperature sensors
Voltage	0x02	Voltage sensors
Current	0x03	Current sensors
Fan	0x04	Fan speed sensors
Physical Security	0x05	Intrusion sensors
Platform Security	0x06	Security sensors
Processor	0x07	CPU sensors
Power Supply	0x08	PSU sensors
Power Unit	0x09	Power unit sensors
Memory	0x0C	Memory sensors
System Event	0x12	System events
OEM	0xC0+	Vendor-specific

Configuration

Entity Manager Sensor Configuration

```
{
  "Name": "CPU_Temp",
  "Type": "TMP75",
  "Bus": 0,
  "Address": "0x48",
  "Thresholds": [
    {
      "Direction": "greater than",
      "Name": "upper critical",
      "Severity": 1,
      "Value": 95
    },
    {
      "Direction": "greater than",
      "Name": "upper non critical",
      "Severity": 0,
      "Value": 85
    }
  ]
}
```

IPMI Sensor Configuration

```
# phosphor-ipmi-host sensor configuration
# /usr/share/ipmi-providers/sensor-defs.yaml

0x01:
  sensorType: 0x01
  sensorName: CPU_Temp
  sensorReadingType: 0x01
  entityId: 0x03
  entityInstance: 0x01
```

Threshold Events

SDR Threshold Support

Threshold Events			
Upper Critical	——	Deassert/Assert	
Upper Non-Crit	——	Deassert/Assert	
Lower Non-Crit	——	Deassert/Assert	
Lower Critical	——	Deassert/Assert	

Monitor Threshold Events

```
# Watch SEL for threshold events
ipmitool sel list

# Example threshold event:
# Sensor CPU_Temp | Upper Critical going high | Asserted
```

Reading Values

Raw to Display Conversion

```
Display Value = (M × raw + B × 10^Bexp) × 10^Rexp

Where:
  M = Multiplier
  B = Additive offset
  Bexp = B exponent
  Rexp = Result exponent
```

Example

```
Temperature sensor:
raw value = 0x2D (45)
M = 1, B = 0, Bexp = 0, Rexp = 0
Display = (1 × 45 + 0) × 1 = 45°C
```

Troubleshooting

SDR Not Showing

```
# Check sensor exists in D-Bus
busctl tree xyz.openbmc_project.Sensor

# Check IPMI host daemon
systemctl status phosphor-ipmi-host
journalctl -u phosphor-ipmi-host

# Check sensor has required interfaces
busctl introspect xyz.openbmc_project.HwmonTempSensor \
    /xyz/openbmc_project/sensors/temperature/CPU_Temp
```

Wrong Readings

```
# Compare D-Bus value with IPMI
busctl get-property xyz.openbmc_project.HwmonTempSensor \
    /xyz/openbmc_project/sensors/temperature/CPU_Temp \
    xyz.openbmc_project.Sensor.Value Value

# vs
ipmitool sensor get CPU_Temp

# Check M, B, Exp values match sensor scaling
```

Missing Thresholds

```
# Verify thresholds in D-Bus
busctl introspect xyz.openbmc_project.HwmonTempSensor \
    /xyz/openbmc_project/sensors/temperature/CPU_Temp | grep Threshold

# Check Entity Manager configuration
cat /usr/share/entity-manager/configurations/*.json | jq '.Exposes[] | select(.Name=="CPU_Temp")'
```

D-Bus to SDR

Query Sensors via D-Bus

```
# List all sensors
busctl tree xyz.openbmc_project.Sensor

# Get sensor reading
busctl get-property xyz.openbmc_project.HwmonTempSensor \
    /xyz/openbmc_project/sensors/temperature/CPU_Temp \
    xyz.openbmc_project.Sensor.Value Value

# Get thresholds
busctl get-property xyz.openbmc_project.HwmonTempSensor \
    /xyz/openbmc_project/sensors/temperature/CPU_Temp \
    xyz.openbmc_project.Sensor.Threshold.Critical CriticalHigh
```

Deep Dive

Advanced implementation details for SDR developers.

SDR Record Type 01h Format (Full Sensor Record)

SDR Type 01h - Full Sensor Record

RECORD HEADER (5 bytes)

Record ID LSB	Record ID MSB	SDR Version (0x51)	Record Type (0x01)	Record Length (varies)
------------------	------------------	-----------------------	-----------------------	---------------------------

RECORD KEY BYTES (3 bytes)

Byte 5: Sensor Owner ID (I2C slave addr or software ID)
 Byte 6: Sensor Owner LUN (bits 1:0), Channel (bits 7:4)
 Byte 7: Sensor Number (unique within owner)

RECORD BODY - Entity & Sensor Information

Byte 8: Entity ID (processor=0x03, memory=0x20, etc.)
 Byte 9: Entity Instance (0x60-0x7F = device-relative)
 Byte 10: Sensor Initialization
 [7] Sensor Init Scanning enabled
 [6] Sensor Init Event generation enabled
 [5] Sensor Init Thresholds accessible
 [4] Sensor Init Hysteresis accessible
 [3] Sensor Init Type accessible
 [2] Sensor Init Event msg ctrl accessible
 [1] Sensor Init Settle time required
 Byte 11: Sensor Capabilities
 [7:6] Event message control support
 [5:4] Threshold access support
 [3:2] Hysteresis support
 [1] Auto re-arm support
 [0] Ignore sensor if entity not present

SENSOR TYPE & READING TYPE

Byte 12: Sensor Type (Temperature=0x01, Voltage=0x02, etc.)
 Byte 13: Event/Reading Type Code
 0x01 = Threshold-based
 0x6F = Sensor-specific discrete

READING FACTORS & LINEARIZATION (for threshold sensors)

Byte 21: M (LSB of multiplier)
 Byte 22: M (MSB 2 bits) + Tolerance (6 bits)
 Byte 23: B (LSB of offset)
 Byte 24: B (MSB 2 bits) + Accuracy (6 bits LSB)
 Byte 25: Accuracy (4 bits MSB) + Accuracy Exp (2 bits)
 Byte 26: R Exp (4 bits) + B Exp (4 bits) - signed

Formula: $y = (M \times \text{raw} + B \times 10^{\text{Bexp}}) \times 10^{\text{Rexp}}$

Example: Temperature sensor, raw=0x50 (80)

M=1, B=0, Bexp=0, Rexp=0

$y = (1 \times 80 + 0) \times 1 = 80^{\circ}\text{C}$

Example: Voltage sensor 12V rail, raw=0x78 (120)

M=100, B=0, Bexp=0, Rexp=-3
 $y = (100 \times 120 + 0) \times 0.001 = 12.0V$

THRESHOLDS

Byte 36: Upper Non-Recoverable threshold (raw value)
Byte 37: Upper Critical threshold
Byte 38: Upper Non-Critical threshold
Byte 39: Lower Non-Recoverable threshold
Byte 40: Lower Critical threshold
Byte 41: Lower Non-Critical threshold
Byte 42: Positive-going Threshold Hysteresis
Byte 43: Negative-going Threshold Hysteresis

SENSOR ID STRING (variable length)

Byte 47: ID String Type/Length
[7:6] = 11 binary (8-bit ASCII + Latin 1)
[4:0] = length (0-31 characters)
Bytes 48+: ID String data (e.g., "CPU_Temp")

D-Bus to SDR Mapping

D-Bus Sensor to SDR Translation

D-BUS SENSOR OBJECT

Path: /xyz/openbmc_project/sensors/temperature/CPU_Temp

Interfaces:

```
xyz.openbmc_project.Sensor.Value
├── Value: 45.0 (double)      → SDR raw reading
├── Unit: "xyz...Unit.DegreesC" → SDR unit type
├── Scale: 0 (int64)         → SDR R exponent
├── MaxValue: 127.0         → SDR nominal max
└── MinValue: -40.0         → SDR nominal min

xyz.openbmc_project.Sensor.Threshold.Critical
├── CriticalHigh: 95.0      → SDR Upper Critical
├── CriticalLow: -20.0      → SDR Lower Critical
├── CriticalAlarmHigh: false → Threshold event state
└── CriticalAlarmLow: false  → Threshold event state

xyz.openbmc_project.Sensor.Threshold.Warning
├── WarningHigh: 85.0      → SDR Upper Non-Critical
├── WarningLow: 0.0        → SDR Lower Non-Critical
├── WarningAlarmHigh: false → Threshold event state
└── WarningAlarmLow: false  → Threshold event state
```

SDR GENERATION ALGORITHM (phosphor-ipmi-host):

1. ENUMERATE D-BUS SENSORS

```
// Query ObjectMapper for all sensors
busctl call xyz.openbmc_project.ObjectMapper
  /xyz/openbmc_project/object_mapper
  xyz.openbmc_project.ObjectMapper GetSubTree
  sias "/xyz/openbmc_project/sensors" 0 1
  "xyz.openbmc_project.Sensor.Value"
```

2. CALCULATE M, B, R_EXP FOR EACH SENSOR

```
// Determine scaling from MinValue to MaxValue
// Raw values 0-255 must map to display range

double range = maxValue - minValue; // e.g., 127 - (-40) = 167
double M = range / 255.0;           // e.g., 0.655
double B = minValue;                // e.g., -40

// Quantize M and B to SDR format (10-bit signed)
int16_t M_sdr = (int16_t)(M * scaleFactor);
int16_t B_sdr = (int16_t)(B * scaleFactor);
int8_t R_exp = calculateExponent(range);
int8_t B_exp = calculateExponent(B);
```

3. CONVERT THRESHOLDS TO RAW VALUES

```
// Reverse the display→raw formula
// raw = (display - B × 10^Bexp) / (M × 10^Rexp)

uint8_t upperCritRaw = (uint8_t)((criticalHigh - B) / M);
uint8_t upperWarnRaw = (uint8_t)((warningHigh - B) / M);
// etc.
```

4. ASSIGN SENSOR NUMBER

```
// Sensor numbers assigned sequentially or from config
// Must be unique per sensor owner

static uint8_t nextSensorNum = 0x01;
sdrRecord.sensorNumber = nextSensorNum++;

// Or lookup from YAML/JSON configuration
sdrRecord.sensorNumber = sensorConfig[sensorName].number;
```

5. BUILD SDR RECORD

```
FullSensorRecord sdr = {};
sdr.recordType = 0x01;
sdr.sensorNumber = sensorNum;
sdr.entityId = getEntityId(sensorPath); // 0x03 for CPU
sdr.sensorType = getSensorType(unit); // 0x01 for temp
sdr.eventReadingType = 0x01; // Threshold
sdr.M = M_sdr;
sdr.B = B_sdr;
sdr.Rexp = R_exp;
sdr.Bexp = B_exp;
sdr.upperCritical = upperCritRaw;
sdr.upperNonCritical = upperWarnRaw;
// ... set other fields
strncpy(sdr.idString, sensorName, 16);
```

SDR Repository Commands

SDR Repository IPMI Commands

GET SDR REPOSITORY INFO (0x20)

Response:

Byte	Description
1	Completion code
2	SDR version (0x51 = IPMI 2.0)
3-4	Record count (LSB first)
5-6	Free space (0xFFFF = unspecified)
7-10	Most recent addition timestamp
11-14	Most recent erase timestamp
15	Operation support
	[7] Overflow flag
	[5] Modal SDR update support
	[4] Non-modal SDR update support
	[3] Delete SDR command supported
	[2] Partial Add SDR command supported
	[1] Reserve SDR Repository supported
	[0] Get SDR Allocation Info supported

RESERVE SDR REPOSITORY (0x22)

Request: (empty)

Response: 2-byte reservation ID (used for subsequent Get SDR)

GET SDR (0x23)

Request:

Byte 1-2	Reservation ID (from Reserve SDR Repository)
Byte 3-4	Record ID (0x0000 = first, 0xFFFF = last)
Byte 5	Offset into record
Byte 6	Bytes to read (0xFF = entire record)

Response:

Byte 1	Completion code
Byte 2-3	Next Record ID (0xFFFF = end of repository)
Byte 4+	SDR record data

ITERATION EXAMPLE (ipmitool sdr list):

1. Reserve SDR Repository → get reservation ID 0x1234
2. Get SDR (res=0x1234, rec=0x0000, offset=0, len=0xFF)
→ Record data, Next ID = 0x0001
3. Get SDR (res=0x1234, rec=0x0001, offset=0, len=0xFF)
→ Record data, Next ID = 0x0002
4. ... repeat until Next ID = 0xFFFF ...

Source Code Reference

Key implementation files in [phosphor-host-ipmid](#):

File	Description
sensorhandler.cpp	Sensor reading command handlers (Get Sensor Reading, etc.)
sensorcommands.cpp	SDR-related commands (Get SDR, Reserve SDR Repository)
sensordatahandler.cpp	D-Bus to SDR data translation
types.hpp	SDR structure definitions
scripts/sensor-gen.py	Static SDR generation from YAML configuration

References

- [IPMI Specification](#)
- [phosphor-host-ipmid](#) - D-Bus based IPMI daemon for host commands
- [phosphor-net-ipmid](#) - Network IPMI server

Tested on: OpenBMC master, QEMU romulus

Linux Debug Tools Guide

Configure and use Linux kernel and application debugging tools for OpenBMC development.

Table of Contents

1. [Overview](#)
 1. [Tool Selection Guide](#)
 2. [Tool Comparison](#)
2. [Early Boot Debugging](#)
 1. [Kernel Configuration](#)
 2. [Command Line Options](#)
 1. [ARM \(ASPEED BMC\)](#)
 2. [x86 \(QEMU\)](#)
 3. [Yocto Configuration](#)
 4. [Kernel Config Fragment](#)
 5. [Troubleshooting Early Boot](#)
3. [AddressSanitizer \(ASan\)](#)
 1. [What ASan Detects](#)
 2. [Yocto Build Configuration](#)
 1. [Global ASan Build](#)
 2. [Per-Recipe ASan \(Recommended\)](#)
 3. [Meson-Based Recipes](#)
 4. [CMake-Based Recipes](#)
 3. [Runtime Configuration](#)
 4. [systemd Service Configuration](#)
 5. [Interpreting ASan Reports](#)
 6. [Common ASan Issues](#)
4. [UndefinedBehaviorSanitizer \(UBSan\)](#)
 1. [What UBSan Detects](#)
 2. [Yocto Configuration](#)
 3. [Selective Checks](#)
 4. [Runtime Options](#)
 5. [Example UBSan Report](#)
5. [ThreadSanitizer \(TSan\)](#)
 1. [Requirements](#)
 2. [What TSan Detects](#)
 3. [Yocto Configuration](#)
 4. [Runtime Options](#)
 5. [Suppression File Format](#)

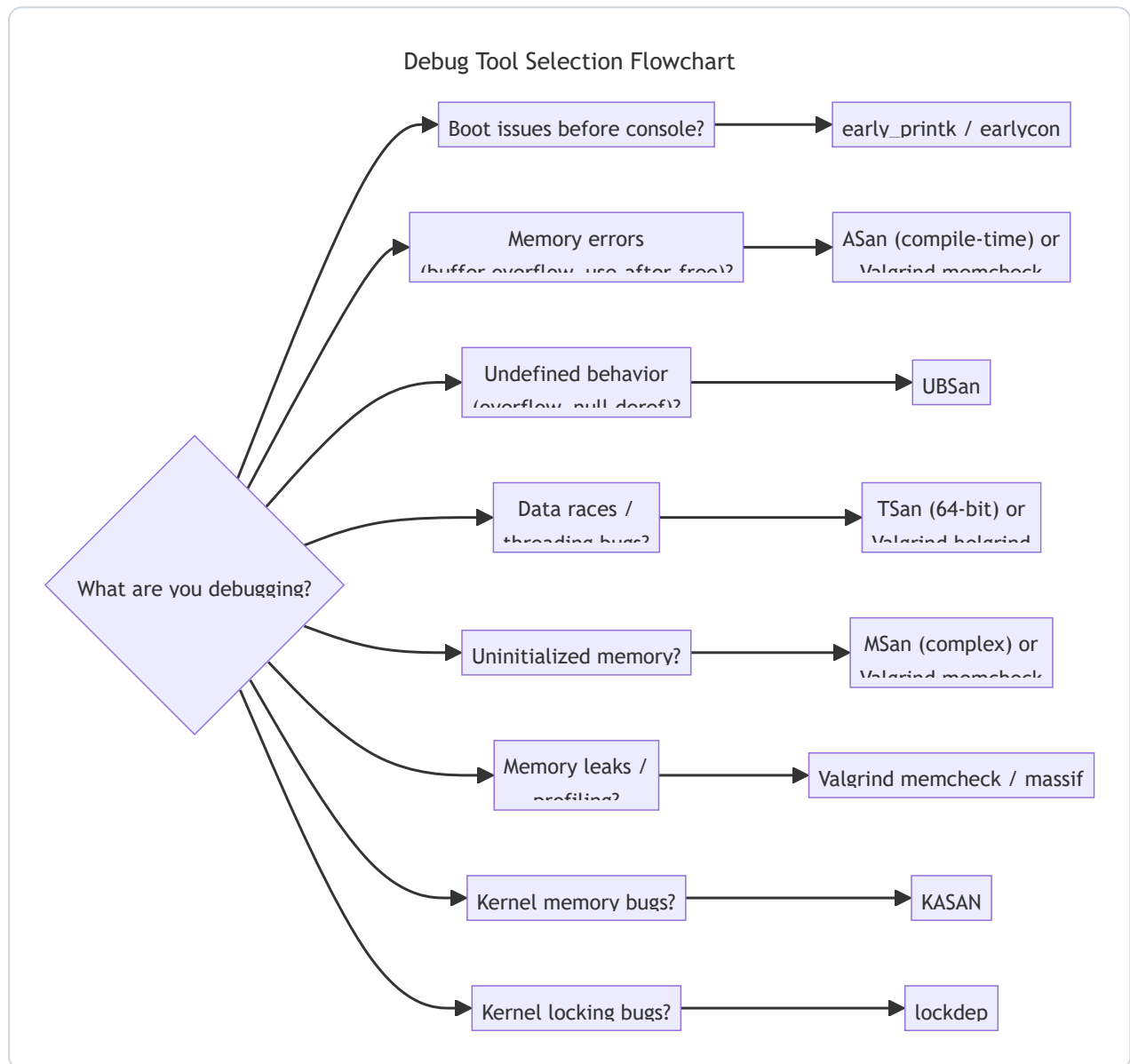
- 6. [Example TSan Report](#)
- 6. [MemorySanitizer \(MSan\)](#)
 - 1. [What MSan Detects](#)
 - 2. [Limitations](#)
 - 3. [Basic Configuration](#)
 - 4. [When to Use MSan](#)
 - 5. [Alternative: Valgrind](#)
- 7. [Valgrind](#)
 - 1. [Tool Suite](#)
 - 2. [Yocto Configuration](#)
 - 3. [Memcheck \(Memory Errors\)](#)
 - 4. [Memcheck Output](#)
 - 5. [Cachegrind \(Cache Profiling\)](#)
 - 6. [Callgrind \(Call Graph\)](#)
 - 7. [Helgrind \(Thread Errors\)](#)
 - 8. [DRD \(Data Race Detector\)](#)
 - 9. [Massif \(Heap Profiler\)](#)
 - 10. [Suppression File](#)
 - 11. [Running OpenBMC Services](#)
- 8. [Kernel Debug Options](#)
 - 1. [KASAN \(Kernel AddressSanitizer\)](#)
 - 1. [Yocto Kernel Config](#)
 - 2. [KASAN Output](#)
 - 2. [Kernel UBSan](#)
 - 3. [KCSAN \(Kernel Concurrency Sanitizer\)](#)
 - 4. [KMSAN \(Kernel MemorySanitizer\)](#)
 - 5. [Lockdep \(Lock Validator\)](#)
 - 1. [Lockdep Output](#)
 - 6. [Combined Kernel Debug Config](#)
- 9. [Performance Considerations](#)
 - 1. [Overhead Comparison](#)
 - 2. [Recommendations by Environment](#)
 - 3. [Resource Requirements](#)
- 10. [Troubleshooting](#)
 - 1. [ASan: No Symbols in Stack Trace](#)
 - 2. [ASan: Out of Memory](#)
 - 3. [TSan: False Positives](#)
 - 4. [Valgrind: Slow Startup](#)
 - 5. [Valgrind: Too Many Errors](#)
 - 6. [Kernel: KASAN Flooding Logs](#)
- 11. [Quick Reference](#)

1. [Enable ASan for a Recipe](#)
 2. [Enable ASan + UBSan \(Meson\)](#)
 3. [Run with Valgrind](#)
 4. [Enable KASAN](#)
 12. [Deep Dive](#)
 1. [AddressSanitizer Memory Layout](#)
 2. [Valgrind Binary Translation](#)
 3. [KASAN Kernel Implementation](#)
 4. [Source Code References](#)
 13. [Examples](#)
 14. [References](#)
-

Overview

OpenBMC development requires debugging at multiple levels—from early kernel boot to application runtime. This guide covers essential Linux debugging tools including sanitizers, Valgrind, and kernel debug options.

Tool Selection Guide



► ASCII-art version (for comparison)

Tool Comparison

Tool	Detects	Overhead	Compatibility
ASan	Memory errors	2x slowdown, 2-3x memory	Incompatible with TSan, MSan
UBSan	Undefined behavior	1.2x slowdown	Compatible with ASan
TSan	Data races	5-15x slowdown	64-bit only, incompatible with ASan
MSan	Uninitialized memory	3x slowdown	Requires full instrumentation
Valgrind	Memory, threads, cache	10-50x slowdown	No recompilation needed
KASAN	Kernel memory errors	1.5-3x slowdown	Kernel only
KCSAN	Kernel data races	2-5x slowdown	Kernel only

Early Boot Debugging

When the BMC fails to boot before the normal console is available, `early_printk` and `earlycon` provide debug output directly to the serial port.

Kernel Configuration

```
# Enable early printk support
CONFIG_EARLY_PRINTK=y

# ARM-specific low-level debug
CONFIG_DEBUG_LL=y
CONFIG_DEBUG_LL_UART_NONE=n

# For ASPEED BMC (AST2500/AST2600)
CONFIG_DEBUG_UART_PHYS=0x1e784000
CONFIG_DEBUG_UART_VIRT=0xf8784000
CONFIG_DEBUG_UART_8250_SHIFT=2
```

Command Line Options

ARM (ASPEED BMC)

```
# AST2500/AST2600 UART5 (typical BMC debug port)
earlycon=uart8250,mmio32,0x1e784000,115200n8

# With explicit flow control disabled
earlycon=uart8250,mmio32,0x1e784000,115200n8 console=ttyS4,115200
```

x86 (QEMU)

```
# Standard COM1
earlycon=uart8250,io,0x3f8,115200n8

# Memory-mapped UART
earlycon=uart,mmio32,0xfe034000,115200n8
```

Yocto Configuration

```
# In your machine .conf file

# Add earlycon to kernel command line
KERNEL_EXTRA_ARGS:append = " earlycon=uart8250,mmio32,0x1e784000,115200n8"

# Or in APPEND for QEMU machines
APPEND += "earlycon=uart8250,io,0x3f8"
```

Kernel Config Fragment

Create a kernel config fragment for debug builds:

```
# meta-mylayer/recipes-kernel/linux/files/debug.cfg
CONFIG_EARLY_PRINTK=y
CONFIG_DEBUG_LL=y
CONFIG_DEBUG_INFO=y
CONFIG_PRINTK_TIME=y
CONFIG_DYNAMIC_DEBUG=y
```

```
# In linux-aspeed_%.bbappend
SRC_URI += "file://debug.cfg"
```

Troubleshooting Early Boot

```
# If earlycon output stops, check:
# 1. UART address matches your SoC
# 2. Baud rate matches your terminal
# 3. Console= parameter takes over after earlycon

# Enable kernel messages timing
printk.time=1

# Increase kernel log verbosity
loglevel=8 debug
```

AddressSanitizer (ASan)

ASan detects memory errors at runtime with relatively low overhead. It's the recommended first tool for debugging memory corruption.

What ASan Detects

Bug Type	Example
Heap buffer overflow	Reading/writing past malloc'd buffer
Stack buffer overflow	Array out-of-bounds on stack
Global buffer overflow	Out-of-bounds on global arrays
Use-after-free	Accessing freed memory
Use-after-return	Accessing stack memory after return
Use-after-scope	Accessing out-of-scope stack variable
Double-free	Freeing already freed memory
Memory leaks	Unreachable allocated memory (via LeakSanitizer)

Yocto Build Configuration

Global ASan Build

```
# In local.conf - enable ASan for all packages (not recommended for production)
CFLAGS:append = " -fsanitize=address -fno-omit-frame-pointer -g"
CXXFLAGS:append = " -fsanitize=address -fno-omit-frame-pointer -g"
LDFLAGS:append = " -fsanitize=address"
```

Per-Recipe ASan (Recommended)

```
# In a .bbappend file for the target recipe
# e.g., phosphor-logging_%.bbappend

CFLAGS:append = " -fsanitize=address -fno-omit-frame-pointer"
CXXFLAGS:append = " -fsanitize=address -fno-omit-frame-pointer"
LDFLAGS:append = " -fsanitize=address"

# Ensure debug symbols
DEBUG_BUILD = "1"
```

Meson-Based Recipes

```
# For meson-based recipes (bmcweb, phosphor-dbus-interfaces, etc.)
EXTRA_OEMESON:append:pn-bmcweb = " -Db_sanitize=address"
EXTRA_OEMESON:append:pn-phosphor-logging = " -Db_sanitize=address"
```

CMake-Based Recipes

```
# For CMake-based recipes
EXTRA_OECMAKE:append:pn-sdbusplus = " \
    -DCMAKE_C_FLAGS='-fsanitize=address -fno-omit-frame-pointer' \
    -DCMAKE_CXX_FLAGS='-fsanitize=address -fno-omit-frame-pointer' \
    "
```

Runtime Configuration

```
# Basic ASan options
export ASAN_OPTIONS="detect_leaks=1:halt_on_error=0:log_path=/tmp/asan"

# Full options for debugging
export ASAN_OPTIONS="\
detect_leaks=1:\
halt_on_error=0:\
log_path=/tmp/asan:\
symbolize=1:\
print_stats=1:\
check_initialization_order=1:\
detect_stack_use_after_return=1:\
strict_string_checks=1:\
detect_invalid_pointer_pairs=2"

# For better stack traces
export ASAN_OPTIONS="$ASAN_OPTIONS:fast_unwind_on_malloc=0"

# Use suppressions file
export ASAN_OPTIONS="$ASAN_OPTIONS:suppressions=/etc/asan.supp"
```

systemd Service Configuration

```
# /etc/systemd/system/phosphor-logging.service.d/asan.conf
[Service]
Environment="ASAN_OPTIONS=detect_leaks=1:halt_on_error=0:log_path=/tmp/asan-logging"
```

Interpreting ASan Reports

```
==12345==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x602000000014
READ of size 4 at 0x602000000014 thread T0
    #0 0x4c3b5f in process_buffer /src/handler.cpp:45
    #1 0x4c2a12 in handle_request /src/handler.cpp:123
    #2 0x4c1234 in main /src/main.cpp:67
    #3 0x7f... in __libc_start_main

0x602000000014 is located 0 bytes to the right of 20-byte region [0x602000000000,0x602000000014)
allocated by thread T0 here:
    #0 0x493d2f in malloc
    #1 0x4c3a11 in allocate_buffer /src/handler.cpp:30
    #2 0x4c29f3 in handle_request /src/handler.cpp:120

SUMMARY: AddressSanitizer: heap-buffer-overflow /src/handler.cpp:45 in process_buffer
```

Reading the report:

1. **Error type:** `heap-buffer-overflow`
2. **Access type:** `READ of size 4`
3. **Location:** Stack trace shows `process_buffer` at line 45
4. **Memory info:** Accessing byte 20 of a 20-byte allocation (off by one)
5. **Allocation:** Shows where the buffer was allocated

Common ASan Issues

```
# Issue: ASan not finding symbols
# Solution: Ensure debug symbols and symbolizer
export ASAN_SYMBOLIZER_PATH=/usr/bin/llvm-symbolizer

# Issue: False positives with system libraries
# Solution: Use suppressions
echo "leak:libglib" >> /etc/asan.supp

# Issue: Out of memory with ASan
# Solution: Increase virtual memory or reduce shadow scale
export ASAN_OPTIONS="quarantine_size_mb=64:malloc_context_size=10"
```

UndefinedBehaviorSanitizer (UBSan)

UBSan detects undefined behavior that can cause subtle, hard-to-reproduce bugs.

What UBSan Detects

Check	Flag	Description
Alignment	<code>-fsanitize=alignment</code>	Misaligned pointer dereference
Bool	<code>-fsanitize=bool</code>	Loading invalid bool value
Bounds	<code>-fsanitize=bounds</code>	Array index out of bounds
Enum	<code>-fsanitize=enum</code>	Invalid enum value
Float-cast-overflow	<code>-fsanitize=float-cast-overflow</code>	Float to int overflow
Float-divide-by-zero	<code>-fsanitize=float-divide-by-zero</code>	Floating-point division by zero
Integer-divide-by-zero	<code>-fsanitize=integer-divide-by-zero</code>	Integer division by zero
Null	<code>-fsanitize=null</code>	Null pointer dereference
Object-size	<code>-fsanitize=object-size</code>	Memory access exceeds object size
Pointer-overflow	<code>-fsanitize=pointer-overflow</code>	Pointer arithmetic overflow
Return	<code>-fsanitize=return</code>	Missing return in non-void function
Shift	<code>-fsanitize=shift</code>	Shift by negative or too large amount
Signed-integer-overflow	<code>-fsanitize=signed-integer-overflow</code>	Signed integer overflow
Unreachable	<code>-fsanitize=unreachable</code>	Reaching <code>__builtin_unreachable()</code>
VLA-bound	<code>-fsanitize=vla-bound</code>	Negative VLA size
Vptr	<code>-fsanitize=vptr</code>	Invalid virtual function call

Yocto Configuration

```
# Enable all UBSan checks
CFLAGS:append:pn-target = " -fsanitize=undefined -fno-sanitize-recover=all"
CXXFLAGS:append:pn-target = " -fsanitize=undefined -fno-sanitize-recover=all"

# Meson-based recipes
EXTRA_OEMESON:append:pn-target = " -Db_sanitize=undefined"

# Combined ASan + UBSan (recommended)
EXTRA_OEMESON:append:pn-bmcweb = " -Db_sanitize=address,undefined"
```

Selective Checks

```
# Enable only specific checks (faster, less noise)
CFLAGS:append = " -fsanitize=signed-integer-overflow,null,bounds"

# Disable specific checks
CFLAGS:append = " -fsanitize=undefined -fno-sanitize=vptr"
```

Runtime Options

```
# Print stack traces for UBSan errors
export UBSAN_OPTIONS="print_stacktrace=1"

# Halt on first error (useful for debugging)
export UBSAN_OPTIONS="print_stacktrace=1:halt_on_error=1"

# Log to file
export UBSAN_OPTIONS="print_stacktrace=1:log_path=/tmp/ubsan"

# Use suppressions
export UBSAN_OPTIONS="suppressions=/etc/ubsan.supp"
```

Example UBSan Report

```
/src/calculation.cpp:42:15: runtime error: signed integer overflow:
2147483647 + 1 cannot be represented in type 'int'
  #0 0x4a2b3f in calculate /src/calculation.cpp:42
  #1 0x4a1234 in process /src/process.cpp:88
  #2 0x4a0567 in main /src/main.cpp:23

SUMMARY: UndefinedBehaviorSanitizer: signed-integer-overflow /src/calculation.cpp:42
```

ThreadSanitizer (TSan)

TSan detects data races and other threading bugs. It requires 64-bit builds and is incompatible with ASan.

Requirements

- **64-bit architecture only** (x86_64, aarch64)
- **Incompatible with ASan/MSan** - use one at a time
- **Higher overhead** than ASan (5-15x slowdown, 5-10x memory)

What TSan Detects

Bug Type	Description
Data races	Concurrent unsynchronized access to shared memory
Lock order violations	Potential deadlocks from inconsistent lock ordering
Thread leaks	Threads not joined before program exit
Destroying locked mutex	Mutex destroyed while still held
Use of invalid mutex	Operations on uninitialized/destroyed mutex
Signal-unsafe functions	Calling non-async-signal-safe functions in handlers

Yocto Configuration

```
# TSan requires 64-bit build
MACHINE = "qemux86-64"

# Per-recipe TSan
CFLAGS:append:pn-target = " -fsanitize=thread -fPIE"
CXXFLAGS:append:pn-target = " -fsanitize=thread -fPIE"
LDFLAGS:append:pn-target = " -fsanitize=thread -pie"

# Meson-based
EXTRA_OEMESON:append:pn-target = " -Db_sanitize=thread"
```

Runtime Options

```
# Basic TSan options
export TSAN_OPTIONS="history_size=7:second_deadlock_stack=1"

# Detailed output
export TSAN_OPTIONS="\
history_size=7:\
second_deadlock_stack=1:\
print_suppressions=1:\
report_bugs=1:\
log_path=/tmp/tsan"

# Suppressions
export TSAN_OPTIONS="suppressions=/etc/tsan.supp"
```

Suppression File Format

```
# /etc/tsan.supp

# Suppress races in third-party library
race:libthirdparty.so

# Suppress specific function
race:known_benign_race_function

# Suppress by source file
race:legacy_module.cpp

# Suppress deadlock warnings for intentional lock order
deadlock:custom_lock_hierarchy

# Suppress thread leak for daemon threads
thread:background_worker_thread
```

Example TSan Report

```
=====
WARNING: ThreadSanitizer: data race (pid=12345)
  Write of size 4 at 0x7f8b12345678 by thread T2:
    #0 increment_counter /src/counter.cpp:15
    #1 worker_thread /src/worker.cpp:45

  Previous read of size 4 at 0x7f8b12345678 by thread T1:
    #0 get_counter /src/counter.cpp:20
    #1 monitor_thread /src/monitor.cpp:30

  Location is global '_g_counter' of size 4 at 0x7f8b12345678

  Thread T2 (tid=12347, running) created by main thread at:
    #0 pthread_create
    #1 start_worker /src/main.cpp:50

  Thread T1 (tid=12346, running) created by main thread at:
    #0 pthread_create
    #1 start_monitor /src/main.cpp:48

SUMMARY: ThreadSanitizer: data race /src/counter.cpp:15 in increment_counter
=====
```

MemorySanitizer (MSan)

MSan detects use of uninitialized memory. It's powerful but requires special build setup.

What MSan Detects

- Reading uninitialized stack memory
- Reading uninitialized heap memory
- Passing uninitialized values to functions
- Using uninitialized values in conditionals
- Copying uninitialized memory

Limitations

MSan requires **all code** including libraries to be MSan-instrumented. This makes it impractical for full OpenBMC builds but useful for isolated component testing.

Limitation	Impact
Full instrumentation required	All libraries must be rebuilt with MSan
Incompatible with ASan/TSan	Cannot combine with other sanitizers
High complexity	Requires custom toolchain setup
False positives	Uninstrumented code causes false reports

Basic Configuration

```
# Only practical for specific isolated testing
CFLAGS:append:pn-target = " -fsanitize=memory -fPIE -fno-omit-frame-pointer"
CXXFLAGS:append:pn-target = " -fsanitize=memory -fPIE -fno-omit-frame-pointer"
LDFLAGS:append:pn-target = " -fsanitize=memory -pie"
```

When to Use MSan

- Testing isolated components with no external dependencies
- Debugging specific uninitialized memory bugs
- Running in controlled test environments with MSan-instrumented libc

Alternative: Valgrind

For detecting uninitialized memory in real OpenBMC environments, use **Valgrind memcheck** with `--track-origins=yes` instead of MSan.

Valgrind

Valgrind is a runtime instrumentation framework that doesn't require recompilation. It's slower but works with any binary.

Tool Suite

Tool	Purpose	Use Case
memcheck	Memory error detection	Leaks, invalid access, uninitialized
cachegrind	Cache profiling	Performance optimization
callgrind	Call graph profiling	Hotspot analysis
helgrind	POSIX thread errors	Data races without TSan
drd	Thread error detection	Alternative to helgrind
massif	Heap profiler	Memory usage analysis

Yocto Configuration

```
# Add Valgrind to image
IMAGE_INSTALL:append = " valgrind"

# Enable debug symbols for better output
DEBUG_BUILD = "1"
EXTRA_IMAGE_FEATURES += "dbg-pkgs"

# Include debug info packages
IMAGE_INSTALL:append = " valgrind-dbg"
```

Memcheck (Memory Errors)

```
# Basic memory check
valgrind --tool=memcheck /usr/bin/phosphor-logging

# Full leak check with origins
valgrind --tool=memcheck \
    --leak-check=full \
    --show-leak-kinds=all \
    --track-origins=yes \
    --log-file=/tmp/valgrind-memcheck.log \
    /usr/bin/phosphor-logging

# Check child processes too
valgrind --tool=memcheck \
    --trace-children=yes \
    --leak-check=full \
    /usr/bin/entity-manager

# Generate suppressions for known issues
valgrind --tool=memcheck \
    --gen-suppressions=all \
    --log-file=/tmp/valgrind-supp.log \
    /usr/bin/bmcweb
```

Memcheck Output

```
==12345== Memcheck, a memory error detector
==12345== Invalid read of size 4
==12345==    at 0x4C2B1A: process_data (handler.cpp:42)
==12345==    by 0x4C1234: main (main.cpp:15)
==12345== Address 0x5204044 is 0 bytes after a block of size 20 alloc'd
==12345==    at 0x4A2B6CD: malloc (vg_replace_malloc.c:381)
==12345==    by 0x4C2A11: allocate_buffer (handler.cpp:30)

==12345== LEAK SUMMARY:
==12345==    definitely lost: 48 bytes in 2 blocks
==12345==    indirectly lost: 0 bytes in 0 blocks
==12345==    possibly lost: 120 bytes in 4 blocks
==12345==    still reachable: 1,024 bytes in 10 blocks
==12345==    suppressed: 0 bytes in 0 blocks
```

Cachegrind (Cache Profiling)

```
# Profile cache behavior
valgrind --tool=cachegrind \
  --cachegrind-out-file=/tmp/cachegrind.out \
  /usr/bin/bmcweb &

# Let it run, then stop and analyze
cg_annotate /tmp/cachegrind.out

# Compare two runs
cg_diff /tmp/cachegrind.out.1 /tmp/cachegrind.out.2
```

Callgrind (Call Graph)

```
# Generate call graph data
valgrind --tool=callgrind \
  --callgrind-out-file=/tmp/callgrind.out \
  /usr/bin/ipmid &

# Run for a while, then analyze
callgrind_annotate /tmp/callgrind.out

# Visualize with KCachegrind (on host)
# Copy callgrind.out to host and open with kcachegrind
```

Helgrind (Thread Errors)

```
# Check for data races and lock issues
valgrind --tool=helgrind \
  --log-file=/tmp/helgrind.log \
  /usr/bin/phosphor-state-manager

# Include lock order information
valgrind --tool=helgrind \
  --track-lockorders=yes \
  --history-level=full \
  /usr/bin/phosphor-state-manager
```

DRD (Data Race Detector)

```
# Alternative to helgrind
valgrind --tool=drd \
  --log-file=/tmp/drd.log \
  /usr/bin/phosphor-state-manager

# With specific options
valgrind --tool=drd \
  --check-stack-var=yes \
  --trace-barrier=yes \
  /usr/bin/phosphor-state-manager
```

Massif (Heap Profiler)

```
# Profile heap usage over time
valgrind --tool=massif \
  --massif-out-file=/tmp/massif.out \
  --time-unit=B \
  /usr/bin/entity-manager &

# Analyze results
ms_print /tmp/massif.out
```

Suppression File

```
# /etc/valgrind.supp

# Suppress known leaks in glib
{
  glib_type_init_leak
  Memcheck:Leak
  match-leak-kinds: possible
  fun:calloc
  ...
  fun:g_type_init*
}

# Suppress false positive in dbus
{
  dbus_connection_leak
  Memcheck:Leak
  match-leak-kinds: reachable
  fun:malloc
  ...
  fun:dbus_connection_open*
}

# Suppress systemd internal allocations
{
  systemd_internal
  Memcheck:Leak
  match-leak-kinds: reachable
  ...
  obj:/lib/libsystemd.so*
}
```

```
# Use suppression file
valgrind --suppressions=/etc/valgrind.supp \
  --tool=memcheck \
  /usr/bin/bmcweb
```

Running OpenBMC Services

```
# Stop service, run under Valgrind, then restart normally
systemctl stop phosphor-logging

valgrind --tool=memcheck \
    --leak-check=full \
    --log-file=/tmp/valgrind-logging.log \
    /usr/bin/phosphor-log-manager &

# Test the service
busctl tree xyz.openbmc_project.Logging

# Stop Valgrind version and restart normal service
kill %1
systemctl start phosphor-logging
```

Kernel Debug Options

For debugging kernel-level issues, Linux provides kernel-integrated sanitizers.

KASAN (Kernel AddressSanitizer)

```
# Enable KASAN
CONFIG_KASAN=y

# Choose implementation
CONFIG_KASAN_GENERIC=y      # Software, works everywhere
# CONFIG_KASAN_SW_TAGS=y    # ARM64 with TBI
# CONFIG_KASAN_HW_TAGS=y    # ARM64 with MTE hardware

# Additional options
CONFIG_KASAN_INLINE=y       # Faster but larger kernel
CONFIG_KASAN_VMALLOCS=y     # Check vmalloc allocations
CONFIG_KASAN_STACK=y        # Check stack variables
CONFIG_STACKTRACE=y         # Better stack traces
```

Yocto Kernel Config

```
# In linux-aspeed_%.bbappend
SRC_URI += "file://kasan.cfg"

# kasan.cfg contents:
# CONFIG_KASAN=y
# CONFIG_KASAN_GENERIC=y
# CONFIG_KASAN_INLINE=y
# CONFIG_STACKTRACE=y
```

KASAN Output

```
BUG: KASAN: slab-out-of-bounds in driver_function+0x1234/0x5678
Write of size 4 at addr ffff888012345678 by task kworker/0:1/123
```

```
CPU: 0 PID: 123 Comm: kworker/0:1 Not tainted 5.15.0 #1
```

```
Hardware name: ASPEED AST2600 EVB
```

```
Call Trace:
```

```
dump_stack+0x12/0x34
print_address_description+0x56/0x78
kasan_report+0x9a/0xbc
driver_function+0x1234/0x5678
...
```

```
Allocated by task 100:
```

```
kasan_save_stack+0x12/0x34
kmem_cache_alloc+0x56/0x78
allocator_function+0x9a/0xbc
...
```

Kernel UBSan

```
# Enable kernel UBSan
CONFIG_UBSAN=y
CONFIG_UBSAN_SANITIZE_ALL=y

# Specific checks
CONFIG_UBSAN_BOUNDS=y           # Array bounds
CONFIG_UBSAN_SHIFT=y           # Shift overflow
CONFIG_UBSAN_DIV_ZERO=y        # Division by zero
CONFIG_UBSAN_UNREACHABLE=y      # Unreachable code
CONFIG_UBSAN_ALIGNMENT=y       # Alignment issues (may be noisy on ARM)
```

KCSAN (Kernel Concurrency Sanitizer)

```
# Enable KCSAN for data race detection
CONFIG_KCSAN=y
CONFIG_KCSAN_STRICT=y           # Stricter checking
CONFIG_KCSAN_WEAK_MEMORY=y      # Detect weak memory ordering bugs
CONFIG_KCSAN_REPORT_ONCE_IN_MS=1000 # Rate limit reports

# Additional options
CONFIG_KCSAN_REPORT_VALUE_CHANGE_ONLY=n # Report all races
CONFIG_KCSAN_ASSUME_PLAIN_WRITES_ATOMIC=n # Stricter plain access checking
```

KMSAN (Kernel MemorySanitizer)

```
# Enable KMSAN (requires Linux 6.1+)
CONFIG_KMSAN=y

# Note: KMSAN has significant overhead and is
# primarily for kernel development/testing
```

Lockdep (Lock Validator)

```
# Enable lock debugging
CONFIG_PROVE_LOCKING=y      # Prove lock correctness
CONFIG_LOCK_STAT=y          # Lock statistics
CONFIG_DEBUG_LOCK_ALLOC=y   # Debug lock allocation
CONFIG_DEBUG_ATOMIC_SLEEP=y # Detect sleeping in atomic
CONFIG_DEBUG_MUTEXES=y      # Debug mutex operations
CONFIG_DEBUG_SPINLOCK=y     # Debug spinlock operations
CONFIG_DEBUG_RT_MUTEXES=y   # Debug RT mutexes
```

Lockdep Output

```
=====
WARNING: possible circular locking dependency detected
-----
kworker/0:1/123 is trying to acquire lock:
ffff888012345678 (&dev->mutex){+..}-{3:3}, at: device_operation+0x12/0x34

but task is already holding lock:
ffff888087654321 (&subsys->lock){+..}-{3:3}, at: subsys_function+0x56/0x78

which lock already depends on the new lock.

the existing dependency chain (in reverse order) is:
-> #1 (&subsys->lock){+..}-{3:3}:
    lock_acquire+0x12/0x34
    mutex_lock+0x56/0x78
    device_callback+0x9a/0xbc

-> #0 (&dev->mutex){+..}-{3:3}:
    lock_acquire+0x12/0x34
    mutex_lock+0x56/0x78
    device_operation+0x12/0x34
```

Combined Kernel Debug Config

```
# meta-mylayer/recipes-kernel/linux/files/kernel-debug.cfg

# Basic debug
CONFIG_DEBUG_KERNEL=y
CONFIG_DEBUG_INFO=y
CONFIG_DEBUG_INFO_DWARF4=y
CONFIG_FRAME_POINTER=y
CONFIG_STACKTRACE=y

# Memory debugging
CONFIG_KASAN=y
CONFIG_KASAN_GENERIC=y
CONFIG_KASAN_INLINE=y
CONFIG_DEBUG_PAGEALLOC=y
CONFIG_DEBUG_SLAB=y
CONFIG_SLUB_DEBUG=y

# Undefined behavior
CONFIG_UBSAN=y
CONFIG_UBSAN_SANITIZE_ALL=y

# Lock debugging
CONFIG_PROVE_LOCKING=y
CONFIG_DEBUG_LOCK_ALLOC=y
CONFIG_DEBUG_ATOMIC_SLEEP=y
CONFIG_DEBUG_MUTEXES=y
CONFIG_DEBUG_SPINLOCK=y

# Dynamic debug
CONFIG_DYNAMIC_DEBUG=y
CONFIG_PRINTK_TIME=y

# Fault injection (for testing)
CONFIG_FAULT_INJECTION=y
CONFIG_FAILSLAB=y
CONFIG_FAIL_PAGE_ALLOC=y
```


Performance Considerations

Overhead Comparison

Tool	CPU Overhead	Memory Overhead	Notes
ASan	2x	2-3x	Acceptable for development
UBSan	1.2x	Minimal	Low overhead, use widely
TSan	5-15x	5-10x	64-bit only
MSan	3x	2x	Requires full instrumentation
Valgrind memcheck	10-50x	2x	No recompilation
KASAN	1.5-3x	1.5-2x	Kernel only
KCSAN	2-5x	Moderate	Kernel only
lockdep	1.5-2x	Moderate	Kernel only

Recommendations by Environment

Environment	Recommended Tools
Development	ASan + UBSan, Valgrind as needed
CI/Testing	ASan + UBSan for all builds
Performance testing	Clean builds (no sanitizers)
Race debugging	TSan (64-bit) or Helgrind
Kernel debugging	KASAN + lockdep
Production	None (debug builds should not ship)

Resource Requirements

```
# ASan increases memory usage ~2-3x
# Ensure BMC has sufficient RAM or use QEMU with more memory

# QEMU with increased memory for sanitizer testing
qemu-system-arm -m 1024 ... # Instead of default 512MB

# Valgrind typically needs 2x memory
# May need to increase /tmp size for logs
mount -o remount,size=100M /tmp
```

Troubleshooting

ASan: No Symbols in Stack Trace

```
# Ensure debug symbols are built
DEBUG_BUILD = "1"
EXTRA_IMAGE_FEATURES += "dbg-pkgs"

# Set symbolizer path
export ASAN_SYMBOLIZER_PATH=/usr/bin/llvm-symbolizer

# Or use addr2line
ASAN_OPTIONS="symbolize=0" ./program 2>&1 | asan_symbolize.py
```

ASan: Out of Memory

```
# Reduce quarantine size
export ASAN_OPTIONS="quarantine_size_mb=32"

# Reduce redzone size
export ASAN_OPTIONS="redzone=32"

# Reduce malloc context
export ASAN_OPTIONS="malloc_context_size=5"
```

TSan: False Positives

```
# Create suppression file for known benign races
cat > /etc/tsan.supp << 'EOF'
race:benign_counter_function
race:third_party_lib.so
EOF

export TSAN_OPTIONS="suppressions=/etc/tsan.supp"
```

Valgrind: Slow Startup

```
# Use --trace-children=no if not needed
valgrind --trace-children=no ...

# Reduce history for helgrind
valgrind --tool=helgrind --history-level=none ...
```

Valgrind: Too Many Errors

```
# Stop after N errors
valgrind --error-limit=yes --errors-for-leak-kinds=definite ...

# Use suppressions
valgrind --suppressions=/etc/valgrind.supp ...

# Focus on definite leaks only
valgrind --show-leak-kinds=definite ...
```

Kernel: KASAN Flooding Logs

```
# Rate limit KASAN reports
echo 1 > /sys/module/kasan/parameters/report_threshold

# Or in kernel command line
kasan.report_threshold=1
```

Quick Reference

Enable ASan for a Recipe

```
# recipe_%.bbappend
CFLAGS:append = " -fsanitize=address -fno-omit-frame-pointer"
CXXFLAGS:append = " -fsanitize=address -fno-omit-frame-pointer"
LDFLAGS:append = " -fsanitize=address"
```

Enable ASan + UBSan (Meson)

```
EXTRA_OEMESON:append:pn-recipe = " -Db_sanitize=address,undefined"
```

Run with Valgrind

```
valgrind --leak-check=full --track-origins=yes /usr/bin/program
```

Enable KASAN

```
CONFIG_KASAN=y
CONFIG_KASAN_GENERIC=y
```

Deep Dive

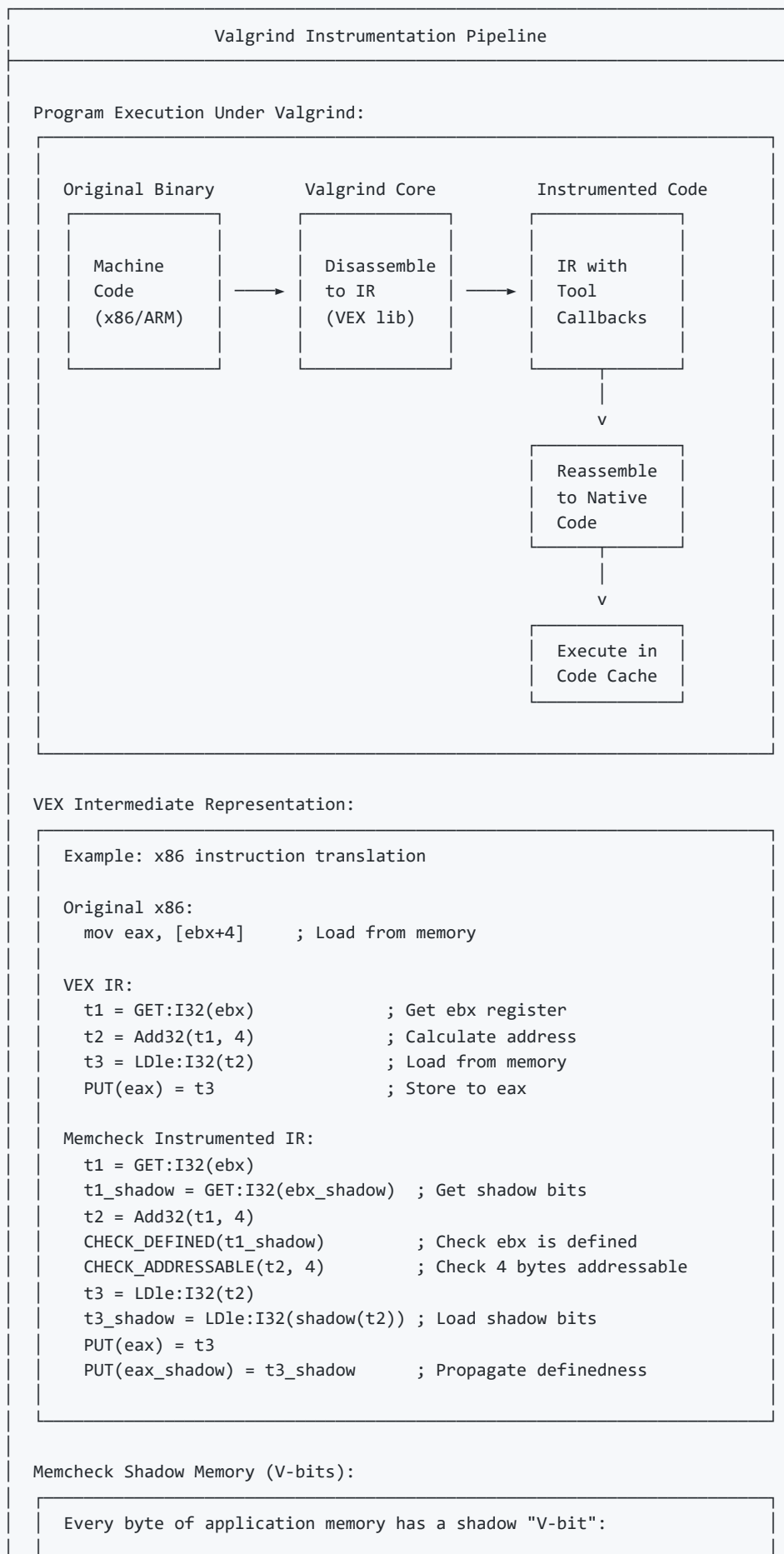
This section provides detailed technical information for developers who want to understand how sanitizers and debugging tools work internally.

AddressSanitizer Memory Layout



```
    if ((ptr & 7) + access_size > shadow_value) {  
        __asan_report_store(ptr, access_size); // Report error  
    }  
}  
*ptr = value; // Original store
```

Valgrind Binary Translation



V-bit = 0: Byte is defined (initialized)
V-bit = 1: Byte is undefined (uninitialized)

Additionally, A-bits track addressability:

A-bit = 0: Address is not valid (unmapped/freed)
A-bit = 1: Address is valid (allocated)

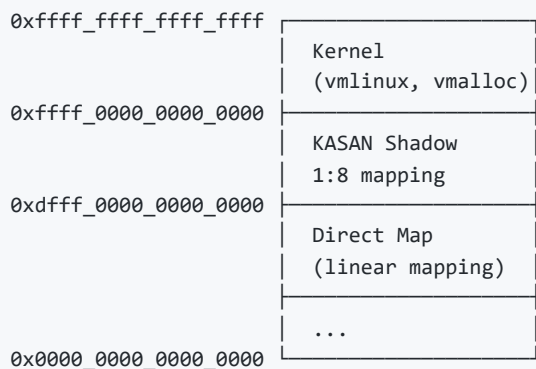
Memory operations:

- malloc(): Set A-bits=1, V-bits=1 (addressable but undefined)
- write: Set V-bits=0 for written bytes
- free(): Set A-bits=0 (no longer addressable)
- read: Check A-bits=1 and report if V-bits=1

Kernel AddressSanitizer Internals

KASAN Shadow Memory Layout (ARM64):

Kernel Virtual Address Space (48-bit):



Shadow Address Calculation:

```
shadow = (addr >> KASAN_SHADOW_SCALE_SHIFT) + KASAN_SHADOW_OFFSET
KASAN_SHADOW_SCALE_SHIFT = 3 (1:8 ratio)
```

KASAN Hook Points:

Memory Allocator Hooks:

SLAB/SLUB Allocator:

```
kmalloc() → kasan_kmalloc() → poison redzones
kfree()   → kasan_slab_free() → poison entire object
```

Page Allocator:

```
alloc_pages() → kasan_alloc_pages() → unpoison range
free_pages()  → kasan_free_pages()  → poison range
```

vmalloc:

```
vmalloc() → kasan_unpoison_vmalloc()
vfree()   → kasan_poison_vmalloc()
```

Compiler Instrumentation (GCC/Clang):

Every memory access instrumented with check:

```
// Generated for: *ptr = val;
__asan_store4(ptr); // Check 4-byte store is valid
*ptr = val;         // Original store
```

```
// Generated for: val = *ptr;
__asan_load4(ptr); // Check 4-byte load is valid
val = *ptr;        // Original load
```

Check implementation (simplified):

```
void __asan_load4(void *addr) {
    u8 shadow = *(u8*)((addr >> 3) + KASAN_SHADOW_OFFSET);
```



```

    if (unlikely(shadow && shadow < ((addr & 7) + 4))) {
        kasan_report(addr, 4, false); // Report invalid read
    }
}

```

Quarantine (Use-After-Free Detection):

Instead of immediately freeing memory, KASAN quarantines it:

kfree(ptr):

1. Poison the memory (set shadow to 0xFB)
2. Add to quarantine queue
3. When quarantine full, actually free oldest entries

Quarantine Queue:



└ Oldest freed (will be actually freed when quarantine full)

CONFIG_KASAN_QUARANTINE_SIZE = 8MB (default)

Source Code References

Component	Repository	Key Files
LLVM ASan Runtime	llvm/compiler-rt	compiler-rt/lib/asan/
GCC ASan Runtime	gcc/libsanitizer	libsanitizer/asan/
Valgrind Core	valgrind/valgrind	coregrind/ , VEX/
Valgrind Memcheck	valgrind/valgrind	memcheck/mc_main.c , memcheck/mc_translate.c
Linux KASAN	torvalds/linux	mm/kasan/ , include/linux/kasan.h
Linux KCSAN	torvalds/linux	kernel/kcsan/ , include/linux/kcsan.h
Linux Lockdep	torvalds/linux	kernel/locking/lockdep.c

Examples

Working examples are available in the [examples/debug](#) directory:

- [asan-local.conf](#) - AddressSanitizer local.conf configuration
- [kernel-debug.cfg](#) - Kernel debug options configuration
- [tsan.suppress](#) - ThreadSanitizer suppression file
- [valgrind.suppress](#) - Valgrind suppression file

References

- [AddressSanitizer Documentation](#) - Clang ASan reference
 - [UndefinedBehaviorSanitizer](#) - Clang UBSan reference
 - [ThreadSanitizer](#) - Clang TSan reference
 - [MemorySanitizer](#) - Clang MSan reference
 - [Valgrind Documentation](#) - Complete Valgrind manual
 - [KASAN Documentation](#) - Kernel ASan
 - [KCSAN Documentation](#) - Kernel Concurrency Sanitizer
 - [Lockdep Design](#) - Lock validator design
 - [GCC Sanitizer Options](#) - GCC sanitizer flags
-

Tested on: OpenBMC master with QEMU romulus/witherspoon. Kernel options tested on Linux 5.15+. Valgrind tested on ARM (limited feature support) and x86_64.

eSPI Guide

Configure and use eSPI (Enhanced Serial Peripheral Interface) for BMC-to-host communication on OpenBMC.

Table of Contents

1. [Overview](#)
 1. [eSPI vs LPC Comparison](#)
 2. [eSPI Architecture](#)
 3. [eSPI Channel Types](#)
 4. [Channel Selection Guide](#)
2. [Virtual Wire Channel](#)
 1. [Virtual Wire Signal Table](#)
 2. [Common Virtual Wires Explained](#)
 3. [Virtual Wire to GPIO Mapping](#)
 4. [Device Tree Configuration for Virtual Wires](#)
 5. [ASPEED eSPI Virtual Wire Driver](#)
 6. [Virtual Wire Troubleshooting](#)
3. [eSPI Console/UART](#)
 1. [eSPI UART Operation](#)
 2. [Kernel Configuration for eSPI UART](#)
 3. [Device Tree for eSPI UART](#)
 4. [Using eSPI UART](#)
 5. [SOL over eSPI OOB Channel](#)
4. [OOB Message Channel](#)
 1. [OOB Channel Overview](#)
 2. [OOB Use Cases](#)
 3. [Kernel Configuration for OOB](#)
 4. [Device Tree for OOB Channel](#)
 5. [Sending/Receiving OOB Messages](#)
5. [Flash Access Channel & eDAF](#)
 1. [eDAF Architecture](#)
 2. [eDAF Boot Flow](#)
 3. [Flash Layout for eDAF](#)
 4. [MTD Partition Configuration](#)
 5. [U-Boot Configuration for eDAF](#)
 6. [Device Tree for Flash Access Channel](#)
 7. [eDAF Troubleshooting](#)
6. [Kernel Drivers & Device Tree](#)
 1. [ASPEED eSPI Kernel Drivers](#)

- 2. [Kernel Configuration Options](#)
 - 3. [Complete Device Tree Example](#)
 - 4. [Device Tree Properties Reference](#)
 - 5. [Pinmux Configuration](#)
 - 7. [Platform-Specific Notes](#)
 - 1. [Intel Platform Considerations](#)
 - 2. [AMD Platform Considerations](#)
 - 3. [Intel vs AMD Summary](#)
 - 8. [Troubleshooting](#)
 - 1. [Step-by-Step eSPI Debugging](#)
 - 2. [Common Failure Modes](#)
 - 3. [Relevant Log Messages](#)
 - 4. [Debug Interfaces](#)
 - 5. [Reading eSPI Status Registers](#)
 - 9. [References](#)
 - 1. [Specifications](#)
 - 2. [Kernel Documentation](#)
 - 3. [OpenBMC Resources](#)
-

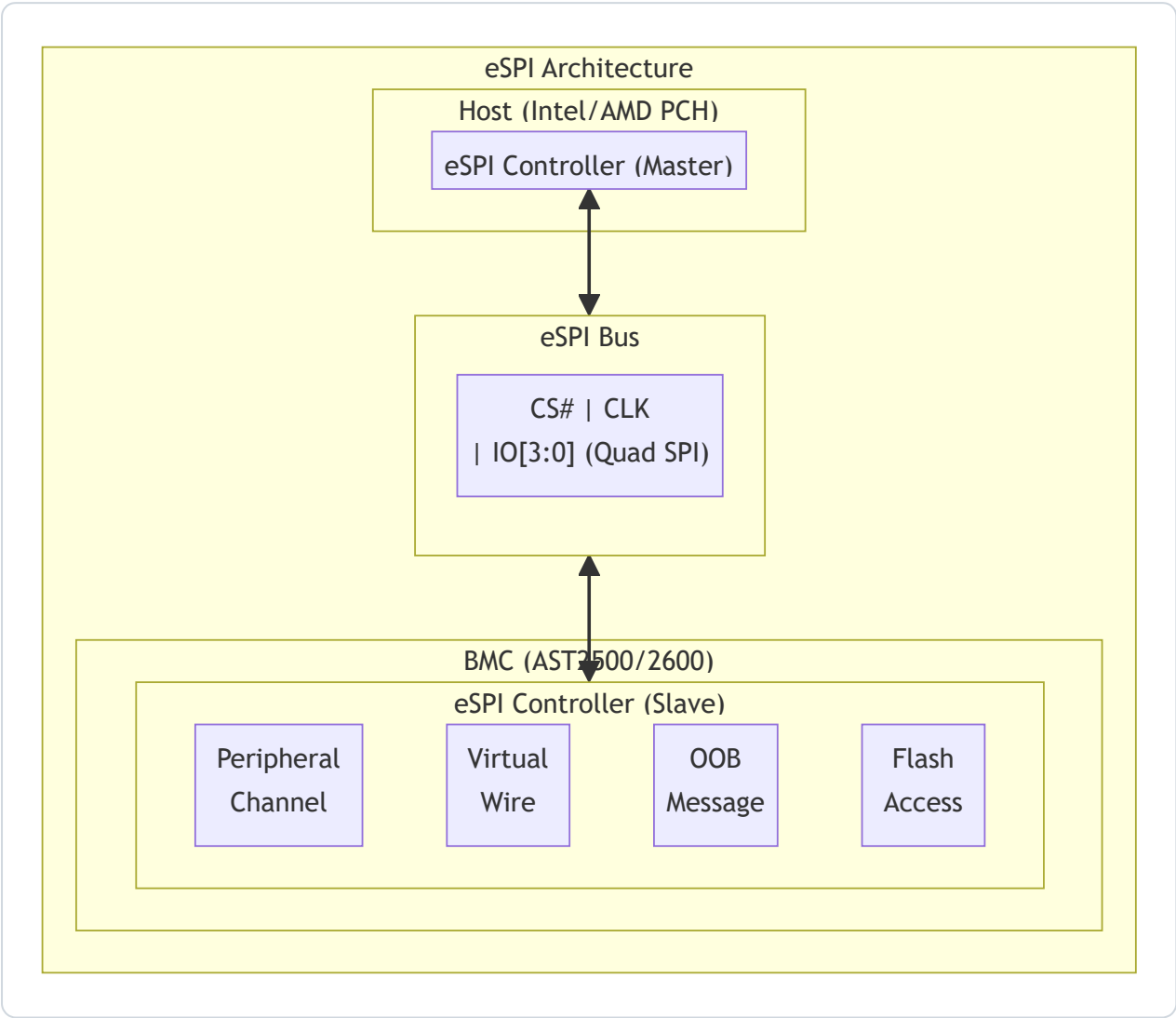
Overview

eSPI (Enhanced Serial Peripheral Interface) is Intel's successor to LPC (Low Pin Count), providing higher bandwidth and lower pin count for BMC-to-host communication. Modern Intel and AMD platforms use eSPI exclusively for BMC connectivity.

eSPI vs LPC Comparison

Feature	LPC	eSPI
Pins	13 signals	4-6 signals
Bandwidth	16.67 MB/s (33 MHz)	66 MB/s (66 MHz quad I/O)
Voltage	3.3V only	1.8V or 3.3V
Channels	Single	4 independent channels
Alert	SERIRQ	Alert# pin + in-band
Flash Access	Separate SPI	Integrated flash channel

eSPI Architecture



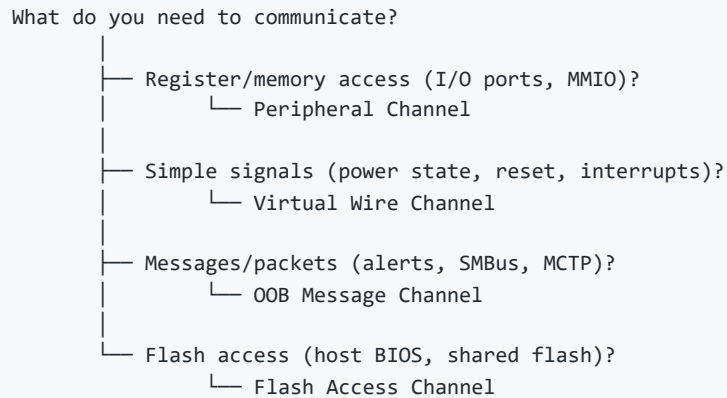
► ASCII-art version (for comparison)

eSPI Channel Types

eSPI provides four independent channels, each serving different communication needs:

Channel	Purpose	Use Cases
Peripheral	Memory-mapped I/O and I/O cycles	LPC-compatible peripherals, IPMI KCS, POST codes
Virtual Wire	GPIO-like signals	Power states (SLP_S3/S4/S5), PLTRST#, interrupts
OOB Message	SMBus-like messaging	Platform alerts, tunneled SMBus, MCTP
Flash Access	SPI flash sharing	Host BIOS on BMC-attached flash (eDAF)

Channel Selection Guide



Virtual Wire Channel

Virtual wires provide GPIO-like signaling between host and BMC, replacing dedicated LPC signals with software-defined wires.

Virtual Wire Signal Table

Index	Signal	Direction	Active	Purpose
2	SLP_S3#	Host→BMC	Low	S3 sleep state indicator
3	SLP_S4#	Host→BMC	Low	S4 sleep state indicator
4	SLP_S5#	Host→BMC	Low	S5 sleep state indicator
5	SLP_A#	Host→BMC	Low	SLP_A signal
6	SLP_LAN#	Host→BMC	Low	LAN sleep signal
7	SLP_WLAN#	Host→BMC	Low	WLAN sleep signal
41	PLTRST#	Host→BMC	Low	Platform reset
42	SUS_STAT#	Host→BMC	Low	Suspend status
43	SUS_PWRDN_ACK	BMC→Host	High	Suspend power down ack
44	SUS_WARN#	Host→BMC	Low	Suspend warning
45	OOB_RST_WARN	Host→BMC	High	OOB reset warning
46	OOB_RST_ACK	BMC→Host	High	OOB reset acknowledge
47	HOST_RST_WARN	Host→BMC	High	Host reset warning
48	HOST_RST_ACK	BMC→Host	High	Host reset acknowledge
64-127	GPIO	Bi-dir	Config	Platform-specific GPIOs

Common Virtual Wires Explained

Power State Signals (SLP_S3#, SLP_S4#, SLP_S5#):

These signals indicate ACPI sleep states:

- **SLP_S3#**: Low when system enters S3 (suspend to RAM)
- **SLP_S4#**: Low when system enters S4 (hibernate)
- **SLP_S5#**: Low when system enters S5 (soft off)

Power State Detection:

SLP_S3#	SLP_S4#	SLP_S5#	System State
High	High	High	S0 (Running)
Low	High	High	S3 (Suspend RAM)
High	Low	High	S4 (Hibernate)
High	High	Low	S5 (Soft Off)
Low	Low	Low	Mechanical Off

Platform Reset (PLTRST#):

PLTRST# indicates platform reset state:

- Low during host reset sequence
- High when host completes reset
- BMC uses this to detect host boot status

Virtual Wire to GPIO Mapping

OpenBMC maps virtual wires to GPIO interfaces for integration with existing services:

```
# Virtual wires appear as GPIO lines
gpioinfo | grep -i espi
# gpiochip1 - 8 lines:
#   line 0: "espi-vw-slp-s3" input
#   line 1: "espi-vw-slp-s4" input
#   line 2: "espi-vw-slp-s5" input
#   line 3: "espi-vw-pltrst" input

# Read virtual wire state
gpioget gpiochip1 0 # Read SLP_S3#

# Monitor virtual wire changes
gpiomon gpiochip1 0 1 2 3
```

Device Tree Configuration for Virtual Wires

```
// ASPEED AST2600 eSPI Virtual Wire configuration

&espi {
    status = "okay";

    espi_vw: espi-vw {
        compatible = "aspeed,ast2600-espi-vw";
        status = "okay";

        /* Map system virtual wires to GPIO lines */
        gpio-controller;
        #gpio-cells = <2>;
        gpio-line-names =
            "espi-vw-slp-s3",    /* VW index 2 */
            "espi-vw-slp-s4",    /* VW index 3 */
            "espi-vw-slp-s5",    /* VW index 4 */
            "espi-vw-pltrst",    /* VW index 41 */
            "espi-vw-sus-stat",  /* VW index 42 */
            "espi-vw-sus-warn",  /* VW index 44 */
            "espi-vw-oob-rst-warn", /* VW index 45 */
            "espi-vw-host-rst-warn"; /* VW index 47 */
    };
};
```

ASPEED eSPI Virtual Wire Driver

Kernel Configuration:

```
CONFIG_ASPEED_ESPI=y
CONFIG_ASPEED_ESPI_VW=y
```

Driver Module Parameters:

```
# View current parameters
cat /sys/module/aspeed_espi_vw/parameters/*

# Virtual wire index mapping configured via device tree
```

Sysfs Interface:

```
# Virtual wire status
cat /sys/bus/platform/devices/*.espi-vw/status

# Debug information
cat /sys/kernel/debug/aspeed-espi-vw/vw_status
```

Virtual Wire Troubleshooting

Issue: Virtual wire signals not changing


```
# 1. Verify eSPI controller is enabled
cat /sys/bus/platform/devices/*.espi/enabled
# Should show "1"

# 2. Check eSPI negotiation completed
dmesg | grep -i espi
# Look for "eSPI channel X enabled"

# 3. Verify virtual wire channel enabled
cat /sys/bus/platform/devices/*.espi/vw_channel_enabled

# 4. Read raw virtual wire registers
devmem 0x1e6ee080 # ASPEED VW status register (example)

# 5. Check for interrupt activity
cat /proc/interrupts | grep espi
```

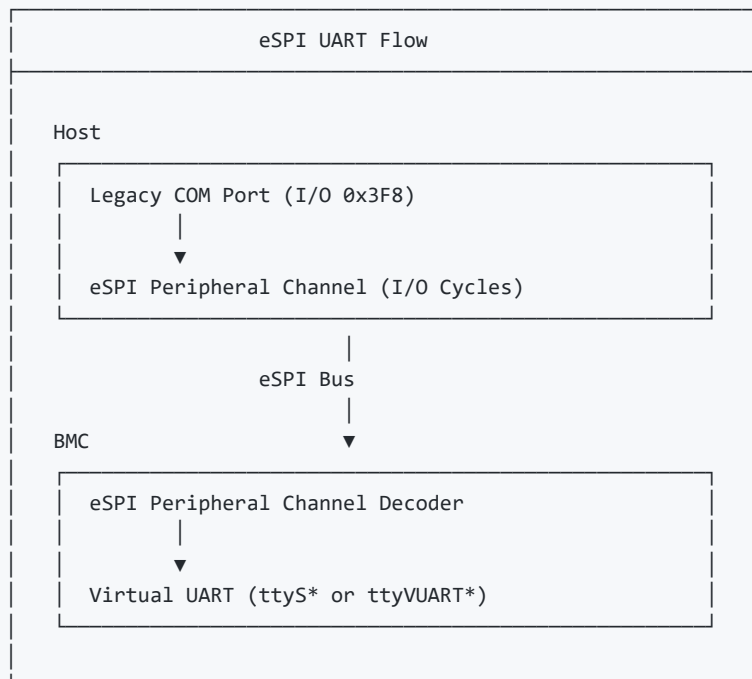
Issue: Wrong polarity or stuck signals

```
# Check device tree gpio-active-low settings
# Verify host chipset VW configuration
# Check for electrical issues on eSPI bus
```

eSPI Console/UART

eSPI can provide UART functionality through the peripheral channel, replacing dedicated UART pins.

eSPI UART Operation



Kernel Configuration for eSPI UART

```
# ASPEED eSPI UART support
CONFIG_ASPEED_ESPI=y
CONFIG_ASPEED_ESPI_PERIPHERAL=y
CONFIG_SERIAL_8250=y
CONFIG_SERIAL_8250_ASPEED_VUART=y
```

Device Tree for eSPI UART

```
// ASPEED eSPI with UART over peripheral channel

&espi {
    status = "okay";

    espi_peri: espi-peripheral {
        compatible = "aspeed,ast2600-espi-peripheral";
        status = "okay";

        /* UART over eSPI peripheral channel */
        uart-over-espi;
        uart-io-base = <0x3f8>; /* COM1 I/O address */
    };
};

/* Virtual UART configuration */
&vuart1 {
    status = "okay";
    aspeed,sirq = <4>; /* IRQ 4 for COM1 */
    aspeed,lpc-io-reg = <0x3f8>;
};
```

Using eSPI UART

```
# List serial devices
ls -la /dev/ttyS* /dev/ttyVUART*

# Configure serial parameters
stty -F /dev/ttyVUART0 115200 cs8 -cstopb -parenb

# Connect to host console
screen /dev/ttyVUART0 115200

# Or use obmc-console
systemctl status obmc-console@ttyVUART0
```

SOL over eSPI OOB Channel

Serial over LAN can also use eSPI OOB channel for IPMI SOL:

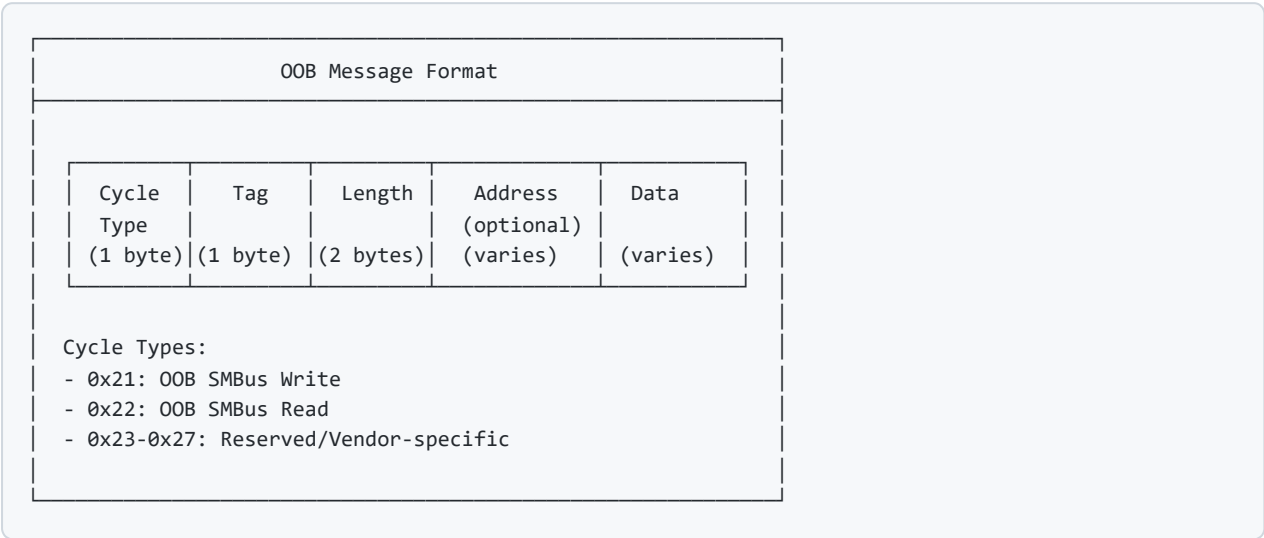
```
# Configure IPMI SOL to use eSPI-backed UART
# In phosphor-ipmi-host configuration

# SOL activation uses standard IPMI commands
ipmitool -I lanplus -H <bmc-ip> -U root -P password sol activate
```

OOB Message Channel

The OOB (Out-of-Band) Message Channel provides SMBus-like packet communication between host and BMC.

OOB Channel Overview



OOB Use Cases

Use Case	Description
Platform Alerts	BMC sends alerts to host (thermal, power, errors)
Tunneled SMBus	SMBus transactions over eSPI (sensor access)
MCTP over eSPI	PLDM/MCTP messages over OOB channel
Vendor Commands	Platform-specific communication

Kernel Configuration for OOB

```
CONFIG_ASPEED_ESPI=y
CONFIG_ASPEED_ESPI_OOB=y
```

Device Tree for OOB Channel

```
&espi {
    status = "okay";

    espi_oob: espi-oob {
        compatible = "aspeed,ast2600-espi-oob";
        status = "okay";

        /* OOB channel configuration */
        oob-channel-enable;
    };
};
```

Sending/Receiving OOB Messages

```
# OOB interface (platform-specific)
# Typically accessed via character device or netlink

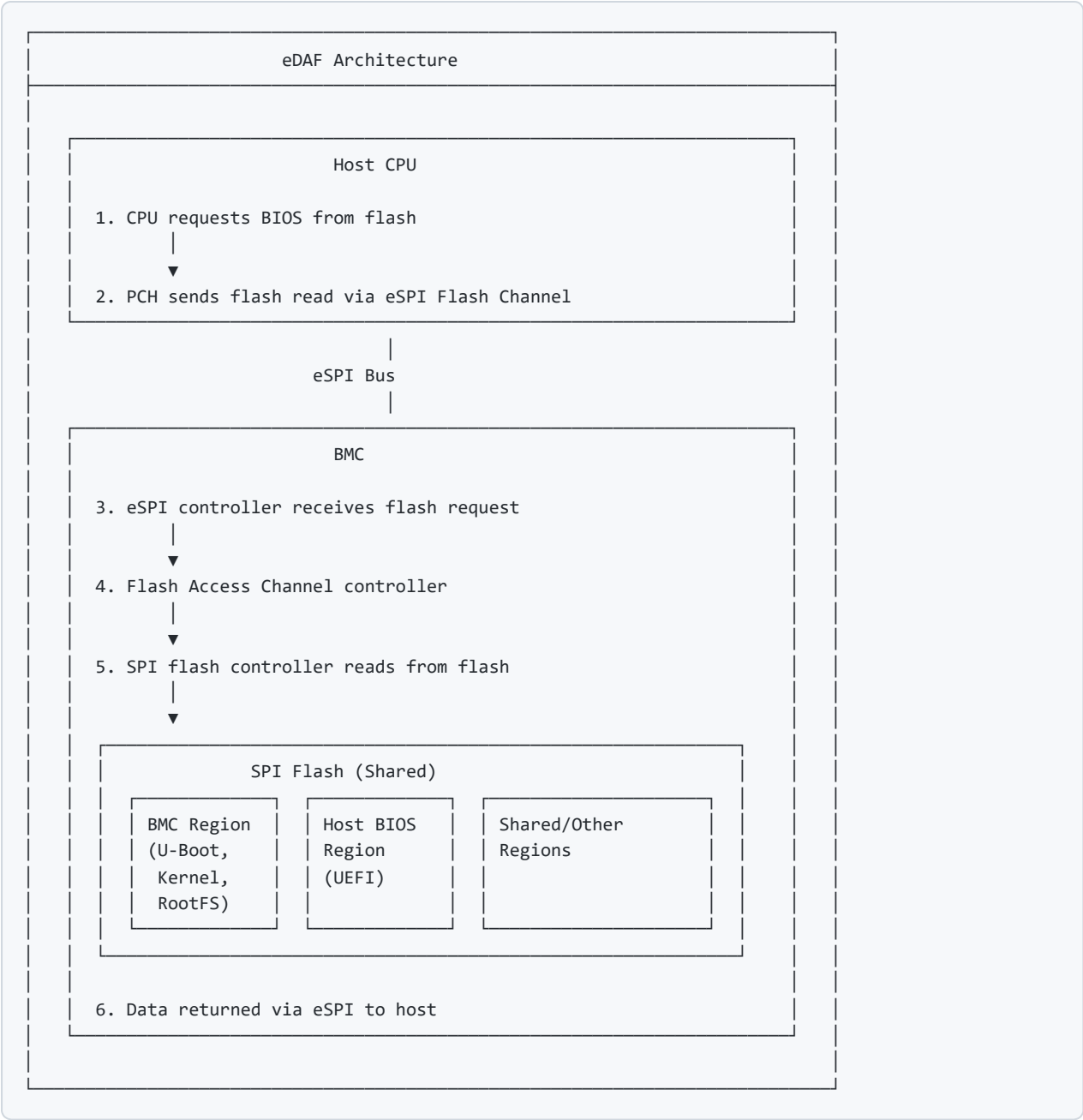
# Check OOB channel status
cat /sys/bus/platform/devices/*.espi-oob/status

# Debug OOB messages
echo 1 > /sys/kernel/debug/aspeed-espi-oob/trace_enable
cat /sys/kernel/debug/aspeed-espi-oob/trace
```

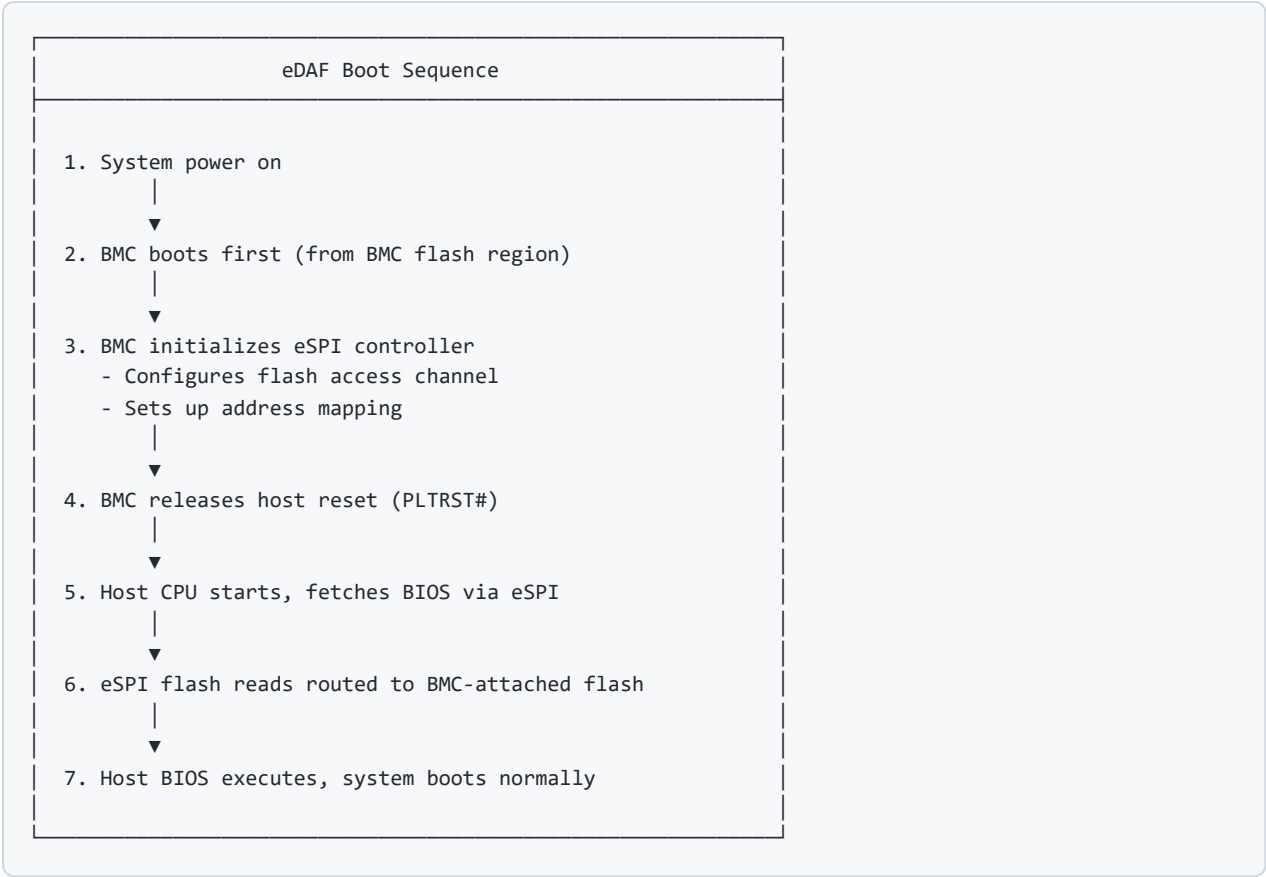
Flash Access Channel & eDAF

The Flash Access Channel enables the host to access SPI flash attached to the BMC, enabling eDAF (eSPI Direct Attached Flash) boot.

eDAF Architecture



eDAF Boot Flow



Flash Layout for eDAF

Shared SPI Flash Layout (64MB example)			
Offset	Size	Region	Access
0x00000000	512KB	U-Boot	BMC only
0x00080000	64KB	U-Boot Env	BMC only
0x00090000	~7MB	BMC Kernel	BMC only
0x00800000	24MB	BMC RootFS	BMC only
0x02000000	16MB	Host BIOS	Host via eSPI
0x03000000	16MB	Host BIOS (B)	Host via eSPI

MTD Partition Configuration

```
// Device tree flash partition for eDAF

&fmc {
    status = "okay";
    flash@0 {
        status = "okay";
        m25p,fast-read;
        label = "bmc";
        spi-max-frequency = <50000000>;

        partitions {
            compatible = "fixed-partitions";
            #address-cells = <1>;
            #size-cells = <1>;

            u-boot@0 {
                reg = <0x0 0x80000>;
                label = "u-boot";
            };

            u-boot-env@80000 {
                reg = <0x80000 0x10000>;
                label = "u-boot-env";
            };

            kernel@90000 {
                reg = <0x90000 0x770000>;
                label = "kernel";
            };

            rofs@800000 {
                reg = <0x800000 0x1800000>;
                label = "rofs";
            };

            /* Host BIOS region - accessed via eSPI flash channel */
            host-bios@2000000 {
                reg = <0x2000000 0x1000000>;
                label = "host-bios";
            };

            host-bios-backup@3000000 {
                reg = <0x3000000 0x1000000>;
                label = "host-bios-backup";
            };
        };
    };
};
```

U-Boot Configuration for eDAF

```
# U-Boot environment for eDAF systems

# Ensure BMC doesn't interfere with host BIOS region
# Set flash protection for host region during normal operation

# U-Boot defconfig additions:
CONFIG_ASPEED_ESPI=y
CONFIG_ASPEED_ESPI_FLASH=y

# Environment variables
flash_protect=yes
host_bios_offset=0x2000000
host_bios_size=0x1000000
```

Device Tree for Flash Access Channel

```
&espi {
    status = "okay";

    espi_flash: espi-flash {
        compatible = "aspeed,ast2600-espi-flash";
        status = "okay";

        /* Flash access channel configuration */
        flash-channel-enable;

        /* Address mapping for host flash access */
        /* Host sees flash starting at 0x0, mapped to BMC flash offset */
        flash-mafs-mode; /* Master Attached Flash Sharing */

        /* Or SAF mode for more control */
        /* flash-saf-mode; */
    };
};
```

eDAF Troubleshooting

Issue: Host fails to boot (no BIOS execution)

```
# 1. Verify eSPI flash channel is enabled
cat /sys/bus/platform/devices/*.espi-flash/status

# 2. Check flash access channel negotiation
dmesg | grep -i "espi.*flash"
# Look for "flash channel enabled"

# 3. Verify host BIOS region is accessible
hexdump -C /dev/mtd/host-bios | head
# Should show valid BIOS header (not 0xFF)

# 4. Check eSPI timing
# Host may request flash before BMC is ready
# Verify BMC boot timing vs host release

# 5. Monitor flash access
echo 1 > /sys/kernel/debug/aspeed-espi-flash/trace
cat /sys/kernel/debug/aspeed-espi-flash/trace
```


Issue: Flash corruption or read errors

```
# Check for SPI flash errors
dmesg | grep -i spi
dmesg | grep -i mtd

# Verify flash chip detection
cat /sys/class/mtd/mtd*/name
cat /sys/class/mtd/mtd*/size

# Test flash read
dd if=/dev/mtd/host-bios of=/tmp/bios.bin bs=4096 count=1
hexdump -C /tmp/bios.bin
```

Kernel Drivers & Device Tree

ASPEED eSPI Kernel Drivers

Driver	Module	Purpose
<code>aspeed-espi-ctrl</code>	<code>aspeed_espi</code>	Main eSPI controller
<code>aspeed-espi-vw</code>	<code>aspeed_espi_vw</code>	Virtual wire channel
<code>aspeed-espi-oob</code>	<code>aspeed_espi_oob</code>	OOB message channel
<code>aspeed-espi-flash</code>	<code>aspeed_espi_flash</code>	Flash access channel
<code>aspeed-espi-peri</code>	<code>aspeed_espi_peri</code>	Peripheral channel

Kernel Configuration Options

```
# Main eSPI support
CONFIG_ASPEED_ESPI=y

# Individual channel support
CONFIG_ASPEED_ESPI_PERIPHERAL=y
CONFIG_ASPEED_ESPI_VW=y
CONFIG_ASPEED_ESPI_OOB=y
CONFIG_ASPEED_ESPI_FLASH=y

# Debug options
CONFIG_ASPEED_ESPI_DEBUG=y
```

Complete Device Tree Example

```
// Complete ASPEED AST2600 eSPI configuration

/ {
    compatible = "aspeed,ast2600";

    /* eSPI pin configuration */
    pinctrl {
        espi_default: espi_default {
            function = "ESPI";
            groups = "ESPI";
        };
    };
};

&espi {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&espi_default>;

    /* eSPI clock and timing */
    espi,clock-frequency = <66000000>; /* 66 MHz */

    /* Peripheral channel */
    espi_peri: espi-peripheral {
        compatible = "aspeed,ast2600-espi-peripheral";
        status = "okay";

        /* I/O address decoding */
        io-addr = <0x60 0x64>; /* KCS interface */
        io-addr = <0x80>;      /* POST code port */

        /* Memory-mapped regions */
        mmio-addr = <0xfed1c000 0x1000>; /* HECI */
    };

    /* Virtual wire channel */
    espi_vw: espi-vw {
        compatible = "aspeed,ast2600-espi-vw";
        status = "okay";

        gpio-controller;
        #gpio-cells = <2>;
        gpio-line-names =
            "espi-vw-slp-s3",
            "espi-vw-slp-s4",
            "espi-vw-slp-s5",
            "espi-vw-pltrst",
            "espi-vw-sus-stat",
            "espi-vw-sus-warn",
            "espi-vw-oob-rst-warn",
            "espi-vw-host-rst-warn";
    };

    /* OOB message channel */
    espi_oob: espi-oob {
        compatible = "aspeed,ast2600-espi-oob";
        status = "okay";
    };

    /* Flash access channel */
    espi_flash: espi-flash {
```

```

        compatible = "aspeed,ast2600-espi-flash";
        status = "okay";
        flash-mafs-mode;
    };
};

```

Device Tree Properties Reference

Property	Type	Description
<code>status</code>	string	"okay" to enable, "disabled" to disable
<code>compatible</code>	string	Driver matching string
<code>espi,clock-frequency</code>	u32	eSPI bus clock in Hz
<code>io-addr</code>	u32 array	Peripheral channel I/O addresses
<code>mmio-addr</code>	u32 array	Peripheral channel MMIO regions
<code>gpio-controller</code>	flag	Enable GPIO interface for VW
<code>flash-mafs-mode</code>	flag	Master Attached Flash Sharing
<code>flash-saf-mode</code>	flag	Slave Attached Flash mode

Pinmux Configuration

```

// ASPEED AST2600 eSPI pinmux

&pinctrl {
    /* eSPI uses dedicated pins on AST2600 */
    /* No explicit pinmux needed for eSPI function */

    /* For AST2500, explicit pinmux may be required */
    espi_pins: espi_pins {
        function = "ESPI";
        groups = "ESPI";
        /* Pins: ESPI_CLK, ESPI_CS#, ESPI_IO[3:0], ESPI_ALERT#, ESPI_RST# */
    };
};

```

Platform-Specific Notes

Intel Platform Considerations

Virtual Wire Mapping (Intel PCH):

Intel Signal	VW Index	Description
SLP_S3#	2	S3 sleep
SLP_S4#	3	S4 sleep
SLP_S5#	4	S5 sleep
SLP_A#	5	Deep sleep
PLTRST#	41	Platform reset
PME#	65	Power management event
WAKE#	66	Wake signal

Intel ME/CSME Interaction:

```
# Intel ME may use OOB channel for HECI over eSPI
# Ensure HECI MMIO region is configured in peripheral channel

# Device tree for HECI
espi_peri: espi-peripheral {
    mmio-addr = <0xfed1c000 0x1000>; /* HECI1 */
    mmio-addr = <0xfed1d000 0x1000>; /* HECI2 */
};
```

AMD Platform Considerations

AMD PSP Interaction:

AMD platforms may have different eSPI behavior due to PSP (Platform Security Processor):

```
# AMD may use different VW indices for some signals
# Check AMD BIOS specification for exact mapping

# AMD-specific timing requirements
# PSP initialization may affect eSPI availability
```

Virtual Wire Differences:

AMD Signal	VW Index	Notes
SLP_S3#	2	Same as Intel
SLP_S5#	4	Same as Intel
PLTRST#	41	Same as Intel
AMD-specific	64+	Platform dependent

Intel vs AMD Summary

Aspect	Intel	AMD
VW Indices 2-47	Standard ACPI signals	Generally compatible
VW Indices 64+	Vendor-specific	Vendor-specific
Flash Channel	Supported	Supported
OOB Channel	HECI tunneling	PSP communication
Timing	ME initialization	PSP initialization

Troubleshooting

Step-by-Step eSPI Debugging

```
# 1. Verify eSPI controller detected
dmesg | grep -i espi
# Expected: "aspeed-espi XXXXX: eSPI controller initialized"

# 2. Check eSPI negotiation
cat /sys/bus/platform/devices/*.espi/capabilities
# Shows negotiated channels and frequencies

# 3. Verify channel status
for ch in vw oob flash peripheral; do
    echo "=== $ch channel ==="
    cat /sys/bus/platform/devices/*espi-$ch/status 2>/dev/null
done

# 4. Check for errors
dmesg | grep -i "espi.*error\|espi.*fail"

# 5. Monitor eSPI activity
cat /proc/interrupts | grep espi
```

Common Failure Modes

Symptom	Possible Cause	Resolution
No eSPI in dmesg	Driver not loaded	Check kernel config
Channel not enabled	Negotiation failed	Check host eSPI config
VW signals stuck	Polarity mismatch	Verify device tree
Flash timeouts	Timing issues	Check clock frequency
Host boot failure	eDAF not ready	Verify BMC boot timing

Relevant Log Messages

```
# Successful initialization
aspeed-espi: eSPI controller initialized
aspeed-espi: Channel capabilities: VW OOB Flash Peripheral
aspeed-espi-vw: Virtual wire channel enabled

# Common errors
aspeed-espi: Timeout waiting for channel enable
aspeed-espi: eSPI reset detected
aspeed-espi-flash: Flash read timeout
```

Debug Interfaces

Sysfs:

```
# eSPI controller status
/sys/bus/platform/devices/*.espi/
├─ capabilities      # Negotiated capabilities
├─ enabled           # Controller enable status
├─ frequency         # Operating frequency
└─ status            # Overall status

# Per-channel status
/sys/bus/platform/devices/*.espi-vw/
├─ status            # Channel status
└─ vw_gpio/          # GPIO interface for VW
```

Debugfs:

```
# Enable debugfs (if not mounted)
mount -t debugfs none /sys/kernel/debug

# eSPI debug info
/sys/kernel/debug/aspeed-espi/
├─ registers         # Register dump
├─ statistics         # Transaction counters
└─ trace              # Transaction trace (if enabled)

# Enable tracing
echo 1 > /sys/kernel/debug/aspeed-espi/trace_enable
cat /sys/kernel/debug/aspeed-espi/trace
```

Reading eSPI Status Registers

```
# ASPEED AST2600 eSPI registers (base 0x1E6EE000)
# Use with caution - direct register access

# eSPI Control Register
devmem 0x1E6EE000

# eSPI Interrupt Status
devmem 0x1E6EE008

# eSPI Interrupt Enable
devmem 0x1E6EE00C

# Virtual Wire Status
devmem 0x1E6EE080

# Flash Channel Status
devmem 0x1E6EE0C0
```

References

Specifications

- [Intel eSPI Interface Base Specification](#) - Official eSPI protocol specification
- [Intel eSPI Specification PDF \(Rev 1.6\)](#) - Direct PDF download

Kernel Documentation

- [Linux SPI Documentation](#) - General SPI subsystem
- [ASPEED eSPI Driver Patches](#) - Kernel mailing list

OpenBMC Resources

- [OpenBMC Kernel](#) - ASPEED eSPI drivers in drivers/soc/aspeed/
- [ASPEED SDK](#) - Vendor examples and reference implementations

Tested on: ASPEED AST2500/AST2600 EVB with Intel and AMD host platforms

Unit Testing Guide

Write and run unit tests for OpenBMC services using GTest/GMock.

Table of Contents

1. [Overview](#)
 1. [Why Unit Test?](#)
2. [GTest/GMock Setup with Meson](#)
 1. [Meson Configuration](#)
 2. [Project Structure](#)
 3. [Test Main Entry Point](#)
3. [Writing GTest Tests](#)
 1. [Basic Test Structure](#)
 2. [Test Fixtures \(TEST_F\)](#)
 3. [Common Assertions](#)
4. [GMock Basics](#)
 1. [Creating Mock Classes](#)
 2. [Using Mock Objects](#)
 3. [Common GMock Matchers](#)
5. [D-Bus Mocking with sdbus++](#)
 1. [Why Mock D-Bus?](#)
 2. [sdbus++ Mock Utilities](#)
 3. [Example: Mocking D-Bus Method Calls](#)
 4. [Mocking Property Gets/Sets](#)
 5. [Verifying D-Bus Interactions](#)
6. [Running Tests](#)
 1. [Via Meson \(Local Build\)](#)
 2. [Via Bitbake](#)
 3. [With SDK \(Local Development\)](#)
 4. [Interpreting Test Output](#)
7. [Test Coverage](#)
 1. [Enabling Coverage in Meson](#)
 2. [Generating Coverage Reports](#)
 3. [Coverage Report Interpretation](#)
8. [Practical Example: phosphor-logging](#)
 1. [Test Structure](#)
 2. [Example Test from phosphor-logging](#)
 3. [Running phosphor-logging Tests](#)
9. [Try It Yourself](#)

1. [Standalone Examples \(No OpenBMC Required\)](#)
 2. [Real OpenBMC Test Suites](#)
 1. [phosphor-logging \(Event Logging\)](#)
 2. [dbus-sensors \(Sensor Daemons\)](#)
 3. [bmcweb \(Redfish Server\)](#)
 10. [Best Practices](#)
 1. [Test Organization](#)
 2. [Test Independence](#)
 3. [Mock Appropriately](#)
 4. [Coverage Goals](#)
 11. [Next Steps](#)
 12. [References](#)
-

Overview

Unit testing is essential for OpenBMC development. All code submitted upstream must include unit tests. OpenBMC uses:

- **GTest** - Google Test framework for C++ unit tests
- **GMock** - Google Mock for creating mock objects
- **Meson** - Build system with native test support

This guide covers writing tests, mocking D-Bus interfaces, and running tests in OpenBMC.

Why Unit Test?

Benefit	Description
Upstream Requirement	Code without tests is rejected
Catch Regressions	Automated testing prevents bugs
Document Behavior	Tests show expected usage
Enable Refactoring	Safe code changes with test coverage

GTest/GMock Setup with Meson

Meson Configuration

Add test configuration to your service's `meson.build`:

```

# Enable testing
gtest_dep = dependency('gtest', main: true, required: true)
gmock_dep = dependency('gmock', required: true)

# Define test executable
test_src = [
    'test/main_test.cpp',
    'test/service_test.cpp',
]

test_exe = executable(
    'test-myservice',
    test_src,
    dependencies: [
        gtest_dep,
        gmock_dep,
        sdbusplus_dep,
        phosphor_logging_dep,
    ],
    include_directories: include_directories('src'),
)

# Register test with Meson
test('myservice-tests', test_exe)

```

Project Structure

Organize test files alongside source:

```

my-service/
├─ meson.build
├─ src/
│   ├─ main.cpp
│   └─ service.cpp
└─ test/
    ├─ main_test.cpp      # GTest main entry
    ├─ service_test.cpp  # Service unit tests
    └─ mocks/
        └─ mock_dbus.hpp # D-Bus mocks

```

Test Main Entry Point

Create `test/main_test.cpp`:

```

#include <gtest/gtest.h>

int main(int argc, char** argv)
{
    ::testing::InitGoogleTest(&argc, argv);
    return RUN_ALL_TESTS();
}

```

Writing GTest Tests

Basic Test Structure

```
#include <gtest/gtest.h>
#include "service.hpp"

// Simple test case
TEST(ServiceTest, InitializesCorrectly)
{
    Service svc;
    EXPECT_TRUE(svc.isInitialized());
}

// Test with expected value
TEST(ServiceTest, ReturnsDefaultValue)
{
    Service svc;
    EXPECT_EQ(svc.getValue(), 0);
}
```

Test Fixtures (TEST_F)

Use fixtures for common setup/teardown:

```
#include <gtest/gtest.h>
#include "sensor_manager.hpp"

class SensorManagerTest : public ::testing::Test
{
protected:
    void SetUp() override
    {
        // Runs before each test
        manager = std::make_unique<SensorManager>();
    }

    void TearDown() override
    {
        // Runs after each test
        manager.reset();
    }

    std::unique_ptr<SensorManager> manager;
};

TEST_F(SensorManagerTest, AddsSensorSuccessfully)
{
    EXPECT_TRUE(manager->addSensor("CPU_Temp", 50.0));
    EXPECT_EQ(manager->getSensorCount(), 1);
}

TEST_F(SensorManagerTest, RejectsDuplicateSensor)
{
    manager->addSensor("CPU_Temp", 50.0);
    EXPECT_FALSE(manager->addSensor("CPU_Temp", 60.0));
}
```

Common Assertions

Assertion	Description
<code>EXPECT_TRUE(cond)</code>	Condition is true
<code>EXPECT_FALSE(cond)</code>	Condition is false
<code>EXPECT_EQ(a, b)</code>	Values are equal
<code>EXPECT_NE(a, b)</code>	Values are not equal
<code>EXPECT_LT(a, b)</code>	$a < b$
<code>EXPECT_GT(a, b)</code>	$a > b$
<code>EXPECT_THROW(expr, type)</code>	Expression throws exception
<code>EXPECT_NO_THROW(expr)</code>	Expression doesn't throw
<code>ASSERT_*</code>	Same as EXPECT but stops test on failure

GMock Basics

Creating Mock Classes

```
#include <gmock/gmock.h>

// Interface to mock
class SensorInterface
{
public:
    virtual ~SensorInterface() = default;
    virtual double getValue() = 0;
    virtual void setValue(double value) = 0;
    virtual bool isValid() const = 0;
};

// Mock implementation
class MockSensor : public SensorInterface
{
public:
    MOCK_METHOD(double, getValue, (), (override));
    MOCK_METHOD(void, setValue, (double value), (override));
    MOCK_METHOD(bool, isValid, (), (const, override));
};
```

Using Mock Objects

```
#include <gmock/gmock.h>
#include <gtest/gtest.h>

using ::testing::Return;
using ::testing::_;

TEST(ControllerTest, ReadsFromSensor)
{
    MockSensor mockSensor;

    // Set up expectations
    EXPECT_CALL(mockSensor, getValue())
        .WillOnce(Return(45.5));

    Controller ctrl(&mockSensor);
    EXPECT_EQ(ctrl.readTemperature(), 45.5);
}

TEST(ControllerTest, HandlesSensorFailure)
{
    MockSensor mockSensor;

    EXPECT_CALL(mockSensor, isValid())
        .WillOnce(Return(false));

    Controller ctrl(&mockSensor);
    EXPECT_THROW(ctrl.readTemperature(), std::runtime_error);
}
```

Common GMock Matchers

Matcher	Description
<code>Return(value)</code>	Return specified value
<code>_</code>	Match any argument
<code>Eq(value)</code>	Argument equals value
<code>Gt(value)</code>	Argument greater than value
<code>WillOnce(action)</code>	Perform action once
<code>WillRepeatedly(action)</code>	Perform action every time
<code>Times(n)</code>	Expect exactly n calls

D-Bus Mocking with sdbus++

Why Mock D-Bus?

Unit tests should be isolated. Mocking D-Bus:

- Avoids requiring running D-Bus daemon
- Controls responses for testing edge cases

- Enables testing without real hardware

sdbus++ Mock Utilities

OpenBMC's sdbus++ provides mock helpers. Include the mock headers:

```
#include <sdbusplus/test/sdbus_mock.hpp>
```

Example: Mocking D-Bus Method Calls

```
#include <gtest/gtest.h>
#include <gmock/gmock.h>
#include <sdbusplus/test/sdbus_mock.hpp>

using ::testing::_;
using ::testing::Return;

class DBusServiceTest : public ::testing::Test
{
protected:
    sdbusplus::SdBusMock sdbusMock;

    void SetUp() override
    {
        // Set up mock bus
    }
};

TEST_F(DBusServiceTest, CallsRemoteMethod)
{
    // Expect D-Bus method call
    EXPECT_CALL(sdbusMock,
        sd_bus_call_method(_, _, _, _, _, _, _))
        .WillOnce(Return(0));

    auto bus = sdbusplus::get_mocked_new(&sdbusMock);
    MyService svc(bus);

    EXPECT_NO_THROW(svc.callRemoteMethod());
}
```

Mocking Property Gets/Sets

```
TEST_F(DBusServiceTest, GetsPropertyValue)
{
    // Mock property read
    EXPECT_CALL(sdbusMock,
        sd_bus_get_property(_, _, _, _, _, _))
        .WillOnce([](sd_bus*, const char*, const char*,
            const char*, sd_bus_error*, sd_bus_message**,
            const char*) {
            // Return mocked property value
            return 0;
        });

    auto bus = sdbusplus::get_mocked_new(&sdbusMock);
    MyService svc(bus);

    EXPECT_EQ(svc.getRemoteProperty(), expectedValue);
}
```

Verifying D-Bus Interactions

```
TEST_F(DBusServiceTest, EmitsSignalOnChange)
{
    EXPECT_CALL(sdbusMock,
        sd_bus_emit_signal(_, _, _, _, _))
        .Times(1);

    auto bus = sdbusplus::get_mocked_new(&sdbusMock);
    MyService svc(bus);

    svc.updateValue(newValue); // Should emit signal
}
```

Running Tests

Via Meson (Local Build)

```
# Configure build with tests enabled
meson setup builddir -Dtests=enabled

# Build and run tests
meson test -C builddir

# Run with verbose output
meson test -C builddir -v

# Run specific test
meson test -C builddir myservice-tests
```

Via Bitbake

```
# Run tests for a specific recipe
bitbake phosphor-logging -c test

# Run tests and keep build directory
bitbake phosphor-logging -c test -f

# Check test results
cat tmp/work/*/phosphor-logging/*/temp/log.do_test
```

With SDK (Local Development)

```
# Source SDK environment
source /opt/openbmc-phosphor/*/environment-setup-*

# Build and test locally
meson setup builddir --cross-file=cross.txt
meson test -C builddir
```

Interpreting Test Output

```
[=====] Running 5 tests from 2 test suites.
[-----] 3 tests from ServiceTest
[ RUN      ] ServiceTest.InitializesCorrectly
[      OK   ] ServiceTest.InitializesCorrectly (0 ms)
[ RUN      ] ServiceTest.ReturnsDefaultValue
[      OK   ] ServiceTest.ReturnsDefaultValue (0 ms)
[ RUN      ] ServiceTest.HandlesError
[  FAILED   ] ServiceTest.HandlesError (1 ms)
[-----] 2 tests from ControllerTest
...
[=====] 5 tests from 2 test suites ran. (10 ms total)
[  PASSED   ] 4 tests.
[  FAILED   ] 1 test.
```

Test Coverage

Enabling Coverage in Meson

Add to `meson.build`:

```
if get_option('coverage')
    add_project_arguments('-fprofile-arcs', '-ftest-coverage',
                          language: 'cpp')
    add_project_link_arguments('-lgcov', language: 'cpp')
endif
```

Add to `meson_options.txt`:

```
option('coverage', type: 'boolean', value: false,
       description: 'Enable code coverage')
```


Generating Coverage Reports

```
# Build with coverage enabled
meson setup builddir -Dcoverage=true
meson compile -C builddir

# Run tests
meson test -C builddir

# Generate coverage report with lcov
lcov --capture --directory builddir \
    --output-file coverage.info

# Generate HTML report
genhtml coverage.info --output-directory coverage-report

# View report
xdg-open coverage-report/index.html
```

Coverage Report Interpretation

Metric	Target	Description
Line Coverage	>80%	Lines executed by tests
Function Coverage	>90%	Functions called by tests
Branch Coverage	>70%	Decision branches taken

Practical Example: phosphor-logging

The `phosphor-logging` repository demonstrates OpenBMC testing patterns.

Test Structure

```
phosphor-logging/
├─ test/
│   ├── meson.build
│   ├── log_manager_test.cpp
│   ├── elog_test.cpp
│   └─ remote_logging_test.cpp
```

Example Test from phosphor-logging

```
#include <gtest/gtest.h>
#include "log_manager.hpp"

class LogManagerTest : public ::testing::Test
{
protected:
    void SetUp() override
    {
        // Create temporary log directory
        tempDir = std::filesystem::temp_directory_path() / "test-logs";
        std::filesystem::create_directories(tempDir);
    }

    void TearDown() override
    {
        std::filesystem::remove_all(tempDir);
    }

    std::filesystem::path tempDir;
};

TEST_F(LogManagerTest, CreatesLogEntry)
{
    LogManager manager(tempDir);

    auto id = manager.createEntry(
        "Test message",
        Entry::Level::Informational);

    EXPECT_GT(id, 0);
    EXPECT_TRUE(manager.hasEntry(id));
}

TEST_F(LogManagerTest, DeletesLogEntry)
{
    LogManager manager(tempDir);
    auto id = manager.createEntry("Test", Entry::Level::Error);

    EXPECT_TRUE(manager.deleteEntry(id));
    EXPECT_FALSE(manager.hasEntry(id));
}
```

Running phosphor-logging Tests

```
# Clone and build
git clone https://github.com/openbmc/phosphor-logging
cd phosphor-logging

# Build with tests
meson setup builddir -Dtests=enabled
meson compile -C builddir

# Run tests
meson test -C builddir -v
```

Try It Yourself

Standalone Examples (No OpenBMC Required)

Build and run the example tests from this tutorial:

```
# Clone the tutorial repository
git clone https://github.com/MichaelTien8901/openbmc-guide-tutorial.git
cd openbmc-guide-tutorial/docs/examples/testing

# Option 1: CMake (auto-downloads GTest)
mkdir build && cd build
cmake ..
make
ctest --output-on-failure

# Option 2: Make (requires GTest installed)
# sudo apt install libgtest-dev libgmock-dev
make
make test
```

Expected output:

```
[=====] Running 7 tests from 2 test suites.
[-----] 2 tests from SensorManagerBasicTest
[ RUN      ] SensorManagerBasicTest.InitiallyEmpty
[          OK ] SensorManagerBasicTest.InitiallyEmpty (0 ms)
...
[ PASSED   ] 7 tests.
```

Real OpenBMC Test Suites

After learning the basics, try running tests from actual OpenBMC repositories:

phosphor-logging (Event Logging)

```
git clone https://github.com/openbmc/phosphor-logging
cd phosphor-logging

# Install dependencies (on Ubuntu)
sudo apt install meson ninja-build pkg-config \
    libsdplusplus-dev libphosphor-dbus-interfaces-dev \
    libcereal-dev nlohmann-json3-dev

# Build with tests
meson setup builddir -Dtests=enabled
meson compile -C builddir

# Run tests
meson test -C builddir -v
```

dbus-sensors (Sensor Daemons)

```
git clone https://github.com/openbmc/dbus-sensors
cd dbus-sensors
meson setup builddir -Dtests=enabled
meson compile -C builddir
meson test -C builddir -v
```

bmcweb (Redfish Server)

```
git clone https://github.com/openbmc/bmcweb
cd bmcweb
meson setup builddir -Dtests=enabled
meson compile -C builddir
meson test -C builddir -v
```

If dependency installation is complex, use the OpenBMC SDK which includes all required libraries pre-installed.

Best Practices

Test Organization

- One test file per source file (`service.cpp` → `service_test.cpp`)
- Group related tests in test fixtures
- Use descriptive test names: `TEST(Component, WhatItDoes)`

Test Independence

- Each test should be independent
- Don't rely on test execution order
- Clean up resources in TearDown

Mock Appropriately

- Mock external dependencies (D-Bus, filesystem, network)
- Don't mock the code under test
- Verify mock expectations

Coverage Goals

- Aim for >80% line coverage
 - Test error paths, not just happy paths
 - Cover boundary conditions
-

Next Steps

- [Robot Framework Guide](#) - Integration testing
 - [Development Workflow](#) - devtool for rapid iteration
-

References

- [Google Test Documentation](#)
 - [Google Mock Documentation](#)
 - [phosphor-logging Tests](#)
 - [sdbusplus Test Utilities](#)
-

Tested on: OpenBMC master branch with Meson build system

Robot Framework Guide

Integration testing for OpenBMC using Robot Framework.

Table of Contents

1. [Overview](#)
 1. [Unit Tests vs Integration Tests](#)
2. [Environment Setup](#)
 1. [Install Robot Framework](#)
 2. [Clone openbmc-test-automation](#)
 3. [Environment Variables](#)
3. [Hello World Test](#)
 1. [Minimal Robot Test](#)
 2. [Running the Test](#)
 3. [Test Output](#)
4. [Robot Test File Structure](#)
 1. [File Organization](#)
 2. [Common Settings](#)
5. [openbmc-test-automation Structure](#)
 1. [Directory Layout](#)
 2. [Key Resource Files](#)
 3. [Using Existing Keywords](#)
6. [Test Execution](#)
 1. [Running Tests Against QEMU](#)
 2. [Running Tests Against Real Hardware](#)
 3. [Running Specific Test Suites](#)
 4. [Test Output Options](#)
 5. [Argument Files \(test_lists/\)](#)
7. [First Failure Data Capture \(FFDC\)](#)
 1. [Collection Plugins](#)
 2. [Wiring FFDC Into a Suite](#)
 3. [Plug-in Package Framework](#)
8. [GUI Test Automation](#)
 1. [Test Layout](#)
 2. [Locators](#)
 3. [Running GUI Tests](#)
9. [Common Test Patterns](#)
 1. [Redfish API Test](#)
 2. [IPMI Command Test](#)

3. SSH-Based Test

10. Example Test Files

11. Troubleshooting

1. Common Issues

2. Debug Tips

3. Checking BMC State

12. Next Steps

13. References

Overview

Robot Framework is the standard integration testing framework for OpenBMC. The [openbmc-test-automation](#) repository contains comprehensive tests for:

- Redfish API validation
- IPMI command testing
- Power state management
- Sensor and inventory verification
- Firmware update testing

This guide covers setting up Robot Framework and writing tests for OpenBMC.

Unit Tests vs Integration Tests

Aspect	Unit Tests (GTest)	Integration Tests (Robot)
Scope	Single function/class	Full system behavior
Dependencies	Mocked	Real services
Speed	Fast (ms)	Slower (seconds)
Environment	Build machine	Running BMC (QEMU/HW)
Purpose	Code correctness	System behavior

Environment Setup

Install Robot Framework

```
# Install Python 3 and pip
sudo apt install python3 python3-pip

# Install Robot Framework
pip3 install robotframework

# Install additional libraries
pip3 install robotframework-sshlibrary
pip3 install robotframework-requests
pip3 install robotframework-httplibrary

# Verify installation
robot --version
```

Clone openbmc-test-automation

```
# Clone the test repository
git clone https://github.com/openbmc/openbmc-test-automation.git
cd openbmc-test-automation

# Install Python dependencies
pip3 install -r requirements.txt
```

Environment Variables

Set up connection parameters:

```
# BMC connection (QEMU example)
export OPENBMC_HOST=localhost
export OPENBMC_SSH_PORT=2222
export OPENBMC_HTTPS_PORT=2443

# Credentials
export OPENBMC_USERNAME=root
export OPENBMC_PASSWORD=openBmc

# For real hardware
export OPENBMC_HOST=192.168.1.100
export OPENBMC_SSH_PORT=22
export OPENBMC_HTTPS_PORT=443
```

Hello World Test

Minimal Robot Test

Create `hello_openbmc.robot` :


```

*** Settings ***
Documentation      Hello World test for OpenBMC
Library            Collections
Library            RequestsLibrary

*** Variables ***
${OPENBMC_HOST}    localhost
${OPENBMC_PORT}    2443
${OPENBMC_USERNAME} root
${OPENBMC_PASSWORD} 0penBmc

*** Test Cases ***
Verify BMC Redfish Service Root
[Documentation]    Verify Redfish service is accessible
[Tags]            smoke

    # Create session
    Create Session    openbmc    https://${OPENBMC_HOST}:${OPENBMC_PORT}
    ...    verify=${False}

    # Get service root
    ${resp}=    GET On Session    openbmc    /redfish/v1
    ...    expected_status=200

    # Verify response
    Should Be Equal As Strings    ${resp.status_code}    200
    Dictionary Should Contain Key    ${resp.json()}    @odata.id

Verify BMC Manager Resource
[Documentation]    Verify BMC Manager is accessible
[Tags]            smoke

    Create Session    openbmc    https://${OPENBMC_HOST}:${OPENBMC_PORT}
    ...    auth=${OPENBMC_AUTH}    verify=${False}

    ${resp}=    GET On Session    openbmc    /redfish/v1/Managers/bmc
    ...    expected_status=200

    Should Be Equal As Strings    ${resp.json()}["Status"]["State"]}    Enabled

*** Keywords ***
${OPENBMC_AUTH}
[Return]    ${OPENBMC_USERNAME}    ${OPENBMC_PASSWORD}

```

Running the Test

```

# Start QEMU with OpenBMC (in another terminal)
qemu-system-arm -m 1G -M ast2600-evb -nographic \
    -drive file=obmc-phosphor-image-ast2600-evb.static.mtd,format=raw,if=mtd \
    -net nic -net user,hostfwd=tcp::2222-:22,hostfwd=tcp::2443-:443

# Run the test
robot hello_openbmc.robot

# Run with verbose output
robot -L DEBUG hello_openbmc.robot

```

Test Output

Robot generates three output files:

- `output.xml` - Machine-readable results
- `log.html` - Detailed execution log
- `report.html` - Summary report

```
# View report
xdg-open report.html
```

Robot Test File Structure

File Organization

```
*** Settings ***
Documentation      Description of test suite
Library            LibraryName
Resource           resource_file.robot
Suite Setup        Setup Keyword
Suite Teardown     Teardown Keyword

*** Variables ***
${VARIABLE}        value
@{LIST}            item1    item2
&{DICT}            key1=value1    key2=value2

*** Test Cases ***
Test Case Name
  [Documentation]    Test description
  [Tags]             tag1    tag2
  Keyword 1
  Keyword 2    ${argument}

*** Keywords ***
Custom Keyword
  [Arguments]    ${arg1}    ${arg2}
  Log    ${arg1}
  [Return]    ${result}
```

Common Settings

Setting	Purpose
<code>Library</code>	Import Python/Robot library
<code>Resource</code>	Import resource file with keywords
<code>Suite Setup</code>	Run before all tests
<code>Suite Teardown</code>	Run after all tests
<code>Test Setup</code>	Run before each test
<code>Test Teardown</code>	Run after each test

openbmc-test-automation Structure

Directory Layout

```
openbmc-test-automation/
├─ robot_framework/           # Test suites
│  └─ redfish/                # Redfish API tests
│     └─ ipmi/                # IPMI tests
│        └─ ssh/              # SSH-based tests
│           └─ gui/            # Web UI tests
├─ lib/                       # Keyword libraries
│  └─ bmc_redfish.py          # Redfish keywords
│     └─ ipmi_client.py       # IPMI keywords
│        └─ utils.py          # Utility functions
├─ data/                      # Test data
├─ templates/                 # Configuration templates
└─ requirements.txt            # Python dependencies
```

Key Resource Files

File	Purpose
<code>lib/resource.robot</code>	Common variables and keywords
<code>lib/bmc_redfish_resource.robot</code>	Redfish-specific keywords
<code>lib/ipmi_client.robot</code>	IPMI keywords
<code>lib/connection_client.robot</code>	SSH connection handling

Using Existing Keywords

```
*** Settings ***
Resource    ../lib/resource.robot
Resource    ../lib/bmc_redfish_resource.robot

*** Test Cases ***
Test Power On Host
    [Documentation]    Power on the host via Redfish
    [Tags]            power

    Redfish Power On
    Wait Until Keyword Succeeds    3 min    10 sec
    ...    Is Host Running
```

Test Execution

Running Tests Against QEMU

```
# Start QEMU (terminal 1)
qemu-system-arm -m 1G -M ast2600-evb -nographic \
  -drive file=openbmc-phosphor-image.static.mtd,format=raw,if=mtd \
  -net nic \
  -net user,hostfwd=tcp::2222-:22,hostfwd=tcp::2443-:443,hostfwd=udp::2623-:623

# Run tests (terminal 2)
cd openbmc-test-automation

robot -v OPENBMC_HOST:localhost \
  -v SSH_PORT:2222 \
  -v HTTPS_PORT:2443 \
  -v OPENBMC_USERNAME:root \
  -v OPENBMC_PASSWORD:OpenBmc \
  redfish/service_root/test_service_root.robot
```

Running Tests Against Real Hardware

```
robot -v OPENBMC_HOST:192.168.1.100 \
  -v SSH_PORT:22 \
  -v HTTPS_PORT:443 \
  -v OPENBMC_USERNAME:root \
  -v OPENBMC_PASSWORD:OpenBmc \
  redfish/service_root/test_service_root.robot
```

Running Specific Test Suites

```
# Run single test file
robot redfish/account_service/test_accounts.robot

# Run tests with specific tag
robot --include smoke redfish/

# Run tests excluding a tag
robot --exclude destructive redfish/

# Run specific test case
robot --test "Verify Service Root" redfish/service_root/
```

Test Output Options

```
# Custom output directory
robot -d results/ test_suite.robot

# Custom report name
robot -r custom_report.html test_suite.robot

# Log level
robot -L DEBUG test_suite.robot
robot -L TRACE test_suite.robot
```

Argument Files (test_lists/)

Passing long lists of suites, tags, and variables on the command line gets unwieldy, so the repository ships **Robot argument files** in the `test_lists/` directory. Each file is a curated set of `robot` arguments for one scenario:

Argument file	Purpose
<code>HW_CI</code>	Suites safe to run against real hardware in CI
<code>CT_basic_run</code>	A basic continuous-test smoke run
<code>QEMU_CI</code>	Suites suitable for QEMU-based CI
<code>skip_test_witherspoon</code> , <code>skip_test_palmetto</code>	Platform-specific skip lists

Point `robot` at one with `--argumentfile`:

```
robot --argumentfile test_lists/HW_CI \  
-v OPENBMC_HOST:192.168.1.100 \  
-v OPENBMC_USERNAME:root \  
-v OPENBMC_PASSWORD:0penBmc \  
redfish/
```

You can supply `--argumentfile` more than once – for example, combine a run list with a platform skip list to exclude tests that do not apply to your board.

First Failure Data Capture (FFDC)

When a test fails you usually need the BMC’s logs and state *from the moment of failure* to debug it. `openbmc-test-automation` automates this with **First Failure Data Capture (FFDC)**: on a failed test, a teardown keyword collects logs, journals, dumps, and inventory from the target and saves them alongside the Robot results.

Collection Plugins

FFDC data is gathered by protocol plugins in `ffdc/plugins/`, each of which knows how to pull data over one transport:

Plugin	Transport	Typical use
<code>ssh_execution.py</code>	SSH	Run commands on the BMC and capture stdout
<code>scp_execution.py</code>	SCP	Copy log files and dumps off the BMC
<code>telnet_execution.py</code>	Telnet	Collect data over serial/console-style access
<code>redfish.py</code>	Redfish	Walk and dump the Redfish resource tree

The `redfish.py` plugin uses `redfishtool` to recursively enumerate the service: `enumerate_request()` issues GET requests and `walk_nested_dict()` follows every `@odata.id` and `Members` link, producing a full JSON snapshot of the tree.

Wiring FFDC Into a Suite

FFDC runs from teardown keywords so it fires automatically on failure. The Robot glue lives in `lib/openbmc_ffdc.robot`, and `lib/ffdc_cli_robot_script.py` drives the plugin-based collector:

```
*** Settings ***
Resource      ../lib/openbmc_ffdc.robot
Test Teardown  FFDC On Test Case Fail

*** Test Cases ***
Power On Host
    [Documentation]  Power on and capture FFDC if the test fails.
    [Tags]          power
    Redfish Power On
    Wait Until Keyword Succeeds    3 min    10 sec    Is Host Running
```

If `Power On Host` fails, `FFDC On Test Case Fail` collects logs through the plugins above before the suite moves on.

Plug-in Package Framework

FFDC’s collectors are one use of a more general **plug-in package framework**. `bin/process_plug_in_packages.py` discovers and runs plug-in packages, invoking their *call-point* programs (scripts run at defined points in the test flow, such as setup, cleanup, or FFDC collection), and `bin/validate_plug_ins.py` checks that a package is well-formed. Reusable plug-in packages ship under `bin/plugin_packages/`. This lets you extend a run with custom setup, teardown, or data-collection logic without modifying the test suites themselves.

GUI Test Automation

Beyond API-level tests, `openbmc-test-automation` drives the OpenBMC web UI end-to-end in a real browser using **SeleniumLibrary**.

Test Layout

The current GUI suites target the Vue-based **webui-vue** front end and live under `gui/gui_test/`, organized by menu (for example `gui/gui_test/operations_menu/` and `gui/gui_test/settings_menu/`). Earlier AngularJS-era GUI tests (for the legacy `phosphor-webui`) have largely been retired as the web UI moved to Vue.

Locators

Rather than scattering XPathS through the tests, element locators are centralized under `gui/data/`:

File	Contents
<code>gui/data/gui_variables.py</code>	XPath and string constants for the Vue GUI
<code>gui/data/resource_variables.py</code>	Shared resource locators and identifiers

Tests prefer stable `data-test-id` attributes rendered by webui-vue, so locators survive cosmetic markup changes.

Running GUI Tests

GUI tests need a Selenium-driven browser and a matching WebDriver (for example ChromeDriver). Setup steps are documented in [docs/gui_setup_reference.md](#). A run looks like:

```
robot -v OPENBMC_HOST:${OPENBMC_HOST} \
      -v OPENBMC_USERNAME:root \
      -v OPENBMC_PASSWORD:0penBmc \
      gui/gui_test/
```

GUI tests are browser- and timing-sensitive, so they run slower and break more easily than Redfish or IPMI tests. Keep them out of fast smoke runs and gate them behind their own tags or argument file.

Common Test Patterns

Redfish API Test

```
*** Settings ***
Library           RequestsLibrary
Library           Collections

*** Variables ***
${REDFISH_BASE}   /redfish/v1

*** Test Cases ***
Get System Information Via Redfish
[Documentation]    Retrieve system info via Redfish
[Tags]            redfish    system

    # Authenticate
    ${auth}=       Create List    ${OPENBMC_USERNAME}    ${OPENBMC_PASSWORD}
    Create Session  bmc    https://${OPENBMC_HOST}:${HTTPS_PORT}
    ...    auth=${auth}    verify=${False}

    # Get system resource
    ${resp}=       GET On Session  bmc    ${REDFISH_BASE}/Systems/system
    Should Be Equal As Integers    ${resp.status_code}    200

    # Verify fields
    ${json}=       Set Variable    ${resp.json()}
    Should Be Equal    ${json["PowerState"]}    On
    Should Not Be Empty    ${json["SerialNumber"]}

Patch System Asset Tag
[Documentation]    Update system asset tag
[Tags]            redfish    system    patch

    ${auth}=       Create List    ${OPENBMC_USERNAME}    ${OPENBMC_PASSWORD}
    Create Session  bmc    https://${OPENBMC_HOST}:${HTTPS_PORT}
    ...    auth=${auth}    verify=${False}

    # Update asset tag
    ${payload}=     Create Dictionary    AssetTag=TestAsset123
    ${headers}=     Create Dictionary    Content-Type=application/json

    ${resp}=       PATCH On Session  bmc    ${REDFISH_BASE}/Systems/system
    ...    json=${payload}    headers=${headers}

    Should Be Equal As Integers    ${resp.status_code}    200
```

Under the hood: `bmc_redfish` and `mTLS`. The higher-level Redfish keywords in the suite are backed by `lib/bmc_redfish.py`, whose `bmc_redfish` class extends `redfish_plus` (a thin wrapper over the DMTF `redfish` Python library). Its `enumerate()` keyword recursively walks the service (via `walk_nested_dict()`) to snapshot the whole tree. For mutual-TLS testing, set `MTLS_ENABLED=True` so the client authenticates with a certificate instead of a session token, together with `VALID_CERT` and `CERT_DIR_PATH` (defined in `lib/redfish_plus.py` and `lib/resource.robot`). See `docs/redfish_request_via_mTLS.md` for the full mTLS setup.

IPMI Command Test

```
*** Settings ***
Library           OperatingSystem
Library           Process

*** Variables ***
${IPMI_CMD}       ipmitool -I lanplus -H ${OPENBMC_HOST} -p 623 -U ${OPENBMC_USERNAME} -P ${OPENBMC_PASSWORD}

*** Test Cases ***

Get BMC Info Via IPMI
[Documentation]    Get BMC device info via IPMI
[Tags]            ipmi

    ${result}=    Run Process    ${IPMI_CMD} mc info    shell=True
    Should Be Equal As Integers    ${result.rc}    0
    Should Contain    ${result.stdout}    Firmware Revision

Get Sensor List Via IPMI
[Documentation]    List sensors via IPMI
[Tags]            ipmi    sensors

    ${result}=    Run Process    ${IPMI_CMD} sensor list    shell=True
    Should Be Equal As Integers    ${result.rc}    0
    Should Not Be Empty    ${result.stdout}

Chassis Power Status
[Documentation]    Check chassis power status
[Tags]            ipmi    power

    ${result}=    Run Process    ${IPMI_CMD} chassis power status    shell=True
    Should Be Equal As Integers    ${result.rc}    0
    Should Match Regexp    ${result.stdout}    Chassis Power is (on|off)
```

SSH-Based Test

```
*** Settings ***
Library          SSHLibrary

*** Test Cases ***
Verify BMC Version Via SSH
    [Documentation]    Check BMC version via SSH
    [Tags]            ssh

    # Connect to BMC
    Open Connection    ${OPENBMC_HOST}    port=${SSH_PORT}
    Login              ${OPENBMC_USERNAME}    ${OPENBMC_PASSWORD}

    # Get version info
    ${output}=         Execute Command    cat /etc/os-release
    Should Contain     ${output}    openbmc-phosphor

    # Check service status
    ${output}=         Execute Command    systemctl is-active bmcweb
    Should Be Equal    ${output.strip()}    active

    [Teardown]        Close Connection

Execute obmcutil Command
    [Documentation]    Run obmcutil via SSH
    [Tags]            ssh    power

    Open Connection    ${OPENBMC_HOST}    port=${SSH_PORT}
    Login              ${OPENBMC_USERNAME}    ${OPENBMC_PASSWORD}

    ${output}=         Execute Command    obmcutil state
    Should Contain     ${output}    BMCState

    [Teardown]        Close Connection
```

Example Test Files

Working examples are available in the [examples/robot](#) directory:

- [hello_openbmc.robot](#) - Basic connectivity and Redfish authentication tests
 - [redfish_sensors.robot](#) - Sensor reading and validation tests
 - [ssh_commands.robot](#) - SSH command execution tests
-

Troubleshooting

Common Issues

Issue	Cause	Solution
Connection refused	BMC not running	Start QEMU or check network
SSL certificate error	Self-signed cert	Use <code>verify=\${False}</code>
Authentication failed	Wrong credentials	Verify username/password
Timeout waiting for host	Slow boot	Increase timeout values
IPMI connection failed	Port not forwarded	Check QEMU port forwarding (UDP 623)

Debug Tips

```
# Run with debug logging
robot -L DEBUG test.robot

# Trace library calls
robot -L TRACE test.robot

# Dry run (syntax check)
robot --dryrun test.robot

# List tests without running
robot --list test.robot
```

Checking BMC State

```
# SSH to BMC
ssh -p 2222 root@localhost

# Check service status
systemctl status bmcweb
systemctl status phosphor-ipmi-host

# View logs
journalctl -u bmcweb -f
```

Next Steps

- [Unit Testing Guide](#) - GTest/GMock for unit tests
- [Verification Guide](#) - Complete testing checklist
- [Redfish Guide](#) - Redfish API details

References

- [Robot Framework Documentation](#)

- [openbmc-test-automation Repository](#)
 - [RequestsLibrary Documentation](#)
 - [SSHLibrary Documentation](#)
-

Tested on: Robot Framework 6.x with OpenBMC on QEMU ast2600-evb

Linux Kernel Driver Development

Develop, debug, and integrate Linux kernel drivers for OpenBMC — from kernel patching and out-of-tree modules to userspace alternatives and end-to-end sensor binding.

Table of Contents

1. [Overview](#)
2. [How OpenBMC Structures Kernel Recipes](#)
 1. [Layer Hierarchy](#)
 2. [Key Files in meta-aspeed](#)
 3. [The Base Kernel Recipe](#)
3. [Workflow 1: devtool \(Interactive Development\)](#)
 1. [Step-by-Step](#)
 2. [Interactive Kernel Configuration](#)
4. [Workflow 2: bbappend with Patch Files \(Permanent Integration\)](#)
 1. [Creating Patch Files](#)
 2. [Directory Structure](#)
 3. [Writing the bbappend](#)
 4. [Real-World Example: Vesnin Platform](#)
 5. [Patch Application Order](#)
5. [Kernel Configuration Fragments](#)
 1. [Creating Fragments](#)
 2. [Applying Fragments](#)
 3. [Validating Configuration](#)
6. [Device Tree Patching](#)
 1. [DTS Naming Convention](#)
 2. [Creating a New Device Tree](#)
 3. [Adding DTS to the Build](#)
 4. [Patching an Existing Device Tree](#)
7. [Testing Kernel Changes](#)
 1. [QEMU Testing](#)
 2. [TFTP Netboot \(Without Reflashing\)](#)
 3. [Out-of-Tree Kernel Build](#)
8. [Verifying Your Patches](#)
 1. [Check Kernel Version and Patches](#)
 2. [Verify Driver is Loaded](#)
 3. [Verify Kernel Config](#)
 4. [BitBake Validation](#)
9. [Common Patch Scenarios](#)
 1. [Scenario 1: Adding a New hwmon Driver](#)

2. [Scenario 2: Fixing an Existing Driver Bug](#)
 3. [Scenario 3: Enabling a Kernel Subsystem](#)
10. [Out-of-Tree Kernel Modules](#)
 1. [When to Use Out-of-Tree vs In-Tree](#)
 2. [Recipe Layout in Your Meta-Layer](#)
 3. [Kbuild Makefile](#)
 4. [Module Source \(Minimal Example\)](#)
 5. [Yocto Recipe Using module.bbclass](#)
 6. [Module Autoloading](#)
 7. [Building and Testing](#)
11. [Userspace Driver Alternatives](#)
 1. [Decision Framework: Kernel Driver vs Userspace](#)
 2. [libgpod — GPIO Access](#)
 3. [i2c-dev / i2c-tools — Direct I2C Access](#)
 4. [spidev — SPI Userspace Access](#)
 5. [UIO — Userspace I/O for Register-Mapped Devices](#)
 6. [/dev/mem and devmem2 — Direct Physical Memory Access](#)
12. [Driver Debugging](#)
 1. [dynamic_debug — Runtime Debug Prints](#)
 2. [ftrace — Probe Sequence Tracing](#)
 3. [debugfs Interfaces](#)
 4. [dmesg Patterns for Common Probe Failures](#)
13. [I2C/SPI Sensor Binding Walkthrough](#)
 1. [Step 1: Device Tree Node](#)
 2. [Step 2: Kernel Driver Binding](#)
 3. [Step 3: hwmon sysfs Verification](#)
 4. [Step 4: Entity-Manager JSON Configuration](#)
 5. [Step 5: dbus-sensors Pickup and D-Bus Exposure](#)
 6. [Step 6: Redfish Sensor Visibility](#)
 7. [Cross-References](#)
14. [Upstreaming Best Practices](#)
 1. [OpenBMC Upstream-First Policy](#)
 2. [Commit Message Format](#)
 3. [Submission to Upstream Linux](#)
 4. [Submission to OpenBMC Kernel Tree](#)
15. [Troubleshooting](#)
 1. [Issue: Patch Fails to Apply](#)
 2. [Issue: Config Option Not Taking Effect](#)
 3. [Issue: devtool modify Uses Wrong defconfig](#)
 4. [Issue: Kernel Boots But Driver Not Found](#)
16. [Quick Reference](#)

1. [Essential Commands](#)
2. [Patch Workflow Summary](#)

17. [References](#)

Overview

When developing or modifying drivers for OpenBMC, you often need to patch the Linux kernel image itself. Unlike userspace services that live in separate recipes, kernel changes — new drivers, bug fixes, device tree modifications, or configuration changes — require a structured patching workflow through Yocto/BitBake.

OpenBMC maintains its own kernel tree at [openbmc/linux](#) with a development branch naming convention of `dev-X.Y` (e.g., `dev-6.18`). The kernel recipe pins to a specific commit hash for reproducibility, and your patches are applied on top during the build.

Key concepts covered:

- How OpenBMC organizes kernel recipes across meta-layers
- Two workflows for kernel patching: `devtool` (interactive) and `bbappend` (permanent)
- Creating and applying kernel configuration fragments
- Device tree patching for new hardware
- Testing kernel changes on QEMU before flashing hardware
- Building out-of-tree kernel modules as standalone Yocto recipes
- Userspace driver alternatives (libgpiod, i2c-dev, spidev, UIO)
- Driver debugging techniques (dynamic_debug, ftrace, debugfs)
- End-to-end I2C/SPI sensor binding walkthrough
- Best practices for upstreaming patches

Kernel patches directly affect system stability. Always test on QEMU before deploying to hardware. A bad kernel patch can brick a BMC, requiring physical recovery.

How OpenBMC Structures Kernel Recipes

Understanding the layer hierarchy helps you know where to place your patches.

Layer Hierarchy

Platform/Vendor Layer (meta-ibm, meta-google, etc.) — Machine-specific patches + config fragments
Distribution Layer (meta-phosphor) — Phosphor-specific kernel configs (GPIO, etc.)
BSP Layer (meta-aspeed, meta-nuvoton) — Base kernel recipe, SoC defconfig, kernel source

Each layer adds its own `.bbappend` files and config fragments on top of the base recipe. The build system merges them in layer priority order.

Key Files in meta-aspeed

File	Purpose
<code>recipes-kernel/linux/linux-aspeed_git.bb</code>	Pins kernel branch, version, and commit hash
<code>recipes-kernel/linux/linux-aspeed.inc</code>	SRC_URI, license, build settings, conditional features
<code>recipes-kernel/linux/linux-aspeed/aspeed-g4/defconfig</code>	AST2400 (ARMv5) base kernel config
<code>recipes-kernel/linux/linux-aspeed/aspeed-g5/defconfig</code>	AST2500 (ARMv6) base kernel config
<code>recipes-kernel/linux/linux-aspeed/aspeed-g6/defconfig</code>	AST2600 (ARMv7) base kernel config

The Base Kernel Recipe

The `linux-aspeed_git.bb` recipe is minimal — it pins the source and delegates to the include:

```
KBRANCH ?= "dev-6.18"
LINUX_VERSION ?= "6.18.8"
SRCREV = "3fcb5927beef6fc8d9faa16b90c01fc6e67f2065"

require linux-aspeed.inc
```

The include file (`linux-aspeed.inc`) sets up the source fetch, defconfig selection, and conditional features:

```
KSRC ?= "git://github.com/openbmc/linux;protocol=https;branch=${KBRANCH}"
SRC_URI = "${KSRC}"
SRC_URI += "file://defconfig"

# Conditional: TPM2 support
SRC_URI += "${@bb.utils.contains('MACHINE_FEATURES', 'tpm2', 'file://tpm2.cfg', '', d)}"

# Conditional: UBI filesystem
SRC_URI += "${@bb.utils.contains('DISTRO_FEATURES', 'ubi', 'file://ubi.cfg', '', d)}"

KCONFIG_MODE = "--alldefconfig"
inherit kernel linux-yocto
```

Workflow 1: devtool (Interactive Development)

Use `devtool` when you are actively developing a kernel change — editing code, testing iteratively, and refining before committing. This is the recommended workflow for driver development.

Step-by-Step

```
# 1. Source the OpenBMC build environment for your machine
cd openbmc
. setup romulus

# 2. Check out the kernel source into a workspace
devtool modify linux-aspeed
```

This clones the kernel into `build/workspace/sources/linux-aspeed/` and sets up BitBake to build from your local checkout instead of the cached source.

```
# 3. Navigate to the kernel source
cd build/workspace/sources/linux-aspeed

# 4. Make your driver changes
vim drivers/hwmon/my_sensor.c
vim drivers/hwmon/Makefile
vim drivers/hwmon/Kconfig

# 5. Commit your changes (REQUIRED – uncommitted changes are ignored!)
git add drivers/hwmon/my_sensor.c drivers/hwmon/Makefile drivers/hwmon/Kconfig
git commit -s -m "hwmon: my_sensor: Add support for XYZ sensor chip"
```

You **must** commit your changes to git. The `devtool` workflow ignores uncommitted modifications in the working tree.

```
# 6. Build just the kernel to test compilation
devtool build linux-aspeed

# 7. Or build the full image
bitbake obmc-phosphor-image

# 8. When satisfied, export patches to your meta-layer
devtool finish linux-aspeed ~/openbmc/meta-mymachine

# OR create just a bbappend without removing the workspace
devtool update-recipe -a ~/openbmc/meta-mymachine linux-aspeed

# 9. Clean up the workspace when done
devtool reset linux-aspeed
```

Interactive Kernel Configuration

Use `menuconfig` to explore and change kernel options interactively:

```
# Open the ncurses-based config editor
bitbake linux-aspeed -c menuconfig

# After saving, generate a defconfig
bitbake linux-aspeed -c savedefconfig
```

The `devtool modify` command checks out the branch specified in the recipe (e.g., `dev-6.18`). Unlike some recipes, it does **not** create a separate `devtool` branch — patches are applied directly on the checked-out branch.

Workflow 2: bbappend with Patch Files (Permanent Integration)

Use this approach when you have a finalized patch ready to integrate into your machine layer. This is the standard method for production systems.

Creating Patch Files

Generate patches from your kernel git tree using `git format-patch` :

```
# Single patch from the last commit
git format-patch -1

# Multiple patches from the last 3 commits
git format-patch -3

# Patches since branching from a base
git format-patch origin/dev-6.18..HEAD
```

This produces numbered patch files like `0001-hwmon-my_sensor-Add-support-for-XYZ.patch` .

Directory Structure

Place patches in your meta-layer following this convention:

```
meta-vendor/
meta-platform/
  recipes-kernel/
    linux/
      linux-aspeed_%.bbappend      # The append file
      linux-aspeed/                # Directory matching recipe name
      my-platform.cfg              # Config fragment
      0001-hwmon-add-xyz-driver.patch # Kernel source patch
      0002-dts-add-platform.patch  # Device tree patch
```

Writing the bbappend

Create `linux-aspeed_%.bbappend` (the `%` wildcard matches any version):

```
FILESEXTRAPATHS:prepend := "${THISDIR}/${PN}:"

SRC_URI += " \
    file://0001-hwmon-add-xyz-driver.patch \
    file://0002-dts-add-platform.patch \
    file://my-platform.cfg \
"
```

The `FILESEXTRAPATHS:prepend` line tells BitBake to search the `linux-aspeed/` subdirectory (matching `${PN}`) alongside the append file for the referenced files.

Real-World Example: Vesnin Platform

From `meta-yadro/meta-vesnin` :

```
FILESEXTRAPATHS:prepend := "${THISDIR}/${PN}:"
SRC_URI:append:vesnin = " \
    file://vesnin.cfg \
    file://0002-vesnin-remap-aspeed-uart.patch \
"
```

This adds a UART remapping patch and a platform-specific config fragment, but only for the `vesnin` machine.

Patch Application Order

Patches listed in `SRC_URI` with `file://` are **not guaranteed** to apply in the order listed. To enforce ordering, use a `.scc` (Series Configuration Control) file:

```
# my-patches.scc
patch 0001-first-fix.patch
patch 0002-second-fix.patch
patch 0003-third-fix.patch
```

Then reference the `.scc` file in `SRC_URI` instead of individual patches.

Kernel Configuration Fragments

Rather than maintaining a complete defconfig (thousands of lines), use **configuration fragments** — small `.cfg` files containing only the options you need to change.

Creating Fragments

Method 1: diffconfig (recommended)

```
# Start from a clean config
bitbake linux-aspeed -c kernel_configme -f

# Make changes interactively
bitbake linux-aspeed -c menuconfig

# Generate a fragment of just your changes
bitbake linux-aspeed -c diffconfig
# Output: fragment.cfg in the kernel work directory
```

Method 2: Manual creation

```
cat > my-driver.cfg << 'EOF'
CONFIG_I2C_SLAVE=y
CONFIG_SENSORS_LM75=y
CONFIG_SENSORS_TMP421=m
# CONFIG_SENSORS_FAKE is not set
EOF
```

Applying Fragments

Add the fragment via your `.bbappend`:

```
FILESEXTRAPATHS:prepend := "${THISDIR}/${PN}:"
SRC_URI += "file://my-driver.cfg"
```

Validating Configuration

Verify that all fragment options were applied correctly:

```
bitbake linux-aspeed -c kernel_configcheck -f
```

This produces warnings when a requested option does not appear in the final `.config`, helping you catch silent configuration failures.

Start `.cfg` files with a blank line — some Yocto versions silently skip the first line. The `KCONFIG_MODE = "--alldefconfig"` setting in `linux-aspeed.inc` means any option not explicitly set gets its default value.

Device Tree Patching

Device trees describe hardware to the kernel. For a new platform or hardware modification, you often need to add or patch a device tree source (DTS) file.

DTS Naming Convention

OpenBMC follows the upstream Linux convention:

```
aspeed-bmc-<vendor>-<platform>.dts
```

Examples: `aspeed-bmc-opp-romulus.dts`, `aspeed-bmc-ibm-rainier.dts`, `aspeed-bmc-amd-onyx.dts`

All DTS files live in `arch/arm/boot/dts/aspeed/` in the kernel tree and include base SoC definitions:

SoC	Include	Generation
AST2400	<code>aspeed-g4.dtsi</code>	G4 (ARMv5)
AST2500	<code>aspeed-g5.dtsi</code>	G5 (ARMv6)
AST2600	<code>aspeed-g6.dtsi</code>	G6 (ARMv7)

Creating a New Device Tree

```
// SPDX-License-Identifier: GPL-2.0-or-later
// aspeed-bmc-vendor-platform.dts
/dts-v1/;

#include "aspeed-g6.dtsi"
#include <dt-bindings/gpio/aspeed-gpio.h>
#include <dt-bindings/leds/common.h>

/ {
    model = "Vendor Platform BMC";
    compatible = "vendor,platform-bmc", "aspeed,ast2600";

    chosen {
        stdout-path = &uart5;
        bootargs = "console=ttyS4,115200n8 root=/dev/ram rw";
    };

    leds {
        compatible = "gpio-leds";
        fault {
            gpios = <&gpio ASPEED_GPIO(N, 2) GPIO_ACTIVE_LOW>;
        };
    };
};

&i2c0 {
    status = "okay";

    temperature-sensor@48 {
        compatible = "ti,tmp175";
        reg = <0x48>;
    };
};

&uart5 {
    status = "okay";
};
```

Adding DTS to the Build

Two files need changes:

1. Add to the kernel Makefile ([arch/arm/boot/dts/aspeed/Makefile](#)):

```
dtb-$(CONFIG_ARCH_ASPEED) += aspeed-bmc-vendor-platform.dtb
```

2. Set in your machine configuration:

```
KERNEL_DEVICETREE = "aspeed-bmc-vendor-platform.dtb"
```

Patching an Existing Device Tree

Generate a patch, then add it via [.bbappend](#):

```
# In the kernel tree
git add arch/arm/boot/dts/aspeed/aspeed-bmc-vendor-platform.dts
git add arch/arm/boot/dts/aspeed/Makefile
git commit -s -m "ARM: dts: aspeed: Add Vendor Platform BMC device tree"
git format-patch -1
```

```
# linux-aspeed_%.bbappend
FILESEXTRAPATHS:prepend := "${THISDIR}/${PN}:"
SRC_URI += "file://0001-ARM-dts-aspeed-Add-Vendor-Platform-BMC-device-tree.patch"
```

Device tree source files are an **exception** to the upstream-first rule in OpenBMC. You can carry DTS patches in your meta-layer without waiting for upstream acceptance. However, you should still aim to upstream them eventually.

Testing Kernel Changes

QEMU Testing

Always validate kernel patches on QEMU before deploying to hardware:

```
# Build the full image with your kernel changes
bitbake obmc-phosphor-image

# Boot with QEMU
qemu-system-arm -M ast2600-evb \
  -drive file=tmp/deploy/images/ast2600-evb/flash-ast2600-evb,format=raw,if=mtd \
  -nographic
```

Or boot kernel and DTB separately:

```
qemu-system-arm -M ast2600-evb \
  -kernel tmp/deploy/images/ast2600-evb/zImage \
  -dtb tmp/deploy/images/ast2600-evb/aspeed-bmc-vendor-platform.dtb \
  -initrd tmp/deploy/images/ast2600-evb/obmc-phosphor-initramfs-ast2600-evb.cpio.gz \
  -nographic
```

TFTP Netboot (Without Reflashing)

For faster iteration, serve the kernel image over TFTP instead of reflashing:

```
# Copy FIT image to TFTP server
cp tmp/deploy/images/<machine>/fitImage /tftpboot/

# In U-Boot console on the BMC:
setenv ipaddr 192.168.0.80
setenv serverip 192.168.0.11
tftp 0x83000000 fitImage
bootm 0x83000000
```

Use `0x43000000` for AST2400 boards. Use `0x83000000` for AST2500/AST2600. This address is the kernel load address in DRAM.

Out-of-Tree Kernel Build

For the fastest compile-test cycle, build the kernel outside of BitBake entirely:

```
# Configure for AST2600
make ARCH=arm O=obj CROSS_COMPILE=arm-linux-gnueabihf- aspeed_g6_defconfig

# Build
make ARCH=arm O=obj CROSS_COMPILE=arm-linux-gnueabihf- -j$(nproc)

# Build with initramfs baked in
make ARCH=arm O=obj CROSS_COMPILE=arm-linux-gnueabihf- \
  CONFIG_INITRAMFS_SOURCE=/path/to/obmc-phosphor-image-<machine>.cpio.gz
```

The cross-compiler (`arm-linux-gnueabihf-`) is available from your Yocto SDK or your distribution's `gcc-arm-linux-gnueabihf` package.

Verifying Your Patches

After building, verify that your changes took effect.

Check Kernel Version and Patches

```
# On the running BMC
uname -a
cat /proc/version
```

Verify Driver is Loaded

```
# Check if your driver module is loaded
lsmod | grep my_driver

# Check kernel ring buffer for driver messages
dmesg | grep my_driver

# Check device tree nodes
ls /proc/device-tree/
cat /proc/device-tree/compatible
```

Verify Kernel Config

```
# If /proc/config.gz is enabled
zcat /proc/config.gz | grep CONFIG_MY_OPTION
```

BitBake Validation

```
# Verify config fragments were applied
bitbake linux-aspeed -c kernel_configcheck -f

# Verify patch application succeeded
bitbake linux-aspeed -c patch
```

Common Patch Scenarios

Scenario 1: Adding a New hwmon Driver

A new temperature sensor on the I2C bus needs a kernel driver.

1. Add driver source to `drivers/hwmon/` and update `Kconfig` / `Makefile`
2. Add the device node to your platform DTS
3. Create a config fragment enabling the driver
4. Package as patches in your meta-layer

```
meta-vendor/recipes-kernel/linux/  
linux-aspeed_%.bbappend  
linux-aspeed/  
    0001-hwmon-add-xyz-sensor-driver.patch    # Driver code  
    0002-dts-add-xyz-sensor-to-platform.patch  # DTS node  
    enable-xyz-sensor.cfg                     # CONFIG_SENSORS_XYZ=y
```

Scenario 2: Fixing an Existing Driver Bug

A known upstream fix hasn't been backported to the OpenBMC kernel branch yet.

```
# Cherry-pick from upstream Linux  
cd build/workspace/sources/linux-aspeed  
git cherry-pick <upstream-commit-hash>  
  
# If it doesn't apply cleanly, resolve conflicts and commit  
git add -u  
git commit -s  
  
# Export the patch  
git format-patch -1
```

Scenario 3: Enabling a Kernel Subsystem

You need USB gadget support that is disabled by default.

```
# Create a config fragment  
cat > usb-gadget.cfg << 'EOF'  
  
CONFIG_USB_GADGET=y  
CONFIG_USB_CONFIGFS=y  
CONFIG_USB_CONFIGFS_MASS_STORAGE=y  
EOF
```

No kernel source patches needed — just the `.cfg` fragment and a `.bbappend`.

Out-of-Tree Kernel Modules

When your driver does not belong in the upstream kernel tree — vendor-specific hardware, proprietary logic, or rapid prototyping — build it as an **out-of-tree kernel module**. This keeps your driver in its own Yocto recipe, decoupled from the kernel source and build cycle.

When to Use Out-of-Tree vs In-Tree

Factor	Out-of-Tree Module	In-Tree Patch
Upstream acceptance	Not planned or not possible	Planned or already accepted
Build coupling	Independent recipe, faster rebuilds	Tied to full kernel rebuild
Development speed	Fast iteration — rebuild only your module	Slower — full kernel compile on change
Kernel API stability	Must track API changes across kernel versions	Automatically consistent
Deployment	Can be updated independently of kernel	Requires full firmware image update

For most BMC production drivers, prefer in-tree patches (upstream-first policy). Use out-of-tree modules for vendor-specific hardware abstraction layers, debug/test modules, or drivers still in active prototyping.

Recipe Layout in Your Meta-Layer

```
meta-vendor/  
  meta-platform/  
    recipes-kernel/  
      my-hwmon-module/  
        my-hwmon-module_1.0.bb      # Yocto recipe  
        files/  
          Makefile                  # Kbuild Makefile  
          my_hwmon_driver.c         # Module source  
          my-hwmon-module.conf      # Autoload config
```

Kbuild Makefile

The out-of-tree module Makefile uses standard Kbuild syntax:

```
# files/Makefile  
obj-m := my_hwmon_driver.o
```

For multi-file modules:

```
obj-m := my_hwmon_driver.o  
my_hwmon_driver-objs := main.o i2c_ops.o sysfs_attrs.o
```

Module Source (Minimal Example)

```
// files/my_hwmon_driver.c
#include <linux/module.h>
#include <linux/i2c.h>
#include <linux/hwmon.h>

static int my_hwmon_probe(struct i2c_client *client)
{
    dev_info(&client->dev, "my_hwmon_driver probed at 0x%02x\n", client->addr);
    return 0;
}

static const struct of_device_id my_hwmon_of_match[] = {
    { .compatible = "vendor,my-sensor" },
    { }
};
MODULE_DEVICE_TABLE(of, my_hwmon_of_match);

static struct i2c_driver my_hwmon_driver = {
    .driver = {
        .name = "my-hwmon-driver",
        .of_match_table = my_hwmon_of_match,
    },
    .probe = my_hwmon_probe,
};
module_i2c_driver(my_hwmon_driver);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Example out-of-tree hwmon driver for BMC");
```

Yocto Recipe Using module.bbclass

```
# my-hwmon-module_1.0.bb
SUMMARY = "Out-of-tree hwmon driver for vendor sensor"
LICENSE = "GPL-2.0-only"
LIC_FILES_CHKSUM = "file://my_hwmon_driver.c;beginline=1;endline=1;md5=..."

SRC_URI = " \
    file://Makefile \
    file://my_hwmon_driver.c \
    file://my-hwmon-module.conf \
"

S = "${WORKDIR}"

inherit module

# Autoload at boot
KERNEL_MODULE_AUTOLOAD += "my_hwmon_driver"

# Install autoload config
do_install:append() {
    install -d ${D}${sysconfdir}/modules-load.d
    install -m 0644 ${WORKDIR}/my-hwmon-module.conf \
        ${D}${sysconfdir}/modules-load.d/
}
```

The `inherit module` line pulls in `module.bbclass`, which handles cross-compilation against the kernel headers, module signing (if enabled), and packaging into the correct `/lib/modules/<version>/` directory.

Module Autoloading

Create a `.conf` file for `/etc/modules-load.d/`:

```
# files/my-hwmon-module.conf
# Load vendor hwmon driver at boot
my_hwmon_driver
```

Alternatively, use `KERNEL_MODULE_AUTOLOAD` in the recipe (shown above) which writes the appropriate config automatically.

Building and Testing

```
# Build just the module recipe
bitbake my-hwmon-module

# Build the full image (includes the module)
bitbake obmc-phosphor-image

# On the running BMC, verify the module
lsmod | grep my_hwmon
modinfo my_hwmon_driver
```

Userspace Driver Alternatives

Not every hardware interaction requires a kernel driver. For simple register reads, GPIO toggling, or I2C device testing, userspace access is faster to develop and easier to debug. This section helps you decide when a userspace approach is sufficient.

Decision Framework: Kernel Driver vs Userspace

Criterion	Kernel Driver	Userspace Access
Interrupt handling	Required — only kernel can register IRQ handlers	Not possible (polling only)
Latency requirements	Microsecond-level response	Millisecond-level acceptable
Shared device access	Kernel manages arbitration	Single process at a time
DMA support	Full DMA engine access	Not available
Upstream acceptance	Required for OpenBMC mainline	Not applicable — stays in your layer
Development speed	Slower (compile, deploy, reboot)	Fast (edit, run)
Debugging	Kernel oops, printk, ftrace	gdb, printf, strace
hwmon/D-Bus integration	Automatic via hwmon subsystem	Manual — must write your own bridge

If your device needs to appear as a standard hwmon sensor on D-Bus (for `dbus-sensors/entity-manager` integration), you almost always need a kernel driver. Userspace approaches are best for prototyping, diagnostics, and one-off access.

libgpiod — GPIO Access

libgpiod v2 is the recommended way to access GPIO lines from userspace. It replaces the deprecated sysfs GPIO interface (</sys/class/gpio/>).

Command-line tools:

```
# List all GPIO chips
gpiodetect

# Show lines on a specific chip
gpioinfo gpiochip0

# Read a GPIO line value
gpioget gpiochip0 42

# Set a GPIO line (active-high)
gpioset gpiochip0 42=1

# Monitor a line for events (rising/falling edge)
gpiomon gpiochip0 42
```

C API snippet:

```
#include <gpiod.h>

struct gpiod_chip *chip = gpiod_chip_open("/dev/gpiochip0");
struct gpiod_line_settings *settings = gpiod_line_settings_new();
gpiod_line_settings_set_direction(settings, GPIOD_LINE_DIRECTION_INPUT);

struct gpiod_line_config *config = gpiod_line_config_new();
static const unsigned int offset = 42;
gpiod_line_config_add_line_settings(config, &offset, 1, settings);

struct gpiod_line_request *request =
    gpiod_chip_request_lines(chip, NULL, config);

enum gpiod_line_value value =
    gpiod_line_request_get_value(request, offset);

gpiod_line_request_release(request);
gpiod_line_config_free(config);
gpiod_line_settings_free(settings);
gpiod_chip_close(chip);
```

OpenBMC uses libgpiod extensively in [phosphor-gpio-monitor](#) and [phosphor-buttons](#) . See the [GPIO Management Guide](#) for the full OpenBMC GPIO architecture.

i2c-dev / i2c-tools — Direct I2C Access

The [i2c-dev](#) kernel module exposes I2C buses as [/dev/i2c-N](#) character devices.

```
# List I2C buses
i2cdetect -l

# Scan for devices on bus 0
i2cdetect -y 0

# Read a register (bus 0, device 0x48, register 0x00)
i2cget -y 0 0x48 0x00 w

# Write a register
i2cset -y 0 0x48 0x01 0x60 w

# Dump all registers
i2cdump -y 0 0x48
```

Using `i2c-tools` on a bus where a kernel driver is already bound can cause bus contention and unpredictable behavior. Use `i2cdetect` to check if addresses show `UU` (already claimed by a driver).

spidev — SPI Userspace Access

The `spidev` driver exposes SPI devices as `/dev/spidevB.C` (bus B, chip-select C).

Device tree binding:

```
&spi1 {
    status = "okay";

    spidev@0 {
        compatible = "linux,spidev";
        reg = <0>;
        spi-max-frequency = <10000000>;
    };
};
```

```
# Test SPI communication (requires spi-tools package)
spi-config -d /dev/spidev1.0 -q
spi-pipe -d /dev/spidev1.0 -s 1000000 < data.bin > response.bin
```

UIO — Userspace I/O for Register-Mapped Devices

UIO maps device registers directly into userspace memory, allowing you to write a full driver without kernel code. Useful for FPGA registers or vendor-specific control blocks.

Device tree binding:

```
my-device@1e6e0000 {
    compatible = "generic-uio";
    reg = <0x1e6e0000 0x1000>;
};
```

Kernel config fragment:

```
CONFIG_UIO=y
CONFIG_UIO_PDRV_GENIRQ=y
```

Userspace access:

```
#include <sys/mman.h>
#include <fcntl.h>

int fd = open("/dev/uio0", O_RDWR);
void *regs = mmap(NULL, 0x1000, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

uint32_t val = ((volatile uint32_t *)regs)[0]; // Read register 0x00
((volatile uint32_t *)regs)[1] = 0xDEADBEEF; // Write register 0x04

munmap(regs, 0x1000);
close(fd);
```

/dev/mem and devmem2 — Direct Physical Memory Access

For quick register checks during debugging, `devmem2` reads and writes physical addresses directly.

```
# Read SCU register on AST2600 (Silicon Revision)
devmem2 0x1e6e2004 w

# Write a register
devmem2 0x1e6e2000 w 0x1688A8A8
```

`/dev/mem` access bypasses all kernel protections. A wrong write can corrupt memory, hang the SoC, or brick the BMC. Use only for debugging on development systems. Production firmware should disable `/dev/mem` access (`CONFIG_STRICT_DEVMEM=y`).

Driver Debugging

Debugging kernel drivers on a BMC is constrained: limited flash storage rules out large trace buffers, there is typically no JTAG/kgdb access, and the BMC may be the only management interface to the system. These techniques work within those constraints.

dynamic_debug — Runtime Debug Prints

The `dynamic_debug` facility lets you enable `dev_dbg()` and `pr_debug()` messages at runtime without rebuilding the kernel. This is the single most useful driver debugging tool on a BMC.

Enable for a specific module:

```
# Enable all debug prints in the lm75 driver
echo "module lm75 +p" > /sys/kernel/debug/dynamic_debug/control

# Enable for a specific file
echo "file drivers/hwmon/lm75.c +p" > /sys/kernel/debug/dynamic_debug/control

# Enable for a specific function
echo "func lm75_probe +p" > /sys/kernel/debug/dynamic_debug/control

# Include function name and line number in output
echo "module lm75 +pflm" > /sys/kernel/debug/dynamic_debug/control
```

Flags: `p` = print, `f` = function name, `l` = line number, `m` = module name, `t` = thread ID

View output:

```
dmesg -w # Follow kernel log in real time
```

Kernel config required:

```
CONFIG_DYNAMIC_DEBUG=y
```

To enable debug messages early in boot (before userspace is up), add `dyndbg="module lm75 +p"` to the kernel command line in U-Boot or the device tree `chosen` node.

ftrace — Probe Sequence Tracing

Use `ftrace` with the `function_graph` tracer to visualize the call chain during driver probe — invaluable for understanding why a probe fails or hangs.

```
# Enable function_graph tracer
echo function_graph > /sys/kernel/debug/tracing/current_tracer

# Filter to your driver's functions
echo "lm75_*" > /sys/kernel/debug/tracing/set_ftrace_filter

# Or trace the entire I2C subsystem
echo "i2c_*" >> /sys/kernel/debug/tracing/set_ftrace_filter

# Start tracing
echo 1 > /sys/kernel/debug/tracing/tracing_on

# Trigger a probe (rebind the driver)
echo "0-0048" > /sys/bus/i2c/drivers/lm75/unbind
echo "0-0048" > /sys/bus/i2c/drivers/lm75/bind

# Stop and read the trace
echo 0 > /sys/kernel/debug/tracing/tracing_on
cat /sys/kernel/debug/tracing/trace
```

Example output:

```
1)          | lm75_probe() {
1) 0.834 us  |   devm_regmap_init_i2c();
1) 0.417 us  |   regmap_read();
1)          |   devm_hwmon_device_register_with_info() {
1) 1.250 us  |     hwmon_device_register_with_info();
1) 1.667 us  |   }
1) 4.584 us  | }
```

On memory-constrained BMCs, reduce the trace buffer: `echo 512 > /sys/kernel/debug/tracing/buffer_size_kb` (default is often 1408 KB per CPU).

debugfs Interfaces

Several kernel subsystems expose debugging information through debugfs:

GPIO state:

```
cat /sys/kernel/debug/gpio
# Shows all registered GPIO chips, claimed lines, direction, and value
```

I2C bus information:

```
# If CONFIG_I2C_DEBUG_CORE is enabled
ls /sys/kernel/debug/i2c/
```

Regmap register dumps:

```
# If your driver uses regmap, its registers are exposed automatically
cat /sys/kernel/debug/regmap/0-0048/registers
```

Clock tree (useful for SoC debugging):

```
cat /sys/kernel/debug/clk/clk_summary
```

dmesg Patterns for Common Probe Failures

When a driver fails to probe, `dmesg` contains the clues. Here are the patterns to look for:

Missing device tree node:

```
# No output at all for your driver – it was never matched
# Check: does your DTS node have the right compatible string?
cat /proc/device-tree/soc/apb/bus@1e78a000/i2c-bus@100/temperature-sensor@48/compatible
```

Wrong compatible string:

```
# Driver loads but never probes – compatible mismatch
# Compare driver's of_match_table with DTS compatible
cat /sys/bus/i2c/drivers/lm75/module/drivers
```

Deferred probe:

```
lm75 0-0048: probe deferral - supplier not ready
# A dependency (regulator, clock, GPIO) isn't available yet
# Check: cat /sys/kernel/debug/devices_deferred
```

I2C communication failure:

```
lm75 0-0048: Failed to read register 0x00: -6
# -6 = ENXIO (no device at that address)
# Verify with: i2cdetect -y 0
```

Missing kernel config:

```
# modprobe: FATAL: Module lm75 not found
# Check: zcat /proc/config.gz | grep LM75
```

I2C/SPI Sensor Binding Walkthrough

This end-to-end walkthrough traces the full path for adding an I2C hwmon sensor to an OpenBMC system, from device tree to Redfish visibility. We use the **TI TMP175** temperature sensor as the running example — it is

supported by the `lm75` kernel driver and works in QEMU.



Step 1: Device Tree Node

Add the sensor to your platform's device tree on the appropriate I2C bus:

```
&i2c0 {
    status = "okay";

    temperature-sensor@48 {
        compatible = "ti,tmp175";
        reg = <0x48>;
    };
};
```

Key fields:

- **compatible**: Must match a string in the kernel driver's `of_device_id` table. For TMP175, the `lm75` driver matches `"ti,tmp175"`.
- **reg**: The 7-bit I2C address. TMP175 defaults to `0x48` (A0=A1=A2=GND).
- **Node name**: Convention is `<function>@<addr>`, e.g., `temperature-sensor@48`.

Step 2: Kernel Driver Binding

The kernel's I2C subsystem matches the DTS `compatible` string against registered drivers. For TMP175:

```
Driver: drivers/hwmon/lm75.c
Compatible table entry: { .compatible = "ti,tmp175", .data = &tmp175 }
```

At boot, when the I2C bus is initialized, the kernel:

1. Parses the DTS node and creates an `i2c_client` for address `0x48`
2. Matches `"ti,tmp175"` to the `lm75` driver
3. Calls `lm75_probe()`, which reads chip ID registers and creates an hwmon device

Verify binding succeeded:

```
# Check driver bound to device
ls -la /sys/bus/i2c/devices/0-0048/driver
# Should show: ... -> ../../../../bus/i2c/drivers/lm75

# Check dmesg for probe
dmesg | grep lm75
# lm75 0-0048: hwmon0: sensor 'tmp175'
```

Step 3: hwmon sysfs Verification

Once the driver probes successfully, it exposes temperature readings through the hwmon sysfs interface:

```
# Find the hwmon device
ls /sys/class/hwmon/
# hwmon0

# Check which driver owns it
cat /sys/class/hwmon/hwmon0/name
# tmp175

# Read temperature (in millidegrees Celsius)
cat /sys/class/hwmon/hwmon0/temp1_input
# 25000 (= 25.0°C)

# Read all attributes
ls /sys/class/hwmon/hwmon0/
# name temp1_input temp1_max temp1_max_hyst ...
```

The `temp1_input` value is in millidegrees Celsius. Divide by 1000 for the human-readable temperature (25000 = 25.0°C).

Step 4: Entity-Manager JSON Configuration

Entity-manager discovers hardware based on JSON configuration files. Create an entry for your platform that describes the sensor:

```
{
  "Exposes": [
    {
      "Name": "Baseboard Temp",
      "Type": "TMP175",
      "Bus": 0,
      "Address": "0x48"
    }
  ],
  "Name": "My Platform Baseboard",
  "Probe": "TRUE"
}
```

Place this JSON in your meta-layer's entity-manager configurations directory:

```
meta-vendor/meta-platform/
  recipes-phosphor/configuration/
    entity-manager/
      my-platform-baseboard.json
```

See the [Entity Manager Guide](#) for advanced probe expressions and configuration patterns.

Step 5: dbus-sensors Pickup and D-Bus Exposure

The `dbus-sensors` package includes `hwmontempensor`, which monitors entity-manager for matching configurations and exposes hwmon readings on D-Bus.

Once entity-manager publishes the configuration, `hwmontempensor` automatically:

1. Matches the `Type` field to its supported sensor types
2. Opens the corresponding `/sys/class/hwmon/` sysfs path
3. Creates a D-Bus object at `/xyz/openbmc_project/sensors/temperature/Baseboard_Temp`

4. Polls the sysfs value and updates the D-Bus property

Verify on D-Bus:

```
# List temperature sensors
busctl tree xyz.openbmc_project.HwmonTempSensor

# Read the sensor value
busctl get-property xyz.openbmc_project.HwmonTempSensor \
    /xyz/openbmc_project/sensors/temperature/Baseboard_Temp \
    xyz.openbmc_project.Sensor.Value Value
# d 25.0
```

See the [D-Bus Sensors Guide](#) and [hwmon Sensors Guide](#) for sensor types and thresholds.

Step 6: Redfish Sensor Visibility

BMCWeb exposes D-Bus sensor objects as Redfish Sensor resources automatically:

```
# Query chassis sensors
curl -k -u root:0penBmc \
    https://localhost:2443/redfish/v1/Chassis/chassis/Sensors/temperature_Baseboard_Temp
```

Expected response (excerpt):

```
{
  "@odata.id": "/redfish/v1/Chassis/chassis/Sensors/temperature_Baseboard_Temp",
  "Name": "Baseboard Temp",
  "Reading": 25.0,
  "ReadingUnits": "Cel",
  "ReadingType": "Temperature",
  "Status": {
    "State": "Enabled",
    "Health": "OK"
  }
}
```

Cross-References

For deeper detail on each layer of the stack:

- [Entity Manager Guide](#) — JSON probe expressions, multi-board configs
- [D-Bus Sensors Guide](#) — sensor daemon architecture, threshold configuration
- [hwmon Sensors Guide](#) — hwmon sysfs attributes, supported chip types
- [I2C Device Integration Guide](#) — I2C bus configuration, device troubleshooting

Upstreaming Best Practices

OpenBMC Upstream-First Policy

The OpenBMC kernel tree's development policy is **"code must be upstream first."** Code enters the OpenBMC kernel in order of preference:

1. **Upstream release integration** — arrives automatically when OpenBMC advances kernel versions

2. **Backporting** — cherry-picking accepted upstream commits to the current OpenBMC branch
3. **Temporary carry patches** — held in the OpenBMC tree while upstream review is ongoing (least preferred)

Commit Message Format

Follow the Linux kernel commit message conventions:

```
subsystem: component: Short description (under 50 chars)

Detailed explanation of the change. Explain the problem being solved
and why this approach was chosen. Wrap at 72 characters.

Tested on ast2600-evb QEMU and physical hardware.

Signed-off-by: Your Name <your.email@example.com>
```

Submission to Upstream Linux

```
# Format patches for upstream
git format-patch -1 --cc=openbmc@lists.ozlabs.org

# Send via email to the subsystem maintainer
git send-email --to=maintainer@kernel.org \
  --cc=openbmc@lists.ozlabs.org \
  0001-my-patch.patch
```

Submission to OpenBMC Kernel Tree

For patches carried temporarily in the OpenBMC tree:

```
git format-patch \
  --subject-prefix="PATCH linux dev-6.18" \
  --to=openbmc@lists.ozlabs.org \
  -1
```

Push to Gerrit for code review:

```
git push gerrit HEAD:refs/for/dev-6.18
```

Troubleshooting

Issue: Patch Fails to Apply

Symptom: BitBake errors with “Patch failed” during `do_patch`

Cause: The patch was generated against a different kernel version or conflicts with other patches in the layer stack.

Solution:

1. Check the base commit your patch was generated from
2. Use `devtool modify linux-aspeed` to get the exact source BitBake uses
3. Regenerate the patch against that source:

```
cd build/workspace/sources/linux-aspeed
# Apply your changes, commit, then:
git format-patch -1
```

Issue: Config Option Not Taking Effect

Symptom: Your `.cfg` fragment sets `CONFIG_MY_OPTION=y` but the final `.config` has it unset

Cause: A dependency is missing, or another config overrides yours.

Solution:

1. Run `bitbake linux-aspeed -c kernel_configcheck -f` to see warnings
2. Check dependencies in `menuconfig` — the option may require a parent option
3. Add the parent option to your fragment as well

Issue: devtool modify Uses Wrong defconfig

Symptom: After `devtool modify`, the kernel builds with an incorrect or minimal config

Cause: Known issue ([openbmc/openbmc#3294](https://github.com/openbmc/openbmc/issues/3294)) where devtool may not apply machine-specific defconfig correctly.

Solution:

1. Manually copy the correct defconfig:

```
cp meta-aspeed/recipes-kernel/linux/linux-aspeed/aspeed-g6/defconfig \
  build/workspace/sources/linux-aspeed/.config
```

2. Or use the bbappend workflow instead of devtool for the final integration

Issue: Kernel Boots But Driver Not Found

Symptom: `modprobe my_driver` returns "Module not found" or device is not detected at boot

Cause: Driver not enabled in kernel config, or device tree node missing/incorrect.

Solution:

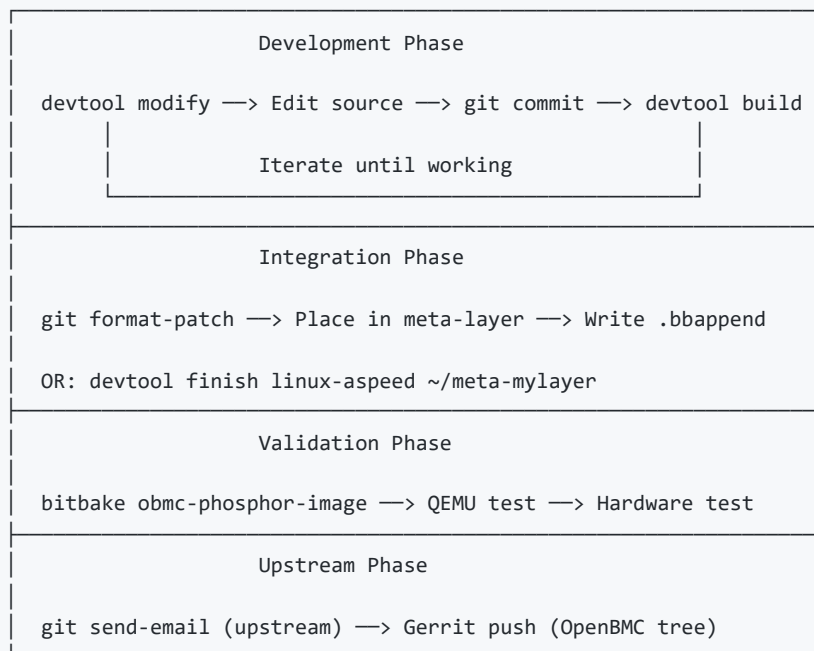
1. Verify config: `zcat /proc/config.gz | grep MY_DRIVER`
 2. Verify DTS node: `ls /proc/device-tree/soc/...`
 3. Check `dmesg` for probe failures: `dmesg | grep -i error`
-

Quick Reference

Essential Commands

Task	Command
Check out kernel for editing	<code>devtool modify linux-aspeed</code>
Build kernel only	<code>devtool build linux-aspeed</code>
Interactive config editor	<code>bitbake linux-aspeed -c menuconfig</code>
Generate config diff	<code>bitbake linux-aspeed -c diffconfig</code>
Validate config fragments	<code>bitbake linux-aspeed -c kernel_configcheck -f</code>
Save current config as defconfig	<code>bitbake linux-aspeed -c savedefconfig</code>
Export patches to layer	<code>devtool finish linux-aspeed ~/meta-mylayer</code>
Clean up workspace	<code>devtool reset linux-aspeed</code>
Force re-patch	<code>bitbake linux-aspeed -c patch -f</code>
Full rebuild from scratch	<code>bitbake linux-aspeed -c cleansstate && bitbake linux-aspeed</code>
Build out-of-tree module	<code>bitbake my-hwmon-module</code>
Enable dynamic debug for module	<code>echo "module lm75 +p" > /sys/kernel/debug/dynamic_debug/control</code>
Start ftrace function_graph	<code>echo function_graph > /sys/kernel/debug/tracing/current_tracer</code>
Check GPIO state via debugfs	<code>cat /sys/kernel/debug/gpio</code>
Scan I2C bus	<code>i2cdetect -y 0</code>
Read I2C register	<code>i2cget -y 0 0x48 0x00 w</code>
Check deferred probes	<code>cat /sys/kernel/debug/devices_deferred</code>

Patch Workflow Summary



References

- [OpenBMC Kernel Development Guide](#) — official kernel development policy and workflow
- [OpenBMC Yocto Development](#) — devtool workflow for OpenBMC
- [OpenBMC Cheatsheet](#) — quick reference for devtool, BitBake, and TFTP testing
- [OpenBMC Add New System](#) — adding platforms with DTS and kernel config
- [Yocto Kernel Development Manual](#) — devtool, patches, and config fragments
- [Yocto devtool Reference](#) — complete devtool command reference
- [Linux Kernel Submitting Patches](#) — upstream kernel submission guide
- [OpenBMC Linux Kernel Repository](#) — kernel source tree
- [Yocto module.bbclass](#) — out-of-tree kernel module recipe class
- [libgpod Documentation](#) — userspace GPIO access library (v2 API)
- [Linux UIO Documentation](#) — Userspace I/O framework
- [Linux ftrace Documentation](#) — function tracer and function_graph tracer
- [Linux dynamic_debug](#) — runtime debug print control

Secure Boot & Image Signing

Establish a hardware root of trust on AST2600-based OpenBMC platforms using OTP key provisioning, secure boot chain verification, and build-time image signing.

Table of Contents

1. [Overview](#)
2. [Architecture](#)
 1. [AST2600 Secure Boot Chain](#)
 2. [Verification Flow Summary](#)
 3. [Key Dependencies](#)
3. [OTP Key Provisioning](#)
 1. [Step 1: Generate an RSA Key Pair](#)
 2. [Step 2: Install the socsec Tool](#)
 3. [Step 3: Create the OTP Configuration](#)
 4. [Step 4: Generate the OTP Image](#)
 5. [Step 5: Program OTP Fuses](#)
 6. [Security Strap Configuration](#)
4. [Image Signing Workflow](#)
 1. [Signing U-Boot SPL](#)
 2. [Signing U-Boot Proper](#)
 3. [Signing the Kernel FIT Image](#)
 4. [Verifying Signatures Offline](#)
5. [Build-Time Signing Integration](#)
 1. [Yocto Configuration](#)
 2. [Recipe Integration](#)
 3. [Kernel FIT Signing in the Build](#)
 4. [Build and Verify](#)
6. [Development vs Production Key Management](#)
 1. [Key Hierarchy](#)
 2. [Development Key Practices](#)
 3. [Production Key Practices](#)
 4. [OTP Key Rotation](#)
7. [Security Jumper Handling](#)
 1. [Jumper Configuration](#)
 2. [Recovery Procedure](#)
8. [Code Examples](#)
 1. [Example 1: Key Generation Script](#)
 2. [Example 2: Build-Time Signing Verification Script](#)

9. Troubleshooting

1. [Issue: Board does not boot after OTP programming](#)
2. [Issue: socsec reports "signature verification failed"](#)
3. [Issue: FIT image signature verification fails in U-Boot](#)
4. [Issue: OTP programming fails with "protected region" error](#)
5. [Debug Commands](#)

10. References

1. [Official Resources](#)
2. [Related Guides](#)
3. [External Documentation](#)

Overview

Secure boot ensures that every piece of firmware executed on the BMC is cryptographically verified before it runs. On ASPEED AST2600 platforms, the secure boot chain begins at the hardware level with a Root of Trust stored in One-Time Programmable (OTP) fuses and extends through each boot stage up to the Linux kernel.

This guide walks you through the complete secure boot lifecycle: understanding the chain of trust, generating and provisioning cryptographic keys, integrating image signing into the Yocto build, and managing keys across development and production environments. You will learn how each boot stage verifies the next, how the ASPEED `socsec` tool signs firmware images, and how OTP fuse programming permanently binds a platform to a specific Root of Trust.

Secure boot is essential for production BMC deployments where firmware integrity must be guaranteed. Without it, an attacker with flash access can replace BMC firmware with a malicious image. With secure boot enabled, the hardware rejects any firmware that does not carry a valid signature from the provisioned Root of Trust key.

Key concepts covered:

- AST2600 secure boot chain of trust (OTP through kernel)
- RSA key pair generation and OTP fuse provisioning
- ASPEED `socsec` tool for image signing
- Yocto build integration for automated signing
- Development vs production key management
- Security strap configuration and jumper handling

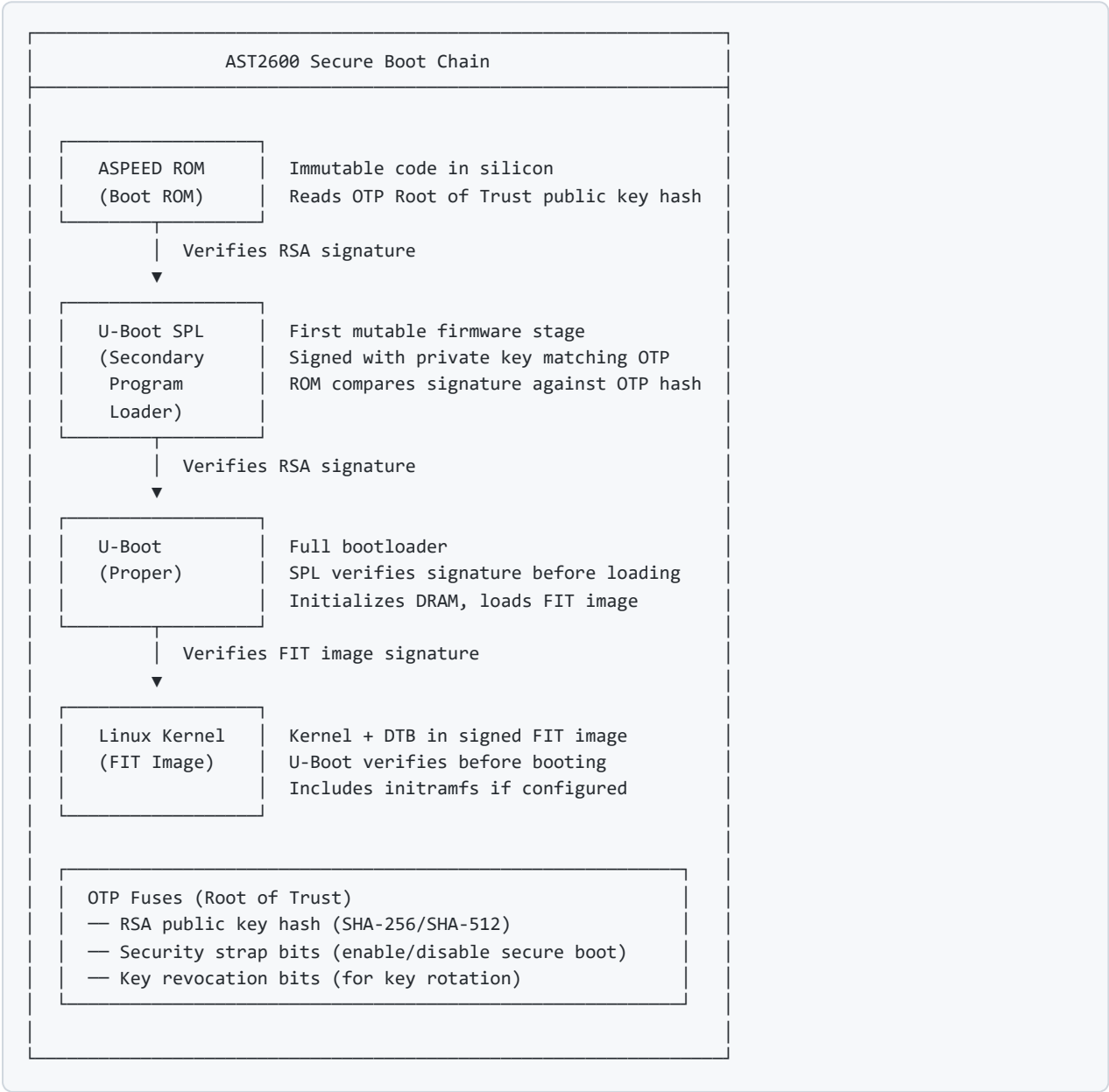
OTP programming is irreversible. Once you burn a public key hash into OTP fuses, you cannot change or revoke it. A mistake during OTP provisioning can permanently brick the platform. Always verify your key material and OTP configuration on development boards before touching production hardware.

Requires real hardware. While you can test the signing workflow and build signed images using QEMU, OTP programming and actual secure boot verification require a physical AST2600 board. QEMU does not emulate OTP fuses or the ROM-based signature verification.

Architecture

AST2600 Secure Boot Chain

The AST2600 implements a multi-stage verified boot process. Each stage verifies the cryptographic signature of the next stage before transferring control. If any verification fails, the boot process halts.



Verification Flow Summary

Boot Stage	Verified By	Signature Type	Key Source
U-Boot SPL	ASPEED Boot ROM	RSA-2048/4096 + SHA-256	OTP fuse (public key hash)
U-Boot	U-Boot SPL	RSA-2048/4096 + SHA-256	Embedded in SPL image
Linux Kernel (FIT)	U-Boot	RSA-2048/4096 + SHA-256	Embedded in U-Boot DTB

Key Dependencies

- **ASPEED Boot ROM:** The immutable first-stage bootloader in silicon. It reads OTP fuses to determine whether secure boot is enabled and which public key hash to use for verification.
- **socsec:** ASPEED's command-line tool for signing images, generating OTP configuration, and programming fuses. Part of the `socsec` Python package.
- **OpenSSL:** Used for RSA key pair generation. The private key signs images; the public key hash is burned into OTP.
- **U-Boot FIT signing:** U-Boot's Flattened Image Tree (FIT) format supports embedded RSA signatures for kernel and DTB verification.

OTP Key Provisioning

OTP (One-Time Programmable) fuses provide the hardware Root of Trust for the secure boot chain. You program the SHA-256 hash of your RSA public key into the OTP region. The Boot ROM reads this hash at every boot to verify the SPL signature.

Step 1: Generate an RSA Key Pair

Generate a 4096-bit RSA key pair using OpenSSL. This key pair is the foundation of your entire secure boot chain.

```
# Create a directory for your secure boot keys
mkdir -p ~/secure-boot-keys && cd ~/secure-boot-keys

# Generate 4096-bit RSA private key
openssl genrsa -out rsa4096_private.pem 4096

# Extract the public key
openssl rsa -in rsa4096_private.pem -pubout -out rsa4096_public.pem

# Verify the key pair
openssl rsa -in rsa4096_private.pem -check -noout
```

Use RSA-4096 for production deployments. RSA-2048 is acceptable for development and testing but provides a smaller security margin for long-lived platforms.

For development boards, you may choose RSA-2048 for faster signing and verification:

```
# Development-only: 2048-bit key (faster, lower security margin)
openssl genrsa -out rsa2048_dev_private.pem 2048
openssl rsa -in rsa2048_dev_private.pem -pubout -out rsa2048_dev_public.pem
```

Step 2: Install the socsec Tool

The ASPEED `socsec` tool handles OTP image generation, image signing, and OTP programming. Install it from PyPI or the ASPEED SDK.

```
# Install socsec from PyPI
pip3 install socsec

# Verify installation
socsec --version

# Alternatively, clone from ASPEED's repository
git clone https://github.com/AspeedTech-BMC/socsec.git
cd socsec && pip3 install .
```

Step 3: Create the OTP Configuration

The OTP configuration file defines which fuse bits to program, including the public key hash and security strap settings. Create a JSON configuration for your platform.

```
{
  "version": "A3",
  "soc": "2600",
  "otp_info": {
    "conf": {
      "secure_boot_enable": true,
      "secure_boot_header_offset": "0x20",
      "rsa_key_order": "big_endian",
      "sha_algorithm": "SHA256",
      "retire_key_id": 0
    },
    "strap": {
      "secure_boot": true,
      "boot_from_emmc": false,
      "enable_watchdog": true,
      "uart_debug_disable": false
    },
    "key": {
      "key_type": "RSA4096_SHA256",
      "number_of_keys": 1,
      "keys": [
        {
          "key_file": "rsa4096_public.pem",
          "key_id": 0
        }
      ]
    }
  }
}
```

Save this file as `otp_config.json` in your secure boot keys directory.

The `version` field must match your AST2600 silicon revision (A1, A3, etc.). Check your chip marking or datasheet. Using the wrong version causes OTP programming to fail or produce incorrect fuse values.

Step 4: Generate the OTP Image

Use `socsec` to generate the OTP image from your configuration. This produces a binary that contains the fuse values to program.

```
# Generate OTP image
socsec make_otp_image \
  --config otp_config.json \
  --output otp_image.bin

# Verify the generated OTP image contents
socsec verify_otp_image \
  --config otp_config.json \
  --image otp_image.bin
```

The tool reads your public key, computes its SHA-256 hash, and embeds it in the OTP image along with the strap and configuration bits.

Step 5: Program OTP Fuses

This step is irreversible. Once OTP fuses are programmed, they cannot be erased or modified. Double-check your configuration and key material before proceeding. Perform this step on a development board first.

Program the OTP fuses using `socsec` over the ASPEED debug UART or JTAG interface:

```
# Connect to the AST2600 via debug UART
# Ensure the board is in OTP programming mode (security jumper set)

# Program OTP fuses
socsec otp_prog \
  --image otp_image.bin \
  --port /dev/ttyUSB0 \
  --baud 115200

# Read back and verify OTP contents
socsec otp_read \
  --port /dev/ttyUSB0 \
  --baud 115200 \
  --output otp_readback.bin
```

After programming, compare the readback with your expected values:

```
# Diff the programmed values against expected
socsec otp_verify \
  --config otp_config.json \
  --image otp_readback.bin
```

Security Strap Configuration

The AST2600 has hardware strap pins that interact with OTP security settings. These straps control boot behavior during development.

Strap Pin	Function	Development	Production
<code>SCU510[1]</code>	Secure boot enable	0 (disabled)	1 (enabled via OTP)
<code>SCU510[2]</code>	Boot source select	SPI (default)	SPI
<code>SCU510[4]</code>	UART debug	Enabled	Disabled
<code>SCU510[6]</code>	JTAG debug	Enabled	Disabled

During development, keep UART and JTAG debug enabled in the strap configuration. Disable them only in production OTP images. If you disable debug interfaces in OTP and your signed image fails to boot, you lose all debug access to the board.

Image Signing Workflow

Once OTP fuses are provisioned with your public key hash, every firmware image must be signed with the corresponding private key. The ASPEED `socsec` tool signs U-Boot SPL and U-Boot images, while U-Boot itself handles FIT image (kernel) signing.

Signing U-Boot SPL

The Boot ROM expects the SPL image to contain a signature header. Use `socsec` to wrap the SPL binary with a verified boot header.

```
# Sign the U-Boot SPL binary
socsec sign \
  --image u-boot-spl.bin \
  --key rsa4096_private.pem \
  --algorithm RSA4096_SHA256 \
  --soc 2600 \
  --output u-boot-spl-signed.bin

# Verify the signature offline (before flashing)
socsec verify \
  --image u-boot-spl-signed.bin \
  --key rsa4096_public.pem \
  --soc 2600
```

The signed SPL binary includes:

- A signature header with the RSA signature
- The original SPL code
- Padding to align with the Boot ROM's expectations

Signing U-Boot Proper

U-Boot SPL verifies U-Boot proper using the same key infrastructure. Sign the U-Boot binary similarly:

```
# Sign U-Boot proper
socsec sign \
  --image u-boot.bin \
  --key rsa4096_private.pem \
  --algorithm RSA4096_SHA256 \
  --soc 2600 \
  --output u-boot-signed.bin

# Verify offline
socsec verify \
  --image u-boot-signed.bin \
  --key rsa4096_public.pem \
  --soc 2600
```

Signing the Kernel FIT Image

The Linux kernel, device tree, and optional initramfs are packaged into a FIT (Flattened Image Tree) image. U-Boot verifies the FIT image signature before booting.

FIT signing uses U-Boot's built-in mechanism with `mkimage`:

```
# Create a FIT image description (ITS file)
# The ITS file references the kernel, DTB, and signing key

# Sign the FIT image with mkimage
mkimage -f fit-image.its \
    -K u-boot.dtb \
    -k ~/secure-boot-keys \
    -r \
    fitImage

# The -K flag writes the public key into the U-Boot DTB
# The -k flag specifies the directory containing the private key
# The -r flag marks the key as "required" for verification
```

The U-Boot DTB now contains the public key node. When U-Boot boots, it reads this key from its own DTB and uses it to verify the FIT image signature before launching the kernel.

Verifying Signatures Offline

Always verify your signed images before flashing them to hardware:

```
# Verify SPL signature
socsec verify --image u-boot-spl-signed.bin --key rsa4096_public.pem --soc 2600
echo "SPL verification: $?"

# Verify U-Boot signature
socsec verify --image u-boot-signed.bin --key rsa4096_public.pem --soc 2600
echo "U-Boot verification: $?"

# Verify FIT image signature using fit_check_sign (from U-Boot tools)
fit_check_sign -f fitImage -k u-boot.dtb
echo "FIT verification: $?"
```

Add offline verification as a CI/CD step. Reject any build that produces images failing signature verification. This catches signing key mismatches and corrupted images before they reach hardware.

Build-Time Signing Integration

The OpenBMC Yocto build system integrates secure boot signing into the image build process. You configure signing parameters in your machine configuration, and the build automatically signs all boot-stage images.

Yocto Configuration

Add the following variables to your machine's `local.conf` or `machine.conf`:

```

# Enable secure boot signing
SOCSEC_SIGN_ENABLE = "1"

# Path to the signing private key
SOCSEC_SIGN_KEY = "/path/to/secure-boot-keys/rsa4096_private.pem"

# Signing algorithm
SOCSEC_SIGN_ALGO = "RSA4096_SHA256"

# SOC type
SOCSEC_SIGN_SOC = "2600"

# Enable FIT image signing for the kernel
UBOOT_SIGN_ENABLE = "1"
UBOOT_SIGN_KEYNAME = "rsa4096"
UBOOT_SIGN_KEYDIR = "/path/to/secure-boot-keys"
UBOOT_MKIMAGE_DTCOPTS = "-I dts -O dtb -p 2000"
UBOOT_SIGN_IMG_KEYNAME = "rsa4096"
FIT_SIGN_INDIVIDUAL = "1"

```

Recipe Integration

The `u-boot-aspeed-sdk` recipe (or your machine's U-Boot recipe) picks up the signing configuration and invokes `socsec` during the build. The typical integration looks like this in the recipe's bbappend:

```

# meta-<your-machine>/recipes-bsp/u-boot/u-boot-aspeed-sdk_%.bbappend

DEPENDS += "socsec-native"

do_deploy:append() {
    if [ "${SOCSEC_SIGN_ENABLE}" = "1" ]; then
        # Sign U-Boot SPL
        socsec sign \
            --image ${DEPLOYDIR}/u-boot-spl.bin \
            --key ${SOCSEC_SIGN_KEY} \
            --algorithm ${SOCSEC_SIGN_ALGO} \
            --soc ${SOCSEC_SIGN_SOC} \
            --output ${DEPLOYDIR}/u-boot-spl-signed.bin

        # Sign U-Boot proper
        socsec sign \
            --image ${DEPLOYDIR}/u-boot.bin \
            --key ${SOCSEC_SIGN_KEY} \
            --algorithm ${SOCSEC_SIGN_ALGO} \
            --soc ${SOCSEC_SIGN_SOC} \
            --output ${DEPLOYDIR}/u-boot-signed.bin

        # Replace unsigned images with signed versions
        install -m 0644 ${DEPLOYDIR}/u-boot-spl-signed.bin \
            ${DEPLOYDIR}/u-boot-spl.bin
        install -m 0644 ${DEPLOYDIR}/u-boot-signed.bin \
            ${DEPLOYDIR}/u-boot.bin
    fi
}

```

Kernel FIT Signing in the Build

The kernel recipe uses U-Boot's `mkimage` to sign the FIT image during the build. Enable this in your kernel bbappend:


```
# meta-<your-machine>/recipes-kernel/linux/linux-aspeed_%.bbappend

inherit kernel-fitimage

# These variables drive FIT signing
UBOOT_SIGN_ENABLE = "1"
UBOOT_SIGN_KEYNAME = "rsa4096"
UBOOT_SIGN_KEYDIR = "/path/to/secure-boot-keys"

# Include both kernel and DTB in the signed FIT
FIT_SIGN_INDIVIDUAL = "1"
KERNEL_CLASSES = "kernel-fitimage"
KERNEL_IMAGETYPE = "fitImage"
```

Build and Verify

Run the full build and verify the output images carry valid signatures:

```
# Build the image with signing enabled
cd openbmc
. setup <your-machine>
bitbake obmc-phosphor-image

# Check the deploy directory for signed images
ls -la build/tmp/deploy/images/<your-machine>/u-boot-spl.bin
ls -la build/tmp/deploy/images/<your-machine>/u-boot.bin
ls -la build/tmp/deploy/images/<your-machine>/fitImage

# Verify signatures offline
socsec verify \
    --image build/tmp/deploy/images/<your-machine>/u-boot-spl.bin \
    --key ~/secure-boot-keys/rsa4096_public.pem \
    --soc 2600
```

The build system does not program OTP fuses. OTP provisioning is always a separate, explicit step performed directly on hardware. The build only produces signed images that are compatible with the OTP-provisioned key.

Development vs Production Key Management

Managing cryptographic keys is the most critical aspect of secure boot operations. A compromised private key allows an attacker to sign malicious firmware that the hardware accepts as legitimate. The irreversible nature of OTP programming amplifies the stakes: once a public key hash is fused, the platform trusts that key permanently (unless key rotation slots are available).

Key Hierarchy

Maintain separate key sets for development and production with clearly defined boundaries:



Development Key Practices

Use development keys for all prototyping and CI/CD workflows. These keys are known and shared among the team.

```
# Generate a clearly labeled development key pair
openssl genrsa -out dev_rsa4096_private.pem 4096
openssl rsa -in dev_rsa4096_private.pem -pubout -out dev_rsa4096_public.pem

# Store in version control (development keys ONLY)
# NEVER store production keys in version control
cp dev_rsa4096_private.pem ~/openbmc/meta-mymachine/conf/keys/
cp dev_rsa4096_public.pem ~/openbmc/meta-mymachine/conf/keys/
```

Never reuse development keys for production. Development keys stored in version control or on developer workstations are considered compromised by definition. Production hardware must use keys generated and stored exclusively in an HSM.

Production Key Practices

Production keys must never leave the HSM. Use a signing service that accepts unsigned images and returns signed images without exposing the private key.

```

# Production signing workflow (conceptual)
# The private key never leaves the HSM

# 1. Build unsigned images
bitbake obmc-phosphor-image # with SOCSEC_SIGN_ENABLE = "0"

# 2. Submit unsigned images to the signing service
curl -X POST https://signing-service.internal/sign \
  -F "image=@u-boot-spl.bin" \
  -F "algorithm=RSA4096_SHA256" \
  -F "key_id=production_slot0" \
  -o u-boot-spl-signed.bin

# 3. Verify the signature using the public key
socsec verify \
  --image u-boot-spl-signed.bin \
  --key production_rsa4096_public.pem \
  --soc 2600

```

OTP Key Rotation

The AST2600 OTP supports multiple key slots, allowing you to rotate keys without replacing hardware. However, key rotation is constrained by the number of available OTP slots (typically 4).

Operation	Possible?	Notes
Add a new key to an empty slot	Yes	Program the new key hash into the next available slot
Retire an old key	Yes	Set the retire bit for the old key's slot ID
Modify an existing key	No	OTP fuses are write-once; bits can be set but not cleared
Erase all keys	No	OTP is permanently programmed
Add keys beyond slot limit	No	Once all slots are used, no more keys can be added

To rotate to a new key:

```
# 1. Generate a new key pair
openssl genrsa -out rsa4096_rotation1_private.pem 4096
openssl rsa -in rsa4096_rotation1_private.pem -pubout -out rsa4096_rotation1_public.pem

# 2. Update OTP config to add the new key at slot 1 and retire slot 0
# In otp_config.json:
# "retire_key_id": 0,
# "keys": [
#   { "key_file": "rsa4096_public.pem", "key_id": 0 },
#   { "key_file": "rsa4096_rotation1_public.pem", "key_id": 1 }
# ]

# 3. Generate and program the updated OTP image
socsec make_otp_image --config otp_config_rotation.json --output otp_rotation.bin
socsec otp_prog --image otp_rotation.bin --port /dev/ttyUSB0 --baud 115200

# 4. Sign all new images with the rotation key
socsec sign \
  --image u-boot-spl.bin \
  --key rsa4096_rotation1_private.pem \
  --algorithm RSA4096_SHA256 \
  --soc 2600 \
  --key-id 1 \
  --output u-boot-spl-signed.bin
```

Plan your key rotation strategy before the initial OTP provisioning. Decide how many slots to reserve for rotation and document the key lifecycle for your platform.

Security Jumper Handling

During development, you need the ability to disable secure boot for debugging and reflashing unsigned images. The AST2600 supports a hardware security jumper that overrides OTP strap settings.

Jumper Configuration

Most AST2600 evaluation boards provide a jumper header for secure boot override:

Jumper State	Boot Behavior	Use Case
Open (default)	OTP straps control boot	Production / secure boot active
Closed	Secure boot bypassed	Development / recovery

When the security jumper is closed:

- The Boot ROM skips SPL signature verification
- Unsigned images boot normally
- UART and JTAG debug remain available regardless of OTP settings
- OTP fuse contents are not modified (the jumper only bypasses enforcement)

Label development boards clearly with their jumper state and OTP provisioning status. A board with OTP fuses programmed but the security jumper closed looks identical to an unprovisioned board during boot, which can cause confusion during testing.

Recovery Procedure

If a board with secure boot enabled fails to boot a signed image (for example, due to a corrupted flash), use the security jumper to recover:

```
# 1. Power off the board
# 2. Close the security jumper (bypass secure boot)
# 3. Power on – board boots unsigned recovery image or enters UART recovery mode
# 4. Reflash a correctly signed image via UART or SPI programmer

# UART recovery using ASPEED's uart_flash tool
socsec uart_flash \
    --port /dev/ttyUSB0 \
    --baud 115200 \
    --image flash-<machine>.bin

# 5. Power off
# 6. Open the security jumper (re-enable secure boot)
# 7. Power on – board boots the newly flashed signed image
```

The security jumper must be removed (open) on production systems. A closed security jumper completely defeats the purpose of secure boot. Include jumper state verification in your manufacturing and deployment checklists.

Code Examples

Example 1: Key Generation Script

A complete script for generating development key pairs and OTP configuration:

```

#!/bin/bash
# generate-dev-keys.sh
# Generate development secure boot keys for AST2600

set -euo pipefail

KEY_DIR="${1:-./dev-keys}"
KEY_SIZE="${2:-4096}"
SOC_VERSION="${3:-A3}"

echo "Generating ${KEY_SIZE}-bit RSA development keys in ${KEY_DIR}..."

mkdir -p "${KEY_DIR}"

# Generate RSA private key
openssl genrsa -out "${KEY_DIR}/dev_rsa${KEY_SIZE}_private.pem" "${KEY_SIZE}"

# Extract public key
openssl rsa \
    -in "${KEY_DIR}/dev_rsa${KEY_SIZE}_private.pem" \
    -pubout \
    -out "${KEY_DIR}/dev_rsa${KEY_SIZE}_public.pem"

# Display key fingerprint for verification
echo "Public key fingerprint:"
openssl rsa \
    -pubin \
    -in "${KEY_DIR}/dev_rsa${KEY_SIZE}_public.pem" \
    -outform DER 2>/dev/null | sha256sum

# Generate OTP configuration
cat > "${KEY_DIR}/otp_config_dev.json" << OTPEOF
{
    "version": "${SOC_VERSION}",
    "soc": "2600",
    "otp_info": {
        "conf": {
            "secure_boot_enable": true,
            "secure_boot_header_offset": "0x20",
            "rsa_key_order": "big_endian",
            "sha_algorithm": "SHA256",
            "retire_key_id": 0
        },
        "strap": {
            "secure_boot": true,
            "boot_from_emmc": false,
            "enable_watchdog": true,
            "uart_debug_disable": false
        },
        "key": {
            "key_type": "RSA${KEY_SIZE}_SHA256",
            "number_of_keys": 1,
            "keys": [
                {
                    "key_file": "dev_rsa${KEY_SIZE}_public.pem",
                    "key_id": 0
                }
            ]
        }
    }
}
OTPEOF

```

```
echo "Development keys and OTP config generated in ${KEY_DIR}/"  
echo "WARNING: These are DEVELOPMENT keys. Never use them for production."
```

See the complete example at [examples/secure-boot/](#).

Example 2: Build-Time Signing Verification Script

A script to verify all signed images after a build completes:

```

#!/bin/bash
# verify-signed-images.sh
# Verify all secure boot signatures after an OpenBMC build

set -euo pipefail

DEPLOY_DIR="${1:?Usage: $0 <deploy-dir> <public-key>}"
PUBLIC_KEY="${2:?Usage: $0 <deploy-dir> <public-key>}"
SOC="2600"
ERRORS=0

echo "Verifying signed images in ${DEPLOY_DIR}..."

# Verify U-Boot SPL
echo -n " U-Boot SPL: "
if socsec verify --image "${DEPLOY_DIR}/u-boot-spl.bin" \
    --key "${PUBLIC_KEY}" --soc "${SOC}" 2>/dev/null; then
    echo "PASS"
else
    echo "FAIL"
    ERRORS=$((ERRORS + 1))
fi

# Verify U-Boot proper
echo -n " U-Boot:      "
if socsec verify --image "${DEPLOY_DIR}/u-boot.bin" \
    --key "${PUBLIC_KEY}" --soc "${SOC}" 2>/dev/null; then
    echo "PASS"
else
    echo "FAIL"
    ERRORS=$((ERRORS + 1))
fi

# Verify FIT image (kernel)
echo -n " FIT Image:  "
if [ -f "${DEPLOY_DIR}/fitImage" ]; then
    if fit_check_sign -f "${DEPLOY_DIR}/fitImage" \
        -k "${DEPLOY_DIR}/u-boot.dtb" 2>/dev/null; then
        echo "PASS"
    else
        echo "FAIL"
        ERRORS=$((ERRORS + 1))
    fi
else
    echo "SKIP (fitImage not found)"
fi

echo ""
if [ "${ERRORS}" -eq 0 ]; then
    echo "All signature verifications passed."
    exit 0
else
    echo "ERROR: ${ERRORS} verification(s) failed!"
    exit 1
fi

```


Troubleshooting

Issue: Board does not boot after OTP programming

Symptom: Board appears completely dead after programming OTP fuses. No serial output, no response.

Cause: The OTP security straps enable secure boot, but the flash contains unsigned images (or images signed with a different key).

Solution:

1. Close the security jumper to bypass secure boot
2. Power cycle the board
3. Flash correctly signed images via UART recovery or SPI programmer
4. Open the security jumper and power cycle again

Issue: socsec reports “signature verification failed”

Symptom: `socsec verify` fails on a signed image that previously verified correctly.

Cause: The image was modified after signing (e.g., by a post-processing step in the build), or you are verifying with the wrong public key.

Solution:

1. Confirm you are using the correct public key that matches the signing private key
2. Check that no build steps modify the image after the signing step
3. Re-sign the image and verify immediately:

```
socsec sign --image u-boot-spl.bin --key private.pem \  
--algorithm RSA4096_SHA256 --soc 2600 --output signed.bin  
socsec verify --image signed.bin --key public.pem --soc 2600
```

Issue: FIT image signature verification fails in U-Boot

Symptom: U-Boot prints `Bad Data Hash` or `RSA signature verification failed` when booting the kernel.

Cause: The FIT image was signed with a key that does not match the public key embedded in U-Boot’s DTB, or the FIT image was modified after signing.

Solution:

1. Verify that the `UBOOT_SIGN_KEYNAME` in your build configuration matches the key filename (without extension)
2. Rebuild both U-Boot and the FIT image together (the public key is embedded in U-Boot’s DTB during FIT signing)
3. Ensure the `-r` flag is passed to `mkimage` to mark the key as required

Issue: OTP programming fails with “protected region” error

Symptom: `socsec otp_prog` reports a write-protection error.

Cause: The OTP region you are attempting to write has already been programmed. OTP bits can only transition from 0 to 1, never from 1 to 0.

Solution:

1. Read the current OTP contents: `socsec otp_read --port /dev/ttyUSB0 --baud 115200 --output current_otp.bin`
2. Compare against your target configuration to identify conflicting bits
3. If the existing OTP data conflicts with your new key, you cannot change it. The board is permanently bound to the previously programmed key.

Debug Commands

```
# Check OTP fuse contents from a running BMC (Linux)
devmem 0x1e6f2000 32  # OTP configuration word 0
devmem 0x1e6f2004 32  # OTP configuration word 1

# Read security strap register
devmem 0x1e6e2510 32  # SCU510 – hardware strap status

# Check U-Boot verified boot status (from U-Boot shell)
# At U-Boot prompt:
env print verified_boot
iminfo ${loadaddr}      # Show FIT image info including signature status

# View kernel boot log for FIT verification
dmesg | grep -i "verified\|signature\|secure"
```

References

Official Resources

- [ASPEED socsec Repository](#)
- [U-Boot Verified Boot Documentation](#)
- [OpenBMC U-Boot Repository](#)
- [ASPEED SDK Documentation](#)

Related Guides

- [Firmware Update Guide](#)
- [SPDM Guide](#)

External Documentation

- [ASPEED AST2600 Datasheet](#) (requires registration)
- [U-Boot FIT Image Signing](#)
- [Yocto kernel-fitimage Class](#)
- [NIST SP 800-57 Key Management Recommendations](#)

Tested on: AST2600-EVB hardware. OTP programming and secure boot verification require physical hardware and cannot be validated on QEMU. Last updated: 2026-02-06

Multi-Host Support

Configure a single BMC to manage multiple host systems using IPMB bridging, per-host state management, D-Bus/systemd multiplexing, or satellite Bridge ICs (OpenBIC).

Table of Contents

1. [Overview](#)
2. [Architecture](#)
 1. [Multi-Host System Topology](#)
 2. [IPMB Communication Model](#)
 3. [D-Bus Object Path Convention](#)
 4. [Redfish Resource Mapping](#)
 5. [Key Dependencies](#)
3. [Configuration](#)
 1. [Machine Configuration for Multi-Host](#)
 2. [IPMB Bridge Daemon Configuration](#)
 1. [ipmbd Service Template](#)
 2. [Enable IPMB for Each Host](#)
 3. [IPMB Channel Configuration](#)
 4. [I2C Device Tree Configuration](#)
 5. [Per-Host State Manager Configuration](#)
 1. [Host State Template](#)
 2. [Chassis State Template](#)
 3. [Enable Per-Host Services](#)
 6. [Build-Time Options](#)
4. [Porting Guide](#)
 1. [Step 1: Prerequisites](#)
 2. [Step 2: Update Device Tree](#)
 3. [Step 3: Configure Machine Layer](#)
 4. [Step 4: Create Systemd Target Dependencies](#)
 5. [Step 5: Add IPMB Bridge Recipe Append](#)
 6. [Step 6: Build and Verify](#)
5. [Code Examples](#)
 1. [Example 1: Querying Per-Host State via D-Bus](#)
 2. [Example 2: Power Control via Redfish](#)
 3. [Example 3: Per-Host IPMB Communication](#)
 4. [Example 4: Host State Monitor Script](#)
 5. [Example 5: Per-Host Sensor Namespace](#)
6. [Satellite Processor Approach \(OpenBIC\)](#)
 1. [What is a Bridge IC \(BIC\)?](#)

2. [BIC vs Direct IPMB Architecture](#)
 3. [Satellite Processor Topology](#)
 4. [Communication Stack](#)
 5. [PLDM Sensor Data Flow](#)
 6. [OpenBIC Hardware: ASPEED AST1030](#)
 7. [Reference Platform: Meta Yosemite v3.5](#)
 8. [When to Choose Each Approach](#)
7. [Troubleshooting](#)
 1. [Issue: IPMB Bridge Fails to Start](#)
 2. [Issue: Host State Manager Shows “Unknown” State](#)
 3. [Issue: IPMB Messages Time Out](#)
 4. [Issue: Redfish Shows Only One System](#)
 5. [Issue: Per-Host Sensors Not Appearing](#)
 6. [Debug Commands](#)
8. [References](#)
 1. [Official Resources](#)
 2. [OpenBIC / Satellite Processor Resources](#)
 3. [Related Guides](#)
 4. [External Documentation](#)
-

Overview

Multi-host support allows a single BMC to manage two or more host systems from one firmware image. This is common in high-density server platforms such as twin-server chassis, multi-node sleds, and blade enclosures where dedicating a BMC per host is impractical due to cost, power, or space constraints.

In a multi-host OpenBMC deployment, the BMC communicates with each host over dedicated IPMB (Intelligent Platform Management Bus) links — one I2C bus per host — and maintains independent state machines, sensor namespaces, and Redfish resource trees for each host. The systemd service manager uses template units to instantiate per-host services, while D-Bus object paths are indexed by host number to keep each host’s state isolated.

This guide walks you through the complete multi-host architecture: how IPMB bridges messages between the BMC and each host, how systemd template units and D-Bus object paths provide per-host isolation, how to configure your machine layer for N hosts, and how Redfish exposes each host as a separate ComputerSystem resource.

Key concepts covered:

- Single-BMC-to-N-host architecture over IPMB
- IPMB bridge daemon configuration (I2C bus assignment, address routing)
- Per-host state management with systemd template units
- D-Bus object path multiplexing (`/xyz/openbmc_project/state/host0` , `host1` , ...)
- Redfish ComputerSystem mapping for each host
- Yocto/BitBake configuration for multi-host builds

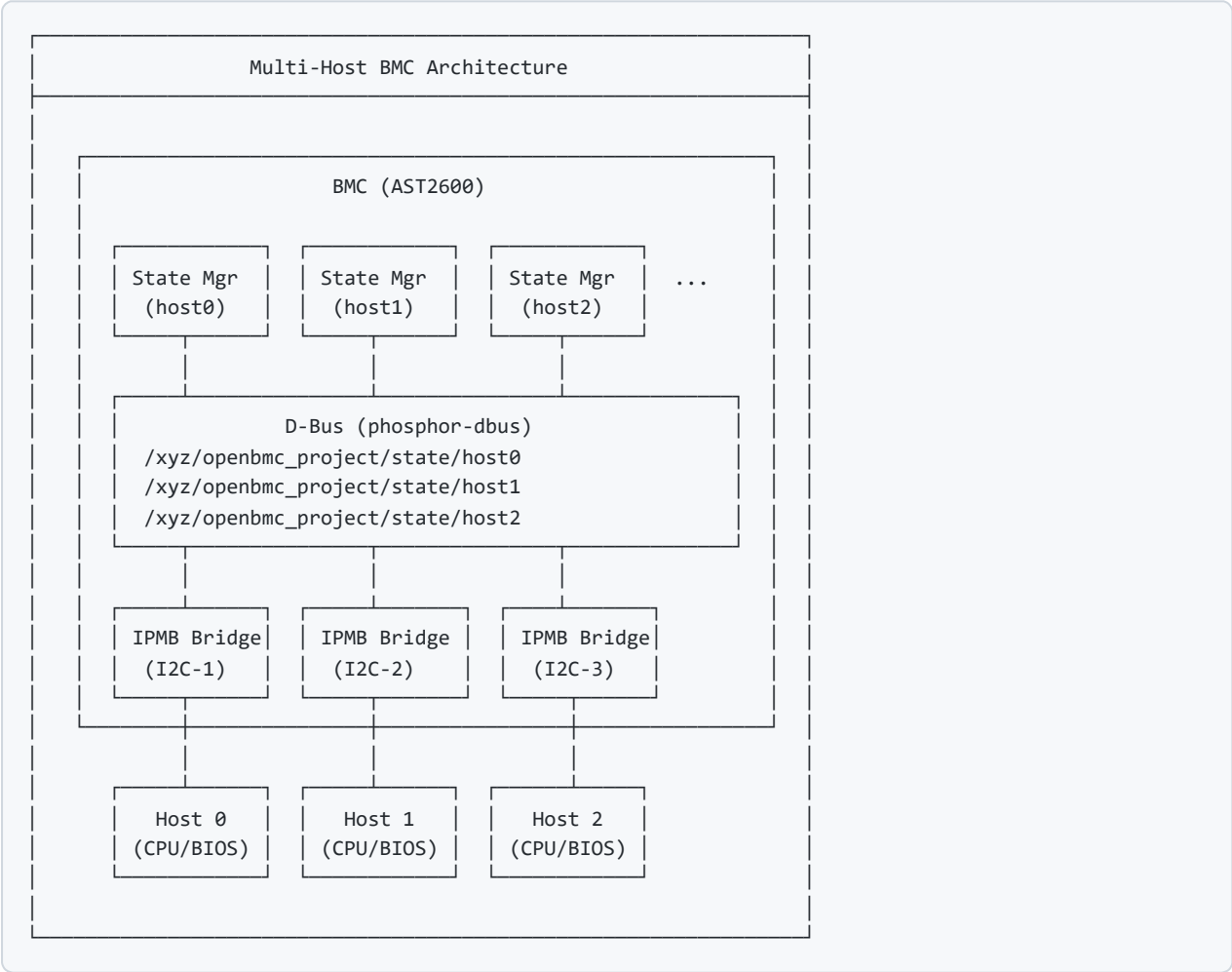
Multi-host support requires careful hardware design. Each host needs a dedicated I2C bus for IPMB communication. Verify your baseboard schematic provides separate I2C controllers or mux channels for each host before implementing the software configuration.

Multi-host is distinct from multi-node MCTP/PLDM topologies. This guide covers two approaches: (1) IPMI-based multi-host where each host has a traditional IPMB link to the BMC, and (2) satellite processor (OpenBIC) multi-host where Bridge ICs act as local agents offloading the BMC. For PLDM-based multi-endpoint management, see the [MCTP & PLDM Guide](#).

Architecture

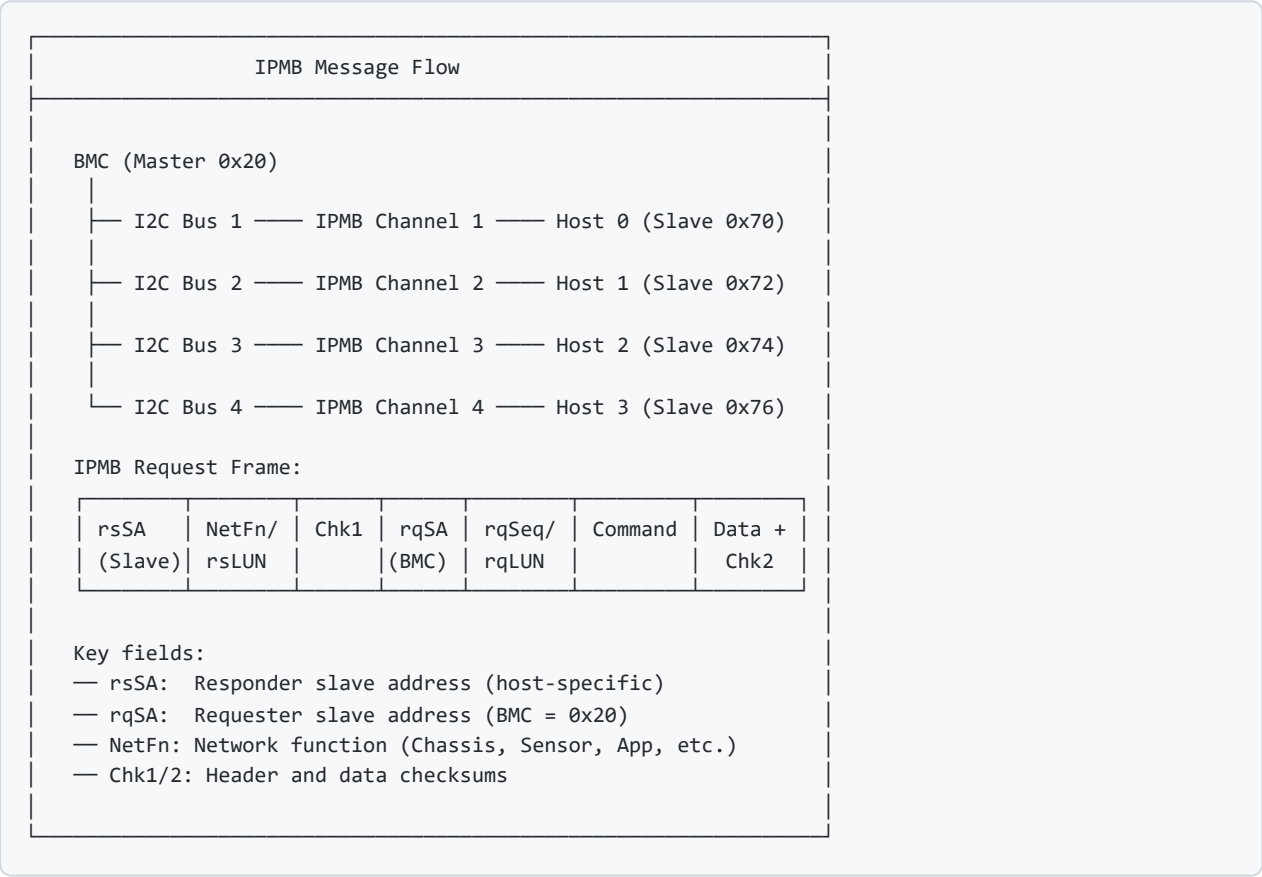
Multi-Host System Topology

A multi-host BMC manages N hosts through dedicated IPMB channels. Each host has its own I2C bus, power control GPIOs, and an independent state machine on the BMC side.



IPMB Communication Model

Each host communicates with the BMC over a dedicated I2C bus using the IPMB protocol. The BMC acts as the IPMB master (address 0x20) and each host baseboard management controller or satellite controller responds at a host-specific slave address.



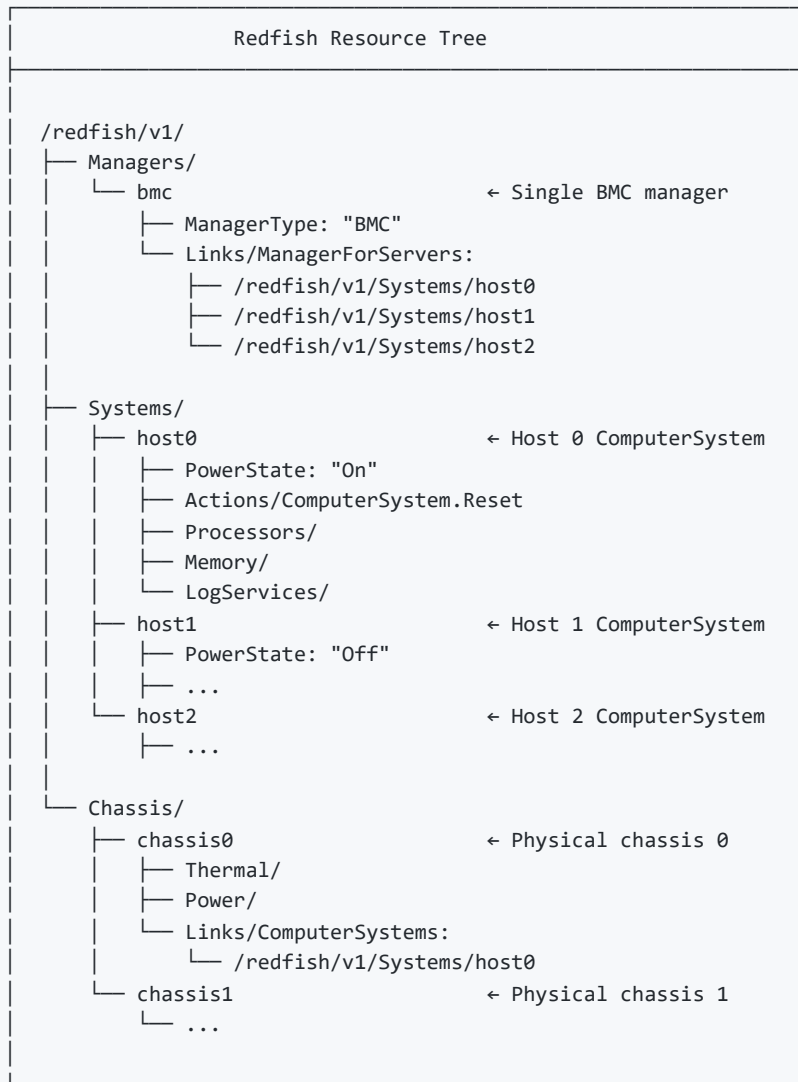
D-Bus Object Path Convention

Multi-host OpenBMC uses a numeric suffix on D-Bus object paths to identify each host. All phosphor services that are host-aware follow this naming convention.

D-Bus Object Path	Purpose
/xyz/openbmc_project/state/host0	Host 0 power/boot state
/xyz/openbmc_project/state/host1	Host 1 power/boot state
/xyz/openbmc_project/state/chassis0	Chassis 0 power state
/xyz/openbmc_project/state/chassis1	Chassis 1 power state
/xyz/openbmc_project/control/host0/boot	Host 0 boot settings
/xyz/openbmc_project/control/host1/boot	Host 1 boot settings
/xyz/openbmc_project/sensors/temperature/host0_cpu_temp	Host 0 CPU temperature
/xyz/openbmc_project/sensors/temperature/host1_cpu_temp	Host 1 CPU temperature
/xyz/openbmc_project/inventory/system/host0	Host 0 inventory
/xyz/openbmc_project/inventory/system/host1	Host 1 inventory

Redfish Resource Mapping

Each host maps to a separate Redfish ComputerSystem resource. The BMC itself is exposed as a single Manager that manages all hosts.



Key Dependencies

- **phosphor-state-manager**: Provides per-host state machines (host, chassis, BMC states)
- **ipmb-bridge / ipmbd**: Routes IPMI messages between the BMC and each host over I2C
- **phosphor-host-ipmid**: Handles IPMI commands, instantiated per host
- **bmcweb**: Serves Redfish with per-host ComputerSystem resources
- **phosphor-gpio-monitor**: Monitors per-host GPIO signals (power button, reset, etc.)

Configuration

Machine Configuration for Multi-Host

Define the number of hosts and IPMB bus assignments in your machine configuration.

```
# meta-mymachine/conf/machine/mymachine.conf

# Number of host instances (0-indexed: host0, host1, ..., hostN-1)
OBMC_HOST_INSTANCES = "0 1 2 3"

# Map each host to an I2C bus for IPMB communication
OBMC_HOST_IPMB_BUSES = "1 2 3 4"

# IPMB slave addresses per host (7-bit format)
OBMC_HOST_IPMB_ADDRESSES = "0x70 0x72 0x74 0x76"

# Include multi-host support packages
OBMC_MACHINE_FEATURES += "obmc-host-state-mgmt obmc-chassis-state-mgmt"

# Enable per-host state manager instances
VIRTUAL-RUNTIME_obmc-host-state-manager = "phosphor-state-manager-host"
VIRTUAL-RUNTIME_obmc-chassis-state-manager = "phosphor-state-manager-chassis"
```

IPMB Bridge Daemon Configuration

The IPMB bridge daemon (`ipmbd`) runs one instance per host, each bound to a specific I2C bus. Configuration is typically done through systemd template units.

ipmbd Service Template

```
# /lib/systemd/system/ipmbd@.service
[Unit]
Description=IPMB Bridge Daemon for host %i
After=sys-subsystem-i2c-devices-i2c\x2d%i.device
BindsTo=sys-subsystem-i2c-devices-i2c\x2d%i.device

[Service]
Type=simple
ExecStart=/usr/bin/ipmbd %i 0x20
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Enable IPMB for Each Host

```
# Enable ipmbd for I2C buses 1-4 (corresponding to hosts 0-3)
systemctl enable ipmbd@1.service
systemctl enable ipmbd@2.service
systemctl enable ipmbd@3.service
systemctl enable ipmbd@4.service
```

IPMB Channel Configuration

Define IPMB channels in the channel configuration JSON. Each channel maps to an IPMB bus and host instance.


```

{
  "channels": [
    {
      "channel": 1,
      "type": "ipmb",
      "name": "IPMB_HOST0",
      "bus": 1,
      "slave_address": "0x70",
      "host_instance": 0
    },
    {
      "channel": 2,
      "type": "ipmb",
      "name": "IPMB_HOST1",
      "bus": 2,
      "slave_address": "0x72",
      "host_instance": 1
    },
    {
      "channel": 3,
      "type": "ipmb",
      "name": "IPMB_HOST2",
      "bus": 3,
      "slave_address": "0x74",
      "host_instance": 2
    },
    {
      "channel": 4,
      "type": "ipmb",
      "name": "IPMB_HOST3",
      "bus": 4,
      "slave_address": "0x76",
      "host_instance": 3
    }
  ]
}

```

I2C Device Tree Configuration

Each IPMB bus requires an I2C controller in the device tree. On AST2600, assign dedicated I2C controllers for each host.

```

/* IPMB I2C buses for multi-host */
&i2c1 {
    status = "okay";
    /* IPMB to Host 0 */
    multi-master;
    ipmb@20 {
        compatible = "ipmb-dev";
        reg = <0x20>;
    };
};

&i2c2 {
    status = "okay";
    /* IPMB to Host 1 */
    multi-master;
    ipmb@20 {
        compatible = "ipmb-dev";
        reg = <0x20>;
    };
};

&i2c3 {
    status = "okay";
    /* IPMB to Host 2 */
    multi-master;
    ipmb@20 {
        compatible = "ipmb-dev";
        reg = <0x20>;
    };
};

&i2c4 {
    status = "okay";
    /* IPMB to Host 3 */
    multi-master;
    ipmb@20 {
        compatible = "ipmb-dev";
        reg = <0x20>;
    };
};

```

The `multi-master` property enables I2C multi-master mode, which is required for IPMB because both the BMC and the host can initiate transactions on the same bus.

Per-Host State Manager Configuration

The phosphor-state-manager uses systemd template units to create independent state machines per host.

Host State Template

```
# /lib/systemd/system/phosphor-host-state-manager@.service
[Unit]
Description=Phosphor Host State Manager for host %i
Wants=mapper-wait@-xyz-openbmc_project-state-host%i.service
After=mapper-wait@-xyz-openbmc_project-state-host%i.service

[Service]
Type=dbus
BusName=xyz.openbmc_project.State.Host%i
ExecStart=/usr/bin/phosphor-host-state-manager --host %i
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Chassis State Template

```
# /lib/systemd/system/phosphor-chassis-state-manager@.service
[Unit]
Description=Phosphor Chassis State Manager for chassis %i

[Service]
Type=dbus
BusName=xyz.openbmc_project.State.Chassis%i
ExecStart=/usr/bin/phosphor-chassis-state-manager --chassis %i
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Enable Per-Host Services

```
# Enable state managers for all hosts
for i in 0 1 2 3; do
    systemctl enable phosphor-host-state-manager@${i}.service
    systemctl enable phosphor-chassis-state-manager@${i}.service
done
```

Build-Time Options

Option	Default	Description
<code>OBMC_HOST_INSTANCES</code>	<code>"0"</code>	Space-separated list of host instance numbers
<code>OBMC_HOST_IPMB_BUSSES</code>	<code>"1"</code>	I2C bus numbers for IPMB, one per host
<code>OBMC_HOST_IPMB_ADDRESSES</code>	<code>"0x70"</code>	7-bit I2C slave addresses for each host
<code>OBMC_CHASSIS_INSTANCES</code>	<code>"0"</code>	Chassis instance numbers (may differ from host count)
<code>OBMC_HOST_STATE_MANAGER_MULTI</code>	<code>"1"</code>	Set to number of hosts to enable multi-host state manager

Porting Guide

Follow these steps to enable multi-host support on your platform.

Step 1: Prerequisites

Ensure you have:

- ☐ Dedicated I2C buses from the BMC to each host (one per host)
- ☐ Per-host GPIO signals for power control (power button, reset, etc.)
- ☐ A working single-host OpenBMC image for your platform
- ☐ Device tree source for your baseboard

Step 2: Update Device Tree

Add I2C controller nodes for each IPMB bus and configure multi-master mode.

```
/* In your machine device tree overlay */
/* aspeed-bmc-mymachine.dts */

&i2c1 {
    status = "okay";
    multi-master;
    /* Host 0 IPMB */
};

&i2c2 {
    status = "okay";
    multi-master;
    /* Host 1 IPMB */
};

/* Add per-host power control GPIOs */
&gpio0 {
    host0-power-enable {
        gpio-hog;
        gpios = <ASPEED_GPIO(A, 0) GPIO_ACTIVE_HIGH>;
        output-low;
        line-name = "host0-power-enable";
    };

    host1-power-enable {
        gpio-hog;
        gpios = <ASPEED_GPIO(A, 1) GPIO_ACTIVE_HIGH>;
        output-low;
        line-name = "host1-power-enable";
    };
};
```

Step 3: Configure Machine Layer

Update your machine configuration to declare host instances and IPMB mappings.

```
# meta-mymachine/conf/machine/mymachine.conf

OBMC_HOST_INSTANCES = "0 1"

# Map host instances to I2C buses
OBMC_HOST_IPMB_BUSES = "1 2"
OBMC_HOST_IPMB_ADDRESSES = "0x70 0x72"

# Per-host power GPIO configuration
OBMC_POWER_GPIO_HOST0 = "host0-power-enable"
OBMC_POWER_GPIO_HOST1 = "host1-power-enable"
```

Step 4: Create Systemd Target Dependencies

Define per-host boot targets so that services start in the correct order for each host.

```
# meta-mymachine/recipes-phosphor/state/files/host0-boot.target
[Unit]
Description=Host 0 Boot Target
Wants=ipmbd@1.service
Wants=phosphor-host-state-manager@0.service
Wants=phosphor-chassis-state-manager@0.service
After=ipmbd@1.service
```

```
# meta-mymachine/recipes-phosphor/state/files/host1-boot.target
[Unit]
Description=Host 1 Boot Target
Wants=ipmbd@2.service
Wants=phosphor-host-state-manager@1.service
Wants=phosphor-chassis-state-manager@1.service
After=ipmbd@2.service
```

Step 5: Add IPMB Bridge Recipe Append

Create a bbappend to install per-host ipmbd service configurations.

```
# meta-mymachine/recipes-phosphor/ipmi/ipmb-bridge_%.bbappend

FILESEXTRAPATHS:prepend := "${THISDIR}/${PN}:"

SRC_URI += " \
    file://ipmb-channels.json \
"

do_install:append() {
    install -d ${D}${datadir}/ipmb-bridge
    install -m 0644 ${WORKDIR}/ipmb-channels.json \
        ${D}${datadir}/ipmb-bridge/
}
```

Step 6: Build and Verify

```
# Build the multi-host image
bitbake obmc-phosphor-image

# After flashing, verify all host state managers are running
systemctl status phosphor-host-state-manager@0
systemctl status phosphor-host-state-manager@1

# Verify IPMB bridges
systemctl status ipmbd@1
systemctl status ipmbd@2

# Check D-Bus objects
busctl tree xyz.openbmc_project.State.Host0
busctl tree xyz.openbmc_project.State.Host1
```

Code Examples

Example 1: Querying Per-Host State via D-Bus

```
# Get power state for each host
for i in 0 1 2 3; do
    echo "Host ${i} state:"
    busctl get-property xyz.openbmc_project.State.Host${i} \
        /xyz/openbmc_project/state/host${i} \
        xyz.openbmc_project.State.Host \
        CurrentHostState
done

# Example output:
# Host 0 state: s "xyz.openbmc_project.State.Host.HostState.Running"
# Host 1 state: s "xyz.openbmc_project.State.Host.HostState.Off"
# Host 2 state: s "xyz.openbmc_project.State.Host.HostState.Running"
# Host 3 state: s "xyz.openbmc_project.State.Host.HostState.Off"
```

Example 2: Power Control via Redfish

```
# Power on Host 1
curl -k -u root:OpenBmc -X POST \
  https://bmc-ip/redfish/v1/Systems/host1/Actions/ComputerSystem.Reset \
  -H "Content-Type: application/json" \
  -d '{"ResetType": "On"}'

# Graceful shutdown Host 0
curl -k -u root:OpenBmc -X POST \
  https://bmc-ip/redfish/v1/Systems/host0/Actions/ComputerSystem.Reset \
  -H "Content-Type: application/json" \
  -d '{"ResetType": "GracefulShutdown"}'

# Get all systems
curl -k -u root:OpenBmc \
  https://bmc-ip/redfish/v1/Systems/

# Example response (abbreviated):
# {
#   "@odata.id": "/redfish/v1/Systems",
#   "Members": [
#     {"@odata.id": "/redfish/v1/Systems/host0"},
#     {"@odata.id": "/redfish/v1/Systems/host1"},
#     {"@odata.id": "/redfish/v1/Systems/host2"},
#     {"@odata.id": "/redfish/v1/Systems/host3"}
#   ],
#   "Members@odata.count": 4
# }
```

Example 3: Per-Host IPMB Communication

```
# Send an IPMI Get Device ID command to Host 0 via IPMB channel 1
ipmitool -I ipmb -H localhost -t 0x70 -b 1 raw 0x06 0x01

# Send the same command to Host 1 via IPMB channel 2
ipmitool -I ipmb -H localhost -t 0x72 -b 2 raw 0x06 0x01

# Get chassis status for Host 0
ipmitool -I ipmb -H localhost -t 0x70 -b 1 chassis status

# Get sensor list from Host 1
ipmitool -I ipmb -H localhost -t 0x72 -b 2 sdr list
```

Example 4: Host State Monitor Script

```
#!/bin/bash
# monitor-hosts.sh - Monitor power state of all hosts
# See: docs/examples/multi-host/

HOSTS="0 1 2 3"
INTERVAL=5

while true; do
    clear
    echo "=== Multi-Host Status ($(date)) ==="
    echo ""
    for h in ${HOSTS}; do
        STATE=$(busctl get-property \
            xyz.openbmc_project.State.Host${h} \
            /xyz/openbmc_project/state/host${h} \
            xyz.openbmc_project.State.Host \
            CurrentHostState 2>/dev/null | awk '{print $2}' | tr -d ' "')

        CHASSIS=$(busctl get-property \
            xyz.openbmc_project.State.Chassis${h} \
            /xyz/openbmc_project/state/chassis${h} \
            xyz.openbmc_project.State.Chassis \
            CurrentPowerState 2>/dev/null | awk '{print $2}' | tr -d ' "')

        # Extract short state name
        HOST_SHORT="${STATE##*.}"
        CHASSIS_SHORT="${CHASSIS##*.}"

        printf " Host %-2d : %-12s Chassis: %-12s\n" \
            "${h}" "${HOST_SHORT:-Unknown}" "${CHASSIS_SHORT:-Unknown}"
    done
    echo ""
    echo "Refreshing every ${INTERVAL}s. Press Ctrl+C to stop."
    sleep ${INTERVAL}
done
```

Example 5: Per-Host Sensor Namespace

```
# List sensors for a specific host
busctl tree xyz.openbmc_project.HostSensors${HOST_ID}

# Read Host 0 CPU temperature
busctl get-property xyz.openbmc_project.Sensor.Host0 \
    /xyz/openbmc_project/sensors/temperature/host0_cpu_temp \
    xyz.openbmc_project.Sensor.Value Value

# Read Host 1 DIMM temperature
busctl get-property xyz.openbmc_project.Sensor.Host1 \
    /xyz/openbmc_project/sensors/temperature/host1_dimm0_temp \
    xyz.openbmc_project.Sensor.Value Value

# Via Redfish - Host 0 thermal sensors
curl -k -u root:0penBmc \
    https://bmc-ip/redfish/v1/Chassis/chassis0/Thermal

# Via Redfish - Host 1 thermal sensors
curl -k -u root:0penBmc \
    https://bmc-ip/redfish/v1/Chassis/chassis1/Thermal
```


See working examples in the [examples/multi-host](#) directory:

- `monitor-hosts.sh` - Monitor power state across all hosts
 - `ipmb-health-check.sh` - Verify IPMB connectivity to each host
 - `multi-host-power-cycle.sh` - Sequential power cycle all hosts
 - `config/ipmb-channels.json` - IPMB channel configuration example
 - `config/multi-host.conf` - Machine configuration snippet
-

Satellite Processor Approach (OpenBIC)

An alternative to direct IPMB-per-host is using **satellite processors** — small Bridge ICs (BICs) placed on each server board that act as local BMC agents. The open-source [OpenBIC](#) project provides firmware for these Bridge ICs, enabling a single BMC to manage many hosts by delegating per-host monitoring and control to dedicated satellite processors.

What is a Bridge IC (BIC)?

A Bridge IC is a small microcontroller (typically an ASPEED AST1030) deployed on each server sled or server board. It runs OpenBIC firmware (based on Zephyr RTOS) and provides:

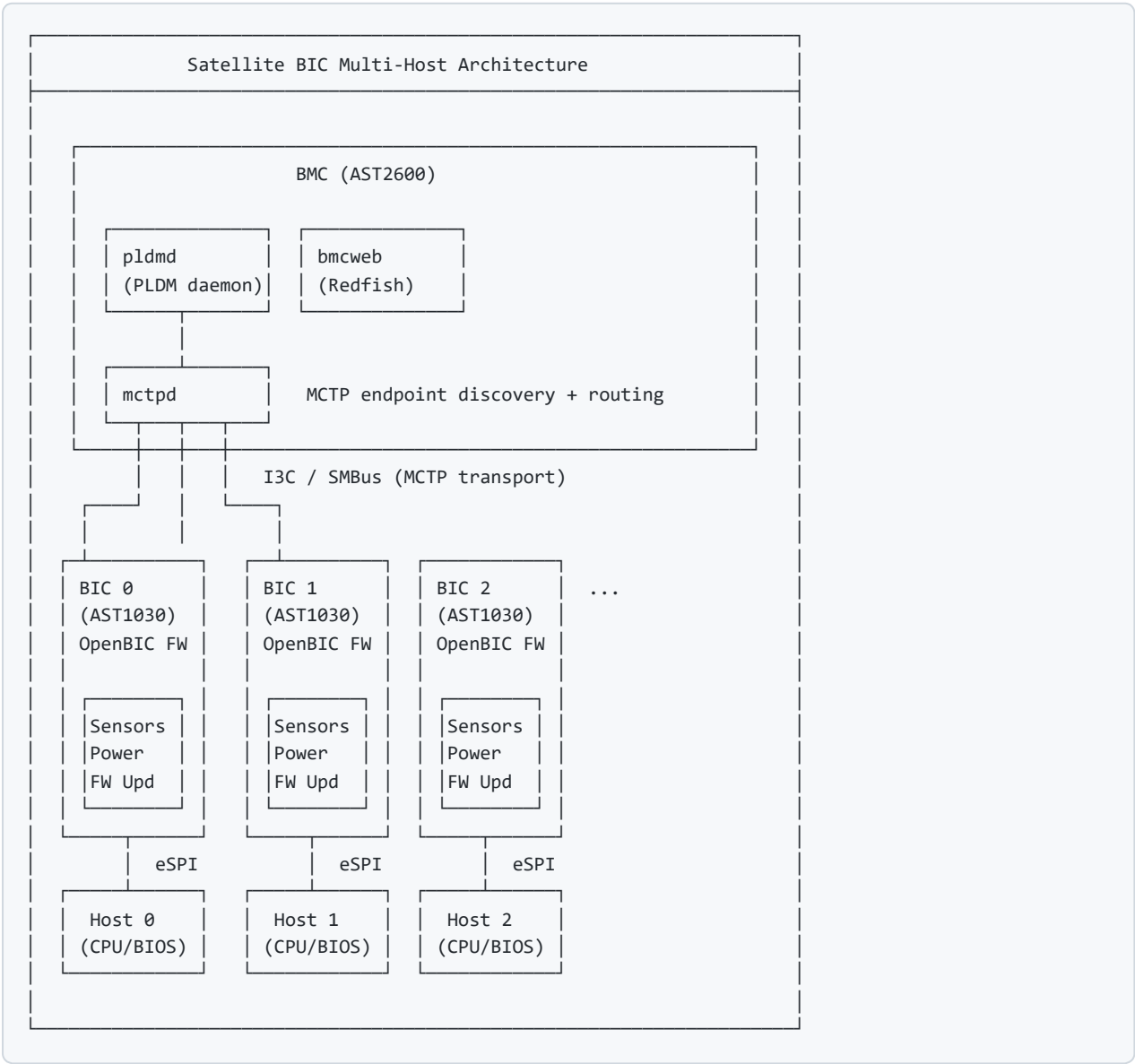
- **Local sensor monitoring** — reads host CPU temperatures, voltages, and power sensors directly, reducing I2C traffic to the BMC
- **Event logging** — captures and buffers host events locally before forwarding to the BMC
- **Host power sequencing** — manages local power-on/off sequences for its host
- **Firmware update agent** — handles local component firmware updates (BIOS, CPLD, etc.)
- **Protocol bridging** — translates between the host's eSPI/LPC interface and the BMC's management bus

The BIC acts as a local agent of the BMC, sharing the BMC's management burden in complex multi-host architectures.

BIC vs Direct IPMB Architecture

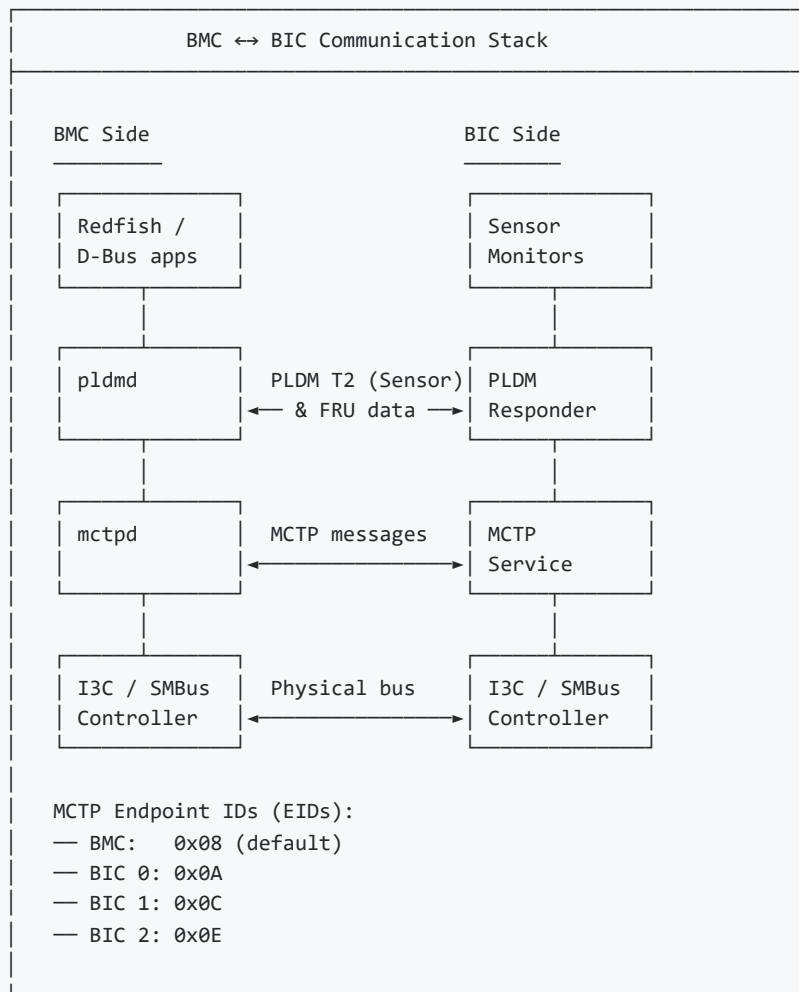
Aspect	Direct IPMB (Traditional)	Satellite BIC (OpenBIC)
Host connection	BMC connects directly to each host via I2C/IPMB	BIC on each sled connects to its host; BMC talks to BICs
Scalability	Limited by BMC I2C bus count	Scales to many hosts — each BIC handles its own host
Sensor polling	BMC polls all sensors for all hosts	BIC polls local sensors, BMC queries BIC for aggregated data
Protocol	IPMB (IPMI over I2C)	MCTP over I3C or SMBus, with PLDM for sensor/FRU data
Failure isolation	BMC I2C hang affects all hosts on that bus	BIC failure only affects its local host
Hardware cost	Lower (no extra IC)	Higher (one AST1030 per sled)
Firmware complexity	Simpler — single firmware image	Two firmware stacks: OpenBMC on BMC + OpenBIC on each BIC
Example platforms	Traditional twin-server boards	Meta Yosemite v3/v3.5/v4, OCP multi-node sleds

Satellite Processor Topology



Communication Stack

In the BIC architecture, the BMC communicates with each Bridge IC using **MCTP** (Management Component Transport Protocol) over **I3C** or **SMBus**, with **PLDM** (Platform Level Data Model) as the application-layer protocol for sensor data, FRU information, and firmware updates.



PLDM Sensor Data Flow

Each BIC monitors local sensors and exposes them to the BMC via PLDM Type 2 (Monitoring and Control). The BMC's `pldmd` daemon discovers BIC sensors through Platform Descriptor Records (PDRs) and makes them available on D-Bus for bmcweb/Redfish.

```
# On the BMC – discover PLDM endpoints
pldmtool platform GetPDR -d 0 # List PDRs from terminus 0 (BIC 0)

# Read a sensor from a BIC via PLDM
pldmtool platform GetSensorReading -i <sensor_id> -t <terminus_id>

# Sensors from BICs appear on D-Bus like any other sensor
busctl get-property xyz.openbmc_project.PLDM \
  /xyz/openbmc_project/sensors/temperature/BIC_0_CPU_Temp \
  xyz.openbmc_project.Sensor.Value Value
```

OpenBIC Hardware: ASPEED AST1030

The [AST1030](#) is ASPEED's purpose-built BIC chip:

Feature	AST1030
CPU	Arm Cortex-M4F
Memory	Internal flash and SRAM
Host interface	eSPI
BMC interface	I2C, I3C, SMBus
USB	USB 1.1 device
RTOS	Zephyr (via OpenBIC)
Package	256-pin TFBGA (13mm x 13mm)

Reference Platform: Meta Yosemite v3.5

The [Yosemite v3.5](#) is a well-known OCP platform that uses the BIC architecture:

- **Baseboard:** AST2600 BMC running OpenBMC
- **4 server slots:** Each slot has an AST1030 BIC running OpenBIC
- **Form factor:** Sled that fits into a 40U chassis (3 sleds per chassis = 12 hosts per chassis)
- **QEMU support:** The `fbby35` machine is emulated in QEMU for development

```
# Run Yosemite v3.5 in QEMU (BMC + one BIC)
qemu-system-arm -machine fby35 \
  -drive file=obmc-bmc.mtd,format=raw,if=mtd \
  -drive file=obmc-bic.mtd,format=raw,if=mtd \
  -nographic
```

When to Choose Each Approach

Use direct IPMB when:

- You have 2-4 hosts in a simple twin-server or blade chassis
- Your platform uses traditional IPMI and you need backward compatibility
- Hardware cost is a primary constraint
- You already have a working single-host OpenBMC and just need to multiply it

Use satellite BICs (OpenBIC) when:

- You have 4+ hosts per BMC and need to scale management
- You need per-host failure isolation (a hung sensor poll on one host must not affect others)
- Your platform uses modern MCTP/PLDM protocols
- You want local firmware update capability on each sled
- You are building an OCP-compliant multi-node server platform

The two approaches are not mutually exclusive. Some platforms use BICs for server sleds while still using IPMB for legacy expansion cards or chassis management controllers within the same system.

Troubleshooting

Issue: IPMB Bridge Fails to Start

Symptom: `ipmbd@1.service` fails with "No such device" or "Permission denied."

Cause: The I2C bus device node does not exist, or the device tree does not enable the I2C controller.

Solution:

1. Verify the I2C bus exists:

```
ls /dev/i2c-*
i2cdetect -l
```

2. Check the device tree enables the controller:

```
# Look for I2C status in device tree
cat /sys/firmware/devicetree/base/ahb/apb/i2c@1e78a080/status
```

3. Ensure `multi-master` is set in the device tree for IPMB buses.

Issue: Host State Manager Shows "Unknown" State

Symptom: `busctl get-property` returns no value or an error for a host instance.

Cause: The state manager template unit is not instantiated for that host number, or `OBMC_HOST_INSTANCES` does not include the host.

Solution:

1. Check whether the service is running:

```
systemctl status phosphor-host-state-manager@1
```

2. Verify `OBMC_HOST_INSTANCES` in your machine config includes the host.
3. Ensure the D-Bus service file matches the host instance number:

```
busctl list | grep State.Host
```

Issue: IPMB Messages Time Out

Symptom: IPMI commands sent over IPMB return timeout errors.

Cause: I2C bus contention, incorrect slave address, or the host-side IPMB responder is not running.

Solution:

1. Verify the target address is correct with `i2cdetect`:

```
i2cdetect -y 1 # Check bus 1 for Host 0
```

2. Check for I2C errors in the kernel log:

```
dmesg | grep -i i2c
```

3. Confirm the host-side IPMB controller is active (requires host console access).
4. Try reducing I2C clock speed if the bus length is long:

```
&i2c1 {  
    clock-frequency = <100000>; /* 100 kHz standard mode */  
};
```

Issue: Redfish Shows Only One System

Symptom: `/redfish/v1/Systems/` returns only `host0` even though multiple hosts are configured.

Cause: bmcweb only exposes ComputerSystem resources for hosts whose state managers are registered on D-Bus.

Solution:

1. Verify all host state managers are active:

```
busctl list | grep State.Host
```

2. Check bmcweb logs for discovery issues:

```
journalctl -u bmcweb -f
```

3. Ensure `OBMC_HOST_INSTANCES` is correctly set and the image was rebuilt with the multi-host configuration.

Issue: Per-Host Sensors Not Appearing

Symptom: Sensors for Host 1 or higher-numbered hosts are missing from D-Bus and Redfish.

Cause: Sensor configuration files are not templated for multiple hosts, or the sensor daemon is not host-aware.

Solution:

1. Check sensor configuration includes per-host entries:

```
ls /usr/share/entity-manager/configurations/ | grep -i host
```

2. Verify entity-manager detected the host:

```
busctl tree xyz.openbmc_project.EntityManager
```

3. For IPMB-sourced sensors, confirm the IPMB bridge for that host is operational.

Debug Commands

```
# Check all state manager services
systemctl list-units 'phosphor-*-state-manager@*'

# Check all IPMB bridge services
systemctl list-units 'ipmbd@*'

# View logs for a specific host state manager
journalctl -u phosphor-host-state-manager@1 -f

# View IPMB bridge logs
journalctl -u ipmbd@2 -f

# List all host-related D-Bus objects
busctl tree xyz.openbmc_project.State.Host0
busctl tree xyz.openbmc_project.State.Host1

# Monitor D-Bus signals for host state changes
busctl monitor xyz.openbmc_project.State.Host0

# Test I2C connectivity to a specific host
i2ctransfer -y 1 w1@0x70 0x00
```

References

Official Resources

- [phosphor-state-manager \(GitHub\)](#) - Host and chassis state management
- [ipmb-bridge \(GitHub\)](#) - IPMB message routing daemon
- [phosphor-host-ipmid \(GitHub\)](#) - Host-side IPMI daemon
- [bmcweb \(GitHub\)](#) - Redfish/REST API server
- [D-Bus Interface Definitions](#) - State interface YAML definitions

OpenBIC / Satellite Processor Resources

- [OpenBIC \(GitHub\)](#) - Open-source Bridge IC firmware framework
- [OpenBIC Documentation](#) - API and MCTP service documentation
- [ASPEED BIC Products](#) - AST1030/AST1035 Bridge IC hardware
- [Yosemite v3.5 QEMU Emulation](#) - BMC + BIC emulation for development

Related Guides

- [MCTP & PLDM Guide](#) - Alternative multi-endpoint management via PLDM
- [Logging Guide](#) - Per-host event logging and SEL

External Documentation

- [IPMI Specification v2.0](#) - IPMB protocol specification
- [DMTF Redfish Specification](#) - ComputerSystem and Manager resource definitions
- [systemd Template Units](#) - Template unit documentation (`@.service`)

Tested on: OpenBMC master branch with AST2600-EVB, dual-host configuration. QEMU can simulate the D-Bus and systemd aspects but does not provide physical IPMB I2C buses. Use QEMU for state manager and Redfish validation; use hardware for end-to-end IPMB testing.

Performance Optimization Guide

Profile and optimize boot time, memory usage, and runtime performance on resource-constrained BMC hardware.

Table of Contents

1. [Overview](#)
2. [BMC Resource Constraints](#)
 1. [Hardware Budgets by SoC Generation](#)
 2. [Typical Memory Budget](#)
 3. [Per-Service Memory Targets](#)
3. [Boot Time Profiling](#)
 1. [systemd-analyze and Critical Chain](#)
 2. [Blame Analysis](#)
 3. [SVG Boot Chart and Kernel Time](#)
 4. [Automating Boot Time Tracking](#)
4. [Runtime Memory Profiling](#)
 1. [Per-Process RSS Tracking](#)
 2. [Detailed Memory Maps with smaps](#)
 3. [Tracking Memory Over Time](#)
 4. [Detecting Memory Leaks](#)
5. [Optimization Strategies](#)
 1. [Strategy 1: Lazy Initialization](#)
 2. [Strategy 2: D-Bus Communication Patterns](#)
 3. [Strategy 3: Python-to-C++ Migration](#)
 4. [Strategy 4: Image Size Reduction](#)
 5. [Strategy 5: Reduce Service Count](#)
6. [Kernel Configuration Optimization](#)
 1. [Disabling Debug Options](#)
 2. [Disabling Tracing](#)
 3. [Removing Unused Drivers](#)
 4. [Applying Kernel Config Fragments](#)
 5. [Optimization Impact Summary](#)
7. [Optimization Workflow](#)
8. [Troubleshooting](#)
 1. [Issue: Boot Time Regressed After Image Update](#)
 2. [Issue: Memory Usage Grows Over Time](#)
 3. [Issue: Flash Image Exceeds Partition Size](#)
 4. [Issue: systemd-analyze Shows Inaccurate Times](#)
9. [Quick Reference](#)

- 1. [Boot Time Commands](#)
- 2. [Memory Commands](#)
- 3. [Image Size Commands](#)

10. [References](#)

Overview

BMC systems-on-chip (SoCs) operate under tight resource constraints. A typical AST2600 provides a dual-core ARM Cortex-A7 at 800 MHz with 512 MB–1 GB of DDR4 RAM and 32–128 MB of SPI NOR flash. Every megabyte of RAM and every second of boot time matters because the BMC must be operational before the host system can power on.

This guide walks you through profiling boot time, tracking runtime memory consumption, and applying concrete optimization strategies to keep your OpenBMC image within budget.

Key concepts covered:

- Hardware resource budgets and how to allocate them across services
- Boot time profiling with `systemd-analyze` and critical-chain analysis
- Runtime memory profiling with `/proc/smaps`, RSS tracking, and leak detection
- Optimization strategies: lazy initialization, D-Bus batching, Python-to-C++ migration
- Kernel configuration tuning to reduce image size and memory footprint

Optimization work should be guided by measurements, not assumptions. Always profile before and after each change to verify the impact.

BMC Resource Constraints

Hardware Budgets by SoC Generation

Resource	AST2500 (G5)	AST2600 (G6)	Notes
CPU	ARM1176 single-core 800 MHz	Cortex-A7 dual-core 800 MHz	G6 adds SMP
RAM	256–512 MB DDR3	512 MB–1 GB DDR4	Shared with video engine
SPI Flash	32–64 MB	32–128 MB	Holds kernel + rootfs + U-Boot
Root FS	~20 MB (typical)	~30 MB (typical)	Compressed squashfs or UBI

Typical Memory Budget

On a running AST2600 system with a standard OpenBMC image:

Typical RAM Usage (~512 MB system)	
Kernel + modules + slab caches	~60--80 MB
Core BMC services	~80--120 MB
└─ bmcweb (Redfish)	~15--25 MB RSS
└─ entity-manager	~8--15 MB RSS
└─ ipmid	~10--15 MB RSS
└─ phosphor-logging	~8--12 MB RSS
└─ Other services (~20 daemons)	~40--55 MB RSS
systemd + journald + tmpfs	~25--50 MB
Free / available	~260--350 MB

These numbers are approximate and vary by image configuration, number of sensors, and host platform complexity. Always measure your specific build.

Per-Service Memory Targets

Service Category	Target RSS	Concern Threshold
Small daemon (state-manager, led-manager)	< 10 MB	> 15 MB
Medium daemon (entity-manager, ipmid)	< 15 MB	> 25 MB
Large daemon (bmcweb, phosphor-logging)	< 25 MB	> 40 MB
Python services (if any)	< 20 MB	> 30 MB

Boot Time Profiling

A BMC that boots slowly delays host power-on and extends maintenance windows. Reaching the Redfish endpoint within 30–60 seconds of power application is a common target.

systemd-analyze and Critical Chain

```
# Total boot time breakdown
systemd-analyze
# Startup finished in 3.512s (kernel) + 18.234s (userspace) = 21.746s

# Show the critical boot path (longest dependency chain)
systemd-analyze critical-chain
# multi-user.target @18.102s
# └─phosphor-bmc-state-ready.target @18.050s
#   └─mapper-wait@-xyz-openbmc_project-state-bmc.service @17.892s +145ms
#     └─xyz.openbmc_project.State.BMC.service @15.234s +2.651s
#       └─phosphor-mapper.service @12.891s +2.310s
#         └─dbus.service @3.456s +120ms
```

Each line shows the service, its activation timestamp (@), and how long it took to start (+).

Blame Analysis

```
# Show services ordered by startup time
systemd-analyze blame
# 4.210s phosphor-read-eeeprom@...service
# 3.891s entity-manager.service
# 2.651s xyz.openbmc_project.State.BMC.service
# 2.310s phosphor-mapper.service
# 1.892s bmcweb.service
```

Focus on the top 5–10 services in the blame list. Optimizing a service that takes 100 ms has negligible impact compared to one that takes 4 seconds.

SVG Boot Chart and Kernel Time

```
# Generate visual boot chart (copy to host for viewing)
systemd-analyze plot > /tmp/boot-chart.svg
scp -P 2222 root@localhost:/tmp/boot-chart.svg .

# Measure kernel boot time from dmesg timestamps
dmesg | grep "Run /sbin/init"
# [ 3.456789] Run /sbin/init as init process
```

Automating Boot Time Tracking

```
#!/bin/bash
# boot-metrics.sh - Run on the BMC after each boot
echo "=== Boot Metrics $(date) ==="
systemd-analyze
echo "--- Critical Chain ---"
systemd-analyze critical-chain --no-pager
echo "--- Top 15 Blame ---"
systemd-analyze blame --no-pager | head -15
echo "--- Memory ---"
free -m
```

Runtime Memory Profiling

Per-Process RSS Tracking

Resident Set Size (RSS) measures the actual physical memory a process occupies:

```
# List all processes sorted by RSS (descending)
ps aux --sort=-rss | head -20

# Track RSS of key OpenBMC services
for svc in bmcweb phosphor-log-manager entity-manager ipmid; do
    pid=$(pidof $svc 2>/dev/null)
    if [ -n "$pid" ]; then
        rss=$(awk '/VmRSS/{print $2}' /proc/$pid/status)
        printf "%-30s PID=%-6s RSS=%s kB\n" "$svc" "$pid" "$rss"
    fi
done
```

Detailed Memory Maps with smaps

The `/proc/<pid>/smaps_rollup` file distinguishes shared libraries from private heap:

```
pid=$(pidof bmcweb)
cat /proc/$pid/smaps_rollup
# Rss:                18432 kB
# Pss:                12288 kB    <-- Proportional share (accounts for sharing)
# Private_Dirty:      8704 kB    <-- Heap + private data (primary target)
# Shared_Clean:       8192 kB    <-- Shared library text
```

Field	Meaning	Optimization Target
<code>Rss</code>	Total physical memory	Reduce overall footprint
<code>Pss</code>	Proportional share	More accurate for shared libs
<code>Private_Dirty</code>	Heap + writable private data	Primary optimization target
<code>Shared_Clean</code>	Shared library code pages	Reduce by linking fewer libs

Tracking Memory Over Time

Monitor a service for memory growth that might indicate a leak:

```
#!/bin/bash
# mem-track.sh <process-name> <interval-seconds> <count>
PROC=$1; INTERVAL=${2:-10}; COUNT=${3:-60}
echo "timestamp,pid,rss_kb,vsz_kb"
for i in $(seq 1 $COUNT); do
    pid=$(pidof $PROC 2>/dev/null)
    if [ -n "$pid" ]; then
        rss=$(awk '/VmRSS/{print $2}' /proc/$pid/status)
        vsz=$(awk '/VmSize/{print $2}' /proc/$pid/status)
        echo "$(date +%s),$pid,$rss,$vsz"
    fi
    sleep $INTERVAL
done
```

A steadily increasing RSS over hours or days strongly suggests a memory leak. Occasional small increases are normal due to caching, but monotonic growth requires investigation.

Detecting Memory Leaks

For leak detection without recompiling, use Valgrind:

```
systemctl stop bmcweb
valgrind --tool=memcheck --leak-check=full \
    --show-leak-kinds=definite,possible \
    --log-file=/tmp/valgrind-bmcweb.log \
    /usr/bin/bmcweb &

# Exercise the service, then stop and examine
kill %1
cat /tmp/valgrind-bmcweb.log | grep "definitely lost"
```

For compile-time leak detection, use AddressSanitizer. See the [Linux Debug Tools Guide](#) for details.

Optimization Strategies

Strategy 1: Lazy Initialization

Many services load all data at startup. Lazy initialization defers work until first use.

Before (eager):

```
SensorManager(sdbusplus::bus_t& bus) : bus_(bus) {
    loadAllSensorConfigs();           // 200ms
    initializeAllThresholds();        // 150ms
    registerAllDbusObjects();         // 300ms
}
```

After (lazy):

```
SensorManager(sdbusplus::bus_t& bus) : bus_(bus) {
    registerMinimalDbusInterface(); // 50ms
}
void ensureInitialized(const std::string& sensorPath) {
    if (initialized_.count(sensorPath) == 0) {
        loadSensorConfig(sensorPath);
        initializeThreshold(sensorPath);
        initialized_.insert(sensorPath);
    }
}
```

Impact: Reduces service startup time. The trade-off is a small latency on first access to each resource.

Strategy 2: D-Bus Communication Patterns

D-Bus round trips are expensive on BMC hardware. Each call involves serialization, context switches, and deserialization.

Batch property reads:

```
// Bad: N individual GetObject calls
for (const auto& path : sensorPaths) {
    bus.call(bus.new_method_call("xyz.openbmc_project.ObjectMapper", ...));
}
// Better: Single GetSubTree call
auto subtree = mapper::getSubTree(bus, "/xyz/openbmc_project/sensors",
                                   interfaces, depth);
```

Use PropertiesChanged signals instead of polling:

```
// Bad: Polling every second
while (running) { value = readPropertyFromDbus(path, "Value"); sleep(1); }

// Better: React to changes
sdbusplus::bus::match_t match(bus,
    sdbusplus::bus::match::rules::propertiesChanged(
        sensorPath, "xyz.openbmc_project.Sensor.Value"),
    [](sdbusplus::message_t& msg) { /* handle change */ });
```

Strategy 3: Python-to-C++ Migration

Python services consume significantly more resources than equivalent C++ implementations:

Metric	Python Service	C++ Equivalent	Improvement
RSS at idle	15–25 MB	3–8 MB	3–5x less RAM
Startup time	1–3 s	0.1–0.5 s	3–10x faster
CPU per D-Bus call	~2 ms	~0.2 ms	~10x faster

Migration is a significant effort. Target services that run persistently and handle high-frequency events. One-time scripts are fine to leave in Python.

Strategy 4: Image Size Reduction

```
# In local.conf or your distro configuration
IMAGE_FEATURES:remove = "debug-tweaks tools-debug tools-profile"
INHIBIT_PACKAGE_STRIP = "0"
DEBUG_BUILD = "0"
FULL_OPTIMIZATION = "-O2 -pipe"

# Disable optional features in large services
EXTRA_OEMESON:append:pn-bmcweb = " -Dredfish-dbus-log=disabled"
```

Strategy 5: Reduce Service Count

Each daemon consumes baseline RSS for its runtime, D-Bus connection, and event loop. Remove services you do not need:

Service	When to Remove	Savings
<code>phosphor-ipmi-host</code>	No IPMI over KCS/BT required	~10–15 MB
<code>phosphor-ipmi-net</code>	No IPMI-over-LAN required	~8–12 MB
<code>obmc-ikvm</code>	No remote KVM feature	~5–10 MB
<code>phosphor-certificate-manager</code>	No TLS certificate management	~5–8 MB
<code>phosphor-post-code-manager</code>	No POST code logging	~4–6 MB

```
# Remove IPMI if using Redfish only
IMAGE_INSTALL:remove = "phosphor-ipmi-host phosphor-ipmi-net ipmitool"
```

Kernel Configuration Optimization

The default AST2600 defconfig includes many features useful for development but unnecessary in production.

Disabling Debug Options

```
# production-strip-debug.cfg

# CONFIG_DEBUG_KERNEL is not set
# CONFIG_DEBUG_INFO is not set
# CONFIG_DEBUG_FS is not set
# CONFIG_PROVE_LOCKING is not set
# CONFIG_DEBUG_LOCK_ALLOC is not set
# CONFIG_DEBUG_MUTEXES is not set
# CONFIG_KASAN is not set
# CONFIG_KMEMLEAK is not set
# CONFIG_SLUB_DEBUG is not set
```

Disabling Tracing

```
# production-strip-tracing.cfg

# CONFIG_FTRACE is not set
# CONFIG_FUNCTION_TRACER is not set
# CONFIG_DYNAMIC_FTRACE is not set
# CONFIG_KPROBES is not set
# CONFIG_UPROBES is not set
# CONFIG_PROFILING is not set
# CONFIG_PERF_EVENTS is not set
```

Removing Unused Drivers

```
# production-minimal-drivers.cfg

# CONFIG_NET_VENDOR_INTEL is not set
# CONFIG_NET_VENDOR_REALTEK is not set
# CONFIG_EXT4_FS is not set           # If using squashfs + tmpfs only
# CONFIG_BTRFS_FS is not set
# CONFIG_INPUT_MOUSEDEV is not set
# CONFIG_USB_STORAGE is not set
# CONFIG_SOUND is not set
```

Be conservative when removing drivers. Test thoroughly on your actual hardware after each change. A missing driver for an on-board device will cause silent failures.

Applying Kernel Config Fragments

```
# meta-vendor/recipes-kernel/linux/linux-aspeed_%.bbappend
FILESEXTRAPATHS:prepend := "${THISDIR}/${PN}:"
SRC_URI += " \
    file://production-strip-debug.cfg \
    file://production-strip-tracing.cfg \
    file://production-minimal-drivers.cfg \
"
```

```
# Validate all options took effect
bitbake linux-aspeed -c kernel_configcheck -f
```

Optimization Impact Summary

Category	Options Disabled	Image Savings	RAM Savings
Debug infrastructure	<code>DEBUG_KERNEL</code> , <code>DEBUG_INFO</code> , <code>DEBUG_FS</code>	2–5 MB	10–30 MB
Tracing and profiling	<code>FTRACE</code> , <code>KPROBES</code> , <code>PERF_EVENTS</code>	1–3 MB	5–15 MB
Unused drivers	Platform-specific	0.5–2 MB	1–5 MB
Unused filesystems	Platform-specific	0.2–1 MB	0.5–2 MB
Total potential		4–11 MB	17–52 MB

Optimization Workflow

Follow this systematic process to avoid wasted effort:

Optimization Workflow

1. MEASURE

systemd-analyze / free -m / df -h

2. IDENTIFY

critical-chain / blame / RSS ranking

3. OPTIMIZE

Lazy init, D-Bus batching, remove services,
kernel config stripping, Python -> C++ migration

4. VERIFY

Re-measure, regression test, document impact

5. REPEAT until targets are met

Keep a log of your optimization results. Record the build commit, the change made, and before/after measurements. This prevents regressions and helps justify changes during code review.

Troubleshooting

Issue: Boot Time Regressed After Image Update

Symptom: `systemd-analyze` shows significantly longer boot time after updating packages.

Cause: A new or updated service on the critical boot path, or new `Before=` / `Wants=` dependencies.

Solution:

1. Compare `systemd-analyze blame` output before and after
2. Run `systemd-analyze critical-chain` to find the new bottleneck

3. Inspect dependencies: `systemctl show <service> -p After -p Wants -p Before`

Issue: Memory Usage Grows Over Time

Symptom: `free -m` shows available memory steadily decreasing over days.

Cause: Memory leak in a userspace service, or unbounded caching.

Solution:

1. Identify the growing process: `ps aux --sort=-rss | head -10`
2. Track with the `mem-track.sh` script from the profiling section
3. Check log entry counts: `busctl tree xyz.openbmc_project.Logging | wc -l`
4. Run the suspect under Valgrind (see [Linux Debug Tools Guide](#))

Issue: Flash Image Exceeds Partition Size

Symptom: Build fails with “image too large” or BMC fails to boot after flashing.

Cause: Added packages or debug symbols pushed rootfs beyond the flash partition limit.

Solution:

1. Examine rootfs: `du -sh tmp/work/*/obmc-phosphor-image/*/rootfs/*/ | sort -rh | head -20`
2. Remove unnecessary packages from `IMAGE_INSTALL`
3. Ensure `DEBUG_BUILD = "0"` and symbols are stripped

Issue: systemd-analyze Shows Inaccurate Times

Symptom: Reported boot times do not match wall-clock observations.

Cause: `systemd-analyze` measures from kernel handoff to init, not from power-on. It does not account for U-Boot or firmware initialization.

Solution: For total power-on-to-ready time, use serial log timestamps. Add kernel boot time (`dmesg` first timestamp to init) plus userspace time for the complete picture.

Quick Reference

Boot Time Commands

Task	Command
Total boot time	<code>systemd-analyze</code>
Critical boot path	<code>systemd-analyze critical-chain</code>
Services by start time	<code>systemd-analyze blame</code>
Visual boot chart	<code>systemd-analyze plot > /tmp/boot.svg</code>
Kernel boot time	<code>dmesg \ grep "Run /sbin/init"</code>

Memory Commands

Task	Command
System memory overview	<code>free -m</code>
Per-process RSS	<code>ps aux --sort=-rss \\\ head -20</code>
Detailed memory map	<code>cat /proc/<pid>/smaps_rollup</code>
Kernel slab usage	<code>cat /proc/slabinfo \\\ sort -k3 -rn \\\ head</code>

Image Size Commands

Task	Command
Flash partition usage	<code>df -h</code> (on running BMC)
Kernel image size	<code>ls -lh tmp/deploy/images/*/zImage*</code>
Package count	<code>opkg list-installed \\\ wc -l</code> (on running BMC)

References

- [systemd-analyze man page](#) – Boot time analysis tool
- [OpenBMC Development Environment](#) – Development environment reference
- [Yocto Image Size Optimization](#) – Build quality and size optimization
- [Linux Kernel Size Tuning](#) – Embedded Linux kernel tuning
- [sdbusplus Repository](#) – C++ D-Bus bindings used in OpenBMC
- [Systemd Boot Ordering](#) – Prerequisite guide on systemd dependencies
- [Linux Debug Tools](#) – Sanitizers and Valgrind for leak detection

Tested on: OpenBMC master with QEMU ast2600-evb. Memory figures measured on a representative AST2600 build. Actual values vary by image configuration and platform.

BIOS Firmware Management

Manage host BIOS/UEFI firmware on OpenBMC – flash updates via SPI with GPIO mux control, BIOS configuration through biosconfig_manager, and multi-host firmware coordination.

Table of Contents

1. [Overview](#)
2. [Architecture](#)
 1. [BIOS Flash Update Flow](#)
 2. [BIOS Configuration Flow](#)
 3. [D-Bus Interfaces](#)
 4. [Key Dependencies](#)
3. [BIOS Flash Update](#)
 1. [SPI Flash Access with GPIO Mux](#)
 1. [GPIO Mux Configuration](#)
 2. [obmc-flash-bios Service](#)
 2. [Redfish UpdateService](#)
 3. [Build-Time Configuration](#)
4. [BIOS Configuration with biosconfig_manager](#)
 1. [How It Works](#)
 2. [D-Bus Operations](#)
 1. [Reading Current BIOS Attributes](#)
 2. [Setting Pending Attributes](#)
 3. [Redfish Bios Resource](#)
 4. [BIOS Configuration JSON Schema](#)
5. [PLDM-Based BIOS Configuration \(DSP0247\)](#)
 1. [How PLDM BIOS Configuration Works](#)
 2. [The Three PLDM BIOS Tables](#)
 3. [PLDM BIOS Boot Sequence](#)
 4. [PLDM BIOS Attribute JSON Configuration](#)
 5. [Using pldmtool for BIOS Operations](#)
 6. [PLDM vs IPMI for BIOS Configuration](#)
 7. [Troubleshooting PLDM BIOS](#)
6. [Multi-Host Firmware Management](#)
 1. [Multi-Host D-Bus Layout](#)
 2. [Redfish Multi-Host Endpoints](#)
 3. [Multi-Host Build Configuration](#)
7. [Porting Guide](#)
 1. [Step 1: Prerequisites](#)
 2. [Step 2: Configure Device Tree](#)

3. [Step 3: Create Flash Scripts](#)
 4. [Step 4: Enable biosconfig_manager](#)
 5. [Step 5: Verify](#)
 8. [Troubleshooting](#)
 1. [Issue: BIOS Flash Fails with "Device Busy"](#)
 2. [Issue: Pending Attributes Not Applied After Reboot](#)
 3. [Issue: Redfish Returns 404 for /Systems/system/Bios](#)
 4. [Debug Commands](#)
 9. [References](#)
 1. [Official Resources](#)
 2. [Related Guides](#)
 3. [External Documentation](#)
-

Overview

OpenBMC provides a complete framework for managing host BIOS firmware from the BMC. This includes flashing BIOS images to the host SPI flash, reading and modifying BIOS configuration settings remotely, and coordinating firmware updates across multi-host platforms.

The BMC accesses host BIOS flash through an SPI connection controlled by a GPIO multiplexer. When the host is powered off, the BMC asserts GPIO lines to route the SPI bus to itself, performs the flash operation, then releases the bus back to the host. This out-of-band access enables firmware recovery even when the host is unbootable.

BIOS configuration management uses the `biosconfig_manager` service, which exposes BIOS settings through D-Bus and the Redfish `Bios` resource. Administrators can read current settings, stage changes as pending attributes, and apply them on the next host boot – all without physically accessing the machine or entering the BIOS setup menu.

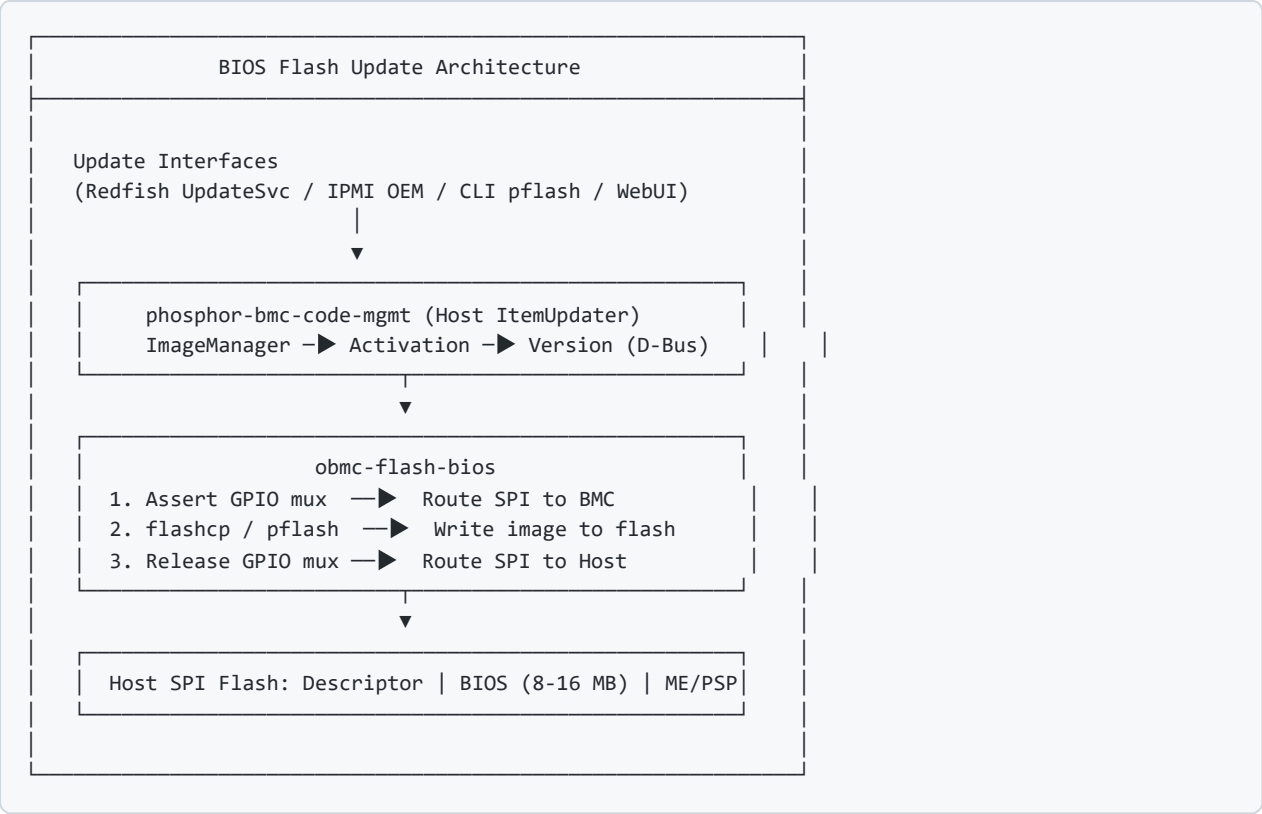
Key concepts covered:

- SPI flash access with GPIO mux switching (obmc-flash-bios)
 - Redfish UpdateService for BIOS image uploads
 - biosconfig_manager for remote BIOS configuration
 - PLDM Type 3 (DSP0247) BIOS Control and Configuration — BIOS tables, pldmtool, attribute JSON
 - Pending attributes and apply-on-next-boot workflow
 - Multi-host firmware management patterns
-

Architecture

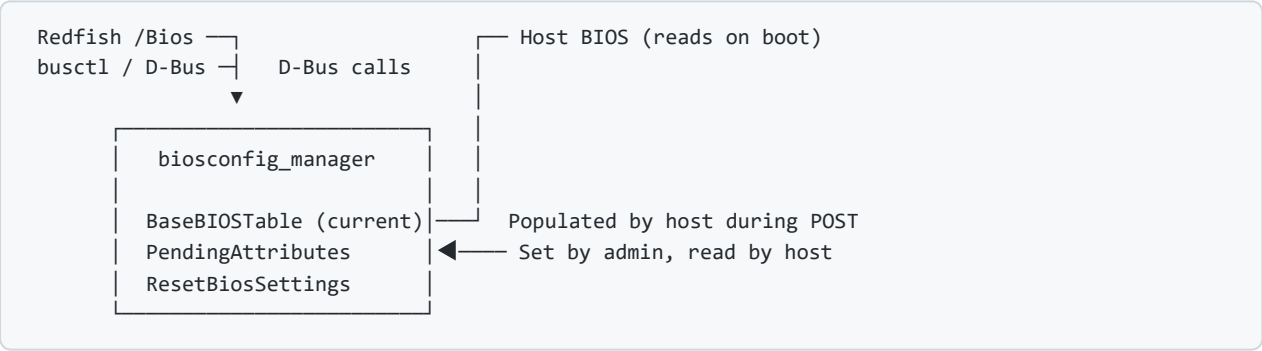
BIOS Flash Update Flow

The BMC updates host BIOS firmware by taking control of the SPI bus through a GPIO multiplexer, writing the image to flash, and releasing the bus back to the host processor.



BIOS Configuration Flow

The `biosconfig_manager` stores BIOS attributes on D-Bus and exposes them through Redfish. Pending changes are applied by the host BIOS on the next boot cycle.



D-Bus Interfaces

Interface	Object Path	Description
<code>xyz.openbmc_project.BIOSConfig.Manager</code>	<code>/xyz/openbmc_project/bios_config/manager</code>	BIOS configuration management
<code>xyz.openbmc_project.Software.Version</code>	<code>/xyz/openbmc_project/software/<id></code>	BIOS firmware version tracking
<code>xyz.openbmc_project.Software.Activation</code>	<code>/xyz/openbmc_project/software/<id></code>	BIOS update activation control

Key Dependencies

- phosphor-bmc-code-mgmt:** Core firmware management with Host ItemUpdater for BIOS flash operations.

- **obmc-flash-bios**: Shell scripts and systemd services that control GPIO mux switching and invoke flash tools.
- **biosconfig_manager**: D-Bus service that stores BIOS attributes and exposes them through the Redfish Bios resource.
- **bmcweb**: Redfish server that maps `UpdateService` and `Systems/{id}/Bios` endpoints to D-Bus calls.

BIOS Flash Update

SPI Flash Access with GPIO Mux

The BMC and host processor share access to the host SPI flash through a hardware multiplexer. The BMC controls which device owns the SPI bus by asserting or deasserting GPIO lines.

Never flash BIOS while the host is running. The host must be powered off before the BMC takes control of the SPI bus. Flashing while the host is active corrupts the running firmware and can damage the flash device.

The refactored device-updater framework provides a `bios` updater (`BIOSSoftwareManager` / `SPIDevice`) configured through the Entity Manager interface `xyz.openbmc_project.Configuration.SPIFlash` (keys `SPIControllerIndex` , `SPIDeviceIndex` , mux outputs, `Layout`). It powers the host off, binds the `spi-aspeed-smc` / `spi-nor` drivers, then writes the image with one of `flashrom` (for Intel IFD layouts), `flashcp` , or a flat MTD write, and supports a `--dryrun` flag to exercise the flow without touching hardware. See the `/openbmc-guide-tutorial/docs/05-advanced/23-device-firmware-update-guide.html` for the shared updater framework. (Verify against your phosphor-bmc-code-mgmt version.)

GPIO Mux Configuration

Define the GPIO lines and flash device in your machine layer's device tree:

```
/* AST2600 device tree: GPIO mux for host SPI */
gpio_bios_mux: bios-mux {
    compatible = "gpio-mux";
    gpios = <&gpio0 ASPEED_GPIO(AA, 0) GPIO_ACTIVE_HIGH>;
    /* HIGH = BMC owns SPI, LOW = Host owns SPI */
};

&fmc {
    status = "okay";
    flash@1 {
        /* Host BIOS SPI flash behind mux */
        status = "okay";
        spi-max-frequency = <50000000>;
    };
};
```

obmc-flash-bios Service

The `obmc-flash-bios` systemd service manages the GPIO switching and flash operations. The script follows a strict sequence: verify host power state, assert the mux, write flash, release the mux.


```
#!/bin/bash
# obmc-flash-bios -- Flash BIOS image to host SPI
BIOS_MTD="/dev/mtd/bios"
GPIO_MUX="/sys/class/gpio/gpio<N>/value"

# Verify host is powered off
host_state=$(busctl get-property xyz.openbmc_project.State.Host \
    /xyz/openbmc_project/state/host0 \
    xyz.openbmc_project.State.Host CurrentHostState | awk '{print $2}')
[ "$host_state" != '"xyz.openbmc_project.State.Host.HostState.Off"' ] && exit 1

echo 1 > "$GPIO_MUX"          # Assert GPIO mux -- route SPI to BMC
sleep 0.5
flashcp -v "$1" "$BIOS_MTD" # Flash the BIOS image
rc=$?
echo 0 > "$GPIO_MUX"          # Release GPIO mux -- route SPI to Host
exit $rc
```

Redfish UpdateService

Upload a BIOS image through the Redfish UpdateService endpoint. The BMC validates the image, triggers the flash operation, and tracks progress through a task resource.

```
# Upload BIOS firmware image via Redfish
curl -k -u root:openBmc \
    -X POST \
    -H "Content-Type: application/octet-stream" \
    --data-binary @bios-firmware.bin \
    https://${BMC_IP}/redfish/v1/UpdateService
```

For large BIOS images (16 MB+), use the [MultipartHttpPushUri](#) endpoint with a target to avoid HTTP timeout issues.

```
# Multipart upload targeting the BIOS resource
curl -k -u root:openBmc \
    -X POST \
    -F 'UpdateParameters={"Targets":["/redfish/v1/Systems/system/Bios"]};type=application/json' \
    -F 'UpdateFile=@bios-firmware.bin;type=application/octet-stream' \
    https://${BMC_IP}/redfish/v1/UpdateService/update

# Monitor update progress
curl -k -u root:openBmc \
    https://${BMC_IP}/redfish/v1/TaskService/Tasks/0

# Verify BIOS firmware inventory after update
curl -k -u root:openBmc \
    https://${BMC_IP}/redfish/v1/UpdateService/FirmwareInventory/bios
```

Build-Time Configuration

Enable host BIOS upgrade support in your Yocto configuration:

```
# In your machine .conf or local.conf
EXTRA_OEMESON:pn-phosphor-bmc-code-mgmt += " \
    -Dhost-bios-upgrade=enabled \
    "

IMAGE_INSTALL:append = " \
    phosphor-software-manager \
    obmc-flash-bios \
    "
```

BIOS Configuration with biosconfig_manager

The `biosconfig_manager` service provides remote BIOS configuration management. It stores BIOS attribute definitions and their current values on the BMC, allows administrators to stage pending changes, and signals the host firmware to apply them on the next boot.

How It Works

1. The host BIOS populates the BMC with its current attribute table (via IPMI or PLDM) during boot.
2. The BMC stores these as `BaseBIOSTable` attributes on D-Bus.
3. An administrator sets `PendingAttributes` through Redfish or D-Bus.
4. On the next host boot, the BIOS reads pending attributes from the BMC and applies them.
5. The BIOS reports updated current values back to the BMC.

The `biosconfig_manager` does not flash BIOS firmware. It manages BIOS *settings* (boot order, memory timing, virtualization, security features). For BIOS *image* updates, use the UpdateService workflow described above.

D-Bus Operations

Reading Current BIOS Attributes

```
# List the BaseBIOSTable (all current BIOS attributes)
busctl get-property xyz.openbmc_project.BIOSConfigManager \
    /xyz/openbmc_project/bios_config/manager \
    xyz.openbmc_project.BIOSConfig.Manager \
    BaseBIOSTable
```

Each `BaseBIOSTable` entry contains:

Field	Type	Description
<code>AttributeType</code>	string	<code>Enumeration</code> , <code>String</code> , <code>Integer</code> , <code>Boolean</code>
<code>ReadOnly</code>	boolean	Whether the attribute can be modified
<code>DisplayName</code>	string	Human-readable name
<code>CurrentValue</code>	variant	Current setting value
<code>DefaultValue</code>	variant	Factory default value
<code>Options</code>	array	Valid values for enumeration types

Setting Pending Attributes

```
# Stage a BIOS attribute change (applied on next boot)
busctl set-property xyz.openbmc_project.BIOSConfigManager \
  /xyz/openbmc_project/bios_config/manager \
  xyz.openbmc_project.BIOSConfig.Manager \
  PendingAttributes a{s\{sv\}} 2 \
  "HyperThreading" "s" "Enabled" \
  "VTx" "s" "Enabled"

# Reset all BIOS settings to factory defaults
busctl call xyz.openbmc_project.BIOSConfigManager \
  /xyz/openbmc_project/bios_config/manager \
  xyz.openbmc_project.BIOSConfig.Manager \
  ResetBiosSettings s \
  "xyz.openbmc_project.BIOSConfig.Manager.ResetFlag.FactoryDefaults"
```

Redfish Bios Resource

The Redfish `Bios` resource at `/redfish/v1/Systems/system/Bios` maps directly to the `biosconfig_manager` D-Bus interface.

```
# Read current BIOS attributes
curl -k -u root:openBmc \
  https://${BMC_IP}/redfish/v1/Systems/system/Bios

# Stage pending BIOS attribute changes (applied on next boot)
curl -k -u root:openBmc \
  -X PATCH \
  -H "Content-Type: application/json" \
  -d '{
    "Attributes": {
      "BootOrder": "PXE,HDD,USB",
      "QuietBoot": "Disabled"
    }
  }' \
  https://${BMC_IP}/redfish/v1/Systems/system/Bios/Settings

# Reset all BIOS settings to factory defaults
curl -k -u root:openBmc \
  -X POST \
  -H "Content-Type: application/json" \
  -d '{"ResetType": "ResetAll"}' \
  https://${BMC_IP}/redfish/v1/Systems/system/Bios/Actions/Bios.ResetBios
```

After staging pending attributes, reboot the host to apply changes. The BIOS reads pending attributes from the BMC during POST and clears them after successful application.

BIOS Configuration JSON Schema

Define your platform's BIOS attributes in a JSON file that `biosconfig_manager` loads at startup:

```
{
  "BaseBIOSTable": {
    "BootOrder": {
      "AttributeType": "Enumeration",
      "ReadOnly": false,
      "DisplayName": "Boot Order",
      "CurrentValue": "HDD,PXE,USB",
      "DefaultValue": "HDD,PXE,USB",
      "Options": ["HDD,PXE,USB", "PXE,HDD,USB", "USB,HDD,PXE"]
    },
    "HyperThreading": {
      "AttributeType": "Enumeration",
      "ReadOnly": false,
      "DisplayName": "Hyper-Threading",
      "CurrentValue": "Enabled",
      "DefaultValue": "Enabled",
      "Options": ["Enabled", "Disabled"]
    }
  }
}
```

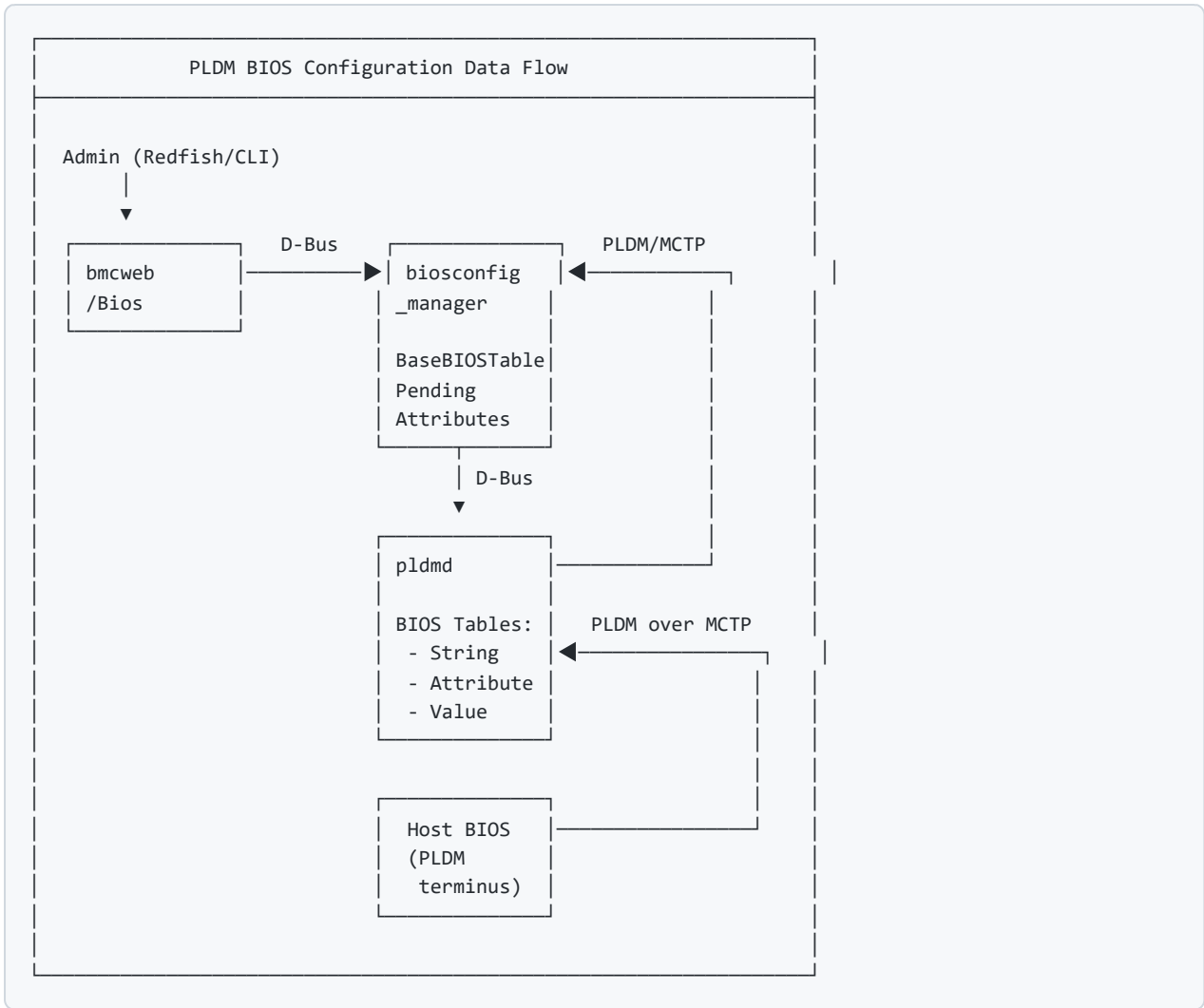
See the complete example at [examples/bios-config/](#).

PLDM-Based BIOS Configuration (DSP0247)

For platforms that use PLDM (Platform Level Data Model) between the BMC and host, BIOS configuration is handled through **PLDM Type 3 — BIOS Control and Configuration**, defined in DMTF specification [DSP0247](#). This is the modern alternative to IPMI-based BIOS attribute exchange and is used on IBM POWER, ARM, and other PLDM-enabled platforms.

How PLDM BIOS Configuration Works

The host BIOS and the BMC's `pldmd` daemon exchange BIOS attributes using three PLDM BIOS tables and a set of PLDM commands. The `pldmd` daemon acts as the bridge between the host and the `biosconfig_manager` D-Bus service.



The Three PLDM BIOS Tables

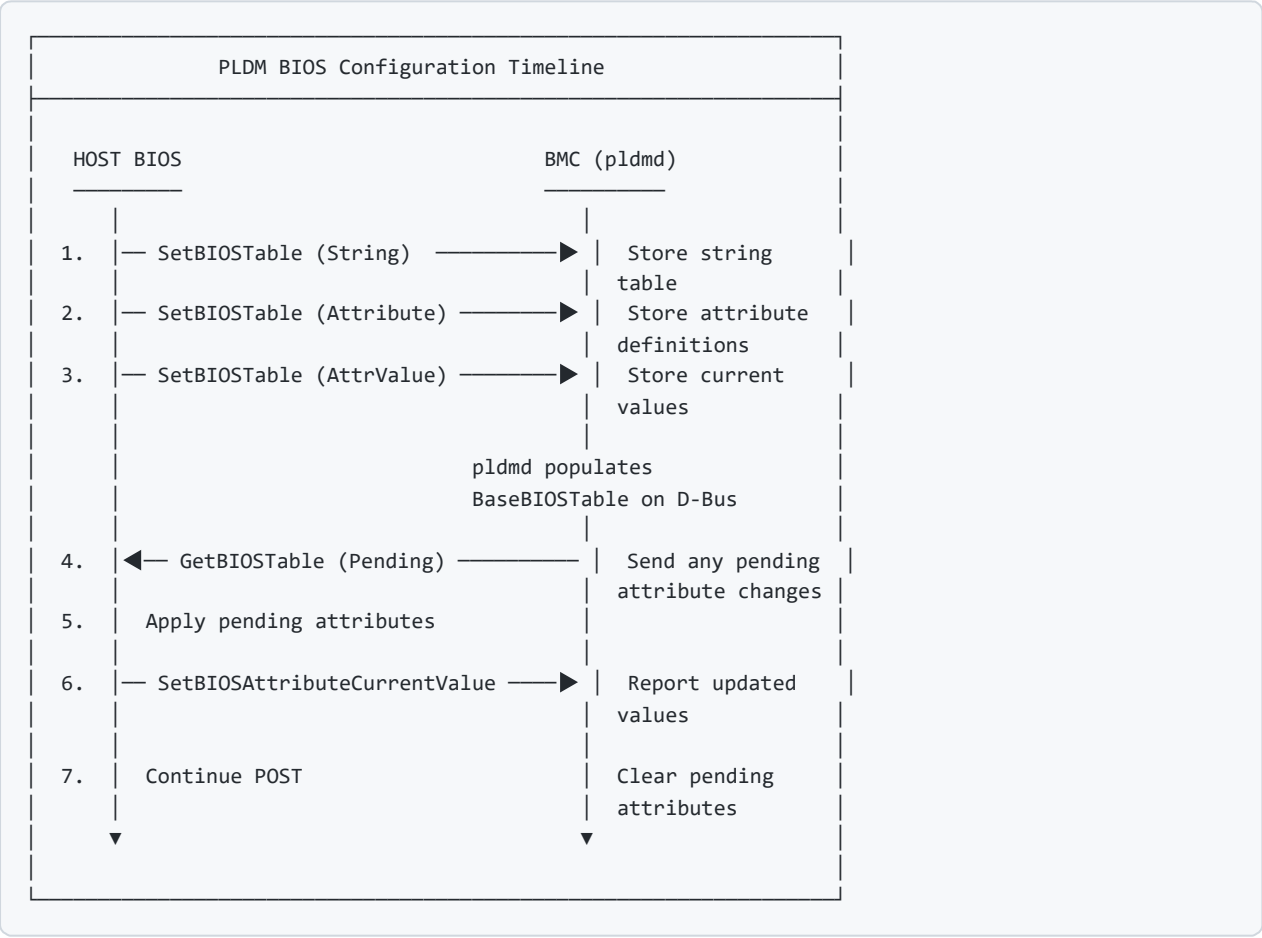
PLDM BIOS uses three interrelated tables to represent all BIOS configuration data:

Table	PLDM Table Type	Purpose
String Table	0	Dictionary of all strings used by attributes (names, option labels, help text)
Attribute Table	1	Schema for each attribute: type (enum/integer/string), constraints, default values
Attribute Value Table	2	Current runtime values for each attribute

These tables are exchanged between the host BIOS and the BMC using PLDM commands:

PLDM Command	Direction	Description
<code>GetBIOSTable</code>	BMC → Host	BMC requests a BIOS table from the host
<code>SetBIOSTable</code>	Host → BMC	Host sends a complete BIOS table to the BMC
<code>GetBIOSAttributeCurrentValueByHandle</code>	BMC → Host	Read a single attribute's current value
<code>SetBIOSAttributeCurrentValue</code>	Host → BMC or BMC → Host	Update a single attribute value

PLDM BIOS Boot Sequence



PLDM BIOS Attribute JSON Configuration

For PLDM-based systems, OEM vendors define BIOS attributes in JSON files that `pldmd` parses to initialize the `BaseBIOSTable`. These files define the attribute schema — `pldmd` converts them to PLDM BIOS table format.

```
{
  "entries": [
    {
      "attribute_type": "enum",
      "attribute_name": "HyperThreading",
      "possible_values": ["Enabled", "Disabled"],
      "default_values": ["Enabled"],
      "display_name": "Hyper-Threading Technology",
      "help_text": "Enable or disable CPU Hyper-Threading",
      "read_only": false,
      "dbus": {
        "object_path": "/xyz/openbmc_project/bios_config/manager",
        "interface": "xyz.openbmc_project.BIOSConfig.Manager",
        "property_name": "HyperThreading",
        "property_type": "string"
      }
    },
    {
      "attribute_type": "integer",
      "attribute_name": "MemoryFrequency",
      "lower_bound": 1600,
      "upper_bound": 4800,
      "scalar_increment": 400,
      "default_value": 3200,
      "display_name": "Memory Operating Frequency (MHz)",
      "help_text": "Set DDR memory operating frequency",
      "read_only": false
    },
    {
      "attribute_type": "string",
      "attribute_name": "AssetTag",
      "string_type": "ASCII",
      "minimum_string_length": 0,
      "maximum_string_length": 64,
      "default_string": "",
      "display_name": "System Asset Tag",
      "read_only": false
    }
  ]
}
```

Install this file in your machine layer:

```
# meta-<machine>/recipes-phosphor/pldm/pldm_%.bbappend
FILESEXTRAPATHS:prepend := "${THISDIR}/${PN}:"
SRC_URI += "file://bios_attrs.json"

do_install:append() {
  install -d ${D}${datadir}/pldm/bios
  install -m 0644 ${WORKDIR}/bios_attrs.json \
    ${D}${datadir}/pldm/bios/
}
```

Using pldmtool for BIOS Operations

The `pldmtool` CLI on the BMC provides direct access to PLDM BIOS commands for debugging and testing.

```
# Get the BIOS String Table (type 0)
pldmttool bios GetBIOSTable -t 0

# Get the BIOS Attribute Table (type 1) -- shows attribute definitions
pldmttool bios GetBIOSTable -t 1

# Get the BIOS Attribute Value Table (type 2) -- shows current values
pldmttool bios GetBIOSTable -t 2

# Get a single attribute's current value by handle
pldmttool bios GetBIOSAttributeCurrentValueByHandle -a HyperThreading

# Set a BIOS attribute value
pldmttool bios SetBIOSAttributeCurrentValue -a HyperThreading -d Disabled
```

Use `pldmttool bios GetBIOSTable -t 1` to discover all available BIOS attributes and their valid values. This is the PLDM equivalent of viewing the BIOS setup menu.

PLDM vs IPMI for BIOS Configuration

Aspect	IPMI (Traditional)	PLDM (Modern)
Transport	IPMB / KCS / BT	MCTP over I3C, SMBus, or PCIe VDM
Specification	Vendor-specific OEM commands	DMTF DSP0247 (standardized)
Attribute discovery	Not standardized — requires vendor docs	Self-describing via PLDM BIOS tables
Data model	Flat key-value	Typed attributes (enum, int, string) with constraints
Pending attributes	Vendor-dependent	Standardized pending table exchange
Security	No built-in auth	SPDM integration possible over MCTP
Platforms	Legacy x86, older BMC designs	IBM POWER, ARM SBMR, OCP servers

Both IPMI and PLDM ultimately populate the same `biosconfig_manager` D-Bus interface. Redfish consumers do not need to know which protocol the host uses — the Redfish `/Systems/system/Bios` endpoint works identically regardless of the underlying transport.

Troubleshooting PLDM BIOS

BIOS tables not populated after host boot:

```
# Check if pldmd discovered the host as a PLDM terminus
pldmttool base GetTID

# Check if BIOS tables were received
journalctl -u pldmd -f | grep -i bios

# Verify MCTP connectivity to the host
busctl tree xyz.openbmc_project.MCTP
```

Pending attributes not reaching the host:


```
# Verify pending attributes are set on D-Bus
busctl get-property xyz.openbmc_project.BIOSConfigManager \
    /xyz/openbmc_project/bios_config/manager \
    xyz.openbmc_project.BIOSConfig.Manager \
    PendingAttributes

# Check pldmd logs for pending attribute transfer
journalctl -u pldmd --no-pager | grep -i pending
```

Attribute type mismatch errors: Ensure your `bios_attrs.json` matches the host BIOS attribute types exactly. An enum attribute on the host cannot be set with an integer value from the BMC. Use `pldmdtool bios GetBIOSTable -t 1` to verify attribute definitions.

Multi-Host Firmware Management

In multi-host platforms (blade servers, multi-node chassis), the BMC manages BIOS firmware for multiple host processors independently. Each host has its own SPI flash, GPIO mux, and D-Bus object path.

Multi-Host D-Bus Layout

Each host gets a separate D-Bus object path for both firmware management and BIOS configuration:

Host	Firmware Object	BIOSConfig Object
Host 0	<code>/xyz/openbmc_project/software/host0_bios</code>	<code>/xyz/openbmc_project/bios_config/manager0</code>
Host 1	<code>/xyz/openbmc_project/software/host1_bios</code>	<code>/xyz/openbmc_project/bios_config/manager1</code>
Host N	<code>/xyz/openbmc_project/software/hostN_bios</code>	<code>/xyz/openbmc_project/bios_config/managerN</code>

Redfish Multi-Host Endpoints

```
# Access BIOS settings for a specific host
curl -k -u root:openBmc \
    https://${BMC_IP}/redfish/v1/Systems/system0/Bios
curl -k -u root:openBmc \
    https://${BMC_IP}/redfish/v1/Systems/system1/Bios

# Update BIOS firmware for a specific host
curl -k -u root:openBmc \
    -X POST \
    -F 'UpdateParameters={"Targets":["/redfish/v1/Systems/system1/Bios"]};type=application/json' \
    -F 'UpdateFile=@bios-firmware.bin;type=application/octet-stream' \
    https://${BMC_IP}/redfish/v1/UpdateService/update
```

Multi-Host Build Configuration

```
# In your machine .conf for multi-host support
OBMC_HOST_INSTANCES = "0 1 2 3"
OBMC_HOST_BIOS_FLASH_INSTANCES = "0 1 2 3"
BIOS_MUX_GPIO_host0 = "AA0"
BIOS_MUX_GPIO_host1 = "AA1"
BIOS_MUX_GPIO_host2 = "AA2"
BIOS_MUX_GPIO_host3 = "AA3"
```

Never flash two hosts simultaneously if they share a SPI controller. The GPIO mux ensures only one host flash is accessible at a time, so your update orchestration must serialize flash operations.

Porting Guide

Follow these steps to enable BIOS firmware management on your platform.

Step 1: Prerequisites

Ensure you have:

- ☐ A working OpenBMC build for your platform
- ☐ GPIO mux hardware between BMC SPI controller and host SPI flash
- ☐ Knowledge of your platform's GPIO pin assignments for the mux
- ☐ Host BIOS that supports BMC-based configuration (IPMI or PLDM interface)

Step 2: Configure Device Tree

Add SPI flash and GPIO mux definitions to your machine device tree (`meta-<machine>/recipes-kernel/linux/files/<machine>.dts`). Enable `flash@1` under `&fmc` for the host BIOS SPI flash behind the GPIO mux, as shown in the GPIO Mux Configuration section above.

Step 3: Create Flash Scripts

Add the `obmc-flash-bios` script and systemd service to your machine layer:

```
# meta-<machine>/recipes-phosphor/flash/obmc-flash-bios.bbappend
FILESEXTRAPATHS:prepend := "${THISDIR}/${PN}:"
SRC_URI += "file://obmc-flash-bios file://obmc-flash-bios@.service"
do_install:append() {
    install -d ${D}${sbindir}
    install -m 0755 ${WORKDIR}/obmc-flash-bios ${D}${sbindir}/
    install -d ${D}${systemd_system_unitdir}
    install -m 0644 ${WORKDIR}/obmc-flash-bios@.service ${D}${systemd_system_unitdir}/
}
```

Step 4: Enable biosconfig_manager

```
# meta-<machine>/recipes-phosphor/bios/biosconfig-manager.bbappend
FILESEXTRAPATHS:prepend := "${THISDIR}/${PN}:"
SRC_URI += "file://bios-attrs.json"
do_install:append() {
    install -d ${D}${datadir}/biosconfig
    install -m 0644 ${WORKDIR}/bios-attrs.json ${D}${datadir}/biosconfig/
}
```

Step 5: Verify

1. Build the image: `bitbake obmc-phosphor-image`
2. Check services: `systemctl status xyz.openbmc_project.biosconfig_manager.service`
3. Verify D-Bus: `busctl tree xyz.openbmc_project.BIOSConfigManager`

4. Test Redfish: `curl -k -u root:OpenBmc https://localhost/redfish/v1/Systems/system/Bios`

Troubleshooting

Issue: BIOS Flash Fails with “Device Busy”

Symptom: `flashcp` returns “Device or resource busy”.

Cause: GPIO mux not asserted, or another process holds the MTD device.

Solution: Verify the host is off (`obmcutil state`), check the GPIO mux value (`cat /sys/class/gpio/gpio<N>/value`) should be `1`), and check for other MTD users (`fuser /dev/mtd/bios`).

Issue: Pending Attributes Not Applied After Reboot

Symptom: Changes staged via `/Bios/Settings` remain pending after host reboot.

Cause: Host BIOS does not support the BMC attribute interface, or the IPMI/PLDM channel is misconfigured.

Solution: Verify pending attributes are stored on D-Bus (`busctl get-property ... PendingAttributes`), confirm the host BIOS supports the attribute protocol (consult vendor docs), and check `biosconfig_manager` logs (`journalctl -u xyz.openbmc_project.biosconfig_manager.service -f`).

Issue: Redfish Returns 404 for /Systems/system/Bios

Symptom: The Bios resource endpoint returns “ResourceNotFound”.

Cause: `biosconfig_manager` is not running or has not populated the `BaseBIOSTable` .

Solution: Check the service status (`systemctl status xyz.openbmc_project.biosconfig_manager.service`), verify the D-Bus object exists (`busctl introspect xyz.openbmc_project.BIOSConfigManager /xyz/openbmc_project/bios_config/manager`), and ensure the BIOS attribute JSON file is installed at the expected path.

Debug Commands

```
systemctl status obmc-flash-bios@0.service           # Flash service
journalctl -u xyz.openbmc_project.biosconfig_manager.service --no-pager # Config logs
busctl tree xyz.openbmc_project.BIOSConfigManager    # D-Bus objects
gpioget gpiochip0 <line_number>                     # GPIO mux state
cat /proc/mtd                                         # MTD partitions
obmcutil state                                        # Host power state
```

References

Official Resources

- [phosphor-bmc-code-mgmt Repository](#)
- [biosconfig_manager Repository](#)
- [D-Bus BIOSConfig Interface](#)
- [OpenBMC Host Firmware Update Documentation](#)

Related Guides

- [Firmware Update Guide](#)
- [Secure Boot & Image Signing](#)
- [MCTP & PLDM Guide](#) - PLDM transport and sensor monitoring

External Documentation

- [DMTF Redfish Bios Resource \(DSP0268\)](#)
- [PLDM for BIOS Control and Configuration \(DSP0247\)](#)
- [ASPEED AST2600 SPI Controller Documentation](#)
- [OpenBMC Remote BIOS Configuration Design](#)

Tested on: QEMU ast2600-evb, OpenBMC commit [497ca5d](#) Last updated: 2026-02-06

Debug Dump Collection

Collect, manage, and extend BMC diagnostic dumps using phosphor-debug-collector and the dreport framework.

Table of Contents

1. [Overview](#)
2. [Architecture](#)
 1. [Dump Collection Architecture](#)
 2. [Dump Types](#)
 3. [D-Bus Interfaces](#)
 4. [Key Dependencies](#)
3. [Setup & Configuration](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [Meson Build Options](#)
 3. [Runtime Verification](#)
4. [BMC Dump Collection](#)
 1. [Via Redfish DumpService](#)
 1. [Create a BMC Dump](#)
 2. [Poll for Completion](#)
 3. [List Available Dumps](#)
 4. [Download a Dump](#)
 5. [Delete a Dump](#)
 2. [Via D-Bus](#)
 1. [Create a BMC Dump](#)
 2. [Monitor Dump Progress](#)
 3. [List Dump Entries](#)
 4. [Delete a Dump Entry](#)
 3. [Via Command Line \(dreport\)](#)
 4. [Dump Contents](#)
5. [Custom dreport Plugins](#)
 1. [Plugin API](#)
 1. [Plugin Naming Convention](#)
 2. [Plugin Script Structure](#)
 2. [dreport Utility Functions](#)
 3. [Writing a Custom Plugin](#)
 4. [Installing Custom Plugins via Yocto](#)
 5. [Testing Plugins](#)
6. [Dump Storage Management](#)
 1. [Storage Limits](#)

2. [Retention Behavior](#)
 3. [Monitoring Storage Usage](#)
 4. [Customizing Limits](#)
 5. [Manual Cleanup](#)
 7. [Troubleshooting](#)
 1. [Issue: Dump Creation Fails with “No Space”](#)
 2. [Issue: Dump Manager Service Not Running](#)
 3. [Issue: Custom Plugin Not Executing](#)
 4. [Issue: Dump Download Returns Empty or Corrupt File](#)
 5. [Debug Commands](#)
 8. [Examples](#)
 9. [References](#)
 1. [Official Resources](#)
 2. [Related Guides](#)
 3. [External Documentation](#)
-

Overview

When a BMC daemon crashes, a host fails to boot, or sensors report unexpected values, you need a way to capture the full system state for offline analysis. OpenBMC provides **phosphor-debug-collector**, a dump management service that orchestrates diagnostic data collection through the **dreport** framework.

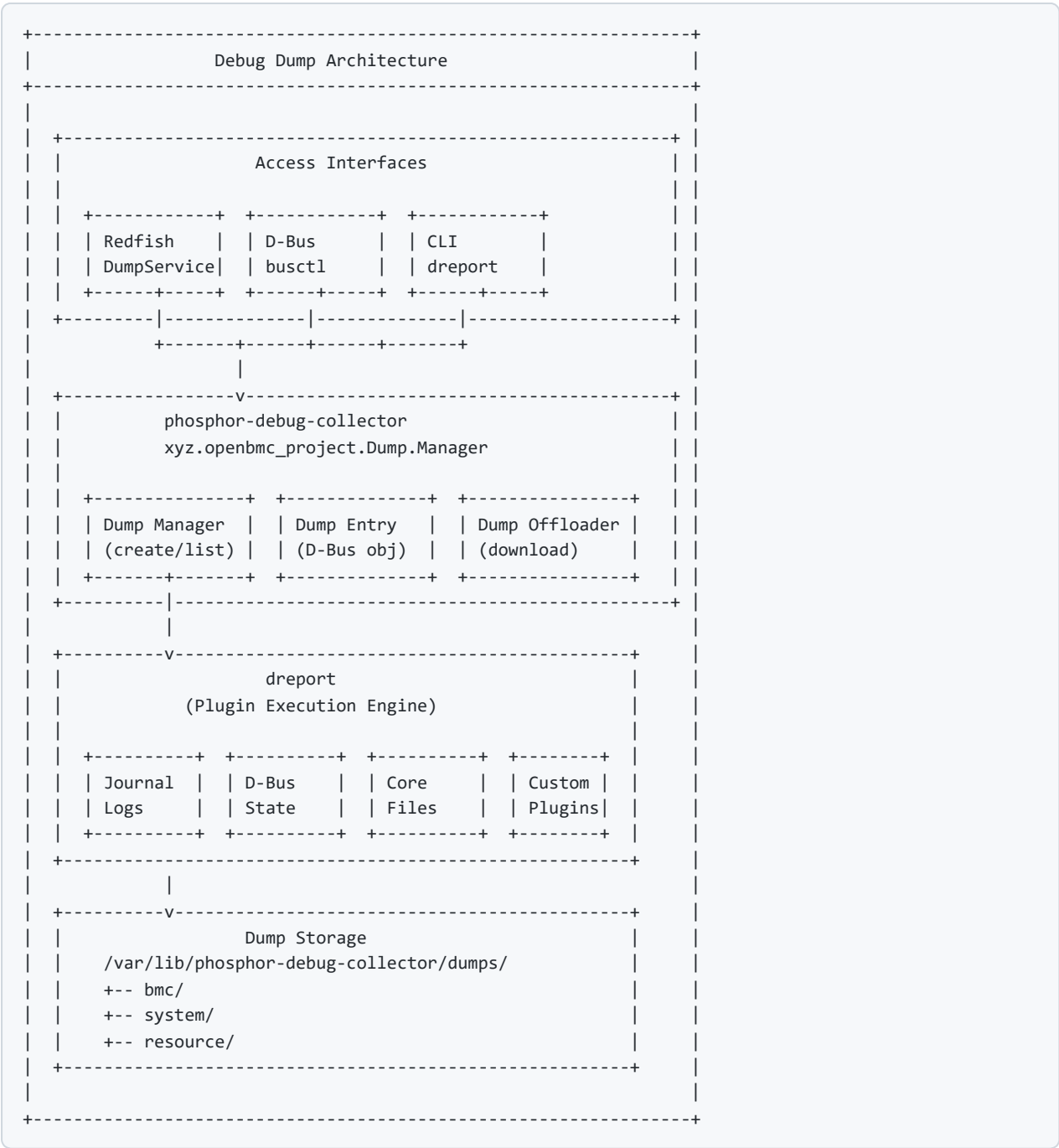
phosphor-debug-collector exposes dump operations on D-Bus and Redfish, while dreport executes a pipeline of plugin scripts that gather journal logs, core files, D-Bus object trees, hardware register snapshots, and any platform-specific data you define. The resulting tarball can be downloaded via Redfish, SCP, or the BMC command line.

This guide walks you through the dump collection architecture, shows you how to trigger and retrieve dumps through Redfish and D-Bus, explains how to write custom dreport plugins for vendor-specific diagnostics, and covers storage management to keep dump files from exhausting flash space.

Key concepts covered:

- phosphor-debug-collector architecture and dump types
- BMC dump collection via Redfish DumpService and D-Bus
- dreport framework and plugin pipeline
- Writing and installing custom dreport plugins
- Dump storage limits, retention policies, and cleanup

Source Reference: [phosphor-debug-collector](#) contains the dump manager, dreport scripts, and default plugins.



Dump Type	D-Bus Path	Contents	Trigger
BMC Dump	<code>/xyz/openbmc_project/dump/bmc</code>	Journal logs, D-Bus state, core files, config	User request, daemon crash
System Dump	<code>/xyz/openbmc_project/dump/system</code>	Host memory, SBE state, hardware registers	Host failure, checkstop
Resource Dump	<code>/xyz/openbmc_project/dump/resource</code>	Specific hardware resource data	Targeted diagnostics

BMC dumps are the most commonly used type. They capture everything running on the BMC itself and do not require host cooperation. Start here for most debugging scenarios.

D-Bus Interfaces

Interface	Object Path	Description
<code>xyz.openbmc_project.Dump.Create</code>	<code>/xyz/openbmc_project/dump/bmc</code>	Create a new dump
<code>xyz.openbmc_project.Dump.Entry</code>	<code>/xyz/openbmc_project/dump/bmc/entry/<id></code>	Individual dump metadata
<code>xyz.openbmc_project.Dump.Manager</code>	<code>/xyz/openbmc_project/dump/bmc</code>	List and manage dumps
<code>xyz.openbmc_project.Object.Delete</code>	<code>/xyz/openbmc_project/dump/bmc/entry/<id></code>	Delete a dump entry

Key Dependencies

- **phosphor-logging**: Event logs that may trigger automatic dump collection
- **bmcweb**: Serves Redfish DumpService endpoints for remote dump operations
- **dreport**: Shell script framework that executes plugin scripts to gather data
- **systemd-coredump**: Captures core dumps from crashing daemons

Setup & Configuration

Build-Time Configuration (Yocto)

```
# In your machine .conf or local.conf

# Include dump collection packages
IMAGE_INSTALL:append = " \
    phosphor-debug-collector \
"

# Configure dump options via meson overrides
EXTRA_OEMESON:pn-phosphor-debug-collector = " \
    -Dbmc_dump_total_size=67108864 \
    -Dbmc_dump_max_num=10 \
"
```

Meson Build Options

Option	Default	Description
<code>bmc_dump_total_size</code>	67108864 (64 MB)	Maximum total size of all BMC dumps in bytes
<code>bmc_dump_max_num</code>	10	Maximum number of BMC dump entries
<code>bmc_dump_path</code>	<code>/var/lib/phosphor-debug-collector/dumps/</code>	Storage path for dump files

Runtime Verification

```
# Check dump manager service
systemctl status phosphor-dump-manager

# View service logs
journalctl -u phosphor-dump-manager -f

# Verify D-Bus service is registered
busctl status xyz.openbmc_project.Dump.Manager

# Check dump storage directory
ls -la /var/lib/phosphor-debug-collector/dumps/
```

BMC Dump Collection

Via Redfish DumpService

The Redfish DumpService provides the standard interface for creating, listing, and downloading dumps from remote management tools.

Create a BMC Dump

```
# Trigger a new BMC dump
curl -k -u root:openBmc -X POST \
  -H "Content-Type: application/json" \
  -d '{"DiagnosticDataType": "Manager"}' \
  https://localhost/redfish/v1/Managers/bmc/LogServices/Dump/Actions/LogService.CollectDiagnosticData
```

The response includes a task URI you can poll for completion:

```
{
  "@odata.id": "/redfish/v1/TaskService/Tasks/0",
  "@odata.type": "#Task.v1_4_3.Task",
  "Id": "0",
  "TaskState": "Running"
}
```

Poll for Completion

```
# Check task status
curl -k -u root:openBmc \
  https://localhost/redfish/v1/TaskService/Tasks/0
```

Dump generation can take 30 seconds to several minutes depending on the number of dreport plugins and the amount of data collected. Poll the task endpoint until `TaskState` shows `Completed`.

List Available Dumps

```
# List all BMC dump entries
curl -k -u root:openBmc \
  https://localhost/redfish/v1/Managers/bmc/LogServices/Dump/Entries
```

Download a Dump

```
# Download dump tarball by entry ID
curl -k -u root:openBmc -o bmc_dump_1.tar.xz \
  https://localhost/redfish/v1/Managers/bmc/LogServices/Dump/Entries/1/attachment
```

Delete a Dump

```
# Delete a specific dump entry
curl -k -u root:openBmc -X DELETE \
  https://localhost/redfish/v1/Managers/bmc/LogServices/Dump/Entries/1

# Clear all dumps
curl -k -u root:openBmc -X POST \
  https://localhost/redfish/v1/Managers/bmc/LogServices/Dump/Actions/LogService.ClearLog
```

Via D-Bus

Use D-Bus calls for programmatic dump collection from scripts or services running on the BMC itself.

Create a BMC Dump

```
# Create a user-initiated BMC dump
busctl call xyz.openbmc_project.Dump.Manager \
    /xyz/openbmc_project/dump/bmc \
    xyz.openbmc_project.Dump.Create \
    CreateDump "a{sv}" 0
```

The call returns the object path of the new dump entry:

```
o "/xyz/openbmc_project/dump/bmc/entry/1"
```

Monitor Dump Progress

```
# Watch for dump completion signal
busctl monitor xyz.openbmc_project.Dump.Manager \
    --match "type='signal',interface='org.freedesktop.DBus.Properties',member='PropertiesChanged',path_namespace='/xyz/openbmc_project/dump/bmc/entry'"
```

List Dump Entries

```
# List all dump entries in the D-Bus tree
busctl tree xyz.openbmc_project.Dump.Manager

# Get properties of a specific dump entry
busctl introspect xyz.openbmc_project.Dump.Manager \
    /xyz/openbmc_project/dump/bmc/entry/1

# Get the dump size
busctl get-property xyz.openbmc_project.Dump.Manager \
    /xyz/openbmc_project/dump/bmc/entry/1 \
    xyz.openbmc_project.Dump.Entry \
    Size
```

Delete a Dump Entry

```
# Delete a specific dump
busctl call xyz.openbmc_project.Dump.Manager \
    /xyz/openbmc_project/dump/bmc/entry/1 \
    xyz.openbmc_project.Object.Delete \
    Delete
```

Via Command Line (dreport)

For direct on-BMC debugging, you can invoke dreport manually:

```
# Generate a user-initiated dump
dreport -d /tmp/dumps -n manual_dump -t user -v

# Report type options:
# user      - User-initiated BMC dump (default)
# core      - Triggered by a core dump
# elog      - Triggered by an error log event

# Examine the generated dump
ls -la /tmp/dumps/
tar -tvf /tmp/dumps/manual_dump_*.tar.xz
```

Dump Contents

A typical BMC dump tarball contains:

Directory/File	Description
<code>dreport.log</code>	dreport execution log and plugin output
<code>journal/</code>	Full journald logs from the BMC
<code>dbus/</code>	D-Bus object tree snapshots
<code>core/</code>	Core dump files (if triggered by a crash)
<code>config/</code>	System configuration files
<code>proc/</code>	Process information (<code>/proc</code> snapshots)
<code>platform/</code>	Platform-specific data from custom plugins

```
# Extract and examine a dump
mkdir /tmp/dump_analysis
tar -xf bmc_dump_1.tar.xz -C /tmp/dump_analysis
ls /tmp/dump_analysis/

# Review journal logs from the dump
less /tmp/dump_analysis/journal/journal.log

# Check D-Bus state at dump time
less /tmp/dump_analysis/dbus/dbus_tree.txt
```

Custom dreport Plugins

The dreport framework executes plugin scripts during dump generation. Each plugin collects a specific category of data. You can write custom plugins to capture platform-specific diagnostics such as FPGA register dumps, vendor ASIC state, or custom hardware telemetry.

Plugin API

dreport plugins are executable shell scripts that follow a naming convention and write their output to a designated directory. dreport discovers and runs them automatically based on the dump type being generated.

Plugin Naming Convention

pl_<type><priority>_<description>

Field	Description	Values
pl_	Prefix (required)	Always pl_
type	Dump type code	u = user, c = core, e = elog
priority	Execution order (2 digits)	00 (highest) to 99 (lowest)
description	Brief identifier	Lowercase, underscores

Examples:

- pl_u01_journal – User dump, high priority, collects journal logs
- pl_u50_dbus_state – User dump, medium priority, collects D-Bus tree
- pl_c10_core_files – Core dump, high priority, collects core files
- pl_u80_fpga_regs – User dump, low priority, collects FPGA registers

Plugin names must be unique. If two plugins share the same name, only one will execute. Always verify your plugin name does not conflict with existing plugins.

Plugin Script Structure

Every dreport plugin receives these environment variables:

Variable	Description
DREPORT_INCLUDE	Path to dreport utility functions (source this)
EPOCH_TIME	Timestamp of the dump request
name	Name of the dump
dump_dir	Output directory for this dump
dump_id	Numeric ID of this dump

```
#!/bin/bash
# pl_u80_fpga_regs - Collect FPGA register dump

# Source dreport utility functions
. "$DREPORT_INCLUDE"/functions

# Define the output file
desc="FPGA Register Dump"
file_name="fpga_registers"

# Use add_cmd_output to run a command and capture output
add_cmd_output "devmem2 0x1e6e2000 w" "${file_name}_ctrl.log" "${desc} - Control"
add_cmd_output "devmem2 0x1e6e2004 w" "${file_name}_status.log" "${desc} - Status"

# Or use add_copy_file to include an existing file
add_copy_file "/sys/kernel/debug/fpga/registers" "fpga_debug_regs.txt" "${desc}"
```

dreport Utility Functions

When you source `$DREPORT_INCLUDE/functions`, you get access to these helper functions:

Function	Arguments	Description
<code>add_cmd_output</code>	<code>"command" "filename" "description"</code>	Run a command and save its stdout to the dump
<code>add_copy_file</code>	<code>"source_path" "dest_name" "description"</code>	Copy a file into the dump tarball
<code>log_summary</code>	<code>"message"</code>	Write a message to the dreport summary log

Writing a Custom Plugin

Here is a complete example that collects I2C device state for a platform with custom power regulators:

```
#!/bin/bash
# pl_u70_i2c_regulators - Dump I2C regulator state
#
# Collects register dumps from VR controllers on I2C bus 3
# for post-mortem voltage regulator analysis.

. "$DREPORT_INCLUDE"/functions

desc="I2C Voltage Regulator State"

# Dump I2C bus scan results
add_cmd_output "i2cdetect -y 3" "i2c_bus3_scan.log" "${desc} - Bus Scan"

# Read specific regulator registers
for addr in 0x20 0x21 0x22; do
    name="vr_${addr}"
    add_cmd_output "i2cdump -y 3 ${addr}" "${name}_regs.log" "${desc} - ${addr}"
done

# Include any relevant kernel messages
add_cmd_output "dmesg | grep -i regulator" "regulator_dmesg.log" "${desc} - Kernel"

log_summary "I2C regulator state collected for bus 3"
```

Installing Custom Plugins via Yocto

To deploy your custom plugin in the BMC image, create a bbappend for phosphor-debug-collector:

```
# meta-myplatform/recipes-phosphor/dump/phosphor-debug-collector_%.bbappend

FILESEXTRAPATHS:prepend := "${THISDIR}/files:"

SRC_URI += " \
    file://pl_u70_i2c_regulators \
    file://pl_u80_fpga_regs \
"

do_install:append() {
    # Install custom dreport plugins
    install -d ${D}${dreport_plugin_dir}
    install -m 0755 ${WORKDIR}/pl_u70_i2c_regulators \
        ${D}${dreport_plugin_dir}/
    install -m 0755 ${WORKDIR}/pl_u80_fpga_regs \
        ${D}${dreport_plugin_dir}/
}
```

The `dreport_plugin_dir` variable is defined by phosphor-debug-collector and points to the correct plugin installation path. Use it instead of hardcoding the directory.

Testing Plugins

You can test a plugin directly on the BMC without generating a full dump:

```
# Set up the environment that dreport provides
export DREPORT_INCLUDE=/usr/share/dreport.d/include.d
export dump_dir=/tmp/test_plugin_output
export dump_id=999
export name="test"
export EPOCH_TIME=$(date +%s)

mkdir -p "$dump_dir"

# Run the plugin directly
bash /usr/share/dreport.d/plugins.d/pl_u70_i2c_regulators

# Check the output
ls -la "$dump_dir"
```

Dump Storage Management

BMC flash storage is limited. Without proper management, dump files can consume all available space and cause service failures. phosphor-debug-collector enforces both count and size limits.

Storage Limits

Setting	Default	Description
Maximum dump count	10	Oldest dump deleted when limit reached
Maximum total size	64 MB	New dumps rejected when space exhausted
Storage path	<code>/var/lib/phosphor-debug-collector/dumps/</code>	On persistent filesystem

Retention Behavior

When a new dump is requested and the count or size limit would be exceeded:

1. The dump manager checks total dump count against `bmc_dump_max_num`
2. If the count limit is reached, the **oldest** dump entry is automatically deleted
3. If the total size limit (`bmc_dump_total_size`) would be exceeded even after count-based cleanup, the dump request fails with an error
4. Deleted dump entries are removed from both D-Bus and the filesystem

Monitoring Storage Usage

```
# Check current dump count and sizes
busctl tree xyz.openbmc_project.Dump.Manager

# Get size of each dump entry
for entry in $(busctl tree xyz.openbmc_project.Dump.Manager \
  | grep "/xyz/openbmc_project/dump/bmc/entry/"); do
  size=$(busctl get-property xyz.openbmc_project.Dump.Manager \
    "$entry" xyz.openbmc_project.Dump.Entry Size 2>/dev/null)
  echo "$entry: $size"
done

# Check filesystem usage
df -h /var/lib/phosphor-debug-collector/
du -sh /var/lib/phosphor-debug-collector/dumps/*
```

Customizing Limits

Override the defaults in your platform layer:

```
# meta-myplatform/recipes-phosphor/dump/phosphor-debug-collector_%.bbappend

# Allow more dumps with larger total budget (128 MB)
EXTRA_OEMESON:append = " \
  -Dbmc_dump_max_num=20 \
  -Dbmc_dump_total_size=134217728 \
"
```


Increasing dump limits consumes flash storage. On platforms with limited flash (e.g., 32 MB SPI NOR), keep the total dump size well below the available free space on the data partition. Monitor [/var/lib/phosphor-debug-collector/](#) usage after changing these values.

Manual Cleanup

```
# Delete all dumps via Redfish
curl -k -u root:OpenBmc -X POST \
  https://localhost/redfish/v1/Managers/bmc/LogServices/Dump/Actions/LogService.ClearLog

# Delete all dumps via D-Bus
for entry in $(busctl tree xyz.openbmc_project.Dump.Manager \
  | grep "/xyz/openbmc_project/dump/bmc/entry/"); do
  busctl call xyz.openbmc_project.Dump.Manager \
    "$entry" xyz.openbmc_project.Object.Delete Delete
done

# Direct filesystem cleanup (last resort)
rm -rf /var/lib/phosphor-debug-collector/dumps/bmc/*
systemctl restart phosphor-dump-manager
```

Troubleshooting

Issue: Dump Creation Fails with “No Space”

Symptom: Redfish returns an error or D-Bus call fails when creating a new dump.

Cause: The total dump size limit has been reached, or the filesystem is full.

Solution:

1. Check current dump storage usage:

```
du -sh /var/lib/phosphor-debug-collector/dumps/
df -h /var/lib/phosphor-debug-collector/
```

2. Delete old dumps to free space:

```
curl -k -u root:OpenBmc -X DELETE \
  https://localhost/redfish/v1/Managers/bmc/LogServices/Dump/Entries/1
```

3. If the filesystem is full from non-dump files, investigate with:

```
du -sh /var/lib/* | sort -rh | head -10
```

Issue: Dump Manager Service Not Running

Symptom: D-Bus calls to [xyz.openbmc_project.Dump.Manager](#) fail with “service not found.”

Cause: The phosphor-dump-manager service failed to start or crashed.

Solution:

1. Check the service status:

```
systemctl status phosphor-dump-manager
```

2. Review journal logs for errors:

```
journalctl -u phosphor-dump-manager --no-pager -n 50
```

3. Verify the package is installed:

```
opkg list-installed | grep phosphor-debug-collector
```

Issue: Custom Plugin Not Executing

Symptom: Your custom dreport plugin output is missing from the dump tarball.

Cause: Plugin naming, permissions, or dump type mismatch.

Solution:

1. Verify the plugin name follows the convention (`pl_<type><priority>_<name>`):

```
ls -la /usr/share/dreport.d/plugins.d/pl_u*
```

2. Confirm the plugin is executable:

```
chmod +x /usr/share/dreport.d/plugins.d/pl_u80_fpga_regs
```

3. Check the dump type letter matches – `u` for user dumps, `c` for core dumps
4. Test the plugin manually as described in [Testing Plugins](#)

Issue: Dump Download Returns Empty or Corrupt File

Symptom: The downloaded tarball is 0 bytes or fails to extract.

Cause: The dump generation is still in progress, or the dump file was corrupted on disk.

Solution:

1. Verify the dump is complete before downloading:

```
curl -k -u root:0penBmc \
  https://localhost/redfish/v1/Managers/bmc/LogServices/Dump/Entries/1 \
  | python3 -m json.tool | grep -i status
```

2. Check that the file exists and has nonzero size:

```
ls -la /var/lib/phosphor-debug-collector/dumps/bmc/
```

Debug Commands

```
# Check dump manager service
systemctl status phosphor-dump-manager

# View dump manager logs
journalctl -u phosphor-dump-manager -f

# List all D-Bus dump objects
busctl tree xyz.openbmc_project.Dump.Manager

# List installed dreport plugins
ls -la /usr/share/dreport.d/plugins.d/

# Run dreport in verbose mode
dreport -d /tmp/debug_test -n test -t user -v
```

Examples

Working examples are available in the [examples/debug-dump](#) directory:

- `pl_u80_fpga_regs` – Custom dreport plugin for FPGA register collection
- `pl_u70_i2c_regulators` – Custom dreport plugin for I2C voltage regulator state
- `collect-dump-redfish.sh` – Script to create, poll, and download a BMC dump via Redfish
- `list-dumps.sh` – Script to list all dump entries with sizes

References

Official Resources

- [phosphor-debug-collector](#) – Dump manager and dreport framework
- [D-Bus Dump Interfaces](#) – Dump D-Bus interface definitions
- [OpenBMC Documentation](#) – General OpenBMC documentation

Related Guides

- [Logging Guide](#) – Event logging, SEL, and error management
- [Linux Debug Tools](#) – ASan, Valgrind, and kernel debugging

External Documentation

- [Redfish DumpService Schema](#) – DMTF Redfish log service specification
- [DMTF Redfish Specification](#) – Full Redfish standard

Tested on: QEMU ast2600-evb, OpenBMC master branch Last updated: 2026-02-06

POST Code Monitoring

Capture, store, and expose host POST codes through the OpenBMC stack, from LPC snoop hardware to Redfish APIs.

Table of Contents

1. [Overview](#)
2. [Architecture](#)
 1. [POST Code Capture Pipeline](#)
 2. [D-Bus Interfaces](#)
 3. [Key Dependencies](#)
3. [Setup and Configuration](#)
 1. [Build-Time Configuration \(Yocto\)](#)
 2. [Kernel and Device Tree Configuration](#)
 3. [Runtime Verification](#)
4. [Capturing POST Codes](#)
 1. [How LPC Snoop Works](#)
 2. [Reading POST Codes via D-Bus](#)
 3. [Querying POST Code History](#)
5. [Redfish POST Code Exposure](#)
 1. [PostCodes LogService](#)
 2. [Entry Format](#)
 3. [Example Redfish Response](#)
 4. [Clearing POST Code Logs](#)
6. [Boot Progress State Mapping](#)
 1. [BootProgress D-Bus Property](#)
 2. [Standard Boot Progress States](#)
 3. [Boot Progress via Redfish](#)
 4. [Customizing Boot Progress Mapping](#)
7. [Multi-Host POST Code Collection](#)
 1. [Multi-Host Architecture](#)
 2. [Yocto Configuration for Multi-Host](#)
 3. [Multi-Host D-Bus Paths](#)
 4. [Device Tree for Multi-Host](#)
8. [Troubleshooting](#)
 1. [No POST Codes Appearing](#)
 2. [POST Code History Missing](#)
 3. [Redfish PostCodes Endpoint Returns 404](#)
 4. [POST Codes Stop After Boot](#)

9. Examples

10. References

1. [OpenBMC Repositories](#)
2. [Specifications](#)
3. [Related Guides](#)

Overview

POST codes (Power-On Self-Test codes) are single-byte or multi-byte values that the host firmware writes to I/O port 0x80 during boot. Each code indicates which initialization stage the BIOS or UEFI firmware has reached. When a system hangs during boot, the last POST code captured by the BMC tells you exactly where the host stalled.

OpenBMC provides a complete POST code pipeline. The LPC snoop hardware on the ASPEED BMC captures every write to port 0x80, the `postd` daemon reads those values from the kernel driver, `phosphor-post-code-manager` stores them organized by boot cycle, and `bmcweb` exposes the data through standard Redfish LogService endpoints. This pipeline gives operators full visibility into host boot progress without requiring physical seven-segment displays or serial console access.

You use POST code monitoring whenever you need to diagnose host boot failures, track boot progress remotely, or integrate boot telemetry into data center management workflows. Combined with the BootProgress D-Bus property, POST codes provide both fine-grained numeric codes and human-readable boot stage descriptions.

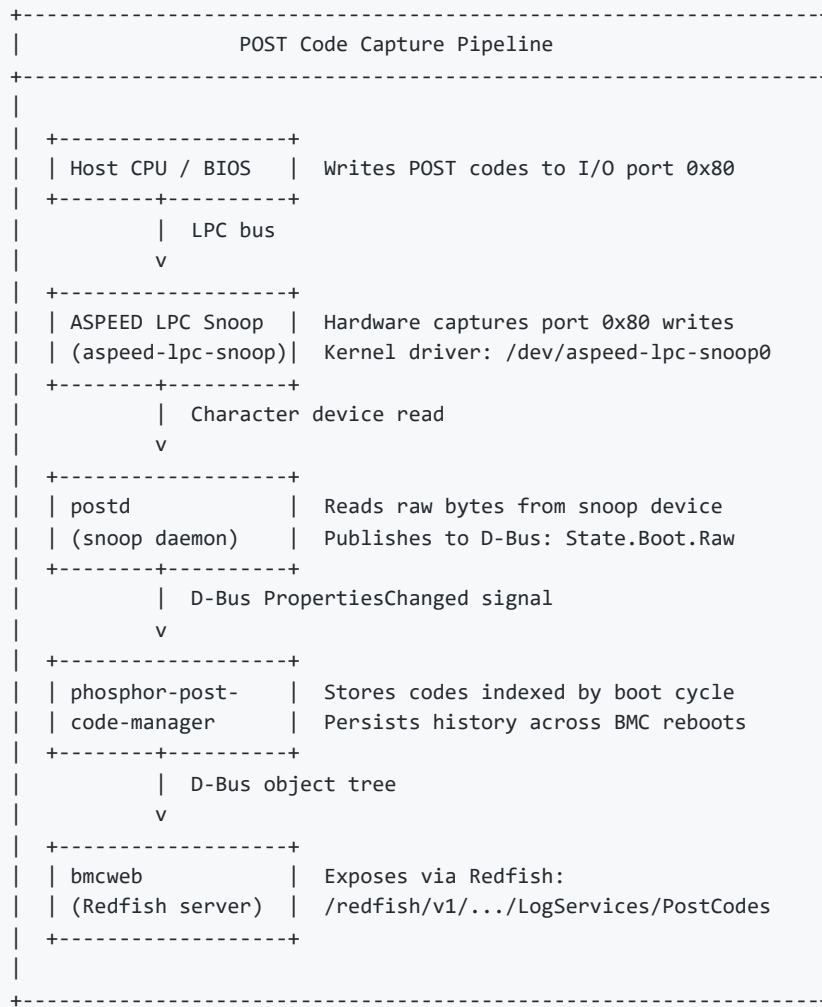
Key concepts covered:

- LPC snoop capture and the kernel `aspeed-lpc-snoop` driver
- The `postd` and `phosphor-post-code-manager` daemons
- Redfish POST code exposure via PostCodes LogService
- Boot progress state mapping through D-Bus properties
- Multi-host POST code collection

Architecture

POST Code Capture Pipeline

The POST code pipeline flows from hardware capture to Redfish exposure through four stages.



D-Bus Interfaces

Interface	Object Path	Description
<code>xyz.openbmc_project.State.Boot.Raw</code>	<code>/xyz/openbmc_project/state/boot/raw0</code>	Current POST code value
<code>xyz.openbmc_project.State.Boot.PostCode</code>	<code>/xyz/openbmc_project/State/Boot/PostCode0</code>	POST code history by boot cycle
<code>xyz.openbmc_project.State.Boot.Progress</code>	<code>/xyz/openbmc_project/state/host0</code>	High-level boot progress state

Key Dependencies

- **aspeed-lpc-snoop kernel driver:** Captures LPC port 0x80 writes in hardware
- **phosphor-host-postd:** Reads the snoop device and publishes to D-Bus
- **phosphor-post-code-manager:** Aggregates and persists POST code history
- **bmcweb:** Translates D-Bus POST code objects into Redfish responses

Setup and Configuration

Build-Time Configuration (Yocto)

```
# In your machine .conf or local.conf

# Include POST code packages
IMAGE_INSTALL:append = " \
    phosphor-host-postd \
    phosphor-post-code-manager \
"

# The LPC snoop driver is typically built into the kernel
# for ASPEED platforms. Verify it is enabled:
KERNEL_FEATURES:append = " features/aspeed/lpc-snoop.scc"
```

Kernel and Device Tree Configuration

Ensure the ASPEED LPC snoop driver is enabled in your kernel (`CONFIG_ASPEED_LPC_SNOOP=y`) and the snoop node is enabled in your platform device tree:

```
&lpc_snoop {
    status = "okay";
    snoop-ports = <0x80>;
};
```

Most ASPEED reference platforms (ast2500-evb, ast2600-evb) already have the LPC snoop node enabled. Check your platform DTS before adding a duplicate entry.

Runtime Verification

```
# Verify the snoop device exists
ls -l /dev/aspeed-lpc-snoop0

# Check postd service
systemctl status phosphor-host-postd

# Check post-code-manager service
systemctl status phosphor-post-code-manager

# View service logs
journalctl -u phosphor-host-postd -f
journalctl -u phosphor-post-code-manager -f
```

Capturing POST Codes

How LPC Snoop Works

The ASPEED BMC includes dedicated hardware that monitors the LPC bus for writes to specific I/O port addresses. When the host writes a byte to port 0x80, the snoop hardware latches the value into a FIFO buffer. The kernel driver exposes this FIFO as a character device at `/dev/aspeed-lpc-snoop0`.

The `postd` daemon performs a blocking read on this device. Each time a new POST code arrives, `postd` updates the `Value` property on the `xyz.openbmc_project.State.Boot.Raw` D-Bus interface. The `phosphor-post-code-manager` daemon monitors this property via D-Bus signal matching and appends each new code to the current boot cycle's history.

Reading POST Codes via D-Bus

```
# Get the current (most recent) POST code
busctl get-property xyz.openbmc_project.State.Boot.Raw \
    /xyz/openbmc_project/state/boot/raw0 \
    xyz.openbmc_project.State.Boot.Raw \
    Value

# Watch POST codes arrive in real time
busctl monitor xyz.openbmc_project.State.Boot.Raw
```

Querying POST Code History

The post-code-manager stores codes organized by boot cycle number. Boot cycle 1 is the most recent complete boot, cycle 2 is the one before that, and so on.

```
# Get POST codes from the current boot cycle
busctl call xyz.openbmc_project.State.Boot.PostCode0 \
    /xyz/openbmc_project/State/Boot/PostCode0 \
    xyz.openbmc_project.State.Boot.PostCode \
    GetPostCodesWithTimeStamp q 1

# Get the number of stored boot cycles
busctl get-property xyz.openbmc_project.State.Boot.PostCode0 \
    /xyz/openbmc_project/State/Boot/PostCode0 \
    xyz.openbmc_project.State.Boot.PostCode \
    CurrentBootCycleCount
```

The `GetPostCodesWithTimeStamp` method returns an array of `(timestamp, code)` pairs. The timestamp is a microsecond-precision value that lets you calculate the time between boot stages.

Redfish POST Code Exposure

PostCodes LogService

OpenBMC exposes POST codes through the Redfish LogService at the standard path. This is the primary interface for remote management tools.

```
# Get the PostCodes LogService
curl -k -u root:openBmc \
    https://localhost/redfish/v1/Systems/system/LogServices/PostCodes

# List POST code entries
curl -k -u root:openBmc \
    https://localhost/redfish/v1/Systems/system/LogServices/PostCodes/Entries

# Get a specific entry
curl -k -u root:openBmc \
    https://localhost/redfish/v1/Systems/system/LogServices/PostCodes/Entries/B1-1
```


Entry Format

Each Redfish POST code entry includes:

Field	Description	Example
<code>Id</code>	Boot cycle and sequence index	<code>B1-1</code> (Boot 1, code 1)
<code>MessageId</code>	Redfish message identifier	<code>OpenBMC.0.2.BIOSPOSTCode</code>
<code>Created</code>	Timestamp of the POST code	<code>2026-02-06T10:15:30+00:00</code>
<code>MessageArgs</code>	Array containing the POST code value	<code>["0x19"]</code>
<code>Severity</code>	Always <code>OK</code> for POST codes	<code>OK</code>

Example Redfish Response

```
{
  "@odata.id": "/redfish/v1/Systems/system/LogServices/PostCodes/Entries/B1-3",
  "@odata.type": "#LogEntry.v1_9_0.LogEntry",
  "Id": "B1-3",
  "Name": "POST Code Log Entry",
  "EntryType": "Event",
  "Severity": "OK",
  "Created": "2026-02-06T10:15:32.451+00:00",
  "MessageId": "OpenBMC.0.2.BIOSPOSTCode",
  "MessageArgs": [
    "0xA2"
  ],
  "Message": "BIOS POST Code: 0xA2"
}
```

Clearing POST Code Logs

```
# Clear all POST code entries
curl -k -u root:openBmc -X POST \
  https://localhost/redfish/v1/Systems/system/LogServices/PostCodes/Actions/LogService.ClearLog
```

Clearing POST code logs deletes all stored boot cycle history. This action cannot be undone. Consider exporting log data before clearing if you need it for later analysis.

Boot Progress State Mapping

BootProgress D-Bus Property

In addition to raw numeric POST codes, OpenBMC maps host boot stages to human-readable progress states. The `phosphor-state-manager` exposes a `BootProgress` property that higher-level tools can use without needing to interpret vendor-specific POST code tables.

```
# Get current boot progress
busctl get-property xyz.openbmc_project.State.Host \
  /xyz/openbmc_project/state/host0 \
  xyz.openbmc_project.State.Boot.Progress \
  BootProgress
```

Standard Boot Progress States

D-Bus Enumeration	Description	Typical POST Code Range
Unspecified	Boot state unknown or not yet started	–
PrimaryProcInit	Primary processor initialization	0x01 - 0x0F
BusInit	System bus (PCI, USB) initialization	0x10 - 0x2F
MemoryInit	Memory detection and training	0x30 - 0x4F
SecondaryProcInit	Secondary processor initialization	0x50 - 0x5F
PCIInit	PCI resource enumeration and configuration	0x60 - 0x7F
OSStart	Operating system handoff beginning	0x80 - 0x8F
OSRunning	Operating system is running	0x90+
SystemInitComplete	All firmware initialization complete	0xA0 - 0xAF
SystemSetup	BIOS setup / configuration menu active	0xB0 - 0xBF

POST code-to-progress-state mappings are vendor-specific. The ranges listed above are common conventions, but your BIOS vendor may use different assignments. Consult your BIOS documentation for exact mappings.

Boot Progress via Redfish

The boot progress state is also available through the Redfish Systems resource:

```
# Query boot progress through Redfish
curl -k -u root:openBmc \
  https://localhost/redfish/v1/Systems/system | \
  python3 -m json.tool | grep -A 5 BootProgress
```

Example response fragment:

```
{
  "BootProgress": {
    "LastState": "SystemHardwareInitializationComplete",
    "LastStateTime": "2026-02-06T10:16:45+00:00"
  }
}
```

Customizing Boot Progress Mapping

To map your platform's POST codes to standard boot progress states, create a platform-specific configuration in your machine layer:

```
{
  "CodeMap": {
    "0x13": "PrimaryProcInit",
    "0x34": "MemoryInit",
    "0x62": "PCIInit",
    "0x85": "OSStart",
    "0xA0": "SystemInitComplete"
  }
}
```

Multi-Host POST Code Collection

Multi-Host Architecture

In multi-host (multi-node) platforms, each host has an independent POST code pipeline. The BMC manages separate snoop devices, `postd` instances, and post-code-manager instances for each host. Each pipeline follows the same four-stage flow described in the Architecture section, with instance-numbered D-Bus paths and Redfish endpoints.

Yocto Configuration for Multi-Host

```
# Enable multi-host POST code support
# In your machine .conf
OBMC_HOST_INSTANCES = "0 1 2 3"

# Each instance gets its own systemd service
# phosphor-host-postd@0.service
# phosphor-host-postd@1.service
# etc.
```

Multi-Host D-Bus Paths

Each host instance uses a numbered suffix on D-Bus paths. Replace `N` with the host index (0, 1, 2, ...):

Resource	D-Bus Path
Current POST code	<code>/xyz/openbmc_project/state/boot/rawN</code>
POST code history	<code>/xyz/openbmc_project/State/Boot/PostCodeN</code>
Boot progress	<code>/xyz/openbmc_project/state/hostN</code>

```
# Read Host 1 current POST code
busctl get-property xyz.openbmc_project.State.Boot.Raw \
  /xyz/openbmc_project/state/boot/raw1 \
  xyz.openbmc_project.State.Boot.Raw Value

# Read Host 1 boot cycle history
busctl call xyz.openbmc_project.State.Boot.PostCode1 \
  /xyz/openbmc_project/State/Boot/PostCode1 \
  xyz.openbmc_project.State.Boot.PostCode \
  GetPostCodesWithTimeStamp q 1
```

Device Tree for Multi-Host

```
&lpc_snoop {
    status = "okay";
    snoop-ports = <0x80 0x81>;
    /* Port 0x80 -> /dev/aspeed-lpc-snoop0 (Host 0) */
    /* Port 0x81 -> /dev/aspeed-lpc-snoop1 (Host 1) */
};
```

Some multi-host platforms use separate LPC buses rather than separate I/O ports. In that case, each LPC bus has its own snoop node in the device tree. Check your platform's hardware design to determine the correct configuration.

Troubleshooting

No POST Codes Appearing

Symptom: The `Value` property on `State.Boot.Raw` never changes, even when the host is booting.

Solution:

```
# 1. Verify the snoop device exists
ls -l /dev/aspeed-lpc-snoop*

# 2. Check the kernel driver is loaded
dmesg | grep -i snoop

# 3. Verify postd is running and connected
systemctl status phosphor-host-postd
journalctl -u phosphor-host-postd --no-pager -n 20

# 4. Read the snoop device directly (bypasses postd)
# This blocks until a POST code arrives
xxd /dev/aspeed-lpc-snoop0

# 5. Check the device tree has snoop enabled
cat /sys/firmware/devicetree/base/ahb/apb/lpc/lpc-snoop/status
```

If `xxd /dev/aspeed-lpc-snoop0` produces no output even during host boot, the issue is at the hardware or kernel driver level. Verify that the LPC bus is physically connected and the host is actually writing to port 0x80.

POST Code History Missing

Symptom: Real-time POST codes appear in D-Bus but `GetPostCodesWithTimeStamp` returns empty results.

Solution:

```
# 1. Check post-code-manager service
systemctl status phosphor-post-code-manager
journalctl -u phosphor-post-code-manager --no-pager -n 20

# 2. Check persistent storage
ls -la /var/lib/phosphor-post-code-manager/
```

Redfish PostCodes Endpoint Returns 404

Symptom: Querying `/redfish/v1/Systems/system/LogServices/PostCodes` returns a 404 error.

Solution:

```
# 1. Verify bmcweb is running
systemctl status bmcweb

# 2. Check that post-code-manager D-Bus objects exist
busctl tree xyz.openbmc_project.State.Boot.PostCode0

# 3. Confirm bmcweb was built with POST code support
# The PostCodes LogService requires BMCWEB_ENABLE_REDFISH_POSTCODE
```

POST Codes Stop After Boot

This is usually normal behavior. Most BIOS implementations stop writing to port 0x80 once they hand off to the OS bootloader.

Examples

Working examples are available in the [examples/post-code](#) directory:

- `read-postcode.sh` - Read current and historical POST codes via D-Bus
- `monitor-boot.sh` - Monitor POST codes and boot progress in real time
- `redfish-postcode.sh` - Query POST code history through Redfish API

References

OpenBMC Repositories

- [phosphor-host-postd](#) - LPC snoop reader daemon
- [phosphor-post-code-manager](#) - POST code history storage
- [phosphor-dbus-interfaces \(State.Boot.Raw\)](#) - D-Bus interface definitions
- [bmcweb](#) - Redfish server with PostCodes LogService

Specifications

- [DMTF Redfish LogService Schema](#) - Redfish log service specification
- [DMTF Redfish ComputerSystem Schema](#) - BootProgress property definition

Related Guides

- [Logging Guide](#) - Event logs, SEL, and debug dumps
- [eSPI Guide](#) - LPC/eSPI host communication

Tested on: OpenBMC master, QEMU ast2600-evb. Multi-host features require platform-specific hardware support.

PLDM Platform Monitoring & Control (Type 2)

Use PLDM Type 2 (DSP0248) for sensor monitoring, effector control, Platform Descriptor Records (PDRs), and event handling on OpenBMC.

Table of Contents

1. [Overview](#)
2. [Architecture](#)
 1. [How PLDM Monitoring Works in OpenBMC](#)
 2. [Discovery and Monitoring Flow](#)
 3. [Key Components](#)
3. [Platform Descriptor Records \(PDRs\)](#)
 1. [PDR Types](#)
 2. [PDR Structure \(Common Header\)](#)
 3. [Numeric Sensor PDR](#)
 4. [State Sensor PDR](#)
 5. [Entity Types \(Common\)](#)
 6. [State Sets \(Common\)](#)
 7. [Querying PDRs with pldmtool](#)
4. [Sensors](#)
 1. [Numeric Sensors](#)
 1. [Reading Numeric Sensors](#)
 2. [Sensor States](#)
 3. [How Numeric Sensors Map to D-Bus](#)
 2. [State Sensors](#)
 1. [Reading State Sensors](#)
 3. [Sensor Base Units](#)
5. [Effecters](#)
 1. [Numeric Effecters](#)
 2. [State Effecters](#)
 3. [Effector PDR to D-Bus Mapping](#)
 4. [Driving Remote Effecters from D-Bus](#)
6. [BMC as a PLDM Responder](#)
7. [Event Handling](#)
 1. [Event Types](#)
 2. [Sensor Event Flow](#)
 3. [PDR Repository Change Event](#)
8. [D-Bus and Redfish Integration](#)
 1. [Sensor D-Bus Object Tree](#)
 2. [Redfish Mapping](#)

9. Configuration

1. [pldmd Platform Monitoring Options](#)
2. [Sensor Polling Interval](#)
3. [Entity Manager Integration](#)

10. Troubleshooting

1. [Issue: No PLDM Sensors Appearing on D-Bus](#)
2. [Issue: Sensor Reads Return “unavailable”](#)
3. [Issue: Missing Thresholds in Redfish](#)
4. [Issue: Event Messages Not Received](#)
5. [Debug Commands Summary](#)

11. References

1. [DMTF Specifications](#)
2. [OpenBMC Repositories](#)
3. [Related Guides](#)

Overview

PLDM for Platform Monitoring and Control (DMTF [DSP0248](#)) is the most heavily used PLDM type in OpenBMC. It replaces IPMI's SDR/sensor model with a richer, self-describing system based on Platform Descriptor Records (PDRs).

With PLDM Type 2, the BMC can:

- **Discover sensors and effecters** automatically through PDR exchange — no manual SDR configuration
- **Read numeric sensors** (temperature, voltage, current, power, fan speed) from any PLDM-capable device
- **Read state sensors** (presence, operational state, health status) for logical entities
- **Control effecters** to set fan speeds, power states, or trigger device actions
- **Receive asynchronous events** when sensor thresholds are crossed or states change

This is how the BMC monitors GPUs, NICs, NVMe drives, power supplies, Bridge ICs (OpenBIC), and other platform devices that speak PLDM over MCTP.

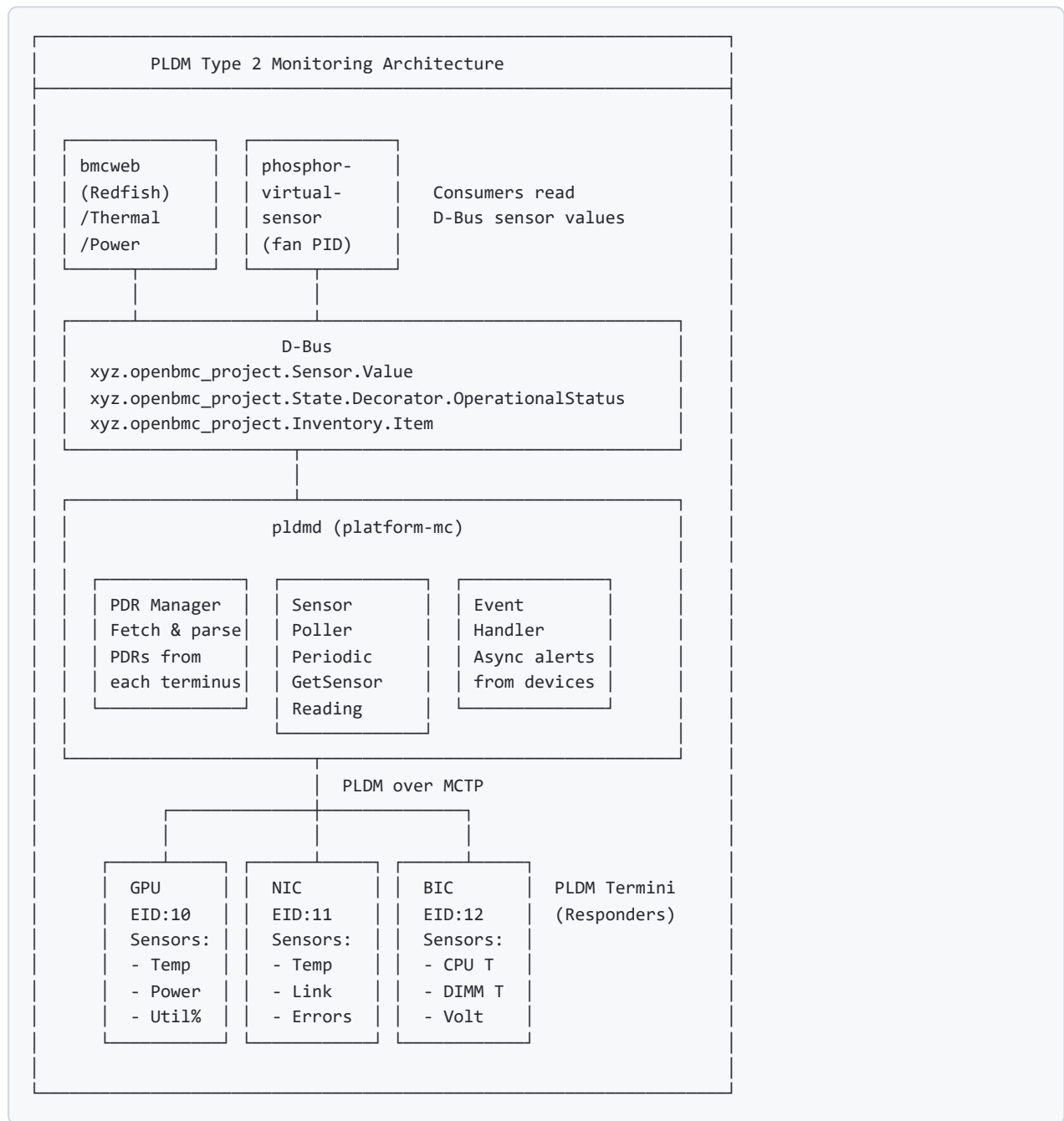
Key concepts covered:

- Platform Descriptor Records (PDRs) — the self-describing sensor/effector database
- Numeric and state sensors — reading values from PLDM termini
- Effecters — controlling devices through PLDM
- Event handling — asynchronous threshold and state change notifications
- D-Bus and Redfish integration — how PLDM sensors appear to consumers
- `pldmtool platform` commands for debugging

This guide focuses on PLDM Type 2 specifically. For MCTP transport configuration and general PLDM concepts, see the [MCTP & PLDM Guide](#). For BIOS configuration (Type 3), see the [BIOS Firmware Management Guide](#).

Architecture

How PLDM Monitoring Works in OpenBMC



Discovery and Monitoring Flow

- MCTP endpoint discovery** — `mctpd` discovers new MCTP endpoints (EIDs)
- PLDM terminus initialization** — `pldmd` queries each endpoint for supported PLDM types
- PDR fetch** — `pldmd` retrieves all PDRs from the terminus using `GetPDR` commands
- Sensor/effector creation** — PDRs are parsed; D-Bus sensor and effector objects are created
- Polling loop** — `pldmd` periodically reads sensor values via `GetSensorReading` / `GetStateSensorReadings`
- Event handling** — Devices send `PlatformEventMessage` asynchronously when thresholds are crossed

Key Components

Component	Role
pldmd (platform-mc)	Core PLDM monitoring engine — fetches PDRs, polls sensors, handles events
libpldm	C library for encoding/decoding PLDM messages
mctpd	MCTP transport — delivers PLDM messages to/from devices
pldmtool	CLI for debugging — manual PDR queries and sensor reads

Platform Descriptor Records (PDRs)

PDRs are the foundation of PLDM monitoring. Each PLDM terminus (device) maintains a local PDR repository that describes all its sensors, effecters, and entity relationships. The BMC fetches these PDRs to learn what the device can report and control.

PDR Types

PDR Type	ID	Description
Terminus Locator	1	Identifies the terminus and its MCTP EID
Numeric Sensor	2	Defines a numeric sensor (temp, voltage, power, etc.)
Numeric Sensor Init	3	Initial values for a numeric sensor
State Sensor	4	Defines a state sensor (presence, health, operational state)
Sensor Auxiliary Names	6	Human-readable sensor names
Effector Auxiliary Names	13	Human-readable effector names
Numeric Effector	9	Defines a numeric effector (fan speed, voltage set point)
State Effector	11	Defines a state effector (power control, LED control)
Entity Association	15	Parent-child relationships between entities
FRU Record Set	20	Links FRU data to entities
Compact Numeric Sensor	21	Space-efficient numeric sensor (PLDM 1.3+)

PDR Structure (Common Header)

Every PDR starts with a common header:

PDR Common Header		
Record Handle	(4 bytes)	Unique ID in the repo
PDR Header Version	(1 byte)	Always 0x01
PDR Type	(1 byte)	See PDR Types table above
Record Change #	(2 bytes)	Incremented on change
Data Length	(2 bytes)	Length of type-specific data
Type-specific PDR data (variable)		

Numeric Sensor PDR

A Numeric Sensor PDR fully describes one numeric sensor:

Numeric Sensor PDR (Type 2)	
PLDMTerminusHandle	Which terminus owns this sensor
SensorID	Unique sensor ID within terminus
EntityType	What entity (CPU, DIMM, fan, etc.)
EntityInstanceNumber	Which instance of that entity
SensorInit	Initialization behavior
SensorAuxNamesPDR	Handle of aux names PDR
BaseUnit	Units (Celsius, Volts, Watts, RPM)
UnitModifier	Scale factor (10^{modifier})
RateUnit	Per-second, per-minute, etc.
SensorDataSize	uint8, sint8, uint16, sint16, uint32
Resolution	Sensor resolution
Offset	Value offset
RangeFieldSupport	Which thresholds are defined
WarningHigh	Warning high threshold
CriticalHigh	Critical high threshold
FatalHigh	Fatal high threshold
WarningLow	Warning low threshold
CriticalLow	Critical low threshold
FatalLow	Fatal low threshold

State Sensor PDR

State Sensor PDR (Type 4)	
PLDMTerminusHandle	Which terminus owns this sensor
SensorID	Unique sensor ID within terminus
EntityType	What entity
EntityInstanceNumber	Which instance
CompositeCount	Number of state sets
For each state set:	
StateSetID	Which state set (see State Sets)
PossibleStatesSize	How many possible states
PossibleStates	Bitmask of valid states

Entity Types (Common)

Entity Type	ID	Description
Processor	135	CPU/SoC
Memory Module	142	DIMM
Fan	29	Cooling fan
Power Supply	120	PSU
System Board	64	Motherboard
Add-in Card	69	PCIe card (GPU, NIC)
Drive Bay	137	Storage bay
Connector	82	Physical connector
Chassis	45	Physical enclosure

State Sets (Common)

State Set	ID	Possible States
Health State	2	Normal, Non-Critical, Critical, Fatal, Unknown
Operational State	5	Enabled, Disabled, Unavailable, Shutting Down, In Test
Presence	11	Present, Not Present
Link State	18	Up, Down, Error, Unknown
Boot Progress	196	PCI Init, OS Boot, Base Board Init, ...

`pldmtool` groups its subcommands by PLDM type: **base** (`GetPLDMTypes` , `GetPLDMVersion` , `GetPLDMCommands` , `GetTID` / `SetTID`), **bios** (`GetDateTime` / `SetDateTime` , `GetBIOSTable` , `Get` / `SetBIOSAttributeCurrentValue`), **fru** (`GetFruRecordTableMetadata` , `GetFruRecordTable` , `GetFRURecordByOption`), **fw_update** (`GetStatus` , `QueryDeviceIdentifiers` , `RequestUpdate` , `ActivateFirmware` , ...), and **platform** (used below). A **raw** subcommand sends arbitrary request bytes for protocol debugging.

Querying PDRs with pldmtool

```
# Get PDR repository info (total PDR count, repo size)
pldmtool platform GetPDRRepositoryInfo

# Example output:
# {
#   "repositoryState": "available",
#   "recordCount": 42,
#   "repositorySize": 8192,
#   "largestRecordSize": 256
# }

# Fetch all PDRs sequentially (start from record handle 0)
pldmtool platform GetPDR -d 0    # First PDR
pldmtool platform GetPDR -d 1    # Second PDR
# ... continue until nextRecordHandle = 0

# Get a specific PDR by record handle
pldmtool platform GetPDR -d 17

# Example Numeric Sensor PDR output:
# {
#   "PDRType": "Numeric Sensor PDR",
#   "PLDMTerminusHandle": 1,
#   "sensorID": 5,
#   "entityType": "Processor",
#   "entityInstanceNumber": 0,
#   "baseUnit": "degreesCelsius",
#   "sensorDataSize": "sint16",
#   "warningHigh": 85.0,
#   "criticalHigh": 95.0,
#   "fatalHigh": 105.0
# }

# Example State Sensor PDR output:
# {
#   "PDRType": "State Sensor PDR",
#   "sensorID": 25,
#   "entityType": "Fan",
#   "entityInstanceNumber": 0,
#   "stateSetID": "Operational State",
#   "possibleStates": ["Enabled", "Disabled", "Unavailable"]
# }
```

Sensors

Numeric Sensors

Numeric sensors report continuous values like temperature, voltage, current, power, and fan speed.

Reading Numeric Sensors

```
# Read a numeric sensor value
pldmtool platform GetSensorReading -i <sensorID> -m <mctpEID>

# Example: Read GPU temperature (sensor ID 5, MCTP EID 10)
pldmtool platform GetSensorReading -i 5 -m 10

# Example output:
# {
#   "sensorDataSize": "sint16",
#   "sensorOperationalState": "enabled",
#   "sensorEventMessageEnable": "eventsEnabled",
#   "presentState": "normal",
#   "previousState": "normal",
#   "presentReading": 72
# }
```

Sensor States

Each numeric sensor reading includes the current threshold state:

State	Meaning
<code>unknown</code>	Sensor not yet read
<code>normal</code>	Within normal range
<code>warningLow</code> / <code>warningHigh</code>	Exceeded warning threshold
<code>criticalLow</code> / <code>criticalHigh</code>	Exceeded critical threshold
<code>fatalLow</code> / <code>fatalHigh</code>	Exceeded fatal threshold
<code>upperNonRecoverable</code> / <code>lowerNonRecoverable</code>	Non-recoverable condition

How Numeric Sensors Map to D-Bus

When `pldmd` discovers a numeric sensor via PDR, it creates a D-Bus object:

```
D-Bus Object Path:
  /xyz/openbmc_project/sensors/<type>/<terminus>_<name>

Interfaces:
  xyz.openbmc_project.Sensor.Value
    - Value (double): current reading
    - Unit (string): "xyz.openbmc_project.Sensor.Value.Unit.DegreesC"
    - MaxValue / MinValue (double): sensor range

  xyz.openbmc_project.Sensor.Threshold.Warning
    - WarningHigh (double)
    - WarningLow (double)
    - WarningAlarmHigh (bool): currently in alarm?

  xyz.openbmc_project.Sensor.Threshold.Critical
    - CriticalHigh (double)
    - CriticalLow (double)
    - CriticalAlarmHigh (bool)
```

```
# Read a PLDM sensor via D-Bus (after pldmd creates it)
busctl get-property xyz.openbmc_project.PLDM \
    /xyz/openbmc_project/sensors/temperature/GPU_0_Temp \
    xyz.openbmc_project.Sensor.Value Value

# Read via Redfish
curl -k -u root:0penBmc \
    https://${BMC_IP}/redfish/v1/Chassis/chassis/Thermal
```

State Sensors

State sensors report discrete states rather than continuous values — for example, device presence, operational state, or health status.

Reading State Sensors

```
# Read a state sensor
pldmtool platform GetStateSensorReadings -i <sensorID> -m <mctpEID>

# Example: Read fan presence (sensor ID 25, MCTP EID 10)
pldmtool platform GetStateSensorReadings -i 25 -m 10

# Example output:
# {
#   "compositeSensorCount": 1,
#   "sensorOpState": ["enabled"],
#   "presentState": ["present"],
#   "previousState": ["present"],
#   "eventState": ["present"]
# }
```

Sensor Base Units

Unit	ID	Examples
Degrees Celsius	2	CPU temp, DIMM temp, board temp
Volts	5	12V rail, 3.3V, VCore
Amps	6	PSU current, CPU current
Watts	7	System power, CPU power, GPU power
RPM	18	Fan speed
Percentage	21	Utilization, health percentage
Counts	1	Error counters, correctable ECC errors

Effecters

Effecters are the control counterpart to sensors. While sensors read values, effecters write values to devices — setting fan speeds, controlling power states, or triggering LED patterns.

Numeric Effecters

```
# Set a numeric effector value (e.g., fan speed setpoint)
pldmttool platform SetNumericEffectorValue -i <effectorID> -m <mctpEID> -d <value>

# Example: Set fan 0 to 5000 RPM
pldmttool platform SetNumericEffectorValue -i 10 -m 12 -d 5000

# Read current effector value
pldmttool platform GetNumericEffectorValue -i <effectorID> -m <mctpEID>
```

State Effecters

```
# Set a state effector (e.g., power control)
pldmttool platform SetStateEffectorStates -i <effectorID> -m <mctpEID> -c 1 -d <stateValue>

# Example: Set host power state to "power on" (effector ID 1, state 1)
pldmttool platform SetStateEffectorStates -i 1 -m 12 -c 1 -d 1

# Get current state effector value
pldmttool platform GetStateEffectorStates -i <effectorID> -m <mctpEID>
```

Effector PDR to D-Bus Mapping

Numeric effecters map to D-Bus control interfaces, and state effecters typically map to operational state controls. The `pldmd` platform-mc module creates the appropriate D-Bus objects when effector PDRs are discovered.

Driving Remote Effecters from D-Bus

Beyond exposing effecters for external callers, `pldmd` can drive an effector on a remote terminus automatically in response to a **local D-Bus property change**. The `HostEffectorParser` (in the platform-mc `dbus_to_terminus_effecters` module) reads a JSON map that binds a D-Bus property to a state or numeric effector. When the watched property changes, `pldmd` issues the matching `SetStateEffectorStates` (0x39) or `SetNumericEffectorValue` (0x31) command to the terminus.

Because many host effecters are only meaningful while the host is up, each entry carries a `checkHostState` flag (default `true`). When set, `pldmd` forwards the effector write only if the host is in a running-class state (for example `SystemInitComplete`, `OSRunning`, or `SystemSetup`), preventing writes to a powered-off or still-booting host.

```
// dbus_to_terminus_effector.json – bind a D-Bus property to a state effector
{
  "entries": [
    {
      "mctp_eid": 9,
      "effector_info": {
        "effectorID": 4,
        "entityType": 33,
        "compositeEffectorCount": 1
      },
      "effecters": [
        {
          "dbus_info": {
            "object_path": "/xyz/openbmc_project/control/host0/boot",
            "interface": "xyz.openbmc_project.Control.Boot.Mode",
            "property_name": "BootMode",
            "property_type": "string",
            "property_values": [
              "xyz.openbmc_project.Control.Boot.Mode.Modes.Regular"
            ]
          },
          "state": { "id": 196, "state_values": [2] }
        }
      ]
    }
  ]
}
```

When `BootMode` changes to `Regular`, `pldmd` maps it to boot-progress state `2` and sends `SetStateEffectersStates` for effector 4 on EID 9. A numeric mapping instead specifies `effectorDataSize`, `resolution`, `offset`, and `unitModifier`, and `pldmd` sends `SetNumericEffectorValue`.

BMC as a PLDM Responder

Everything above treats the BMC as a PLDM *requester*. `pldmd` is also a *responder* that answers PLDM commands from the host:

- **Base responder** — handles `GetPLDMTypes`, `GetPLDMCommands`, `GetPLDMVersion`, and `GetTID`, so a host can discover what the BMC supports.
- **FRU responder** (`FruImpl` / `FruParser`) — builds PLDM FRU record tables from D-Bus inventory using JSON that maps D-Bus interfaces/properties to FRU record fields, and supports hot add/remove.
- **BIOS responder** (`BIOSConfig`) — loads attribute JSON, builds the String / Attribute / AttributeValue tables, persists them, and syncs with `xyz.openbmc_project.BIOSConfig.Manager` (`BaseBIOSTable`, `PendingAttributes`). See </openbmc-guide-tutorial/docs/05-advanced/16-bios-firmware-management-guide.html> for the BIOS-config side.

Vendors extend any of these through the `oem_platform` / `oem_fru` / `oem_bios` handler base classes. (Verify class names against your `pldm` version.)

Event Handling

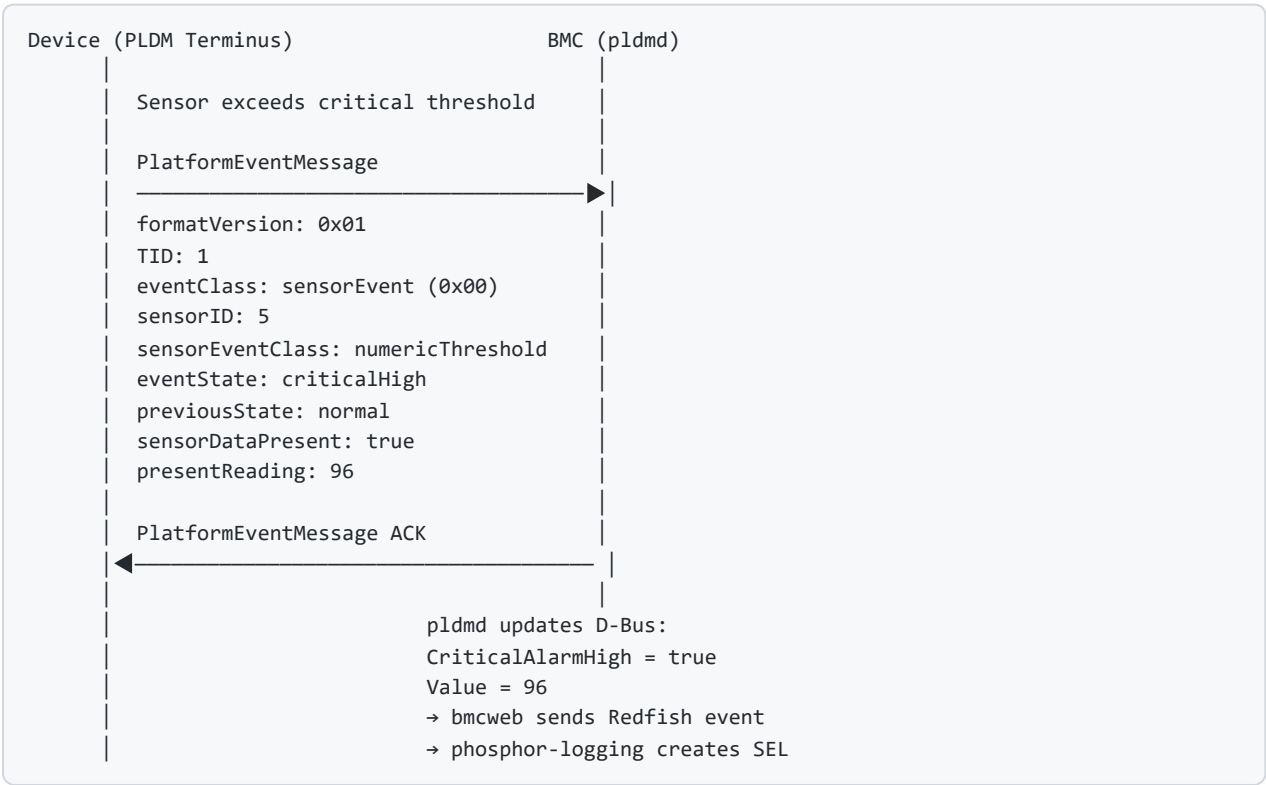
PLDM devices can send asynchronous event notifications to the BMC using the `PlatformEventMessage` command (0x0A). This is more efficient than polling — devices only send messages when something interesting happens.

Event Types

Event Type	Class	Description
<code>sensorEvent</code>	0x00	Numeric or state sensor crossed a threshold or changed state
<code>effectorEvent</code>	0x01	Effector state changed
<code>redfishTaskExecutedEvent</code>	0x02	A Redfish task completed on the device
<code>redfishMessageEvent</code>	0x03	A Redfish-format event from the device
<code>pldmPDRRepositoryChgEvent</code>	0x04	Device's PDR repo changed (sensors added/removed)
<code>pldmMessagePollEvent</code>	0x05	Device has queued events to be polled
<code>heartbeatTimerElapsed</code>	0x06	Heartbeat timer expired
<code>oemEvent</code>	0xFF	Vendor-specific event

`pldmd`'s `EventManager` also handles `PLDM_CPER_EVENT` (Common Platform Error Records): the CPER payload is written to a temporary file and a D-Bus dump entry is created (`createCperDumpEntry`), tying platform faults into the dump/log infrastructure. Large or fragmented events are reassembled via `pollForPlatformEventMessage` . (Verify against your `pldm` version.)

Sensor Event Flow



PDR Repository Change Event

When a device's sensor configuration changes at runtime (hotplug, firmware update), it sends a `pldmPDRRepositoryChgEvent` . The BMC then re-fetches the PDR repository to discover new or removed sensors.

```
# Monitor PLDM events in real-time
journalctl -u pldmd -f | grep -i event

# Check D-Bus for sensor alarm states
busctl get-property xyz.openbmc_project.PLDM \
  /xyz/openbmc_project/sensors/temperature/GPU_0_Temp \
  xyz.openbmc_project.Sensor.Threshold.Critical \
  CriticalAlarmHigh
```

D-Bus and Redfish Integration

Sensor D-Bus Object Tree

`pldmd` creates sensor objects under the standard OpenBMC sensor hierarchy:

```
# PLDM sensor D-Bus tree
busctl tree xyz.openbmc_project.PLDM

# /xyz/openbmc_project
# └─ sensors
#   └─ temperature
#     ├── PLDM_Device_0_CPU_Temp
#     ├── PLDM_Device_0_DIMM0_Temp
#     └── PLDM_Device_1_Board_Temp
#   └─ voltage
#     ├── PLDM_Device_0_P12V
#     └── PLDM_Device_0_VCore
#   └─ power
#     └── PLDM_Device_0_Total_Power
#   └─ fan_tach
#     ├── PLDM_Device_0_Fan0
#     └── PLDM_Device_0_Fan1
# └─ state
#   └── PLDM_Device_0_Operational
# └─ inventory
#   └── PLDM_Device_0
```

Redfish Mapping

PLDM sensors appear in standard Redfish resources:

PLDM Sensor Type	Redfish Resource
Temperature	<code>/redfish/v1/Chassis/{id}/Thermal</code> → <code>Temperatures[]</code>
Voltage	<code>/redfish/v1/Chassis/{id}/Power</code> → <code>Voltages[]</code>
Power	<code>/redfish/v1/Chassis/{id}/Power</code> → <code>PowerControl[]</code>
Fan speed	<code>/redfish/v1/Chassis/{id}/Thermal</code> → <code>Fans[]</code>
State sensors	<code>/redfish/v1/Chassis/{id}</code> → <code>Status.Health</code>

```
# Query all thermal sensors (includes PLDM-sourced sensors)
curl -k -u root:openBmc \
  https://{BMC_IP}/redfish/v1/Chassis/chassis/Thermal

# Example response showing a PLDM GPU temperature sensor:
# {
#   "Temperatures": [
#     {
#       "Name": "GPU_0_Temp",
#       "ReadingCelsius": 72,
#       "UpperThresholdNonCritical": 85,
#       "UpperThresholdCritical": 95,
#       "UpperThresholdFatal": 105,
#       "Status": {"State": "Enabled", "Health": "OK"}
#     }
#   ]
# }
```

Configuration

pldmd Platform Monitoring Options

```
# pldmd is started with platform-mc enabled by default
# Key meson build options:
# -Dplatform-mc=enabled      Enable platform monitoring & control
# -Dsensor-polling-time=1    Sensor polling interval in seconds
```

Sensor Polling Interval

The default polling interval determines how frequently `pldmd` reads sensor values from each terminus. Shorter intervals give more responsive monitoring but increase I2C/MCTP bus traffic.

Use Case	Recommended Interval	Notes
Thermal monitoring	1-2 seconds	Fast response for fan PID control
Power monitoring	2-5 seconds	Adequate for power capping
Inventory/presence	10-30 seconds	Rarely changes
Non-critical sensors	5-10 seconds	Reduce bus load

Entity Manager Integration

For platforms that use `entity-manager` for configuration, PLDM sensors can be associated with physical entities through entity-manager JSON configs:

```
{
  "Exposes": [
    {
      "Name": "GPU_0_Temp",
      "Type": "PLDMNumericSensor",
      "TerminusID": 1,
      "SensorID": 5,
      "Thresholds": [
        {
          "Direction": "greater than",
          "Name": "upper critical",
          "Severity": 1,
          "Value": 95
        }
      ]
    }
  ]
}
```

Troubleshooting

Issue: No PLDM Sensors Appearing on D-Bus

Symptom: `busctl tree xyz.openbmc_project.PLDM` shows no sensor objects.

Cause: PDR fetch failed, terminus not discovered, or platform-mc disabled.

Solution:

```
# Check if MCTP endpoints are discovered
busctl tree xyz.openbmc_project.MCTP

# Check pldmd status and logs
systemctl status pldmd
journalctl -u pldmd -f | grep -i "pdr\|sensor\|terminus"

# Manually query terminus for PLDM types
pldmtool base GetPLDMTypes -m <eid>

# Manually fetch PDRs to verify device responds
pldmtool platform GetPDRRepositoryInfo -m <eid>
pldmtool platform GetPDR -d 0 -m <eid>
```

Issue: Sensor Reads Return "unavailable"

Symptom: Sensor values show as NaN or unavailable on D-Bus/Redfish.

Cause: The device returned `sensorOperationalState = unavailable` or MCTP communication timed out.

Solution:

```
# Read the sensor directly to check operational state
pldmtool platform GetSensorReading -i <sensorID> -m <eid>

# Check MCTP connectivity
mctp endpoint

# Check for I2C errors if using I2C/SMBus transport
dmesg | grep -i "i2c\|mctp"

# Check if terminus is still responding
pldmtool base GetTID -m <eid>
```

Issue: Missing Thresholds in Redfish

Symptom: Redfish thermal sensors show readings but no threshold values.

Cause: The device's Numeric Sensor PDR does not populate threshold fields, or `RangeFieldSupport` bitmap is 0.

Solution:

```
# Check the PDR for threshold support
pldmtool platform GetPDR -d <sensorPDRHandle> -m <eid>
# Look for "rangeFieldSupport" and threshold values in the output

# If the device doesn't provide thresholds, you can set them
# via entity-manager configuration on the BMC side
```

Issue: Event Messages Not Received

Symptom: Sensor alarms don't trigger even when values exceed thresholds.

Cause: The device may not support async events, or event messaging is disabled.

Solution:

```
# Check device capabilities for event support
pldmtool platform GetPDR -d <sensorPDRHandle>
# Look for "sensorEventMessageEnable"

# Check if pldmd is receiving events
journalctl -u pldmd -f | grep -i "event\|platform_event"

# Verify the device supports PlatformEventMessage
pldmtool base GetPLDMCommands -t 2 -m <eid>
```

Debug Commands Summary

```
# Full PLDM platform debugging workflow
pldmtool base GetPLDMTypes -m <eid>           # What PLDM types?
pldmtool base GetPLDMCommands -t 2 -m <eid>   # What Type 2 commands?
pldmtool platform GetPDRRepositoryInfo -m <eid> # How many PDRs?
pldmtool platform GetPDR -d 0 -m <eid>        # Fetch first PDR
pldmtool platform GetSensorReading -i 1 -m <eid> # Read sensor 1
pldmtool platform GetStateSensorReadings -i 2 -m <eid> # Read state sensor

# Monitor sensor changes via D-Bus
busctl monitor xyz.openbmc_project.PLDM

# View all PLDM sensor values
busctl tree xyz.openbmc_project.PLDM | grep sensors
```

References

DMTF Specifications

- [DSP0248 — PLDM for Platform Monitoring and Control](#) — The core specification
- [DSP0245 — PLDM IDs and Codes](#) — Entity types, state sets, sensor units
- [DSP0240 — PLDM Base Specification](#) — Message format, terminus discovery

OpenBMC Repositories

- [openbmc/pldm](#) — PLDM daemon (includes platform-mc module)
- [openbmc/libpldm](#) — PLDM message encode/decode library
- [openbmc/phosphor-dbus-interfaces](#) — D-Bus sensor interfaces

Related Guides

- [MCTP & PLDM Guide](#) — MCTP transport and general PLDM overview
- [PLDM Firmware Update Guide](#) — PLDM Type 5 firmware update
- [SDR Guide](#) — IPMI SDR-based sensors (legacy alternative)

Tested on: OpenBMC master branch. PLDM monitoring requires real hardware with PLDM-capable devices or the PLDM emulator ([spdm-emu](#)). QEMU does not emulate PLDM terminus, but you can test D-Bus integration by manually populating sensor objects.

PLDM Firmware Update (Type 5)

Update device firmware using PLDM Type 5 (DSP0267) — package creation, update agent workflow, component transfer, activation, and integration with OpenBMC's update infrastructure.

Table of Contents

1. [Overview](#)
2. [Architecture](#)
 1. [Roles](#)
 2. [Update Flow Overview](#)
 3. [State Machine](#)
3. [PLDM Commands](#)
 1. [UA → FD Commands \(BMC sends to device\)](#)
 2. [FD → UA Commands \(Device sends to BMC\)](#)
 3. [Inventory Commands \(pre-update discovery\)](#)
4. [Firmware Update Package](#)
 1. [Package Structure](#)
 2. [Component Classifications](#)
 3. [Activation Methods](#)
5. [Creating Firmware Update Packages](#)
 1. [Using pldm_fwup_pkg_creator.py](#)
 2. [Metadata JSON Format](#)
 3. [Example: Package a GPU Firmware Update](#)
 4. [Device Descriptor Types](#)
6. [Complete Update Flow](#)
 1. [Step-by-Step Protocol Exchange](#)
 2. [Transfer Size and Chunking](#)
7. [Redfish Integration](#)
 1. [Upload via Redfish UpdateService](#)
 2. [Monitor Update Progress](#)
 3. [Query Device Firmware Versions](#)
8. [Multi-Component Updates](#)
9. [Build Configuration](#)
 1. [Enable PLDM Firmware Update in OpenBMC](#)
 2. [pldm Firmware Update Meson Options](#)
10. [Troubleshooting](#)
 1. [Issue: Package Not Accepted by Device](#)
 2. [Issue: Transfer Stalls](#)
 3. [Issue: Verify or Apply Fails](#)
 4. [Issue: Activation Requires Manual Reset](#)

5. Debug Commands Summary

11. References

1. [DMTF Specifications](#)
2. [OpenBMC Repositories](#)
3. [Related Guides](#)
4. [OCP Standards](#)

Overview

PLDM for Firmware Update (DMTF [DSP0267](#)) provides a standardized protocol for updating firmware on any PLDM-capable device — GPUs, NICs, NVMe drives, power supplies, Bridge ICs (OpenBIC), FPGAs, and more. It eliminates the need for vendor-specific update tools by defining a common update flow.

In OpenBMC, the `pldmd` daemon acts as the **Update Agent (UA)** — the entity that initiates and drives the firmware update process. The device being updated is the **Firmware Device (FD)**. The UA sends the firmware package, the FD pulls data from the UA, verifies the image, applies it, and optionally activates the new firmware.

Key concepts covered:

- PLDM firmware update package format (component images + metadata)
- Update Agent (UA) / Firmware Device (FD) roles and state machines
- The complete update flow: request → component transfer → verify → apply → activate
- Package creation with `pldm_fwup_pkg_creator.py`
- Integration with Redfish UpdateService
- Multi-component and multi-device update patterns

This guide covers PLDM-based firmware update for platform devices. For host BIOS flash update via SPI/GPIO mux, see the [BIOS Firmware Management Guide](#). For the general OpenBMC firmware update framework, see the [Firmware Update Guide](#).

Architecture

Roles

Role	Entity	Description
Update Agent (UA)	BMC (<code>pldmd</code>)	Initiates updates, serves firmware data to the FD
Firmware Device (FD)	Target device (GPU, NIC, BIC, etc.)	Receives firmware, verifies, applies, and activates

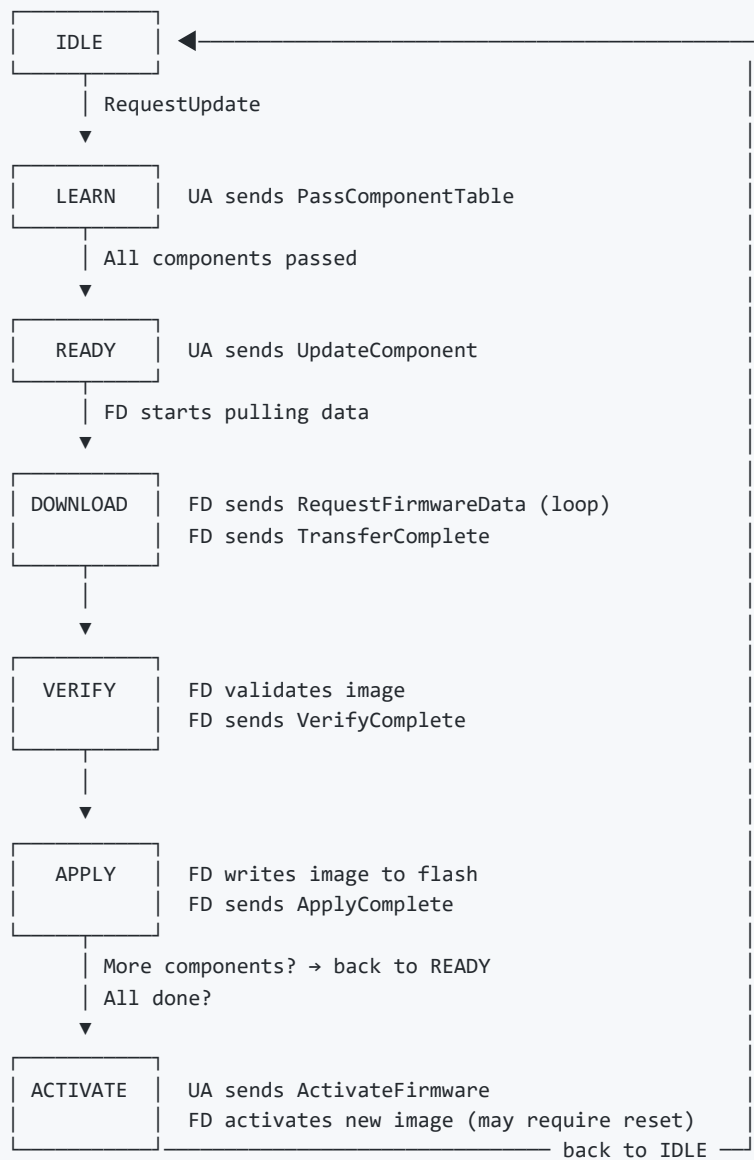
The key design principle: the **FD pulls data from the UA**. The UA does not push firmware directly — instead, the FD requests specific byte ranges of the component image from the UA via `RequestFirmwareData` commands.

Update Flow Overview



State Machine

The Firmware Device progresses through these states during an update:



PLDM Commands

UA → FD Commands (BMC sends to device)

Command	Code	Description
RequestUpdate	0x10	Initiate update, specify max transfer size
PassComponentTable	0x13	Send component metadata for FD to review
UpdateComponent	0x14	Start transferring a specific component
ActivateFirmware	0x1A	Activate new firmware (self-contained or pending reset)
GetStatus	0x1B	Query FD's current update state
CancelUpdateComponent	0x1C	Cancel current component transfer
CancelUpdate	0x1D	Cancel entire update process

FD → UA Commands (Device sends to BMC)

Command	Code	Description
<code>RequestFirmwareData</code>	0x15	Pull a chunk of component image from the UA
<code>TransferComplete</code>	0x16	All data received for current component
<code>VerifyComplete</code>	0x17	Image verification succeeded or failed
<code>ApplyComplete</code>	0x18	Image applied to storage

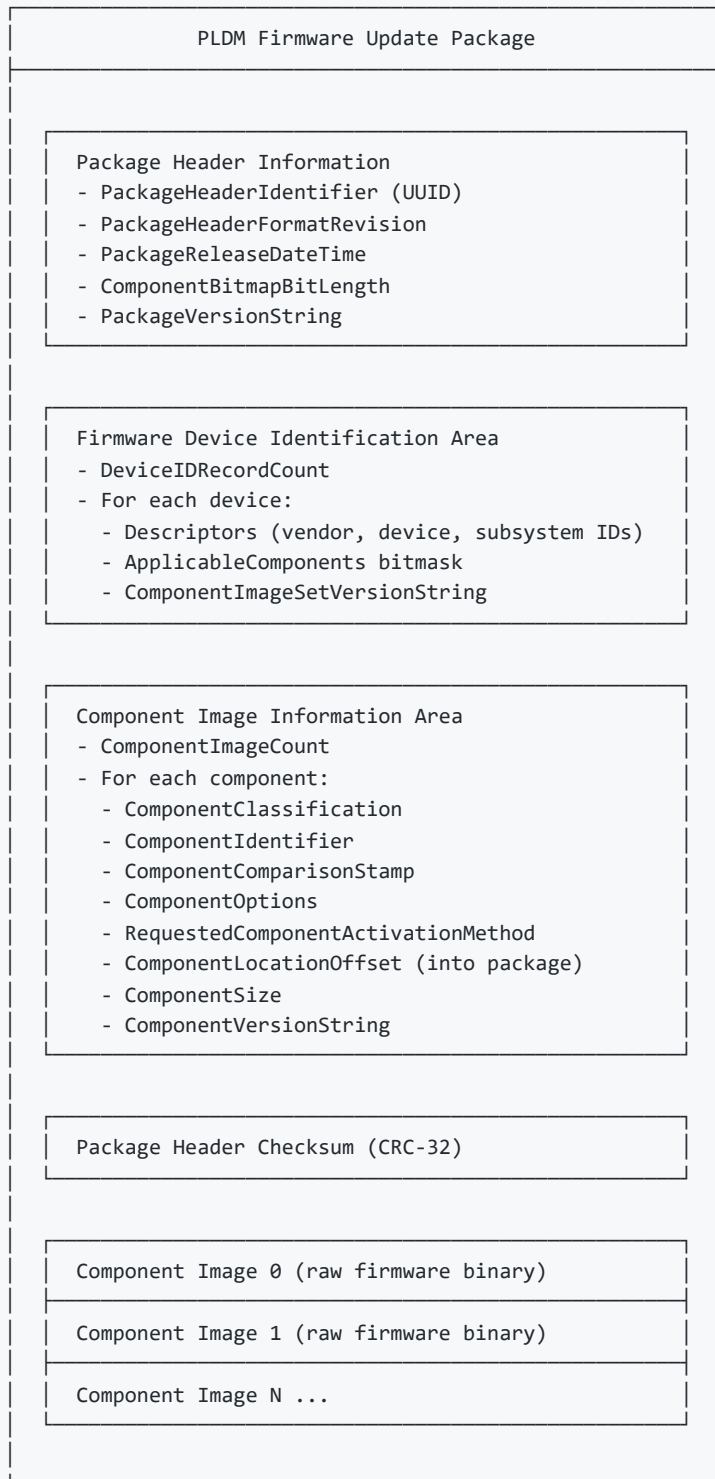
Inventory Commands (pre-update discovery)

Command	Code	Description
<code>QueryDeviceIdentifiers</code>	0x01	Get device descriptors (vendor, model, serial)
<code>GetFirmwareParameters</code>	0x02	Get current firmware versions and component info

Firmware Update Package

Package Structure

A PLDM firmware update package bundles one or more component images with metadata:



Component Classifications

Classification	ID	Description
Unknown	0x0000	Unspecified
Other	0x0001	Vendor-defined
Firmware	0x000A	Device firmware
BIOS/UEFI	0x000C	Host BIOS image
CPLD	0x0010	CPLD bitstream
Diagnostic	0x000D	Diagnostic software
Operating System	0x000E	OS image
Middleware	0x000F	Middleware software

Activation Methods

Method	Bit	Description
Automatic	0	FD activates immediately after apply
Self-Contained	1	FD can activate without external trigger
Medium-Specific Reset	2	Requires bus reset (PCIe, I2C)
System Reboot	3	Requires full system reboot
DC Power Cycle	4	Requires power cycle
AC Power Cycle	5	Requires AC power cycle
Pending Image	6	FD holds image; UA sends ActivateFirmware later

Creating Firmware Update Packages

Using pldm_fwup_pkg_creator.py

OpenBMC provides a Python script to create PLDM firmware update packages from raw component images and a metadata JSON file.

```
# Location in the pldm repository
# tools/fw-update/pldm_fwup_pkg_creator.py

# Usage
python3 pldm_fwup_pkg_creator.py \
    <output_package_name> \
    <metadata_json> \
    <component_image_1> [component_image_2] ...
```

Metadata JSON Format

```
{
  "PackageHeaderInformation": {
    "PackageHeaderIdentifier": "f018878c-cb7d-4943-9800-a02f059aca02",
    "PackageHeaderFormatRevision": 1,
    "PackageReleaseDateTime": "2026-02-07T00:00:00Z",
    "ComponentBitmapBitLength": 8,
    "PackageVersionString": "1.0.0"
  },
  "FirmwareDeviceIdentificationArea": [
    {
      "DeviceUpdateOptionFlags": 0,
      "ApplicableComponents": [0],
      "ComponentImageSetVersionString": "2.1.0",
      "FirmwareDevicePackageDataLength": 0,
      "Descriptors": [
        {
          "DescriptorType": 0,
          "DescriptorData": "0x10DE"
        },
        {
          "DescriptorType": 1,
          "DescriptorData": "0x1234"
        }
      ]
    }
  ],
  "ComponentImageInformationArea": [
    {
      "ComponentClassification": 10,
      "ComponentIdentifier": 1,
      "ComponentComparisonStamp": 0,
      "ComponentOptions": 0,
      "RequestedComponentActivationMethod": 6,
      "ComponentVersionString": "2.1.0"
    }
  ]
}
```

Example: Package a GPU Firmware Update

```
# Create a single-component package for a GPU firmware
python3 pldm_fwup_pkg_creator.py \
  gpu_firmware_v2.1.0.pldm \
  gpu_metadata.json \
  gpu_firmware.bin

# Create a multi-component package (firmware + CPLD)
python3 pldm_fwup_pkg_creator.py \
  gpu_full_update_v2.1.0.pldm \
  gpu_multi_metadata.json \
  gpu_firmware.bin \
  gpu_cpld.bin
```

Device Descriptor Types

Descriptors in the metadata identify which device the package targets:

Type	ID	Description	Example
PCI Vendor ID	0	PCIe vendor	<code>0x10DE</code> (NVIDIA)
PCI Device ID	1	PCIe device	<code>0x1234</code>
PCI Subsystem Vendor ID	2	Subsystem vendor	<code>0x10DE</code>
PCI Subsystem ID	3	Subsystem device	<code>0x5678</code>
PCI Revision ID	4	Revision	<code>0x01</code>
IANA Enterprise ID	5	IANA enterprise number	<code>0x00A015</code>
UUID	6	Device UUID	RFC 4122 format
Vendor Defined	0xFFFF	OEM-specific	Arbitrary

Complete Update Flow

Step-by-Step Protocol Exchange





Transfer Size and Chunking

The `MaxTransferSize` negotiated in `RequestUpdate` determines the maximum bytes per `RequestFirmwareData` exchange. Typical values:

Transport	Typical MaxTransferSize	Notes
MCTP over I2C/SMBus	256 bytes	Limited by I2C transaction size
MCTP over PCIe VDM	4096 bytes	PCIe allows larger payloads
MCTP over I3C	1024 bytes	I3C higher bandwidth than I2C

Larger transfer sizes significantly reduce update time. A 2 MB firmware image with 256-byte chunks requires ~8000 round-trips, while 4096-byte chunks need only ~500.

Redfish Integration

Upload via Redfish UpdateService

```
# Upload a PLDM firmware update package via Redfish
curl -k -u root:openBmc \
  -X POST \
  -H "Content-Type: application/octet-stream" \
  --data-binary @gpu_firmware_v2.1.0.pldm \
  https://${BMC_IP}/redfish/v1/UpdateService

# Multipart upload targeting a specific device
curl -k -u root:openBmc \
  -X POST \
  -F 'UpdateParameters={"Targets":["/redfish/v1/Chassis/GPU_0"]};type=application/json' \
  -F 'UpdateFile=@gpu_firmware_v2.1.0.pldm;type=application/octet-stream' \
  https://${BMC_IP}/redfish/v1/UpdateService/update
```

Monitor Update Progress

```
# Check update task progress
curl -k -u root:openBmc \
  https://${BMC_IP}/redfish/v1/TaskService/Tasks/0

# Example response:
# {
#   "TaskState": "Running",
#   "PercentComplete": 65,
#   "Messages": [
#     {"Message": "Transferring component 1 of 1"},
#     {"Message": "Downloaded 1.3 MB of 2.0 MB"}
#   ]
# }

# Check firmware inventory after update
curl -k -u root:openBmc \
  https://${BMC_IP}/redfish/v1/UpdateService/FirmwareInventory
```

Query Device Firmware Versions

```
# Query current firmware versions via PLDM inventory
pldmtool fw_update GetFirmwareParameters -m <eid>

# Example output:
# {
#   "ActiveComponentImageSetVersionString": "2.0.0",
#   "PendingComponentImageSetVersionString": "2.1.0",
#   "ComponentParameterEntries": [
#     {
#       "ComponentClassification": "Firmware",
#       "ComponentIdentifier": 1,
#       "ActiveComponentVersionString": "2.0.0",
#       "PendingComponentVersionString": "2.1.0"
#     }
#   ]
# }
```

Multi-Component Updates

A single PLDM firmware package can contain multiple component images — for example, main firmware + CPLD bitstream + configuration data. The UA updates each component sequentially:

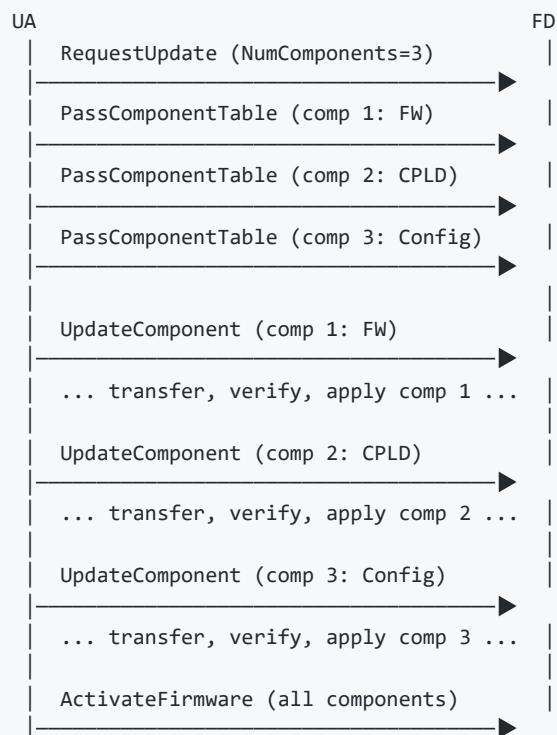


Image signature verification is performed by the Firmware Device, not the BMC. The UA performs only compatibility checks (matching device descriptors). The FD is responsible for cryptographic validation of the image before applying it.

Build Configuration

Enable PLDM Firmware Update in OpenBMC

```
# In your machine .conf or local.conf
EXTRA_OEMESON:pn-pldm += " \
    -Dfirmware-update=enabled \
"

# Include pldm in the image
IMAGE_INSTALL:append = " pldm"

# Optional: include the package creator tool for development
IMAGE_INSTALL:append = " pldm-tools"
```

pldmd Firmware Update Meson Options

Option	Default	Description
<code>firmware-update</code>	<code>enabled</code>	Enable PLDM Type 5 firmware update
<code>maximum-transfer-size</code>	<code>4096</code>	Max bytes per RequestFirmwareData

Troubleshooting

Issue: Package Not Accepted by Device

Symptom: `RequestUpdate` or `PassComponentTable` returns an error.

Cause: Device descriptor mismatch — the package targets a different device.

Solution:

```
# Query what the device reports as its identifiers
pldmtool fw_update QueryDeviceIdentifiers -m <eid>

# Compare with your package metadata JSON
# Vendor ID, Device ID, and descriptors must match exactly

# Check if the device supports the requested activation method
pldmtool fw_update GetFirmwareParameters -m <eid>
```

Issue: Transfer Stalls

Symptom: Update progress stops at a certain percentage.

Cause: MCTP transport timeout, I2C bus error, or FD internal error.

Solution:

```
# Check PLDM update status
pldmtool fw_update GetStatus -m <eid>

# Check pldmd logs for transfer errors
journalctl -u pldmd -f | grep -i "fw_update\|firmware\|transfer"

# Check MCTP connectivity
pldmtool base GetTID -m <eid>

# Check for I2C/bus errors
dmesg | grep -i "i2c\|mctp"
```

Issue: Verify or Apply Fails

Symptom: `VerifyComplete` or `ApplyComplete` returns a failure result.

Cause: Image signature invalid, incompatible firmware version, or flash write error on the FD.

Solution:

```
# Check the completion code in pldm logs
journalctl -u pldm --no-pager | grep -i "verify\|apply"

# VerifyResult codes:
# 0 = Success
# 1 = GeneralError
# 2 = VersionMismatch
# 3 = SecurityCheckFail
# 4 = IncompleteUpdate

# If signature fails, verify the image is correctly signed
# for the target device's trust chain
```

Issue: Activation Requires Manual Reset

Symptom: New firmware is applied but device still runs old version.

Cause: The FD's `RequestedComponentActivationMethod` requires a reset that hasn't occurred.

Solution:

```
# Check what activation method the device requires
pldmtool fw_update GetFirmwareParameters -m <eid>
# Look for ActivationMethod field

# If "Medium-Specific Reset" – reset the bus
# If "DC Power Cycle" – power cycle the device
# If "System Reboot" – reboot the host

# After reset, verify new firmware is active
pldmtool fw_update GetFirmwareParameters -m <eid>
# ActiveComponentVersionString should show new version
```

Debug Commands Summary

```
# Pre-update discovery
pldmtool fw_update QueryDeviceIdentifiers -m <eid>
pldmtool fw_update GetFirmwareParameters -m <eid>

# During update
pldmtool fw_update GetStatus -m <eid>

# Monitor pldm update progress
journalctl -u pldm -f | grep -i fw_update

# Cancel a stuck update
pldmtool fw_update CancelUpdate -m <eid>
```

References

DMTF Specifications

- [DSP0267 — PLDM for Firmware Update](#) — Core firmware update specification
- [DSP0240 — PLDM Base Specification](#) — Message format and base commands
- [DSP0245 — PLDM IDs and Codes](#) — Descriptor types, component classifications

OpenBMC Repositories

- [openbmc/pldm](#) — PLDM daemon (fw-update module)
- [openbmc/pldm/tools/fw-update](#) — Package creator script
- [NVIDIA/PLDM-unpack](#) — Tool to inspect/unpack PLDM firmware packages

Related Guides

- [MCTP & PLDM Guide](#) — MCTP transport and general PLDM concepts
- [PLDM Platform Monitoring Guide](#) — PLDM Type 2 sensors and effecters
- [Firmware Update Guide](#) — General OpenBMC firmware update framework
- [BIOS Firmware Management Guide](#) — SPI-based BIOS flash and PLDM BIOS config

OCP Standards

- [OCP GPU Firmware Update Specification](#) — GPU-specific PLDM firmware update guidance

Tested on: OpenBMC master branch. PLDM firmware update requires real hardware with PLDM-capable firmware devices. Use `pldm_fwup_pkg_creator.py` to create test packages for development.

DICE Security Guide

Understand the Device Identifier Composition Engine (DICE) — a hardware root-of-trust standard for device identity and firmware attestation, and how it integrates with OpenBMC's SPDM attestation infrastructure.

Table of Contents

1. [Overview](#)
 1. [Why DICE Matters for OpenBMC](#)
 2. [DICE vs. TPM](#)
 2. [DICE Architecture](#)
 1. [Core Concepts](#)
 2. [The DICE Derivation Chain](#)
 3. [The Certificate Chain](#)
 4. [The Critical Security Property](#)
 3. [DICE and SPDM Integration](#)
 1. [Authentication Flow](#)
 2. [What the BMC Learns](#)
 3. [OpenBMC SPDM Daemon Integration](#)
 4. [BMC as a DICE Attester](#)
 1. [BMC DICE Boot Chain](#)
 2. [Relationship to AST2600 Secure Boot](#)
 5. [DICE Provisioning](#)
 1. [Manufacturing-Time Setup](#)
 2. [Certificate Authority Setup](#)
 6. [Attestation Policy](#)
 1. [Defining Expected Measurements](#)
 2. [Policy Actions](#)
 3. [Firmware Update and DICE](#)
 7. [DICE Profiles](#)
 1. [Open DICE Profile \(Google\)](#)
 2. [TCG DICE Attestation Architecture](#)
 3. [Android DICE](#)
 8. [Relationship to Other OpenBMC Security Features](#)
 9. [Summary](#)
 10. [References](#)
-

Overview

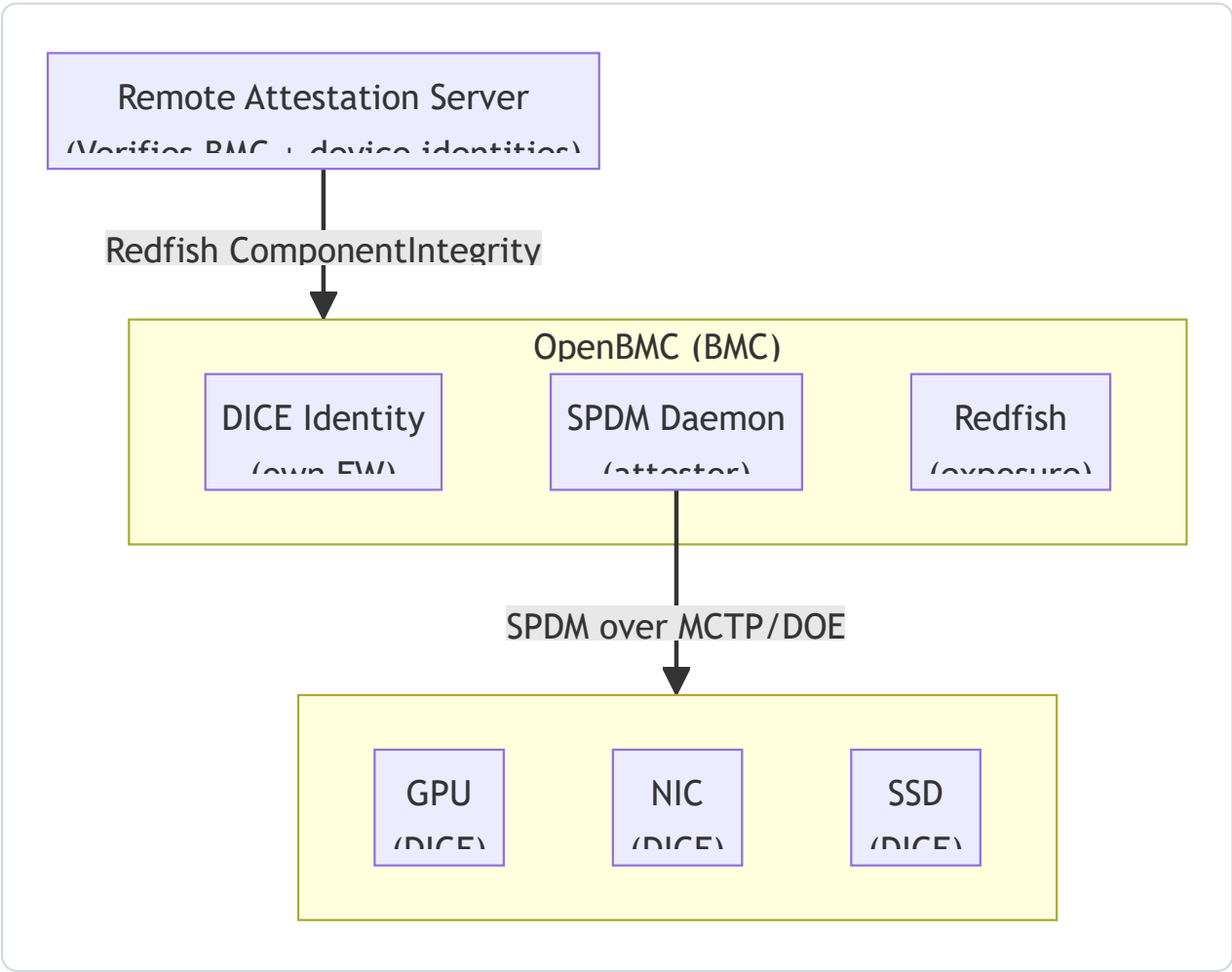
DICE (Device Identifier Composition Engine) is a TCG (Trusted Computing Group) standard that provides hardware-rooted cryptographic device identity and firmware attestation. It is designed for components where a full TPM is impractical — SSDs, NICs, GPUs, power supplies, BMCs themselves, and other embedded controllers.

DICE creates **unique cryptographic identities that are bound to the device’s firmware**. If firmware changes, the identity changes. This property makes DICE ideal for supply chain security and runtime firmware attestation.

Why DICE Matters for OpenBMC

OpenBMC operates in two DICE-related roles:

Role	Description
Verifier	The BMC attests DICE-enabled platform components (GPUs, NICs, SSDs) via SPDM, where those devices use DICE to generate their SPDM keys
Attester	The BMC itself can use DICE for its own firmware identity, allowing the host or a remote verifier to attest the BMC



DICE vs. TPM

	DICE	TPM
Hardware cost	Minimal — ROM + fuse	Separate chip or firmware TPM
Suitable for	SSDs, NICs, GPUs, MCUs, BMCs	PCs, servers, laptops
Identity	Firmware-coupled (changes with FW update)	Stable (Endorsement Key persists)
Measurement	Built into boot chain — each layer measures next	Explicit PCR extend operations
Attestation	Certificate chain from boot layers	TPM quote over PCR values
Sealing	CDI-derived sealing keys	TPM seal/unseal with PCR policy
Standard	TCG DICE specs	TCG TPM 2.0 spec

DICE Architecture

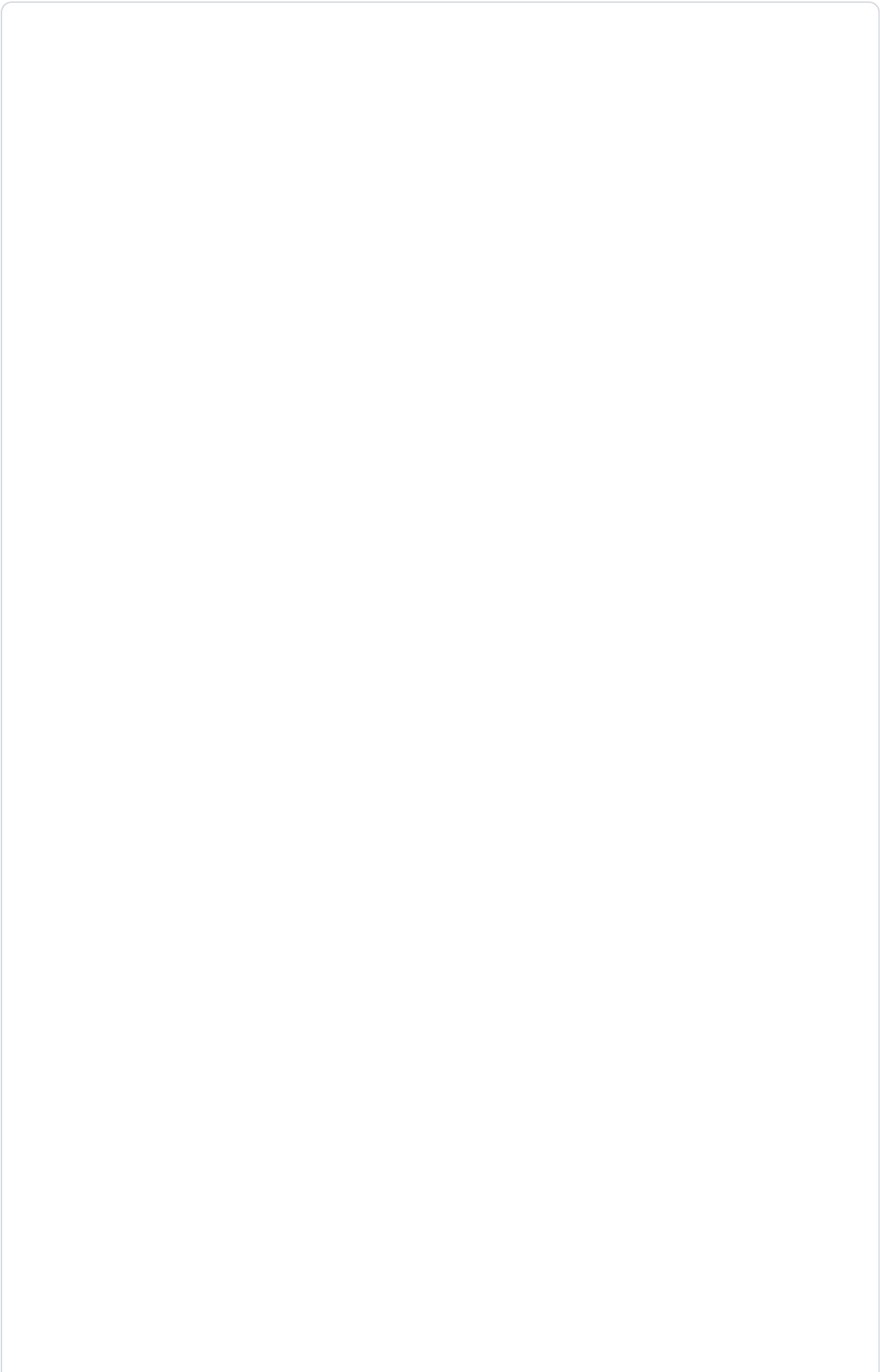
Core Concepts

DICE relies on three foundational elements:

Term	What It Is
UDS (Unique Device Secret)	Per-device random secret fused in hardware at manufacturing. Never accessible to software — only the DICE ROM engine uses it.
CDI (Compound Device Identifier)	Derived from UDS + measurement of the first mutable firmware. Unique to this device AND this firmware version.
TCI (TCB Component Identifier)	The measurement (hash) of a firmware layer — its code, configuration, and security-relevant state.

The DICE Derivation Chain

The fundamental operation: each boot layer **measures the next** and derives a unique secret for it. If any firmware changes, all downstream secrets change.



Hardware (Immutable ROM)

UDS (Unique Device Secret)

- Fused at manufacturing
- Unique per device
- Never readable by software after DICE engine runs

DICE Engine

CDI

Layer 0 (First Mutable Firmware)

DeviceID Key Pair = $\text{KDF}(\text{CDI}, \text{'identity'})$

- Represents: this device + this ROM
- Stable across L0 firmware updates

Alias Key Pair = $\text{KDF}(\text{CDI}, \text{Hash}(\text{Layer 1 firmware image}))$

- Represents: this device + this ROM + this L0 + this L1
- Changes if Layer 1 firmware is updated

Issue Alias Certificate (signed by DeviceID private key)

Alias Key + Certificate Chain

Layer 1 (Main Firmware / RTOS / Linux)

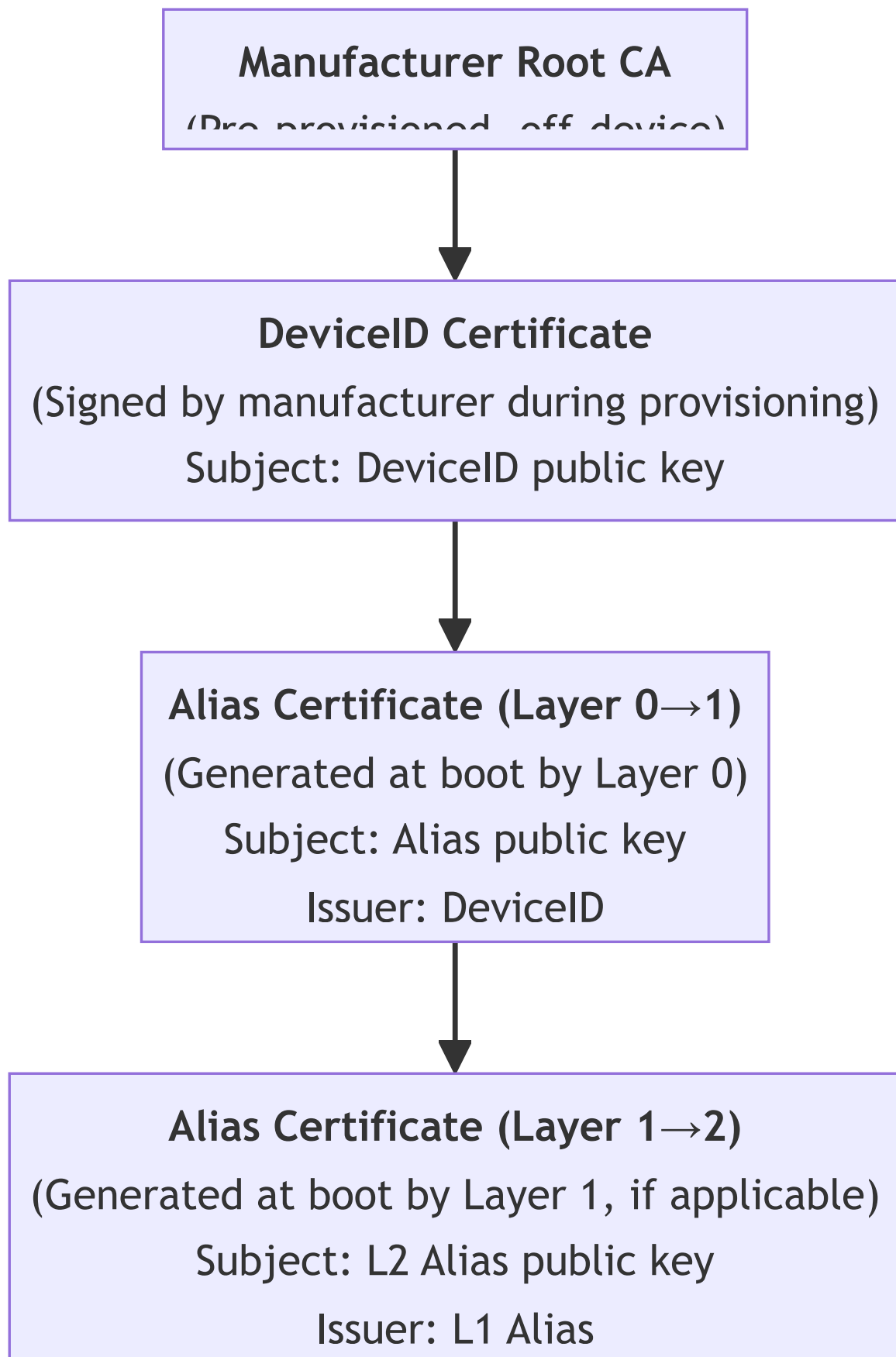
Uses Alias key for:

USES Alias Key For:

- SPDm challenge responses
- TLS client authentication
- Firmware attestation signing
- Secure session establishment

The Certificate Chain

Each DICE layer produces a certificate that chains to the previous layer:



A verifier (the BMC via SPDH, or a remote server via Redfish) validates the entire chain:

1. **Root CA** is trusted (pre-configured)
2. **DeviceID cert** chains to root CA
3. **Alias cert** chains to DeviceID
4. **TCI values** in certificate extensions match expected firmware hashes

The Critical Security Property

Each layer **wipes its secrets** before passing control to the next:

- Hardware wipes UDS after computing CDI
- Layer 0 wipes CDI after computing DeviceID and Alias keys
- Layer 0 wipes DeviceID private key after issuing Alias cert

If Layer 1 (main firmware) is compromised, the attacker gets the Alias key for **this firmware version only**. They cannot:

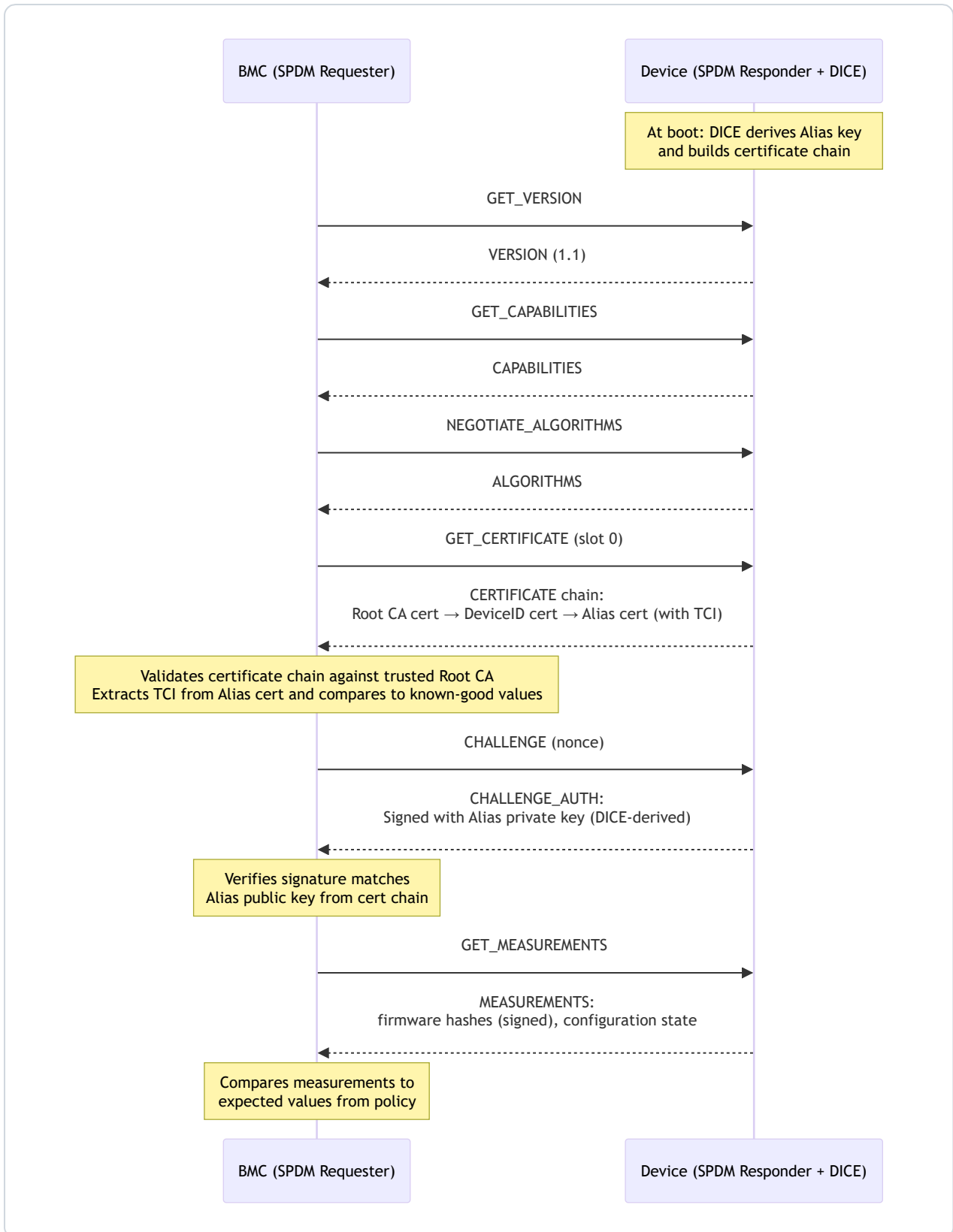
- Forge an Alias key for a different firmware version
- Recover the DeviceID private key
- Recover the UDS

Updating firmware automatically rotates the Alias identity, invalidating any compromised keys.

DICE and SPDM Integration

DICE provides the **key material and certificates**; SPDM provides the **protocol** to present them to a verifier. This is how the BMC verifies DICE-enabled devices:

Authentication Flow



What the BMC Learns

SPDM Response	DICE-Provided Content	What It Proves
<code>GET_CERTIFICATE</code>	Alias cert chain (DeviceID → Alias)	Device is genuine, manufactured by trusted vendor
<code>CHALLENGE_AUTH</code>	Signature from DICE-derived Alias key	Device holds the private key for this firmware version
<code>GET_MEASUREMENTS</code>	Firmware hashes from DICE layers	Firmware has not been tampered with
TCI in Alias cert extension	Hash of measured firmware	Firmware identity is cryptographically bound to the device key

OpenBMC SPDM Daemon Integration

The OpenBMC SPDM daemon (`spdm`) handles the SPDM protocol. For DICE-enabled devices, the daemon:

- 1. Discovers SPDM-capable devices via MCTP or PCIe DOE
- 2. Runs the SPDM authentication flow (above)
- 3. Validates the DICE certificate chain against configured trusted CAs
- 4. Extracts TCI measurements from certificate extensions
- 5. Compares measurements against expected values (policy)
- 6. Exposes results via D-Bus and Redfish `ComponentIntegrity`

```
# Check SPDM attestation status for a DICE-enabled device
busctl get-property xyz.openbmc_project.SPDM \
  /xyz/openbmc_project/spdm/device/gpu0 \
  xyz.openbmc_project.SPDM.Device \
  AuthenticationStatus

# Query via Redfish
curl -k -u admin:password \
  https://{BMC_IP}/redfish/v1/ComponentIntegrity/gpu0
```

Example Redfish `ComponentIntegrity` response for a DICE-attested device:


```

{
  "@odata.id": "/redfish/v1/ComponentIntegrity/gpu0",
  "ComponentIntegrityType": "SPDM",
  "ComponentIntegrityTypeVersion": "1.1.0",
  "SPDMIdentity": {
    "RequesterAuthentication": {
      "VerificationStatus": "Success"
    },
    "ResponderAuthentication": {
      "ComponentCertificate": {
        "@odata.id": "/redfish/v1/Managers/bmc/Certificates/SPDM_gpu0"
      },
      "VerificationStatus": "Success"
    }
  },
  "ComponentIntegrityEnabled": true,
  "TargetComponentURI": "/redfish/v1/Systems/system/Processors/gpu0",
  "Status": {
    "State": "Enabled",
    "Health": "OK"
  }
}

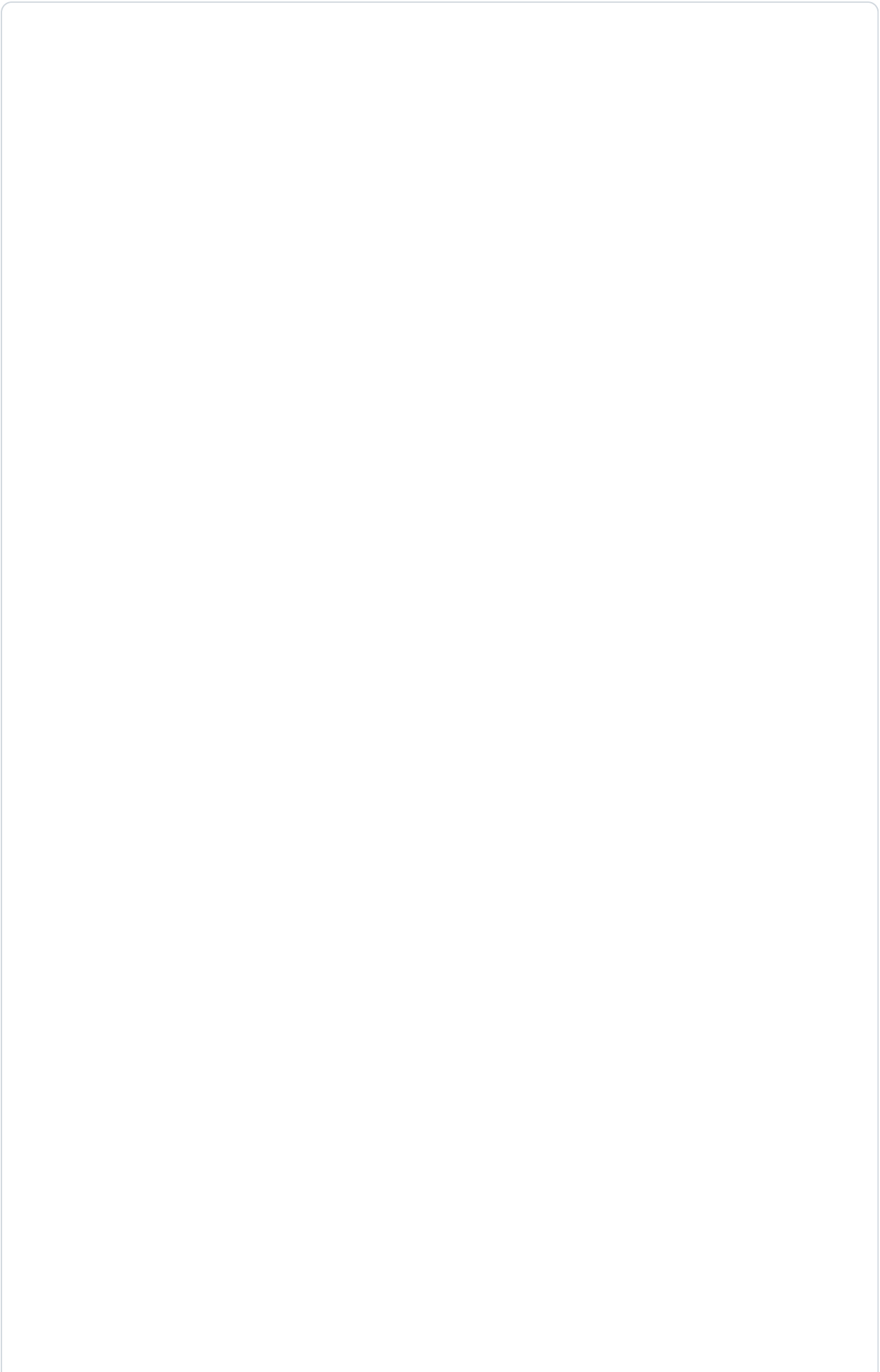
```

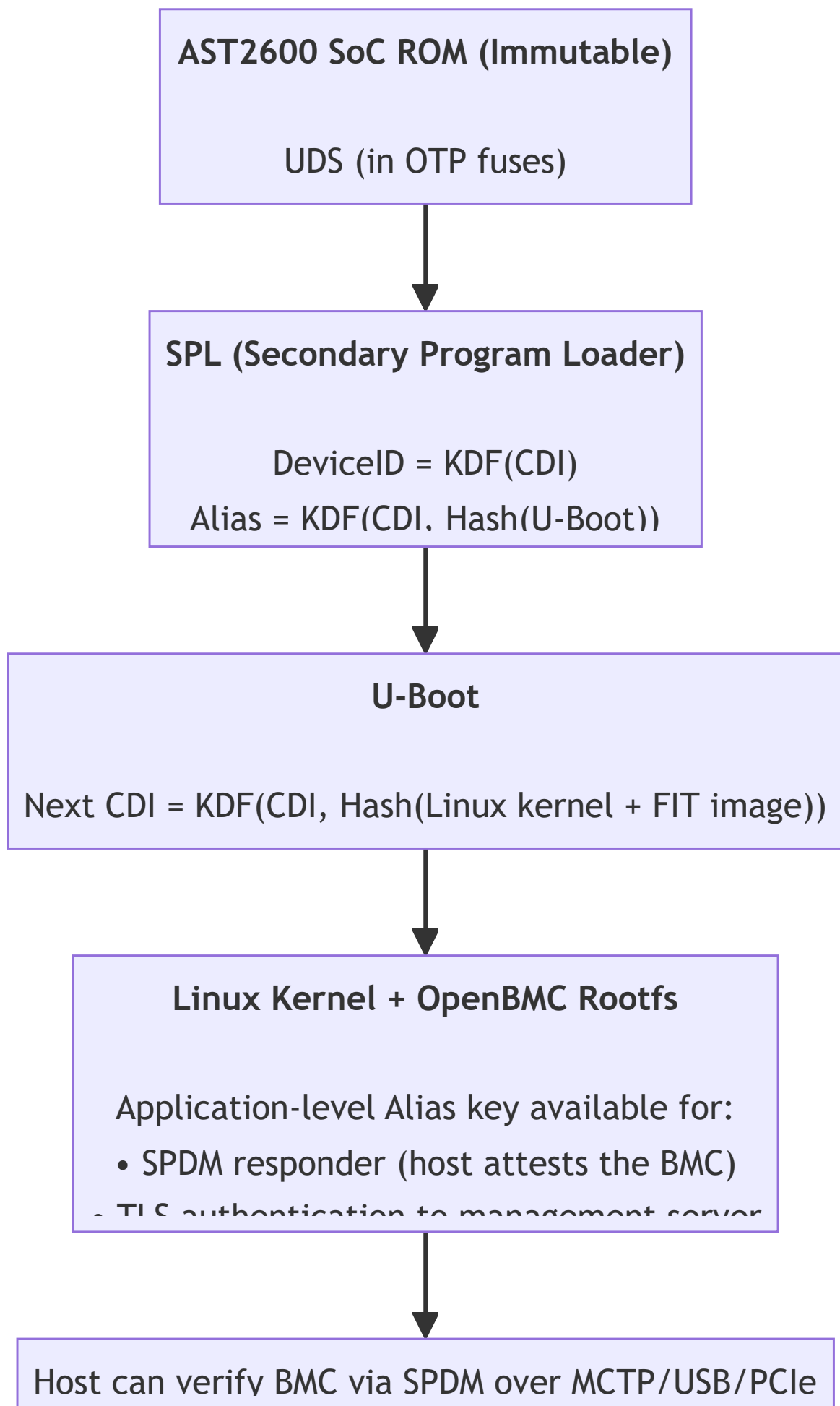
BMC as a DICE Attester

The BMC itself can use DICE to establish its own firmware identity. This enables the host or a remote management server to verify the BMC firmware hasn't been tampered with.

BMC DICE Boot Chain

On an AST2600-based OpenBMC system:





Relationship to AST2600 Secure Boot

DICE and [Secure Boot](#) are complementary:

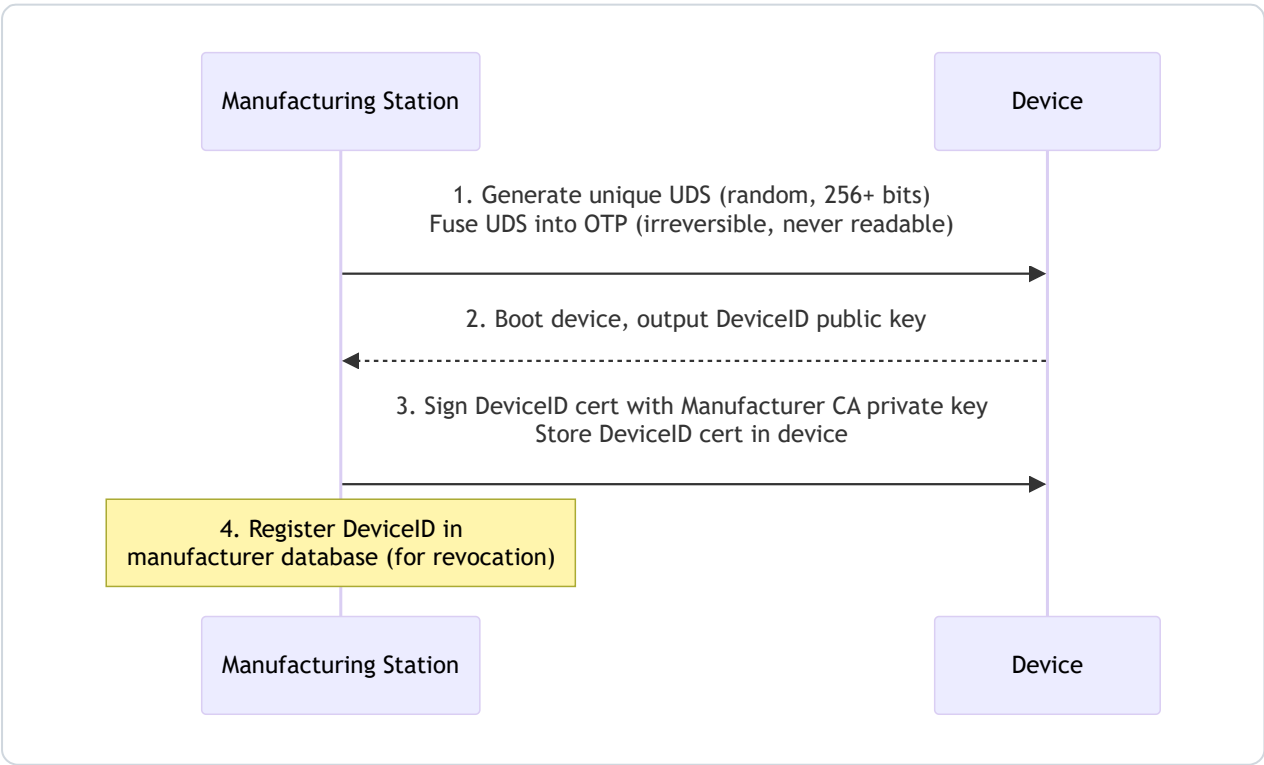
Feature	Secure Boot	DICE
Purpose	Prevent unauthorized firmware from running	Prove which firmware IS running
Mechanism	Signature verification at each boot stage	Measurement and key derivation at each boot stage
Failure mode	Refuses to boot (hard stop)	Boots but with wrong identity (attestation fails remotely)
What it guarantees	Only signed firmware executes	Firmware identity is cryptographically verifiable
OTP keys	Verification keys (public)	UDS (secret)

Best practice: **use both together**. Secure boot prevents unauthorized code from executing. DICE proves to remote verifiers exactly which authorized code is running.

DICE Provisioning

Manufacturing-Time Setup

DICE requires one-time provisioning at the factory:



Certificate Authority Setup

For an OpenBMC deployment verifying DICE devices, configure the trusted CA certificates:

```
# Install manufacturer root CA for each device vendor
# The BMC uses these to validate DICE certificate chains

# GPU vendor root CA
busctl call xyz.openbmc_project.Certs.Manager.Authority.SPDM \
  /xyz/openbmc_project/certs/authority/spdm \
  xyz.openbmc_project.Certs.Install \
  Install s "/tmp/gpu_vendor_root_ca.pem"

# NIC vendor root CA
busctl call xyz.openbmc_project.Certs.Manager.Authority.SPDM \
  /xyz/openbmc_project/certs/authority/spdm \
  xyz.openbmc_project.Certs.Install \
  Install s "/tmp/nic_vendor_root_ca.pem"

# SSD vendor root CA
busctl call xyz.openbmc_project.Certs.Manager.Authority.SPDM \
  /xyz/openbmc_project/certs/authority/spdm \
  xyz.openbmc_project.Certs.Install \
  Install s "/tmp/ssd_vendor_root_ca.pem"
```

Or via Redfish:

```
# Install trusted CA certificate
curl -k -u admin:password \
  -X POST \
  -H "Content-Type: application/json" \
  -d '{"CertificateString": "<PEM-encoded-cert>", "CertificateType": "PEM"}' \
  https://${BMC_IP}/redfish/v1/Managers/bmc/Truststore/Certificates
```

Attestation Policy

Defining Expected Measurements

The BMC needs to know what firmware measurements to expect for each DICE-enabled device. This is typically a JSON policy file:

```
{
  "devices": {
    "gpu0": {
      "vendor": "NVIDIA",
      "expected_measurements": {
        "firmware_version": "535.129.03",
        "tci_layer0": "sha256:a1b2c3d4e5f6....",
        "tci_layer1": "sha256:f6e5d4c3b2a1...."
      },
      "trusted_ca": "/etc/spdm/certs/nvidia_root_ca.pem",
      "policy": "enforce"
    },
    "nic0": {
      "vendor": "Broadcom",
      "expected_measurements": {
        "firmware_version": "230.0.148.0",
        "tci_layer0": "sha256:1a2b3c4d5e6f...."
      },
      "trusted_ca": "/etc/spdm/certs/broadcom_root_ca.pem",
      "policy": "log_only"
    }
  }
}
```

Policy Actions

Policy	On Attestation Success	On Attestation Failure
enforce	Device operational, Health=OK	Device isolated, Redfish alert, Health=Critical
log_only	Device operational, Health=OK	Device operational, event logged, Health=Warning
disabled	No attestation performed	No attestation performed

Firmware Update and DICE

When a device firmware is updated, the DICE Alias key **automatically changes** (because the firmware measurement changes). The attestation policy must be updated with the new expected TCI values:

1. Vendor releases new firmware + new expected TCI hash
2. Admin updates policy file with new TCI
3. Device firmware is updated (via PLDM FW Update or vendor tool)
4. Device reboots – DICE derives new Alias key
5. BMC re-attests – new Alias cert has new TCI
6. BMC verifies new TCI matches updated policy
7. Attestation passes with new firmware identity

If the policy is not updated before firmware update, attestation will **fail** because the TCI will not match. Plan policy updates as part of your firmware update workflow.

DICE Profiles

Open DICE Profile (Google)

The [Open Profile for DICE](#) is a concrete implementation specification by Google. It standardizes:

- **CDI derivation:** HKDF-SHA256/SHA384/SHA512
- **Key types:** Ed25519, P-256, P-384
- **Certificate format:** X.509 v3 with DICE-specific extensions, or CBOR (for constrained devices)
- **Measurement format:** Standardized TCI structure

Used by Android (for hardware key attestation) and Pigweed (for embedded MCUs).

TCG DICE Attestation Architecture

The TCG's [DICE Attestation Architecture](#) (also called "Implicit Identity Based Device Attestation") defines:

- How DICE identities map to SPDH certificate slots
- Measurement reporting formats
- Certificate extension OIDs for TCI values
- Interaction with platform TPMs (hybrid DICE + TPM scenarios)

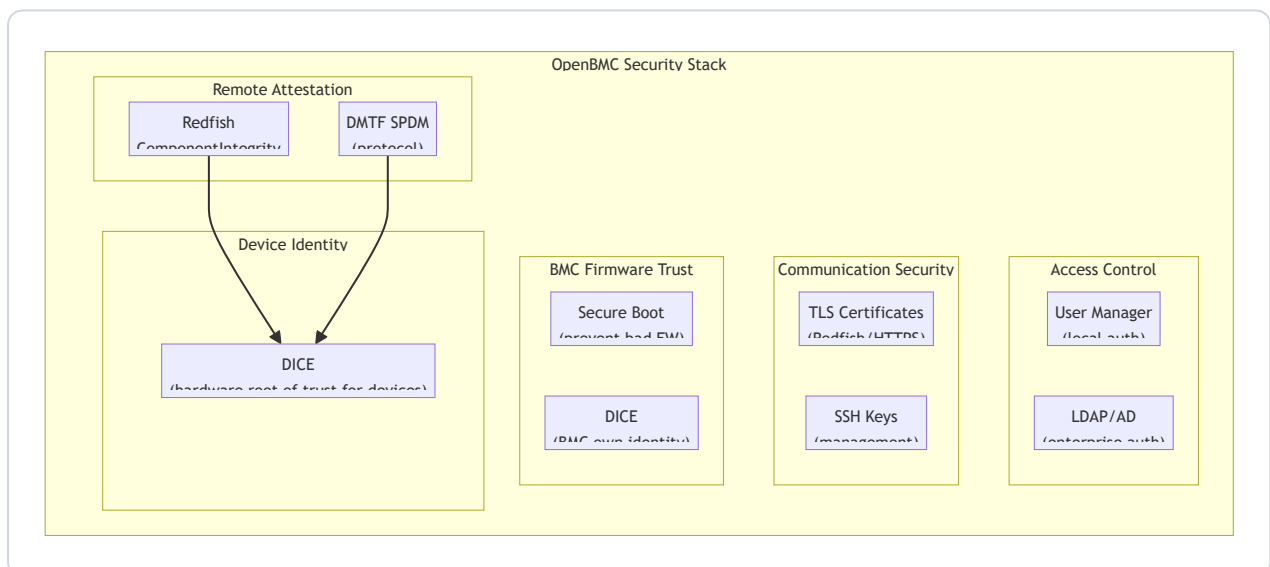
Android DICE

[Android DICE](#) extends the DICE chain through the full Android boot:

ROM → Bootloader → ABL → Kernel → microdroid → App

Each layer has its own CDI and certificate. Android uses this for hardware-backed key attestation (KeyMint/Keymaster) to prove to remote services which OS version and security patch level is running.

Relationship to Other OpenBMC Security Features



Feature	Guide	Relationship to DICE
SPDM	SPDM Guide	Transport protocol for DICE attestation — carries certificates and measurements
Secure Boot	Secure Boot Guide	Complementary — Secure Boot prevents bad FW; DICE proves which FW is running
Certificate Manager	Certificate Manager Guide	Manages trusted CA certs used to validate DICE chains
MCTP/PLDM	MCTP/PLDM Guide	Physical transport for SPDM messages to DICE-enabled devices
Firmware Update	Firmware Update Guide	FW updates change DICE identity — policy must be coordinated

Summary

Concept	Description
UDS	Per-device hardware secret, fused at manufacturing, never software-accessible
CDI	Derived from UDS + firmware measurement, unique to device + firmware version
DeviceID	Stable device identity key, derived from CDI
Alias Key	Firmware-version-specific key, used for SPDM responses
Certificate Chain	Root CA → DeviceID → Alias, validates with standard X.509
TCI	Firmware measurement hash, embedded in certificate extensions
Key rotation	Automatic on firmware update — new FW = new Alias key
BMC role (verifier)	Uses SPDM to validate DICE chains from platform devices
BMC role (attester)	Uses DICE for its own identity, allowing host/remote verification

References

- [TCG DICE Overview](#) - What is DICE
- [TCG DICE Hardware Requirements](#) - Hardware specification
- [TCG DICE Layering Architecture](#) - Multi-layer derivation
- [TCG DICE Attestation Architecture](#) - Implicit identity based attestation
- [Open Profile for DICE \(Google\)](#) - Concrete implementation spec
- [Android DICE](#) - Android's DICE implementation
- [DICE - Microsoft Research](#) - Microsoft's DICE research
- [DMTF SPDM Specification](#) - Protocol for transporting DICE attestation
- [OpenBMC Redfish SPDM Attestation Design](#) - OpenBMC integration
- [ARM TF-M DICE Protection Environment](#) - ARM Cortex-M DICE implementation

DICE is a device firmware standard. OpenBMC developers primarily interact with DICE through the SPDm attestation flow — the BMC validates DICE certificate chains from devices. Implementing DICE itself is a device firmware task. For the SPDm protocol details, see the [SPDM Guide](#).

SGPIO (Serial GPIO)

Expand BMC signal count over a few shared pins using the ASPEED serial GPIO master — define it in the device tree, enable the kernel driver, and use it from userspace exactly like parallel GPIO.

Table of Contents

1. [Overview](#)
2. [SGPIO vs Parallel GPIO](#)
3. [Architecture](#)
 1. [The Serial Master Protocol](#)
 2. The `ngpios` Doubling Rule
4. [Defining SGPIO in the Device Tree](#)
 1. [Compatible Strings](#)
 2. [Properties](#)
 3. [The SoC Controller Nodes \(already in the `.dtsi`\)](#)
 4. [Enabling and Naming Lines in Your Board DTS](#)
5. [The Kernel Driver](#)
 1. [Driver and Kconfig](#)
 2. [What the Driver Does](#)
6. [Using SGPIO from Userspace](#)
 1. [Discover Lines](#)
 2. [Read Inputs and Drive Outputs](#)
 3. [Monitoring with phosphor-multi-gpio-monitor](#)
7. [Nuvoton NPCM SGPIO \(SIOX Hardware\)](#)
 1. [Hardware: SIOX, Not True SGPIO](#)
 2. [Where the Driver and Binding Live](#)
 3. [Compatible Strings](#)
 4. [How Nuvoton Differs from ASPEED](#)
 5. [Device Tree Node](#)
 6. [Pinmux](#)
 7. [Enabling and Using It](#)
8. [Porting Guide](#)
9. [Code Examples](#)
 1. [Example 1: Minimal AST2500 Enablement \(Intel Vegman style\)](#)
 2. [Example 2: Minimal AST2600 Enablement](#)
 3. [Example 3: Reading an SGPIO Input with libgpiod \(C\)](#)
 4. [Example 4: phosphor-multi-gpio-monitor for Host Status](#)
10. [Troubleshooting](#)
 1. [Issue: Controller Fails to Probe](#)
 2. [Issue: Lines Appear but Inputs Are Stuck](#)

3. [Issue: `gpioset` on an SGPIO Line Has No Effect](#)
4. [Issue: Edge Events Missed or Delayed](#)
5. [Debug Commands](#)

11. References

1. [Official Resources](#)
 2. [Real-Board Examples](#)
 3. [Related Guides](#)
 4. [External Documentation](#)
-

Overview

SGPIO (Serial GPIO) is a hardware feature on ASPEED BMC SoCs that shifts many GPIO signals in and out over a small, fixed set of pins instead of dedicating one SoC pin per signal. A server platform often needs to read dozens of host-side status signals — CPU presence, THERMTRIP, VRHOT, FIVR fault, DIMM status — and drive backplane LEDs. Wiring each of those to its own BMC pin would exhaust the pinmux. SGPIO solves this by serializing the signals onto a clocked bus.

The ASPEED SGPIO controller is a **master**: it generates a shift clock and a load/latch strobe, shifts BMC output bits out, and shifts external input bits in. The other end is typically the host PCH (acting as an SGPIO source) or a chain of shift registers on a backplane. From Linux's point of view, the controller registers as an ordinary **gpiochip**, so every userspace tool you already know — `gpioinfo`, `gpioget`, `gpioset`, `giomon`, `libgpiod`, and `phosphor-multi-gpio-monitor` — works on SGPIO lines with **no API changes**.

This guide shows you how to declare the SGPIO controller in the device tree, enable and understand the `gpio-aspeed-sgpio` kernel driver, and use the resulting lines from userspace — contrasting each step with ordinary parallel GPIO so you know when to reach for SGPIO and what changes when you do. ASPEED (AST2500/AST2600) is the primary worked example; a dedicated [Nuvoton NPCM SGPIO](#) section covers the NPCM7xx/NPCM8xx controller — whose underlying silicon is actually the **SIOX** (Serial I/O eXpansion) block with a *different, half-duplex* signaling scheme, even though its Linux driver is named `sgpio`.

Key concepts covered:

- What serial GPIO is and the 4-wire master protocol (clock, load, data-out, data-in)
- How SGPIO differs from parallel GPIO — pins, latency, direction model, use cases
- Declaring the SGPIO controller in the device tree (`compatible`, `ngpios`, `bus-frequency`, `pinctrl`)
- The `ngpios` doubling rule and the even-input / odd-output line layout
- Enabling and understanding the `gpio-aspeed-sgpio` kernel driver
- Using SGPIO lines from userspace and `phosphor-multi-gpio-monitor` — identical to GPIO
- Porting to Nuvoton NPCM, whose `nuvoton,input-ngpios` / `nuvoton,output-ngpios` model differs from ASPEED

SGPIO is **hardware-dependent**. The controller and its gpiochip register under Linux regardless, so you can discover lines with `gpioinfo`, but meaningful *input values* require the physical serial link to the host or shift registers. The `ast2600-evb` QEMU target does not emulate the SGPIO serial bus, so this guide is verified only as far as line discovery in QEMU; live signal values require real hardware.

SGPIO vs Parallel GPIO

If you have read the [GPIO Management Guide](#), SGPIO will feel familiar once it is up — the difference is almost entirely in *how the lines reach the SoC*, not in how you use them. Use this table to decide which one a given signal belongs on.

Aspect	Parallel GPIO	SGPIO (Serial GPIO)
Wiring	One dedicated SoC pin per signal	3–4 shared pins (clock, load, data-out, data-in) carry <i>all</i> signals
Signal count	Limited by free pins on the package	Up to 80 lines (AST2400/AST2500) or 128 lines (AST2600 master 1) per controller
Transport	Direct electrical level on the pin	State serially shifted on a clocked bus at bus-frequency
Direction	Each pin independently input or output	Each hardware line is fixed : one input and one output (see the doubling rule)
Latency	Immediate — the register reflects the pin now	Bounded by the shift period; state refreshes once per frame
Edge events	Hardware-accurate, low-latency IRQ	Edge/level IRQ is supported, but timestamp granularity is bounded by the shift clock
Typical use	Buttons, resets, presence local to the BMC board	Host/PCH/CPU status <i>into</i> the BMC; backplane LED/presence fan-out
Device tree	Controller already in the SoC .dts i ; you just name lines	Dedicated sgpio / sgpiom node you must enable, size (n gpios), and clock (bus-frequency)
Userspace API	libgpiod / gpiochip	Identical — same gpiochip, same libgpiod, same tools

Rule of thumb: reach for SGPIO when you have many low-rate *status* signals (presence, fault, LEDs) that originate off the BMC board — especially host/PCH status. Keep latency-critical or BMC-local signals (power button, reset, watchdog strobes) on parallel GPIO.

The “bounded latency / no precise edge timing” characteristic follows from the serial, divided-clock design: a signal is only sampled when its bit is shifted through. The driver *does* deliver edge interrupts, so event-driven monitoring works — but do not rely on SGPIO for sub-microsecond edge timestamps the way you might with a parallel GPIO IRQ.

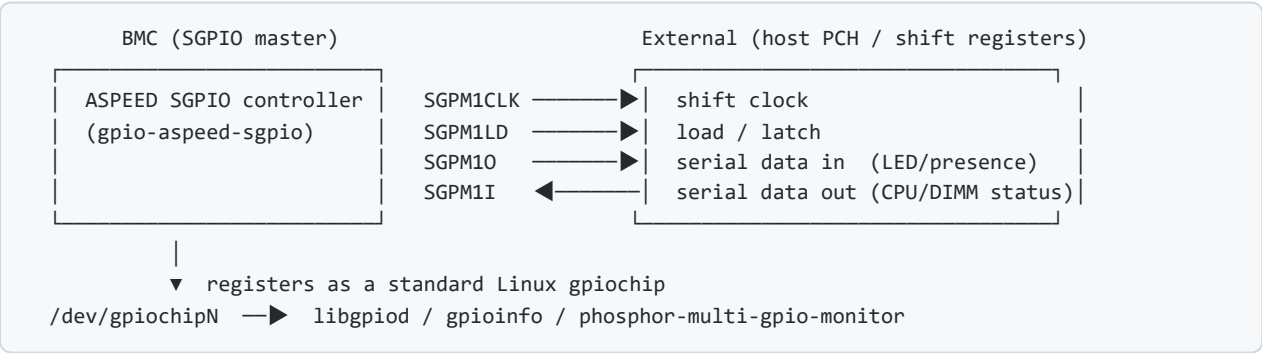
Architecture

The Serial Master Protocol

The ASPEED SGPIO master drives a small fixed pin group and clocks signal state through it. On the AST2600, master 1 ([SGPM1](#)) uses four balls:

Signal	AST2600 ball	Pad (GPIO)	Role
<code>SGPM1CLK</code>	A18	GPIOH0	Shift clock — generated by the BMC master
<code>SGPM1LD</code>	B18	GPIOH1	Load / latch strobe — frames each shift cycle
<code>SGPM1O</code>	C18	GPIOH2	Serial data out (BMC → external, e.g. LEDs)
<code>SGPM1I</code>	A17	GPIOH3	Serial data in (external → BMC, e.g. CPU status)

The shift clock is derived from the APB bus clock divided by a programmable value set from the `bus-frequency` property. Each load/shift cycle moves the full bank of input bits into the controller’s input register and the full bank of output bits out to the external shift register or device.



The ASPEED SoC also exposes **slave** pin groups (`SGPS1` / `SGPS2`) for the case where the BMC is an SGPIO *slave* to an external master. The mainline `gpio-aspeed-sgpio` driver implements the **master** side only. This guide covers the master path, which is the common OpenBMC use case.

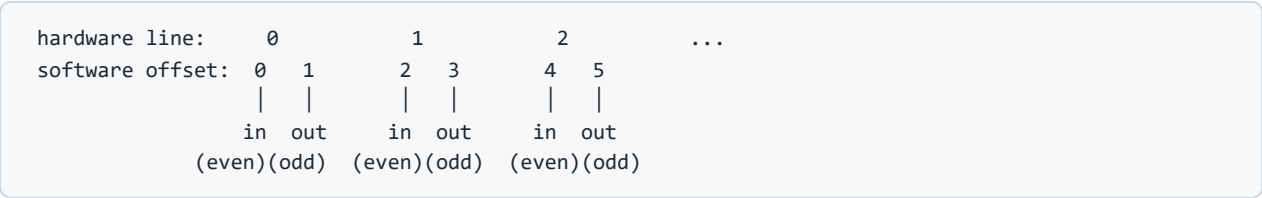
The `ngpios` Doubling Rule

This is the single most important thing to understand about ASPEED SGPIO, and the biggest difference from parallel GPIO.

The `ngpios` property declares the number of **hardware** SGPIO lines. The driver then exposes **two software GPIO lines per hardware line** — one input and one output — so the registered gpiochip has `2 × ngpios` lines. The two lines are **interleaved**, not split into halves:

- **Even** offset (0, 2, 4, ...) = the **input** half of a hardware line (read host/PCH/CPU status here)
- **Odd** offset (1, 3, 5, ...) = the **output** half of a hardware line (drive an LED/strobe here)

So `ngpios = <80>` produces **160** userspace GPIO lines, and `ngpios = <128>` produces **256**.



Because direction is fixed by this even/odd model, you cannot freely flip an SGPIO line between input and output the way you can a parallel GPIO. Each hardware line is *permanently* one input GPIO plus one output GPIO. Real platforms that only consume host status populate the **even** (input) `gpio-line-names` and leave the **odd** (output) names as empty strings `""`.

Defining SGPIO in the Device Tree

There are two parts: the **SoC controller node** (already present, but disabled, in the ASPEED `.dtsi`) and your **board-level enablement** that sizes it, clocks it, and names its lines.

Compatible Strings

SoC	Compatible string	Notes
AST2400	<code>aspeed,ast2400-sgpio</code>	Single controller, up to 80 lines
AST2500	<code>aspeed,ast2500-sgpio</code>	Single controller, up to 80 lines (shares driver data with AST2400)
AST2600	<code>aspeed,ast2600-sgpiom</code>	Two controllers (<code>sgpiom0</code> / <code>sgpiom1</code>); note the <code>-sgpiom</code> suffix
AST2700	<code>aspeed,ast2700-sgpiom</code>	Present in current mainline

Watch the suffix: AST2400/AST2500 use `-sgpio` , while AST2600/AST2700 use `-sgpio**m**` (master). A proposed patch once added `aspeed,ast2600-sgpiom1` / `-sgpiom2` compatibles, but **mainline does not use these** — both AST2600 nodes share the single `aspeed,ast2600-sgpiom` string. Do not invent per-instance compatibles.

Properties

Property	Required	Description
<code>compatible</code>	yes	One of the strings above
<code>reg</code>	yes	Controller register window
<code>gpio-controller</code>	yes	Marks this node as a GPIO provider
<code>#gpio-cells</code>	yes	Always <code><2></code> (line offset + flags)
<code>interrupts</code>	yes	Controller interrupt line
<code>interrupt-controller</code>	yes	The chip is also an IRQ provider (edge/level events)
<code>#interrupt-cells</code>	yes	Always <code><2></code>
<code>ngpios</code>	yes	Number of hardware lines — see the doubling rule . Must be a multiple of 8.
<code>clocks</code>	yes	Source clock for the shift-clock divider (APB / APB2)
<code>bus-frequency</code>	yes	SGPM shift-clock frequency in Hz
<code>pinctrl-0</code> / <code>pinctrl-names</code>	in practice	Routes the SGPM pins (not in the binding's <i>required</i> list, but needed to actually drive the bus)

`ngpios` **must be a multiple of 8** — the driver rejects any other value with `-EINVAL` ("Number of GPIOs not multiple of 8"). It is *not* enough for it to be even. Also note the driver does **not** enforce the per-controller hardware maximum (80 or 128); choosing a value that exceeds your SoC's capability is your responsibility in the board DTS.

The SoC Controller Nodes (already in the `.dtsi`)

You normally do **not** write these — they live in the ASPEED SoC include files with `status = "disabled"`. They are shown here so you know what you are enabling.

AST2600 (`aspeed-g6.dtsi`) — two masters:

```
sgpiom0: sgpiom@1e780500 {
    #gpio-cells = <2>;
    gpio-controller;
    compatible = "aspeed,ast2600-sgpiom";
    reg = <0x1e780500 0x100>;
    interrupts = <GIC_SPI 51 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&syscon ASPEED_CLK_APB2>;
    #interrupt-cells = <2>;
    interrupt-controller;
    bus-frequency = <12000000>;
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_sgpm1_default>;
    status = "disabled";          /* you enable this in the board DTS */
};

/* sgpiom1 @ 0x1e780600 is identical except:
 *   interrupts = <GIC_SPI 70 IRQ_TYPE_LEVEL_HIGH>;
 *   pinctrl-0 = <&pinctrl_sgpm2_default>;
 */
```

AST2500 (`aspeed-g5.dtsi`) — single controller:

```
sgpio: sgpio@1e780200 {
    #gpio-cells = <2>;
    compatible = "aspeed,ast2500-sgpio";
    gpio-controller;
    interrupts = <40>;
    reg = <0x1e780200 0x0100>;
    clocks = <&syscon ASPEED_CLK_APB>;
    #interrupt-cells = <2>;
    interrupt-controller;
    bus-frequency = <12000000>;
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_sgpm_default>;
    status = "disabled";
};
```

Enabling and Naming Lines in Your Board DTS

This is the part **you** write. Reference the SoC node by phandle, set `status = "okay"`, size it with `ngpios`, set the `bus-frequency` your hardware can carry, and assign `gpio-line-names`. Remember the even/odd layout: input names on even indices, output names (or `" "`) on odd.

AST2600 example — host/CPU status into the BMC:

```

&sgpiom0 {
    status = "okay";
    ngpios = <128>;          /* 128 hardware lines -> 256 software GPIOs */
    bus-frequency = <1000000>; /* 1 MHz shift clock; lower it for long backplane runs */

    gpio-line-names =
        /* hw line 0 */ "CPU1_PRESENCE",    "", /* even=in, odd=out(unused) */
        /* hw line 1 */ "CPU1_THERMTRIP",    "",
        /* hw line 2 */ "CPU1_VRHOT",        "",
        /* hw line 3 */ "CPU1_FIVR_FAULT",    "",
        /* hw line 4 */ "CPU1_MEM_ABCD_VRHOT", "",
        /* hw line 5 */ "CPU1_MEM_EFGH_VRHOT", "",
        /* hw line 6 */ "",                  "FAULT_LED_DRIVE"; /* output example */
        /* ... continue for all 128 hardware lines (256 names) ... */
};

```

The `gpio-line-names` array must list **two** names per hardware line and cover the full `2 × ngpios` count (160 for `ngpios=80`, 256 for `ngpios=128`). A short array silently leaves the trailing lines unnamed. Keep the input name on the even index and either an output name or `""` on the following odd index.

Lowering `bus-frequency` for real boards: the `.dtsi` default of 12 MHz is rarely used on production hardware. Long backplane cable runs and signal integrity push the clock down — mainline boards run anywhere from 2 MHz (Intel Vegman) to 500 kHz (IBM System1). The constraint is purely that the computed divider fits a 16-bit field; the exact floor depends on your APB clock.

The Kernel Driver

Driver and Kconfig

Item	Value
Source file	<code>drivers/gpio/gpio-aspeed-sgpio.c</code>
Kconfig symbol	<code>CONFIG_GPIO_ASPEED_SGPIO</code>
Binds via	<code>compatible</code> strings in its <code>of_match_table</code> (see above)

Enable it in your kernel config fragment (see the [Kernel Driver Development Guide](#) for how fragments are layered in OpenBMC):

```

# meta-<vendor>/recipes-kernel/linux/linux-aspeed/sgpio.cfg
CONFIG_GPIO_ASPEED_SGPIO=y

```

```

# linux-aspeed_%.bbappend
FILESEXTRAPATHS:prepend := "${THISDIR}/${PN}:"
SRC_URI += "file://sgpio.cfg"

```

Verify it is built and bound after boot:


```
# Driver present?
zcat /proc/config.gz | grep GPIO_ASPEED_SGPIO      # CONFIG_GPIO_ASPEED_SGPIO=y

# Controller bound? (look for the sgpiom register address)
ls /sys/bus/platform/drivers/aspeed-sgpio/
dmesg | grep -i sgpio
```

What the Driver Does

Understanding a few driver behaviors explains the userspace experience:

- **Registers a standard gpiochip.** It reads `ngpios`, sets `chip.ngpio = ngpios * 2`, and registers an ordinary `gpio_chip` plus an irqchip. That is *why* every libgpiod tool works unchanged.
- **Even = input, odd = output.** The direction of each software line is decided by `offset % 2` (`aspeed_sgpio_input()` returns true for even offsets). You cannot change it.
- **On-demand register access.** `get` / `set` perform direct `ioread32` / `iowrite32` against the controller's data registers — there is **no polling kernel timer**. An input read returns the value last shifted into the input register.
- **Edge and level interrupts.** The driver provides an irqchip supporting `IRQ_TYPE_EDGE_RISING`, `IRQ_TYPE_EDGE_FALLING`, `IRQ_TYPE_EDGE_BOTH`, and the level types, dispatched through `handle_edge_irq` / `handle_level_irq`. This is what makes `gpiomon` and `phosphor-multi-gpio-monitor` event-driven monitoring work on SGPIO lines. The irqchip is flagged `IRQCHIP_IMMUTABLE`.
- **Clock divider.** From `bus-frequency` it computes `div = apb_freq / (sgpio_freq * 2) - 1` (so the shift frequency is `PCLK / (2 * (div + 1))`) and rejects a divider that overflows the 16-bit field with `-EINVAL` — the practical reason a too-low `bus-frequency` fails to probe.

There is no SGPIO-specific character device or sysfs interface. The driver's whole job is to make a serial bus *look like a parallel gpiochip*. Once probed, forget it is serial — until you are debugging timing.

Using SGPIO from Userspace

Here is the payoff: **using SGPIO is identical to using parallel GPIO**. The only thing you must keep in mind is the even-input / odd-output line layout.

Discover Lines

```
# List every gpiochip; the SGPIO controller appears alongside the parallel ones
gpiodetect
# gpiochip0 [1e780000.gpio]    (232 lines)  <- parallel GPIO
# gpiochip1 [1e780500.sgpio]  (256 lines)  <- SGPIO master 0 (ngpios=128 -> 256)

# Show SGPIO lines with their names and current state
gpioinfo gpiochip1 | grep -v unnamed
# line 0: "CPU1_PRESENCE"  input  active-high
# line 2: "CPU1_THERMTRIP" input  active-high
# line 4: "CPU1_VRHOT"     input  active-high
```

Prefer **named-line** access over raw chip+offset. Because OpenBMC tools resolve a line by its `gpio-line-names` string across all chips, you never have to hard-code which `gpiochipN` the SGPIO controller landed on — and that number can shift when other GPIO providers probe.

Read Inputs and Drive Outputs

```
# Read a host-status input by NAME (resolves to the even line, whichever chip it is on)
gpioget $(gpiofind "CPU1_THERMTRIP")
# 0    -> not asserted

# ...or by explicit chip + EVEN offset (input half of hardware line 1)
gpioget gpiochip1 2

# Drive an output by name (an ODD line – the output half of a hardware line)
gpioset $(gpiofind "FAULT_LED_DRIVE")=1

# Watch an SGPIO input for edges, exactly like a parallel GPIO
piomon --num-events=1 --falling-edge $(gpiofind "CPU1_THERMTRIP")
```

Trying to `gpioset` an **even** (input) offset, or `gpioget` -and-expect-to-drive an **odd** (output) offset, will not do what you want. Even = input, odd = output is fixed by the hardware. If `gpioset` on a name fails, you almost certainly named the input half; the drivable line is the next (odd) offset.

Monitoring with phosphor-multi-gpio-monitor

Because SGPIO lines are ordinary named gpiochip lines, `phosphor-multi-gpio-monitor` configures them with the **same JSON** described in the [GPIO Management Guide](#) — just point `LineName` at an SGPIO input name. Here a host THERMTRIP arriving over SGPIO triggers a logging target:

```
{
  "GpioConfigs": [
    {
      "Name": "Cpu1Thermtrip",
      "LineName": "CPU1_THERMTRIP",
      "GpioNum": 0,
      "ChipId": "",
      "EventMon": "FALLING",
      "Targets": {
        "0": ["log-cpu-thermtrip@0.target"]
      },
      "Continue": true
    }
  ]
}
```

The monitor neither knows nor cares that `CPU1_THERMTRIP` is an SGPIO line rather than a parallel one — it requests the line by name through the same libgpiod chardev path. This is the central practical takeaway of this guide.

Nuvoton NPCM SGPIO (SIOX Hardware)

Nuvoton NPCM7xx and NPCM8xx BMCs expose a serial GPIO expander too, and Linux drives it through a gpiochip — so everything *above* the device tree (the gpiochip abstraction, libgpiod, `phosphor-multi-gpio-monitor`) works the same way. But there are **two** important differences from ASPEED: the device-tree model is different enough that you cannot copy the node, **and the underlying hardware is not the same kind of serial GPIO at all.**

Hardware: SIOX, Not True SGPIO

The NPCM peripheral is the **SIOX (Serial I/O eXpansion)** block, not an SFF-8485-style SGPIO master like ASPEED's. The Linux driver and binding are *named* `sgpio` (`gpio-npcm-sgpio.c` , `nuvoton,...-sgpio`), but the silicon is SIOX. The giveaway is everywhere in the pinmux: the pin groups are `iox1` / `iox2` / `ioxh` — **IOX = I/O eXpansion** — and the interface is designed to clock external `74HC595` (output, serial-in/parallel-out) and `74HC165` (input, parallel-in/serial-out) shift-register chains.

The signaling difference matters for performance:

	True SGPIO (ASPEED)	SIOX (Nuvoton NPCM)
Duplex	Full-duplex	Half-duplex
How bits move	Input and output bits are clocked on opposite edges of the same shift clock — one clock period moves an input bit <i>and</i> an output bit concurrently	Shifts a full 8 bits in one direction, then 8 bits in the other direction in separate phases
Throughput at a given clock	Both directions per clock	~half — refreshing both the input byte and the output byte takes roughly twice the clocks

Do not assume Nuvoton SGPIO has the same refresh latency as ASPEED SGPIO at the same clock rate. Because SIOX is half-duplex (one direction at a time), the effective update rate for a given shift clock is **about half** that of a full-duplex SGPIO master. Size your timing budget for status signals accordingly, and keep genuinely fast signals on parallel GPIO. The same “*edge events may be missed/delayed*” caveat applies, more so.

Why call it “SGPIO” at all, then? Because from Linux's point of view both are just serial gpiochips, and the upstream driver/binding adopted the `sgpio` name. This guide groups them together for that reason — but when you reason about *signal timing or schematic design*, treat the Nuvoton part as SIOX with HC595/HC165 expanders, not as an SFF-8485 SGPIO.

Where the Driver and Binding Live

Item	Value
Driver source	<code>drivers/gpio/gpio-npcm-sgpio.c</code>
Kconfig symbol	<code>CONFIG_GPIO_NPCM_SGPIO</code>
Mainline Linux	Present since v6.8
OpenBMC kernel	In-tree on <code>openbmc/linux</code> since <code>dev-6.6</code> (current <code>dev-6.18</code> has it)
meta-nuvoton	Enabled via defconfig — <code>CONFIG_GPIO_NPCM_SGPIO=y</code> in <code>npcm8xx_defconfig</code> (not the generic <code>defconfig</code>); no out-of-tree kernel patch
Device tree binding	<code>Documentation/devicetree/bindings/gpio/nuvoton,sgpio.yaml</code>

You were right to expect a patch: in an **early (v4) submission**, the Nuvoton SGPIO DTS node carried a `bus-frequency` property and a `pinctrl-0 = <&iox1_pins>`. **Both were dropped before merge**. The shipping upstream driver and binding have neither — so do not copy a `bus-frequency` / `pinctrl-0` from an old patch onto a modern NPCM SGPIO node; it will not match the binding. The driver and binding are now fully upstream.

No in-tree device-tree node exists for NPCM SGPIO — not in mainline, `openbmc/linux`, or U-Boot. Unlike ASPEED (where the `sgpio` / `sgpiom` node ships disabled in the SoC `.dtsi` and you just enable it), on Nuvoton you must **author the entire `sgpio` node yourself** from the binding example. Enable the controller's clock and add it under the SoC's APB3 bus.

Compatible Strings

SoC	Compatible string
NPCM7xx	<code>nuvoton,npcm750-sgpio</code>
NPCM8xx	<code>nuvoton,npcm845-sgpio</code>

How Nuvoton Differs from ASPEED

This is the part that trips people up. The Nuvoton controller does **not** use a single `ngpios` with the even/odd doubling rule. Instead it declares input and output counts separately, and lays the lines out as two contiguous ranges with **outputs first**.

Aspect	ASPEED (<code>gpio-aspeed-sgpio</code>)	Nuvoton (<code>gpio-npcm-sgpio</code>)
Hardware block	SGPIO master (SFF-8485 style), full-duplex	SIOX (Serial I/O eXpansion), half-duplex — see above
Line-count property	Single <code>ngpios</code> (hardware lines)	Two: <code>nuvoton,input-ngpios</code> and <code>nuvoton,output-ngpios</code> (each ≤ 64)
Total gpiochip lines	<code>2 × ngpios</code>	<code>input-ngpios + output-ngpios</code>
Direction layout	Interleaved: even = input, odd = output	Contiguous: offsets <code>0 ... nout-1</code> = outputs , then <code>n out ... nout+nin-1</code> = inputs
Direction	Fixed (each hw line is 1 in + 1 out)	Fixed (each line is input-only or output-only)
Shift clock	<code>bus-frequency</code> property (you set it)	No property — auto-derived from APB3, kept < 8 MHz
pinctrl on the node	<code>pinctrl-0 = <&pinctrl_sgpm1_default></code>	Not on the node — mux via the <code>iox1</code> / <code>iox2</code> pin groups separately
Max lines / controller	80 (AST2400/2500) or 128 (AST2600 master 0) hardware	64 input + 64 output = 128 total
Controllers per SoC	1 (AST2500) / 2 (AST2600)	2 (both NPCM7xx and NPCM8xx)

Outputs occupy the low offsets, inputs the high offsets — the opposite mental model from “inputs first”. With `output-ngpios = <64>` and `input-ngpios = <64>`, lines `0-63` are outputs and lines `64-127` are inputs. Only the input range is interrupt-capable. Get this backwards and `gpioset` / `gpioget` will silently target the wrong half.

Device Tree Node

This is the **merged, authoritative** NPCM7xx example from the upstream binding — note there is no `bus-frequency` and no `pinctrl-0`:

```
/* Add under the APB3 bus in your board DTS. NPCM7xx (nuvoton,npcm750-sgpio). */
sgpio1: gpio@101000 {
    compatible = "nuvoton,npcm750-sgpio";
    reg = <0x101000 0x200>;
    clocks = <&clk NPCM7XX_CLK_APB3>;
    interrupts = <GIC_SPI 19 IRQ_TYPE_LEVEL_HIGH>;
    gpio-controller;
    #gpio-cells = <2>;
    nuvoton,input-ngpios = <64>;    /* lines 64..127 -> inputs */
    nuvoton,output-ngpios = <64>;   /* lines 0..63   -> outputs */
};
```

For **NPCM8xx**, swap the compatible string and the clock macro:

```
sgpio1: gpio@<base> {
    compatible = "nuvoton,npcm845-sgpio";
    reg = <0x<base> 0x200>;          /* confirm the NPCM8xx base from your SoC datasheet/reference DTS */
    clocks = <&clk NPCM8XX_CLK_APB3>;
    interrupts = <GIC_SPI 19 IRQ_TYPE_LEVEL_HIGH>;
    gpio-controller;
    #gpio-cells = <2>;
    nuvoton,input-ngpios = <64>;
    nuvoton,output-ngpios = <64>;
};
```

The NPCM8xx register base is intentionally left as `<base>` above: no public in-tree NPCM8xx SGPIO node exists to copy a verified address from. Take the base (and the second module’s address) from the NPCM8xx datasheet or your platform’s reference device tree. The NPCM7xx module 1 base `0x101000` (size `0x200`) is the only address confirmed from upstream.

Both fragments, with the outputs-first `gpio-line-names` layout filled in, are in `docs/examples/sgpio/sgpio-npcm.dtsi`.

Pinmux

The SIOX interface uses four pins (data-out, data-in, shift-clock, load/latch), muxed through the **IOX** (I/O eXpansion) pin groups — the naming that reveals this is the SIOX block. On both NPCM7xx and NPCM8xx (`pinctrl-npcm7xx` / `pinctrl-npcm8xx`):

Group	Pins	Use
<code>iox1</code>	0, 1, 2, 3	Serial I/O eXpansion 1 (SIOX module 1)
<code>iox2</code>	4, 5, 6, 7	Serial I/O eXpansion 2 (SIOX module 2)

Select the matching group in your pinctrl configuration (e.g. reference `iox1` from the board’s `pinctrl` node) so the SIOX pins leave the package.

Enabling and Using It

```
# meta-<vendor>/recipes-kernel/linux/linux-nuvoton/<machine>.cfg (or rely on npc8xx_defconfig)
CONFIG_GPIO_NPCM_SGPIO=y
```

From userspace it is the same gpiochip story as ASPEED — just remember the outputs-first / inputs-second layout when you read `gpioinfo`:

```
gpiodetect | grep -i sgpio
gpioinfo $(gpiodetect | awk '/sgpio/{print $1}')
# line 0: unnamed output    <- outputs occupy the LOW offsets
# ...
# line 64: "CPU1_THERMTRIP" input  <- inputs occupy the HIGH offsets

# Read an input (high offset), drive an output (low offset)
gpioget $(gpiofind "CPU1_THERMTRIP")
gpiochipN 5=1
```

The clock cannot be set from the device tree on Nuvoton — the driver derives the SGPIO shift clock from APB3 and keeps it under 8 MHz automatically. There is nothing to tune in DT; if you need a slower shift for a long backplane, that is a hardware/board concern, not a `bus-frequency` knob.

Porting Guide

Follow these steps to bring up SGPIO on your platform:

1. **Confirm the use case.** SGPIO is for many off-board status/LED signals — most often host/PCH/CPU status into the BMC. Latency-critical or BMC-local signals stay on parallel GPIO.
2. **Enable the kernel driver.** Add `CONFIG_GPIO_ASPEED_SGPIO=y` via a `.cfg` fragment and `.bbappend` (see [The Kernel Driver](#)).
3. **Enable the controller in your board DTS.** Reference `&sgpiom0` (AST2600) or `&sgpio` (AST2500), set `status = "okay"`, choose `ngpios` (multiple of 8, within the hardware max), and pick a `bus-frequency` your wiring supports.
4. **Route the pins.** Make sure the SGPM pinctrl group (`pinctrl1_sgpm1_default` etc.) is selected so the clock/load/data balls leave the package.
5. **Name the lines.** Fill `gpio-line-names` with input names on even indices and output names (or `" "`) on odd indices, covering all `2 × ngpios` entries.
6. **Build, flash, and verify:**

```
bitbake obmc-phosphor-image

# After boot, confirm the chip and lines:
gpiodetect | grep sgpio
gpioinfo $(gpiodetect | awk '/sgpio/{print $1}') | grep -v unnamed

# Drive the bus and read a known signal (needs the physical serial link):
gpioget $(gpiofind "CPU1_PRESENCE")
```

If `gpioinfo` lists the SGPIO lines but every input reads a stuck value, the gpiochip registered but the **serial bus is not actually shifting** — check the pinmux routing, the `bus-frequency` divider (too low → probe fails; too high → the far end cannot keep up), and that the external device/shift-register chain is powered and clocked.

Code Examples

Example 1: Minimal AST2500 Enablement (Intel Vegman style)

A real-board pattern: an AST2500 reading CPU/PCH status, 80 hardware lines at 2 MHz, only input names populated.

```
&sgpio {
    status = "okay";
    ngpios = <80>;                /* -> 160 software GPIOs */
    bus-frequency = <2000000>;
    gpio-line-names =
        "CPU1_PRESENCE",        "",
        "CPU1_THERMTRIP",       "",
        "CPU1_VRHOT",           "",
        "CPU1_FIVR_FAULT",      "",
        "CPU1_MEM_ABCD_VRHOT",  "",
        "CPU1_MEM_EFGH_VRHOT",  "";
    /* ...pad to 160 names total... */
};
```

Example 2: Minimal AST2600 Enablement

The shortest possible bring-up — enable, size, and clock master 0. Line names can be added later; `gpioinfo` will show 256 lines immediately.

```
&sgpiom0 {
    status = "okay";
    ngpios = <128>;                /* -> 256 software GPIOs */
    bus-frequency = <500000>;      /* 500 kHz, conservative for long runs */
};
```

Example 3: Reading an SGPIO Input with libgpiod (C)

SGPIO needs no special libgpiod calls — this is the ordinary named-line read pattern. See the [Kernel Driver Development Guide](#) for the full libgpiod walkthrough.

```

#include <gpiod.h>
#include <stdio.h>

int main(void)
{
    /* Resolve the line by name across all chips – works regardless of
     * which gpiochip the SGPIO controller happens to be. */
    struct gpiod_line *line = gpiod_line_find("CPU1_THERMTRIP");
    if (!line) {
        fprintf(stderr, "CPU1_THERMTRIP not found (check gpio-line-names)\n");
        return 1;
    }

    if (gpiod_line_request_input(line, "sgpio-reader") < 0) {
        perror("request_input");
        return 1;
    }

    int value = gpiod_line_get_value(line); /* even/input line: read-only */
    printf("CPU1_THERMTRIP = %d\n", value);

    gpiod_line_release(line);
    return 0;
}

```

Example 4: phosphor-multi-gpio-monitor for Host Status

See the inline [monitoring example](#) above and the working config in [docs/examples/sgpio/sgpio-host-status.json](#), alongside the other fragments under [docs/examples/sgpio/](#).

Troubleshooting

Issue: Controller Fails to Probe

Symptom: `dmesg | grep sgpio` shows an error and no `gpiochip` for the SGPIO controller.

Cause: Most often an invalid `ngpios` or an out-of-range `bus-frequency`.

Solution:

1. Confirm `ngpios` is a **multiple of 8**. The driver rejects anything else with "Number of GPIOs not multiple of 8".
2. Confirm `bus-frequency` is not so low that the divider overflows 16 bits (`apb / (2 × freq) - 1 > 0xFFFF` → `-EINVAL`). Raise it.
3. Confirm `clocks` points at the right APB source (`ASPEED_CLK_APB2` on AST2600, `ASPEED_CLK_APB` on AST2500).

Issue: Lines Appear but Inputs Are Stuck

Symptom: `gpioinfo` lists the named SGPIO lines, but every input reads the same value and never changes.

Cause: The gpiochip registered, but the serial bus is not actually shifting valid data — pinmux not routed, far end unpowered, or clock mismatch.

Solution:

1. Verify the SGPM pinctrl group is selected so the clock/load/data balls leave the package:


```
cat /sys/kernel/debug/pinctrl/*/pinmux-pins | grep -i sgpm
```

2. Confirm the external device or shift-register chain is powered and its clock expectations match your `bus-frequency`. Lower the clock for long cable runs.
3. Scope `SGPM1CLK` and `SGPM1LD` — no clock means no shifting.

Issue: `gpioset` on an SGPIO Line Has No Effect

Symptom: `gpioset` returns success but the output never changes; or it errors on a name you expected to drive.

Cause: You targeted an **even** (input) line. Only **odd** offsets are outputs.

Solution: Drive the **odd** line — the output half of the hardware line. If you named the signal on the even index, the drivable line is the next offset (even `N` → output `N+1`). Confirm with:

```
gpioinfo gpiochip1 | sed -n '1,8p' # even=input, odd=output
```

Issue: Edge Events Missed or Delayed

Symptom: `gpiomon` / `phosphor-multi-gpio-monitor` reacts late or misses brief pulses on an SGPIO input.

Cause: SGPIO state refreshes only once per shift frame; sub-shift-period pulses can be missed, and edge timestamps are coarse.

Solution: Raise `bus-frequency` if signal integrity allows, or move genuinely fast/latency-critical signals to a parallel GPIO. SGPIO is best for level-ish status, not high-rate pulse trains.

Debug Commands

```
# Is the driver built and the controller bound?
zcat /proc/config.gz | grep GPIO_ASPEED_SGPIO
ls /sys/bus/platform/drivers/aspeed-sgpio/
dmesg | grep -i sgpio

# Enumerate chips and lines
gpiodetect
gpioinfo $(gpiodetect | awk '/sgpio/{print $1}')

# Pinmux routing for the SGPM group
cat /sys/kernel/debug/pinctrl/*/pinmux-pins | grep -i sgpm

# Read / watch a named SGPIO line
gpioget $(gpiofind "CPU1_PRESENCE")
gpiomon $(gpiofind "CPU1_THERMTRIP")
```

References

Official Resources

- Linux `gpio-aspeed-sgpio.c` driver
- Device tree binding: `aspeed,sgpio.yaml`
- Legacy binding text: `sgpio-aspeed.txt`

- ASPEED AST2600 dtsi (`sgpiom0` / `sgpiom1`)
- ASPEED AST2500 dtsi (`sgpio`)
- Nuvoton `gpio-npcm-sgpio.c` driver (mainline $\geq$ v6.8)
- Nuvoton device tree binding: `nuvoton,sgpio.yaml`
- OpenBMC `gpio-npcm-sgpio.c` (in-tree, `dev-6.6`)
- meta-nuvoton `npcm8xx_defconfig` (enables `CONFIG_GPIO_NPCM_SGPIO`)

Real-Board Examples

- Intel Vegman N110 (AST2500, host status SGPIO)
- IBM System1 (AST2600, 128-line master)

Related Guides

- [GPIO Management Guide](#) — phosphor-multi-gpio-monitor, line-name discovery, the JSON config SGPIO reuses
- [Linux Kernel Driver Development](#) — kernel config fragments, device tree patching, libgpiod
- [Buttons Guide](#) — parallel-GPIO buttons and presence

External Documentation

- [libgpiod Documentation](#)
- [Linux GPIO Subsystem](#)

Tested on: Line discovery verified against the ASPEED and Nuvoton SGPIO drivers and bindings; SGPIO serial signaling requires physical hardware (host/PCH or shift registers) and is **not** emulated by the `ast2600-evb` QEMU target. Facts are drawn from upstream Linux — ASPEED: `gpio-aspeed-sgpio.c` , `aspeed,sgpio.yaml` , AST2500/AST2600 dtsi, Vegman/System1 board DTS; Nuvoton: `gpio-npcm-sgpio.c` (mainline $\geq$ v6.8, `openbmc/linux` $\geq$ `dev-6.6`), `nuvoton,sgpio.yaml` , and `npcm8xx_defconfig` . Last updated: 2026-06-27 </content>

</invoke>

Device Firmware Update

Use the modern, device-agnostic firmware-update framework in phosphor-bmc-code-mgmt to update CPLDs, voltage regulators, EEPROM-backed devices, and to inventory TPMs - all driven by Entity Manager configuration.

Table of Contents

1. [Overview](#)
 2. [Architecture](#)
 1. [Component Diagram](#)
 2. [Discovery and configuration flow](#)
 3. [Coroutine-based update state machine](#)
 4. [ScopedBmcMux \(RAII bus mux\)](#)
 5. [PLDM component matching](#)
 6. [D-Bus Interfaces](#)
 3. [CPLD Firmware Update](#)
 1. [Interface and factory](#)
 2. [Supported vendors](#)
 3. [Configuration](#)
 4. [I2C Voltage Regulator Firmware Update](#)
 1. [VoltageRegulator base and factory](#)
 2. [Supported families](#)
 3. [Configuration](#)
 5. [EEPROM Device Update](#)
 1. [Update mechanism: at24 bind → dd → unbind](#)
 2. [DeviceVersion providers](#)
 3. [Configuration](#)
 6. [TPM Device Management](#)
 1. [What it actually does](#)
 2. [Configuration](#)
 7. [Troubleshooting](#)
 1. [Issue: Device never appears in the firmware inventory](#)
 2. [Issue: Update starts but fails immediately](#)
 3. [Issue: TPM update request is rejected](#)
 4. [Debug Commands](#)
 8. [References](#)
 1. [Official Resources](#)
 2. [Related Guides](#)
 3. [External Documentation](#)
-

Overview

The [Firmware Update Guide](#) covers the classic BMC/host image path (`ItemUpdater` , `ImageManager` , `Activation`) built around A/B flash images. Alongside that, `phosphor-bmc-code-mgmt` has grown a second, newer subsystem: a **device-agnostic update framework** whose job is to update the many small programmable devices that hang off the BMC - complex programmable logic devices (CPLDs), I2C voltage regulators, EEPROM-backed retimers, and similar parts.

Instead of one daemon per board, this framework provides a shared C++ **common library** of base classes, and a family of small per-device-class daemons (one for CPLDs, one for I2C voltage regulators, one for EEPROM devices, one for TPMs). Each daemon discovers its devices from **Entity Manager** configuration at runtime, exposes them on D-Bus using the standard `xyz.openbmc_project.Software.*` interfaces, and drives the flash using `sdbusplus::async` coroutines. Because the D-Bus surface is the same as the BMC updater, the same Redfish `UpdateService` and `busctl` workflows apply to every device.

You would use this framework when you are bringing up a new platform that carries programmable satellite devices and you want each one to appear in the Redfish firmware inventory and be updatable through the same pipeline as the BMC - without writing a bespoke updater for every chip.

Key concepts covered:

- The `common/` library base classes: `Device` , `Software` , `SoftwareManager`
- `sdbusplus::async` coroutines and the `ScopedBmcMux` RAIL bus-mux helper
- PLDM component matching by compatible string and vendor IANA
- Per-device daemons: `cpld` , `i2c-vr` , `eeeprom-device` , and the read-only `tpm`

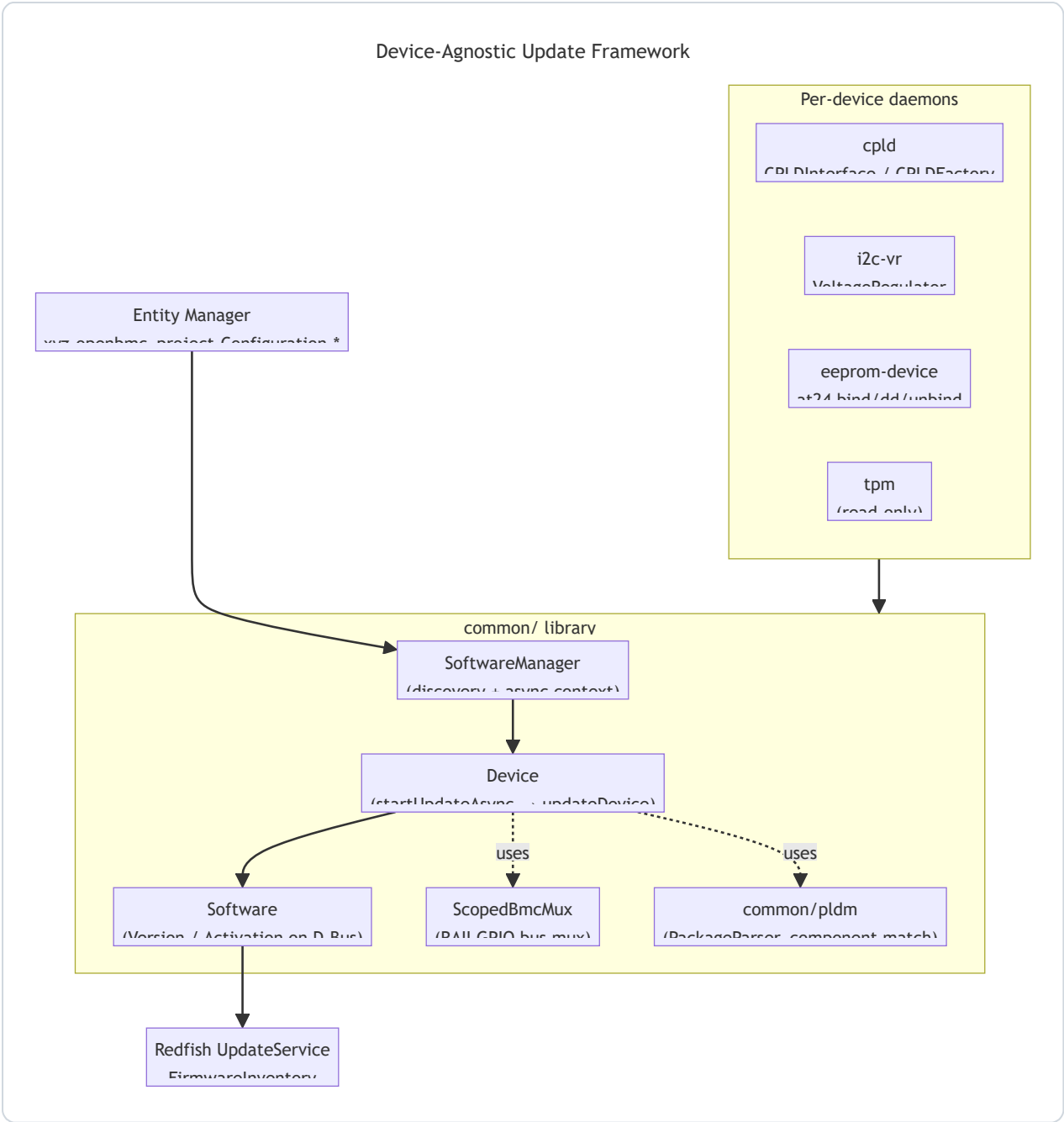
This framework is under active upstream development. Class names, the set of supported chip families, and the exact D-Bus surface evolve quickly - always confirm against the current `openbmc/phosphor-bmc-code-mgmt` tree for your build.

Architecture

Every device daemon is a thin wrapper around three shared base classes that live in `common/` (`common/include/` for headers, `common/src/` for implementations):

Base class	File	Responsibility
<code>SoftwareManager</code>	<code>common/src/software_manager.cpp</code>	Discovers devices from Entity Manager, owns the <code>sdbusplus::async::context</code> , and instantiates one <code>Device</code> per matching config
<code>Device</code>	<code>common/src/device.cpp</code>	Represents one updatable device; drives the update state machine (<code>startUpdateAsync</code>) and defers the actual flash to a device-specific <code>updateDevice()</code> override
<code>Software</code>	<code>common/src/software.cpp</code>	Represents one firmware image/version on D-Bus (<code>xyz.openbmc_project.Software.*</code>)

Component Diagram



► ASCII-art version (for comparison)

Discovery and configuration flow

`SoftwareManager` takes an `sdbusplus::async::context& ctx` in its constructor and watches Entity Manager for `InterfacesAdded` / `InterfacesRemoved` on the inventory tree (service `xyz.openbmc_project.EntityManager`, path root `/xyz/openbmc_project/inventory`). When a matching configuration appears, it reads the device's `<config>.FirmwareInfo` sub-interface:

Property	Type	Purpose
<code>VendorIANA</code>	<code>uint64</code>	Vendor IANA Enterprise Number used for PLDM component matching
<code>CompatibleHardware</code>	<code>string</code>	Compatible string (dotted, e.g. <code>com.meta.Hardware...</code>) used for PLDM component matching
<code>Type</code>	<code>string</code>	Selects the device-specific implementation (chip family)
<code>Name</code>	<code>string</code>	Human-readable name

These are packaged into a `SoftwareConfig` and handed to a new `Device`.

Coroutine-based update state machine

The framework is asynchronous end to end. `Device::startUpdateAsync` is a coroutine:

```
// common/src/device.cpp (signature, upstream master)
sdbusplus::async::task<bool> Device::startUpdateAsync(
    sdbusplus::message::unix_fd image,
    RequestedApplyTimes applyTime,
    std::unique_ptr<Software> softwarePendingIn);
```

Inside, it `co_await`s each step - parsing the incoming PLDM package, selecting the matching component image, then calling the device-specific `updateDevice(image, size)` override, and finally publishing the new `Software` version. Daemons spawn work with `ctx.spawn(...)` and never block the event loop.

ScopedBmcMux (RAII bus mux)

Many satellite devices sit behind a mux or a bus that is normally owned by the host. Flashing them safely means routing the bus to the BMC for the duration of the write and handing it back afterward - even if the update fails partway. The framework provides a RAII helper in `common/include/gpio_controller.hpp`:

```
// A RAII wrapper: mux GPIO lines to BMC on construction,
// back to device on destruction.
class ScopedBmcMux
{
public:
    explicit ScopedBmcMux(GPIOGroup& group);    // muxToBMC()
    ~ScopedBmcMux();                          // muxToDevice()
};
```

`GPIOGroup` provides the underlying `muxToBMC()`, `muxToDevice()`, and `releaseAll()` operations. A `Device` implementation constructs a `ScopedBmcMux` on the stack at the start of a flash; when the object leaves scope (success or exception), the destructor restores the bus to the device automatically.

This RAII pattern is why the update path is exception-safe: you never have to remember to un-mux the bus in an error branch. Prefer stack-scoped `ScopedBmcMux` over calling `muxToBMC()` / `muxToDevice()` by hand.

PLDM component matching

Incoming images are PLDM firmware update packages. `common/pldm` contains a `PackageParser` (supporting PLDM firmware package revision up to 1.0.0) that walks the package's component image set. A component is applied to a device only when **both** identifiers agree:

- the device's `CompatibleHardware` string matches the component's descriptor, and
- the device's `VendorIANA` matches the component's vendor.

This is what lets a single uploaded package carry firmware for several different satellite devices and land each component on the correct part.

D-Bus Interfaces

Interface	Object Path	Description
<code>xyz.openbmc_project.Software.Version</code>	<code>/xyz/openbmc_project/software/<id></code>	Reports the device firmware version
<code>xyz.openbmc_project.Software.Activation</code>	<code>/xyz/openbmc_project/software/<id></code>	Activation state during/after update
<code>xyz.openbmc_project.Software.Update</code>	<code>/xyz/openbmc_project/software/<id></code>	<code>StartUpdate</code> entry point (takes an image FD + apply time)

CPLD Firmware Update

The `cpld/` daemon (`xyz.openbmc_project.Software.CPLD.service`) updates complex programmable logic devices over I2C. Its core files are `cpld.cpp`, `cpld_interface.cpp`, and `cpld_software_manager.cpp`.

Interface and factory

Vendor support is organized around two types in `cpld_interface.hpp`:

- `CPLDInterface` - the abstract base each vendor implements (erase/program/verify for that silicon).
- `CPLDFactory` - a registry that maps a chip-type string to a constructor. Vendor back-ends register themselves with `registerCPLD(chipType, creator)`, and the daemon calls `create(chipType, ...)` to obtain the right implementation. Internally it is a `std::unordered_map<std::string, Creator>`.

Supported vendors

Vendor	Family	Implementation files
Altera (Intel)	MAX10	<code>cpld/altera/max10_base_cpld.*</code> , <code>max10_standard_cpld.*</code> , <code>max10_cpld_factory.*</code>
Lattice	XO3 (e.g. LCMXO3LF-4300C)	<code>cpld/lattice/lattice_xo3_cpld.*</code> , <code>lattice_base_cpld.*</code> , <code>lattice_cpld_factory.*</code>
Lattice	XO5 (base / standard / T-series)	<code>cpld/lattice/lattice_xo5_base_cpld.*</code> , <code>lattice_xo5_standard_cpld.*</code> , <code>lattice_xo5_tseries_cpld.*</code>

The upstream tree ships Altera MAX10 and Lattice XO3/XO5 back-ends today. Other families referenced informally (for example Lattice XO2) are not present as dedicated implementations at the time of writing - verify against the `cpld/altera/` and `cpld/lattice/` directories for your build.

Configuration

Entity Manager describes the CPLD with a `Type` that selects the factory back-end and a `FirmwareInfo` block for PLDM matching:

```
{
  "Name": "Harma_MB_CPLD",
  "Type": "LatticeLCMX03LF_4300CFirmware",
  "Bus": 5,
  "Address": "0x40",
  "FirmwareInfo": {
    "VendorIANA": 40981,
    "CompatibleHardware": "com.meta.Hardware.Harma.CPLD.LCMX03LF_4300C_mb"
  }
}
```

This surfaces on D-Bus as `xyz.openbmc_project.Configuration.LatticeLCMX03LF_4300CFirmware` (the interface name is the configuration `Type`).

I2C Voltage Regulator Firmware Update

The `i2c-vr/` daemon (`xyz.openbmc_project.Software.I2CVR.service`) programs voltage regulators / PMBus controllers. Core files: `i2cvr_device.cpp` , `i2cvr_software_manager.cpp` , and the `vr.cpp` / `vr.hpp` base.

VoltageRegulator base and factory

`vr.hpp` defines the abstract `VoltageRegulator` base and a `VRType` enum enumerating the supported controllers. Instances are created through a factory:

```
// i2c-vr/vr.hpp (upstream master)
std::unique_ptr<VoltageRegulator> create(
    sdbusplus::async::context& ctx, enum VRType vrType,
    uint16_t bus, uint16_t address);

bool stringToEnum(std::string& vrStr, VRType& vrType);
```

Supported families

The `VRType` enum and the per-vendor subdirectories (`xdpe1x2xx/` , `xdp71x/` , `tda38640a/` , `isl69269/` , `mps/` , `tps25990/`) cover:

Vendor	Controllers (VRType)
Infineon	<code>XDPE1X2XX</code> , <code>XDP71X</code> , <code>TDA38640A</code>
Renesas / Intersil	<code>ISL69269</code> , <code>RAA22XGen2</code> , <code>RAA22XGen3p5</code>
MPS (Monolithic Power)	<code>MP2X6XX</code> , <code>MP292X</code> , <code>MP297X</code> , <code>MP5998</code> , <code>MP994X</code> , <code>MPQ87XX</code>
Texas Instruments	<code>TPS25990</code>
Richtek	<code>RS31390</code>

Voltage-regulator configuration memory has a limited number of write cycles. Most of these controllers store their configuration in OTP (one-time programmable) or MTP (multi-time programmable) memory with a small, finite endurance - often only a handful of reprogramming cycles. A failed or repeated update can permanently exhaust the part, and a corrupt VR image can leave a rail unable to power on. Update VRs only with a vendor-validated image, confirm the power state is safe first, and never loop the update in test scripts.

Configuration

```
{
  "Name": "vr_p0_vddcr",
  "Type": "XDPE1X2XXFirmware",
  "Bus": 3,
  "Address": "0x76",
  "FirmwareInfo": {
    "VendorIANA": 40981,
    "CompatibleHardware": "com.example.Hardware.Vr.XDPE192C4"
  }
}
```

EEPROM Device Update

The `eeeprom-device/` daemon (`xyz.openbmc_project.Software.EEPROMDevice.service`) updates devices whose firmware lives in an attached I2C EEPROM - PCIe retimers being the canonical case. Core files: `eeeprom_device.cpp`, `eeeprom_device_software_manager.cpp`, `eeeprom_device_version.cpp`.

Update mechanism: at24 bind → dd → unbind

Rather than bit-banging PMBus, this daemon reuses the kernel's `at24` EEPROM driver and the `dd` tool:

1. **Bind** the `at24` driver to the device so a sysfs `eeeprom` file appears:

```
// eeeprom_device.cpp (upstream master)
auto bindPath = driverPath + "/bind";           // /sys/bus/i2c/drivers/at24/bind
std::ofstream ofbind(bindPath, std::ofstream::out);
ofbind << i2cDeviceId;                         // e.g. "3-0050"
```

2. **Write** the image to the resulting EEPROM node:

```
std::string cmd = "dd if=" + path +
    " of=" + eepromPath + " bs=1k";    // of=/sys/bus/i2c/devices/{bus}-{addr}/eeprom
auto success = co_await asyncSystem(ctx, cmd);
```

3. **Unbind** the driver so the device returns to normal operation:

```
auto unbindPath = driverPath + "/unbind";    // /sys/bus/i2c/drivers/at24/unbind
std::ofstream ofunbind(unbindPath, std::ofstream::out);
ofunbind << i2cDeviceId;
```

The image is staged to a temporary file under `/tmp/` first, then `dd` 'd onto the EEPROM node.

DeviceVersion providers

Reading back the *running* firmware version usually cannot be done from the raw EEPROM - it requires talking to the device itself. `eeprom_device_version.hpp` defines a `DeviceVersion` base with a factory:

```
// eeprom-device/eeprom_device_version.hpp (upstream master)
class DeviceVersion
{
public:
    virtual std::string getVersion() = 0;
    virtual std::optional<HostPowerInf::HostState>
        getHostStateToQueryVersion() = 0;
};

std::unique_ptr<DeviceVersion> getVersionProvider(
    const std::string& chipModel, uint16_t bus, uint8_t address);
```

`getHostStateToQueryVersion()` lets a provider declare that the version can only be read in a particular host power state. The Astera Labs **PT5161L** retimer ships as one such provider.

Configuration

```
{
    "Name": "Harma_Retimer0",
    "Type": "PT5161LFirmware",
    "Bus": 1,
    "Address": "0x24",
    "FirmwareInfo": {
        "VendorIANA": 40981,
        "CompatibleHardware": "com.meta.Hardware.Harma.pt5161l.Retimer"
    }
}
```

This surfaces as `xyz.openbmc_project.Configuration.EEPROMDevice`.

TPM Device Management

The `tpm/` daemon (`xyz.openbmc_project.Software.TPM.service`) participates in the same framework, but with an important limitation.

TPM firmware update is NOT implemented today. The concrete TPM implementation in `tpm/tpm2/tpm2.hpp` declares `isUpdateSupported()` to `return false`, and `TPM2Interface::updateFirmware()` immediately logs `"TPM2 firmware update is not supported"` and returns failure. The daemon exists to **report** TPM firmware version and manufacturer - not to flash the TPM.

The relevant upstream code is unambiguous:

```
// tpm/tpm2/tpm2.hpp
bool isUpdateSupported() const final
{
    // Currently, we do not support TPM2 firmware updates
    return false;
}
```

```
// tpm/tpm2/tpm2.cpp
sdplus::async::task<bool> TPM2Interface::updateFirmware(
    const uint8_t* image, size_t image_size)
{
    (void)image;
    (void)image_size;
    error("TPM2 firmware update is not supported");
    co_return false;
}
```

What it actually does

The daemon reads TPM properties by shelling out to `tpm2_getcap` against the TPM resource manager, then publishes the version on D-Bus:

```
/usr/bin/tpm2_getcap properties-fixed --tcti device:/dev/tpmrm0 | grep -A1 <property>
```

It validates the manufacturer against a known set and formats the firmware version:

Manufacturer	ID (<code>TPM2_PT_MANUFACTURER</code>)	Version property
Infineon (IFX)	<code>0x49465800</code>	<code>TPM2_PT_FIRMWARE_VERSION_1</code>
Nuvoton	<code>0x4E544300</code>	<code>TPM2_PT_FIRMWARE_VERSION_1</code> (+ <code>_VERSION_2</code>)

The version is composed by bit-shifting the raw value (`fwVer >> 16` for major, `fwVer & 0xFFFF` for minor).

Configuration

TPMs are described with the `xyz.openbmc_project.Configuration.TPM2Firmware` interface (note the distinct name from the other devices), including a `VendorIANA`:

```
{
    "Name": "System_TPM",
    "Type": "TPM2Firmware",
    "VendorIANA": 40981,
    "CompatibleHardware": "com.example.Hardware.Tpm"
}
```

Because `isUpdateSupported()` is `false`, `SoftwareManager` still surfaces the TPM's version in the firmware inventory, but any attempt to start an update is rejected. Treat the TPM entry as read-only until upstream adds an

updater back-end.

Troubleshooting

Issue: Device never appears in the firmware inventory

Symptom: The daemon is running but `busctl tree xyz.openbmc_project.Software.<Device>` shows no object for your part.

Cause: Entity Manager did not publish a matching `xyz.openbmc_project.Configuration.<Type>` interface, or the `Type` string does not match a registered factory back-end.

Solution:

1. Confirm the config is on D-Bus:

```
busctl call xyz.openbmc_project.ObjectMapper /xyz/openbmc_project/object_mapper \
xyz.openbmc_project.ObjectMapper GetSubTree sias "/xyz/openbmc_project/inventory" 0 1 \
"xyz.openbmc_project.Configuration.EEPROMDevice"
```

2. Verify the `Type` string exactly matches a factory registration (`CPLDFactory` / `VRType` / provider name).
3. Check the daemon log: `journalctl -u xyz.openbmc_project.Software.CPLD -f`.

Issue: Update starts but fails immediately

Symptom: `StartUpdate` returns, but `Activation` goes to `Failed`.

Cause: PLDM component matching found no component for this device (mismatched `CompatibleHardware` or `VendorIANA`), or the bus could not be muxed to the BMC.

Solution:

1. Confirm the package actually contains a component whose descriptor matches the device's `CompatibleHardware` string and `VendorIANA`.
2. Check for GPIO/mux errors in the log - a failed `ScopedBmcMux` construction aborts the flash.
3. For EEPROM devices, verify the `at24` driver bound: `ls /sys/bus/i2c/devices/<bus>-<addr>/eeprom`.

Issue: TPM update request is rejected

Symptom: Attempting to update the TPM returns an error / "not supported".

Cause: This is expected. `isUpdateSupported()` returns `false`; the TPM daemon is read-only.

Solution: None required - use the entry to read the TPM version only. Track upstream for a future updater back-end.

Debug Commands

```
# List all device-update daemons
systemctl list-units 'xyz.openbmc_project.Software.*'

# Follow a specific daemon
journalctl -u xyz.openbmc_project.Software.I2CVR -f

# Inspect a discovered device's version
busctl introspect xyz.openbmc_project.Software.CPLD \
  /xyz/openbmc_project/software/<id> \
  xyz.openbmc_project.Software.Version

# Read TPM version (read-only device)
tpm2_getcap properties-fixed --tcti device:/dev/tpmrm0
```

References

Official Resources

- [phosphor-bmc-code-mgmt Repository](#)
- [common/ firmware library](#)
- [cpld/ daemon](#) · [i2c-vr/ daemon](#) · [eeprom-device/ daemon](#) · [tpm/ daemon](#)
- [Software D-Bus interfaces](#)

Related Guides

- [Firmware Update Guide](#)
- [BIOS Firmware Management](#)
- [PLDM Firmware Update Guide](#)
- [MCTP & PLDM Guide](#)

External Documentation

- [PLDM Firmware Update Specification \(DMTF DSP0267\)](#)
- [Entity Manager](#)

Verified against: [openbmc/phosphor-bmc-code-mgmt](#) master branch (2026-07). Class names and supported-chip lists change frequently upstream - reconfirm against the current tree before relying on them. Last updated: 2026-07-15

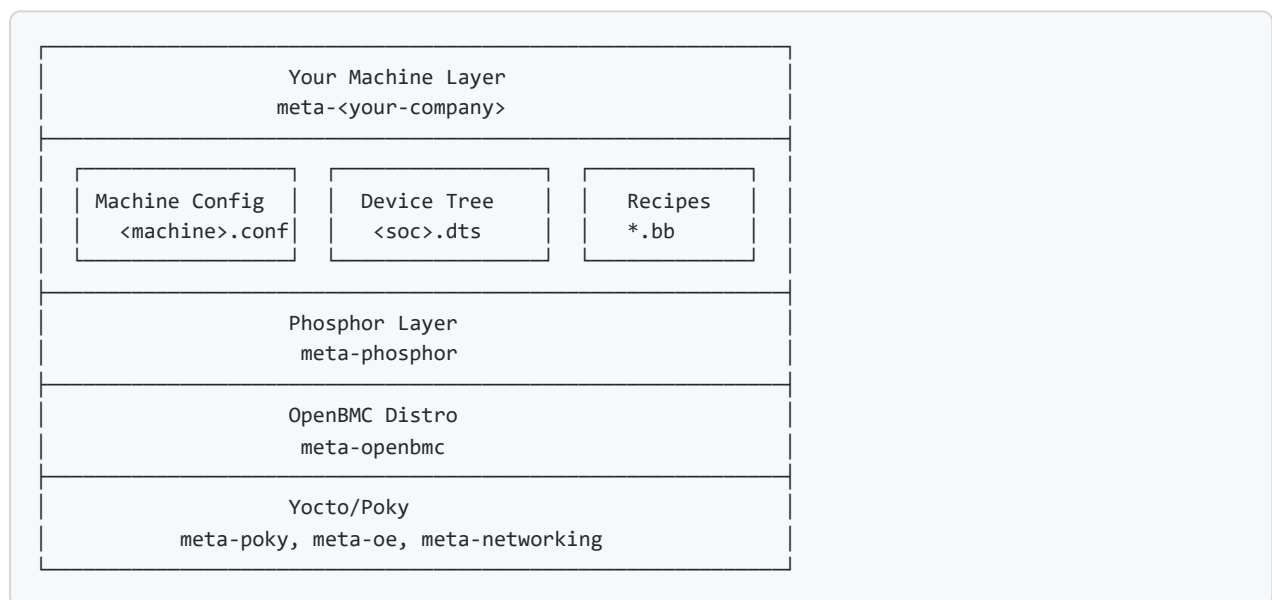
Platform Porting

Port OpenBMC to your custom hardware platform.

What You'll Learn

- Create a machine-specific layer
- Configure device tree for your BMC SoC
- Set up U-Boot for your platform
- Enable and configure OpenBMC services
- Verify and validate your port
- Port to ARM-based server platforms (NVIDIA, Ampere)

Porting Overview



Porting Workflow

1. **Plan** - Identify hardware, map features to OpenBMC services
2. **Create Layer** - Set up meta- structure
3. **Device Tree** - Configure BMC SoC peripherals
4. **U-Boot** - Configure bootloader
5. **Enable Services** - Configure OpenBMC features
6. **Test** - Verify each feature works
7. **Iterate** - Fix issues, add features

Guides in This Section

Guide	Description	Phase
Porting Reference	Complete checklist	Planning
Machine Layer	Create your layer	Setup
Device Tree	BMC SoC configuration	Configuration
U-Boot	Bootloader setup	Configuration
Verification	Testing procedures	Validation
ARM Platform Guide	ARM server platforms	Advanced
Entity Manager Advanced	Dynamic hardware config	Advanced
Flash Layout Optimization	SPI flash partitioning, image size reduction	Advanced
AST2700 Enablement	ASPEED AST2700 next-gen SoC porting	Advanced

Supported BMC SoCs

OpenBMC supports several BMC system-on-chip platforms:

SoC	Vendor	Common Platforms
AST2400	ASPEED	Legacy systems
AST2500	ASPEED	Current mainstream
AST2600	ASPEED	Latest generation
NPCM7xx	Nuvoton	Alternative platforms

For ARM-based server platforms (NVIDIA Grace, Ampere, etc.), see the [ARM Platform Guide](#).

Prerequisites

- Complete all previous sections
- Strong Linux kernel knowledge
- Device tree experience
- Yocto/BitBake proficiency
- Access to your target hardware

Porting requires hardware access. While some testing can be done in QEMU, final validation needs real hardware.

Getting Help

- [OpenBMC Mailing List](#)
- [OpenBMC Discord](#)
- Review existing machine layers in the OpenBMC repository

Table of contents

- [Porting Reference](#)
- [Machine Layer Guide](#)
- [Device Tree Guide](#)
- [U-Boot Guide](#)
- [Verification Guide](#)
- [ARM Platform Guide](#)
- [Entity Manager Advanced](#)
- [Flash Layout & Optimization Guide](#)
- [AST2700 Enablement](#)

Porting Reference

Complete guide for porting OpenBMC to a new platform.

Table of Contents

1. [Overview](#)
2. [Prerequisites](#)
3. [Porting Checklist](#)
 1. [1. Layer Setup](#)
 2. [2. Machine Configuration](#)
 3. [3. Device Tree](#)
 4. [4. U-Boot](#)
 5. [5. Kernel](#)
 6. [6. OpenBMC Services](#)
 7. [7. Verification](#)
4. [Creating the Machine Layer](#)
 1. [Layer Structure](#)
 2. [layer.conf](#)
 3. [machine.conf](#)
5. [Device Tree Configuration](#)
 1. [Example Device Tree](#)
6. [GPIO Configuration](#)
 1. [GPIO Pin Mapping](#)
 2. [Power Control Configuration](#)
7. [Entity Manager Configuration](#)
 1. [Board Configuration](#)
 2. [Recipe for Configuration](#)
8. [Verification Testing](#)
 1. [Boot Test](#)
 2. [Sensor Test](#)
 3. [Power Control Test](#)
 4. [Network Test](#)
9. [Common Pitfalls](#)
 1. [Device Tree Issues](#)
 2. [I2C Issues](#)
 3. [Service Failures](#)
10. [Debugging During Porting](#)
 1. [Early Boot Debugging](#)
 2. [Kernel Debug Configuration](#)

3. [Memory and Runtime Debugging](#)

11. [eSPI Configuration](#)

12. [References](#)

Overview

Porting OpenBMC to a new platform involves creating a machine layer, configuring hardware-specific components, and integrating with the build system.

ARM Server Platforms: For ARM-based servers (NVIDIA Grace, Ampere, etc.), see the [ARM Platform Guide](#) which covers ARM-specific boot flow, meta-arm integration, and NVIDIA OpenBMC as a reference.



Porting Architecture

Your Machine Layer (meta-myplatform)

conf/machine/
myboard.conf

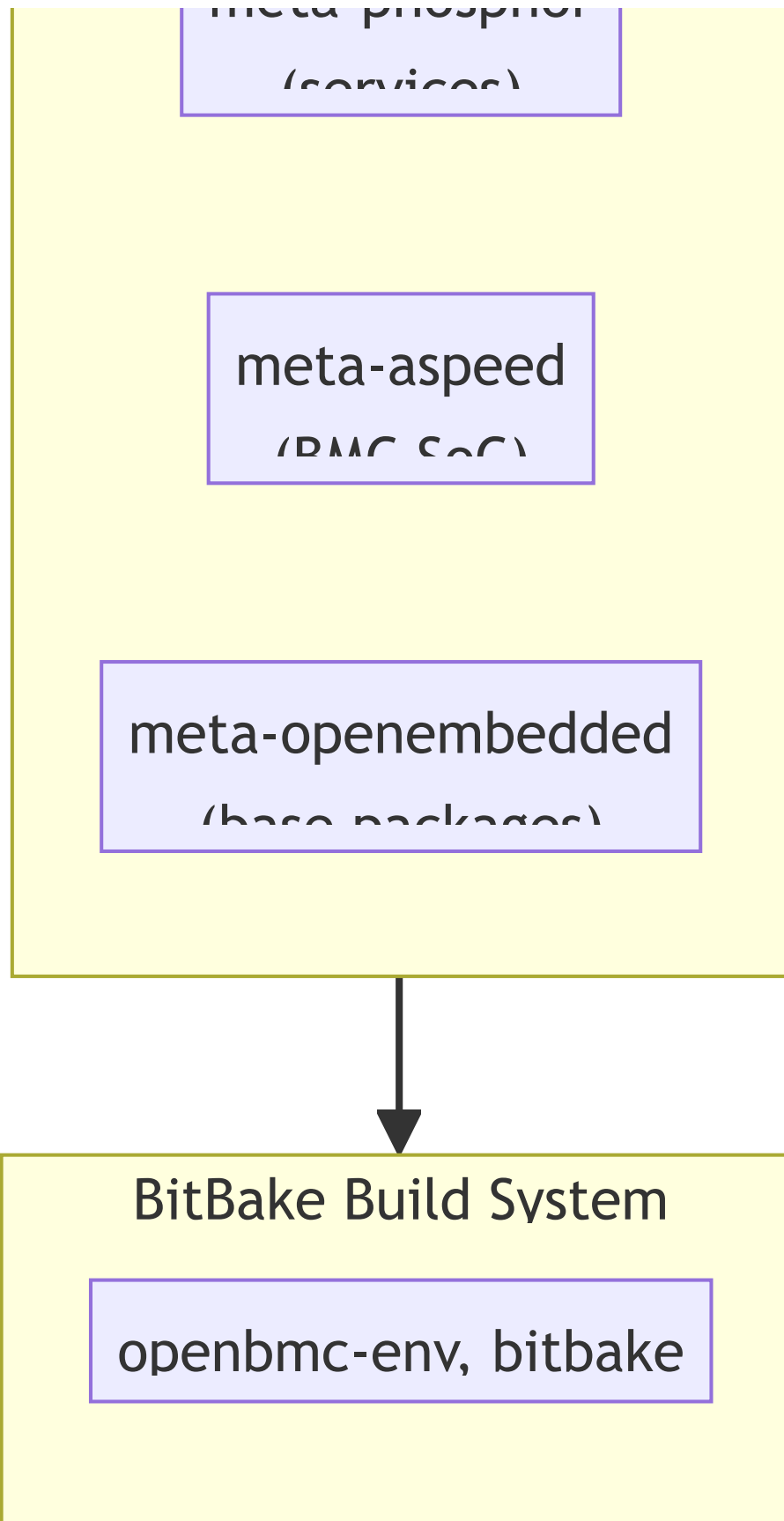
recipes-
(customization)

conf/layer.conf
(layer metadata)



OpenBMC Base Layers

meta-phosphor



► ASCII-art version (for comparison)

Prerequisites

Before porting, ensure you have:

- ☐ BMC SoC datasheet and documentation
 - ☐ Host system specifications
 - ☐ Hardware schematic
 - ☐ Working OpenBMC development environment
 - ☐ QEMU or hardware for testing
-

Porting Checklist

1. Layer Setup

- ☐ Create meta-myplatform layer
- ☐ Create conf/layer.conf
- ☐ Create conf/machine/myboard.conf
- ☐ Add to bblayers.conf

2. Machine Configuration

- ☐ Set MACHINE architecture (armv7/armv8)
- ☐ Configure BMC SoC (ASPEED/Nuvoton)
- ☐ Set kernel and U-Boot preferences
- ☐ Configure flash layout

3. Device Tree

- ☐ Create or modify device tree
- ☐ Configure GPIO mappings
- ☐ Configure I2C buses
- ☐ Configure SPI/UART/etc.

4. U-Boot

- ☐ Configure U-Boot for platform
- ☐ Set boot arguments
- ☐ Configure flash partitions

5. Kernel

- ☐ Enable required drivers
- ☐ Configure hwmon sensors
- ☐ Configure GPIO subsystem

6. OpenBMC Services

- ☐ Configure Entity Manager
- ☐ Configure sensor mappings
- ☐ Configure LED groups
- ☐ Configure power control

7. Verification

- ☐ Boot test on hardware/QEMU
- ☐ Sensor readings
- ☐ Power control
- ☐ Network connectivity
- ☐ Redfish/IPMI access

Creating the Machine Layer

Layer Structure

```
meta-myplatform/
├── conf/
│   ├── layer.conf
│   └── machine/
│       └── myboard.conf
├── recipes-bsp/
│   └── u-boot/
│       └── u-boot-aspeed_%.bbappend
├── recipes-kernel/
│   └── linux/
│       └── linux-aspeed_%.bbappend
├── recipes-phosphor/
│   ├── configuration/
│   │   └── entity-manager/
│   │       └── myboard-entity-config.bb
│   ├── sensors/
│   ├── leds/
│   └── power/
└── README.md
```

layer.conf

```
# conf/layer.conf

BBPATH .= ":${LAYERDIR}"

BBFILES += "${LAYERDIR}/recipes-*/*/*.bb \
           ${LAYERDIR}/recipes-*/*/*.bbappend"

BBFILE_COLLECTIONS += "meta-myplatform"
BBFILE_PATTERN_meta-myplatform = "^${LAYERDIR}/"
LAYERDEPENDS_meta-myplatform = " \
    core \
    meta-phosphor \
    meta-aspeed \
"
LAYERSERIES_COMPAT_meta-myplatform = "kirkstone mickledore scarthgap"
```

machine.conf

```
# conf/machine/myboard.conf

KERNEL_DEVICETREE = "aspeed/aspeed-bmc-myplatform-myboard.dtb"
PREFERRED_PROVIDER_virtual/bootloader = "u-boot-aspeed"
PREFERRED_PROVIDER_u-boot = "u-boot-aspeed"
PREFERRED_PROVIDER_u-boot-fw-utils = "u-boot-aspeed-fw-utils"

# Distro features
DISTRO_FEATURES:append = " obmc-phosphor-fan-mgmt"
DISTRO_FEATURES:append = " obmc-phosphor-chassis-mgmt"
DISTRO_FEATURES:append = " obmc-host-state-mgmt"

# Image features
IMAGE_INSTALL:append = " \
    entity-manager-myboard \
    phosphor-led-manager \
    phosphor-fan-control \
"

# Flash layout
FLASH_SIZE = "32768" # 32MB
require conf/machine/include/aspeed.inc
require conf/machine/include/obmc-bsp-common.inc

# Console settings
SERIAL_CONSOLES = "115200;ttyS4"

# Machine-specific features
MACHINE_FEATURES += "obmc-phosphor-fan-mgmt"
MACHINE_FEATURES += "obmc-phosphor-chassis-mgmt"
MACHINE_FEATURES += "obmc-host-state-mgmt"
```


Device Tree Configuration

Example Device Tree

```
// aspeed-bmc-myplatform-myboard.dts

/dts-v1/;

#include "aspeed-g5.dtsi"
#include <dt-bindings/gpio/aspeed-gpio.h>

/ {
    model = "MyPlatform MyBoard BMC";
    compatible = "myplatform,myboard-bmc", "aspeed,ast2500";

    chosen {
        stdout-path = &uart5;
        bootargs = "console=ttyS4,115200 earlyprintk";
    };

    memory@80000000 {
        reg = <0x80000000 0x20000000>; /* 512MB */
    };

    reserved-memory {
        #address-cells = <1>;
        #size-cells = <1>;
        ranges;

        video_memory: video {
            size = <0x04000000>; /* 64MB for video */
            alignment = <0x01000000>;
            compatible = "shared-dma-pool";
            reusable;
        };
    };

    leds {
        compatible = "gpio-leds";

        identify {
            label = "identify";
            gpios = <&gpio ASPEED_GPIO(A, 0) GPIO_ACTIVE_HIGH>;
            default-state = "off";
        };

        power {
            label = "power";
            gpios = <&gpio ASPEED_GPIO(A, 1) GPIO_ACTIVE_HIGH>;
            default-state = "on";
        };

        fault {
            label = "fault";
            gpios = <&gpio ASPEED_GPIO(A, 2) GPIO_ACTIVE_HIGH>;
            default-state = "off";
        };
    };

    gpio-keys {
        compatible = "gpio-keys";
    };
};
```

```

        power-button {
            label = "power-button";
            gpios = <&gpio ASPEED_GPIO(B, 0) GPIO_ACTIVE_LOW>;
            linux,code = <116>; /* KEY_POWER */
        };

        reset-button {
            label = "reset-button";
            gpios = <&gpio ASPEED_GPIO(B, 1) GPIO_ACTIVE_LOW>;
            linux,code = <0x198>; /* KEY_RESTART */
        };
    };

    &fmc {
        status = "okay";
        flash@0 {
            status = "okay";
            m25p,fast-read;
            label = "bmc";
            spi-max-frequency = <50000000>;
            #include "openbmc-flash-layout.dtsi"
        };
    };

    &spi1 {
        status = "okay";
        pinctrl-names = "default";
        pinctrl-0 = <&pinctrl_spi1_default>;
        flash@0 {
            status = "okay";
            m25p,fast-read;
            label = "pnor";
            spi-max-frequency = <100000000>;
        };
    };

    &uart5 {
        status = "okay";
    };

    &mac0 {
        status = "okay";
        pinctrl-names = "default";
        pinctrl-0 = <&pinctrl_rmii1_default>;
        use-ncsi;
    };

    &i2c0 {
        status = "okay";

        tmp75@48 {
            compatible = "ti,tmp75";
            reg = <0x48>;
            label = "cpu_temp";
        };
    };

    &i2c1 {
        status = "okay";

        eeprom@50 {
            compatible = "atmel,24c256";
            reg = <0x50>;

```

```

        label = "fru";
    };
};

&i2c2 {
    status = "okay";

    psu@58 {
        compatible = "pmbus";
        reg = <0x58>;
    };
};

&pwm_tacho {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_pwm0_default &pinctrl_pwm1_default>;

    fan@0 {
        reg = <0x00>;
        aspeed,fan-tach-ch = /bits/ 8 <0x00>;
    };

    fan@1 {
        reg = <0x01>;
        aspeed,fan-tach-ch = /bits/ 8 <0x01>;
    };
};

&video {
    status = "okay";
    memory-region = <&video_memory>;
};

&uart {
    status = "okay";
};

```

GPIO Configuration

GPIO Pin Mapping

Create a GPIO mapping document:

GPIO Pin Mapping for MyBoard

Power Control

Signal	GPIO	Direction	Active
POWER_BTN_IN	GPIOB0	Input	Low
POWER_OUT	GPIOB2	Output	High
RESET_BTN_IN	GPIOB1	Input	Low
RESET_OUT	GPIOB3	Output	High

Status Signals

Signal	GPIO	Direction	Active
POST_COMPLETE	GPIOC0	Input	High
S0_SLP_S3	GPIOC1	Input	Low
S0_SLP_S5	GPIOC2	Input	Low

LEDs

Signal	GPIO	Direction	Active
LED_IDENTIFY	GPIOA0	Output	High
LED_POWER	GPIOA1	Output	High
LED_FAULT	GPIOA2	Output	High

Power Control Configuration

```
// recipes-phosphor/configuration/power-config/power-config.json
{
  "gpio_configs": [
    {
      "name": "power_button",
      "line_name": "POWER_BTN_IN",
      "direction": "input",
      "active_low": true
    },
    {
      "name": "power_out",
      "line_name": "POWER_OUT",
      "direction": "output",
      "active_low": false
    },
    {
      "name": "reset_button",
      "line_name": "RESET_BTN_IN",
      "direction": "input",
      "active_low": true
    },
    {
      "name": "reset_out",
      "line_name": "RESET_OUT",
      "direction": "output",
      "active_low": false
    }
  ]
}
```

Entity Manager Configuration

Board Configuration

```
// recipes-phosphor/configuration/entity-manager/myboard.json
{
  "Exposes": [
    {
      "Name": "CPU_Temp",
      "Type": "TMP75",
      "Bus": 0,
      "Address": "0x48",
      "Thresholds": [
        {"Direction": "greater than", "Name": "upper critical", "Severity": 1, "Value": 95},
        {"Direction": "greater than", "Name": "upper non critical", "Severity": 0, "Value":
85},
        {"Direction": "less than", "Name": "lower critical", "Severity": 1, "Value": 0},
        {"Direction": "less than", "Name": "lower non critical", "Severity": 0, "Value": 5}
      ]
    },
    {
      "Name": "PSU1",
      "Type": "pmbus",
      "Bus": 2,
      "Address": "0x58"
    },
    {
      "Name": "FRU",
      "Type": "EEPROM",
      "Bus": 1,
      "Address": "0x50"
    }
  ],
  "Name": "MyBoard Baseboard",
  "Probe": "TRUE"
}
```

Recipe for Configuration

```
# recipes-phosphor/configuration/entity-manager/myboard-entity-config.bb
SUMMARY = "MyBoard Entity Manager Configuration"
LICENSE = "Apache-2.0"
LIC_FILES_CHKSUM = "file://${COREBASE}/meta/files/common-licenses/Apache-2.0;md5=89aea4e17d99a7cacdb
eed46a0096b10"

inherit allarch

SRC_URI = " \
    file://myboard.json \
"

S = "${WORKDIR}"

do_install() {
    install -d ${D}${datadir}/entity-manager/configurations
    install -m 0644 ${S}/myboard.json \
        ${D}${datadir}/entity-manager/configurations/
}

FILES:${PN} = "${datadir}/entity-manager/configurations/*"
```

Verification Testing

Boot Test

```
# Check boot messages
dmesg | less

# Verify services are running
systemctl status phosphor-*

# Check D-Bus objects
busctl tree xyz.openbmc_project.ObjectMapper
```

Sensor Test

```
# Check sensors
busctl tree xyz.openbmc_project.Sensor

# Read temperature sensor
busctl get-property xyz.openbmc_project.HwmonTempSensor \
    /xyz/openbmc_project/sensors/temperature/CPU_Temp \
    xyz.openbmc_project.Sensor.Value Value

# Via Redfish
curl -k -u root:0penBmc \
    https://localhost/redfish/v1/Chassis/chassis/Sensors
```

Power Control Test

```
# Check state
obmcutil state

# Power on
obmcutil poweron

# Power off
obmcutil poweroff
```

Network Test

```
# Check interface
ip addr show eth0

# Test connectivity
ping -c 3 192.168.1.1
```

Common Pitfalls

Device Tree Issues

```
# Symptom: Device not found
# Check: Device tree binding
# Solution: Verify compatible string matches driver

# Symptom: Wrong GPIO polarity
# Check: GPIO flags in device tree
# Solution: Set GPIO_ACTIVE_LOW/HIGH correctly
```

I2C Issues

```
# Symptom: Device not responding
# Check: I2C bus and address
i2cdetect -y 0

# Solution: Verify address and bus in device tree
```

Service Failures

```
# Check service status
systemctl status phosphor-pid-control

# Check logs
journalctl -u phosphor-pid-control
```

Debugging During Porting

When bringing up a new platform, kernel and service debugging is essential.

Early Boot Debugging

For kernel bring-up, enable early console output:

```
# In device tree chosen node
chosen {
    bootargs = "console=ttyS4,115200 earlyprintk";
};

# Or via U-Boot
fw_setenv bootargs "console=ttyS4,115200 earlyprintk earlycon=uart8250,mmio32,0x1e784000"
```

Kernel Debug Configuration

Add kernel debug options for porting:

```
# Recommended for porting
CONFIG_DEBUG_INFO=y
CONFIG_KASAN=y           # Kernel Address Sanitizer
CONFIG_PROVE_LOCKING=y   # Lock debugging
CONFIG_DEBUG_ATOMIC_SLEEP=y
CONFIG_STACKTRACE=y
```

Memory and Runtime Debugging

For debugging services and applications on your new platform:

```
# Enable ASan for a specific recipe
CFLAGS:append:pn-<recipe> = " -fsanitize=address -fno-omit-frame-pointer"
LDFLAGS:append:pn-<recipe> = " -fsanitize=address"

# Run service under Valgrind
valgrind --leak-check=full /usr/bin/<service>
```

For comprehensive debugging tools including ASan, UBSan, TSan, Valgrind, and kernel debug options (KASAN, KCSAN, lockdep), see the [Linux Debug Tools Guide](#).

eSPI Configuration

For Intel and AMD platforms using eSPI (Enhanced Serial Peripheral Interface) instead of LPC, additional configuration is required:

- **Virtual Wires:** Map host power state signals (SLP_S3, SLP_S5, PLTRST)
- **Flash Channel:** Configure eDAF if host BIOS is on BMC-attached flash
- **Console:** Set up eSPI UART for host console redirection

See the [eSPI Guide](#) for complete eSPI configuration including device tree examples, kernel drivers, and troubleshooting.

References

- [OpenBMC Machine Layer Examples](#)

- [ASPEED SDK Documentation](#)
 - [Yocto Project Documentation](#)
 - [Add New System Guide](#)
-

Prerequisites: Strong Yocto/BitBake knowledge, hardware access or accurate specifications

Machine Layer Guide

Create a Yocto machine layer for your OpenBMC platform.

Table of Contents

1. [Overview](#)
 2. [Layer Structure](#)
 3. [Creating layer.conf](#)
 4. [Machine Configuration](#)
 1. [Basic machine.conf](#)
 2. [Feature Options](#)
 5. [Adding the Layer to Build](#)
 1. [Update bblayers.conf](#)
 2. [Set Machine](#)
 6. [Common Customizations](#)
 1. [Add Entity Manager Configuration](#)
 2. [Customize LED Configuration](#)
 7. [Building](#)
 8. [Example Machine Layer](#)
 9. [Next Steps](#)
 10. [References](#)
-

Overview

A machine layer defines your specific hardware platform for OpenBMC. It contains machine configuration, recipes, and customizations.

Layer Structure

```
meta-myplatform/
├── conf/
│   ├── layer.conf                # Layer metadata
│   └── machine/
│       └── myboard.conf          # Machine configuration
├── recipes-bsp/
│   └── u-boot/
│       └── u-boot-aspeed_%.bbappend # U-Boot customization
├── recipes-kernel/
│   └── linux/
│       ├── linux-aspeed_%.bbappend # Kernel config
│       └── linux-aspeed/
│           └── myboard.cfg          # Kernel fragments
├── recipes-phosphor/
│   ├── configuration/
│   │   └── entity-manager/         # Hardware configuration
│   ├── leds/                      # LED configuration
│   ├── sensors/                   # Sensor customization
│   └── gpio/                      # GPIO configuration
└── README.md
```

Creating layer.conf

```
# conf/layer.conf

# Add layer to BBPATH
BBPATH .= ":${LAYERDIR}"

# Recipe locations
BBFILES += " \
    ${LAYERDIR}/recipes-*/*/.bb \
    ${LAYERDIR}/recipes-*/*/.bbappend \
"

# Layer identification
BBFILE_COLLECTIONS += "meta-myplatform"
BBFILE_PATTERN_meta-myplatform = "^${LAYERDIR}/"
BBFILE_PRIORITY_meta-myplatform = "10"

# Layer dependencies
LAYERDEPENDS_meta-myplatform = " \
    core \
    meta-phosphor \
    meta-aspeed \
"

# Compatible Yocto versions
LAYERSERIES_COMPAT_meta-myplatform = "kirkstone mickledore scarthgap"
```

Machine Configuration

Basic machine.conf

```
# conf/machine/myboard.conf

#@TYPE: Machine
#@NAME: MyBoard BMC
#@DESCRIPTION: Machine configuration for MyBoard

# Kernel device tree
KERNEL_DEVICETREE = "aspeed/aspeed-bmc-myplatform-myboard.dtb"

# U-Boot configuration
PREFERRED_PROVIDER_virtual/bootloader = "u-boot-aspeed"
PREFERRED_PROVIDER_u-boot = "u-boot-aspeed"
PREFERRED_PROVIDER_u-boot-fw-utils = "u-boot-aspeed-fw-utils"

# Flash size (KB)
FLASH_SIZE = "32768"

# Console
SERIAL_CONSOLES = "115200;ttyS4"

# Machine features - enable OpenBMC capabilities
MACHINE_FEATURES += " \
    obmc-phosphor-fan-mgmt \
    obmc-phosphor-chassis-mgmt \
    obmc-host-state-mgmt \
"

# Distro features
DISTRO_FEATURES:append = " obmc-ikvm"
DISTRO_FEATURES:append = " obmc-virtual-media"

# Packages to install
IMAGE_INSTALL:append = " \
    entity-manager \
    dbus-sensors \
    phosphor-pid-control \
    myboard-entity-config \
"

# Include base configuration
require conf/machine/include/aspeed.inc
require conf/machine/include/obmc-bsp-common.inc
```

Feature Options

Feature	Description
<code>obmc-phosphor-fan-mgmt</code>	Fan control and monitoring
<code>obmc-phosphor-chassis-mgmt</code>	Chassis power control
<code>obmc-host-state-mgmt</code>	Host state management
<code>obmc-ikvm</code>	Remote KVM support
<code>obmc-virtual-media</code>	Virtual media support

Adding the Layer to Build

Update bblayers.conf

```
# Add layer path
echo 'BBLAYERS += "/path/to/meta-myplatform"' >> build/conf/bblayers.conf
```

Set Machine

```
# In local.conf
echo 'MACHINE = "myboard"' >> build/conf/local.conf
```

Common Customizations

Add Entity Manager Configuration

```
# recipes-phosphor/configuration/entity-manager/myboard-config.bb

SUMMARY = "MyBoard Entity Manager Configuration"
LICENSE = "Apache-2.0"
LIC_FILES_CHKSUM = "file://${COREBASE}/meta/files/common-licenses/Apache-2.0;md5=89aea4e17d99a7cacdb
eed46a0096b10"

inherit allarch

SRC_URI = "file://myboard.json"

S = "${WORKDIR}"

do_install() {
    install -d ${D}${datadir}/entity-manager/configurations
    install -m 0644 ${S}/myboard.json ${D}${datadir}/entity-manager/configurations/
}

FILES:${PN} = "${datadir}/entity-manager/configurations/*"
```

Customize LED Configuration

```
# recipes-phosphor/leds/myboard-led-config.bb

SUMMARY = "MyBoard LED Configuration"
LICENSE = "Apache-2.0"

inherit allarch

SRC_URI = "file://led.yaml"

do_install() {
    install -d ${D}${datadir}/phosphor-led-manager
    install -m 0644 ${S}/led.yaml ${D}${datadir}/phosphor-led-manager/
}
```

Building

```
# Initialize environment
. setup myboard

# Build image
bitbake obmc-phosphor-image
```

Example Machine Layer

Working examples are available in the [examples/porting](#) directory:

- [meta-myplatform/](#) - Complete template machine layer with Entity Manager configuration
- [README.md](#) - Quick start guide for customizing the template

Next Steps

- [Device Tree](#) - Configure BMC SoC peripherals
- [Entity Manager Advanced](#) - Dynamic hardware configuration

References

- [Yocto Machine Configuration](#)
- [OpenBMC Machine Examples](#)
- [meta-phosphor Documentation](#)

Prerequisites: Yocto/BitBake experience required

Device Tree Guide

Configure the Linux device tree for your BMC platform.

Table of Contents

1. [Overview](#)
 2. [Device Tree Structure](#)
 3. [GPIO Configuration](#)
 1. [LED GPIOs](#)
 2. [Button GPIOs](#)
 4. [I2C Bus Configuration](#)
 5. [SPI Flash Configuration](#)
 6. [Network Configuration](#)
 1. [NCSI \(Sideband\)](#)
 2. [Dedicated Ethernet](#)
 7. [PWM/Fan Configuration](#)
 8. [Console/UART Configuration](#)
 9. [Video Capture \(KVM\)](#)
 10. [Adding Device Tree to Build](#)
 11. [Debugging Device Tree](#)
 12. [References](#)
-

Overview

The device tree describes your hardware to the Linux kernel, including GPIO pins, I2C buses, SPI flash, and peripherals.

Device Tree Structure

Source Reference: OpenBMC Device Tree examples

- ASPEED DTS: [linux/arch/arm/boot/dts/aspeed/](#)
- Reference: [aspeed-bmc-opp-romulus.dts](#)

```

/dts-v1/;

#include "aspeed-g5.dtsi"
#include <dt-bindings/gpio/aspeed-gpio.h>

/ {
    model = "MyPlatform MyBoard BMC";
    compatible = "myplatform,myboard-bmc", "aspeed,ast2500";

    chosen {
        stdout-path = &uart5;
        bootargs = "console=ttyS4,115200 earlyprintk";
    };

    memory@80000000 {
        reg = <0x80000000 0x20000000>; /* 512MB */
    };
};

```

GPIO Configuration

LED GPIOs

```

leds {
    compatible = "gpio-leds";

    identify {
        label = "identify";
        gpios = <&gpio ASPEED_GPIO(A, 0) GPIO_ACTIVE_HIGH>;
        default-state = "off";
    };

    power {
        label = "power";
        gpios = <&gpio ASPEED_GPIO(A, 1) GPIO_ACTIVE_HIGH>;
        default-state = "on";
    };

    fault {
        label = "fault";
        gpios = <&gpio ASPEED_GPIO(A, 2) GPIO_ACTIVE_HIGH>;
        default-state = "off";
    };
};

```


Button GPIOs

```
gpio-keys {
    compatible = "gpio-keys";

    power-button {
        label = "power-button";
        gpios = <&gpio ASPEED_GPIO(B, 0) GPIO_ACTIVE_LOW>;
        linux,code = <116>; /* KEY_POWER */
    };

    reset-button {
        label = "reset-button";
        gpios = <&gpio ASPEED_GPIO(B, 1) GPIO_ACTIVE_LOW>;
        linux,code = <0x198>; /* KEY_RESTART */
    };
};
```

I2C Bus Configuration

```
&i2c0 {
    status = "okay";

    /* Temperature sensor */
    tmp75@48 {
        compatible = "ti,tmp75";
        reg = <0x48>;
    };
};

&i2c1 {
    status = "okay";

    /* FRU EEPROM */
    eeprom@50 {
        compatible = "atmel,24c256";
        reg = <0x50>;
    };
};

&i2c2 {
    status = "okay";

    /* Power supply */
    psu@58 {
        compatible = "pmbus";
        reg = <0x58>;
    };
};
```

SPI Flash Configuration

```
&fmc {
    status = "okay";
    flash@0 {
        status = "okay";
        m25p,fast-read;
        label = "bmc";
        spi-max-frequency = <50000000>;
        #include "openbmc-flash-layout.dtsi"
    };
};

&spi1 {
    status = "okay";
    flash@0 {
        status = "okay";
        label = "pnor";
        spi-max-frequency = <100000000>;
    };
};
```

Network Configuration

NCSI (Sideband)

```
&mac0 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_rmii1_default>;
    use-ncsi;
};
```

Dedicated Ethernet

```
&mac1 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_rgmii2_default>;
    phy-mode = "rgmii";
    phy-handle = <&phy1>;
};
```

PWM/Fan Configuration

```
&pwm_tacho {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_pwm0_default &pinctrl_pwm1_default>;

    fan@0 {
        reg = <0x00>;
        aspeed,fan-tach-ch = /bits/ 8 <0x00>;
    };

    fan@1 {
        reg = <0x01>;
        aspeed,fan-tach-ch = /bits/ 8 <0x01>;
    };
};
```

Console/UART Configuration

```
&uart5 {
    status = "okay";
};

&vuart {
    status = "okay";
};
```

Video Capture (KVM)

```
reserved-memory {
    video_memory: video {
        size = <0x04000000>; /* 64MB */
        alignment = <0x01000000>;
        compatible = "shared-dma-pool";
        reusable;
    };
};

&video {
    status = "okay";
    memory-region = <&video_memory>;
};
```

Adding Device Tree to Build

```
# recipes-kernel/linux/linux-aspeed_%.bbappend

FILESEXTRAPATHS:prepend := "${THISDIR}/linux-aspeed:"

SRC_URI += "file:///aspeed-bmc-myplatform-myboard.dts"

do_configure:append() {
    cp ${WORKDIR}/aspeed-bmc-myplatform-myboard.dts \
        ${S}/arch/arm/boot/dts/aspeed/
}
```

Debugging Device Tree

```
# View compiled device tree
dtc -I dtb -O dts /sys/firmware/fdt

# Check GPIO assignments
cat /sys/kernel/debug/gpio

# Check I2C buses
i2cdetect -l
i2cdetect -y 0
```

References

- [Linux Device Tree Documentation](#)
 - [ASPEED Device Tree Bindings](#)
 - [Device Tree Specification](#)
-

Prerequisites: Linux kernel and device tree knowledge required

U-Boot Guide

Configure the U-Boot bootloader for your BMC platform.

Table of Contents

1. [Overview](#)
 2. [U-Boot Configuration](#)
 1. [Default Configuration](#)
 2. [Board Configuration](#)
 3. [Environment Variables](#)
 1. [Key Variables](#)
 2. [Boot Commands](#)
 4. [Dual Boot Support](#)
 1. [Partition Layout](#)
 2. [Boot Selection Logic](#)
 5. [Adding U-Boot to Build](#)
 1. [Recipe Append](#)
 6. [U-Boot Commands](#)
 1. [Common Operations](#)
 2. [Flash Operations](#)
 3. [Debug](#)
 7. [Verified Boot](#)
 1. [Enable Secure Boot](#)
 2. [Sign Images](#)
 8. [Troubleshooting](#)
 1. [Boot Fails](#)
 2. [Environment Corrupt](#)
 9. [References](#)
-

Overview

U-Boot is the bootloader for OpenBMC. It initializes hardware, loads the Linux kernel, and supports dual-image boot for failsafe recovery.

U-Boot Configuration

Default Configuration

```
# configs/ast2500_openbmc_defconfig

CONFIG_ARM=y
CONFIG_ARCH_ASPEED=y
CONFIG_SYS_TEXT_BASE=0x00000000
CONFIG_ASPEED_AST2500=y
CONFIG_TARGET_MYBOARD=y
CONFIG_DEFAULT_DEVICE_TREE="aspeed-bmc-myplatform-myboard"
CONFIG_FIT=y
CONFIG_FIT_SIGNATURE=y
CONFIG_SPL=y
```

Board Configuration

```
// include/configs/myboard.h

#ifndef __CONFIG_MYBOARD_H
#define __CONFIG_MYBOARD_H

#include <configs/aspeed-common.h>

#define CONFIG_MACH_TYPE    MACH_TYPE_ASPEED

/* Memory */
#define CONFIG_SYS_SDRAM_SIZE    0x20000000 /* 512MB */

/* Flash */
#define CONFIG_SYS_MAX_FLASH_BANKS    1
#define CONFIG_SYS_MAX_FLASH_SECT    512

/* Console */
#define CONFIG_CONS_INDEX    5 /* UART5 */
#define CONFIG_BAUDRATE    115200

/* Boot */
#define CONFIG_BOOTCOMMAND    "run bootcmd_obmc"
#define CONFIG_BOOTARGS    "console=ttyS4,115200 earlyprintk"

#endif
```

Environment Variables

Key Variables

Variable	Description	Example
<code>bootcmd</code>	Boot command	<code>run bootcmd_obmc</code>
<code>bootargs</code>	Kernel arguments	<code>console=ttyS4,115200</code>
<code>bootside</code>	Active boot partition	<code>a</code> or <code>b</code>
<code>bootcount</code>	Boot attempt counter	<code>0</code>
<code>bootlimit</code>	Max boot attempts	<code>3</code>

Boot Commands

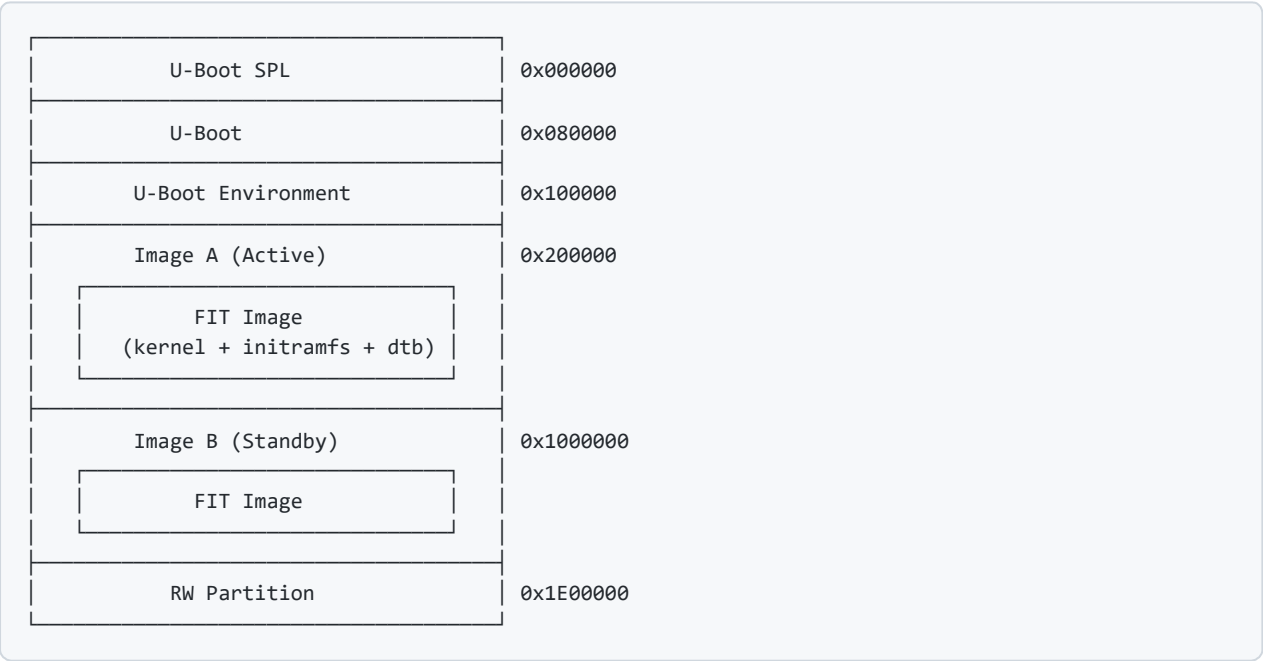
```
# Default boot command
bootcmd_obmc=run set_bootargs; bootm ${kerneladdr}

# Set boot arguments
set_bootargs=setenv bootargs console=ttyS4,115200 root=/dev/ram rw

# Load kernel
loadkernel=sf probe 0; sf read ${kerneladdr} ${kerneloff} ${kernelsize}
```

Dual Boot Support

Partition Layout



Boot Selection Logic

```
# Check boot side
if test "${bootside}" = "a"; then
    kerneladdr=0x20200000
else
    kerneladdr=0x21000000
fi

# Increment boot counter
setexpr bootcount ${bootcount} + 1

# Check boot limit
if test ${bootcount} -ge ${bootlimit}; then
    # Switch to other side
    if test "${bootside}" = "a"; then
        setenv bootside b
    else
        setenv bootside a
    fi
    setenv bootcount 0
fi
```

Adding U-Boot to Build

Recipe Append

```
# recipes-bsp/u-boot/u-boot-aspeed_%.bbappend

FILESEXTRAPATHS:prepend := "${THISDIR}/u-boot-aspeed:"

SRC_URI += " \
    file://myboard_defconfig \
    file://myboard.h \
"

do_configure:prepend() {
    install -m 0644 ${WORKDIR}/myboard_defconfig \
        ${S}/configs/
    install -m 0644 ${WORKDIR}/myboard.h \
        ${S}/include/configs/
}
```


U-Boot Commands

Common Operations

```
# Print environment
printenv

# Set variable
setenv bootargs console=ttyS4,115200

# Save environment
saveenv

# Boot kernel
bootm ${kerneladdr}

# Reset
reset
```

Flash Operations

```
# Probe SPI flash
sf probe 0

# Read flash
sf read 0x83000000 0x200000 0x800000

# Erase flash
sf erase 0x200000 0x800000

# Write flash
sf write 0x83000000 0x200000 0x800000
```

Debug

```
# Memory dump
md 0x80000000 100

# I2C scan
i2c bus
i2c dev 0
i2c probe

# GPIO
gpio status
```

Verified Boot

Enable Secure Boot

```
# In defconfig
CONFIG_FIT_SIGNATURE=y
CONFIG_RSA=y
CONFIG_FIT_SIGNATURE_ENFORCE=y
```

Sign Images

```
# Sign FIT image with key
mkimage -F -k keys/ -K u-boot.dtb -r image.fit
```

Troubleshooting

Boot Fails

```
# Check boot counter
printenv bootcount

# Force boot side
setenv bootside a
saveenv
reset
```

Environment Corrupt

```
# Reset to defaults
env default -a
saveenv
```

References

- [U-Boot Documentation](#)
 - [ASPEED U-Boot](#)
 - [OpenBMC U-Boot Integration](#)
-

Prerequisites: Bootloader and embedded systems experience required

Verification Guide

Test and validate your OpenBMC port.

Table of Contents

1. [Overview](#)
 2. [Verification Checklist](#)
 1. [Boot & Basic](#)
 2. [Power Control](#)
 3. [Sensors](#)
 4. [Thermal Management](#)
 5. [Remote Access](#)
 3. [Boot Verification](#)
 1. [U-Boot Check](#)
 2. [Kernel Boot](#)
 3. [BMC State](#)
 4. [Power Control Tests](#)
 1. [Basic Power Operations](#)
 2. [GPIO Verification](#)
 5. [Sensor Verification](#)
 1. [List Sensors](#)
 2. [Verify Readings](#)
 3. [Threshold Test](#)
 6. [Fan Control Tests](#)
 1. [Manual Control](#)
 2. [Verify Fan Speed](#)
 7. [Network Tests](#)
 1. [Basic Connectivity](#)
 2. [Service Access](#)
 8. [Redfish API Tests](#)
 9. [IPMI Tests](#)
 10. [Common Issues](#)
 1. [Sensors Not Appearing](#)
 2. [Power Control Fails](#)
 3. [Network Issues](#)
 11. [Automated Testing](#)
 1. [Robot Framework](#)
 12. [Next Steps](#)
 13. [References](#)
-

Overview

Systematic verification ensures your OpenBMC port works correctly. This guide provides test procedures for each major subsystem.

Verification Checklist

Boot & Basic

- ☐ U-Boot initializes correctly
- ☐ Kernel boots without errors
- ☐ Root filesystem mounts
- ☐ BMC reaches Ready state
- ☐ Console access works
- ☐ Network connectivity

Power Control

- ☐ Power on host
- ☐ Power off host
- ☐ Graceful shutdown
- ☐ Force power off
- ☐ Power cycle
- ☐ Reset host

Sensors

- ☐ Temperature sensors read correctly
- ☐ Voltage sensors read correctly
- ☐ Fan tach sensors work
- ☐ Thresholds trigger events
- ☐ Sensors appear in Redfish/IPMI

Thermal Management

- ☐ Fan PWM control works
- ☐ PID control adjusts fans
- ☐ Thermal zones configured
- ☐ Over-temperature protection

Remote Access

- ☐ SSH login
- ☐ WebUI access
- ☐ Redfish API

- ☐ IPMI over LAN
 - ☐ KVM (if applicable)
 - ☐ Virtual media (if applicable)
-

Boot Verification

U-Boot Check

```
# During boot, watch for:
# - Memory initialization
# - Flash detection
# - Kernel loading

# At U-Boot prompt:
printenv
md 0x80000000 100 # Memory check
```

Kernel Boot

```
# Watch console for:
# - Device tree loading
# - Driver initialization
# - Filesystem mount

# Check dmesg after boot
dmesg | grep -i error
dmesg | grep -i fail
```

BMC State

```
# Check BMC state
obmcutil state

# Expected:
# BMC: Ready
# Chassis: Off (initially)
# Host: Off (initially)
```

Power Control Tests

Basic Power Operations

```
# Power on
obmcutil poweron
sleep 10
obmcutil state # Check host is Running

# Power off
obmcutil poweroff
sleep 10
obmcutil state # Check host is Off

# Power cycle
obmcutil poweron
sleep 30
obmcutil powercycle
sleep 10
obmcutil state
```

GPIO Verification

```
# Check power GPIOs
cat /sys/kernel/debug/gpio | grep -i power

# Manual GPIO test (careful!)
gpioset gpiochip0 10=1 # Assert power
gpioget gpiochip0 11   # Read status
```

Sensor Verification

List Sensors

```
# D-Bus sensors
busctl tree xyz.openbmc_project.Sensor

# Redfish sensors
curl -k -u root:0penBmc \
  https://localhost/redfish/v1/Chassis/chassis/Sensors

# IPMI sensors
ipmitool sensor list
```

Verify Readings

```
# Read specific sensor
busctl get-property xyz.openbmc_project.HwmonTempSensor \
  /xyz/openbmc_project/sensors/temperature/CPU_Temp \
  xyz.openbmc_project.Sensor.Value Value

# Check hwmon
cat /sys/class/hwmon/hwmon*/temp*_input
```

Threshold Test

```
# Force threshold crossing (test environment only)
# Watch for events in Redfish logs
curl -k -u root:0penBmc \
  https://localhost/redfish/v1/Systems/system/LogServices/EventLog/Entries
```

Fan Control Tests

Manual Control

```
# Set manual mode
busctl set-property xyz.openbmc_project.State.FanCtrl \
  /xyz/openbmc_project/control/fanpwm/zone0 \
  xyz.openbmc_project.Control.Mode \
  Manual b true

# Set PWM
busctl set-property xyz.openbmc_project.Hwmon.external \
  /xyz/openbmc_project/control/fanpwm/Pwm_0 \
  xyz.openbmc_project.Control.FanPwm \
  Target t 50
```

Verify Fan Speed

```
# Read tachometer
busctl get-property xyz.openbmc_project.FanSensor \
  /xyz/openbmc_project/sensors/fan_tach/Fan0 \
  xyz.openbmc_project.Sensor.Value Value
```

Network Tests

Basic Connectivity

```
# Check interface
ip addr show eth0

# Ping gateway
ping -c 3 192.168.1.1

# DNS resolution
nslookup example.com
```

Service Access

```
# SSH
ssh -p 22 root@bmc-ip

# HTTPS
curl -k https://bmc-ip/

# IPMI
ipmitool -I lanplus -H bmc-ip -U root -P 0penBmc power status
```

Redfish API Tests

```
# Service root
curl -k -u root:0penBmc https://localhost/redfish/v1/

# System info
curl -k -u root:0penBmc https://localhost/redfish/v1/Systems/system

# Manager info
curl -k -u root:0penBmc https://localhost/redfish/v1/Managers/bmc

# Chassis info
curl -k -u root:0penBmc https://localhost/redfish/v1/Chassis/chassis
```

IPMI Tests

```
# Power status
ipmitool -I lanplus -H bmc-ip -U root -P 0penBmc power status

# Sensor list
ipmitool -I lanplus -H bmc-ip -U root -P 0penBmc sensor list

# FRU data
ipmitool -I lanplus -H bmc-ip -U root -P 0penBmc fru print

# SEL
ipmitool -I lanplus -H bmc-ip -U root -P 0penBmc sel list
```

Common Issues

Sensors Not Appearing

```
# Check Entity Manager
journalctl -u entity-manager

# Verify configuration
cat /usr/share/entity-manager/configurations/*.json

# Check dbus-sensors
journalctl -u dbus-sensors
```


Power Control Fails

```
# Check GPIO configuration
cat /sys/kernel/debug/gpio

# Check state manager
journalctl -u phosphor-state-manager-host

# Verify device tree GPIO nodes
```

Network Issues

```
# Check driver
dmesg | grep -i eth

# Check systemd-networkd
systemctl status systemd-networkd
journalctl -u systemd-networkd
```

Automated Testing

Robot Framework

```
*** Test Cases ***
Verify BMC State Is Ready
    ${state}=    Get BMC State
    Should Be Equal    ${state}    Ready

Verify Power On Works
    Power On Host
    Wait Until Host Is Running
    ${state}=    Get Host State
    Should Be Equal    ${state}    Running
```

Next Steps

- [Robot Framework Guide](#) - Detailed integration testing guide
- [Unit Testing Guide](#) - GTest/GMock unit tests

References

- [OpenBMC Test Documentation](#)
- [Robot Framework](#)

Prerequisites: Working OpenBMC build on target hardware

ARM Platform Guide

Port OpenBMC to ARM-based server platforms with NVIDIA OpenBMC as reference.

Table of Contents

1. [Overview](#)
 2. [ARM vs ASPEED Architecture](#)
 1. [Key Differences](#)
 2. [Platform Communication](#)
 3. [meta-arm Layer Integration](#)
 1. [Layer Stack](#)
 2. [Adding meta-arm to Your Build](#)
 3. [meta-arm Components](#)
 4. [Machine Configuration](#)
 1. [ARM Machine Template](#)
 2. [Key ARM Machine Variables](#)
 5. [NVIDIA OpenBMC Reference](#)
 1. [Repository Structure](#)
 2. [NVIDIA Layer Configuration](#)
 3. [NVIDIA Machine Configuration Example](#)
 6. [ARM Boot Flow](#)
 1. [Boot Sequence](#)
 2. [Boot Components](#)
 3. [TF-A Configuration](#)
 4. [U-Boot for ARM Platforms](#)
 7. [ARM Device Tree](#)
 1. [Device Tree Structure](#)
 2. [Common ARM Bindings](#)
 8. [MCTP/PLDM Integration](#)
 1. [Common Configuration](#)
 9. [Example ARM Machine Layer](#)
 10. [Troubleshooting](#)
 1. [Common Issues](#)
 2. [Debug Tips](#)
 11. [References](#)
-

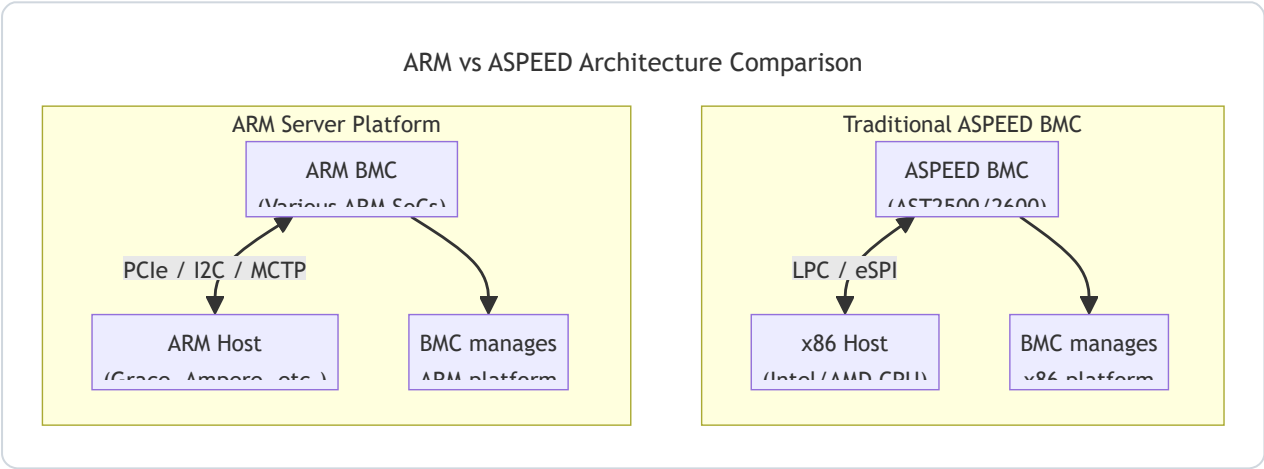
Overview

This guide covers porting OpenBMC to ARM-based server platforms, using NVIDIA’s OpenBMC fork as a reference implementation. While the [Porting Reference](#) focuses on ASPEED-based BMCs managing x86 hosts, this guide addresses ARM-native server platforms where both the host and BMC may use ARM processors.

Reference Repository: [NVIDIA OpenBMC](#) (develop branch)

ARM vs ASPEED Architecture

Key Differences



► ASCII-art version (for comparison)

Aspect	ASPEED BMC	ARM Platform
BMC SoC	AST2500/AST2600 (ARM core)	Various ARM SoCs
Host CPU	x86 (Intel/AMD)	ARM (Grace, Ampere, etc.)
Host Interface	LPC, eSPI	PCIe, I2C, MCTP
Boot Firmware	ASPEED SDK	TF-A, OP-TEE, U-Boot
Console	VUART, LPC UART	Physical UART, SSH
POST Codes	LPC snoop	Platform-specific

Platform Communication

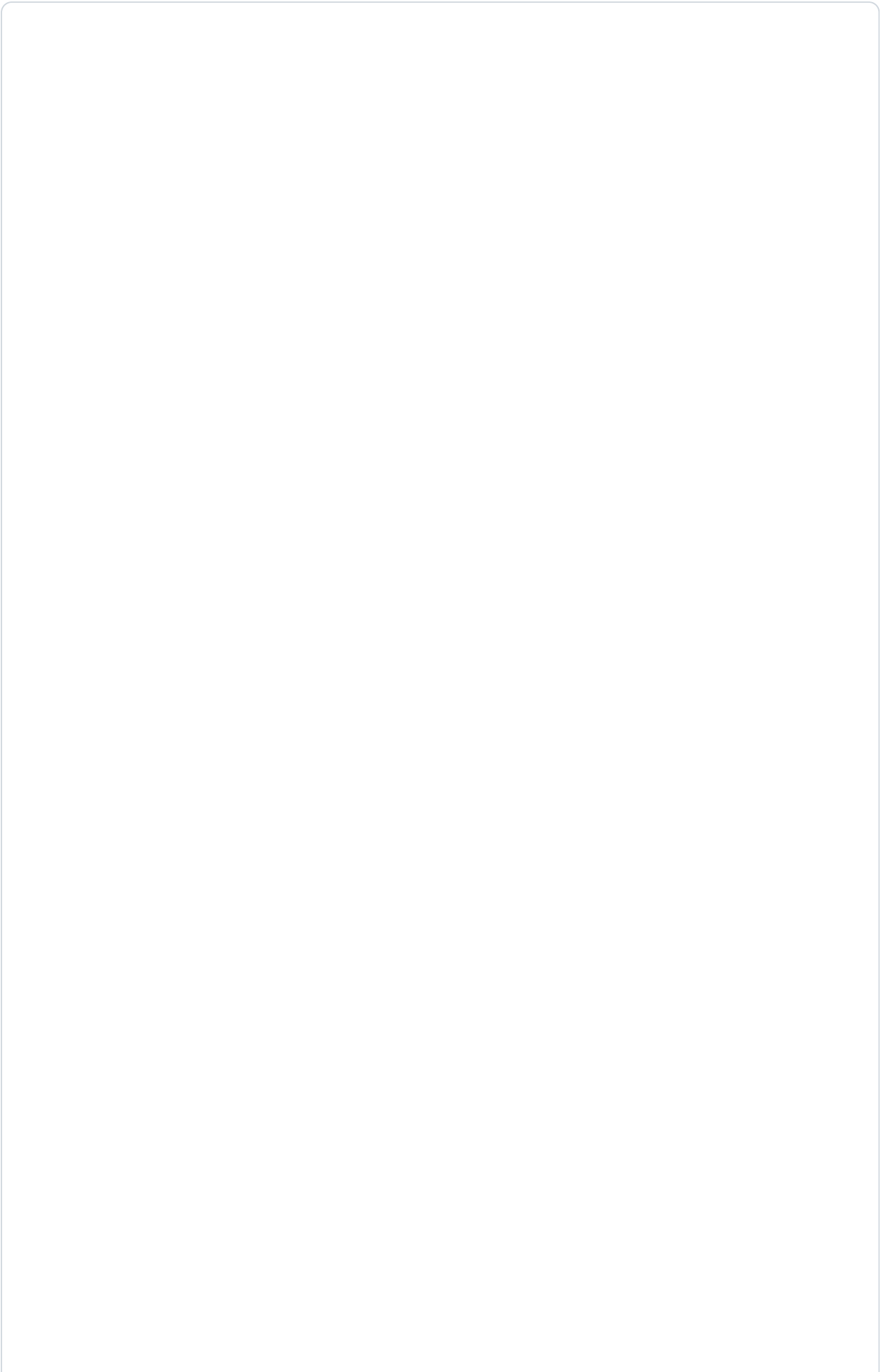
ARM platforms typically use different host-BMC communication methods:

Interface	Description	Use Case
PCIe	High-bandwidth link	Firmware updates, data transfer
I2C/SMBus	Low-speed control bus	Sensor polling, FRU access
MCTP	Management Component Transport	PLDM, SPDAM messaging
GPIO	Direct signal control	Power control, reset, alerts
Network	Out-of-band management	Redfish, IPMI-over-LAN

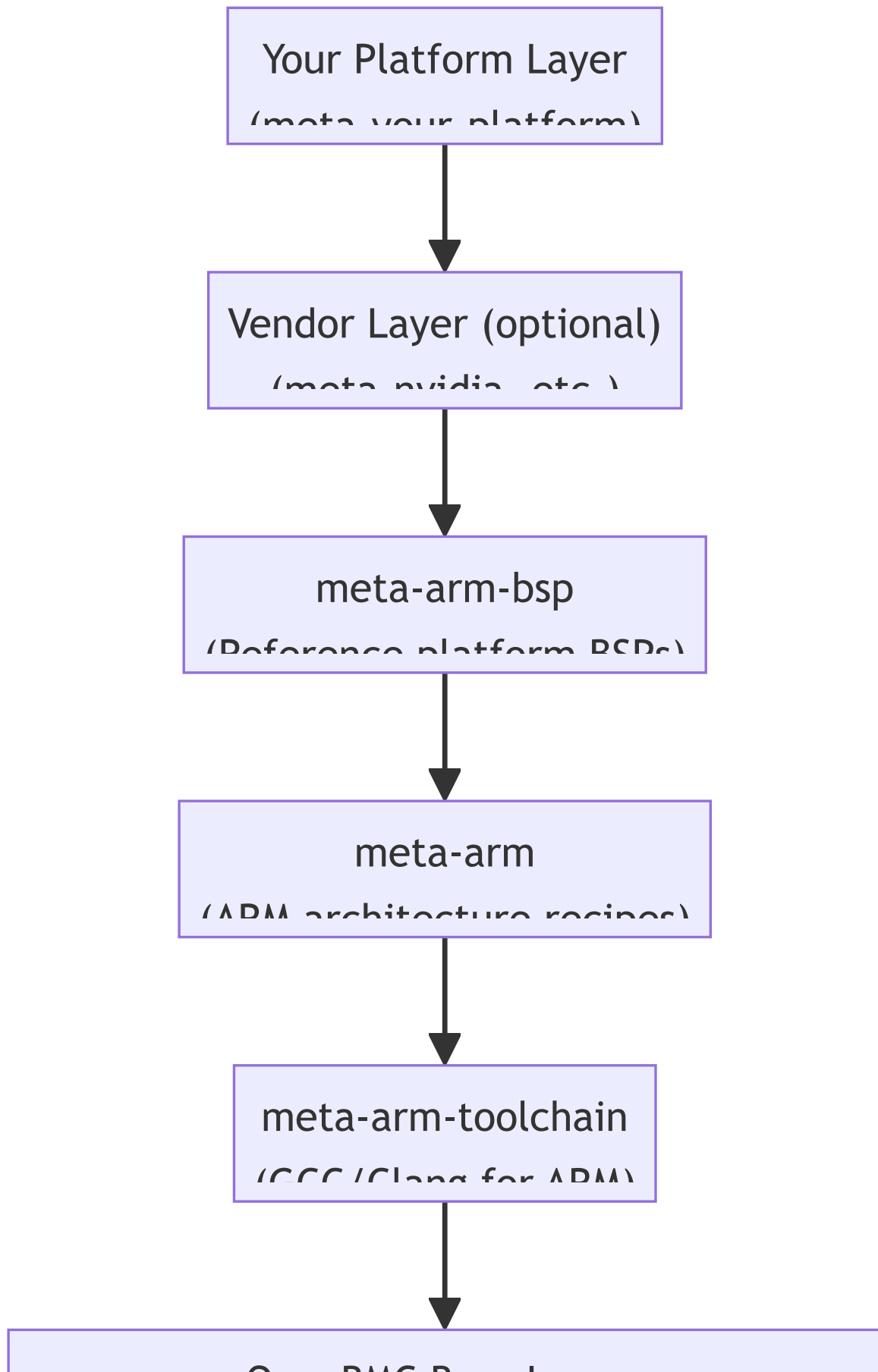
meta-arm Layer Integration

ARM platforms use the Yocto Project’s meta-arm layer stack for base support.

Layer Stack



meta-arm Layer Stack



OpenBMC Base Layers

(meta-phosphor meta-openembedded poky)

► ASCII-art version (for comparison)

Adding meta-arm to Your Build

```
# Clone meta-arm layers
cd /path/to/openbmc
git clone https://git.yoctoproject.org/meta-arm

# Add to bblayers.conf
cat >> conf/bblayers.conf << 'EOF'
BBLAYERS += "${BSPDIR}/meta-arm/meta-arm"
BBLAYERS += "${BSPDIR}/meta-arm/meta-arm-toolchain"
# Optional: Reference BSPs
# BBLAYERS += "${BSPDIR}/meta-arm/meta-arm-bsp"
EOF
```

meta-arm Components

Layer	Purpose	Key Recipes
meta-arm	Core ARM support	TF-A, OP-TEE, firmware
meta-arm-toolchain	ARM compilers	arm-gnu-toolchain, GCC
meta-arm-bsp	Reference platforms	FVP, Juno, N1SDP

Machine Configuration

ARM Machine Template

Create your machine configuration based on ARM patterns:


```

# conf/machine/myarm-bmc.conf

#@TYPE: Machine
#@NAME: My ARM BMC Platform
#@DESCRIPTION: OpenBMC for ARM-based server

# ARM64 architecture
DEFAULTTUNE = "cortexa53"
require conf/machine/include/arm/armv8a/tune-cortexa53.inc

# Kernel configuration
PREFERRED_PROVIDER_virtual/kernel = "linux-aspeed"
KERNEL_IMAGETYPE = "Image"
KERNEL_DEVICETREE = "${KMACHINE}.dtb"

# U-Boot configuration
PREFERRED_PROVIDER_virtual/bootloader = "u-boot"
PREFERRED_PROVIDER_u-boot = "u-boot"
UBOOT_MACHINE = "myarm_bmc_defconfig"

# Flash layout (adjust for your platform)
FLASH_SIZE = "65536" # 64MB
FLASH_UBOOT_OFFSET = "0"
FLASH_KERNEL_OFFSET = "1024"
FLASH_ROFS_OFFSET = "10240"
FLASH_RWFS_OFFSET = "49152"

# Console configuration
SERIAL_CONSOLES = "115200;ttyAMA0"

# Machine features
MACHINE_FEATURES = "efi"

# OpenBMC features - customize for your platform
OBMC_MACHINE_FEATURES += "\
    obmc-bmc-state-mgmt \
    obmc-phosphor-fan-mgmt \
    obmc-phosphor-flash-mgmt \
"

# Entity Manager for inventory
VIRTUAL-RUNTIME_obmc-inventory-manager = "entity-manager"

```

Key ARM Machine Variables

Variable	Description	Example
<code>DEFAULTTUNE</code>	CPU tuning	<code>cortexa53</code> , <code>cortexa72</code>
<code>KERNEL_IMAGETYPE</code>	Kernel format	<code>Image</code> (ARM64), <code>zImage</code> (ARM32)
<code>SERIAL_CONSOLES</code>	Console device	<code>115200;ttyAMA0</code>
<code>MACHINE_FEATURES</code>	Platform features	<code>efi</code> , <code>optee</code>

NVIDIA OpenBMC Reference

NVIDIA's OpenBMC fork provides a production-quality reference for ARM platforms.

Repository Structure

```
NVIDIA/openbmc (develop branch)
├─ meta-nvidia/
│   └─ conf/
│       └─ layer.conf                # Layer configuration
│   └─ meta-common/                 # Shared components
│   └─ meta-prime/                   # Server platform support
│       └─ meta-graceblackwell/      # Grace Blackwell
│           └─ meta-gb200nv1/        # GB200 NVL machine
│               └─ meta-bmc/
│                   └─ conf/machine/
│                       └─ gb200nv1-bmc.conf
│                           └─ gb200nv1-bmc-ut3.conf
├─ recipes-phosphor/                # OpenBMC customizations
└─ recipes-nvidia/                  # NVIDIA-specific recipes
```

Reference: [meta-nvidia/conf/layer.conf](https://github.com/NVIDIA/openbmc/blob/develop/meta-nvidia/conf/layer.conf)

NVIDIA Layer Configuration

The NVIDIA layer configuration demonstrates vendor layer patterns:

```
# Reference: meta-nvidia/conf/layer.conf
# https://github.com/NVIDIA/openbmc/blob/develop/meta-nvidia/conf/layer.conf

# Layer name and compatibility
BBFILE_COLLECTIONS += "nvidia-layer"
LAYERSERIES_COMPAT_nvidia-layer = "whinlatter walnascar"

# Recipe patterns
BBFILES += "${LAYERDIR}/recipes-*/*/*.bb \
            ${LAYERDIR}/recipes-*/*/*.bbappend"
```

Key patterns (NVIDIA-specific):

- Uses nested meta-* layers for platform hierarchy
- Separate layers for BMC, HMC (Hardware Management Controller), PMC (Power Management Controller)
- Repository mirroring for internal GitLab → GitHub

Generic ARM patterns:

- Standard layer.conf structure
- Recipe organization by function (recipes-phosphor, recipes-kernel, etc.)

NVIDIA Machine Configuration Example

From [meta-nvidia/meta-prime/meta-graceblackwell/meta-gb200nv1/meta-bmc/conf/machine/gb200nv1-bmc.conf](https://github.com/NVIDIA/openbmc/blob/develop/meta-nvidia/meta-prime/meta-graceblackwell/meta-gb200nv1/meta-bmc/conf/machine/gb200nv1-bmc.conf):

```

# Reference: gb200nvl-bmc.conf
# NVIDIA GB200 NVL BMC - uses ASPEED AST2600 as BMC SoC

# Device tree
KERNEL_DEVICETREE = "aspeed-bmc-nvidia-gb200nvl-bmc.dtb"

# U-Boot
UBOOT_MACHINE = "ast2600_openbmc_spl_defconfig"
SPL_BINARY = "spl/u-boot-spl.bin"

# Flash layout (64MB)
FLASH_SIZE = "65536"
FLASH_UBOOT_OFFSET:flash-65536 = "0"
FLASH_KERNEL_OFFSET:flash-65536 = "1024"
FLASH_ROFS_OFFSET:flash-65536 = "10240"

# Console
SERIAL_CONSOLES = "115200;ttyS4"

# Use Entity Manager
VIRTUAL-RUNTIME_obmc-inventory-manager = "entity-manager"

# Feature selection
OBMC_IMAGE_EXTRA_INSTALL:remove = "phosphor-snmp"

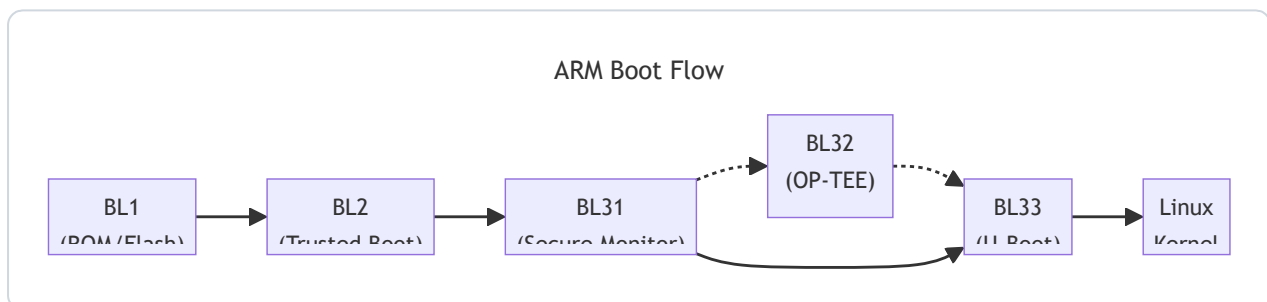
```

Note: The GB200 NVL uses an ASPEED AST2600 as the BMC SoC managing an ARM Grace Hopper host. This is a hybrid architecture.

ARM Boot Flow

ARM platforms use a standardized boot flow with Trusted Firmware-A (TF-A).

Boot Sequence



Legend:

- **BL1** - Boot Loader Stage 1 (ROM or first-stage flash)
- **BL2** - Trusted Boot Firmware
- **BL31** - EL3 Runtime Firmware (Secure Monitor)
- **BL32** - Secure Payload (OP-TEE, optional)
- **BL33** - Non-secure Bootloader (U-Boot)

► ASCII-art version (for comparison)

Boot Components

Stage	Component	Description
BL1	ROM/Flash	First boot code, initializes BL2
BL2	TF-A	Trusted boot, loads BL31/BL32/BL33
BL31	TF-A Runtime	Secure Monitor, handles SMC calls
BL32	OP-TEE	Trusted execution environment (optional)
BL33	U-Boot	Standard bootloader, loads Linux

TF-A Configuration

```
# recipes-bsp/trusted-firmware-a/trusted-firmware-a_%.bbappend

# Platform-specific TF-A configuration
TFA_PLATFORM = "myarm"
TFA_BUILD_TARGET = "bl31"

# Optional: Enable secure boot
TFA_MBEDTLS_DIR = "${STAGING_DIR_HOST}${libdir}"
TFA_ARM_ARCH = "8.0"
```

U-Boot for ARM Platforms

```
# recipes-bsp/u-boot/u-boot_%.bbappend

# ARM-specific U-Boot configuration
UBOOT_MACHINE = "myarm_bmc_defconfig"

# Enable FIT images for secure boot
UBOOT_MKIMAGE_DTCOPTS = "-I dts -O dtb -p 2000"
UBOOT_SIGN_ENABLE = "1"
```

ARM Device Tree

ARM platforms use standard Device Tree conventions with some BMC-specific patterns.

Device Tree Structure

```
// myarm-bmc.dts
/dts-v1/;

#include "arm-soc-base.dtsi"

/ {
    model = "My ARM BMC Platform";
    compatible = "vendor,myarm-bmc", "arm,cortex-a53";

    chosen {
        stdout-path = "serial0:115200n8";
    };

    aliases {
        serial0 = &uart0;
        i2c0 = &i2c0;
    };
};

// I2C bus for sensor/FRU access
&i2c0 {
    status = "okay";
    clock-frequency = <100000>;

    // Temperature sensor
    tmp75@48 {
        compatible = "ti,tmp75";
        reg = <0x48>;
    };

    // FRU EEPROM
    eeprom@50 {
        compatible = "atmel,24c64";
        reg = <0x50>;
    };
};

// GPIO for platform control
&gpio0 {
    status = "okay";

    power-button {
        gpios = <&gpio0 10 GPIO_ACTIVE_LOW>;
        linux,code = <KEY_POWER>;
    };

    reset-button {
        gpios = <&gpio0 11 GPIO_ACTIVE_LOW>;
        linux,code = <KEY_RESTART>;
    };
};
```

Common ARM Bindings

Peripheral	Binding	Example
UART	<code>arm,pl011</code>	Console, host UART
I2C	<code>arm,versatile-i2c</code>	Sensors, EEPROMs
SPI	<code>arm,pl022</code>	Flash access
GPIO	<code>arm,pl061</code>	Power control
Watchdog	<code>arm,sp805</code>	System watchdog

MCTP/PLDM Integration

ARM platforms often use MCTP/PLDM for host-BMC communication.

See the [MCTP/PLDM Guide](#) for detailed information.

Common Configuration

```
// Entity Manager MCTP endpoint configuration
{
  "Exposes": [
    {
      "Name": "Host MCTP Endpoint",
      "Type": "MCTPEndpoint",
      "Bus": 1,
      "Address": "0x08",
      "MessageTypes": ["PLDM", "SPDM"]
    }
  ]
}
```

Example ARM Machine Layer

Working examples are available in the [examples/porting/meta-arm-example](#) directory:

- `conf/machine/` - ARM-specific machine configuration template
- `recipes-kernel/` - Kernel customization for ARM platforms
- `README.md` - ARM platform porting quick start

Troubleshooting

Common Issues

Issue	Cause	Solution
No console output	Wrong UART device	Check <code>SERIAL_CONSOLES</code> in machine.conf
Kernel panic on boot	Missing DT bindings	Verify device tree compatible strings
I2C devices not found	Bus numbering mismatch	Check DT aliases and bus assignments
TF-A build fails	Missing platform files	Implement platform-specific TF-A port

Debug Tips

```
# Check boot messages
dmesg | grep -i arm

# Verify device tree
cat /proc/device-tree/model

# List I2C buses
i2cdetect -l

# Check MCTP endpoints
busctl tree xyz.openbmc_project.MCTP
```

References

- [NVIDIA OpenBMC Repository](#)
- [meta-arm Layer](#)
- [Trusted Firmware-A Documentation](#)
- [ARM Architecture Reference](#)
- [OpenBMC Porting Reference](#)
- [MCTP/PLDM Guide](#)
- [Entity Manager Guide](#)

Tested on: Reference documentation only - requires platform-specific hardware for validation.

Entity Manager Advanced Configuration

Master dynamic hardware configuration with advanced probing, FRU integration, and real-world patterns.

Table of Contents

1. [Overview](#)
 1. [When to Use This Guide](#)
 2. [Advanced Probe Patterns](#)
 1. [Regex Probes](#)
 2. [Multi-Condition Probes \(AND\)](#)
 3. [OR Condition Probes](#)
 4. [GPIO-Based Hardware Detection](#)
 5. [I2C Address Probes with Wildcards](#)
 3. [FruDevice Integration](#)
 1. [FruDevice Role](#)
 2. [FRU EEPROM Structure](#)
 3. [FruDevice Probe Configuration](#)
 4. [Referencing FRU Fields in Exposes](#)
 4. [Inventory Manager vs Entity Manager](#)
 1. [Comparison Table](#)
 2. [Decision Criteria](#)
 3. [Migration Path: Static to Dynamic](#)
 5. [Real-World Configuration Examples](#)
 1. [Temperature Sensor Configuration](#)
 2. [Fan PWM/Tach Configuration](#)
 3. [Voltage Regulator Configuration](#)
 6. [Debugging and Troubleshooting](#)
 1. [Entity Manager Journal Logs](#)
 2. [busctl Commands for Inspection](#)
 3. [Common Issues and Solutions](#)
 7. [Next Steps](#)
-

Overview

This guide covers advanced Entity Manager topics for complex hardware configurations. For basic Entity Manager concepts, see the [Entity Manager Guide](#).

Entity Manager dynamically discovers hardware using JSON configuration files with “probes” that match system conditions. When probes match, Entity Manager exposes D-Bus objects that other services (sensors, fan control) use.

When to Use This Guide

Scenario	This Guide Helps
Complex probe conditions (regex, multi-field)	✓
FRU EEPROM integration	✓
Migrating from static to dynamic config	✓
Debugging Entity Manager issues	✓
Basic Entity Manager setup	Use basic guide

Advanced Probe Patterns

Probes determine when a configuration applies. Advanced patterns allow precise hardware matching.

Regex Probes

Use regex patterns to match variable hardware identifiers:

```
{
  "Exposes": [...],
  "Name": "Motherboard Sensors",
  "Probe": "xyz.openbmc_project.FruDevice({'PRODUCT_PRODUCT_NAME': '.*Motherboard.*'})"
}
```

Regex syntax:

Pattern	Matches
<code>.*</code>	Any characters (wildcard)
<code>^ABC</code>	Starts with "ABC"
<code>XYZ\$</code>	Ends with "XYZ"
<code>V[0-9]+</code>	"V" followed by digits
<code>(A B)</code>	Either "A" or "B"

Example: Match multiple product variants:

```
{
  "Probe": "xyz.openbmc_project.FruDevice({'PRODUCT_PRODUCT_NAME': '^((Server-2U|Server-4U).*)$'})"
}
```

Multi-Condition Probes (AND)

Match multiple conditions simultaneously using array syntax:

```
{
  "Probe": [
    "xyz.openbmc_project.FruDevice({'PRODUCT_MANUFACTURER': 'ACME'})",
    "xyz.openbmc_project.FruDevice({'PRODUCT_PRODUCT_NAME': 'PowerBoard'})"
  ]
}
```

All conditions must match (logical AND). This ensures the configuration only applies when both manufacturer AND product name match.

OR Condition Probes

For OR logic, use the `OR` keyword:

```
{
  "Probe": "OR",
  "ProbeValue": [
    "xyz.openbmc_project.FruDevice({'PRODUCT_PRODUCT_NAME': 'BoardTypeA'})",
    "xyz.openbmc_project.FruDevice({'PRODUCT_PRODUCT_NAME': 'BoardTypeB'})"
  ]
}
```

GPIO-Based Hardware Detection

Detect hardware presence using GPIO pins:

```
{
  "Probe": "xyz.openbmc_project.Inventory.Decorator.Asset({'Name': 'Riser Card'})",
  "Name": "Riser Card Sensors",
  "Exposes": [
    {
      "Name": "Riser_Temp",
      "Type": "TMP75",
      "Bus": 6,
      "Address": "0x48"
    }
  ]
}
```

For GPIO presence detection, use the GpioPresence daemon which creates inventory objects based on GPIO state:

```
{
  "Name": "GPU Presence",
  "Type": "GpioPresence",
  "Pin": "GPU_PRSENT_N",
  "ActiveLow": true
}
```

I2C Address Probes with Wildcards

Match devices across multiple I2C buses:

```
{
  "Probe": "xyz.openbmc_project.FruDevice({'BUS': '.*', 'ADDRESS': '0x50'})"
}
```

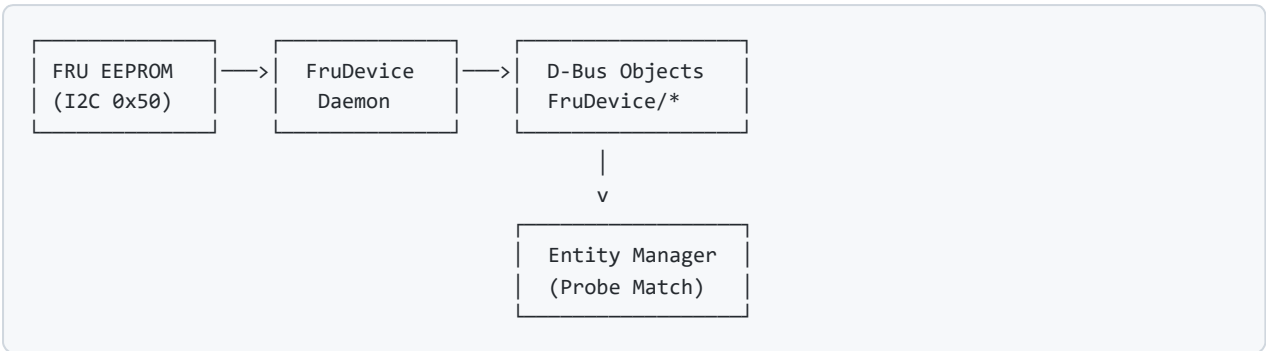
Specific bus range:

```
{
  "Probe": "xyz.openbmc_project.FruDevice({'BUS': '[2-5]', 'ADDRESS': '0x50'})"
}
```

FruDevice Integration

FruDevice daemon scans I2C buses for FRU EEPROMs and exposes their contents via D-Bus.

FruDevice Role



FruDevice scans configured I2C buses at boot and exposes discovered FRU data as D-Bus objects that Entity Manager can probe against.

FRU EEPROM Structure

Standard IPMI FRU format (IPMI Platform Management FRU Information Storage Definition):

Area	Common Fields
Board Info	BOARD_MANUFACTURER, BOARD_PRODUCT_NAME, BOARD_SERIAL_NUMBER, BOARD_PART_NUMBER
Product Info	PRODUCT_MANUFACTURER, PRODUCT_PRODUCT_NAME, PRODUCT_PART_NUMBER, PRODUCT_VERSION, PRODUCT_SERIAL_NUMBER
Chassis Info	CHASSIS_TYPE, CHASSIS_PART_NUMBER, CHASSIS_SERIAL_NUMBER

FruDevice Probe Configuration

Configure FruDevice to scan specific I2C buses in your machine layer:

```
{
  "Exposes": [
    {
      "Name": "Motherboard FRU",
      "Type": "EEPROM",
      "Bus": 2,
      "Address": "0x50"
    }
  ],
  "Name": "Baseboard FRU Config",
  "Probe": "TRUE"
}
```

Referencing FRU Fields in Exposes

Use `$` syntax to reference FRU fields dynamically:

```
{
  "Probe": "xyz.openbmc_project.FruDevice({'PRODUCT_PRODUCT_NAME': 'PowerModule'})",
  "Name": "$PRODUCT_SERIAL_NUMBER Power Module",
  "Exposes": [
    {
      "Name": "$BOARD_PRODUCT_NAME Voltage",
      "Type": "ADC",
      "Index": 0,
      "ScaleFactor": 0.001
    }
  ]
}
```

The `$(FIELD_NAME)` syntax substitutes the actual FRU field value at runtime, enabling unique naming for multiple identical modules.

Inventory Manager vs Entity Manager

Choose the right approach for your platform:

Comparison Table

Aspect	Inventory Manager (Static)	Entity Manager (Dynamic)
Configuration	YAML files at build time	JSON files with runtime probes
Hardware Discovery	None - predefined	FRU EEPROM, GPIO, I2C scan
Hot-plug Support	No	Yes
Configuration Complexity	Simple	More complex
Flexibility	Low	High
Debugging	Easy - static	Requires understanding probes
Best For	Fixed hardware, prototypes	Production with variants

Decision Criteria

Use Inventory Manager (Static) when:

- Hardware configuration is fixed and known at build time
- Rapid prototyping or proof-of-concept
- Simple platforms without FRU EEPROMs
- Legacy platforms being maintained

Use Entity Manager (Dynamic) when:

- Multiple hardware variants share one image
- Hot-pluggable components (risers, drives)
- FRU EEPROMs identify components
- Production platforms with SKU variants

Migration Path: Static to Dynamic

1. **Identify static configurations** in your YAML inventory files
 2. **Document hardware variants** and how they differ
 3. **Add FRU EEPROMs** to boards (or use existing ones)
 4. **Create Entity Manager JSON** with probes matching FRU data
 5. **Test each variant** to verify correct probe matching
 6. **Remove static YAML** configurations
 7. **Update recipes** to install JSON configs instead of YAML
-

Real-World Configuration Examples

Temperature Sensor Configuration

Configure hwmon temperature sensors discovered via Entity Manager:

```

{
  "Exposes": [
    {
      "Address": "0x48",
      "Bus": 6,
      "Name": "CPU0 Temp",
      "Thresholds": [
        {
          "Direction": "greater than",
          "Name": "upper critical",
          "Severity": 1,
          "Value": 95
        },
        {
          "Direction": "greater than",
          "Name": "upper non critical",
          "Severity": 0,
          "Value": 85
        }
      ],
      "Type": "TMP75"
    },
    {
      "Address": "0x49",
      "Bus": 6,
      "Name": "CPU1 Temp",
      "Thresholds": [
        {
          "Direction": "greater than",
          "Name": "upper critical",
          "Severity": 1,
          "Value": 95
        },
        {
          "Direction": "greater than",
          "Name": "upper non critical",
          "Severity": 0,
          "Value": 85
        }
      ],
      "Type": "TMP75"
    }
  ],
  "Name": "CPU Temperature Sensors",
  "Probe": "xyz.openbmc_project.FruDevice({'PRODUCT_PRODUCT_NAME': '.*Motherboard.*'})"
}

```

Fan PWM/Tach Configuration

Configure fan control with PWM output and tachometer input:

```

{
  "Exposes": [
    {
      "Connector": {
        "Name": "System Fan 1",
        "Pwm": 0,
        "Tachs": [0]
      },
      "Name": "Fan 1",
      "Type": "AspeedFan"
    },
    {
      "Connector": {
        "Name": "System Fan 2",
        "Pwm": 1,
        "Tachs": [1]
      },
      "Name": "Fan 2",
      "Type": "AspeedFan"
    },
    {
      "Connector": {
        "Name": "System Fan 3",
        "Pwm": 2,
        "Tachs": [2, 3]
      },
      "Name": "Fan 3",
      "Type": "AspeedFan"
    }
  ],
  "Name": "Chassis Fans",
  "Probe": "xyz.openbmc_project.FruDevice({'PRODUCT_PRODUCT_NAME': 'Chassis'})"
}

```

Note: Dual tach configuration (`"Tachs": [2, 3]`) is for fans with redundant tachometer sensors.

Voltage Regulator Configuration

Configure voltage monitoring via ADC or PMBus:

```

{
  "Exposes": [
    {
      "Address": "0x40",
      "Bus": 3,
      "Name": "P12V",
      "Type": "ADM1275",
      "Thresholds": [
        {
          "Direction": "greater than",
          "Name": "upper critical",
          "Severity": 1,
          "Value": 13.2
        },
        {
          "Direction": "less than",
          "Name": "lower critical",
          "Severity": 1,
          "Value": 10.8
        }
      ]
    },
    {
      "Address": "0x41",
      "Bus": 3,
      "Name": "P3V3",
      "Type": "ADM1275",
      "Thresholds": [
        {
          "Direction": "greater than",
          "Name": "upper critical",
          "Severity": 1,
          "Value": 3.6
        },
        {
          "Direction": "less than",
          "Name": "lower critical",
          "Severity": 1,
          "Value": 3.0
        }
      ]
    }
  ],
  "Name": "Power Regulators",
  "Probe": "xyz.openbmc_project.FruDevice({'PRODUCT_PRODUCT_NAME': 'PowerBoard'})"
}

```

Debugging and Troubleshooting

Entity Manager Journal Logs

View Entity Manager activity:


```
# Follow Entity Manager logs
journalctl -f -u xyz.openbmc_project.EntityManager

# View recent Entity Manager activity
journalctl -u xyz.openbmc_project.EntityManager --since "5 minutes ago"

# Check for probe matching
journalctl -u xyz.openbmc_project.EntityManager | grep -i "probe"
```

Common log patterns:

Log Message	Meaning
Probe matched	Configuration activated
Probe failed	Conditions not met
Exposing object	D-Bus object created
Failed to find	Referenced object missing

busctl Commands for Inspection

Inspect Entity Manager exposed objects:

```
# List all Entity Manager objects
busctl tree xyz.openbmc_project.EntityManager

# View specific object properties
busctl introspect xyz.openbmc_project.EntityManager \
    /xyz/openbmc_project/inventory/system/board/Motherboard

# Get a specific property
busctl get-property xyz.openbmc_project.EntityManager \
    /xyz/openbmc_project/inventory/system/board/Motherboard \
    xyz.openbmc_project.Inventory.Decorator.Asset Name
```

Inspect FruDevice objects:

```
# List all discovered FRU devices
busctl tree xyz.openbmc_project.FruDevice

# View FRU data
busctl introspect xyz.openbmc_project.FruDevice \
    /xyz/openbmc_project/FruDevice/Motherboard_Fru
```

Common Issues and Solutions

Issue	Cause	Solution
Probe never matches	FRU field name mismatch	Check exact field names with <code>busctl introspect</code> on FruDevice
Probe matches wrong device	Too broad regex	Make regex more specific or add AND conditions
Sensors not appearing	hwmon driver not loaded	Check <code>dmesg</code> for I2C errors, verify device tree
Duplicate objects	Multiple configs match	Add discriminating probe conditions
Objects disappear after reboot	FruDevice scan timing	Ensure FruDevice starts before Entity Manager
\$FIELD substitution empty	Field doesn't exist in FRU	Verify FRU contents with <code>busctl</code>

Debug workflow:

1. Verify FRU detection:

```
busctl tree xyz.openbmc_project.FruDevice
```

2. Check FRU field names:

```
busctl introspect xyz.openbmc_project.FruDevice /xyz/openbmc_project/FruDevice/<name>
```

3. Test probe manually:

```
# Check if your probe conditions exist
busctl call xyz.openbmc_project.ObjectMapper \
  /xyz/openbmc_project/object_mapper \
  xyz.openbmc_project.ObjectMapper GetSubTree \
  sias "/" 0 1 "xyz.openbmc_project.FruDevice"
```

4. Review Entity Manager logs:

```
journalctl -u xyz.openbmc_project.EntityManager -n 100
```

Next Steps

- [Machine Layer](#) - Create your platform layer
- [Verification](#) - Test your configuration
- [D-Bus Sensors Guide](#) - Sensor daemon configuration

Tested on: OpenBMC master branch with AST2600-EVB

Flash Layout & Optimization Guide

Understand OpenBMC flash partitioning, image construction, and techniques for reducing firmware image size.

Table of Contents

1. [Overview](#)
2. [Flash Layout Diagrams](#)
 1. [Static MTD Layout \(32MB\)](#)
 2. [Static MTD Layout \(64MB\)](#)
 3. [UBI Layout \(32MB\)](#)
 4. [UBI Layout \(64MB\)](#)
 5. [Static vs UBI Comparison](#)
3. [Image Construction Process](#)
 1. [image_types_phosphor.bbclass Walkthrough](#)
 1. [Key Configuration Variables](#)
 2. [Machine Configuration Example](#)
 2. [Assembly Process](#)
 1. [Stage 1: Build Individual Components](#)
 2. [Stage 2: Assemble Final Image](#)
 3. [Stage 3: Output Artifacts](#)
 3. [Partition Size Calculations](#)
 4. [Verifying Image Sizes](#)
4. [Image Size Reduction Techniques](#)
 1. [Size Reduction Checklist](#)
 2. [Technique 1: Remove Unused Packages](#)
 1. [Package Size Reference](#)
 3. [Technique 2: Disable IMAGE_FEATURES](#)
 4. [Technique 3: Kernel Configuration Trimming](#)
 1. [Create a Kernel Config Fragment](#)
 2. [Apply the Fragment](#)
 5. [Technique 4: Filesystem Compression](#)
 1. [Compression Options](#)
 6. [Technique 5: Strip Debug Symbols](#)
 7. [Technique 6: Minimize WebUI and Static Assets](#)
 8. [Size Reduction Summary](#)
5. [A/B Update Partitioning](#)
 1. [A/B Flash Layout \(32MB\)](#)
 2. [A/B Flash Layout \(64MB\)](#)
 3. [U-Boot Environment Variables for A/B Boot](#)

4. [Boot Selection Logic](#)
 5. [Rollback Flow](#)
 6. [Configuring A/B in Your Machine Layer](#)
 6. [eMMC vs NOR Flash Comparison](#)
 1. [Comparison Table](#)
 2. [NOR Flash Filesystem Layout](#)
 3. [eMMC Filesystem Layout](#)
 4. [When to Use Each](#)
 7. [Code Examples](#)
 1. [Example 1: Image Size Analysis Script](#)
 2. [Example 2: Custom Flash Layout Configuration](#)
 3. [Example 3: Verifying Flash Layout on Target](#)
 8. [Troubleshooting](#)
 1. [Issue: Image Exceeds Flash Partition Size](#)
 2. [Issue: JFFS2 Mount Fails After Flash Update](#)
 3. [Issue: A/B Rollback Loop](#)
 4. [Issue: UBI Volume Attach Fails](#)
 5. [Debug Commands](#)
 9. [References](#)
 1. [Official Resources](#)
 2. [Related Guides](#)
 3. [External Documentation](#)
-

Overview

Every OpenBMC image must fit inside the SPI NOR flash attached to the BMC SoC. The flash stores U-Boot, the Linux kernel (packaged as a FIT image), a read-only root filesystem (squashfs), and a read-write data partition. Understanding how these pieces are arranged and assembled is essential for porting to new hardware, reducing image size, and enabling reliable firmware updates.

This guide walks you through the complete flash lifecycle: how the Yocto build system constructs the final firmware image, how the partitions are laid out on flash, and how to shrink or restructure the image for your platform. You also learn how to configure A/B dual-image updates for rollback-safe firmware deployment.

Key concepts covered:

- Static MTD and UBI flash layouts for 32MB and 64MB configurations
- Image construction via `image_types_phosphor.bbclass`
- Image size reduction checklist (packages, kernel, compression)
- A/B dual-image partitioning and rollback flow
- eMMC vs NOR flash storage comparison

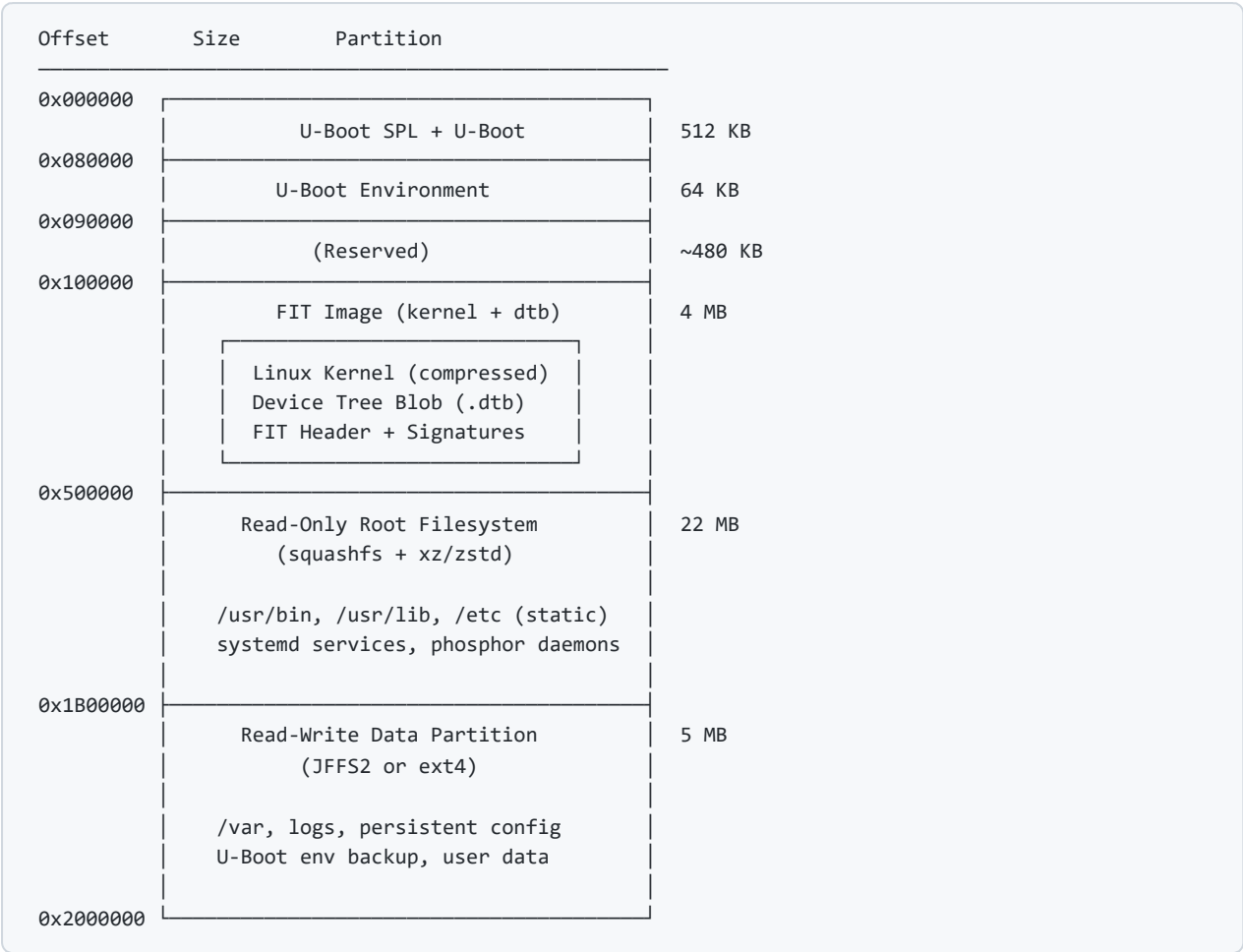
Full validation of flash layout changes requires real hardware. The architectural concepts in this guide are testable conceptually, but partition offsets and boot behavior must be verified on your target BMC platform.

Flash Layout Diagrams

OpenBMC supports two primary flash layouts: **static MTD** (flat partitions on raw flash) and **UBI** (managed volumes with wear leveling). The choice depends on your platform requirements and flash size.

Static MTD Layout (32MB)

The static layout divides the flash into fixed partitions at compile-time offsets. This is the simpler and more common approach for 32MB (0x2000000) SPI NOR flash.



Static MTD Layout (64MB)

Platforms with 64MB (0x4000000) flash gain more space for the root filesystem and read-write data. This is common on AST2600-based systems.

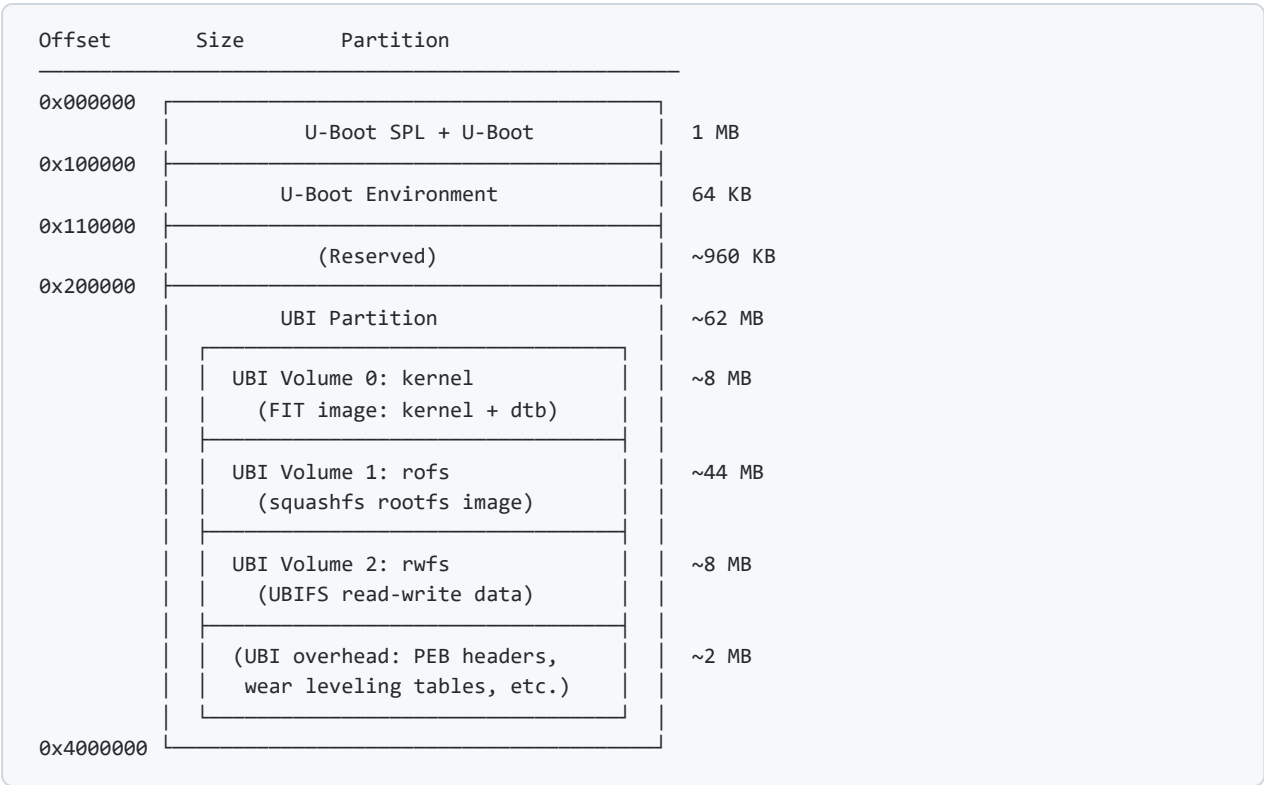
Offset	Size	Partition
0x000000		
		U-Boot SPL + U-Boot
0x100000	1 MB	
		U-Boot Environment
0x110000	64 KB	
		(Reserved)
0x200000	~960 KB	
		FIT Image (kernel + dtb)
	8 MB	
		Linux Kernel (compressed)
		Device Tree Blob (.dtb)
		FIT Header + Signatures
0xA00000		
		Read-Only Root Filesystem (squashfs + xz/zstd)
	44 MB	
		Full phosphor stack, debug tools, WebUI assets, additional daemons
0x3600000		
		Read-Write Data Partition (JFFS2 or ext4)
	10 MB	
		/var, logs, persistent config, crash dumps, extended user data
0x4000000		

UBI Layout (32MB)

UBI (Unsorted Block Images) provides wear leveling and dynamic volume sizing on top of raw NAND or NOR flash. OpenBMC uses UBI on platforms that require better flash endurance.

Offset	Size	Partition
0x000000		
		U-Boot SPL + U-Boot
0x080000	512 KB	
		U-Boot Environment
0x090000	64 KB	
		(Reserved)
0x100000	~480 KB	
		UBI Partition
	~31 MB	
		UBI Volume 0: kernel (FIT image: kernel + dtb)
	~4 MB	
		UBI Volume 1: rofs (squashfs rootfs image)
	~22 MB	
		UBI Volume 2: rwfs (UBIFS read-write data)
	~4 MB	
		(UBI overhead: PEB headers, wear leveling tables, etc.)
	~1 MB	
0x2000000		

UBI Layout (64MB)



Static vs UBI Comparison

Aspect	Static MTD	UBI
Wear leveling	None (fixed erase blocks)	Built-in across all PEBs
Partition sizing	Fixed at compile time	Dynamic volume allocation
Read-write FS	JFFS2 (slow mount)	UBIFS (faster mount)
Flash overhead	None	~2-5% for UBI metadata
Update method	Write raw MTD partitions	Update UBI volumes
Complexity	Simple, well understood	More complex, better endurance
Typical use	SPI NOR, short lifecycle	SPI NOR/NAND, long lifecycle

Use static MTD for platforms where simplicity matters and flash writes are infrequent. Use UBI when the read-write partition receives heavy writes (logging, configuration changes) and flash endurance is a concern.

Image Construction Process

The Yocto build system assembles the final firmware image through the `image_types_phosphor.bbclass` class. This section explains how each component is built and combined into the final flashable image.

image_types_phosphor.bbclass Walkthrough

The `image_types_phosphor.bbclass` file in `meta-phosphor` defines the image assembly logic. It reads partition offset variables from your machine configuration and combines individual components into a single binary.

Key Configuration Variables

Set these variables in your machine configuration file (`conf/machine/<machine>.conf`):

Variable	Unit	Description	32MB Example	64MB Example
<code>FLASH_SIZE</code>	KB	Total flash size	<code>32768</code>	<code>65536</code>
<code>FLASH_UBOOT_OFFSET</code>	KB	U-Boot start offset	<code>0</code>	<code>0</code>
<code>FLASH_KERNEL_OFFSET</code>	KB	FIT image start offset	<code>1024</code>	<code>2048</code>
<code>FLASH_ROFS_OFFSET</code>	KB	Root filesystem start	<code>5120</code>	<code>10240</code>
<code>FLASH_RWFS_OFFSET</code>	KB	Read-write partition start	<code>27648</code>	<code>55296</code>

All offset values are in **kilobytes**, not bytes. The build system converts them to byte offsets internally. For example, `FLASH_KERNEL_OFFSET = "1024"` corresponds to byte offset `0x100000`.

Machine Configuration Example

```
# conf/machine/myboard.conf

# Flash layout for 32MB SPI NOR
FLASH_SIZE = "32768"

# Partition offsets (in KB)
FLASH_UBOOT_OFFSET = "0"
FLASH_KERNEL_OFFSET = "1024"
FLASH_ROFS_OFFSET = "5120"
FLASH_RWFS_OFFSET = "27648"

# Image type selection
OBMC_IMAGE_TYPE = "static" # or "ubi"
```

For a 64MB platform:

```
# conf/machine/myboard-64.conf

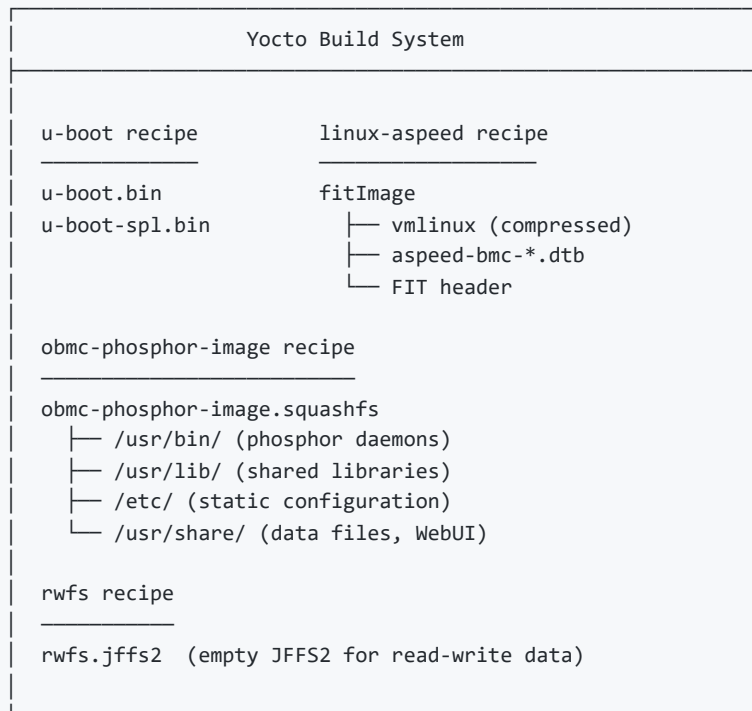
# Flash layout for 64MB SPI NOR
FLASH_SIZE = "65536"

# Partition offsets (in KB)
FLASH_UBOOT_OFFSET = "0"
FLASH_KERNEL_OFFSET = "2048"
FLASH_ROFS_OFFSET = "10240"
FLASH_RWFS_OFFSET = "55296"
```

Assembly Process

The build system constructs the final image through a multi-stage pipeline. Each component is built independently, then combined into the final flashable binary.

Stage 1: Build Individual Components



Stage 2: Assemble Final Image

The `image_types_phosphor.bbclass` combines all components using `dd` commands with calculated offsets:

```
# Simplified assembly logic from image_types_phosphor.bbclass

# Create empty flash image (all 0xFF, matching erased NOR flash)
dd if=/dev/zero bs=1k count=${FLASH_SIZE} | tr '\000' '\377' > flash.img

# Write U-Boot at offset 0
dd if=u-boot-spl.bin of=flash.img bs=1k seek=${FLASH_UBOOT_OFFSET} conv=notrunc

# Write FIT image (kernel + dtb) at kernel offset
dd if=fitImage of=flash.img bs=1k seek=${FLASH_KERNEL_OFFSET} conv=notrunc

# Write squashfs root filesystem at rofs offset
dd if=obmc-phosphor-image.squashfs of=flash.img bs=1k \
  seek=${FLASH_ROFS_OFFSET} conv=notrunc

# Write empty JFFS2 read-write partition at rwfs offset
dd if=rwfs.jffs2 of=flash.img bs=1k seek=${FLASH_RWFS_OFFSET} conv=notrunc

# Result: flash.img is the complete .static.mtd image
```

The final output file is named `obmc-phosphor-image-<machine>.static.mtd` for static layouts or `obmc-phosphor-image-<machine>.ubi.mtd` for UBI layouts.

Stage 3: Output Artifacts

After the build completes, find the image artifacts in the deploy directory:

```
ls tmp/deploy/images/<machine>/

# Key output files:
# obmc-phosphor-image-<machine>.static.mtd - Complete flash image (static)
# obmc-phosphor-image-<machine>.ubi.mtd    - Complete flash image (UBI)
# fitImage-<machine>.bin                  - Standalone FIT image
# u-boot-spl.bin                          - U-Boot SPL binary
# u-boot.bin                              - U-Boot main binary
# obmc-phosphor-image-<machine>.squashfs   - Root filesystem
```

Partition Size Calculations

Calculate available space for each partition to verify your image fits:

```
Kernel space = FLASH_ROFS_OFFSET - FLASH_KERNEL_OFFSET
RootFS space = FLASH_RWFS_OFFSET - FLASH_ROFS_OFFSET
RWFS space   = FLASH_SIZE       - FLASH_RWFS_OFFSET
```

For a 32MB static layout:

```
Kernel space = 5120 - 1024 = 4096 KB (4 MB)
RootFS space = 27648 - 5120 = 22528 KB (22 MB)
RWFS space   = 32768 - 27648 = 5120 KB (5 MB)
```

If your squashfs rootfs exceeds the RootFS space, the build produces an image that overlaps partitions. The build system does not always catch this error. Always verify your image sizes after building.

Verifying Image Sizes

Run this check after every build:

```
# Check individual component sizes
ls -lh tmp/deploy/images/<machine>/fitImage-*.bin
ls -lh tmp/deploy/images/<machine>/obmc-phosphor-image-<machine>.squashfs

# Compare against partition limits
KERNEL_MAX=$((5120 - 1024)) # 4096 KB for 32MB layout
ROFS_MAX=$((27648 - 5120))  # 22528 KB for 32MB layout

KERNEL_SIZE=$(stat -c%s tmp/deploy/images/<machine>/fitImage-*.bin)
ROFS_SIZE=$(stat -c%s tmp/deploy/images/<machine>/obmc-phosphor-image-*.squashfs)

echo "Kernel: $((($KERNEL_SIZE / 1024)) KB / ${KERNEL_MAX} KB"
echo "RootFS: $((($ROFS_SIZE / 1024)) KB / ${ROFS_MAX} KB"
```

Image Size Reduction Techniques

When your image exceeds the available flash space, apply these techniques in order of impact and risk. Start with package removal, then move to kernel trimming and compression changes.

Size Reduction Checklist

Use this prioritized checklist when you need to shrink your image:

- ☐ Remove unused packages from `IMAGE_INSTALL`
- ☐ Disable unnecessary `IMAGE_FEATURES`
- ☐ Strip debug symbols from all binaries
- ☐ Trim kernel configuration (disable unused subsystems)
- ☐ Switch to more aggressive filesystem compression
- ☐ Remove unused kernel modules
- ☐ Minimize WebUI assets (or remove WebUI entirely)
- ☐ Remove documentation and man pages from rootfs

Technique 1: Remove Unused Packages

The most effective way to reduce image size is to remove packages you do not need. Use `IMAGE_INSTALL:remove` in your machine configuration or image recipe.

```
# In your machine layer: recipes-phosphor/images/obmc-phosphor-image.bbappend

# Remove packages not needed on your platform
IMAGE_INSTALL:remove = "\
    phosphor-snmp \
    phosphor-ipmi-ipmb \
    phosphor-host-postd \
    phosphor-post-code-manager \
    obmc-ikvm \
    phosphor-webui \
"

# Remove debug and development tools from production images
IMAGE_INSTALL:remove = "\
    gdb \
    strace \
    tcpdump \
    openssh-sftp-server \
    ipmitool \
"
```

Run `bitbake -g obmc-phosphor-image && cat recipe-depends.dot | grep IMAGE_INSTALL` to see all packages pulled into the image. Alternatively, inspect `tmp/deploy/images/<machine>/obmc-phosphor-image-<machine>.manifest` for an installed package list with sizes.

Package Size Reference

Common packages and their approximate installed sizes:

Package	Approx. Size	Notes
<code>phosphor-webui</code>	3-5 MB	Web interface assets
<code>obmc-ikvm</code>	1-2 MB	KVM-over-IP
<code>phosphor-snmp</code>	200-500 KB	SNMP agent
<code>gdb</code>	3-5 MB	GNU debugger
<code>strace</code>	500 KB	System call tracer
<code>openssh-sftp-server</code>	200 KB	SFTP subsystem
<code>tcpdump</code>	800 KB	Network packet capture
<code>ipmitool</code>	500 KB	IPMI command-line tool

Technique 2: Disable IMAGE_FEATURES

OpenBMC inherits several Yocto `IMAGE_FEATURES` that increase image size. Disable features you do not need:

```
# In your image recipe or local.conf

# Remove debug-related features
IMAGE_FEATURES:remove = "debug-tweaks"
IMAGE_FEATURES:remove = "tools-debug"
IMAGE_FEATURES:remove = "tools-profile"
IMAGE_FEATURES:remove = "dev-pkgs"
IMAGE_FEATURES:remove = "dbg-pkgs"

# If you do not need SSH access during development
# IMAGE_FEATURES:remove = "ssh-server-openssh"
```

Removing `debug-tweaks` disables passwordless root login. Make sure you have an alternative authentication method configured before deploying to hardware.

Technique 3: Kernel Configuration Trimming

The Linux kernel is the second-largest component in the FIT image. Trim unused subsystems to reduce kernel and module sizes.

Create a Kernel Config Fragment

```
# meta-<your-machine>/recipes-kernel/linux/linux-aspeed/<machine>-size.cfg

# ===== Disable Debug Options =====
# CONFIG_DEBUG_INFO is not set
# CONFIG_DEBUG_FS is not set
# CONFIG_DEBUG_KERNEL is not set
# CONFIG_DEBUG_BUGVERBOSE is not set
# CONFIG_SCHED_DEBUG is not set
# CONFIG_FTRACE is not set
# CONFIG_FUNCTION_TRACER is not set
# CONFIG_STACK_TRACER is not set

# ===== Disable Unused Filesystems =====
# CONFIG_EXT4_FS is not set
# CONFIG_BTRFS_FS is not set
# CONFIG_XFS_FS is not set
# CONFIG_NFS_FS is not set
# CONFIG_NFSD is not set
# CONFIG_CIFS is not set
# CONFIG_FAT_FS is not set

# ===== Disable Unused Drivers =====
# CONFIG_SOUND is not set
# CONFIG_USB_GADGET is not set
# CONFIG_WIRELESS is not set
# CONFIG_WLAN is not set
# CONFIG_BLUETOOTH is not set
# CONFIG_INPUT_MOUSEDEV is not set
# CONFIG_INPUT_JOYDEV is not set
# CONFIG_VT is not set

# ===== Disable Unused Network Protocols =====
# CONFIG_IPV6 is not set           (only if IPv6 not needed)
# CONFIG_NETFILTER is not set     (only if iptables not needed)
# CONFIG_BRIDGE is not set
# CONFIG_DECNET is not set
# CONFIG_ECONET is not set
```

Apply the Fragment

```
# meta-<your-machine>/recipes-kernel/linux/linux-aspeed_%.bbappend

FILESEXTRAPATHS:prepend := "${THISDIR}/linux-aspeed:"

SRC_URI += "file://<machine>-size.cfg"
```

After changing kernel configuration, rebuild and check the FIT image size. Disabling `CONFIG_DEBUG_INFO` alone can save 5-10 MB from the uncompressed kernel. The compressed FIT image savings are typically 1-3 MB.

Technique 4: Filesystem Compression

The root filesystem compression algorithm significantly affects the final image size. OpenBMC uses squashfs with configurable compression.

Compression Options

Compression	Ratio	Decompress Speed	CPU Usage	Image Size (typical)
<code>squashfs+gzip</code>	Good	Fast	Low	~20 MB
<code>squashfs+xz</code>	Best	Slower	High	~16 MB
<code>squashfs+zstd</code>	Very Good	Fast	Medium	~17 MB
<code>squashfs+lzo</code>	Fair	Fastest	Lowest	~22 MB

Configure compression in your machine layer:

```
# In your machine.conf or local.conf

# Use xz for maximum compression (best for 32MB flash)
IMAGE_FSTYPES = "squashfs-xz"
EXTRA_IMAGECMD:squashfs-xz = "-comp xz -Xbcj arm -b 256K"

# Use zstd for good compression with faster boot (recommended for 64MB flash)
IMAGE_FSTYPES = "squashfs-zstd"
EXTRA_IMAGECMD:squashfs-zstd = "-comp zstd -Xcompression-level 19 -b 256K"
```

Use `squashfs+xz` when you need to squeeze every byte for 32MB flash. Use `squashfs+zstd` when you have 64MB flash and prefer faster boot times. The `-Xbcj arm` option for xz enables ARM bytecode filtering, which improves compression of ARM binaries by 2-5%.

Technique 5: Strip Debug Symbols

Ensure all binaries are stripped in production images:

```
# In your local.conf or machine.conf

# Strip all installed binaries
INHIBIT_PACKAGE_STRIP = "0"
INHIBIT_PACKAGE_DEBUG_SPLIT = "1"

# Remove .debug directories from rootfs
PACKAGE_DEBUG_SPLIT_STYLE = "debug-without-src"
```

Technique 6: Minimize WebUI and Static Assets

The WebUI can consume several megabytes. If you use Redfish API exclusively, remove it:

```
# Remove WebUI entirely
IMAGE_INSTALL:remove = "phosphor-webui"

# Or, if keeping WebUI, ensure assets are compressed
# The WebUI recipe typically includes pre-built compressed assets
```

Size Reduction Summary

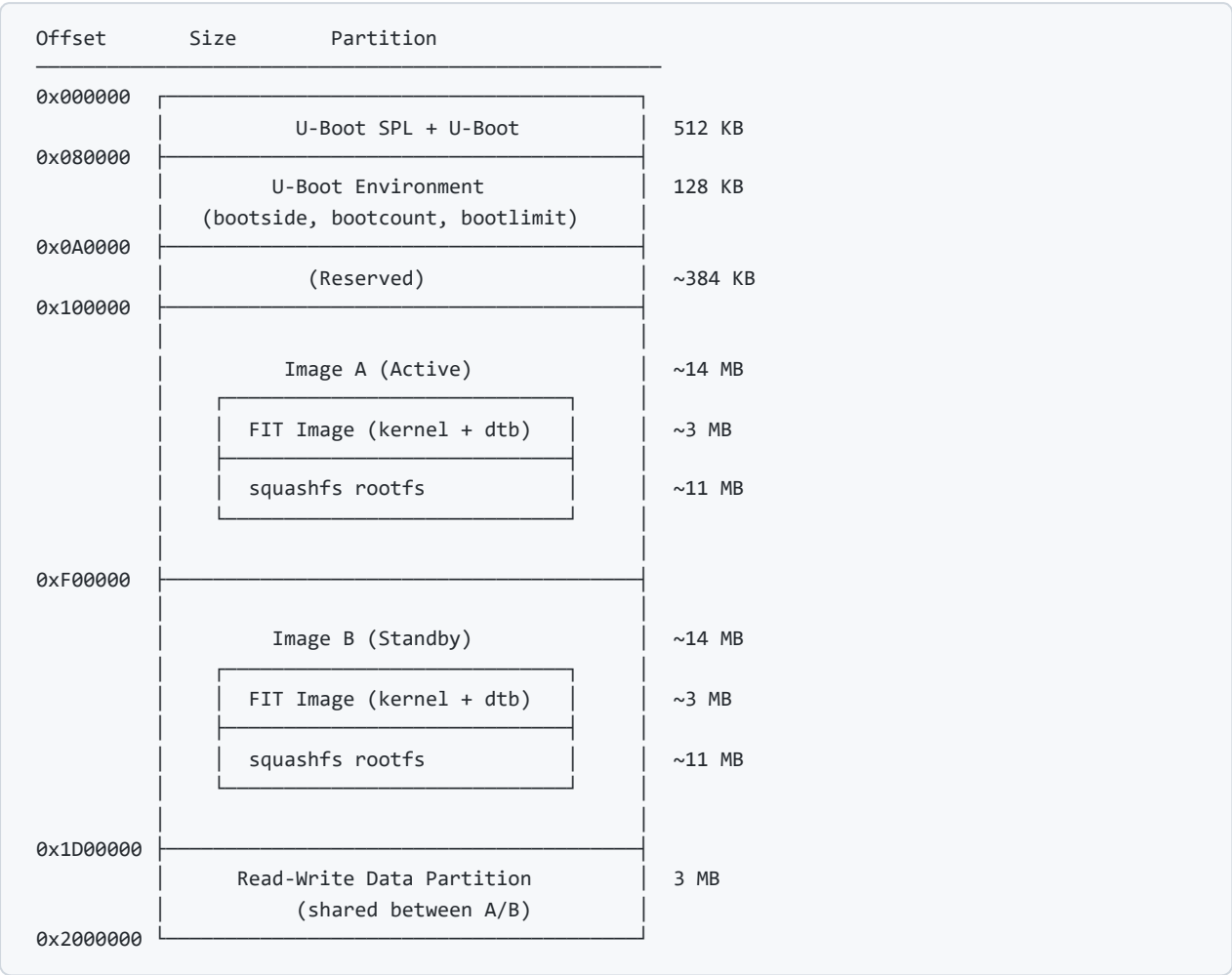
Expected savings from each technique (approximate, based on typical 32MB platform):

Technique	Typical Savings	Risk Level
Remove unused packages	2-10 MB	Low
Disable IMAGE_FEATURES	1-3 MB	Low
Kernel config trim	1-3 MB	Medium
squashfs+gzip to squashfs+xz	3-5 MB	Low
Strip debug symbols	1-2 MB	Low
Remove WebUI	3-5 MB	Low
Total possible savings	11-28 MB	

A/B Update Partitioning

A/B dual-image partitioning provides reliable firmware updates with automatic rollback. The flash stores two complete firmware images, and U-Boot selects which one to boot. If the newly updated image fails to boot, the system automatically falls back to the known-good image.

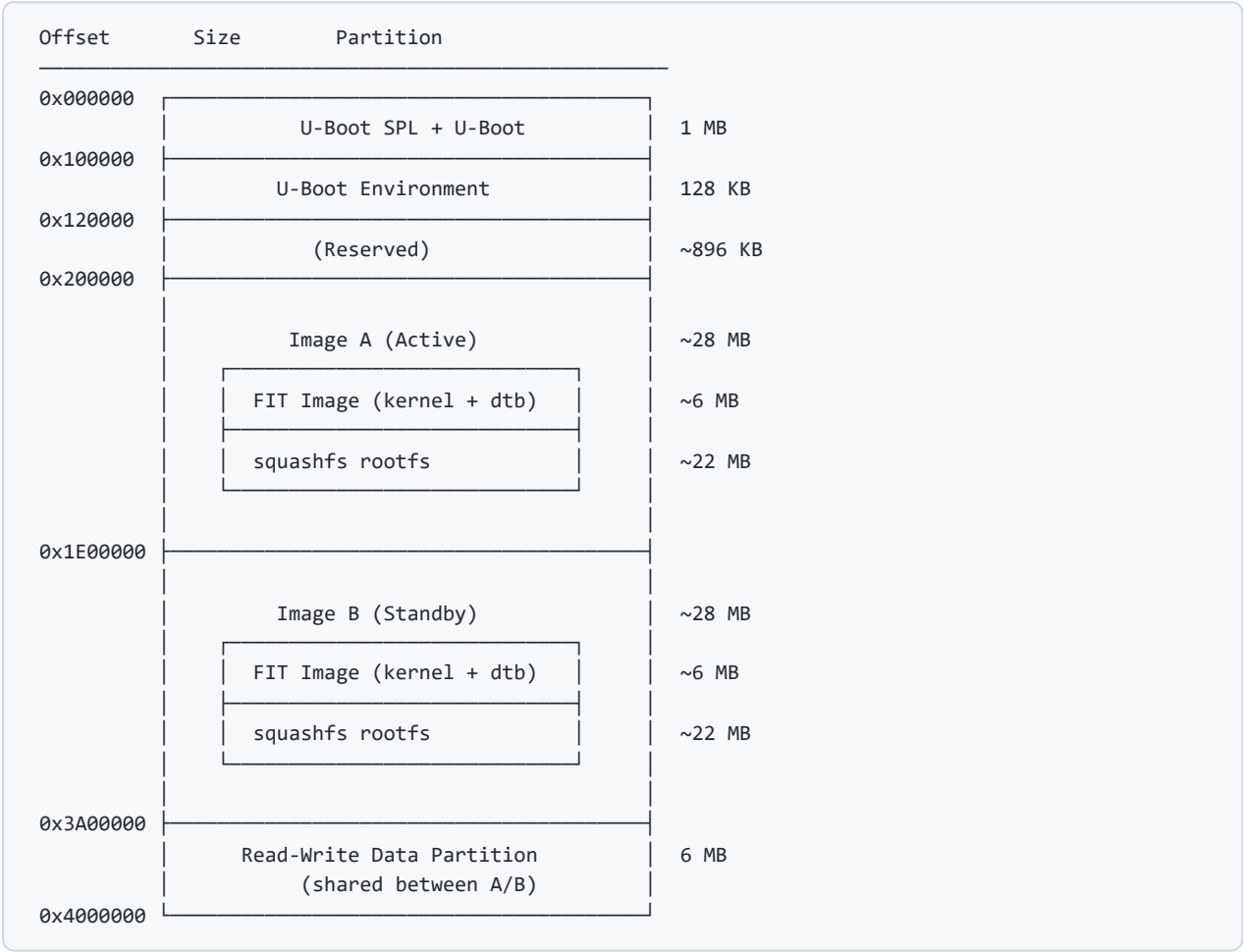
A/B Flash Layout (32MB)



A/B partitioning on 32MB flash is tight. Each image slot has roughly half the space of a single-image layout. Aggressive image size reduction (see previous section) is essential to fit both images.

A/B Flash Layout (64MB)

With 64MB flash, A/B partitioning is comfortable:



U-Boot Environment Variables for A/B Boot

U-Boot manages the A/B boot selection through environment variables. These variables persist across reboots in the U-Boot environment partition.

Variable	Description	Values
<code>bootside</code>	Currently selected boot image	<code>a</code> or <code>b</code>
<code>bootcount</code>	Number of consecutive boot attempts	Integer (0-N)
<code>bootlimit</code>	Maximum boot attempts before rollback	Integer (default: 3)
<code>bootcmd</code>	Top-level boot command	<code>run bootcmd_obmc</code>
<code>kerneladdr_a</code>	Load address for Image A kernel	Platform-specific
<code>kerneladdr_b</code>	Load address for Image B kernel	Platform-specific

Boot Selection Logic

The U-Boot boot command implements A/B selection with automatic rollback:

```
# U-Boot environment script for A/B boot

# Top-level boot command
bootcmd_obmc=run select_image; run load_kernel; run set_bootargs; bootm ${loadaddr}

# Select image based on bootside variable
select_image=\
if test "${bootside}" = "a"; then \
    setenv kerneloff ${kerneloff_a}; \
    setenv rootfsoff ${rootfsoff_a}; \
    echo "Booting Image A"; \
else \
    setenv kerneloff ${kerneloff_b}; \
    setenv rootfsoff ${rootfsoff_b}; \
    echo "Booting Image B"; \
fi

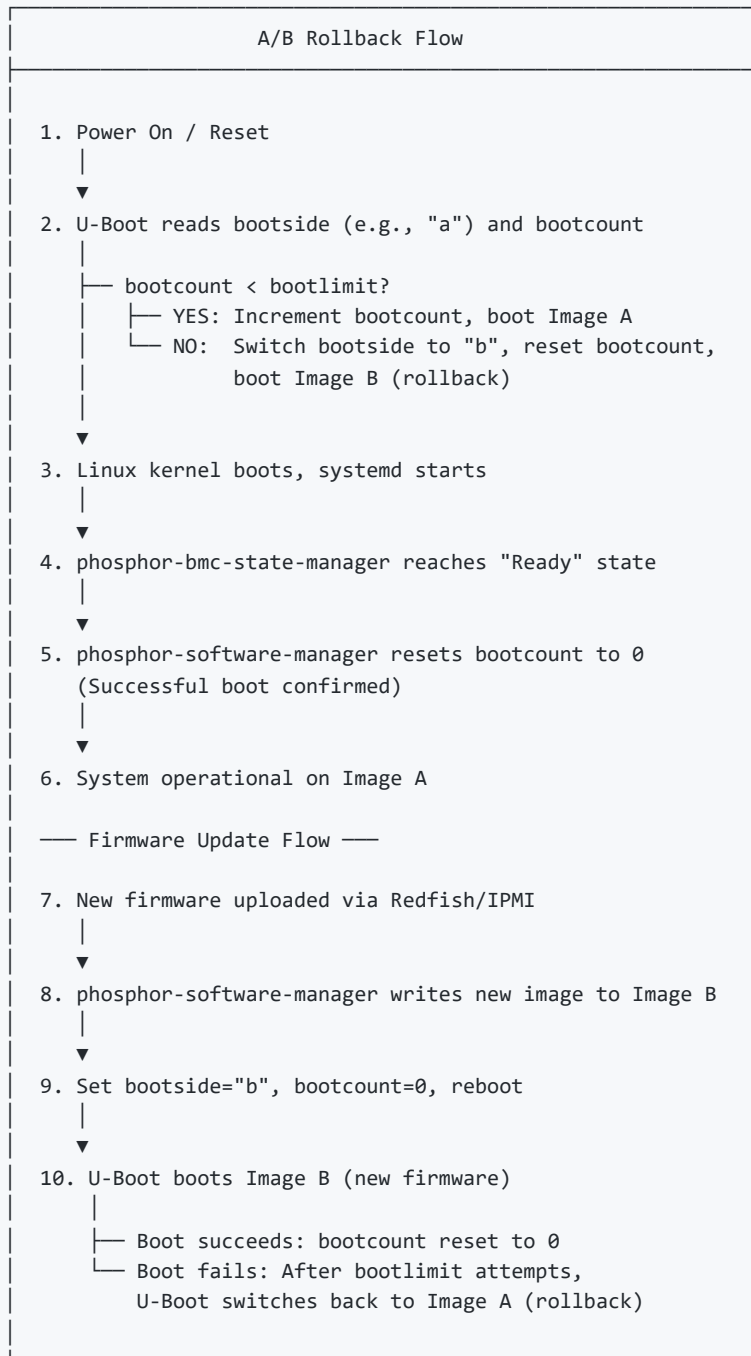
# Load kernel from selected partition
load_kernel=sf probe 0; sf read ${loadaddr} ${kerneloff} ${kernelsize}

# Set kernel boot arguments with selected rootfs
set_bootargs=setenv bootargs console=ttyS4,115200 root=/dev/mtdblock${rootmtd} ro

# Partition offsets for each image
kerneloff_a=0x100000
rootfsoff_a=0x400000
kerneloff_b=0xF00000
rootfsoff_b=0x1200000
```

Rollback Flow

The rollback mechanism works through a boot counter managed between U-Boot and the OpenBMC userspace:



Configuring A/B in Your Machine Layer

Enable A/B update support in your machine configuration:

```
# conf/machine/myboard.conf

# Enable dual image support
OBMC_MACHINE_FEATURES += "obmc-phosphor-flash-mgmt"

# Flash layout for A/B partitioning (64MB example)
FLASH_SIZE = "65536"
FLASH_UBOOT_OFFSET = "0"

# Image A offsets
FLASH_KERNEL_OFFSET = "2048"
FLASH_ROFS_OFFSET = "10240"

# Image B offsets (set by alternate image configuration)
# These are managed by phosphor-software-manager

# Shared read-write partition
FLASH_RWFS_OFFSET = "59392"
```

```
# recipes-phosphor/flash/phosphor-software-manager_%.bbappend

# Enable dual image support
EXTRA_OEMESON:append = " -Dfwupd-type=static-dual"
```

The `phosphor-software-manager` handles writing the update to the alternate partition. You do not manually manage Image B offsets in the machine configuration. The software manager calculates the alternate partition location based on the flash layout.

eMMC vs NOR Flash Comparison

Some newer BMC platforms offer eMMC storage as an alternative to traditional SPI NOR flash. This section compares the two options to help you choose the right storage technology.

Comparison Table

Aspect	SPI NOR Flash	eMMC
Typical capacity	32 MB - 128 MB	4 GB - 64 GB
Read speed	50-100 MB/s	100-300 MB/s
Write speed	1-5 MB/s (page write)	30-90 MB/s
Erase block size	4-64 KB	512 KB (managed internally)
Wear leveling	None (manual via UBI)	Built-in controller
Endurance	100K erase cycles (typical)	3K-10K P/E cycles (per cell)
Boot time	Fast (XIP capable)	Slightly slower (needs init)
Execute in place (XIP)	Yes	No
Cost per MB	High	Low
Power failure safety	Good (atomic page writes)	Varies (needs FTL)
OpenBMC support	Mature, well tested	Emerging, platform-specific

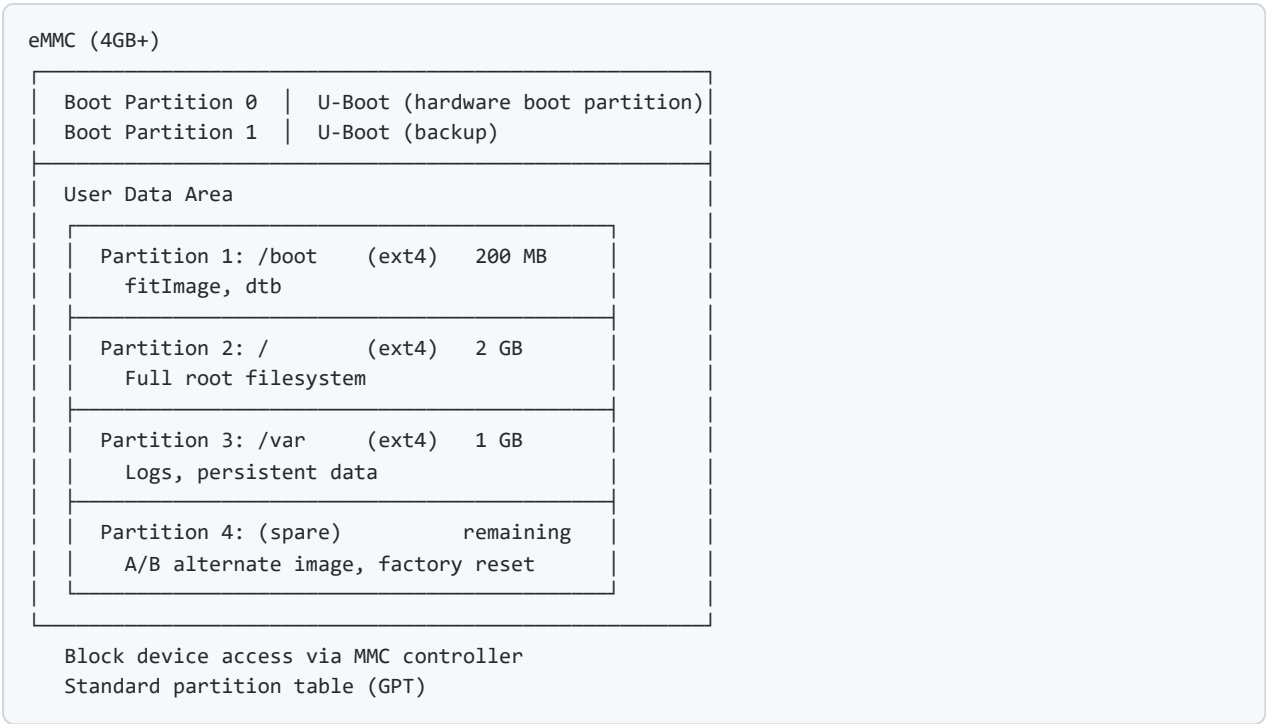
NOR Flash Filesystem Layout

SPI NOR Flash (32MB)

U-Boot	MTD raw
Kernel	MTD raw
RootFS	squashfs
RWFS	JFFS2/UBIFS

Direct memory-mapped access
Simple, predictable boot path

eMMC Filesystem Layout



When to Use Each

Choose SPI NOR when:

- Your platform has 32-64 MB flash and does not need more
- You need the fastest possible boot time (XIP support)
- You want the most mature and tested OpenBMC support
- Your design is cost-sensitive and does not need large storage
- You prefer the simplest possible boot path

Choose eMMC when:

- You need more than 128 MB of storage (diagnostics, crash dumps, logs)
- Your platform performs frequent write operations to persistent storage
- You want built-in wear leveling without UBI overhead
- You plan to store large assets (BIOS images, firmware bundles)
- Your SoC has a built-in eMMC controller (AST2600 supports eMMC)

The AST2600 supports both SPI NOR and eMMC boot. You can use SPI NOR as the primary boot device with eMMC for extended storage, giving you the reliability of NOR flash boot with the capacity of eMMC.

Code Examples

Example 1: Image Size Analysis Script

Use this script to identify the largest packages in your OpenBMC image:

```
#!/bin/bash
# analyze-image-size.sh - Analyze OpenBMC image size contributors
# Usage: ./analyze-image-size.sh <build-dir> <machine>

BUILD_DIR=${1:-.}
MACHINE=${2:-$(cat ${BUILD_DIR}/conf/local.conf | grep ^MACHINE | awk -F' "' '{print $2}')}
DEPLOY="${BUILD_DIR}/tmp/deploy/images/${MACHINE}"
MANIFEST="${DEPLOY}/obmc-phosphor-image-${MACHINE}.manifest"

echo "=== Flash Image Size Analysis ==="
echo "Machine: ${MACHINE}"
echo ""

# Total image size
if [ -f "${DEPLOY}/obmc-phosphor-image-${MACHINE}.static.mtd" ]; then
    TOTAL=$(stat -c%s "${DEPLOY}/obmc-phosphor-image-${MACHINE}.static.mtd")
    echo "Total flash image: $((($TOTAL / 1024 / 1024)) MB"
fi

# Individual component sizes
echo ""
echo "=== Component Sizes ==="
for f in fitImage*.bin *.squashfs u-boot*.bin; do
    FILE="${DEPLOY}/${f}"
    if [ -f "${FILE}" ]; then
        SIZE=$(stat -c%s "${FILE}")
        printf " %-50s %8d KB\n" "${f}" "$((($SIZE / 1024))"
    fi
done

# Package manifest (top 20 largest)
echo ""
echo "=== Top 20 Packages by Installed Size ==="
if [ -f "${MANIFEST}" ]; then
    # The manifest lists package-name arch version
    # Use opkg to get sizes from the package database
    ROOTFS="${BUILD_DIR}/tmp/work/*-openbmc-linux/obmc-phosphor-image/*/rootfs"
    if [ -d ${ROOTFS} ]; then
        du -sk ${ROOTFS}/usr/bin/* ${ROOTFS}/usr/lib/* ${ROOTFS}/usr/sbin/* \
            2>/dev/null | sort -rn | head -20 | \
            awk '{printf " %-50s %8d KB\n", $2, $1}'
    fi
fi

echo ""
echo "=== Filesystem Space Check ==="
echo "Compare sizes against partition limits in your machine.conf"
```

See the complete example at [examples/flash-layout/](#).

Example 2: Custom Flash Layout Configuration

This example shows a minimal machine configuration with A/B support for a 64MB platform:

```
# conf/machine/example-bmc.conf - Flash layout with A/B support

#@TYPE: Machine
#@NAME: Example BMC with A/B flash
#@DESCRIPTION: 64MB flash with dual-image support

require conf/machine/include/ast2600.inc
require conf/machine/include/obmc-bsp-common.inc

# Flash configuration
FLASH_SIZE = "65536"
FLASH_UBOOT_OFFSET = "0"
FLASH_KERNEL_OFFSET = "2048"
FLASH_ROFS_OFFSET = "10240"
FLASH_RWFS_OFFSET = "59392"

# Use static image with dual support
OBMC_IMAGE_TYPE = "static"

# Compression for fitting A/B images
EXTRA_IMAGECMD:squashfs-xz = "-comp xz -Xbcj arm -b 256K"

# Enable flash management
OBMC_MACHINE_FEATURES += "obmc-phosphor-flash-mgmt"
```

Example 3: Verifying Flash Layout on Target

Run these commands on a booted OpenBMC system to inspect the actual flash layout:

```
# View MTD partition table
cat /proc/mtd
# Output:
# dev:      size  erasesize  name
# mtd0: 00060000 00010000 "u-boot"
# mtd1: 00020000 00010000 "u-boot-env"
# mtd2: 00400000 00010000 "kernel"
# mtd3: 01600000 00010000 "rofs"
# mtd4: 00500000 00010000 "rwfs"

# Check mounted filesystems
mount | grep mtd
# /dev/mtdblock3 on / type squashfs (ro)
# /dev/mtdblock4 on /var type jffs2 (rw)

# Check flash usage
df -h
# Filesystem      Size  Used Avail Use% Mounted on
# /dev/root        22M   18M    0 100% /
# /dev/mtdblock4  5.0M   1.2M   3.8M  24% /var

# Read flash info via sysfs
for mtd in /sys/class/mtd/mtd*/; do
    echo "$(basename $mtd): $(cat ${mtd}name) - $(cat ${mtd}size) bytes"
done
```

Troubleshooting

Issue: Image Exceeds Flash Partition Size

Symptom: Build completes but the system fails to boot, or you see warnings about image size during build.

Cause: The squashfs rootfs or FIT image has grown beyond the space allocated between partition offsets.

Solution:

1. Check component sizes: `ls -lh tmp/deploy/images/<machine>/*.squashfs`
2. Compare against available space: `FLASH_RWFS_OFFSET - FLASH_ROFS_OFFSET` (in KB)
3. Apply image size reduction techniques from the section above
4. If size reduction is insufficient, increase `FLASH_RWFS_OFFSET` or use a larger flash part

Issue: JFFS2 Mount Fails After Flash Update

Symptom: System boots but `/var` or other read-write paths are not available. Kernel log shows JFFS2 errors.

Cause: The read-write partition was not properly erased before writing the new JFFS2 image, or the partition offset changed between firmware versions.

Solution:

1. Erase the RWFS partition from U-Boot: `sf erase <rwfs_offset> <rwfs_size>`
2. Verify partition offsets match between old and new firmware
3. Consider using UBI layout for better flash management

Issue: A/B Rollback Loop

Symptom: System continuously switches between Image A and Image B without successfully booting.

Cause: Both images are corrupted or have the same boot-blocking issue. The boot counter reaches `bootlimit` on each side.

Solution:

1. Interrupt U-Boot at the serial console (press a key during countdown)
2. Manually set the boot side: `setenv bootside a; setenv bootcount 0; saveenv`
3. Boot manually: `run bootcmd_obmc`
4. If both images are bad, reflash via UART or JTAG using a known-good image

Issue: UBI Volume Attach Fails

Symptom: Boot fails with UBI errors like `ubi0 error: ubi_io_read` or `UBI error: cannot attach`.

Cause: The UBI metadata on flash is corrupted, often from a power loss during a UBI write or from flashing a static image to a UBI-formatted partition.

Solution:

1. Reformat the UBI partition from U-Boot:

```
sf probe 0
sf erase 0x100000 0x1F00000 # Erase entire UBI area
```


2. Reflash the complete UBI image
3. Ensure you flash UBI images (`.ubi.mtd`) to UBI-formatted flash, not static images

Debug Commands

```
# Check MTD partition layout
cat /proc/mtd

# Read flash ID and status
mtd_debug info /dev/mtd0

# Dump flash content for analysis
nanddump /dev/mtd2 -f /tmp/kernel-dump.bin

# Check UBI device info
ubininfo -a

# View U-Boot environment from Linux
fw_printenv

# Modify U-Boot environment from Linux
fw_setenv bootside a
fw_setenv bootcount 0

# Check image version and activation status
busctl tree xyz.openbmc_project.Software.BMC.Updater
busctl introspect xyz.openbmc_project.Software.BMC.Updater \
    /xyz/openbmc_project/software
```

References

Official Resources

- [OpenBMC image_types_phosphor.bbclass](#)
- [phosphor-bmc-code-mgmt \(Software Manager\)](#)
- [OpenBMC Flash Layout Documentation](#)

Related Guides

- [U-Boot Guide](#)
- [Machine Layer Guide](#)
- [Firmware Update Guide](#)
- [Porting Reference](#)

External Documentation

- [MTD Subsystem Documentation](#)
- [UBI - Unsorted Block Images](#)
- [SquashFS Documentation](#)
- [Yocto Project Image Creation](#)

Tested on: Architectural reference documentation - requires real hardware for full flash layout validation. Last updated: 2025-02-06

AST2700 Enablement Guide

Port OpenBMC to the ASPEED AST2700, the first 64-bit ARM BMC SoC with Caliptra silicon root of trust.

Table of Contents

1. [Overview](#)
2. [AST2600 vs AST2700 Comparison](#)
 1. [Hardware Comparison Table](#)
 2. [Architecture Diagram](#)
3. [32-bit to 64-bit Migration](#)
 1. [Key Migration Changes](#)
 2. [Machine Configuration for AST2700](#)
 3. [Pointer Size and Code Portability](#)
 4. [Kernel Configuration Changes](#)
4. [New Peripheral Interfaces](#)
 1. [CAN FD \(Controller Area Network Flexible Data-Rate\)](#)
 1. [Device Tree and Usage](#)
 2. [LTPI \(LVDS Tunneling Protocol Interface\)](#)
 3. [USB 3.2 Gen1](#)
 4. [I3C \(Improved Inter-Integrated Circuit\)](#)
5. [Caliptra Silicon Root of Trust](#)
 1. [Caliptra Components](#)
 2. [Measured Boot Flow](#)
 3. [DICE Certificate Chain](#)
 4. [Caliptra Mailbox Interface](#)
 5. [Enabling Caliptra in OpenBMC](#)
6. [Upstream Linux Kernel Driver Readiness](#)
 1. [Driver Readiness Table](#)
 2. [Kernel Source Strategy](#)
7. [Troubleshooting](#)
 1. [Issue: Kernel Fails to Boot on AST2700](#)
 2. [Issue: Userspace Segfaults After 64-bit Migration](#)
 3. [Issue: CAN or LTPI Drivers Not Found](#)
 4. [Issue: Caliptra Mailbox Timeout](#)
 5. [Debug Commands](#)
8. [References](#)
 1. [Official Resources](#)
 2. [Caliptra Resources](#)
 3. [Related Guides](#)

Overview

The ASPEED AST2700 is ASPEED's next-generation BMC system-on-chip. It replaces the single-core 32-bit ARM Cortex-A7 of the AST2600 with a quad-core 64-bit ARM Cortex-A35 application processor alongside a dedicated real-time Cortex-M4F core. This architectural leap brings significantly more compute power, wider memory addressing, and new peripheral interfaces that align with emerging datacenter management standards.

Enabling the AST2700 in OpenBMC involves more than just adding a new machine configuration. The transition from 32-bit to 64-bit ARM changes the kernel architecture, compiler tuning, and pointer sizes across the entire userspace. New peripheral controllers for CAN bus, LTPI (LVDS Tunneling Protocol Interface), and USB 3.2 require kernel driver support and device tree bindings. Most notably, the AST2700 integrates Caliptra, an open-source silicon root of trust developed by the Open Compute Project, which enables measured boot and DICE-based identity certificates.

Key concepts covered:

- AST2600 vs AST2700 hardware comparison and migration path
- 64-bit ARM (aarch64) kernel and userspace configuration
- New peripheral interfaces: CAN, LTPI, USB 3.2, I3C
- Caliptra silicon root of trust and measured boot
- Upstream Linux kernel driver readiness for AST2700

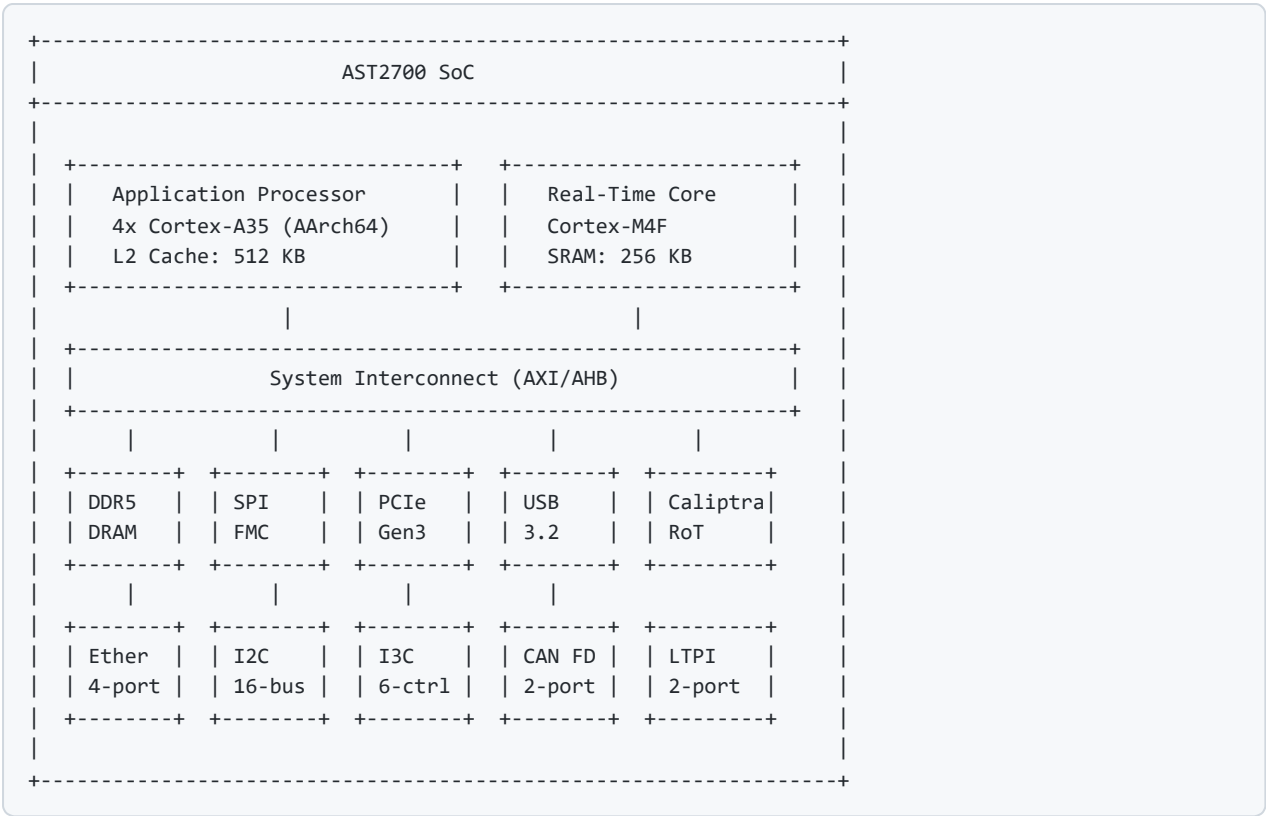
The AST2700 is a new SoC and upstream Linux kernel support is still evolving. Some drivers may be available only in ASPEED's vendor kernel tree at the time of writing. Always check the latest upstream kernel status before starting your port.

AST2600 vs AST2700 Comparison

Hardware Comparison Table

Feature	AST2600	AST2700
CPU (Application)	1x ARM Cortex-A7 (32-bit)	4x ARM Cortex-A35 (64-bit)
CPU (Real-time)	N/A	1x ARM Cortex-M4F
Architecture	ARMv7-A (armhf)	ARMv8-A (aarch64)
Max Clock	1.2 GHz	1.6 GHz
L2 Cache	256 KB	512 KB (shared)
DRAM	DDR4, up to 2 GB (32-bit bus)	DDR5, up to 8 GB (64-bit bus)
SPI NOR Flash	2x FMC + 2x SPI	3x FMC + 2x SPI
eMMC	Yes (5.1)	Yes (5.1)
Ethernet	4x RGMII/RMII	4x RGMII/RMII + SGMII
USB	USB 2.0 Host/Hub	USB 3.2 Gen1 + USB 2.0
PCIe	PCIe Gen2 x1 (RC)	PCIe Gen3 x2 (RC/EP)
CAN Bus	N/A	2x CAN FD
LTPI	N/A	2x LTPI
I2C	16 buses	16 buses
I3C	N/A	6x I3C controllers
GPIO	~228 pins	~256 pins
ADC	16-channel, 10-bit	16-channel, 12-bit
Video Engine	2D, JPEG/ASPEED codec	2D, JPEG/ASPEED codec (improved)
Security	Secure Boot (OTP)	Caliptra Silicon RoT, Secure Boot
Process Node	40nm	12nm

Architecture Diagram



The Cortex-M4F real-time core runs independently from the Cortex-A35 cluster. It handles low-latency tasks such as fan control PID loops, GPIO event monitoring, and power sequencing. Communication between the two cores uses shared memory and inter-processor interrupts (IPI).

32-bit to 64-bit Migration

The transition from AST2600 (ARMv7-A, 32-bit) to AST2700 (ARMv8-A, 64-bit) affects every layer of the software stack: the kernel, bootloader, userspace libraries, and Yocto build configuration.

Key Migration Changes

Aspect	AST2600 (32-bit)	AST2700 (64-bit)
DEFAULTTUNE	<code>arm1176jzs</code> or <code>cortexa7hf-neon</code>	<code>cortexa35</code>
Kernel ARCH	<code>arm</code>	<code>arm64</code>
Kernel image	<code>zImage</code>	<code>Image</code>
Pointer size	4 bytes	8 bytes
<code>size_t</code> / <code>off_t</code>	32-bit	64-bit
Userspace ABI	<code>arm-openbmc-linux-gnueabi</code>	<code>aarch64-openbmc-linux</code>
Compiler flags	<code>-marm -march=armv7-a</code>	<code>-march=armv8-a</code>

Machine Configuration for AST2700

```
# conf/machine/ast2700-bmc.conf

#@TYPE: Machine
#@NAME: AST2700 BMC
#@DESCRIPTION: OpenBMC for ASPEED AST2700-based platforms

# 64-bit ARM Cortex-A35 tuning
DEFAULTTUNE = "cortexa35"
require conf/machine/include/arm/armv8a/tune-cortexa35.inc

# Kernel configuration
PREFERRED_PROVIDER_virtual/kernel = "linux-aspeed"
KERNEL_IMAGETYPE = "Image"
KERNEL_DEVICETREE = "aspeed/aspeed-bmc-${MACHINE}.dtb"

# U-Boot configuration
PREFERRED_PROVIDER_virtual/bootloader = "u-boot-aspeed"
PREFERRED_PROVIDER_u-boot = "u-boot-aspeed"
UBOOT_MACHINE = "ast2700_openbmc_defconfig"
SPL_BINARY = "spl/u-boot-spl.bin"

# Flash layout (64MB SPI NOR)
FLASH_SIZE = "65536"
FLASH_UBOOT_OFFSET = "0"
FLASH_KERNEL_OFFSET = "2048"
FLASH_ROFS_OFFSET = "10240"
FLASH_RWFS_OFFSET = "55296"

# Console
SERIAL_CONSOLES = "115200;ttyS4"

# Machine features
MACHINE_FEATURES = "efi"
OBMC_MACHINE_FEATURES += "\
    obmc-bmc-state-mgmt \
    obmc-phosphor-fan-mgmt \
    obmc-phosphor-flash-mgmt \
"

# Entity Manager for dynamic inventory
VIRTUAL-RUNTIME_obmc-inventory-manager = "entity-manager"

# Include AST2700 base configuration
require conf/machine/include/ast2700.inc
require conf/machine/include/obmc-bsp-common.inc
```

Do not mix 32-bit and 64-bit tunings. Setting `DEFAULTTUNE = "cortexa35"` ensures the entire sysroot is compiled for AArch64. If you accidentally inherit a 32-bit tune file, the build may succeed but produce a non-bootable image.

Pointer Size and Code Portability

The move from 32-bit to 64-bit pointers affects C/C++ code that assumes pointer sizes:

```
// BAD: Assumes pointer fits in 32-bit integer
uint32_t addr = (uint32_t)some_pointer; // Truncates on 64-bit

// GOOD: Use uintptr_t for pointer-to-integer conversions
uintptr_t addr = (uintptr_t)some_pointer; // Safe on both

// BAD: Assumes size_t is 32-bit
printf("size: %u\n", some_size_t_value);

// GOOD: Use %zu for size_t
printf("size: %zu\n", some_size_t_value); // Portable
```

Search your platform-specific recipes for `uint32_t` casts of pointers, hardcoded `4` as `sizeof(void*)`, and `%u` / `%lu` format strings for `size_t` values. These are the most common sources of 64-bit migration bugs.

Kernel Configuration Changes

```
# meta-<machine>/recipes-kernel/linux/linux-aspeed/ast2700.cfg

# ===== Architecture =====
CONFIG_ARM64=y
CONFIG_ARCH_ASPEED=y
CONFIG_MACH_ASPEED_G7=y

# ===== SMP Support (quad-core) =====
CONFIG_SMP=y
CONFIG_NR_CPUS=4

# ===== AST2700 Peripheral Drivers =====
CONFIG_SERIAL_8250=y
CONFIG_SERIAL_8250_ASPEED=y
CONFIG_I2C_ASPEED=y
CONFIG_SPI_ASPEED_SMC=y
CONFIG_PINCTRL_ASPEED_G7=y
CONFIG_GPIO_ASPEED=y
CONFIG_SENSORS_ASPEED=y
CONFIG_WATCHDOG_ASPEED=y
CONFIG_USB_XHCI_HCD=y
CONFIG_CAN=y
CONFIG_CAN_ASPEED=y

# ===== Real-Time Core Communication =====
CONFIG_MAILBOX=y
CONFIG_ASPEED_MBOX=y

# ===== Security =====
CONFIG_CRYPT=y
CONFIG_CRYPT_SHA256=y
CONFIG_CRYPT_SHA384=y
CONFIG_CRYPT_SHA512=y
```

Apply this fragment in your kernel bbappend:

```
# meta-<machine>/recipes-kernel/linux/linux-aspeed_%.bbappend

FILESEXTRAPATHS:prepend := "${THISDIR}/linux-aspeed:"
SRC_URI += "file://ast2700.cfg"
```


New Peripheral Interfaces

The AST2700 introduces several new peripheral controllers not present on the AST2600.

CAN FD (Controller Area Network Flexible Data-Rate)

The AST2700 includes two CAN FD controllers for communication with platform management controllers, power supplies, and chassis management devices. CAN FD supports data rates up to 8 Mbps with 64-byte payloads.

Device Tree and Usage

```
&can0 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_can0_default>;
    can-transceiver {
        max-bitsrate = <8000000>;
    };
};

# Bring up CAN interface
ip link set can0 type can bitrate 500000 dbitsrate 4000000 fd on
ip link set can0 up

# Send and monitor CAN FD frames
cansend can0 123##1.DEADBEEF
candump can0
```

LTPI (LVDS Tunneling Protocol Interface)

LTPI is an OCP-specified interface for tunneling low-speed management buses (I2C, GPIO, UART) over a single LVDS differential pair. It reduces cable count between the BMC and host or satellite boards, as defined in the DC-SCM 2.0 specification.

```
&lt;lt;pi0 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_lt;pi0_default>;
    aspeed,mode = "bmc"; /* "bmc" or "host" */
    i2c-channels = <4>; /* Tunneled I2C buses */
    gpio-channels = <16>; /* Tunneled GPIO pins */
    uart-channels = <1>; /* Tunneled UARTs */
};
```

Use Case	Description
DC-SCM 2.0	Single-cable BMC-to-host board connection
Satellite BMC	Remote I2C/GPIO access to expansion chassis
Cable reduction	Replace multiple ribbon cables with one LVDS pair

LTPI driver support is currently available only in ASPEED’s vendor kernel tree. Track upstream submission at the [linux-aspeed mailing list](#).

USB 3.2 Gen1

The AST2700 upgrades from USB 2.0 to USB 3.2 Gen1 (5 Gbps), enabling faster virtual media, firmware updates, and diagnostic data transfer.

```
&usb {
    status = "okay";
    xhci@1e6a0000 {
        compatible = "aspeed,ast2700-xhci";
        reg = <0x1e6a0000 0x1000>;
        interrupts = <GIC_SPI 14 IRQ_TYPE_LEVEL_HIGH>;
        phys = <&usb3_phy>;
        phy-names = "usb3-phy";
    };
};
```

```
# Kernel configuration for USB 3.2
CONFIG_USB_XHCI_HCD=y
CONFIG_USB_XHCI_PLATFORM=y
CONFIG_USB_ASPEED_XHCI=y
CONFIG_USB_GADGET=y
CONFIG_USB_CONFIGFS_MASS_STORAGE=y
```

I3C (Improved Inter-Integrated Circuit)

The AST2700 includes six I3C controllers for next-generation sensors and memory devices using the MIPI I3C protocol.

```
&i3c0 {
    status = "okay";
    i3c-scl-hz = <12500000>; /* 12.5 MHz I3C mode */
    i2c-scl-hz = <400000>; /* 400 KHz I2C fallback */

    temperature-sensor@48,39200000000 {
        reg = <0x48 0x392 0x00000000>;
        assigned-address = <0x48>;
    };
};
```

I3C is backward-compatible with I2C. You can connect legacy I2C sensors to an I3C bus in mixed mode. The controller automatically handles the protocol difference for each device.

Caliptra Silicon Root of Trust

The AST2700 integrates Caliptra, an open-source silicon root of trust (RoT) developed by the Open Compute Project (OCP). Caliptra provides hardware-rooted security features that are immutable after manufacturing.

Caliptra Components

Component	Function
DICE Engine	Generates identity certificates bound to firmware measurements
SHA-384/512	Hashes firmware images for measured boot
ECC-384	Signs and verifies certificates and attestation data
HMAC Engine	Derives keys from compound device identifiers
Key Vault	Stores sealed keys accessible only to authorized firmware
Mailbox	Hardware interface for BMC firmware communication

Measured Boot Flow

Caliptra implements a measured boot chain where each firmware stage is hashed before execution. Measurements are stored in Platform Configuration Registers (PCRs) inside the Caliptra hardware.

Boot Stage	Measured By	PCR	Description
Caliptra FMC	Caliptra ROM	PCR[0]	First Mutable Code, DICE CDI derived
Caliptra Runtime	Caliptra FMC	PCR[1]	Runtime firmware, certificate chain extended
U-Boot SPL	Caliptra RT	PCR[2]	First BMC boot stage
U-Boot	U-Boot SPL	PCR[3]	Hash sent to Caliptra via mailbox
FIT Image	U-Boot	PCR[4]	Kernel + device tree blob

After the kernel boots, Caliptra Runtime remains active and serves attestation requests via SPDH over MCTP.

DICE Certificate Chain

Caliptra generates a DICE (Device Identifier Composition Engine) certificate chain that cryptographically binds device identity to the specific firmware running on it.

Certificate	Issuer	Contains
DeviceID Cert	Manufacturer CA	Unique device identity (from OTP fuses)
Alias Cert (FMC)	DeviceID	Measurement of FMC firmware
Alias Cert (RT)	FMC Alias	Measurement of Runtime firmware
Alias Cert (BMC)	RT Alias	Measurement of U-Boot + Kernel

The DICE certificate chain changes any time firmware is updated, because the measurements change. Remote attestation services use this chain to verify platform integrity before granting access to secrets or workloads.

Caliptra Mailbox Interface

The BMC firmware communicates with Caliptra through a hardware mailbox for sending measurements, requesting certificates, and performing cryptographic operations.

```

/* Caliptra mailbox register map (simplified) */
#define CALIPTRA_MBOX_CMD      0x1e6f0000 /* Command register */
#define CALIPTRA_MBOX_DLEN    0x1e6f0004 /* Data length */
#define CALIPTRA_MBOX_DATAIN   0x1e6f0008 /* Data input FIFO */
#define CALIPTRA_MBOX_DATAOUT  0x1e6f000c /* Data output FIFO */
#define CALIPTRA_MBOX_STATUS   0x1e6f0010 /* Status register */

/* Common mailbox commands */
#define CALIPTRA_CMD_STASH_MEASUREMENT 0x434D5348 /* "CMSh" */
#define CALIPTRA_CMD_GET_IDEV_CERT     0x49444556 /* "IDev" */
#define CALIPTRA_CMD_GET_FMC_ALIAS_CERT 0x464D4341 /* "FMCA" */

```

Enabling Caliptra in OpenBMC

```

# Kernel configuration for Caliptra
CONFIG_ASPEED_CALIPTRA=y
CONFIG_ASPEED_CALIPTRA_MBOX=y
CONFIG_TCG_TPM=y
CONFIG_TCG_CALIPTRA=y

```

```

# recipes-bsp/u-boot/u-boot-aspeed_%.bbappend
# Enable Caliptra measured boot in U-Boot
EXTRA_OEMAKE:append = " CALIPTRA=1"

```

Caliptra's runtime firmware supports SPDm for remote attestation, allowing a verifier to request measurement logs and DICE certificates over MCTP:

```

# Check Caliptra attestation readiness
busctl tree xyz.openbmc_project.SPDm

# Query Caliptra device identity
busctl get-property xyz.openbmc_project.SPDm \
    /xyz/openbmc_project/spdm/caliptra \
    xyz.openbmc_project.SPDm.Responder CertificateChain

```

If your platform does not require measured boot, you can leave Caliptra in its default pass-through mode. The boot flow proceeds normally without measurement enforcement, but you lose remote attestation capability.

Upstream Linux Kernel Driver Readiness

Driver Readiness Table

Peripheral	Driver	Upstream Status	Kernel Version	Notes
UART (16550)	8250_aspeed	Merged	v6.6+	Standard 16550-compatible
I2C	i2c-aspeed	Merged	v6.8+	AST2700 register updates
SPI FMC	spi-aspeed-smc	Merged	v6.8+	New FMC3 controller
GPIO	gpio-aspeed	Merged	v6.9+	Extended pin count (256)
Pinctrl	pinctrl-aspeed-g7	Merged	v6.9+	New G7 pin mux tables
Watchdog	aspeed_wdt	Merged	v6.8+	Dual watchdog support
ADC	aspeed_adc	Merged	v6.10+	12-bit resolution
PECI	peci-aspeed	Merged	v6.8+	Enhanced PECI 4.0
Ethernet	ftgmac100	Merged	v6.9+	SGMII PHY support
eMMC/SD	sdhci-aspeed	Merged	v6.8+	eMMC 5.1 support
PWM/Fan	aspeed-pwm-tacho	Merged	v6.10+	Updated for G7
USB 3.2 xHCI	xhci-aspeed	In review	–	Expected v6.12+
CAN FD	can-aspeed	In review	–	Expected v6.12+
I3C	i3c-aspeed	In review	–	Expected v6.13+
LTPI	ltpi-aspeed	Not submitted	–	Vendor tree only
Caliptra Mbox	caliptra-aspeed	In review	–	OCP working group
Video Engine	aspeed-video	Partial	v6.9+	G7 updates pending
MCTP/PCIe EP	mctp-pcie-aspeed	Partial	v6.10+	Gen3 EP pending
GIC-600	irq-gic-v3	Merged	v6.6+	Standard ARM GICv3
Timer	arm_arch_timer	Merged	v6.6+	Standard ARM timer

Drivers marked “In review” or “Not submitted” require ASPEED’s vendor kernel tree or backported patches. Track status at the [linux-aspeed mailing list](#) and [LKML](#).

Kernel Source Strategy

Strategy	When to Use	Pros	Cons
Upstream mainline	All required drivers merged	Community support, long-term maintenance	May lack newest features
ASPEED vendor kernel	Need CAN, LTPI, Caliptra, USB 3.2	Full feature set	Vendor maintenance burden
Upstream + backports	Most drivers merged, need a few extras	Best of both worlds	Patch maintenance

```
# Using ASPEED vendor kernel (recommended for early AST2700 enablement)
# recipes-kernel/linux/linux-aspeed_%.bbappend

FILESEXTRAPATHS:prepend := "${THISDIR}/linux-aspeed:"

SRC_URI = "git://github.com/AspeedTech-BMC/linux;branch=aspeed-master-v6.6"
SRCREV = "${AUTOREV}"

SRC_URI += "\
    file://ast2700.cfg \
    file://ast2700-can.cfg \
    file://ast2700-caliptra.cfg \
"
```

Start with the ASPEED vendor kernel for initial bring-up, then migrate to upstream mainline as drivers are merged. Track which patches you carry and their upstream submission status.

Troubleshooting

Issue: Kernel Fails to Boot on AST2700

Symptom: U-Boot loads the FIT image but the kernel hangs after "Starting kernel..."

Cause: Kernel compiled for 32-bit ARM (`zImage`) instead of 64-bit ARM (`Image`), or missing GICv3 device tree configuration.

Solution:

1. Verify `KERNEL_IMAGETYPE = "Image"` in machine configuration (not `zImage`)
2. Confirm `DEFAULTTUNE = "cortexa35"` is set
3. Check that the device tree includes the GICv3 interrupt controller node
4. Verify U-Boot uses `booti` command (not `bootz`)

Issue: Userspace Segfaults After 64-bit Migration

Symptom: OpenBMC daemons crash with segmentation faults that did not occur on AST2600.

Cause: Code that casts pointers to 32-bit integers or assumes 32-bit pointer width.

Solution:

1. Rebuild with `-Wall -Wpointer-to-int-cast` to catch truncation warnings

2. Search for `(uint32_t)` casts applied to pointer values
3. Replace `%u` format specifiers for `size_t` with `%zu`
4. Use `sizeof()` instead of hardcoded `4` for pointer sizes

Issue: CAN or LTPI Drivers Not Found

Symptom: `ip link show` does not list CAN interfaces, or LTPI tunneled buses are not visible.

Cause: Upstream kernel does not yet include these drivers.

Solution:

1. Switch to the ASPEED vendor kernel branch
2. Verify the driver is enabled in your kernel config fragment
3. Check `dmesg | grep -i can` or `dmesg | grep -i ltpi` for probe messages
4. Confirm device tree nodes have `status = "okay"` and correct pin control

Issue: Caliptra Mailbox Timeout

Symptom: U-Boot or kernel reports "Caliptra mailbox timeout" during boot.

Cause: Caliptra firmware not loaded, mailbox misconfigured, or Caliptra core held in reset.

Solution:

1. Verify Caliptra firmware is loaded into the correct flash region
2. Check that Caliptra reset is deasserted in the SCU (System Control Unit)
3. Inspect mailbox status register at `0x1e6f0010` for error flags
4. Ensure `caliptra_reserved` memory region does not overlap other allocations

Debug Commands

```
# Check CPU architecture and core count
lscpu
# Should show: Architecture: aarch64, CPU(s): 4

# Verify device tree model
cat /proc/device-tree/model

# Check for AST2700-specific peripherals
dmesg | grep -i ast2700
dmesg | grep -i caliptra
dmesg | grep -i "can\|ltpi\|xhci\|i3c"

# List all I2C/I3C buses (including LTPI-tunneled)
i2cdetect -l

# Verify 64-bit userspace
file /usr/bin/bmcweb
# Should show: ELF 64-bit LSB executable, ARM aarch64
```

References

Official Resources

- [ASPEED AST2700 Product Page](#)
- [OpenBMC meta-aspeed Layer](#)
- [ASPEED Linux Kernel Tree](#)
- [ASPEED U-Boot Tree](#)

Caliptra Resources

- [Caliptra Specification \(OCP\)](#)
- [Caliptra RTL \(CHIPS Alliance\)](#)
- [Caliptra Software \(CHIPS Alliance\)](#)
- [DICE Architecture \(TCG\)](#)

Related Guides

- [Porting Reference](#)
- [Machine Layer Guide](#)
- [Device Tree Guide](#)
- [U-Boot Guide](#)
- [ARM Platform Guide](#)
- [Flash Layout & Optimization Guide](#)

External Documentation

- [ARM Cortex-A35 Technical Reference](#)
- [Linux ARM64 Documentation](#)
- [OCP DC-SCM 2.0 Specification](#)
- [MIPI I3C Specification](#)

Tested on: Reference documentation based on ASPEED AST2700 EVB and vendor SDK materials. Upstream kernel driver status reflects early 2026. Always verify current driver availability before starting your port. Last updated: 2026-02-06

Appendix

Reference material, infrastructure details, and supplementary documentation for OpenBMC development.

Contents

Document	Description
Jenkins CI/CD Infrastructure	OpenBMC's Jenkins-based build and test system
AMI OneTree vs OpenBMC	Technical comparison of AMI MegaRAC OneTree and upstream OpenBMC
Yocto BitBake Build Optimization	Practical guide to speeding up Yocto/BitBake builds for OpenBMC

Table of contents

- [Jenkins CI/CD Infrastructure](#)
- [AMI OneTree vs OpenBMC](#)
- [Yocto BitBake Build Optimization](#)

OpenBMC Jenkins CI/CD Infrastructure

Overview

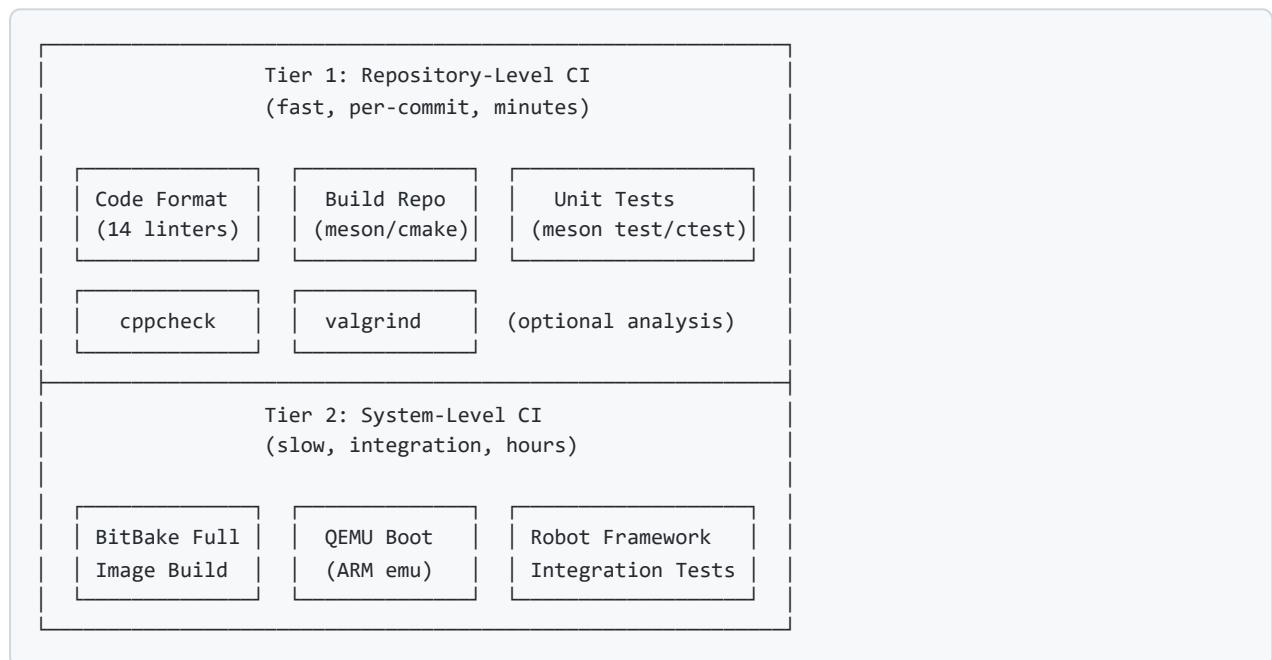
OpenBMC uses a Jenkins-based CI/CD system — not GitHub Actions — for all build and test automation. The infrastructure is centralized in the [openbmc/openbmc-build-scripts](https://github.com/openbmc/openbmc-build-scripts) repository and operates at <https://jenkins.openbmc.org>, tightly integrated with Gerrit for code review.

This appendix documents the CI architecture, job structure, Docker test infrastructure, and Gerrit integration.

Architecture

Two-Tier CI Model

OpenBMC implements a two-tier testing strategy:



Tier 1: Repository-Level CI

Triggered on every Gerrit patchset upload. Runs via `run-unit-test-docker.sh` → `unit-test.py`:

- **Code formatting** — 14 linters/formatters (clang-format, black, shellcheck, eslint, prettier, markdownlint, etc.)
- **Build** — compiles the changed repo using its build system (meson, cmake, or autotools)
- **Unit tests** — runs `meson test` / `make check` / `ctest`
- **Static analysis** — optional cppcheck, valgrind, sanitizer, clang-tidy passes
- **Coverage reports** — generated for tracking test coverage
- Runs on both x86_64 and ppc64le architectures

Tier 2: System-Level CI

Full image build + QEMU boot + Robot Framework testing:

1. **BitBake image build** (`build-setup.sh`) — full Yocto build for a target machine
2. **QEMU boot** (`boot-qemu.sh`) — boots the firmware image in ARM emulation
3. **Robot Framework tests** (`run-robot.sh`) — Redfish API, IPMI, SSH, boot validation tests
4. Duration: tens of minutes to hours

Jenkins Job Map

Job	Script	Purpose
<code>ci-repository</code>	<code>run-unit-test-docker.sh</code>	Unit tests for any repo
<code>ci-repository-ppc64le</code>	<code>run-unit-test-docker.sh</code>	Unit tests on ppc64le
<code>ci-openbmc</code>	<code>build-setup.sh</code>	Full BitBake image build
<code>ci-build-seed</code>	<code>jenkins/build-seed</code>	Pre-populate sstate caches for ~19 machines
<code>run-ci-in-qemu</code>	<code>run-qemu-robot-test.sh</code>	Boot QEMU + Robot tests
<code>openbmc-userid-validation</code>	<code>jenkins/userid-validation</code>	Gerrit user authorization
<code>latest-master</code>	<code>build-setup.sh</code>	Nightly full image builds
<code>latest-master-sdk</code>	<code>build-setup.sh</code>	Nightly SDK builds
<code>latest-qemu-ppc64le</code>	<code>qemu-build.sh</code>	Build QEMU for ppc64le
<code>latest-qemu-x86</code>	<code>qemu-build.sh</code>	Build QEMU for x86
<code>latest-unit-test-coverage</code>	<code>get_unit_test_report.py</code>	Coverage reports
<code>release-tag</code>	<code>build-setup.sh</code>	Tagged release builds
<code>ci-openbmc-build-scripts</code>	<code>jenkins/run-build-script-ci</code>	Self-test of build scripts

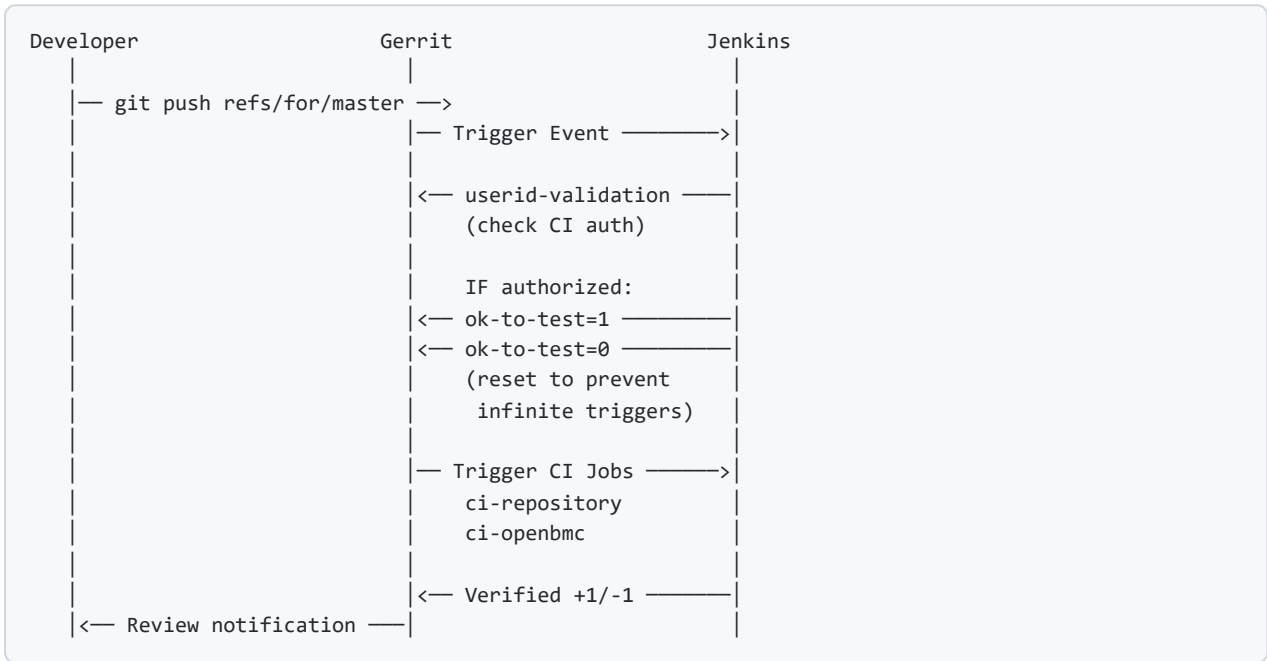
Repository Structure

```
openbmc-build-scripts/
├── build-setup.sh           # Full BitBake image build (Tier 2)
├── run-unit-test-docker.sh  # Unit test Docker runner (Tier 1)
├── run-qemu-robot-test.sh   # QEMU + Robot Framework test runner
├── run-rootfs-size-docker.sh # Root filesystem size analysis
├── qemu-build.sh           # QEMU compilation in Docker
├── jenkins/
│   ├── build-seed          # Seed job for all machine builds
│   ├── run-build-script-ci # Self-test of build scripts
│   ├── run-meta-ci         # Meta-layer CI (deprecated)
│   └── userid-validation    # Gerrit user authorization check
├── scripts/
│   ├── build-unit-test-docker # Docker image builder (Python)
│   ├── unit-test.py          # Unit test runner (Python)
│   ├── dbus-unit-test.py     # D-Bus session test wrapper
│   ├── format-code.sh        # 14-tool code formatting suite
│   ├── boot-qemu.sh          # QEMU launch script
│   ├── boot-qemu-test.exp     # Expect-based QEMU boot validation
│   ├── test-qemu             # Multi-platform QEMU smoke test
│   ├── run-robot.sh          # Robot Framework test executor
│   ├── get_unit_test_report.py # Unit test coverage reporter
│   └── repositories.txt      # Test repository list
├── config/
│   ├── .gitlint             # Commit message rules
│   ├── eslint.config.js     # JSON linting
│   ├── markdownlint.yaml    # Markdown linting
│   └── prettier.yaml        # Multi-format formatting
└── tools/
    ├── owners               # OWNERS file parser for Gerrit
    └── config-clang-tidy     # Clang-tidy configuration
```

Gerrit Integration

Workflow

OpenBMC uses Gerrit (not GitHub PRs) for code review. Jenkins integrates via the Gerrit Trigger plugin:



User Authorization

The `userid-validation` script is a critical gatekeeper. When a Gerrit patchset is uploaded:

1. **Identify the committer** — uses `gerrit query` via SSH to get the uploader's username
2. **Reset the vote** — sets `ok-to-test=0` to prevent Jenkins infinite retriggers
3. **Add OWNERS-based reviewers** — parses OWNERS files and automatically adds reviewers via the Gerrit REST API
4. **Check group membership** — enumerates members of 60+ organization-specific `ci-authorized` groups:
 - `amd/ci-authorized`, `ampere/ci-authorized`, `arm/ci-authorized`, `aspeed/ci-authorized`
 - `google/ci-authorized`, `ibm/ci-authorized`, `intel/ci-authorized`, `nvidia/ci-authorized`
 - And 50+ more organizations
5. **Grant or deny CI** — if found, sets `ok-to-test=1` with "User approved, CI ok to start", then immediately resets to `ok-to-test=0`

This implements the delegated CI authorization model from the [ci-authorization design document](#), where each organization manages its own `ci-authorized` Gerrit group.

Unit Testing Pipeline

The unit testing pipeline involves three layers:

Layer 1: Docker Entry Point (`run-unit-test-docker.sh`)

- Validates `WORKSPACE`, build scripts, and test package directories
- Invokes `scripts/build-unit-test-docker` to build/cache the Docker image
- Launches a Docker container with the test package mounted
- Inside the container, runs `dbus-unit-test.py` → `unit-test.py`

Key environment variables:

Variable	Purpose
<code>UNIT_TEST_PKG</code>	Repository being tested
<code>BRANCH</code>	Branch to build from (defaults to master)
<code>TEST_ONLY</code>	Skip analysis tools
<code>NO_FORMAT_CODE</code>	Skip formatting checks
<code>NO_CPPCHECK</code>	Skip static analysis
<code>INTERACTIVE</code>	Run bash shell for debugging

Layer 2: Docker Image Builder (`scripts/build-unit-test-docker`)

- Builds 30+ OpenBMC dependency packages in parallel using threads
- Core libraries: sdbusplus, phosphor-dbus-interfaces, phosphor-logging, libpdm
- Each package specifies build type: autoconf, cmake, meson, make, or custom
- Uses Docker content-hash-based caching with `YYYY-lww` (ISO week) tagging
- Supports both Docker and Podman container runtimes

Layer 3: Test Runner (`scripts/unit-test.py`)

- Detects build system: Meson, Autotools, or CMake
- Implements a `DepTree` class for dependency ordering with cycle detection
- For the target package:
 1. Downloads and builds all dependencies
 2. Configures with testing flags enabled
 3. Builds the software
 4. Executes unit tests
 5. Runs optional analysis: cppcheck, valgrind, sanitizers, clang-tidy
 6. Generates code coverage reports

Layer 4: D-Bus Wrapper (`scripts/dbus-unit-test.py`)

- Launches a `dbus-daemon` in session mode with a custom socket
- Sets `DBUS_SESSION_BUS_ADDRESS` for OpenBMC services that require D-Bus
- Cleans up the D-Bus daemon on exit

Full Image Build (`build-setup.sh`)

Handles the complete BitBake image build for Tier 2 CI:

- Generates a Dockerfile for either Fedora or Ubuntu containers
- Configures locale, proxy settings, user/group mappings
- Creates an inner build script that runs BitBake inside the container
- Supports `CONTAINER_ONLY` mode for just building the Docker image
- Handles both Docker and Podman runtimes

QEMU Testing Pipeline

QEMU Build (`qemu-build.sh`)

- Compiles QEMU from source in a Docker container (Ubuntu Jammy base)
- Targets ARM architecture with minimal features (no X11, USB, VNC)

QEMU Boot (`scripts/boot-qemu.sh`)

- Launches QEMU with configurable architecture (ppc64le or x86_64)
- Port forwarding: SSH (22), HTTPS (443), HTTP (80), IPMI (623)
- Handles multiple image formats: `.ubi.mtd` , `.static.mtd` , `rootfs.ext4`
- Machine-specific configurations (e.g., Tacoma requires four NICs)

QEMU Boot Validation (`scripts/boot-qemu-test.exp`)

- Expect script that spawns `boot-qemu.sh`
- Waits for login prompt, authenticates with `root@openBmc`
- Outputs `OPENBMC-READY` marker when system is booted

Robot Framework Tests (`run-qemu-robot-test.sh`)

- Builds a Docker image with QEMU and Robot Framework pre-installed
- Starts QEMU container in detached mode
- Polls Docker logs for `OPENBMC-READY` marker (timeout: 300s default)
- Launches a separate Robot test container against the QEMU instance
- Tests include: Redfish API, IPMI, SSH connectivity, boot validation
- Supports architectures: ppc64le, x86_64, aarch64

Multi-Platform Smoke Test (`scripts/test-qemu`)

Downloads pre-built firmware from Jenkins and validates login for:

Platform	SoC
Palmetto	AST2400
Romulus	AST2500
P10BMC	AST2600

Code Formatting Suite

`scripts/format-code.sh` enforces consistency across 14 tools:

Tool	File Types	Purpose
<code>clang_format</code>	C/C++	Code formatting
<code>clang_tidy</code>	C/C++	Static analysis config
<code>black</code>	Python	Code formatting (79-char lines)
<code>flake8</code>	Python	Style checking
<code>isort</code>	Python	Import organization
<code>beautysh</code>	bash/sh	Shell script formatting
<code>shellcheck</code>	bash/sh	Shell static analysis
<code>eslint</code>	JSON	JSON linting
<code>prettier</code>	JSON/MD/YAML	Multi-format beautifier
<code>markdownlint</code>	Markdown	Markdown validation
<code>meson</code>	Meson files	Build file formatting
<code>commit_gitlint</code>	Commit messages	Format validation (72-char lines)
<code>commit_spelling</code>	Commit messages	Spell-check via codespell

Features `--enable` and `--disable` flags, `.linter-ignore` file support, and TTY-aware colored output.

Seed Job (Cache Warming)

The `build-seed` job pre-builds sstate caches for ~19 supported machines:

```
anacapa bletchley bletchley15 catalina clemente e3c246d4i
evb-npcm845 gb200nvl-obmc gbs harma minerva p10bmc romulus
santabarbara ventura ventura2 witherspoon yosemite4 yosemite5
```

This dramatically speeds up per-commit CI builds since BitBake reuses cached artifacts instead of building everything from scratch. If a build fails for one machine, it continues to the next.

Docker Infrastructure

Image Hierarchy

```
Ubuntu Base (public.ecr.aws/ubuntu)
├─ Unit Test Image (openbmc/ubuntu-unit-test)
│   ├── 30+ pre-built OpenBMC packages
│   ├── Build tools (meson, cmake, autotools)
│   ├── Analysis tools (cppcheck, valgrind, clang-tidy)
│   └─ Formatting tools (clang-format, black, shellcheck, etc.)
├─ QEMU Robot Test Image (openbmc/ubuntu-robot-qemu)
│   ├── QEMU with ARM support
│   ├── Robot Framework + libraries
│   ├── Redfish tools (redfish, redfishtool)
│   ├── Firefox + geckodriver (Selenium tests)
│   └─ ipmitool, sshpass, socat
├─ BitBake Build Image (generated by build-setup.sh)
│   ├── Fedora or Ubuntu base
│   └─ BitBake build environment
└─ Rootfs Size Image (openbmc/ubuntu-rootfs-size)
    └─ squashfs-tools, Python 3
```

Caching and Cleanup

- **Tagging convention** — images use `YYYY-Www` (ISO week) tags combined with Dockerfile content hashes
- **Cleanup** — `scripts/clean-unit-test-docker` runs weekly to remove stale images
- **Cache invalidation** — changing the Dockerfile content hash forces a rebuild
- **Runtime support** — all scripts support both Docker and Podman

Key Environment Variables

Variable	Used By	Purpose
<code>WORKSPACE</code>	All scripts	Jenkins workspace directory
<code>UNIT_TEST_PKG</code>	<code>run-unit-test-docker.sh</code>	Repository name to test
<code>GERRIT_PROJECT</code>	<code>userid-validation</code>	Gerrit project path
<code>GERRIT_BRANCH</code>	Multiple	Branch under review
<code>GERRIT_CHANGE_NUMBER</code>	<code>userid-validation</code>	Gerrit change number
<code>SSH_KEY</code>	<code>userid-validation</code>	Jenkins SSH key for Gerrit
<code>target</code>	<code>build-setup.sh</code>	Machine target for BitBake
<code>DOCKER_REG</code>	All Docker scripts	Docker registry override
<code>http_proxy</code>	All Docker scripts	Proxy configuration
<code>UPSTREAM_WORKSPACE</code>	<code>run-qemu-robot-test.sh</code>	Path to QEMU artifacts
<code>QEMU_RUN_TIMER</code>	<code>run-qemu-robot-test.sh</code>	QEMU runtime before shutdown
<code>INTERACTIVE</code>	<code>run-unit-test-docker.sh</code>	Drop to bash shell for debugging

References

- [Jenkins Instance](#)
- [openbmc-build-scripts Repository](#)
- [CI Authorization Design Document](#)
- [OpenBMC Contributing Guide](#)
- [Gerrit Setup Guide](#)
- [Adding a System to Hardware CI](#)

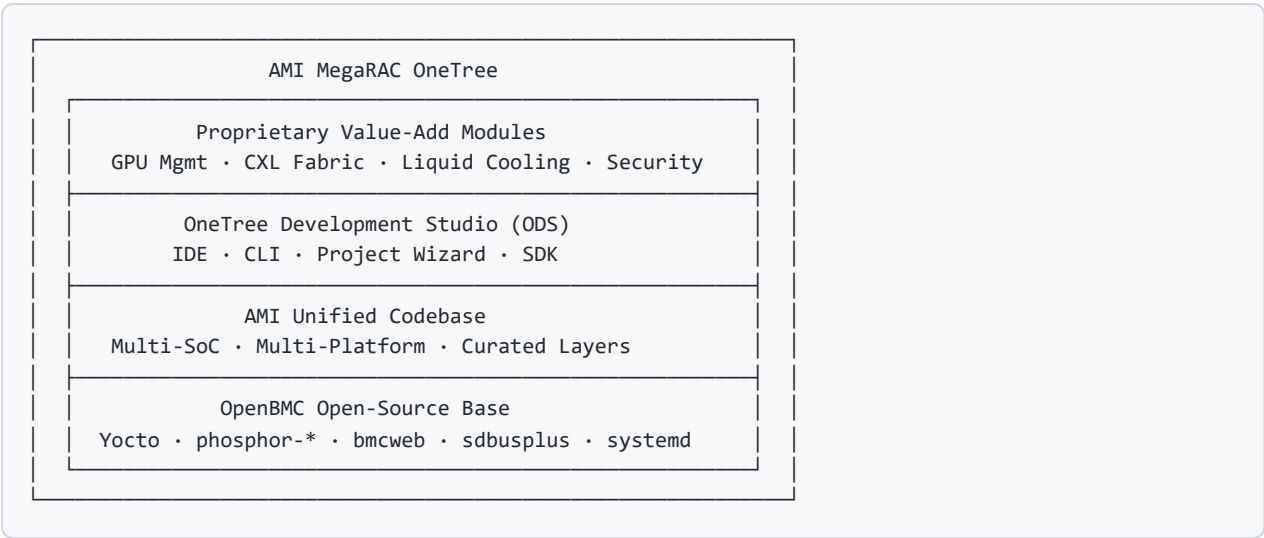
AMI MegaRAC OneTree vs Upstream OpenBMC

Overview

AMI [MegaRAC OneTree](#) is a commercial BMC firmware platform built on top of the open-source [OpenBMC](#) project. OneTree packages OpenBMC’s open-source base with proprietary value-add modules, a unified multi-platform codebase, dedicated development tooling, and commercial support.

This appendix provides a technical comparison of the two platforms across build systems, development tools, testing, code architecture, and security.

Relationship



AMI adopts an “**Open Source Base + Commercial Value-Added Modules**” strategy: the open base provides a standard framework for rapid deployment, while proprietary IP modules are selectable based on customer needs.

Build System

	OpenBMC	AMI OneTree
Foundation	Yocto / OpenEmbedded + BitBake	Same Yocto/BitBake underneath, abstracted by tooling
Build command	<code>bitbake obmc-phosphor-image</code>	OneTree Development Studio (ODS) GUI or CLI wraps BitBake
Repo management	<code>repo init</code> / <code>kas</code> to fetch 100+ individual git repos	Unified “OneTree” mono-repo managing multiple platforms in a single codebase
Layers	Community meta-layers (<code>meta-phosphor</code> , <code>meta-openembedded</code> , vendor layers like <code>meta-ibm</code> , <code>meta-facebook</code>)	AMI-curated layers + proprietary <code>meta-ami</code> layer
Machine targets	One machine config per vendor fork (e.g., <code>romulus</code> , <code>witherspoon</code> , <code>yosemite4</code>)	Multi-platform from one tree — ASPEED AST2500/2600/2700, Nuvoton Arbel
Build reproducibility	Depends on vendor pinning; community repos use rolling HEAD	Formal SDK releases (e.g., SDK v9.08) with pinned versions

OpenBMC Build Workflow

```
# Typical OpenBMC build
git clone https://github.com/openbmc/openbmc.git
cd openbmc

# Set up environment for a specific machine
. setup romulus

# Build the full image (can take 1-3 hours on first build)
bitbake obmc-phosphor-image
```

OneTree Build Workflow

OneTree Development Studio provides a GUI-driven workflow for project creation, feature selection, and build configuration. Alternatively, the ODS CLI (v3.0) provides command-line access to the same capabilities for CI/CD integration.

AMI explicitly markets OneTree as reducing the Yocto learning curve — developers interact with a higher-level abstraction rather than writing BitBake recipes directly.

Development Tools & IDE

	OpenBMC	AMI OneTree
IDE	No official IDE — use any editor (vim, VS Code, etc.)	OneTree Development Studio (ODS) — dedicated IDE with GUI for project creation, feature selection, platform config, and build
CLI tooling	Yocto <code>devtool</code> for recipe modification, <code>bitbake</code> directly	ODS CLI (v3.0) wraps build commands for streamlined workflows
Recipe workflow	Manual: <code>devtool modify</code> , <code>devtool add</code> , edit <code>.bb</code> / <code>.bbappend</code> files	GUI-driven feature selection and configuration, abstracts recipe-level details
Learning curve	Steep — requires understanding Yocto layers, BitBake syntax, recipe classes, <code>devtool</code> , OpenEmbedded	Reduced — ODS abstracts Yocto complexity
SDK	Yocto eSDK generated per build	Formal SDK releases (e.g., SDK v9.08) with unified kernel for AST2600, AST27x0, Arbel

OpenBMC devtool Workflow

```
# Modify an existing package source
devtool modify -n phosphor-logging /path/to/local/source

# Build the modified package
bitbake phosphor-logging

# Test in QEMU
runqemu romulus nographic

# Finish and generate patches
devtool finish phosphor-logging meta-phosphor
```

OneTree Development Studio Features

- **Project creation wizard** — select SoC, platform, and feature set via GUI
- **Feature customization** — enable/disable BMC capabilities without editing recipes
- **Platform configuration** — configure hardware-specific settings
- **Build optimization** — managed build with dependency resolution
- **Firmware update tool** — integrated image deployment and update

Testing & Emulation

	OpenBMC	AMI OneTree
QEMU	Built-in QEMU machine models; Romulus is CI default	QEMU support inherited from OpenBMC; additional platform-specific configs
CI system	Jenkins at jenkins.openbmc.org — compiles commits, runs <code>make check</code> per repo, builds full image, runs Robot Framework in QEMU	Internal AMI QA pipeline with multiple QA cycles, third-party code scanning, in-house test suites
Test framework	Robot Framework (integration), GoogleTest/pytest (unit)	Same frameworks plus AMI proprietary test suites
CI gating	Two-tier: repo-level <code>make check</code> + system-level QEMU image tests	Multi-stage internal validation before release

OpenBMC CI Two-Tier Model

Tier 1: Repository-Level CI (fast, per-commit) └ <code>make check</code>, unit tests, static analysis Tier 2: System-Level CI (slower, full image) └ <code>bitbake full image</code> → QEMU boot → Robot Framework tests
--

OpenBMC CI runs automatically for org members. Non-member commits require a reviewer to manually trigger the CI run.

Code Review & Contribution

	OpenBMC	AMI OneTree
Code hosting	GitHub (source) + Gerrit (code review), per-repo	GitHub under ocp-hm-openbmc-opf-ami ; internal review for proprietary code
Review process	Open Gerrit reviews with Jenkins CI gating	Internal AMI review for proprietary modules; upstream contributions follow OpenBMC Gerrit process
Contribution model	Fully open — anyone can submit patches	Dual: open-source base contributions go upstream; proprietary modules are AMI-internal

Code Architecture

Both platforms share the same core architecture since OneTree is built on OpenBMC:

Component	OpenBMC	AMI OneTree
Init system	systemd	systemd
IPC	D-Bus (sdbusplus)	D-Bus (sdbusplus)
Services	<code>phosphor-*</code> daemons	<code>phosphor-*</code> + AMI proprietary extensions
Web server	<code>bmcweb</code> (Redfish)	<code>bmcweb</code> + AMI OEM Redfish schema extensions
Package format	<code>ipk</code> via <code>opkg</code>	<code>ipk</code> via <code>opkg</code>
Protocols	MCTP, PLDM, IPMI, Redfish	All of the above + SPDM, NVMe-MI, SMBPBI, CXL fabric management

OneTree Additional Capabilities

AMI adds proprietary modules for use cases beyond upstream OpenBMC:

- **Accelerator/GPU management** — monitor and manage GPGPUs
- **CXL fabric management** — memory expansion and pooling
- **Advanced liquid cooling management** — for high-density AI infrastructure
- **Enhanced telemetry** — extended RAS and real-time monitoring

Silicon Support

SoC	OpenBMC	AMI OneTree
ASPEED AST2500	Supported (community)	Supported
ASPEED AST2600	Supported (community)	Supported
ASPEED AST2700	In progress (community)	Supported (unified kernel)
Nuvoton NPCM7xx	Supported (community)	Supported
Nuvoton Arbel	Limited	Supported (unified kernel)

OneTree’s unified kernel approach means a single codebase supports AST2600, AST27x0, and Arbel, reducing porting effort across SoC families.

Security

	OpenBMC	AMI OneTree
Code scanning	Community-driven; varies by vendor	Third-party scanning + in-house security test suites
Secure boot	Available but vendor-specific	Standardized secure boot across platforms
Certifications	None (community project)	OCP SAFE certified, ISO 9001:2015
Security audits	Community review	Eclypsium identified OneTree as optimal among OpenBMC builds for security posture

Fragmentation: The Core Problem OneTree Solves

Upstream OpenBMC suffers from **fragmentation** — major vendors (Meta, Google, IBM, etc.) maintain separate forks with vendor-specific features in isolated repositories. This makes it difficult to:

- Share features across platforms
- Maintain a consistent security baseline
- Onboard new platforms without duplicating integration work

OneTree addresses this with a **unified mono-repo architecture** that manages multiple platforms, SoCs, and feature sets in a single codebase, while still tracking upstream OpenBMC.

OpenBMC Ecosystem (Fragmented):

- └─ openbmc/openbmc (upstream)
- └─ facebook/openbmc (Meta fork)
- └─ google/openbmc (Google fork)
- └─ IBM phosphor repos
- └─ ...many vendor forks

AMI OneTree (Unified):

- └─ OneTree mono-repo
 - └─ OpenBMC upstream base
 - └─ Multi-SoC platform support
 - └─ AMI proprietary modules
 - └─ Customer customization layer

Summary

Aspect	OpenBMC	AMI OneTree
Best for	Hyperscalers with strong in-house firmware teams	ODMs/OEMs wanting faster time-to-market
Flexibility	Maximum — full control over everything	Reduced — abstractions trade flexibility for productivity
Support	Community (self-service)	Commercial (AMI global engineering)
Cost	Free (open source)	Commercial license
Time-to-market	Longer — must integrate everything yourself	Shorter — pre-integrated platform with tooling

References

- [AMI MegaRAC OneTree Product Page](#)
- [OneTree Development Studio Introduction](#)
- [MegaRAC OneTree Version 3.0 Announcement](#)
- [MegaRAC OneTree v2.1 Release](#)
- [Eclipsium OpenBMC Security Analysis](#)
- [AMI OneTree GitHub \(OCP\)](#)
- [OpenBMC GitHub](#)
- [OpenBMC Yocto Development Docs](#)
- [OpenBMC Development Guide](#)

Yocto BitBake Build Optimization

Overview

A first-time OpenBMC build can take 1–3 hours depending on hardware, network speed, and configuration. Subsequent builds are faster thanks to caching, but poorly tuned settings can still waste significant time. This appendix covers practical optimizations — from hardware selection to BitBake tuning — that can dramatically reduce build times for both initial and incremental OpenBMC builds.

Hardware Recommendations

Build performance is primarily bound by CPU, RAM, and disk I/O. Invest in these areas for the biggest gains.

Minimum vs Recommended Specs

Resource	Minimum	Recommended	Notes
CPU	4 cores	8–16 cores	More cores = more parallel tasks
RAM	16 GB	32–64 GB	BitBake + compiler processes consume memory quickly
Storage	100 GB HDD	250+ GB NVMe SSD	I/O is often the bottleneck
Swap	8 GB	16 GB	Safety net for memory-intensive recipes

Builds on spinning HDDs can be 3–5x slower than on NVMe SSDs. If your build is slow and you are on an HDD, upgrading storage is the single biggest improvement you can make.

Storage Best Practices

- **Dedicated build disk** — put the build directory on its own disk to avoid contention with the OS
- **NVMe SSD** — preferred over SATA SSD for the build directory
- **ext4 filesystem** — use ext4 over ext2/ext3 for improved performance with extents
- **Disable journaling** (optional, for dedicated build disks only):

```
sudo tune2fs -O ^has_journal /dev/sdX
```

- **Optimized mount options** — reduce unnecessary disk writes:

```
noatime,barrier=0,commit=6000
```

- **tmpfs for /tmp** — mount /tmp as tmpfs to speed up temporary file operations:

```
sudo mount -t tmpfs -o size=8G tmpfs /tmp
```

Using tmpfs for the entire TMPDIR provides limited additional benefit because GCC's `-pipe` flag already keeps intermediate files in memory. However, tmpfs for `/tmp` is still worthwhile.

BitBake Parallel Build Settings

These variables control how many tasks and compilations run simultaneously.

Key Variables

Variable	Purpose	Default	Recommended
<code>BB_NUMBER_THREADS</code>	Max simultaneous BitBake tasks	Number of CPU cores	<code>nproc</code> or <code>nproc - 1</code>
<code>PARALLEL_MAKE</code>	Parallel <code>make</code> jobs per recipe	<code>-j</code> + number of CPU cores	<code>-j \$(nproc)</code>
<code>BB_NUMBER_PARSE_THREADS</code>	Threads for recipe parsing	Number of CPU cores	<code>nproc</code>
<code>PARALLEL_MAKEINST</code>	Parallel <code>make install</code> jobs	Same as <code>PARALLEL_MAKE</code>	Leave as default

Configuration in local.conf

```
# conf/local.conf

# Match to your CPU core count (example: 8-core system)
BB_NUMBER_THREADS = "8"
PARALLEL_MAKE = "-j 8"
BB_NUMBER_PARSE_THREADS = "8"
```

Or use dynamic calculation:

```
# Automatically scale to available cores
BB_NUMBER_THREADS = "${@oe.utils.cpu_count()}"
PARALLEL_MAKE = "-j ${@oe.utils.cpu_count()}"
```

Scaling Considerations

Research from Yocto Project benchmarks shows that small builds (minimal images) do not scale well beyond ~8 cores due to package inter-dependencies. Larger builds like full OpenBMC images benefit more from additional cores. Going beyond 16 threads on a single machine typically yields diminishing returns.

If your system is running out of memory during builds, **reduce** `BB_NUMBER_THREADS` rather than adding swap:

```
# Memory-constrained system (16 GB RAM)
BB_NUMBER_THREADS = "4"
PARALLEL_MAKE = "-j 4"
```

Shared State Cache (sstate-cache)

The shared state cache is BitBake’s most powerful optimization. It stores the output of individual build tasks and reuses them when inputs haven’t changed.

How It Works

```
Recipe Task (e.g., do_compile for phosphor-logging)
|
| Hash inputs (source, config, dependencies)
|
| Hash matches sstate? —YES—> Skip task, use cached output
|
| Hash doesn't match? —NO—> Build task, store result in sstate
```

Shared sstate-cache Directory

By default, each build gets its own sstate-cache. Share it across builds to avoid redundant work:

```
# conf/local.conf – shared sstate directory
SSTATE_DIR = "/var/cache/yocto/sstate-cache"
```

A typical OpenBMC build generates 4–8 GB of sstate data. With a warm cache, rebuild times drop from hours to minutes.

Remote sstate Mirrors

Pull pre-built artifacts from a shared server to skip building common packages:

```
# conf/local.conf – use a remote sstate mirror
SSTATE_MIRRORS ?= "\
    file://.* https://your-server.example.com/sstate/PATH;downloadfilename=PATH \
"
```

The Yocto Project provides a public sstate mirror:

```
SSTATE_MIRRORS ?= "\
    file://.* https://sstate.yoctoproject.org/all/PATH;downloadfilename=PATH \
"
```

For team environments, set up an HTTP server hosting your sstate-cache directory. Every developer benefits from artifacts built by any team member or CI system.

OpenBMC CI Seed Builds

OpenBMC's Jenkins CI runs a `build-seed` job that pre-populates sstate caches for ~19 supported machines (romulus, witherspoon, p10bmc, etc.). This is why CI builds for submitted patches are much faster than first-time local builds — they reuse the seeded cache.

Download Directory (DL_DIR)

BitBake downloads source tarballs and git repositories to the `DL_DIR`. Sharing this across builds prevents redundant downloads.

Shared Download Directory

```
# conf/local.conf
DL_DIR = "/var/cache/yocto/downloads"
```

Generate Mirror Tarballs

Create tarballs from git fetches so they can be mirrored more easily:

```
BB_GENERATE_MIRROR_TARBALLS = "1"
```

Pre-fetch All Sources

Download everything before building (useful for preparing offline builds):

```
bitbake obmc-phosphor-image --runonly=fetch
```

Offline Builds

Verify that all sources are cached, then build without network access:

```
# Test that everything is cached
BB_NO_NETWORK = "1"
bitbake obmc-phosphor-image
```

Source Mirrors

Point to an organizational mirror to avoid relying on upstream servers:

```
# conf/local.conf
SOURCE_MIRROR_URL ?= "file:///var/cache/yocto/downloads/"
INHERIT += "own-mirrors"
```

Hash Equivalence

Hash equivalence is an advanced caching optimization that recognizes when two different task input hashes produce functionally identical output. This avoids unnecessary rebuilds when cosmetic changes (like timestamps or build paths) alter the input hash without changing the actual result.

How It Works

```
Task A (hash: abc123) — produces —> Output X
Task B (hash: def456) — produces —> Output X (same!)

Hash equivalence server records: abc123 ≡ def456
Next time hash def456 is seen, reuse cached Output X from abc123
```

Configuration

```
# conf/local.conf

# Enable hash equivalence
BB_SIGNATURE_HANDLER = "OEEquivHash"

# Run a local hash equivalence server (auto-started by BitBake)
BB_HASHSERVE = "auto"

# Optionally connect to the Yocto Project upstream server
BB_HASHSERVE_UPSTREAM = "wss://hashserv.yoctoproject.org/ws"
```

Running Your Own Server

For teams, run a shared hash equivalence server:

```
# Start a persistent hash equivalence server
bitbake-hashserv --bind 0.0.0.0:8687 --database /var/cache/yocto/hashserv.db

# In local.conf on developer machines:
BB_HASHSERVE = "your-server.example.com:8687"
BB_SIGNATURE_HANDLER = "OEEquivHash"
```

The hash equivalence server must be maintained alongside the shared state cache. If you share sstate but not hash equivalence data (or vice versa), the benefit is reduced.

Persistent Cache Directory

Store hash equivalence databases across builds:

```
PERSISTENT_DIR = "/var/cache/yocto/persistent"
```

rm_work: Reducing Disk Usage

The `rm_work` class removes a recipe's work directory after it finishes building, freeing disk space as the build progresses.

Configuration

```
# conf/local.conf
INHERIT += "rm_work"

# Exclude specific recipes you want to debug or inspect
RM_WORK_EXCLUDE += "phosphor-logging bmcweb"
```

Benefits and Trade-offs

Benefit	Trade-off
Reduces peak disk usage by 50–70%	Cannot inspect build artifacts after the fact
Faster cleanup of TMPDIR	Rebuilding an excluded recipe requires re-fetching/building
Slightly faster builds (less data in cache to index)	Harder to debug build failures

Use `rm_work` on CI servers and space-constrained systems. Exclude recipes you are actively developing so you can inspect their build output.

Reducing Build Scope

Strip Unnecessary DISTRO_FEATURES

Remove features you don't need to avoid building unnecessary packages:

```
# conf/local.conf
DISTRO_FEATURES:remove = "x11 wayland bluetooth wifi nfc 3g"
```

Disable Debug Packages

Skip generating `-dbg` packages to save packaging time:

```
INHIBIT_PACKAGE_DEBUG_SPLIT = "1"
```

Packaging Backend

IPK is the fastest packaging backend (and the OpenBMC default). If your configuration doesn't specify one, ensure it's set:

```
PACKAGE_CLASSES = "package_ipk"
```

Distributed Compilation with icecc

[Icecream \(icecc\)](#) distributes compilation jobs across a network of machines, turning multiple build hosts into a single compilation cluster.

Setup

1. Install icecc on all machines in the cluster:

```
sudo apt install icecc    # Debian/Ubuntu
sudo dnf install icecream # Fedora
```

2. Start the scheduler on one machine:

```
sudo systemctl start icecc-scheduler
```

3. Start the daemon on all machines (including the scheduler host):

```
sudo systemctl start iceccd
```

4. Configure BitBake:

```
# conf/local.conf
INHERIT += "icecc"
ICECC_PARALLEL_MAKE = "-j 32"    # Total jobs across cluster
PARALLEL_MAKE = "-j 8"          # Fallback for non-icecc recipes
```

Considerations

- icecc changes task hashes — sstate built with icecc is **not** reusable without icecc
- To toggle icecc without invalidating sstate, always inherit the class and use `ICECC_DISABLED`:

```
INHERIT += "icecc"
ICECC_DISABLED = "1" # Set to "0" to enable
```

- Blacklist recipes that fail with distributed compilation:

```
ICECC_USER_PACKAGE_BL = "some-problematic-recipe"
```

Uninative: Avoiding Host Tool Rebuilds

Uninative provides a pre-built native C library that prevents unnecessary rebuilds when host system tools are updated.

Without uninative, updating your host's glibc (e.g., via `apt upgrade`) can invalidate the entire sstate cache, forcing a full rebuild. Uninative locks the native toolchain to a specific version.

Configuration

Uninative is typically enabled by default in modern Yocto/OpenBMC setups:

```
# Usually already set in your distro config
INHERIT += "uninative"
```

If you see unexpected full rebuilds after a host OS update, verify that uninative is enabled. This is one of the most common causes of unnecessary rebuild cycles.

Build Monitoring and Profiling

Enable buildstats

Track per-recipe build times to identify bottlenecks:

```
# conf/local.conf
INHERIT += "buildstats"
```

Build statistics are written to `tmp/buildstats/` and can be analyzed to find the slowest recipes:


```
# Find the 10 slowest recipes in the last build
cd tmp/buildstats/
find . -name "do_compile" -exec sh -c \
  'echo "${cat "$1" | grep "Elapsed" | awk "{print \$3}" } $1" _ {} \; \
  | sort -rn | head -10
```

BitBake Build Time Summary

BitBake prints a task execution summary at the end of each build. Pay attention to:

NOTE: Tasks Summary: Attempted 4532 tasks of which 4201 didn't need to be rerun and all succeeded.

If the number of “didn’t need to be rerun” is low relative to total tasks, your sstate cache is not being utilized effectively.

Quick Reference: Optimized local.conf

Here is a consolidated example of optimization settings for a typical 8-core, 32 GB RAM development machine:

```
# conf/local.conf – OpenBMC Build Optimization

# — Parallelism —
BB_NUMBER_THREADS = "8"
PARALLEL_MAKE = "-j 8"

# — Shared Caches —
DL_DIR = "/var/cache/yocto/downloads"
SSTATE_DIR = "/var/cache/yocto/sstate-cache"

# — Hash Equivalence —
BB_SIGNATURE_HANDLER = "OEEquivHash"
BB_HASHSERVE = "auto"

# — Disk Space —
INHERIT += "rm_work"
RM_WORK_EXCLUDE += "phosphor-logging bmcweb"

# — Performance Monitoring —
INHERIT += "buildstats"

# — Packaging —
PACKAGE_CLASSES = "package_ipk"
INHIBIT_PACKAGE_DEBUG_SPLIT = "1"
```

Optimization Impact Summary

Approximate build time improvements (cumulative, on a first-time full OpenBMC build):

Optimization	Typical Impact	Effort
NVMe SSD instead of HDD	3–5x faster I/O	Hardware purchase
Shared sstate-cache (warm)	60–90% time reduction	One-time config
Shared DL_DIR	10–20 min saved on first build	One-time config
Parallel build tuning	10–30% improvement	One-time config
Hash equivalence	5–15% fewer rebuilds	One-time config
rm_work	50–70% less disk usage	One-time config
icecc cluster (4 machines)	2–4x faster compilation	Infrastructure setup
Remote sstate mirror	Near-instant rebuilds for unchanged recipes	Server setup

References

- [Yocto Project: Speeding Up a Build](#) — official Yocto optimization guide
- [Yocto Project: Hash Equivalence Server Setup](#) — hash equivalence configuration
- [Yocto Project: Variables Glossary](#) — complete reference for all BitBake variables
- [Yocto Project Wiki: Build Performance](#) — community benchmarks and filesystem tips
- [Improving Yocto Build Time \(The Good Penguin\)](#) — practical optimization walkthrough
- [OpenBMC Yocto Development Docs](#) — OpenBMC-specific Yocto usage
- [OpenBMC Cheatsheet](#) — quick reference for OpenBMC builds
- [Icecream Distributed Compiler](#) — icecc project for distributed builds

Contributing to OpenBMC Guide Tutorial

Thank you for your interest in contributing! This guide ensures consistency across all documentation.

Quick Start

1. **Fork** the repository on GitHub
2. **Clone** your fork locally
3. **Create a branch** for your changes
4. **Make your changes** following the guidelines below
5. **Test locally** with `bundle exec jekyll serve`
6. **Submit a pull request**

Ways to Contribute

Report Issues

Found an error or missing information? [Open an issue](#) with:

- Clear description of the problem
- Link to the affected page
- Suggested fix (if you have one)

Improve Existing Guides

- Fix typos, broken links, or outdated information
- Add missing configuration examples
- Improve code samples with better explanations
- Update for newer OpenBMC versions

Add New Content

- Write guides for undocumented features
- Add troubleshooting sections
- Create new code examples
- Expand porting documentation for different platforms

Guide Structure

Every guide should follow this template:

```

---
layout: default
title: <Guide Title>
parent: <Section Name>
nav_order: <number>
difficulty: beginner|intermediate|advanced
prerequisites:
  - <prerequisite-guide-1>
  - <prerequisite-guide-2>
---

# <Guide Title>

## Overview

Brief introduction (2-3 paragraphs):
- What this component/feature does
- Why it matters
- When you would use it

## Architecture

- Component diagram (Mermaid or ASCII)
- Key interfaces and D-Bus paths
- Data flow explanation

## Configuration

### Required Files

| File | Location | Purpose |
|-----|-----|-----|
| ... | ... | ... |

### Configuration Example

```json
{
 "example": "configuration"
}

```

## Porting Guide

---

Step-by-step instructions:

1. **Prerequisites:** What must be in place first
2. **Configuration:** Files to create/modify
3. **Recipe changes:** Bitbake/Yocto modifications
4. **Verification:** How to test it works

## Code Examples

---

Link to examples in the [docs/examples/](#) directory.

# Troubleshooting

---

## Common Issue 1

**Symptom:** Description of what goes wrong **Cause:** Why it happens **Solution:** How to fix it

## References

---

- [Official Repo](#)
- [D-Bus Interfaces](#)
- Related guides: [Guide 1](#), [Guide 2](#) ``

## Style Guidelines

---

### Writing

Do	Don't
Use <b>active voice</b> : "Configure the sensor"	"The sensor should be configured"
Use <b>present tense</b> : "This creates a file"	"This will create a file"
Be <b>concise</b> : Avoid unnecessary words	Use filler phrases
Use <b>you/your</b> : Address the reader directly	Use passive constructions

### Code Examples

All code examples must be tested on QEMU ast2600-evb or romulus before submitting.

- Include the OpenBMC commit hash or version tested against
- Use syntax highlighting with the correct language tag
- Keep examples minimal but complete
- Ensure examples are copy-paste ready

### Formatting

Use **callouts** for important information:

```
{: .warning }
This action cannot be undone.

{: .note }
This is optional but recommended.

{: .tip }
Here's a helpful shortcut.
```

Use **tables** for structured data and **numbered lists** for sequential steps.

## Links

- **Internal links:** Use Jekyll's link tag

```
[Environment Setup](/openbmc-guide-tutorial/docs/01-getting-started/02-environment-setup.html)
```

- **External links:** Use absolute URLs
- **OpenBMC source:** Prefer GitHub permalinks with commit hash

## Adding Code Examples

1. Create a directory under `docs/examples/<category>/`
2. Include:
  - Complete, buildable source code
  - `README.md` with build/test instructions
  - `meson.build` or appropriate build configuration
3. Test on QEMU before submitting

### Example Directory Structure

```
docs/examples/
├── sensors/
│ ├── README.md
│ ├── meson.build
│ └── custom_sensor.cpp
├── dbus/
└── ...
```

## Testing Changes Locally

```
Install dependencies (first time only)
bundle install

Serve locally with live reload
bundle exec jekyll serve --livereload

Build without serving
bundle exec jekyll build
```

Visit <http://localhost:4000/openbmc-guide-tutorial/> to preview your changes.

### Docker Alternative

```
docker run --rm -it -v "$PWD:/srv/jekyll" -p 4000:4000 \
jekyll/jekyll:4.3 jekyll serve --host 0.0.0.0
```

## Pull Request Checklist

Before submitting, verify:

- ☐ Content follows the guide template structure
- ☐ Code examples are tested on QEMU
- ☐ All links work (internal and external)
- ☐ Spelling and grammar are correct
- ☐ Difficulty and prerequisites are set appropriately
- ☐ Related guides are cross-referenced
- ☐ Mermaid diagrams render correctly (if used)

## Commit Messages

---

Use clear, descriptive commit messages:

```
Add fan control troubleshooting section
```

- Document common PID tuning issues
- Add thermal runaway debugging steps
- Include dbus-monitor examples

## Review Process

---

1. Submit your pull request
2. Maintainers review for accuracy and style
3. Address any feedback
4. Once approved, changes are merged and auto-deployed

## Questions?

---

- [Open an issue](#) for questions about contributing
- Check existing issues for common questions
- Reference the [OpenBMC docs](#) for authoritative information

---

This is a community project. All contributions are welcome, from fixing typos to adding complete guides. Every improvement helps the OpenBMC community.